

# C# Y .NET 10

## Parte 0. Entorno de desarrollo

# 2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                            |    |
|--------------------------------------------|----|
| Tabla de ilustraciones.....                | 2  |
| Acerca del autor.....                      | 3  |
| Licencia de este libro .....               | 3  |
| Licencia del software .....                | 3  |
| Marcas registradas .....                   | 4  |
| Dedicatoria .....                          | 5  |
| Introducción.....                          | 6  |
| Visual Studio Community Edition 2026 ..... | 7  |
| Explorador de soluciones .....             | 15 |
| Lugar donde se guardan los proyectos ..... | 16 |

## Tabla de ilustraciones

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Ilustración 1: Sitio oficial de Visual Studio 2026 .....                                         | 7  |
| Ilustración 2: Visual Studio Community 2026.....                                                 | 8  |
| Ilustración 3: Instalar el "Desarrollo de escritorio de .NET" .....                              | 9  |
| Ilustración 4: Pantalla de inicio de Visual Studio 2026. Se presiona "Crear un proyecto" .....   | 10 |
| Ilustración 5: Se selecciona "Aplicación de consola" .....                                       | 11 |
| Ilustración 6: Se le pone un nombre al proyecto.....                                             | 12 |
| Ilustración 7: Usar .NET 10 y se marca la opción "No usar instrucciones de nivel superior" ..... | 13 |
| Ilustración 8: Se crea una aplicación por defecto. Se da clic en el botón de ejecutar.....       | 14 |
| Ilustración 9: Ejecución del programa.....                                                       | 14 |
| Ilustración 10: Explorador de soluciones. ....                                                   | 15 |
| Ilustración 11: Sitio donde se guardan los proyectos.....                                        | 16 |
| Ilustración 12: Archivos al interior de cada proyecto .....                                      | 16 |
| Ilustración 13: Código fuente. ....                                                              | 17 |
| Ilustración 14: Comprimir carpeta.....                                                           | 18 |
| Ilustración 15: Carpeta comprimida .....                                                         | 19 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL "Lesser General Public License" (Wikipedia, 2017)





## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

# Introducción

¿Por qué C#? Se enumeran algunas razones:

Lenguaje fuertemente tipado por lo que obliga a estar atento en la escritura de operaciones matemáticas y validar que los resultados sean correctos minimizando el riesgo de cálculos incorrectos que pueden aparecer al mezclar datos de diferente tipo.

Los entornos de desarrollo son bastante maduros y completos.

Puede ser usado para hacer aplicaciones de escritorio, Web e inclusive videojuegos usando motores como Godot o Unity.

Es multiplataforma.

Es compilado por lo que su velocidad de ejecución es muy rápida.

Tiene protección de memoria, luego las operaciones con arreglos son mucho más confiables, ante un error de desbordamiento, C# se detiene y muestra en qué instrucción hubo el problema.

Tiene recolector de basura, es decir, no hay que preocuparse por liberar la memoria de los objetos que ya no se usan, C# se encarga automáticamente de esto.

Es libre. C# está regulado por ECMA International.

El entorno de desarrollo será en principio Microsoft Visual Studio 2026 Community Edition, en sucesivas actualizaciones, se documentará en los anexos otras herramientas de desarrollo.

# Visual Studio Community Edition 2026

El entorno de desarrollo que ofrece Microsoft para C# es Visual Studio 2026, el sitio oficial es <https://visualstudio.microsoft.com/es/vs/> :

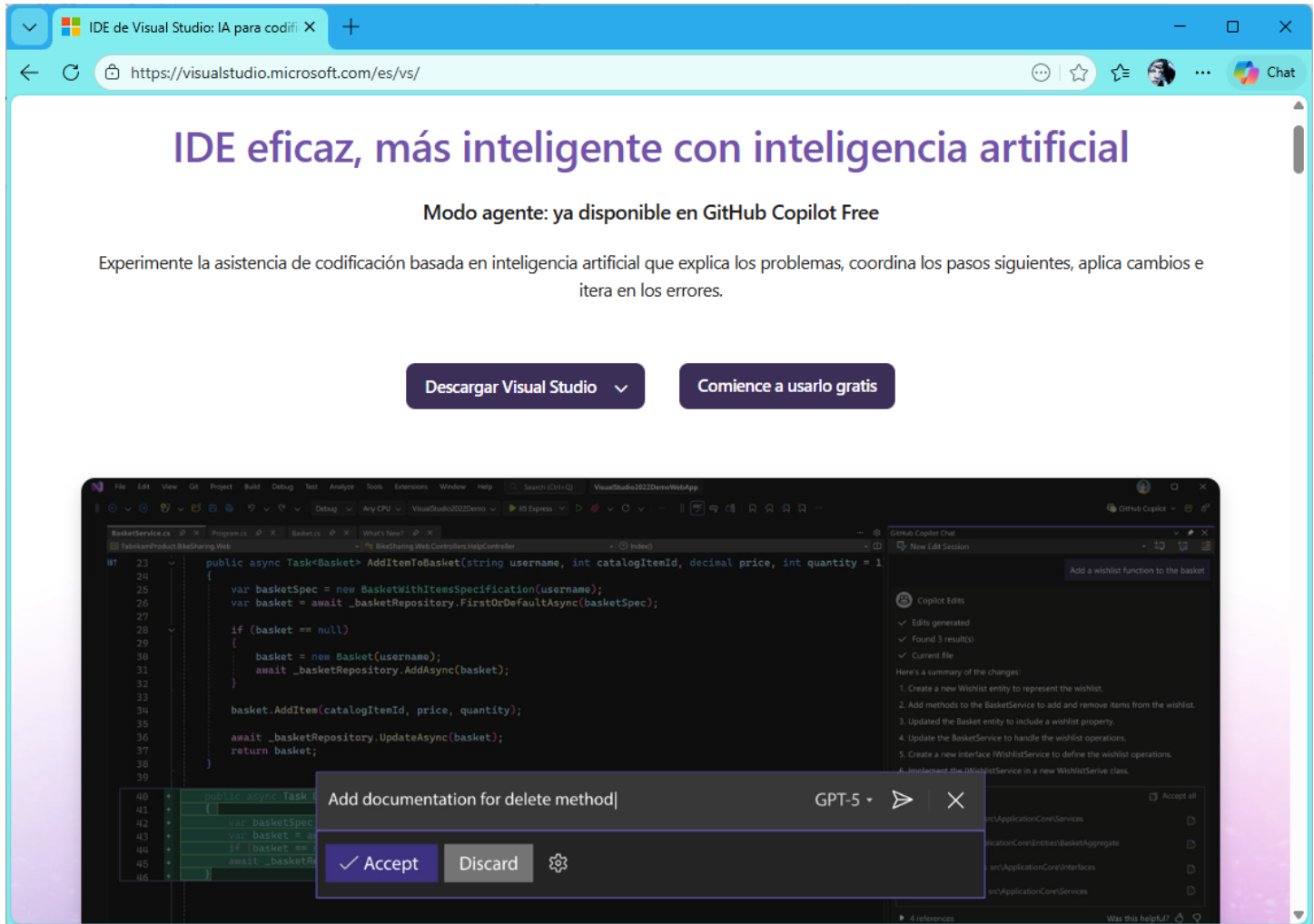


Ilustración 1: Sitio oficial de Visual Studio 2026

Se va a trabajar con la Community Edition:



Ilustración 2: Visual Studio Community 2026

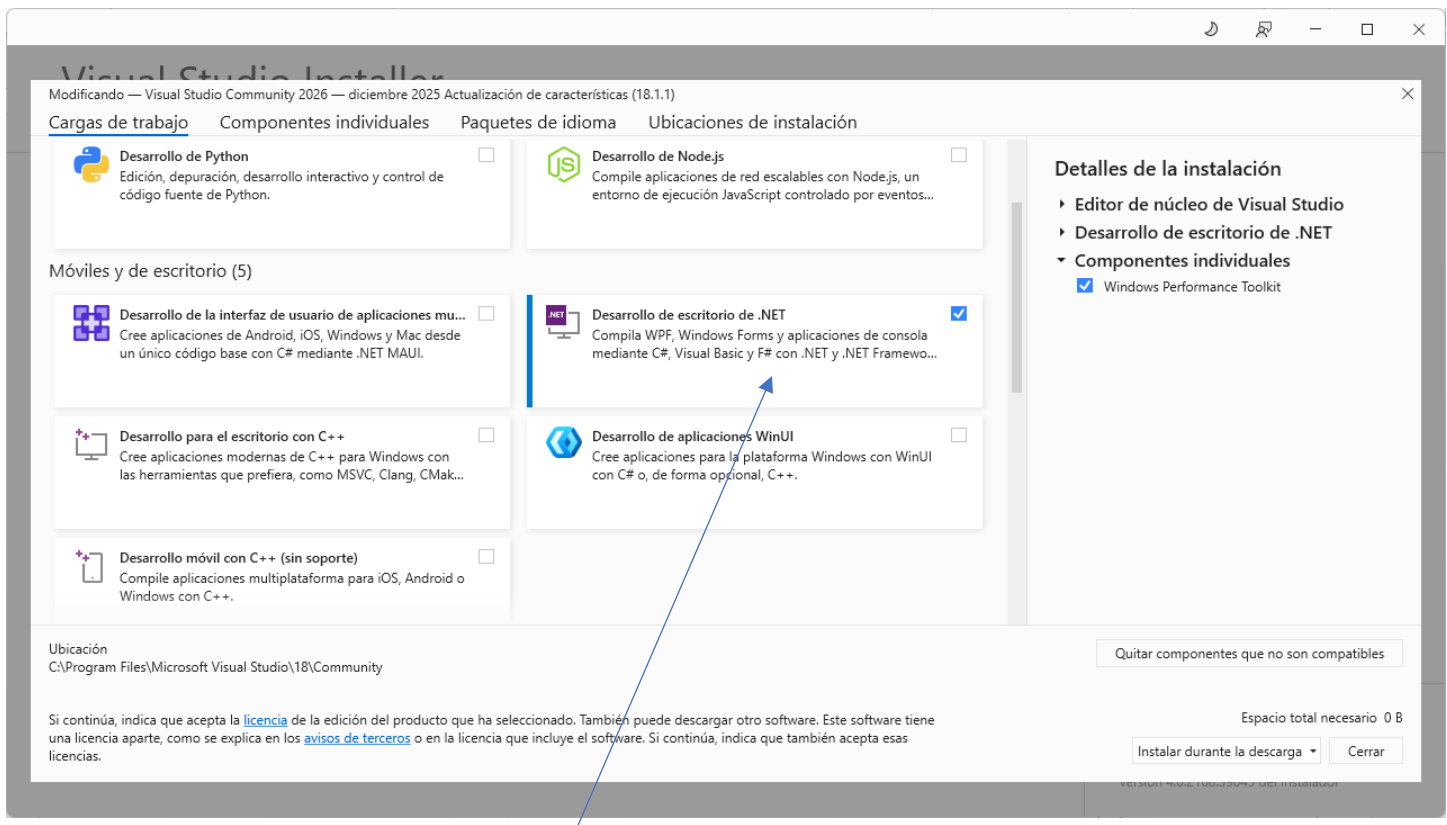


Ilustración 3: Instalar el "Desarrollo de escritorio de .NET"

Una vez instalado el entorno de desarrollo, al ejecutar aparece esta pantalla:



*Ilustración 4: Pantalla de inicio de Visual Studio 2026. Se presiona "Crear un proyecto"*

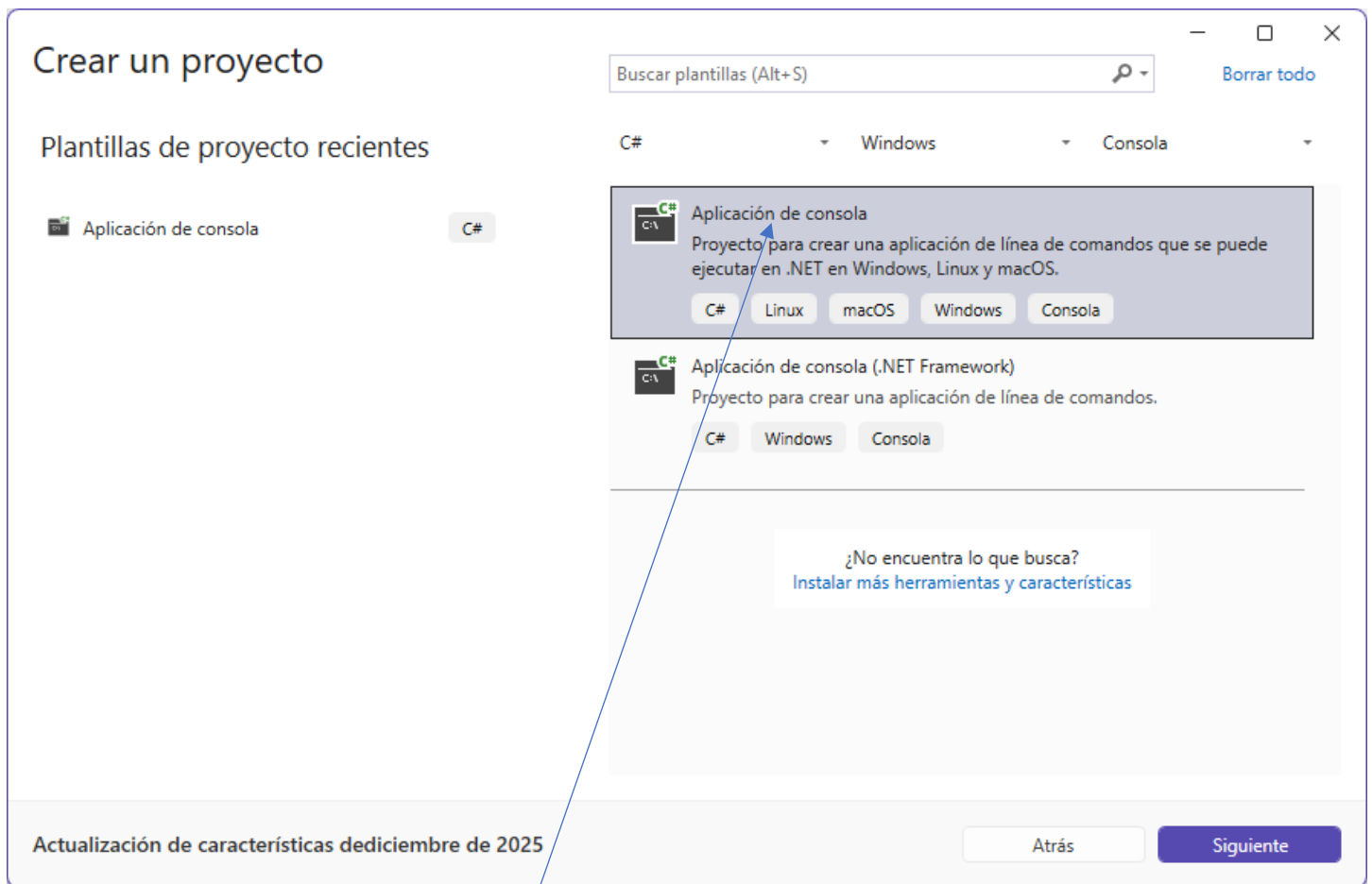


Ilustración 5: Se selecciona "Aplicación de consola"



—□×

# Configure su nuevo proyecto

Aplicación de consola

C#LinuxmacOSWindowsConsola

Nombre del proyecto

Ejemplo

Ubicación

C:\Users\engin\source\repos

...

Nombre de la solución ⓘ

Ejemplo

☐ Colocar la solución y el proyecto en el mismo directorio

Proyecto se creará en "C:\Users\engin\source\repos\Ejemplo\Ejemplo"

Actualización de características dediciembre de 2025

Atrás

Siguiente

Ilustración 6: Se le pone un nombre al proyecto.

Información adicional

Aplicación de consola C# Linux macOS Windows Console

Framework ⓘ

.NET 10.0 (Compatibilidad a largo plazo)

☒ No usar instrucciones de nivel superior ⓘ

☐ Habilitar publicación de AOT native ⓘ

Actualización de características dediciembre de 2025

Atrás Crear

Ilustración 7: Usar .NET 10 y se marca la opción "No usar instrucciones de nivel superior"

Es recomendado activar el "No usar instrucciones de nivel superior" para que en el código fuente aparezcan todas las instrucciones mínimas requeridas para que compile y ejecute un programa en C#.

En "Habilitar publicación de native AOT", se explica mejor aquí: <https://dev.to/bytehide/native-aot-the-future-of-net-app-development-47ha> y <https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/>

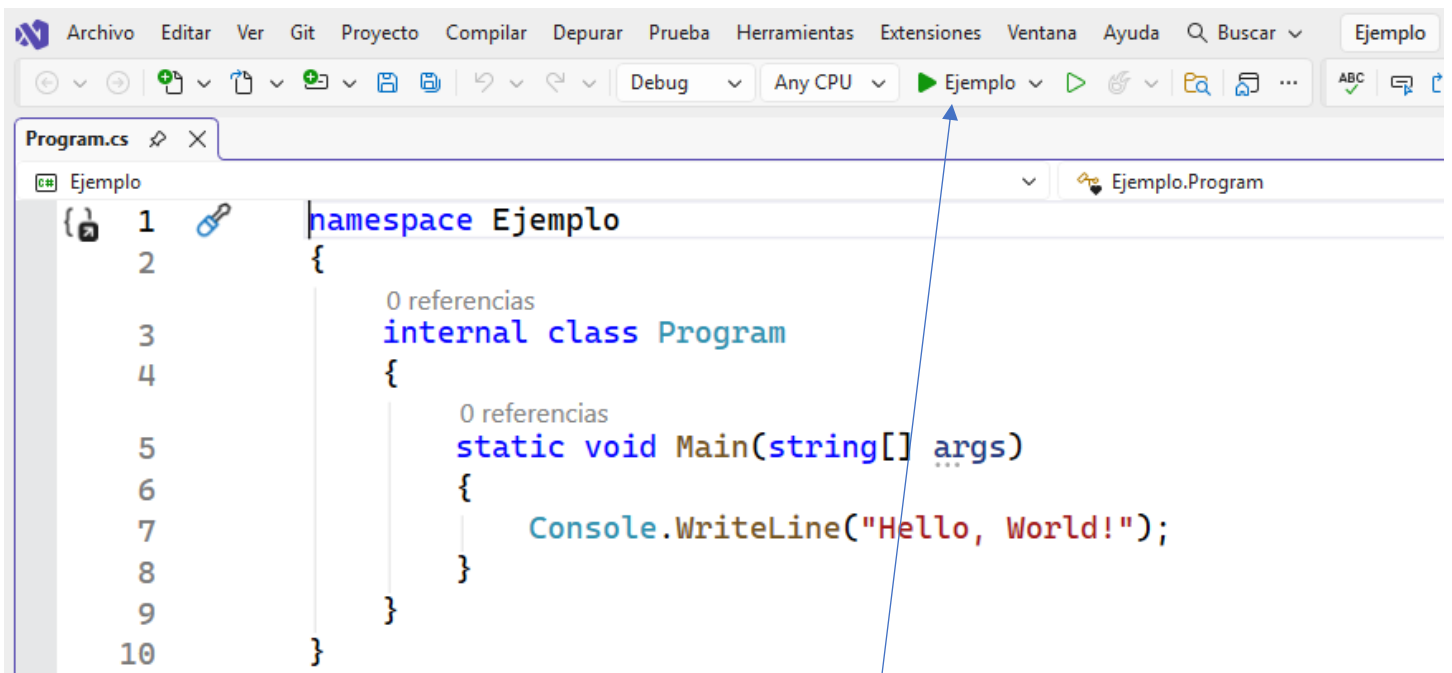


Ilustración 8: Se crea una aplicación por defecto. Se da clic en el botón de ejecutar

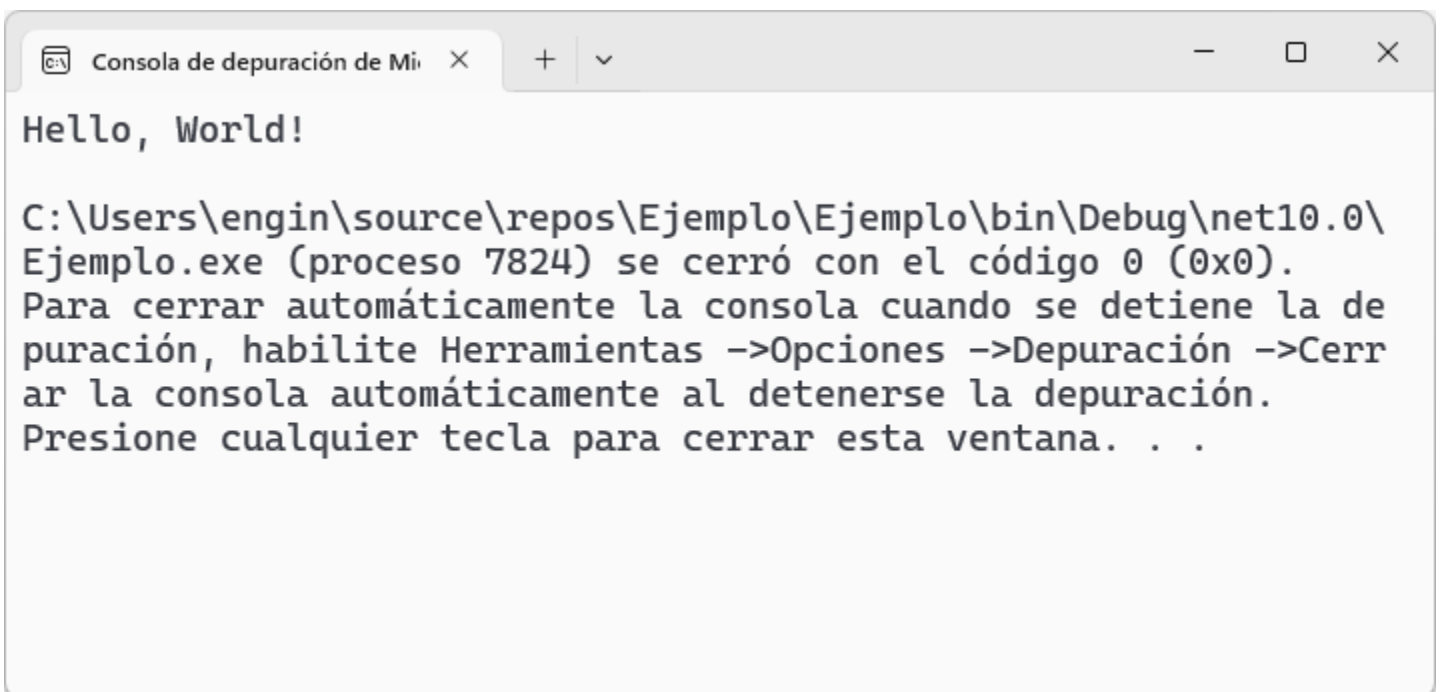
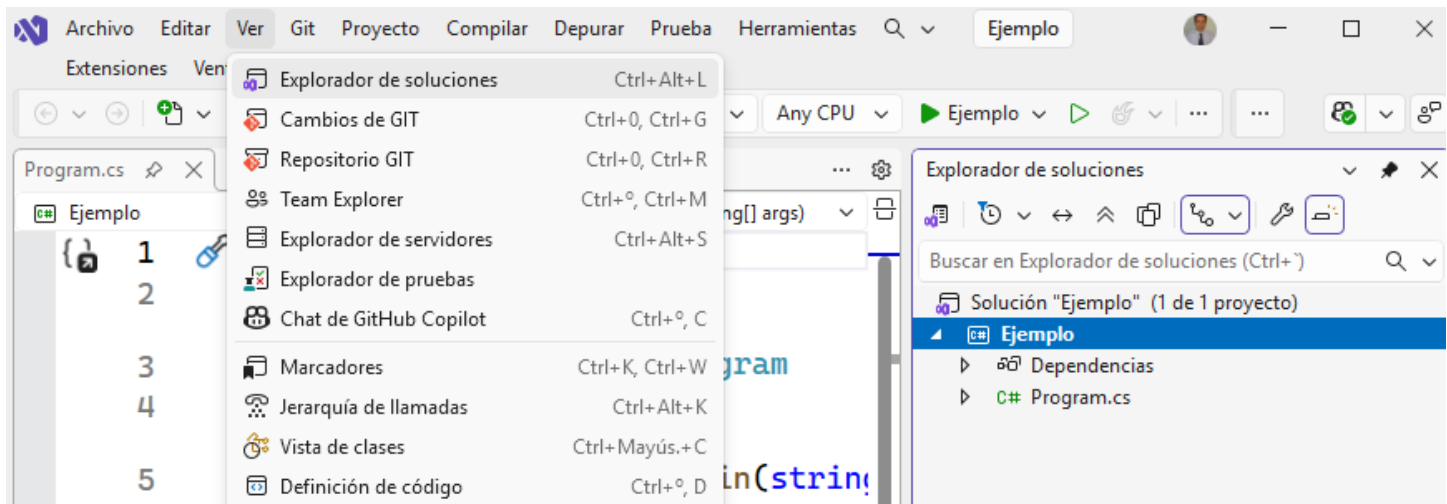


Ilustración 9: Ejecución del programa

## Explorador de soluciones

Una de las ventanas más usadas de ese entorno de desarrollo. Debe irse por “Ver” y luego “Explorador de soluciones”

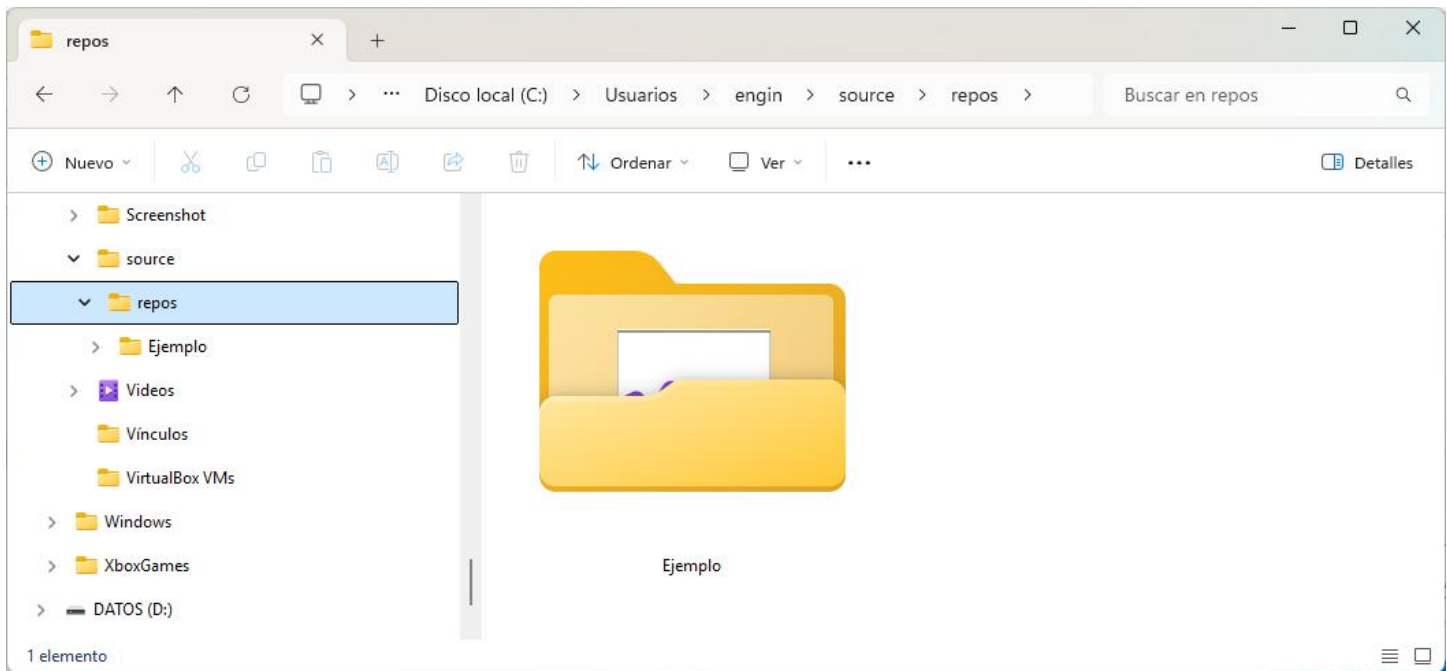


*Ilustración 10: Explorador de soluciones.*

En esa ventana estarán visibles los archivos de C# (extensión .cs)

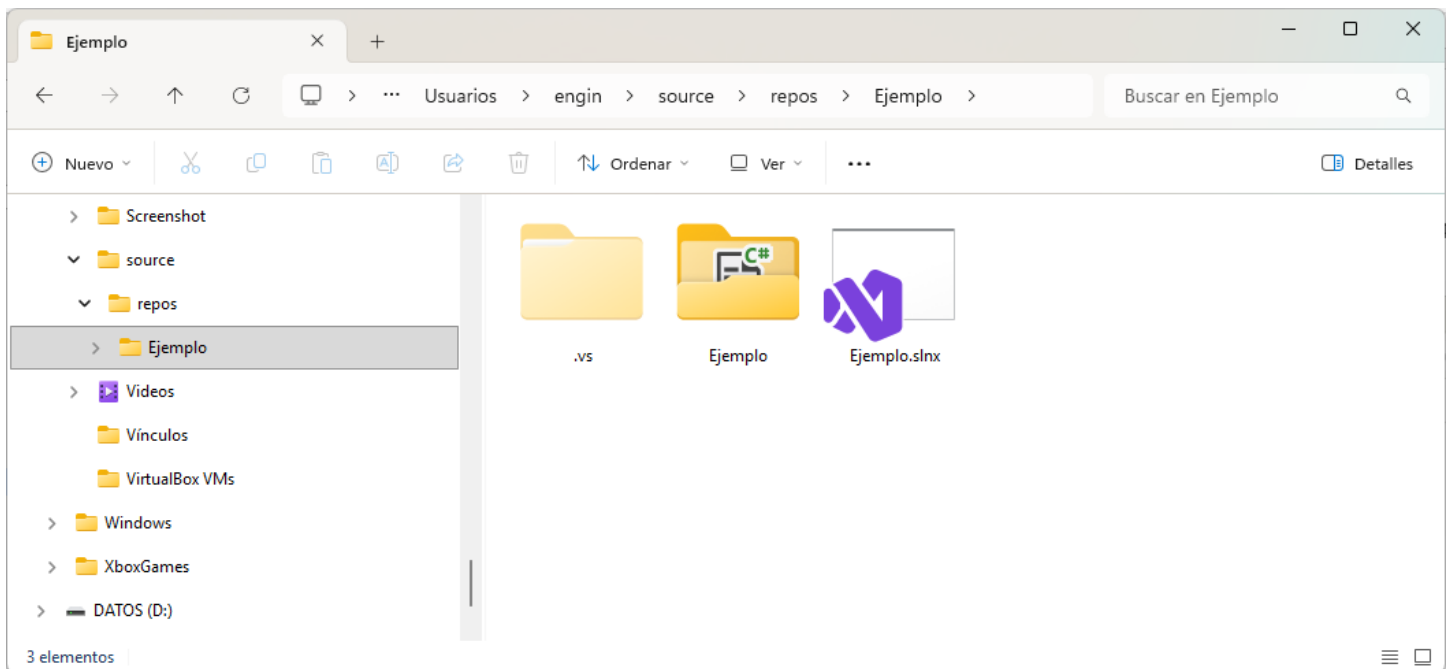
## Lugar donde se guardan los proyectos

Usualmente los proyectos se guardan en C:\Usuarios\**su\_usuario**\source\repos



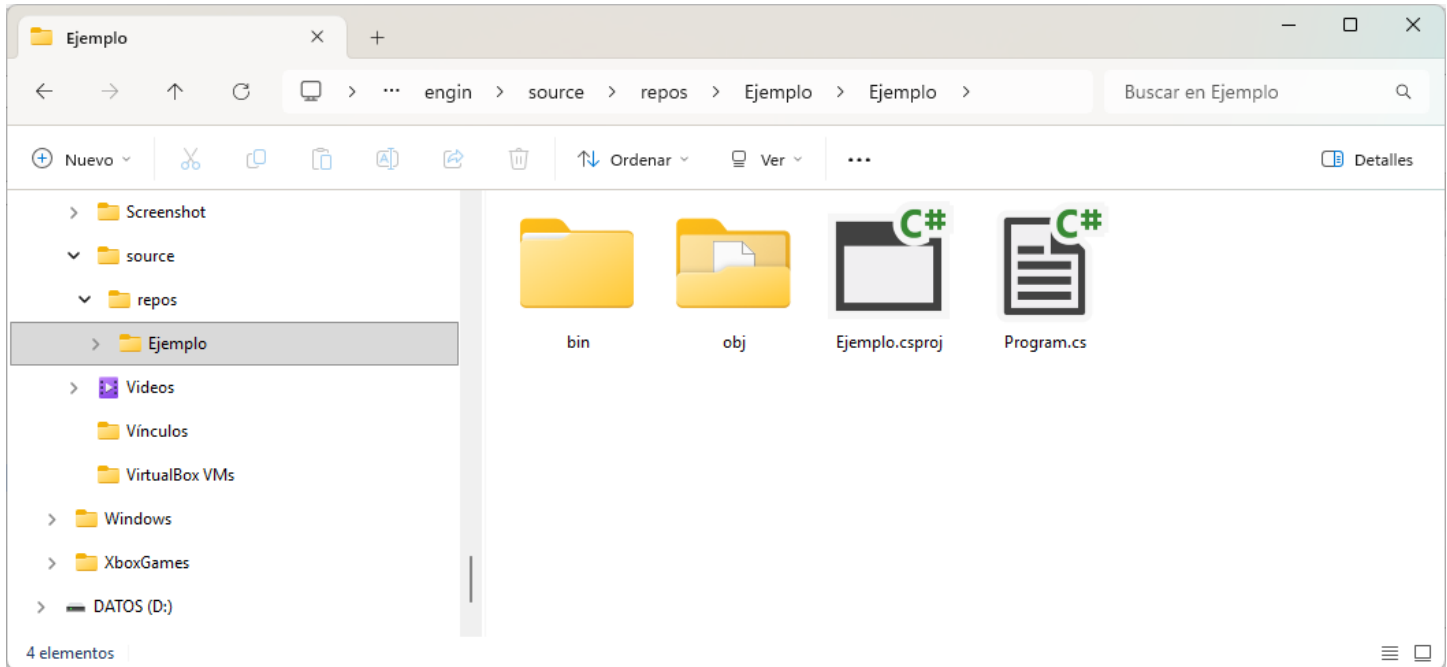
*Ilustración 11: Sitio donde se guardan los proyectos.*

En el interior de cada carpeta o directorio, se ve una serie de archivos.



*Ilustración 12: Archivos al interior de cada proyecto*

El archivo con extensión .slnx es el que requiere Visual Studio 2026 para poder abrir el proyecto. En la carpeta con el mismo nombre del proyecto está el código fuente.



*Ilustración 13: Código fuente.*

Para tener una copia del proyecto y los archivos requeridos, se comprime toda la carpeta `C:\Usuarios\su_usuario\source\repos\su_proyecto`

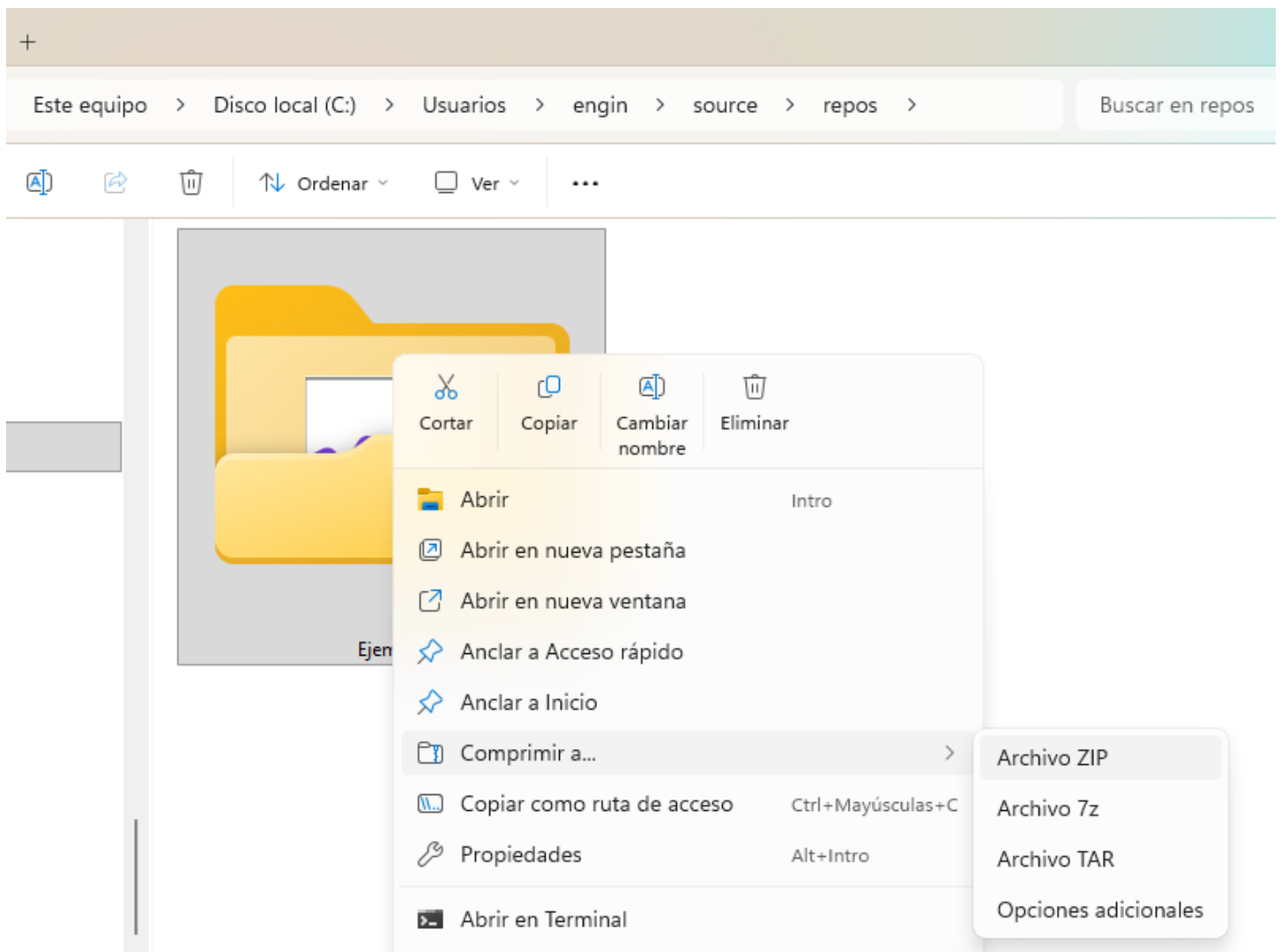
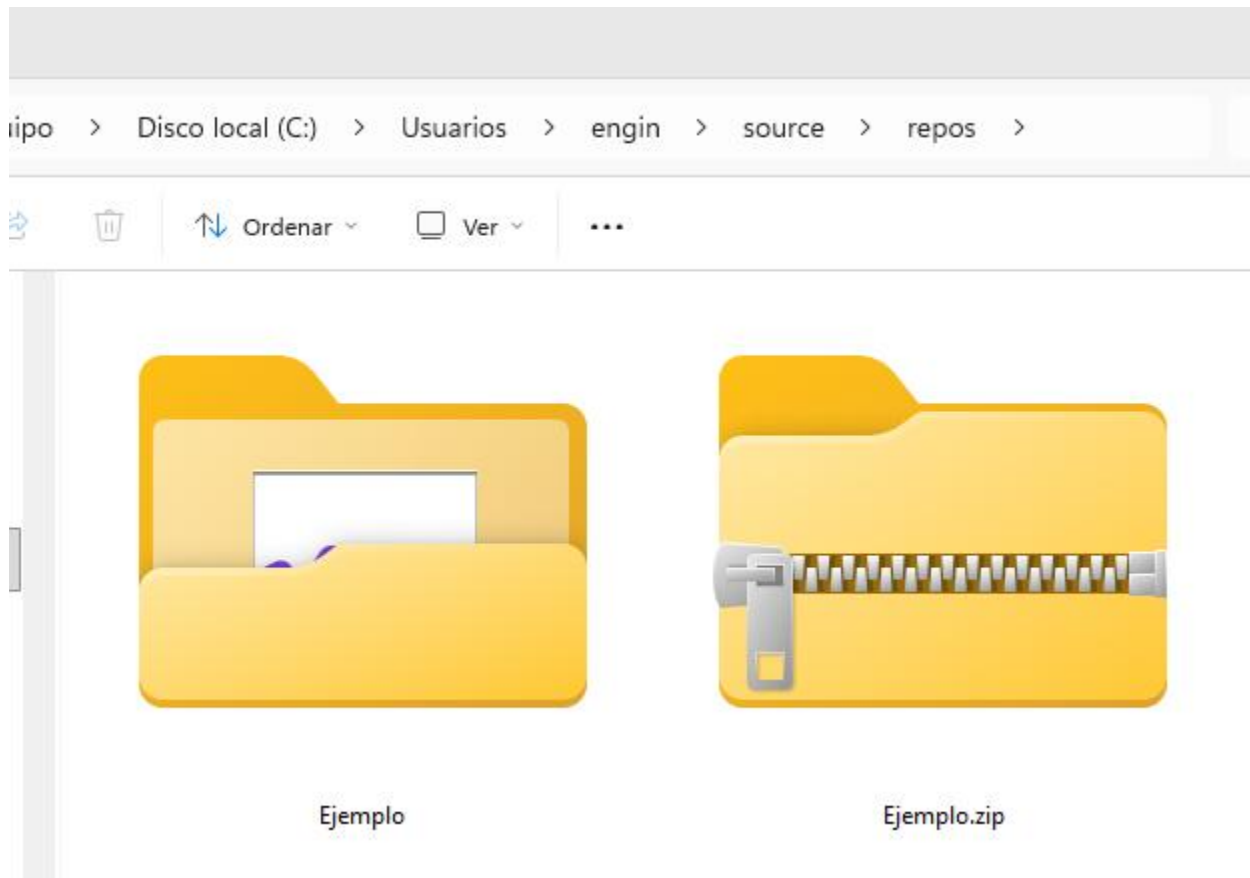


Ilustración 14: Comprimir carpeta

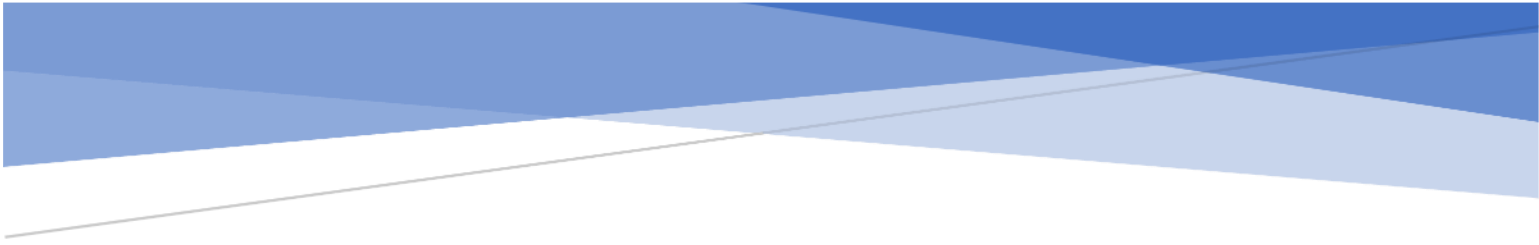
Y es esa carpeta comprimida la que se debe llevar a otro PC para poder seguir trabajando en el proyecto.



*Ilustración 15: Carpeta comprimida*

**¡OJO!** Cuando se lleve la carpeta comprimida a otro lado, debe primero ser descomprimida y así se puede seguir trabajando con Visual Studio 2026. **NO** se debe trabajar directamente con la carpeta comprimida.





# C# Y .NET 10

## Parte 01. Básico

2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                             |    |
|-------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                 | 4  |
| Acerca del autor.....                                       | 6  |
| Licencia de este libro .....                                | 6  |
| Licencia del software .....                                 | 6  |
| Marcas registradas .....                                    | 7  |
| Dedicatoria .....                                           | 8  |
| Comentarios en el código .....                              | 9  |
| El tradicional "Hola Mundo" .....                           | 10 |
| Variables .....                                             | 12 |
| Imprimiendo con formato .....                               | 18 |
| Operaciones matemáticas.....                                | 23 |
| Diferencias de precisión entre float, double y decimal..... | 25 |
| Función de potencia.....                                    | 26 |
| Orden de evaluación de los operadores.....                  | 27 |
| Problemas al usar cast .....                                | 28 |
| De formato algebraico a C# .....                            | 30 |
| Diferencias entre usar el cast y la conversión.....         | 32 |
| Conversión de números reales con el punto decimal .....     | 33 |
| Fallos en la conversión .....                               | 34 |
| Constantes matemáticas .....                                | 36 |
| Funciones trigonométricas .....                             | 38 |
| Funciones hiperbólicas .....                                | 39 |
| Otras funciones matemáticas .....                           | 40 |
| Manejo del NaN (Not a Number) y el infinito.....            | 42 |
| Leer un número por consola.....                             | 44 |
| Si condicional .....                                        | 45 |
| Uso del if...else, if...else if .....                       | 47 |
| Uso del AND &&.....                                         | 49 |
| Uso del OR    .....                                         | 50 |
| Uso del switch .....                                        | 51 |
| Operadores booleanos .....                                  | 52 |
| Precedencia de los operadores.....                          | 58 |
| El operador "?", un si condicional .....                    | 59 |

|                                                     |    |
|-----------------------------------------------------|----|
| Captura de error con try...catch.....               | 60 |
| Uso de TryParse para conversión.....                | 61 |
| Ciclo for.....                                      | 63 |
| Ciclo while .....                                   | 65 |
| Ciclo do...while .....                              | 67 |
| Romper ciclo con break .....                        | 69 |
| Uso del continue en ciclos.....                     | 70 |
| Ciclos anidados .....                               | 71 |
| Rompiendo ciclos anidados .....                     | 72 |
| Uso del goto.....                                   | 73 |
| Diferencias de precisión entre float y double ..... | 74 |
| Funciones .....                                     | 75 |
| Funciones con doble salida .....                    | 77 |
| Funciones iterativas y recursivas.....              | 79 |
| Ámbito de las variables .....                       | 80 |
| Manejo de cifras.....                               | 82 |

## Tabla de ilustraciones

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Ilustración 1: Ejecución del programa "Hola Mundo". Impresión en consola..... | 10 |
| Ilustración 2: Diferencia entre Write y Writeln.....                          | 11 |
| Ilustración 3: Variables de tipo entero .....                                 | 13 |
| Ilustración 4: Valores de tipo "double" .....                                 | 14 |
| Ilustración 5: Valores de tipo "char" .....                                   | 15 |
| Ilustración 6: Valores de tipo "string" .....                                 | 16 |
| Ilustración 7: Valores almacenados por variables de tipo "float" .....        | 17 |
| Ilustración 8: Números impresos con determinados formatos .....               | 19 |
| Ilustración 9: Impresión de números de tipo double con formato .....          | 20 |
| Ilustración 10: Valores tipo "double" con formato .....                       | 21 |
| Ilustración 11: Valores tipo "double" con formato de redondeo .....           | 22 |
| Ilustración 12: Resultados de operaciones .....                               | 24 |
| Ilustración 13: Diferencias entre float, double y decimal .....               | 25 |
| Ilustración 14: Función de potencia .....                                     | 26 |
| Ilustración 15: Orden de evaluación de los operadores .....                   | 27 |
| Ilustración 16: Problemas del uso del "cast" .....                            | 28 |
| Ilustración 17: Problemas del uso del "cast" .....                            | 29 |
| Ilustración 18: Formato algebraico a C# .....                                 | 31 |
| Ilustración 19: Diferencias entre usar el cast y la conversión .....          | 32 |
| Ilustración 20: Conversión de números reales con el punto decimal.....        | 33 |
| Ilustración 21: Mensaje de error .....                                        | 35 |
| Ilustración 22: Constantes matemáticas .....                                  | 37 |
| Ilustración 23: Funciones trigonométricas .....                               | 38 |
| Ilustración 24: Funciones hiperbólicas .....                                  | 39 |
| Ilustración 25: Otras funciones matemáticas .....                             | 41 |
| Ilustración 26: Manejo del NaN (Not a Number) y el infinito .....             | 43 |
| Ilustración 27: Leer un número por consola.....                               | 44 |
| Ilustración 28: Si condicional .....                                          | 46 |
| Ilustración 29: Uso de if...else .....                                        | 47 |
| Ilustración 30: Uso de if... else if ... else .....                           | 48 |
| Ilustración 31: Uso del AND && .....                                          | 49 |
| Ilustración 32: Uso del OR   .....                                            | 50 |
| Ilustración 33: Uso del switch .....                                          | 51 |
| Ilustración 34: Operadores booleanos.....                                     | 52 |
| Ilustración 35: Tablas de verdad .....                                        | 54 |
| Ilustración 36: Tablas de verdad usando && y    .....                         | 56 |
| Ilustración 37: Expresión booleana .....                                      | 57 |
| Ilustración 38: Precedencia de los operadores.....                            | 58 |
| Ilustración 40: Captura de error con try...catch.....                         | 60 |
| Ilustración 41: Uso de TryParse para conversión.....                          | 62 |
| Ilustración 42: Ciclo for.....                                                | 64 |
| Ilustración 43: Ciclo while.....                                              | 66 |
| Ilustración 44: Ciclo do...while .....                                        | 68 |
| Ilustración 45: Romper ciclo con break .....                                  | 69 |
| Ilustración 46: Uso del continue en ciclos.....                               | 70 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| Ilustración 47: Ciclos anidados .....                               | 71 |
| Ilustración 48: Rompiendo ciclos anidados .....                     | 72 |
| Ilustración 49: Uso del goto .....                                  | 73 |
| Ilustración 50: Diferencias de precisión entre float y double ..... | 74 |
| Ilustración 51: Función que retorna dato de tipo bool.....          | 75 |
| Ilustración 52: Funciones .....                                     | 76 |
| Ilustración 53: Funciones con doble salida.....                     | 77 |
| Ilustración 55: Funciones iterativas y recursivas.....              | 79 |
| Ilustración 56: Ámbito de las variables .....                       | 80 |
| Ilustración 57: Ámbito de las variables .....                       | 81 |
| Ilustración 58: Manejo de cifras.....                               | 91 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelif@hotmai.com](mailto:enginelif@hotmai.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Sonic,  
Meimei, Frac y mis recordados Sally, Capuchina, Tinita,  
Tammy, Vikingo y Michu.



## Comentarios en el código

En C#, similar a lenguajes como C, C++ o Java, los comentarios pueden ser de una línea usando los caracteres `//` o de varias líneas iniciando con `/*` y finalizando con `*/`

A/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Este es un comentario de una sola línea

        /* Este es un
           comentario de
           varias líneas */
    }
}
```

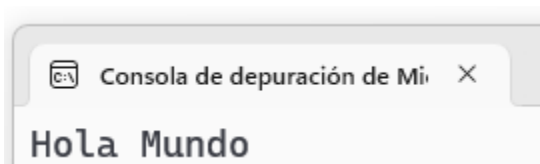
## El tradicional “Hola Mundo”

Para mostrar datos en la consola se utiliza la instrucción `Console.WriteLine` o `Console.Write`, la diferencia entre ambas instrucciones es que la primera imprime y hace un salto de renglón, la segunda no hace ese salto de renglón.

A/002.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Imprime en consola
        Console.Write("Hola Mundo");
    }
}
```

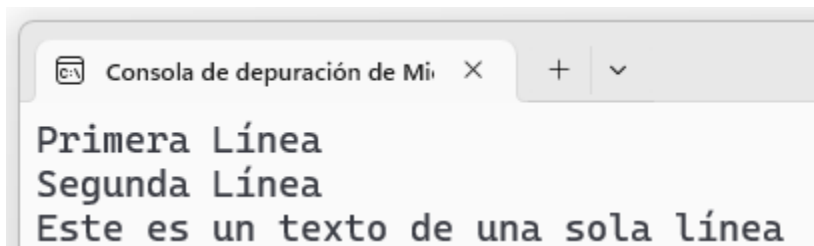


*Ilustración 1: Ejecución del programa “Hola Mundo”. Impresión en consola.*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Imprime en consola en dos líneas
        Console.WriteLine("Primera Línea");
        Console.WriteLine("Segunda Línea");

        //Imprime en la misma línea
        Console.Write("Este es un texto");
        Console.Write(" de una sola línea");
    }
}
```



*Ilustración 2: Diferencia entre Write y WriteLine*

## Variables

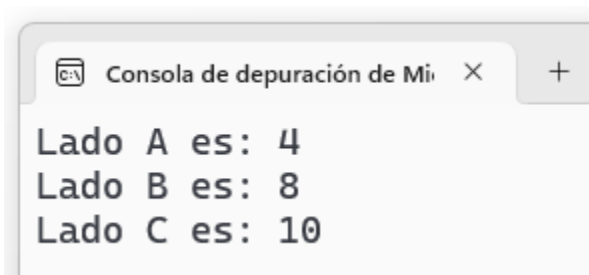
C# es un lenguaje de programación tipado, luego al crear las variables se debe poner su tipo:

| <b>Tipo</b> | <b>Descripción</b>                                                                                                                                                           | <b>Valor mínimo</b>  | <b>Valor máximo</b>  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------------|
| byte        | Número entero sin signo                                                                                                                                                      | 0                    | 255                  |
| sbyte       | Número entero con signo                                                                                                                                                      | -128                 | 127                  |
| short       | Número entero con signo                                                                                                                                                      | -32768               | 32767                |
| ushort      | Número entero sin signo                                                                                                                                                      | 0                    | 65535                |
| int         | Número entero con signo<br>(32 bits)                                                                                                                                         | -2147483648          | 2147483647           |
| uint        | Número entero sin signo<br>(32 bits)                                                                                                                                         | 0                    | 4294967295           |
| long        | Número entero con signo<br>(64 bits)                                                                                                                                         | -9223372036854775808 | 9223372036854775807  |
| ulong       | Número entero sin signo<br>(64 bits)                                                                                                                                         | 0                    | 18446744073709551615 |
| float       | Número real de precisión<br>simple (32 bits)                                                                                                                                 |                      |                      |
| double      | Número real de precisión<br>doble (64 bits)                                                                                                                                  |                      |                      |
| decimal     | Número real de precisión<br>más alta (128 bits)                                                                                                                              |                      |                      |
| char        | Carácter (1 byte)                                                                                                                                                            |                      |                      |
| bool        | Booleano (1 bit)                                                                                                                                                             |                      |                      |
| string      | Almacenamiento de<br>caracteres alfanuméricos                                                                                                                                |                      |                      |
| var         | Es una forma de crear<br>una variable local que no<br>requiere especificar el<br>tipo, ya que el<br>compilador de C# lo<br>deduce a partir de la<br>expresión que se asigna. |                      |                      |

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo entero
        int ladoA = 4;
        int ladoB = 8;
        int ladoC = 10;

        //Imprime esas variables
        Console.WriteLine("Lado A es: " + ladoA);
        Console.WriteLine("Lado B es: " + ladoB);
        Console.WriteLine("Lado C es: " + ladoC);
    }
}
```

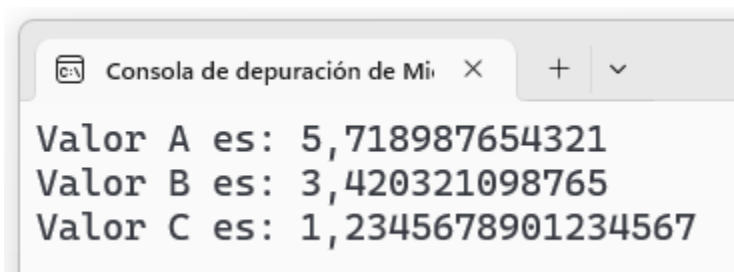


*Ilustración 3: Variables de tipo entero*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo double
        double valA = 5.718987654321;
        double valB = 3.420321098765;
        double valC = 1.2345678901234567890123;

        //Imprime esas variables
        Console.WriteLine("Valor A es: " + valA);
        Console.WriteLine("Valor B es: " + valB);
        Console.WriteLine("Valor C es: " + valC);
    }
}
```

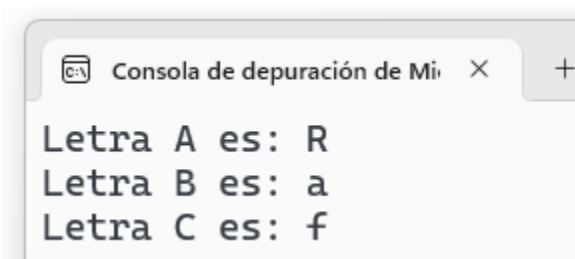


*Ilustración 4: Valores de tipo "double"*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo char
        char letraA = 'R';
        char letraB = 'a';
        char letraC = 'f';

        //Imprime esas variables
        Console.WriteLine("Letra A es: " + letraA);
        Console.WriteLine("Letra B es: " + letraB);
        Console.WriteLine("Letra C es: " + letraC);
    }
}
```



*Ilustración 5: Valores de tipo "char"*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo string
        string cadenaA = "Esta es una prueba";
        string cadenaB = "Programando en C#";
        string cadenaC = "Con Visual Studio 2026";

        //Imprime esas variables
        Console.WriteLine("Cadena A es: " + cadenaA);
        Console.WriteLine("Cadena B es: " + cadenaB);
        Console.WriteLine("Cadena C es: " + cadenaC);
    }
}
```

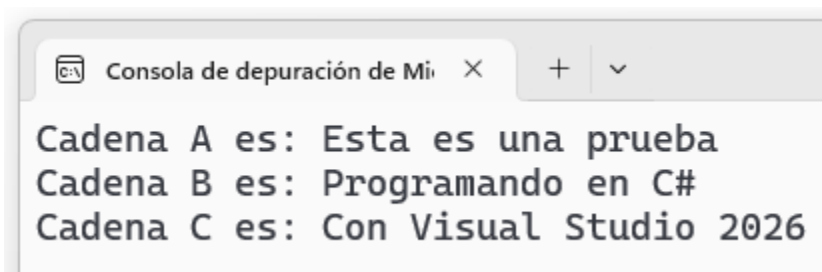


Ilustración 6: Valores de tipo "string"



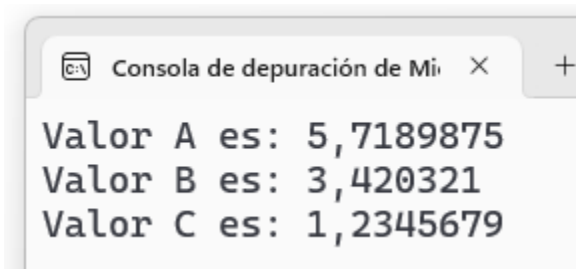
Uso de variables tipo float, como se puede observar, su precisión es baja en comparación con las variables de tipo double.

A/008.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo float.
        //Hay que usar el cast (float)
        float valA = (float)5.718987654321;
        float valB = (float)3.420321098765;
        float valC = (float)1.234567890123456789012;

        //Imprime esas variables
        Console.WriteLine("Valor A es: " + valA);
        Console.WriteLine("Valor B es: " + valB);
        Console.WriteLine("Valor C es: " + valC);
    }
}
```



*Ilustración 7: Valores almacenados por variables de tipo "float"*

Por lo tanto, se recomienda hacer uso mejor de variables tipo double.

## Imprimiendo con formato

Se requiere imprimir los números en consola con un determinado formato. Este es el código:

A/009.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo double
        double valA = 5.718987654321;
        double valB = -3.420821098765;
        double valC = 1.234567890123456789;
        double valD = 7.8;

        //Imprime con un formato de al menos dos decimales
        Console.WriteLine("Valor A es: {0:0.00}", valA);

        //Imprime con un formato de al menos tres decimales
        Console.WriteLine("Valor B es: {0:0.000}", valB);

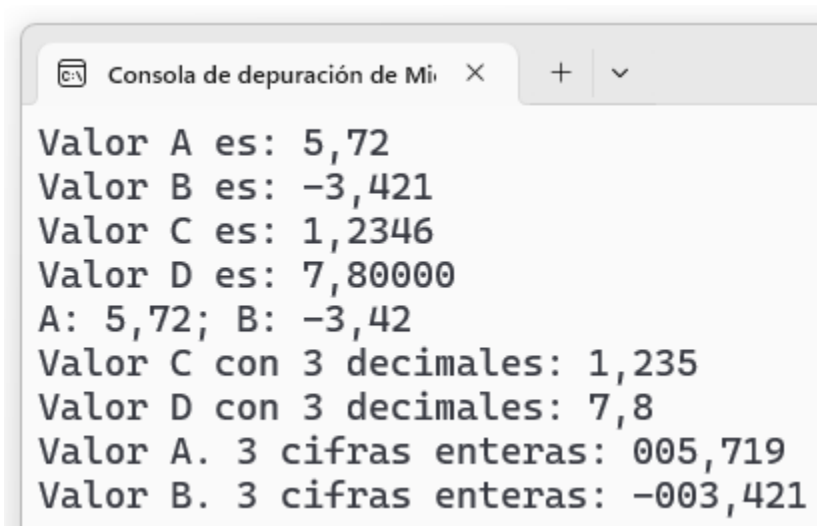
        //Imprime con un formato de al menos cuatro decimales
        Console.WriteLine("Valor C es: {0:0.0000}", valC);

        //Imprime con un formato de al menos cinco decimales
        Console.WriteLine("Valor D es: {0:0.00000}", valD);

        /* Imprime dos variables.
         * Ver el inicio {N: donde N es la posición de la variable
         * en la instrucción de impresión iniciando en cero. */
        Console.WriteLine("A: {0:0.00}; B: {1:0.00}", valA, valB);

        /* Máximo tres decimales */
        Console.WriteLine("Valor C con 3 decimales: {0:0.###}", valC);
        Console.WriteLine("Valor D con 3 decimales: {0:0.###}", valD);

        /* Mínimo tres cifras antes del punto decimal */
        Console.WriteLine("Valor A. 3 cifras enteras: {0:000.###}", valA);
        Console.WriteLine("Valor B. 3 cifras enteras: {0:000.###}", valB);
    }
}
```



Consola de depuración de Mi X + v

```
Valor A es: 5,72
Valor B es: -3,421
Valor C es: 1,2346
Valor D es: 7,80000
A: 5,72; B: -3,42
Valor C con 3 decimales: 1,235
Valor D con 3 decimales: 7,8
Valor A. 3 cifras enteras: 005,719
Valor B. 3 cifras enteras: -003,421
```

*Ilustración 8: Números impresos con determinados formatos*

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo double
        double valA = 17902.8421;
        double valB = -871901372.420821098765;
        double valC = 89341759342.1678;

        //Imprime con un formato de miles
        Console.WriteLine("Impresión con formato de miles y 3 decimales");
        Console.WriteLine("Valor A: {0:0,0.0}", valA);
        Console.WriteLine("Valor A: {0:0,0.000}", valA);

        Console.WriteLine("\r\nValor B: {0:0,0.0}", valB);
        Console.WriteLine("Valor B: {0:0,0.000}", valB);

        Console.WriteLine("\r\nValor C: {0:0,0.0}", valC);
        Console.WriteLine("Valor C: {0:0,0.000}", valC);
    }
}

```

```

Consola de depuración de Mi
Impresión con formato de miles y 3 decimales
Valor A: 17.902,8
Valor A: 17.902,842

Valor B: -871.901.372,4
Valor B: -871.901.372,421

Valor C: 89.341.759.342,2
Valor C: 89.341.759.342,168

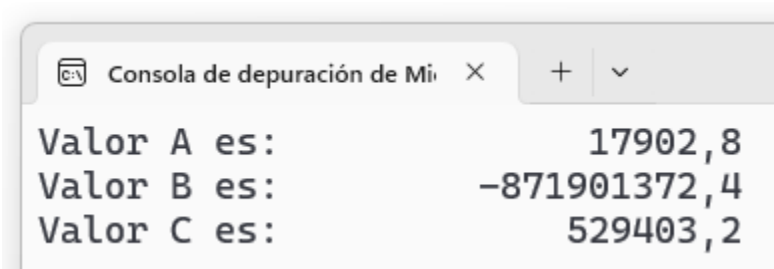
```

*Ilustración 9: Impresión de números de tipo double con formato*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo double
        double valA = 17902.8421;
        double valB = -871901372.420821098765;
        double valC = 529403.1678;

        //Imprime con alineación a la derecha
        //20 espacios para poner el número
        Console.WriteLine("Valor A es: {0,20:0.0}", valA);
        Console.WriteLine("Valor B es: {0,20:0.0}", valB);
        Console.WriteLine("Valor C es: {0,20:0.0}", valC);
    }
}
```



Consola de depuración de Mi... × + v

Valor A es: 17902,8  
Valor B es: -871901372,4  
Valor C es: 529403,2

*Ilustración 10: Valores tipo "double" con formato*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declara variables de tipo double
        double valA = 2.76;
        double valB = 2.04;
        double valC = 0.14;

        //Imprime solo si la cifra es significativa
        //redondea también.
        Console.WriteLine("Valor A (redondea): {0:#.##}", valA);
        Console.WriteLine("Valor B (redondea): {0:#.##}", valB);
        Console.WriteLine("Valor C (redondea): {0:#.##}", valC);
    }
}
```

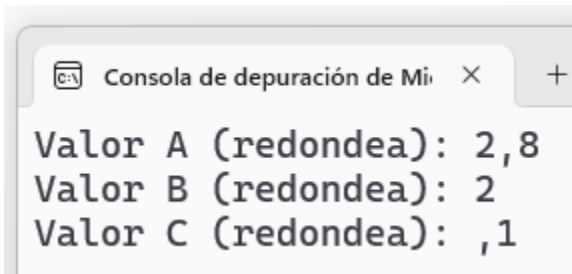


Ilustración 11: Valores tipo "double" con formato de redondeo

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Operador suma
        double valA = 1 + 2 + 3;

        //Operador resta
        double valB = 4 - 5 - 6;

        //Operador multiplicación
        double valC = 7 * 8 * 9;

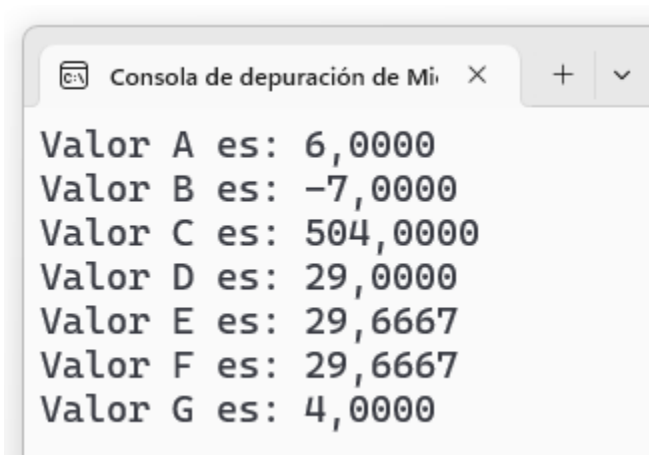
        //Operador división usando enteros
        double valD = 178 / 2 / 3;

        //Operador división usando números reales
        double valE = 178 / 2.0 / 3.0;

        //Operador división usando números enteros haciendo cast
        double valF = (double)178 / 2 / 3;

        //Operador división modular
        double valG = 70 % 6;

        Console.WriteLine("Valor A es: {0:0.0000}", valA);
        Console.WriteLine("Valor B es: {0:0.0000}", valB);
        Console.WriteLine("Valor C es: {0:0.0000}", valC);
        Console.WriteLine("Valor D es: {0:0.0000}", valD);
        Console.WriteLine("Valor E es: {0:0.0000}", valE);
        Console.WriteLine("Valor F es: {0:0.0000}", valF);
        Console.WriteLine("Valor G es: {0:0.0000}", valG);
    }
}
```



A screenshot of a C# debug console window titled 'Consola de depuración de Mi'. The window contains the following text:

```
Valor A es: 6,0000  
Valor B es: -7,0000  
Valor C es: 504,0000  
Valor D es: 29,0000  
Valor E es: 29,6667  
Valor F es: 29,6667  
Valor G es: 4,0000
```

*Ilustración 12: Resultados de operaciones*

En valD el resultado es un entero porque C# lo toma como si fuese una división entera de números enteros por lo que el resultado es entero. En cambio, valE y valF retornan el resultado real, el primero por usar notación de número real (el uso del punto) y el otro por usar cast (double).



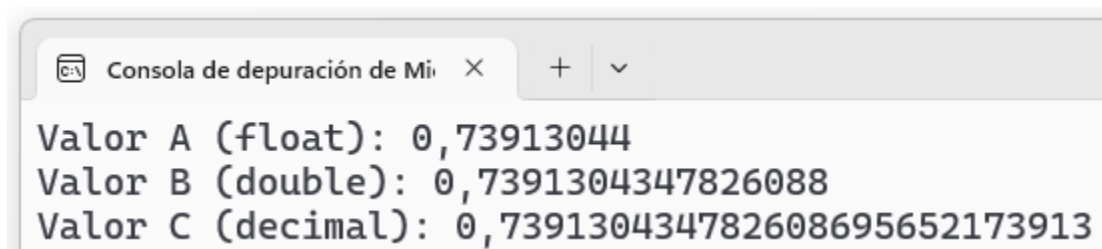
# Diferencias de precisión entre float, double y decimal

A/014.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Compara la precisión entre float y double
        float valA = (float) (1.7 / 2.3);
        double valB = 1.7 / 2.3;
        decimal valC = (decimal)1.7 / (decimal)2.3;

        Console.WriteLine("Valor A (float): " + valA);
        Console.WriteLine("Valor B (double): " + valB);
        Console.WriteLine("Valor C (decimal): " + valC);
    }
}
```



```
Consola de depuración de Mi...
Valor A (float): 0,73913044
Valor B (double): 0,7391304347826088
Valor C (decimal): 0,739130434782608695652173913
```

*Ilustración 13: Diferencias entre float, double y decimal*

# Función de potencia

A/015.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Potencia (^)
        double valA = Math.Pow(2, 5);

        //Raíz cúbica de 729
        double valB = Math.Pow(729, (double)1 / 3);

        //Raíz cuadrada de 64
        double valC = Math.Pow(64, (double)1 / 2);

        //  $4^{(-2)} = 1/(4^2) = 1/16$ 
        double valD = Math.Pow(4, -2);

        //  $4^0$ 
        double valE = Math.Pow(4, 0);

        Console.WriteLine("Valor A es: " + valA);
        Console.WriteLine("Valor B es: " + valB);
        Console.WriteLine("Valor C es: " + valC);
        Console.WriteLine("Valor D es: " + valD);
        Console.WriteLine("Valor E es: " + valE);
    }
}
```

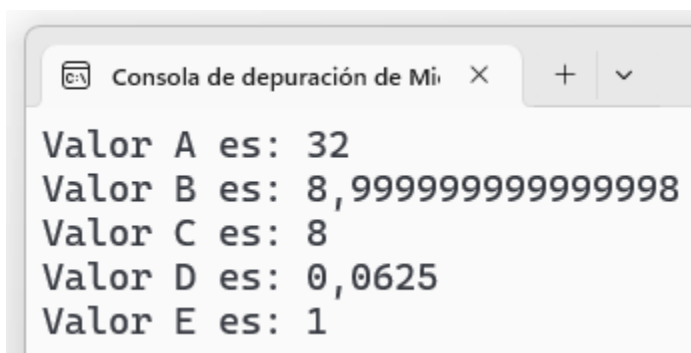


Ilustración 14: Función de potencia

# Orden de evaluación de los operadores

A/016.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Orden de evaluación de los operadores:
        * Primero potencia
        * Segundo multiplicación y división
        * Tercero suma y resta
        * */
        double valA = (double)7 - 1 + 2 * Math.Pow(2, 5);
        double valB = (double)1 + 2 * 3 / 4 - 5;
        double valC = (double)3 + 5 - 2 * 4 / 8;
        double valD = (double)3 * 2 + Math.Pow(3, 2);

        Console.WriteLine("Valor A: " + valA);
        Console.WriteLine("Valor B: " + valB);
        Console.WriteLine("Valor C: " + valC);
        Console.WriteLine("Valor D: " + valD);
    }
}
```

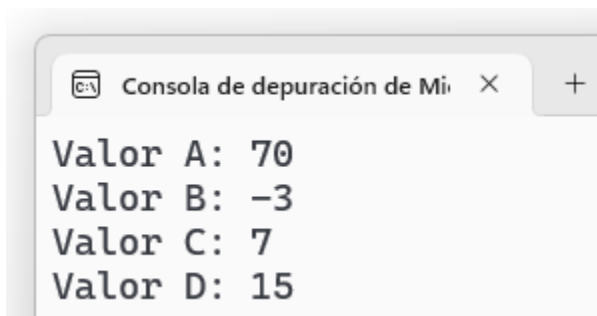


Ilustración 15: Orden de evaluación de los operadores

## Problemas al usar cast

A/017.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Es mejor usar reales que el cast
        double valA = 9.0 / 2.0 - 1.0 + 2.0 * Math.Pow(2.0, 5.0) / 4.0;
        double valB = (double)9 / 2 - 1 + 2 * Math.Pow(2, 5) / 4;

        /*Falla porque se interpreta completamente
        como enteros 9 / 2 es 4 en división entera */
        double valC = 9 / 2 - 1 + 2 * Math.Pow(2, 5) / 4;

        /* Falla el cast porque primero se interpreta la expresión
        como entera y luego se pasa a double */
        double valD = (double)(9 / 2 - 1 + 2 * Math.Pow(2, 5) / 4);

        Console.WriteLine("Valor A es: " + valA);
        Console.WriteLine("Valor B es: " + valB);
        Console.WriteLine("Valor C es: " + valC);
        Console.WriteLine("Valor D es: " + valD);
    }
}
```

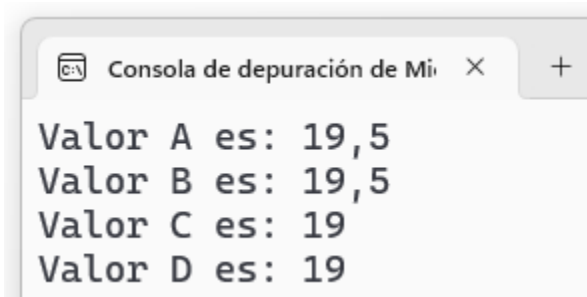


Ilustración 16: Problemas del uso del "cast"

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Falla porque se interpreta como operación de enteros
        double valA = (4 / 3) + (1 / 2) - (5 / 4);

        //Falla porque el cast solo cubre la primera operación
        double valB = (double)4 / 3 + (1 / 2) - (5 / 4);

        //Operación correcta
        double valC = (4.0 / 3.0) + (1.0 / 2.0) - (5.0 / 4.0);

        //Operación correcta pero se deben usar varios cast
        double valD = (double)4 / 3 + (double)1 / 2 - (double)5 / 4;

        //Falla porque el cast solo cubre la primera operación
        double valE = (double)4 / 3 + 1 / 2 - 5 / 4;

        Console.WriteLine("Valor A: " + valA);
        Console.WriteLine("Valor B: " + valB);
        Console.WriteLine("Valor C: " + valC);
        Console.WriteLine("Valor D: " + valD);
        Console.WriteLine("Valor E: " + valE);
    }
}

```

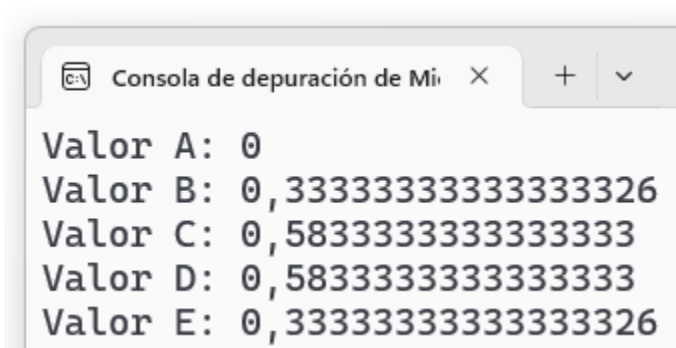


Ilustración 17: Problemas del uso del "cast"

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Pasar correctamente ecuaciones a formato en C#
        double X = 5.7;
        double Y;

        //Ejemplos
        Y = 3 / (X - 1);
        Console.WriteLine("Valor Y es: " + Y);

        Y = 3 / (X - 1) + 2;
        Console.WriteLine("Valor Y es: " + Y);

        Y = 3 / X + X / 7;
        Console.WriteLine("Valor Y es: " + Y);

        Y = 3 / (X * X);
        Console.WriteLine("Valor Y es: " + Y);

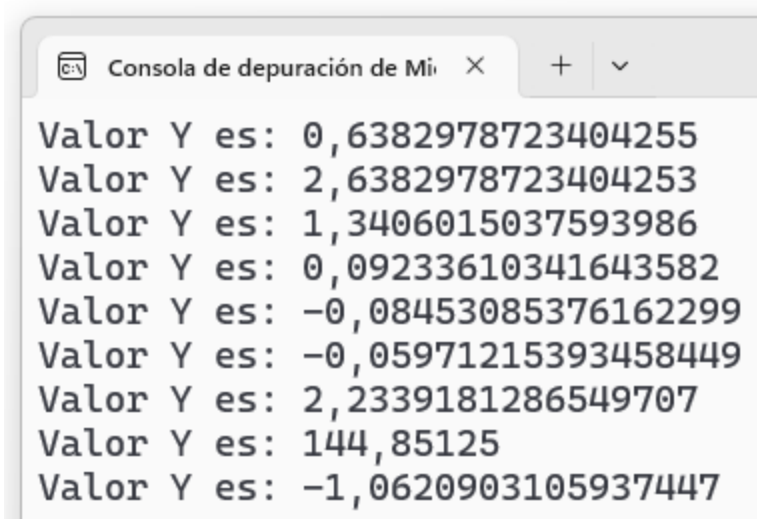
        Y = 3 / (2 - (X * X + 5));
        Console.WriteLine("Valor Y es: " + Y);

        Y = (5 + X) / (6 - X * X * X);
        Console.WriteLine("Valor Y es: " + Y);

        Y = 2 + ((4 / X) / 3);
        Console.WriteLine("Valor Y es: " + Y);

        Y = 5 * ((X * X / 4) / (3 / (X + 5)));
        Console.WriteLine("Valor Y es: " + Y);

        Y = ((X * X) + ((2 * X - 5) / (X + 1))) / (1 - X * X);
        Console.WriteLine("Valor Y es: " + Y);
    }
}
```



A screenshot of a C# console window titled "Consola de depuración de Mi". The window displays eight lines of output, each starting with "Valor Y es:". The values are formatted using the 'G' format specifier, which uses commas as decimal separators and scientific notation for large or small numbers. The values are: 0,6382978723404255; 2,6382978723404253; 1,3406015037593986; 0,09233610341643582; -0,08453085376162299; -0,05971215393458449; 2,2339181286549707; and 144,85125. The last value, 144,85125, is not in scientific notation.

```
Valor Y es: 0,6382978723404255
Valor Y es: 2,6382978723404253
Valor Y es: 1,3406015037593986
Valor Y es: 0,09233610341643582
Valor Y es: -0,08453085376162299
Valor Y es: -0,05971215393458449
Valor Y es: 2,2339181286549707
Valor Y es: 144,85125
Valor Y es: -1,0620903105937447
```

*Ilustración 18: Formato algebraico a C#*

# Diferencias entre usar el cast y la conversión

A/020.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Conversión vs Cast
        double valA = 15.7;

        int valB = Convert.ToInt32(valA); //Redondea
        int valC = (int)valA; //Trunca

        //Ejemplos
        Console.WriteLine("Valor B: " + valB);
        Console.WriteLine("Valor C: " + valC);
    }
}
```

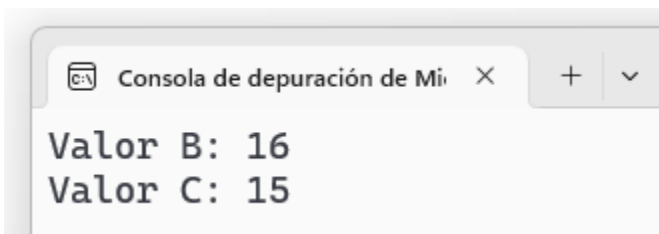


Ilustración 19: Diferencias entre usar el cast y la conversión



# Conversión de números reales con el punto decimal

A/021.cs

```
using System.Globalization;

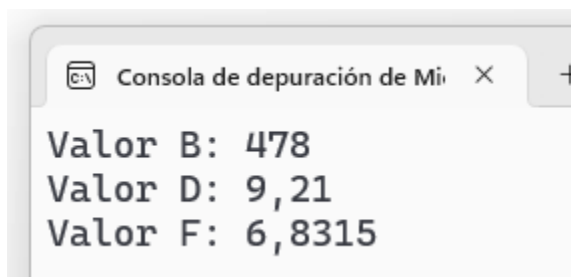
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Conversión de reales
        string valA = "4.78";

        //Da a problemas porque se ignora el punto decimal
        double valB = Convert.ToDouble(valA);
        Console.WriteLine("Valor B: " + valB);

        //Aquí si funciona la conversión
        string valC = "9,21";
        double valD = Convert.ToDouble(valC);
        Console.WriteLine("Valor D: " + valD);

        //Para usar la conversión con punto decimal,
        //se debe hacer uso de CultureInfo
        string valE = "6.8315";
        double valF = double.Parse(valE, CultureInfo.InvariantCulture);
        Console.WriteLine("Valor F: " + valF);
    }
}
```



*Ilustración 20: Conversión de números reales con el punto decimal*

```
using System.Globalization;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fallos de conversión
        string valA = ""; //Una cadena nula o vacía daña la conversión

        //El programa se cae
        double valB = Convert.ToDouble(valA);
        Console.WriteLine("Valor B es: " + valB);

        //El programa se cae
        double valF = double.Parse(valA, CultureInfo.InvariantCulture);
        Console.WriteLine("Valor F es: " + valF);
    }
}
```

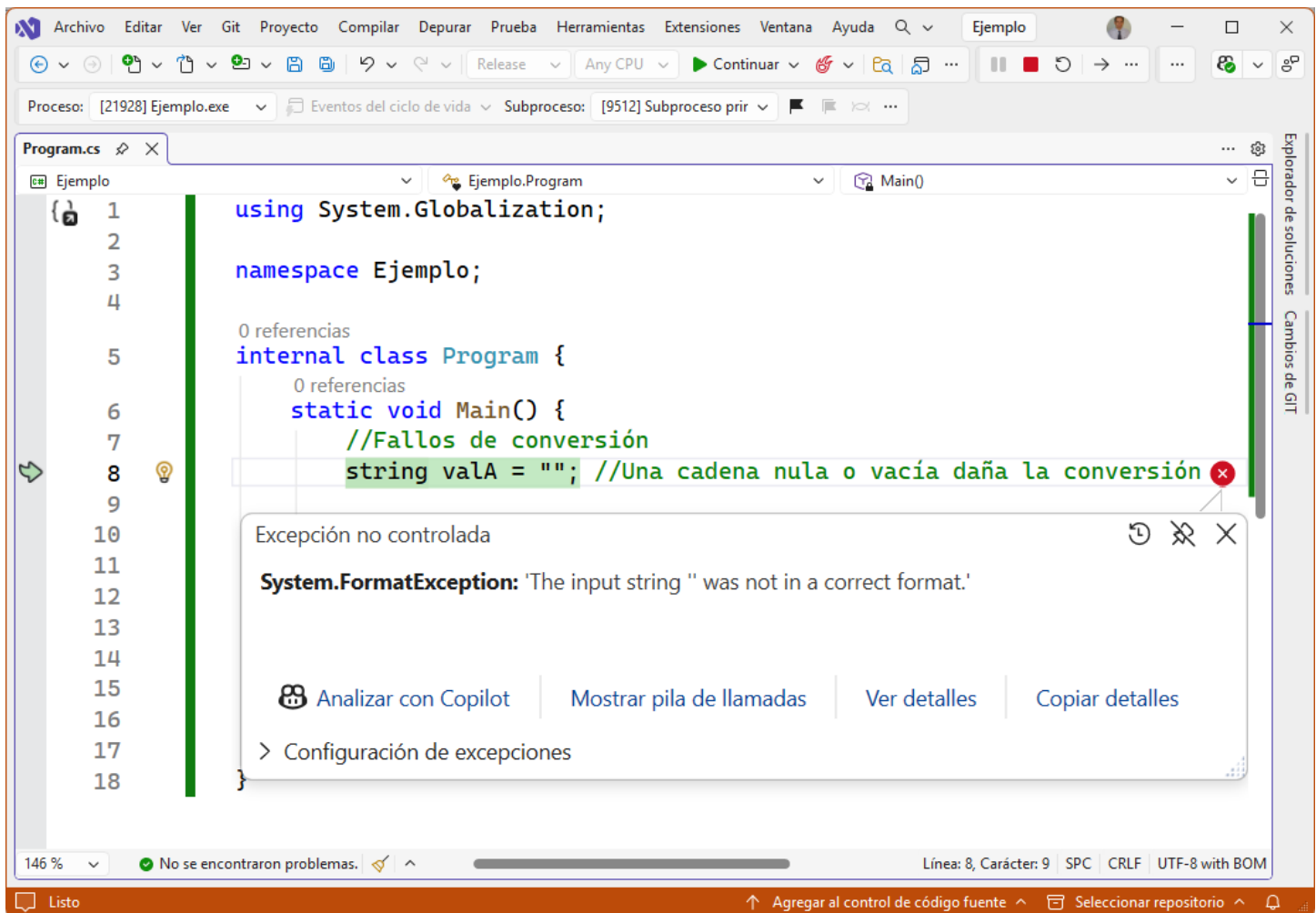


Ilustración 21: Mensaje de error

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Constantes matemáticas
        double valA = Math.PI;
        double valB = Math.E;
        Console.WriteLine("PI es: " + valA);
        Console.WriteLine("E es: " + valB);

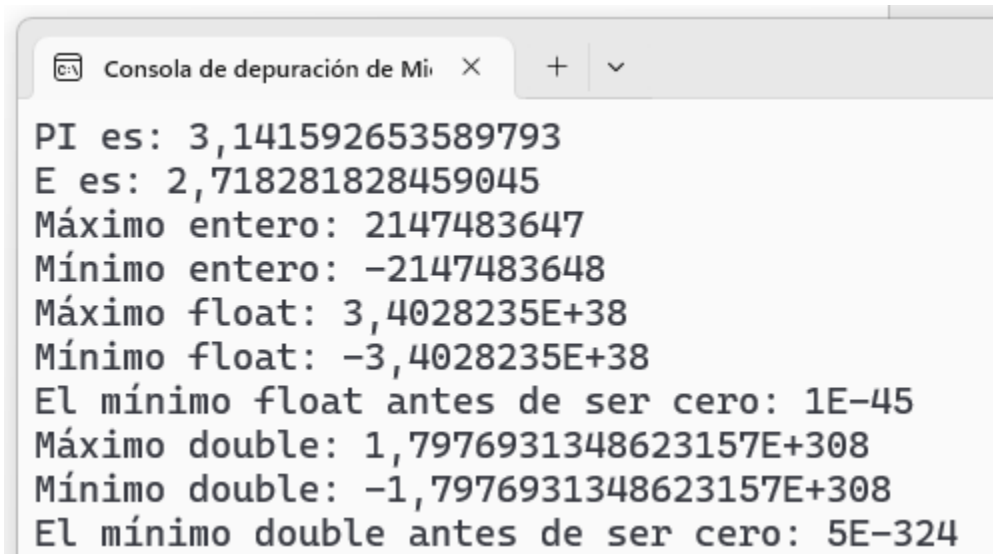
        //Máximo y mínimo valor entero
        int maximoEntero = int.MaxValue;
        int minimoEntero = int.MinValue;
        Console.WriteLine("Máximo entero: " + maximoEntero);
        Console.WriteLine("Mínimo entero: " + minimoEntero);

        //Máximo y mínimo valor float
        float maximofloat = float.MaxValue;
        float minimofloat = float.MinValue;
        float minimoCero = float.Epsilon; //El mínimo valor antes de ser cero
        Console.WriteLine("Máximo float: " + maximofloat);
        Console.WriteLine("Mínimo float: " + minimofloat);
        Console.WriteLine("El mínimo float antes de ser cero: " + minimoCero);

        //Máximo y mínimo valor double
        double maximodouble = double.MaxValue;
        double minimodouble = double.MinValue;

        //El mínimo valor antes de ser cero
        double CeroE = double.Epsilon;

        Console.WriteLine("Máximo double: " + maximodouble);
        Console.WriteLine("Mínimo double: " + minimodouble);
        Console.WriteLine("El mínimo double antes de ser cero: " + CeroE);
    }
}
```



The image shows a screenshot of a debug console window. The title bar at the top reads "Consola de depuración de Mi" followed by a close button (X) and a dropdown menu with a plus sign and a downward arrow. The console contains the following text:

```
PI es: 3,141592653589793
E es: 2,718281828459045
Máximo entero: 2147483647
Mínimo entero: -2147483648
Máximo float: 3,4028235E+38
Mínimo float: -3,4028235E+38
El mínimo float antes de ser cero: 1E-45
Máximo double: 1,7976931348623157E+308
Mínimo double: -1,7976931348623157E+308
El mínimo double antes de ser cero: 5E-324
```

*Ilustración 22: Constantes matemáticas*

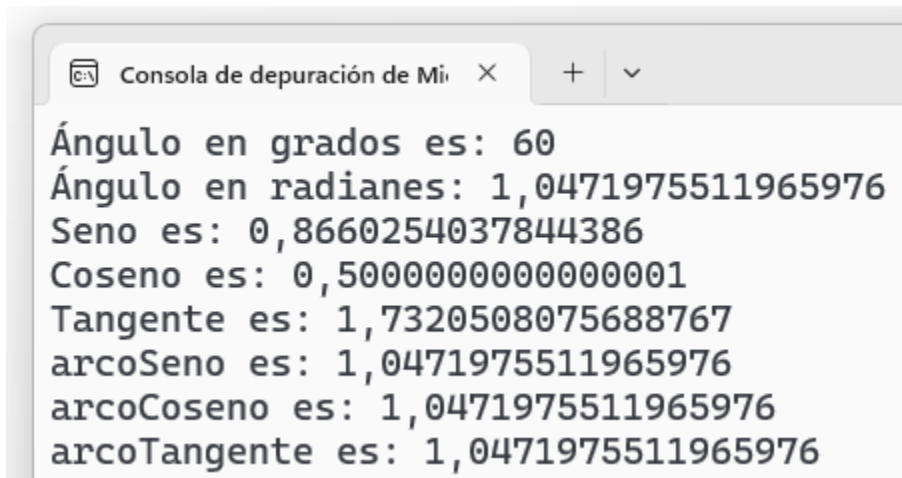
```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Funciones trigonométricas
        double anguloGrados = 60;
        double anguloRadian = anguloGrados * Math.PI / 180;

        double valSeno = Math.Sin(anguloRadian);
        double valCoseno = Math.Cos(anguloRadian);
        double valTangente = Math.Tan(anguloRadian);

        double arcoSeno = Math.Asin(valSeno);
        double arcoCoseno = Math.Acos(valCoseno);
        double arcoTangente = Math.Atan(valTangente);

        Console.WriteLine("Ángulo en grados es: " + anguloGrados);
        Console.WriteLine("Ángulo en radianes: " + anguloRadian);
        Console.WriteLine("Seno es: " + valSeno);
        Console.WriteLine("Coseno es: " + valCoseno);
        Console.WriteLine("Tangente es: " + valTangente);
        Console.WriteLine("arcoSeno es: " + arcoSeno);
        Console.WriteLine("arcoCoseno es: " + arcoCoseno);
        Console.WriteLine("arcoTangente es: " + arcoTangente);
    }
}
```



Consola de depuración de Mi... X + v

Ángulo en grados es: 60  
Ángulo en radianes: 1,0471975511965976  
Seno es: 0,8660254037844386  
Coseno es: 0,5000000000000001  
Tangente es: 1,7320508075688767  
arcoSeno es: 1,0471975511965976  
arcoCoseno es: 1,0471975511965976  
arcoTangente es: 1,0471975511965976

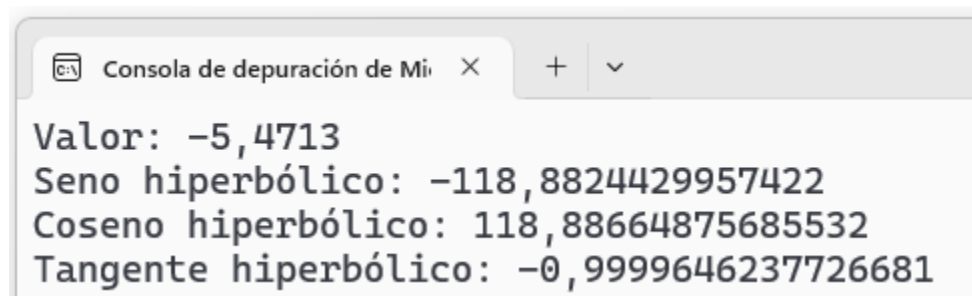
Ilustración 23: Funciones trigonométricas

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Funciones hiperbólicas
        double valorReal = -5.4713;

        double valSenoH = Math.Sinh(valorReal);
        double valCosenoH = Math.Cosh(valorReal);
        double valTangenteH = Math.Tanh(valorReal);

        Console.WriteLine("Valor: " + valorReal);
        Console.WriteLine("Seno hiperbólico: " + valSenoH);
        Console.WriteLine("Coseno hiperbólico: " + valCosenoH);
        Console.WriteLine("Tangente hiperbólico: " + valTangenteH);
    }
}
```



Consola de depuración de Mi... X + v

Valor: -5,4713  
Seno hiperbólico: -118,8824429957422  
Coseno hiperbólico: 118,88664875685532  
Tangente hiperbólico: -0,9999646237726681

Ilustración 24: Funciones hiperbólicas

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Otras funciones matemáticas
        double valorReal = 5.0713;
        double valAbsoluto = Math.Abs(valorReal); //valor absoluto
        double valTecho = Math.Ceiling(valorReal); //entero superior
        double valExponencial = Math.Exp(valorReal);
        double valPiso = Math.Floor(valorReal);
        double valLogaritmoNatural = Math.Log(valorReal);
        double valLogaritmoBase10 = Math.Log10(valorReal);
        double valRedondea = Math.Round(valorReal);
        double valSigno = Math.Sign(valorReal);
        double valRaizC = Math.Sqrt(valorReal); //raiz cuadrada
        double valTrunca = Math.Truncate(valorReal);

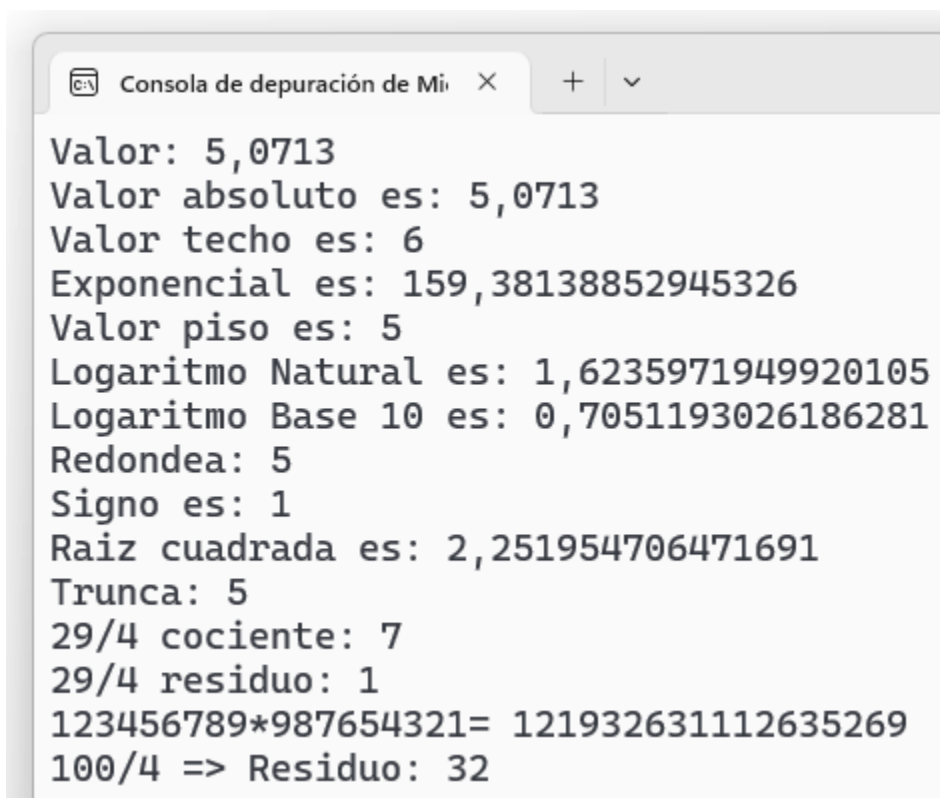
        Console.WriteLine("Valor: " + valorReal);
        Console.WriteLine("Valor absoluto es: " + valAbsoluto);
        Console.WriteLine("Valor techo es: " + valTecho);
        Console.WriteLine("Exponencial es: " + valExponencial);
        Console.WriteLine("Valor piso es: " + valPiso);
        Console.WriteLine("Logaritmo Natural es: " + valLogaritmoNatural);
        Console.WriteLine("Logaritmo Base 10 es: " + valLogaritmoBase10);
        Console.WriteLine("Redondea: " + valRedondea);
        Console.WriteLine("Signo es: " + valSigno);
        Console.WriteLine("Raiz cuadrada es: " + valRaizC);
        Console.WriteLine("Trunca: " + valTrunca);

        //Funciones con salidas
        var (cociente, residuo) = Math.DivRem(29, 4);
        Console.WriteLine("29/4 cociente: " + cociente);
        Console.WriteLine("29/4 residuo: " + residuo);

        //Multiplicación enorme
        long valMultiplica = Math.BigMul(123456789, 987654321);
        Console.WriteLine("123456789*987654321= " + valMultiplica);

        //División modular
        decimal dividendo = 100, divisor = 34;
        decimal Otroresiduo = dividendo % divisor;
        Console.WriteLine("100/4 => Residuo: " + Otroresiduo);
    }
}
```





A screenshot of a web browser's developer console. The console has a tab titled 'Consola de depuración de Mi' with a close button (X) and expand/collapse icons (+ and v). The console displays the following text:

```
Valor: 5,0713
Valor absoluto es: 5,0713
Valor techo es: 6
Exponencial es: 159,38138852945326
Valor piso es: 5
Logaritmo Natural es: 1,6235971949920105
Logaritmo Base 10 es: 0,7051193026186281
Redondea: 5
Signo es: 1
Raiz cuadrada es: 2,251954706471691
Trunca: 5
29/4 cociente: 7
29/4 residuo: 1
123456789*987654321= 121932631112635269
100/4 => Residuo: 32
```

*Ilustración 25: Otras funciones matemáticas*

## Manejo del NaN (Not a Number) y el infinito

Cuando algunas operaciones matemáticas generan error (por ejemplo, raíz cuadrada de un número negativo), la función retorna NaN (Not a Number). También hay operaciones que dan infinito como la división entre cero, en ese caso en consola se muestra como si fuese el número 8 (ocho), y es más bien un infinito girado 90 grados.

A/027.cs

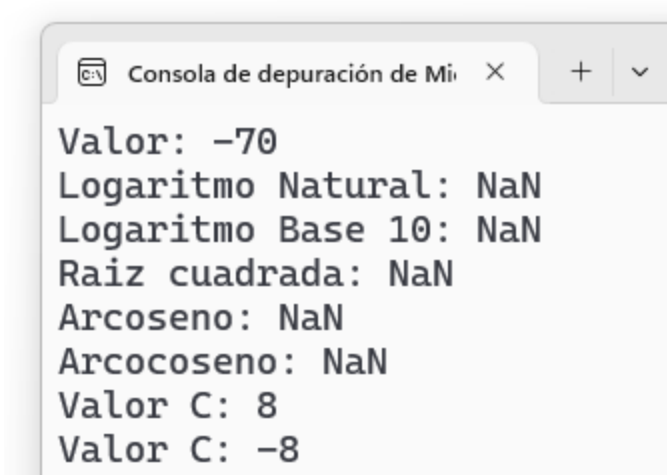
```
namespace Ejemplo;

internal class Program {
    static void Main() {
        // Genera un NaN: Not a Number
        double valorReal = -70;
        double valLogaritmoNatural = Math.Log(valorReal);
        double valLogaritmoBase10 = Math.Log10(valorReal);
        double valRaizC = Math.Sqrt(valorReal); //raiz cuadrada
        double arcoSeno = Math.Asin(valorReal);
        double arcoCoseno = Math.Acos(valorReal);

        Console.WriteLine("Valor: " + valorReal);
        Console.WriteLine("Logaritmo Natural: " + valLogaritmoNatural);
        Console.WriteLine("Logaritmo Base 10: " + valLogaritmoBase10);
        Console.WriteLine("Raiz cuadrada: " + valRaizC);
        Console.WriteLine("Arcoseno: " + arcoSeno);
        Console.WriteLine("Arcocoseno: " + arcoCoseno);

        //Genera infinito positivo
        float valA = 10;
        float valB = 0;
        float valC = valA / valB;
        Console.WriteLine("Valor C: " + valC);

        //Genera infinito negativo
        valA = -10;
        valB = 0;
        valC = valA / valB;
        Console.WriteLine("Valor C: " + valC);
    }
}
```



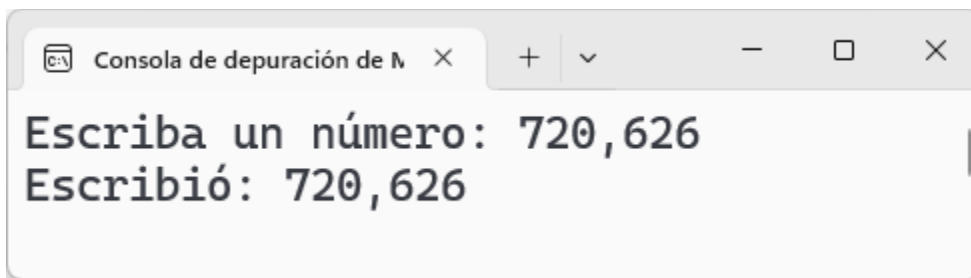
*Ilustración 26: Manejo del NaN (Not a Number) y el infinito*

## Leer un número por consola

A/028.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Leer un número por consola
        Console.Write("Escriba un número: ");
        double valorReal = Convert.ToDouble(Console.ReadLine());
        Console.WriteLine("Escribió: " + valorReal);
    }
}
```



*Ilustración 27: Leer un número por consola*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee dos números por consola
        Console.Write("Escriba un primer número: ");
        double valorA = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba un segundo número: ");
        double valorB = Convert.ToDouble(Console.ReadLine());

        //Si condicional
        if (valorA > valorB) {
            Console.WriteLine(valorA + " es mayor que " + valorB);
        }

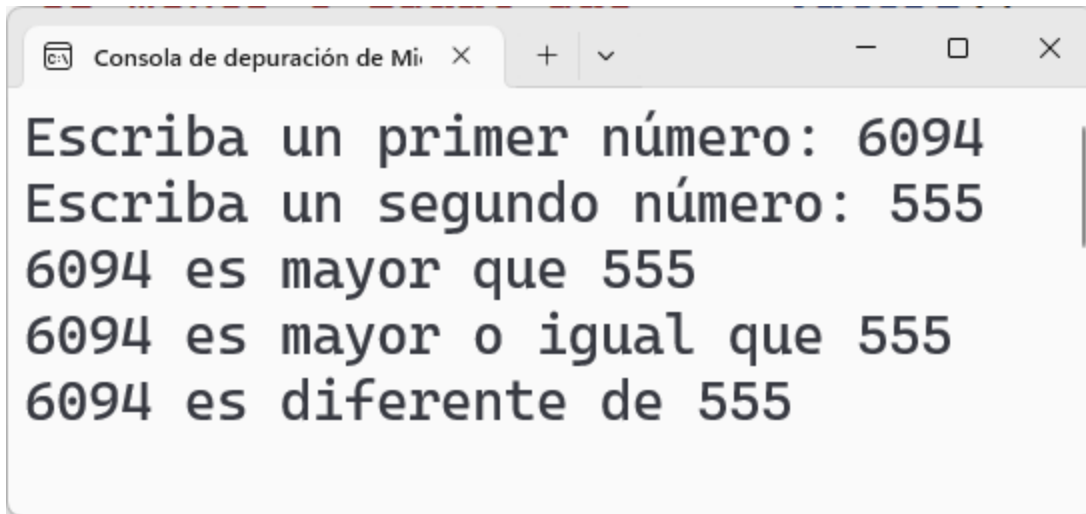
        if (valorA >= valorB) {
            Console.WriteLine(valorA + " es mayor o igual que " + valorB);
        }

        if (valorA < valorB) {
            Console.WriteLine(valorA + " es menor que " + valorB);
        }

        if (valorA <= valorB) {
            Console.WriteLine(valorA + " es menor o igual que " + valorB);
        }

        if (valorA == valorB) {
            Console.WriteLine(valorA + " es igual a " + valorB);
        }

        if (valorA != valorB) {
            Console.WriteLine(valorA + " es diferente de " + valorB);
        }
    }
}
```



```
Consola de depuración de Mi X + v - □ X  
Escriba un primer número: 6094  
Escriba un segundo número: 555  
6094 es mayor que 555  
6094 es mayor o igual que 555  
6094 es diferente de 555
```

*Ilustración 28: Si condicional*

## Uso del if...else, if...else if

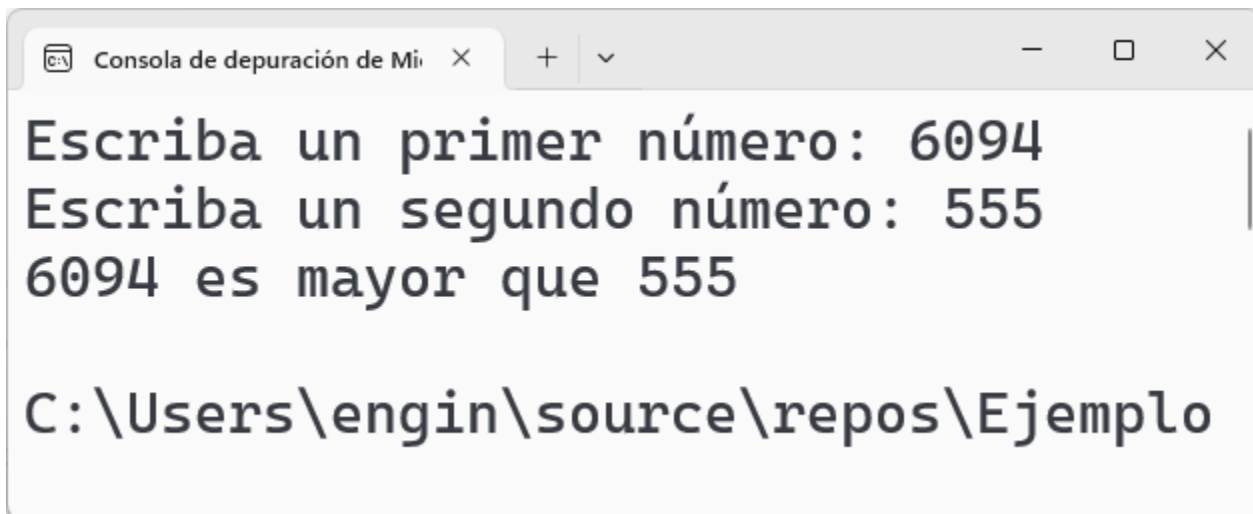
A/030.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee dos números por consola
        Console.Write("Escriba un primer número: ");
        double valorA = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba un segundo número: ");
        double valorB = Convert.ToDouble(Console.ReadLine());

        //Si condicional
        if (valorA > valorB) {
            Console.WriteLine(valorA + " es mayor que " + valorB);
        }
        else { //de lo contrario
            Console.WriteLine(valorA + " es menor o igual que " + valorB);
        }
    }
}
```



```
Consola de depuración de Mi  X  +  v  -  □  X

Escriba un primer número: 6094
Escriba un segundo número: 555
6094 es mayor que 555

C:\Users\engin\source\repos\Ejemplo
```

Ilustración 29: Uso de if...else

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee dos números por consola
        Console.Write("Escriba un primer número: ");
        double valorA = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba un segundo número: ");
        double valorB = Convert.ToDouble(Console.ReadLine());

        //Si condicional
        if (valorA > valorB) {
            Console.WriteLine(valorA + " es mayor que " + valorB);
        }
        else if (valorA < valorB) {
            Console.WriteLine(valorA + " es menor que " + valorB);
        }
        else {
            Console.WriteLine(valorA + " es igual a " + valorB);
        }
    }
}
```

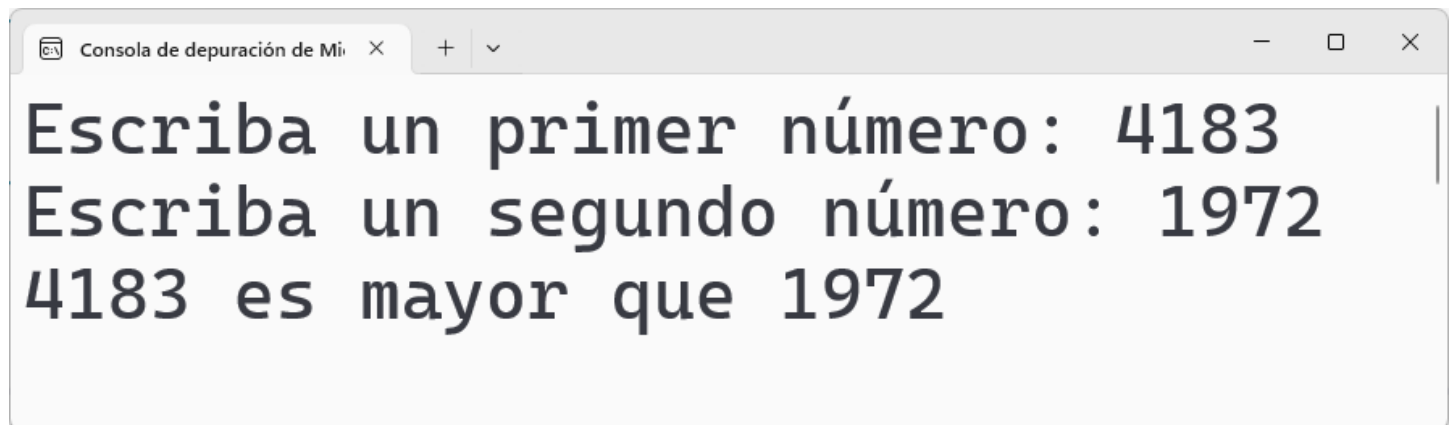


Ilustración 30: Uso de if... else if ... else



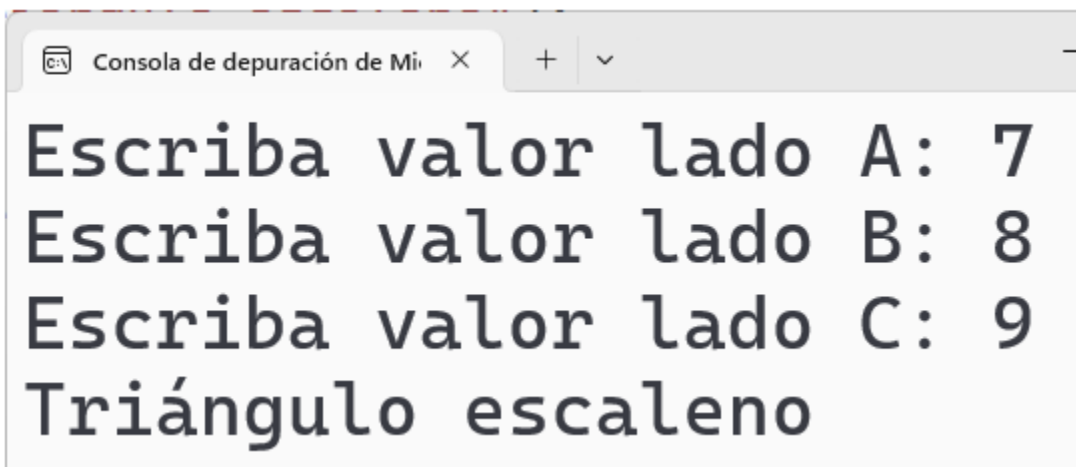
```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee los lados de un triángulo
        Console.Write("Escriba valor lado A: ");
        double ladoA = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba valor lado B: ");
        double ladoB = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba valor lado C: ");
        double ladoC = Convert.ToDouble(Console.ReadLine());

        //Si condicional, uso del AND &&
        if (ladoA == ladoB && ladoA == ladoC) {
            Console.WriteLine("Triángulo equilátero");
        }
        else if (ladoA != ladoB && ladoA != ladoC && ladoB != ladoC) {
            Console.WriteLine("Triángulo escaleno");
        }
        else {
            Console.WriteLine("Triángulo isósceles");
        }
    }
}
```



Consola de depuración de Mi

Escriba valor lado A: 7  
Escriba valor lado B: 8  
Escriba valor lado C: 9  
Triángulo escaleno

Ilustración 31: Uso del AND &&

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee los lados de un triángulo
        Console.Write("Escriba valor lado A: ");
        double ladoA = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba valor lado B: ");
        double ladoB = Convert.ToDouble(Console.ReadLine());

        Console.Write("Escriba valor lado C: ");
        double ladoC = Convert.ToDouble(Console.ReadLine());

        //Si condicional, uso del OR ||
        if (ladoA == ladoB || ladoA == ladoC || ladoB == ladoC) {
            Console.WriteLine("Triángulo equilátero o isósceles");
        }
        else {
            Console.WriteLine("Triángulo escaleno");
        }
    }
}
```

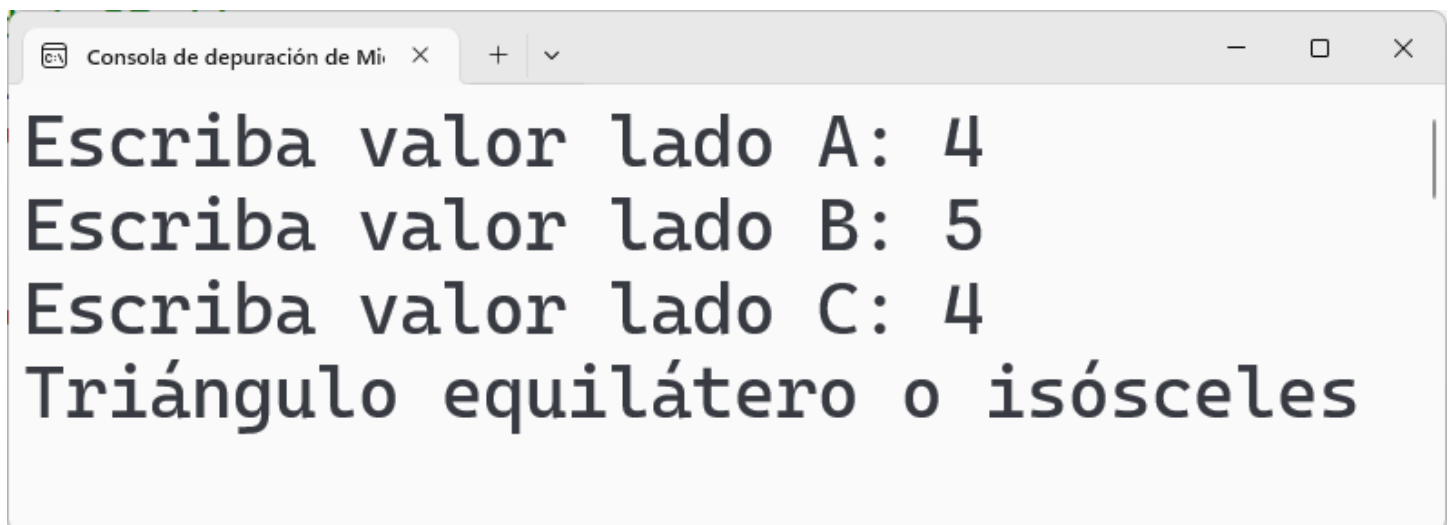


Ilustración 32: Uso del OR ||

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Lee un valor entero
        Console.Write("Escriba un valor entero: ");
        int valor = Convert.ToInt32(Console.ReadLine());

        switch (valor) {
            case 1: Console.WriteLine("Escribió uno"); break;
            case 2:
                Console.WriteLine("Escribió dos, ");
                Console.WriteLine("que es un número par");
                break; //Hay que terminar con break
            case 3: //Esto sería el equivalente a un OR
            case 4:
            case 5:
                Console.WriteLine("Escribió 3 o 4 o 5");
                break;
            default:
                Console.WriteLine("Fuera del rango de 1 a 5");
                break; //Inclusive el default requiere break
        }
    }
}
```

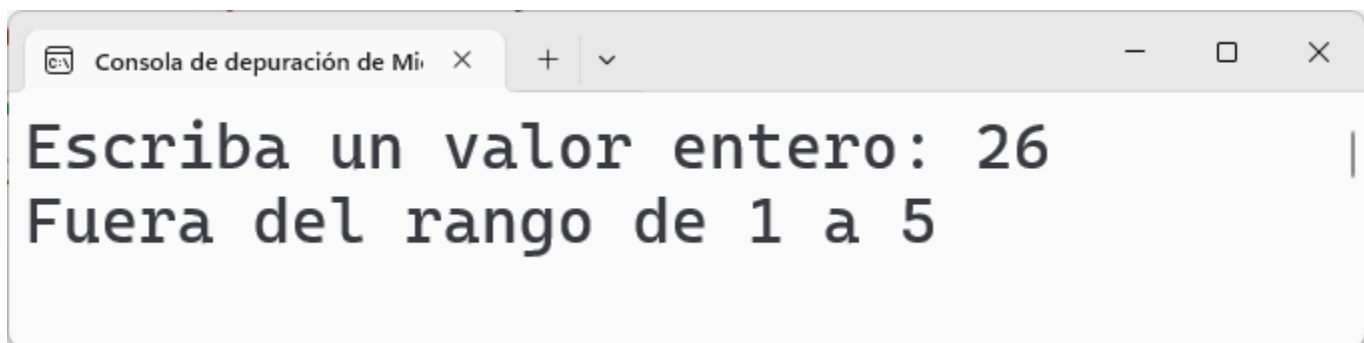


Ilustración 33: Uso del switch

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Operadores booleanos
        bool valA = true;
        bool valB = false;

        bool Resultado1 = valA & valB; //Operador Y
        bool Resultado2 = valA | valB; //Operador O
        bool Resultado3 = valA ^ valB; //Operador XOR
        bool Resultado4 = !valA; //Operador negación

        Console.WriteLine(Resultado1);
        Console.WriteLine(Resultado2);
        Console.WriteLine(Resultado3);
        Console.WriteLine(Resultado4);
    }
}
```

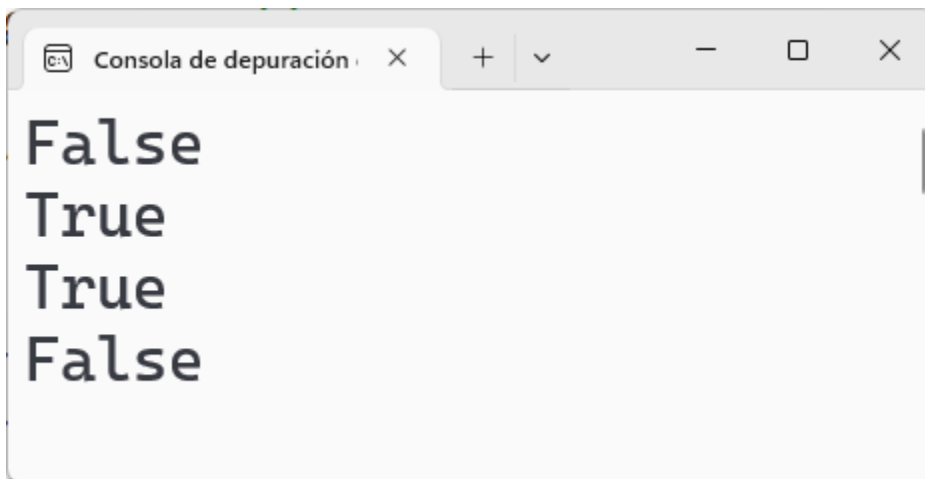


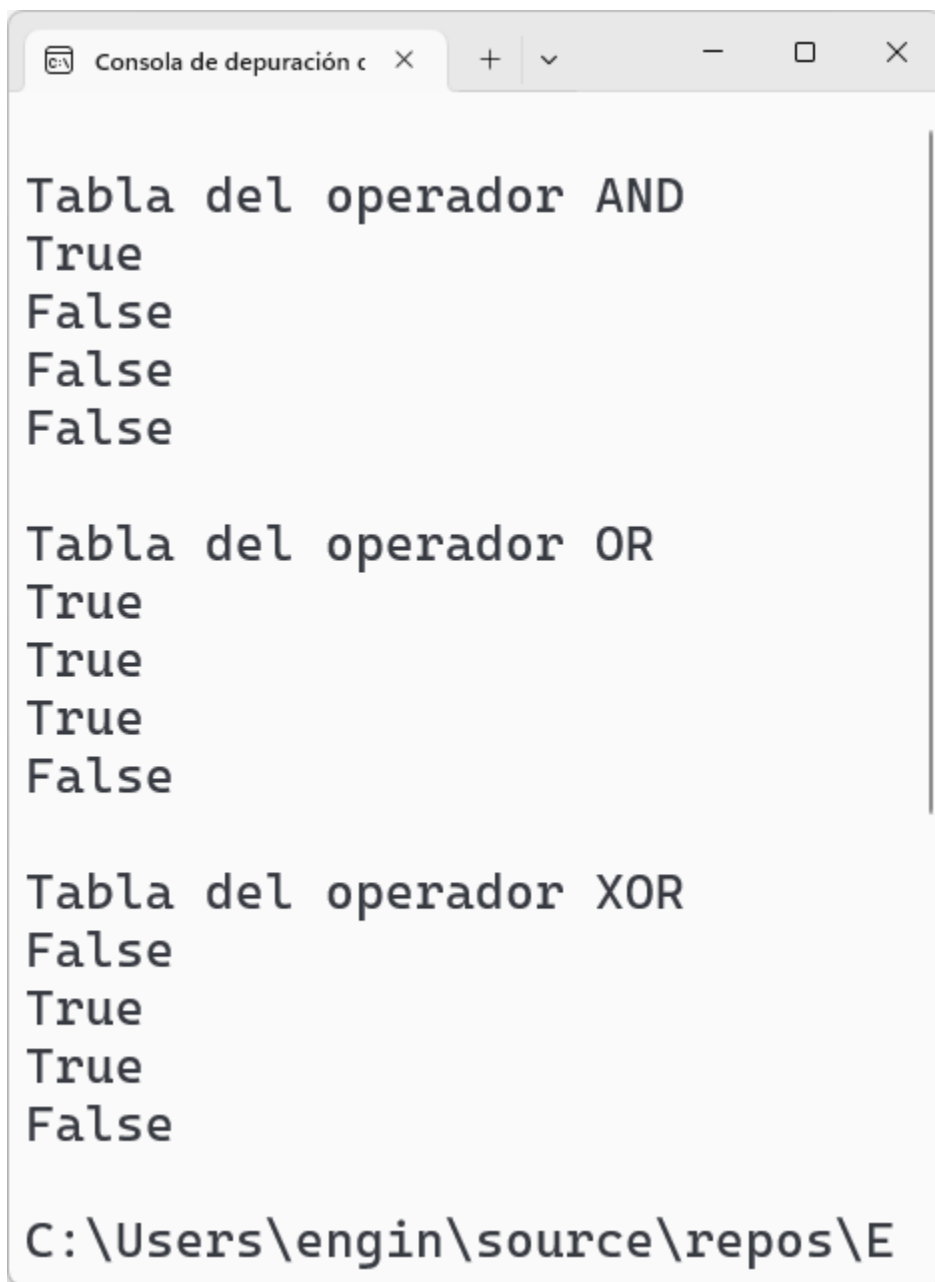
Ilustración 34: Operadores booleanos

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Tabla del AND
        Console.WriteLine("\r\nTabla del operador AND");
        Console.WriteLine(true & true);
        Console.WriteLine(true & false);
        Console.WriteLine(false & true);
        Console.WriteLine(false & false);

        //Tabla del OR
        Console.WriteLine("\r\nTabla del operador OR");
        Console.WriteLine(true | true);
        Console.WriteLine(true | false);
        Console.WriteLine(false | true);
        Console.WriteLine(false | false);

        //Tabla del XOR
        Console.WriteLine("\r\nTabla del operador XOR");
        Console.WriteLine(true ^ true);
        Console.WriteLine(true ^ false);
        Console.WriteLine(false ^ true);
        Console.WriteLine(false ^ false);
    }
}
```



```
Consola de depuración c  X  +  v  -  □  X

Tabla del operador AND
True
False
False
False

Tabla del operador OR
True
True
True
False

Tabla del operador XOR
False
True
True
False

C:\Users\engin\source\repos\E
```

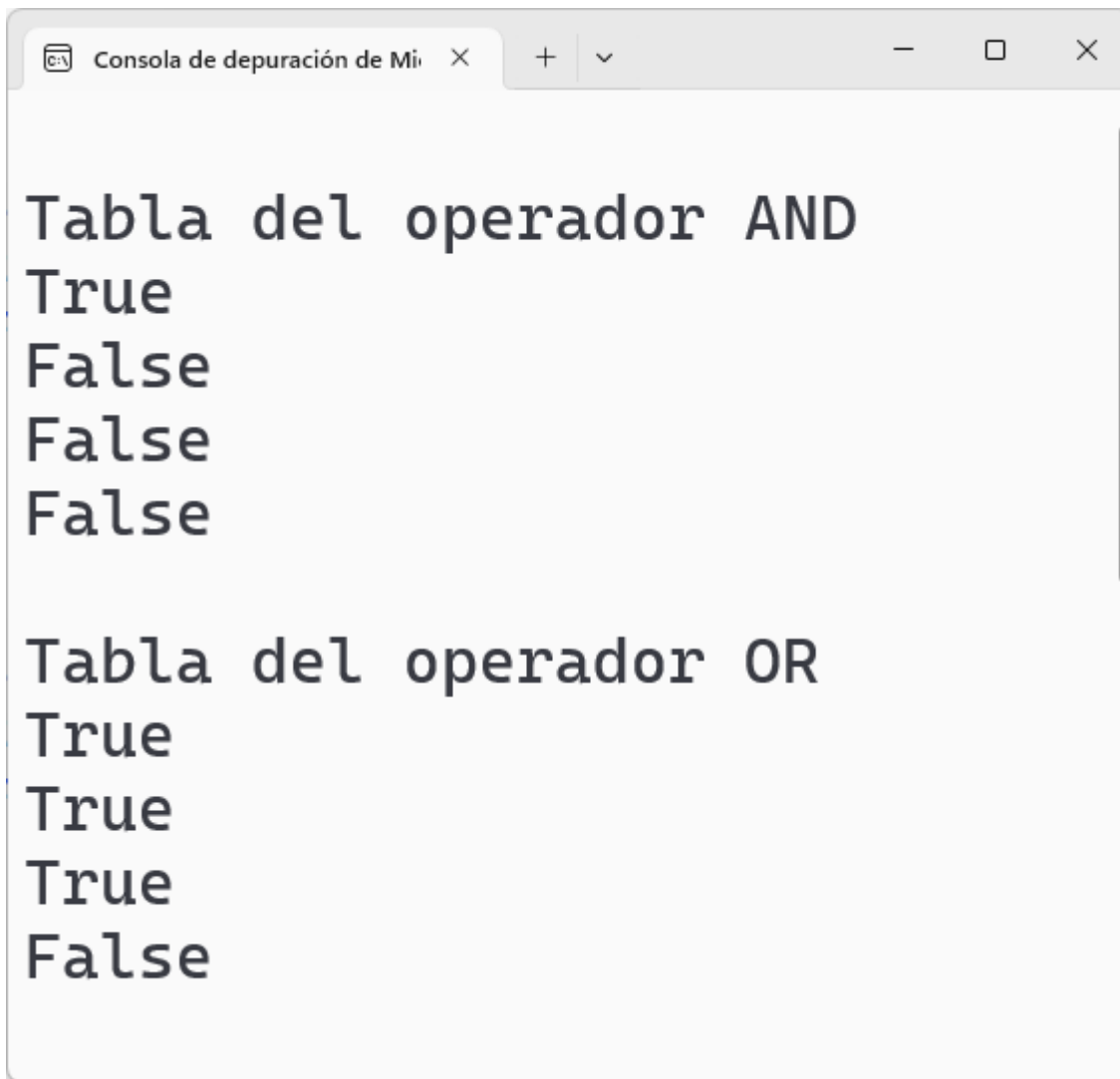
Ilustración 35: Tablas de verdad

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Tabla del AND
        Console.WriteLine("\r\nTabla del operador AND");
        Console.WriteLine(true && true);
        Console.WriteLine(true && false);
        Console.WriteLine(false && true);
        Console.WriteLine(false && false);

        //Tabla del OR
        Console.WriteLine("\r\nTabla del operador OR");
        Console.WriteLine(true || true);
        Console.WriteLine(true || false);
        Console.WriteLine(false || true);
        Console.WriteLine(false || false);

        /* La diferencia entre & y &&, | y || es que cuando
        * se usa & se evalúan ambos operandos a la izquierda
        * y derecha del &, en cambio, cuando se usa && se
        * evalúa primero el operando de la izquierda y si da
        * falso, ya se sabe que toda la expresión es falsa.
        * Luego en un si condicional, se ejecuta más rápido
        * si se usa && en vez de &. Similar se aplica para
        * || o | , si se usa en un si condicional evalúa el
        * primer operando, si es verdadero, toda la expresión
        * es verdadera. */
    }
}
```



The image shows a browser window with the developer console open. The console has a tab labeled 'Consola de depuración de Mi' and standard window controls. It displays two sections of text:

**Tabla del operador AND**  
True  
False  
False  
False

**Tabla del operador OR**  
True  
True  
True  
False

Ilustración 36: Tablas de verdad usando && y ||



```
namespace Ejemplo;

internal class Program {
    static void Main() {
        Console.WriteLine("\r\nCompleja expresión booleana");
        Console.WriteLine(true && true || false && true || true || false);
        Console.WriteLine(true || false && true || false && true && true);
        Console.WriteLine(!false && !true || !true && false && !false);

        /* No se recomienda en una expresión lógica poner dos o más
        * operadores lógicos. Llega a confundir.
        * Hacer uso de paréntesis. */
    }
}
```

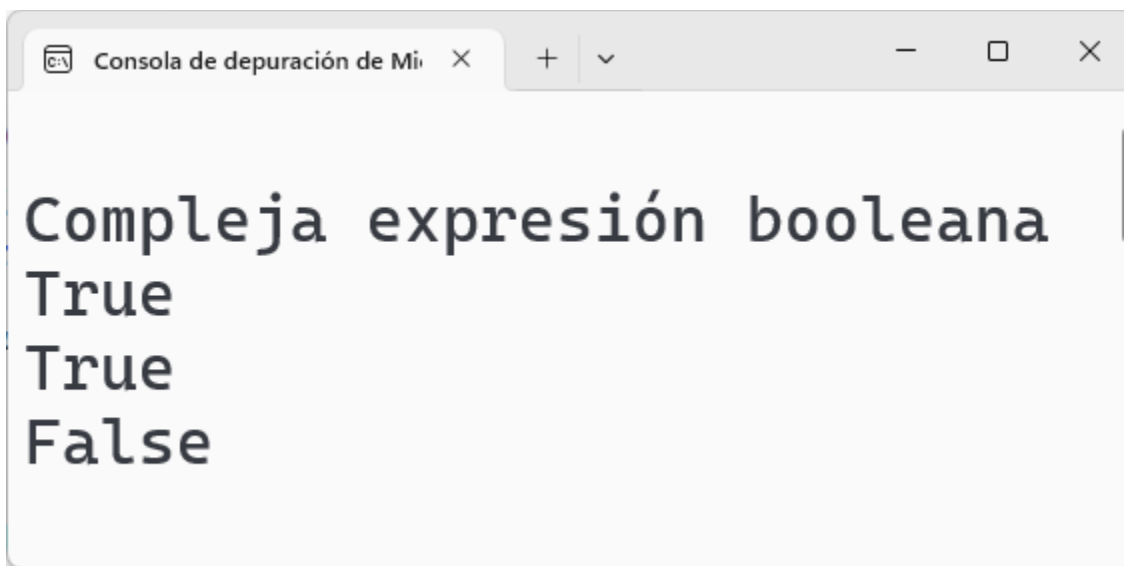


Ilustración 37: Expresión booleana

## Precedencia de los operadores

A/039.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        Console.WriteLine("\r\nCompleja expresión booleana");
        Console.WriteLine(true && true || false && true || true || false);
        Console.WriteLine(true || false && true || false && true && true);
        Console.WriteLine(!false && !true || !true && false && !false);

        /* No se recomienda en una expresión lógica poner dos o más
        * operadores lógicos. Llega a confundir.
        * Hacer uso de paréntesis. */
    }
}
```

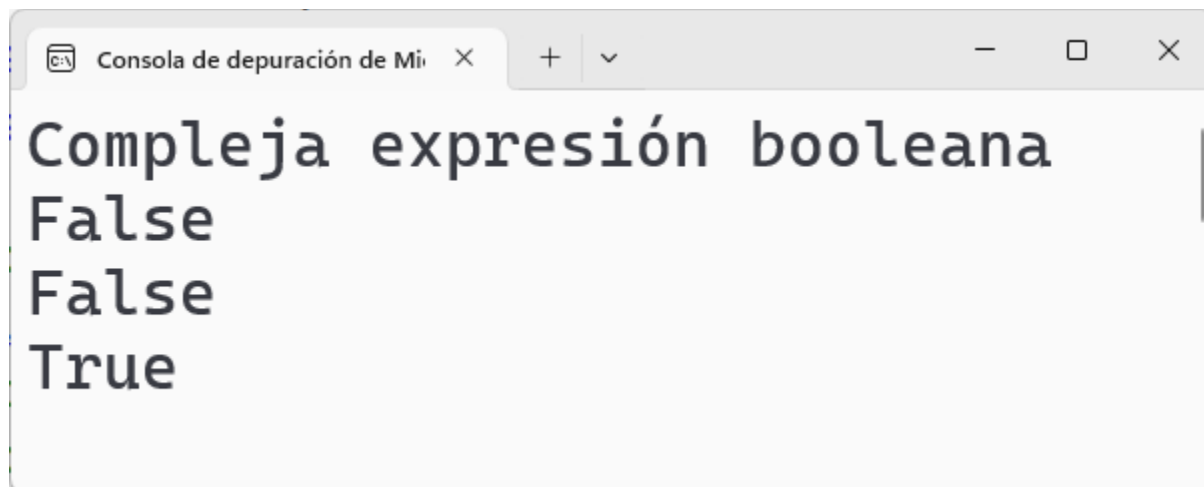


Ilustración 38: Precedencia de los operadores

## El operador "?", un si condicional

Usado en una asignación, funciona de esta manera:

Variable = condición ? valor si es verdadero: valor si es falso;

A/040.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Operador ?
        int valA = 10;
        int valB = 13;
        int Mayor = valA > valB ? valA : valB;
        Console.WriteLine(Mayor);
    }
}
```

## Captura de error con try...catch

Se captura el error y así se evita que el programa colapse.

A/041.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Uso del try...catch
        int valA = 17;
        int valB = 0;
        int resultado;
        try {
            //Intenta dividir entre cero
            resultado = valA / valB;
            Console.WriteLine("Resultado: " + resultado);
        }
        catch { //Captura el error
            Console.WriteLine("División entre cero");
        }
    }
}
```

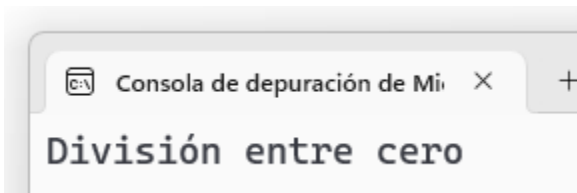


Ilustración 39: Captura de error con try...catch

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Uso del TryParse. Trata de convertir un string a entero
        string Numero = "150";
        if (Int32.TryParse(Numero, out int valorEntero)) {
            Console.WriteLine("Conversión. valorEntero: " + valorEntero);
        }
        else {
            Console.WriteLine("1. No se puede convertir a entero");
        }

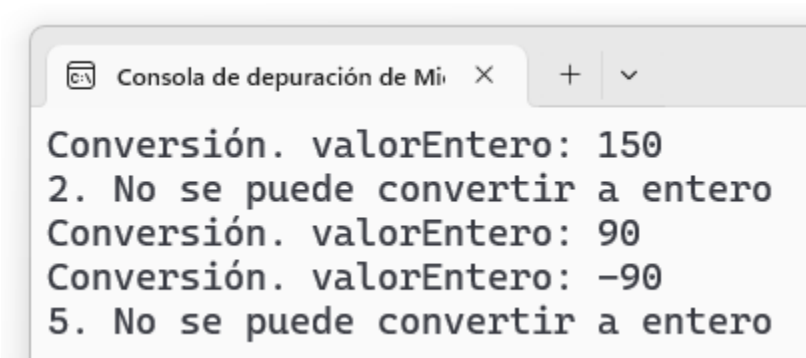
        //Segundo ejemplo
        Numero = "4.178"; //Un punto en la cadena
        if (Int32.TryParse(Numero, out valorEntero)) {
            Console.WriteLine("Conversión. valorEntero: " + valorEntero);
        }
        else {
            Console.WriteLine("2. No se puede convertir a entero");
        }

        //Tercer ejemplo
        Numero = "    90  "; //espacios en la cadena
        if (Int32.TryParse(Numero, out valorEntero)) {
            Console.WriteLine("Conversión. valorEntero: " + valorEntero);
        }
        else {
            Console.WriteLine("3. No se puede convertir a entero");
        }

        //Cuarto ejemplo
        Numero = "-90";
        if (Int32.TryParse(Numero, out valorEntero)) {
            Console.WriteLine("Conversión. valorEntero: " + valorEntero);
        }
        else {
            Console.WriteLine("4. No se puede convertir a entero");
        }

        //Quinto ejemplo
        Numero = "- 90"; //Espacio intermedio
        if (Int32.TryParse(Numero, out valorEntero)) {
            Console.WriteLine("Conversión. valorEntero: " + valorEntero);
        }
    }
}
```

```
    else {  
        Console.WriteLine("5. No se puede convertir a entero");  
    }  
}
```



*Ilustración 40: Uso de TryParse para conversión*

# Ciclo for

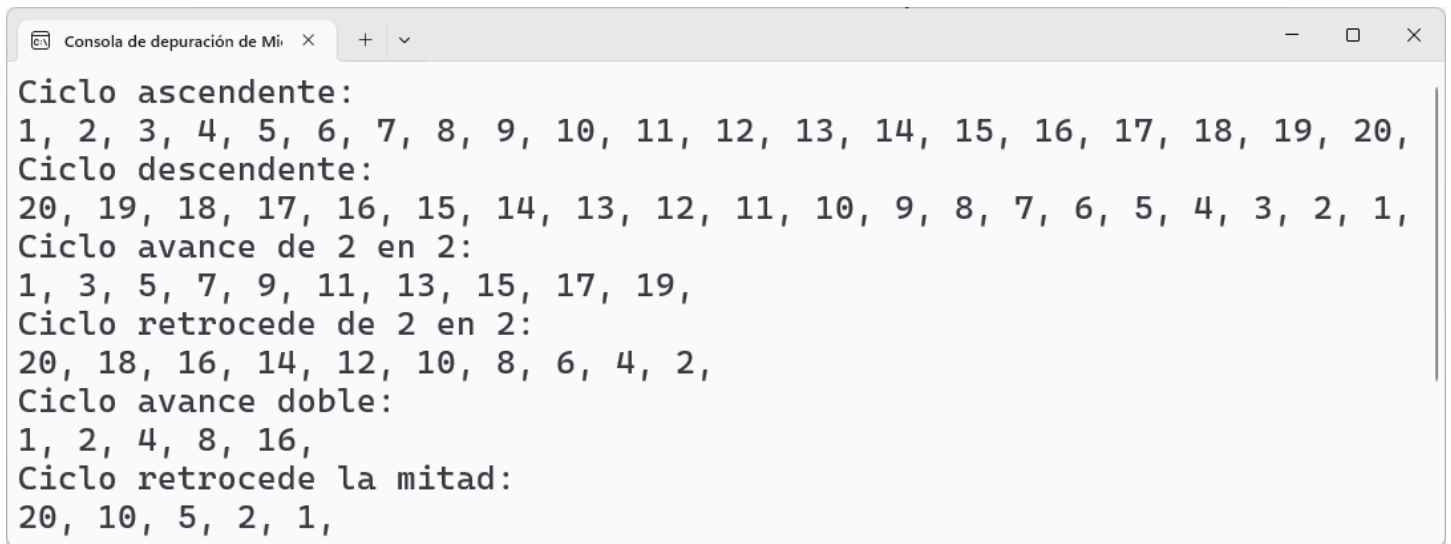
Su sintaxis es:

```
for (variable = valor inicio; condición para que se mantenga el ciclo;  
incremento/decremento de esa variable) { }
```

A/043.cs

```
namespace Ejemplo;  
  
internal class Program {  
    static void Main() {  
        //Ciclo for ascendente  
        Console.WriteLine("Ciclo ascendente:");  
        for (int Contador = 1; Contador <= 20; Contador++) {  
            Console.Write(Contador + ", ");  
        }  
  
        //Ciclo for descendente  
        Console.WriteLine("\r\nCiclo descendente:");  
        for (int Contador = 20; Contador >= 1; Contador--) {  
            Console.Write(Contador + ", ");  
        }  
  
        //Ciclo for ascendente, avance de 2 en 2  
        Console.WriteLine("\r\nCiclo avance de 2 en 2:");  
        for (int Contador = 1; Contador <= 20; Contador += 2) {  
            Console.Write(Contador + ", ");  
        }  
  
        //Ciclo for descendente, retrocede de 2 en 2  
        Console.WriteLine("\r\nCiclo retrocede de 2 en 2:");  
        for (int Contador = 20; Contador >= 1; Contador -= 2) {  
            Console.Write(Contador + ", ");  
        }  
  
        //Ciclo for ascendente, avance doble  
        Console.WriteLine("\r\nCiclo avance doble:");  
        for (int Contador = 1; Contador <= 20; Contador *= 2) {  
            Console.Write(Contador + ", ");  
        }  
  
        //Ciclo for descendente, retrocede la mitad  
        Console.WriteLine("\r\nCiclo retrocede la mitad:");  
        for (int Contador = 20; Contador >= 1; Contador /= 2) {  
            Console.Write(Contador + ", ");  
        }  
    }  
}
```

}



```
Consola de depuración de Mi x + v
Ciclo ascendente:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
Ciclo descendente:
20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
Ciclo avance de 2 en 2:
1, 3, 5, 7, 9, 11, 13, 15, 17, 19,
Ciclo retrocede de 2 en 2:
20, 18, 16, 14, 12, 10, 8, 6, 4, 2,
Ciclo avance doble:
1, 2, 4, 8, 16,
Ciclo retrocede la mitad:
20, 10, 5, 2, 1,
```

*Ilustración 41: Ciclo for*



## Ciclo while

Su sintaxis es:

```
while (condición para que se mantenga en el ciclo) {  
}
```

A/044.cs

```
namespace Ejemplo;  
  
internal class Program {  
    static void Main() {  
        int Contador;  
  
        //Ciclo while ascendente  
        Console.WriteLine("Ciclo ascendente:");  
        Contador = 1;  
        while (Contador <= 20) {  
            Console.Write(Contador + ", ");  
            Contador++;  
        }  
  
        //Ciclo while descendente  
        Console.WriteLine("\r\nCiclo descendente:");  
        Contador = 20;  
        while (Contador >= 1) {  
            Console.Write(Contador + ", ");  
            Contador--;  
        }  
  
        //Ciclo while ascendente, avance de 2 en 2  
        Console.WriteLine("\r\nCiclo avance de 2 en 2:");  
        Contador = 1;  
        while (Contador <= 20) {  
            Console.Write(Contador + ", ");  
            Contador += 2;  
        }  
  
        //Ciclo while descendente, retrocede de 2 en 2  
        Console.WriteLine("\r\nCiclo retrocede de 2 en 2:");  
        Contador = 20;  
        while (Contador >= 1) {  
            Console.Write(Contador + ", ");  
            Contador -= 2;  
        }  
  
        //Ciclo while ascendente, avance doble
```

```

    Console.WriteLine("\r\nCiclo avance doble:");
    Contador = 1;
    while (Contador <= 20) {
        Console.Write(Contador + ", ");
        Contador *= 2;
    }

    //Ciclo while descendente, retrocede la mitad
    Console.WriteLine("\r\nCiclo retrocede la mitad:");
    Contador = 20;
    while (Contador >= 1) {
        Console.Write(Contador + ", ");
        Contador /= 2;
    }
}
}

```

```

Consola de depuración de Mi
Ciclo ascendente:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
Ciclo descendente:
20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
Ciclo avance de 2 en 2:
1, 3, 5, 7, 9, 11, 13, 15, 17, 19,
Ciclo retrocede de 2 en 2:
20, 18, 16, 14, 12, 10, 8, 6, 4, 2,
Ciclo avance doble:
1, 2, 4, 8, 16,
Ciclo retrocede la mitad:
20, 10, 5, 2, 1,

```

Ilustración 42: Ciclo while

# Ciclo do...while

Su sintaxis es:

```
do {
```

```
} while (condición para que se mantenga en el ciclo)
```

A/045.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        int Contador;

        //Ciclo do-while ascendente
        Console.WriteLine("Ciclo ascendente:");
        Contador = 1;
        do {
            Console.Write(Contador + ", ");
            Contador++;
        } while (Contador <= 20);

        //Ciclo do-while descendente
        Console.WriteLine("\r\nCiclo descendente:");
        Contador = 20;
        do {
            Console.Write(Contador + ", ");
            Contador--;
        } while (Contador >= 1);

        //Ciclo do-while ascendente, avance de 2 en 2
        Console.WriteLine("\r\nCiclo avance de 2 en 2:");
        Contador = 1;
        do {
            Console.Write(Contador + ", ");
            Contador += 2;
        } while (Contador <= 20);

        //Ciclo do-while descendente, retrocede de 2 en 2
        Console.WriteLine("\r\nCiclo retrocede de 2 en 2:");
        Contador = 20;
        do {
            Console.Write(Contador + ", ");
            Contador -= 2;
        } while (Contador >= 1);
    }
}
```

```

//Ciclo do-while ascendente, avance doble
Console.WriteLine("\r\nCiclo avance doble:");
Contador = 1;
do {
    Console.Write(Contador + ", ");
    Contador *= 2;
} while (Contador <= 20);

//Ciclo do-while descendente, retrocede la mitad
Console.WriteLine("\r\nCiclo retrocede la mitad:");
Contador = 20;
do {
    Console.Write(Contador + ", ");
    Contador /= 2;
} while (Contador >= 1);
}
}

```

```

Consola de depuración de Mi
Ciclo ascendente:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
Ciclo descendente:
20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1,
Ciclo avance de 2 en 2:
1, 3, 5, 7, 9, 11, 13, 15, 17, 19,
Ciclo retrocede de 2 en 2:
20, 18, 16, 14, 12, 10, 8, 6, 4, 2,
Ciclo avance doble:
1, 2, 4, 8, 16,
Ciclo retrocede la mitad:
20, 10, 5, 2, 1,

```

Ilustración 43: Ciclo do...while

## Romper ciclo con break

A/046.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        int Contador;

        //Rompe el ciclo con break
        Console.WriteLine("Ciclo ascendente:");
        Contador = 1;
        do {
            //Si Contador es múltiplo de 13
            //se rompe el ciclo con break
            if (Contador % 13 == 0) break;
            Console.Write(Contador + ", ");
            Contador++;
        } while (Contador <= 20);
    }
}
```

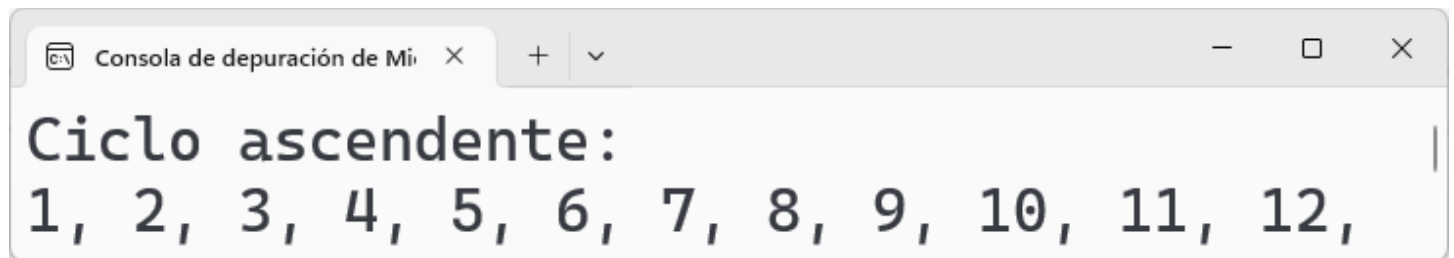


Ilustración 44: Romper ciclo con break

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        int Contador;

        Console.WriteLine("Ciclo ascendente:");
        Contador = 0;
        do {
            Contador++;

            //Si Contador es par entonces va
            //a la siguiente iteración, no
            //ejecuta lo que está después.
            if (Contador % 2 == 0) continue;

            Console.Write(Contador + ", ");
        } while (Contador <= 20);
    }
}
```

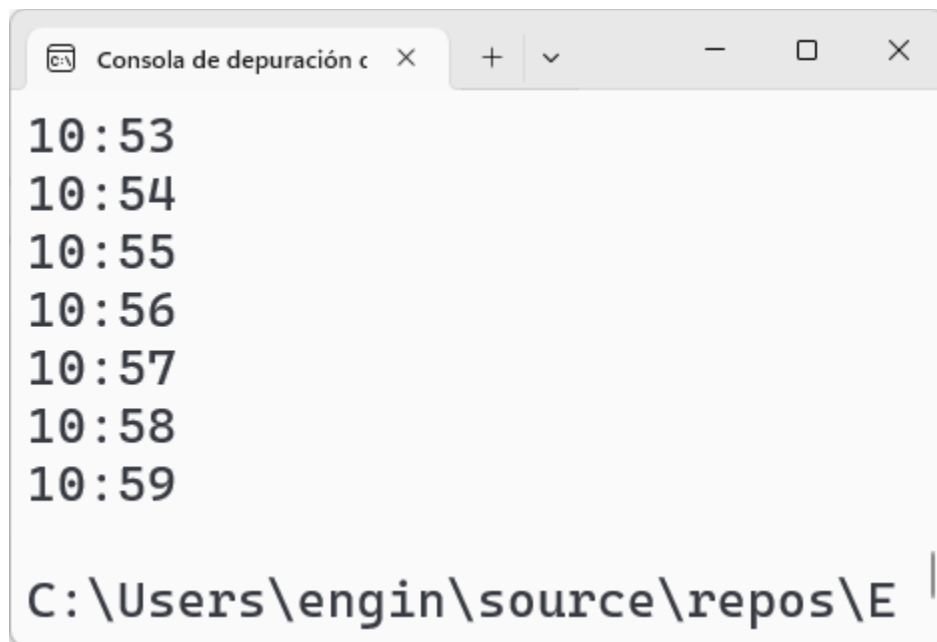


Ilustración 45: Uso del continue en ciclos

```
namespace Ejemplo;

internal class Program {
    static void Main() {

        //Ciclos anidados simulando
        //un minuterero y un segundero
        for (int minuto = 0; minuto <= 10; minuto++) {
            for (int segundo = 0; segundo < 60; segundo++) {
                Console.WriteLine(minuto + ":" + segundo);
            }
        }
    }
}
```



```
10:53
10:54
10:55
10:56
10:57
10:58
10:59

C:\Users\engin\source\repos\E
```

Ilustración 46: Ciclos anidados

```
namespace Ejemplo;

internal class Program {
    static void Main() {

        for (int numA = 1; numA <= 3; numA++) {
            for (int numB = 1; numB <= 6; numB++) {
                for (int numC = 1; numC <= 9; numC++) {
                    Console.Write(numA + "|");
                    Console.WriteLine(numB + "|" + numC);
                    //Sólo rompe el ciclo más interno
                    if (numC == 5) break;
                }
            }
        }
        Console.WriteLine("Final");
    }
}
```

```
3|5|5
3|6|1
3|6|2
3|6|3
3|6|4
3|6|5
Final
```

Ilustración 47: Rompiendo ciclos anidados



```
namespace Ejemplo;

internal class Program {
    static void Main() {

        for (int numA = 1; numA <= 3; numA++) {
            for (int numB = 1; numB <= 6; numB++) {
                for (int numC = 1; numC <= 9; numC++) {
                    Console.Write(numA + "|");
                    Console.WriteLine(numB + "|" + numC);
                    //Sale de todos los ciclos
                    if (numC == 5) goto afuera;
                }
            }
        }

        afuera: Console.WriteLine("Final");
    }
}
```

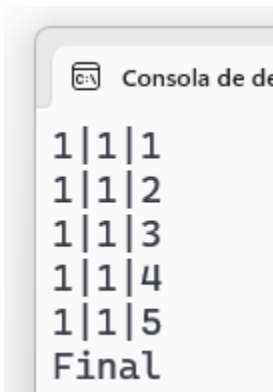


Ilustración 48: Uso del goto

# Diferencias de precisión entre float y double

A/051.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Diferencias en precisión entre float y double

        float acumA = 0;
        float divideA = 7;
        for (float numA = 1; numA <= 100000; numA++) {
            acumA += 1 / divideA;
        }

        double acumB = 0;
        double divideB = 7;
        for (double numB = 1; numB <= 100000; numB++) {
            acumB += 1 / divideB;
        }

        Console.WriteLine("Resultados: " + acumA + " y " + acumB);
    }
}
```

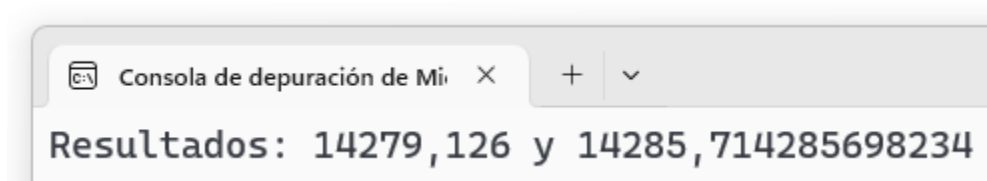


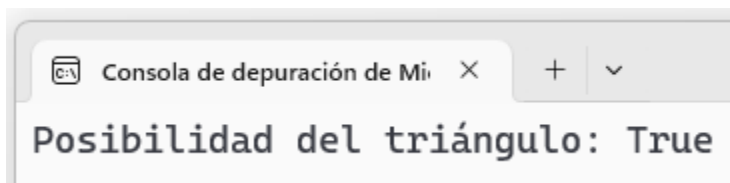
Ilustración 49: Diferencias de precisión entre float y double

Muy recomendado: usar double.

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Uso de funciones
        double ladoA = 3;
        double ladoB = 4;
        double ladoC = 5;
        bool Posible = TrianguloPosible(ladoA, ladoB, ladoC);
        Console.WriteLine("Posibilidad del triángulo: " + Posible);
    }

    //Retorna true si el triángulo es posible
    static bool TrianguloPosible(double valA, double valB, double valC) {
        return valA + valB >= valC &&
            valA + valC >= valB &&
            valB + valC >= valA;
    }
}
```



*Ilustración 50: Función que retorna dato de tipo bool*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Uso de funciones
        double ladoA = 3;
        double ladoB = 4;
        double ladoC = 5;
        double Area = AreaTrianguloHeron(ladoA, ladoB, ladoC);
        Console.WriteLine("Area triángulo es: " + Area);
    }

    //Fórmula de Herón para el cálculo del área
    //de un triángulo dado los lados
    static double AreaTrianguloHeron(double valA, double valB, double valC) {
        double s = (valA + valB + valC) / 2;
        double area = Math.Sqrt(s * (s - valA) * (s - valB) * (s - valC));
        return area;
    }
}
```

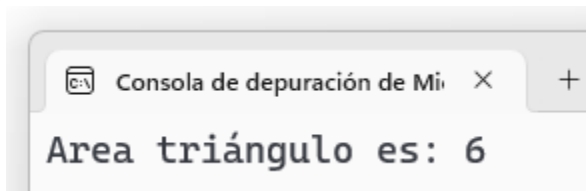


Ilustración 51: Funciones

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Función con doble salida. Nuevo en C# 7.0
        double radio = 1;
        double perimetro, area;
        area = DatosCirculo(radio, out perimetro);
        Console.WriteLine("Area del círculo es: " + area);
        Console.WriteLine("Perímetro del círculo es: " + perimetro);
    }

    //Retorna dos valores: el área y el perímetro del círculo
    static double DatosCirculo(double Radio, out double Perimetro) {
        double area = Math.PI * Radio * Radio;
        Perimetro = 2 * Math.PI * Radio;
        return area;
    }
}
```

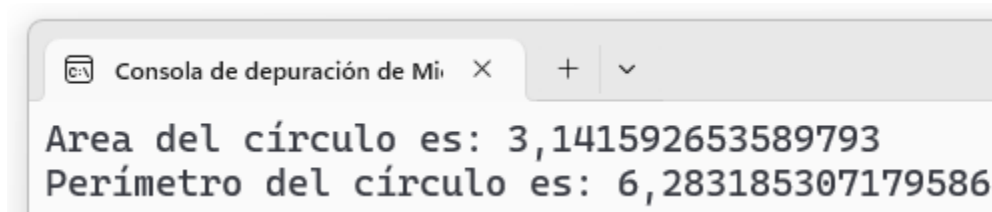


Ilustración 52: Funciones con doble salida

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Función con doble salida. Nuevo en C# 7.0
        double radio = 1;
        Circulo(radio, out double perimetro, out double area);
        Console.WriteLine("Area del círculo: " + area);
        Console.WriteLine("Perímetro del círculo: " + perimetro);
    }

    //Retorna dos valores: el área y el perímetro del círculo
    static void Circulo(double R, out double Perimetro, out double Area) {
        Area = Math.PI * R * R;
        Perimetro = 2 * Math.PI * R;
    }
}
```

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Función iterativa y recursiva
        long valor = 10;
        long factorialA, factorialB;

        factorialA = CalculaFactorialIterativa(valor);
        factorialB = CalculaFactorialRecursivo(valor);

        Console.WriteLine("factorialA es: " + factorialA);
        Console.WriteLine("factorialB es: " + factorialB);
    }

    //Retorna el factorial de un número, de forma iterativa
    static long CalculaFactorialIterativa(long numero) {
        long resultado = 1;
        for (long num = 2; num <= numero; num++) {
            resultado *= num;
        }
        return resultado;
    }

    //Retorna el factorial de un número, de forma recursiva
    static long CalculaFactorialRecursivo(long numero) {
        if (numero == 1) return 1;
        return numero * CalculaFactorialRecursivo(numero - 1);
    }
}
```

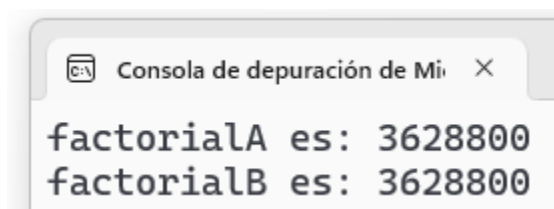


Ilustración 53: Funciones iterativas y recursivas

```
namespace Ejemplo;

internal class Program {
    //Una variable que es conocida por todas las funciones
    static int valor;

    static void Main() {
        //Ámbito de las variables
        valor = 17;
        Console.WriteLine("Valor al iniciar: " + valor);
        SubrutinaA();
        Console.WriteLine("Después de SubrutinaA(): " + valor);
        SubrutinaB();
        Console.WriteLine("Después de SubrutinaB(): " + valor);
    }

    //Un procedimiento
    static void SubrutinaA() {
        valor = 590;
    }

    //Otro procedimiento
    static void SubrutinaB() {
        valor = 3246;
    }
}
```

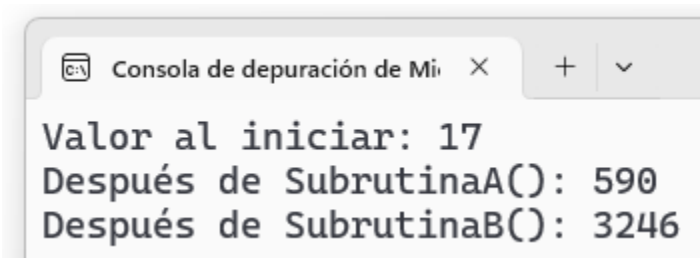


Ilustración 54: Ámbito de las variables



```

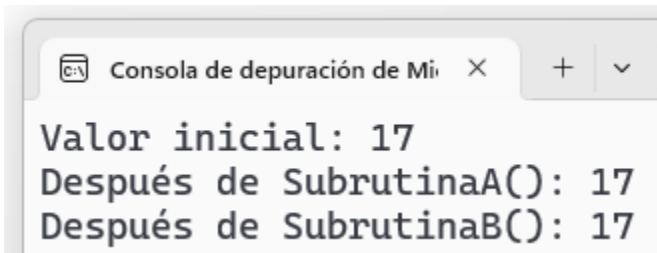
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Ámbito de las variables. Variable local
        int valor = 17;
        Console.WriteLine("Valor inicial: " + valor);
        SubrutinaA();
        Console.WriteLine("Después de SubrutinaA(): " + valor);
        SubrutinaB();
        Console.WriteLine("Después de SubrutinaB(): " + valor);
    }

    //Un procedimiento.
    //La variable "valor" es otra distinta a la del "Main"
    static void SubrutinaA() {
        int valor = 590;
    }

    //Otro procedimiento. La variable
    //"valor" es otra distinta a la del "Main"
    static void SubrutinaB() {
        int valor = 3246;
    }
}

```



*Ilustración 55: Ámbito de las variables*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Funciones de números
        int Numero = 16832929;
        Console.Write("Número: ");
        Console.WriteLine(Numero);

        Console.Write("Impar: ");
        Console.WriteLine(EsImpar(Numero));

        Console.Write("Par: ");
        Console.WriteLine(EsPar(Numero));

        Console.Write("Total de cifras: ");
        Console.WriteLine(TotalCifras(Numero));

        Console.Write("Dos últimas cifras: ");
        Console.WriteLine(RetornadosultimasCifras(Numero));

        Console.Write("Antepenúltima cifra: ");
        Console.WriteLine(AntepenultimaCifra(Numero));

        Console.Write("Penúltima cifra: ");
        Console.WriteLine(PenultimaCifra(Numero));

        Console.Write("Última cifra: ");
        Console.WriteLine(UltimaCifra(Numero));

        Console.Write("Cifra más alta: ");
        Console.WriteLine(LaCifraMasAlta(Numero));

        Console.Write("Cifra más baja: ");
        Console.WriteLine(LaCifraMasBaja(Numero));

        Console.Write("Total de cifras iguales a 5 es: ");
        Console.WriteLine(Cifrashalladas(Numero, 5));

        Console.Write("Al invertirlo es: ");
        Console.WriteLine(InvierteNumero(Numero));

        Console.Write("Es palíndromo: ");
        Console.WriteLine(EsPalindromo(Numero));
    }
}
```

```
Console.Write("Tercera cifra es: ");
Console.WriteLine(CifraPosicion(Numero, 3));

Console.Write("Primera cifra es: ");
Console.WriteLine(PrimeraCifra(Numero));

Console.Write("Suma de las cifras es: ");
Console.WriteLine(SumaCifras(Numero));

Console.Write("Suma de cifras pares es: ");
Console.WriteLine(SumaCifrasPares(Numero));

Console.Write("Suma de cifras impares es: ");
Console.WriteLine(SumaCifrasImpares(Numero));

Console.Write("Multiplicación de cifras es: ");
Console.WriteLine(MultiplicaCifras(Numero));

Console.Write("Multiplicación de cifras pares es: ");
Console.WriteLine(MultiplicaCifrasPares(Numero));

Console.Write("Multiplicación de cifras impares es: ");
Console.WriteLine(MultiplicaCifrasImpares(Numero));

Console.Write("Todas las cifras son pares: ");
Console.WriteLine(TodasCifrasPares(Numero));

Console.Write("Todas las cifras son impares: ");
Console.WriteLine(TodasCifrasImpares(Numero));

Console.Write("Total cifras pares: ");
Console.WriteLine(TotalCifrasPares(Numero));

Console.Write("Total cifras impares: ");
Console.WriteLine(TotalCifrasImpares(Numero));

Console.Write("Solo hay cifras menores o iguales a 5: ");
Console.WriteLine(SoloCifrasMenorIgual(Numero, 5));

Console.Write("Solo hay cifras mayores o iguales a 5: ");
Console.WriteLine(SoloCifrasMayorIgual(Numero, 5));

Console.Write("Usa distintas cifras: ");
Console.WriteLine(DistintasCifras(Numero));

Console.Write("Usa distintas cifras pares: ");
Console.WriteLine(DistintasCifrasPares(Numero));
```

```

    Console.Write("Usa distintas cifras impares: ");
    Console.WriteLine(DistintasCifrasImpares(Numero));

    Console.Write("Extrayendo cifras pares: ");
    Console.WriteLine(NumeroSoloPares(Numero));

    Console.Write("Extrayendo cifras impares: ");
    Console.WriteLine(NumeroSoloImpares(Numero));
}

// Retorna true si un número es impar
static bool EsImpar(int Numero) {
    return Numero % 2 == 1;
}

// Retorna true si un número es par
static bool EsPar(int Numero) {
    return Numero % 2 == 0;
}

// Retorna el número de Cifras de un número
static int TotalCifras(int Numero) {
    if (Numero == 0) return 1;
    return (int)Math.Floor(Math.Log10(Math.Abs(Numero))) + 1;
}

// Retorna las dos últimas Cifras del número
static int RetornadosultimasCifras(int Numero) {
    return Numero % 100;
}

// Retorna la antepenúltima Cifra de un número entero
static int AntepenultimaCifra(int Numero) {
    return (Numero / 100) % 10;
}

// Retorna la penúltima Cifra de un número entero
static int PenultimaCifra(int Numero) {
    return (Numero / 10) % 10;
}

// Retorna la última Cifra de un número entero
static int UltimaCifra(int Numero) {
    return Numero % 10;
}

// Retorna la Cifra más alta
static int LaCifraMasAlta(int Numero) {

```

```

    int Copia = Numero;
    int Cifra = 0;
    while (Copia != 0) {
        if (Copia % 10 > Cifra) Cifra = Copia % 10;
        Copia /= 10;
    }
    return Cifra;
}

// Retorna la Cifra más baja
static int LaCifraMasBaja(int Numero) {
    if (Numero == 0) return 0;
    int Copia = Numero;
    int Cifra = 9;
    while (Copia != 0) {
        if (Copia % 10 < Cifra) Cifra = Copia % 10;
        Copia /= 10;
    }
    return Cifra;
}

// Dice cuántas Cifras de determinado número hay en el número enviado
static int Cifrashalladas(int Numero, int Cifra) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        if (Copia % 10 == Cifra) Acumula++;
        Copia /= 10;
    }
    return Acumula;
}

// Invierte un número
static int InvierteNumero(int Numero) {
    int resultado = 0;
    while (Numero != 0) {
        resultado = resultado * 10 + (Numero % 10);
        Numero /= 10;
    }
    return resultado;
}

// Retorna true si el número es palíndromo
static bool EsPalindromo(int Numero) {
    return Numero == InvierteNumero(Numero);
}

//Retorna la Cifra de una determinada posición

```

```

static int CifraPosicion(int Numero, int Posicion) {
    int totalCifras = TotalCifras(Numero);
    if (Posicion < 1 || Posicion > totalCifras) return 0;
    int divisor = (int)Math.Pow(10, totalCifras - Posicion);
    return (Numero / divisor) % 10;
}

// Retorna la primera Cifra de un número
static int PrimeraCifra(int Numero) {
    return CifraPosicion(Numero, 1);
}

// Retorna la sumatoria de las Cifras de un número
static int SumaCifras(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        Acumula += Cifra;
        Copia /= 10;
    }
    return Acumula;
}

// Retorna la sumatoria de las Cifras pares de un número
static int SumaCifrasPares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 == 0) Acumula += Cifra;
        Copia /= 10;
    }
    return Acumula;
}

// Retorna la sumatoria de las Cifras impares de un número
static int SumaCifrasImpares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 != 0) Acumula += Cifra;
        Copia /= 10;
    }
    return Acumula;
}

```

```

// Retorna el producto de las Cifras de un número
static int MultiplicaCifras(int Numero) {
    if (Numero == 0) return 0;
    int Copia = Numero;
    int Acumula = 1;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        Acumula *= Cifra;
        Copia /= 10;
    }
    return Acumula;
}

// Retorna el producto de las Cifras pares de un número
static int MultiplicaCifrasPares(int Numero) {
    int Copia = Numero;
    int Acumula = 1;
    bool HayPar = false;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 == 0) {
            Acumula *= Cifra;
            HayPar = true;
        }
        Copia /= 10;
    }
    if (HayPar) return Acumula;
    return 0;
}

// Retorna el producto de las Cifras impares de un número
static int MultiplicaCifrasImpares(int Numero) {
    int Copia = Numero;
    int Acumula = 1;
    bool HayImpar = false;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 != 0) {
            Acumula *= Cifra;
            HayImpar = true;
        }
        Copia /= 10;
    }
    if (HayImpar) return Acumula;
    return 0;
}

// Retorna true si todas las Cifras son pares

```

```

static bool TodasCifrasPares(int Numero) {
    int Copia = Numero;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 != 0) return false;
        Copia /= 10;
    }
    return true;
}

// Retorna true si todas las Cifras son impares
static bool TodasCifrasImpares(int Numero) {
    int Copia = Numero;
    if (Copia == 0) return false;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 == 0) return false;
        Copia /= 10;
    }
    return true;
}

// Retorna el número de Cifras pares
static int TotalCifrasPares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 == 0) Acumula++;
        Copia /= 10;
    }
    return Acumula;
}

// Retorna el número de Cifras impares
static int TotalCifrasImpares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 != 0) Acumula++;
        Copia /= 10;
    }
    return Acumula;
}

// Retorna true si el número tiene sólo Cifras menores o iguales a Cifra
static bool SoloCifrasMenorIgual(int Numero, int Cifra) {

```



```

    int Copia = Numero;
    while (Copia != 0) {
        if (Copia % 10 > Cifra) return false;
        Copia /= 10;
    }
    return true;
}

// Retorna true si el número tiene sólo Cifras mayores o iguales a Cifra
static bool SoloCifrasMayorIgual(int Numero, int Cifra) {
    int Copia = Numero;
    while (Copia != 0) {
        if (Copia % 10 < Cifra) return false;
        Copia /= 10;
    }
    return true;
}

// Retorna true si todas las Cifras son distintas
static bool DistintasCifras(int Numero) {
    string s = Numero.ToString();
    return s.Length == s.Distinct().Count();
}

// Retorna si todas las Cifras pares son distintas
static bool DistintasCifrasPares(int Numero) {
    for (int Cifra = 0; Cifra <= 8; Cifra += 2) {
        int Copia = Numero;
        int Cuenta = 0;
        while (Copia != 0) {
            if (Copia % 10 == Cifra) Cuenta++;
            if (Cuenta > 1) return false;
            Copia /= 10;
        }
    }
    return true;
}

// Retorna si todas las Cifras impares son distintas
static bool DistintasCifrasImpares(int Numero) {
    for (int Cifra = 1; Cifra <= 9; Cifra += 2) {
        int Copia = Numero;
        int Cuenta = 0;
        while (Copia != 0) {
            if (Copia % 10 == Cifra) Cuenta++;
            if (Cuenta > 1) return false;
            Copia /= 10;
        }
    }
}

```

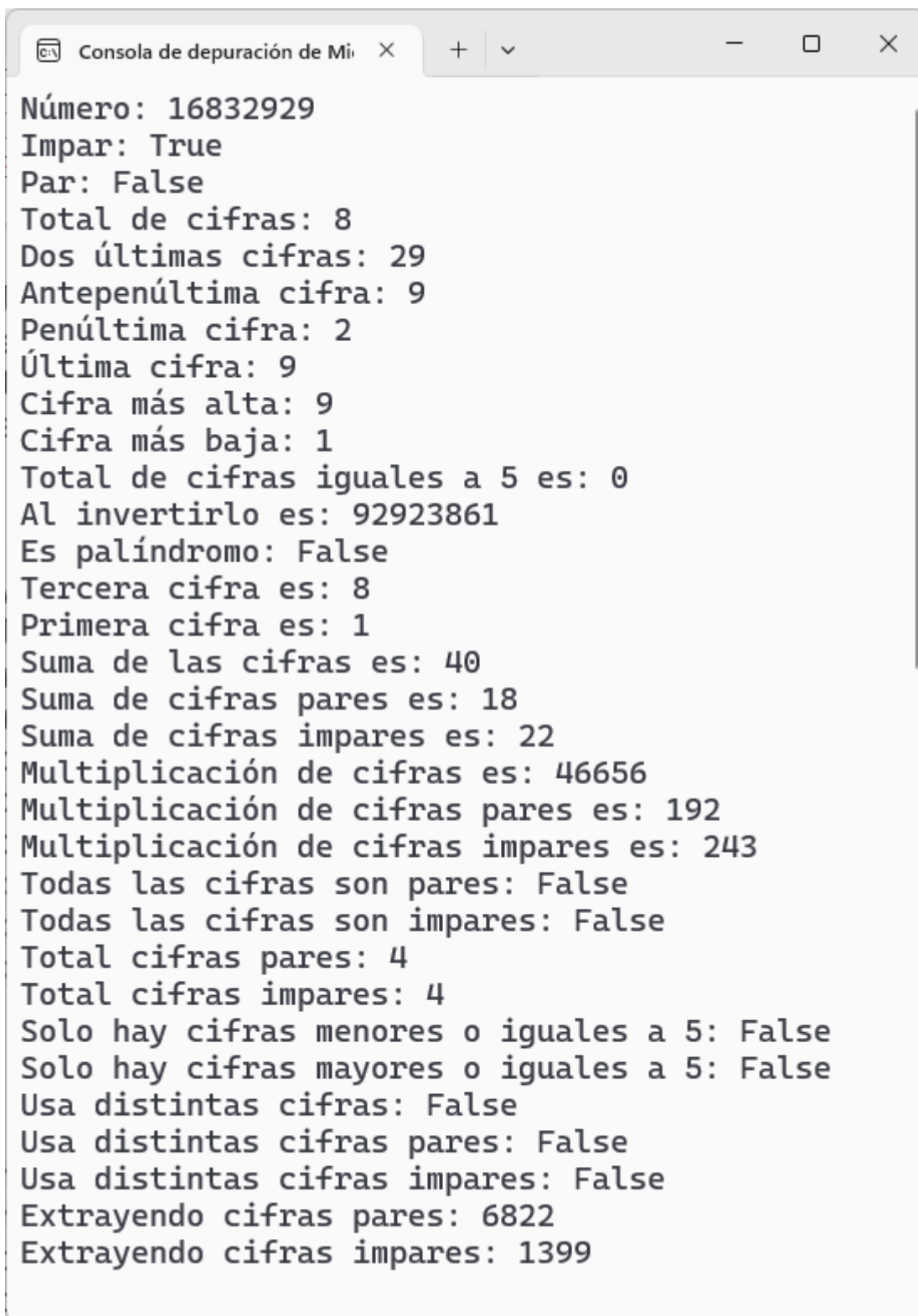
```

    }
    return true;
}

//Retorna un número con solo las Cifras pares
static int NumeroSoloPares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    int Posicion = 1;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 == 0) {
            Acumula += Posicion * Cifra;
            Posicion *= 10;
        }
        Copia /= 10;
    }
    return Acumula;
}

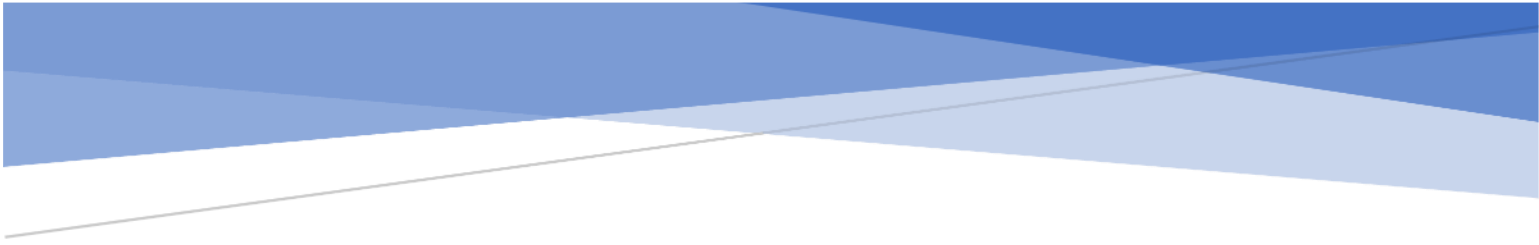
//Retorna un número con solo las Cifras impares
static int NumeroSoloImpares(int Numero) {
    int Copia = Numero;
    int Acumula = 0;
    int Posicion = 1;
    while (Copia != 0) {
        int Cifra = Copia % 10;
        if (Cifra % 2 != 0) {
            Acumula += Posicion * Cifra;
            Posicion *= 10;
        }
        Copia /= 10;
    }
    return Acumula;
}
}

```



```
Consola de depuración de Mi X + v - □ X
Número: 16832929
Impar: True
Par: False
Total de cifras: 8
Dos últimas cifras: 29
Antepenúltima cifra: 9
Penúltima cifra: 2
Última cifra: 9
Cifra más alta: 9
Cifra más baja: 1
Total de cifras iguales a 5 es: 0
Al invertirlo es: 92923861
Es palíndromo: False
Tercera cifra es: 8
Primera cifra es: 1
Suma de las cifras es: 40
Suma de cifras pares es: 18
Suma de cifras impares es: 22
Multiplicación de cifras es: 46656
Multiplicación de cifras pares es: 192
Multiplicación de cifras impares es: 243
Todas las cifras son pares: False
Todas las cifras son impares: False
Total cifras pares: 4
Total cifras impares: 4
Solo hay cifras menores o iguales a 5: False
Solo hay cifras mayores o iguales a 5: False
Usa distintas cifras: False
Usa distintas cifras pares: False
Usa distintas cifras impares: False
Extrayendo cifras pares: 6822
Extrayendo cifras impares: 1399
```

Ilustración 56: Manejo de cifras



# C# Y .NET 10

## Parte 02. Cadenas

2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                     | 3  |
| Acerca del autor.....                                           | 4  |
| Licencia de este libro .....                                    | 4  |
| Licencia del software .....                                     | 4  |
| Marcas registradas .....                                        | 5  |
| Dedicatoria .....                                               | 6  |
| Inmutabilidad de las cadenas .....                              | 7  |
| Declaración de cadenas.....                                     | 7  |
| Uso de @ en cadenas.....                                        | 9  |
| Constantes con cadenas.....                                     | 10 |
| Copia de cadenas .....                                          | 11 |
| Caracteres especiales .....                                     | 12 |
| Acceder a un determinado carácter .....                         | 13 |
| Tamaño de la cadena y recorrerla .....                          | 14 |
| Subcadenas .....                                                | 15 |
| Reemplazar caracteres .....                                     | 16 |
| Encontrar subcadenas o caracteres .....                         | 18 |
| Convertir a mayúsculas y minúsculas .....                       | 20 |
| StringBuilder, más veloz y se puede modificar su contenido..... | 21 |
| StringBuilder, métricas de velocidad .....                      | 22 |
| Eliminar caracteres al inicio y al final .....                  | 24 |
| Comparar cadenas. Forma no recomendada .....                    | 26 |
| Comparar cadenas. Forma recomendada.....                        | 27 |
| Comparar cadenas. Ignorando las mayúsculas y minúsculas.....    | 28 |

## Tabla de ilustraciones

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Ilustración 1: Declaración de cadenas.....                                      | 8  |
| Ilustración 2: Uso de @.....                                                    | 9  |
| Ilustración 3: No se puede cambiar una constante.....                           | 10 |
| Ilustración 4: Copia de cadenas.....                                            | 11 |
| Ilustración 5: Caracteres especiales .....                                      | 12 |
| Ilustración 6: Acceder a un determinado carácter.....                           | 13 |
| Ilustración 7: Tamaño de la cadena y recorrerla .....                           | 14 |
| Ilustración 8: Subcadenas .....                                                 | 15 |
| Ilustración 9: Reemplazar caracteres.....                                       | 17 |
| Ilustración 10: Encontrar subcadenas o caracteres .....                         | 19 |
| Ilustración 11: Convertir a mayúsculas y minúsculas.....                        | 20 |
| Ilustración 12: StringBuilder, más veloz y se puede modificar su contenido..... | 21 |
| Ilustración 13: StringBuilder, métricas de velocidad .....                      | 23 |
| Ilustración 14: Eliminar caracteres al inicio y al final .....                  | 25 |
| Ilustración 15: Comparar cadenas. Forma no recomendada.....                     | 26 |
| Ilustración 16: Comparar cadenas. Forma recomendada .....                       | 27 |
| Ilustración 17: Comparar cadenas. Ignorando las mayúsculas y minúsculas.....    | 28 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifemail@hotmail.com](mailto:enginelifemail@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>



## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac, Sonic,  
Meimei y mis recordados Sally, Capuchina, Tinita, Tammy,  
Vikingo y Michu.

# Inmutabilidad de las cadenas

En C#, las cadenas son inmutables. Es decir, que una variable (de tipo string) una vez iniciada con una cadena, esta no puede cambiar ni una letra directamente. ¿Por qué? El manejo de cadenas es simplificado con la inmutabilidad porque ya no hay riesgo que estas cambien de forma inesperada. Es útil en el caso de manejo de hilos porque si hay varios accediendo a la misma cadena, esta no va a cambiar. Otro beneficio de la inmutabilidad es que pueden ser almacenadas en caché, por lo que se optimiza el rendimiento.

## Declaración de cadenas

En C# hay un rico conjunto de funciones para manipular cadenas (strings), incluyendo una específica para mejorar el desempeño: `StringBuilder`

B/001.cs

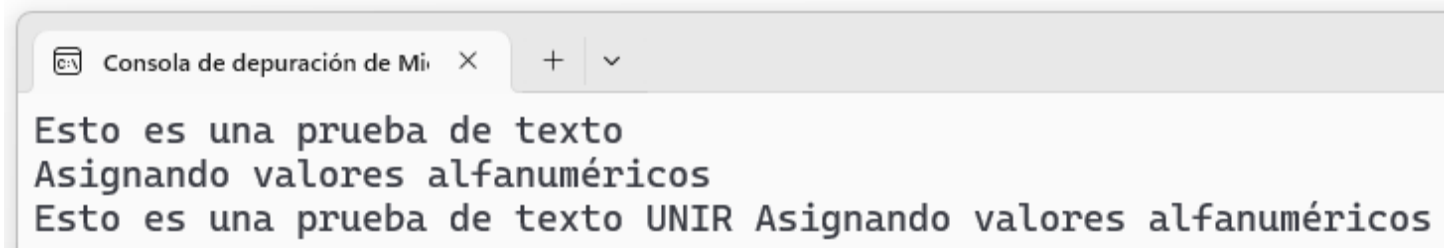
```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Declarar variable de tipo cadena (string). Dos formas.
        System.String cadenaA;
        string cadenaB;

        //Asignando valores de cadena
        cadenaA = "Esto es una prueba de texto";
        cadenaB = "Asignando valores alfanuméricos";

        //Imprimiendo por consola
        Console.WriteLine(cadenaA);
        Console.WriteLine(cadenaB);

        //Uniendo dos cadenas o concatenar
        string cadenaC = cadenaA + " UNIR " + cadenaB;
        Console.WriteLine(cadenaC);
    }
}
```



*Ilustración 1: Declaración de cadenas*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Rutas. Forma antigua
        string RutaA = "C:\\Users\\engin\\OneDrive\\";

        //Ruta. Forma moderna
        string RutaB = @"C:\Users\engin\OneDrive\";

        //Imprimiendo por consola
        Console.WriteLine(RutaA);
        Console.WriteLine(RutaB);
    }
}
```

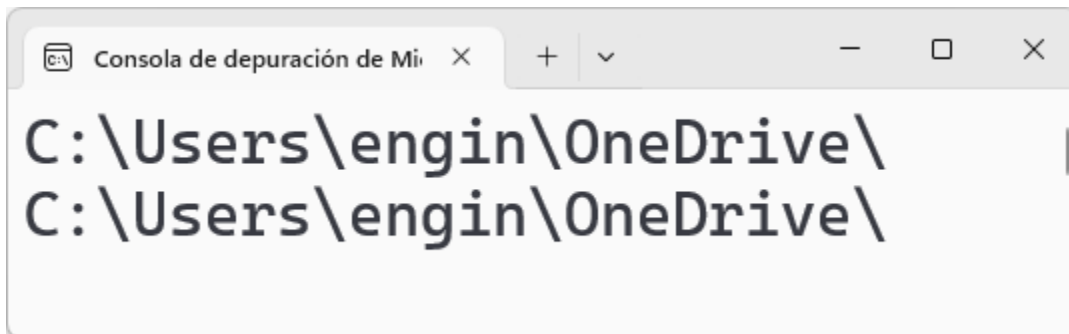


Ilustración 2: Uso de @

# Constantes con cadenas

Las constantes una vez definidas con valores, no se pueden cambiar.

B/003.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Constante
        const string cadena = "abcdefghijkl";

        //Se intenta cambiar la constante y da error en compilación
        cadena = "Hola Mundo";

        //Imprimiendo por consola
        Console.WriteLine(cadena);
    }
}
```

namespace Ejemplo;

0 referencias

internal class Program {

0 referencias

static void Main() {

//Constante

const string cadena = "abcdefghijkl";

//Se intenta cambiar la constante y da error en compilación

cadena = "Hola Mundo";



//1

Cor

}

}

(constante local) string cadena = "abcdefghijkl"

Constante

Describir con Copilot

CS0131: La parte izquierda de una asignación debe ser una variable, una propiedad o un indizador

Mostrar posibles correcciones (Alt+Entrar o Ctrl+.)

Ilustración 3: No se puede cambiar una constante

## Copia de cadenas

C# trata los strings como si fuesen un tipo de dato nativo en la práctica, por eso se copia el contenido de una variable a otra al usarse el operador de asignación (=).

B/004.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Inmutabilidad
        string cadenaA = "abcdefghijkl";

        //Se copian los datos de cadenaA en cadenaB
        string cadenaB = cadenaA;

        //Se agregan datos a cadenaA
        //¿Qué sucederá con cadenaB?
        cadenaA += "pqrstuvwxyz";

        //Imprimiendo por consola
        Console.WriteLine(cadenaA);
        Console.WriteLine(cadenaB);
    }
}
```

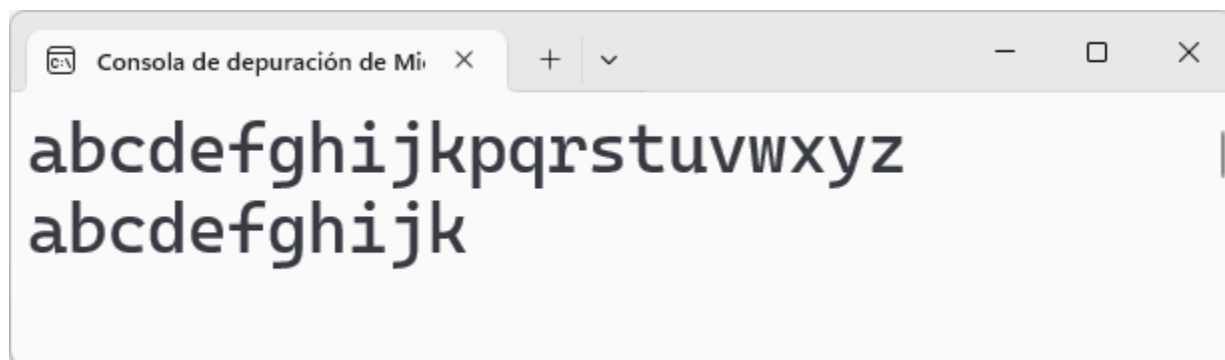


Ilustración 4: Copia de cadenas

# Caracteres especiales

Los caracteres especiales no se imprimen, sino que generan otro comportamiento al intentar "imprimirse".

B/005.cs

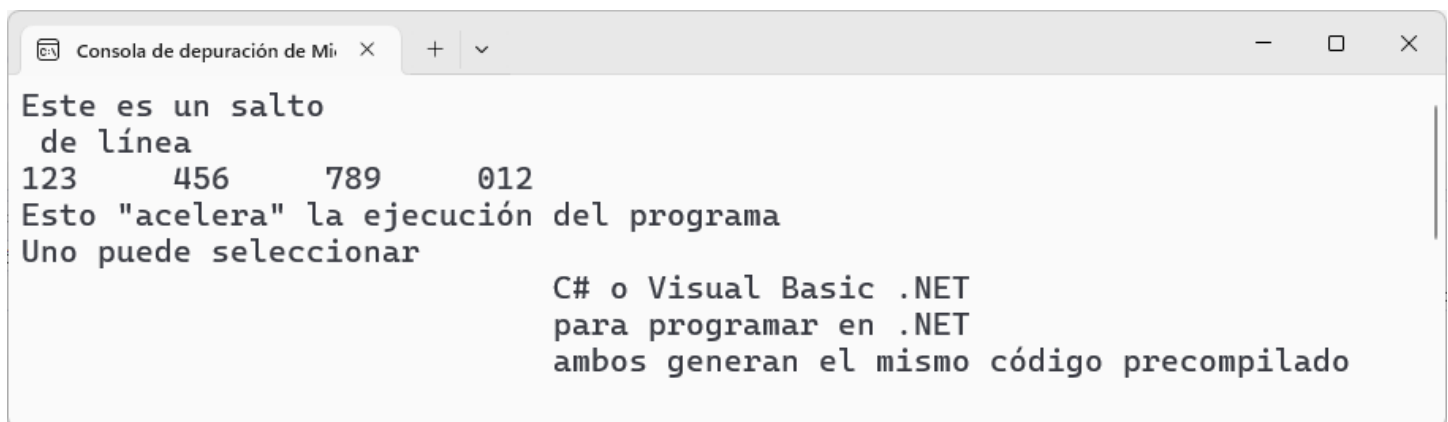
```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Caracteres especiales. Salto de línea
        string cadenaA = "Este es un salto \r\n de línea";
        Console.WriteLine(cadenaA);

        //Caracteres especiales. Tabuladores
        string cadenaB = "123\t456\t789\t012";
        Console.WriteLine(cadenaB);

        //Caracteres especiales. Imprimir las comillas dobles
        string cadenaC = "Esto \"acelera\" la ejecución del programa";
        Console.WriteLine(cadenaC);

        //Usando el verbatim (toma los caracteres internos)
        string cadenaD = @"Uno puede seleccionar
                           C# o Visual Basic .NET
                           para programar en .NET
                           ambos generan el mismo código precompilado";
        Console.WriteLine(cadenaD);
    }
}
```



```
Este es un salto
de línea
123      456      789      012
Esto "acelera" la ejecución del programa
Uno puede seleccionar
                           C# o Visual Basic .NET
                           para programar en .NET
                           ambos generan el mismo código precompilado
```

Ilustración 5: Caracteres especiales

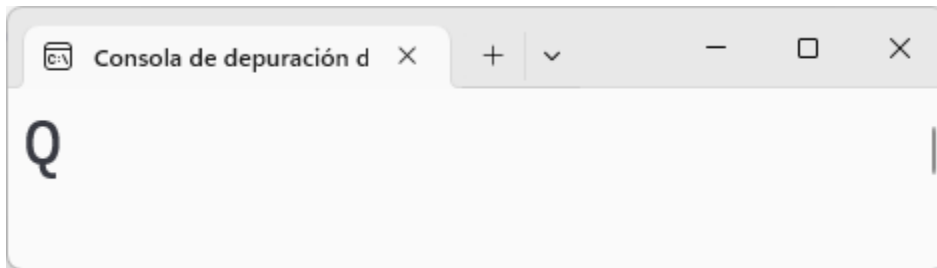
## Acceder a un determinado carácter

En C# la primera letra en un string está en la posición cero.

B/006.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Acceder a determinado caracter
        string cadena = "QWERTYUIOPabcdefghijklmnopqrstuvwxyz";
        char letra = cadena[0]; //Accede a la primera letra
        Console.WriteLine(letra);
    }
}
```



*Ilustración 6: Acceder a un determinado carácter*



## Tamaño de la cadena y recorrerla

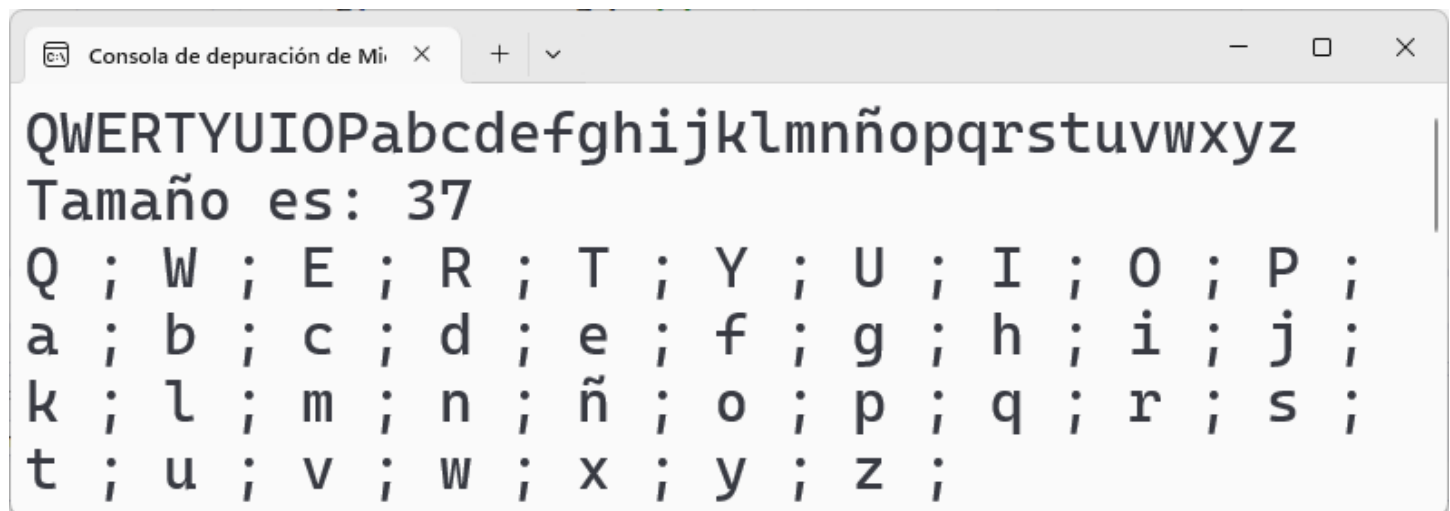
Para obtener el tamaño en caracteres de una cadena se hace uso de la instrucción `Length`. En cuanto a su recorrido, se inicia en cero y la última letra sería el tamaño restándole uno.

B/007.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Tamaño de cadena y recorrerla
        string cadena = "QWERTYUIOPabcdefghijklmnopqrstuvwxyz";
        int tamano = cadena.Length;
        Console.WriteLine(cadena);
        Console.WriteLine("Tamaño es: " + tamano.ToString());

        //Recorre la cadena
        for (int posicion = 0; posicion < tamano; posicion++) {
            char letra = cadena[posicion]; //va de letra en letra
            Console.Write(letra.ToString() + " ; ");
        }
    }
}
```



```
QWERTYUIOPabcdefghijklmnopqrstuvwxyz
Tamaño es: 37
Q ; W ; E ; R ; T ; Y ; U ; I ; O ; P ;
a ; b ; c ; d ; e ; f ; g ; h ; i ; j ;
k ; l ; m ; n ; ñ ; o ; p ; q ; r ; s ;
t ; u ; v ; w ; x ; y ; z ;
```

Ilustración 7: Tamaño de la cadena y recorrerla

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Subcadenas
        string cadena = "abcdefghijklmnñopqrstuvwxyz";

        //Del caracter 3 en adelante
        string subCadA = cadena.Substring(3);
        Console.WriteLine(subCadA);

        //Del caracter 7 traiga 4 caracteres
        string subCadB = cadena.Substring(7, 4);
        Console.WriteLine(subCadB);
    }
}
```

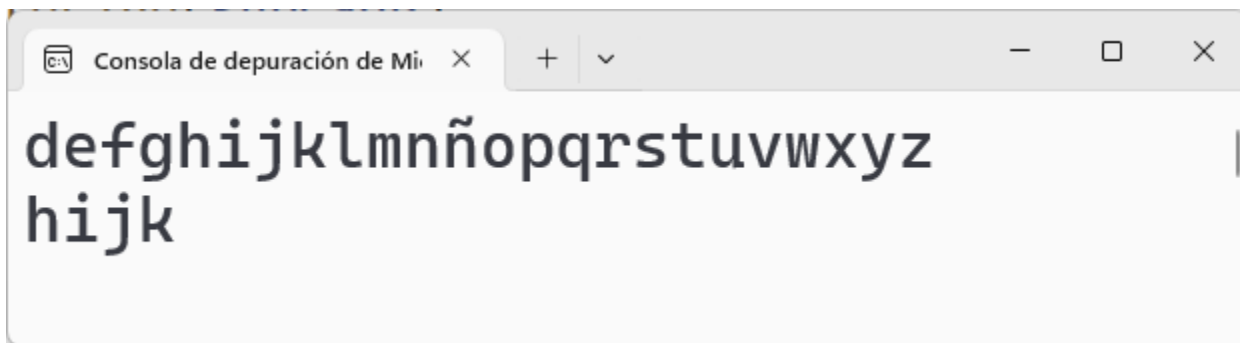


Ilustración 8: Subcadenas

# Reemplazar caracteres

B/009.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Reemplazar
        string cadena = "manzana y naranja";
        Console.WriteLine(cadena);

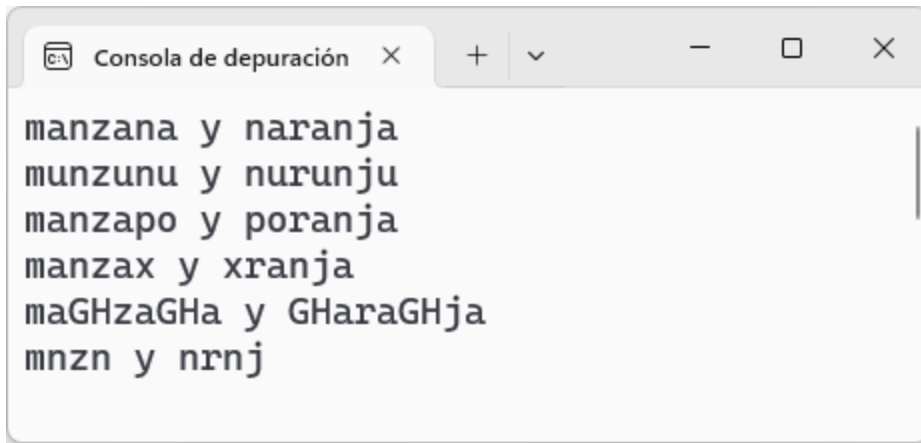
        //Cambia en toda la cadena una letra por otra
        string ReemplazaA = cadena.Replace('a', 'u');
        Console.WriteLine(ReemplazaA);

        //Cambia en toda la cadena una subcadena por otra subcadena
        string ReemplazaB = cadena.Replace("na", "po");
        Console.WriteLine(ReemplazaB);

        //Cambia en toda la cadena una subcadena por una letra
        string ReemplazaC = cadena.Replace("na", "x");
        Console.WriteLine(ReemplazaC);

        //Cambia en toda la cadena una letra por una subcadena
        string ReemplazaD = cadena.Replace("n", "GH");
        Console.WriteLine(ReemplazaD);

        //Cambia en toda la cadena una letra por vacío
        string ReemplazaE = cadena.Replace("a", "");
        Console.WriteLine(ReemplazaE);
    }
}
```



*Ilustración 9: Reemplazar caracteres*

Considerar el siguiente código en C#:

```
string original = "Hola";  
string modificada = original.Replace("H", "M");
```

```
Console.WriteLine(original); // Salida: Hola
```

```
Console.WriteLine(modificada); // Salida: Mola
```

En este ejemplo, el método `Replace` no modifica la cadena original, sino que crea una nueva cadena con el resultado de la operación. La cadena original permanece intacta, demostrando su inmutabilidad.

## Encontrar subcadenas o caracteres

B/010.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Encontrar subcadenas
        string cadena = "manzana y naranja";
        Console.WriteLine(cadena);

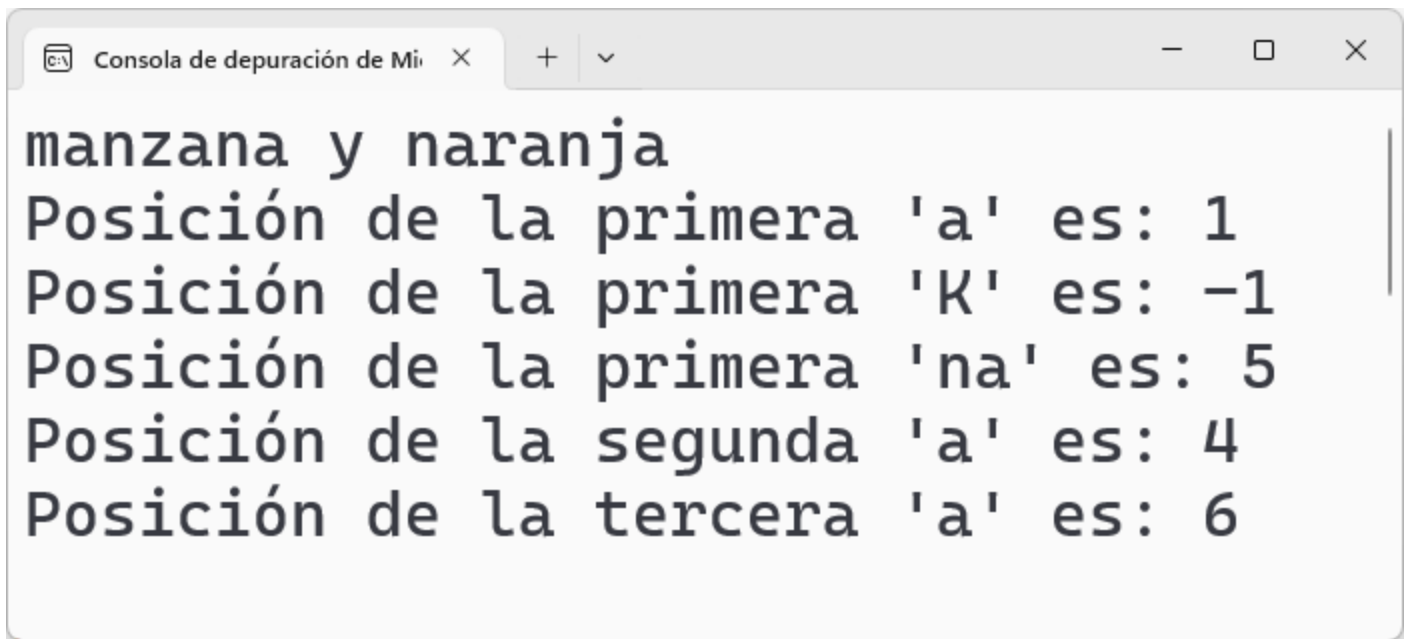
        //Busca la primera posición de la letra "a"
        int posA = cadena.IndexOf('a');
        Console.WriteLine("Posición de la primera 'a' es: " + posA);

        //Busca una letra que no existe
        int posB = cadena.IndexOf('K');
        Console.WriteLine("Posición de la primera 'K' es: " + posB);

        //Busca la primera posición de la subcadena "na"
        int posC = cadena.IndexOf("na");
        Console.WriteLine("Posición de la primera 'na' es: " + posC);

        //Busca la segunda posición de la letra "a"
        int posD = cadena.IndexOf('a', posA + 1);
        Console.WriteLine("Posición de la segunda 'a' es: " + posD);

        //Busca la tercera posición de la letra "a"
        int posE = cadena.IndexOf('a', posD + 1);
        Console.WriteLine("Posición de la tercera 'a' es: " + posE);
    }
}
```



The image shows a web browser window with a single tab titled 'Consola de depuración de Mi'. The developer console is open, displaying the following text:

```
manzana y naranja
Posición de la primera 'a' es: 1
Posición de la primera 'K' es: -1
Posición de la primera 'na' es: 5
Posición de la segunda 'a' es: 4
Posición de la tercera 'a' es: 6
```

*Ilustración 10: Encontrar subcadenas o caracteres*

# Convertir a mayúsculas y minúsculas

B/011.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Encontrar subcadenas
        string cadena = "manzana y naranja";
        Console.WriteLine(cadena);

        //Busca la primera posición de la letra "a"
        int posA = cadena.IndexOf('a');
        Console.WriteLine("Posición de la primera 'a' es: " + posA);

        //Busca una letra que no existe
        int posB = cadena.IndexOf('K');
        Console.WriteLine("Posición de la primera 'K' es: " + posB);

        //Busca la primera posición de la subcadena "na"
        int posC = cadena.IndexOf("na");
        Console.WriteLine("Posición de la primera 'na' es: " + posC);

        //Busca la segunda posición de la letra "a"
        int posD = cadena.IndexOf('a', posA + 1);
        Console.WriteLine("Posición de la segunda 'a' es: " + posD);

        //Busca la tercera posición de la letra "a"
        int posE = cadena.IndexOf('a', posD + 1);
        Console.WriteLine("Posición de la tercera 'a' es: " + posE);
    }
}
```

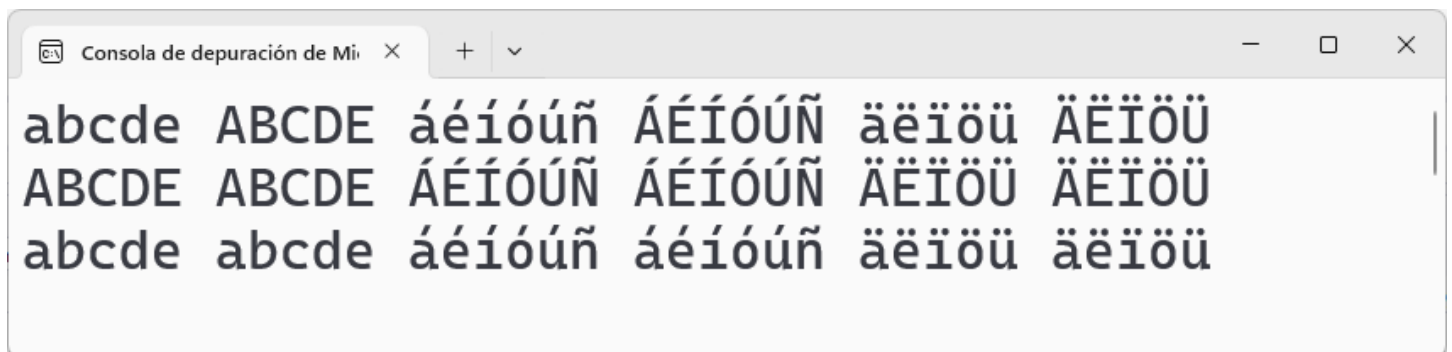


Ilustración 11: Convertir a mayúsculas y minúsculas

# StringBuilder, más veloz y se puede modificar su contenido

B/012.cs

```
using System.Text;
//Librería requerida para StringBuilder

namespace Ejemplo;

internal class Program {
    static void Main() {
        //StringBuilder, mayor velocidad y se puede modificar
        StringBuilder cadenaRapida = new("Prueba");
        Console.WriteLine(cadenaRapida);

        //Cambia el primer caracter
        cadenaRapida[0] = 'e';
        Console.WriteLine(cadenaRapida);

        //Agrega caracteres
        for (int Numero = 0; Numero <= 9; Numero++)
            cadenaRapida.Append(Numero);

        Console.WriteLine(cadenaRapida);
    }
}
```

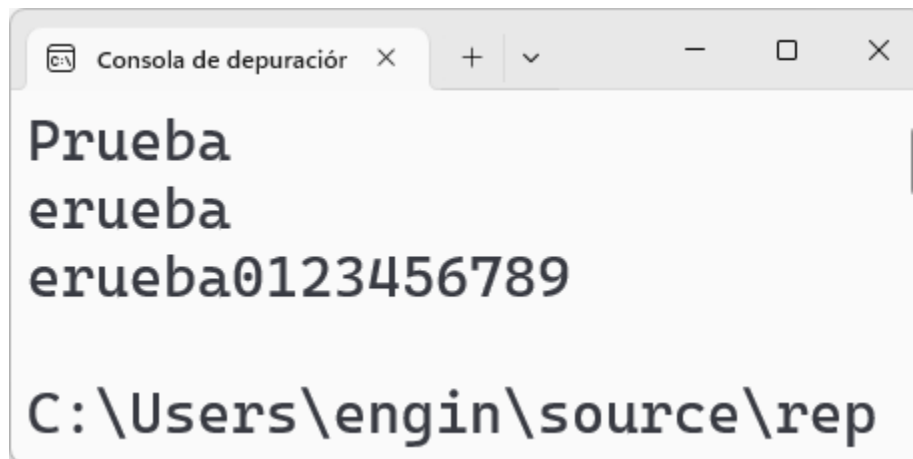


Ilustración 12: StringBuilder, más veloz y se puede modificar su contenido



# StringBuilder, métricas de velocidad

B/013.cs

```
using System.Diagnostics;
using System.Text;

namespace Ejemplo {
    internal class Program {
        static void Main() {
            //StringBuilder, comparando la velocidad
            StringBuilder cadenaRapida = new();
            string cadenaClasica = "";
            int NumLetras = 90000;

            //Medidor de tiempos
            Stopwatch cronometro = new();

            //Agrega caracteres a un StringBuilder
            cronometro.Reset();
            cronometro.Start();

            for (int numero = 1; numero <= NumLetras; numero++)
                cadenaRapida.Append(numero);

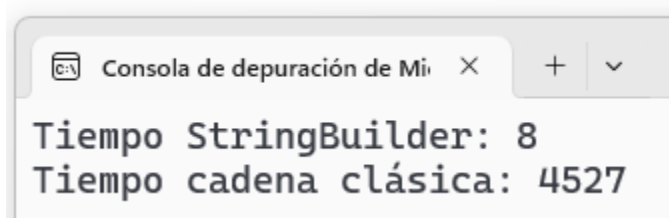
            long TBuilder = cronometro.ElapsedMilliseconds;

            //Agrega caracteres a un string
            cronometro.Reset();
            cronometro.Start();

            for (int numero = 1; numero <= NumLetras; numero++)
                cadenaClasica += numero;

            long TClasica = cronometro.ElapsedMilliseconds;

            if (cadenaClasica == cadenaRapida.ToString()) {
                Console.WriteLine("Tiempo StringBuilder: " + TBuilder);
                Console.WriteLine("Tiempo cadena clásica: " + TClasica);
            }
            else
                Console.WriteLine("Prueba errónea");
        }
    }
}
```



*Ilustración 13: StringBuilder, métricas de velocidad*

# Eliminar caracteres al inicio y al final

B/014.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Cortando espacios
        string cadenaA = "    Con espacios    ";
        Console.WriteLine "[" + cadenaA + "]";

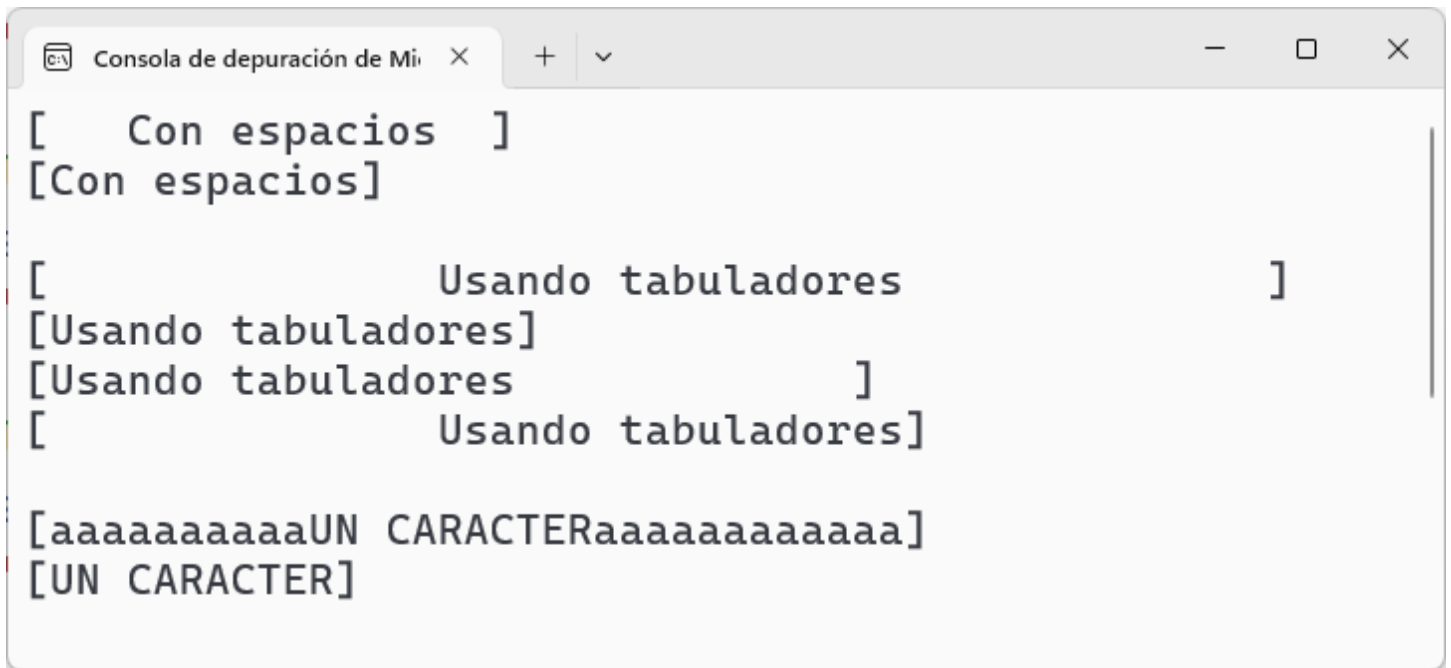
        //Quita los espacios de inicio y fin
        string cadenaB = cadenaA.Trim(' ');
        Console.WriteLine "[" + cadenaB + "]\r\n");

        //¿Y si son tabuladores? Los retira: \t
        string cadenaC = "\t\tUsando tabuladores\t\t";
        Console.WriteLine "[" + cadenaC + "]";
        string cadenaD = cadenaC.Trim('\t');
        Console.WriteLine "[" + cadenaD + "]";

        //Retirar los tabuladores de la izquierda
        string cadenaE = cadenaC.TrimStart('\t');
        Console.WriteLine "[" + cadenaE + "]";

        //Retirar los tabuladores de la derecha
        string cadenaF = cadenaC.TrimEnd('\t');
        Console.WriteLine "[" + cadenaF + "]\r\n");

        //Retirar otro caracter
        cadenaA = "aaaaaaaaaaUN CARACTERaaaaaaaaaaaaa";
        Console.WriteLine "[" + cadenaA + "]";
        string cadenaG = cadenaA.Trim('a');
        Console.WriteLine "[" + cadenaG + "]";
    }
}
```



The image shows a debug console window titled 'Consola de depuración de Mi'. It contains several lines of text demonstrating string formatting with spaces and tabs. The text is as follows:

```
[  Con espacios  ]  
[Con espacios]  
  
[          Usando tabuladores          ]  
[Usando tabuladores]  
[Usando tabuladores          ]  
[          Usando tabuladores]  
  
[aaaaaaaaaaaaUN CARACTERaaaaaaaaaaaaa]  
[UN CARACTER]
```

*Ilustración 14: Eliminar caracteres al inicio y al final*

## Comparar cadenas. Forma no recomendada

No se recomienda hacer uso del operador `==` al usar cadenas.

B/015.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Comparar cadenas
        string cadenaA = "abcdefghij";
        string cadenaB = "Abcdefghij";
        string cadenaC = "abcdefghij ";
        string cadenaD = "abcdefg hij";

        //Forma 1 de comparar. No recomendada.
        if (cadenaA == cadenaB)
            Console.WriteLine("1. Iguales");
        else
            Console.WriteLine("1. Diferentes");

        if (cadenaA == cadenaC)
            Console.WriteLine("2. Iguales");
        else
            Console.WriteLine("2. Diferentes");

        if (cadenaA == cadenaD)
            Console.WriteLine("3. Iguales");
        else
            Console.WriteLine("3. Diferentes");
    }
}
```



Ilustración 15: Comparar cadenas. Forma no recomendada

## Comparar cadenas. Forma recomendada

Para comparar es mejor usar el operador Equals

B/016.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Comparar cadenas
        string cadenaA = "abcdefghijkl";
        string cadenaB = "Abcdefghij";
        string cadenaC = "abcdefghijkl ";
        string cadenaD = "abcdefg hij";

        //Forma 2 de comparar. Recomendada.
        if (cadenaA.Equals(cadenaB))
            Console.WriteLine("1. Iguales");
        else
            Console.WriteLine("1. Diferentes");

        if (cadenaA.Equals(cadenaC))
            Console.WriteLine("2. Iguales");
        else
            Console.WriteLine("2. Diferentes");

        if (cadenaA.Equals(cadenaD))
            Console.WriteLine("3. Iguales");
        else
            Console.WriteLine("3. Diferentes");
    }
}
```



Ilustración 16: Comparar cadenas. Forma recomendada

# Comparar cadenas. Ignorando las mayúsculas y minúsculas

B/017.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Comparar cadenas
        string cadenaA = "abcdefghij";
        string cadenaB = "Abcdefghij";
        string cadenaC = "aBCDEfghiJ";
        string cadenaD = "ABCDEFGHIj";

        //Forma 2 de comparar ignorando mayúsculas y minúsculas
        if (cadenaA.Equals(cadenaB, StringComparison.OrdinalIgnoreCase))
            Console.WriteLine("1. Iguales");
        else
            Console.WriteLine("1. Diferentes");

        if (cadenaA.Equals(cadenaC, StringComparison.OrdinalIgnoreCase))
            Console.WriteLine("2. Iguales");
        else
            Console.WriteLine("2. Diferentes");

        if (cadenaA.Equals(cadenaD, StringComparison.OrdinalIgnoreCase))
            Console.WriteLine("3. Iguales");
        else
            Console.WriteLine("3. Diferentes");
    }
}

//Más información: https://docs.microsoft.com/en-us/dotnet/csharp/how-to/compare-strings
```

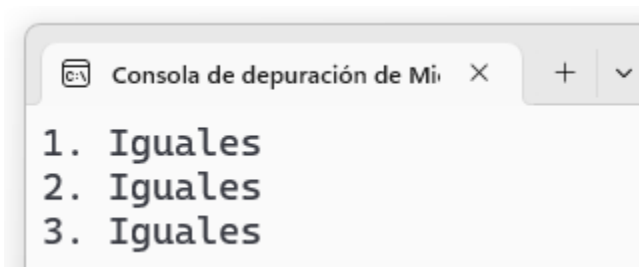
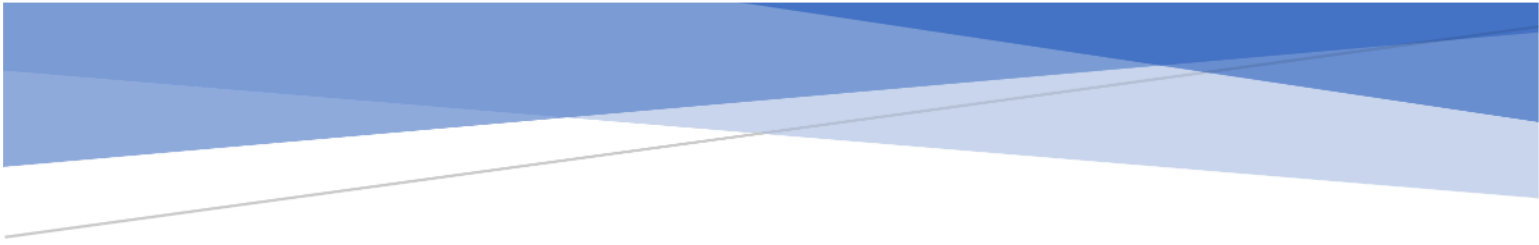


Ilustración 17: Comparar cadenas. Ignorando las mayúsculas y minúsculas



# C# Y .NET 10

## Parte 03. Arreglos estáticos

# 2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com



## Contenido

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                                        | 3  |
| Acerca del autor.....                                                              | 4  |
| Licencia de este libro .....                                                       | 4  |
| Licencia del software .....                                                        | 4  |
| Marcas registradas .....                                                           | 5  |
| Dedicatoria .....                                                                  | 6  |
| Declaración y asignar valores .....                                                | 7  |
| Otra forma de definir y asignar.....                                               | 9  |
| Arreglo unidimensional de cadenas .....                                            | 10 |
| Arreglo unidimensional. Tamaños.....                                               | 11 |
| Recorrer arreglos de cadenas .....                                                 | 12 |
| Uso de la instrucción foreach para recorrer un arreglo .....                       | 13 |
| Ordenar un arreglo de cadenas.....                                                 | 14 |
| Ordenar un arreglo de cadenas. Las mayúsculas, minúsculas, tildes y diéresis ..... | 15 |
| Ordenar un arreglo de tipo double .....                                            | 16 |
| Funciones genéricas para arreglos unidimensionales .....                           | 17 |
| Algoritmos de ordenación .....                                                     | 19 |
| Métrica de velocidad: Algoritmos de ordenación .....                               | 24 |
| Arreglo bidimensional .....                                                        | 29 |
| Arreglo tridimensional.....                                                        | 31 |
| Arreglo de arreglos.....                                                           | 32 |
| Métrica de velocidad: arreglo bidimensional vs arreglo de arreglos .....           | 34 |
| Métrica de velocidad: Arreglos multidimensionales.....                             | 36 |
| Tipo implícito de arreglo .....                                                    | 39 |
| Tipo implícito en arreglo de arreglos.....                                         | 40 |
| Convertir una cadena en un arreglo de cadenas al dividirla .....                   | 41 |

## Tabla de ilustraciones

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| Ilustración 1: Declaración y asignar valores .....                                              | 8  |
| Ilustración 2: Otra forma de definir y asignar.....                                             | 9  |
| Ilustración 3: Arreglo unidimensional de cadenas .....                                          | 10 |
| Ilustración 4: Arreglo unidimensional. Tamaños.....                                             | 11 |
| Ilustración 5: Recorrer arreglos de cadenas .....                                               | 12 |
| Ilustración 6: Uso de la instrucción foreach para recorrer un arreglo.....                      | 13 |
| Ilustración 7: Ordenar un arreglo de cadenas .....                                              | 14 |
| Ilustración 8: Ordenar un arreglo de cadenas. Las mayúsculas, minúsculas, tildes y diéresis ... | 15 |
| Ilustración 9: Ordenar un arreglo de tipo double .....                                          | 16 |
| Ilustración 10: Funciones genéricas para arreglos unidimensionales .....                        | 18 |
| Ilustración 11: Algoritmos de ordenación .....                                                  | 23 |
| Ilustración 12: Métrica de velocidad: Algoritmos de ordenación .....                            | 28 |
| Ilustración 13: Arreglo bidimensional.....                                                      | 30 |
| Ilustración 14: Arreglo tridimensional .....                                                    | 31 |
| Ilustración 15: Arreglo de arreglos .....                                                       | 33 |
| Ilustración 16: Métrica de velocidad: arreglo bidimensional vs arreglo de arreglos .....        | 35 |
| Ilustración 17: Métrica de velocidad: arreglos multidimensionales .....                         | 38 |
| Ilustración 18: Tipo implícito de arreglo .....                                                 | 39 |
| Ilustración 19: Tipo implícito en arreglo de arreglos .....                                     | 40 |
| Ilustración 20: Convertir una cadena en un arreglo de cadenas al dividirla.....                 | 41 |
| Ilustración 21: Cuando hay más de un espacio intermedio .....                                   | 43 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

# Declaración y asignar valores

Los arreglos unidimensionales empiezan en la posición 0

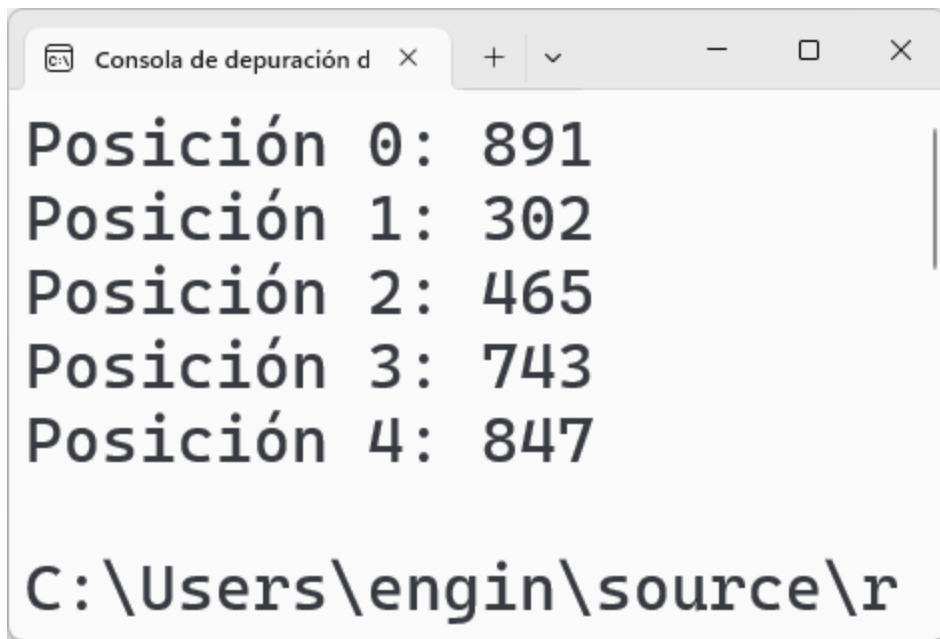
C/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional. Definición.
        int[] arreglo = new int[5];

        //Asignando valores a cada posición del arreglo
        arreglo[0] = 891;
        arreglo[1] = 302;
        arreglo[2] = 465;
        arreglo[3] = 743;
        arreglo[4] = 847;

        //Imprimiendo valores
        Console.WriteLine("Posición 0: " + arreglo[0]);
        Console.WriteLine("Posición 1: " + arreglo[1]);
        Console.WriteLine("Posición 2: " + arreglo[2]);
        Console.WriteLine("Posición 3: " + arreglo[3]);
        Console.WriteLine("Posición 4: " + arreglo[4]);
    }
}
```



The image shows a screenshot of a Windows Debug Console window. The title bar reads 'Consola de depuración d' followed by a close button. The window contains five lines of text representing array elements: 'Posición 0: 891', 'Posición 1: 302', 'Posición 2: 465', 'Posición 3: 743', and 'Posición 4: 847'. Below these, there is a file path: 'C:\Users\engin\source\r'. The text is in a monospaced font.

```
Posición 0: 891
Posición 1: 302
Posición 2: 465
Posición 3: 743
Posición 4: 847

C:\Users\engin\source\r
```

*Ilustración 1: Declaración y asignar valores*

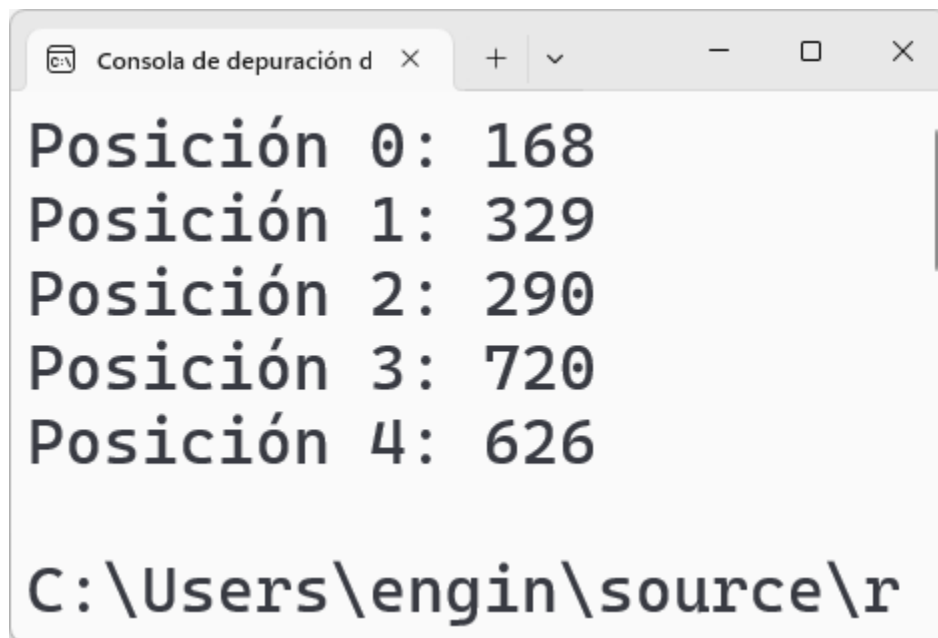
## Otra forma de definir y asignar

C/002.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional. Definición y asignación.
        int[] arreglo = { 168, 329, 290, 720, 626 };

        //Imprimiendo valores
        Console.WriteLine("Posición 0: " + arreglo[0]);
        Console.WriteLine("Posición 1: " + arreglo[1]);
        Console.WriteLine("Posición 2: " + arreglo[2]);
        Console.WriteLine("Posición 3: " + arreglo[3]);
        Console.WriteLine("Posición 4: " + arreglo[4]);
    }
}
```



```
Consola de depuración d
Posición 0: 168
Posición 1: 329
Posición 2: 290
Posición 3: 720
Posición 4: 626

C:\Users\engin\source\r
```

Ilustración 2: Otra forma de definir y asignar



# Arreglo unidimensional de cadenas

C/003.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional de cadenas.
        string[] cadenas = ["Sally", "Suini", "Grisú", "Arian"];

        //Imprimiendo valores
        Console.WriteLine("Posición 0: " + cadenas[0]);
        Console.WriteLine("Posición 1: " + cadenas[1]);
        Console.WriteLine("Posición 2: " + cadenas[2]);
        Console.WriteLine("Posición 3: " + cadenas[3]);
    }
}
```



*Ilustración 3: Arreglo unidimensional de cadenas*

# Arreglo unidimensional. Tamaños

C/004.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional. Tamaños.
        string[] cadenas = ["naranja", "manzana", "pera", "coco"];
        int[] numeros = new int[8];

        //Tamaños de los arreglos
        int TCadenas = cadenas.Length;
        int TNumeros = numeros.Length;

        //Imprime
        Console.WriteLine("Tamaño arreglo cadenas: " + TCadenas);
        Console.WriteLine("Tamaño arreglo numeros: " + TNumeros);
    }
}
```

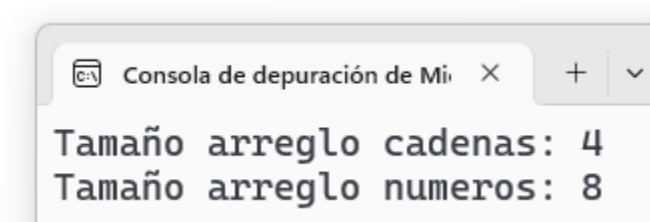


Ilustración 4: Arreglo unidimensional. Tamaños

# Recorrer arreglos de cadenas

C/005.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional
        string[] cadenas = ["naranja", "manzana", "pera", "coco"];

        //Tamaño
        int TCadenas = cadenas.Length;

        //Recorre el arreglo y lo imprime
        for (int pos = 0; pos < TCadenas; pos++) {
            Console.WriteLine(cadenas[pos]);
        }
    }
}
```

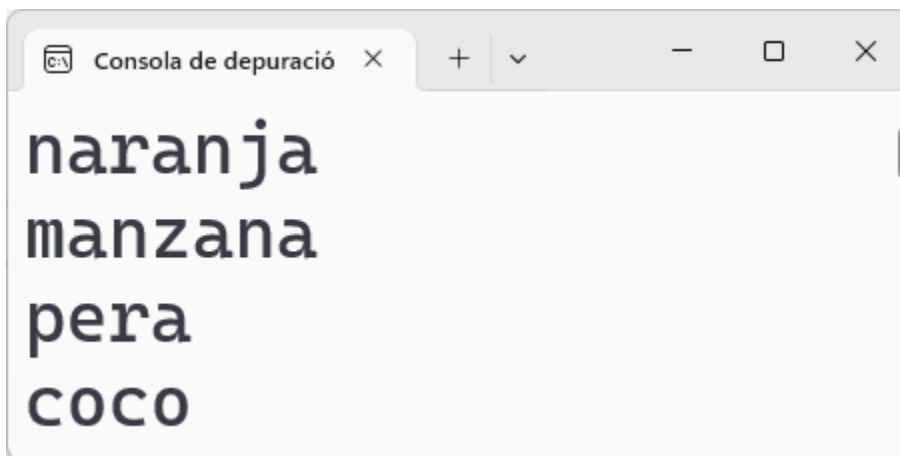


Ilustración 5: Recorrer arreglos de cadenas

## Uso de la instrucción foreach para recorrer un arreglo

C/006.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional
        string[] cadenas = ["naranja", "manzana", "pera", "coco"];

        //Recorre el arreglo y lo imprime
        foreach (string texto in cadenas) {
            Console.WriteLine(texto);
        }
    }
}
```

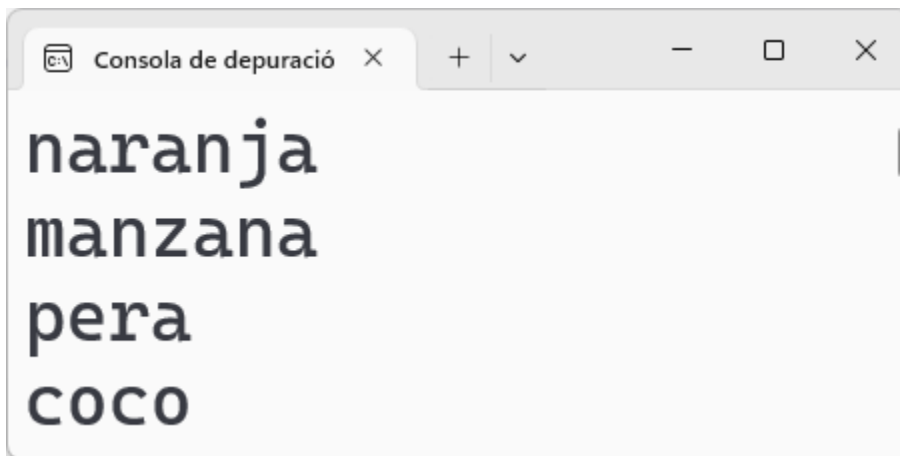


Ilustración 6: Uso de la instrucción foreach para recorrer un arreglo

# Ordenar un arreglo de cadenas

C/007.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional
        string[] cadenas = ["mil", "ufr", "ac", "ari", "an"];

        //Ordena el arreglo
        Array.Sort(cadenas);

        //Recorre el arreglo y lo imprime
        foreach (string texto in cadenas) {
            Console.WriteLine(texto);
        }
    }
}
```

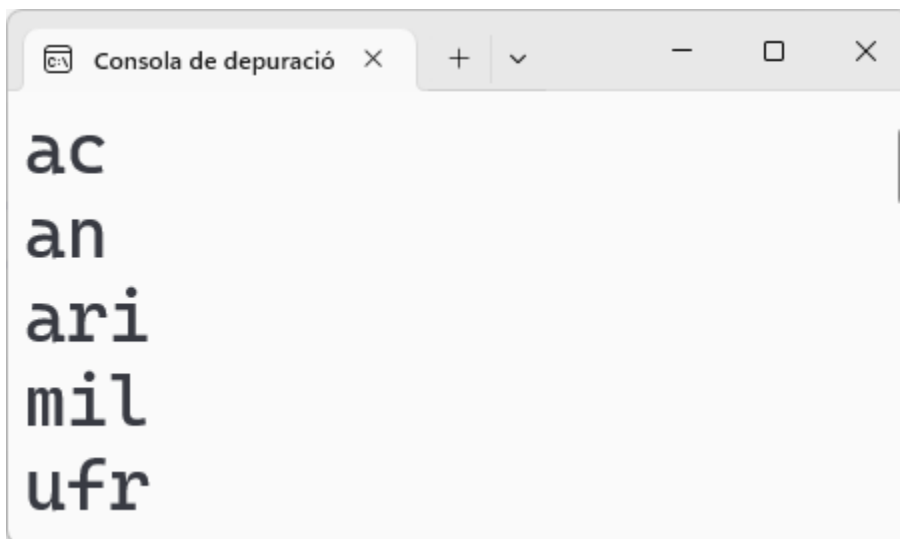


Ilustración 7: Ordenar un arreglo de cadenas

## Ordenar un arreglo de cadenas. Las mayúsculas, minúsculas, tildes y diéresis

C/008.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Arreglo unidimensional
        string[] cadenas = ["áa", "aa", "äa", "Aa", "aá"];

        //Ordena el arreglo
        Array.Sort(cadenas);

        //Recorre el arreglo y lo imprime
        foreach (string texto in cadenas) {
            Console.WriteLine(texto);
        }
    }
}
```

En caso de probar las tildes, diéresis, mayúsculas y minúsculas, se obtendría este resultado:

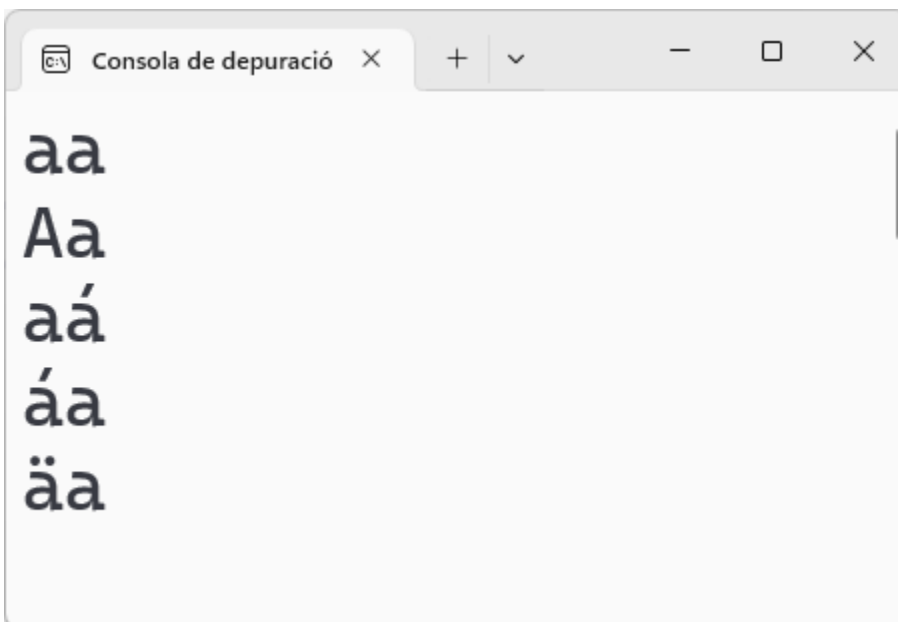


Ilustración 8: Ordenar un arreglo de cadenas. Las mayúsculas, minúsculas, tildes y diéresis

## Ordenar un arreglo de tipo double

C/009.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        Random Azar = new();

        //Llena un arreglo con valores aleatorios
        double[] Numeros = new double[10];
        for (int cont = 0; cont < Numeros.Length; cont++)
            Numeros[cont] = Azar.NextDouble();

        //Ordena el arreglo
        Array.Sort(Numeros);

        //Recorre el arreglo y lo imprime
        foreach (double unvalor in Numeros) {
            Console.WriteLine(unvalor);
        }
    }
}
```

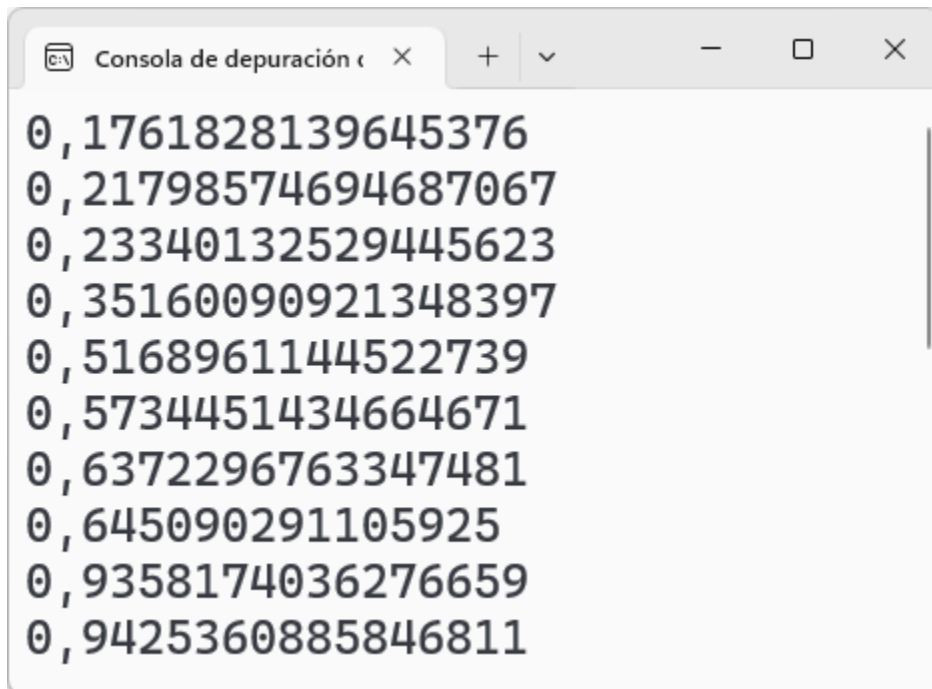


Ilustración 9: Ordenar un arreglo de tipo double

# Funciones genéricas para arreglos unidimensionales

Se crean funciones que se envía como parámetro el arreglo

C/010.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Arreglo unidimensional. Funciones genéricas
        int[] numeros = new int[50];

        //Llena con valores al azar
        int minimo = 1, maximo = 30;
        LlenaArreglo(numeros, minimo, maximo);
        ImprimeArreglo(numeros);

        //Busca un valor determinado y retorna su posición
        int valorBusca = 17;
        int pos = PosArregloDato(numeros, valorBusca);
        if (pos == -1)
            Console.WriteLine("Valor no encontrado");
        else {
            Console.Write("Valor " + valorBusca);
            Console.WriteLine(" en posición: " + pos);
        }
    }

    //Llena el arreglo con valores al azar
    //entre min y max (ambos incluidos)
    static void LlenaArreglo(int[] arreglo, int min, int max) {
        Random azar = new();
        for (int pos = 0; pos < arreglo.Length; pos++) {
            arreglo[pos] = azar.Next(min, max + 1);
        }
    }

    //Imprime el arreglo en consola
    static void ImprimeArreglo(int[] arreglo) {
        for (int pos = 0; pos < arreglo.Length; pos++) {
            Console.Write(arreglo[pos] + " ; ");
        }
        Console.WriteLine(" ");
    }

    //Retorna la posición del dato en el arreglo
}
```



```

//o retorna -1 si no lo pos
static int PosArregloDato(int[] arreglo, int valor) {
    for (int pos = 0; pos < arreglo.Length; pos++) {
        if (arreglo[pos] == valor)
            return pos;
    }
    return -1;
}
}

```

The screenshot shows a 'Consola de depuración' (Debug Console) window. It contains a list of numbers arranged in a grid-like fashion, separated by semicolons. The numbers are: 20, 26, 19, 12, 19, 30, 3, 0, 2, 17, 1, 7, 21, 15, 5, 3, 14, 27, 3, 8, 16, 8, 29, 14, 14, 21, 19, 29, 8, 22, 25, 26, 16, 10, 13, 3, 17, 15, 26, 21, 2, 9, 23, 3, 19, 6, 21, 23, 24, 3, 18, 11. Below this list, it says 'Valor 17 en posición: 8', indicating that the value 17 was found at index 8.

Ilustración 10: Funciones genéricas para arreglos unidimensionales

# Algoritmos de ordenación

Se prueban diversos algoritmos de ordenación enviando arreglos como parámetros y estos arreglos son modificados por las funciones mismas. Al enviar un arreglo como parámetro a una función o procedimiento, este es enviado por referencia (**no se copian los valores** como sucede con los tipos de datos nativos como int, double, float, string). Luego se debe tener especial cuidado con esto porque si la función o procedimiento modifica el arreglo, quedaría modificado también desde el sitio en que se llamó a la función o procedimiento.

C/011.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Ordenación
        int Limite = 10;
        int[] Listado = new int[Limite];

        int minimo = 10, maximo = 99;

        //Ordena por Inserción
        Console.WriteLine("\nOrdena por inserción");
        LlenaArreglo(Listado, minimo, maximo);
        ImprimeArreglo("Original", Listado);
        Insercion(Listado);
        ImprimeArreglo("Ordenado", Listado);

        //Ordena por Selección
        Console.WriteLine("\nOrdena por selección");
        LlenaArreglo(Listado, minimo, maximo);
        Array.Copy(Listado, 0, Listado, 0, Listado.Length);
        ImprimeArreglo("Original", Listado);
        Seleccion(Listado);
        ImprimeArreglo("Ordenado", Listado);

        //Ordena por Burbuja
        Console.WriteLine("\nOrdena por burbuja");
        LlenaArreglo(Listado, minimo, maximo);
        ImprimeArreglo("Original", Listado);
        Burbuja(Listado);
        ImprimeArreglo("Ordenado", Listado);

        //Ordena por Shell
        Console.WriteLine("\nOrdena por shell");
        LlenaArreglo(Listado, minimo, maximo);
        ImprimeArreglo("Original", Listado);
```

```

Shell(Listado);
ImprimeArreglo("Ordenado", Listado);

//Ordena por QuickSort
Console.WriteLine("\nOrdena por quicksort");
LlenaArreglo(Listado, minimo, maximo);
ImprimeArreglo("Original", Listado);
QuickSort(Listado, 0, Listado.Length - 1);
ImprimeArreglo("Ordenado", Listado);
}

//Llena el arreglo con valores al azar
//entre min y max (ambos incluidos)
static void LlenaArreglo(int[] arreglo, int min, int max) {
    Random azar = new Random();
    for (int pos = 0; pos < arreglo.Length; pos++) {
        arreglo[pos] = azar.Next(min, max + 1);
    }
}

//Imprime el arreglo en consola
static void ImprimeArreglo(string Texto, int[] arreglo) {
    Console.Write(Texto + ": ");
    for (int pos = 0; pos < arreglo.Length; pos++) {
        Console.Write(arreglo[pos] + ";");
    }
    Console.WriteLine(" ");
}

//Ordenamiento por Insert
static void Insercion(int[] arreglo) {
    int j;
    for (int i = 1; i < arreglo.Length; i++) {
        int tmp = arreglo[i];
        for (j = i - 1; j >= 0 && arreglo[j] > tmp; j--) {
            arreglo[j + 1] = arreglo[j];
        }
        arreglo[j + 1] = tmp;
    }
}

//Ordenamiento por Selección
static void Seleccion(int[] arreglo) {
    for (int i = 0; i < arreglo.Length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < arreglo.Length; j++) {
            if (arreglo[j] < arreglo[min]) {
                min = j;
            }
        }
        //Intercambio
        int tmp = arreglo[i];
        arreglo[i] = arreglo[min];
        arreglo[min] = tmp;
    }
}

```

```

        }
    }
    if (i != min) {
        int aux = arreglo[i];
        arreglo[i] = arreglo[min];
        arreglo[min] = aux;
    }
}

//Ordenamiento por Burbuja
static void Burbuja(int[] arreglo) {
    int n = arreglo.Length;
    int tmp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (arreglo[j] > arreglo[j + 1]) {
                tmp = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = tmp;
            }
        }
    }
}

//Ordenamiento por Shell
static void Shell(int[] arr) {
    int incr = arr.Length;
    do {
        incr /= 2;
        for (int k = 0; k < incr; k++) {
            for (int i = incr + k; i < arr.Length; i += incr) {
                int j = i;
                while (j - incr >= 0 && arr[j] < arr[j - incr]) {
                    int tmp = arr[j];
                    arr[j] = arr[j - incr];
                    arr[j - incr] = tmp;
                    j -= incr;
                }
            }
        }
    } while (incr > 1);
}

//Ordenación por QuickSort
static void QuickSort(int[] arreglo, int primero, int ultimo) {
    int i, j, central;
    int pivote;

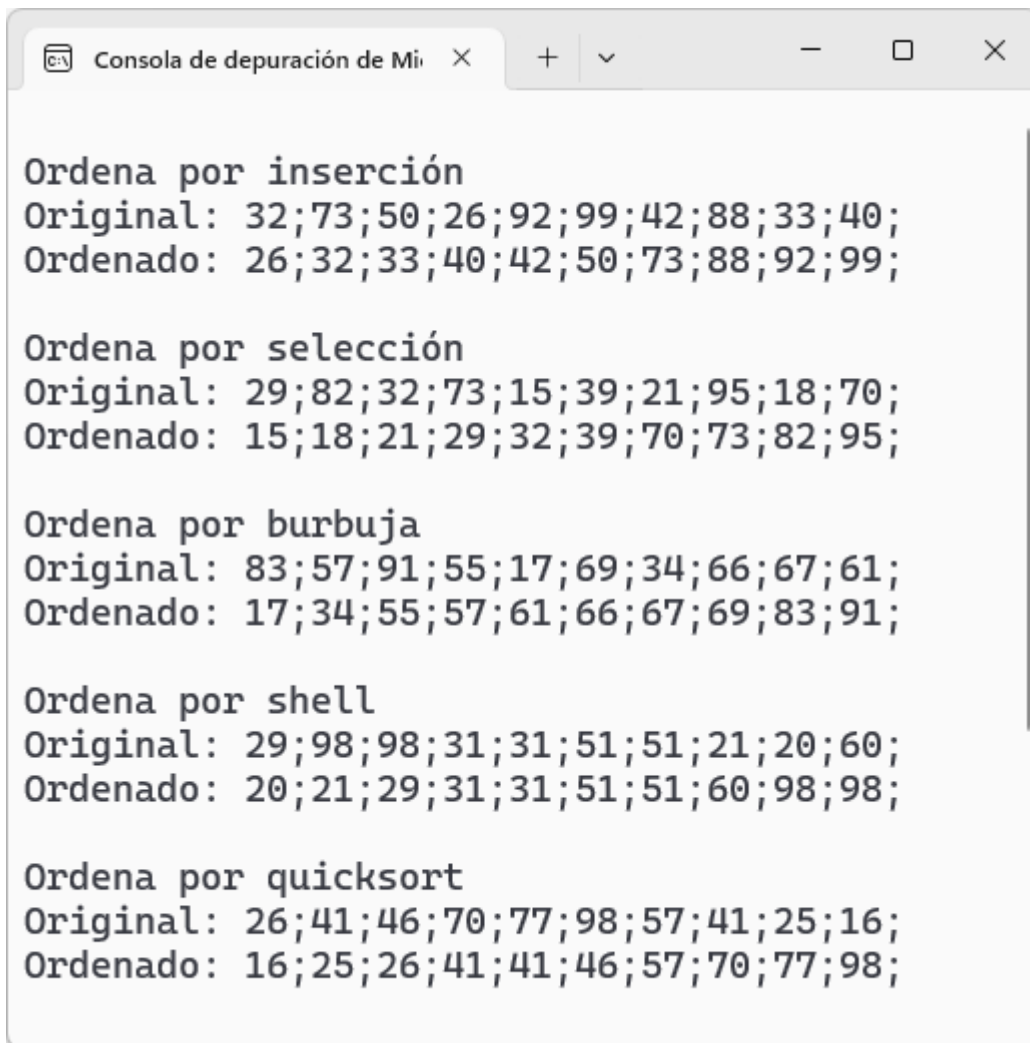
```

```

    central = (primero + ultimo) / 2;
    pivote = arreglo[central];
    i = primero;
    j = ultimo;
    do {
        while (arreglo[i] < pivote) i++;
        while (arreglo[j] > pivote) j--;
        if (i <= j) {
            int tmp = arreglo[i];
            arreglo[i] = arreglo[j];
            arreglo[j] = tmp;
            i++;
            j--;
        }
    } while (i <= j);

    if (primero < j) {
        QuickSort(arreglo, primero, j);
    }
    if (i < ultimo) {
        QuickSort(arreglo, i, ultimo);
    }
}
}

```



```
Consola de depuración de Mi X + v - □ X

Ordena por inserción
Original: 32;73;50;26;92;99;42;88;33;40;
Ordenado: 26;32;33;40;42;50;73;88;92;99;

Ordena por selección
Original: 29;82;32;73;15;39;21;95;18;70;
Ordenado: 15;18;21;29;32;39;70;73;82;95;

Ordena por burbuja
Original: 83;57;91;55;17;69;34;66;67;61;
Ordenado: 17;34;55;57;61;66;67;69;83;91;

Ordena por shell
Original: 29;98;98;31;31;51;51;21;20;60;
Ordenado: 20;21;29;31;31;51;51;60;98;98;

Ordena por quicksort
Original: 26;41;46;70;77;98;57;41;25;16;
Ordenado: 16;25;26;41;41;46;57;70;77;98;
```

*Ilustración 11: Algoritmos de ordenación*

```
using System.Diagnostics;

namespace Ejemplo;

internal class Program {
    static void Main() {
        int Limite = 30000;
        int[] Original = new int[Limite];
        int[] LShell = new int[Limite];
        int[] LInsercion = new int[Limite];
        int[] LSeleccion = new int[Limite];
        int[] LBurbuja = new int[Limite];
        int[] LQuickSort = new int[Limite];

        //Medidor de tiempos
        double TPShell = 0, TPIns = 0, TPSel = 0, TPBur = 0, TPQuick = 0;

        //Para disminuir oscilaciones en el tiempo, se hacen
        //N pruebas con cada grupo de pruebas
        int TotalPruebas = 20;
        for (int prueba = 1; prueba <= TotalPruebas; prueba++) {
            LlenaArreglo(Original, 10, 90);

            //Ordenación por método Shell
            Array.Copy(Original, 0, LShell, 0, Original.Length);
            long Inicia = Stopwatch.GetTimestamp();
            Shell(LShell);
            TimeSpan Transcurrido = Stopwatch.GetElapsedTime(Inicia);
            TPShell += Transcurrido.TotalMilliseconds;

            //Ordenación por método Inserción
            Array.Copy(Original, 0, LInsercion, 0, Original.Length);
            Inicia = Stopwatch.GetTimestamp();
            Insercion(LInsercion);
            Transcurrido = Stopwatch.GetElapsedTime(Inicia);
            TPIns += Transcurrido.TotalMilliseconds;

            //Ordenación por método Selección
            Array.Copy(Original, 0, LSeleccion, 0, Original.Length);
            Inicia = Stopwatch.GetTimestamp();
            Seleccion(LSeleccion);
            Transcurrido = Stopwatch.GetElapsedTime(Inicia);
            TPSel += Transcurrido.TotalMilliseconds;
        }
    }
}
```

```

//Ordenación por método Burbuja
Array.Copy(Original, 0, LBurbuja, 0, Original.Length);
Inicia = Stopwatch.GetTimestamp();
Burbuja(LBurbuja);
Transcurrido = Stopwatch.GetElapsedTime(Inicia);
TPBur += Transcurrido.TotalMilliseconds;

//Ordenación por método QuickSort
Array.Copy(Original, 0, LQuickSort, 0, Original.Length);
Inicia = Stopwatch.GetTimestamp();
QuickSort(LQuickSort, 0, LQuickSort.Length - 1);
Transcurrido = Stopwatch.GetElapsedTime(Inicia);
TPQuick += Transcurrido.TotalMilliseconds;

//Verifica que los arreglos ordenados coinciden
for (int cont = 0; cont < Original.Length; cont++) {
    if (LShell[cont] != LInsercion[cont] ||
        LInsercion[cont] != LSeleccion[cont] ||
        LSeleccion[cont] != LBurbuja[cont] ||
        LBurbuja[cont] != LQuickSort[cont])
        Console.WriteLine("Error en la prueba");
}
}

double TS = (double)TPShell / TotalPruebas;
double TI = (double)TPIns / TotalPruebas;
double TL = (double)TPSel / TotalPruebas;
double TB = (double)TPBur / TotalPruebas;
double TQ = (double)TPQuick / TotalPruebas;

Console.WriteLine("Número de elementos a ordenar: " + Limite);
Console.WriteLine("Tiempo promedio en milisegundos");
Console.WriteLine("ShellSort: " + TS);
Console.WriteLine("InsertSort: " + TI);
Console.WriteLine("Selección: " + TL);
Console.WriteLine("Burbuja: " + TB);
Console.WriteLine("QuickSort: " + TQ);
}

//Llena el arreglo con valores al azar entre min y max
static void LlenaArreglo(int[] arreglo, int min, int max) {
    Random azar = new();
    for (int posicion = 0; posicion < arreglo.Length; posicion++) {
        arreglo[posicion] = azar.Next(min, max + 1);
    }
}

//Ordenamiento por Insert

```



```

static void Insercion(int[] arreglo) {
    int j;
    for (int i = 1; i < arreglo.Length; i++) {
        int tmp = arreglo[i];
        for (j = i - 1; j >= 0 && arreglo[j] > tmp; j--) {
            arreglo[j + 1] = arreglo[j];
        }
        arreglo[j + 1] = tmp;
    }
}

//Ordenamiento por Selección
static void Seleccion(int[] arreglo) {
    for (int i = 0; i < arreglo.Length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < arreglo.Length; j++) {
            if (arreglo[j] < arreglo[min]) {
                min = j;
            }
        }
        if (i != min) {
            int aux = arreglo[i];
            arreglo[i] = arreglo[min];
            arreglo[min] = aux;
        }
    }
}

//Ordenamiento por Burbuja
static void Burbuja(int[] arreglo) {
    int n = arreglo.Length;
    int tmp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1; j++) {
            if (arreglo[j] > arreglo[j + 1]) {
                tmp = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = tmp;
            }
        }
    }
}

//Ordenamiento por Shell
static void Shell(int[] arr) {
    int incr = arr.Length;
    do {
        incr /= 2;
    }
}

```

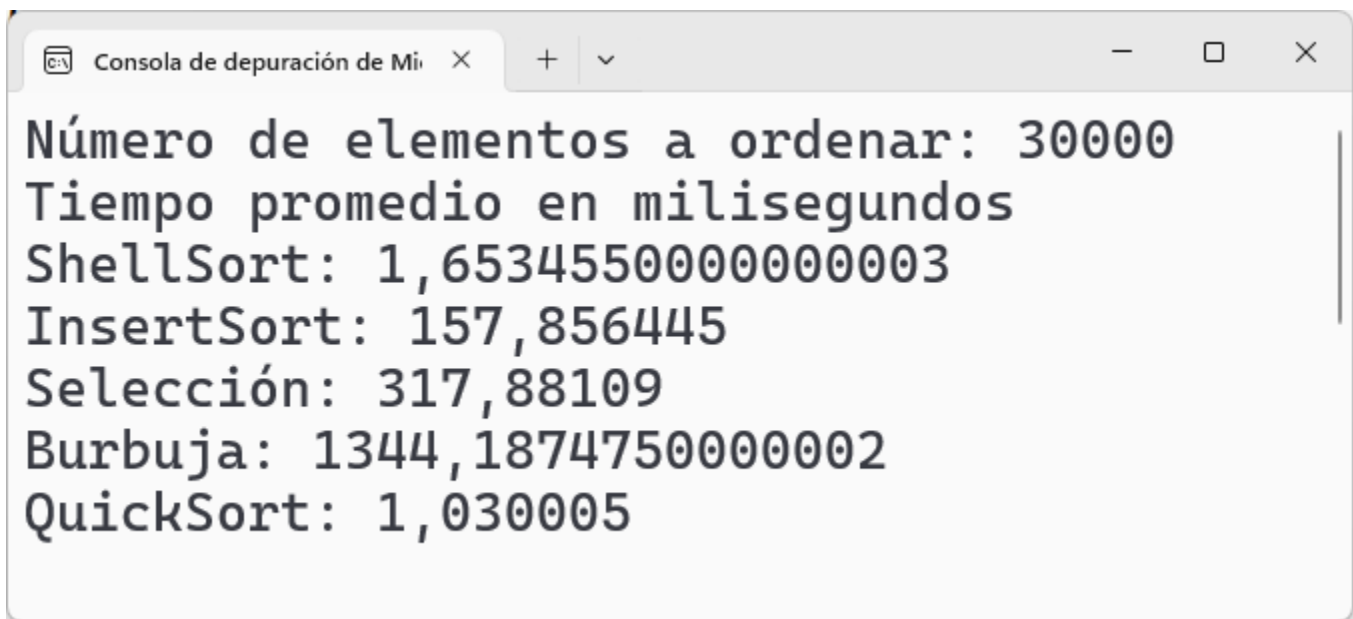
```

        for (int k = 0; k < incr; k++) {
            for (int i = incr + k; i < arr.Length; i += incr) {
                int j = i;
                while (j - incr >= 0 && arr[j] < arr[j - incr]) {
                    int tmp = arr[j];
                    arr[j] = arr[j - incr];
                    arr[j - incr] = tmp;
                    j -= incr;
                }
            }
        }
    } while (incr > 1);
}

//Ordenación por QuickSort
static void QuickSort(int[] arreglo, int primero, int ultimo) {
    int i, j, central;
    int pivote;
    central = (primero + ultimo) / 2;
    pivote = arreglo[central];
    i = primero;
    j = ultimo;
    do {
        while (arreglo[i] < pivote) i++;
        while (arreglo[j] > pivote) j--;
        if (i <= j) {
            int tmp = arreglo[i];
            arreglo[i] = arreglo[j];
            arreglo[j] = tmp;
            i++;
            j--;
        }
    } while (i <= j);

    if (primero < j) {
        QuickSort(arreglo, primero, j);
    }
    if (i < ultimo) {
        QuickSort(arreglo, i, ultimo);
    }
}
}

```



A screenshot of a debug console window titled 'Consola de depuración de Mi'. The window contains the following text:

```
Número de elementos a ordenar: 30000  
Tiempo promedio en milisegundos  
ShellSort: 1,6534550000000003  
InsertSort: 157,856445  
Selección: 317,88109  
Burbuja: 1344,1874750000002  
QuickSort: 1,030005
```

*Ilustración 12: Métrica de velocidad: Algoritmos de ordenación*

# Arreglo bidimensional

C/013.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        int TotalFilas = 5;
        int TotalColumnas = 10;

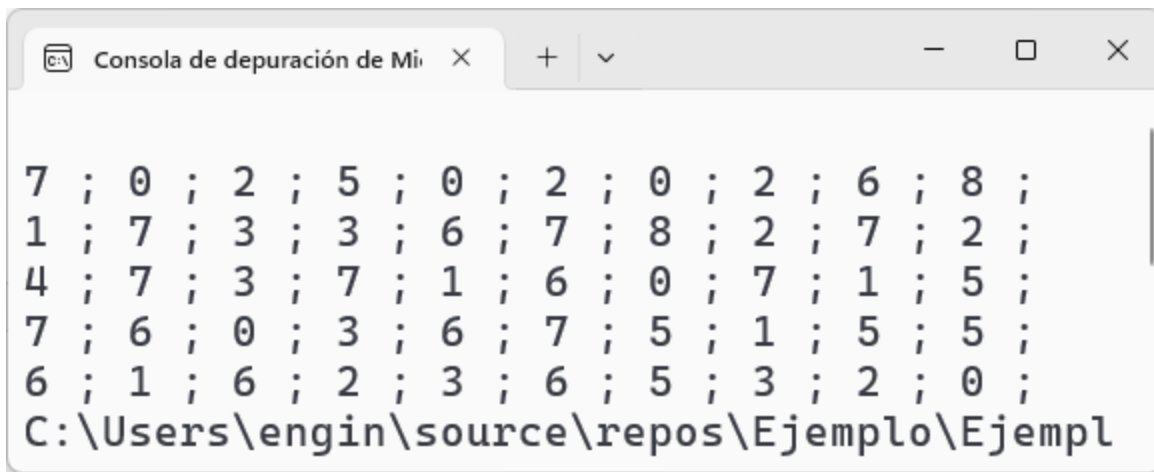
        //Declara un arreglo bidimensional
        int[,] Tablero = new int[TotalFilas, TotalColumnas];

        /* Llena ese arreglo bidimensional
           Retorna el número de filas (la primera dimensión)
           Tablero.GetLength(0)

           Retorna el número de columnas (la segunda dimensión)
           Tablero.GetLength(1)

           Un arreglo bidimensional inicia en [0,0] y
           termina en [TotalFilas-1, TotalColumnas-1]
        */
        Random azar = new();
        for (int fila = 0; fila < Tablero.GetLength(0); fila++)
            for (int col = 0; col < Tablero.GetLength(1); col++)
                Tablero[fila, col] = azar.Next(0, 9);

        //Imprime ese arreglo bidimensional
        for (int fila = 0; fila < Tablero.GetLength(0); fila++) {
            Console.WriteLine(" ");
            for (int col = 0; col < Tablero.GetLength(1); col++)
                Console.Write(Tablero[fila, col] + " ; ");
        }
    }
}
```



The image shows a screenshot of a Windows Debug Console window. The title bar reads "Consola de depuración de Mi" followed by a close button. The window contains a 5x10 grid of numbers separated by semicolons, and a file path below it.

```
7 ; 0 ; 2 ; 5 ; 0 ; 2 ; 0 ; 2 ; 6 ; 8 ;  
1 ; 7 ; 3 ; 3 ; 6 ; 7 ; 8 ; 2 ; 7 ; 2 ;  
4 ; 7 ; 3 ; 7 ; 1 ; 6 ; 0 ; 7 ; 1 ; 5 ;  
7 ; 6 ; 0 ; 3 ; 6 ; 7 ; 5 ; 1 ; 5 ; 5 ;  
6 ; 1 ; 6 ; 2 ; 3 ; 6 ; 5 ; 3 ; 2 ; 0 ;  
C:\Users\engin\source\repos\Ejemplo\Ejempl
```

*Ilustración 13: Arreglo bidimensional*

# Arreglo tridimensional

C/014.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        int TamanoX = 3;
        int TamanoY = 4;
        int TamanoZ = 5;

        //Declara un arreglo Tridimensional
        int[, ,] Solido = new int[TamanoX, TamanoY, TamanoZ];

        /* Llena ese arreglo tridimensional
           Solido.GetLength(0) Retorna la primera dimensión
           Solido.GetLength(1) Retorna la segunda dimensión
           Solido.GetLength(2) Retorna la tercera dimensión

           Un arreglo tridimensional inicia en [0,0,0]
        */
        for (int posX = 0; posX < Solido.GetLength(0); posX++)
            for (int posY = 0; posY < Solido.GetLength(1); posY++)
                for (int posZ = 0; posZ < Solido.GetLength(2); posZ++)
                    Solido[posX, posY, posZ] = (posX + 1) * 100 + posY * 10 +
posZ;

        //Imprime ese arreglo tridimensional
        for (int posX = 0; posX < Solido.GetLength(0); posX++) {
            Console.WriteLine(" ");
            for (int posY = 0; posY < Solido.GetLength(1); posY++) {
                Console.Write("[");
                for (int posZ = 0; posZ < Solido.GetLength(2); posZ++)
                    Console.Write(Solido[posX, posY, posZ] + ";");
                Console.Write("]  ");
            }
        }
    }
}
```

Consola de depuración de Mi

|                        |                        |                        |                        |
|------------------------|------------------------|------------------------|------------------------|
| [100;101;102;103;104;] | [110;111;112;113;114;] | [120;121;122;123;124;] | [130;131;132;133;134;] |
| [200;201;202;203;204;] | [210;211;212;213;214;] | [220;221;222;223;224;] | [230;231;232;233;234;] |
| [300;301;302;303;304;] | [310;311;312;313;314;] | [320;321;322;323;324;] | [330;331;332;333;334;] |

Ilustración 14: Arreglo tridimensional

# Arreglo de arreglos

C/015.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Arreglo de arreglos
        * NO confundirlos con arreglos bidimensionales.
        * Se entiende mejor haciendo analogía con
        * un tronco y ramas */

        //Defino un arreglo (el tronco)
        int[][] arreglo = new int[5][];

        //Defino las ramas
        arreglo[0] = new int[7]; //Tendrá 7 elementos
        arreglo[1] = new int[3]; //Tendrá 3 elementos
        arreglo[2] = new int[9]; //Tendrá 9 elementos
        arreglo[3] = new int[4]; //Tendrá 4 elementos
        arreglo[4] = new int[8]; //Tendrá 8 elementos

        //Llenando un arreglo de arreglos
        Random azar = new();
        for (int tronco = 0; tronco < arreglo.Length; tronco++)
            for (int rama = 0; rama < arreglo[tronco].Length; rama++)
                arreglo[tronco][rama] = azar.Next(0, 9);

        //Imprime ese arreglo de arreglos
        for (int tronco = 0; tronco < arreglo.Length; tronco++) {
            Console.WriteLine(" ");
            for (int rama = 0; rama < arreglo[tronco].Length; rama++)
                Console.Write(arreglo[tronco][rama] + " ; ");
        }
    }
}
```



The image shows a screenshot of a web browser's developer console, specifically the 'Console' tab. The title bar of the console window reads 'Consola de depuración de Mi' followed by a close button (X). The console contains five lines of text, each representing a row of a 5x7 grid. The numbers are displayed in a monospaced font with semicolons separating them. The grid data is as follows:

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 3 | 8 | 4 | 1 | 1 | 0 | 8 |
| 5 | 6 | 2 |   |   |   |   |
| 6 | 4 | 6 | 6 | 1 | 8 | 8 |
| 5 | 0 | 6 | 0 |   |   |   |
| 2 | 8 | 2 | 0 | 5 | 2 | 5 |

*Ilustración 15: Arreglo de arreglos*



# Métrica de velocidad: arreglo bidimensional vs arreglo de arreglos

C/016.cs

```
using System.Diagnostics;

namespace Ejemplo;

internal class Program {
    static void Main() {
        /* ¿Qué es más rápido?
         * ¿Un arreglo bidimensional o un arreglo de arreglos */

        //Limite ancho*alto de ambos arreglos
        int Limite = 80;

        //Arreglo Bidimensional
        int[,] tablero = new int[Limite, Limite];

        //Arreglo de arreglos
        int[][] arreglo = new int[Limite][];
        for (int rama = 0; rama < arreglo.Length; rama++)
            arreglo[rama] = new int[Limite];

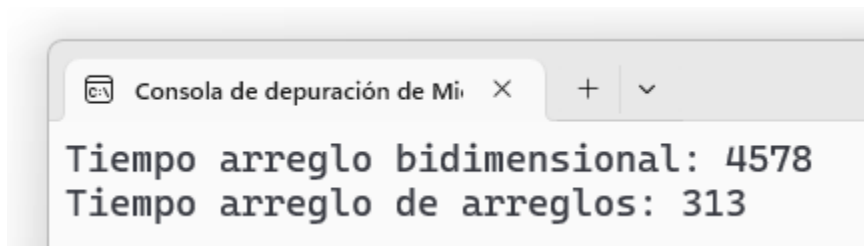
        //Medidor de tiempos
        Stopwatch cronometro = new();

        //Llenando un arreglo bidimensional
        int valor = 0;
        cronometro.Reset();
        cronometro.Start();
        for (int fila = 0; fila < tablero.GetLength(0); fila++)
            for (int col = 0; col < tablero.GetLength(1); col++)
                tablero[fila, col] = valor++;
        long TBidim = cronometro.ElapsedTicks;

        //Llenando un arreglo de arreglos
        valor = 0;
        cronometro.Reset();
        cronometro.Start();
        for (int conjunto = 0; conjunto < arreglo.Length; conjunto++)
            for (int rama = 0; rama < arreglo[conjunto].Length; rama++)
                arreglo[conjunto][rama] = valor++;
        long TArreglo = cronometro.ElapsedTicks;

        //Imprime los tiempos
        Console.WriteLine("Tiempo arreglo bidimensional: " + TBidim);
        Console.WriteLine("Tiempo arreglo de arreglos: " + TArreglo);
    }
}
```

```
}  
}
```



*Ilustración 16: Métrica de velocidad: arreglo bidimensional vs arreglo de arreglos*

```
using System.Diagnostics;

namespace Ejemplo;

internal class Program {
    static void Main() {
        Random Azar = new();

        //Limite de todos los arreglos
        int Lim = 60;

        //Arreglos
        int[,] dosDim = new int[Lim, Lim];
        int[,,,] tresDim = new int[Lim, Lim, Lim, Lim];
        int[,,,] cuatroDim = new int[Lim, Lim, Lim, Lim];
        int[,,,] cincoDim = new int[Lim, Lim, Lim, Lim, Lim];

        //Medidor de tiempos
        Stopwatch cronometro = new();

        //Evitar optimización de ciclos
        //haciendo que acumule los valores
        long Acumula = 0;

        //Llenando un arreglo bidimensional
        int valor = 0;
        cronometro.Reset();
        cronometro.Start();
        for (int a = 0; a < dosDim.GetLength(0); a++)
            for (int b = 0; b < dosDim.GetLength(1); b++)
                dosDim[a, b] = valor++;
        long TBidim = cronometro.ElapsedMilliseconds;

        int posA = Azar.Next(dosDim.GetLength(0));
        int posB = Azar.Next(dosDim.GetLength(1));
        Acumula += dosDim[posA, posB];

        //Llenando un arreglo tridimensional
        valor = 0;
        cronometro.Reset();
        cronometro.Start();
        for (int a = 0; a < tresDim.GetLength(0); a++)
            for (int b = 0; b < tresDim.GetLength(1); b++)
                for (int c = 0; c < tresDim.GetLength(2); c++)
```

```

        tresDim[a, b, c] = valor++;
long TTridim = cronometro.ElapsedMilliseconds;

posA = Azar.Next(tresDim.GetLength(0));
posB = Azar.Next(tresDim.GetLength(1));
int posC = Azar.Next(tresDim.GetLength(2));
Acumula += tresDim[posA, posB, posC];

//Llenando un arreglo de cuatro dimensiones
valor = 0;
cronometro.Reset();
cronometro.Start();
for (int a = 0; a < cuatroDim.GetLength(0); a++)
    for (int b = 0; b < cuatroDim.GetLength(1); b++)
        for (int c = 0; c < cuatroDim.GetLength(2); c++)
            for (int d = 0; d < cuatroDim.GetLength(3); d++)
                cuatroDim[a, b, c, d] = valor++;
long TCuatrodim = cronometro.ElapsedMilliseconds;

posA = Azar.Next(cuatroDim.GetLength(0));
posB = Azar.Next(cuatroDim.GetLength(1));
posC = Azar.Next(cuatroDim.GetLength(2));
int posD = Azar.Next(cuatroDim.GetLength(3));
Acumula += cuatroDim[posA, posB, posC, posD];

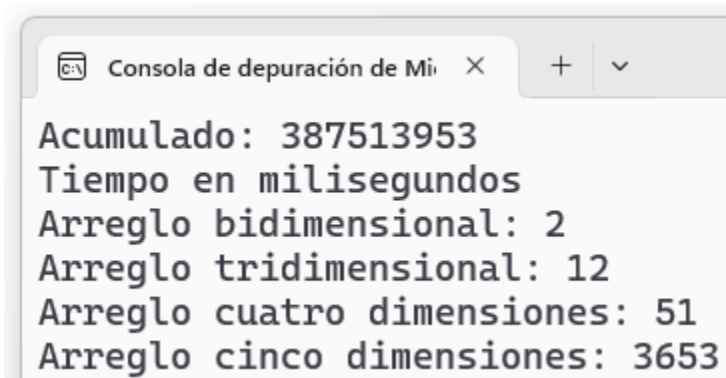
//Llenando un arreglo de cinco dimensiones
valor = 0;
cronometro.Reset();
cronometro.Start();
for (int a = 0; a < cincoDim.GetLength(0); a++)
    for (int b = 0; b < cincoDim.GetLength(1); b++)
        for (int c = 0; c < cincoDim.GetLength(2); c++)
            for (int d = 0; d < cincoDim.GetLength(3); d++)
                for (int e = 0; e < cincoDim.GetLength(4); e++)
                    cincoDim[a, b, c, d, e] = valor++;
long TCincodim = cronometro.ElapsedMilliseconds;

posA = Azar.Next(cincoDim.GetLength(0));
posB = Azar.Next(cincoDim.GetLength(1));
posC = Azar.Next(cincoDim.GetLength(2));
posD = Azar.Next(cincoDim.GetLength(3));
int posE = Azar.Next(cincoDim.GetLength(4));
Acumula += cincoDim[posA, posB, posC, posD, posE];

//Imprime los tiempos
Console.WriteLine("Acumulado: " + Acumula);
Console.WriteLine("Tiempo en milisegundos");
Console.WriteLine("Arreglo bidimensional: " + TBidim);

```

```
Console.WriteLine("Arreglo tridimensional: " + TTridim);  
Console.WriteLine("Arreglo cuatro dimensiones: " + TCuatrodim);  
Console.WriteLine("Arreglo cinco dimensiones: " + TCincodim);  
}  
}
```



The screenshot shows a Windows Debug Console window titled 'Consola de depuración de Mi'. It displays the following output:

```
Acumulado: 387513953  
Tiempo en milisegundos  
Arreglo bidimensional: 2  
Arreglo tridimensional: 12  
Arreglo cuatro dimensiones: 51  
Arreglo cinco dimensiones: 3653
```

*Ilustración 17: Métrica de velocidad: arreglos multidimensionales*

## Tipo implícito de arreglo

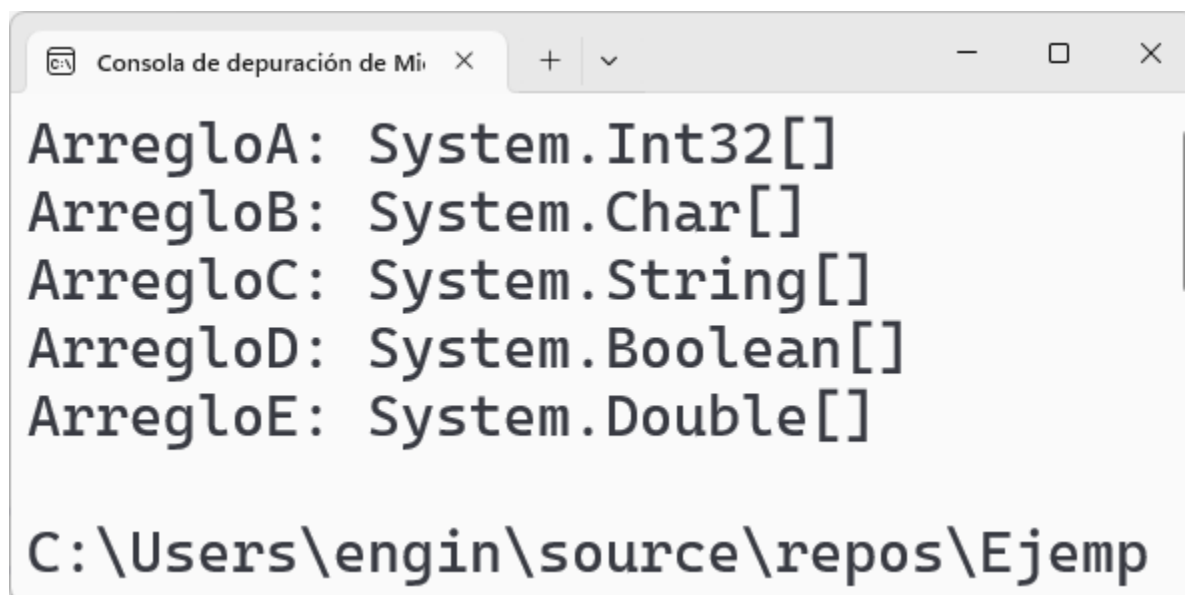
Al inicializar un arreglo que se ha declarado de tipo "var", este es capaz de determinar el tipo de dato al inicializarlo.

C/018.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Tipo implícito de arreglo
        var arregloA = new[] { 16, 83, 29, 29 };
        var arregloB = new[] { 'R', 'a', 'f', 'a', 'e', 'l' };
        var arregloC = new[] { "Esta", "es", "una", "prueba" };
        var arregloD = new[] { true, false, false, true };
        var arregloE = new[] { 3.1, 8.9, 2.3 };

        //Imprime los tipos
        Console.WriteLine("ArregloA: " + arregloA.GetType());
        Console.WriteLine("ArregloB: " + arregloB.GetType());
        Console.WriteLine("ArregloC: " + arregloC.GetType());
        Console.WriteLine("ArregloD: " + arregloD.GetType());
        Console.WriteLine("ArregloE: " + arregloE.GetType());
    }
}
```



```
ArregloA: System.Int32[]
ArregloB: System.Char[]
ArregloC: System.String[]
ArregloD: System.Boolean[]
ArregloE: System.Double[]

C:\Users\engin\source\repos\Ejemp
```

Ilustración 18: Tipo implícito de arreglo

## Tipo implícito en arreglo de arreglos

C/019.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Tipo implícito en arreglo de arreglos
        var arregloA = new[] {
            new[] { 16, 83, 29, 29 },
            new[] { 72, 6, 26 }
        };

        var arregloB = new[] {
            new[] { 'a', 'e', 'i', 'o' },
            new[] { 'q', 'w', 'e' },
            new[] { 'r', 't', 'y', 'u', 'o', 'p' }
        };

        //Imprime los tipos
        Console.WriteLine("Tipo ArregloA: " + arregloA.GetType());
        Console.WriteLine("Tipo ArregloB: " + arregloB.GetType());
    }
}
```

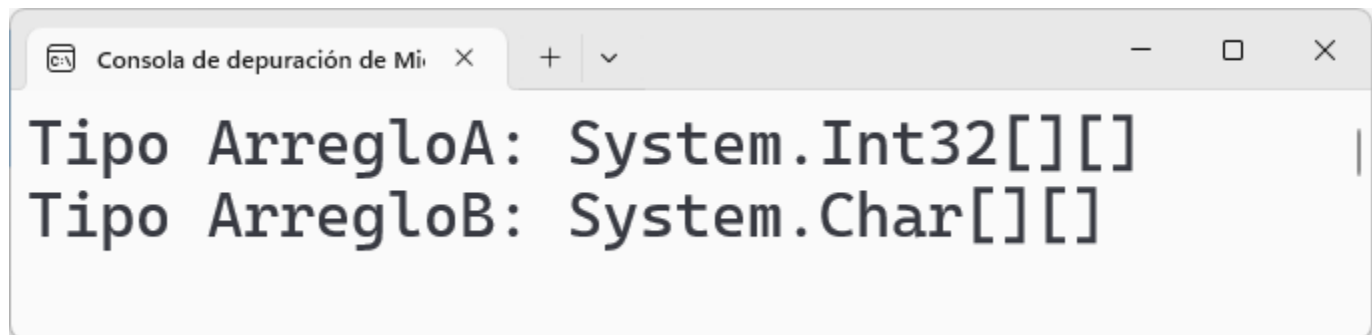


Ilustración 19: Tipo implícito en arreglo de arreglos

## Convertir una cadena en un arreglo de cadenas al dividirla

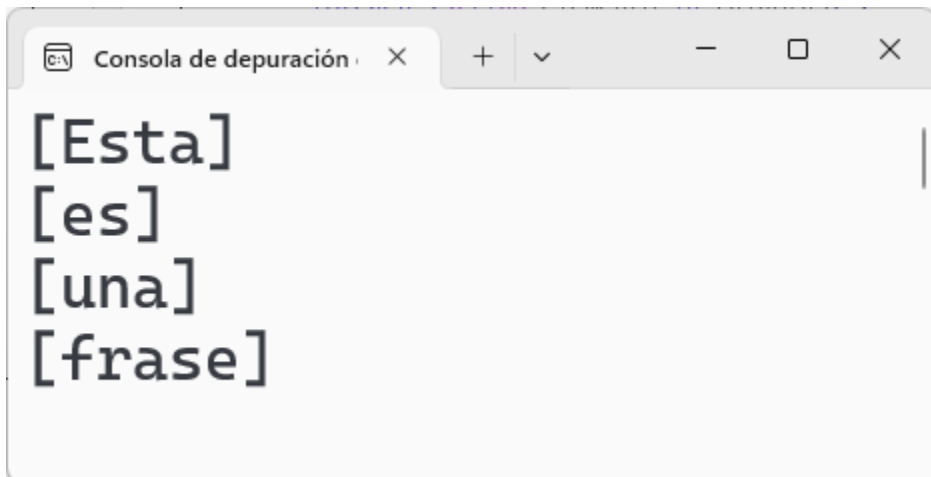
C/020.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una cadena
        string Cadena = "Esta es una frase";

        //Se divide en un arreglo de palabras
        string[] Palabras = Cadena.Split(' ');

        foreach (string elemento in Palabras) {
            Console.WriteLine "[" + elemento + "]");
        }
    }
}
```



*Ilustración 20: Convertir una cadena en un arreglo de cadenas al dividirla*



Sin embargo, si hay más de un espacio intermedio.

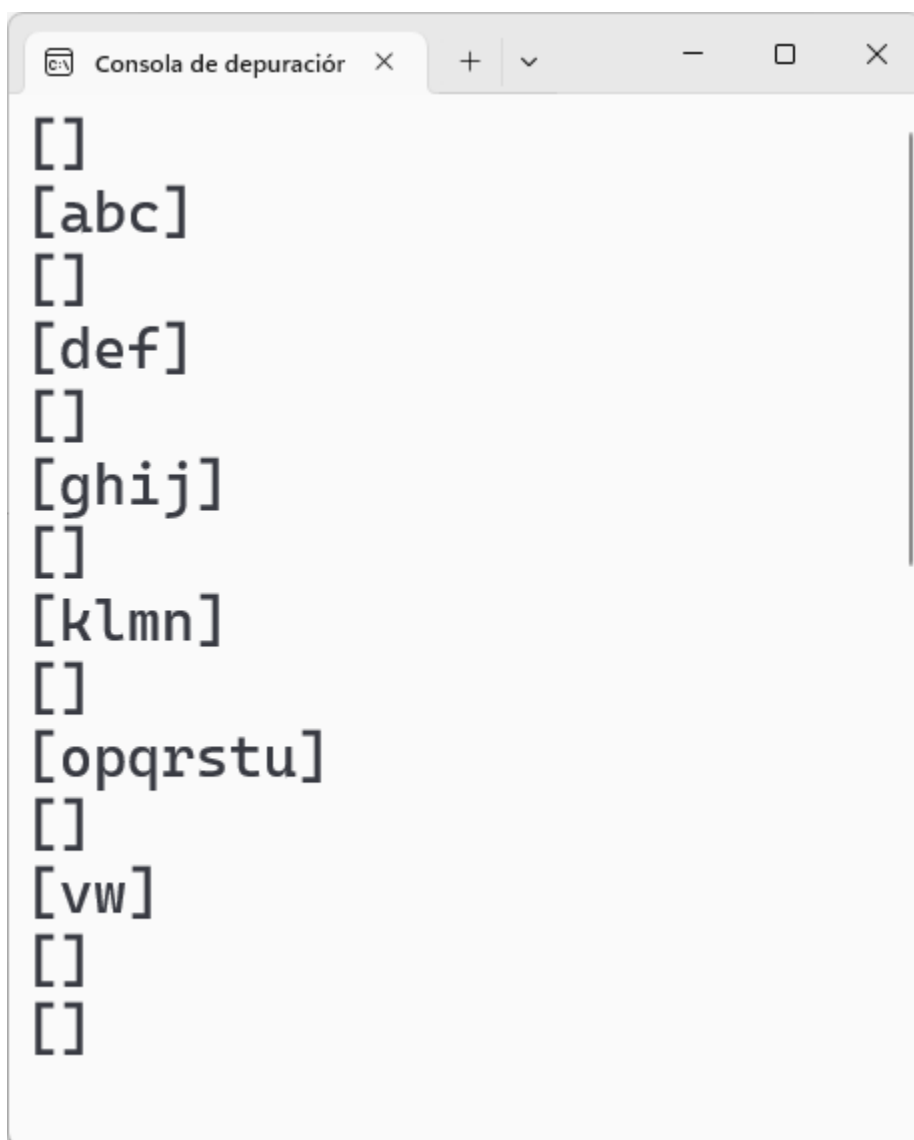
C/021.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una cadena
        string Cadena = " abc def ghij klmn opqrstu vw ";

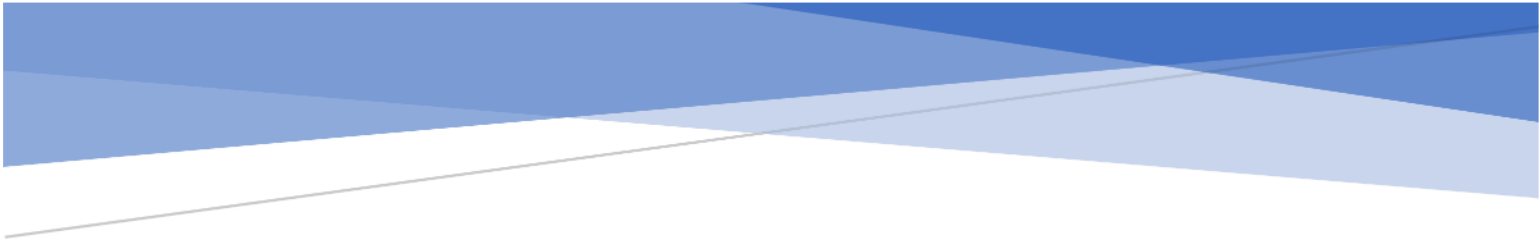
        //Se divide en un arreglo de palabras
        string[] Palabras = Cadena.Split(' ');

        //¡OJO! que cuando hay más de un espacio
        //intermedio, este se interpreta como una palabra
        foreach (string elemento in Palabras) {
            Console.WriteLine "[" + elemento + "]");
        }
    }
}
```



```
[  
[]  
[abc]  
[]  
[def]  
[]  
[ghij]  
[]  
[klmn]  
[]  
[opqrstu]  
[]  
[vw]  
[]  
[]
```

*Ilustraci3n 21: Cuando hay m1s de un espacio intermedio*



# C# Y .NET 10

## Parte 4.

### Programación

### Orientada a Objetos

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

# Contenido

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                       | 5  |
| Acerca del autor.....                                             | 7  |
| Licencia de este libro .....                                      | 7  |
| Licencia del software .....                                       | 7  |
| Marcas registradas .....                                          | 8  |
| Dedicatoria .....                                                 | 9  |
| Definir una clase .....                                           | 10 |
| Errores al tratar de acceder a atributos o métodos privados ..... | 12 |
| Los atributos deben ser privados. Accediendo a ellos.....         | 13 |
| Forma reducida de los getters y setters.....                      | 15 |
| Uso de los getters/setters .....                                  | 16 |
| Otro uso de los getters y setters .....                           | 19 |
| Forma de inicializar los objetos llamando los setters.....        | 21 |
| Dos variables refiriendo al mismo objeto.....                     | 24 |
| Sobrecarga de métodos .....                                       | 27 |
| Por número de parámetros.....                                     | 27 |
| Por tipo de parámetros.....                                       | 29 |
| Constructores.....                                                | 31 |
| Constructor sin parámetros.....                                   | 31 |
| Constructor con parámetros.....                                   | 32 |
| Sobrecarga de constructores .....                                 | 34 |
| Usando el constructor para copiar objetos .....                   | 36 |
| Un constructor puede llamar a otros métodos .....                 | 38 |
| Herencia.....                                                     | 39 |
| Clases abstractas y herencia .....                                | 42 |
| Nivel de protección en los métodos y atributos.....               | 45 |
| Private .....                                                     | 45 |
| Protected .....                                                   | 47 |
| Public .....                                                      | 49 |
| Herencia y métodos iguales en clase madre e hija .....            | 51 |
| Polimorfismo.....                                                 | 52 |
| Polimorfismo, virtual, override.....                              | 53 |
| Herencia y constructores .....                                    | 55 |

|                                                        |     |
|--------------------------------------------------------|-----|
| Llamando a métodos de clases madres .....              | 57  |
| Evitar la herencia .....                               | 59  |
| Clases estáticas .....                                 | 61  |
| Métodos estáticos.....                                 | 62  |
| Constructor static .....                               | 64  |
| Cuidado con el constructor static.....                 | 66  |
| Interface .....                                        | 68  |
| Interface múltiple .....                               | 70  |
| Interface (coinciden métodos) .....                    | 73  |
| Implementación explícita para resolver ambigüedad..... | 74  |
| Herencia múltiple con Interface .....                  | 75  |
| Enums.....                                             | 77  |
| Cambiando los valores de las constantes en enums.....  | 78  |
| Ejemplo 1.....                                         | 78  |
| Ejemplo 2.....                                         | 79  |
| Structs .....                                          | 80  |
| Un struct se puede copiar fácilmente .....             | 82  |
| Métodos en un struct .....                             | 84  |
| Structs y constructores.....                           | 86  |
| Clases parciales .....                                 | 88  |
| Destruyores .....                                      | 90  |
| Patrones de diseño .....                               | 92  |
| Factory Method .....                                   | 92  |
| Abstract Factory .....                                 | 95  |
| Singleton .....                                        | 100 |
| Builder.....                                           | 102 |
| Adapter .....                                          | 106 |
| Composite.....                                         | 109 |
| Facade.....                                            | 111 |
| Modelo Vista Controlador .....                         | 113 |
| SOLID .....                                            | 116 |
| Arreglos de objetos .....                              | 120 |
| Llenado con ciclos.....                                | 122 |
| Ordenando .....                                        | 124 |
| Delegados.....                                         | 127 |
| Segundo ejemplo .....                                  | 129 |

|                                                |     |
|------------------------------------------------|-----|
| Almacenando objetos en medio persistente ..... | 131 |
| Uso de JSON.....                               | 131 |
| Uso de XML .....                               | 132 |
| Genéricos .....                                | 133 |
| Métodos genéricos.....                         | 134 |
| Segundo ejemplo .....                          | 135 |

## Tabla de ilustraciones

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Ilustración 1: Atributos/Métodos privados y públicos .....                          | 11 |
| Ilustración 2: Intento de acceder a un atributo privado.....                        | 12 |
| Ilustración 3: Los atributos deben ser privados. Accediendo a ellos. ....           | 14 |
| Ilustración 4: Forma reducida de los getters y setters .....                        | 15 |
| Ilustración 5: Uso de los getters/setters.....                                      | 18 |
| Ilustración 6: Otro uso de los getters y setters.....                               | 20 |
| Ilustración 7: Forma de inicializar los objetos llamando los setters .....          | 23 |
| Ilustración 8: Ambas variables apuntan al mismo objeto instanciado .....            | 25 |
| Ilustración 9: Dos variables refiriendo al mismo objeto.....                        | 26 |
| Ilustración 10: Sobrecarga de métodos.....                                          | 28 |
| Ilustración 11: Sobrecarga de métodos.....                                          | 30 |
| Ilustración 12: Constructor sin parámetros.....                                     | 31 |
| Ilustración 13: Constructor con parámetros.....                                     | 33 |
| Ilustración 14: Sobrecarga de constructores .....                                   | 35 |
| Ilustración 15: Usando el constructor para copiar objetos .....                     | 37 |
| Ilustración 16: Un constructor puede llamar a otros métodos .....                   | 38 |
| Ilustración 17: No se puede crear instancia .....                                   | 44 |
| Ilustración 18: "Private" impide usar métodos o atributos en clases hijas .....     | 46 |
| Ilustración 19: "Protected" permite uso en clases hijas, pero no en instancias..... | 48 |
| Ilustración 20: Herencia y métodos iguales en clase madre e hija .....              | 51 |
| Ilustración 21: Polimorfismo clásico.....                                           | 52 |
| Ilustración 22: virtual, override .....                                             | 53 |
| Ilustración 23: Polimorfismo .....                                                  | 54 |
| Ilustración 24: Constructores y herencia .....                                      | 56 |
| Ilustración 25: Llamando a métodos de clases madres .....                           | 58 |
| Ilustración 26: Evitar la herencia .....                                            | 60 |
| Ilustración 27: Clases estáticas .....                                              | 61 |
| Ilustración 28: Métodos estáticos.....                                              | 63 |
| Ilustración 29: Constructor static.....                                             | 65 |
| Ilustración 30: Cuidado con el constructor static .....                             | 67 |
| Ilustración 31: Interface.....                                                      | 69 |
| Ilustración 32: Interface múltiple.....                                             | 72 |
| Ilustración 33: Dos "interface" con el mismo método .....                           | 73 |
| Ilustración 34: Implementación explícita en interfaces .....                        | 74 |
| Ilustración 35: Herencia e Interface .....                                          | 76 |
| Ilustración 36: Enums.....                                                          | 77 |
| Ilustración 37: Cambiando los valores de las constantes en enums.....               | 78 |
| Ilustración 38: Cambiando los valores de las constantes en enums.....               | 79 |
| Ilustración 39: Structs .....                                                       | 81 |
| Ilustración 40: Un struct se puede copiar fácilmente .....                          | 83 |
| Ilustración 41: Métodos en un struct .....                                          | 85 |
| Ilustración 42: Structs y constructores .....                                       | 87 |
| Ilustración 43: Clases parciales .....                                              | 89 |
| Ilustración 44: Destructores.....                                                   | 91 |
| Ilustración 45: Factory Method .....                                                | 94 |

|                                                    |     |
|----------------------------------------------------|-----|
| Ilustración 46: Abstract Factory .....             | 99  |
| Ilustración 47: Singleton .....                    | 101 |
| Ilustración 48: Builder .....                      | 105 |
| Ilustración 49: Adapter .....                      | 108 |
| Ilustración 50: Composite.....                     | 110 |
| Ilustración 51: Facade.....                        | 112 |
| Ilustración 52: Modelo Vista Controlador.....      | 115 |
| Ilustración 53: Uso de SOLID .....                 | 119 |
| Ilustración 54: Arreglo de objetos .....           | 121 |
| Ilustración 55: Arreglo de objetos .....           | 123 |
| Ilustración 56: Ordenando arreglo de objetos ..... | 126 |
| Ilustración 57: Uso de delegados .....             | 128 |
| Ilustración 58: Uso de delegados .....             | 130 |



## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelif@hotmai.com](mailto:enginelif@hotmai.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

## Definir una clase

Al definir una clase en C#, se pueden hacer uso de atributos privados (con la palabra reservada `private`), atributos públicos (con la palabra reservada `public`, pero no es recomendado), métodos privados y públicos.

D/001.cs

```
namespace Ejemplo;

//Esta es una clase propia con sus atributos
//y métodos (encapsulación)
internal class MiClase {
    //Atributos privados
    private int Numero;
    private char Letra;
    private string Cadena;
    private double Valor;

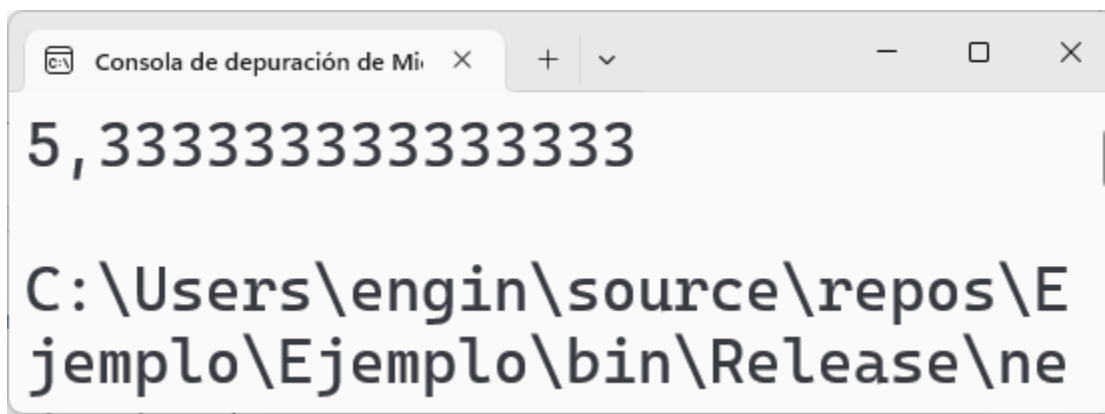
    //Atributos públicos (no recomendado)
    public int Acumula;
    public char Caracter;

    //Métodos privados
    private double Maximo(double numA, double numB, double numC) {
        double max = numA;
        if (max < numB) max = numB;
        if (max < numC) max = numC;
        return max;
    }

    //Métodos públicos
    public double Promedio(double numA, double numB, double numC) {
        return (numA + numB + numC) / 3;
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        MiClase Objeto = new();

        //Solo puede llamar al método público de MiClase
        double resultado = Objeto.Promedio(1, 7, 8);
        Console.WriteLine(resultado);
    }
}
```



*Ilustración 1: Atributos/Métodos privados y públicos*

## Errores al tratar de acceder a atributos o métodos privados

Si se intenta hacer uso de un método privado o acceder a un atributo privado, se generará un error en tiempo de compilación.

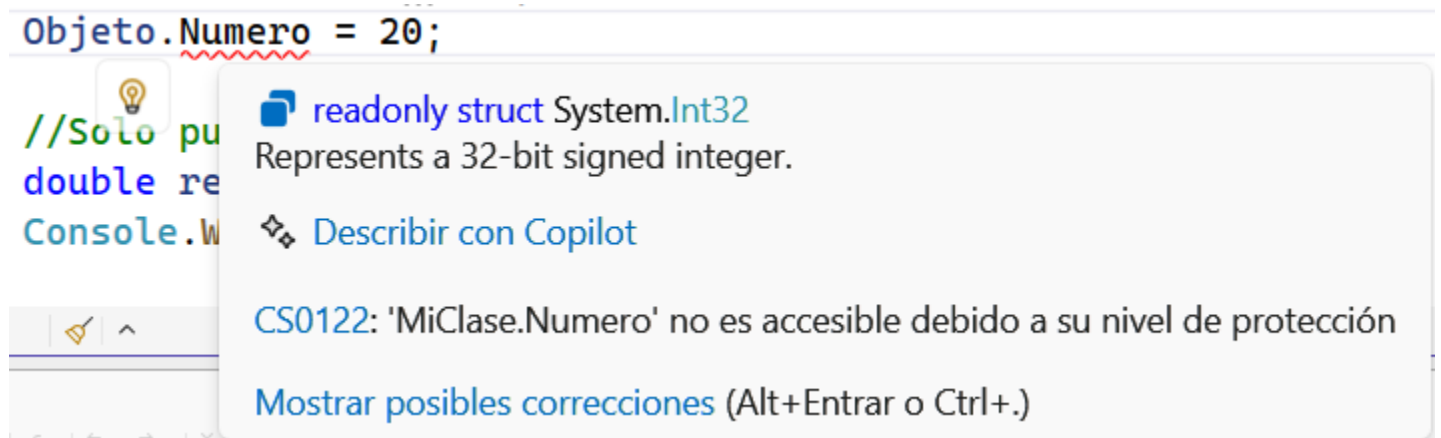


Ilustración 2: Intento de acceder a un atributo privado

## Los atributos deben ser privados. Accediendo a ellos.

La recomendación es que los atributos de una clase siempre sean privados. Así que, para acceder a ellos desde una instancia, se debe hacer a través de métodos públicos de lectura y escritura. Esos métodos son conocidos en el medio como getters y setters.

Cada atributo se le crea un método que en su interior tiene el "get" (para leer) o el "set" (para darle valor)

D/002.cs

```
namespace Ejemplo;

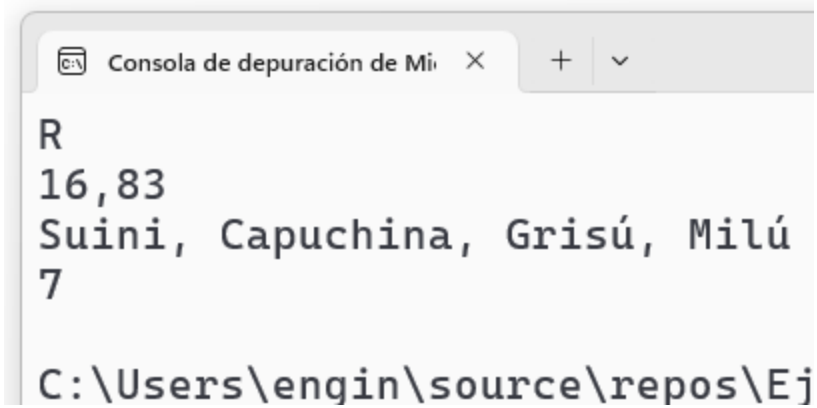
//Esta es una clase propia con sus atributos
//y métodos (encapsulación)
class MiClase {
    //Atributos privados
    private int numero;
    private char letra;
    private string cadena;
    private double valor;

    //Los getters y setters
    public int Numero { get => numero; set => numero = value; }
    public char Letra { get => letra; set => letra = value; }
    public string Cadena { get => cadena; set => cadena = value; }
    public double Valor { get => valor; set => valor = value; }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        MiClase Objeto = new();

        //Llama los setters
        Objeto.Cadena = "Suini, Capuchina, Grisú, Milú";
        Objeto.Numero = 7;
        Objeto.Letra = 'R';
        Objeto.Valor = 16.83;

        //Usa los getters
        Console.WriteLine(Objeto.Letra);
        Console.WriteLine(Objeto.Valor);
        Console.WriteLine(Objeto.Cadena);
        Console.WriteLine(Objeto.Numero);
    }
}
```



*Ilustración 3: Los atributos deben ser privados. Accediendo a ellos.*

Más información en: <https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/classes-and-structs/properties>



```
namespace Ejemplo;

//Esta es una clase propia con sus atributos
//y métodos (encapsulación)
class MiClase {
    //Otra forma de definir atributos con los getters y setters
    public int Numero { get; set; }
    public char Letra { get; set; }
    public string Cadena { get; set; }
    public double Valor { get; set; }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        MiClase Objeto = new();

        //Llama los setters
        Objeto.Cadena = "Suini, Capuchina, Grisú, Milú";
        Objeto.Cadena += ", Sally, Vikingo, Arian, Frac";
        Objeto.Numero = 7;
        Objeto.Letra = 'R';
        Objeto.Valor = 93.5;

        //Usa los getters
        Console.WriteLine(Objeto.Letra);
        Console.WriteLine(Objeto.Valor);
        Console.WriteLine(Objeto.Cadena);
        Console.WriteLine(Objeto.Numero);
    }
}
```

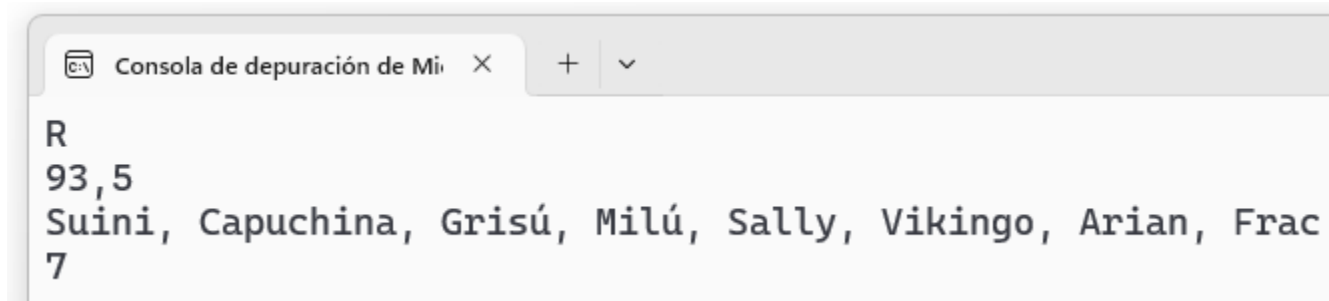


Ilustración 4: Forma reducida de los getters y setters

## Uso de los getters/setters

Observe que el atributo (como variable) es privado, por lo general se escribe en minúsculas, luego se crea el acceso a ese atributo, copiando el nombre del atributo pero poniendo la primera en mayúsculas (es recomendado hacerlo así, no obligatorio). Es en ese acceso donde se pone el código para el get y el set.

D/004.cs

```
namespace Ejemplo;

//Esta es una clase propia con sus atributos
//y métodos (encapsulación)
class MiClase {
    //Atributos privados. Un uso de los getters y setters
    private int numero;
    private char letra;
    private string cadena;
    private double valor;

    //Puede auditar cuando se leyó o
    //cambió el valor de un atributo
    public int Numero {
        get {
            Console.WriteLine("Lee numero: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            return numero;
        }
        set {
            Console.WriteLine("Cambia numero: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            numero = value;
        }
    }

    public char Letra {
        get {
            Console.WriteLine("Lee letra: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            return letra;
        }
        set {
            Console.WriteLine("Cambia letra: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            letra = value;
        }
    }
}
```

```

    }
}

public string Cadena {
    get {
        Console.WriteLine("Lee cadena: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        return cadena;
    }
    set {
        Console.WriteLine("Cambia cadena: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        cadena = value;
    }
}

public double Valor {
    get {
        Console.WriteLine("Lee valor: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        return valor;
    }
    set {
        Console.WriteLine("Cambia valor: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        valor = value;
    }
}
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        MiClase Objeto = new();

        //Llama los setters
        Objeto.Cadena = "Suini, Capuchina, Grisú, Milú";
        Objeto.Numero = 7;
        Objeto.Letra = 'R';
        Objeto.Valor = 93.5;

        //Usa los getters
        char una letra = Objeto.Letra;
    }
}

```

```

double unvalor = Objeto.Valor;
string unacadena = Objeto.Cadena;
int unnumero = Objeto.Numero;

Console.WriteLine("Letra es: " + una letra);
Console.WriteLine("Valor es: " + unvalor);
Console.WriteLine("Cadena es: " + unacadena);
Console.WriteLine("Número es: " + unnumero);
}
}

```

```

Cambia cadena:
2024-07-10 11:07:35 a. m.
Cambia numero:
2024-07-10 11:07:35 a. m.
Cambia letra:
2024-07-10 11:07:35 a. m.
Cambia valor:
2024-07-10 11:07:35 a. m.
Lee letra:
2024-07-10 11:07:35 a. m.
Lee valor:
2024-07-10 11:07:35 a. m.
Lee cadena:
2024-07-10 11:07:35 a. m.
Lee numero:
2024-07-10 11:07:35 a. m.
Letra es: R
Valor es: 93,5
Cadena es: Suini, Capuchina, Grisú, Milú
Número es: 7

C:\Users\engin\source\repos\Ejemplo\Ejemplo

```

Ilustración 5: Uso de los getters/setters

## Otro uso de los getters y setters

Para validar el dato de entrada

D/005.cs

```
namespace Ejemplo;

//Esta es una clase propia con sus
//atributos y métodos (encapsulación)
class MiClase {
    //Atributos privados. Un uso de los getters y setters
    private int edad;

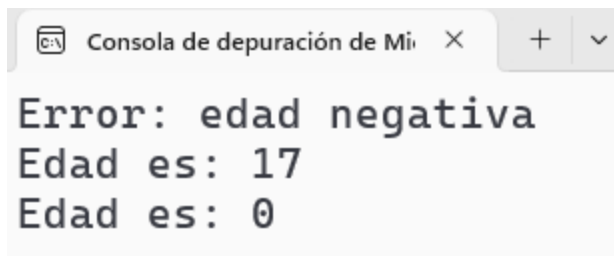
    //Puede validar el dato de inicialización
    public int Edad {
        get {
            return edad;
        }
        set {
            if (value < 0)
                Console.WriteLine("Error: edad negativa");
            else
                edad = value;
        }
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        MiClase Objeto = new();
        MiClase Otro = new();

        //Llama los setters
        Objeto.Edad = 17;
        Otro.Edad = -8;

        Console.WriteLine("Edad es: " + Objeto.Edad);
        Console.WriteLine("Edad es: " + Otro.Edad);
    }
}
```

Al intentar dar un valor a un atributo el setter valida si ese dato es válido. Dado el caso, asigna el valor, de lo contrario, muestra un mensaje y el dato no es asignado.



*Ilustración 6: Otro uso de los getters y setters*

```
namespace Ejemplo;

//Esta es una clase propia con sus
//atributos y métodos (encapsulación)
class MiClase {
    //Atributos privados. Un uso de los getters y setters
    private int numero;
    private char letra;
    private string cadena;
    private double valor;

    //Puede auditar cuando se leyó o cambió
    //el valor de un atributo
    public int Numero {
        get {
            Console.WriteLine("Lee numero: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            return numero;
        }
        set {
            Console.WriteLine("Cambia numero: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            numero = value;
        }
    }

    public char Letra {
        get {
            Console.WriteLine("Lee letra: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            return letra;
        }
        set {
            Console.WriteLine("Cambia letra: ");
            string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
            Console.WriteLine(T);
            letra = value;
        }
    }

    public string Cadena {
```

```

    get {
        Console.WriteLine("Lee cadena: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        return cadena;
    }
    set {
        Console.WriteLine("Cambia cadena: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        cadena = value;
    }
}

public double Valor {
    get {
        Console.WriteLine("Lee valor: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        return valor;
    }
    set {
        Console.WriteLine("Cambia valor: ");
        string T = DateTime.Now.ToString("yyyy-MM-dd h:mm:ss tt");
        Console.WriteLine(T);
        valor = value;
    }
}
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase.
        //Otra forma de inicializar los atributos.
        MiClase Objeto = new MiClase {

            //Llama los setters
            Cadena = "Suini, Capuchina, Grisú, Milú, Sally, Vikingo",
            Numero = 7,
            Letra = 'R',
            Valor = 93.5
        };

        //Usa los getters
        char una letra = Objeto.Letra;
        double unvalor = Objeto.Valor;
        string unacadena = Objeto.Cadena;
    }
}

```



```

    int unnumero = Objeto.Numero;

    Console.WriteLine("Letra es: " + una letra);
    Console.WriteLine("Valor es: " + unvalor);
    Console.WriteLine("Cadena es: " + unacadena);
    Console.WriteLine("Número es: " + unnumero);
}
}

```

```

Cambia cadena:
2024-07-10 11:10:42 a. m.
Cambia numero:
2024-07-10 11:10:42 a. m.
Cambia letra:
2024-07-10 11:10:42 a. m.
Cambia valor:
2024-07-10 11:10:42 a. m.
Lee letra:
2024-07-10 11:10:42 a. m.
Lee valor:
2024-07-10 11:10:42 a. m.
Lee cadena:
2024-07-10 11:10:42 a. m.
Lee numero:
2024-07-10 11:10:42 a. m.
Letra es: R
Valor es: 93,5
Cadena es: Suini, Capuchina, Grisú, Milú, Sally, Vikingo
Número es: 7

```

*Ilustración 7: Forma de inicializar los objetos llamando los setters*

## Dos variables refiriendo al mismo objeto

D/007.cs

```
namespace Ejemplo;

//Esta es una clase propia con sus
//atributos y métodos (encapsulación)
class MiClase {
    //Otra forma de definir atributos con
    //los getters y setters
    public int Numero { get; set; }
    public char Letra { get; set; }
    public string Cadena { get; set; }
    public double Valor { get; set; }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase.
        MiClase Mascotas = new MiClase {
            Cadena = "Suini, Capuchina, Grisú, Milú",
            Numero = 7,
            Letra = 'R',
            Valor = 93.5
        };

        //Crea una variable de tipo MiClase
        MiClase otra;

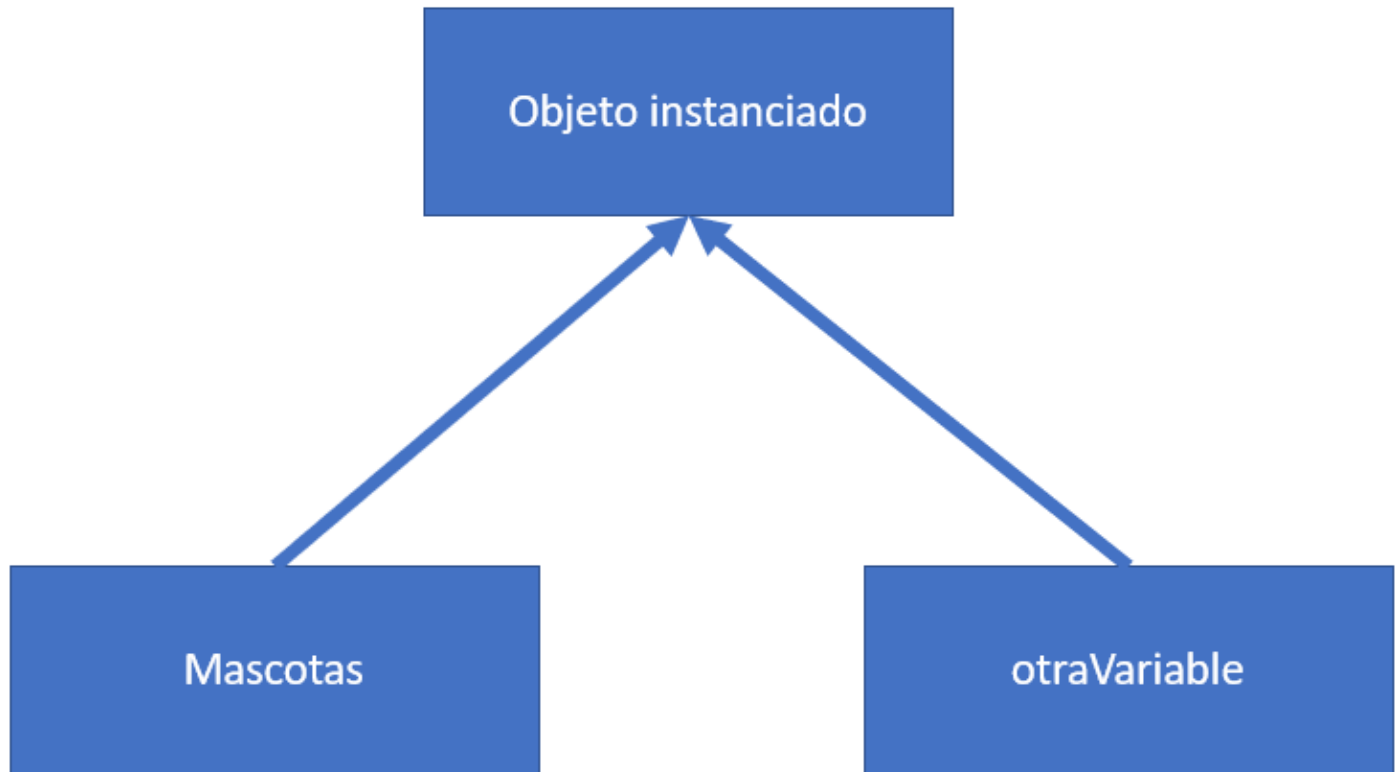
        //Asigna el primer objeto a esa variable
        otra = Mascotas;

        //¿Qué sucede? Que tenemos dos variables
        //apuntando al mismo objeto en memoria

        //Se imprimen los valores de ambas variables
        Console.WriteLine("Letra en Mascotas: " + Mascotas.Letra);
        Console.WriteLine("Valor en Mascotas: " + Mascotas.Valor);
        Console.WriteLine("Letra en otraVariable: " + otra.Letra);
        Console.WriteLine("Valor en otraVariable: " + otra.Valor);

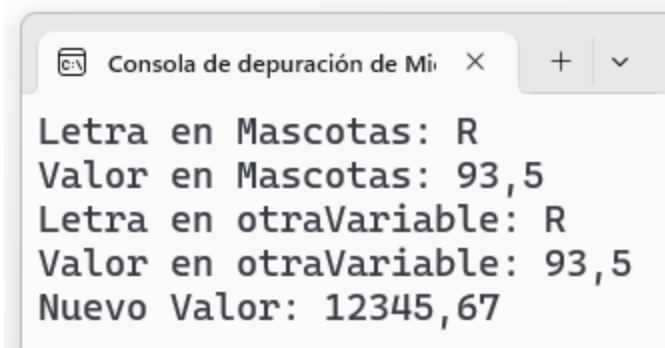
        //Si se modifican los valores en "otra"
        //afecta a Mascotas porque ambas apuntan
        //al mismo objeto en memoria
        otra.Valor = 12345.67;
        Console.WriteLine("Nuevo Valor: " + Mascotas.Valor);
    }
}
```

```
}  
}
```



*Ilustración 8: Ambas variables apuntan al mismo objeto instanciado*

Dos variables apuntando al mismo objeto en memoria, eso es una copia superficial o "Shallow Copy".



The image shows a debug console window titled "Consola de depuración de Mi". It contains five lines of text output:

```
Letra en Mascotas: R
Valor en Mascotas: 93,5
Letra en otraVariable: R
Valor en otraVariable: 93,5
Nuevo Valor: 12345,67
```

*Ilustración 9: Dos variables refiriendo al mismo objeto*

# Sobrecarga de métodos

Dependiendo del número y tipo de parámetros C# sabe que método usar así tenga el mismo nombre.

A continuación, una clase con tres métodos con el mismo nombre, pero cada uno de los métodos tiene diferente número de parámetros.

## Por número de parámetros

D/008.cs

```
namespace Ejemplo;

class Geometria {
    //Calcula el área del círculo
    public double Area(double radio) {
        return Math.PI * Math.Pow(radio, 2);
    }

    //Calcula el área del rectángulo
    public double Area(double baseR, double alturaR) {
        return baseR * alturaR;
    }

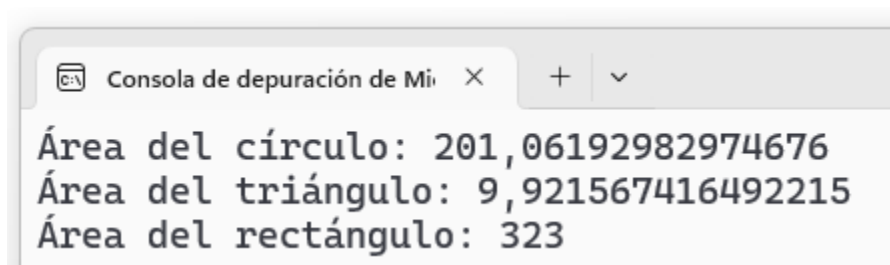
    //Calcula el área del triángulo
    public double Area(double ladoA, double ladoB, double ladoC) {
        double S = (ladoA + ladoB + ladoC) / 2;
        return Math.Sqrt(S * (S - ladoA) * (S - ladoB) * (S - ladoC));
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto Geometria
        Geometria geometria = new();

        //Dependiendo del número de parámetros
        //llama a un método u otro
        double areaCirculo = geometria.Area(8);
        double areaTriangulo = geometria.Area(4, 5, 6);
        double areaRectangulo = geometria.Area(17, 19);

        Console.WriteLine("Área del círculo: " + areaCirculo);
        Console.WriteLine("Área del triángulo: " + areaTriangulo);
        Console.WriteLine("Área del rectángulo: " + areaRectangulo);
    }
}
```

```
}  
}
```



*Ilustración 10: Sobrecarga de métodos*

## Por tipo de parámetros

C# selecciona el método dependiendo del tipo de parámetros. A continuación, una clase que implementa varios métodos con el mismo nombre, sólo que varía el tipo de parámetro.

D/009.cs

```
namespace Ejemplo;

class MiClase {
    private int valor;
    private string cadena;
    private double costo;

    public void UnMetodo(int valor, string cadena) {
        this.valor = valor;
        this.cadena = cadena;
        Console.WriteLine("Un método B");
    }

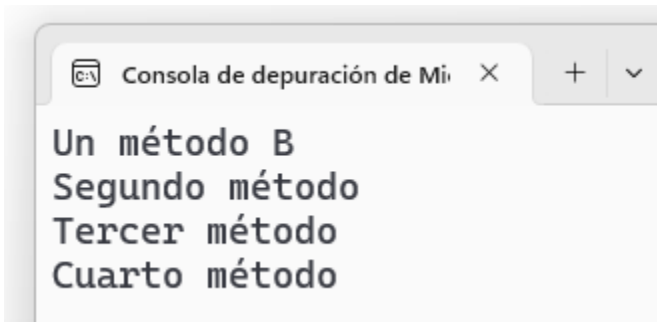
    public void UnMetodo(string cadena, int valor) {
        this.cadena = cadena;
        this.valor = valor;
        Console.WriteLine("Segundo método");
    }

    public void UnMetodo(double costo, int valor) {
        this.costo = costo;
        this.valor = valor;
        Console.WriteLine("Tercer método");
    }

    public void UnMetodo(string cadena, double costo) {
        this.cadena = cadena;
        this.costo = costo;
        Console.WriteLine("Cuarto método");
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto
        MiClase objetoA = new();
        objetoA.UnMetodo(48, "Rafael");
        objetoA.UnMetodo("Alberto", 26);
        objetoA.UnMetodo(1994.06, 48);
        objetoA.UnMetodo("Moreno Parra", 1683.29);
    }
}
```

```
}  
}
```



*Ilustración 11: Sobrecarga de métodos*



# Constructores

En C# los constructores se escriben con "public Nombre\_de\_la\_clase". Los constructores tienen estas características:

1. Deben tener el mismo nombre de la clase
2. Se ejecutan cuando el objeto es instanciado
3. No pueden retornar valores
4. Sólo ejecutan una sola vez (cuando el objeto se instancia)

## Constructor sin parámetros

D/010.cs

```
namespace Ejemplo;

class MiClase {
    public MiClase() {
        Console.WriteLine("Constructor por defecto");
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto
        MiClase instancia = new();
    }
}
```

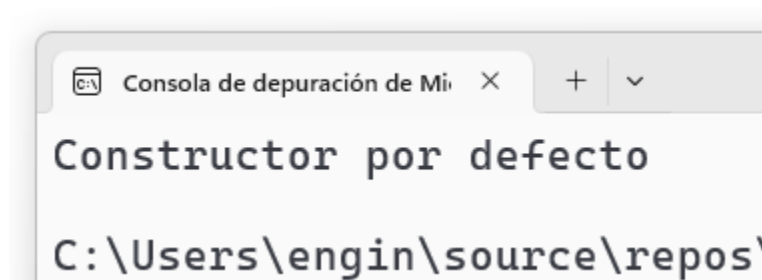


Ilustración 12: Constructor sin parámetros

## Constructor con parámetros

Se envía los datos del objeto al instanciarlo.

D/011.cs

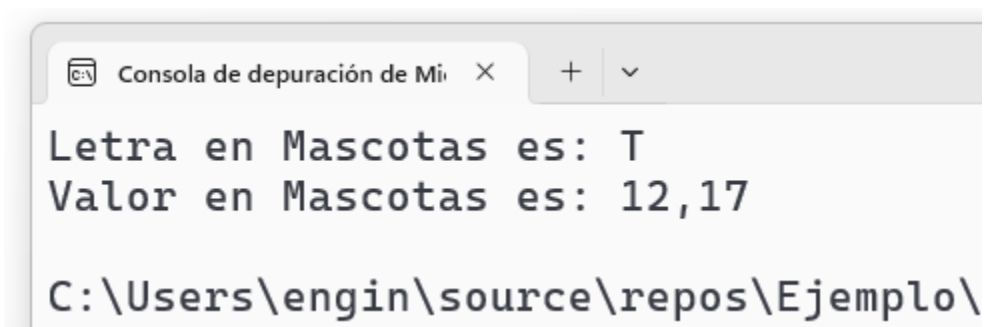
```
namespace Ejemplo;

//Esta es una clase propia con
//sus atributos y métodos (encapsulación)
class MiClase {
    //Un constructor
    public MiClase(int Numero, char Letra, string Cadena, double Valor) {
        //Se asigna así this.atributo = valor parámetro
        this.Numero = Numero;
        this.Letra = Letra;
        this.Cadena = Cadena;
        this.Valor = Valor;
    }

    //Otra forma de definir atributos con los getters y setters
    public int Numero { get; set; }
    public char Letra { get; set; }
    public string Cadena { get; set; }
    public double Valor { get; set; }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
        //llamando el constructor
        MiClase Mascotas = new(2016, 'T', "Tammy", 12.17);

        //Se imprimen los valores de ambas variables
        Console.WriteLine("Letra en Mascotas es: " + Mascotas.Letra);
        Console.WriteLine("Valor en Mascotas es: " + Mascotas.Valor);
    }
}
```



The image shows a screenshot of a Visual Studio debug console window. The title bar at the top reads "Consola de depuración de Mi" followed by a close button (X) and a dropdown menu with a plus sign and a downward arrow. The console output consists of three lines of text: "Letra en Mascotas es: T", "Valor en Mascotas es: 12,17", and "C:\Users\engin\source\repos\Ejemplo\".

```
Letra en Mascotas es: T
Valor en Mascotas es: 12,17
C:\Users\engin\source\repos\Ejemplo\
```

*Ilustración 13: Constructor con parámetros*

## Sobrecarga de constructores

A continuación, una clase con varios constructores:

D/012.cs

```
namespace Ejemplo;

class MiClase {
    private int valor;
    private string cadena;
    private double costo;
    public MiClase() {
        Console.WriteLine("Constructor por defecto");
    }

    public MiClase(int valor) {
        this.valor = valor;
        this.cadena = "por defecto";
        this.costo = 0;
        Console.WriteLine("Constructor B");
    }

    public MiClase(string cadena, int valor) {
        this.cadena = cadena;
        this.valor = valor;
        this.costo = 0;
        Console.WriteLine("Tercer Constructor");
    }

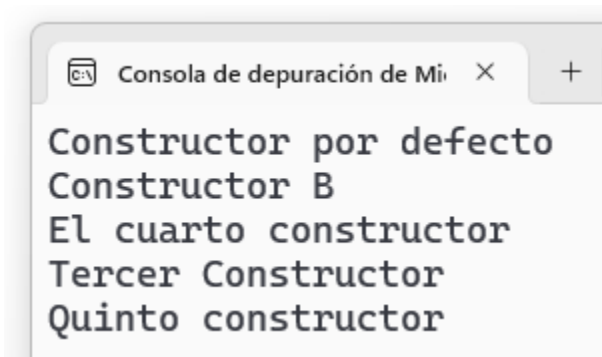
    public MiClase(double costo, int valor) {
        this.cadena = "por defecto";
        this.costo = costo;
        this.valor = valor;
        Console.WriteLine("El cuarto constructor");
    }

    public MiClase(string cadena, double costo, int valor) {
        this.cadena = cadena;
        this.costo = costo;
        this.valor = valor;
        Console.WriteLine("Quinto constructor");
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
```

```
//Instancia o crea un objeto
MiClase objetoA = new();
MiClase objetoB = new(48);
MiClase objetoC = new(1972.06, 26);
MiClase objetoD = new("Ramp", 48);
MiClase objetoE = new("Moreno Parra", 1683.29, 29);
}
```

Dependiendo del número de parámetros, se llama a un constructor o a otro



*Ilustración 14: Sobrecarga de constructores*

## Usando el constructor para copiar objetos

La asignación de una variable objeto a otra variable objeto da como resultado que ambas variables apunten al mismo objeto en memoria, ese se le conoce como una copia superficial "Shallow Copy". ¿Cómo entonces generar una copia total del objeto, es decir, copiar los valores de los atributos también conocida como copia profunda o "Deep Copy"? Usualmente se busca el término "clonar el objeto", en el pasado, se usaba la instrucción ICloneable (considerada obsoleta o mejor no usarla: <https://stackoverflow.com/questions/536349/why-no-icloneable> ). A continuación, se muestra una técnica para hacer una copia profunda:

Se pone un método al que se le puede llamar CopiarObjeto() que retorna una nueva instancia de la clase y se le envía por el **constructor** los valores que tienen en los atributos.

D/013.cs

```
namespace Ejemplo;
//Esta es una clase propia con sus
//atributos y métodos (encapsulación)
internal class MiClase {
    //Un constructor
    public MiClase(int Num, char Car, string Cad, double Valor) {
        //Se asigna así this.atributo = valor parámetro
        this.Numero = Num;
        this.Letra = Car;
        this.Cadena = Cad;
        this.Valor = Valor;
    }

    //Método que permite copiar un objeto
    public MiClase CopiarObjeto() {
        MiClase copia = new MiClase(Numero, Letra, Cadena, Valor);
        return copia;
    }

    //Otra forma de definir atributos
    //con los getters y setters
    public int Numero { get; set; }
    public char Letra { get; set; }
    public string Cadena { get; set; }
    public double Valor { get; set; }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Instancia o crea un objeto de MiClase
```

```

//llamando el constructor
MiClase Mascotas = new(2016, 'T', "Tammy", 12.17);

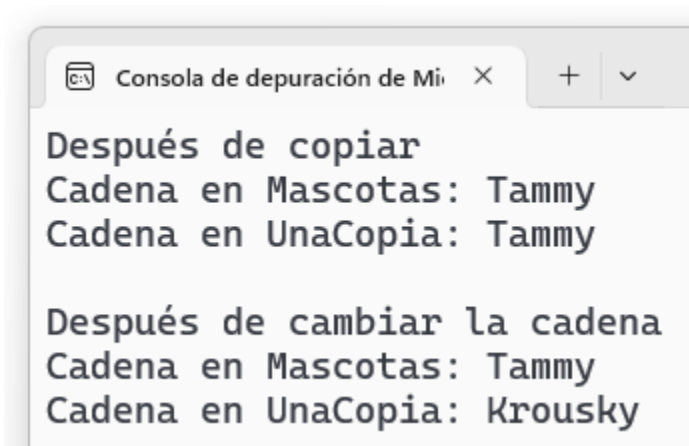
//Hace una copia de ese objeto
MiClase UnaCopia = Mascotas.CopiarObjeto();

//Se imprimen los valores de los dos objetos
Console.WriteLine("Después de copiar");
Console.WriteLine("Cadena en Mascotas: " + Mascotas.Cadena);
Console.WriteLine("Cadena en UnaCopia: " + UnaCopia.Cadena);

//Cambia el valor de cadena en UnaCopia
UnaCopia.Cadena = "Krousky";

//Imprime de nuevo los valores
Console.WriteLine("\r\nDespués de cambiar la cadena");
Console.WriteLine("Cadena en Mascotas: " + Mascotas.Cadena);
Console.WriteLine("Cadena en UnaCopia: " + UnaCopia.Cadena);
}
}

```



*Ilustración 15: Usando el constructor para copiar objetos*

## Un constructor puede llamar a otros métodos

Al instanciar una clase, el constructor puede llamar a otros métodos.

D/014.cs

```
namespace Ejemplo;

internal class MiClase {
    //Constructor
    public MiClase() {
        //Llama a otros métodos
        MetodoA();
        MetodoB();
    }

    public void MetodoA() {
        Console.WriteLine("Ha llamado método A");
    }

    private void MetodoB() {
        Console.WriteLine("Ha llamado método B");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        MiClase objeto = new();
    }
}
```

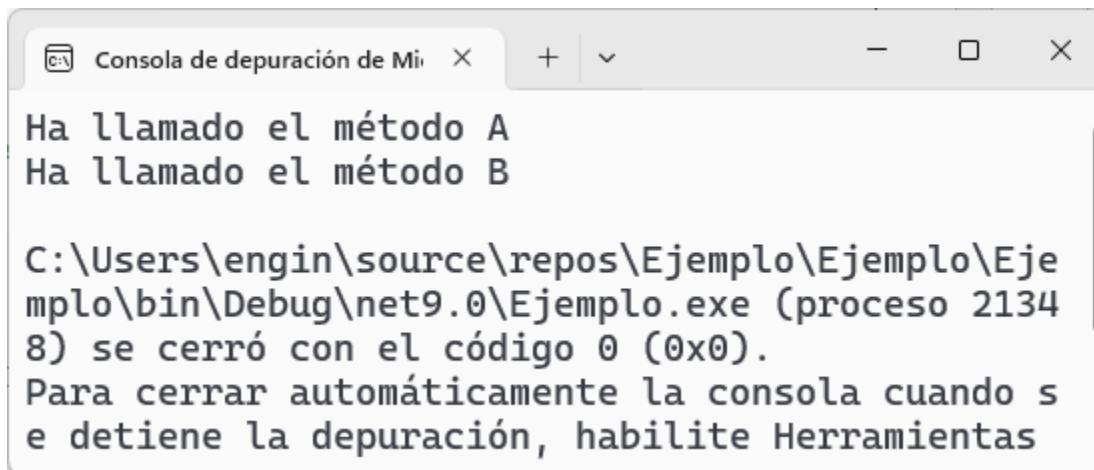


Ilustración 16: Un constructor puede llamar a otros métodos



# Herencia

En C# se implementa así: *nombre clase:clase madre*

D/015.cs

```
namespace Ejemplo;

class Mascota {
    public string SerialChip { get; set; }
    public string Nombre { get; set; }
    public DateTime FechaNacimiento { get; set; }
    public char Sexo { get; set; } //Macho, Hembra

    //Nombre del propietario
    public string Propietario { get; set; }

    //Teléfono del propietario
    public string Telefono { get; set; }

    //Correo electrónico del propietario
    public string Correo { get; set; }
    public double Peso { get; set; }

    //0. Azul, 1. Verde, 2. Dorado, 3. Dispar
    public int ColorOjos { get; set; }

    public int EsperanzaVidaMinimo { get; set; }
    public int EsperanzaVidaMaximo { get; set; }

    //0. Baja, 1. Media, 2. Alta
    public int NecesidadAtencion { get; set; }

    public string Raza { get; set; }
}

class Gato : Mascota {
    public string PatronPelo { get; set; }
    public int TendenciaPerderPelo { get; set; }

    //Asociación de Criadores de Gatos
    public string ReconocimientoCFA { get; set; }

    //Asociación Americana de Criadores de Gatos
    public string ReconocimientoACFA { get; set; }

    //Fédération Internationale Féline
    public string ReconocimientoFIFe { get; set; }
}
```

```

    //Asociación Internacional de Gatos
    public string ReconocimientoTICA { get; set; }
}

class Perro : Mascota {
    //Real Sociedad Canina de España
    public string ReconocimientoRSCE { get; set; }

    //United Kennel Club
    public string ReconocimientoUKC { get; set; }

    //Crianza
    public string CriadoPara { get; set; }

    public double AlturaALaCruz { get; set; }

    //0. Ninguna, 1. Baja, 2. Moderada
    public int TendenciaBabear { get; set; }

    public int TendenciaRoncar { get; set; }
    public int TendenciaLadrar { get; set; }
    public int TendenciaExcavar { get; set; }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        Mascota objMascota = new();
        Gato objGato = new();
        Perro objPerro = new();

        //Da valores a la instancia de mascota
        objMascota.Correo = "enginelifelife@hotmail.com";
        objMascota.ColorOjos = 1;

        //Da valores a la instancia de gato
        objGato.Correo = "ramsoftware@gmail.com";
        objGato.Propietario = "Rafael Alberto Moreno Parra";
        objGato.Nombre = "Sally";
        objGato.Sexo = 'H';
        objGato.PatronPelo = "Tricolor";

        //Da valores a la instancia de perro
        objPerro.Raza = "Pastor Alemán";
        objPerro.Sexo = 'M';
        objPerro.Nombre = "Firuláis";
        objPerro.TendenciaLadrar = 1;
    }
}

```

}  
}

## Clases abstractas y herencia

Una clase abstracta solo permite heredar, no se puede instanciar. Si se intenta instanciar dará un mensaje de error en tiempo de compilación. La palabra reservada "abstract" es para definir clases abstractas.

D/016.cs

```
namespace Ejemplo;

//Esta clase solo puede heredar, no se puede instanciar
abstract class Mascota {
    public string SerialChip { get; set; }

    public string Nombre { get; set; }

    public DateTime FechaNacimiento { get; set; }

    //Macho, Hembra
    public char Sexo { get; set; }

    //Nombre del propietario
    public string Propietario { get; set; }

    //Teléfono del propietario
    public string Telefono { get; set; }

    //Correo electrónico del propietario
    public string Correo { get; set; }

    public double Peso { get; set; }

    //0. Azul, 1. Verde, 2. Dorado, 3. Dispar
    public int ColorOjos { get; set; }

    public int EsperanzaVidaMinimo { get; set; }

    public int EsperanzaVidaMaximo { get; set; }

    //0. Baja, 1. Media, 2. Alta
    public int NecesidadAtencion { get; set; }

    public string Raza { get; set; }
}

class Perro : Mascota {
    //Real Sociedad Canina de España
    public string ReconocimientoRSCE { get; set; }
}
```

```

//United Kennel Club
public string ReconocimientoUKC { get; set; }

//Crianza
public string CriadoPara { get; set; }

public double AlturaALaCruz { get; set; }

//0. Ninguna, 1. Baja, 2. Moderada
public int TendenciaBabear { get; set; }

public int TendenciaRoncar { get; set; }
public int TendenciaLadrrar { get; set; }
public int TendenciaExcavar { get; set; }
}

class Gato : Mascota {
    public string PatronPelo { get; set; }
    public int TendenciaPerderPelo { get; set; }

    //Asociación de Criadores de Gatos
    public string ReconocimientoCFA { get; set; }

    //Asociación Americana de Criadores de Gatos
    public string ReconocimientoACFA { get; set; }

    //Fédération Internationale Féline
    public string ReconocimientoFIFe { get; set; }

    //Asociación Internacional de Gatos
    public string ReconocimientoTICA { get; set; }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        Mascota objMascota = new();
        Gato objGato = new();
        Perro objPerro = new();

        //Da valores a la instancia de mascota
        objMascota.Correo = "enginelifelife@hotmail.com";
        objMascota.ColorOjos = 1;

        //Da valores a la instancia de gato
        objGato.Correo = "ramsoftware@gmail.com";
        objGato.Propietario = "Rafael Alberto Moreno Parra";
        objGato.Nombre = "Sally";
    }
}

```

```

objGato.Sexo = 'H';
objGato.PatronPelo = "Tricolor";

//Da valores a la instancia de perro
objPerro.Raza = "Pastor Alemán";
objPerro.Sexo = 'M';
objPerro.Nombre = "Firuláis";
objPerro.TendenciaLadrar = 1;
}
}

```

```

static void Main() {
    Mascota objMascota = new();
    Gato objGato = new();
    Perro objPerro = new()
    //Da valores a la instancia
    objMascota.Correo = "el correo";
    objMascota.ColorOjos = 1;
}

```

class Ejemplo.Mascota

CS0144: No se puede crear una instancia de la interfaz o el tipo abstracto "Mascota"

Mostrar posibles correcciones (Alt+Entrar o Ctrl+.)

*Ilustración 17: No se puede crear instancia*

## Nivel de protección en los métodos y atributos

### Private

En la clase madre, los atributos o métodos con el atributo private no pueden ser usados por las clases hijas.

D/017.cs

```
namespace Ejemplo;

//Clase madre con atributos privados
class Mascota {
    private string Nombre { get; set; }

    //Macho, Hembra
    private char Sexo { get; set; }

    //Nombre del propietario
    private string Dueno { get; set; }
}

class Gato : Mascota {
    public string PatronPelo { get; set; }
    public int TendenciaPerderPelo { get; set; }
    public void DatosGato(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

class Perro : Mascota {
    //Crianza
    public string CriadoPara { get; set; }
    public double AlturaALaCruz { get; set; }

    public void DatosPerro(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        Gato objGato = new();
    }
}
```

```

    Perro objPerro = new();

    //Da valores a la instancia de gato
    objGato.DatosGato("Sally", 'H', "Rafael Moreno");

    //Da valores a la instancia de perro
    objPerro.DatosPerro("Kitty", 'H', "Chloe Perry");
}
}

```

```

public void DatosPerro(string Nombre, char Sexo, string Dueno) {
    this.Nombre = Nombre;
    this.S
    this.D
}

```

 [class System.String](#)  
 Represents text as a sequence of UTF-16 code units.  
 [Describir con Copilot](#)

CS0122: 'Mascota.Nombre' no es accesible debido a su nivel de protección  
 Mostrar posibles correcciones (Alt+Entrar o Ctrl+.)

Ilustración 18: "Private" impide usar métodos o atributos en clases hijas



## Protected

Si la clase madre tiene atributos o métodos con la palabra "protected", significa que esos atributos pueden ser accedidos por las clases hijas, pero no pueden ser accedidos por instancias.

D/018.cs

```
namespace Ejemplo;

//Clase madre con atributos privados
class Mascota {
    protected string Nombre { get; set; }

    //Macho, Hembra
    protected char Sexo { get; set; }

    //Nombre del propietario
    protected string Dueno { get; set; }
}

class Gato : Mascota {
    public string PatronPelo { get; set; }
    public int TendenciaPerderPelo { get; set; }
    public void DatosGato(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

class Perro : Mascota {
    //Crianza
    public string CriadoPara { get; set; }
    public double AlturaALaCruz { get; set; }

    public void DatosPerro(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        Mascota objMascota = new();
        Gato objGato = new();
        Perro objPerro = new();
    }
}
```

```

//Da valores a la instancia de gato
objGato.DatosGato("Sally", 'H', "Rafael Moreno");

//Da valores a la instancia de perro
objPerro.DatosPerro("Kitty", 'H', "Chloe Perry");

//Intenta acceder a los métodos protegidos de Mascota
objMascota.Nombre = "Milú";
}
}

```

```

//Intenta acceder a los métodos protegidos de Mascota
objMascota.Nombre = "Milú";

```



 `class System.String`

Represents text as a sequence of UTF-16 code units.

 [Describir con Copilot](#)

[CS0122](#): 'Mascota.Nombre' no es accesible debido a su nivel de protección

[Mostrar posibles correcciones](#) (Alt+Entrar o Ctrl+.)

*Ilustración 19: "Protected" permite uso en clases hijas, pero no en instancias*

## Public

Cuando los atributos o métodos tienen la palabra “public”, entonces pueden ser accedidos por las clases hijas y también por las instancias.

D/019.cs

```
namespace Ejemplo;

//Clase madre con atributos privados
class Mascota {
    public string Nombre { get; set; }

    //Macho, Hembra
    public char Sexo { get; set; }

    //Nombre del propietario
    public string Dueno { get; set; }
}

class Gato : Mascota {
    public string PatronPelo { get; set; }
    public int TendenciaPerderPelo { get; set; }
    public void DatosGato(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

class Perro : Mascota {
    //Crianza
    public string CriadoPara { get; set; }
    public double AlturaALaCruz { get; set; }

    public void DatosPerro(string Nombre, char Sexo, string Dueno) {
        this.Nombre = Nombre;
        this.Sexo = Sexo;
        this.Dueno = Dueno;
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        Mascota objMascota = new();
        Gato objGato = new();
        Perro objPerro = new();
    }
}
```

```
//Da valores a la instancia de gato
objGato.DatosGato("Sally", 'H', "Rafael Moreno");

//Da valores a la instancia de perro
objPerro.DatosPerro("Kitty", 'H', "Chloe Perry");

//Intenta acceder a los métodos protegidos de Mascota
objMascota.Nombre = "Milú";
}
}
```

## Herencia y métodos iguales en clase madre e hija

¿Qué sucede si un método están en la clase madre y se escribe un método con el mismo nombre en la clase hija y luego se instancia la clase hija y se ejecuta ese método? El término se le conoce como ocultamiento de método.

D/020.cs

```
namespace Ejemplo;

internal class Madre {
    public void Procedimiento() {
        Console.WriteLine("En la clase madre");
    }
}

internal class Hija : Madre {
    public new void Procedimiento() {
        Console.WriteLine("En la clase hija");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        Hija objHija = new();
        objHija.Procedimiento();
    }
}
```

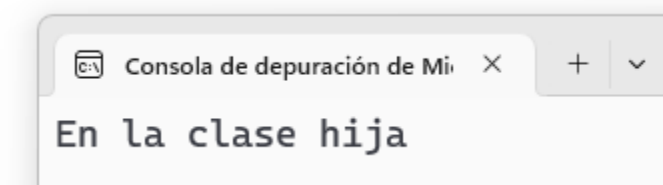


Ilustración 20: Herencia y métodos iguales en clase madre e hija

## Polimorfismo

Obsérvese este código:

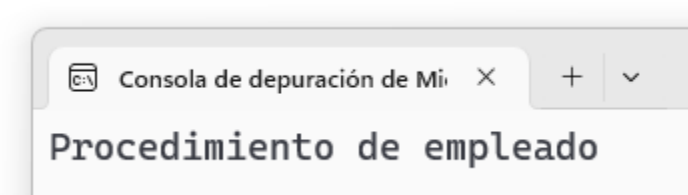
```
namespace Ejemplo;

internal class Empleado {
    public void Procedimiento() {
        Console.WriteLine("Procedimiento de empleado");
    }
}

internal class Operario : Empleado {
    public void Procedimiento() {
        Console.WriteLine("Operario, ejecuta");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Todo operario es un empleado
        Empleado objeto = new Operario();
        objeto.Procedimiento();
    }
}
```

Al ejecutar sucede esto:



*Ilustración 21: Polimorfismo clásico*

En C#, una referencia de tipo base (clase Madre) puede apuntar a un objeto de tipo derivado (clase Hija). Esto se llama "upcasting", y es seguro porque todo Operario es un Empleado. Sin embargo, al llamar a "Procedimiento" se ejecuta el método de la clase Madre. ¿Cómo ejecutar el "Procedimiento" de la clase Hija? Usando "virtual" y "override"

## Polimorfismo, virtual, override

Ahora este código:

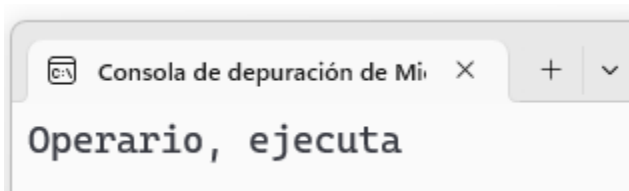
```
namespace Ejemplo;

internal class Empleado {
    public virtual void Procedimiento() {
        Console.WriteLine("Procedimiento de empleado");
    }
}

internal class Operario : Empleado {
    public override void Procedimiento() {
        Console.WriteLine("Operario, ejecuta");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Todo operario es un empleado
        Empleado objeto = new Operario();
        objeto.Procedimiento();
    }
}
```

Al ejecutar sucede esto:



*Ilustración 22: virtual, override*

Este patrón permite **polimorfismo**, es decir, que se pueda tratar diferentes tipos derivados como si fueran del tipo base, y aun así obtener comportamientos específicos.

```

namespace Ejemplo;

internal class Empleado {
    public virtual void Procedimiento() {
        Console.WriteLine("Procedimiento de empleado");
    }
}

internal class Operario : Empleado {
    public override void Procedimiento() {
        Console.WriteLine("Ejecuta método de Operario");
    }
}

internal class Ingeniero : Empleado {
    public override void Procedimiento() {
        Console.WriteLine("Ejecuta método de Ingeniero");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Todo operario es un empleado
        Empleado objetoA = new Operario();
        Empleado objetoB = new Ingeniero();
        objetoA.Procedimiento();
        objetoB.Procedimiento();
    }
}

```

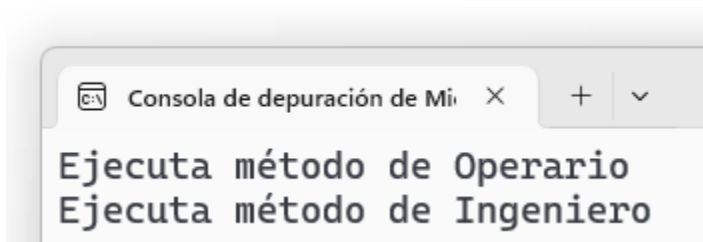


Ilustración 23: Polimorfismo



## Herencia y constructores

¿Qué sucede si la clase abuela o madre o hija tienen todos constructores? ¿Se ejecutan todos?  
¿En qué orden?

D/021.cs

```
namespace Ejemplo;

class Abuela {
    //Constructor
    public Abuela() {
        Console.WriteLine("Constructor de la clase abuela");
    }

    //Método
    public void Mostrar() {
        Console.WriteLine("Mostrar en Abuela");
    }
}

class Madre : Abuela {
    //Constructor
    public Madre() {
        Console.WriteLine("Constructor de la clase madre");
    }

    public new void Mostrar() {
        Console.WriteLine("Mostrar en Madre");
    }
}

class Hija : Madre {
    //Constructor
    public Hija() {
        Console.WriteLine("Constructor de la clase hija");
    }

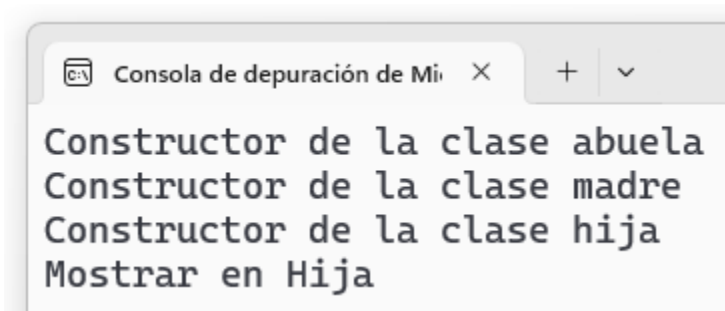
    public new void Mostrar() {
        Console.WriteLine("Mostrar en Hija");
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia a la hija
        Hija objHija = new();
    }
}
```

```
//Ejecuta método  
objHija.Mostrar();  
}  
}
```

La ejecución del programa hace lo siguiente:

1. Ejecuta el constructor de la clase abuela
2. Ejecuta el constructor de la clase madre
3. Ejecuta el constructor de la clase hija
4. Si un método, diferente al constructor, tiene el mismo nombre en las clases abuela, madre e hija. Al ser ejecutado por la instancia de la clase hija, sólo ejecutará el método de la clase hija.



*Ilustración 24: Constructores y herencia*

## Llamando a métodos de clases madres

Desde el método de la clase hija se hace uso de la instrucción "base" y así se llama al método de la clase madre

D/022.cs

```
namespace Ejemplo;

internal class Abuela {
    //Constructor
    public Abuela() {
        Console.WriteLine("Constructor de la clase abuela");
    }

    //Método
    public void Mostrar() {
        Console.WriteLine("Mostrar en Abuela");
    }
}

internal class Madre : Abuela {
    //Constructor
    public Madre() {
        Console.WriteLine("Constructor de la clase madre");
    }

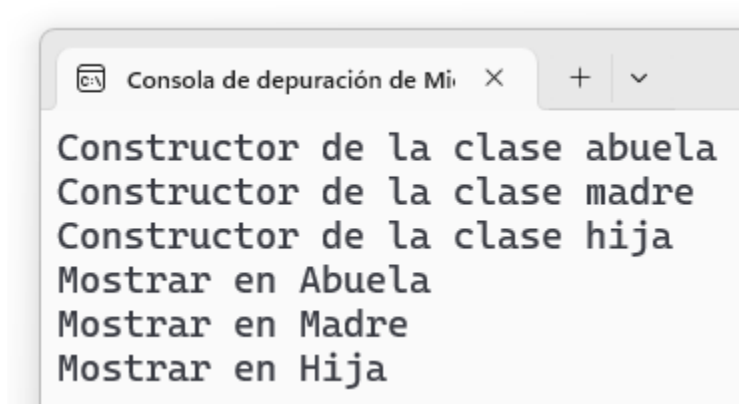
    public new void Mostrar() {
        base.Mostrar(); //Llama al método de la clase abuela
        Console.WriteLine("Mostrar en Madre");
    }
}

internal class Hija : Madre {
    //Constructor
    public Hija() {
        Console.WriteLine("Constructor de la clase hija");
    }

    public new void Mostrar() {
        base.Mostrar(); //Llama al método de la clase madre
        Console.WriteLine("Mostrar en Hija");
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
```

```
//Instancia a la hija  
Hija objHija = new();  
  
//Ejecuta método  
objHija.Mostrar();  
}  
}
```



*Ilustración 25: Llamando a métodos de clases madres*

## Evitar la herencia

Con la palabra reservada "sealed" se evita que de esa clase se pueda heredar. Si se intenta habrá un error en tiempo de compilación.

D/023.cs

```
namespace Ejemplo;

sealed class Madre {
    public void Aviso() {
        Console.WriteLine("Método en clase madre");
    }
}

class Hija : Madre {
    public void Mensaje() {
        Console.WriteLine("Método en clase hija");
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        Hija objHija = new();
        objHija.Mensaje();
    }
}
```

1 referencia

```
sealed class Madre {
```

0 referencias

```
    public void Aviso() {
```

```
        Console.WriteLine("Método en clase madre");
```

```
    }
```

```
}
```

2 referencias

```
class Hija : Madre {
```

1 referencia

```
    public void
```

```
        Console.
```

```
    }
```

```
}
```

```
//Inicia la apli
```

  class Ejemplo.Madre

 Describir con Copilot

CS0509: 'Hija': no puede derivar del tipo sellado 'Madre'

[Mostrar posibles correcciones](#) (Alt+Entrar o Ctrl+.)

Ilustración 26: Evitar la herencia

# Clases estáticas

Una clase estática no requiere instanciarse para ser usada.

D/024.cs

```
namespace Ejemplo;

//Clase estática, no necesita instanciarse
static class Geometria {
    //Todos los métodos deben ser static
    public static double AreaTriangulo(double baseT, double alturaT) {
        return baseT * alturaT / 2;
    }

    public static double AreaTriangulo(double ladoA, double ladoB,
                                       double ladoC) {
        double s = (ladoA + ladoB + ladoC) / 2;
        return Math.Sqrt(s * (s - ladoA) * (s - ladoB) * (s - ladoC));
    }

    public static double AreaCirculo(double radio) {
        return Math.PI * radio * radio;
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Hace uso de la clase sin instanciarla
        double unRadio = 7;
        double AreaUnCirculo = Geometria.AreaCirculo(unRadio);
        Console.WriteLine("Área círculo es: " + AreaUnCirculo);

        double AreaTri = Geometria.AreaTriangulo(3, 4, 5);
        Console.WriteLine("Área triángulo es: " + AreaTri);
    }
}
```

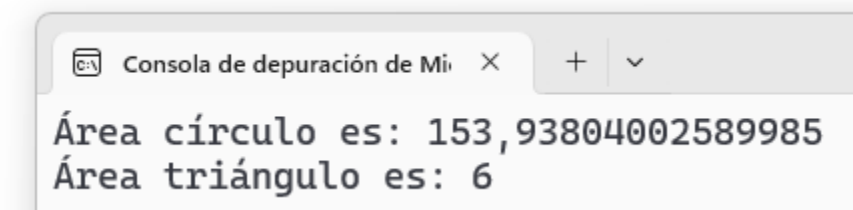


Ilustración 27: Clases estáticas

## Métodos estáticos

Una clase tradicional puede tener métodos estáticos y estos métodos pueden ser accedidos sin necesidad de instanciar la clase, los otros métodos no estáticos si requieren que se instancie la clase.

D/025.cs

```
namespace Ejemplo;

//Clase con métodos estáticos
internal class Geometria {
    //Estos métodos estáticos pueden ser usados sin instanciar la clase
    public static double AreaTriangulo(double baseT, double alturaT) {
        return baseT * alturaT / 2;
    }

    public static double AreaTriangulo(double ladoA, double ladoB,
                                       double ladoC) {
        double s = (ladoA + ladoB + ladoC) / 2;
        return Math.Sqrt(s * (s - ladoA) * (s - ladoB) * (s - ladoC));
    }

    public static double AreaCirculo(double radio) {
        return Math.PI * radio * radio;
    }

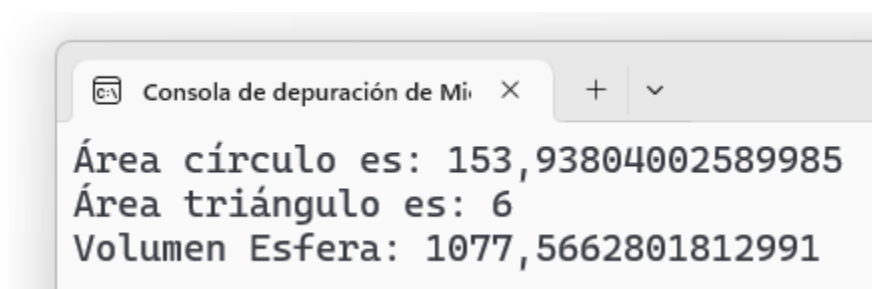
    //Este método requiere instanciar la clase
    public double VolumenEsfera(double radio) {
        return 4 / 3 * Math.PI * Math.Pow(radio, 3);
    }
}

internal class Program {
    static void Main() {
        double unRadio = 7;
        double AreaUnCirculo = Geometria.AreaCirculo(unRadio);
        Console.WriteLine("Área círculo es: " + AreaUnCirculo);

        double AreaTri = Geometria.AreaTriangulo(3, 4, 5);
        Console.WriteLine("Área triángulo es: " + AreaTri);

        //Instancio la clase
        Geometria objGeometria = new();
        double Esfera = objGeometria.VolumenEsfera(7);
        Console.WriteLine("Volumen Esfera: " + Esfera);
    }
}
```





*Ilustración 28: Métodos estáticos*

## Constructor static

Para inicializar los atributos static de una clase, se hace uso de los constructores static. Una clase no static puede tener un constructor static y un constructor normal, el primero se ejecuta siempre al usarse la clase o instanciarse el objeto, en cambio, el constructor normal sólo se ejecuta al instanciarse la clase.

D/026.cs

```
namespace Ejemplo;

//Clase con métodos estáticos
class Geometria {
    public static int valorEntero;
    public static double valorReal;
    public static string unaCadena;

    //Constructor static (para inicializar atributos static)
    static Geometria() {
        valorEntero = 7;
        valorReal = 16.832;
        unaCadena = "Rafael";
        Console.WriteLine("Se ha ejecutado el constructor static");
    }

    //Constructor de clase
    public Geometria() {
        Console.WriteLine("Ejecuta el constructor de la clase");
    }

    //Este método estático puede ser usado sin instanciar la clase
    public static double AreaCirculo(double radio) {
        return Math.PI * radio * radio;
    }

    //Este método requiere instanciar la clase
    public double VolumenEsfera(double radio) {
        return 4 / 3 * Math.PI * Math.Pow(radio, 3);
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Acediendo a métodos estáticos
        double AreaUnCirculo = Geometria.AreaCirculo(7);
        Console.WriteLine("Área círculo es: " + AreaUnCirculo);
    }
}
```

```

//Accediendo a atributos estáticos
Console.WriteLine("Cadena es: " + Geometria.unaCadena);
Console.WriteLine("Valor entero es: " + Geometria.valorEntero);
Console.WriteLine("Valor real es: " + Geometria.valorReal);

//Se instancia la clase
Geometria objGeometria = new Geometria();
double Esfera = objGeometria.VolumenEsfera(7);
Console.WriteLine("Volumen Esfera: " + Esfera);
}
}

```

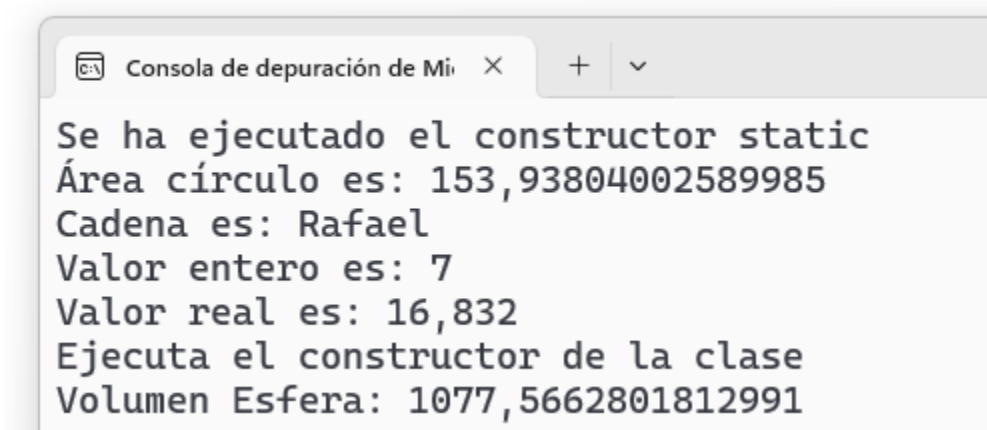


Ilustración 29: Constructor static

## Cuidado con el constructor static

El constructor static se ejecuta en el momento que es usada la clase por primera vez (por ejemplo, cuando se instancia), no se vuelve a usar más.

D/027.cs

```
namespace Ejemplo;

//Clase con métodos estáticos
internal class Geometria {
    public static int valorEntero;
    public static double valorReal;
    public static string unaCadena;

    //Constructor static (para inicializar atributos static)
    static Geometria() {
        valorEntero = 7;
        valorReal = 16.832;
        unaCadena = "Rafael";
        Console.WriteLine("Se ha ejecutado el constructor static");
    }

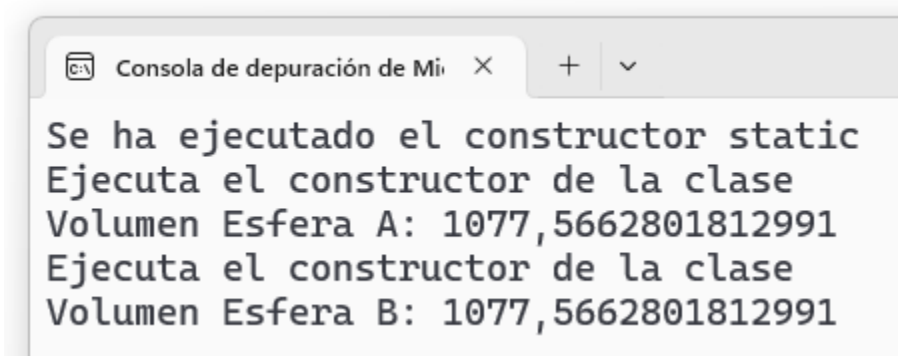
    //Constructor de clase
    public Geometria() {
        Console.WriteLine("Ejecuta el constructor de la clase");
    }

    //Este método estático puede ser usado sin instanciar la clase
    public static double AreaCirculo(double radio) {
        return Math.PI * radio * radio;
    }

    //Este método requiere instanciar la clase
    public double VolumenEsfera(double radio) {
        return 4 / 3 * Math.PI * Math.Pow(radio, 3);
    }
}

//Inicia la aplicación aquí
internal class Program {
    static void Main() {
        //Se instancia la clase la primera vez
        Geometria objGeometria = new();
        double Esfera = objGeometria.VolumenEsfera(7);
        Console.WriteLine("Volumen Esfera A: " + Esfera);
    }
}
```

```
//Se instancia la clase la segunda vez
Geometria objOtro = new();
double OtraEsfera = objOtro.VolumenEsfera(7);
Console.WriteLine("Volumen Esfera B: " + OtraEsfera);
}
}
```



*Ilustración 30: Cuidado con el constructor static*

# Interface

Con la palabra reservada "interface" se crean las definiciones de métodos y propiedades que deben ser escritos en las clases que implementen esa "interface". El estándar solicita que el nombre de las "interface" empiece por I. Nota: Las interfaces no implementan ningún código, obligan a la clase que hereda a implementar los métodos.

D/028.cs

```
namespace Ejemplo;

//Declara una interface (el estándar dice
//que debe empezar con la letra "I")
interface IMetodosRequeridos {
    //Métodos requeridos en las clases que
    //implementen esta interface
    double Area();
    double Perimetro();
}

//Esta clase debe implementar lo que dice la interface
class Circulo : IMetodosRequeridos {
    public double Radio { get; set; }

    public Circulo(double Radio) {
        this.Radio = Radio;
    }

    //Implementa los métodos señalados por la interface
    public double Area() {
        return Math.PI * Radio * Radio;
    }

    public double Perimetro() {
        return 2 * Math.PI * Radio;
    }
}

//Esta clase debe implementar lo que dice la interface
class Cuadrado : IMetodosRequeridos {
    public double Lado { get; set; }

    public Cuadrado(double Lado) {
        this.Lado = Lado;
    }

    //Implementa los métodos señalados por la interface
```

```

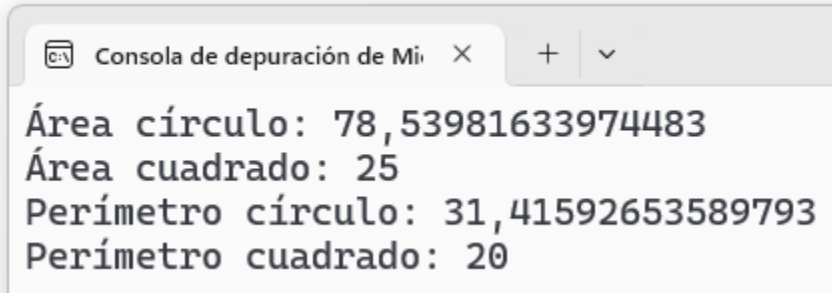
    public double Area() {
        return Lado * Lado;
    }

    public double Perimetro() {
        return 4 * Lado;
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia las clases
        Cuadrado cuad = new(5);
        Circulo circ = new(5);

        //Imprime los valores
        Console.WriteLine("Área círculo: " + circ.Area());
        Console.WriteLine("Área cuadrado: " + cuad.Area());
        Console.WriteLine("Perímetro círculo: " + circ.Perimetro());
        Console.WriteLine("Perímetro cuadrado: " + cuad.Perimetro());
    }
}

```



Área círculo: 78,53981633974483  
 Área cuadrado: 25  
 Perímetro círculo: 31,41592653589793  
 Perímetro cuadrado: 20

*Ilustración 31: Interface*

## Interface múltiple

Una clase puede "heredar" de dos o más interfaces.

D/029.cs

```
namespace Ejemplo;

//Declara una interface (el estándar
//dice que debe empezar con la letra "I")
interface ICalculos {
    //Métodos requeridos en las clases
    //que implementen esta interface
    double Area();
    double Perimetro();
}

//Otra interface para obligar a mostrar los resultados
interface IMuestra {
    void VerArea();
    void VerPerimetro();
}

//Implementa de varios Interface
class Circulo : ICalculos, IMuestra {
    public double Radio { get; set; }

    public Circulo(double Radio) {
        this.Radio = Radio;
    }

    //Implementa los métodos señalados por la interface
    public double Area() {
        return Math.PI * Radio * Radio;
    }

    public double Perimetro() {
        return 2 * Math.PI * Radio;
    }

    public void VerArea() {
        Console.WriteLine("Área círculo: " + Area());
    }

    public void VerPerimetro() {
        Console.WriteLine("Perímetro círculo: " + Perimetro());
    }
}
```



```

}

class Cuadrado : ICalculos, IMuestra {
    public double Lado { get; set; }

    public Cuadrado(double Lado) {
        this.Lado = Lado;
    }

    //Implementa los métodos señalados por la interface
    public double Area() {
        return Lado * Lado;
    }

    public double Perimetro() {
        return 4 * Lado;
    }

    public void VerArea() {
        Console.WriteLine("Área cuadrado: " + Area());
    }

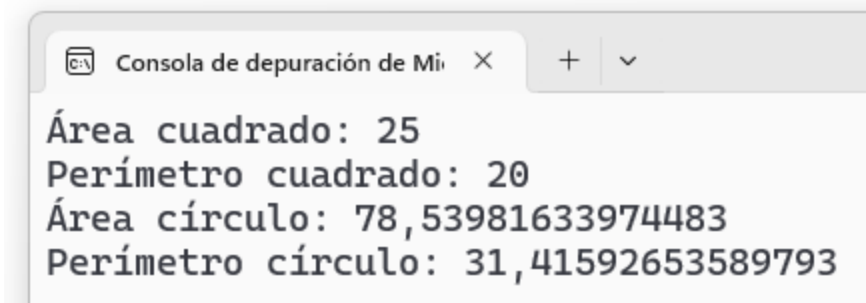
    public void VerPerimetro() {
        Console.WriteLine("Perímetro cuadrado: " + Perimetro());
    }
}

//Inicia la aplicación aquí
class Program {
    static void Main() {
        //Instancia las clases
        Cuadrado objCuadrado = new(5);
        Circulo objCirculo = new(5);

        //Imprime los valores
        objCuadrado.VerArea();
        objCuadrado.VerPerimetro();

        objCirculo.VerArea();
        objCirculo.VerPerimetro();
    }
}

```



*Ilustración 32: Interface múltiple*

## Interface (coinciden métodos)

Si una clase implementa dos o más interfaces que tienen métodos con el mismo nombre y firma, se puede generar una ambigüedad.

```
namespace Ejemplo;

internal interface InterfazA {
    public void Imprime();
}

internal interface InterfazB {
    public void Imprime();
}

internal class MiClase : InterfazA, InterfazB {
    public void Imprime() {
        Console.WriteLine("Esto es una prueba");
    }
}

internal class Program {
    static void Main(string[] args) {
        MiClase objeto = new();
        objeto.Imprime();
    }
}
```

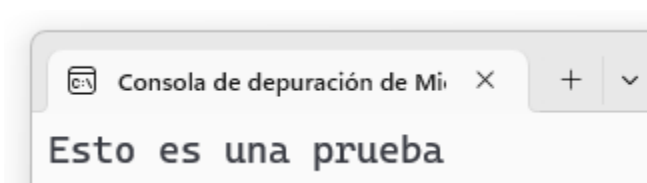


Ilustración 33: Dos "interface" con el mismo método

## Implementación explícita para resolver ambigüedad

Para resolver la ambigüedad en el caso de “heredar” dos interfaces con el mismo método, se hace uso de la implementación explícita:

```
namespace Ejemplo;

internal interface InterfazA {
    public void Imprime();
}

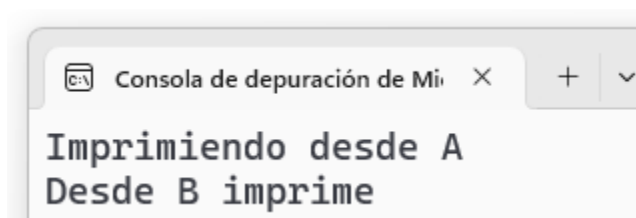
internal interface InterfazB {
    public void Imprime();
}

internal class MiClase : InterfazA, InterfazB {
    void InterfazA.Imprime() {
        Console.WriteLine("Imprimiendo desde A");
    }

    void InterfazB.Imprime() {
        Console.WriteLine("Desde B imprime");
    }
}

internal class Program {
    static void Main() {
        InterfazA objA = new MiClase();
        InterfazB objB = new MiClase();

        objA.Imprime();
        objB.Imprime();
    }
}
```



*Ilustración 34: Implementación explícita en interfaces*

## Herencia múltiple con Interface

Sólo se puede heredar de **una clase** pero se puede "heredar" de varios Interface.

D/030.cs

```
namespace Ejemplo;

interface IMetodos {
    void MetodoA();
    void MetodoB();
}

interface IProcedimientos {
    void ProcedimientoA();
    void ProcedimientoB();
}

class Madre {
    public void Aviso() {
        Console.WriteLine("Método de clase madre");
    }
}

//Hereda de Madre e implementa de IMetodos e IProcedimientos
class Hija : Madre, IMetodos, IProcedimientos {
    public void Mensaje() {
        Console.WriteLine("En clase hija");
    }

    public void MetodoA() {
        Console.WriteLine("En MetodoA");
    }

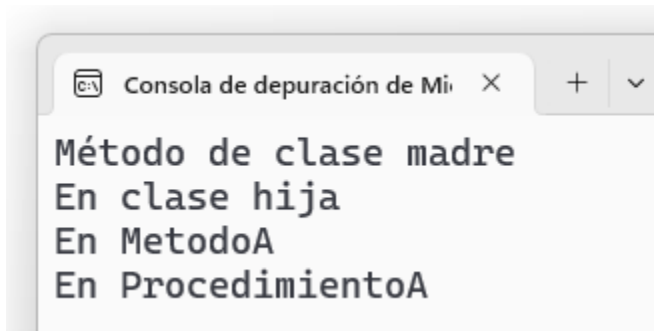
    public void MetodoB() {
        Console.WriteLine("En MetodoB");
    }

    public void ProcedimientoA() {
        Console.WriteLine("En ProcedimientoA");
    }

    public void ProcedimientoB() {
        Console.WriteLine("En ProcedimientoB");
    }
}

//Inicia la aplicación aquí
```

```
class Program {  
    static void Main() {  
        Hija objHija = new();  
        objHija.Aviso();  
        objHija.Mensaje();  
        objHija.MetodoA();  
        objHija.ProcedimientoA();  
    }  
}
```



*Ilustración 35: Herencia e Interface*

# Enums

Es una "clase especial" para guardar constantes

D/031.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una "clase especial" para almacenar constantes
    enum Meses {
        Enero, //0
        Febrero, //1
        Marzo, //2
        Abril, //3
        Mayo, //4
        Junio, //5
        Julio, //6
        Agosto, //7
        Septiembre, //8
        Octubre, //9
        Noviembre, //10
        Diciembre //11
    }

    static void Main() {
        Meses unMes = Meses.Junio;
        Console.WriteLine(unMes);
        Console.WriteLine((int)unMes);
    }
}
```

Por defecto, las constantes de un enum comienzan a enumerarse desde cero, pero eso pueden cambiarse.

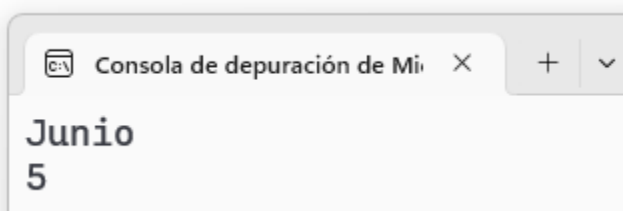


Ilustración 36: Enums

## Cambiando los valores de las constantes en enums

### Ejemplo 1

El método para que cada constante tenga su propio valor es constante = valor

D/032.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una "clase especial" para almacenar constantes
    enum Meses {
        Enero = 1,
        Febrero = 2,
        Marzo = 3,
        Abril = 4,
        Mayo = 5,
        Junio = 6,
        Julio = 7,
        Agosto = 8,
        Septiembre = 9,
        Octubre = 10,
        Noviembre = 11,
        Diciembre = 12
    }

    static void Main() {
        Meses unMes = Meses.Junio;
        Console.WriteLine(unMes);
        Console.WriteLine((int)unMes);
    }
}
```

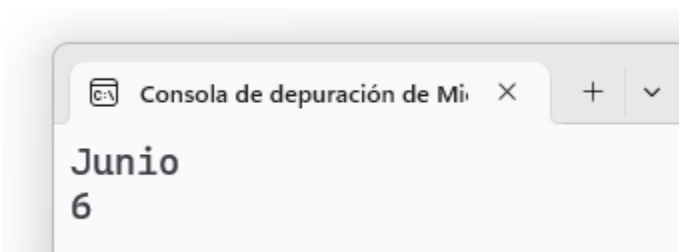


Ilustración 37: Cambiando los valores de las constantes en enums



```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una "clase especial" para almacenar constantes
    enum Valores {
        valorA = 89,
        valorB = 12,
        valorC = 47,
        valorD = 63
    }

    static void Main() {
        Valores unosValores = Valores.valorC;
        Console.WriteLine(unosValores);
        Console.WriteLine((int)unosValores);
    }
}
```

Nota 1: Los valores sólo pueden ser de tipo entero, long, byte, sbyte, short, ushort, uint, long, ulong

Nota 2: Los valores no pueden ser reales o cadenas o caracteres

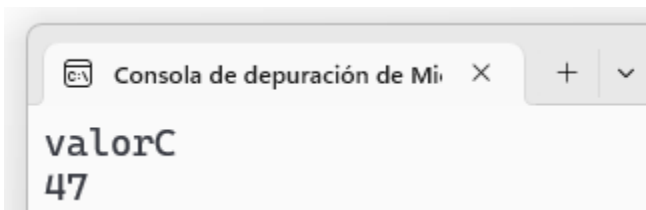


Ilustración 38: Cambiando los valores de las constantes en enums

## Structs

C# trae las estructuras, que tienen un gran parecido a las clases, a tal punto que podrían ser su reemplazo en varias ocasiones porque tienen características interesantes como poder copiar el contenido de un struct en otro con el simple operador de asignación (algo que requiere un tratamiento si se usaran clases). Los structs tienen sus diferencias con respecto a las clases: sólo pueden definir constructores propios (con parámetros de entrada), pero no se puede crear un constructor simple, no pueden heredar, ni se puede heredar de estos, tampoco se pueden hacer structs abstractos.

D/034.cs

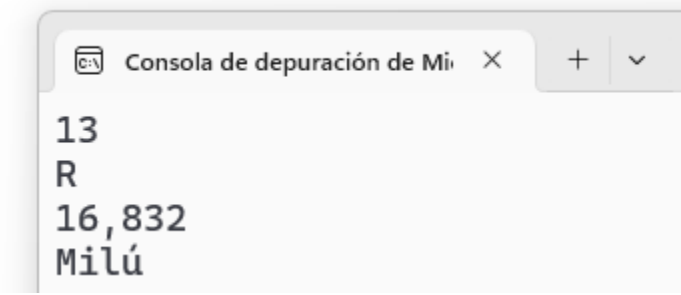
```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una estructura
    struct Valores {
        public int valorA;
        public char valorB;
        public double valorC;
        public string valorD;
    }

    static void Main() {
        //Crea una variable de tipo struct
        Valores unosValores;
        unosValores.valorA = 13;
        unosValores.valorB = 'R';
        unosValores.valorC = 16.832;
        unosValores.valorD = "Milú";

        //Puede imprimir esos valores
        Console.WriteLine(unosValores.valorA);
        Console.WriteLine(unosValores.valorB);
        Console.WriteLine(unosValores.valorC);
        Console.WriteLine(unosValores.valorD);
    }
}
```



*Ilustración 39: Structs*

## Un struct se puede copiar fácilmente

Los valores de una variable struct se pueden copiar en otra variable struct de la misma estructura usando el operador de asignación (=). Si se modifica el original, no afecta a la copia.

D/035.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una estructura
    struct Valores {
        public int valorA;
        public char valorB;
        public double valorC;
        public string valorD;
    }

    static void Main() {
        //Crea una variable de tipo struct
        Valores unosValores;
        unosValores.valorA = 13;
        unosValores.valorB = 'R';
        unosValores.valorC = 16.832;
        unosValores.valorD = "Milú";

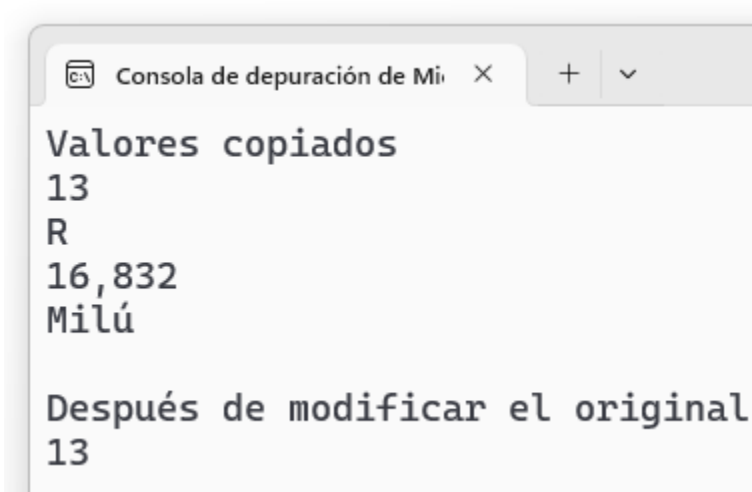
        //Crea una segunda variable y le asigna la primera
        //creando una copia
        Valores otro;
        otro = unosValores;

        //Puede imprimir esos valores
        Console.WriteLine("Valores copiados");
        Console.WriteLine(otro.valorA);
        Console.WriteLine(otro.valorB);
        Console.WriteLine(otro.valorC);
        Console.WriteLine(otro.valorD);

        //Modifica la original
        unosValores.valorA = -9876;

        //Imprime la copia
        Console.WriteLine("\nDespués de modificar el original");
        Console.WriteLine(otro.valorA);
    }
}
```

}



*Ilustración 40: Un struct se puede copiar fácilmente*

## Métodos en un struct

Un struct puede tener métodos.

D/036.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

    //Una estructura
    struct Valores {
        private int valorA;
        private char valorB;
        private double valorC;
        private string valorD;

        public void Asigna(int A, char B, double C, string D) {
            this.valorA = A;
            this.valorB = B;
            this.valorC = C;
            this.valorD = D;
        }

        public void ImprimeValores() {
            Console.WriteLine(valorA);
            Console.WriteLine(valorB);
            Console.WriteLine(valorC);
            Console.WriteLine(valorD);
        }

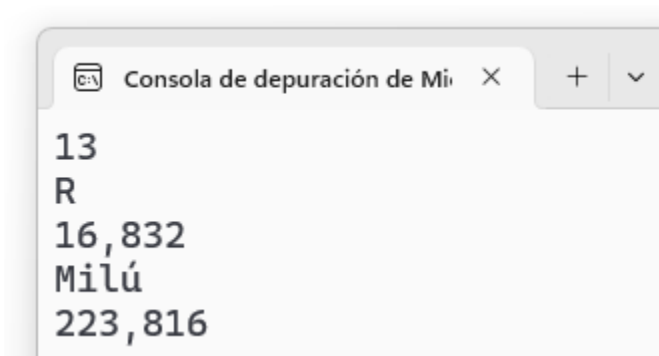
        public double RetornaValor(double numero) {
            return valorA * valorC + numero;
        }
    }

    static void Main() {
        //Crea una variable de tipo struct y
        //la inicializa por defecto
        Valores unosValores = default;

        //Llama a los métodos del struct
        unosValores.Asigna(13, 'R', 16.832, "Milú");
        unosValores.ImprimeValores();

        //Y a la función del struct
        Console.WriteLine(unosValores.RetornaValor(5));
    }
}
```

```
}  
}
```



*Ilustración 41: Métodos en un struct*

## Structs y constructores

Un struct puede tener un constructor que tenga parámetros (no se puede generar un constructor sin parámetros). Ejemplo:

D/037.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {

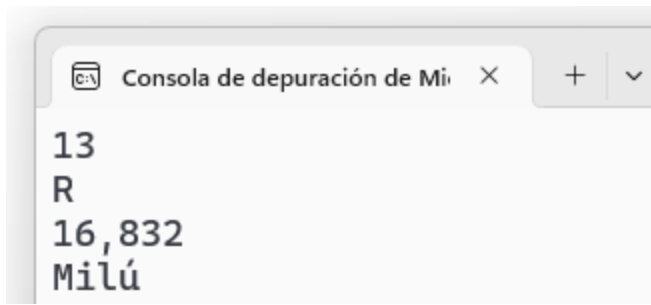
    //Una estructura
    struct Valores {
        private int valorA;
        private char valorB;
        private double valorC;
        private string valorD;

        public Valores(int valA, char valB, double valC, string valD) {
            this.valorA = valA;
            this.valorB = valB;
            this.valorC = valC;
            this.valorD = valD;
        }

        public void ImprimeValores() {
            Console.WriteLine(valorA);
            Console.WriteLine(valorB);
            Console.WriteLine(valorC);
            Console.WriteLine(valorD);
        }
    }

    static void Main() {
        //Crea una variable de tipo struct y
        //la inicializa con un constructor
        Valores unosValores = new Valores(13, 'R', 16.832, "Milú");
        unosValores.ImprimeValores();
    }
}
```





*Ilustración 42: Structs y constructores*

## Clases parciales

C# puede tener partes de una misma clase en diferentes archivos. Útil si dos o más programadores quieren trabajar en la misma clase al tiempo. En Visual Studio, se hace para separar la parte lógica de un formulario, de la parte de diseño GUI de ese mismo formulario.

D/038a.cs

```
namespace Ejemplo;

partial class MiClase {
    public MiClase(int valA, double valB, char valC, string valD) {
        ValorA = valA;
        ValorB = valB;
        ValorC = valC;
        ValorD = valD;
    }

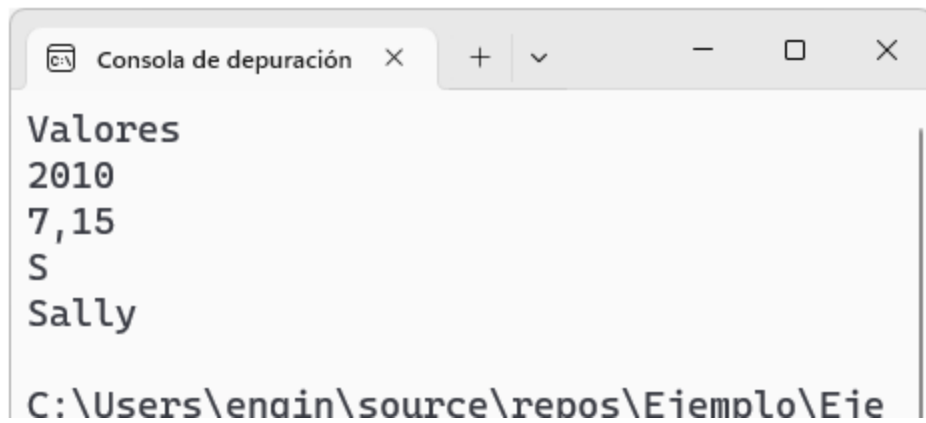
    public void Imprime() {
        Console.WriteLine("Valores");
        Console.WriteLine(ValorA);
        Console.WriteLine(ValorB);
        Console.WriteLine(ValorC);
        Console.WriteLine(ValorD);
    }
}

//Inicia la aplicación aquí
class Program {
    public static void Main() {
        MiClase objClase = new MiClase(2010, 7.15, 'S', "Sally");
        objClase.Imprime();
    }
}
```

D/038b.cs

```
namespace Ejemplo;

partial class MiClase {
    public int ValorA { get; set; }
    public double ValorB { get; set; }
    public char ValorC { get; set; }
    public string ValorD { get; set; }
}
```



*Ilustración 43: Clases parciales*

## Destruyores

Por lo general, C# se encarga automáticamente de liberar la memoria de los objetos que ya no pueden ser usados (por ejemplo, cuando se crea un objeto dentro de una función con una variable local y luego termina la función). Aun así, en raras ocasiones, es necesario tener un método que se ejecuta cuando el objeto es eliminado, sería la contraparte del constructor y es conocido como el destructor. A pesar de su existencia, los destructores no pueden llamarse explícitamente. Eso sucede cuando el "Garbage Collector" lo considere oportuno. Entonces una forma de ejecutar el destructor es llamando explícitamente al Garbage Collector (con las siglas GC) para que haga limpieza y liberación de memoria.

Los destructores se nombran iniciando con el símbolo ~ seguido del nombre de la clase. No tienen parámetros.

D/039a.cs

```
namespace Ejemplo;

//Inicia la aplicación aquí
internal class Program {
    public static void Main() {
        Procedimiento();

        //Ejecuta el Garbage Collector
        GC.Collect(); //Limpia todo
        GC.WaitForPendingFinalizers(); //Espera que se limpie todo

        Console.WriteLine("Termina el programa");
    }

    public static void Procedimiento() {
        //Se instancia la clase con una variable local
        MiClase objClase = new MiClase(2010, 7.15, 'S', "Sally");
        objClase.Imprime();

        //Aquí debería ejecutarse el destructor de esa clase
    }
}
```

D/039b.cs

```
namespace Ejemplo;

partial class MiClase {
    public int ValorA { get; set; }
    public double ValorB { get; set; }
    public char ValorC { get; set; }
```

```

public string ValorD { get; set; }

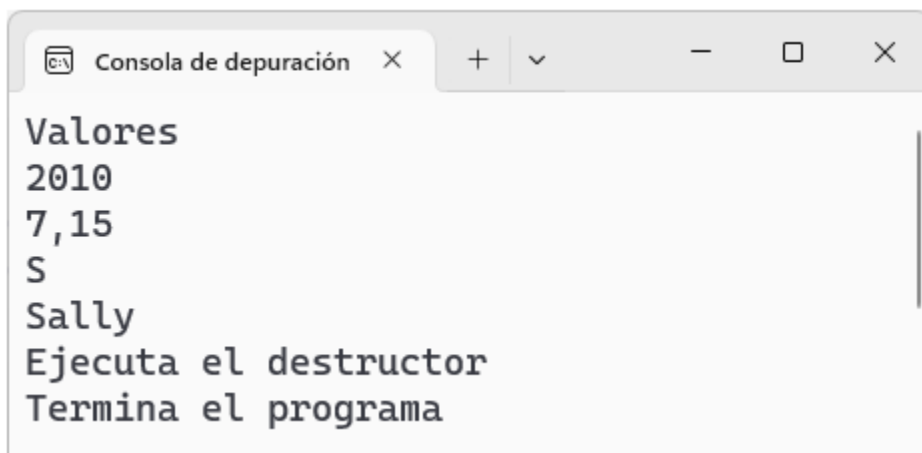
public MiClase(int valA, double valB, char valC, string valD) {
    ValorA = valA;
    ValorB = valB;
    ValorC = valC;
    ValorD = valD;
}

public void Imprime() {
    Console.WriteLine("Valores");
    Console.WriteLine(ValorA);
    Console.WriteLine(ValorB);
    Console.WriteLine(ValorC);
    Console.WriteLine(ValorD);
}

//Destructor
~MiClase() {
    Console.WriteLine("Ejecuta el destructor");
}
}

```

Nota: Por lo publicado en diversos foros sobre los destructores y el uso del Garbage Collector, esto debe hacerlo con mucho cuidado, así que se recomienda no hacer uso de destructores, ni llamar al Garbage Collector.



*Ilustración 44: Destructores*

# Patrones de diseño

A continuación, algunos patrones de diseño implementados en C#:

## Factory Method

El patrón de diseño Factory Method es un patrón creacional que se utiliza para crear objetos sin tener que especificar su clase exacta.

Se necesita crear objetos de una clase, pero no se sabe qué clase exacta necesita hasta que se ejecuta el programa. El patrón Factory Method permite crear objetos sin tener que especificar su clase exacta. En lugar de crear objetos directamente, se utiliza un método de fábrica para crear objetos. Este método de fábrica se encarga de crear el objeto correcto según los parámetros que se le pasen.

```

namespace Ejemplo;
//Patrón: Factory Method

//Interface que obliga a definir el método dibujar
interface IFigura {
    void Dibujar();
}

class Circulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Se hace el dibujo de un círculo");
    }
}

class Rectangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Estoy dibujando un rectángulo");
    }
}

class Triangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Ahora se dibuja un triángulo");
    }
}

class FabricaFiguras {
    //Dependiendo del parámetro retorna uno u otro objeto
    public IFigura GetFigura(string TipoFigura) {
        if (TipoFigura.Equals("CIRCULO"))
            return new Circulo();

        if (TipoFigura.Equals("RECTANGULO"))
            return new Rectangulo();

        if (TipoFigura.Equals("TRIANGULO"))
            return new Triangulo();

        return null;
    }
}

class Program {
    static void Main() {
        FabricaFiguras objeto = new();

        //Obtiene un objeto círculo
    }
}

```

```
IFigura Figura1 = objeto.GetFigura("CIRCULO");

//Llama el método de dibujar del objeto círculo
Figura1.Dibujar();

//Obtiene un objeto rectángulo
IFigura Figura2 = objeto.GetFigura("RECTANGULO");

//Llama el método de dibujar del objeto rectángulo
Figura2.Dibujar();

//Obtiene un objeto triángulo
IFigura Figura3 = objeto.GetFigura("TRIANGULO");

//Llama el método de dibujar del objeto triángulo
Figura3.Dibujar();
}
}
```

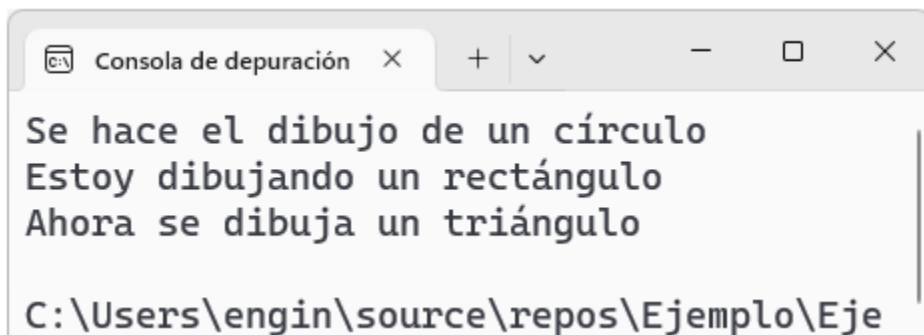


Ilustración 45: Factory Method



## Abstract Factory

El patrón de diseño Abstract Factory es un patrón creacional que se utiliza para crear familias de objetos relacionados o dependientes sin especificar su clase concreta.

Es necesario crear un conjunto de objetos que trabajen juntos, pero no se sabe qué objetos específicos se necesitan hasta que se ejecuta el programa: El patrón Abstract Factory permite crear una fábrica abstracta que define una interfaz para crear objetos relacionados o dependientes. Luego, puede crear fábricas concretas que implementan la fábrica abstracta y crean objetos específicos. De esta manera, puede crear diferentes familias de objetos relacionados o dependientes sin tener que cambiar el código del cliente.

D/041.cs

```
namespace Ejemplo;
//Patrón: Abstract Factory

//Interface que obliga a definir el método dibujar
public interface IFigura {
    void Dibujar();
}

class Rectangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Estoy dibujando un rectángulo");
    }
}

class Triangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Ahora se dibuja un triángulo");
    }
}

class Circulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Se hace el dibujo de un círculo");
    }
}

public interface IColor {
    void Rellenar();
}

class Rojo : IColor {
    public void Rellenar() {
        Console.WriteLine("Pinta de rojo");
    }
}
```

```

}

class Verde : IColor {
    public void Rellenar() {
        Console.WriteLine("Un verde es pintado");
    }
}

class Azul : IColor {
    public void Rellenar() {
        Console.WriteLine("Ahora de azul es rellenado");
    }
}

public abstract class Fabrica {
    public abstract IFigura GetFigura(string TipoFigura);
    public abstract IColor GetColor(string color);
}

public class FabricaFiguras : Fabrica {
    //Dependiendo del parámetro retorna uno u otro objeto
    public override IFigura GetFigura(string TipoFigura) {

        if (TipoFigura.Equals("CIRCULO"))
            return new Circulo();

        if (TipoFigura.Equals("RECTANGULO"))
            return new Rectangulo();

        if (TipoFigura.Equals("TRIANGULO"))
            return new Triangulo();

        return null;
    }

    public override IColor GetColor(string color) {
        return null;
    }
}

class FabricaColores : Fabrica {
    //Dependiendo del parámetro retorna uno u otro objeto
    public override IFigura GetFigura(string TipoFigura) {
        return null;
    }

    public override IColor GetColor(string color) {

```

```

        if (color.Equals("ROJO"))
            return new Rojo();

        if (color.Equals("VERDE"))
            return new Verde();

        if (color.Equals("AZUL"))
            return new Azul();

        return null;
    }
}

class CreaFabricas {
    public static Fabrica GetFabrica(string seleccion) {

        if (seleccion.Equals("FIGURA"))
            return new FabricaFiguras();

        if (seleccion.Equals("COLOR"))
            return new FabricaColores();

        return null;
    }
}

class Program {
    static void Main() {
        //Trae una determinada fábrica
        //en este caso de FIGURA
        Fabrica fig = CreaFabricas.GetFabrica("FIGURA");

        //Obtenida la fábrica, se solicita
        //un tipo de objeto de esa fábrica
        IFigura figural = fig.GetFigura("CIRCULO");

        //Llama un método de ese objeto
        //dado por la fábrica en particular
        figural.Dibujar();

        //Obtenida la fábrica, se solicita
        //un tipo de objeto de esa fábrica
        IFigura figura2 = fig.GetFigura("RECTANGULO");

        //Llama un método de ese objeto dado
        //por la fábrica en particular
        figura2.Dibujar();
    }
}

```

```

//Obtenida la fábrica, se solicita
//un tipo de objeto de esa fábrica
IFigura figura3 = fig.GetFigura("TRIANGULO");

//Llama un método de ese objeto dado
//por la fábrica en particular
figura3.Dibujar();

//Trae una determinada fábrica
//en este caso de COLOR
Fabrica color = CreaFabricas.GetFabrica("COLOR");

//Obtenida la fábrica, se solicita
//un tipo de objeto de esa fábrica
IColor color1 = color.GetColor("ROJO");

//Llama un método de ese objeto dado
//por la fábrica en particular
color1.Rellenar();

//Obtenida la fábrica, se solicita
//un tipo de objeto de esa fábrica
IColor color2 = color.GetColor("VERDE");

//Llama un método de ese objeto dado
//por la fábrica en particular
color2.Rellenar();

//Obtenida la fábrica, se solicita un
//tipo de objeto de esa fábrica
IColor color3 = color.GetColor("AZUL");

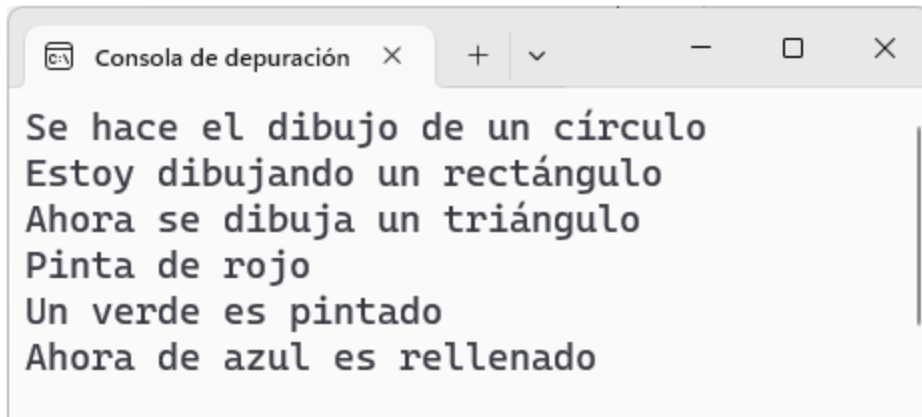
//Llama un método de ese objeto dado por
//la fábrica en particular
color3.Rellenar();
}

```

```

}

```



A screenshot of a debug console window titled "Consola de depuración". The window has a standard toolbar with icons for adding, collapsing, zooming, and closing. The console contains the following text:

```
Se hace el dibujo de un círculo  
Estoy dibujando un rectángulo  
Ahora se dibuja un triángulo  
Pinta de rojo  
Un verde es pintado  
Ahora de azul es relleno
```

*Ilustración 46: Abstract Factory*

## Singleton

El patrón de diseño Singleton es un patrón creacional que se utiliza para garantizar que una clase tenga exactamente una instancia y proporcionar un punto de acceso global a esta. En otras palabras, el patrón Singleton se utiliza para restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto.

D/042.cs

```
namespace Ejemplo;
//Patrón: Singleton

class ObjetoUnico {
    //Genera un objeto de ObjetoUnico
    private static ObjetoUnico instancia = new ObjetoUnico();

    //Hace el constructor privado por lo que
    //no puede ser instanciado
    private ObjetoUnico() { }

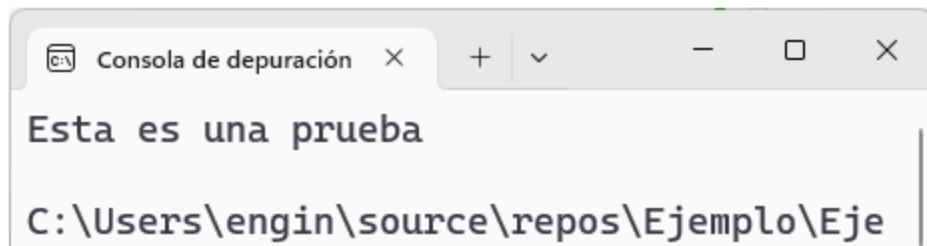
    //Retorna la única instancia de esta clase
    public static ObjetoUnico GetInstancia() {
        return instancia;
    }

    public void Mensaje() {
        Console.WriteLine("Esta es una prueba");
    }
}

class Program {
    static void Main() {
        //Quite el comentario de esta instrucción
        //y generará un error al compilar
        //ObjetoUnico pruebaObjeto = new ObjetoUnico();

        //Obtiene el único objeto instanciable
        ObjetoUnico miObjeto = ObjetoUnico.GetInstancia();

        //Muestra un mensaje
        miObjeto.Mensaje();
    }
}
```



*Ilustración 47: Singleton*

## Builder

El patrón de diseño Builder es un patrón creacional que se utiliza para crear objetos complejos paso a paso. Este patrón permite la creación de diferentes tipos y representaciones de un objeto utilizando el mismo proceso de construcción.

Cuando es necesario crear un objeto complejo que requiere una inicialización laboriosa, paso a paso, de muchos campos y objetos anidados: Si se intenta crear el objeto pasando argumentos a un constructor, puede que se termine con un constructor con muchos parámetros, muchos de los cuales no se usarían en la mayoría de los casos. El patrón Builder permite abstraer el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto, de tal forma que el mismo proceso de construcción pueda crear representaciones diferentes.

D/043.cs

```
namespace Ejemplo;
//Patrón: Builder

public interface IEmpacado {
    //Obligar a hacer el método Empaque()
    string Empaque();
}

class Envoltura : IEmpacado {
    public string Empaque() {
        return "Empaque Ecológico";
    }
}

public class Botella : IEmpacado {
    public string Empaque() {
        return "Botella biodegradable";
    }
}

//Todo producto en la comida tendrá estos
//ítems: Nombre, como se empaca, precio
public interface Item {
    string Nombre();
    IEmpacado Empacando();
    float Precio();
}

public abstract class Hamburguesa : Item {
    public IEmpacado Empacando() {
        return new Envoltura();
    }
    public abstract float Precio();
    public abstract string Nombre();
}
```



```

}

public class HamburguesaPollo : Hamburguesa {
    public override float Precio() {
        return 7000;
    }

    public override string Nombre() {
        return "Hamburguesa de pollo";
    }
}

class HamburguesaVegetariana : Hamburguesa {
    public override float Precio() {
        return 5000;
    }

    public override string Nombre() {
        return "Hamburguesa vegetariana";
    }
}

public abstract class BebidaFria : Item {
    public IEmpacado Empacando() {
        return new Botella();
    }

    public abstract float Precio();
    public abstract string Nombre();
}

class Malteada : BebidaFria {
    public override float Precio() {
        return 4700;
    }

    public override string Nombre() {
        return "Malteada";
    }
}

class CaFeFrio : BebidaFria {
    public override float Precio() {
        return 4000;
    }

    public override string Nombre() {
        return "Café frío";
    }
}

```

```

}

class Comida {
    private List<Item> items = new List<Item>();

    public void AddItem(Item item) {
        items.Add(item);
    }

    public float GetCosto() {
        float costo = 0.0f;
        foreach (Item item in items) {
            costo += item.Precio();
        }
        return costo;
    }

    public void MostrarItems() {
        foreach (Item item in items) {
            Console.Write("Item: " + item.Nombre());
            Console.Write(", Empaque: " + item.Empacando().Empaque());
            Console.WriteLine(", Precio: " + item.Precio());
        }
    }
}

//Prepara la comida dependiendo si es vegetariana o no
class FabricaComida {
    public Comida Vegetariano() {
        Comida miComida = new();
        miComida.AddItem(new HamburguesaVegetariana());
        miComida.AddItem(new CaFeFrio());
        return miComida;
    }

    public Comida NoVegetariano() {
        Comida miComida = new();
        miComida.AddItem(new HamburguesaPollo());
        miComida.AddItem(new Malteada());
        return miComida;
    }
}

class Program {
    static void Main() {
        FabricaComida miComida = new();

        Comida vegetariano = miComida.Vegetariano();
    }
}

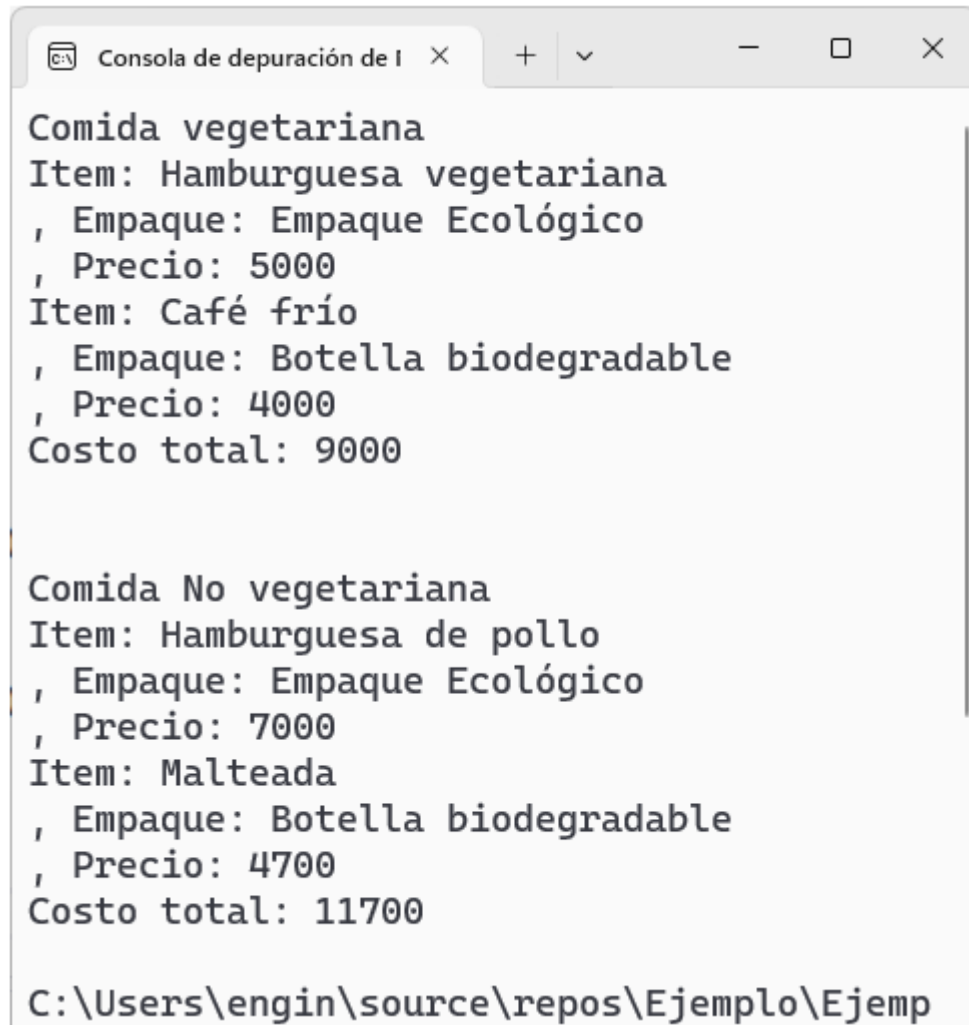
```

```

        Console.WriteLine("Comida vegetariana");
        vegetariano.MostrarItems();
        Console.WriteLine("Costo: " + vegetariano.GetCosto());

        Comida noVegetariano = miComida.NoVegetariano();
        Console.WriteLine("\n\nComida No vegetariana");
        noVegetariano.MostrarItems();
        Console.WriteLine("Costo: " + noVegetariano.GetCosto());
    }
}

```



```

Comida vegetariana
Item: Hamburguesa vegetariana
, Empaque: Empaque Ecológico
, Precio: 5000
Item: Café frío
, Empaque: Botella biodegradable
, Precio: 4000
Costo total: 9000

Comida No vegetariana
Item: Hamburguesa de pollo
, Empaque: Empaque Ecológico
, Precio: 7000
Item: Malteada
, Empaque: Botella biodegradable
, Precio: 4700
Costo total: 11700

C:\Users\engin\source\repos\Ejemplo\Ejemp

```

Ilustración 48: Builder

## Adapter

El patrón de diseño Adapter es un patrón estructural que se utiliza para adaptar una interfaz existente a otra interfaz. En otras palabras, el adaptador actúa como un intermediario entre dos interfaces incompatibles y proporciona una capa adicional de abstracción para permitir que los objetos trabajen juntos.

Se tienen dos clases con interfaces incompatibles, y es necesario que trabajen juntas. El patrón Adapter permite crear una clase intermedia que actúa como un traductor entre las dos interfaces. El adaptador implementa la interfaz del cliente y utiliza la interfaz del servicio para realizar la traducción.

D/044.cs

```
namespace Ejemplo;
//Patrón de diseño: Adapter

public interface IEjecutorMultimedia {
    void Ejecutar(string TipoAudio, string NombreArchivo);
}

public interface IEjecutorAvanzadoArchivosMultimedia {
    void EjecutaVLC(string NombreArchivo);
    void EjecutaMP4(string NombreArchivo);
}

class EjecutorVLC : IEjecutorAvanzadoArchivosMultimedia {
    public void EjecutaVLC(string NombreArchivo) {
        Console.WriteLine("Ejecutando VLC: " + NombreArchivo);
    }
    public void EjecutaMP4(string NombreArchivo) {
    }
}

class EjecutorMP4 : IEjecutorAvanzadoArchivosMultimedia {
    public void EjecutaVLC(string NombreArchivo) {
    }
    public void EjecutaMP4(string NombreArchivo) {
        Console.WriteLine("Ejecutando MP4: " + NombreArchivo);
    }
}

class AdaptadorMultimedia : IEjecutorMultimedia {
    IEjecutorAvanzadoArchivosMultimedia ejecutorAvanzado;

    //Constructor
    public AdaptadorMultimedia(string TipoAudio) {
        if (TipoAudio.Equals("vlc")) {
            ejecutorAvanzado = new EjecutorVLC();
        }
    }
}
```

```

    }
    if (TipoAudio.Equals("mp4")) {
        ejecutorAvanzado = new EjecutorMP4();
    }
}

//Dependiendo del tipo de audio llama a VLC o MP4
public void Ejecutar(string TipoAudio, string NombreArchivo) {
    if (TipoAudio.Equals("vlc")) {
        ejecutorAvanzado.EjecutaVLC(NombreArchivo);
    }
    else if (TipoAudio.Equals("mp4")) {
        ejecutorAvanzado.EjecutaMP4(NombreArchivo);
    }
}
}

class EjecutorAudio : IEjecutorMultimedia {

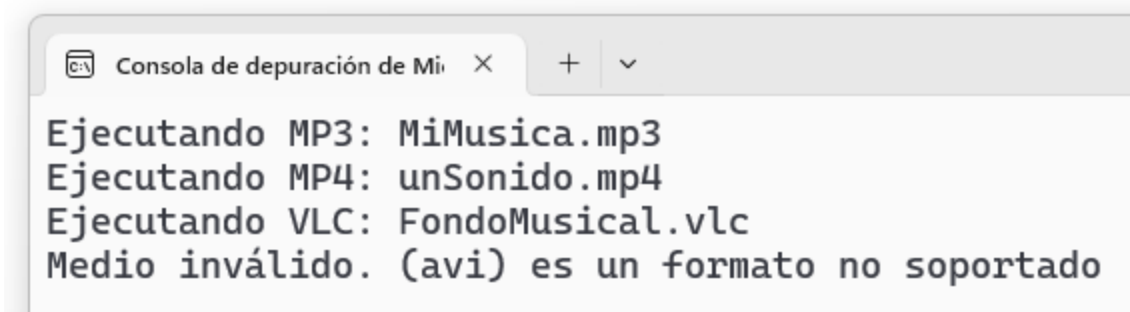
    AdaptadorMultimedia adaptadorMultimedia;

    public void Ejecutar(string TipoAudio, string NombreArchivo) {
        //Archivos MP3
        if (TipoAudio.Equals("mp3")) {
            Console.WriteLine("Ejecutando MP3: " + NombreArchivo);
        } //Otros formatos
        else if (TipoAudio.Equals("vlc") || TipoAudio.Equals("mp4")) {
            adaptadorMultimedia = new AdaptadorMultimedia(TipoAudio);
            adaptadorMultimedia.Ejecutar(TipoAudio, NombreArchivo);
        }
        else {
            Console.Write("Medio inválido. (" + TipoAudio);
            Console.WriteLine(") es un formato no soportado");
        }
    }
}

class Program {
    static void Main() {
        EjecutorAudio Multimedia = new EjecutorAudio();

        Multimedia.Ejecutar("mp3", "MiMusica.mp3");
        Multimedia.Ejecutar("mp4", "unSonido.mp4");
        Multimedia.Ejecutar("vlc", "FondoMusical.vlc");
        Multimedia.Ejecutar("avi", "unAudio.avi");
    }
}

```



*Ilustración 49: Adapter*

## Composite

El patrón de diseño Composite es un patrón estructural que se utiliza para representar jerarquías parte-todo como un árbol. Este patrón permite a los clientes tratar objetos individuales y composiciones de objetos de manera uniforme.

Se tiene una estructura jerárquica de objetos, donde cada objeto puede ser un objeto simple o un objeto compuesto: el patrón Composite permite tratar tanto los objetos simples como los objetos compuestos de la misma manera, como si fueran una instancia única de un objeto.

D/045.cs

```
namespace Ejemplo;
//Patrón de diseño: Composite

public class Empleado {
    private string nombre;
    private string departamento;
    private int salario;
    private List<Empleado> subordinados;

    //Constructor
    public Empleado(string nombre, string departamento, int salario) {
        this.nombre = nombre;
        this.departamento = departamento;
        this.salario = salario;
        subordinados = new List<Empleado>();
    }

    public void Adicionar(Empleado objEmpleado) {
        subordinados.Add(objEmpleado);
    }

    public void Quitar(Empleado objEmpleado) {
        subordinados.Remove(objEmpleado);
    }

    public List<Empleado> GetSubordinados() {
        return subordinados;
    }

    public new string ToString() {
        string Cad = "Empleado => Nombre: " + nombre;
        Cad += ", departamento: " + departamento;
        Cad += ", salario: " + salario;
        return Cad;
    }
}
```

```

internal class Program {
    static void Main() {
        Empleado Gerente = new("Laura", "Gerente", 5000);
        Empleado jefeVentas = new("Patricia", "Ventas", 3000);
        Empleado jefeMercadeo = new("Adriana", "Mercadeo", 3000);
        Empleado disenador1 = new("Sandra", "Marketing", 2000);
        Empleado disenador2 = new("Alejandra", "Marketing", 2000);
        Empleado vendedor1 = new("Francisca", "Ventas", 2000);
        Empleado vendedor2 = new("Flor", "Ventas", 2000);

        Gerente.Adicionar(jefeVentas);
        Gerente.Adicionar(jefeMercadeo);

        jefeVentas.Adicionar(vendedor1);
        jefeVentas.Adicionar(vendedor2);

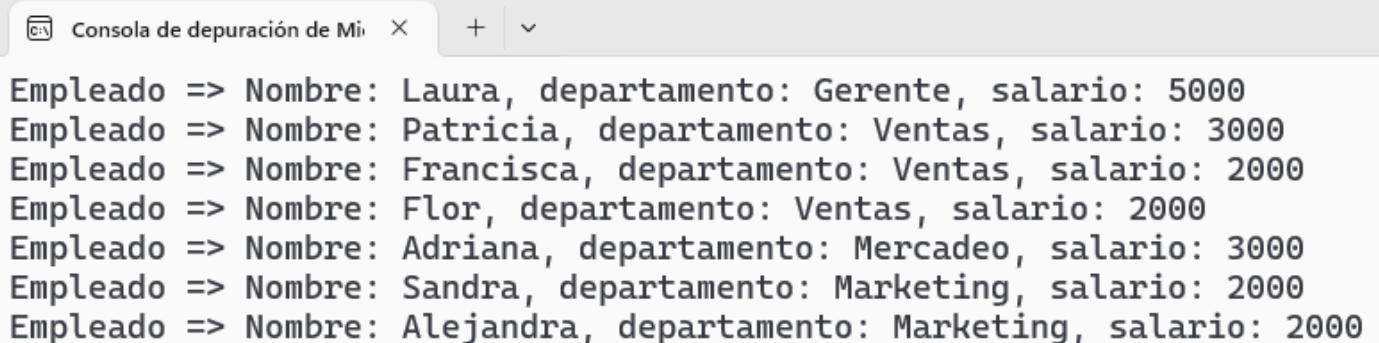
        jefeMercadeo.Adicionar(disenador1);
        jefeMercadeo.Adicionar(disenador2);

        //Imprime todos los empleados de la organización
        Console.WriteLine(Gerente.ToString());

        foreach (Empleado jefe in Gerente.GetSubordinados()) {
            Console.WriteLine(jefe.ToString());

            foreach (Empleado empleado in jefe.GetSubordinados()) {
                Console.WriteLine(empleado.ToString());
            }
        }
    }
}

```



```

Empleado => Nombre: Laura, departamento: Gerente, salario: 5000
Empleado => Nombre: Patricia, departamento: Ventas, salario: 3000
Empleado => Nombre: Francisca, departamento: Ventas, salario: 2000
Empleado => Nombre: Flor, departamento: Ventas, salario: 2000
Empleado => Nombre: Adriana, departamento: Mercadeo, salario: 3000
Empleado => Nombre: Sandra, departamento: Marketing, salario: 2000
Empleado => Nombre: Alejandra, departamento: Marketing, salario: 2000

```

Ilustración 50: Composite



## Facade

El patrón de diseño Facade es un patrón estructural que se utiliza para simplificar la complejidad de un sistema. Imaginarse tener un sistema complejo con muchos subsistemas, cada uno con su propia interfaz. Si un cliente quiere interactuar con el sistema, tendría que conocer todas las interfaces de los subsistemas, lo que puede ser muy complicado. El patrón Facade proporciona una interfaz unificada para un conjunto de interfaces en un subsistema. La idea detrás de este patrón es proporcionar una interfaz simple para un subsistema complejo, reduciendo así la complejidad del sistema y minimizando las comunicaciones y dependencias entre los subsistemas.

D/046.cs

```
namespace Ejemplo;
//Patrón de diseño: Facade

interface IFigura {
    void Dibujar();
}

class Circulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Dibujando un círculo");
    }
}

class Rectangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Traza un rectángulo");
    }
}

class Triangulo : IFigura {
    public void Dibujar() {
        Console.WriteLine("Delinea un triángulo");
    }
}

class HacerFigura {
    private IFigura circulo;
    private IFigura rectangulo;
    private IFigura triangulo;

    public HacerFigura() {
        circulo = new Circulo();
        rectangulo = new Rectangulo();
        triangulo = new Triangulo();
    }
}
```

```

    public void DibujaCirculo() {
        circulo.Dibujar();
    }

    public void DibujaRectangulo() {
        rectangulo.Dibujar();
    }

    public void DibujaTriangulo() {
        triangulo.Dibujar();
    }
}

class Program {
    static void Main() {
        HacerFigura hacefigura = new();

        hacefigura.DibujaCirculo();
        hacefigura.DibujaRectangulo();
        hacefigura.DibujaTriangulo();
    }
}

```

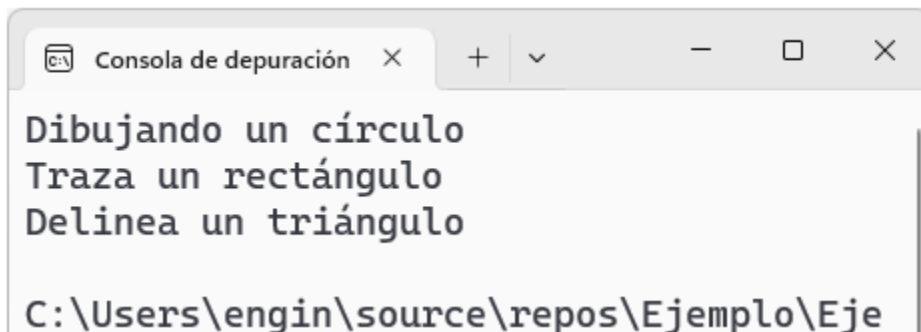


Ilustración 51: Facade

## Modelo Vista Controlador

El patrón de diseño modelo vista controlador (MVC) es una forma de organizar una aplicación en tres componentes principales: el modelo, la vista y el controlador. Cada componente tiene una función específica y se comunica con los otros para lograr una interacción fluida entre el usuario y la aplicación. Una breve explicación de cada componente:

**Modelo:** Es el componente que gestiona los datos y la lógica de la aplicación. Se encarga de almacenar, manipular y validar los datos, así como de interactuar con fuentes externas como bases de datos o APIs. El modelo no tiene conocimiento de la interfaz de usuario, solo se ocupa de los datos.

**Vista:** Es el componente que muestra los datos al usuario. Se encarga de generar la interfaz gráfica de usuario (GUI). La vista recibe los datos del modelo a través del controlador y los presenta de forma atractiva y comprensible. La vista también puede capturar las acciones del usuario, como hacer clic en un botón o introducir un texto, y enviarlas al controlador.

**Controlador:** Es el componente que coordina la comunicación entre el modelo y la vista. Se encarga de procesar las peticiones del usuario, como solicitar una página o enviar un formulario, y de invocar al modelo para obtener o modificar los datos necesarios. El controlador también decide qué vista mostrar al usuario según el resultado del modelo.

D/047.cs

```
namespace Ejemplo;
//Patrón de diseño: Modelo Vista Controlador
class Gente {
    public string Codigo { get; set; }
    public string Nombre { get; set; }
}

class ControladorGente {
    private Gente modelo;
    private VisorGente vista;

    public ControladorGente(Gente modelo, VisorGente vista) {
        this.modelo = modelo;
        this.vista = vista;
    }

    public void setNombreGente(string nombre) {
        modelo.Nombre = nombre;
    }

    public string getNombreGente() {
        return modelo.Nombre;
    }

    public void setCodigoGente(string codigo) {
```

```

        modelo.Codigo = codigo;
    }

    public string getCodigoGente() {
        return modelo.Codigo;
    }

    public void ActualizarVista() {
        vista.ImprimeGente(modelo.Nombre, modelo.Codigo);
    }
}

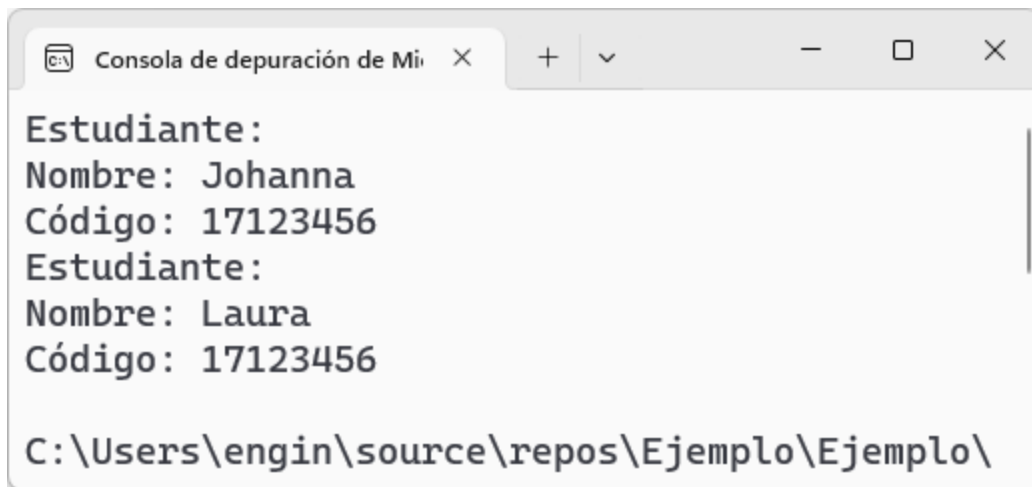
class VisorGente {
    public void ImprimeGente(string Nombre, string Codigo) {
        Console.WriteLine("Gente: ");
        Console.WriteLine("Nombre: " + Nombre);
        Console.WriteLine("Código: " + Codigo);
    }
}

class Program {
    static void Main() {
        Gente modelo = TraeGenteBaseDatos();
        VisorGente vista = new();
        ControladorGente control = new(modelo, vista);

        control.ActualizarVista();
        control.setNombreGente("Laura");
        control.ActualizarVista();
    }

    private static Gente TraeGenteBaseDatos() {
        Gente Gente = new();
        Gente.Nombre = "Johanna";
        Gente.Codigo = "17123456";
        return Gente;
    }
}

```



*Ilustración 52: Modelo Vista Controlador*

# SOLID

SOLID es un conjunto de principios diseñados para mejorar el diseño y la mantenibilidad del código en la Programación Orientada a Objetos (POO). Estos son:

**S:** Single Responsibility Principle (Principio de Responsabilidad Única) Cada clase debe tener una única responsabilidad, es decir, debe realizar una sola tarea o propósito.

**O:** Open/Closed Principle (Principio Abierto/Cerrado) Las clases deben estar abiertas para ser extendidas, pero cerradas para modificaciones. Esto significa que puedes agregar nuevas funcionalidades sin cambiar el código existente.

**L:** Liskov Substitution Principle (Principio de Sustitución de Liskov) Los objetos de una clase derivada deben poder sustituir a los objetos de la clase base sin alterar el funcionamiento del programa.

**I:** Interface Segregation Principle (Principio de Segregación de Interfaces) Las interfaces deben ser específicas y pequeñas, de modo que las clases que implementan esas interfaces no se vean obligadas a usar métodos que no necesitan.

**D:** Dependency Inversion Principle (Principio de Inversión de Dependencias) Las clases deben depender de abstracciones y no de implementaciones concretas, favoreciendo el uso de interfaces o clases abstractas.

Ejemplo:

D/048.cs

```
namespace Ejemplo;
// S: Single Responsibility Principle (SRP)
// sólo se encarga de los datos del pedido.
public class Pedido {
    public int Codigo { get; set; }
    public decimal TotalCosto { get; set; }

    public Pedido(int Codigo, decimal TotalCosto) {
        this.Codigo = Codigo;
        this.TotalCosto = TotalCosto;
    }
}
```

```

// Clase con la responsabilidad de persistir los pedidos
// se encarga de guardar los pedidos, separando la responsabilidad
public class PersistePedidos {
    public void Guardar(Pedido objPedido) {
        Console.WriteLine("Pedido: " + objPedido.Codigo + " guardado con un
total de: " + objPedido.TotalCosto);
    }
}

// O: Open/Closed Principle (OCP)
//La interfaz `IDescuento` permite extender el sistema con nuevos tipos de
descuento
//(`SinDescuento`, `DescuentoTemporada`) sin modificar las clases
existentes.
public interface IDescuento {
    decimal AplicaDescuento(decimal TotalCosto);
}

public class SinDescuento : IDescuento {
    public decimal AplicaDescuento(decimal TotalCosto) {
        return TotalCosto;
    }
}

public class DescuentoTemporada : IDescuento {
    public decimal AplicaDescuento(decimal TotalCosto) {
        return TotalCosto * 0.9m; // 10% de descuento
    }
}

// L: Liskov Substitution Principle (LSP)
//La clase abstracta `Notificacion` asegura que sus subclases
//(`EmailNotificacion`, `SmsNotificacion`) sean intercambiables sin
afectar el comportamiento.
public abstract class Notificacion {
    public abstract void Notificar(string Mensaje);
}

public class EmailNotificacion : Notificacion {
    public override void Notificar(string Mensaje) {
        Console.WriteLine("Correo enviado: " + Mensaje);
    }
}

public class SmsNotificacion : Notificacion {
    public override void Notificar(string Mensaje) {
        Console.WriteLine("SMS enviado: " + Mensaje);
    }
}

```

```

}

// I: Interface Segregation Principle (ISP)
// Las interfaces separadas, como `IProcesoPago`, aseguran que
// las clases solo implementen métodos que necesitan
// (`TarjetaCredito`, `PorPayPal`)
public interface IProcesoPago {
    void Pago(decimal Precio);
}

public class TarjetaCredito : IProcesoPago {
    public void Pago(decimal Precio) {
        Console.WriteLine("Pago con tarjeta crédito de: " + Precio + " fue
procesado.");
    }
}

public class PorPayPal : IProcesoPago {
    public void Pago(decimal Precio) {
        Console.WriteLine("Pago con PayPal de: " + Precio + " fue
procesado.");
    }
}

// D: Dependency Inversion Principle (DIP)
// La clase `ServicioPedido` no depende de implementaciones concretas,
// sino de abstracciones (`IDescuento`, `Notificacion`, `IProcesoPago`),
// lo que mejora la flexibilidad y el testeo
public class ServicioPedido {
    private readonly IDescuento _descuento;
    private readonly Notificacion _notificacion;
    private readonly PersistePedidos _repositorio;
    private readonly IProcesoPago _formadePago;

    public ServicioPedido(IDescuento descuento, Notificacion notificacion,
PersistePedidos repositorio, IProcesoPago formadePago) {
        _descuento = descuento;
        _notificacion = notificacion;
        _repositorio = repositorio;
        _formadePago = formadePago;
    }

    public void ProcesarPedido(Pedido objPedido) {
        // Aplicar descuento
        objPedido.TotalCosto =
_descuento.AplicaDescuento(objPedido.TotalCosto);

        // Procesar pago
    }
}

```



```

        _formadePago.Pago(objPedido.TotalCosto);

        // Guardar pedido
        _repositorio.Guardar(objPedido);

        // Enviar notificación
        _notificacion.Notificar("Pedido " + objPedido.Codigo + " procesado
exitósamente con un total de: " + objPedido.TotalCosto);
    }
}

// Uso
class Program {
    static void Main() {
        var objPedido = new Pedido(1, 100m);

        // Inyectar dependencias
        var Descuento = new DescuentoTemporada();
        var Notificacion = new EmailNotificacion();
        var Persistencia = new PersistePedidos();
        var MododePago = new TarjetaCredito();

        var objServicioPedido = new ServicioPedido(Descuento, Notificacion,
Persistencia, MododePago);

        // Procesar pedido
        objServicioPedido.ProcesarPedido(objPedido);
    }
}

```

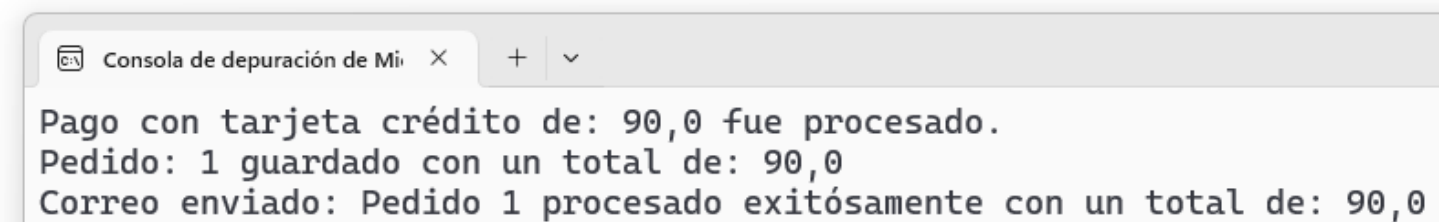


Ilustración 53: Uso de SOLID

# Arreglos de objetos

Es posible tener un conjunto de objetos de la misma clase.

D/049.cs

```
namespace Ejemplo;

class Valores {
    private int Entero;
    private double ValorReal;
    private bool Booleano;
    private char Caracter;
    private string Cadena;

    public Valores(int entero, double valorReal, bool booleano, char
caracter, string cadena) {
        Entero = entero;
        ValorReal = valorReal;
        Booleano = booleano;
        Caracter = caracter;
        Cadena = cadena;
    }

    public void Imprime() {
        Console.WriteLine(Entero + ";" + ValorReal + ";" + Booleano + ";" +
Caracter + ";" + Cadena);
    }
}

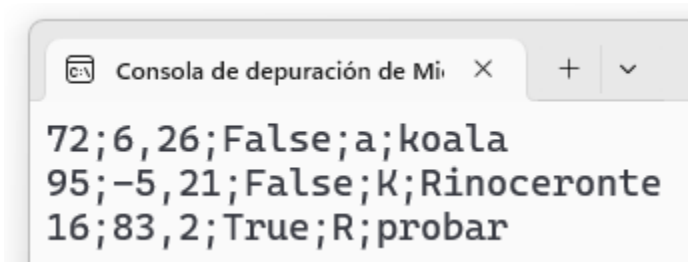
internal class Program {
    static void Main(string[] args) {
        //Arreglo unidimensional de objetos
        Valores[] objetos = new Valores[10];

        //Se crean los objetos
        objetos[0] = new Valores(16, 83.2, true, 'R', "probar");
        objetos[1] = new Valores(72, 6.26, false, 'a', "koala");
        objetos[2] = new Valores(95, -5.21, false, 'K', "Rinoceronte");

        //Se imprimen los objetos
        objetos[1].Imprime();
        objetos[2].Imprime();
        objetos[0].Imprime();

        //¿Que pasaría aquí? Mensaje de error: el objeto no ha sido creado
        //objetos[4].Imprime();
    }
}
```

}



*Ilustración 54: Arreglo de objetos*

## Llenado con ciclos

Creando conjunto de objetos usando ciclos

D/050.cs

```
namespace Ejemplo;

class Valores {
    private int Entero;
    private double ValorReal;

    public Valores(int entero, double valorReal) {
        Entero = entero;
        ValorReal = valorReal;
    }

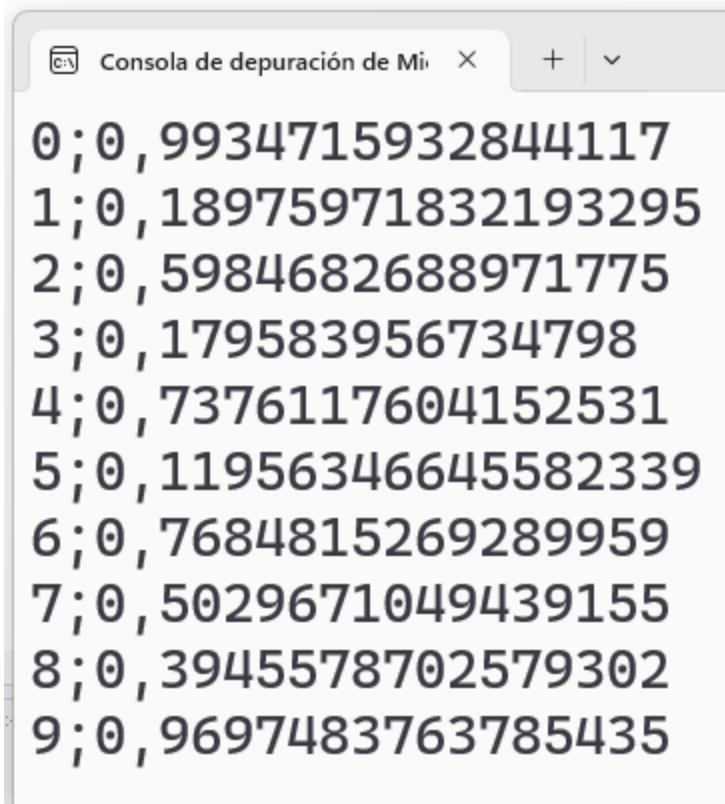
    public void Imprime() {
        Console.WriteLine(Entero + ";" + ValorReal);
    }
}

internal class Program {
    static void Main(string[] args) {
        //Arreglo unidimensional de objetos
        Valores[] objetos = new Valores[10];

        Random Azar = new();

        //Se crean los objetos
        for (int Cont = 0; Cont < objetos.Length; Cont++)
            objetos[Cont] = new Valores(Cont, Azar.NextDouble());

        //Se imprimen los objetos
        for (int Cont = 0; Cont < objetos.Length; Cont++)
            objetos[Cont].Imprime();
    }
}
```



```
0;0,9934715932844117
1;0,18975971832193295
2;0,5984682688971775
3;0,179583956734798
4;0,7376117604152531
5;0,11956346645582339
6;0,7684815269289959
7;0,5029671049439155
8;0,3945578702579302
9;0,9697483763785435
```

*Ilustración 55: Arreglo de objetos*

## Ordenando

Ordenando los objetos dentro de un arreglo dependiendo del valor de algún atributo.

D/051.cs

```
namespace Ejemplo;

class Valores {
    public int Entero;
    public double ValorReal;

    public Valores(int entero, double valorReal) {
        Entero = entero;
        ValorReal = valorReal;
    }

    public void Imprime() {
        Console.WriteLine(Entero + ";" + ValorReal);
    }
}

internal class Program {
    static void Main() {
        //Arreglo unidimensional de objetos
        Valores[] objetos = new Valores[20];

        Random Azar = new();

        //Se crean los objetos
        for (int Cont = 0; Cont < objetos.Length; Cont++)
            objetos[Cont] = new Valores(Azar.Next(100), Azar.NextDouble());

        //Imprime los objetos
        Console.WriteLine("Arreglo de objetos original");
        for (int Cont = 0; Cont < objetos.Length; Cont++)
            objetos[Cont].Imprime();

        //Se procede a ordenar usando ShellSort
        Shell(objetos);

        //Imprime los objetos
        Console.WriteLine("\r\nArreglo de objetos ordenado por atributo
entero");
        for (int Cont = 0; Cont < objetos.Length; Cont++)
            objetos[Cont].Imprime();
    }
}
```

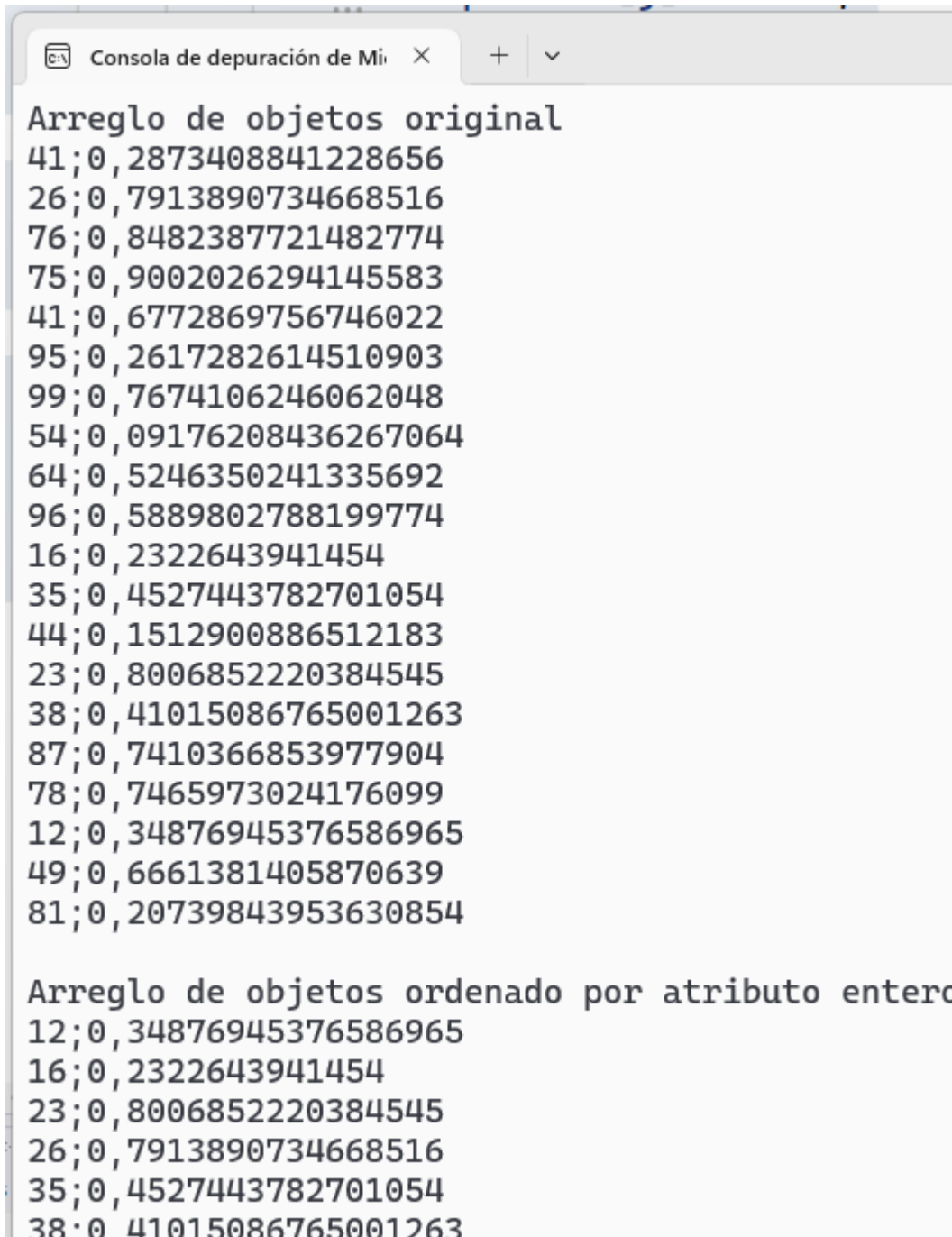
```

//Ordenamiento por Shell
static void Shell(Valores[] arr) {
    int incr = arr.Length;
    do {
        incr /= 2;
        for (int k = 0; k < incr; k++) {
            for (int i = incr + k; i < arr.Length; i += incr) {
                int j = i;
                while (j - incr >= 0 && arr[j].Entero < arr[j - incr].Entero)
                {
                    int tmp = arr[j].Entero;
                    arr[j].Entero = arr[j - incr].Entero;
                    arr[j - incr].Entero = tmp;

                    double tmp2 = arr[j].ValorReal;
                    arr[j].ValorReal = arr[j - incr].ValorReal;
                    arr[j - incr].ValorReal = tmp2;

                    j -= incr;
                }
            }
        }
    } while (incr > 1);
}

```



```
Consola de depuración de Mi X + v

Arreglo de objetos original
41;0,2873408841228656
26;0,7913890734668516
76;0,8482387721482774
75;0,9002026294145583
41;0,6772869756746022
95;0,2617282614510903
99;0,7674106246062048
54;0,09176208436267064
64;0,5246350241335692
96;0,5889802788199774
16;0,2322643941454
35;0,4527443782701054
44;0,1512900886512183
23;0,8006852220384545
38;0,41015086765001263
87;0,7410366853977904
78;0,7465973024176099
12;0,34876945376586965
49;0,6661381405870639
81;0,20739843953630854

Arreglo de objetos ordenado por atributo entero
12;0,34876945376586965
16;0,2322643941454
23;0,8006852220384545
26;0,7913890734668516
35;0,4527443782701054
38;0,41015086765001263
```

Ilustración 56: Ordenando arreglo de objetos



# Delegados

Usados para mejorar la reutilización de código.

D/052.cs

```
namespace Ejemplo;

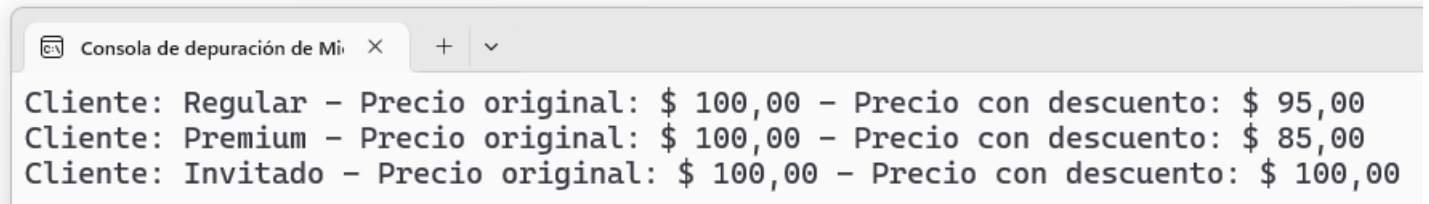
//Inicia la aplicación aquí
internal class Program {
    // 1. Se declara el delegado
    public delegate double EstrategiaDescuento(double precioOriginal);

    // 2. Distintas estrategias de descuento. Obsérvese que debe
    //coincidir el tipo de dato devuelto, el número de parámetros
    //y el tipo.
    public static double DescuentoRegular(double precio) {
        return precio * 0.95; // 5%
    }
    public static double DescuentoPremium(double precio) {
        return precio * 0.85; // 15%
    }
    public static double SinDescuento(double precio) {
        return precio; // 0%
    }

    // 3. Método que aplica el descuento usando el delegado
    public static void AplicarDescuento(string tipoCliente, double precio,
        EstrategiaDescuento descuento) {
        double precioFinal = descuento(precio);
        Console.WriteLine($"Cliente: {tipoCliente} - Precio original:
{precio:C} - Precio con descuento: {precioFinal:C}");
    }

    static void Main() {
        double precioProducto = 100;

        // 4. Se usa el delegado para aplicar distintas estrategias
        AplicarDescuento("Regular", precioProducto, DescuentoRegular);
        AplicarDescuento("Premium", precioProducto, DescuentoPremium);
        AplicarDescuento("Invitado", precioProducto, SinDescuento);
    }
}
```



The image shows a browser's developer console with a single log entry. The log entry contains three lines of text, each representing a different customer type and their corresponding original and discounted prices. The text is as follows:

```
Cliente: Regular - Precio original: $ 100,00 - Precio con descuento: $ 95,00
Cliente: Premium - Precio original: $ 100,00 - Precio con descuento: $ 85,00
Cliente: Invitado - Precio original: $ 100,00 - Precio con descuento: $ 100,00
```

*Ilustración 57: Uso de delegados*

```
namespace Ejemplo;

class Program {
    // 1. Se declara el delegado
    public delegate double Operacion(double valor);

    // 2. Métodos compatibles con el delegado
    public static double Doble(double x) => x * 2;
    public static double Cuadrado(double x) => x * x;
    public static double Raiz(double x) => Math.Sqrt(x);

    // 3. Método que recibe el delegado y aplica la operación
    public static void EjecutarOperacion(double numero, Operacion op) {
        double resultado = op(numero);
        Console.WriteLine($"Resultado: {resultado}");
    }

    static void Main() {
        Console.WriteLine("Ingrese un número:");
        double numero = Convert.ToDouble(Console.ReadLine());

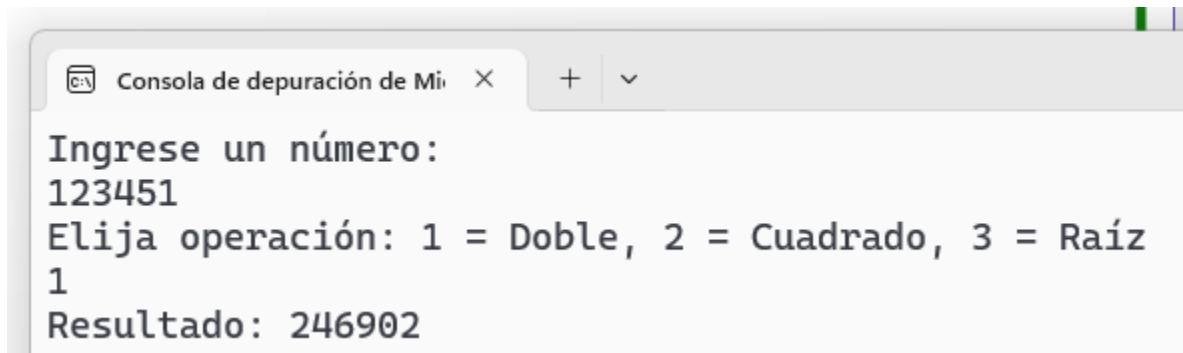
        Console.WriteLine("Operación: 1 = Doble, 2 = Cuadrado, 3 = Raíz");
        string opcion = Console.ReadLine();

        Operacion operacionElegida;

        // 4. Se asigna el método al delegado según la opción
        switch (opcion) {
            case "1":
                operacionElegida = Doble;
                break;
            case "2":
                operacionElegida = Cuadrado;
                break;
            case "3":
                operacionElegida = Raiz;
                break;
            default:
                Console.WriteLine("Opción inválida.");
                return;
        }

        // 5. Ejecuta la operación
        EjecutarOperacion(numero, operacionElegida);
    }
}
```

}



*Ilustración 58: Uso de delegados*

# Almacenando objetos en medio persistente

## Uso de JSON

Se tiene un objeto y se quiere almacenar en un medio persistente. Una forma de hacerlo es usando "serialización" y se guarda como una cadena JSON. La clase se marca con la etiqueta: [Serializable]

D/054.cs

```
using System.Text.Json;

namespace Ejemplo;

[Serializable]
public class Persona {
    public string Nombre { get; set; }
    public int Telefono { get; set; }
}

internal class Program {
    static void Main(string[] args) {
        var persona = new Persona { Nombre = "Rafael", Telefono = 3156789 };

        // Guardar
        var json = JsonSerializer.Serialize(persona);
        File.WriteAllText("persona.json", json);

        // Leer
        var jsonLeido = File.ReadAllText("persona.json");
        var personaLeida = JsonSerializer.Deserialize<Persona>(jsonLeido);

        //Imprime
        Console.WriteLine("Nombre: " + personaLeida.Nombre);
        Console.WriteLine("Teléfono: " + personaLeida.Telefono);
    }
}
```

```
using System.Xml.Serialization;

namespace Ejemplo;

[Serializable]
public class Persona {
    public string Nombre { get; set; }
    public int Telefono { get; set; }
}

internal class Program {
    static void Main(string[] args) {
        var persona = new Persona { Nombre = "Rafael", Telefono = 3124789 };

        var serializer = new XmlSerializer(typeof(Persona));
        using (var writer = new StreamWriter("persona.xml")) {
            serializer.Serialize(writer, persona);
        }

        // Leer
        using (var reader = new StreamReader("persona.xml")) {
            var personaLeida = (Persona)serializer.Deserialize(reader);

            //Imprime
            Console.WriteLine("Nombre: " + personaLeida.Nombre);
            Console.WriteLine("Teléfono: " + personaLeida.Telefono);
        }
    }
}
```

# Genéricos

Los genéricos en C# son una característica del lenguaje que permite definir clases, interfaces, métodos y delegados con tipos parametrizados, es decir, sin especificar el tipo exacto hasta que se usa.

D/056.cs

```
namespace Ejemplo;

public class Caja<T> {
    public T Contenido { get; set; }
}

internal class Program {
    static void Main(string[] args) {
        var cajaInt = new Caja<int> { Contenido = 42 };
        var cajaTexto = new Caja<string> { Contenido = "Hola Rafael" };

        Console.WriteLine("Entero:" + cajaInt.Contenido);
        Console.WriteLine("Texto: " + cajaTexto.Contenido);
    }
}
```

```
namespace Ejemplo;

//Compara dos valores si son iguales
public static class Utilidades {
    public static bool SonIguales<T>(T a, T b) {
        return EqualityComparer<T>.Default.Equals(a, b);
    }
}

internal class Program {
    static void Main(string[] args) {
        bool resultado1 = Utilidades.SonIguales(5, 5);           // true
        bool resultado2 = Utilidades.SonIguales("hola", "adiós"); // false
        bool resultado3 = Utilidades.SonIguales(3.14, 3.14);     // true

        Console.WriteLine(resultado1);
        Console.WriteLine(resultado2);
        Console.WriteLine(resultado3);
    }
}
```

### ¿Por qué es útil?

Funciona con cualquier tipo: int, string, double, DateTime, incluso tipos personalizados.

Usa `EqualityComparer<T>.Default`, que respeta la implementación de `Equals()` o `IEquatable<T>` si está definida.

Evita duplicar lógica para cada tipo.

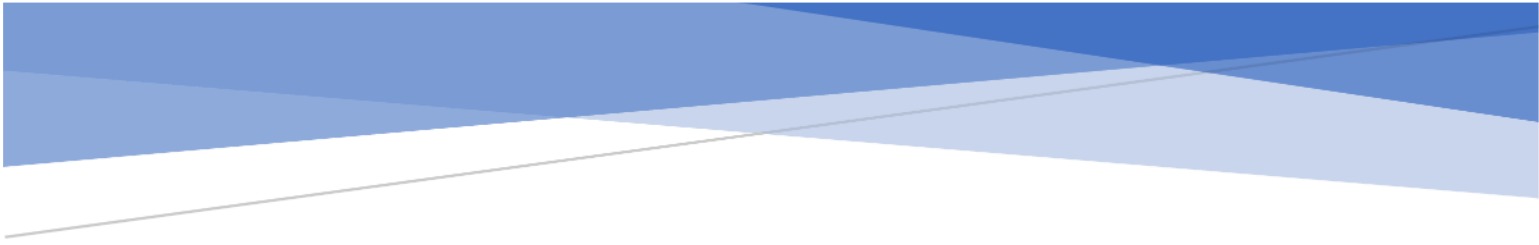


```
namespace Ejemplo
{
    //Compara dos valores si son iguales
    public static class Utilidades {
        public static void Intercambiar<T>(ref T a, ref T b) {
            T temp = a;
            a = b;
            b = temp;
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            int x = 10, y = 20;
            Utilidades.Intercambiar(ref x, ref y);
            // x = 20, y = 10

            string s1 = "uno", s2 = "dos";
            Utilidades.Intercambiar(ref s1, ref s2);
            // s1 = "dos", s2 = "uno"

            Console.WriteLine("x = " + x + " y = " + y);
            Console.WriteLine("s1 = " + s1 + " s2 = " + s2);
        }
    }
}
```



# C# Y .NET 10

## Parte 5. Estructuras de datos dinámicas

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

# Contenido

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                                       | 4  |
| Acerca del autor.....                                                             | 5  |
| Licencia de este libro .....                                                      | 5  |
| Licencia del software .....                                                       | 5  |
| Marcas registradas .....                                                          | 6  |
| Dedicatoria .....                                                                 | 7  |
| Introducción.....                                                                 | 8  |
| List.....                                                                         | 9  |
| Adicionar, tamaño, buscar e imprimir.....                                         | 9  |
| Borrar elemento .....                                                             | 11 |
| Cambiar Elemento .....                                                            | 13 |
| Insertar Elemento.....                                                            | 15 |
| SubLista .....                                                                    | 17 |
| Tres formas de recorrer un List .....                                             | 19 |
| Borrar completamente un List.....                                                 | 21 |
| Borrar un rango en un List .....                                                  | 22 |
| Guardar el List en un arreglo estático .....                                      | 24 |
| Agregar un arreglo estático a un List .....                                       | 25 |
| Inserta un arreglo estático en una determinada posición del List .....            | 26 |
| Insertar varios List dentro de un List .....                                      | 27 |
| Invertir un List .....                                                            | 28 |
| Invertir un rango de datos en el List .....                                       | 29 |
| Ordena un List .....                                                              | 30 |
| Búsqueda Binaria .....                                                            | 31 |
| Capacidad del List .....                                                          | 32 |
| Métricas: Comparativa de desempeño de List vs Arreglo estático vs ArrayList ..... | 34 |
| Lista de objetos .....                                                            | 40 |
| Listas en Listas .....                                                            | 43 |
| Dictionary .....                                                                  | 55 |
| Uso de llaves .....                                                               | 55 |
| Llaves tipo string .....                                                          | 57 |
| Manejo de objetos en un Dictionary .....                                          | 59 |
| Queue (Cola).....                                                                 | 61 |

|                            |    |
|----------------------------|----|
| Objetos en la cola.....    | 63 |
| Stack (Pila) .....         | 65 |
| Objetos en la pila .....   | 67 |
| SortedList .....           | 69 |
| LinkedList .....           | 71 |
| Objetos en LinkedList..... | 74 |
| SortedList .....           | 76 |
| PriorityQueue .....        | 77 |
| SortedList .....           | 78 |
| Persistencia.....          | 79 |
| Usando JSON.....           | 81 |
| Guardando objetos .....    | 82 |

## Tabla de ilustraciones

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Ilustración 1: List Adicionar, tamaño, buscar e imprimir .....                | 10 |
| Ilustración 2: List, borrar elemento .....                                    | 12 |
| Ilustración 3: List, cambiar elemento.....                                    | 14 |
| Ilustración 4: List, insertar elemento .....                                  | 16 |
| Ilustración 5: List, referenciar con una variable, un rango de la lista ..... | 18 |
| Ilustración 6: List, tres formas de recorrerlo .....                          | 20 |
| Ilustración 7: Borrar completamente un List .....                             | 21 |
| Ilustración 8: Borrar un rango en un List.....                                | 23 |
| Ilustración 9: Capacidad del List .....                                       | 33 |
| Ilustración 10: Comparativa.....                                              | 38 |
| Ilustración 11: Lista de objetos .....                                        | 42 |
| Ilustración 12: Uso de listas para simular un sistema de información .....    | 53 |
| Ilustración 13: Uso de listas para simular un sistema de información .....    | 54 |
| Ilustración 14: Dictionary .....                                              | 56 |
| Ilustración 15: Dictionary .....                                              | 58 |
| Ilustración 16: Dictionary, manejo de objetos .....                           | 60 |
| Ilustración 17: Dato definido en la cola .....                                | 62 |
| Ilustración 18: Objetos en la cola.....                                       | 64 |
| Ilustración 19: Dato definido en la pila.....                                 | 66 |
| Ilustración 20: Objetos en la pila .....                                      | 68 |
| Ilustración 21: SortedList .....                                              | 70 |
| Ilustración 22: LinkedList .....                                              | 73 |
| Ilustración 23: Objetos en LinkedList.....                                    | 75 |
| Ilustración 24: SortedSet .....                                               | 76 |
| Ilustración 25: Cola con prioridad .....                                      | 77 |
| Ilustración 26: SortedDictionary .....                                        | 78 |
| Ilustración 27: Guardando objetos .....                                       | 83 |
| Ilustración 28: Datos JSON .....                                              | 84 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

En recuerdo de Michu





# Introducción

En este libro se estudiarán las estructuras de datos implementadas en .NET 10 y su uso en C#. Algunas estructuras no son documentadas como ArrayList o Hashtable porque ya han sido reemplazadas por mejores como List o Dictionary.

# List

## Adicionar, tamaño, buscar e imprimir

En C# está el List, es un contenedor de objetos de un tipo definido

E/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Tamaño la lista
        int tamano = ListaAnimales.Count;
        Console.WriteLine("Tamaño de la lista: " + tamano);

        //Traer un determinado elemento de la lista
        int posicion = 5;
        string texto = ListaAnimales[posicion].ToString();
        Console.WriteLine("\r\nEn posición: " + posicion + " es: " + texto);

        //Nos dice si existe un determinado elemento en la lista
        string buscar = "Pulpo";
        bool Existe = ListaAnimales.Contains(buscar);
        Console.WriteLine("\r\nBusca: " + buscar + " Resultado: " + Existe);

        //Nos dice la posición donde encontró el elemento en la lista
        int posBusca = ListaAnimales.IndexOf(buscar);
        Console.WriteLine("\r\nBusca: " + buscar + " Posición: " + posBusca);

        //Agrega elementos a la lista
    }
}
```

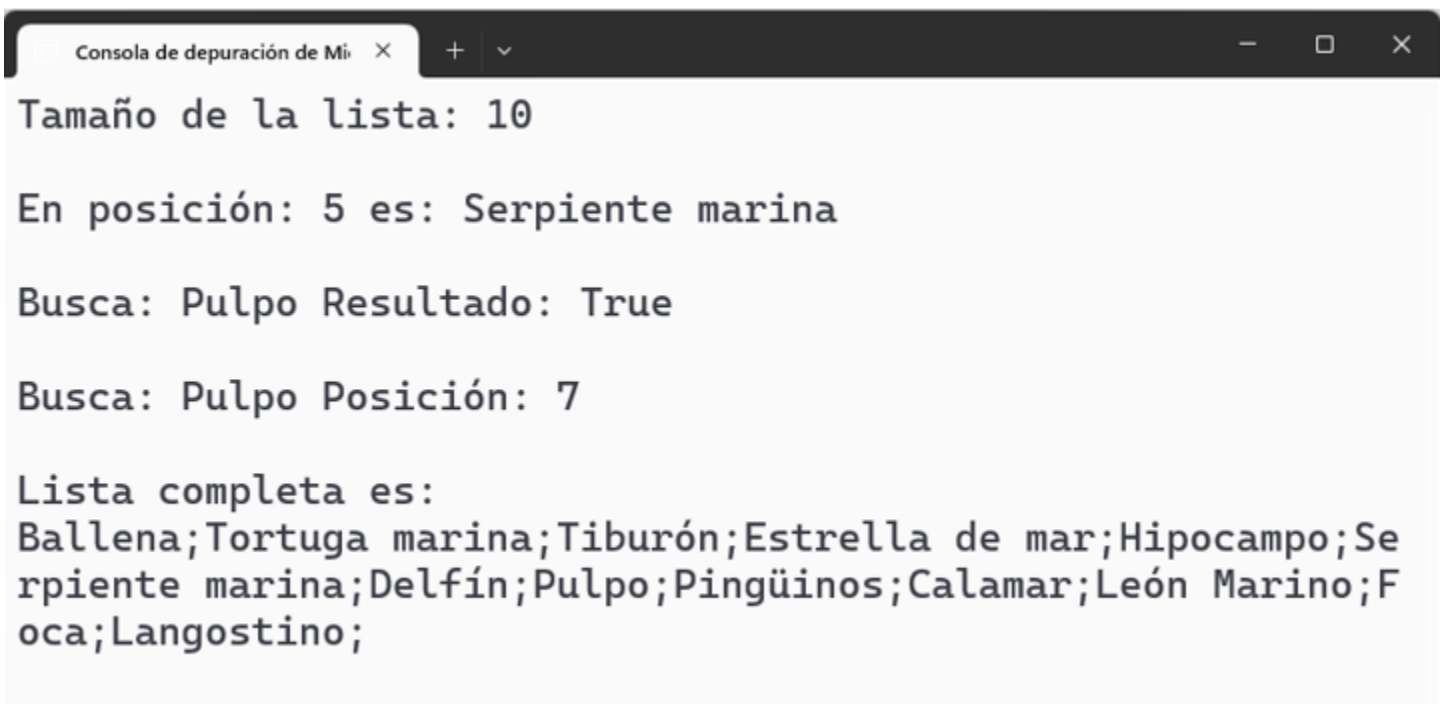
```

ListaAnimales.Add("León Marino");
ListaAnimales.Add("Foca");
ListaAnimales.Add("Langostino");

//Imprime la lista
Console.WriteLine("\r\nLista completa es:");
for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
    Console.Write(ListaAnimales[Cont] + ";");

Console.WriteLine("\r\n");
}
}

```



*Ilustración 1: List Adicionar, tamaño, buscar e imprimir*

Los List empiezan en cero. En el código anterior se muestran las funciones para adicionar al List, determinar el tamaño (número de elementos que tiene), decir si existe un determinado elemento y mostrar la lista.

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Tamaño la lista
        int tamano = ListaAnimales.Count;
        Console.WriteLine("Tamaño de la lista: " + tamano);

        //Traer un determinado elemento de la lista
        int posicion = 5;
        string texto = ListaAnimales[posicion].ToString();
        Console.WriteLine("\r\nEn posición: " + posicion + " es: " + texto);

        //Nos dice si existe un determinado elemento en la lista
        string buscar = "Pulpo";
        bool Existe = ListaAnimales.Contains(buscar);
        Console.WriteLine("\r\nBusca: " + buscar + " Resultado: " + Existe);

        //Nos dice la posición donde encontró el elemento en la lista
        int posBusca = ListaAnimales.IndexOf(buscar);
        Console.WriteLine("\r\nBusca: " + buscar + " Posición: " + posBusca);

        //Agrega elementos a la lista
        ListaAnimales.Add("León Marino");
        ListaAnimales.Add("Foca");
        ListaAnimales.Add("Langostino");

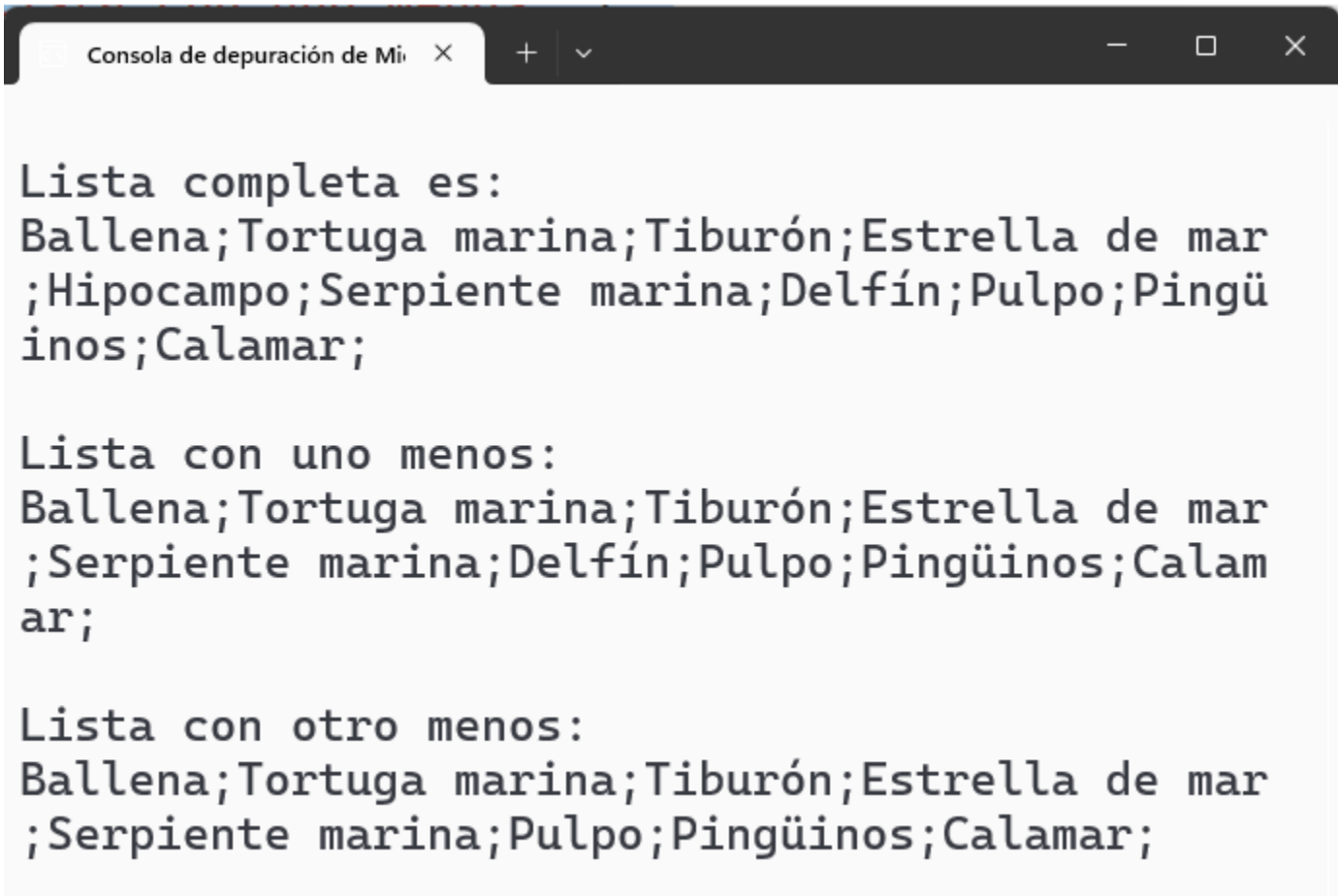
        //Imprime la lista
    }
}
```

```

    Console.WriteLine("\r\nLista completa es:");
    for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
        Console.Write(ListaAnimales[Cont] + ";");

    Console.WriteLine("\r\n");
}
}

```



*Ilustración 2: List, borrar elemento*

Dos técnicas para eliminar elementos de un List, buscando el elemento y eliminándolo, o dada una posición se elimina lo que hay allí.

## Cambiar Elemento

Con la posición de la cadena en la lista, se puede cambiar directamente.

E/003.cs

```
namespace Ejemplo;

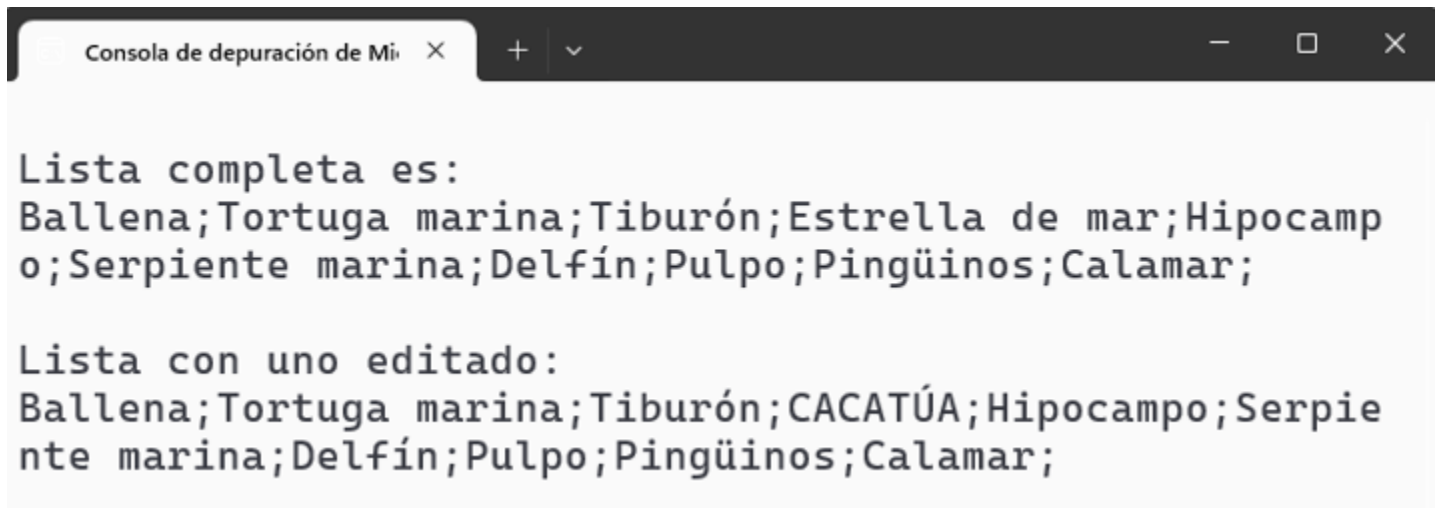
internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Imprime la lista
        Console.WriteLine("\r\nLista completa es:");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");

        //Cambia una cadena en la lista
        ListaAnimales[3] = "CACATÚA";

        //Imprime la lista
        Console.WriteLine("\r\n\r\nLista con uno editado:");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");

        Console.WriteLine("\r\n");
    }
}
```



```
Consola de depuración de Mi X + v - □ X  
  
Lista completa es:  
Ballena;Tortuga marina;Tiburón;Estrella de mar;Hipocampo;  
Serpiente marina;Delfín;Pulpo;Pingüinos;Calamar;  
  
Lista con uno editado:  
Ballena;Tortuga marina;Tiburón;CACATÚA;Hipocampo;Serpiente  
marina;Delfín;Pulpo;Pingüinos;Calamar;
```

*Ilustración 3: List, cambiar elemento*

## Insertar Elemento

Se puede insertar un elemento en la lista simplemente señalando su posición.

E/004.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

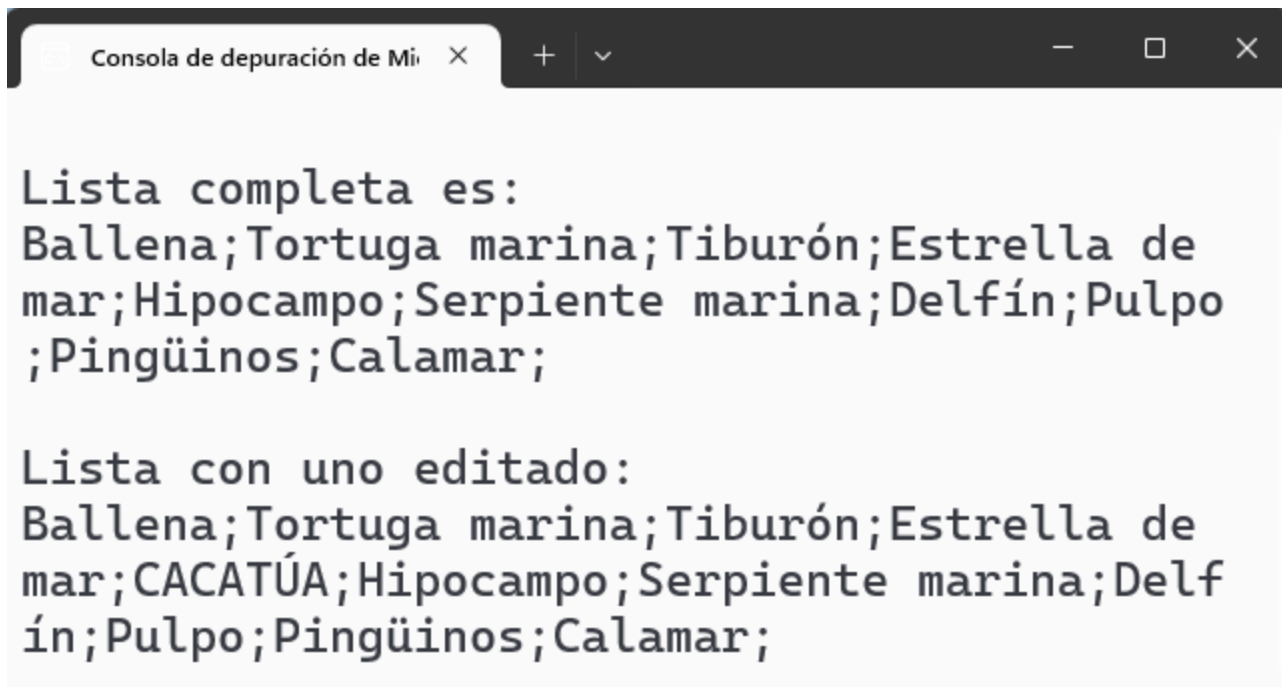
        //Imprime la lista
        Console.WriteLine("\r\nLista completa es:");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");

        //Inserta una cadena en la posición 4 de la lista
        ListaAnimales.Insert(4, "CACATÚA");

        //Imprime la lista
        Console.WriteLine("\r\n\r\nLista con uno editado:");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");

        Console.WriteLine("\r\n");
    }
}
```





```
Consola de depuración de Mi X + v - □ X

Lista completa es:
Ballena;Tortuga marina;Tiburón;Estrella de
mar;Hipocampo;Serpiente marina;Delfín;Pulpo
;Pingüinos;Calamar;

Lista con uno editado:
Ballena;Tortuga marina;Tiburón;Estrella de
mar;CACATÚA;Hipocampo;Serpiente marina;Delf
ín;Pulpo;Pingüinos;Calamar;
```

*Ilustración 4: List, insertar elemento*

## SubLista

List<T>.GetRange crea una nueva lista (una copia de ese segmento si son tipos de datos nativos como string, int, char, double pero no hace copia si es de objetos).

E/005.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Imprime de nuevo la lista original
        Console.WriteLine("\r\n\r\nLista original");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");

        //Genera nueva lista
        int posIni = 5;
        int cantidad = 3;
        List<string> nuevaLista = ListaAnimales.GetRange(posIni, cantidad);

        //Imprime valores de esa nueva lista
        Console.WriteLine("\r\n\r\nNueva lista");
        for (int Cont = 0; Cont < nuevaLista.Count; Cont++)
            Console.Write(nuevaLista[Cont] + ";");

        //Modifica un valor de la nueva lista
        nuevaLista[0] = "CACATÚA";

        //Imprime la lista nueva con el valor alterado
        Console.WriteLine("\r\n\r\nNueva lista, primer valor editado:");
        for (int Cont = 0; Cont < nuevaLista.Count; Cont++)
```

```

        Console.WriteLine(nuevaLista[Cont] + ";");

//Imprime de nuevo la lista original
Console.WriteLine("\r\n\r\nLista original");
for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
    Console.WriteLine(ListaAnimales[Cont] + ";");

Console.WriteLine("\r\n");
    }
}

```

```

Consola de depuración de Mi
+ - □ ×

Lista original
Ballena;Tortuga marina;Tiburón;Estrella de mar;Hi
pocampo;Serpiente marina;Delfín;Pulpo;Pingüinos;C
alamar;

Nueva lista
Serpiente marina;Delfín;Pulpo;

Nueva lista, primer valor editado:
CACATÚA;Delfín;Pulpo;

Lista original
Ballena;Tortuga marina;Tiburón;Estrella de mar;Hi
pocampo;Serpiente marina;Delfín;Pulpo;Pingüinos;C
alamar;

```

Ilustración 5: List, referenciar con una variable, un rango de la lista

```
namespace Ejemplo;

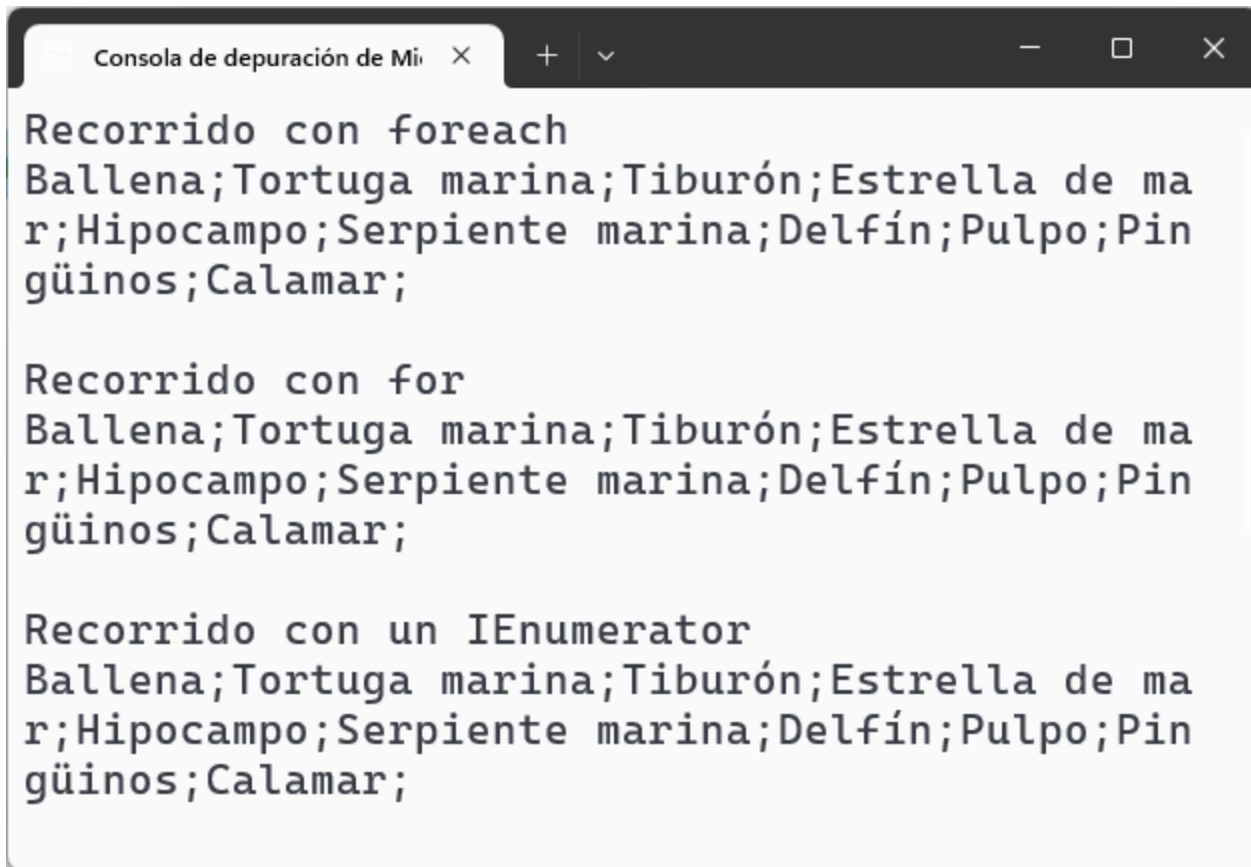
internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Recorrido con foreach
        Console.WriteLine("Recorrido con foreach");
        foreach (Object objeto in ListaAnimales)
            Console.Write(objeto + ";");

        //Recorrido con for
        Console.WriteLine("\r\n\r\nRecorrido con for");
        for (int cont = 0; cont < ListaAnimales.Count; cont++)
            Console.Write(ListaAnimales[cont] + ";");

        //Recorrido con un IEnumerator
        Console.WriteLine("\r\n\r\nRecorrido con un IEnumerator");
        List<string>.Enumerator elemento = ListaAnimales.GetEnumerator();
        while (elemento.MoveNext())
            Console.Write(elemento.Current + ";");

        Console.WriteLine("\r\n");
    }
}
```



The image shows a screenshot of a Visual Studio Code console window. The title bar at the top reads 'Consola de depuración de Mi' followed by a close button. The console contains three blocks of text, each representing a different iteration method:

```
Recorrido con foreach
Ballena;Tortuga marina;Tiburón;Estrella de ma
r;Hipocampo;Serpiente marina;Delfín;Pulpo;Pin
güinos;Calamar;

Recorrido con for
Ballena;Tortuga marina;Tiburón;Estrella de ma
r;Hipocampo;Serpiente marina;Delfín;Pulpo;Pin
güinos;Calamar;

Recorrido con un IEnumerator
Ballena;Tortuga marina;Tiburón;Estrella de ma
r;Hipocampo;Serpiente marina;Delfín;Pulpo;Pin
güinos;Calamar;
```

*Ilustración 6: List, tres formas de recorrerlo*

## Borrar completamente un List

Se utiliza el método `Clear()`

E/007.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales =
        [
            //Inicia con elementos
            "Ballena",
            "Tortuga marina",
            "Tiburón",
            "Estrella de mar",
            "Hipocampo",
            "Serpiente marina",
            "Delfín",
            "Pulpo",
            "Pingüinos",
            "Calamar",
        ];

        //Limpia el List
        Console.WriteLine("(Antes) Elementos: " + ListaAnimales.Count);
        ListaAnimales.Clear();
        Console.WriteLine("(Después) Elementos: " + ListaAnimales.Count);

        Console.WriteLine("\r\n");
    }
}
```

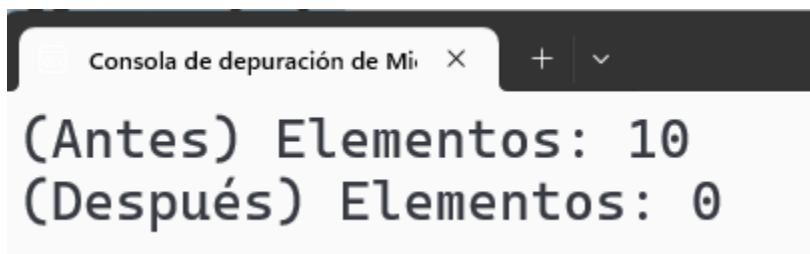


Ilustración 7: Borrar completamente un List

## Borrar un rango en un List

Se utiliza el método RemoveRange()

E/008.cs

```
namespace Ejemplo;

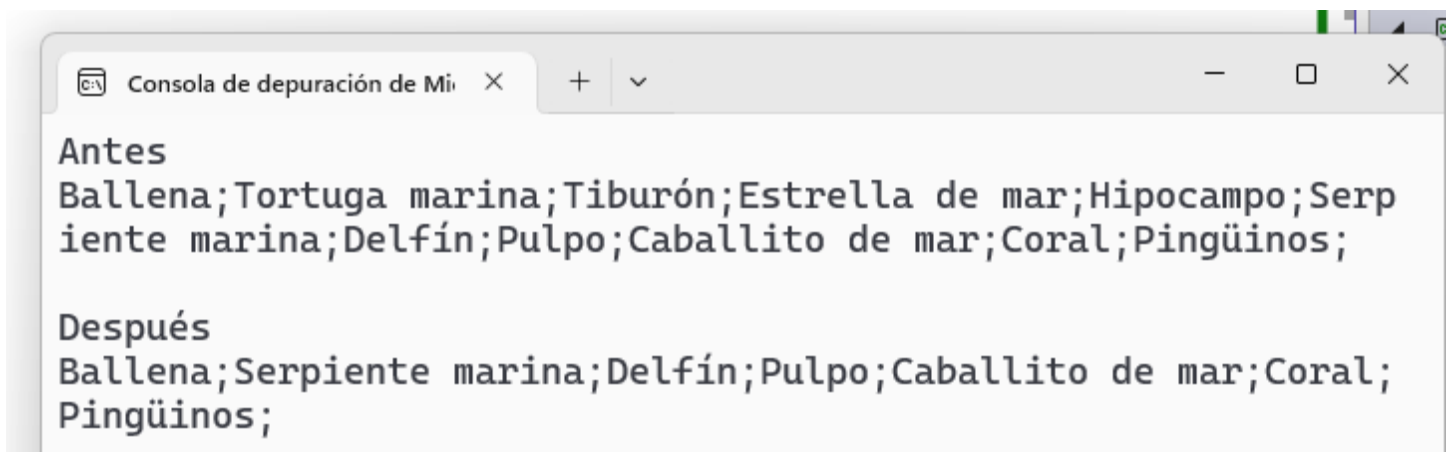
class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales = new();

        //Adiciona elementos a la lista
        ListaAnimales.Add("Ballena");
        ListaAnimales.Add("Tortuga marina");
        ListaAnimales.Add("Tiburón");
        ListaAnimales.Add("Estrella de mar");
        ListaAnimales.Add("Hipocampo");
        ListaAnimales.Add("Serpiente marina");
        ListaAnimales.Add("Delfín");
        ListaAnimales.Add("Pulpo");
        ListaAnimales.Add("Caballito de mar");
        ListaAnimales.Add("Coral");
        ListaAnimales.Add("Pingüinos");

        //Elimina un rango de elementos del List
        Console.WriteLine("Antes");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");
        Console.WriteLine(" ");

        int posicion = 1;
        int cantidad = 4;
        ListaAnimales.RemoveRange(posicion, cantidad);

        Console.WriteLine("\r\nDespués");
        //Imprime valores
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");
        Console.WriteLine(" ");
    }
}
```



*Ilustración 8: Borrar un rango en un List*



## Guardar el List en un arreglo estático

Todos los elementos del List pasan a un arreglo estático, sea de tipo object() o de string.

E/009.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales = new();

        //Adiciona elementos a la lista
        ListaAnimales.Add("Ballena");
        ListaAnimales.Add("Tortuga marina");
        ListaAnimales.Add("Tiburón");
        ListaAnimales.Add("Estrella de mar");
        ListaAnimales.Add("Hipocampo");
        ListaAnimales.Add("Serpiente marina");
        ListaAnimales.Add("Delfín");
        ListaAnimales.Add("Pulpo");
        ListaAnimales.Add("Caballito de mar");
        ListaAnimales.Add("Coral");
        ListaAnimales.Add("Pingüinos");

        //Guarda el List en un arreglo estático
        //de tipo objeto
        Console.WriteLine("Arreglo estático de tipo objeto");
        object[] arregloEstatico =[.. ListaAnimales];
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");
        Console.WriteLine(" ");

        //Guarda el List en un arreglo estático de tipo string
        Console.WriteLine("Arreglo estático de tipo cadena");
        string[] Cadenas =[.. ListaAnimales];
        for (int cont = 0; cont < Cadenas.Length; cont++)
            Console.Write(Cadenas[cont] + ";");
    }
}
```

## Agregar un arreglo estático a un List

Toma el contenido del arreglo estático y lo copia en el List usando el método AddRange()

E/010.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales = new();

        //Adiciona elementos a la lista
        ListaAnimales.Add("Ballena");
        ListaAnimales.Add("Tortuga marina");
        ListaAnimales.Add("Tiburón");
        ListaAnimales.Add("Estrella de mar");
        ListaAnimales.Add("Hipocampo");
        ListaAnimales.Add("Serpiente marina");
        ListaAnimales.Add("Delfín");
        ListaAnimales.Add("Pulpo");
        ListaAnimales.Add("Caballito de mar");
        ListaAnimales.Add("Coral");
        ListaAnimales.Add("Pingüinos");

        //Un arreglo estático
        string[] Cadenas = { "Gato", "Conejo", "Liebre" };

        //Adiciona el arreglo estático al List
        ListaAnimales.AddRange(Cadenas);

        //Imprime el List
        for (int cont = 0; cont < ListaAnimales.Count; cont++)
            Console.Write(ListaAnimales[cont] + "; ");
    }
}
```

## Inserta un arreglo estático en una determinada posición del List

Toma el contenido del arreglo estático, lo copia y lo inserta en alguna posición del List usando el método InsertRange()

E/011.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> ListaAnimales = new();

        //Adiciona elementos a la lista
        ListaAnimales.Add("Ballena");
        ListaAnimales.Add("Tortuga marina");
        ListaAnimales.Add("Tiburón");
        ListaAnimales.Add("Estrella de mar");
        ListaAnimales.Add("Hipocampo");
        ListaAnimales.Add("Serpiente marina");
        ListaAnimales.Add("Delfín");
        ListaAnimales.Add("Pulpo");
        ListaAnimales.Add("Caballito de mar");
        ListaAnimales.Add("Coral");
        ListaAnimales.Add("Pingüinos");

        //Un arreglo estático
        string[] Cadenas = { "Gato", "Conejo", "Liebre" };

        //Inserta el arreglo estático en una determinada
        //posición del List
        int posicionInserta = 4;
        ListaAnimales.InsertRange(posicionInserta, Cadenas);

        //Imprime el List
        for (int cont = 0; cont < ListaAnimales.Count; cont++)
            Console.Write(ListaAnimales[cont] + "; ");
    }
}
```

## Insertar varios List dentro de un List

El contenido de varios List es copiado dentro de otro List. Como es una copia, no importa si se modifica el List fuente.

E/012.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara tres List
        List<string> ListaA = [];
        List<string> ListaB = [];
        List<string> ListaC = [];

        //Adiciona elementos a esos List
        ListaA.Add("A");
        ListaA.Add("B");
        ListaA.Add("C");
        ListaB.Add("7");
        ListaB.Add("8");
        ListaB.Add("9");
        ListaC.Add("qw");
        ListaC.Add("er");
        ListaC.Add("ty");

        //Inserta los dos primeros List en el tercero
        int posicionInserta = 1;
        ListaC.InsertRange(posicionInserta, ListaA);

        posicionInserta = 5;
        ListaC.InsertRange(posicionInserta, ListaB);

        //Imprime el List
        for (int cont = 0; cont < ListaC.Count; cont++)
            Console.Write(ListaC[cont] + "; ");
        Console.WriteLine("\r\n");

        //Modifica ListaA y se chequea si eso afectó a ListaC
        ListaA[0] = "CACATÚA";
        for (int cont = 0; cont < ListaC.Count; cont++)
            Console.Write(ListaC[cont] + "; ");
    }
}
```

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> Listado = [];

        //Adiciona elementos a la lista
        Listado.Add("AB");
        Listado.Add("CD");
        Listado.Add("EF");
        Listado.Add("GH");
        Listado.Add("IJ");
        Listado.Add("KL");
        Listado.Add("MN");
        Listado.Add("OP");

        //Imprime el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
        Console.WriteLine("\r\n");

        //Aplica Reverse()
        Listado.Reverse();

        //Imprime de nuevo el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
    }
}
```

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> Listado = new();

        //Adiciona elementos a la lista
        Listado.Add("AB");
        Listado.Add("CD");
        Listado.Add("EF");
        Listado.Add("GH");
        Listado.Add("IJ");
        Listado.Add("KL");
        Listado.Add("MN");
        Listado.Add("OP");

        //Imprime el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
        Console.WriteLine("\r\n");

        //Aplica Reverse(posicion, cantidad)
        int posicion = 2;
        int cantidad = 4;
        Listado.Reverse(posicion, cantidad);

        //Imprime de nuevo el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
    }
}
```

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> Listado = [];

        //Adiciona elementos a la lista
        Listado.Add("GH");
        Listado.Add("MN");
        Listado.Add("AB");
        Listado.Add("OP");
        Listado.Add("IJ");
        Listado.Add("KL");
        Listado.Add("CD");
        Listado.Add("EF");

        //Imprime el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
        Console.WriteLine("\r\n");

        //Ordena el List
        Listado.Sort();

        //Imprime de nuevo el List
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
    }
}
```

## Búsqueda Binaria

Una vez el List es ordenado, se puede hacer una búsqueda binaria.

E/016.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<string> Listado = [];

        //Adiciona elementos a la lista
        Listado.Add("GH");
        Listado.Add("MN");
        Listado.Add("AB");
        Listado.Add("KL");
        Listado.Add("OP");
        Listado.Add("IJ");
        Listado.Add("CD");
        Listado.Add("EF");

        //Imprime el List
        Console.WriteLine("List Original");
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
        Console.WriteLine("\r\n");

        //Ordena el List
        Listado.Sort();

        //Imprime de nuevo el List
        Console.WriteLine("List Ordenado");
        for (int cont = 0; cont < Listado.Count; cont++)
            Console.Write(Listado[cont] + "; ");
        Console.WriteLine("\r\n");

        //Busca en forma binaria en el List
        string Buscar = "KL";
        int pos = Listado.BinarySearch(Buscar);
        Console.WriteLine("Buscando: " + Buscar);
        Console.WriteLine("Encontrado en: " + pos);
    }
}
```



## Capacidad del List

A medida que el List se le van adicionando elementos, su capacidad va aumentando siempre por encima del número de elementos almacenados.

E/017.cs

```
namespace Ejemplo;

class Program {
    public static void Main() {
        //Declara la lista que almacenará cadenas
        List<double> Listado = new();

        //Para agregar elementos al azar
        Random azar = new();

        //Va agregando elementos al azar,
        //imprime el tamaño y la capacidad
        for (int veces = 1; veces <= 50; veces++) {
            Console.WriteLine("\r\nIteración: " + veces);
            Console.WriteLine("Tamaño: " + Listado.Count);
            Console.WriteLine("Capacidad: " + Listado.Capacity);
            for (int cont = 1; cont <= 30; cont++) {
                Listado.Add(azar.NextDouble());
            }
        }
    }
}
```

```
Iteración: 1
Tamaño: 0
Capacidad: 0

Iteración: 2
Tamaño: 30
Capacidad: 32

Iteración: 3
Tamaño: 60
Capacidad: 64

Iteración: 4
Tamaño: 90
Capacidad: 128

Iteración: 5
Tamaño: 120
Capacidad: 128

Iteración: 6
Tamaño: 150
Capacidad: 256
```

*Ilustración 9: Capacidad del List*

## Métricas: Comparativa de desempeño de List vs Arreglo estático vs ArrayList

La estructura ArrayList es antigua y obsoleta que permite guardar todo tipo de dato, fue reemplazada por estructuras modernas como List. Una de las razones del no uso de ArrayList es su lentitud. Para demostrarlo se hace una comparativa de desempeño, se utilizó el algoritmo de ordenación de burbuja, el cual hace uso intensivo de operaciones de lectura y cambio de valores en la estructura de memoria (por eso es tan lento ese algoritmo), pero será útil para hacer la comparativa. El programa a continuación:

E/018.cs

```
using System.Collections;
using System.Diagnostics;

namespace Ejemplo;

class Program {
    static void Main() {
#ifdef DEBUG
        Console.WriteLine("Modo DEBUG detectado. Las pruebas se deben hacer en RELEASE");
        Environment.Exit(0);
#endif

        /* Prueba de velocidad de los diferentes tipos de estructuras:
         * arreglo estático, ArrayList, List
         * Se usará el método de ordenación de burbuja en el que
         * hace una gran cantidad de lectura y escritura sobre
         * la estructura (por eso es el más lento pero muy bueno para
         * hacer esta comparativa)
         * */

        int minOrden = 500; //Mínimo número de elementos a ordenar
        int maxOrden = 9000; //Máximo número de elementos a ordenar
        int avanceOrden = 500; //Avance de elementos a ordenar

        /* Número de pruebas por ordenamiento
         * Luego el tiempo de ordenar N elementos es el promedio
         * de esas pruebas así se evita que por algún motivo los
         * tiempos tengan picos o valles */
        int numPruebas = 40;

        //Limite es el tamaño de datos que se van a ordenar
        Console.WriteLine(".NET Versión: " + Environment.Version);
        Console.WriteLine("Ordenación. Tiempo promedio en milisegundos");
```

```

Console.WriteLine("Elementos;Arreglo;ArrayList;List");
for (int Lim = minOrden; Lim <= maxOrden; Lim += avanceOrden)
    Ordenamiento(Lim, numPruebas);

Console.WriteLine("\r\nFinal de la prueba");
}

static void Ordenamiento(int Limite, int numPruebas) {
    Random azar = new();

    //Las estructuras usadas: arreglo estático, ArrayList, List
    int[] numerosA = new int[Limite];
    int[] numerosB = new int[Limite];
    ArrayList arraylist = [];
    List<int> list = [];

    //Medidor de tiempos
    Stopwatch temporizador = new();

    //Almacena los tiempos de cada método de ordenación
    long TParreglo = 0, TParrraylist = 0, TPlist = 0;

    //Para disminuir picos o valles en el tiempo,
    //se hacen varias pruebas
    for (int prueba = 1; prueba <= numPruebas; prueba++) {

        //Llena con valores al azar el arreglo
        LlenaAzar(numerosA, azar);

        //Ordenación por Burbuja ArrayList
        arraylist.Clear();
        arraylist.AddRange(numerosA);
        temporizador.Reset();
        temporizador.Start();
        BurbujaArrayList(arraylist);
        TParrraylist += temporizador.ElapsedMilliseconds;

        //Ordenación por Burbuja List
        list.Clear();
        list.AddRange(numerosA);
        temporizador.Reset();
        temporizador.Start();
        BurbujaList(list);
        TPlist += temporizador.ElapsedMilliseconds;

        //Ordenación por Burbuja Arreglo estático
        Array.Copy(numerosA, 0, numerosB, 0, numerosA.Length);
        temporizador.Reset();
    }
}

```

```

    temporizador.Start();
    BurbujaArreglo(numerosB);
    TParreglo += temporizador.ElapsedMilliseconds;

    //Compara las listas ordenadas
    for (int cont = 0; cont < numerosB.Length; cont++) {
        if (numerosB[cont] != list[cont] ||
            list[cont] != Convert.ToInt32(arraylist[cont]))
            Console.WriteLine("Error en la ordenación");
    }
}

double Tarreglo = (double)TParreglo / numPruebas;
double Tarraylist = (double)TParraylist / numPruebas;
double Tlist = (double)TPlist / numPruebas;

Console.Write(Limite + ";" + Tarreglo);
Console.Write ";" + Tarraylist);
Console.WriteLine ";" + Tlist);
}

//Llena el arreglo unidimensional con valores aleatorios
static void LlenaAzar(int[] numerosA, Random azar) {
    for (int cont = 0; cont < numerosA.Length; cont++)
        numerosA[cont] = azar.Next(0, 10000);
}

//Ordenamiento por burbuja usando ArrayList
static void BurbujaArrayList(ArrayList arraylist) {
    int tamano = arraylist.Count;
    object tmp;
    for (int i = 0; i < tamano - 1; i++) {
        for (int j = 0; j < tamano - 1; j++) {
            int iA = Convert.ToInt32(arraylist[j]);
            int iB = Convert.ToInt32(arraylist[j + 1]);
            if (iA > iB) {
                tmp = arraylist[j];
                arraylist[j] = arraylist[j + 1];
                arraylist[j + 1] = tmp;
            }
        }
    }
}

//Ordenamiento por burbuja usando List
static void BurbujaList(List<int> list) {
    int tamano = list.Count;
    int tmp;

```

```

    for (int i = 0; i < tamano - 1; i++) {
        for (int j = 0; j < tamano - 1; j++) {
            if (list[j] > list[j + 1]) {
                tmp = list[j];
                list[j] = list[j + 1];
                list[j + 1] = tmp;
            }
        }
    }
}

//Ordenamiento por burbuja usando arreglo unidimensional estático
static void BurbujaArreglo(int[] arregloestatico) {
    int tamano = arregloestatico.Length;
    int tmp;
    for (int i = 0; i < tamano - 1; i++) {
        for (int j = 0; j < tamano - 1; j++) {
            if (arregloestatico[j] > arregloestatico[j + 1]) {
                tmp = arregloestatico[j];
                arregloestatico[j] = arregloestatico[j + 1];
                arregloestatico[j + 1] = tmp;
            }
        }
    }
}
}

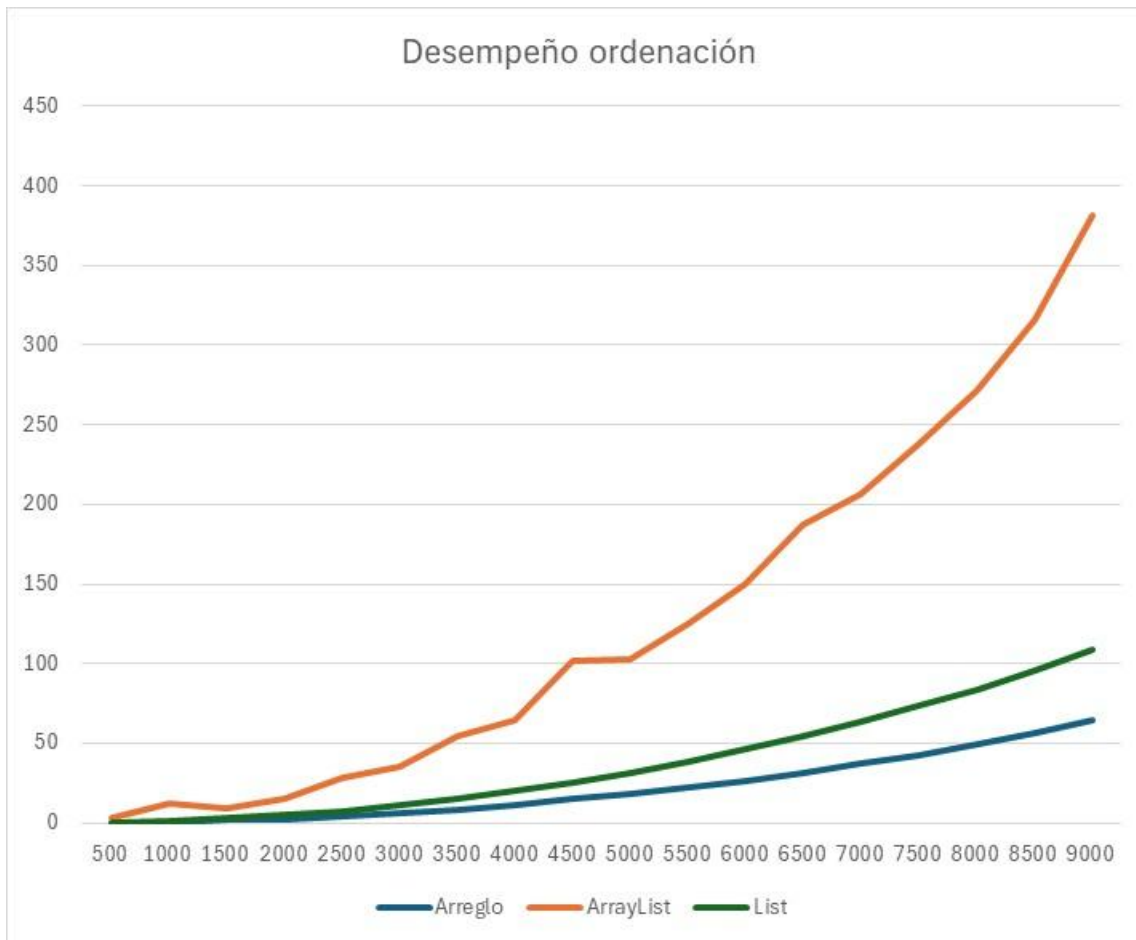
```



```
.NET Versión: 10.0.1
Ordenación. Tiempo promedio en milisegundos
Elementos;Arreglo;ArrayList;List
500;0;3,05;0
1000;0,25;12;1
1500;1,7;8,85;2,85
2000;2,25;15;4,925
2500;4,025;28,45;7,425
3000;6,125;35,225;11,05
3500;8,4;54,8;15,1
4000;11,275;64,2;19,925
4500;14,75;101,6;25,575
5000;18,2;102,85;31,5
5500;22,05;125,3;38,375
6000;26,7;149,575;46,2
6500;31,65;187,7;54,425
7000;36,9;206;63,65
7500;42,825;237,675;73,125
8000;49,675;271,825;83,825
8500;56,725;315,95;95,625
9000;64,725;381,225;109,025

Final de la prueba
```

Ilustración 10: Comparativa



Los arreglos unidimensionales estáticos mostraron mejores tiempos. La estructura dinámica List es ligeramente más lenta y ArrayList es notablemente más lenta.



## Lista de objetos

Un frecuente uso de List es para tener un listado de objetos del mismo tipo, no solo tipos de datos nativo. Se requiere entonces dos clases, en una está definido el tipo de objeto a guardar y en la otra se crean, adicionan, actualizan y borran del List.

E/019.cs

```
namespace Ejemplo;

class MiClase {
    //Atributos variados
    public int Entero { get; set; }
    public double Num { get; set; }
    public char Car { get; set; }
    public string Cad { get; set; }

    //Constructor
    public MiClase(int Entero, double Num, char Car, string Cad) {
        this.Entero = Entero;
        this.Num = Num;
        this.Car = Car;
        this.Cad = Cad;
    }

    //Imprime los valores
    public void Imprime() {
        Console.WriteLine("\r\nEntero: " + Entero);
        Console.WriteLine("Real: " + Num);
        Console.WriteLine("Caracter: " + Car);
        Console.WriteLine("Cadena: [" + Cad + "]");
    }
}

class Program {
    static void Main() {
        List<MiClase> Listado = [];

        //Adiciona objetos a la lista
        Listado.Add(new MiClase(16, 83.29, 'R', "Rui señor"));
        Listado.Add(new MiClase(29, 89.7, 'A', "Águila"));
        Listado.Add(new MiClase(2, 80.19, 'M', "Manatí"));
        Listado.Add(new MiClase(95, 7.21, 'P', "Puma"));

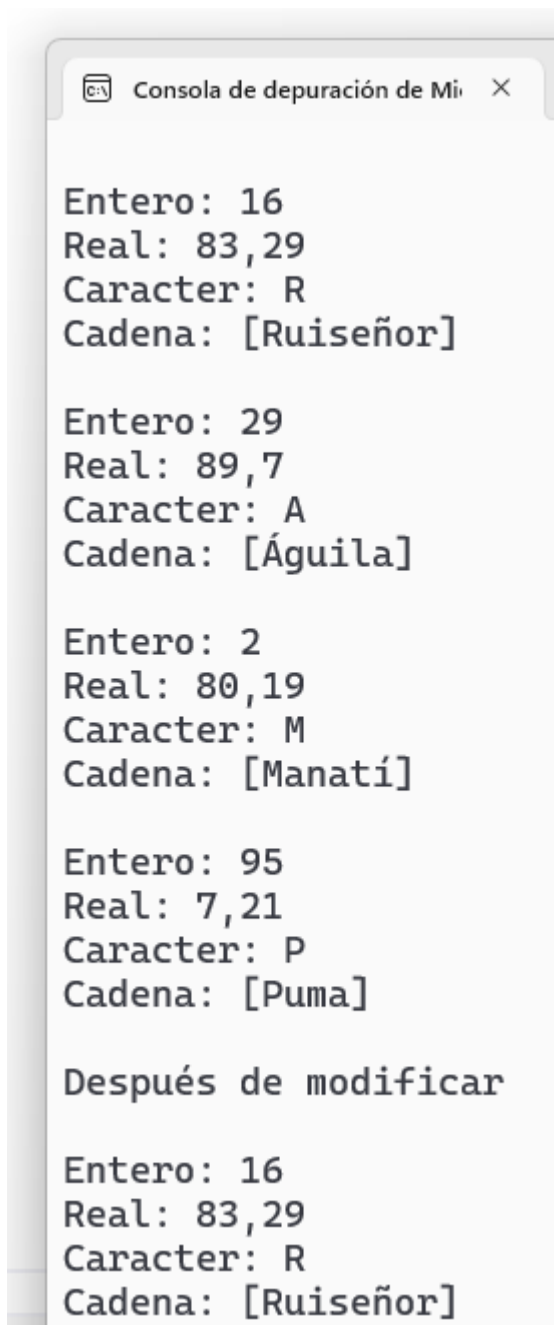
        //Llama al método de imprimir del objeto
        for (int cont = 0; cont < Listado.Count; cont++)
            Listado[cont].Imprime();
    }
}
```

```
//Inserta un objeto
Listado.Insert(1, new MiClase(88, 3.33, 'Z', "QQQQQ"));

//Elimina un objeto
Listado.RemoveAt(3);

//Llama al método de imprimir del objeto
Console.WriteLine("\r\nDespués de modificar");
for (int cont = 0; cont < Listado.Count; cont++)
    Listado[cont].Imprime();

Console.WriteLine("\r\nFinal");
}
}
```



```
Consola de depuración de Mi X

Entero: 16
Real: 83,29
Caracter: R
Cadena: [Ruisenior]

Entero: 29
Real: 89,7
Caracter: A
Cadena: [Águila]

Entero: 2
Real: 80,19
Caracter: M
Cadena: [Manatí]

Entero: 95
Real: 7,21
Caracter: P
Cadena: [Puma]

Después de modificar

Entero: 16
Real: 83,29
Caracter: R
Cadena: [Ruisenior]
```

*Ilustración 11: Lista de objetos*

## Listas en Listas

Un objeto de una lista, a su vez tiene listas. Un ejemplo con series de TV, personajes y actores:

1. La serie tiene un nombre y se puede ver información sobre esta en IMDB.
2. Los personajes son interpretados por actores y actrices.
3. No es nada extraño que los actores participen en diversas series interpretando algún personaje en alguna serie, sólo es ver su ficha en IMDB.

E/020.cs

```
namespace Ejemplo;

//Datos del actor o actriz
class ActorActriz {
    public int Codigo { get; set; }
    public string Nombre { get; set; }
    public string URLIMDB { get; set; }

    //Constructor
    public ActorActriz(int Codigo, string Nombre, string URLIMDB) {
        this.Codigo = Codigo;
        this.Nombre = Nombre;
        this.URLIMDB = "https://www.imdb.com/name/" + URLIMDB;
    }
}

//Datos de la serie de televisión
class Serie {
    public int Codigo { get; set; }
    public string Nombre { get; set; }
    public string URLIMDB { get; set; }

    //Listado de códigos de actores que actúan en la serie
    public List<int> Actor = [];

    //Constructor
    public Serie(int Codigo, string Nombre, string URLIMDB) {
        this.Codigo = Codigo;
        this.Nombre = Nombre;
        this.URLIMDB = "https://www.imdb.com/title/" + URLIMDB;
    }
}

//La parte que simula la capa de persistencia
class Persistencia {
```

```

public List<ActorActriz> Actores;
public List<Serie> Series;

//Carga datos de prueba
public Persistencia() {
    Actores = [];
    Series = [];

    //Un listado de actores y actrices
    ActorAdiciona(78, "Ana María Orozco", "nm0650450");
    ActorAdiciona(81, "Laura Londoño", "nm2256810");
    ActorAdiciona(84, "Carolina Ramírez", "nm1329835");
    ActorAdiciona(93, "Catherine Siachoque", "nm0796171");
    ActorAdiciona(98, "Carmenza González", "nm1863990");
    ActorAdiciona(99, "Andrés Londoño", "nm2150265");

    //Un listado de series
    Series.Add(new Serie(16, "Yo soy Betty, la fea", "tt0233127"));
    Series.Add(new Serie(43, "La reina del flow", "tt8560918"));
    Series.Add(new Serie(60, "Café con Aroma de Mujer", "tt14471346"));
    Series.Add(new Serie(62, "Los Briceño", "tt10348478"));
    Series.Add(new Serie(70, "Distrito Salvaje", "tt8105958"));
    Series.Add(new Serie(72, "Mil Colmillos", "tt9701670"));
    Series.Add(new Serie(83, "Perdida", "tt10064124"));

    //Obsérvese que un mismo actor o actriz puede
    //estar en dos series distintas
    Series[0].Actor.Add(78);
    Series[0].Actor.Add(93);
    Series[0].Actor.Add(98);
    Series[6].Actor.Add(78);
    Series[4].Actor.Add(78);
}

//Adicionar actor
public bool ActorAdiciona(int Codigo, string Nombre, string URL) {
    for (int Cont = 0; Cont < Actores.Count; Cont++) {
        if (Actores[Cont].Codigo == Codigo)
            return false;
    }
    Actores.Add(new ActorActriz(Codigo, Nombre, URL));
    return true;
}

//Editar actor
public bool ActorEdita(int CodigoActor, string Nombre, string URL) {
    for (int Cont = 0; Cont < Actores.Count; Cont++) {
        if (Actores[Cont].Codigo == CodigoActor) {

```

```

        Actores[Cont].Nombre = Nombre;
        Actores[Cont].URLIMDB = URL;
        return true;
    }
}
return false;
}

//Chequea si el actor está trabajando en alguna serie
public bool ActorEnSerie(intCodigoActor) {
    for (int Cont = 0; Cont < Series.Count; Cont++)
        for (int Num = 0; Num < Series[Cont].Actor.Count; Num++)
            if (Series[Cont].Actor[Num] == CodigoActor)
                return true;
    return false;
}

//Borrar actor, si y solo si no está trabajando en alguna serie
public bool ActorBorra(intCodigoActor) {
    if (ActorEnSerie(CodigoActor) == false) {
        for (int Cont = 0; Cont < Actores.Count; Cont++)
            if (Actores[Cont].Codigo == CodigoActor) {
                Actores.RemoveAt(Cont);
                return true;
            }
    }
    return false;
}

//Dado el código, retorna el nombre del actor/actriz
public string NombreActor(intCodigoActor) {
    for (int cont = 0; cont < Actores.Count; cont++) {
        if (Actores[cont].Codigo == CodigoActor)
            return Actores[cont].Nombre;
    }
    return "N/A";
}

//Retorna una lista de series donde el actor trabaja
public List<string> ActorTrabaja(intCodigoActor) {
    List<string> ListaSeries = [];
    for (int cont = 0; cont < Series.Count; cont++) {
        for (int num = 0; num < Series[cont].Actor.Count; num++) {
            if (Series[cont].Actor[num] == CodigoActor)
                ListaSeries.Add(Series[cont].Nombre);
        }
    }
    return ListaSeries;
}

```

```

}

//Adicionar serie
public bool SerieAdiciona(int CodigoSerie, string Nombre, string URL) {
    for (int Cont = 0; Cont < Series.Count; Cont++) {
        if (Series[Cont].Codigo == CodigoSerie)
            return false;
    }
    Series.Add(new Serie(CodigoSerie, Nombre, URL));
    return true;
}

//Editar serie
public bool SerieEdita(int CodigoSerie, string Nombre, string URL) {
    for (int Cont = 0; Cont < Series.Count; Cont++) {
        if (Series[Cont].Codigo == CodigoSerie) {
            Series[Cont].Nombre = Nombre;
            Series[Cont].URLIMDB = URL;
            return true;
        }
    }
    return false;
}

//Borrar serie
public bool SerieBorra(int CodigoSerie) {
    for (int Cont = 0; Cont < Series.Count; Cont++)
        if (Series[Cont].Codigo == CodigoSerie) {
            Series.RemoveAt(Cont);
            return true;
        }
    return false;
}

//Retornar los actores que trabajan en la serie
public List<string> SerieActores(int CodigoSerie) {
    int Pos = PosSerie(CodigoSerie);
    List<string> Nombres = [];
    for (int Cont = 0; Cont < Series[Pos].Actor.Count; Cont++)
        Nombres.Add "[" + Series[Pos].Actor[Cont] + "]" + " " +
NombreActor(Series[Pos].Actor[Cont]));
    return Nombres;
}

//Añade un actor a una serie
public bool SerieAsocia(int CodigoSerie, int CodigoActor) {
    int PosSerial = PosSerie(CodigoSerie);

```

```

        if (PosSerial >= 0) {
            if (Series[PosSerial].Actor.Contains(CodigoActor) == false) {
                Series[PosSerial].Actor.Add(CodigoActor);
                return true;
            }
            else
                return false;
        }
        return false;
    }

    //Quita un actor de una serie
    public bool SerieDisocia(int CodigoSerie, int CodigoActor) {
        int PosSerial = PosSerie(CodigoSerie);
        if (PosSerial >= 0) {
            if (Series[PosSerial].Actor.Contains(CodigoActor) == true) {
                Series[PosSerial].Actor.Remove(CodigoActor);
                return true;
            }
            else
                return false;
        }
        return false;
    }

    //Dado el código de la serie, retorna su posición
    public int PosSerie(int CodigoSerie) {
        for (int Cont = 0; Cont < Series.Count; Cont++)
            if (Series[Cont].Codigo == CodigoSerie) {
                return Cont;
            }
        return -1;
    }
}

//La parte visual del programa
class Visual {
    public Persistencia Datos;

    //Conecta con la capa de persistencia
    public Visual(Persistencia objDatos) {
        Datos = objDatos;
    }

    //Menú principal
    public void Menu() {
        int Opcion;
        do {

```



```

        Console.Clear();
        Console.WriteLine("\nSoftware TV Show 1.7 (Marzo de 2025)");
        Console.WriteLine("1. CRUD de actores y actrices");
        Console.WriteLine("2. CRUD de series");
        Console.WriteLine("3. Salir");
        Console.Write("¿Opción? ");
        Opcion = Convert.ToInt32(Console.ReadLine());
        switch (Opcion) {
            case 1: CRUDactores(); break;
            case 2: CRUDseries(); break;
        }
    } while (Opcion != 3);
}

//Menú de actores y actrices
public void CRUDactores() {
    int Opcion;
    do {
        Console.Clear();
        Console.WriteLine("\nSoftware TV Show. Actores/Actrices");
        for (int Cont = 0; Cont < Datos.Actores.Count; Cont++) {
            Console.Write "[" + Datos.Actores[Cont].Codigo + " ] ";
            Console.Write(Datos.Actores[Cont].Nombre);
            Console.WriteLine(" URL: " + Datos.Actores[Cont].URLIMDB);
        }
        Console.WriteLine(" \n1. Adicionar");
        Console.WriteLine("2. Editar");
        Console.WriteLine("3. Borrar");
        Console.WriteLine("4. ¿En cuáles series trabaja?");
        Console.WriteLine("5. Volver a menú principal");
        Console.Write("¿Opción? ");
        Opcion = Convert.ToInt32(Console.ReadLine());
        switch (Opcion) {
            case 1: ActorAdiciona(); break;
            case 2: ActorEdita(); break;
            case 3: ActorBorra(); break;
            case 4: ActorTrabaja(); break;
        }
    } while (Opcion != 5);
}

//Menú de series de TV
public void CRUDseries() {
    int Opcion;
    do {
        Console.Clear();
        Console.WriteLine("\nSoftware TV Show. Series");
        for (int Cont = 0; Cont < Datos.Series.Count; Cont++) {

```

```

        Console.WriteLine("[ " + Datos.Series[Cont].Codigo + " ] ");
        Console.WriteLine(Datos.Series[Cont].Nombre);
        Console.WriteLine(" URL: " + Datos.Series[Cont].URLIMDB);
    }
    Console.WriteLine("\n1. Adicionar");
    Console.WriteLine("2. Editar");
    Console.WriteLine("3. Borrar");
    Console.WriteLine("4. Detalles de la serie");
    Console.WriteLine("5. Asociar actor a serie");
    Console.WriteLine("6. Disociar actor a serie");
    Console.WriteLine("7. Volver a menú principal");
    Console.Write("¿Opción? ");
    Opcion = Convert.ToInt32(Console.ReadLine());
    switch (Opcion) {
        case 1: SerieAdiciona(); break;
        case 2: SerieEdita(); break;
        case 3: SerieBorra(); break;
        case 4: SerieDetalle(); break;
        case 5: SerieAsocia(); break;
        case 6: SerieDisocia(); break;
    }
} while (Opcion != 7);
}

//Pantalla para adicionar actores
public void ActorAdiciona() {
    Console.WriteLine("\tAdicionar actor al listado");
    Console.Write("¿Código? ");
    int CodigoActor = Convert.ToInt32(Console.ReadLine());
    Console.Write("¿Nombre? ");
    string Nombre = Console.ReadLine();
    Console.Write("¿URL de IMDB? ");
    string URL = Console.ReadLine();
    if (Datos.ActorAdiciona(CodigoActor, Nombre, URL))
        Console.WriteLine("\nActor adicionado.");
    else
        Console.WriteLine("\nError al adicionar el actor. El código ya existe.");
    Console.ReadKey();
}

//Pantalla para editar actores
public void ActorEdita() {
    Console.WriteLine("\tEditar actor");
    Console.Write("¿Cuál? Escriba el número que está entre [ ]: ");
    int CodigoActor = Convert.ToInt32(Console.ReadLine());
    Console.Write("¿Nombre? ");
    string Nombre = Console.ReadLine();

```

```

        Console.WriteLine("¿URL de IMDB? ");
        string URL = Console.ReadLine();
        if (Datos.ActorEdita(CodigoActor, Nombre, URL))
            Console.WriteLine("\nActor editado.");
        else
            Console.WriteLine("\nError al editar el actor");
        Console.ReadKey();
    }

    //Pantalla para borrar actores
    public void ActorBorra() {
        Console.WriteLine("\tBorrar actor o actriz");
        Console.Write("¿Cuál? Escriba el número que está entre [ ]: ");
        int CodigoActor = Convert.ToInt32(Console.ReadLine());
        if (Datos.ActorBorra(CodigoActor))
            Console.WriteLine("\nActor borrado.");
        else
            Console.WriteLine("\nError al borrar el actor. Código erróneo o el actor trabaja en series.");
        Console.ReadKey();
    }

    //Pantalla para mostrar en que series trabaja el actor
    public void ActorTrabaja() {
        List<string> ListaSeries;
        Console.WriteLine("\tListar series donde actúa");
        Console.Write("¿Cuál? Escriba el número que está entre [ ]: ");
        int CodigoActor = Convert.ToInt32(Console.ReadLine());
        ListaSeries = Datos.ActorTrabaja(CodigoActor);
        for (int Cont = 0; Cont < ListaSeries.Count; Cont++)
            Console.WriteLine(ListaSeries[Cont]);
        Console.WriteLine("\nPresione");
        Console.ReadKey();
    }

    //Pantalla para adicionar series
    public void SerieAdiciona() {
        Console.WriteLine("\tAdicionar serie al listado");
        Console.Write("¿Código? ");
        int codigo = Convert.ToInt32(Console.ReadLine());
        Console.Write("¿Nombre? ");
        string nombre = Console.ReadLine();
        Console.Write("¿URL en IMDB? ");
        string url = Console.ReadLine();
        if (Datos.SerieAdiciona(codigo, nombre, url))
            Console.WriteLine("\nSerie adicionada.");
        else
            Console.WriteLine("\nError al adicionar la serie");
    }

```

```

    Console.ReadKey();
}

//Pantalla para editar series
public void SerieEdita() {
    Console.WriteLine("\tEditar serie");
    Console.Write("¿Cuál? Escriba el número que está entre [ ]: ");
    int codigo = Convert.ToInt32(Console.ReadLine());
    Console.Write("¿Nombre? ");
    string nombre = Console.ReadLine();
    Console.Write("¿URL en IMDB? ");
    string url = Console.ReadLine();
    if (Datos.SerieEdita(codigo, nombre, url))
        Console.WriteLine("\nSerie editada.");
    else
        Console.WriteLine("\nError al editar la serie");
    Console.ReadKey();
}

//Pantalla para borrar series
public void SerieBorra() {
    Console.WriteLine("\tBorrar serie");
    Console.Write("¿Cuál? Escriba el número que está entre [ ]: ");
    int codigo = Convert.ToInt32(Console.ReadLine());
    if (Datos.SerieBorra(codigo))
        Console.WriteLine("\nSerie borrada.");
    else
        Console.WriteLine("\nError al borrar la serie.");
    Console.ReadKey();
}

//Pantalla para ver el detalle de la serie
public void SerieDetalle() {
    List<string> ListaActores;

    Console.WriteLine("\t === Detalle de una serie ===");
    Console.Write("¿Cuál? Número[ ]: ");
    int CodigoSerie = Convert.ToInt32(Console.ReadLine());
    int Pos = Datos.PosSerie(CodigoSerie);
    if (Pos >= 0) {
        Console.WriteLine("Nombre: " + Datos.Series[Pos].Nombre);
        Console.WriteLine("URL: " + Datos.Series[Pos].URLIMDB);
        Console.WriteLine("Actores");
        ListaActores = Datos.SerieActores(CodigoSerie);
        for (int cont = 0; cont < ListaActores.Count; cont++)
            Console.WriteLine("\t" + ListaActores[cont]);
    }
    else

```

```

        Console.WriteLine("Error en código de la serie");
        Console.WriteLine("\nENTER para continuar");
        Console.ReadKey();
    }

    //Asociar actor o actriz a una serie
    public void SerieAsocia() {
        Console.WriteLine("\tAsocia un actor o actriz a una serie");
        Console.Write("¿Cuál serie? Número[ ]: ");
        int CodigoSerie = Convert.ToInt32(Console.ReadLine());
        for (int cont = 0; cont < Datos.Actores.Count; cont++) {
            Console.Write "[" + Datos.Actores[cont].Codigo + "] ";
            Console.Write(Datos.Actores[cont].Nombre);
            Console.WriteLine(" URL: " + Datos.Actores[cont].URLIMDB);
        }
        Console.Write("¿Cuál Actor? Número[ ]: ");
        int CodigoActor = Convert.ToInt32(Console.ReadLine());
        if (Datos.SerieAsocia(CodigoSerie, CodigoActor))
            Console.WriteLine("\nActor asociado a la serie.");
        else
            Console.WriteLine("\nError código del actor o ya estaba asociado a la serie");
        Console.ReadKey();
    }

    //Pantalla para disociar actor de alguna serie
    public void SerieDisocia() {
        List<string> ListaActores;

        Console.WriteLine("\t === Disociar actor de la serie ===");
        Console.Write("¿Cuál serie? Número[ ]: ");
        int CodigoSerie = Convert.ToInt32(Console.ReadLine());
        int Pos = Datos.PosSerie(CodigoSerie);
        if (Pos >= 0) {
            ListaActores = Datos.SerieActores(CodigoSerie);
            for (int cont = 0; cont < ListaActores.Count; cont++)
                Console.WriteLine(ListaActores[cont]);

            Console.Write("¿Cuál actor quiere quitar? Número[ ]: ");
            int CodigoActor = Convert.ToInt32(Console.ReadLine());

            if (Datos.SerieDisocia(CodigoSerie, CodigoActor) == true)
                Console.WriteLine("\nActor retirado de la serie.");
            else
                Console.WriteLine("\nError retirando actor de la serie");
        }
        else
            Console.WriteLine("Error en el código de la serie");
    }

```

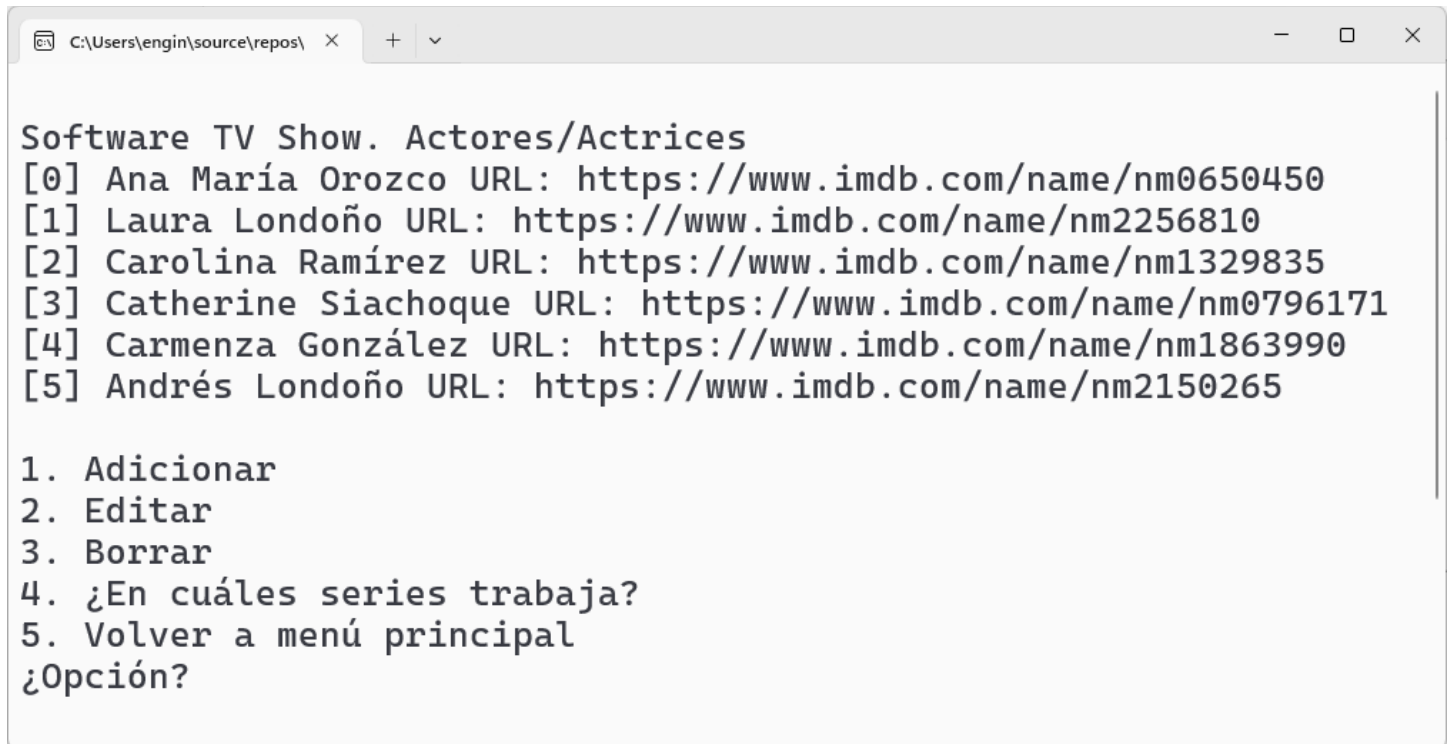
```

        Console.ReadKey();
    }
}

class Program {
    static void Main() {
        //Se debe llamar primero la capa de persistencia
        //(carga datos de ejemplo)
        Persistencia objDatos = new();

        //Luego se llama la capa visual
        Visual objVisual = new(objDatos);
        objVisual.Menu();
    }
}

```



The screenshot shows a Windows command prompt window with the title bar "C:\Users\engin\source\repos\ X". The window displays the following text:

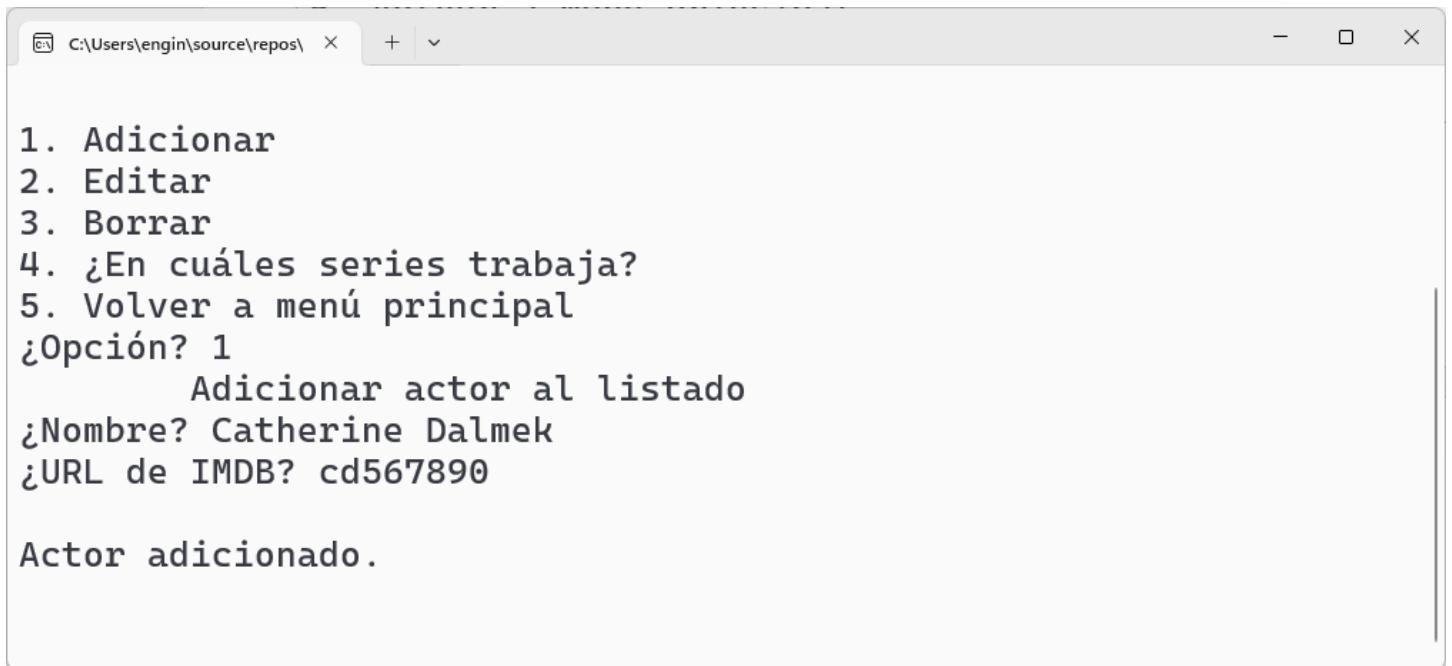
```

Software TV Show. Actores/Actrices
[0] Ana María Orozco URL: https://www.imdb.com/name/nm0650450
[1] Laura Londoño URL: https://www.imdb.com/name/nm2256810
[2] Carolina Ramírez URL: https://www.imdb.com/name/nm1329835
[3] Catherine Siachoque URL: https://www.imdb.com/name/nm0796171
[4] Carmenza González URL: https://www.imdb.com/name/nm1863990
[5] Andrés Londoño URL: https://www.imdb.com/name/nm2150265

1. Adicionar
2. Editar
3. Borrar
4. ¿En cuáles series trabaja?
5. Volver a menú principal
¿Opción?

```

Ilustración 12: Uso de listas para simular un sistema de información



A screenshot of a terminal window with a light gray background. The window has a title bar at the top with a file icon, the path 'C:\Users\engin\source\repos\', and standard window controls (minimize, maximize, close). The terminal content is as follows:

```
1. Adicionar
2. Editar
3. Borrar
4. ¿En cuáles series trabaja?
5. Volver a menú principal
¿Opción? 1
        Adicionar actor al listado
¿Nombre? Catherine Dalmek
¿URL de IMDB? cd567890

Actor adicionado.
```

*Ilustración 13: Uso de listas para simular un sistema de información*

# Dictionary

## Uso de llaves

En una estructura diccionario, hay una llave y un valor (entero, cadena, objeto). Se puede llegar a ese valor usando la llave. Con la instrucción: NombreDiccionario[Llave]. Ejemplo:

E/021.cs

```
namespace Ejemplo;

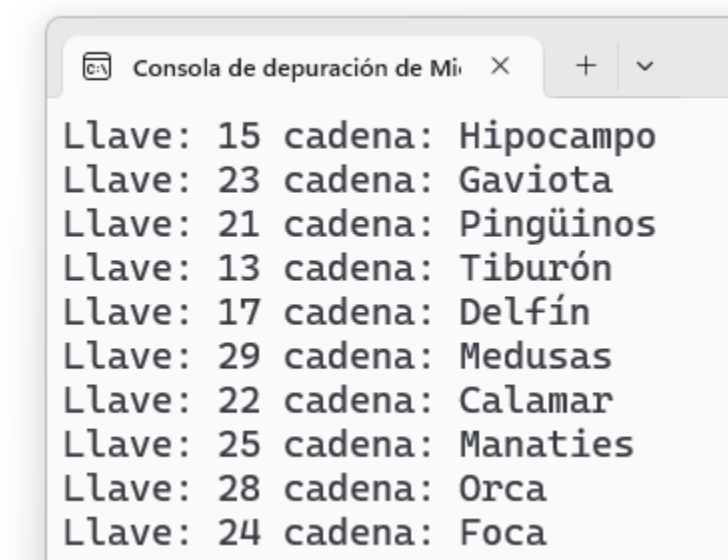
class Program {
    static void Main() {
        Random Azar = new();

        //Se define un diccionario: llave, cadena
        //En este caso la llave es un número entero
        Dictionary<int, string> Animales = new() {
            {11, "Ballena"},
            {12, "Tortuga marina"},
            {13, "Tiburón"},
            {14, "Estrella de mar"},
            {15, "Hipocampo"},
            {16, "Serpiente marina"},
            {17, "Delfín"},
            {18, "Pulpo"},
            {19, "Caballito de mar"},
            {20, "Coral"},
            {21, "Pingüinos"},
            {22, "Calamar"},
            {23, "Gaviota"},
            {24, "Foca"},
            {25, "Manatíes"},
            {26, "Ballena con barba"},
            {27, "Peces Guppy"},
            {28, "Orca"},
            {29, "Medusas"},
            {30, "Mejillones"},
            {31, "Caracoles"}
        };

        for (int cont = 1; cont <= 10; cont++) {
            int Llave = Azar.Next(11, Animales.Count + 11);
            Console.Write("Llave: " + Llave);
            Console.WriteLine(" cadena: " + Animales[Llave]);
        }
    }
}
```



```
}  
}
```



*Ilustración 14: Dictionary*

## Llaves tipo string

También se puede usar una llave de tipo cadena. El diccionario tiene instrucciones de adicionar y borrar.

E/022.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Se define un diccionario: llave, cadena
        //En este caso la llave es una cadena
        Dictionary<string, string> Extension = new() {
            {"exe", "Ejecutable"},
            {"com", "Ejecutable DOS"},
            {"vb", "Visual Basic .NET"},
            {"cs", "C#"},
            {"js", "JavaScript"},
            {"xlsx", "Excel"},
            {"docx", "Word"},
            {"html", "HTML 5"}
        };

        //Otra forma de adicionar
        Extension.Add("pptx", "PowerPoint");

        //Trae un elemento dada una llave
        string Llave = "cs";
        Console.Write("Llave: " + Llave);
        Console.WriteLine(" valor es: " + Extension[Llave]);

        //Tamaño del diccionario
        Console.WriteLine("Tamaño: " + Extension.Count);

        //Elimina un elemento
        Extension.Remove("docx");

        //Tamaño del diccionario
        Console.WriteLine("Después de eliminar: " + Extension.Count);
    }
}
```

```
Llave: cs valor es: C#  
Tamaño: 9  
Después de eliminar: 8
```

*Ilustración 15: Dictionary*

## Manejo de objetos en un Dictionary

Un "Dictionary" puede albergar objetos. Además, tiene una serie de métodos (adicionar, consultar, listar llaves, verificar si existe llave) que se ven a continuación:

E/023.cs

```
namespace Ejemplo;

//Una clase con varios atributos
class MiClase {
    public int Numero { get; set; }
    public double Valor { get; set; }
    public char Car { get; set; }
    public string Cad { get; set; }

    public MiClase(int Numero, double Valor, char Car, string Cad) {
        this.Numero = Numero;
        this.Valor = Valor;
        this.Car = Car;
        this.Cad = Cad;
    }
}

class Program {
    static void Main() {
        //Se define un diccionario: llave, objeto
        //En este caso la llave es una cadena
        var Objetos = new Dictionary<string, MiClase> {
            {"uno", new MiClase(1, 0.2, 'r', "Leafar") },
            {"dos", new MiClase(8, -7.1, 'a', "Otrebla")},
            {"tres", new MiClase(23, -13.6, 'm', "Onerom")},
            {"cuatro", new MiClase(49, 16.83, 'p', "Arrap")}
        };

        //Trae los datos del objeto guardado en el diccionario
        string Llave = "tres";
        Console.Write("Llave: " + Llave);
        Console.WriteLine(" atributo es: " + Objetos[Llave].Cad);

        Console.Write("Llave: " + Llave);
        Console.WriteLine(" atributo es: " + Objetos[Llave].Numero);

        Console.Write("Llave: " + Llave);
        Console.WriteLine(" atributo es: " + Objetos[Llave].Valor);

        //Guarda las llaves en una lista
        Console.WriteLine("\r\nLista de Llaves:");
        var ListaLlaves = new List<string>(Objetos.Keys);
    }
}
```

```

foreach (string Llaves in ListaLlaves) {
    Console.WriteLine("Llave: " + Llaves);
}

//Verifica si existe una llave
Console.WriteLine("\r\nVerifica si existe una llave:");
if (Objetos.ContainsKey("cuatro")) {
    Console.WriteLine(Objetos["cuatro"].Cad);
}
else {
    Console.WriteLine("No existe esa llave");
}
}
}

```

```

Llave: tres atributo es: Onerom
Llave: tres atributo es: 23
Llave: tres atributo es: -13.6

Lista de Llaves:
Llave: uno
Llave: dos
Llave: tres
Llave: cuatro

Verifica si existe una llave:
Arrap

```

*Ilustración 16: Dictionary, manejo de objetos*

## Queue (Cola)

Una cola se parece a un List, la diferencia es que NO se puede acceder a los elementos por un índice, se respeta el orden de llegada, primero en entrar es primero en salir.

E/024.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Se define una cola de tipo string: Queue
        Queue<string> Cola = new();

        //Se agregan elementos a la cola
        Cola.Enqueue("aaa");
        Cola.Enqueue("bbb");
        Cola.Enqueue("ccc");
        Cola.Enqueue("ddd");
        Cola.Enqueue("eee");
        Cola.Enqueue("fff");

        //Número de elementos en la cola
        Console.WriteLine("Número de elementos: " + Cola.Count);

        //Imprimir la cola
        Console.WriteLine("\r\nElementos: ");
        foreach (object elemento in Cola)
            Console.Write(elemento + ", ");

        //Quitar elemento de la cola.
        //Primero en llegar, primero en salir, luego quitaría a "aaa"
        Cola.Dequeue();
        Console.WriteLine("\r\nQuitar un elemento de la cola: ");
        foreach (object elemento in Cola)
            Console.Write(elemento + ", ");

        //Verificar si hay un elemento en la cola
        string Buscar = "ddd";
        if (Cola.Contains(Buscar) == true) {
            Console.WriteLine("\r\n\r\nLa cola contiene: " + Buscar);
        }
        else
            Console.WriteLine("\r\n\r\nLa cola NO contiene: " + Buscar);

        //Obtener el primer elemento de la cola
```

```

//sin borrar ese elemento
string PrimerElemento = Convert.ToString(Cola.Peek());
Console.WriteLine("\r\nPrimer elemento: " + PrimerElemento);

//Leer y borrar la cola
Console.WriteLine("\r\nLee y borra la cola: ");
while (Cola.Count > 0)
    Console.Write(Cola.Dequeue() + "; ");
Console.WriteLine("\r\nNúmero de elementos: " + Cola.Count);
}
}

```

```

Número de elementos: 6

Elementos:
aaa, bbb, ccc, ddd, eee, fff,
Quitar un elemento de la cola:
bbb, ccc, ddd, eee, fff,

La cola contiene: ddd

Primer elemento: bbb

Lee y borra la cola:
bbb; ccc; ddd; eee; fff;
Número de elementos: 0

```

*Ilustración 17: Dato definido en la cola*

## Objetos en la cola

Una cola puede tener objetos personalizados.

E/025.cs

```
namespace Ejemplo;

//Una clase con varios atributos
class MiClase {
    public int Numero { get; set; }
    public double Valor { get; set; }
    public char Car { get; set; }
    public string Cad { get; set; }

    public MiClase(int Numero, double Valor, char Car, string Cad) {
        this.Numero = Numero;
        this.Valor = Valor;
        this.Car = Car;
        this.Cad = Cad;
    }
}

class Program {
    static void Main() {
        //Se define una cola de tipo objeto personalizado
        Queue<MiClase> Cola = new();

        //Se agregan elementos a la cola
        Cola.Enqueue(new MiClase(1, 0.2, 'r', "Leafar"));
        Cola.Enqueue(new MiClase(8, -7.1, 'a', "Otrebla"));
        Cola.Enqueue(new MiClase(23, -13.6, 'm', "Onerom"));
        Cola.Enqueue(new MiClase(49, 16.83, 'p', "Arrap"));

        //Número de elementos en la cola
        Console.WriteLine("Número de elementos: " + Cola.Count);

        //Imprimir la cola
        Console.WriteLine("\r\nElementos: ");
        foreach (MiClase elemento in Cola)
            Console.Write(elemento.Cad + ", ");

        //Quitar elemento de la cola
        //Primero en llegar, primero en salir,
        //luego quitaría a "aaa"
        Cola.Dequeue();
        Console.WriteLine("\r\nAl quitar un elemento de la cola: ");
        foreach (MiClase elemento in Cola)
```



```

        Console.Write(elemento.Cad + ", ");

//Obtener el primer elemento de la cola
//sin borrar ese elemento
MiClase PrimerElemento = Cola.Peek();
Console.WriteLine("\r\n\r\nPrimer: " + PrimerElemento.Cad);

//Leer y borrar la cola
Console.WriteLine("\r\nLee y borra la cola: ");
while (Cola.Count > 0)
    Console.Write(Cola.Dequeue().Cad + "; ");
Console.WriteLine("\r\nNúmero de elementos: " + Cola.Count);

//Agrega elementos a la cola y luego la borra
Cola.Enqueue(new MiClase(7, 6.5, 'z', "qwerty"));
Cola.Enqueue(new MiClase(4, -3.2, 'y', "asdfg"));
Console.WriteLine("\r\nElementos: " + Cola.Count);
Cola.Clear();
Console.WriteLine("Después de borrar: " + Cola.Count);
    }
}

```

```

Número de elementos: 4

Elementos:
Leafar, Otrebla, Onerom, Arrap,
Al quitar un elemento de la cola:
Otrebla, Onerom, Arrap,

Primer: Otrebla

Lee y borra la cola:
Otrebla; Onerom; Arrap;
Número de elementos: 0

Elementos: 2
Después de borrar: 0

```

*Ilustración 18: Objetos en la cola*

## Stack (Pila)

La pila es una estructura que análogo a una pila de platos, cuando se adicionan elementos, estos van quedando encima, por lo que el último en entrar es el primero en salir. Es muy similar a la cola, sólo cambian algunos métodos como el Push (poner) y Pop (retirar).

E/026.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Se define una pila: Stack
        Stack<string> Pila = new();

        //Se agregan elementos a la pila
        Pila.Push("aaa");
        Pila.Push("bbb");
        Pila.Push("ccc");
        Pila.Push("ddd");
        Pila.Push("eee");
        Pila.Push("fff");

        //Número de elementos en la pila
        Console.WriteLine("Número de elementos: " + Pila.Count);

        //Imprimir la pila
        Console.WriteLine("\r\nElementos: ");
        foreach (object elemento in Pila)
            Console.Write(elemento + ", ");

        //Quitar elemento de la pila
        //Último en llegar, primero en salir,
        //luego quitaría a "fff"
        Pila.Pop();
        Console.WriteLine("\r\nAl quitar un elemento de la pila: ");
        foreach (string elemento in Pila)
            Console.Write(elemento + ", ");

        //Verificar si hay un elemento en la pila
        string Buscar = "ddd";
        if (Pila.Contains(Buscar) == true) {
            Console.WriteLine("\r\n\r\nLa pila contiene: " + Buscar);
        }
        else
            Console.WriteLine("\r\n\r\nLa pila NO contiene: " + Buscar);
    }
}
```

```

//Obtener el primer elemento de
//la pila sin borrar ese elemento
string PrimerElemento = Pila.Peek();
Console.WriteLine("\r\nPrimer elemento: " + PrimerElemento);

//Leer y borrar la pila
Console.WriteLine("\r\nLee y borra la pila: ");
while (Pila.Count > 0)
    Console.Write(Pila.Pop() + "; ");
Console.WriteLine("\r\nNúmero de elementos: " + Pila.Count);
}
}

```

Número de elementos: 6

Elementos:

fff, eee, ddd, ccc, bbb, aaa,

Al quitar un elemento de la pila:

eee, ddd, ccc, bbb, aaa,

La pila contiene: ddd

Primer elemento: eee

Lee y borra la pila:

eee; ddd; ccc; bbb; aaa;

Número de elementos: 0

*Ilustración 19: Dato definido en la pila*

```
namespace Ejemplo;

//Una clase con varios atributos
class MiClase {
    public int Numero { get; set; }
    public double Valor { get; set; }
    public char Car { get; set; }
    public string Cad { get; set; }

    public MiClase(int Numero, double Valor, char Car, string Cad) {
        this.Numero = Numero;
        this.Valor = Valor;
        this.Car = Car;
        this.Cad = Cad;
    }
}

class Program {
    static void Main() {
        //Se define una pila de tipo objeto personalizado
        Stack<MiClase> Pila = new();

        //Se agregan elementos a la pila
        Pila.Push(new MiClase(1, 0.2, 'r', "Leafar"));
        Pila.Push(new MiClase(8, -7.1, 'a', "Otrebla"));
        Pila.Push(new MiClase(23, -13.6, 'm', "Onerom"));
        Pila.Push(new MiClase(49, 16.83, 'p', "Arrap"));

        //Número de elementos en la pila
        Console.WriteLine("Número de elementos: " + Pila.Count);

        //Imprimir la pila
        Console.WriteLine("\r\nElementos: ");
        foreach (MiClase elemento in Pila)
            Console.Write(elemento.Cad + ", ");

        //Quitar elemento de la pila
        Pila.Pop(); //Último en llegar, primero en salir
        Console.WriteLine("\r\nAl quitar un elemento de la pila: ");
        foreach (MiClase elemento in Pila)
            Console.Write(elemento.Cad + ", ");

        //Obtener el primer elemento de la pila
        //sin borrar ese elemento
        MiClase Primer = Pila.Peek();
    }
}
```

```

Console.WriteLine("\r\n\r\nElemento más arriba: " + Primer.Cad);

//Leer y borrar la pila
Console.WriteLine("\r\nLee y borra la pila: ");
while (Pila.Count > 0)
    Console.Write(Pila.Pop().Cad + "; ");
Console.WriteLine("\r\nNúmero de elementos: " + Pila.Count);

//Agrega elementos a la pila y luego la borra
Pila.Push(new MiClase(7, 6.5, 'z', "qwerty"));
Pila.Push(new MiClase(4, -3.2, 'y', "asdfg"));
Console.WriteLine("\r\nWlementos: " + Pila.Count);
Pila.Clear();
Console.WriteLine("Después de borrar: " + Pila.Count);
}
}

```

```

Número de elementos: 4

Elementos:
Arrap, Onerom, Otrebla, Leafar,
Al quitar un elemento de la pila:
Onerom, Otrebla, Leafar,

Elemento más arriba: Onerom

Lee y borra la pila:
Onerom; Otrebla; Leafar;
Número de elementos: 0

Wlementos: 2
Después de borrar: 0

```

*Ilustración 20: Objetos en la pila*

## SortedList

SortedList es muy similar a Dictionary, en este caso la lista es ordenada automáticamente por las llaves. Eso es visible al imprimirla.

E/028.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Se define una lista ordenada: llave, cadena
        //En este caso la llave es una cadena
        SortedList<string, string> Extensiones = new() {
            { "exe", "Ejecutable" },
            { "com", "Ejecutable DOS" },
            { "vb", "Visual Basic .NET" },
            { "cs", "C#" },
            { "js", "JavaScript" },
            { "xlsx", "Excel" },
            { "docx", "Word" },
            { "pptx", "PowerPoint" }
        };

        //Imprime la lista ordenada
        foreach (object elemento in Extensiones)
            Console.WriteLine(elemento);

        //Otra forma de adicionar
        Extensiones.Add("html", "HTML 5");

        //Imprime llave y valor
        var ListaLlaves = Extensiones.Keys;
        Console.WriteLine("\r\nImprime llave y valor en separado");
        foreach (string Llave in ListaLlaves) {
            Console.WriteLine("Llave: " + Llave);
            Console.WriteLine(" Valor: " + Extensiones[Llave]);
        }
    }
}
```

```
[com, Ejecutable DOS]
[cs, C#]
[docx, Word]
[exe, Ejecutable]
[html, HTML 5]
[js, JavaScript]
[pptx, PowerPoint]
[vb, Visual Basic .NET]
[xlsx, Excel]
```

Imprime llave y valor en separado

Llave: com Valor: Ejecutable DOS

Llave: cs Valor: C#

Llave: docx Valor: Word

Llave: exe Valor: Ejecutable

Llave: html Valor: HTML 5

Llave: js Valor: JavaScript

Llave: pptx Valor: PowerPoint

Llave: vb Valor: Visual Basic .NET

Llave: xlsx Valor: Excel

*Ilustración 21: SortedList*

# LinkedList

Lista enlazada, no se accede directamente por un índice.

E/029.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Se define una lista enlazada
        LinkedList<string> Lenguajes = new();

        //Agrega al final
        Console.WriteLine("Agregando con AddLast");
        Lenguajes.AddLast("Visual Basic .NET");
        Lenguajes.AddLast("F#");
        Lenguajes.AddLast("C#");
        Lenguajes.AddLast("TypeScript");

        //Imprime esa lista
        foreach (string elemento in Lenguajes)
            Console.Write(elemento + "; ");

        //Agrega al inicio
        Console.WriteLine("\r\n\r\nAgregando con AddFirst");
        Lenguajes.AddFirst("C++");
        Lenguajes.AddFirst("C");

        //Imprime esa lista
        foreach (string elemento in Lenguajes)
            Console.Write(elemento + "; ");

        //Agrega al final
        Lenguajes.AddLast("Python");
        Console.WriteLine("\r\n\r\nAgregando con AddLast");
        foreach (string elemento in Lenguajes)
            Console.Write(elemento + "; ");

        //Cantidad
        Console.WriteLine("\r\n\r\nCantidad es: " + Lenguajes.Count);

        //Elimina primer elemento
        Lenguajes.RemoveFirst();
        Console.WriteLine("\r\nEliminado el primer elemento");
        foreach (string elemento in Lenguajes)
            Console.Write(elemento + "; ");
    }
}
```



```

//Elimina último elemento
Lenguajes.RemoveLast();
Console.WriteLine("\r\n\r\nEliminado el último elemento");
foreach (string elemento in Lenguajes)
    Console.Write(elemento + "; ");

//Elimina determinado elemento
Lenguajes.Remove("F#");
Console.WriteLine("\r\n\r\nEliminado F#");
foreach (string elemento in Lenguajes)
    Console.Write(elemento + "; ");

//Adiciona antes de C#
//Busca el nodo que tiene C#
LinkedListNode<string> nodoPosiciona = Lenguajes.Find("C#");
Lenguajes.AddBefore(nodoPosiciona, "Assembler");
Console.WriteLine("\r\n\r\nAdiciona antes de C#");
foreach (string elemento in Lenguajes)
    Console.Write(elemento + "; ");

//Adiciona después de C#
Lenguajes.AddAfter(nodoPosiciona, "Ada");
Console.WriteLine("\r\n\r\nAdiciona después de C#");
foreach (string elemento in Lenguajes)
    Console.Write(elemento + "; ");
}
}

```

```
Agregando con AddLast
Visual Basic .NET; F#; C#; TypeScript;

Agregando con AddFirst
C; C++; Visual Basic .NET; F#; C#; TypeScript;

Agregando con AddLast
C; C++; Visual Basic .NET; F#; C#; TypeScript; Python;

Cantidad es: 7

Eliminado el primer elemento
C++; Visual Basic .NET; F#; C#; TypeScript; Python;

Eliminado el último elemento
C++; Visual Basic .NET; F#; C#; TypeScript;

Eliminado F#
C++; Visual Basic .NET; C#; TypeScript;

Adiciona antes de C#
C++; Visual Basic .NET; Assembler; C#; TypeScript;

Adiciona después de C#
C++; Visual Basic .NET; Assembler; C#; Ada; TypeScript;
```

*Ilustración 22: LinkedList*

```
namespace Ejemplo;

//Una clase con varios atributos
class MiClase {
    public int Numero { get; set; }
    public double Valor { get; set; }
    public char Car { get; set; }
    public string Cad { get; set; }

    public MiClase(int Numero, double Valor, char Car, string Cad) {
        this.Numero = Numero;
        this.Valor = Valor;
        this.Car = Car;
        this.Cad = Cad;
    }
}

class Program {
    static void Main() {
        //Se define una lista enlazada
        LinkedList<MiClase> Lenguajes = new();

        //Agrega al final
        Console.WriteLine("Agregando con AddLast");
        Lenguajes.AddLast(new MiClase(16, 83.29, 'R', "Lenguaje R"));
        Lenguajes.AddLast(new MiClase(29, 89.7, 'A', "ADA"));
        Lenguajes.AddLast(new MiClase(2, 80.19, 'M', "Máquina"));
        Lenguajes.AddLast(new MiClase(95, 7.21, 'P', "PHP"));

        //Imprime esa lista
        foreach (MiClase elemento in Lenguajes)
            Console.Write(elemento.Cad + "; ");

        //Agrega al inicio
        Lenguajes.AddFirst(new MiClase(78, 12.32, 'S', "C#"));
        Lenguajes.AddFirst(new MiClase(5, -3.1, 'V', "J#"));

        //Imprime esa lista
        Console.WriteLine("\r\n\r\nAgregando con AddFirst");
        foreach (MiClase elemento in Lenguajes)
            Console.Write(elemento.Cad + "; ");

        //Agrega al final
```

```

Lenguajes.AddLast(new MiClase(16, 83.29, 'C', "C++"));
Console.WriteLine("\r\n\r\nAgregando con AddLast");
foreach (MiClase elemento in Lenguajes)
    Console.Write(elemento.Cad + "; ");

//Cantidad
Console.WriteLine("\r\n\r\nCantidad es: " + Lenguajes.Count);

//Elimina primer elemento
Lenguajes.RemoveFirst();
Console.WriteLine("\r\n\r\nEliminado el primer elemento");
foreach (MiClase elemento in Lenguajes)
    Console.Write(elemento.Cad + "; ");

//Elimina último elemento
Lenguajes.RemoveLast();
Console.WriteLine("\r\n\r\nEliminado el último elemento");
foreach (MiClase elemento in Lenguajes)
    Console.Write(elemento.Cad + "; ");
}
}

```

```

Agregando con AddLast
Lenguaje R; ADA; Máquina; PHP;

Agregando con AddFirst
J#; C#; Lenguaje R; ADA; Máquina; PHP;

Agregando con AddLast
J#; C#; Lenguaje R; ADA; Máquina; PHP; C++;

Cantidad es: 7

Eliminado el primer elemento
C#; Lenguaje R; ADA; Máquina; PHP; C++;

Eliminado el último elemento
C#; Lenguaje R; ADA; Máquina; PHP;

```

*Ilustración 23: Objetos en LinkedList*

# SortedSet

Ordenar los elementos y eliminar los repetidos.

E/031.cs

```
namespace Ejemplo;

class Program {
    static void Main() {

        //Ordena y elimina duplicados (enteros)
        SortedSet<int> numeros = [1, 6, 8, 3, 2, 9, 2, 9];

        foreach (var n in numeros)
            Console.Write(n + ", ");

        Console.WriteLine(" ");

        //Ordena y elimina duplicados (char)
        SortedSet<char> letras = ['e', 's', 't', 'a', 'e', 's', 'u', 'n',
'a', 'p', 'r', 'u', 'e', 'b', 'a'];

        foreach (var l in letras)
            Console.Write(l + ", ");
    }
}
```

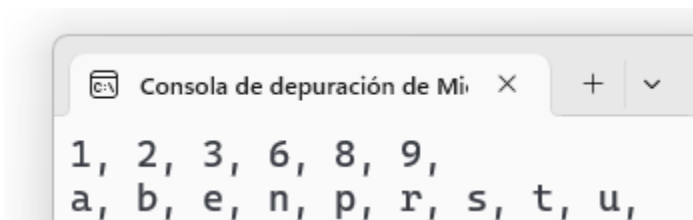


Ilustración 24: SortedSet

# PriorityQueue

Cola con prioridad. El segundo argumento es la prioridad.

E/032.cs

```
namespace Ejemplo;

class Program {
    static void Main() {

        // Crea una PriorityQueue con elementos de cadena y prioridades tipo
        DateTime
        var pq = new PriorityQueue<string, DateTime>();

        // Pone las tareas
        pq.Enqueue("Tarea 1", new DateTime(2026, 03, 25));
        pq.Enqueue("Tarea 2", new DateTime(2026, 01, 15));
        pq.Enqueue("Tarea 3", new DateTime(2026, 01, 24));
        pq.Enqueue("Tarea 4", new DateTime(2026, 04, 19));
        pq.Enqueue("Tarea 5", new DateTime(2026, 02, 06));
        pq.Enqueue("Tarea 6", new DateTime(2026, 02, 18));

        Console.WriteLine("Ordena las tareas según fecha");
        while (pq.Count > 0) {
            pq.TryDequeue(out string Tarea, out DateTime FechaTermina);
            Console.WriteLine($"{Tarea} - Fecha:
{FechaTermina.ToShortDateString()}");
        }
    }
}
```

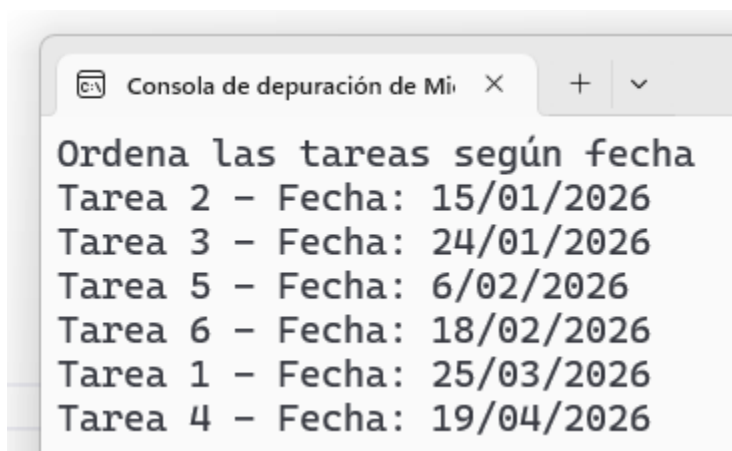


Ilustración 25: Cola con prioridad

# SortedDictionary

Estructura que ordena según el valor del primer parámetro.

E/033.cs

```
namespace Ejemplo;

class Program {
    static void Main() {

        SortedDictionary<int, string> DiccionarioOrdenado = [];

        //Agrega elementos al SortedDictionary
        DiccionarioOrdenado.Add(3, "Koala");
        DiccionarioOrdenado.Add(1, "Tiburón");
        DiccionarioOrdenado.Add(7, "Calamar");
        DiccionarioOrdenado.Add(2, "Arenque");
        DiccionarioOrdenado.Add(5, "Cangrejo");
        DiccionarioOrdenado.Add(4, "Estrella de mar");

        foreach (var Dato in DiccionarioOrdenado) {
            Console.WriteLine($"Key: {Dato.Key}, Value: {Dato.Value}");
        }
    }
}
```

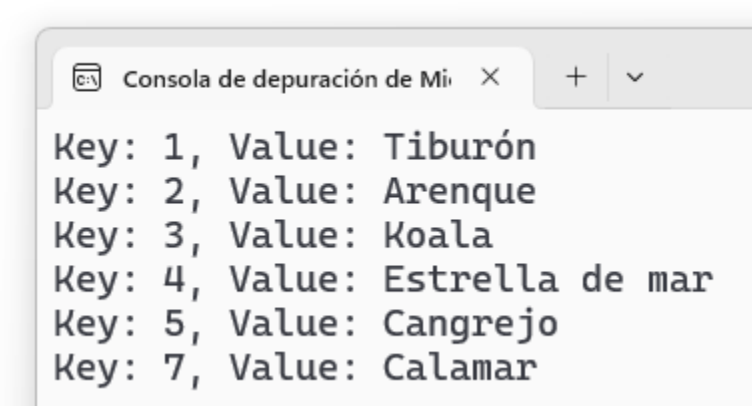


Ilustración 26: SortedDictionary

# Persistencia

Almacenar los registros de un List en un archivo y volverlos a recuperar.

E/034.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Declara la lista
        List<string> ListaAnimales = [];

        //Adiciona elementos a la lista
        ListaAnimales.Add("Ballena");
        ListaAnimales.Add("Tortuga marina");
        ListaAnimales.Add("Tiburón");
        ListaAnimales.Add("Hipocampo");
        ListaAnimales.Add("Delfín");
        ListaAnimales.Add("Pulpo");
        ListaAnimales.Add("Caballito de mar");
        ListaAnimales.Add("Coral");
        ListaAnimales.Add("Pingüinos");

        //Imprime la lista
        Console.WriteLine("LISTA ORIGINAL");
        for (int Cont = 0; Cont < ListaAnimales.Count; Cont++)
            Console.Write(ListaAnimales[Cont] + ";");
        Console.WriteLine("\r\n\r\n");

        //Guarda en medio persistente
        using (StreamWriter Escritor = new("MisDatos.txt")) {
            foreach (object item in ListaAnimales) {
                Escritor.WriteLine(item.ToString());
            }
        }

        //Lee de ese medio persistente y lo guarda
        //en un nuevo List
        List<string> NuevaLista = [];
        using (StreamReader Lector = new("MisDatos.txt")) {
            string Linea;
            while ((Linea = Lector.ReadLine()) != null) {
                NuevaLista.Add(Linea);
            }
        }

        //Imprime la lista leída
    }
}
```



```

    Console.WriteLine("LISTA LEIDA");
    for (int Cont = 0; Cont < NuevaLista.Count; Cont++)
        Console.Write(NuevaLista[Cont] + ";");
    Console.WriteLine("\r\n\r\n");
}
}

```

E/035.cs

```

namespace Ejemplo;

class Program {
    static void Main() {
        List<double> datos = [3.14, -2.71, 1.618, -4.31, 7.89];

        // Guardar como archivo texto plano
        File.WriteAllText("datos.txt", string.Join(";", datos));

        // Leer desde archivo texto plano
        string contenido = File.ReadAllText("datos.txt");
        List<double> datosLeidos = [];
        foreach (var s in contenido.Split(';')) {
            datosLeidos.Add(double.Parse(s));
        }

        //Imprime los datos leídos
        for (int Cont = 0; Cont < datosLeidos.Count; Cont++) {
            Console.WriteLine(datosLeidos[Cont]);
        }
    }
}

```

```
using System.Text.Json;

namespace Ejemplo;

class Program {
    static void Main() {
        List<double> datos = [3.14, -2.71, 1.618, -4.31, 7.89];

        // Guardar como JSON
        string json = JsonSerializer.Serialize(datos);
        File.WriteAllText("datos.json", json);

        // Leer desde JSON
        string jsonLeido = File.ReadAllText("datos.json");
        List<double> datosLeidos =
        JsonSerializer.Deserialize<List<double>>(jsonLeido);

        //Imprime los datos leídos
        for (int Cont = 0; Cont < datosLeidos.Count; Cont++) {
            Console.WriteLine(datosLeidos[Cont]);
        }
    }
}
```

```
using System.Text.Json;

namespace Ejemplo;

class MiClase {
    public int Entero { get; set; }
    public double Real { get; set; }
    public char Caracter { get; set; }
    public bool Booleano { get; set; }
    public string Cadena { get; set; }
}

class Program {
    static void Main() {
        // Crear lista de objetos
        List<MiClase> lista =
        [
            new MiClase { Entero = -1, Real = 3.1416, Caracter = 'A', Booleano =
true, Cadena = "Kakapu" },
            new MiClase { Entero = 38, Real = -2.7164, Caracter = 'K', Booleano =
false, Cadena = "Ballena" },
            new MiClase { Entero = -16, Real = 1.67, Caracter = 'M', Booleano =
false, Cadena = "Ciervo" },
            new MiClase { Entero = 49, Real = -6.112, Caracter = 'R', Booleano =
false, Cadena = "CÓndor" },
            new MiClase { Entero = -83, Real = 9.4676, Caracter = 'T', Booleano =
true, Cadena = "Águila" },
            new MiClase { Entero = 6, Real = -2.6774, Caracter = 'D', Booleano =
false, Cadena = "Gato" },
            new MiClase { Entero = -29, Real = 7.255, Caracter = 'X', Booleano =
true, Cadena = "Perro" },
            new MiClase { Entero = 72, Real = -5.23765, Caracter = 'L', Booleano
= false, Cadena = "Leopardo" },
        ];

        // Serializar a JSON
        string json = JsonSerializer.Serialize(lista, new
JsonSerializerOptions { WriteIndented = true });

        // Guardar en archivo
        File.WriteAllText("datos.json", json);
        Console.WriteLine("Archivo JSON guardado correctamente.");
    }
}
```

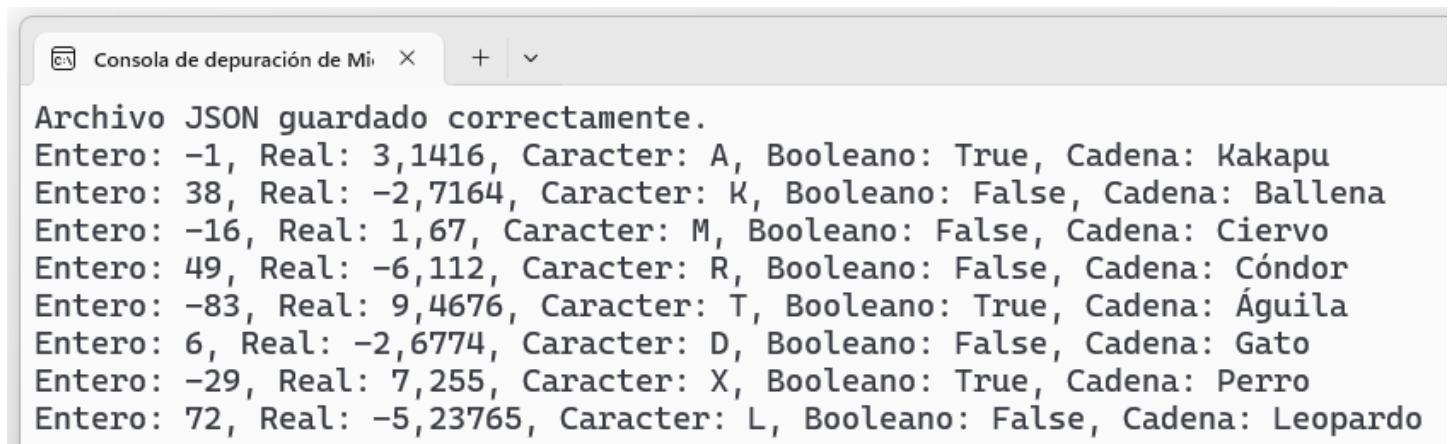
```

// Leer el archivo JSON
string jsonLeido = File.ReadAllText("datos.json");

// Deserializar a lista de objetos
List<MiClase> listaRecuperada =
JsonSerializer.Deserialize<List<MiClase>>(jsonLeido);

// Mostrar los datos
foreach (var obj in listaRecuperada) {
    Console.WriteLine($"Entero: {obj.Entero}, Real: {obj.Real},
Caracter: {obj.Caracter}, Booleano: {obj.Booleano}, Cadena:
{obj.Cadena}");
}
}
}

```



```

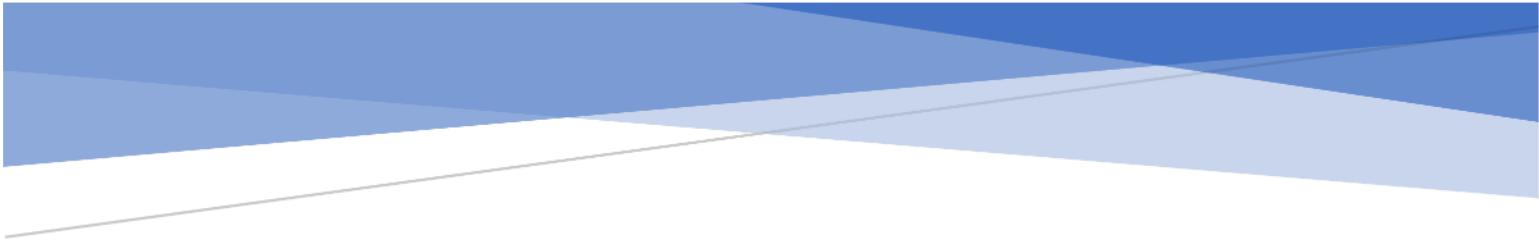
Consola de depuración de Mi
Archivo JSON guardado correctamente.
Entero: -1, Real: 3,1416, Caracter: A, Booleano: True, Cadena: Kakapu
Entero: 38, Real: -2,7164, Caracter: K, Booleano: False, Cadena: Ballena
Entero: -16, Real: 1,67, Caracter: M, Booleano: False, Cadena: Ciervo
Entero: 49, Real: -6,112, Caracter: R, Booleano: False, Cadena: Cóndor
Entero: -83, Real: 9,4676, Caracter: T, Booleano: True, Cadena: Águila
Entero: 6, Real: -2,6774, Caracter: D, Booleano: False, Cadena: Gato
Entero: -29, Real: 7,255, Caracter: X, Booleano: True, Cadena: Perro
Entero: 72, Real: -5,23765, Caracter: L, Booleano: False, Cadena: Leopardo

```

*Ilustración 27: Guardando objetos*

```
1  [
2  {
3      "Entero": -1,
4      "Real": 3.1416,
5      "Caracter": "A",
6      "Booleano": true,
7      "Cadena": "Kakapu"
8  },
9  {
10     "Entero": 38,
11     "Real": -2.7164,
12     "Caracter": "K",
13     "Booleano": false,
14     "Cadena": "Ballena"
15  },
16  {
17     "Entero": -16,
18     "Real": 1.67,
19     "Caracter": "M",
20     "Booleano": false,
21     "Cadena": "Ciervo"
22  },
23 ]
```

Ilustración 28: Datos JSON



# C# Y .NET 10

## Parte 6. LINQ y LAMBDA

# 2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                                 | 4  |
| Acerca del autor.....                                                       | 5  |
| Licencia de este libro .....                                                | 5  |
| Licencia del software .....                                                 | 5  |
| Marcas registradas .....                                                    | 6  |
| En memoria .....                                                            | 7  |
| Filtrar de un List. Ejemplo 1 .....                                         | 8  |
| Filtrar de un List. Ejemplo 2 .....                                         | 9  |
| Ordenación .....                                                            | 10 |
| Ordenación descendente .....                                                | 11 |
| Consulta con salida a texto personalizado .....                             | 12 |
| Contar los registros .....                                                  | 13 |
| Máximo, mínimo y suma .....                                                 | 14 |
| Máximo, mínimo y suma con condiciones .....                                 | 16 |
| Consulta con elementos tipo string.....                                     | 17 |
| Consulta con objetos. Ejemplo 1.....                                        | 18 |
| Consulta con objetos. Ejemplo 2.....                                        | 20 |
| Determinación de tipo de dato.....                                          | 22 |
| Ordenación por un campo y luego por otro .....                              | 24 |
| Agrupación por un campo .....                                               | 26 |
| Hacer un "join" entre listas .....                                          | 29 |
| Un "join" con resultado personalizado .....                                 | 31 |
| Extraer los datos de una lista que no están en otra .....                   | 33 |
| Intersección de dos listas .....                                            | 34 |
| Unir dos listas sin repetir elementos .....                                 | 35 |
| Consulta de texto por algún patrón .....                                    | 36 |
| Ordenar internamente una cadena .....                                       | 38 |
| Ordenar internamente una cadena con diversos caracteres alfanuméricos ..... | 39 |
| Ordenamiento según tamaño de la palabra.....                                | 41 |
| Ordenamiento por la segunda letra de cada palabra .....                     | 43 |
| Invertir el ordenamiento.....                                               | 44 |
| Métricas: Comparativa entre usar LINQ e implementación tradicional .....    | 45 |
| Expresiones LAMBDA .....                                                    | 48 |

Expresión Lambda: Uso de varias líneas..... 49

Expresión Lambda: Retornando tipo de dato..... 50

Expresión Lambda: Acceso a variable externa..... 51

Expresión Lambda: Listas. Pero no funciona. .... 52

Expresión Lambda: Listas. Ahora si funciona..... 53

Expresiones LAMBDA: Listas y variables externas no funciona ..... 54

Expresiones LAMBDA: Uso de static ..... 55

Comparativa entre LINQ y Lambda. Ejemplo 1. .... 56

Comparativa entre LINQ y Lambda. Ejemplo 2. .... 57

Comparativa entre LINQ y Lambda. Ejemplo 3. .... 59

Expresiones Lambda: Promedio ..... 61

JSON y LINQ ..... 62



## Tabla de ilustraciones

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Ilustración 1: LINQ: Filtrar de un arreglo.....                                 | 8  |
| Ilustración 2: LINQ: Filtrar y poner en un List .....                           | 9  |
| Ilustración 3: LINQ: Ordenación .....                                           | 10 |
| Ilustración 4: LINQ: Ordenación descendente .....                               | 11 |
| Ilustración 5: Consulta con salida a texto personalizado .....                  | 12 |
| Ilustración 6: LINQ: Contar los registros .....                                 | 13 |
| Ilustración 7: LINQ: Máximo, mínimo y suma .....                                | 15 |
| Ilustración 8: LINQ: Máximo, mínimo y suma con condiciones .....                | 16 |
| Ilustración 9: LINQ: Consulta con elementos tipo string .....                   | 17 |
| Ilustración 10: LINQ: Consulta con objetos.....                                 | 19 |
| Ilustración 11: LINQ: Consulta con objetos y resultado en un List .....         | 21 |
| Ilustración 12: LINQ: Determinación de tipo de dato .....                       | 23 |
| Ilustración 13: LINQ: Ordenación por un campo y luego por otro .....            | 25 |
| Ilustración 14: LINQ: Agrupación por un campo .....                             | 28 |
| Ilustración 15: LINQ: Hacer un "join" entre listas .....                        | 30 |
| Ilustración 16: LINQ: Un "join" con resultado personalizado .....               | 32 |
| Ilustración 17: LINQ: Extraer los datos de una lista que no están en otra ..... | 33 |
| Ilustración 18: LINQ: Intersección de dos listas.....                           | 34 |
| Ilustración 19: LINQ: Unir dos listas sin repetir elementos .....               | 35 |
| Ilustración 20: LINQ: Consulta de texto por algún patrón .....                  | 37 |
| Ilustración 21: LINQ: Ordenar internamente una cadena .....                     | 38 |
| Ilustración 22: LINQ: Ordenar internamente una cadena .....                     | 40 |
| Ilustración 23: LINQ: Ordenamiento según tamaño de la palabra .....             | 42 |
| Ilustración 24: LINQ: Ordenamiento por la segunda letra de cada palabra .....   | 43 |
| Ilustración 25: LINQ: Invertir el ordenamiento .....                            | 44 |
| Ilustración 26: Comparativa desempeño de LINQ .....                             | 47 |
| Ilustración 27: Ejecuta expresión Lambda .....                                  | 48 |
| Ilustración 28: Expresiones Lambda usando varias líneas .....                   | 49 |
| Ilustración 29: Retornando tipo de dato .....                                   | 50 |
| Ilustración 30: Acceso a variable externa .....                                 | 51 |
| Ilustración 31: No funciona .....                                               | 52 |
| Ilustración 32: Ahora si funciona .....                                         | 53 |
| Ilustración 33: No funciona .....                                               | 54 |
| Ilustración 34: Error al tratar de usar una variable externa .....              | 55 |
| Ilustración 35: Comparativa LINQ y Lambda .....                                 | 56 |
| Ilustración 36: Uso de LINQ con y sin expresiones Lambda .....                  | 58 |
| Ilustración 37: Mismos resultados .....                                         | 60 |
| Ilustración 38: Promedio .....                                                  | 61 |
| Ilustración 39: JSON y LINQ .....                                               | 63 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

En memoria

De mi gato "Vikingo"



# Filtrar de un List. Ejemplo 1

F/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //LINQ, extrae los impares
        List<int> ListaImpares = (from numero in Lista
                                where (numero % 2) == 1
                                select numero).ToList();

        for (int Cont = 0; Cont < ListaImpares.Count; Cont++)
            Console.Write(ListaImpares[Cont] + ", ");
    }
}
```

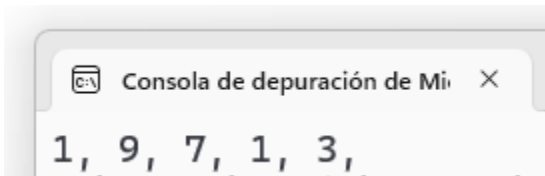


Ilustración 1: LINQ: Filtrar de un arreglo

## Filtrar de un List. Ejemplo 2

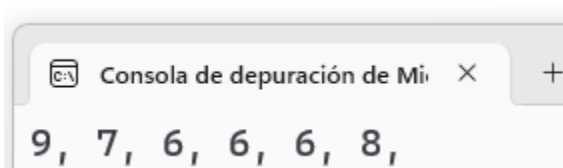
F/002.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //LINQ, extrae los mayores o iguales a 5
        List<int> ListaNumeros = (from numero in Lista
                                where numero >= 5
                                select numero).ToList();

        for (int Cont = 0; Cont < ListaNumeros.Count; Cont++)
            Console.Write(ListaNumeros[Cont] + ", ");
    }
}
```



*Ilustración 2: LINQ: Filtrar y poner en un List*

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Extrayendo solo los números pares
        //en orden ascendente
        List<int> Resultados = (from numero in Lista
                                where (numero % 2) == 0
                                orderby numero
                                select numero).ToList();

        for (int Cont = 0; Cont < Resultados.Count; Cont++)
            Console.Write(Resultados[Cont] + ", ");
    }
}
```

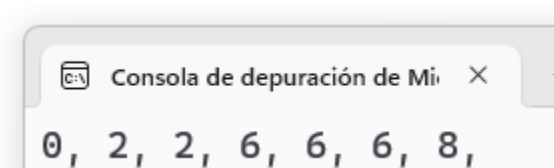


Ilustración 3: LINQ: Ordenación

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Extrayendo solo los números pares en orden
        //descendente.
        List<int> Resultados = (from numero in Lista
                                where (numero % 2) == 0
                                orderby numero descending
                                select numero).ToList();

        for (int Cont = 0; Cont < Resultados.Count; Cont++)
            Console.Write(Resultados[Cont] + ", ");
    }
}
```

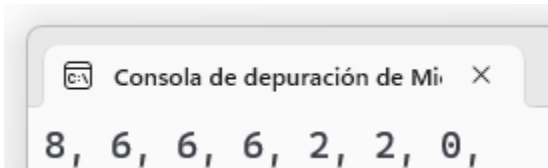


Ilustración 4: LINQ: Ordenación descendente



## Consulta con salida a texto personalizado

F/005.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Extrayendo solo los números pares en
        //orden descendente
        List<string> Resultados =
            (from numero in Lista
             where (numero % 2) == 0
             orderby numero descending
             select $"Valor es: {numero}, ").ToList();

        for (int Cont = 0; Cont < Resultados.Count; Cont++)
            Console.WriteLine(Resultados[Cont]);
    }
}
```

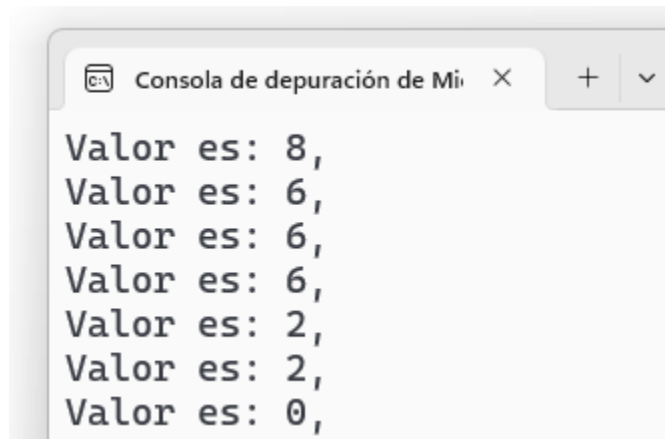


Ilustración 5: Consulta con salida a texto personalizado

# Contar los registros

F/006.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Contando los números pares
        int TotalRegistros =
            (from numero in Lista
             where (numero % 2) == 0
             select numero).Count();

        //Ejecuta la consulta y la imprime
        Console.WriteLine(TotalRegistros);
    }
}
```

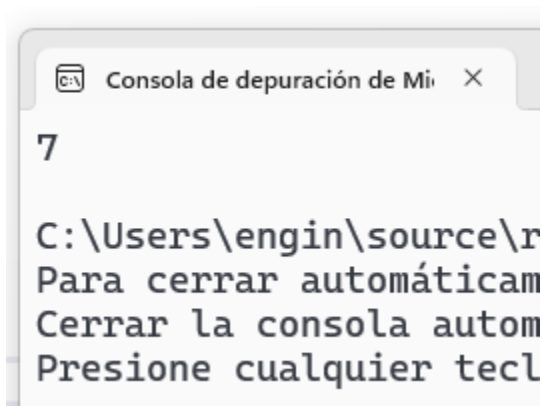


Ilustración 6: LINQ: Contar los registros

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Máximo, mínimo y suma
        int Maximo = (from numero in Lista
                      select numero).Max();

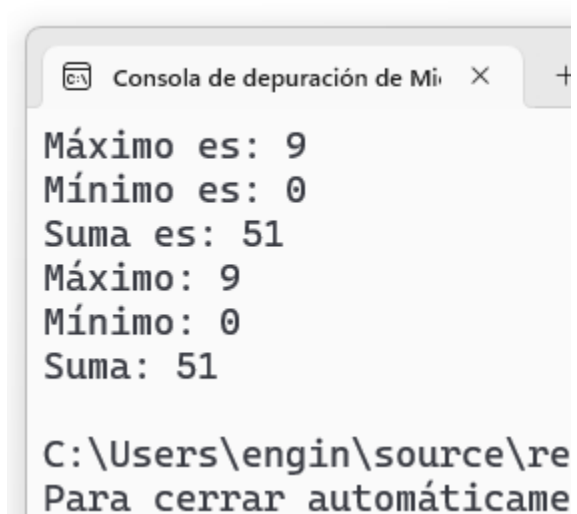
        int Minimo = (from numero in Lista
                     select numero).Min();

        int Suma = (from numero in Lista
                   select numero).Sum();

        //Ejecuta la consulta y la imprime
        Console.WriteLine("Máximo es: " + Maximo);
        Console.WriteLine("Mínimo es: " + Minimo);
        Console.WriteLine("Suma es: " + Suma);

        //Otra forma de hacerlo
        int maximo = Lista.Max();
        int minimo = Lista.Min();
        int suma = Lista.Sum();

        //Imprime
        Console.WriteLine("Máximo: " + maximo);
        Console.WriteLine("Mínimo: " + minimo);
        Console.WriteLine("Suma: " + suma);
    }
}
```



The image shows a screenshot of a Visual Studio debug console window. The title bar reads 'Consola de depuración de Mi' followed by a close button and a plus sign. The console output displays the results of a LINQ query in Spanish. It shows the maximum value ('Máximo es: 9'), the minimum value ('Mínimo es: 0'), and the sum ('Suma es: 51'). Below this, the same values are repeated without the 'es' suffix: 'Máximo: 9', 'Mínimo: 0', and 'Suma: 51'. At the bottom, a file path 'C:\Users\engin\source\re' is partially visible, followed by the text 'Para cerrar automáticamente'.

```
Consola de depuración de Mi X +  
Máximo es: 9  
Mínimo es: 0  
Suma es: 51  
Máximo: 9  
Mínimo: 0  
Suma: 51  
  
C:\Users\engin\source\re  
Para cerrar automáticamente
```

*Ilustración 7: LINQ: Máximo, mínimo y suma*

# Máximo, mínimo y suma con condiciones

F/008.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<int> Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3];

        //Consulta: Máximo y mínimo
        int Maximo = (from numero in Lista
                      where numero % 2 == 0
                      select numero).Max();

        int Minimo = (from numero in Lista
                      where numero % 2 == 0
                      select numero).Min();

        int Suma = (from numero in Lista
                    where numero % 2 == 0
                    select numero).Sum();

        //Ejecuta la consulta y la imprime
        Console.WriteLine("Máximo es: " + Maximo);
        Console.WriteLine("Mínimo es: " + Minimo);
        Console.WriteLine("Suma es: " + Suma);
    }
}
```

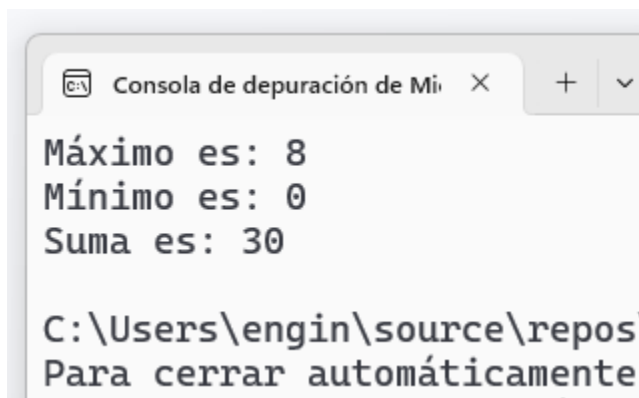


Ilustración 8: LINQ: Máximo, mínimo y suma con condiciones

## Consulta con elementos tipo string

F/009.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos
        List<string> Lista = [ "búho", "loro", "gaviota",
                               "azulejo", "bichofue", "canario" ];

        //Consulta, especies de aves que tengan
        //la letra 'a'
        int Cuenta = (from aves in Lista
                       where aves.Contains("a")
                       select aves).Count();

        //Ejecuta la consulta y la imprime
        Console.WriteLine("Aves con letra a: " + Cuenta);
    }
}
```

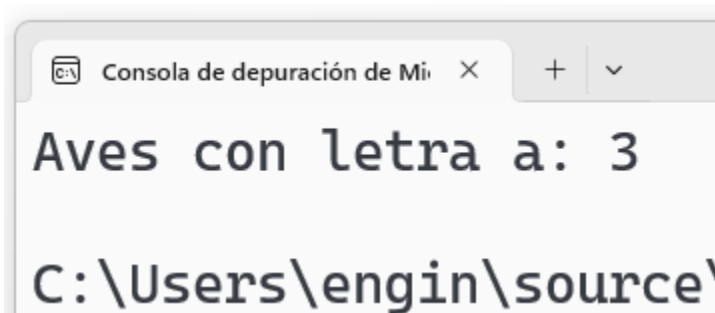


Ilustración 9: LINQ: Consulta con elementos tipo string

## Consulta con objetos. Ejemplo 1.

F/010.cs

```
namespace Ejemplo;

internal class Mascota {
    public intCodigo;
    public string Nombre;
    public int FechaNace; //Formato: aaaammdd

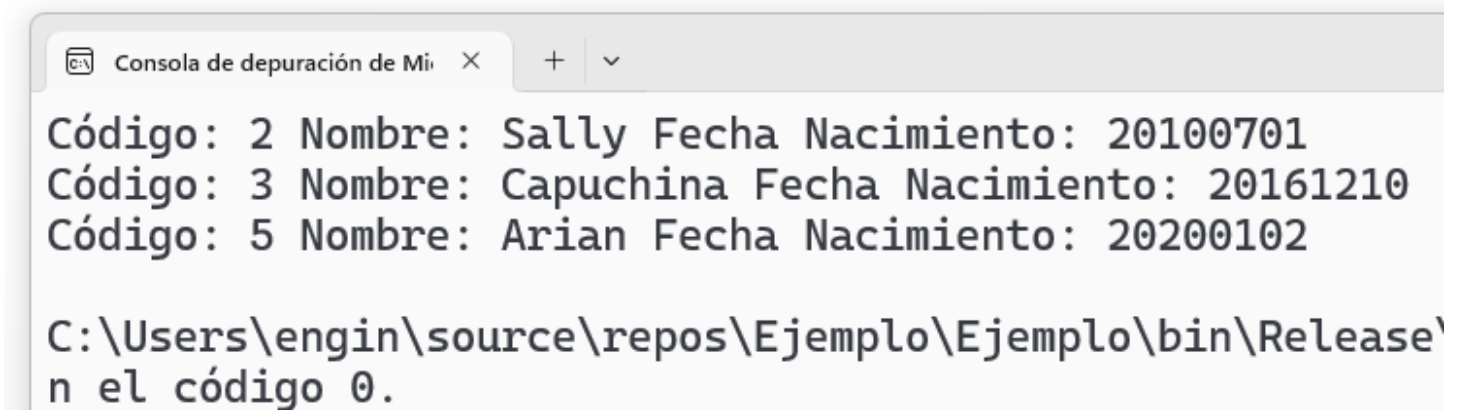
    public Mascota(intCodigo, string Nombre, int FechaNace) {
        this.Codigo = Codigo;
        this.Nombre = Nombre;
        this.FechaNace = FechaNace;
    }

    public void Imprime() {
        Console.WriteLine("Código: " + Codigo);
        Console.WriteLine(" Nombre: " + Nombre);
        Console.WriteLine(" Fecha Nacimiento: " + FechaNace);
    }
}

internal class Program {
    static void Main() {
        //Fuente de datos, una lista
        List<Mascota> listaMascotas = new List<Mascota>();
        listaMascotas.Add(new Mascota(1, "Suini", 20121012));
        listaMascotas.Add(new Mascota(2, "Sally", 20100701));
        listaMascotas.Add(new Mascota(3, "Capuchina", 20161210));
        listaMascotas.Add(new Mascota(4, "Grisú", 20161120));
        listaMascotas.Add(new Mascota(5, "Arian", 20200102));
        listaMascotas.Add(new Mascota(6, "Milú", 20160706));

        //Extraiga los registros donde el nombre tenga la letra 'a'
        List<Mascota> Resultados = (from animal in listaMascotas
                                   where animal.Nombre.Contains("a")
                                   select animal).ToList();

        //Ejecuta la consulta y la imprime
        for (int cont = 0; cont < Resultados.Count; cont++) {
            Resultados[cont].Imprime();
        }
    }
}
```



The image shows a screenshot of a Visual Studio debug console window. The title bar at the top reads 'Consola de depuración de Mi' followed by a close button (X) and a dropdown menu with a plus sign and a downward arrow. The console contains three lines of text representing query results, each on a new line: 'Código: 2 Nombre: Sally Fecha Nacimiento: 20100701', 'Código: 3 Nombre: Capuchina Fecha Nacimiento: 20161210', and 'Código: 5 Nombre: Arian Fecha Nacimiento: 20200102'. Below these, there is a line of code: 'C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Release\' followed by a line break and the text 'n el código 0.'

```
Consola de depuración de Mi X + v
Código: 2 Nombre: Sally Fecha Nacimiento: 20100701
Código: 3 Nombre: Capuchina Fecha Nacimiento: 20161210
Código: 5 Nombre: Arian Fecha Nacimiento: 20200102

C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Release'
n el código 0.
```

*Ilustración 10: LINQ: Consulta con objetos*



## Consulta con objetos. Ejemplo 2.

F/011.cs

```
namespace Ejemplo;

internal class Mascota {
    public intCodigo { get; set; }
    public string Nombre { get; set; }
    public int FechaNace { get; set; } //Formato: aaaammdd

    public Mascota(intCodigo, string Nombre, int FechaNace) {
        this.Codigo = Codigo;
        this.Nombre = Nombre;
        this.FechaNace = FechaNace;
    }

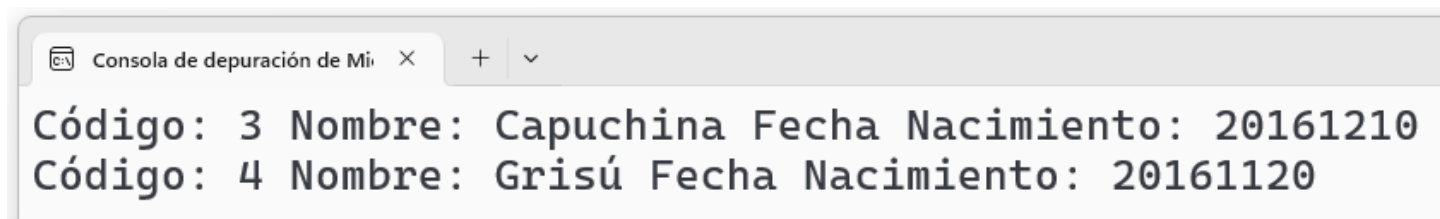
    public void Imprime() {
        Console.WriteLine("Código: " + Codigo);
        Console.WriteLine(" Nombre: " + Nombre);
        Console.WriteLine(" Fecha Nacimiento: " + FechaNace);
    }
}

internal class Program {
    static void Main() {
        //Fuente de datos, una lista
        List<Mascota> listaMascotas = new List<Mascota>();
        listaMascotas.Add(new Mascota(1, "Suini", 20121012));
        listaMascotas.Add(new Mascota(2, "Sally", 20100701));
        listaMascotas.Add(new Mascota(3, "Capuchina", 20161210));
        listaMascotas.Add(new Mascota(4, "Grisú", 20161120));
        listaMascotas.Add(new Mascota(5, "Arian", 20200102));
        listaMascotas.Add(new Mascota(6, "Milú", 20100706));

        //Extraiga los registros donde la fecha de nacimiento
        //esté en un rango
        List<Mascota> Resultados = (from animal in listaMascotas
                                    where animal.FechaNace > 20150101
                                    && animal.FechaNace <= 20161231
                                    select animal).ToList();

        //Ejecuta la consulta y la imprime
        for (int cont = 0; cont < Resultados.Count; cont++) {
            Resultados[cont].Imprime();
        }
    }
}
```

}



The image shows a screenshot of a Visual Studio debug console window. The title bar at the top reads 'Consola de depuración de Mi' followed by a close button (X) and a dropdown menu with a plus sign and a downward arrow. The console output displays two lines of text: 'Código: 3 Nombre: Capuchina Fecha Nacimiento: 20161210' and 'Código: 4 Nombre: Grisú Fecha Nacimiento: 20161120'. The text is in a monospaced font.

```
Código: 3 Nombre: Capuchina Fecha Nacimiento: 20161210  
Código: 4 Nombre: Grisú Fecha Nacimiento: 20161120
```

*Ilustración 11: LINQ: Consulta con objetos y resultado en un List*

# Determinación de tipo de dato

F/012.cs

```
using System.Collections;

namespace Ejemplo;

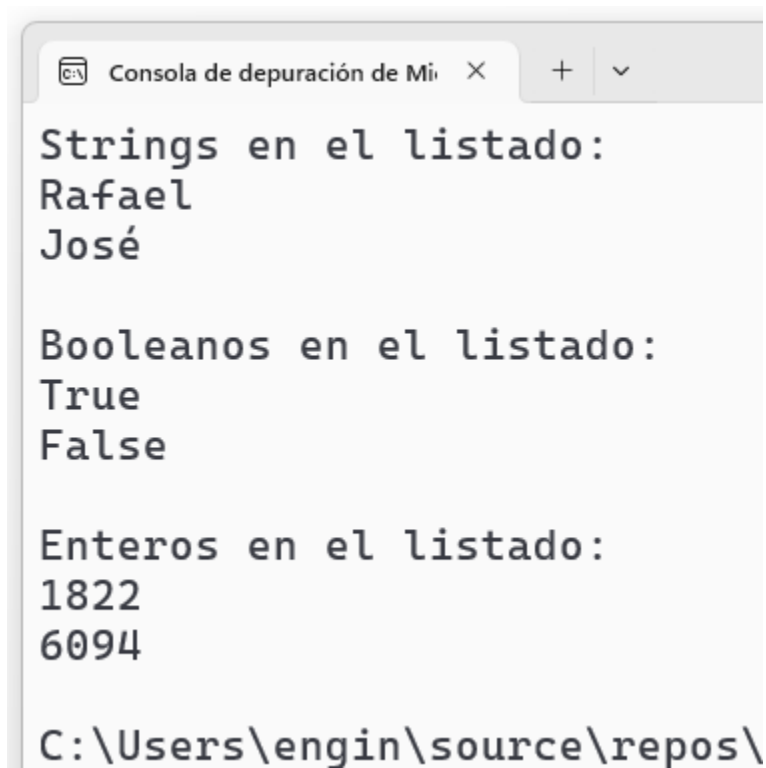
internal class Program {
    static void Main() {
        ArrayList Varios =
        [
            1822,
            'M',
            true,
            639.9,
            "Rafael",
            6094,
            'J',
            false,
            55.5,
            "José",
        ];

        //Muestra los ítems que son strings
        Console.WriteLine("Strings en el listado:");
        List<string> Cadenas = (from nombre in
                                Varios.OfType<string>()
                                select nombre).ToList();
        for (int Cont = 0; Cont < Cadenas.Count; Cont++) {
            Console.WriteLine(Cadenas[Cont]);
        }

        //Muestra los ítems que son booleanos
        Console.WriteLine("\r\nBooleanos en el listado:");
        List<bool> Booleanos = (from valorbool in
                                Varios.OfType<bool>()
                                select valorbool).ToList();
        for (int Cont = 0; Cont < Cadenas.Count; Cont++) {
            Console.WriteLine(Booleanos[Cont]);
        }

        //Muestra los ítems que son enteros
        Console.WriteLine("\r\nEnteros en el listado:");
        List<int> Enteros = (from valorentero in
                                Varios.OfType<int>()
                                select valorentero).ToList();
        for (int Cont = 0; Cont < Cadenas.Count; Cont++) {
```

```
        Console.WriteLine(Enteros[Cont]);  
    }  
}  
}
```



The screenshot shows a Visual Studio debug console window titled 'Consola de depuración de Mi'. It displays the output of a LINQ query. The output is organized into three sections: 'Strings en el listado:' with values 'Rafael' and 'José'; 'Booleanos en el listado:' with values 'True' and 'False'; and 'Enteros en el listado:' with values '1822' and '6094'. The path 'C:\Users\engin\source\repos\' is visible at the bottom of the console window.

```
Strings en el listado:  
Rafael  
José  
  
Booleanos en el listado:  
True  
False  
  
Enteros en el listado:  
1822  
6094  
  
C:\Users\engin\source\repos\
```

*Ilustración 12: LINQ: Determinación de tipo de dato*

## Ordenación por un campo y luego por otro

F/013.cs

```
namespace Ejemplo;

internal class Mascota {
    public string Especie { get; set; }
    public string Nombre { get; set; }
    public Mascota(string Especie, string Nombre) {
        this.Especie = Especie;
        this.Nombre = Nombre;
    }
}

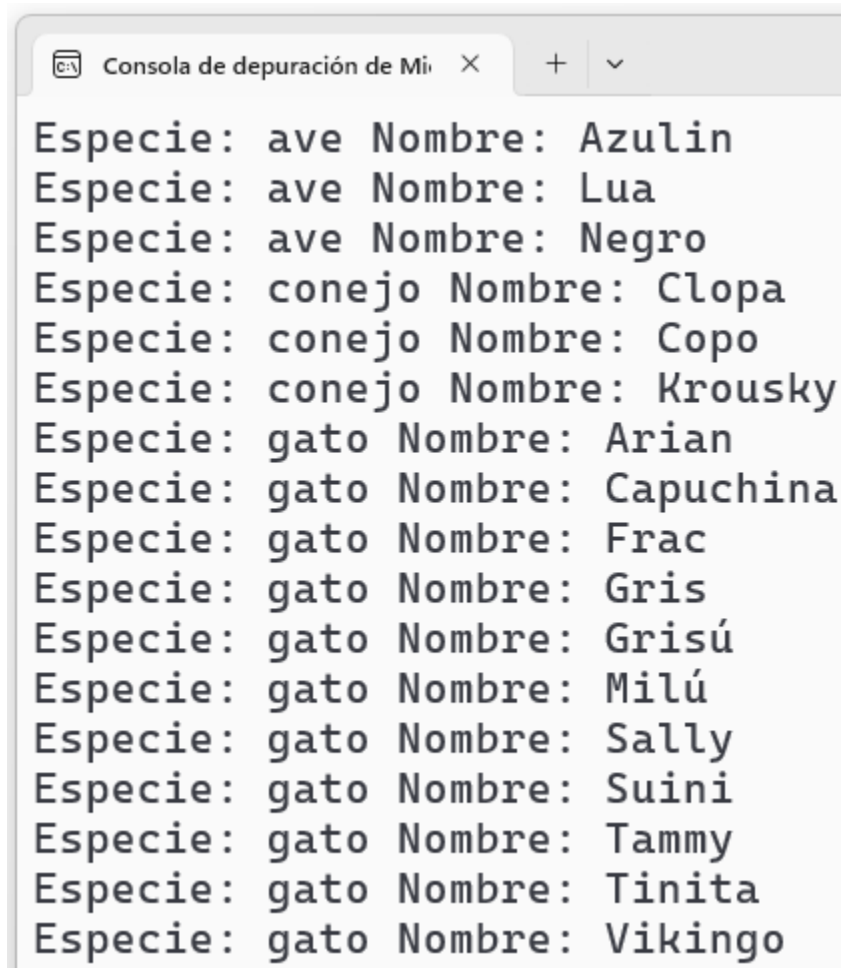
internal class Program {
    static void Main() {
        //Fuente de datos, una lista
        List<Mascota> listaMascotas =
        [
            new Mascota("gato", "Suini"),
            new Mascota("gato", "Gris"),
            new Mascota("gato", "Sally"),
            new Mascota("gato", "Tinita"),
            new Mascota("conejo", "Krousky"),
            new Mascota("gato", "Capuchina"),
            new Mascota("gato", "Tammy"),
            new Mascota("gato", "Grisú"),
            new Mascota("ave", "Lua"),
            new Mascota("conejo", "Copo"),
            new Mascota("gato", "Vikingo"),
            new Mascota("gato", "Arian"),
            new Mascota("gato", "Milú"),
            new Mascota("ave", "Azulin"),
            new Mascota("gato", "Frac"),
            new Mascota("ave", "Negro"),
            new Mascota("conejo", "Clopa"),
        ];

        var ConjuntoGrupos = from animal in listaMascotas
                              group animal by animal.Especie;

        //Itera por grupo, cada grupo tiene una llave
        foreach (var grupo in ConjuntoGrupos) {
            Console.WriteLine("\r\nGrupo: " + grupo.Key);

            // Cada grupo tiene una colección interna
            foreach (Mascota individuo in grupo)
```

```
        Console.WriteLine("Nombre: " + individuo.Nombre);  
    }  
}
```



```
Especie: ave Nombre: Azulín  
Especie: ave Nombre: Lua  
Especie: ave Nombre: Negro  
Especie: conejo Nombre: Clopa  
Especie: conejo Nombre: Copo  
Especie: conejo Nombre: Krousky  
Especie: gato Nombre: Arian  
Especie: gato Nombre: Capuchina  
Especie: gato Nombre: Frac  
Especie: gato Nombre: Gris  
Especie: gato Nombre: Grisú  
Especie: gato Nombre: Milú  
Especie: gato Nombre: Sally  
Especie: gato Nombre: Suini  
Especie: gato Nombre: Tammy  
Especie: gato Nombre: Tinita  
Especie: gato Nombre: Vikingo
```

*Ilustración 13: LINQ: Ordenación por un campo y luego por otro*

## Agrupación por un campo

F/014.cs

```
namespace Ejemplo;

internal class Especie(intCodigo, string Nombre) {
    public int Codigo { get; set; } = Codigo;
    public string Nombre { get; set; } = Nombre;
}

internal class Mascota(int Especie, string Nombre) {
    public int Especie { get; set; } = Especie;
    public string Nombre { get; set; } = Nombre;
}

internal class Program {
    static void Main() {
        List<Especie> listaEspecies =
        [
            new Especie(1, "Gato"),
            new Especie(2, "Conejo"),
            new Especie(3, "Ave"),
        ];

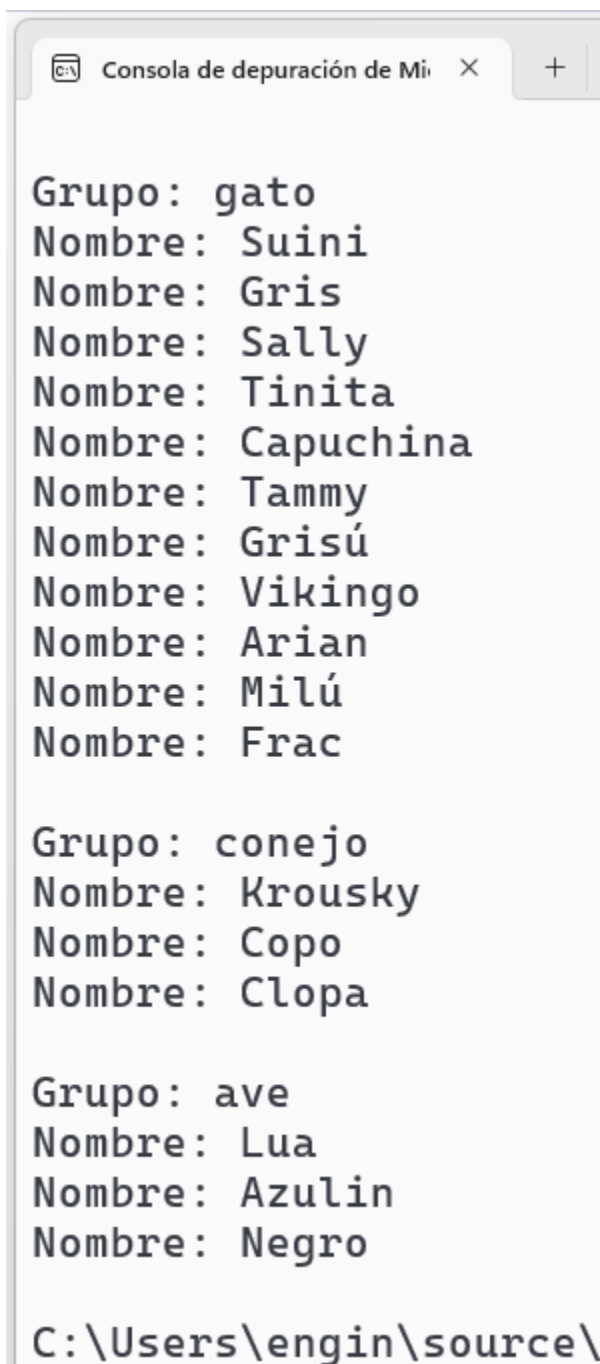
        List<Mascota> listaMascotas =
        [
            new Mascota(1, "Suini"),
            new Mascota(1, "Gris"),
            new Mascota(1, "Sally"),
            new Mascota(1, "Tinita"),
            new Mascota(2, "Krousky"),
            new Mascota(1, "Capuchina"),
            new Mascota(1, "Tammy"),
            new Mascota(1, "Grisú"),
            new Mascota(3, "Lua"),
            new Mascota(2, "Copo"),
            new Mascota(1, "Vikingo"),
            new Mascota(1, "Arian"),
            new Mascota(1, "Milú"),
            new Mascota(3, "Azulin"),
            new Mascota(1, "Frac"),
            new Mascota(3, "Negro"),
            new Mascota(2, "Clopa"),
        ];

        var Consulta = from mascota in listaMascotas
                        join especie in listaEspecies
```

```
        on mascota.Especie equals especie.Codigo
        select new {
            Especie = especie.Nombre,
            Mascota = mascota.Nombre
        };

    foreach (var item in Consulta) {
        Console.Write("La mascota " + item.Mascota);
        Console.WriteLine(" es de la especie " + item.Especie);
    }
}
```





```
Consola de depuración de Mi X +

Grupo: gato
Nombre: Suini
Nombre: Gris
Nombre: Sally
Nombre: Tinita
Nombre: Capuchina
Nombre: Tammy
Nombre: Grisú
Nombre: Vikingo
Nombre: Arian
Nombre: Milú
Nombre: Frac

Grupo: conejo
Nombre: Krousky
Nombre: Copo
Nombre: Clopa

Grupo: ave
Nombre: Lua
Nombre: Azulín
Nombre: Negro

C:\Users\engin\source\
```

*Ilustración 14: LINQ: Agrupación por un campo*

## Hacer un "join" entre listas

F/015.cs

```
namespace Ejemplo;

internal class Especie(intCodigo, string Nombre) {
    public int Codigo { get; set; } = Codigo;
    public string Nombre { get; set; } = Nombre;
}

internal class Mascota(int Especie, string Nombre) {
    public int Especie { get; set; } = Especie;
    public string Nombre { get; set; } = Nombre;
}

internal class Program {
    static void Main() {
        List<Especie> listaEspecies =
        [
            new Especie(1, "Gato"),
            new Especie(2, "Conejo"),
            new Especie(3, "Ave"),
        ];

        List<Mascota> listaMascotas =
        [
            new Mascota(1, "Suini"),
            new Mascota(1, "Gris"),
            new Mascota(1, "Sally"),
            new Mascota(1, "Tinita"),
            new Mascota(2, "Krousky"),
            new Mascota(1, "Capuchina"),
            new Mascota(1, "Tammy"),
            new Mascota(1, "Grisú"),
            new Mascota(3, "Lua"),
            new Mascota(2, "Copo"),
            new Mascota(1, "Vikingo"),
            new Mascota(1, "Arian"),
            new Mascota(1, "Milú"),
            new Mascota(3, "Azulin"),
            new Mascota(1, "Frac"),
            new Mascota(3, "Negro"),
            new Mascota(2, "Clopa"),
        ];

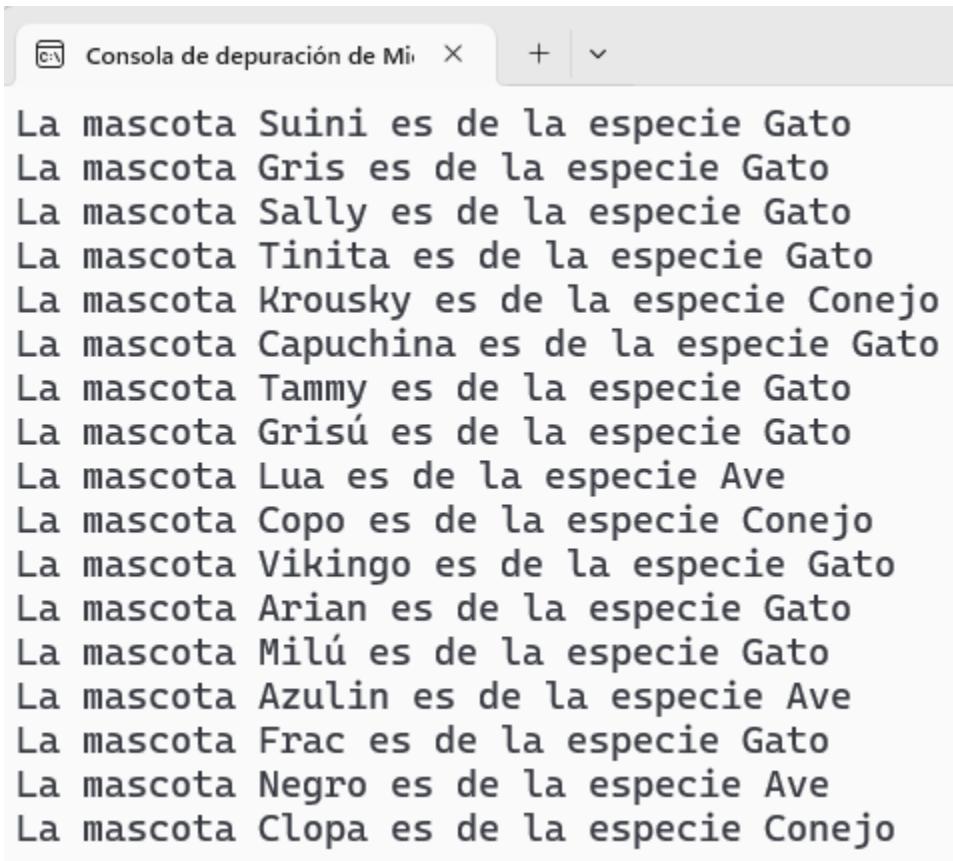
        var Consulta = from mascota in listaMascotas
                        join especie in listaEspecies
```

```

        on mascota.Especie equals especie.Codigo
        select new {
            Especie = especie.Nombre,
            Mascota = mascota.Nombre
        };

    foreach (var item in Consulta) {
        Console.Write("La mascota " + item.Mascota);
        Console.WriteLine(" es de la especie " + item.Especie);
    }
}
}

```



```

La mascota Suini es de la especie Gato
La mascota Gris es de la especie Gato
La mascota Sally es de la especie Gato
La mascota Tinita es de la especie Gato
La mascota Krousky es de la especie Conejo
La mascota Capuchina es de la especie Gato
La mascota Tammy es de la especie Gato
La mascota Grisú es de la especie Gato
La mascota Lua es de la especie Ave
La mascota Copo es de la especie Conejo
La mascota Vikingo es de la especie Gato
La mascota Arian es de la especie Gato
La mascota Milú es de la especie Gato
La mascota Azulin es de la especie Ave
La mascota Frac es de la especie Gato
La mascota Negro es de la especie Ave
La mascota Clopa es de la especie Conejo

```

*Ilustración 15: LINQ: Hacer un "join" entre listas*

## Un "join" con resultado personalizado

F/016.cs

```
namespace Ejemplo;

internal class Especie(intCodigo, string Nombre) {
    public int Codigo { get; set; } = Codigo;
    public string Nombre { get; set; } = Nombre;
}

internal class Mascota(int Especie, string Nombre) {
    public int Especie { get; set; } = Especie;
    public string Nombre { get; set; } = Nombre;
}

internal class Program {
    static void Main() {
        List<Especie> listaEspecies =
        [
            new Especie(1, "Gato"),
            new Especie(2, "Conejo"),
            new Especie(3, "Ave"),
        ];

        List<Mascota> listaMascotas =
        [
            new Mascota(1, "Suini"),
            new Mascota(1, "Gris"),
            new Mascota(1, "Sally"),
            new Mascota(1, "Tinita"),
            new Mascota(2, "Krousky"),
            new Mascota(1, "Capuchina"),
            new Mascota(1, "Tammy"),
            new Mascota(1, "Grisú"),
            new Mascota(3, "Lua"),
            new Mascota(2, "Copo"),
            new Mascota(1, "Vikingo"),
            new Mascota(1, "Arian"),
            new Mascota(1, "Milú"),
            new Mascota(3, "Azulin"),
            new Mascota(1, "Frac"),
            new Mascota(3, "Negro"),
            new Mascota(2, "Clopa"),
        ];

        var Consulta = from mascota in listaMascotas
                        join especie in listaEspecies
```

```

        on mascota.Especie equals especie.Codigo
        select new {
            Mascota = "Nombre Mascota: " + mascota.Nombre,
            Especie = "Especie: " + especie.Nombre
        };

        foreach (var item in Consulta) {
            Console.WriteLine(item.Mascota + " : " + item.Especie);
        }
    }
}

```

```

Nombre Mascota: Suini : Especie: Gato
Nombre Mascota: Gris : Especie: Gato
Nombre Mascota: Sally : Especie: Gato
Nombre Mascota: Tinita : Especie: Gato
Nombre Mascota: Krousky : Especie: Conejo
Nombre Mascota: Capuchina : Especie: Gato
Nombre Mascota: Tammy : Especie: Gato
Nombre Mascota: Grisú : Especie: Gato
Nombre Mascota: Lua : Especie: Ave
Nombre Mascota: Copo : Especie: Conejo
Nombre Mascota: Vikingo : Especie: Gato
Nombre Mascota: Arian : Especie: Gato
Nombre Mascota: Milú : Especie: Gato
Nombre Mascota: Azulin : Especie: Ave
Nombre Mascota: Frac : Especie: Gato
Nombre Mascota: Negro : Especie: Ave
Nombre Mascota: Clopa : Especie: Conejo

```

*Ilustración 16: LINQ: Un "join" con resultado personalizado*

## Extraer los datos de una lista que no están en otra

F/017.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        List<string> Animales = ["Gato", "Condor", "Perro", "Conejo",
                                "Loro"];

        List<string> Mamiferos = ["Perro", "Conejo", "Gato"];

        /* Extrae los animales que no están en mamíferos */
        List<string> Resultado = Animales.Except(Mamiferos).ToList();

        for (int Cont = 0; Cont < Resultado.Count; Cont++) {
            Console.WriteLine(Resultado[Cont]);
        }
    }
}
```

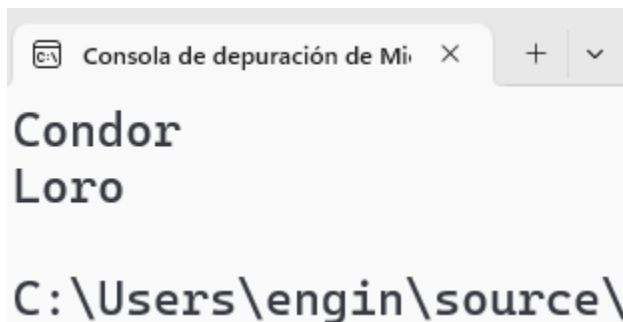


Ilustración 17: LINQ: Extraer los datos de una lista que no están en otra

## Intersección de dos listas

F/018.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        List<string> ColoresA = ["Azul", "Rojo", "Verde", "Violeta"];

        List<string> ColoresB = ["Violeta", "Azul", "Marrón"];

        /* Intersecta los animales que están en ambas listas */
        List<string> Resultado = ColoresA.Intersect(ColoresB).ToList();

        for (int Cont = 0; Cont < Resultado.Count; Cont++) {
            Console.WriteLine(Resultado[Cont]);
        }
    }
}
```

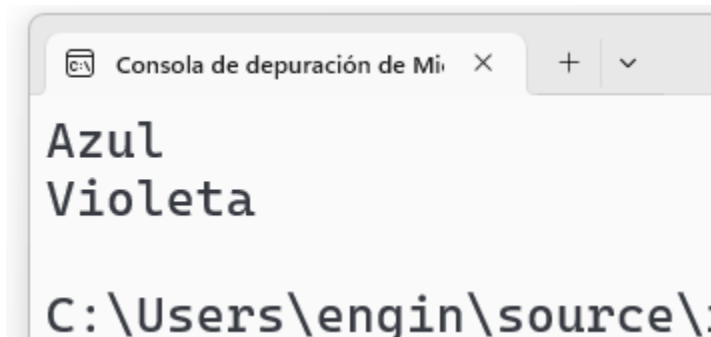


Ilustración 18: LINQ: Intersección de dos listas

## Unir dos listas sin repetir elementos

F/019.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        List<string> ColoresA = ["Azul", "Rojo", "Verde", "Marrón",
"Violeta"];

        List<string> ColoresB = ["Violeta", "Azul", "Marrón", "Naranja"];

        /* Une los valores de ambas listas evitando repetir */
        List<string> Resultado = ColoresA.Union(ColoresB).ToList();

        for (int Cont = 0; Cont < Resultado.Count; Cont++) {
            Console.WriteLine(Resultado[Cont]);
        }
    }
}
```

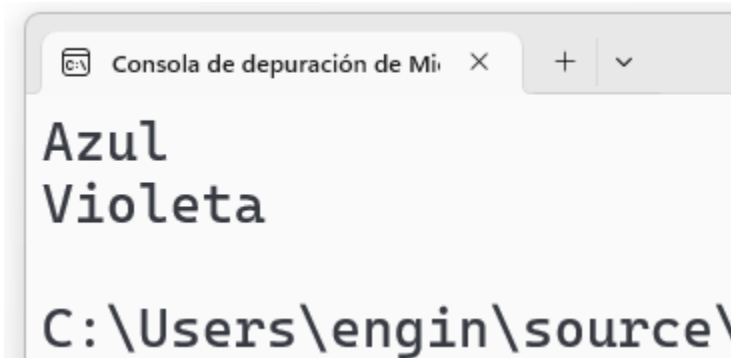


Ilustración 19: LINQ: Unir dos listas sin repetir elementos



## Consulta de texto por algún patrón

F/020.cs

```
namespace Ejemplo;

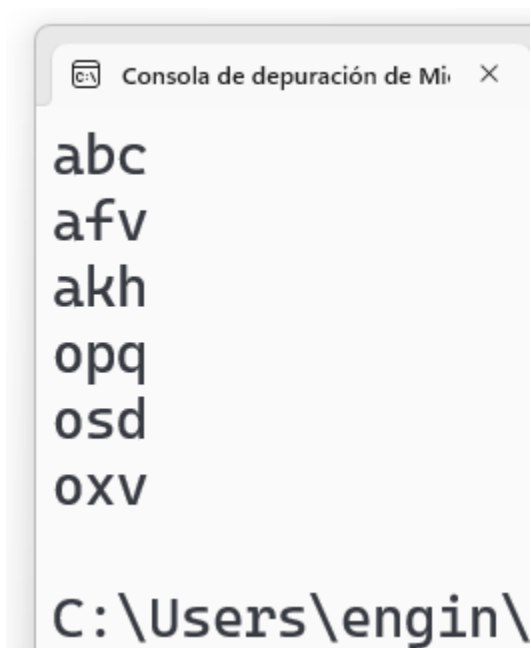
internal class Program {
    static void Main() {
        List<string> Textos =
            [ "abc", "Opq", "Afv", "Tkl", "qaz",
              "Akh", "oSd", "uyt", "oxv" ];

        //Extrae las palabras que empiezan con "a"
        List<string> PalabraMinusculaA = (from palabra in Textos
                                           where palabra.ToLower().StartsWith("a")
                                           select palabra.ToLower()).ToList();

        for (int Cont = 0; Cont < PalabraMinusculaA.Count; Cont++)
            Console.WriteLine(PalabraMinusculaA[Cont]);

        //Extrae las palabras que empiezan con "o"
        //usando la instrucción Let
        List<string> PalabraMinusculaB = (from palabra in Textos
                                           let minuscula = palabra.ToLower()
                                           where minuscula.StartsWith("o")
                                           select minuscula).ToList();

        for (int Cont = 0; Cont < PalabraMinusculaB.Count; Cont++)
            Console.WriteLine(PalabraMinusculaB[Cont]);
    }
}
```



*Ilustración 20: LINQ: Consulta de texto por algún patrón*

## Ordenar internamente una cadena

F/021.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una cadena
        string Cadena = "esta-es-una-prueba-de-ordenamiento";
        Console.WriteLine("Cadena: " + Cadena);

        /* Se ordena usando Linq
         * Si desea ordenar los elementos dentro de una secuencia
         * deberá pasar un método keySelector de identidad que
         * indique que cada elemento de
         * la secuencia es, en sí mismo, una clave. */
        List<char> Resultado = Cadena.OrderBy(str => str).ToList();
        Console.WriteLine("Arreglo con las letras ordenadas");
        for (int Cont = 0; Cont < Resultado.Count; Cont++)
            Console.Write(Resultado[Cont]);

        // Y convierte ese arreglo en cadena
        string Ordenado = String.Concat(Resultado);

        //Imprime
        Console.WriteLine("\r\nOrdenado por letra: " + Ordenado);
    }
}
```

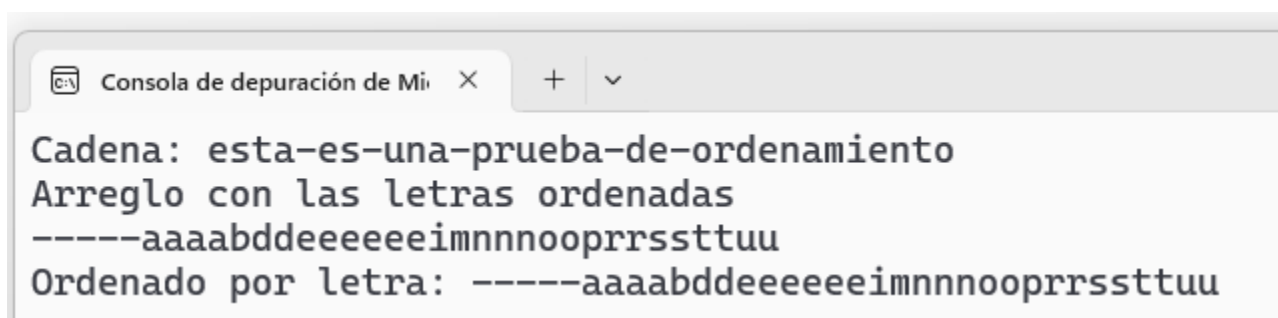


Ilustración 21: LINQ: Ordenar internamente una cadena

## Ordenar internamente una cadena con diversos caracteres alfanuméricos

F/022.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una cadena
        string Cadena = "El-ñandú-es-un-ave-de-Sudamérica.";
        Cadena += "-La-cigüeña-blanca-es-una-especie-de-ave";
        Cadena += "-Ciconiiforme-de-gran-tamaño.";
        Console.WriteLine("Cadena: " + Cadena);

        /* Se ordena usando Linq
         * Si desea ordenar los elementos dentro de una secuencia,
         * deberá pasar un método keySelector de identidad que
         * indique que cada elemento de
         * la secuencia es, en sí mismo, una clave. */
        List<char> Resultado = Cadena.OrderBy(str => str).ToList();

        Console.WriteLine("Arreglo con las letras ordenadas");
        for (int Cont = 0; Cont < Resultado.Count; Cont++)
            Console.Write "[" + Resultado[Cont] + " ] ");

        // Y convierte ese arreglo en cadena
        string Ordenado = String.Concat(Resultado);

        //Imprime
        Console.WriteLine("\r\nOrdenado por letra: " + Ordenado);
    }
}
```

```
Consola de depuración de Mi X + v - □ X
Cadena: El-ñandú-es-un-ave-de-Sudamérica.-La-cigüeña-blanca-es-una-esp
ecie-de-ave-Ciconiiforme-de-gran-tamaño.
Arreglo con las letras ordenadas
[-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-] [-]
] [.] [.] [C] [E] [L] [S] [a] [a] [a] [a] [a] [a] [a] [a] [a] [a] [a] [a]
[a] [a] [b] [c] [c] [c] [c] [c] [d] [d] [d] [d] [d] [e] [e] [e] [e] [e]
] [e] [e] [e] [e] [e] [e] [e] [f] [g] [g] [i] [i] [i] [i] [i] [i] [l]
[l] [m] [m] [m] [n] [n] [n] [n] [n] [n] [o] [o] [o] [p] [r] [r] [r] [s]
] [s] [s] [t] [u] [u] [u] [v] [v] [é] [ñ] [ñ] [ñ] [ú] [ü]
Ordenado por letra: -----..CELSaaaaaaaaaaaaabccccddddddee
eeeeeeeeefggiiiiiiillmmnnnnnnnooprrrrsstuuuvvéñññüü
```

Ilustración 22: LINQ: Ordenar internamente una cadena

# Ordenamiento según tamaño de la palabra

F/023.cs

```
namespace Ejemplo;

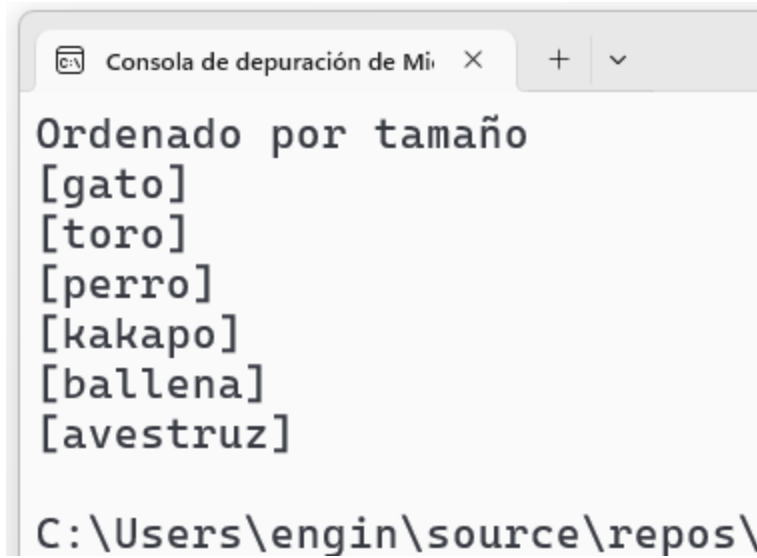
internal class Program {
    static void Main() {
        //Una cadena
        string Cadena = "El-ñandú-es-un-ave-de-Sudamérica.";
        Cadena += "-La-cigüeña-blanca-es-una-especie-de-ave";
        Cadena += "-Ciconiiforme-de-gran-tamaño.";
        Console.WriteLine("Cadena: " + Cadena);

        /* Se ordena usando Linq
         * Si desea ordenar los elementos dentro de una secuencia,
         * deberá pasar un método keySelector de identidad que
         * indique que cada elemento de
         * la secuencia es, en sí mismo, una clave. */
        List<char> Resultado = Cadena.OrderBy(str => str).ToList();

        Console.WriteLine("Arreglo con las letras ordenadas");
        for (int Cont = 0; Cont < Resultado.Count; Cont++)
            Console.Write "[" + Resultado[Cont] + " ] ");

        // Y convierte ese arreglo en cadena
        string Ordenado = String.Concat(Resultado);

        //Imprime
        Console.WriteLine("\r\nOrdenado por letra: " + Ordenado);
    }
}
```



The screenshot shows a Visual Studio console window titled 'Consola de depuración de Mi'. The output text is as follows:

```
Ordenado por tamaño  
[gato]  
[toro]  
[perro]  
[kakapo]  
[ballena]  
[avestruz]  
  
C:\Users\engin\source\repos\
```

*Ilustración 23: LINQ: Ordenamiento según tamaño de la palabra*

## Ordenamiento por la segunda letra de cada palabra

F/024.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una lista de cadenas
        List<string> Cadenas = [ "gato", "perro", "avestruz", "toro",
                                "ballena", "kakapo" ];

        /* Se ordena usando LINQ
           * Ordena por la segunda letra. */
        List<string> Resultado = Cadenas.OrderBy(str => str[1]).ToList();
        Console.WriteLine("Ordenado por la segunda letra");
        for (int Cont = 0; Cont < Resultado.Count; Cont++)
            Console.WriteLine "[" + Resultado[Cont] + " ] ";
    }
}
```

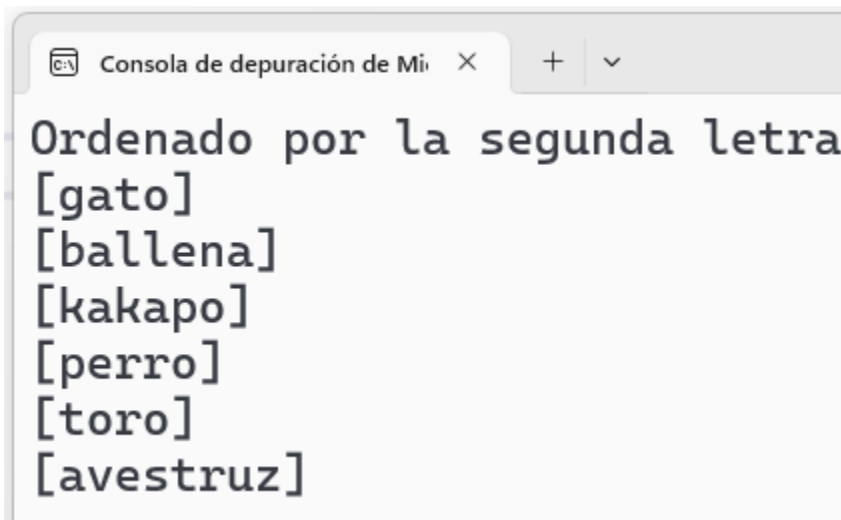


Ilustración 24: LINQ: Ordenamiento por la segunda letra de cada palabra



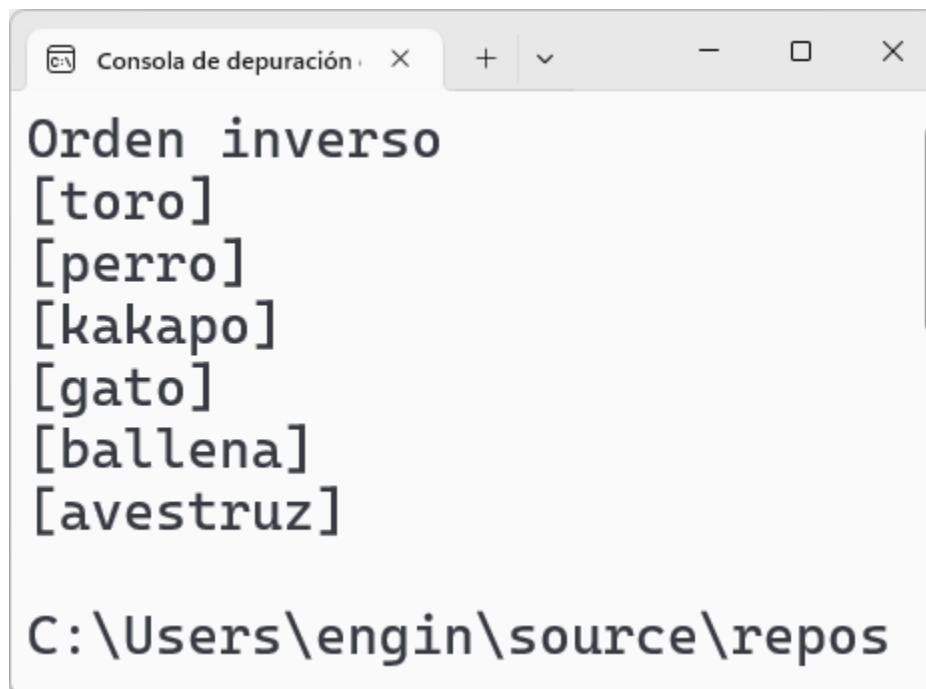
## Invertir el ordenamiento

F/025.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Una lista de cadenas
        List<string> Cadenas = [ "gato", "perro", "avestruz", "toro",
                                "ballena", "kakapo" ];

        /* Se ordena usando LINQ
           * Ordena por la segunda letra. */
        List<string> Resultado = Cadenas.OrderBy(str => str).ToList();
        Resultado.Reverse();
        Console.WriteLine("Orden inverso");
        for (int Cont = 0; Cont < Resultado.Count; Cont++)
            Console.WriteLine "[" + Resultado[Cont] + " ] ";
    }
}
```



```
Consola de depuración
Orden inverso
[toro]
[perro]
[kakapo]
[gato]
[ballena]
[avestruz]
C:\Users\engin\source\repos
```

Ilustración 25: LINQ: Invertir el ordenamiento

# Métricas: Comparativa entre usar LINQ e implementación tradicional

LINQ es una herramienta poderosa porque ahorra la escritura de líneas de código, haciendo más fácil el mantenimiento. Pero ¿cuál es el precio que pagar por esta nueva funcionalidad? A continuación, se muestra una comparativa de desempeño entre usar LINQ y escribir el algoritmo tradicional.

F/026.cs

```
using System.Diagnostics;
/* Comparativa entre LINQ e implementación tradicional */

namespace Ejemplo;

class Program {
    static void Main() {
#ifdef DEBUG
        Console.WriteLine("Modo DEBUG detectado. Las pruebas se deben hacer en RELEASE");
        Environment.Exit(0);
#endif

        Console.WriteLine("Comparativa LINQ vs Programación Clásica\r\n");

        //Generador de números aleatorios único
        Random Azar = new();
        for (int Probar = 1; Probar <= 20; Probar++)
            Pruebas(Azar);
    }

    static public void Pruebas(Random Azar) {
        //Llenar un List<int> con valores al azar
        List<int> Enteros = [];
        for (int Cont = 1; Cont <= 1000000; Cont++) {
            Enteros.Add(Azar.Next(-50, 50));
        }
        List<int> ResultadosLINQ = [];

        //=====
        //Prueba con LINQ
        //=====

        //Medidor de tiempos
        Stopwatch cronometro = new();
        cronometro.Reset();
        cronometro.Start();
    }
}
```

```

ResultadosLINQ =
    (from numero in Enteros
     where (numero % 2) == 1
     select numero).ToList();

long TiempoLINQ = cronometro.ElapsedMilliseconds;

//Imprime el tiempo transcurrido y un valor de la lista
Console.Write("Tiempo LINQ (ms): " + TiempoLINQ);

//=====
//Prueba sin LINQ
//=====
List<int> ResultadosNOLINQ = [];
cronometro.Reset();
cronometro.Start();

//Ejecuta la consulta y guarda el resultado en una lista
foreach (int Valor in Enteros)
    if (Valor % 2 == 1) ResultadosNOLINQ.Add(Valor);

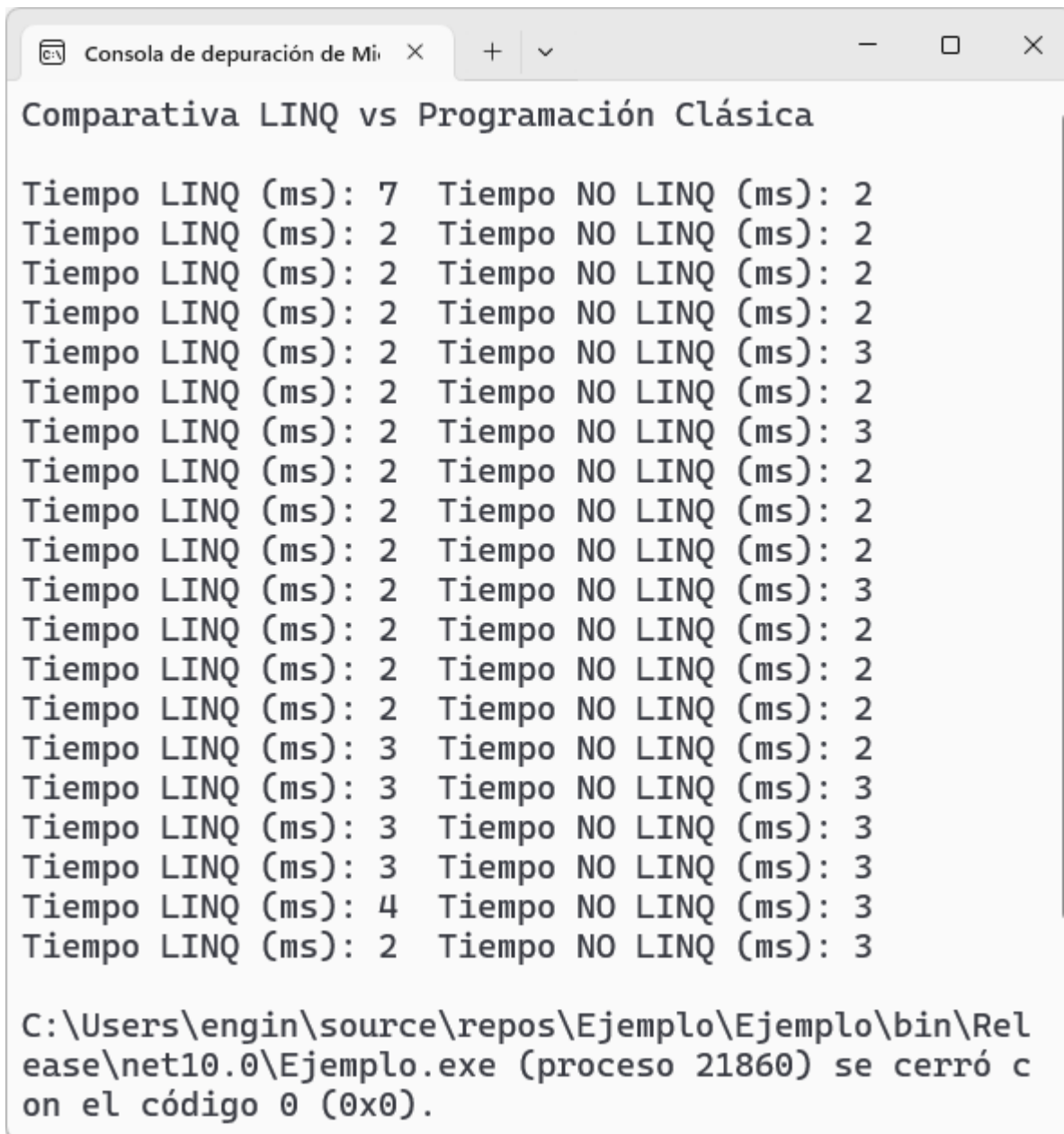
long TiempoNOLINQ = cronometro.ElapsedMilliseconds;

//Imprime el tiempo transcurrido y un valor de la lista
Console.Write("Tiempo NO LINQ (ms): " + TiempoNOLINQ);

//Compara ambas listas
for (int Cont = 0; Cont < ResultadosLINQ.Count; Cont++) {
    if (ResultadosLINQ[Cont] != ResultadosNOLINQ[Cont])
        Console.WriteLine("Fallo en el proceso");
}

Console.WriteLine(" ");
}
}

```



```
Consola de depuración de Mi  X  +  v  -  □  X

Comparativa LINQ vs Programación Clásica

Tiempo LINQ (ms): 7  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 3  Tiempo NO LINQ (ms): 2
Tiempo LINQ (ms): 3  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 3  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 3  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 3  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 4  Tiempo NO LINQ (ms): 3
Tiempo LINQ (ms): 2  Tiempo NO LINQ (ms): 3

C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Release\net10.0\Ejemplo.exe (proceso 21860) se cerró con el código 0 (0x0).
```

Ilustración 26: Comparativa desempeño de LINQ

Como se puede observar, LINQ se desempeña muy similar a la programación clásica.

## Expresiones LAMBDA

Una expresión lambda en C# es una forma concisa de escribir funciones anónimas. Se utilizan principalmente para simplificar el código cuando se trabaja con LINQ.

F/027.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Expresión lambda
        //=====
        var AlCubo = (int valor) => valor * valor * valor;

        //Uso de la expresión lambda
        int Resultado = AlCubo(13);
        Console.WriteLine("Elevado al cubo: " + Resultado);
    }
}
```

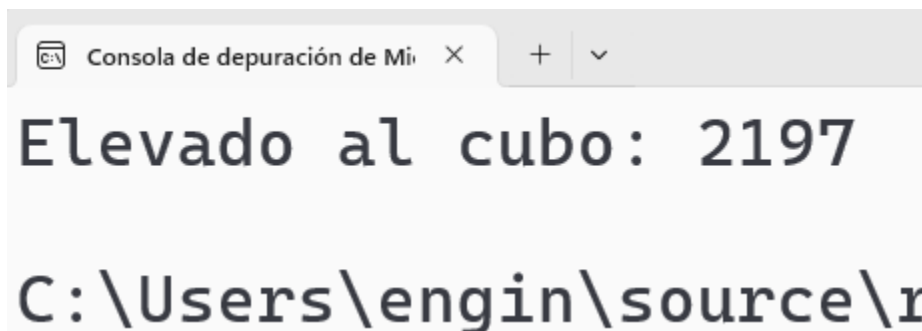


Ilustración 27: Ejecuta expresión Lambda

## Expresión Lambda: Uso de varias líneas

F/028.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Expresión Lambda: Uso del return y { } porque hay
        //más de dos líneas
        //=====
        var AreaTriangulo = (double ladoA, double ladoB, double ladoC) => {
            double s = (ladoA + ladoB + ladoC) / 2;
            return Math.Sqrt(s * (s - ladoA) * (s - ladoB) * (s - ladoC));
        };

        double Area = AreaTriangulo(3, 4, 5);
        Console.WriteLine("Área del triángulo es: " + Area);
    }
}
```

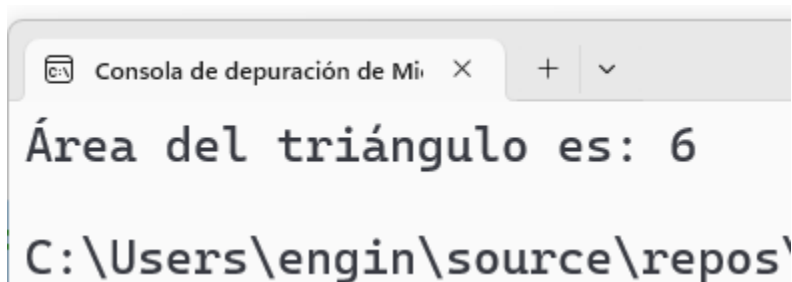


Ilustración 28: Expresiones Lambda usando varias líneas

## Expresión Lambda: Retornando tipo de dato

F/029.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Expresion lambda retornando el tipo de valor
        //=====
        Func<int, long> Factorial = (int numero) => {
            long acumula = 1;
            for (int cont = 1; cont <= numero; cont++) {
                acumula *= cont;
            }
            return acumula;
        };
        long Probar = Factorial(5);
        Console.WriteLine("Factorial es: " + Probar);
    }
}
```

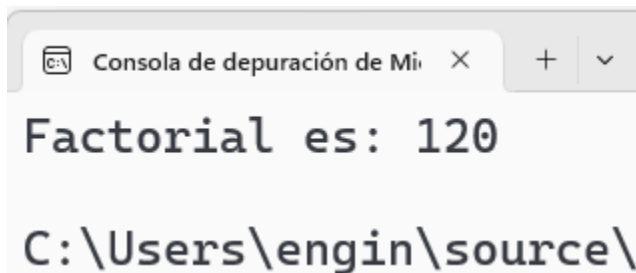


Ilustración 29: Retornando tipo de dato

## Expresión Lambda: Acceso a variable externa

F/030.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Acceso a variable externa a la expresión lambda
        //=====
        double valorExterno = 12.345;

        //Expresión lambda haciendo uso de un valor externo
        var UnCalculo = (double Parametro) => Parametro * valorExterno;

        //Se usa la expresión
        double Resulta = UnCalculo(5);
        Console.WriteLine("Resultado es: " + Resulta);
    }
}
```

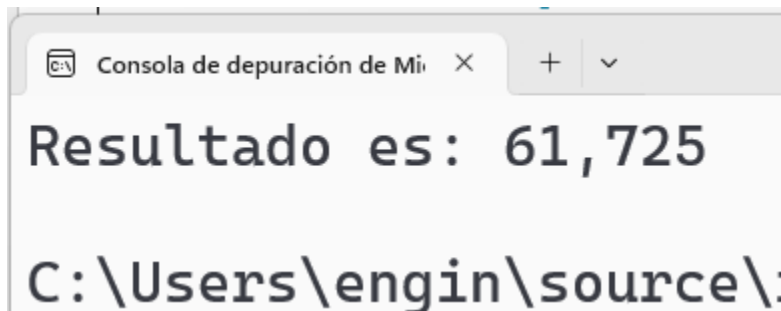


Ilustración 30: Acceso a variable externa



## Expresión Lambda: Listas. Pero no funciona.

F/031.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Listas y variables externas
        //=====

        //Una lista de operaciones Lambda
        var Operaciones = new List<Func<int, int>>();

        //Agrega items a la lista
        for (int Cont = 5; Cont <= 10; Cont++) {
            Operaciones.Add((int Valor) => Valor + Cont);
        }

        //Pero no funciona porque no hace la operación
        foreach (var operacion in Operaciones) {
            int Resultados = operacion(300);
            Console.WriteLine(Resultados);
        }
    }
}
```

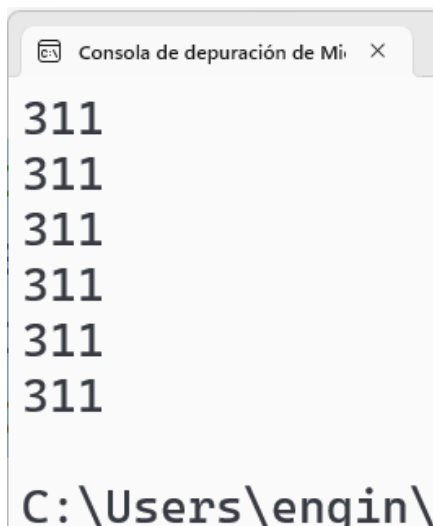


Ilustración 31: No funciona

## Expresión Lambda: Listas. Ahora si funciona.

F/032.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Listas y variables externas
        //=====

        //Una lista de operaciones Lambda
        var Operaciones = new List<Func<int, int>>();

        //Agrega items a la lista, declara internamente a Suma
        for (int Cont = 5; Cont <= 10; Cont++) {
            int Suma = Cont;
            Operaciones.Add((int Valor) => Valor + Suma);
        }

        //En este caso SI funciona
        foreach (var operacion in Operaciones) {
            int Resultados = operacion(300);
            Console.WriteLine(Resultados);
        }
    }
}
```



Ilustración 32: Ahora si funciona

# Expresiones LAMBDA: Listas y variables externas no funciona

F/033.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        //=====
        //Listas y variables externas
        //=====

        //Una lista de operaciones Lambda
        var Operaciones = new List<Func<int, int>>();

        //Agrega items a la lista, declara externamente Suma
        int Suma;
        for (int Cont = 5; Cont <= 10; Cont++) {
            Suma = Cont;
            Operaciones.Add((int Valor) => Valor + Suma);
        }

        //En este caso NO va a funcionar
        foreach (var operacion in Operaciones) {
            int Resultados = operacion(300);
            Console.WriteLine(Resultados);
        }
    }
}
```

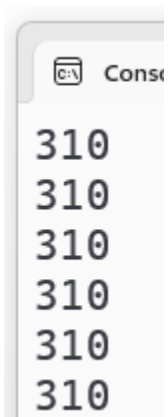


Ilustración 33: No funciona

# Expresiones LAMBDA: Uso de static

F/034.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        int ValorExterno = 13;

        /* El uso de static mejora el desempeño de la expresión lambda
         * pero ya NO se puede hacer uso de variables externas */
        var UnCalculo = static (int Numero) => Numero * ValorExterno;

        int Resultado = UnCalculo(17);
        Console.WriteLine(Resultado);
    }
}
```

Numero \* ValorExterno;



(variable local) **int** ValorExterno

[Describir con Copilot](#)

CS8820: Una función anónima estática no puede contener una referencia a "ValorExterno".

[Mostrar posibles correcciones](#) (Alt+Entrar o Ctrl+.)

*Ilustración 34: Error al tratar de usar una variable externa*

## Comparativa entre LINQ y Lambda. Ejemplo 1.

F/035.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos, un arreglo unidimensional
        int[] Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3, 2];

        Console.WriteLine("Consulta usando LINQ");
        List<int> Resultados = (from numero in Lista
                               where numero > 5
                               select numero).ToList();

        //Ejecuta la consulta y la imprime
        for (int cont = 0; cont < Resultados.Count; cont++)
            Console.Write(Resultados[cont].ToString() + ", ");

        //Hacer lo mismo con una expresión lambda
        Console.WriteLine("\r\nConsulta usando Lambda");
        List<int> Datos = Lista.Where(x => x > 5).ToList();
        foreach (int UnValor in Datos) Console.Write(UnValor + ", ");
    }
}
```

Ambas instrucciones hacen lo mismo, salvo que con Lambda requiere menos líneas de código.

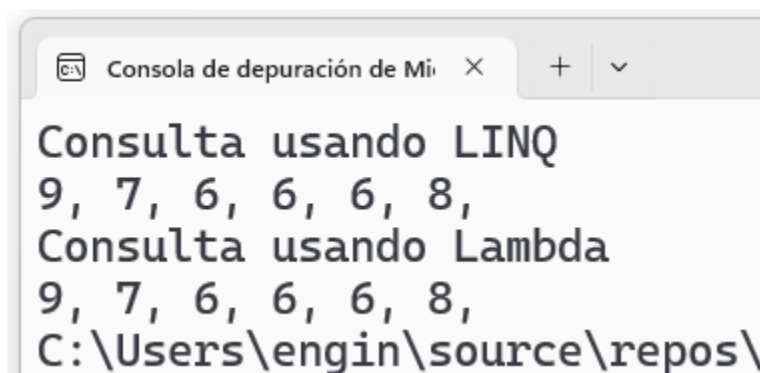


Ilustración 35: Comparativa LINQ y Lambda

## Comparativa entre LINQ y Lambda. Ejemplo 2.

F/036.cs

```
namespace Ejemplo;

internal class Program {
    static int CalcularCuadrado(int num) {
        return num * num;
    }

    static void Main() {
        // Crear una lista de enteros
        List<int> numeros = [3, 1, 2, 7, 6, 9];

        // Usar LINQ con un método en lugar de expresiones lambda
        List<int> cuadrados = numeros.Select(CalcularCuadrado).ToList();

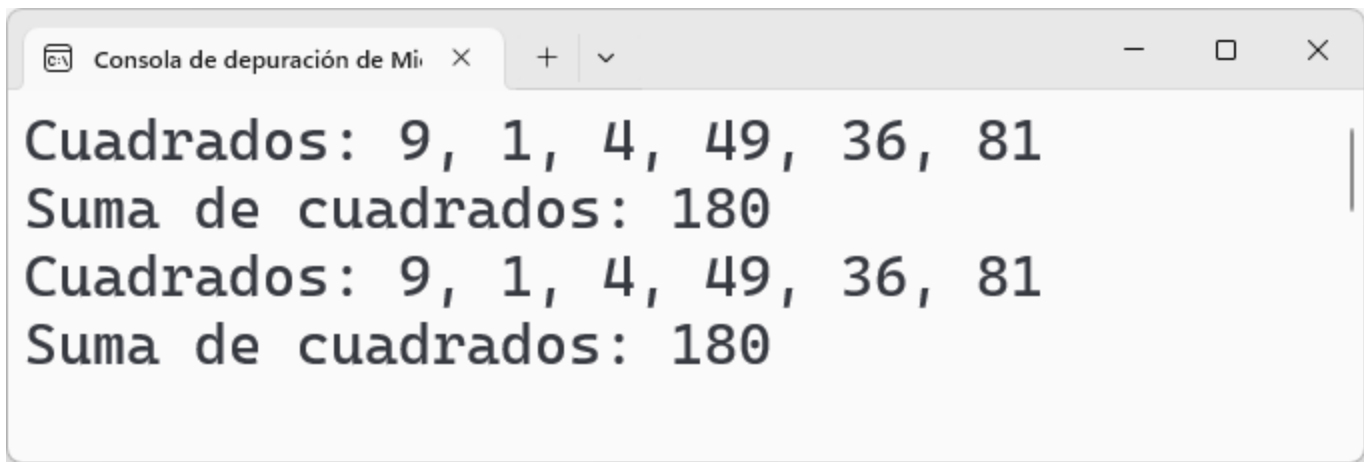
        // Usar LINQ para sumar los cuadrados
        int sumaCuadrados = cuadrados.Sum();

        // Mostrar los resultados
        Console.WriteLine("Cuadrados: " + string.Join(", ", cuadrados));
        Console.WriteLine("Suma de cuadrados: " + sumaCuadrados);

        // Usar LINQ con expresión Lambda
        var CuadradoLINQ = numeros.Select(num => num * num);

        // Usar LINQ para sumar todos los cuadrados
        int SumaCuadradosLINQ = CuadradoLINQ.Sum();

        // Mostrar los resultados
        Console.WriteLine("Cuadrados: " + string.Join(", ", CuadradoLINQ));
        Console.WriteLine("Suma de cuadrados: " + SumaCuadradosLINQ);
    }
}
```



The screenshot shows a Visual Studio debug console window titled 'Consola de depuración de Mi'. It contains two lines of output, each consisting of two parts: a list of squares and their sum. The first line shows the squares 9, 1, 4, 49, 36, 81 and their sum 180. The second line shows the same squares and sum. The window has standard Windows-style controls (minimize, maximize, close) in the top right corner.

```
Cuadrados: 9, 1, 4, 49, 36, 81  
Suma de cuadrados: 180  
Cuadrados: 9, 1, 4, 49, 36, 81  
Suma de cuadrados: 180
```

*Ilustración 36: Uso de LINQ con y sin expresiones Lambda*

## Comparativa entre LINQ y Lambda. Ejemplo 3.

Se presentan dos programas que hacen lo mismo, en uno hace uso de expresiones Lambda con LINQ y el otro sin expresiones Lambda.

Usando expresiones Lambda

F/037a.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        // Crear la lista de enteros
        List<int> numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

        // Agrupar usando LINQ con expresiones lambda
        var Agrupados = numeros.GroupBy(num => num % 2 == 0 ? "Pares" :
"Impares");

        // Mostrar los resultados
        foreach (var grupo in Agrupados) {
            Console.WriteLine($"{grupo.Key}: {string.Join(", ", grupo)}");
        }
    }
}
```



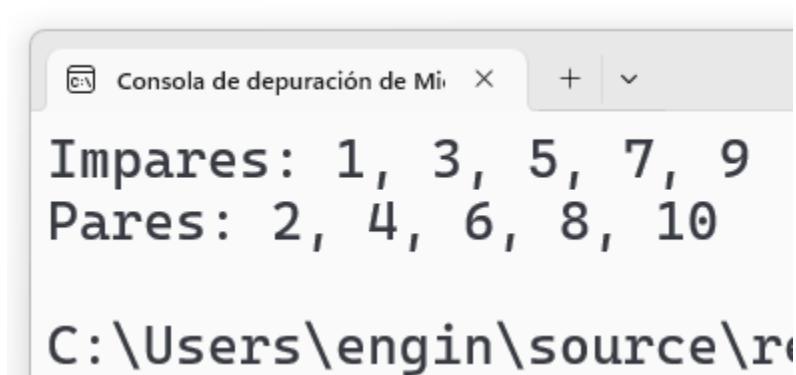
```
namespace Ejemplo;

internal class Program {
    // Método para clasificar números como "Pares" o "Impares"
    static string ClasificarParidad(int num) {
        return num % 2 == 0 ? "Pares" : "Impares";
    }

    static void Main() {
        // Crear la lista de enteros
        List<int> numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

        // Agrupar usando LINQ y un método en lugar de lambda
        var agrupados = numeros.GroupBy(ClasificarParidad);

        // Mostrar los resultados
        foreach (var grupo in agrupados) {
            Console.WriteLine($"{grupo.Key}: {string.Join(", ", grupo)}");
        }
    }
}
```



Consola de depuración de Mi... X + v

Impares: 1, 3, 5, 7, 9  
Pares: 2, 4, 6, 8, 10

C:\Users\engin\source\re...

Ilustración 37: Mismos resultados

## Expresiones Lambda: Promedio

F/038.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Fuente de datos, un arreglo unidimensional
        int[] Lista = [1, 9, 7, 2, 0, 6, 2, 6, 1, 6, 8, 3, 2, 9, 2, 9];

        //Promedio
        double Promedio = Lista.Where(x => x % 2 == 1).Average();
        Console.WriteLine("Promedio valores impares: " + Promedio);
    }
}
```

Consola de depuración de Mi X + v

Promedio valores impares: 5,571428571428571  
C:\Users\engin\source\repos\Ejemplo\Ejemplo\b:  
se cerró con el código 0 (0x0).  
Para cerrar automáticamente la consola cuando

Ilustración 38: Promedio

```
using System.Text.Json.Nodes;

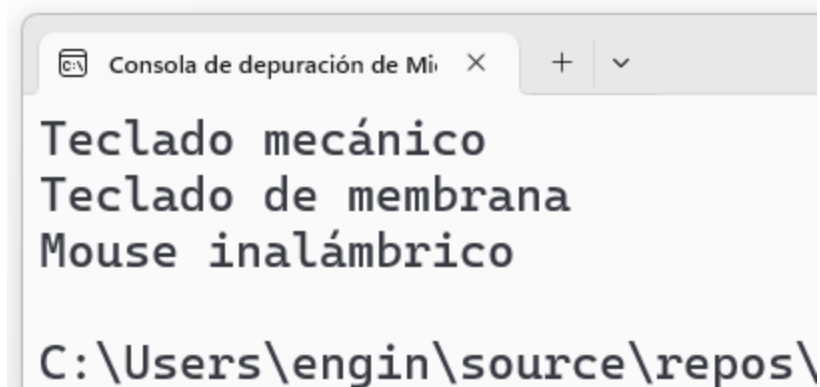
//JSON y LINQ
namespace Ejemplo;

class Program {
    static void Main() {
        string EjemploJSON = @"
        {
            ""Linea"": {
                ""titulo"": ""Ejemplo de LINQ to JSON"",
                ""items"": [
                    { ""titulo"": ""Teclado Mc"", ""descripcion"": ""Teclado
mecánico"" },
                    { ""titulo"": ""Teclado Mb"", ""descripcion"": ""Teclado
de membrana"" },
                    { ""titulo"": ""Mouse Ina"", ""descripcion"": ""Mouse
inalámbrico"" }
                ]
            }
        }";

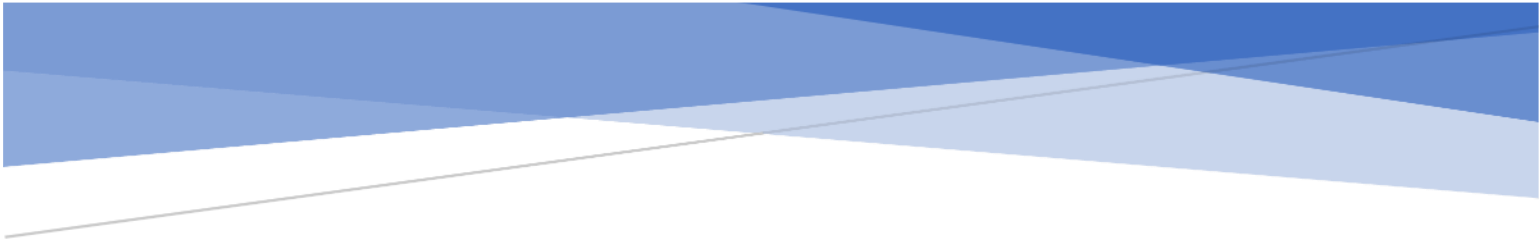
        // Parsear el JSON
        JsonNode rss = JsonNode.Parse(EjemploJSON);

        // Consultar las descripciones de los artículos usando LINQ
        var titulos = rss["Linea"]["items"]
            .ToArray()
            .Select(item => item["descripcion"].ToString());

        // Mostrar los títulos
        foreach (var titulo in titulos) {
            Console.WriteLine(titulo);
        }
    }
}
```



*Ilustración 39: JSON y LINQ*



# C# Y .NET 10

## Parte 7. Estructuras de datos de bajo nivel

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                                | 4  |
| Acerca del autor.....                                                      | 6  |
| Licencia de este libro .....                                               | 6  |
| Licencia del software .....                                                | 6  |
| Marcas registradas .....                                                   | 7  |
| Dedicatoria .....                                                          | 8  |
| Lista simplemente enlazada .....                                           | 9  |
| Enlazamiento continuo .....                                                | 14 |
| Recorriendo lista simplemente enlazada .....                               | 16 |
| Recorriendo la lista usando un método.....                                 | 19 |
| Tamaño de la lista .....                                                   | 22 |
| Traer un determinado nodo .....                                            | 24 |
| Adicionar un nodo en determinada posición .....                            | 26 |
| Borrar un nodo de una determinada posición .....                           | 30 |
| La lista doblemente enlazada .....                                         | 33 |
| Adicionar un nodo en determinada posición .....                            | 36 |
| Borrar un nodo de una determinada posición .....                           | 42 |
| Árbol binario .....                                                        | 48 |
| Primer ejemplo .....                                                       | 48 |
| Segundo ejemplo .....                                                      | 51 |
| Recorrido iterativo (no recursivo).....                                    | 54 |
| Generar árboles binarios al azar.....                                      | 57 |
| Ordenamiento usando un árbol binario.....                                  | 60 |
| Buscar en árbol binario ordenado, número de nodos y altura del árbol ..... | 62 |
| Dibujar un árbol binario.....                                              | 66 |
| Recorrer un árbol binario por niveles .....                                | 69 |
| Árbol N-ario .....                                                         | 73 |
| Recorriéndolo .....                                                        | 75 |
| Grafos.....                                                                | 78 |
| Generando un grafo aleatoriamente.....                                     | 80 |
| Guardando un árbol binario en medio persistente .....                      | 84 |
| Guardando un árbol N-ario en un medio persistente.....                     | 88 |



## Tabla de ilustraciones

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| Ilustración 1: Representación gráfica de una lista simplemente enlazada.....                   | 9  |
| Ilustración 2: El nodo es un objeto con sus atributos, métodos y un apuntador .....            | 9  |
| Ilustración 3: Lista simplemente enlazada .....                                                | 11 |
| Ilustración 4: Se crean tres nodos.....                                                        | 12 |
| Ilustración 5: Conectar el nodo primero con el nodo segundo .....                              | 13 |
| Ilustración 6: Conexión entre nodos .....                                                      | 13 |
| Ilustración 7: Enlazamiento continuo .....                                                     | 15 |
| Ilustración 8: Variable "lista" sostiene la lista simplemente enlazada.....                    | 16 |
| Ilustración 9: Recorriendo lista simplemente enlazada.....                                     | 17 |
| Ilustración 10: La variable "pasea" va de nodo en nodo. ....                                   | 18 |
| Ilustración 11: Recorriendo la lista usando un método.....                                     | 21 |
| Ilustración 12: Tamaño de la lista .....                                                       | 23 |
| Ilustración 13: Traer un determinado nodo .....                                                | 25 |
| Ilustración 14: Paso 1: Lista existente y nodo nuevo a insertar.....                           | 26 |
| Ilustración 15: Paso 2: Ir hasta el punto de inserción.....                                    | 26 |
| Ilustración 16: Paso 3: El nuevo nodo apunta a la posición particular de la lista.....         | 26 |
| Ilustración 17: Paso 4: El nodo de la lista apunta al nuevo nodo .....                         | 27 |
| Ilustración 18: Adicionar un nodo en determinada posición .....                                | 29 |
| Ilustración 19: Paso 1: Lista existente e ir hasta el nodo al que se desee eliminar.....       | 30 |
| Ilustración 20: Paso 2: El nodo que se va a eliminar .....                                     | 30 |
| Ilustración 21: Paso 3: Se cambia el apuntador del nodo anterior al siguiente. ....            | 30 |
| Ilustración 22: Borrar un nodo de una determinada posición .....                               | 32 |
| Ilustración 23: Lista doblemente enlazada.....                                                 | 33 |
| Ilustración 24: La lista doblemente enlazada .....                                             | 35 |
| Ilustración 25: Paso 1: Lista existente y nodo nuevo a insertar. Ir al punto de inserción..... | 36 |
| Ilustración 26: Paso 2: Poner los enlaces .....                                                | 36 |
| Ilustración 27: Paso 3: Poner los enlaces .....                                                | 36 |
| Ilustración 28: Paso 4: Poner los enlaces .....                                                | 37 |
| Ilustración 29: Paso 5: Poner los enlaces .....                                                | 37 |
| Ilustración 30: Adicionar un nodo en determinada posición .....                                | 41 |
| Ilustración 31: Paso 1: Lista existente e ir hasta el nodo al que se desee eliminar.....       | 42 |
| Ilustración 32: Paso 2: Modificar los apuntadores .....                                        | 42 |
| Ilustración 33: Paso 3: Modificar los apuntadores .....                                        | 42 |
| Ilustración 34: Paso 3: El nodo se destruye automáticamente al no tener quien lo sostenga .... | 43 |
| Ilustración 35: Borrar un nodo de una determinada posición .....                               | 47 |
| Ilustración 36: Ejemplo de un árbol binario .....                                              | 48 |
| Ilustración 37: Recorrido de un árbol binario .....                                            | 49 |
| Ilustración 38: Ejemplo de árbol binario.....                                                  | 51 |
| Ilustración 39: Recorrido de un árbol binario .....                                            | 52 |
| Ilustración 40: Árbol binario.....                                                             | 54 |
| Ilustración 41: Recorrido iterativo (no recursivo) .....                                       | 55 |
| Ilustración 42: Generar árboles binarios al azar.....                                          | 59 |
| Ilustración 43: Ordenamiento usando un árbol binario.....                                      | 61 |
| Ilustración 44: Buscar en árbol binario ordenado, número de nodos y altura del árbol .....     | 65 |
| Ilustración 45: Dibujar un árbol binario.....                                                  | 67 |



|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| Ilustración 46: Dibujar un árbol binario.....                                                  | 68 |
| Ilustración 47: Árbol binario de ejemplo.....                                                  | 71 |
| Ilustración 48: Recorrer un árbol binario por niveles .....                                    | 72 |
| Ilustración 49: Árbol N-ario.....                                                              | 74 |
| Ilustración 50: Recorriendo árbol N-ario.....                                                  | 77 |
| Ilustración 51: Grafo generado .....                                                           | 78 |
| Ilustración 52: Grafos .....                                                                   | 79 |
| Ilustración 53: Gafo generado .....                                                            | 82 |
| Ilustración 54: Grafo dibujado con <a href="https://viz-js.com/">https://viz-js.com/</a> ..... | 83 |
| Ilustración 55: Árbol binario guardado usando JSON .....                                       | 86 |
| Ilustración 56: Archivo JSON.....                                                              | 87 |
| Ilustración 57: JSON de un árbol N-ario .....                                                  | 90 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifemail@hotmail.com](mailto:enginelifemail@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

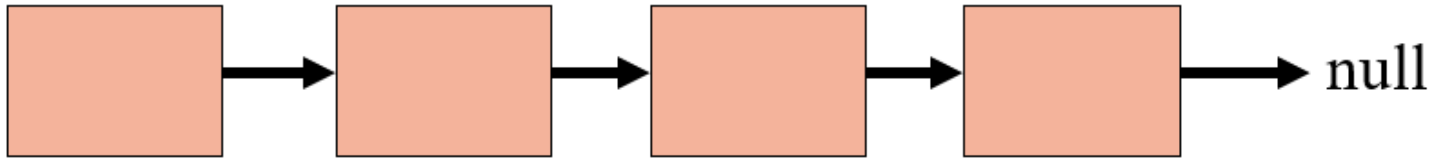
## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

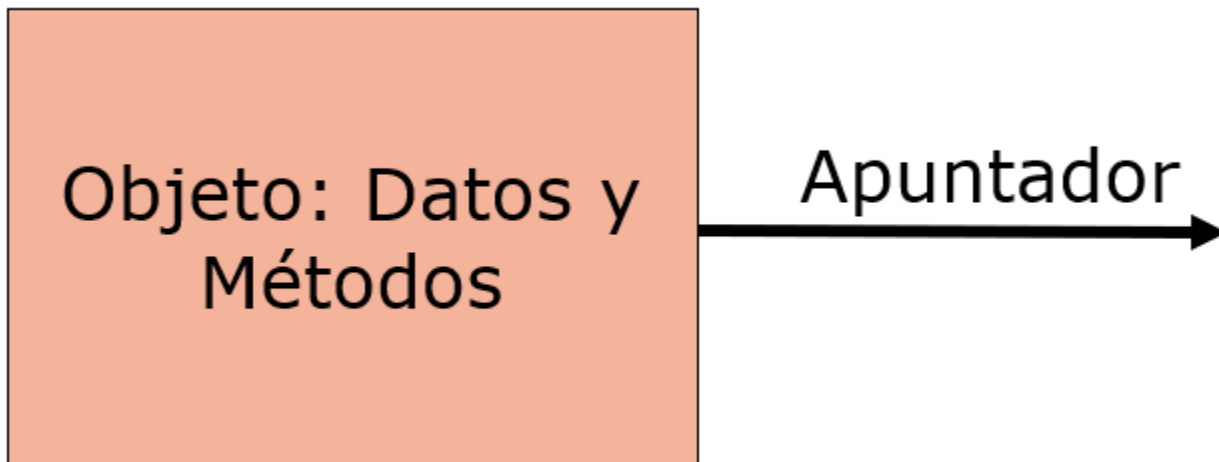
## Lista simplemente enlazada

Se representa de esta forma:



*Ilustración 1: Representación gráfica de una lista simplemente enlazada*

El rectángulo es el objeto con sus datos, métodos y un apuntador, se le conoce como Nodo



*Ilustración 2: El nodo es un objeto con sus atributos, métodos y un apuntador*

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero, double Num) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

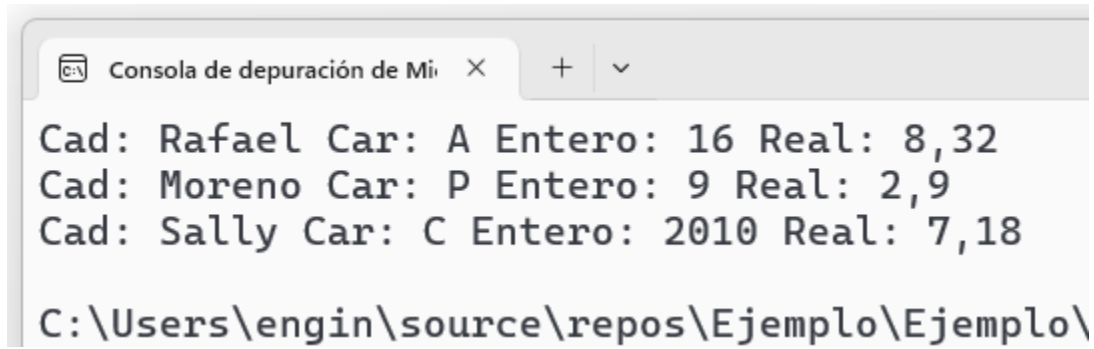
class Program {
    static void Main() {
        //Crea dos nodos separados
        Nodo primero = new("Rafael", 'A', 16, 8.32);
        Nodo segundo = new("Moreno", 'P', 9, 2.9);
        Nodo tercero = new("Sally", 'C', 2010, 7.18);

        //Une el primer nodo con el segundo, creando una simple lista
        primero.Apuntador = segundo;

        //Une el segundo nodo con el tercero, aumentando la lista
        segundo.Apuntador = tercero;

        //Imprime la lista
        primero.Imprime();
        primero.Apuntador.Imprime();
        primero.Apuntador.Apuntador.Imprime();
    }
}
```

}



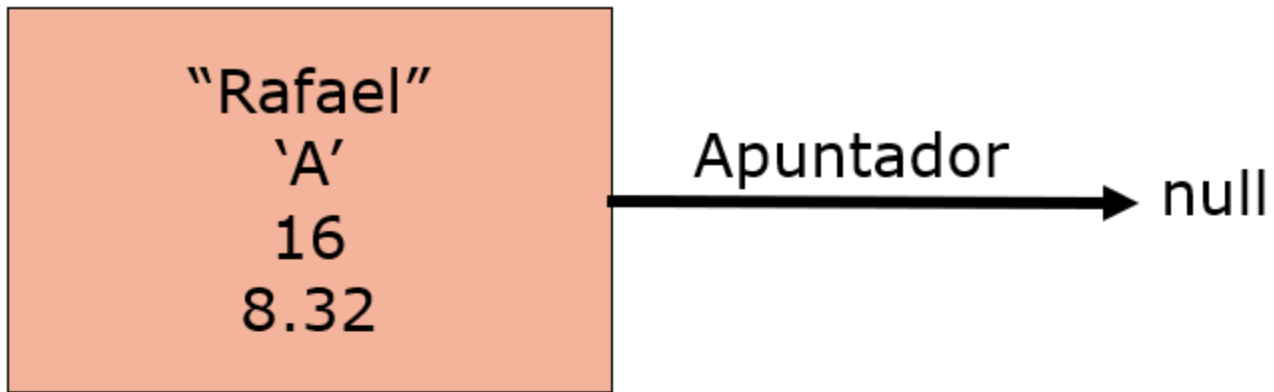
The image shows a screenshot of a Visual Studio debug console window. The title bar reads 'Consola de depuración de Mi' with a close button and a dropdown menu. The console contains three lines of text: 'Cad: Rafael Car: A Entero: 16 Real: 8,32', 'Cad: Moreno Car: P Entero: 9 Real: 2,9', and 'Cad: Sally Car: C Entero: 2010 Real: 7,18'. Below these lines, the beginning of a file path is visible: 'C:\Users\engin\source\repos\Ejemplo\Ejemplo\'.

```
Cad: Rafael Car: A Entero: 16 Real: 8,32
Cad: Moreno Car: P Entero: 9 Real: 2,9
Cad: Sally Car: C Entero: 2010 Real: 7,18
C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

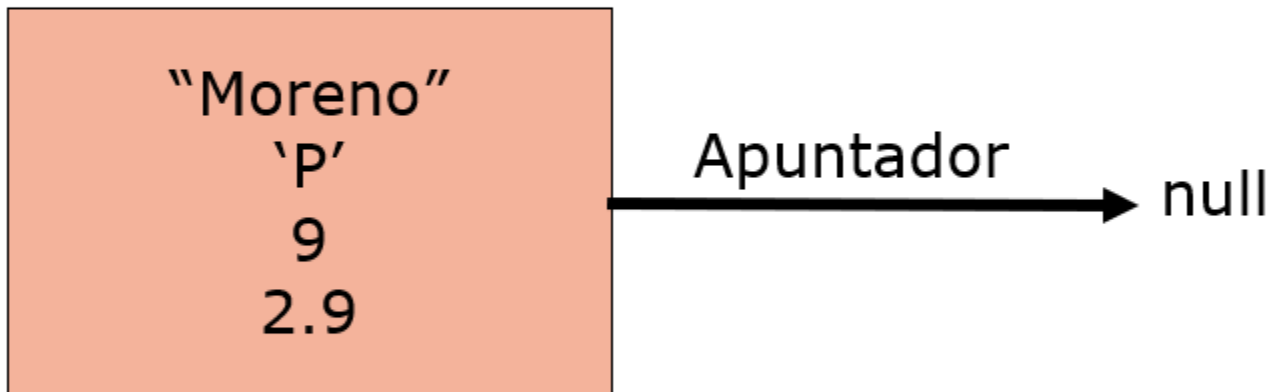
*Ilustración 3: Lista simplemente enlazada*

Se crean tres nodos:

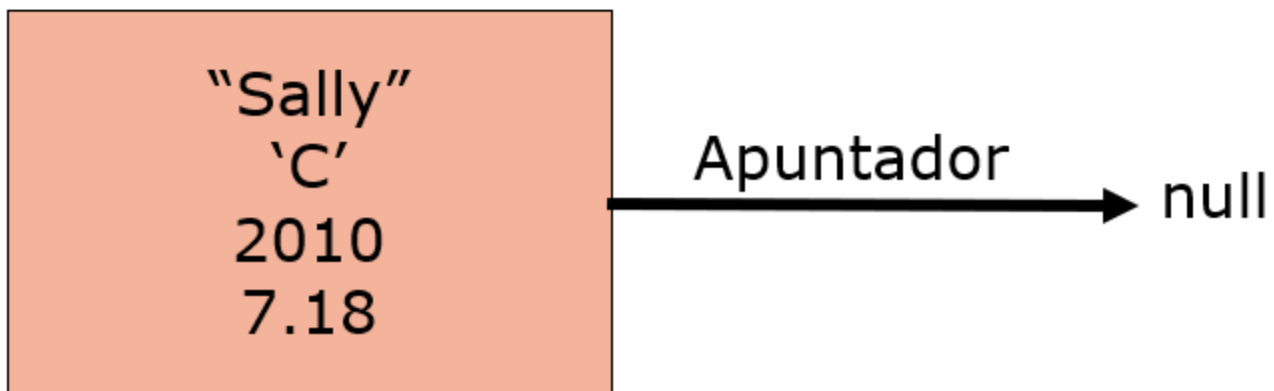
primero



segundo



tercero



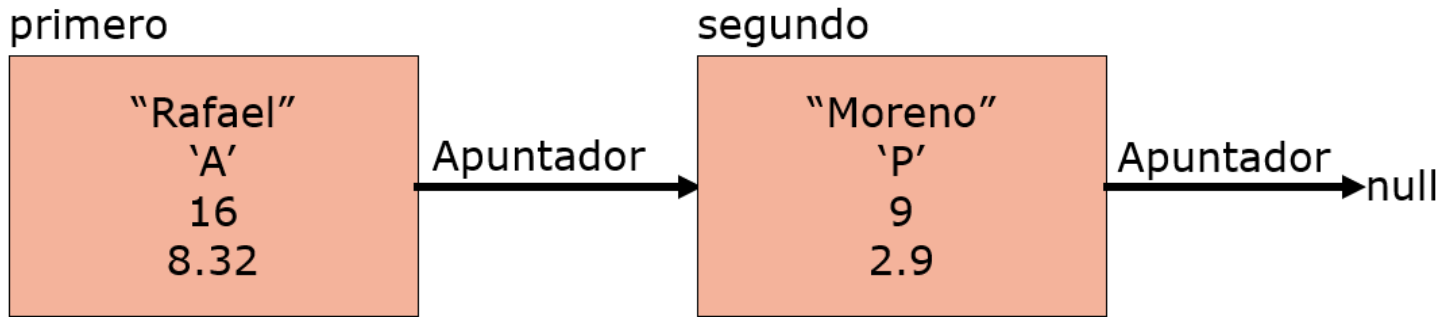
*Ilustración 4: Se crean tres nodos*

Este sería el código en C#:

```
Nodo primero = new Nodo("Rafael", 'A', 16, 8.32);  
Nodo segundo = new Nodo("Moreno", 'P', 9, 2.9);  
Nodo tercero = new Nodo("Sally", 'C', 2010, 7.18);
```



Luego debe conectarse el primer nodo con el segundo nodo:

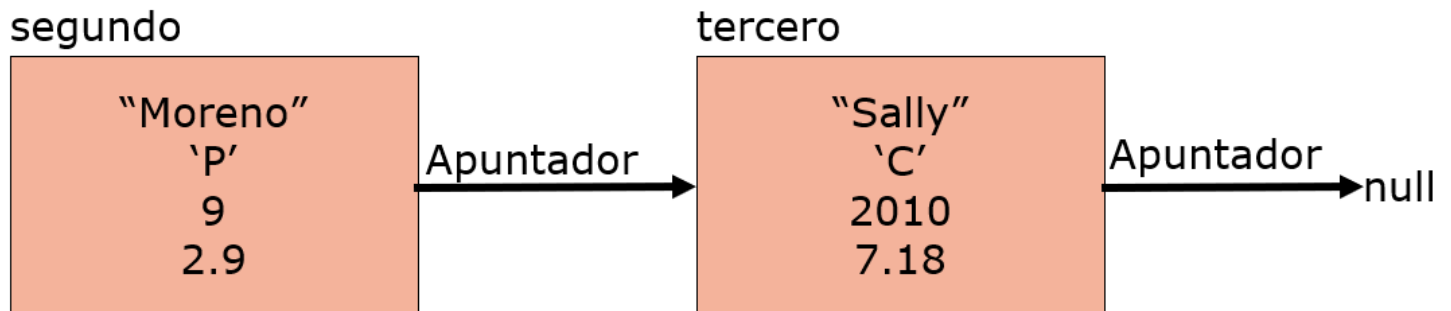


*Ilustración 5: Conectar el nodo primero con el nodo segundo*

Este sería el código en C#:

```
//Une el primer nodo con el segundo, creando una simple lista  
primero.Apuntador = segundo;
```

Luego debe conectarse el segundo nodo con el tercer nodo:



*Ilustración 6: Conexión entre nodos*

Este sería el código en C#:

```
//Une el segundo nodo con el tercero, aumentando la lista  
segundo.Apuntador = tercero;
```

## Enlazamiento continuo

Con una sola variable declarada, se va armando la lista. En primer lugar, se hacen cambios a la clase Nodo, para que en el constructor se pueda enviar el apuntador.

G/002.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

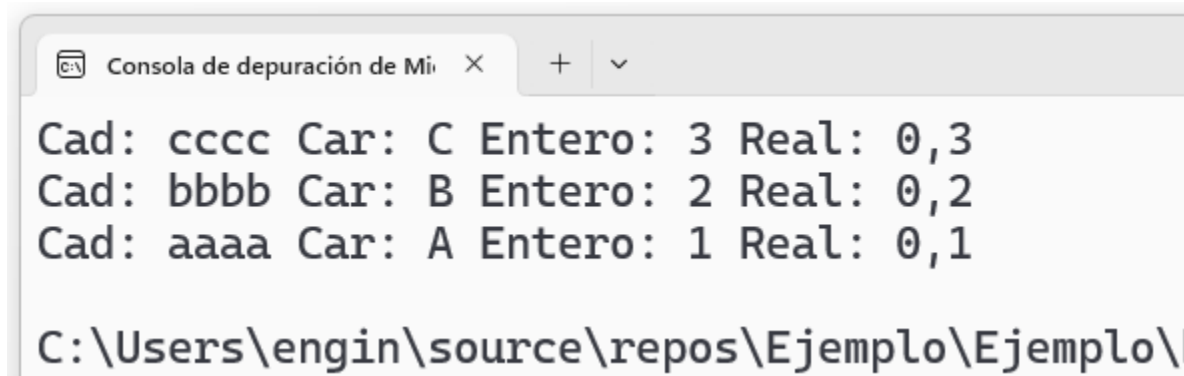
    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.Write("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);

        //Imprime la lista
        lista.Imprime(); //Primer nodo
        lista.Apuntador.Imprime(); //Segundo nodo
        lista.Apuntador.Apuntador.Imprime(); //Tercer nodo
    }
}
```

}



The image shows a screenshot of a Visual Studio debug console window. The title bar at the top reads 'Consola de depuración de Mi' followed by a close button (X) and a dropdown menu with '+' and '-' symbols. The console contains three lines of output: 'Cad: cccc Car: C Entero: 3 Real: 0,3', 'Cad: bbbb Car: B Entero: 2 Real: 0,2', and 'Cad: aaaa Car: A Entero: 1 Real: 0,1'. Below these lines, the file path 'C:\Users\engin\source\repos\Ejemplo\Ejemplo\' is visible, indicating the current file being debugged.

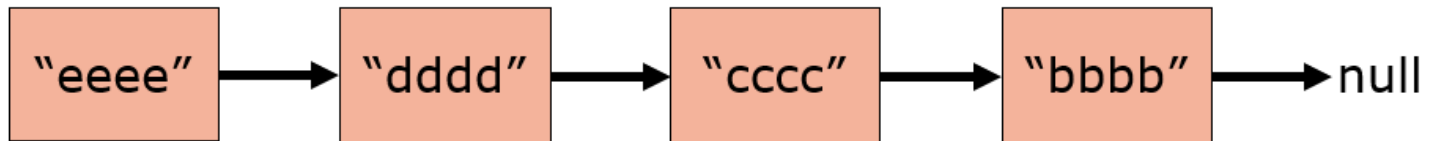
```
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: bbbb Car: B Entero: 2 Real: 0,2  
Cad: aaaa Car: A Entero: 1 Real: 0,1  
  
C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

*Ilustración 7: Enlazamiento continuo*

## Recorriendo lista simplemente enlazada

Para recorrer una lista simplemente enlazada, se debe dejar una variable que sostenga la lista y una segunda es la que la recorre. Es importante eso porque sin la variable que sostiene toda la lista, a medida que va recorriendo nodo a nodo, ise estará borrando!

lista



*Ilustración 8: Variable "lista" sostiene la lista simplemente enlazada*

G/003.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}
```

```

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);
        lista = new("ffff", 'F', 6, 0.6, lista);
        lista = new("gggg", 'G', 7, 0.7, lista);
        lista = new("hhhh", 'H', 8, 0.8, lista);
        lista = new("iiii", 'I', 9, 0.9, lista);

        //Pasea la lista, imprimiéndola
        Nodo pasea = lista;
        while (pasea != null) {
            pasea.Imprime();
            pasea = pasea.Apuntador;
        }
    }
}

```

```

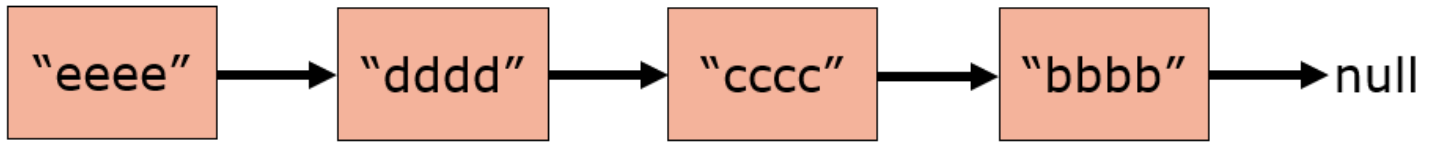
Consola de depuración de Mi
Cad: iiii Car: I Entero: 9 Real: 0,9
Cad: hhhh Car: H Entero: 8 Real: 0,8
Cad: gggg Car: G Entero: 7 Real: 0,7
Cad: ffff Car: F Entero: 6 Real: 0,6
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

C:\Users\engin\source\repos\Ejemplo\Ejemplo\

```

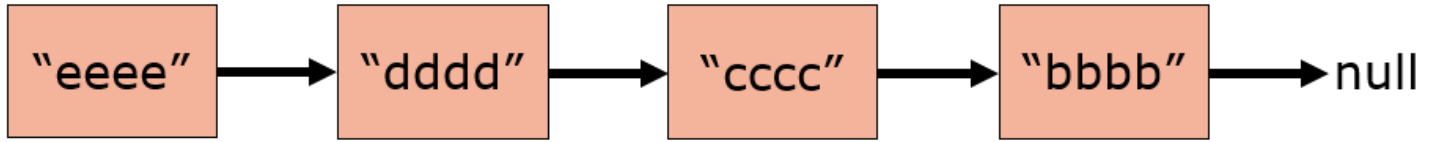
Ilustración 9: Recorriendo lista simplemente enlazada

pasea = lista



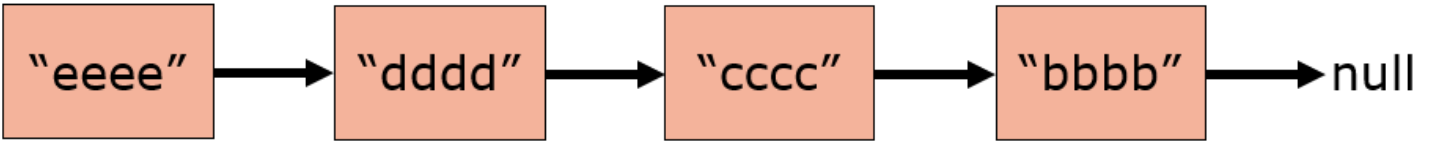
lista

pasea



lista

pasea



*Ilustración 10: La variable "pasea" va de nodo en nodo.*

## Recorriendo la lista usando un método

Se puede crear un método que recorra la lista enviándole por parámetro el apuntador de la lista. En la función se genera una copia de ese apuntador, así que puede recorrerla dentro de la función sin la preocupación de estar destruyendo la lista.

G/004.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.Write("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);
        lista = new("ffff", 'F', 6, 0.6, lista);
        lista = new("gggg", 'G', 7, 0.7, lista);
    }
}
```

```
lista = new("hhhh", 'H', 8, 0.8, lista);
lista = new("iiii", 'I', 9, 0.9, lista);

//Pasea la lista, imprimiéndola
Console.WriteLine("RECORRE PRIMERA VEZ");
ImprimeLista(lista);

Console.WriteLine("\r\nRECORRE SEGUNDA VEZ");
ImprimeLista(lista);
}

static public void ImprimeLista(Nodo pasear) {
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.Apuntador;
    }
}
}
```



```
Consola de depuración de Mi X + v

RECORRE PRIMERA VEZ
Cad: iiii Car: I Entero: 9 Real: 0,9
Cad: hhhh Car: H Entero: 8 Real: 0,8
Cad: gggg Car: G Entero: 7 Real: 0,7
Cad: ffff Car: F Entero: 6 Real: 0,6
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

RECORRE SEGUNDA VEZ
Cad: iiii Car: I Entero: 9 Real: 0,9
Cad: hhhh Car: H Entero: 8 Real: 0,8
Cad: gggg Car: G Entero: 7 Real: 0,7
Cad: ffff Car: F Entero: 6 Real: 0,6
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

Ilustración 11: Recorriendo la lista usando un método

## Tamaño de la lista

Una función puede retornar el tamaño de la lista recorriendo nodo a nodo.

G/005.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.Write("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);
        lista = new("ffff", 'F', 6, 0.6, lista);
        lista = new("gggg", 'G', 7, 0.7, lista);
        lista = new("hhhh", 'H', 8, 0.8, lista);
        lista = new("iiii", 'I', 9, 0.9, lista);
    }
}
```

```

        //Imprime el tamaño de la lista
        Console.WriteLine("Tamaño de la lista es: " + TamanoLista(lista));
    }

    //Imprime la lista
    static public void ImprimeLista(Nodo pasear) {
        while (pasear != null) {
            pasear.Imprime();
            pasear = pasear.Apuntador;
        }
    }

    //Retorna el tamaño de la lista
    static public int TamanoLista(Nodo pasear) {
        int tamano = 0;
        while (pasear != null) {
            tamano++;
            pasear = pasear.Apuntador;
        }
        return tamano;
    }
}

```

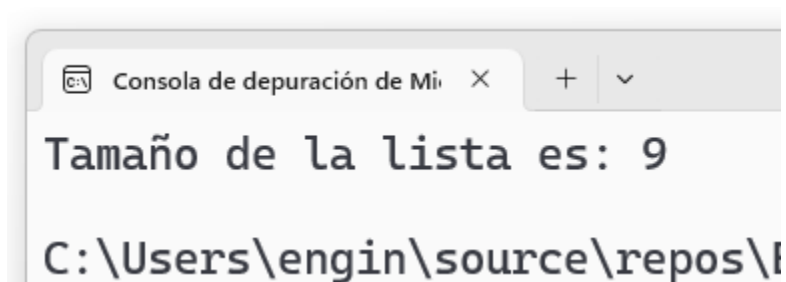


Ilustración 12: Tamaño de la lista

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);
        lista = new("ffff", 'F', 6, 0.6, lista);
        lista = new("gggg", 'G', 7, 0.7, lista);
        lista = new("hhhh", 'H', 8, 0.8, lista);
        lista = new("iiii", 'I', 9, 0.9, lista);

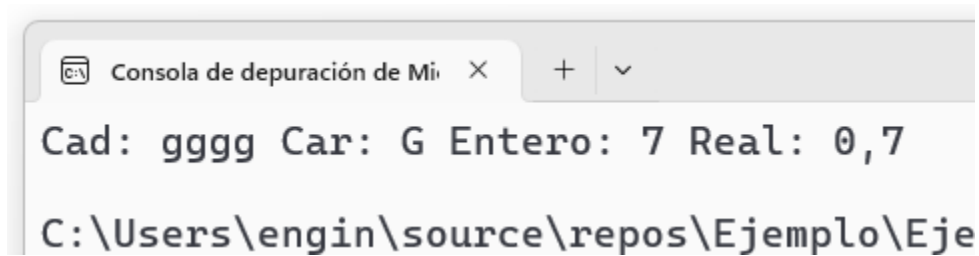
        //Trae un determinado nodo
        Nodo particular = TraeNodo(lista, 2);
    }
}
```

```

        particular.Imprime();
    }

    //Retornar nodo de determinada posición
    static public Nodo TraeNodo(Nodo pasear, int posicion) {
        int ubicacion = 0;
        while (pasear != null) {
            if (ubicacion == posicion) return pasear;
            pasear = pasear.Apuntador;
            ubicacion++;
        }
        return null;
    }
}

```

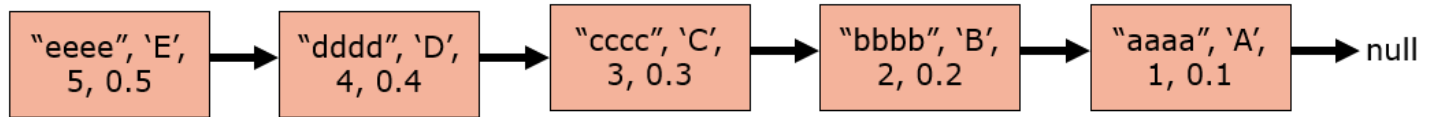


*Ilustración 13: Traer un determinado nodo*

## Adicionar un nodo en determinada posición

Para adicionar un nodo, se deben hacer varias operaciones:

lista



particular

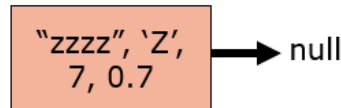
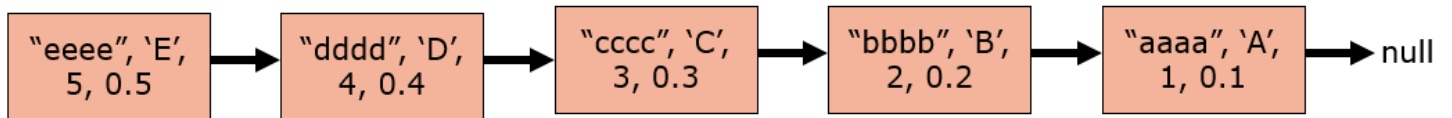


Ilustración 14: Paso 1: Lista existente y nodo nuevo a insertar

lista



particular

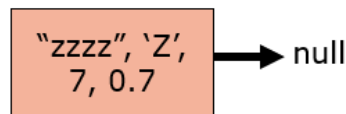


Ilustración 15: Paso 2: Ir hasta el punto de inserción

lista

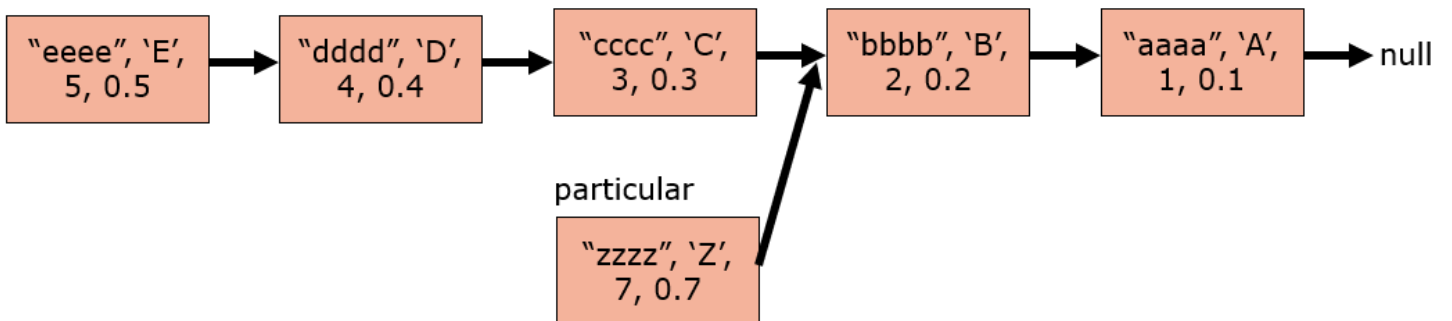


Ilustración 16: Paso 3: El nuevo nodo apunta a la posición particular de la lista

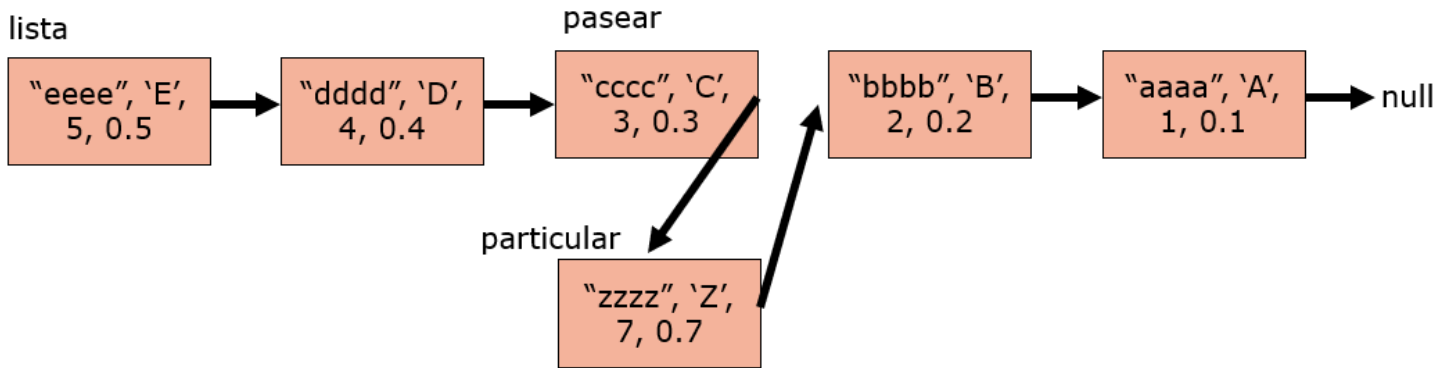


Ilustración 17: Paso 4: El nodo de la lista apunta al nuevo nodo

G/007.cs

```

namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
    }
}

```

```

    Nodo lista = new("aaaa", 'A', 1, 0.1, null);
    lista = new("bbbb", 'B', 2, 0.2, lista);
    lista = new("cccc", 'C', 3, 0.3, lista);
    lista = new("dddd", 'D', 4, 0.4, lista);
    lista = new("eeee", 'E', 5, 0.5, lista);

    //Añade un nodo en una determinada posición
    Nodo particular = new("zzzz", 'Z', 7, 0.7, null);
    lista = AdicionaNodo(particular, lista, 3);
    ImprimeLista(lista);
}

//Adiciona un nodo en determinada posición
static public Nodo AdicionaNodo(Nodo nodo, Nodo lista, int pos) {
    //Si es al inicio de la lista
    if (pos == 0) {
        nodo.Apuntador = lista;
        return nodo;
    }

    //Si es en una ubicación intermedia
    int ubicacion = 0;
    Nodo pasear = lista;
    while (pasear != null) {
        if (ubicacion + 1 == pos) {
            nodo.Apuntador = pasear.Apuntador;
            pasear.Apuntador = nodo;
            return lista;
        }
        pasear = pasear.Apuntador;
        ubicacion++;
    }

    //Si es al final de la lista
    pasear = lista;
    while (pasear.Apuntador != null)
        pasear = pasear.Apuntador;

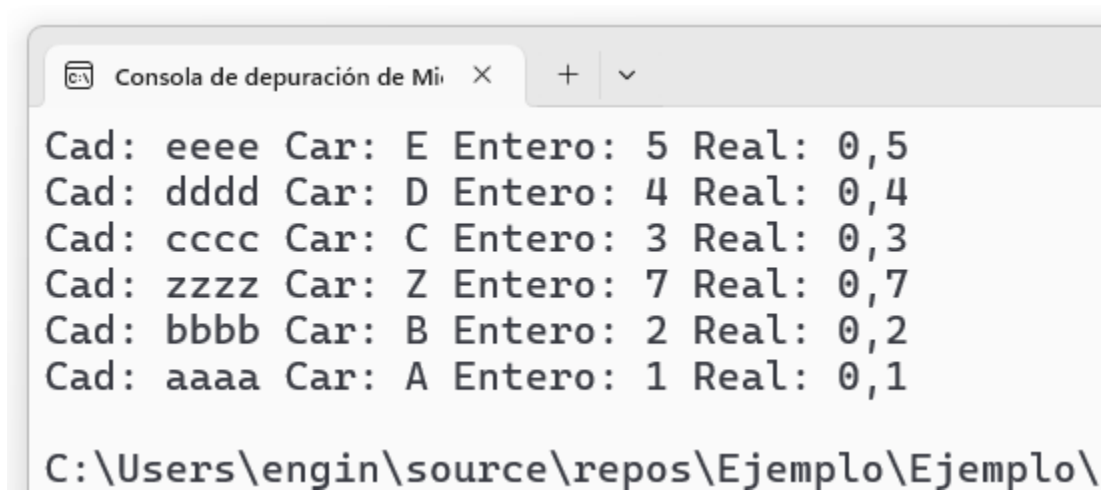
    pasear.Apuntador = nodo;
    return lista;
}

//Imprime la lista
static public void ImprimeLista(Nodo pasear) {
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.Apuntador;
    }
}

```



```
}  
}
```



```
Cad: eeee Car: E Entero: 5 Real: 0,5  
Cad: dddd Car: D Entero: 4 Real: 0,4  
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: zzzz Car: Z Entero: 7 Real: 0,7  
Cad: bbbb Car: B Entero: 2 Real: 0,2  
Cad: aaaa Car: A Entero: 1 Real: 0,1  
  
C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

*Ilustración 18: Adicionar un nodo en determinada posición*

## Borrar un nodo de una determinada posición

Para eliminar un nodo, se deben hacer varias operaciones:

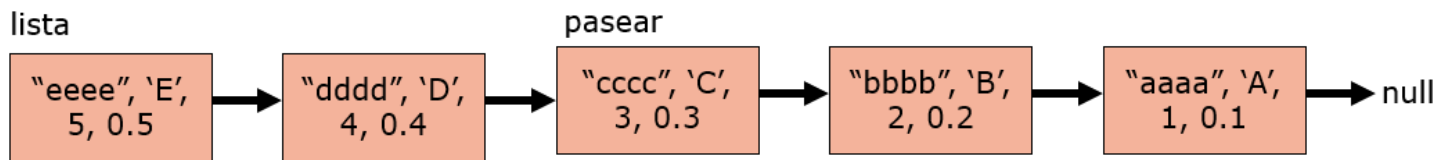


Ilustración 19: Paso 1: Lista existente e ir hasta el nodo al que se desee eliminar

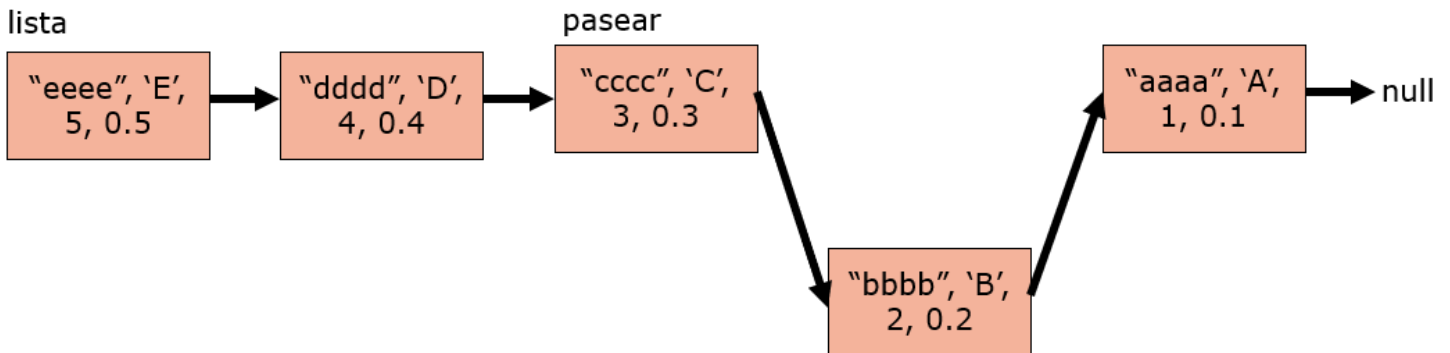


Ilustración 20: Paso 2: El nodo que se va a eliminar

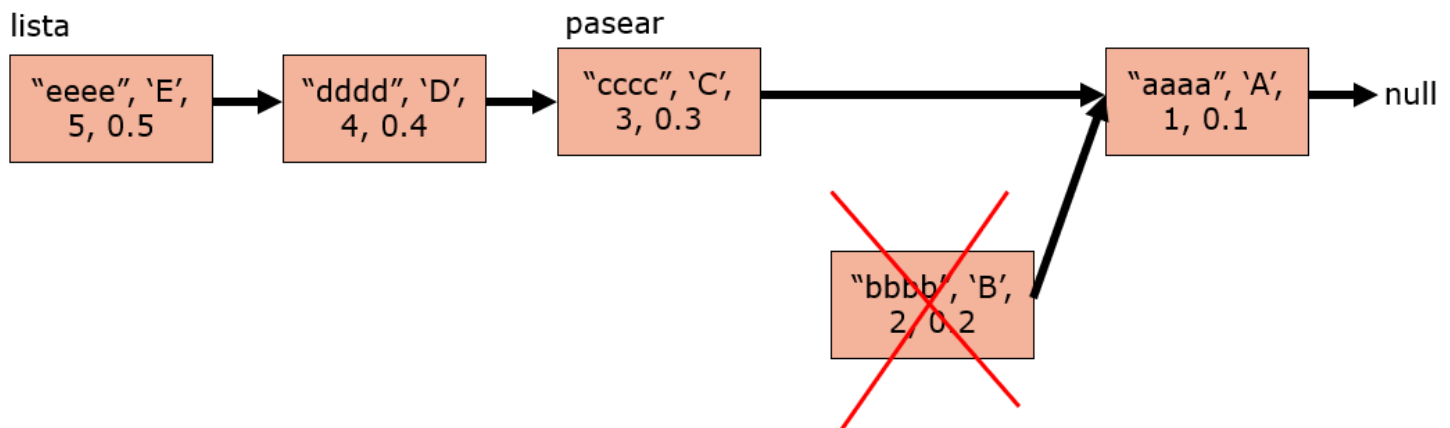


Ilustración 21: Paso 3: Se cambia el apuntador del nodo anterior al siguiente.

```

namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntador para lista simplemente enlazada
    public Nodo Apuntador;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
                double Num, Nodo Apuntador) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        this.Apuntador = Apuntador;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.Write("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);

        //Borra un nodo en una determinada posición
        lista = BorraNodo(lista, 3);
        ImprimeLista(lista);
    }

    //Borra nodo de una determinada posición
    static public Nodo BorraNodo(Nodo lista, int posicion) {
        //Si es al inicio de la lista
        if (posicion == 0) {

```

```

        lista = lista.Apuntador;
        return lista;
    }

    //Si es en una ubicación intermedia
    int ubicacion = 0;
    Nodo pasear = lista;
    while (pasear != null) {
        if (ubicacion + 1 == posicion) {
            pasear.Apuntador = pasear.Apuntador.Apuntador;
            return lista;
        }
        pasear = pasear.Apuntador;
        ubicacion++;
    }

    //Si es al final de la lista
    pasear = lista;
    while (pasear.Apuntador.Apuntador != null)
        pasear = pasear.Apuntador;

    pasear.Apuntador = null;
    return lista;
}

//Imprime la lista
static public void ImprimeLista(Nodo pasear) {
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.Apuntador;
    }
}
}

```

```

Consola de depuración de Mi
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: aaaa Car: A Entero: 1 Real: 0,1

C:\Users\engin\source\repos\Ejemplo\Ejemplo\

```

Ilustración 22: Borrar un nodo de una determinada posición

## La lista doblemente enlazada

Los nodos tienen doble conexión. Eso permite poder desplazarse de izquierda a derecha, o de derecha a izquierda. La variable que sostiene la lista puede estar en cualquier nodo.

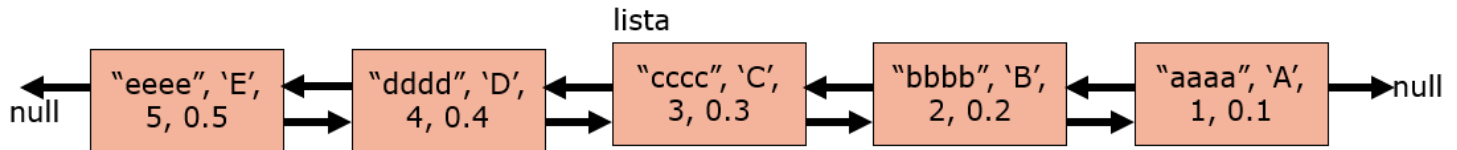


Ilustración 23: Lista doblemente enlazada

G/009.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntadores para listas doblemente enlazadas
    public Nodo NodoIzq;
    public Nodo NodoDer;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo NodoDer) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        NodoIzq = null;
        this.NodoDer = NodoDer;
        if (NodoDer != null) NodoDer.NodoIzq = this;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}
```

```

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);

        //Imprime la lista en ambos sentidos
        ImprimeIzquierdaDerecha(lista);
        ImprimeDerechaIzquierda(lista);
    }

    //Imprime la lista de izquierda a derecha
    static public void ImprimeIzquierdaDerecha(Nodo pasear) {
        Console.WriteLine("\r\nDe izquierda a derecha");

        //Debe ponerse en el primer nodo de la izquierda
        while (pasear.NodoIzq != null) {
            pasear = pasear.NodoIzq;
        }

        //Una vez en el primer nodo de la izquierda, entonces va
        //de izquierda a derecha imprimiendo
        while (pasear != null) {
            pasear.Imprime();
            pasear = pasear.NodoDer;
        }
    }

    //Imprime la lista de derecha a izquierda
    static public void ImprimeDerechaIzquierda(Nodo pasear) {
        Console.WriteLine("\r\nDe derecha a izquierda");

        //Debe ponerse en el primer nodo de la derecha
        while (pasear.NodoDer != null) {
            pasear = pasear.NodoDer;
        }

        //Una vez en el primer nodo de la derecha, entonces va
        //de derecha a izquierda imprimiendo
        while (pasear != null) {
            pasear.Imprime();
            pasear = pasear.NodoIzq;
        }
    }
}

```

}

```
Consola de depuración de Mi X + v

De izquierda a derecha
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

De derecha a izquierda
Cad: aaaa Car: A Entero: 1 Real: 0,1
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: eeee Car: E Entero: 5 Real: 0,5

C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

Ilustración 24: La lista doblemente enlazada

## Adicionar un nodo en determinada posición

Para adicionar un nodo, se deben hacer varias operaciones:

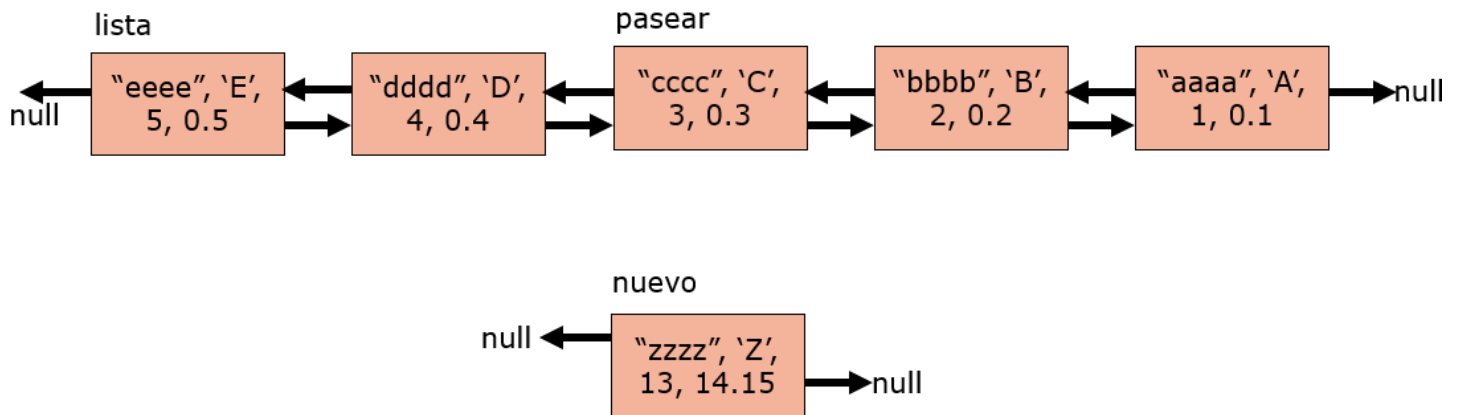


Ilustración 25: Paso 1: Lista existente y nodo nuevo a insertar. Ir al punto de inserción.

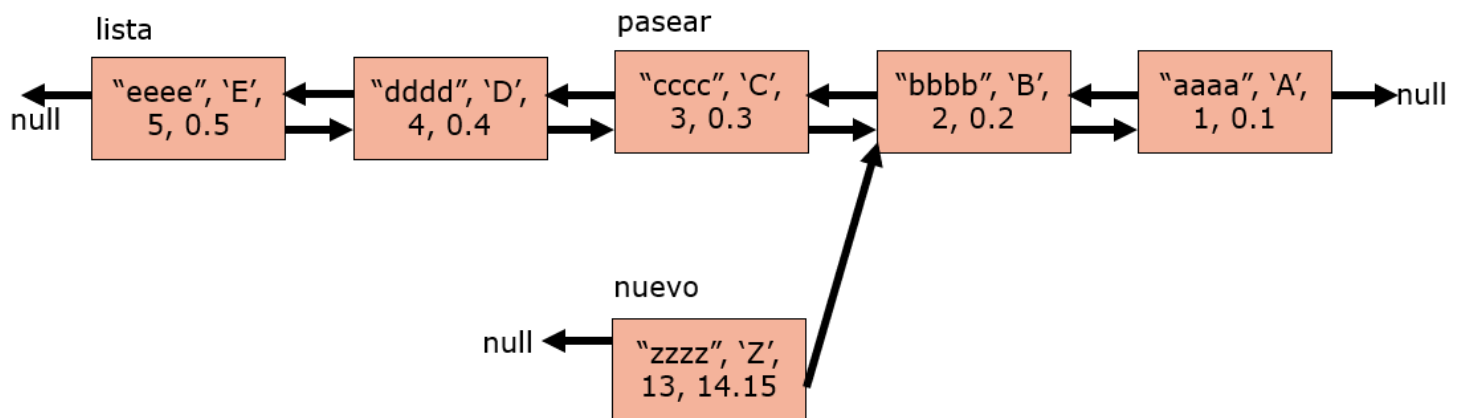


Ilustración 26: Paso 2: Poner los enlaces

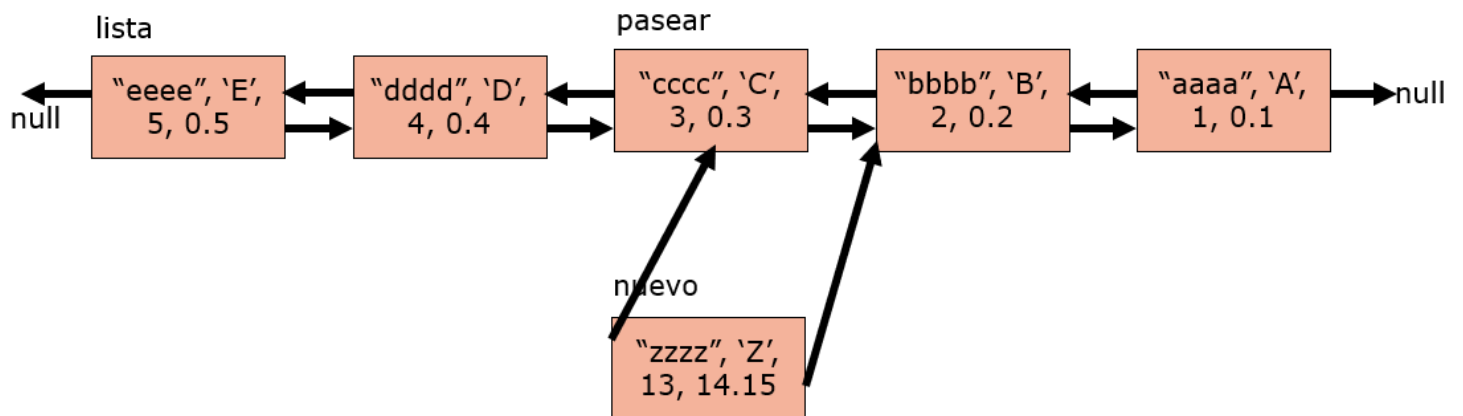


Ilustración 27: Paso 3: Poner los enlaces



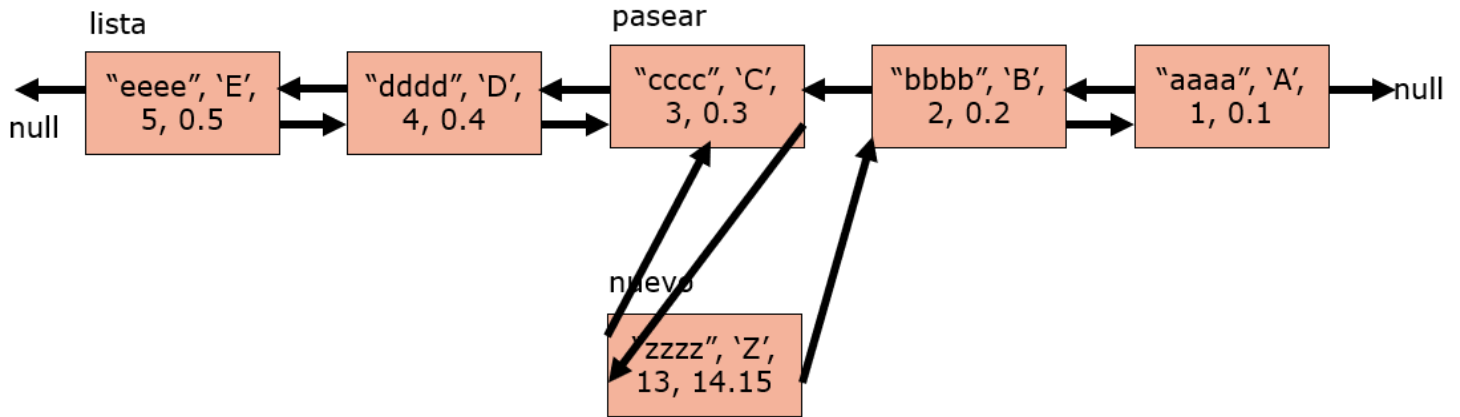


Ilustración 28: Paso 4: Poner los enlaces

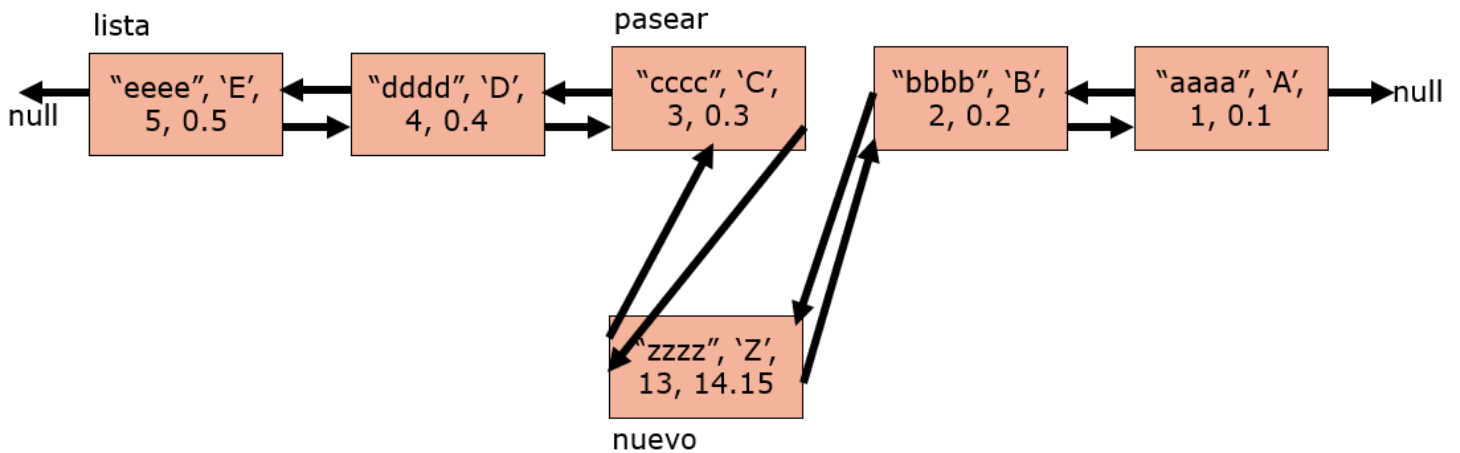


Ilustración 29: Paso 5: Poner los enlaces

G/010.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntadores para listas doblemente enlazadas
    public Nodo NodoIzq;
    public Nodo NodoDer;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo NodoDer) {
```

```

        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        NodoIzq = null;
        this.NodoDer = NodoDer;
        if (NodoDer != null) NodoDer.NodoIzq = this;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);

        //Imprime la lista en ambos sentidos
        ImprimeIzquierdaDerecha(lista);
        ImprimeDerechaIzquierda(lista);

        //Agrega un nodo a la lista doblemente enlazada
        Nodo nuevo = new("zzzz", 'Z', 13, 14.15, null);
        lista = AgregaNodo(nuevo, lista, 3);

        //Imprime la lista en ambos sentidos
        ImprimeIzquierdaDerecha(lista);
        ImprimeDerechaIzquierda(lista);
    }

    //Agrega un nodo a la lista doblemente enlazada
    static public Nodo AgregaNodo(Nodo nuevo, Nodo lista, int posicion) {
        //Debe asegurarse de ponerse en el primer nodo de la izquierda
        while (lista.NodoIzq != null) {
            lista = lista.NodoIzq;
        }

        //Si es al inicio de la lista
        if (posicion == 0) {

```

```

        nuevo.NodoDer = lista;
        lista.NodoIzq = nuevo;
        return nuevo;
    }

    //Si es en una ubicación intermedia
    int ubicacion = 0;
    Nodo pasear = lista;
    while (pasear != null) {
        if (ubicacion + 1 == posicion) {
            nuevo.NodoDer = pasear.NodoDer;
            pasear.NodoDer.NodoIzq = nuevo;
            pasear.NodoDer = nuevo;
            nuevo.NodoIzq = pasear;
            return lista;
        }
        pasear = pasear.NodoDer;
        ubicacion++;
    }

    //Si es al final de la lista
    pasear = lista;
    while (pasear.NodoDer != null)
        pasear = pasear.NodoDer;

    pasear.NodoDer = nuevo;
    nuevo.NodoIzq = pasear;
    return lista;
}

//Imprime la lista de izquierda a derecha
static public void ImprimeIzquierdaDerecha(Nodo pasear) {
    Console.WriteLine("\r\nDe izquierda a derecha");

    //Debe ponerse en el primer nodo de la izquierda
    while (pasear.NodoIzq != null) {
        pasear = pasear.NodoIzq;
    }

    //Una vez en el primer nodo de la izquierda, entonces va
    //de izquierda a derecha imprimiendo
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.NodoDer;
    }
}

```

```
//Imprime la lista de derecha a izquierda
static public void ImprimeDerechaIzquierda(Nodo pasear) {
    Console.WriteLine("\r\nDe derecha a izquierda");

    //Debe ponerse en el primer nodo de la derecha
    while (pasear.NodoDer != null) {
        pasear = pasear.NodoDer;
    }

    //Una vez en el primer nodo de la derecha, entonces va
    //de derecha a izquierda imprimiendo
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.NodoIzq;
    }
}
}
```

```
Consola de depuración de Mi X + v

De izquierda a derecha
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

De derecha a izquierda
Cad: aaaa Car: A Entero: 1 Real: 0,1
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: eeee Car: E Entero: 5 Real: 0,5

De izquierda a derecha
Cad: eeee Car: E Entero: 5 Real: 0,5
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: zzzz Car: Z Entero: 13 Real: 14,15
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: aaaa Car: A Entero: 1 Real: 0,1

De derecha a izquierda
Cad: aaaa Car: A Entero: 1 Real: 0,1
Cad: bbbb Car: B Entero: 2 Real: 0,2
Cad: zzzz Car: Z Entero: 13 Real: 14,15
Cad: cccc Car: C Entero: 3 Real: 0,3
Cad: dddd Car: D Entero: 4 Real: 0,4
Cad: eeee Car: E Entero: 5 Real: 0,5

C:\Users\engin\source\repos\Ejemplo\Ejemp
```

Ilustración 30: Adicionar un nodo en determinada posición

## Borrar un nodo de una determinada posición

Para eliminar un nodo, se deben hacer varias operaciones:

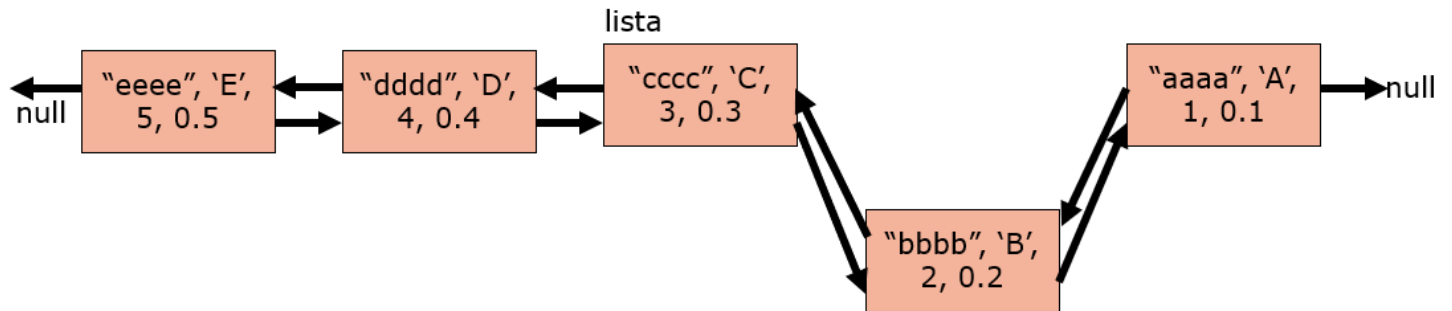


Ilustración 31: Paso 1: Lista existente e ir hasta el nodo al que se desee eliminar

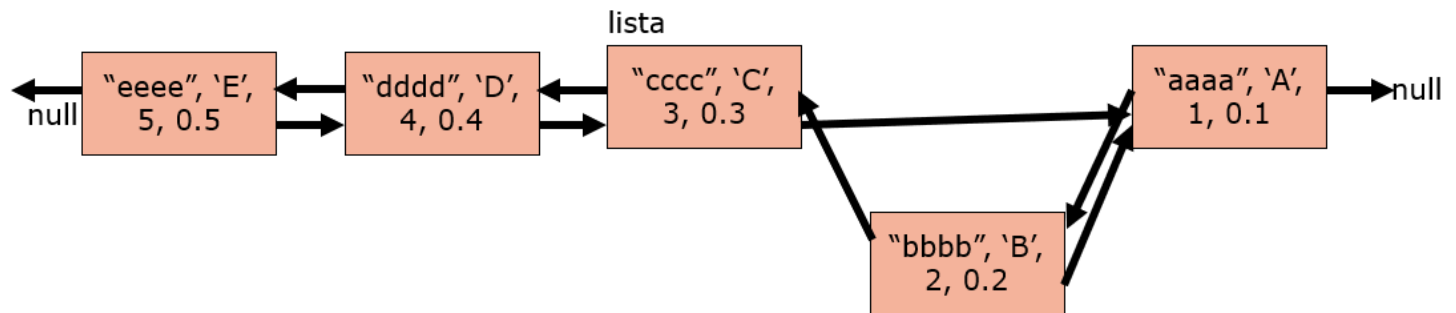


Ilustración 32: Paso 2: Modificar los apuntadores

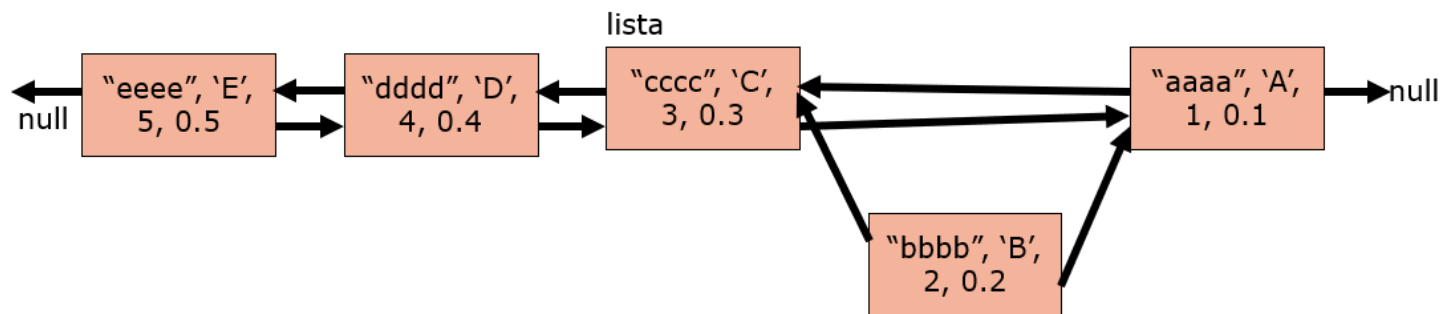


Ilustración 33: Paso 3: Modificar los apuntadores

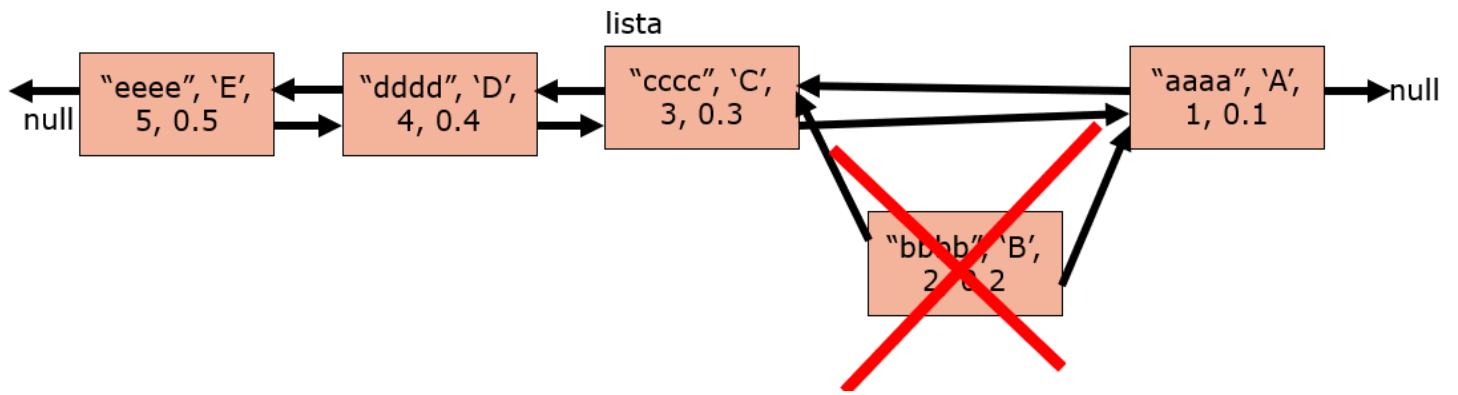


Ilustración 34: Paso 3: El nodo se destruye automáticamente al no tener quien lo sostenga

```

namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Apuntadores para listas doblemente enlazadas
    public Nodo NodoIzq;
    public Nodo NodoDer;

    //Constructor
    public Nodo(string Cad, char Car, int Entero,
        double Num, Nodo NodoDer) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        NodoIzq = null;
        this.NodoDer = NodoDer;
        if (NodoDer != null) NodoDer.NodoIzq = this;
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
    }
}

class Program {
    static void Main() {
        //Crea la lista
        Nodo lista = new("aaaa", 'A', 1, 0.1, null);
        lista = new("bbbb", 'B', 2, 0.2, lista);
        lista = new("cccc", 'C', 3, 0.3, lista);
        lista = new("dddd", 'D', 4, 0.4, lista);
        lista = new("eeee", 'E', 5, 0.5, lista);

        //Imprime la lista en ambos sentidos
        ImprimeIzquierdaDerecha(lista);
        ImprimeDerechaIzquierda(lista);
    }
}

```



```

//Agrega un nodo a la lista doblemente enlazada
lista = BorraNodo(lista, 3);

//Imprime la lista en ambos sentidos
ImprimeIzquierdaDerecha(lista);
ImprimeDerechaIzquierda(lista);
}

//Borra un nodo de la lista doblemente enlazada
static public Nodo BorraNodo(Nodo lista, int posicion) {
    //Debe asegurarse de ponerse en el
    //primer nodo de la izquierda
    while (lista.NodoIzq != null) {
        lista = lista.NodoIzq;
    }

    //Si es al inicio de la lista
    if (posicion == 0) {
        lista = lista.NodoDer;
        lista.NodoIzq = null;
        return lista;
    }

    //Si es en una ubicación intermedia
    int ubicacion = 0;
    Nodo pasear = lista;
    while (pasear != null) {
        if (ubicacion + 1 == posicion) {
            pasear.NodoDer = pasear.NodoDer.NodoDer;
            if (pasear.NodoDer != null)
                pasear.NodoDer.NodoIzq = pasear;

            return lista;
        }
        pasear = pasear.NodoDer;
        ubicacion++;
    }

    //Si es al final de la lista
    pasear = lista;
    while (pasear.NodoDer.NodoDer != null)
        pasear = pasear.NodoDer;

    pasear.NodoDer = null;
    return lista;
}

//Imprime la lista de izquierda a derecha

```

```

static public void ImprimeIzquierdaDerecha(Nodo pasear) {
    Console.WriteLine("\r\nDe izquierda a derecha");

    //Debe ponerse en el primer nodo de la izquierda
    while (pasear.NodoIzq != null) {
        pasear = pasear.NodoIzq;
    }

    //Una vez en el primer nodo de la izquierda, entonces va
    //de izquierda a derecha imprimiendo
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.NodoDer;
    }
}

//Imprime la lista de derecha a izquierda
static public void ImprimeDerechaIzquierda(Nodo pasear) {
    Console.WriteLine("\r\nDe derecha a izquierda");

    //Debe ponerse en el primer nodo de la derecha
    while (pasear.NodoDer != null) {
        pasear = pasear.NodoDer;
    }

    //Una vez en el primer nodo de la derecha, entonces va
    //de derecha a izquierda imprimiendo
    while (pasear != null) {
        pasear.Imprime();
        pasear = pasear.NodoIzq;
    }
}
}

```

### De izquierda a derecha

Cad: eeee Car: E Entero: 5 Real: 0,5  
Cad: dddd Car: D Entero: 4 Real: 0,4  
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: bbbb Car: B Entero: 2 Real: 0,2  
Cad: aaaa Car: A Entero: 1 Real: 0,1

### De derecha a izquierda

Cad: aaaa Car: A Entero: 1 Real: 0,1  
Cad: bbbb Car: B Entero: 2 Real: 0,2  
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: dddd Car: D Entero: 4 Real: 0,4  
Cad: eeee Car: E Entero: 5 Real: 0,5

### De izquierda a derecha

Cad: eeee Car: E Entero: 5 Real: 0,5  
Cad: dddd Car: D Entero: 4 Real: 0,4  
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: aaaa Car: A Entero: 1 Real: 0,1

### De derecha a izquierda

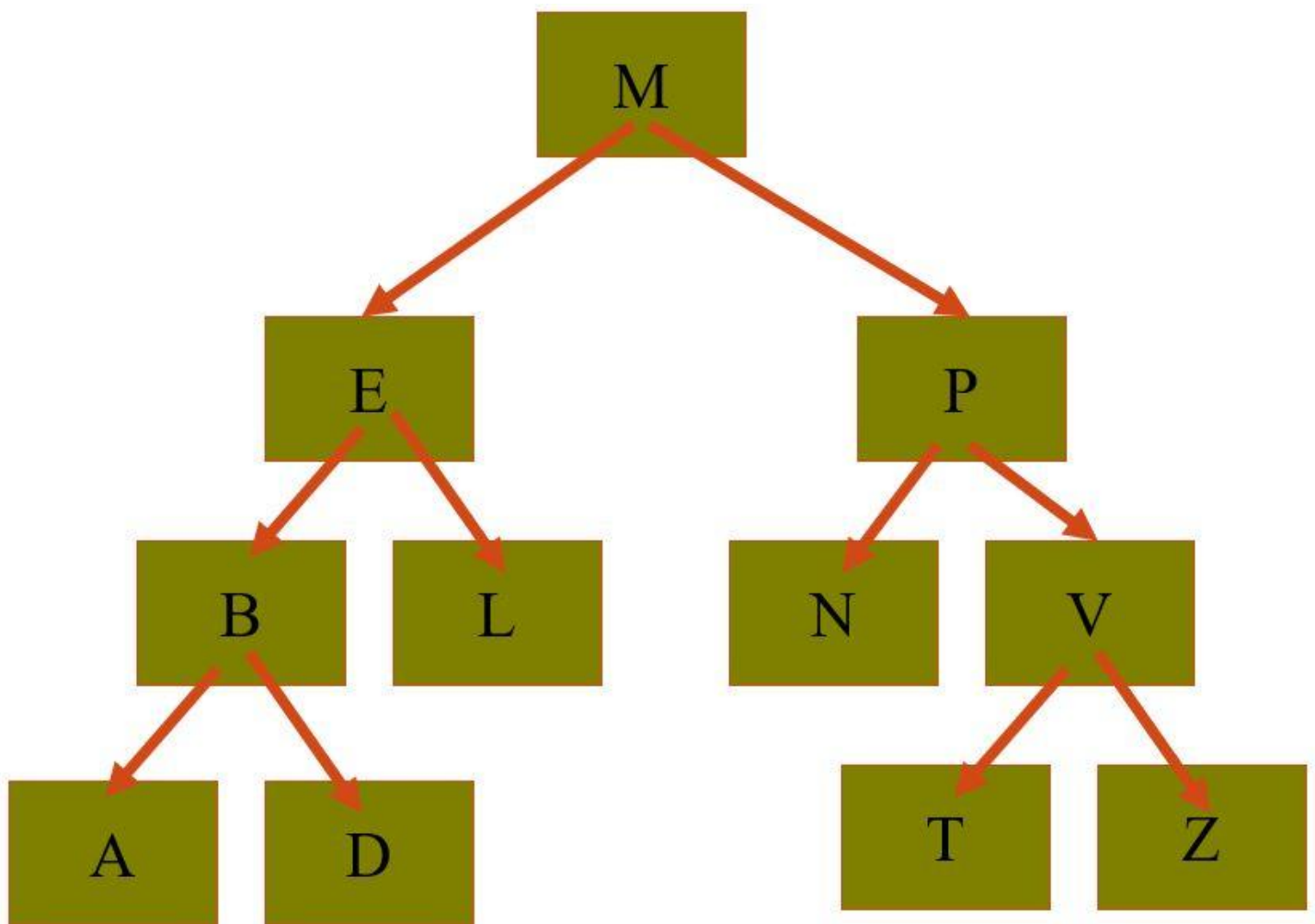
Cad: aaaa Car: A Entero: 1 Real: 0,1  
Cad: cccc Car: C Entero: 3 Real: 0,3  
Cad: dddd Car: D Entero: 4 Real: 0,4  
Cad: eeee Car: E Entero: 5 Real: 0,5

C:\Users\engin\source\repos\Ejemplo\Ejemplo\

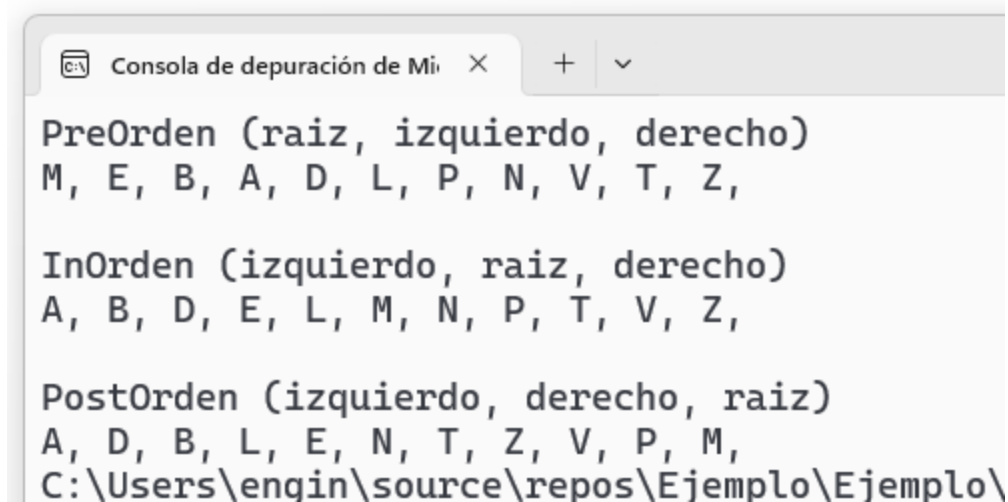
Ilustración 35: Borrar un nodo de una determinada posición

# Árbol binario

## Primer ejemplo



*Ilustración 36: Ejemplo de un árbol binario*



```
Consola de depuración de Mi X + v
PreOrden (raiz, izquierdo, derecho)
M, E, B, A, D, L, P, N, V, T, Z,

InOrden (izquierdo, raiz, derecho)
A, B, D, E, L, M, N, P, T, V, Z,

PostOrden (izquierdo, derecho, raiz)
A, D, B, L, E, N, T, Z, V, P, M,
C:\Users\enqin\source\repos\Ejemplo\Ejemplo\
```

Ilustración 37: Recorrido de un árbol binario

G/012.cs

```
namespace Ejemplo;

//Recorrido de un árbol binario
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(char Letra) {
        this.Letra = Letra;
    }
}

class Program {
    public static void Main() {
        //Crea el árbol
        Nodo Arbol = new('M');
        Arbol.Izquierda = new('E');
        Arbol.Derecha = new('P');
        Arbol.Izquierda.Izquierda = new('B');
        Arbol.Izquierda.Derecha = new('L');
        Arbol.Izquierda.Izquierda.Izquierda = new('A');
        Arbol.Izquierda.Izquierda.Derecha = new('D');
        Arbol.Derecha.Izquierda = new('N');
        Arbol.Derecha.Derecha = new('V');
        Arbol.Derecha.Derecha.Izquierda = new('T');
        Arbol.Derecha.Derecha.Derecha = new('Z');

        //Recorridos
    }
}
```

```

        Console.WriteLine("PreOrden (raiz, izquierdo, derecho)");
        PreOrden(Arbol);

        Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
        InOrden(Arbol);

        Console.WriteLine("\n\nPostOrden (izquierdo, derecho, raiz)");
        PostOrden(Arbol);
    }

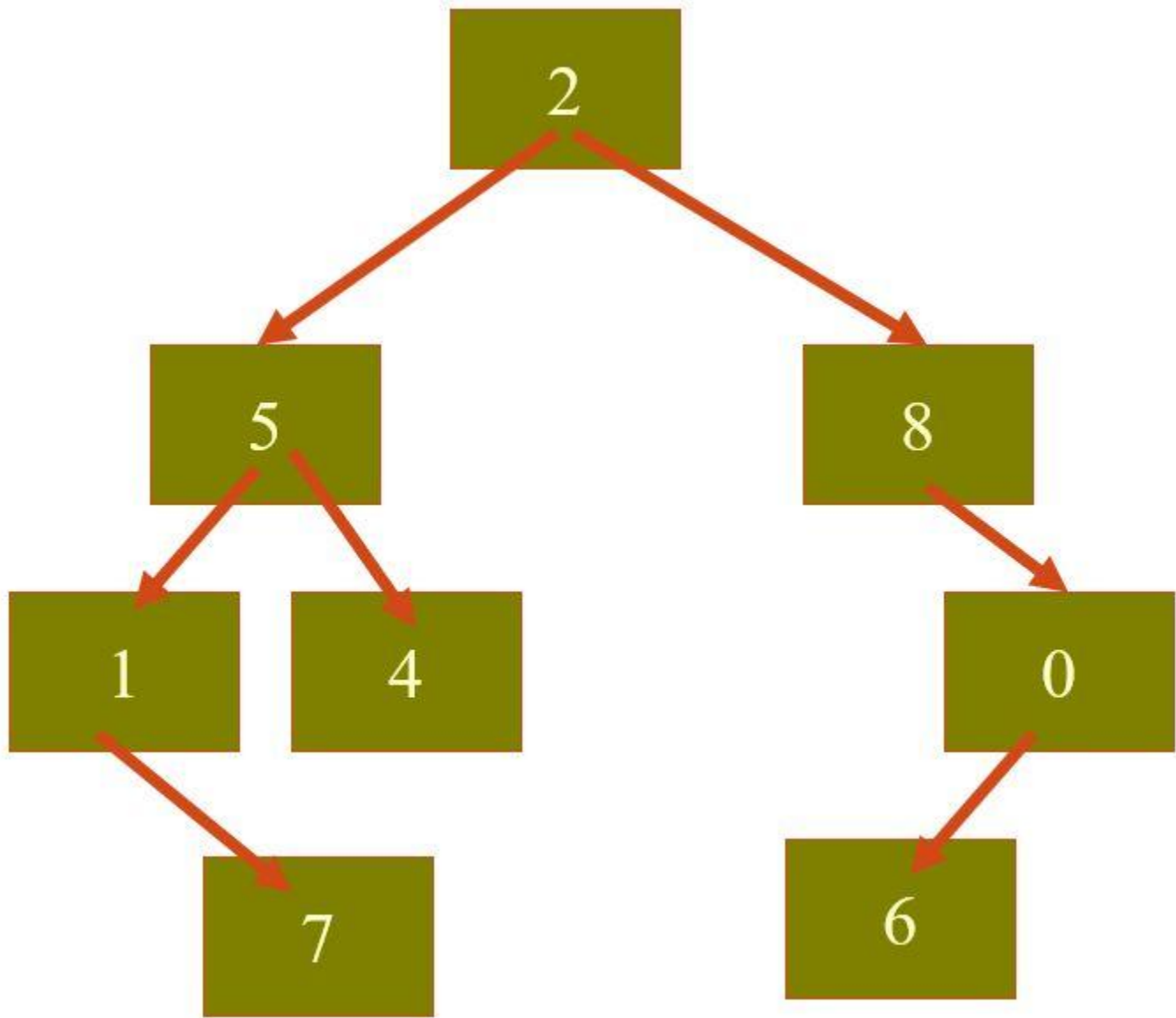
    static void PreOrden(Nodo Arbol) {
        if (Arbol != null) {
            Console.Write(Arbol.Letra + ", ");
            PreOrden(Arbol.Izquierda);
            PreOrden(Arbol.Derecha);
        }
    }

    static void InOrden(Nodo Arbol) {
        if (Arbol != null) {
            InOrden(Arbol.Izquierda);
            Console.Write(Arbol.Letra + ", ");
            InOrden(Arbol.Derecha);
        }
    }

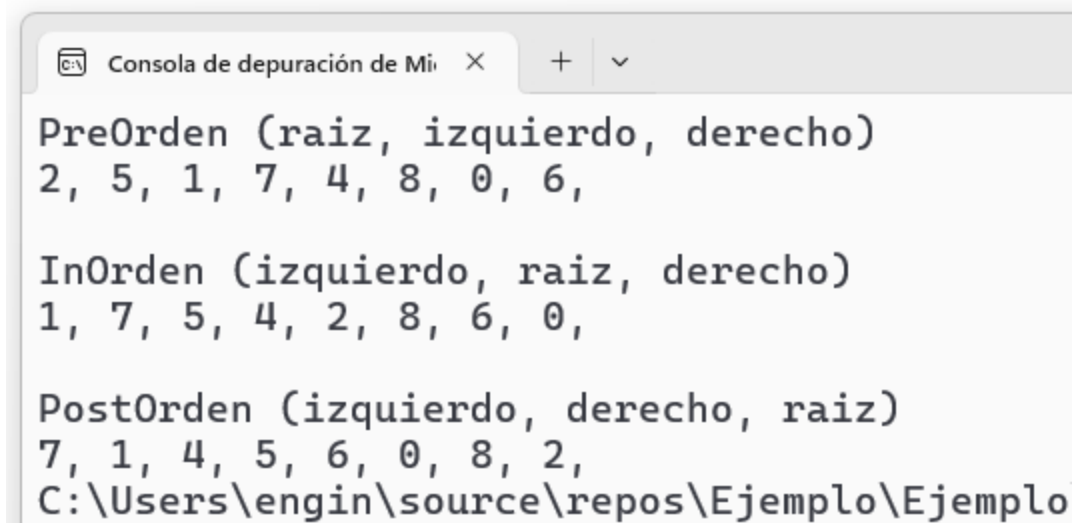
    static void PostOrden(Nodo Arbol) {
        if (Arbol != null) {
            PostOrden(Arbol.Izquierda);
            PostOrden(Arbol.Derecha);
            Console.Write(Arbol.Letra + ", ");
        }
    }
}

```

## Segundo ejemplo



*Ilustración 38: Ejemplo de árbol binario*



```
Consola de depuración de Mi X + v

PreOrden (raiz, izquierdo, derecho)
2, 5, 1, 7, 4, 8, 0, 6,

InOrden (izquierdo, raiz, derecho)
1, 7, 5, 4, 2, 8, 6, 0,

PostOrden (izquierdo, derecho, raiz)
7, 1, 4, 5, 6, 0, 8, 2,
C:\Users\engin\source\repos\Ejemplo\Ejemplo
```

Ilustración 39: Recorrido de un árbol binario

G/013.cs

```
namespace Ejemplo;

//Recorridos de un árbol binario
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(char Letra) {
        this.Letra = Letra;
    }
}

class Program {
    static void Main(string[] args) {
        //Crea el árbol
        Nodo Arbol = new('2');
        Arbol.Izquierda = new('5');
        Arbol.Derecha = new('8');
        Arbol.Izquierda.Izquierda = new('1');
        Arbol.Izquierda.Derecha = new('4');
        Arbol.Izquierda.Izquierda.Derecha = new('7');
        Arbol.Derecha.Derecha = new('0');
        Arbol.Derecha.Derecha.Izquierda = new('6');

        //Recorridos
        Console.WriteLine("PreOrden (raiz, izquierdo, derecho)");
        PreOrden(Arbol);
    }
}
```



```

        Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
        InOrden(Arbol);

        Console.WriteLine("\n\nPostOrden (izquierdo, derecho, raiz)");
        PostOrden(Arbol);
    }

    static void PreOrden(Nodo Arbol) {
        if (Arbol != null) {
            Console.Write(Arbol.Letra + ", ");
            PreOrden(Arbol.Izquierda);
            PreOrden(Arbol.Derecha);
        }
    }

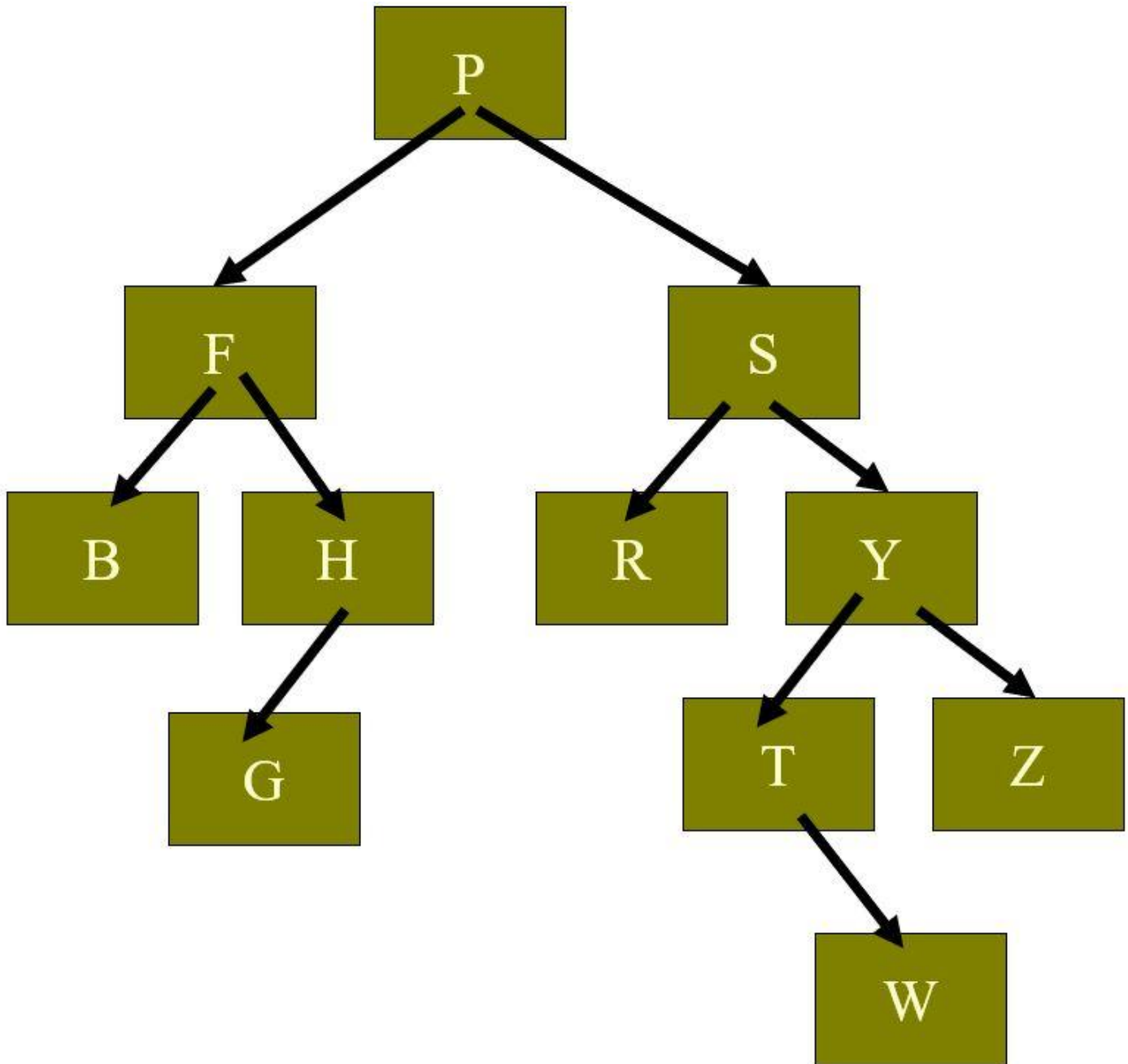
    static void InOrden(Nodo Arbol) {
        if (Arbol != null) {
            InOrden(Arbol.Izquierda);
            Console.Write(Arbol.Letra + ", ");
            InOrden(Arbol.Derecha);
        }
    }

    static void PostOrden(Nodo Arbol) {
        if (Arbol != null) {
            PostOrden(Arbol.Izquierda);
            PostOrden(Arbol.Derecha);
            Console.Write(Arbol.Letra + ", ");
        }
    }
}

```

## Recorrido iterativo (no recursivo)

El siguiente árbol binario será recorrido en forma iterativa. Se requiere para ello el uso de Pilas



*Ilustración 40: Árbol binario*

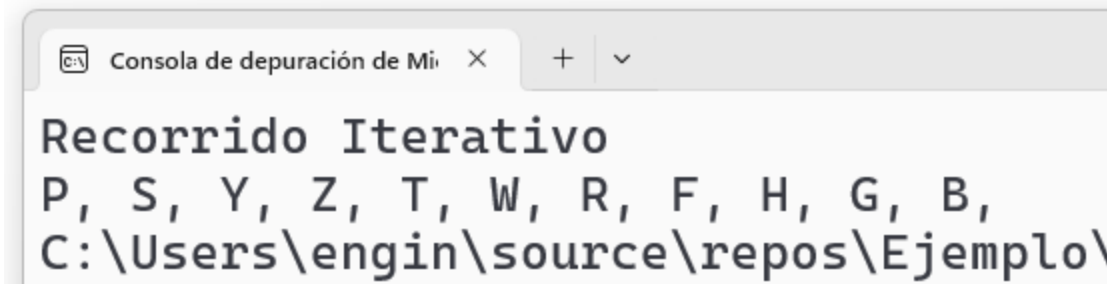


Ilustración 41: Recorrido iterativo (no recursivo)

G/014.cs

```
namespace Ejemplo;

//Recorrido (no recursivo) de un árbol binario
//Nodo de un árbol binario
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(char Letra) {
        this.Letra = Letra;
    }
}

//Nodo de una pila para recorrer iterativamente un árbol binario
class NodoPila {
    public NodoPila Flecha;
    public Nodo Raiz;
    public NodoPila(Nodo Raiz, NodoPila Flecha) {
        this.Raiz = Raiz;
        this.Flecha = Flecha;
    }
}

class Program {
    static void Main(string[] args) {
        //Crea el árbol
        Nodo Arbol = new('P');
        Arbol.Izquierda = new('F');
        Arbol.Derecha = new('S');
        Arbol.Izquierda.Izquierda = new('B');
        Arbol.Izquierda.Derecha = new('H');
        Arbol.Izquierda.Derecha.Izquierda = new('G');
        Arbol.Derecha.Izquierda = new('R');
        Arbol.Derecha.Derecha = new('Y');
        Arbol.Derecha.Derecha.Izquierda = new('T');
```

```

Arbol.Derecha.Derecha.Derecha = new('Z');
Arbol.Derecha.Derecha.Izquierda.Derecha = new('W');

//Recorrido iterativo
Console.WriteLine("Recorrido Iterativo");
Iterativo(Arbol);
}

public static void Iterativo(Nodo arbol) {
    //Usa una pila para guardar
    NodoPila inicia = new NodoPila(arbol, null);
    do {
        //Una variable tmp para ver el nodo del árbol
        Nodo tmp = inicia.Raiz;

        //Imprime el valor del nodo del árbol
        Console.Write(tmp.Letra + ", ");

        //Se quita un elemento de la pila
        inicia = inicia.Flecha;

        if (tmp.Izquierda != null)
            //Si el nodo de árbol tiene un hijo a la
            //izquierda entonces agrega este a la pila
            inicia = new NodoPila(tmp.Izquierda, inicia);

        if (tmp.Derecha != null)
            //Si el nodo de árbol tiene un hijo a la derecha
            //entonces agrega este a la pila
            inicia = new NodoPila(tmp.Derecha, inicia);

    } while (inicia != null); //Hasta que se vacíe la pila
}
}

```

```
namespace Ejemplo;

//Crear un árbol binario al azar

//Nodo de un árbol binario
class Nodo {
    public int Numero { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(int Numero) {
        this.Numero = Numero;
    }
}

class Program {
    static void Main(string[] args) {
        Random azar = new Random();
        Nodo Arbol = new(azar.Next(100));

        //Crea el árbol binario
        for (int cont = 1; cont <= 10; cont++)
            AzarNodoArbol(azar, Arbol);

        //Recorridos
        Console.WriteLine("\n\nPreOrden (raiz, izquierdo, derecho)");
        preOrden(Arbol);

        Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
        inOrden(Arbol);

        Console.WriteLine("\n\nPostOrden (izquierdo, derecho, raiz)");
        postOrden(Arbol);
    }

    //Pone un nodo en una posición al azar
    static void AzarNodoArbol(Random azar, Nodo Raiz) {
        //Por debajo de 0.5 pone una rama a la izquierda
        if (azar.NextDouble() < 0.5) {
            if (Raiz.Izquierda == null)
                Raiz.Izquierda = new(azar.Next(100));
            else
                AzarNodoArbol(azar, Raiz.Izquierda);
        }
    }
}
```

```

    }
    else {
        if (Raiz.Derecha == null)
            Raiz.Derecha = new(azar.Next(100));
        else
            AzarNodoArbol(azar, Raiz.Derecha);
    }
}

static void inOrden(Nodo Arbol) {
    if (Arbol != null) {
        inOrden(Arbol.Izquierda);
        Console.Write(Arbol.Numero + ", ");
        inOrden(Arbol.Derecha);
    }
}

static void preOrden(Nodo Arbol) {
    if (Arbol != null) {
        Console.Write(Arbol.Numero + ", ");
        preOrden(Arbol.Izquierda);
        preOrden(Arbol.Derecha);
    }
}

static void postOrden(Nodo Arbol) {
    if (Arbol != null) {
        postOrden(Arbol.Izquierda);
        postOrden(Arbol.Derecha);
        Console.Write(Arbol.Numero + ", ");
    }
}
}

```

```
Consola de depuración de Mi  X  +  v

PreOrden (raiz, izquierdo, derecho)
64, 86, 60, 44, 93, 94, 19, 34, 89, 51, 3,

InOrden (izquierdo, raiz, derecho)
60, 44, 86, 93, 64, 34, 89, 19, 51, 94, 3,

PostOrden (izquierdo, derecho, raiz)
44, 60, 93, 86, 89, 34, 51, 19, 3, 94, 64,
C:\Users\engin\source\repos\Ejemplo\Ejemplo\
```

*Ilustración 42: Generar árboles binarios al azar*

## Ordenamiento usando un árbol binario

A medida que se van agregando nodos a un árbol binario, los acomoda de tal forma que al leerlo en InOrden aparece ordenado el contenido.

G/016.cs

```
namespace Ejemplo;

//Ordenar con un árbol binario

//Nodo de un árbol binario
class Nodo {
    public int Numero { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(int Numero) {
        this.Numero = Numero;
    }
}

class Program {
    static void Main(string[] args) {
        //Va agregando nodo a nodo y los va ordenando
        Nodo Arbol = new(27);
        AgregaNodo(7, Arbol);
        AgregaNodo(17, Arbol);
        AgregaNodo(2, Arbol);
        AgregaNodo(5, Arbol);
        AgregaNodo(19, Arbol);
        AgregaNodo(15, Arbol);
        AgregaNodo(9, Arbol);
        AgregaNodo(10, Arbol);
        AgregaNodo(-1, Arbol);
        AgregaNodo(18, Arbol);
        AgregaNodo(3, Arbol);

        //Al leer en inorden el arbol, los datos salen ordenados
        Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
        InOrden(Arbol);
    }

    static void AgregaNodo(int Valor, Nodo Raiz) {
        if (Valor <= Raiz.Numero) {
            if (Raiz.Izquierda == null)
```

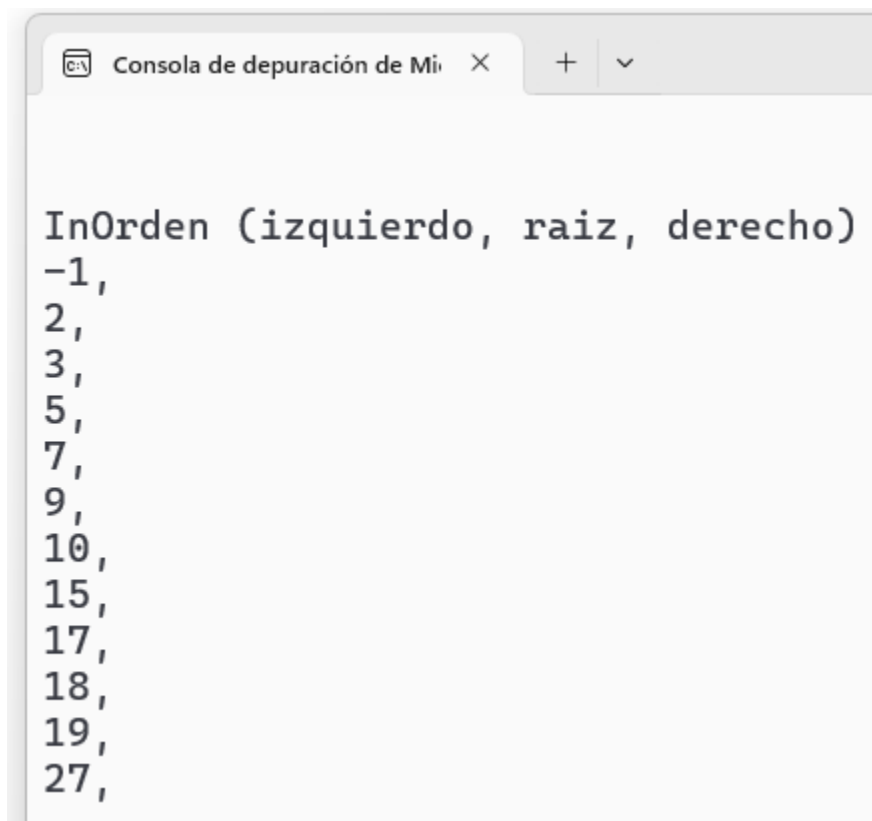


```

        Raiz.Izquierda = new(Valor);
    else
        AgregaNodo(Valor, Raiz.Izquierda);
    }
    else {
        if (Raiz.Derecha == null)
            Raiz.Derecha = new(Valor);
        else
            AgregaNodo(Valor, Raiz.Derecha);
    }
}

static void InOrden(Nodo Arbol) {
    if (Arbol != null) {
        InOrden(Arbol.Izquierda);
        Console.WriteLine(Arbol.Numero + ", ");
        InOrden(Arbol.Derecha);
    }
}
}

```



*Ilustración 43: Ordenamiento usando un árbol binario*

```
namespace Ejemplo;

//Ordenar, buscar en árbol binario ordenado,
//número de nodos y altura del árbol
class Nodo {
    public int Numero { get; set; }
    public Nodo Izquierda;
    public Nodo Derecha;

    public Nodo(int Numero) {
        this.Numero = Numero;
        this.Izquierda = null;
        this.Derecha = null;
    }
}

class Program {

    public static void Main() {

        Nodo Arbol = new(27);
        AgregaNodo(7, Arbol);
        AgregaNodo(17, Arbol);
        AgregaNodo(2, Arbol);
        AgregaNodo(5, Arbol);
        AgregaNodo(19, Arbol);
        AgregaNodo(15, Arbol);
        AgregaNodo(9, Arbol);
        AgregaNodo(10, Arbol);
        AgregaNodo(-1, Arbol);
        AgregaNodo(18, Arbol);
        AgregaNodo(3, Arbol);

        //Al leer en inorden el arbol, los datos salen ordenados
        Console.WriteLine("Valores ordenados");
        InOrden(Arbol);

        //Ahora a buscar un determinado valor
        Console.Write("\r\n\r\nBusca el valor: 185...");
        bool encontrar = BuscaArbol(Arbol, 185);
        if (encontrar)
            Console.WriteLine(" Valor encontrado");
        else
            Console.WriteLine(" Valor NO encontrado");
    }
}
```

```

//Busca valor en el árbol
Console.Write("\r\nBusca el valor: 19...");
Nodo nodoEncuentra = Buscanodo(Arbol, 19);
if (nodoEncuentra != null)
    Console.WriteLine(" Valor encontrado");
else
    Console.WriteLine(" Valor no encontrado");

//Contar los nodos
Console.WriteLine("\r\nTotal nodos: " + CuentaNodosArbol(Arbol));

//Altura del árbol
Console.WriteLine("\nAltura del árbol: " + AlturaArbol(Arbol));
}

public static void AgregaNodo(int Valor, Nodo Raiz) {
    if (Valor <= Raiz.Numero) {
        if (Raiz.Izquierda == null)
            Raiz.Izquierda = new(Valor);
        else
            AgregaNodo(Valor, Raiz.Izquierda);
    }
    else {
        if (Raiz.Derecha == null)
            Raiz.Derecha = new(Valor);
        else
            AgregaNodo(Valor, Raiz.Derecha);
    }
}

//Recorrido del árbol en InOrden
static void InOrden(Nodo Arbol) {
    if (Arbol != null) {
        InOrden(Arbol.Izquierda);
        Console.Write(Arbol.Numero + ", ");
        InOrden(Arbol.Derecha);
    }
}

//Retorna true si encuentra el valor en el árbol binario
static bool BuscaArbol(Nodo Arbol, int valor) {
    if (Arbol != null) {
        if (Arbol.Numero == valor) return true;
        bool encuentraI = BuscaArbol(Arbol.Izquierda, valor);
        bool encuentraD = BuscaArbol(Arbol.Derecha, valor);
        if (encuentraI || encuentraD) return true;
    }
}

```

```

        return false;
    }

    //Retorna el Nodo donde se encuentra el valor buscado
    static Nodo Buscanodo(Nodo Raiz, int valor) {
        if (valor == Raiz.Numero)
            return Raiz;

        if (valor < Raiz.Numero && Raiz.Izquierda != null)
            return Buscanodo(Raiz.Izquierda, valor);

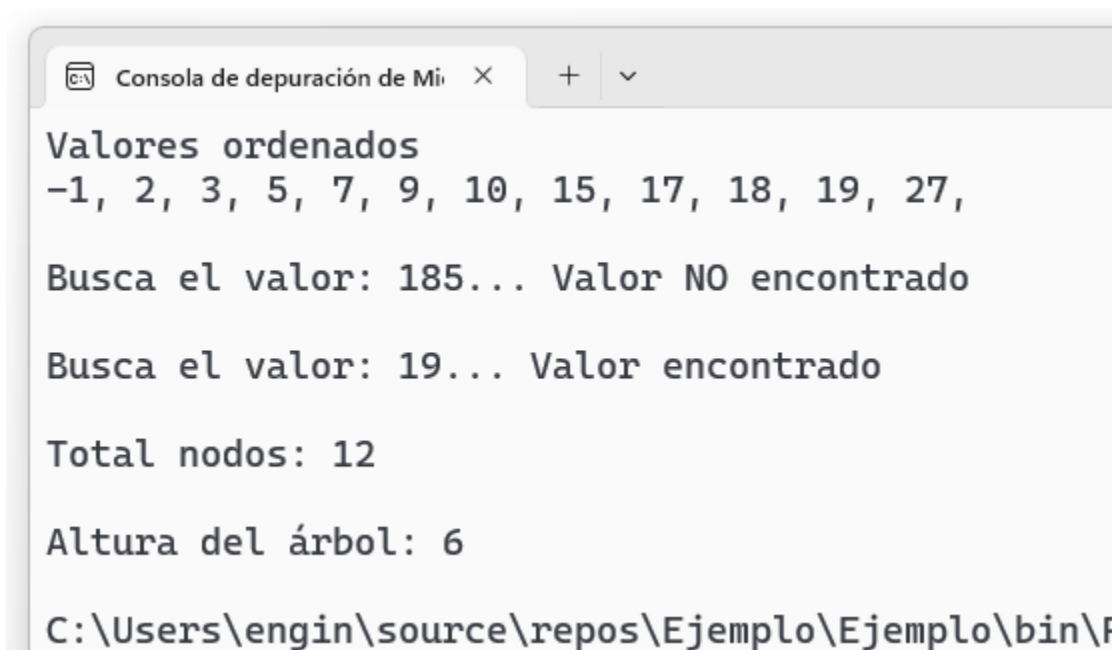
        if (valor > Raiz.Numero && Raiz.Derecha != null)
            return Buscanodo(Raiz.Derecha, valor);

        return null;
    }

    //Cuenta los nodos de un árbol
    static int CuentaNodosArbol(Nodo Arbol) {
        if (Arbol == null) return 0;
        int contarI = CuentaNodosArbol(Arbol.Izquierda);
        int contarD = CuentaNodosArbol(Arbol.Derecha);
        return contarI + contarD + 1;
    }

    //Calcula la altura de un árbol
    static int AlturaArbol(Nodo Arbol) {
        if (Arbol == null) return 0;
        int alturaI = AlturaArbol(Arbol.Izquierda);
        int alturaD = AlturaArbol(Arbol.Derecha);
        if (alturaI > alturaD) return alturaI + 1;
        return alturaD + 1;
    }
}

```



```
Consola de depuración de Mi...  
Valores ordenados  
-1, 2, 3, 5, 7, 9, 10, 15, 17, 18, 19, 27,  
Busca el valor: 185... Valor NO encontrado  
Busca el valor: 19... Valor encontrado  
Total nodos: 12  
Altura del árbol: 6  
C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\...
```

*Ilustración 44: Buscar en árbol binario ordenado, número de nodos y altura del árbol*

## Dibujar un árbol binario

En <http://viz-js.com/> se encuentra este servicio:

G/018.cs

```
namespace Ejemplo;

//Dibujar el árbol en http://viz-js.com/
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda;
    public Nodo Derecha;

    public Nodo(char Letra) {
        this.Letra = Letra;
        this.Izquierda = null;
        this.Derecha = null;
    }
}

class Program {

    public static void Main() {

        //Crea el árbol
        Nodo Arbol = new('P');
        Arbol.Izquierda = new('F');
        Arbol.Derecha = new('S');
        Arbol.Izquierda.Izquierda = new('B');
        Arbol.Izquierda.Derecha = new('H');
        Arbol.Izquierda.Derecha.Izquierda = new('G');
        Arbol.Derecha.Izquierda = new('R');
        Arbol.Derecha.Derecha = new('Y');
        Arbol.Derecha.Derecha.Izquierda = new('T');
        Arbol.Derecha.Derecha.Derecha = new('Z');
        Arbol.Derecha.Derecha.Izquierda.Derecha = new('W');

        //Probarlo en: http://viz-js.com
        Console.WriteLine("digraph testgraph{");
        Dibujar(Arbol);
        Console.WriteLine("}");
    }

    static void Dibujar(Nodo Arbol) {
        if (Arbol != null) {
            if (Arbol.Izquierda != null) {
                Console.Write(Arbol.Letra + "->");
            }
        }
    }
}
```

```

        Console.WriteLine(Arbol.Izquierda.Letra);
        Dibujar(Arbol.Izquierda);
    }
    if (Arbol.Derecha != null) {
        Console.Write(Arbol.Letra + "->");
        Console.WriteLine(Arbol.Derecha.Letra);
        Dibujar(Arbol.Derecha);
    }
}
}
}

```

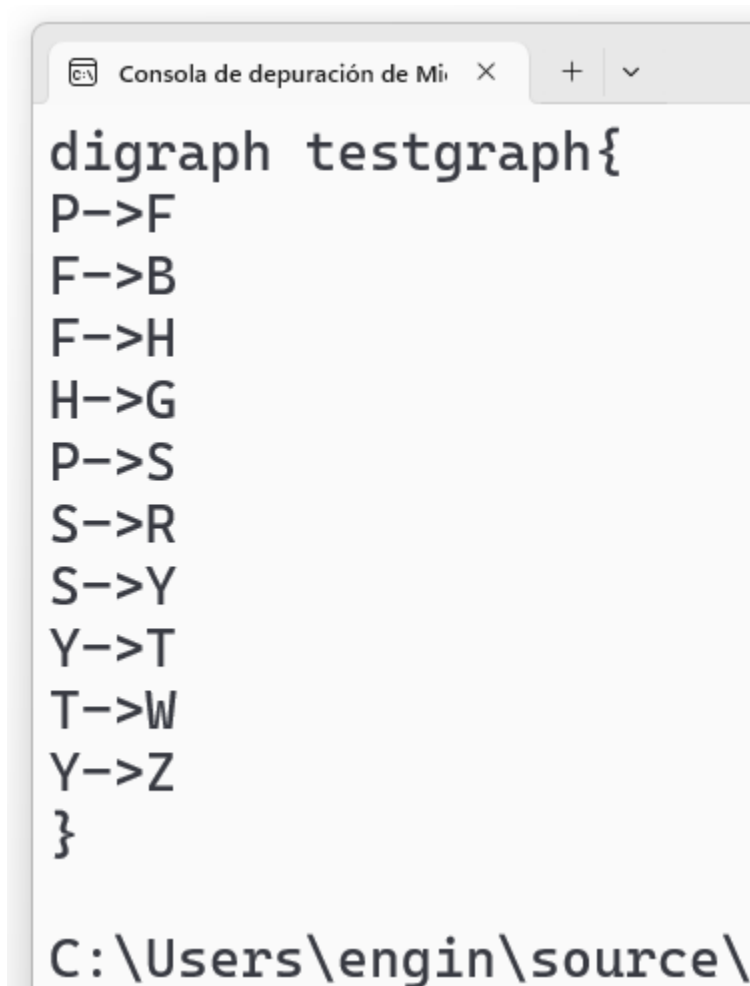


Ilustración 45: Dibujar un árbol binario

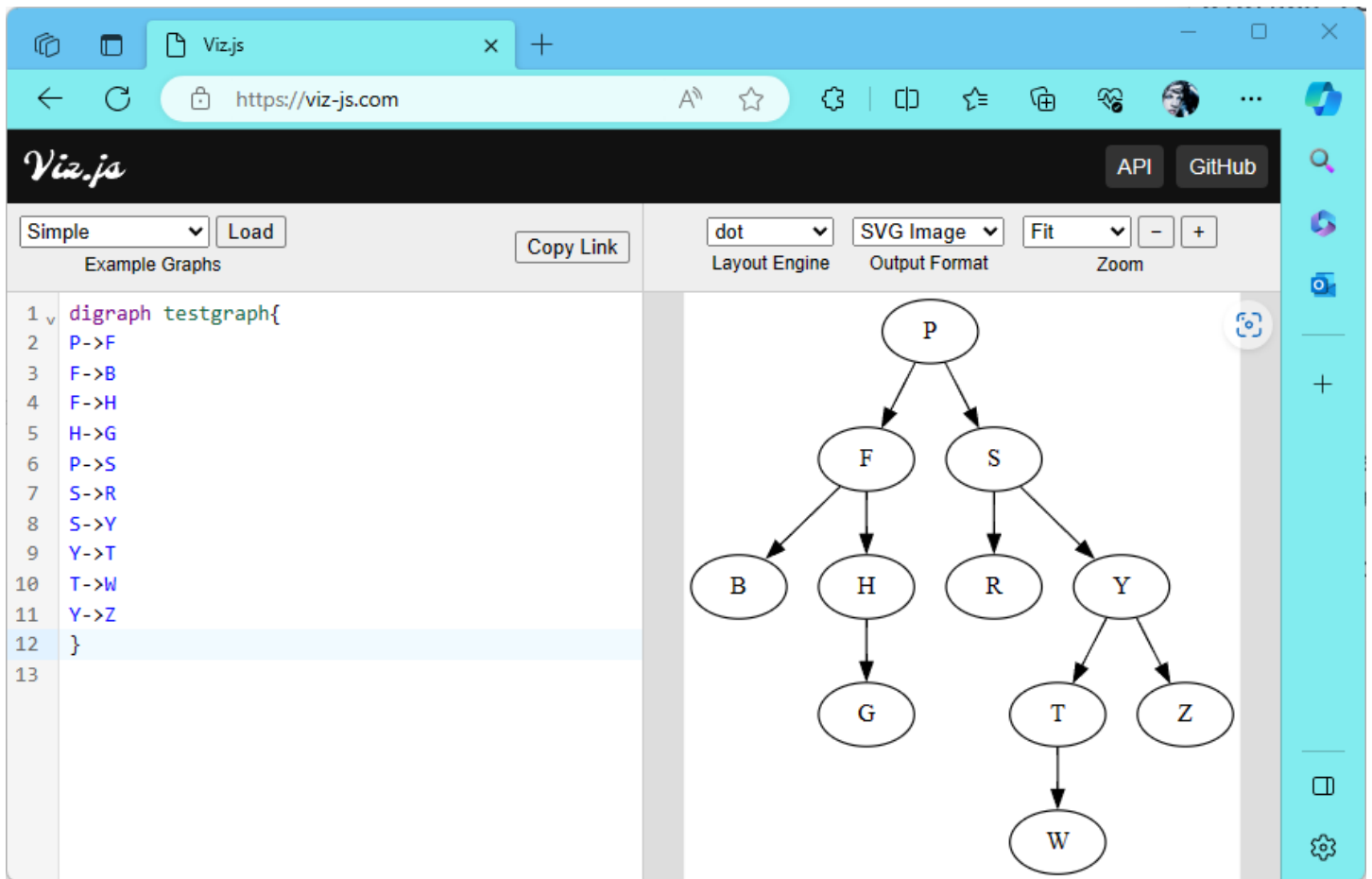


Ilustración 46: Dibujar un árbol binario



## Recorrer un árbol binario por niveles

Se hace uso de una lista para almacenar dos datos de cada nodo del árbol: el nodo y su altura. Luego se recorre varias veces esa lista, mostrando los nodos de cada determinada altura.

G/019.cs

```
namespace Ejemplo;

//Recorrido por niveles de un árbol binario
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda; //Apuntador
    public Nodo Derecha; //Apuntador

    //Constructor
    public Nodo(char Letra) {
        this.Letra = Letra;
    }
}

class NodosNivel {
    public int Altura { get; set; }
    public Nodo nodo;

    public NodosNivel(int Altura, Nodo nodo) {
        this.Altura = Altura;
        this.nodo = nodo;
    }
}

class Program {
    static void Main(string[] args) {
        List<NodosNivel> niveles = [];

        //Crea el árbol
        Nodo Arbol = new('P');
        Arbol.Izquierda = new('F');
        Arbol.Derecha = new('S');
        Arbol.Izquierda.Izquierda = new('B');
        Arbol.Izquierda.Derecha = new('H');
        Arbol.Izquierda.Derecha.Izquierda = new('G');
        Arbol.Derecha.Izquierda = new('R');
        Arbol.Derecha.Derecha = new('Y');
        Arbol.Derecha.Derecha.Izquierda = new('T');
        Arbol.Derecha.Derecha.Derecha = new('Z');
        Arbol.Derecha.Derecha.Izquierda.Derecha = new('W');
```

```

//Recorrido por niveles
Console.WriteLine("Recorrido por niveles");

//Arma la lista con la información de Nodo y Altura
ArmaLista(niveles, Arbol, 0);

//Una vez armada la lista entonces la explora
//usando como llave la altura
bool ExisteNivel;
int Altura = 0;
do {
    ExisteNivel = false;

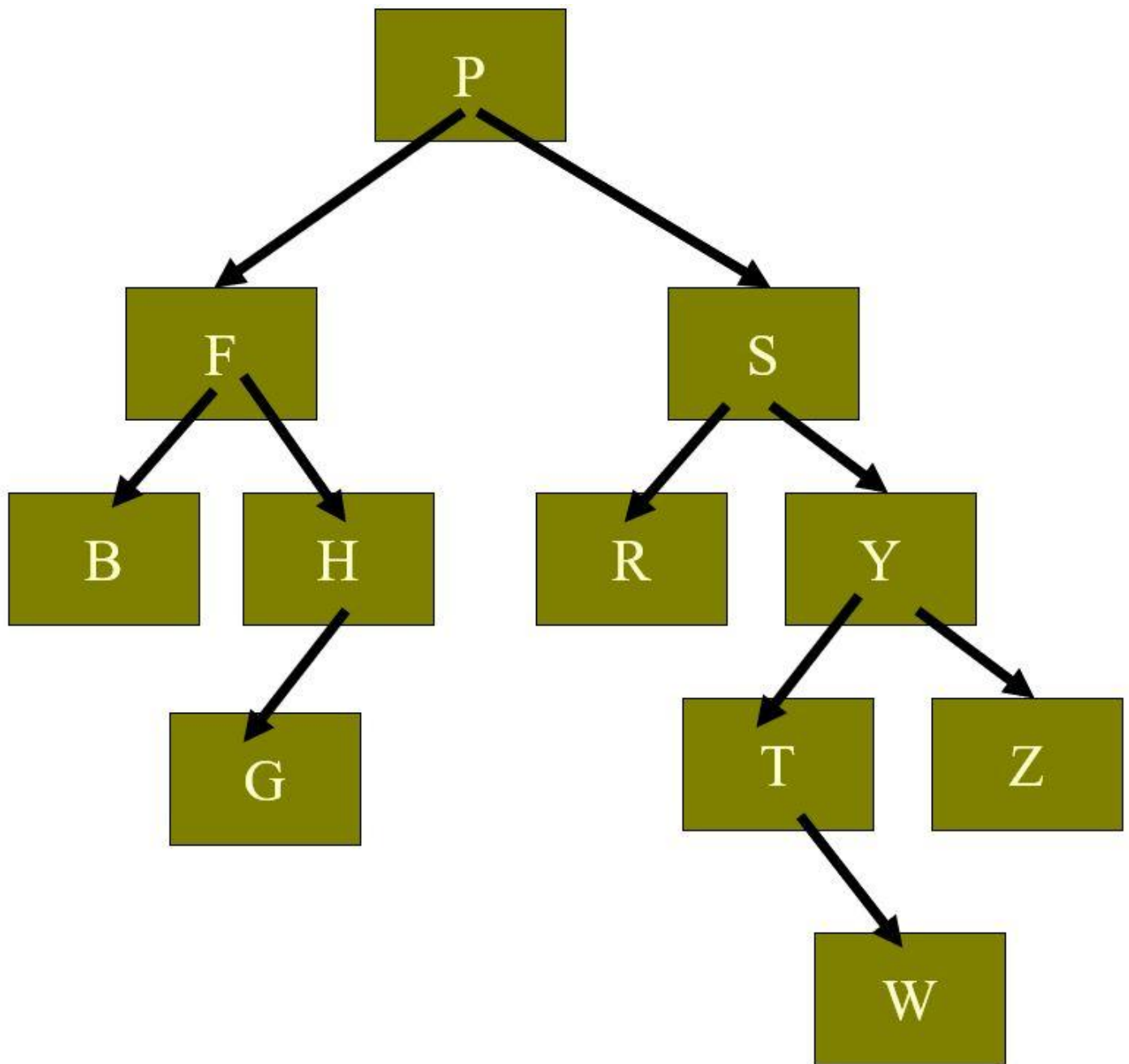
    //Muestra los nodos de esa altura en particular
    for (int cont = 0; cont < niveles.Count; cont++)
        if (niveles[cont].Altura == Altura) {
            Console.Write(niveles[cont].nodo.Letra + " -- ");
            ExisteNivel = true;
        }

    //Salta al siguiente nivel
    Console.WriteLine(" ");
    Altura++;
} while (ExisteNivel);
}

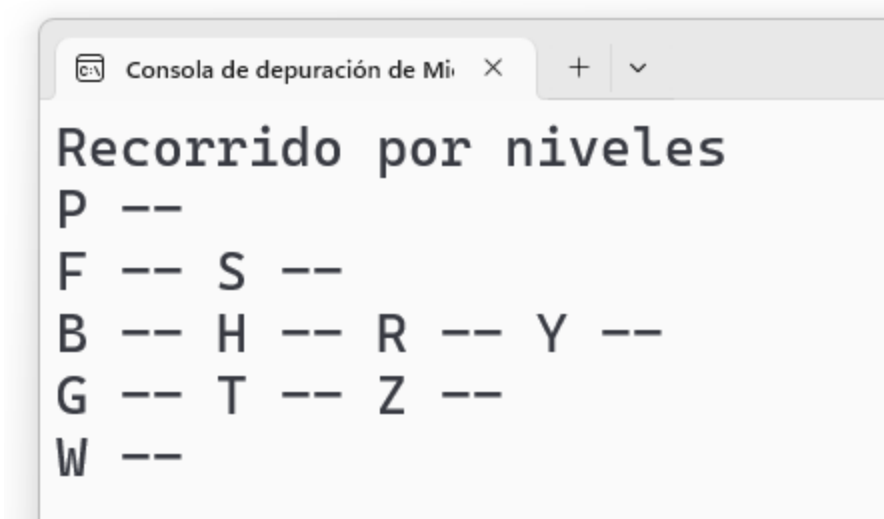
//Arma la lista con la información del nodo y su altura
public static void ArmaLista(List<NodosNivel> niveles,
    Nodo arbol, int altura) {
    niveles.Add(new NodosNivel(altura, arbol));
    if (arbol.Izquierda != null)
        ArmaLista(niveles, arbol.Izquierda, altura + 1);

    if (arbol.Derecha != null)
        ArmaLista(niveles, arbol.Derecha, altura + 1);
}
}

```



*Ilustración 47: Árbol binario de ejemplo*



```
Consola de depuración de Mi × + ▾  
Recorrido por niveles  
P --  
F -- S --  
B -- H -- R -- Y --  
G -- T -- Z --  
W --
```

*Ilustración 48: Recorrer un árbol binario por niveles*

# Árbol N-ario

Un árbol donde puede haber de 0 a N hijos por rama

G/020.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Uso de una Lista para sostener los hijos del nodo
    public List<Nodo> Hijos;

    public Nodo(string Cad, char Car, int Entero, double Num) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        Hijos = []; //Crea la lista vacía
    }

    public void AgregaHijo(Nodo hijo) {
        Hijos.Add(hijo); //Agrega hijo a la lista
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
        Console.WriteLine(" Número de hijos: " + Hijos.Count + "\r\n");
    }
}

class Program {
    static void Main() {
        //Crea la raíz del árbol N-ario
        Nodo arbolN = new("AAAA", 'a', 1, 0.1);

        //Agrega varios hijos a esa raíz
        arbolN.AgregaHijo(new("BBBB", 'b', 2, 0.2));
        arbolN.AgregaHijo(new("CCCC", 'c', 3, 0.3));
    }
}
```

```

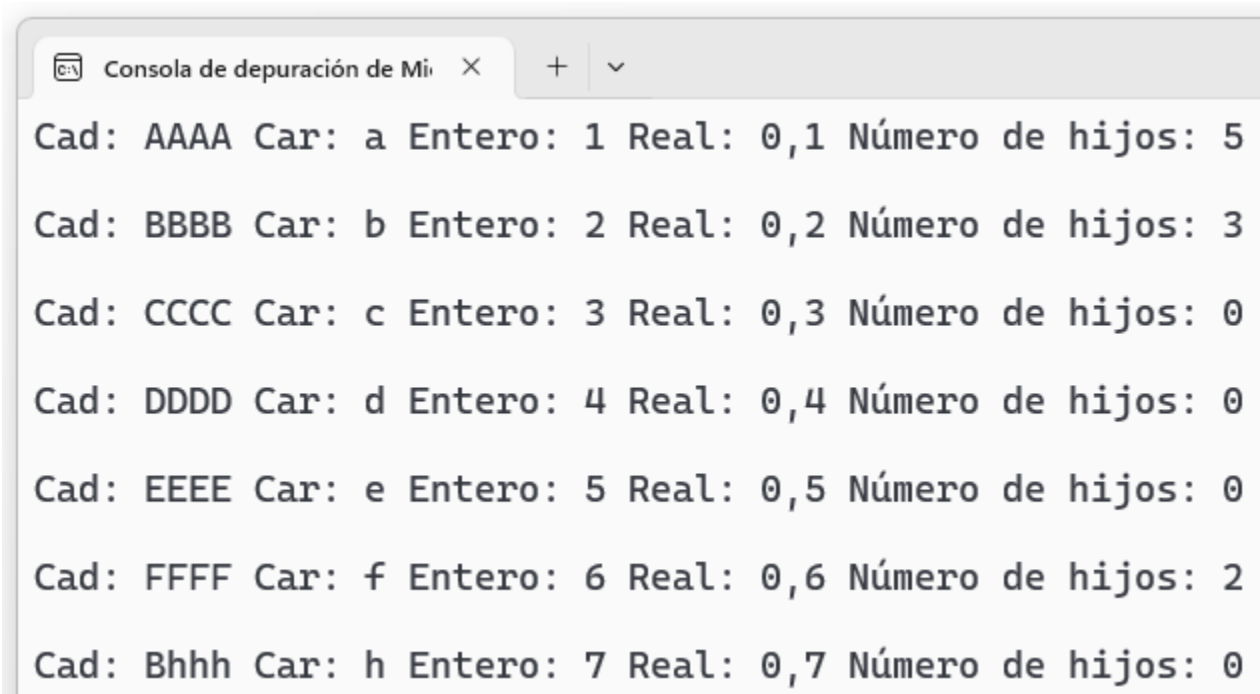
arbolN.AgregaHijo(new("DDDD", 'd', 4, 0.4));
arbolN.AgregaHijo(new("EEEE", 'e', 5, 0.5));
arbolN.AgregaHijo(new("FFFF", 'f', 6, 0.6));

//Agrega varios hijos al nodo "BBBB"
arbolN.Hijos[0].AgregaHijo(new("Bhhh", 'h', 7, 0.7));
arbolN.Hijos[0].AgregaHijo(new("Biii", 'i', 8, 0.8));
arbolN.Hijos[0].AgregaHijo(new("Bjjj", 'j', 9, 0.9));

//Agrega varios hijos al nodo "EEEE"
arbolN.Hijos[3].AgregaHijo(new("Ekkk", 'k', 10, 1.1));
arbolN.Hijos[3].AgregaHijo(new("Elll", 'l', 11, 1.2));

//Imprime el árbol
arbolN.Imprime();
arbolN.Hijos[0].Imprime();
arbolN.Hijos[1].Imprime();
arbolN.Hijos[2].Imprime();
arbolN.Hijos[3].Imprime();
arbolN.Hijos[4].Imprime();
arbolN.Hijos[0].Hijos[0].Imprime();
}
}

```



```

Cad: AAAA Car: a Entero: 1 Real: 0,1 Número de hijos: 5
Cad: BBBB Car: b Entero: 2 Real: 0,2 Número de hijos: 3
Cad: CCCC Car: c Entero: 3 Real: 0,3 Número de hijos: 0
Cad: DDDD Car: d Entero: 4 Real: 0,4 Número de hijos: 0
Cad: EEEE Car: e Entero: 5 Real: 0,5 Número de hijos: 0
Cad: FFFF Car: f Entero: 6 Real: 0,6 Número de hijos: 2
Cad: Bhhh Car: h Entero: 7 Real: 0,7 Número de hijos: 0

```

Ilustración 49: Árbol N-ario

## Recorriéndolo

Se usa un procedimiento recursivo.

G/021.cs

```
namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Uso de una Lista para sostener los hijos del nodo
    public List<Nodo> Hijos;

    public Nodo(string Cad, char Car, int Entero, double Num) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        Hijos = new List<Nodo>(); //Crea la lista vacía
    }

    public void Nuevo(Nodo hijo) {
        Hijos.Add(hijo); //Agrega hijo a la lista
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
        Console.WriteLine(" Número de hijos: " + Hijos.Count);
    }
}

class Program {
    static void Main() {
        //Crea la raíz del árbol N-ario
        Nodo arbolN = new("AAAA", 'a', 1, 0.1);

        //Agrega varios hijos a esa raíz
        arbolN.Nuevo(new("BBBB", 'b', 2, 0.2));
        arbolN.Nuevo(new("CCCC", 'c', 3, 0.3));
        arbolN.Nuevo(new("DDDD", 'd', 4, 0.4));
    }
}
```

```

arbolN.Nuevo(new("EEEE", 'e', 5, 0.5));
arbolN.Nuevo(new("FFFF", 'f', 6, 0.6));

//Agrega varios hijos al nodo "BBBB"
arbolN.Hijos[0].Nuevo(new("Bhhh", 'h', 7, 0.7));
arbolN.Hijos[0].Nuevo(new("Biii", 'i', 8, 0.8));
arbolN.Hijos[0].Nuevo(new("Bjjj", 'j', 9, 0.9));

//Agrega varios hijos al nodo "EEEE"
arbolN.Hijos[3].Nuevo(new("Ekkk", 'k', 10, 1.1));
arbolN.Hijos[3].Nuevo(new("Elll", 'l', 11, 1.2));
arbolN.Hijos[3].Nuevo(new("Emmm", 'm', 12, 1.3));

//Agrega varios hijos al nodo "Biii"
arbolN.Hijos[0].Hijos[1].Nuevo(new("Biiia", 'n', 13, 1.4));
arbolN.Hijos[0].Hijos[1].Nuevo(new("Biiib", 'o', 14, 1.5));
arbolN.Hijos[0].Hijos[1].Nuevo(new("Biiic", 'p', 15, 1.6));
arbolN.Hijos[0].Hijos[1].Nuevo(new("Biiid", 'q', 16, 1.7));
arbolN.Hijos[0].Hijos[1].Nuevo(new("Biiie", 'r', 17, 1.8));

//Imprime el árbol
RecorreArbolN(arbolN);
}

//Recorre el árbol
static void RecorreArbolN(Nodo Arbol) {
    if (Arbol != null) {
        Arbol.Imprime();
        for (int cont = 0; cont < Arbol.Hijos.Count; cont++)
            RecorreArbolN(Arbol.Hijos[cont]);
    }
}
}

```



```
Consola de depuración de Mi X + v
Cad: AAAA Car: a Entero: 1 Real: 0,1 Número de hijos: 5
Cad: BBBB Car: b Entero: 2 Real: 0,2 Número de hijos: 3
Cad: Bhhh Car: h Entero: 7 Real: 0,7 Número de hijos: 0
Cad: Biii Car: i Entero: 8 Real: 0,8 Número de hijos: 5
Cad: Biiia Car: n Entero: 13 Real: 1,4 Número de hijos: 0
Cad: Biiib Car: o Entero: 14 Real: 1,5 Número de hijos: 0
Cad: Biiic Car: p Entero: 15 Real: 1,6 Número de hijos: 0
Cad: Biiid Car: q Entero: 16 Real: 1,7 Número de hijos: 0
Cad: Biiie Car: r Entero: 17 Real: 1,8 Número de hijos: 0
Cad: Bjjj Car: j Entero: 9 Real: 0,9 Número de hijos: 0
Cad: CCCC Car: c Entero: 3 Real: 0,3 Número de hijos: 0
Cad: DDDD Car: d Entero: 4 Real: 0,4 Número de hijos: 0
Cad: EEEE Car: e Entero: 5 Real: 0,5 Número de hijos: 3
Cad: Ekkk Car: k Entero: 10 Real: 1,1 Número de hijos: 0
Cad: Elll Car: l Entero: 11 Real: 1,2 Número de hijos: 0
Cad: Emmm Car: m Entero: 12 Real: 1,3 Número de hijos: 0
Cad: FFFF Car: f Entero: 6 Real: 0,6 Número de hijos: 0
```

Ilustración 50: Recorriendo árbol N-ario

## Grafos

Para generar cualquier otra estructura de nodos interconectados, se hace uso de grafos.

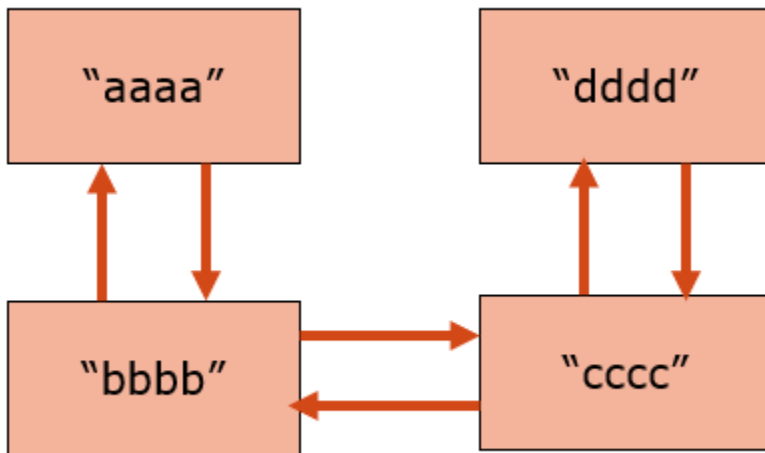


Ilustración 51: Grafo generado

G/022.cs

```
namespace Ejemplo;

//Grafo básico
//Unidad básica en el grafo cuadrado.
//Apuntará Arriba, Abajo, Derecha e Izquierda
class Nodo {
    public string Cad { get; set; }

    //Apuntadores en las 4 direcciones
    public Nodo Arriba;
    public Nodo Abajo;
    public Nodo Derecha;
    public Nodo Izquierda;

    //Constructor
    public Nodo(string Cad) {
        this.Cad = Cad;
    }
}

class Program {
    public static void Main() {
        //Genera los nodos
        Nodo nodoA = new("aaaa");
        Nodo nodoB = new("bbbb");
        Nodo nodoC = new("cccc");
        Nodo nodoD = new("dddd");
```

```

//Une los nodos para crear el grafo
nodoA.Abajo = nodoB;
nodoB.Arriba = nodoA;

nodoB.Derecha = nodoC;
nodoC.Izquierda = nodoB;

nodoC.Arriba = nodoD;
nodoD.Abajo = nodoC;

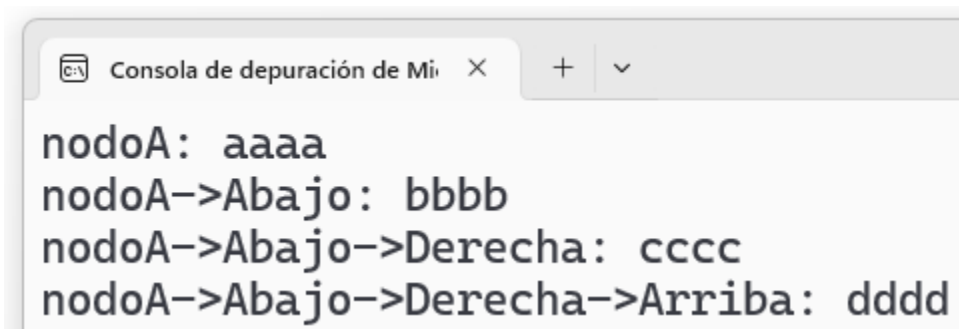
//Imprime
Console.WriteLine("nodoA: " + nodoA.Cad);

Console.WriteLine("nodoA->Abajo: " + nodoA.Abajo.Cad);

Console.Write("nodoA->Abajo->Derecha: ");
Console.WriteLine(nodoA.Abajo.Derecha.Cad);

Console.Write("nodoA->Abajo->Derecha->Arriba: ");
Console.WriteLine(nodoA.Abajo.Derecha.Arriba.Cad);
}
}

```



```

Consola de depuración de Mi
nodoA: aaaa
nodoA->Abajo: bbbb
nodoA->Abajo->Derecha: cccc
nodoA->Abajo->Derecha->Arriba: dddd

```

Ilustración 52: Grafos

## Generando un grafo aleatoriamente

Aquí se hace uso de un truco: se generan los nodos del grafo agregándolos a un List y luego se recorre nodo por nodo del List para interconectar los nodos con otros nodos.

G/023.cs

```
namespace Ejemplo;

//Generando un grafo aleatoriamente
//Unidad básica en el grafo cuadrado.
//Apuntará Arriba, Abajo, Derecha e Izquierda
class Nodo {
    public int Numero { get; set; }

    //Apuntadores en las 4 direcciones
    public Nodo Arriba;
    public Nodo Abajo;
    public Nodo Derecha;
    public Nodo Izquierda;

    //Constructor
    public Nodo(int Numero) {
        this.Numero = Numero;
    }
}

class Program {
    public static void Main() {
        Random azar = new();

        //Usa una lista para guardar los nodos
        List<Nodo> listado = [];

        //Genera los nodos dentro de un List
        int Total = azar.Next(20, 30);
        for (int cont = 1; cont <= Total; cont++) {
            listado.Add(new(cont));
        }

        //Ahora interconecta los nodos al azar
        Total = azar.Next(50, 200);
        for (int cont = 1; cont <= Total; cont++) {
            int nodoA = azar.Next(listado.Count);
            int nodoB;
            do {
                nodoB = azar.Next(listado.Count);
```

```

    } while (nodoA == nodoB);

    switch (azar.Next(4)) {
        case 0:
            listado[nodoA].Arriba = listado[nodoB];
            break;

        case 1:
            listado[nodoA].Abajo = listado[nodoB];
            break;

        case 2:
            listado[nodoA].Izquierda = listado[nodoB];
            break;

        case 3:
            listado[nodoA].Derecha = listado[nodoB];
            break;
    }
}

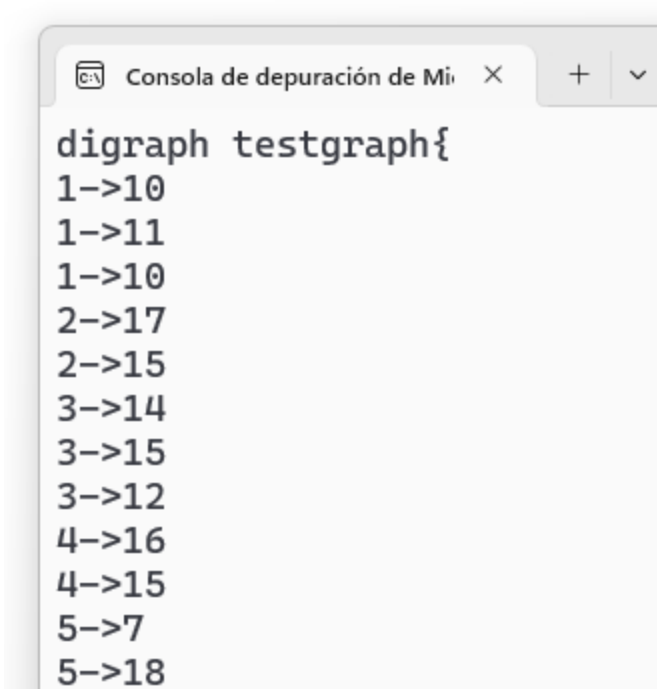
//Imprime el grafo para ser interpretado por viz.js
Console.WriteLine("digraph testgraph{");
for (int cont = 0; cont < listado.Count; cont++) {
    if (listado[cont].Arriba != null) {
        Console.Write(listado[cont].Numero + "->");
        Console.WriteLine(listado[cont].Arriba.Numero);
    }

    if (listado[cont].Abajo != null) {
        Console.Write(listado[cont].Numero + "->");
        Console.WriteLine(listado[cont].Abajo.Numero);
    }

    if (listado[cont].Izquierda != null) {
        Console.Write(listado[cont].Numero + "->");
        Console.WriteLine(listado[cont].Izquierda.Numero);
    }

    if (listado[cont].Derecha != null) {
        Console.Write(listado[cont].Numero + "->");
        Console.WriteLine(listado[cont].Derecha.Numero);
    }
}
Console.WriteLine("}");
}
}

```



The image shows a screenshot of a debug console window titled "Consola de depuración de Mi". The window contains a list of directed edges for a graph, starting with "digraph testgraph{" and followed by 14 edges. The edges are: 1->10, 1->11, 1->10, 2->17, 2->15, 3->14, 3->15, 3->12, 4->16, 4->15, 5->7, and 5->18.

```
digraph testgraph{
1->10
1->11
1->10
2->17
2->15
3->14
3->15
3->12
4->16
4->15
5->7
5->18
}
```

*Ilustración 53: Gafo generado*

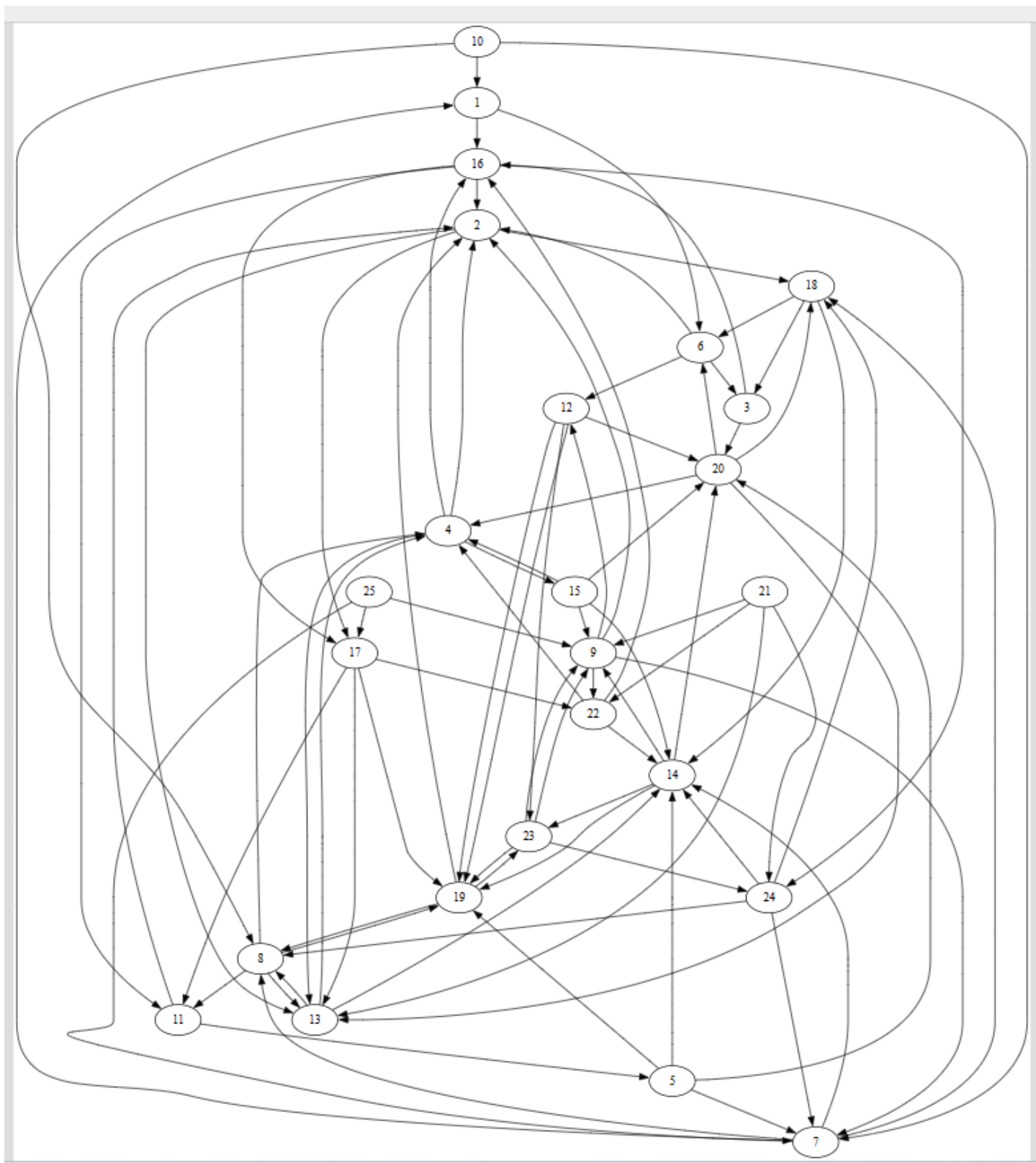


Ilustración 54: Grafo dibujado con <https://viz-js.com/>

## Guardando un árbol binario en medio persistente

Se hace uso de JSON.

¡OJO! Los apuntadores deben ser propiedades (no sólo atributos)

G/024.cs

```
using System.Text.Json;

namespace Ejemplo;

//Recorrido de un árbol binario
class Nodo {
    public char Letra { get; set; }
    public Nodo Izquierda { get; set; } //Apuntador
    public Nodo Derecha { get; set; } //Apuntador

    //Constructor
    public Nodo(char Letra) {
        this.Letra = Letra;
    }
}

class Program {
    public static void Main() {
        //Crea el árbol
        Nodo Arbol = new('M');
        Arbol.Izquierda = new('E');
        Arbol.Derecha = new('P');
        Arbol.Izquierda.Izquierda = new('B');
        Arbol.Izquierda.Derecha = new('L');
        Arbol.Izquierda.Izquierda.Izquierda = new('A');
        Arbol.Izquierda.Izquierda.Derecha = new('D');
        Arbol.Derecha.Izquierda = new('N');
        Arbol.Derecha.Derecha = new('V');
        Arbol.Derecha.Derecha.Izquierda = new('T');
        Arbol.Derecha.Derecha.Derecha = new('Z');

        //Imprime árbol
        Console.WriteLine("ARBOL ORIGINAL");

        //Recorridos
        Console.WriteLine("PreOrden (raiz, izquierdo, derecho)");
        PreOrden(Arbol);

        Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
        InOrden(Arbol);
    }
}
```



```

Console.WriteLine("\n\nPostOrden (izquierdo, derecho, raiz)");
PostOrden(Arbol);

// Guardar como JSON
GuardarComoJSON(Arbol, "arbol.json");
Console.WriteLine("\r\n\r\nÁrbol binario guardado\r\n");

//Recupera el árbol
Nodo Recupera = CargarDesdeJSON("arbol.json");
Console.WriteLine("Árbol binario recuperado de archivo JSON");

Console.WriteLine("PreOrden (raiz, izquierdo, derecho)");
PreOrden(Recupera);

Console.WriteLine("\n\nInOrden (izquierdo, raiz, derecho)");
InOrden(Recupera);

Console.WriteLine("\n\nPostOrden (izquierdo, derecho, raiz)");
PostOrden(Recupera);

}

static void GuardarComoJSON(Nodo raiz, string rutaArchivo) {
    var opciones = new JsonSerializerOptions { WriteIndented = true };
    string json = JsonSerializer.Serialize(raiz, opciones);
    File.WriteAllText(rutaArchivo, json);
}

static Nodo CargarDesdeJSON(string rutaArchivo) {
    string json = File.ReadAllText(rutaArchivo);
    Nodo raiz = JsonSerializer.Deserialize<Nodo>(json);
    return raiz;
}

static void PreOrden(Nodo Arbol) {
    if (Arbol != null) {
        Console.Write(Arbol.Letra + ", ");
        PreOrden(Arbol.Izquierda);
        PreOrden(Arbol.Derecha);
    }
}

static void InOrden(Nodo Arbol) {
    if (Arbol != null) {
        InOrden(Arbol.Izquierda);
        Console.Write(Arbol.Letra + ", ");
        InOrden(Arbol.Derecha);
    }
}

```

```

    }

    static void PostOrden(Nodo Arbol) {
        if (Arbol != null) {
            PostOrden(Arbol.Izquierda);
            PostOrden(Arbol.Derecha);
            Console.Write(Arbol.Letra + ", ");
        }
    }
}

```

```

ARBOL ORIGINAL
PreOrden (raiz, izquierdo, derecho)
M, E, B, A, D, L, P, N, V, T, Z,

InOrden (izquierdo, raiz, derecho)
A, B, D, E, L, M, N, P, T, V, Z,

PostOrden (izquierdo, derecho, raiz)
A, D, B, L, E, N, T, Z, V, P, M,

Árbol binario guardado

Árbol binario recuperado de archivo JSON
PreOrden (raiz, izquierdo, derecho)
M, E, B, A, D, L, P, N, V, T, Z,

InOrden (izquierdo, raiz, derecho)
A, B, D, E, L, M, N, P, T, V, Z,

PostOrden (izquierdo, derecho, raiz)
A, D, B, L, E, N, T, Z, V, P, M,
C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Debug\net9.0\Eje
mplo.exe (proceso 22620) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depu
ración, habilite Herramientas ->Opciones ->Depuración ->Cerrar l
a consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .

```

Ilustración 55: Árbol binario guardado usando JSON

```
1 {
2   "Letra": "M",
3   "Izquierda": {
4     "Letra": "E",
5     "Izquierda": {
6       "Letra": "B",
7       "Izquierda": {
8         "Letra": "A",
9         "Izquierda": null,
10        "Derecha": null
11      },
12      "Derecha": {
13        "Letra": "D",
14        "Izquierda": null,
15        "Derecha": null
16      }
17    },
18    "Derecha": {
19      "Letra": "L",
20      "Izquierda": null,
```

length: 818 lines: 45    Ln: 1 Col: 1 Pos: 1    Windows (CR LF)    UTF-8    INS

Ilustración 56: Archivo JSON

## Guardando un árbol N-ario en un medio persistente

G/025.cs

```
using System.Text.Json;

namespace Ejemplo;

class Nodo {
    //Atributos propios
    public string Cad { get; set; }
    public char Car { get; set; }
    public int Entero { get; set; }
    public double Num { get; set; }

    //Uso de una Lista para sostener los hijos del nodo
    public List<Nodo> Hijos { get; set; } = new List<Nodo>();

    public Nodo(string Cad, char Car, int Entero, double Num) {
        this.Cad = Cad;
        this.Car = Car;
        this.Entero = Entero;
        this.Num = Num;
        Hijos = []; //Crea la lista vacía
    }

    public void AgregaHijo(Nodo hijo) {
        Hijos.Add(hijo); //Agrega hijo a la lista
    }

    //Imprime Contenido
    public void Imprime() {
        Console.WriteLine("Cad: " + Cad + " Car: " + Car);
        Console.WriteLine(" Entero: " + Entero + " Real: " + Num);
        Console.WriteLine(" Número de hijos: " + Hijos.Count + "\r\n");
    }
}

class Program {
    static void Main() {
        //Crea la raíz del árbol N-ario
        Nodo arbolN = new("AAAA", 'a', 1, 0.1);

        //Agrega varios hijos a esa raíz
        arbolN.AgregaHijo(new("BBBB", 'b', 2, 0.2));
        arbolN.AgregaHijo(new("CCCC", 'c', 3, 0.3));
    }
}
```

```

arbolN.AgregaHijo(new("DDDD", 'd', 4, 0.4));
arbolN.AgregaHijo(new("EEEE", 'e', 5, 0.5));
arbolN.AgregaHijo(new("FFFF", 'f', 6, 0.6));

//Agrega varios hijos al nodo "BBBB"
arbolN.Hijos[0].AgregaHijo(new("Bhhh", 'h', 7, 0.7));
arbolN.Hijos[0].AgregaHijo(new("Biii", 'i', 8, 0.8));
arbolN.Hijos[0].AgregaHijo(new("Bjjj", 'j', 9, 0.9));

//Agrega varios hijos al nodo "EEEE"
arbolN.Hijos[3].AgregaHijo(new("Ekkk", 'k', 10, 1.1));
arbolN.Hijos[3].AgregaHijo(new("Elll", 'l', 11, 1.2));

//Imprime el árbol
RecorrerPreorden(arbolN);

//Guarda el árbol N-ario
Console.WriteLine("Guarda árbol N-ario");
GuardarArbolNarioComoJSON(arbolN, "arbol.json");

//Restaura el árbol N-ario
Console.WriteLine("Restaura árbol N-ario");
Nodo Restaura = CargarArbolNarioDesdeJSON("arbol.json");

//Imprime el árbol N-ario restaurado
RecorrerPreorden(Restaura);
}

static void RecorrerPreorden(Nodo nodo) {
    if (nodo == null) return;
    nodo.Imprime();
    foreach (var hijo in nodo.Hijos) {
        RecorrerPreorden(hijo);
    }
}

static void GuardarArbolNarioComoJSON(Nodo raiz, string rutaArchivo) {
    var opciones = new JsonSerializerOptions { WriteIndented = true };
    string json = JsonSerializer.Serialize(raiz, opciones);
    File.WriteAllText(rutaArchivo, json);
}

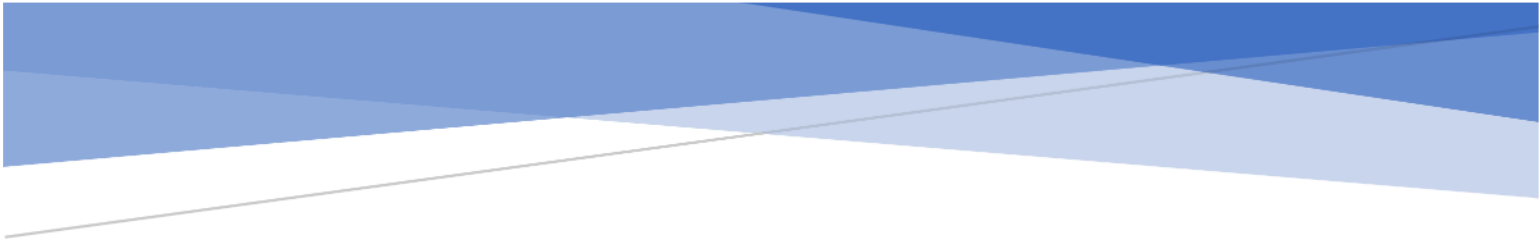
static Nodo CargarArbolNarioDesdeJSON(string rutaArchivo) {
    string json = File.ReadAllText(rutaArchivo);
    return JsonSerializer.Deserialize<Nodo>(json);
}
}

```

```
1 {
2   .. "Cad": .. "AAAA",
3   .. "Car": .. "a",
4   .. "Entero": .. 1,
5   .. "Num": .. 0.1,
6   .. "Hijos": .. [
7     .. {
8       .. "Cad": .. "BBBB",
9       .. "Car": .. "b",
10      .. "Entero": .. 2,
11      .. "Num": .. 0.2,
12      .. "Hijos": .. [
13        .. {
14          .. "Cad": .. "Bhhh",
15          .. "Car": .. "h",
16          .. "Entero": .. 7,
17          .. "Num": .. 0.7,
18          .. "Hijos": .. []
19        .. },
20        .. {
```

length: 1.382 lines: 80 Ln: 1 Col: 1 Pos: 1 Windows (CR LF) UTF-8 INS

Ilustración 57: JSON de un árbol N-ario



# C# Y .NET 10

## Parte 8. Evaluador de expresiones algebraicas

2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| Tabla de ilustraciones.....                                                    | 3   |
| Acerca del autor.....                                                          | 4   |
| Licencia de este libro .....                                                   | 4   |
| Licencia del software .....                                                    | 4   |
| Marcas registradas .....                                                       | 5   |
| Dedicatoria .....                                                              | 6   |
| El problema.....                                                               | 7   |
| Evaluador de expresiones iterativo. Versión 4.1.....                           | 8   |
| El algoritmo.....                                                              | 9   |
| Paso 1: Retirar caracteres que no sean de una expresión algebraica .....       | 9   |
| Paso 2: Convertir la expresión .....                                           | 11  |
| Paso 3: Verificando la sintaxis de la expresión algebraica .....               | 12  |
| Paso 4: Dividiendo la cadena en partes .....                                   | 19  |
| Paso 5: Generando las Piezas desde las Partes .....                            | 22  |
| Paso 6: Evaluando a partir de las Piezas.....                                  | 23  |
| El código completo .....                                                       | 25  |
| Como usar el evaluador de expresiones.....                                     | 41  |
| Chequear: Desempeño con el evaluador interno de C# .....                       | 46  |
| Evaluador de expresiones usando un árbol binario. Programación recursiva ..... | 51  |
| Fase 1. Sumas y restas .....                                                   | 52  |
| Fase 2. Multiplicación y división .....                                        | 62  |
| Fase 3. Potencia .....                                                         | 67  |
| Fase 4. Paréntesis .....                                                       | 72  |
| Fase 5. Variables .....                                                        | 78  |
| Fase 6. Funciones matemáticas.....                                             | 85  |
| Fase 7. Orientado a objetos.....                                               | 95  |
| Fase 8. Evaluación de sintaxis y optimización .....                            | 104 |
| Comparativa de desempeño entre evaluadores .....                               | 116 |



## Tabla de ilustraciones

|                                                          |     |
|----------------------------------------------------------|-----|
| Ilustración 1: Uso del evaluador de expresiones .....    | 41  |
| Ilustración 2: Resultado obtenido con el evaluador ..... | 42  |
| Ilustración 3: Métrica compara ambos evaluadores .....   | 50  |
| Ilustración 6: Árbol binario generado .....              | 53  |
| Ilustración 7: Árbol binario generado .....              | 54  |
| Ilustración 8: Ejemplo de ejecución del programa .....   | 57  |
| Ilustración 9: Interpretación en viz-js.com .....        | 57  |
| Ilustración 10: Resultado al evaluar .....               | 60  |
| Ilustración 11: Validado con la calculadora .....        | 61  |
| Ilustración 12: Árbol binario generado .....             | 61  |
| Ilustración 13: Árbol binario generado .....             | 62  |
| Ilustración 14: Ejecución del evaluador .....            | 66  |
| Ilustración 15: Prueba con calculadora .....             | 66  |
| Ilustración 16: Árbol binario generado .....             | 67  |
| Ilustración 17: Ejemplo de ejecución .....               | 71  |
| Ilustración 18: Prueba con la calculadora .....          | 71  |
| Ilustración 19: Árbol binario generado .....             | 72  |
| Ilustración 20: Árbol binario generado .....             | 83  |
| Ilustración 21: Ejemplo de ejecución .....               | 84  |
| Ilustración 22: Ejemplo de ejecución .....               | 93  |
| Ilustración 23: Árbol binario generado .....             | 94  |
| Ilustración 24: Ejemplo de ejecución .....               | 103 |
| Ilustración 25: Ejemplo de ejecución .....               | 115 |
| Ilustración 26: Comparativa de desempeño .....           | 120 |
| Ilustración 27: Comparativa de desempeño .....           | 120 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

En memoria de Tammy



## El problema

Dada una expresión algebraica almacenada en una cadena "string", esta debe ser interpretada para devolver un valor real. Ejemplo:

Cadena: "3\*4+1" → Resultado: 13

Hay que considerar que las expresiones algebraicas pueden tener:

1. Números reales
2. Variables (de la a .. z)
3. Operadores (suma, resta, multiplicación, división, potencia)
4. Uso de paréntesis
5. Uso de funciones (seno, coseno, tangente, valor absoluto, arcoseno, arcocoseno, arcotangente, logaritmo natural, valor techo, exponencial, raíz cuadrada).

## Evaluador de expresiones iterativo. Versión 4.1

## El algoritmo

Esta es la versión 4.1 del evaluador de expresiones.

La expresión algebraica está almacenada en una variable de tipo cadena (string). Por ejemplo:

```
string Cadena = "0.004 - (1.78 / 3.45 + h) * sen(k ^ x)";
```

La expresión algebraica puede tener números reales, operadores, paréntesis, variables y funciones. Luego hay que interpretarla y evaluarla siguiendo las estrictas reglas matemáticas.

### Paso 1: Retirar caracteres que no sean de una expresión algebraica

Como la expresión algebraica ha sido digitada por un usuario final entonces se quita cualquier caracter(char) que no sea parte de una expresión algebraica. Los únicos caracteres permitidos son: "abcdefghijklmnopqrstuvwxyz0123456789.+\*/^()". Esa parte la hace este código:

```
public class Sintaxis4_1 {  
    /* HashSet estático para caracteres válidos (se crea una sola vez) */  
    private static readonly HashSet<char> CaracteresValidos =  
        [... "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()"];
```

```
    /* Paso 1: Convertir la expresión para poder analizarla */  
    private void Convierte(string ExpresionOriginal) {  
        /* Primero a minúsculas */  
        string Minusculas = ExpresionOriginal.ToLower();  
  
        /* Sólo los caracteres válidos (optimizado con HashSet estático) */  
        StringBuilder ConLetrasValidas = new("(");  
        foreach (char c in Minusculas) {  
            if (CaracteresValidos.Contains(c))  
                ConLetrasValidas.Append(c);  
        }  
        ConLetrasValidas.Append(')');
```

Se agrega un paréntesis que abre al inicio y luego un paréntesis al final.

De ser:  $0.004 - (1.78/3.45 + h) * \sin(k^x)$

Pasa a:  $(0.004 - (1.78/3.45 + h) * \sin(k^x))$

Es necesario ese paso porque el evaluador busca los paréntesis para extraer la expresión interna. Se hace uso de `StringBuilder` para hacerlo más rápido. El uso de la estructura `HashSet` es como filtro para que sólo pasen los caracteres permitidos. Al final la variable `ConLetrasValidas` es la que tiene la expresión limpia.



## Paso 2: Convertir la expresión

Esta es la novedad de la versión 4.1 (la anterior fue la 4.0). Esto permite expresiones así:

1. Convierte de (- a (0- para dar apoyo al menos unario. Por ejemplo:  $4+(-3*2)$  se convierte a  $4+(0-3*2)$
2. Convierte de )( a )\*(. Por ejemplo:  $(7+g)(h/u)$  se convierte a  $(7+g)*(h/u)$
3. Convierte de punto operador a punto cero operador. Ejemplo:  $3.+1$  se convierte a  $3.0+1$
4. Convierte de operador punto a operador cero punto. Ejemplo:  $7-.9$  se convierte a  $7-0.9$
5. Convierte de .( a .0\*(. Ejemplo:  $2.(8+1)$  se convierte a  $2.0*(8+1)$
6. Convierte de ). a )\*0. Ejemplo:  $(9 / 2) .5$  se convierte a  $(9/2)*0.5$
7. Convierte de .) a .0)
8. Convierte de (. a (0.
9. Convierte número letra en número \* letra
10. Convierte número( en número\*(
11. Convierte )numero en )\*numero
12. Convierte de )letra en )\*letra
13. Convierte de letra minúscula( a letra minúscula\*(

Se reemplazan las funciones (que son de tres letras) por una letra mayúscula. Esta es la tabla de conversión:

| <b>Función</b> | <b>Descripción</b> | <b>Letra con que se reemplaza</b> |
|----------------|--------------------|-----------------------------------|
| Sen            | Seno               | A                                 |
| Cos            | Coseno             | B                                 |
| Tan            | Tangente           | C                                 |
| Abs            | Valor absoluto     | D                                 |
| Asn            | Arcoseno           | E                                 |
| Acs            | Arcocoseno         | F                                 |
| Atn            | Arcotangente       | G                                 |
| Log            | Logaritmo Natural  | H                                 |
| Exp            | Exponencial        | I                                 |
| Sqr            | Raíz cuadrada      | J                                 |

### Paso 3: Verificando la sintaxis de la expresión algebraica

Luego se verifica si la expresión cumple con las estrictas reglas sintácticas del álgebra. Se hacen 14 validaciones que son:

1. Doble punto seguido. Ejemplo: 3..1
2. Un punto y sigue una letra. Ejemplo: 3+5.w-8
3. Dos operadores estén seguidos. Ejemplo: 2++4, 5-\*3
4. Un operador seguido de un paréntesis que cierra. Ejemplo: 2-(4+)-7
5. Una letra seguida de número. Ejemplo: 7-a8-6
6. Una letra seguida de punto. Ejemplo: 7-a.-6
7. Una letra seguida de otra letra. Ejemplo: 5+zx\*3
8. Un paréntesis que abre seguido de un operador. Ejemplo: 2-( \*3)
9. Un paréntesis que abre seguido de un paréntesis que cierra. Ejemplo: 7-()-6
10. Inicia con operador. Ejemplo: +3\*5
11. Finaliza con operador. Ejemplo: 3\*5\*
12. No hay correspondencia entre paréntesis. Ejemplo: 3+5)-(7\*2
13. Los paréntesis están desbalanceados. Ejemplo: 3-(2\*4))
14. Doble punto en un número de tipo real. Ejemplo: 7-6.46.1+2

Para hacer esto, se hace uso de una clase dedicada a la conversión y a la revisión de sintaxis.

H/Eval4\_1/Constantes4\_1.cs

```
public static class Constantes4_1 {
    /* Constantes de los diferentes tipos
     * de datos que tendrán las partes y piezas */
    public const int ESFUNCION = 1;
    public const int ESPARABRE = 2;
    public const int ESPARCIERRA = 3;
    public const int ESOPERADOR = 4;
    public const int ESNUMERO = 5;
    public const int ESVARIABLE = 6;
    public const int ESACUMULA = 7;

    public const int POTENCIA = 4;
    public const int DIVIDE = 3;
    public const int MULTIPLICA = 2;
    public const int RESTA = 1;
    public const int SUMA = 0;

    public const string SENO = "A";
    public const string COSENO = "B";
    public const string TANGENTE = "C";
    public const string VALORABSOLUTO = "D";
    public const string ARCOSENO = "E";
    public const string ARCOSENO = "F";
    public const string ARCOTANGENTE = "G";
    public const string LOGARITMONATURAL = "H";
    public const string EXPONENCIAL = "I";
    public const string RAIZCUADRADA = "J";

    public const int NO_FUNCION_COD = -1;
    public const int SENO_COD = 0;
    public const int COSENO_COD = 1;
    public const int TANGENTE_COD = 2;
    public const int VALORABSOLUTO_COD = 3;
    public const int ARCOSENO_COD = 4;
    public const int ARCOSENO_COD = 5;
    public const int ARCOTANGENTE_COD = 6;
    public const int LOGARITMONATURAL_COD = 7;
    public const int EXPONENCIAL_COD = 8;
    public const int RAIZCUADRADA_COD = 9;
}
```

```

public class Sintaxis4_1 {
    /* HashSet estático para caracteres válidos (se crea una sola vez) */
    private static readonly HashSet<char> CaracteresValidos =
        [.. "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()"];

    /* Diccionario para reemplazos de funciones en expresión analizada */
    private static readonly Dictionary<string, string> ReemplazosFunciones
= new()
    {
        {"sen", Constantes4_1.SENO}, {"cos", Constantes4_1.COSENO}, {"tan",
Constantes4_1.TANGENTE},
        {"abs", Constantes4_1.VALORABSOLUTO}, {"asn", Constantes4_1.ARCOSENO},
{"acs", Constantes4_1.ARCOCOSENO},
        {"atn", Constantes4_1.ARCOTANGENTE}, {"log",
Constantes4_1.LOGARITMONATURAL}, {"exp", Constantes4_1.EXPONENCIAL},
        {"sqr", Constantes4_1.RAIZCUADRADA}
    };

    /* Diccionario estático para mensajes de error */
    private static readonly Dictionary<int, string> MensajesError = new()
    {
        {0, "0. Es correcta"},
        {1, "1. Doble punto seguido"},
        {2, "2. Punto y sigue una letra"},
        {3, "3. Dos operadores estén seguidos"},
        {4, "4. Operador seguido de paréntesis que cierra"},
        {5, "5. Letra seguida de número"},
        {6, "6. Letra seguida de punto"},
        {7, "7. Letra seguida de otra letra"},
        {8, "8. Paréntesis que abre y sigue operador"},
        {9, "9. Paréntesis que abre y luego cierra"},
        {10, "10. Inicia con operador"},
        {11, "11. Finaliza con operador"},
        {12, "12. No hay correspondencia entre paréntesis"},
        {13, "13. Paréntesis desbalanceados"},
        {14, "14. Dos o más puntos en número real"}
    };

    /* HashSet estático para operadores (más rápido que switch) */
    private static readonly HashSet<char> Operadores = new() { '+', '-',
'*', '/', '^' };

    /* Arreglo de mensajes de error */
    private int[] TipoError = new int[15];

```

```

/* Expresión convertida y validada */
public string ExpresionConvertida { get; private set; } = string.Empty;

/* Convierte y chequea la expresión */
public bool ChequeaSintaxis(string ExpresionOriginal) {
    /* Paso 1: Convierte la expresión */
    Convierte(ExpresionOriginal);

    /* Inicializa el arreglo de errores */
    Array.Fill(TipoError, 0);

    /* Paso 2: Chequea sintaxis */
    ValidaCaracteresSeguidos(ExpresionConvertida);
    ValidaInicioYFinDeLaExpresion(ExpresionConvertida);
    ValidaParentesisDesbalanceados(ExpresionConvertida);
    ValidaPuntosEnNumeroReal(ExpresionConvertida);

    /* Si una sola celda es 1, hay error */
    for (int Cont = 0; Cont < TipoError.Length; Cont++) {
        if (TipoError[Cont] == 1) {
            return false;
        }
    }
    return true;
}

public List<string> ObtenerMensajesErrores() {
    List<string> mensajes = new();
    for (int i = 1; i < TipoError.Length; i++) {
        if (TipoError[i] == 1) {
            mensajes.Add(MensajeError(i));
        }
    }
    return mensajes;
}

/* Paso 1: Convertir la expresión para poder analizarla */
private void Convierte(string ExpresionOriginal) {
    /* Primero a minúsculas */
    string Minusculas = ExpresionOriginal.ToLower();

    /* Sólo los caracteres válidos (optimizado con HashSet estático) */
    StringBuilder ConLetrasValidas = new("(");
    foreach (char c in Minusculas) {
        if (CaracteresValidos.Contains(c))
            ConLetrasValidas.Append(c);
    }
    ConLetrasValidas.Append(')');
}

```

```

/* Reemplazos de funciones usando diccionario */
foreach (var reemplazo in ReemplazosFunciones) {
    ConLetrasValidas.Replace(reemplazo.Key, reemplazo.Value);
}

/* Agrega +0) donde exista )) porque es
 * necesario para crear las piezas */
ConLetrasValidas.Replace("))", ")+0)");

/* Convierte de (- a (0- para dar apoyo al menos unario */
ConLetrasValidas.Replace("(-", "(0-");

/* Convierte de )( a )( */
ConLetrasValidas.Replace(")", ")*(");

/* Convierte de punto operador a punto cero operador */
ConLetrasValidas.Replace("."+", ".0+");
ConLetrasValidas.Replace(".-", ".0-");
ConLetrasValidas.Replace(".*", ".0*");
ConLetrasValidas.Replace("./", ".0/");
ConLetrasValidas.Replace(".^", ".0^");

/* Convierte de operador punto a operador cero punto */
ConLetrasValidas.Replace("+.", "+0.");
ConLetrasValidas.Replace("-.", "-0.");
ConLetrasValidas.Replace("*. ", "*0.");
ConLetrasValidas.Replace("/.", "/0.");
ConLetrasValidas.Replace("^.", "^0.");

/* Convierte de .( a .0*( */
ConLetrasValidas.Replace(".(", ".0*(");

/* Convierte de ). a )*0. */
ConLetrasValidas.Replace(").", ")*0.");

/* Convierte de .) a .0) */
ConLetrasValidas.Replace(".)", ".0)");

/* Convierte de (. a (0. */
ConLetrasValidas.Replace("(.", "(0.");

/* Convierte a cadena para hacer uso del regex */
ExpresionConvertida = ConLetrasValidas.ToString();

/* Convierte número letra en número * letra */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"(\d)([a-z])", "$1*$2");

```

```

    /* Convierte número( en número*( */
    ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"(\d)\(",
"$1*(");

    /* Convierte )numero en )*numero */
    ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"\) (\d+)",
@"")*$1");

    /* Convierte de )letra en )*letra */
    ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"\) ([a-zA-
Z])", @"")*$1");

    /* Convierte de letra minúscula( a letra minúscula*( */
    ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"([a-z])\(",
@"$1*(");
}

/* Retorna mensaje de error sintáctico */
private string MensajeError(int CodigoError) =>
    MensajesError.TryGetValue(CodigoError, out string? mensaje)
        ? mensaje
        : "Error desconocido";

/* Retorna si el Caracter es un operador matemático */
private static bool EsUnOperador(char Caracter) =>
Operadores.Contains(Caracter);

/* Valida casos de dos caracteres seguidos */
private void ValidaCaracteresSeguidos(string Expresion) {
    for (int Cont = 0; Cont < Expresion.Length - 1; Cont++) {
        char carA = Expresion[Cont];
        char carB = Expresion[Cont + 1];

        if (carA == '.' && carB == '.') TipoError[1] = 1;
        if (carA == '.' && char.IsLower(carB)) TipoError[2] = 1;
        if (EsUnOperador(carA) && EsUnOperador(carB)) TipoError[3] = 1;
        if (EsUnOperador(carA) && carB == ')') TipoError[4] = 1;
        if (char.IsLower(carA) && char.IsDigit(carB)) TipoError[5] = 1;
        if (char.IsLower(carA) && carB == '.') TipoError[6] = 1;
        if (char.IsLower(carA) && char.IsLower(carB)) TipoError[7] = 1;
        if (carA == '(' && EsUnOperador(carB)) TipoError[8] = 1;
        if (carA == '(' && carB == ')') TipoError[9] = 1;
    }
}

/* Valida el inicio y fin de la expresión */

```

```

private void ValidaInicioYFinDeLaExpresion(string Expresion) {
    if (EsUnOperador(Expresion[1])) {
        TipoError[10] = 1;
    }
    if (EsUnOperador(Expresion[Expresion.Length - 2])) {
        TipoError[11] = 1;
    }
}

/* Valida si hay paréntesis desbalanceados */
private void ValidaParentesisDesbalanceados(string Expresion) {
    int Balance = 0;
    for (int Cont = 0; Cont < Expresion.Length; Cont++) {
        if (Expresion[Cont] == '(') Balance++;
        if (Expresion[Cont] == ')') Balance--;
        if (Balance < 0) {
            TipoError[12] = 1;
            return;
        }
    }
    if (Balance != 0) {
        TipoError[13] = 1;
    }
}

/* Validar los puntos decimales de un número real */
private void ValidaPuntosEnNumeroReal(string Expresion) {
    int TotalPuntos = 0;
    for (int Cont = 0; Cont < Expresion.Length; Cont++) {
        if (EsUnOperador(Expresion[Cont])) TotalPuntos = 0;
        if (Expresion[Cont] == '.') TotalPuntos++;
        if (TotalPuntos > 1) {
            TipoError[14] = 1;
            return;
        }
    }
}
}
}
}

```

Si no hay ningún error sintáctico, entonces el programa retorna true.



## Paso 4: Dividiendo la cadena en partes

Se toma la cadena y se divide en partes diferenciadas dentro de un LIST: número real, operador, variable, paréntesis que abre, paréntesis que cierra y función. Por ejemplo:

"(0.4-(1.7/3.4+h)\*A(k^x)+0)"

Queda en:

|   |     |   |   |     |   |     |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|-----|---|---|-----|---|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ( | 0.4 | - | ( | 1.7 | / | 3.4 | + | h | ) | * | A | ( | k | ^ | x | ) | + | 0 | ) |
|---|-----|---|---|-----|---|-----|---|---|---|---|---|---|---|---|---|---|---|---|---|

Las variables y números constantes quedan en una lista dinámica llamada Valores de tipo double (las variables guardan sus respectivos valores allí). De esa forma cuando el evaluador necesite un valor para consultarlo o modificarlo, lo busca directamente en esa lista. Este cambio algorítmico da como resultado, que esta versión del evaluador (la 4.1), sea más rápida que la versión anterior (3.0) del evaluador de expresiones.

### Lista **Valores**

| Posición 0 a 25           | 26  | 27  | 28  | 29 |
|---------------------------|-----|-----|-----|----|
| Variables de la expresión | 0.4 | 1.7 | 3.4 | 0  |

Por lo que la lista de partes queda ahora así:

|   |      |   |   |      |   |      |   |     |   |   |   |   |      |   |      |   |   |      |   |
|---|------|---|---|------|---|------|---|-----|---|---|---|---|------|---|------|---|---|------|---|
| ( | [26] | - | ( | [27] | / | [28] | + | [7] | ) | * | A | ( | [10] | ^ | [23] | ) | + | [29] | ) |
|---|------|---|---|------|---|------|---|-----|---|---|---|---|------|---|------|---|---|------|---|

Lo que está entre [ ] es la posición del valor en la lista de Valores.

En C# esta sería la lista de Valores

```
private List<double> Valores = new List<double>();
```

Las partes se guardan aquí:

```
/* Listado de partes en que se divide la expresión
   Toma una expresión, por ejemplo:
   1.68 + sen( 3 / x ) * ( 2.92 - 9 )
   y la divide en partes así:
   [1.68] [+] [sen() [3] [/] [x] []] [*] [(] [2.92] [-] [9] []]
   Cada parte puede tener un número, un operador, una función,
   un paréntesis que abre o un paréntesis que cierra.
   En esta versión 4.1, las constantes, piezas y variables guardan
   sus valores en la lista Valores.
   En partes, se almacena la posición en Valores */
private List<ParteEvl4_1> Partes = new List<ParteEvl4_1>();
```

Y la clase de las partes es así:

H/Eval4\_1/ParteEvl4\_1.cs

```
/* Partes en que se divide la expresión
   Toma una expresión, por ejemplo:
   1.68 + sen( 3 / x ) * ( 2.92 - 9 )
   y la divide en partes así:
   [1.68] [+] [sen() [3] [/] [x] []] [*] [(] [2.92] [-] [9] []]
   Cada parte puede tener un número, un operador, una función,
   un paréntesis que abre o un paréntesis que cierra.
   En esta versión 4.1, las constantes, piezas y variables guardan
   sus valores en la lista Valores.
   En partes, se almacena la posición en Valores */
public class ParteEvl4_1 {
    /* Acumulador, función, paréntesis que abre,
       * paréntesis que cierra, operador,
       * número, variable */
    public int Tipo;

    /* Código de la función 0:seno, 1:coseno, 2:tangente,
       * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
       * 6: arcotangente, 7: logaritmo natural, 8: exponencial
       * 9: raíz cuadrada */
    public int Funcion;

    /* + suma - resta * multiplicación / división ^ potencia */
    public int Operador;

    /* Posición en lista de valores del número literal
       * por ejemplo: 3.141592 */
    public int posNumero;
```

```

/* Posición en lista de valores del
 * valor de la variable algebraica */
public int posVariable;

/* Posición en lista de valores del valor de la pieza.
 * Por ejemplo:
 * 3 + 2 / 5 se convierte así:
 * |3| |+| |2| | / | |5|
 * |3| |+| |A| A es un identificador de acumulador */
public int posAcumula;

// Diccionario estático para mapeo de operadores
private static readonly Dictionary<char, int> OperadorMap = new()
{
    {'+', 0}, {'-', 1}, {'*', 2}, {'/', 3}, {'^', 4}
};

public ParteEvl4_1(int TipoParte, int Valor) {
    Tipo = TipoParte;
    if (TipoParte == Constantes4_1.ESFUNCION) Funcion = Valor;
    else if (TipoParte == Constantes4_1.ESNUMERO) posNumero = Valor;
    else if (TipoParte == Constantes4_1.ESVARIABLE) posVariable = Valor;
    else if (TipoParte == Constantes4_1.ESPARABRE) Funcion = -1;
}

public ParteEvl4_1(char Operador) {
    Tipo = Constantes4_1.ESOPERADOR;
    this.Operador = OperadorMap[Operador];
}
}

```

## Paso 5: Generando las Piezas desde las Partes

Las Piezas tienen esta estructura: [Pieza] Función Parte1 Operador Parte2

Esta sería la clase que representa las piezas:

H/Eval4\_1/PiezaEvl4\_1.cs

```
public class PiezaEvl4_1 {  
    /* Posición donde se almacena el valor que genera  
     * la pieza al evaluarse */  
    public int PosResultado;  
  
    /* Código de la función 0:seno, 1:coseno, 2:tangente,  
     * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,  
     * 6: arcotangente, 7: logaritmo natural, 8: valor tope,  
     * 9: exponencial, 10: raíz cuadrada */  
    public int Funcion;  
  
    /* Posición donde se almacena la primera parte de la pieza */  
    public int pA;  
  
    /* 0 suma 1 resta 2 multiplicación 3 división 4 potencia */  
    public int Operador;  
  
    /* Posición donde se almacena la segunda parte de la pieza */  
    public int pB;  
}  
}
```

Esas Piezas se construyen desde las Partes. Se sigue el orden de evaluación de los paréntesis y operadores. De:

|   |      |   |   |      |   |      |   |     |   |   |   |   |      |   |      |   |   |
|---|------|---|---|------|---|------|---|-----|---|---|---|---|------|---|------|---|---|
| ( | [26] | - | ( | [27] | / | [28] | + | [7] | ) | * | A | ( | [10] | ^ | [23] | ) | ) |
|---|------|---|---|------|---|------|---|-----|---|---|---|---|------|---|------|---|---|

Queda así:

| Pieza | Función | Parte1 | Operador | Parte2 |
|-------|---------|--------|----------|--------|
| [29]  | A       | [10]   | ^        | [23]   |
| [30]  |         | [27]   | /        | [28]   |
| [31]  |         | [30]   | +        | [7]    |
| [32]  |         | [31]   | *        | [29]   |
| [33]  |         | [26]   | -        | [32]   |

## Paso 6: Evaluando a partir de las Piezas

Yendo de pieza en pieza se evalúa toda la expresión. Este método es el más crítico en cuanto a velocidad por lo que se ha optimizado. ¿Por qué? Lo usual es que se obtenga varios valores de la misma ecuación. Por ejemplo, para hacer un gráfico matemático de una ecuación  $Y=F(X)$ , hay que darle varios valores de  $X$ , la ecuación ya está analizada (convertida en piezas), sólo es cambiar el valor de  $X$ . Este evaluador se ha diseñado considerando que se requieren varios valores de la misma ecuación, por este motivo fue preferible sacrificar tiempo en el análisis (que la convierte en piezas) para darle gran velocidad en la ejecución.

A continuación, el código que evalúa la expresión ya analizada.

```
/* Evalúa la expresión convertida en piezas */
public double Evaluar() {
    double Resulta = 0;
    PiezaEvl4_1 tmp;

    /* Va de pieza en pieza */
    int TotalPiezas = Piezas.Count;
    for (int Posicion = 0; Posicion < TotalPiezas; Posicion++) {
        tmp = Piezas[Posicion];

        // Cacheo de operandos para menos indirections
        double a = Valores[tmp.pA];
        double b = Valores[tmp.pB];

        // Operación binaria (mantiene tu lógica original)
        Resulta = tmp.Operador switch {
            Constantes4_1.SUMA => a + b,
            Constantes4_1.RESTA => a - b,
            Constantes4_1.MULTIPLICA => a * b,
            Constantes4_1.DIVIDE => a / b,
            _ => Math.Pow(a, b),
        };

        switch (tmp.Funcion) {
            case Constantes4_1.SENO_COD: Resulta = Math.Sin(Resulta); break;
            case Constantes4_1.COSENO_COD: Resulta = Math.Cos(Resulta); break;
            case Constantes4_1.TANGENTE_COD: Resulta = Math.Tan(Resulta);
break;
            case Constantes4_1.VALORABSOLUTO_COD: Resulta = Math.Abs(Resulta);
break;
            case Constantes4_1.ARCOSENO_COD: Resulta = Math.Asin(Resulta);
break;
            case Constantes4_1.ARCOCOSENO_COD: Resulta = Math.Acos(Resulta);
break;
        }
    }
}
```

```
        case Constantes4_1.ARCOTANGENTE_COD: Resulta = Math.Atan(Resulta);
break;
        case Constantes4_1.LOGARITMONATURAL_COD: Resulta =
Math.Log(Resulta); break;
        case Constantes4_1.EXPONENCIAL_COD: Resulta = Math.Exp(Resulta);
break;
        case Constantes4_1.RAIZCUADRADA_COD: Resulta = Math.Sqrt(Resulta);
break;
        default:
            break;
    }

    Valores[tmp.PosResultado] = Resulta;
}
return Resulta;
}
```

## El código completo

Este es el código completo del evaluador de expresiones. La versión 4.1 refactorizó todo el código para hacerlo más comprensible. Luego tiene varias clases, cada clase en su propio archivo.

H/Eval4\_1/Constantes4\_1.cs

```
/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */

namespace Ejemplo {
    public static class Constantes4_1 {
        /* Constantes de los diferentes tipos
         * de datos que tendrán las partes y piezas */
        public const int ESFUNCION = 1;
        public const int ESPARABRE = 2;
        public const int ESPARCIERRA = 3;
        public const int ESOPERADOR = 4;
        public const int ESNUMERO = 5;
        public const int ESVARIABLE = 6;
        public const int ESACUMULA = 7;

        public const int POTENCIA = 4;
        public const int DIVIDE = 3;
        public const int MULTIPLICA = 2;
        public const int RESTA = 1;
        public const int SUMA = 0;

        public const string SENO = "A";
        public const string COSENO = "B";
        public const string TANGENTE = "C";
        public const string VALORABSOLUTO = "D";
        public const string ARCOSENO = "E";
        public const string ARCOSENO = "F";
        public const string ARCOTANGENTE = "G";
        public const string LOGARITMONATURAL = "H";
        public const string EXPONENCIAL = "I";
        public const string RAIZCUADRADA = "J";

        public const int NO_FUNCION_COD = -1;
        public const int SENO_COD = 0;
        public const int COSENO_COD = 1;
        public const int TANGENTE_COD = 2;
    }
}
```

```

public const int VALORABSOLUTO_COD = 3;
public const int ARCOSENO_COD = 4;
public const int ARCOSENO_COD = 5;
public const int ARCOTANGENTE_COD = 6;
public const int LOGARITMONATURAL_COD = 7;
public const int EXPONENCIAL_COD = 8;
public const int RAIZCUADRADA_COD = 9;
}
}

```

H/Eval4\_1/ParteEv14\_1.cs

```

/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */

namespace Ejemplo {
    /* Partes en que se divide la expresión
     Toma una expresión, por ejemplo:
     1.68 + sen( 3 / x ) * ( 2.92 - 9 )
     y la divide en partes así:
     [1.68] [+] [sen() [3] [/] [x] []] [*] [(] [2.92] [-] [9] []]
     Cada parte puede tener un número, un operador, una función,
     un paréntesis que abre o un paréntesis que cierra.
     En esta versión 4.1, las constantes, piezas y variables guardan
     sus valores en la lista Valores.
     En partes, se almacena la posición en Valores */
    public class ParteEv14_1 {
        /* Acumulador, función, paréntesis que abre,
         * paréntesis que cierra, operador,
         * número, variable */
        public int Tipo;

        /* Código de la función 0:seno, 1:coseno, 2:tangente,
         * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
         * 6: arcotangente, 7: logaritmo natural, 8: exponencial
         * 9: raíz cuadrada */
        public int Funcion;

        /* + suma - resta * multiplicación / división ^ potencia */
        public int Operador;

        /* Posición en lista de valores del número literal

```



```

    * por ejemplo: 3.141592 */
public int posNumero;

/* Posición en lista de valores del
 * valor de la variable algebraica */
public int posVariable;

/* Posición en lista de valores del valor de la pieza.
 * Por ejemplo:
   3 + 2 / 5 se convierte así:
   |3| |+| |2| | / | |5|
   |3| |+| |A| A es un identificador de acumulador */
public int posAcumula;

// Diccionario estático para mapeo de operadores
private static readonly Dictionary<char, int> OperadorMap = new()
{
    {'+', 0}, {'-', 1}, {'*', 2}, {'/', 3}, {'^', 4}
};

public ParteEvl4_1(int TipoParte, int Valor) {
    Tipo = TipoParte;
    if (TipoParte == Constantes4_1.ESFUNCION) Funcion = Valor;
    else if (TipoParte == Constantes4_1.ESNUMERO) posNumero = Valor;
    else if (TipoParte == Constantes4_1.ESVARIABLE) posVariable = Valor;
    else if (TipoParte == Constantes4_1.ESPARABRE) Funcion = -1;
}

public ParteEvl4_1(char Operador) {
    Tipo = Constantes4_1.ESOPERADOR;
    this.Operador = OperadorMap[Operador];
}
}
}

```

H/Eval4\_1/PiezaEvl4\_1.cs

```

/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */

namespace Ejemplo {
    public class PiezaEvl4_1 {

```

```

/* Posición donde se almacena el valor que genera
 * la pieza al evaluarse */
public int PosResultado;

/* Código de la función 0:seno, 1:coseno, 2:tangente,
 * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
 * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
 * 9: exponencial, 10: raíz cuadrada */
public int Funcion;

/* Posición donde se almacena la primera parte de la pieza */
public int pA;

/* 0 suma 1 resta 2 multiplicación 3 división 4 potencia */
public int Operador;

/* Posición donde se almacena la segunda parte de la pieza */
public int pB;
}
}

```

H/Eval4\_1/Sintaxis4\_1.cs

```

using System.Text;
using System.Text.RegularExpressions;

/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */

namespace Ejemplo {
    public class Sintaxis4_1 {
        /* HashSet estático para caracteres válidos (se crea una sola vez) */
        private static readonly HashSet<char> CaracteresValidos =
            [.. "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()"];

        /* Diccionario para reemplazos de funciones en expresión analizada */
        private static readonly Dictionary<string, string> ReemplazosFunciones
        = new()
        {
            {"sen", Constantes4_1.SENO}, {"cos", Constantes4_1.COSENOS}, {"tan",
Constantes4_1.TANGENTE},
            {"abs", Constantes4_1.VALORABSOLUTO}, {"asn", Constantes4_1.ARCOSENOS},
{"acs", Constantes4_1.ARCOCOSENO},

```

```

        {"atn", Constantes4_1.ARCOTANGENTE}, {"log",
Constantes4_1.LOGARITMONATURAL}, {"exp", Constantes4_1.EXPONENCIAL},
        {"sqr", Constantes4_1.RAIZCUADRADA}
    };

    /* Diccionario estático para mensajes de error */
    private static readonly Dictionary<int, string> MensajesError = new()
    {
        {0, "0. Es correcta"},
        {1, "1. Doble punto seguido"},
        {2, "2. Punto y sigue una letra"},
        {3, "3. Dos operadores estén seguidos"},
        {4, "4. Operador seguido de paréntesis que cierra"},
        {5, "5. Letra seguida de número"},
        {6, "6. Letra seguida de punto"},
        {7, "7. Letra seguida de otra letra"},
        {8, "8. Paréntesis que abre y sigue operador"},
        {9, "9. Paréntesis que abre y luego cierra"},
        {10, "10. Inicia con operador"},
        {11, "11. Finaliza con operador"},
        {12, "12. No hay correspondencia entre paréntesis"},
        {13, "13. Paréntesis desbalanceados"},
        {14, "14. Dos o más puntos en número real"}
    };

    /* HashSet estático para operadores (más rápido que switch) */
    private static readonly HashSet<char> Operadores = new() { '+', '-',
    '*', '/', '^' };

    /* Arreglo de mensajes de error */
    private int[] TipoError = new int[15];

    /* Expresión convertida y validada */
    public string ExpresionConvertida { get; private set; } = string.Empty;

    /* Convierte y chequea la expresión */
    public bool ChequeaSintaxis(string ExpresionOriginal) {
        /* Paso 1: Convierte la expresión */
        Convierte(ExpresionOriginal);

        /* Inicializa el arreglo de errores */
        Array.Fill(TipoError, 0);

        /* Paso 2: Chequea sintaxis */
        ValidaCaracteresSeguidos(ExpresionConvertida);
        ValidaInicioYFinDeLaExpresion(ExpresionConvertida);
        ValidaParentesisDesbalanceados(ExpresionConvertida);
        ValidaPuntosEnNumeroReal(ExpresionConvertida);
    }

```

```

    /* Si una sola celda es 1, hay error */
    for (int Cont = 0; Cont < TipoError.Length; Cont++) {
        if (TipoError[Cont] == 1) {
            return false;
        }
    }
    return true;
}

public List<string> ObtenerMensajesErrores() {
    List<string> mensajes = new();
    for (int i = 1; i < TipoError.Length; i++) {
        if (TipoError[i] == 1) {
            mensajes.Add(MensajeError(i));
        }
    }
    return mensajes;
}

/* Paso 1: Convertir la expresión para poder analizarla */
private void Convierte(string ExpresionOriginal) {
    /* Primero a minúsculas */
    string Minusculas = ExpresionOriginal.ToLower();

    /* Sólo los caracteres válidos (optimizado con HashSet estático) */
    StringBuilder ConLetrasValidas = new("(");
    foreach (char c in Minusculas) {
        if (CaracteresValidos.Contains(c))
            ConLetrasValidas.Append(c);
    }
    ConLetrasValidas.Append(')');

    /* Reemplazos de funciones usando diccionario */
    foreach (var reemplazo in ReemplazosFunciones) {
        ConLetrasValidas.Replace(reemplazo.Key, reemplazo.Value);
    }

    /* Agrega +0) donde exista )) porque es
     * necesario para crear las piezas */
    ConLetrasValidas.Replace("))", ") +0)");

    /* Convierte de (- a (0- para dar apoyo al menos unario */
    ConLetrasValidas.Replace("(-", "(0-");

    /* Convierte de )( a )( */
    ConLetrasValidas.Replace(")(", ")*(");
}

```

```

/* Convierte de punto operador a punto cero operador */
ConLetrasValidas.Replace("+", ".0+");
ConLetrasValidas.Replace("-", ".0-");
ConLetrasValidas.Replace(".", ".0*");
ConLetrasValidas.Replace("/", ".0/");
ConLetrasValidas.Replace(".", ".0^");

/* Convierte de operador punto a operador cero punto */
ConLetrasValidas.Replace("+.", "+0.");
ConLetrasValidas.Replace("-.", "-0.");
ConLetrasValidas.Replace("*. ", "*0.");
ConLetrasValidas.Replace("/.", "/0.");
ConLetrasValidas.Replace("^.", "^0.");

/* Convierte de .( a .0*( */
ConLetrasValidas.Replace(".( ", ".0*( ");

/* Convierte de ). a )*0. */
ConLetrasValidas.Replace("). ", ")*0.");

/* Convierte de .) a .0) */
ConLetrasValidas.Replace(".)", ".0)");

/* Convierte de (. a (0. */
ConLetrasValidas.Replace("(.", "(0.");

/* Convierte a cadena para hacer uso del regex */
ExpresionConvertida = ConLetrasValidas.ToString();

/* Convierte número letra en número * letra */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"(\d)([a-z])", "$1*$2");

/* Convierte número( en número*( */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"(\d)\(", "$1*(");

/* Convierte )numero en )*numero */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"\) (\d+)", @"")*$1");

/* Convierte de )letra en )*letra */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"\) ([a-zA-Z])", @")*$1");

/* Convierte de letra minúscula( a letra minúscula*( */
ExpresionConvertida = Regex.Replace(ExpresionConvertida, @"([a-z])\(", @"$1*(");

```

```

}

/* Retorna mensaje de error sintáctico */
private string MensajeError(int CodigoError) =>
    MensajesError.TryGetValue(CodigoError, out string? mensaje)
        ? mensaje
        : "Error desconocido";

/* Retorna si el Caracter es un operador matemático */
private static bool EsUnOperador(char Caracter) =>
    Operadores.Contains(Caracter);

/* Valida casos de dos caracteres seguidos */
private void ValidaCaracteresSeguidos(string Expresion) {
    for (int Cont = 0; Cont < Expresion.Length - 1; Cont++) {
        char carA = Expresion[Cont];
        char carB = Expresion[Cont + 1];

        if (carA == '.' && carB == '.') TipoError[1] = 1;
        if (carA == '.' && char.IsLower(carB)) TipoError[2] = 1;
        if (EsUnOperador(carA) && EsUnOperador(carB)) TipoError[3] = 1;
        if (EsUnOperador(carA) && carB == ')') TipoError[4] = 1;
        if (char.IsLower(carA) && char.IsDigit(carB)) TipoError[5] = 1;
        if (char.IsLower(carA) && carB == '.') TipoError[6] = 1;
        if (char.IsLower(carA) && char.IsLower(carB)) TipoError[7] = 1;
        if (carA == '(' && EsUnOperador(carB)) TipoError[8] = 1;
        if (carA == '(' && carB == ')') TipoError[9] = 1;
    }
}

/* Valida el inicio y fin de la expresión */
private void ValidaInicioYFinDeLaExpresion(string Expresion) {
    if (EsUnOperador(Expresion[1])) {
        TipoError[10] = 1;
    }
    if (EsUnOperador(Expresion[Expresion.Length - 2])) {
        TipoError[11] = 1;
    }
}

/* Valida si hay paréntesis desbalanceados */
private void ValidaParentesisDesbalanceados(string Expresion) {
    int Balance = 0;
    for (int Cont = 0; Cont < Expresion.Length; Cont++) {
        if (Expresion[Cont] == '(') Balance++;
        if (Expresion[Cont] == ')') Balance--;
        if (Balance < 0) {

```

```

        TipoError[12] = 1;
        return;
    }
}
if (Balance != 0) {
    TipoError[13] = 1;
}
}

/* Validar los puntos decimales de un número real */
private void ValidaPuntosEnNumeroReal(string Expresion) {
    int TotalPuntos = 0;
    for (int Cont = 0; Cont < Expresion.Length; Cont++) {
        if (EsUnOperador(Expresion[Cont])) TotalPuntos = 0;
        if (Expresion[Cont] == '.') TotalPuntos++;
        if (TotalPuntos > 1) {
            TipoError[14] = 1;
            return;
        }
    }
}
}
}
}

```

H/Eval4\_1/Evaluador4\_1.cs

```

using System.Globalization;
using System.Text;

/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */
namespace Ejemplo {

    public class Evaluador4_1 {
        /* Donde guarda los valores de variables, Constantes4_1 y piezas */
        public List<double> Valores;

        /* Listado de partes en que se divide la expresión
         Toma una expresión, por ejemplo:
         1.68 + sen( 3 / x ) * ( 2.92 - 9 )
         y la divide en partes así:
         [1.68] [+] [sen() [3] [/] [x] []] [*] [(] [2.92] [-] [9] []]
        */
    }
}

```

Cada parte puede tener un número, un operador, una función, un paréntesis que abre o un paréntesis que cierra.  
En esta versión 4.1, las Constantes4\_1, piezas y variables guardan sus valores en la lista Valores.

En partes, se almacena la posición en Valores \*/

```
public List<ParteEv14_1> Partes;
```

```
/* Listado de piezas que ejecutan  
Toma las partes y las divide en piezas con  
la siguiente estructura:
```

```
acumula = funcion valor operador valor
```

```
donde valor puede ser un número, una variable o  
un acumulador
```

Siguiendo el ejemplo anterior sería:

```
A = 2.92 - 9
```

```
B = sen( 3 / x )
```

```
C = B * A
```

```
D = 1.68 + C
```

Esas piezas se evalúan de arriba a abajo y así se interpreta la ecuación \*/

```
public List<PiezaEv14_1> Piezas;
```

```
/* Evaluar la sintaxis */
```

```
public Sintaxis4_1 objSintaxis;
```

```
/* Constructor */
```

```
public Evaluador4_1() {  
    this.Valores = [];  
    this.Partes = [];  
    this.Piezas = [];  
    this.objSintaxis = new();  
}
```

```
/* Analiza la expresión */
```

```
public bool Analizar(string ExpresionOriginal) {  
    if (objSintaxis.ChequeaSintaxis(ExpresionOriginal)) {  
        CrearPartes(objSintaxis.ExpresionConvertida);  
        CrearPiezas();  
        return true;  
    }  
    return false;  
}
```

```
/* Da valor a las variables que tendrá
```



```

    * la expresión algebraica */
public void DarValorVariable(char varAlgebra, double Valor) {
    Valores[varAlgebra - 'a'] = Valor;
}

/* Evalúa la expresión convertida en piezas */
public double Evaluar() {
    double Resulta = 0;
    PiezaEvl4_1 tmp;

    /* Va de pieza en pieza */
    int TotalPiezas = Piezas.Count;
    for (int Posicion = 0; Posicion < TotalPiezas; Posicion++) {
        tmp = Piezas[Posicion];

        // Cacheo de operandos para menos indirections
        double a = Valores[tmp.pA];
        double b = Valores[tmp.pB];

        // Operación binaria (mantiene tu lógica original)
        Resulta = tmp.Operador switch {
            Constantes4_1.SUMA => a + b,
            Constantes4_1.RESTA => a - b,
            Constantes4_1.MULTIPLICA => a * b,
            Constantes4_1.DIVIDE => a / b,
            _ => Math.Pow(a, b),
        };

        switch (tmp.Funcion) {
            case Constantes4_1.SENO_COD: Resulta = Math.Sin(Resulta); break;
            case Constantes4_1.COSENO_COD: Resulta = Math.Cos(Resulta); break;
            case Constantes4_1.TANGENTE_COD: Resulta = Math.Tan(Resulta);
break;
            case Constantes4_1.VALORABSOLUTO_COD: Resulta = Math.Abs(Resulta);
break;
            case Constantes4_1.ARCOSENO_COD: Resulta = Math.Asin(Resulta);
break;
            case Constantes4_1.ARCOCOSENO_COD: Resulta = Math.Acos(Resulta);
break;
            case Constantes4_1.ARCOTANGENTE_COD: Resulta = Math.Atan(Resulta);
break;
            case Constantes4_1.LOGARITMONATURAL_COD: Resulta =
Math.Log(Resulta); break;
            case Constantes4_1.EXPONENCIAL_COD: Resulta = Math.Exp(Resulta);
break;
            case Constantes4_1.RAIZCUADRADA_COD: Resulta = Math.Sqrt(Resulta);
break;
            default:

```

```

        break;
    }

    Valores[tmp.PosResultado] = Resulta;
}
return Resulta;
}

/* Convierte las partes en las piezas finales de ejecución */
private void CrearPiezas() {
    Piezas.Clear();

    int Contador = Partes.Count - 1;
    do {
        if (Partes[Contador].Tipo == Constantes4_1.ESPARABRE) {

            /* Evalúa las potencias de izquierda a derecha como Excel */
            GeneraPiezaOpera(Constantes4_1.POTENCIA, Constantes4_1.POTENCIA,
Contador);

            /* Luego evalúa multiplicar y dividir */
            GeneraPiezaOpera(Constantes4_1.MULTIPLICA, Constantes4_1.DIVIDE,
Contador);

            /* Finalmente evalúa sumar y restar */
            GeneraPiezaOpera(Constantes4_1.SUMA, Constantes4_1.RESTA,
Contador);

            /* Agrega la función a la última pieza */
            if (Contador > 0 && Partes[Contador - 1].Tipo ==
Constantes4_1.ESFUNCION) {
                Piezas[Piezas.Count - 1].Funcion = Partes[Contador - 1].Funcion;
                Partes.RemoveAt(Contador - 1);
                Partes.RemoveAt(Contador - 1);
                Partes.RemoveAt(Contador);
            }
            else {
                /* Quita el paréntesis que abre y
                 * el que cierra, dejando el centro */
                Partes.RemoveAt(Contador);
                Partes.RemoveAt(Contador + 1);
            }
        }
        Contador--;
    } while (Contador >= 0);
}

/* Genera las piezas buscando determinado operador */

```

```

private void GeneraPiezaOpera(int OperA, int OperB, int Inicia) {
    int Contador = Inicia + 1;
    do {
        ParteEvl4_1 tmpParte = Partes[Contador];
        if (tmpParte.Tipo == Constantes4_1.ESOPERADOR &&
            (tmpParte.Operador == OperA || tmpParte.Operador == OperB)) {
            ParteEvl4_1 tmpParteIzq = Partes[Contador - 1];
            ParteEvl4_1 tmpParteDer = Partes[Contador + 1];

            /* Crea Pieza */
            PiezaEvl4_1 NuevaPieza = new();
            NuevaPieza.Funcion = -1;

            switch (tmpParteIzq.Tipo) {
                case Constantes4_1.ESNUMERO:
                    NuevaPieza.pA = tmpParteIzq.posNumero;
                    break;

                case Constantes4_1.ESVARIABLE:
                    NuevaPieza.pA = tmpParteIzq.posVariable;
                    break;

                default:
                    NuevaPieza.pA = tmpParteIzq.posAcumula;
                    break;
            }

            NuevaPieza.Operador = tmpParte.Operador;

            switch (tmpParteDer.Tipo) {
                case Constantes4_1.ESNUMERO:
                    NuevaPieza.pB = tmpParteDer.posNumero;
                    break;

                case Constantes4_1.ESVARIABLE:
                    NuevaPieza.pB = tmpParteDer.posVariable;
                    break;

                default:
                    NuevaPieza.pB = tmpParteDer.posAcumula;
                    break;
            }

            /* Añade a lista de piezas y crea una
             * nueva posición en Valores */
            Valores.Add(0);
            NuevaPieza.PosResultado = Valores.Count - 1;
            Piezas.Add(NuevaPieza);
        }
    } while (Contador < Partes.Length);
}

```

```

    /* Elimina la parte del operador y la siguiente */
    Partes.RemoveRange(Contador, 2);

    /* Retorna el contador en uno para tomar
     * la siguiente operación */
    Contador--;

    /* Cambia la parte anterior por parte que acumula */
    tmpParteIzq.Tipo = Constantes4_1.ESACUMULA;
    tmpParteIzq.posAcumula = NuevaPieza.PosResultado;
}
Contador++;
} while (Partes[Contador].Tipo != Constantes4_1.ESPARCIERRA);
}

/* Divide la expresión en partes, yendo de caracter en caracter */
private void CrearPartes(string Expresion) {
    Partes.Clear();

    /* Hace espacio para las 26 variables que
     * puede manejar el evaluador */
    Valores = Enumerable.Repeat(0.0, 26).ToList();

    StringBuilder Numero = new();
    for (int Pos = 0; Pos < Expresion.Length; Pos++) {
        char Letra = Expresion[Pos];
        switch (Letra) {
            case '.':
            case '0':
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
            case '6':
            case '7':
            case '8':
            case '9': /* Si es un dígito o un punto
                       * va acumulando el número */
                Numero.Append(Letra); break;
            case '+':
            case '-':
            case '*':
            case '/':
            case '^':
                /* Si es un operador matemático entonces verifica
                 * si hay un número que se ha acumulado */

```

```

        if (Numero.Length > 0) {
            Valores.Add(double.Parse(Numero.ToString(),
CultureInfo.InvariantCulture));
            Partes.Add(new ParteEv14_1(Constants4_1.ESNUMERO,
Valores.Count - 1));
            Numero.Clear();
        }
        /* Agregar el operador matemático */
        Partes.Add(new ParteEv14_1(Letra));
        break;

    case '(': /* Es paréntesis que abre */
        Partes.Add(new ParteEv14_1(Constants4_1.ESPARABRE, 0));
        break;

    case ')':
        /* Si es un paréntesis que cierra entonces verifica
         * si hay un número que se ha acumulado */
        if (Numero.Length > 0) {
            Valores.Add(double.Parse(Numero.ToString(),
CultureInfo.InvariantCulture));
            Partes.Add(new ParteEv14_1(Constants4_1.ESNUMERO,
Valores.Count - 1));
            Numero.Clear();
        }

        /* Si sólo había un número o variable
         * dentro del paréntesis le agrega + 0
         * (por ejemplo: sen(x) o 3*(2) ) */
        if (Partes[Partes.Count - 2].Tipo == Constants4_1.ESPARABRE) {
            Partes.Add(new ParteEv14_1(Constants4_1.ESOPERADOR, 0));
            Valores.Add(0);
            Partes.Add(new ParteEv14_1(Constants4_1.ESNUMERO,
Valores.Count - 1));
        }

        /* Adiciona el paréntesis que cierra */
        Partes.Add(new ParteEv14_1(Constants4_1.ESPARCIERRA, 0));
        break;

    case 'A': /* Seno */
    case 'B': /* Coseno */
    case 'C': /* Tangente */
    case 'D': /* Valor absoluto */
    case 'E': /* Arcoseno */
    case 'F': /* Arcocoseno */
    case 'G': /* Arcotangente */
    case 'H': /* Logaritmo natural */
    case 'I': /* Exponencial */

```

```
        case 'J':    /* Raíz cuadrada */
            Partes.Add(new ParteEv14_1(Constants4_1.ESFUNCION, Letra -
'A')));
            break;
        default:
            Partes.Add(new ParteEv14_1(Constants4_1.ESVARIABLE, Letra -
'a')));
            break;
    }
}
}
}
```

## Como usar el evaluador de expresiones

En Visual Studio 2026, se añaden las cinco(5) clases Constantes4\_1.cs, ParteEvl4\_1, PiezaEvl4\_1, Sintaxis4\_1, Evaluador4\_1.cs al proyecto:

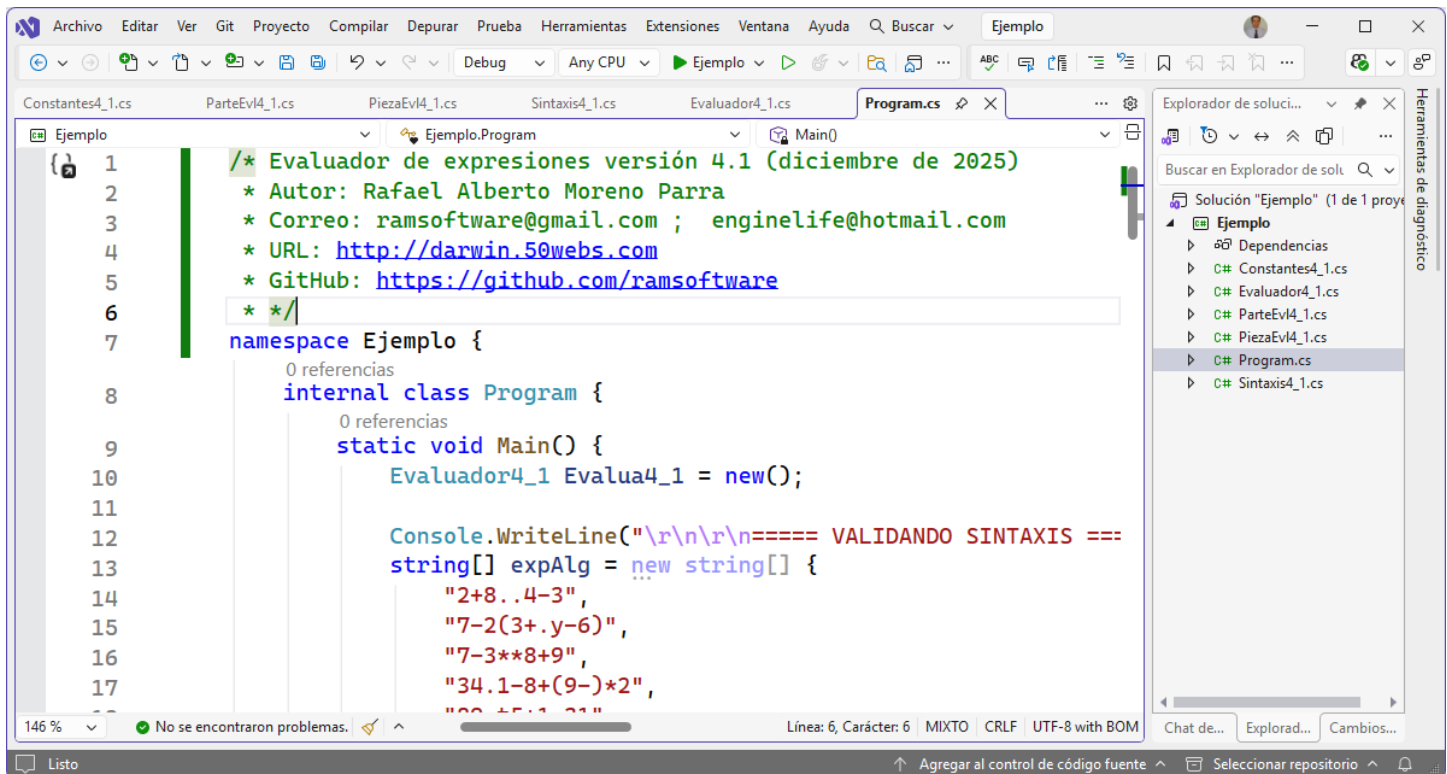


Ilustración 1: Uso del evaluador de expresiones

```
Consola de depuración X + v - □ X

Ejemplo 1: 47,25
Ejemplo 2: -650,02388966401
Ejemplo 3: -14,829744446484295
Ejemplo 4: 14,246169941947016
Ejemplo 5: 12,4

===== MULTIPLES VALORES =====
1,8442468598604296
-5,799724793004208
-3,301894840285532
-5,926209213234891
3,206835420207697
2,7297073821606705
-4,574213907246856
-1,298663405350395
1,762903374035105
-3,250237289264815
-2,116435382250971
-1,7143699021751004
-0,45637651908590526
0,49270576120855925
-3,397399717325367
4,288869053690702
-4,618048303209545
-4,000403625006605
3,56051801654666
-3,0529456328642945
```

*Ilustración 2: Resultado obtenido con el evaluador*



Este es el código fuente y se ha puesto en un proyecto:

H/Eval4\_1/Program.cs

```
/* Evaluador de expresiones versión 4.1 (diciembre de 2025)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ; enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */
namespace Ejemplo {
    internal class Program {
        static void Main() {
            Evaluador4_1 Evalua4_1 = new();

            Console.WriteLine("\r\n\r\n==== VALIDANDO SINTAXIS =====");
            string[] expAlg = new string[] {
                "2+8..4-3",
                "7-2(3+.y-6)",
                "7-3**8+9",
                "34.1-8+(9-)*2",
                "89-t5+1.31",
                "4/p.12+9",
                "2+hu-3.4",
                "9-(+2+4)",
                "9.02-7*(+)+4",
                "/3*7",
                "9-61/",
                "9+3))-((2+9",
                "(7-8)(3+1+(3*4)",
                "9.03.2+8.0121",
                "0.4-cos(3*x+2)/(2*x)-tan(7/0.8*x+3)+9.012/x^2+3*x-7.02*x"
            };

            for (int num = 0; num < expAlg.Length; num++) {
                Console.WriteLine("\r\nExpr " + num + ": " + expAlg[num]);

                if (Evalua4_1.Analizar(expAlg[num])) {
                    Console.WriteLine("Sintaxis correcta");
                }
                else {
                    List<string> Errores = Evalua4_1.objSintaxis.ObtenerMensajesErrores();
                    for (int i = 0; i < Errores.Count; i++) {
                        Console.WriteLine("Error: " + Errores[i]);
                    }
                }
            }

            //Ejemplo 1: Una expresión simple
        }
    }
}
```

```

string Ecuacion = "3+(4)-(5)+(6)-(7)";
Evalua4_1.Analizar(Ecuacion);
double valorY = Evalua4_1.Evaluar();
Console.WriteLine("Ejemplo 1: " + valorY);

//Ejemplo 2: Una expresión simple con números reales
Ecuacion = "7.318/5.0045-9.071^2*8.04961";
Evalua4_1.Analizar(Ecuacion);
valorY = Evalua4_1.Evaluar();
Console.WriteLine("Ejemplo 2: " + valorY);

//Ejemplo 3: Una expresión con paréntesis
Ecuacion = "(3+2)-7";
Evalua4_1.Analizar(Ecuacion);
valorY = Evalua4_1.Evaluar();
Console.WriteLine("Ejemplo 3: " + valorY);

//Ejemplo 4: Una expresión con funciones
Ecuacion = "sen(4.90+2.34)-cos(1.89)";
Ecuacion += "*tan(3)/abs(4-12)+asn(0.12)";
Ecuacion += "-acs(0-0.4)+atn(0.03)*log(1.3)";
Ecuacion += "+sqr(3.4)+exp(2/8)-sqr(9)";
Evalua4_1.Analizar(Ecuacion);
valorY = Evalua4_1.Evaluar();
Console.WriteLine("Ejemplo 4: " + valorY);

//Ejemplo 5: Una expresión con uso de variables
//(deben estar en minúsculas)
Ecuacion = "3*x+2*y";
Evalua4_1.Analizar(Ecuacion);

//Le da valor a las variables
Evalua4_1.DarValorVariable('x', 1.8);
Evalua4_1.DarValorVariable('y', 3.5);
valorY = Evalua4_1.Evaluar();
Console.WriteLine("Ejemplo 5: " + valorY);

//Una expresión con uso de variables
Console.WriteLine("\r\n\r\n==== MULTIPLES VALORES =====");

Ecuacion = "3*cos(2*x+4)-5*sen(4*y-7)";
Evalua4_1.Analizar(Ecuacion);

//Después de ser analizada, se le dan los
//valores a las variables, esto hace que
//el evaluador sea muy rápido
Random Azar = new();
for (int cont = 1; cont <= 20; cont++) {

```

```
double X = Azar.NextDouble();  
double Y = Azar.NextDouble();  
Evalua4_1.DarValorVariable('x', X);  
Evalua4_1.DarValorVariable('y', Y);  
valorY = Evalua4_1.Evaluar();  
Console.WriteLine(valorY);  
}  
}  
}  
}
```

## Chequear: Desempeño con el evaluador interno de C#

.NET 10 tiene un evaluador de expresiones propio, así que una forma de probar que el evaluador está funcionando bien es generando ecuaciones al azar y comparar los resultados entre el evaluador propio de C# y el evaluador 4.1, si son iguales, significa que todo marcha bien. A continuación, el código que genera ecuaciones al azar y usa ambos evaluadores, compara los resultados y además el tiempo que tarda uno y otro.

H/Chequear.zip

```
using System.Data;
using System.Diagnostics;

namespace Chequear {
    internal class Program {

        const int ANCHO_ANALISIS = 10; // ancho de la columna "Análisis"
        const int ANCHO_EVALUA = 10; // ancho de la columna "Evalúa"
        static void Main() {
            Random Azar = new();

            //Versión 2025
            Evaluador4_1 evaluador2025 = new();

            //Arreglos que guardan valores de X, Y, Z
            double[] arregloX = new double[200];
            double[] arregloY = new double[200];
            double[] arregloZ = new double[200];
            for (int cont = 0; cont < arregloX.Length; cont++) {
                arregloX[cont] = Azar.NextDouble();
                arregloY[cont] = Azar.NextDouble();
                arregloZ[cont] = Azar.NextDouble();
            }

            //Prueba evaluador
            long TotalTiempo2025Evalua = 0, TotalTiempo2025Analiza = 0;
            double valor2025, AcumValor2025 = 0;

            //Evaluador interno
            long TotalTiempoInterno = 0;
            double valorInterno, AcumInterno = 0;

            //Toma el tiempo
            Stopwatch temporizador = new();

            for (int num = 1; num <= 1000; num++) {
                string ecuacion = EcuacionAzar(350, Azar);
                //Console.WriteLine(ecuacion);
            }
        }
    }
}
```

```

//Versión 2025. Análisis.
temporizador.Reset();
temporizador.Start();
evaluador2025.Analizar(ecuacion);
temporizador.Stop();
TotalTiempo2025Analiza += temporizador.ElapsedTicks;

//Versión 2025. Evaluación
temporizador.Reset();
temporizador.Start();
valor2025 = 0;
for (int cont = 0; cont < arregloX.Length; cont++) {
    evaluador2025.DarValorVariable('x', arregloX[cont]);
    evaluador2025.DarValorVariable('y', arregloY[cont]);
    evaluador2025.DarValorVariable('z', arregloZ[cont]);
    valor2025 += Math.Abs(evaluador2025.Evaluar());
}
temporizador.Stop();
TotalTiempo2025Evalua += temporizador.ElapsedTicks;

//Compara contra el evaluador de
//expresiones propio que tiene C#
var EvaluadorInterno = new DataTable();
temporizador.Reset();
temporizador.Start();
valorInterno = 0;
for (int cont = 0; cont < arregloX.Length; cont++) {
    string ec1 = ecuacion;
    string ec2 = ec1.Replace("x", arregloX[cont].ToString());
    string ec3 = ec2.Replace("y", arregloY[cont].ToString());
    string ec4 = ec3.Replace("z", arregloZ[cont].ToString());
    string ec5 = ec4.Replace(",", ".");
    var Result = EvaluadorInterno.Compute(ec5, "");
    valorInterno += Math.Abs(Convert.ToDouble(Result));
}
temporizador.Stop();
TotalTiempoInterno += temporizador.ElapsedTicks;

if (Math.Abs(valor2025) > 10000000) continue;
if (double.IsNaN(valor2025) || double.IsInfinity(valor2025))
    continue;

AcumValor2025 += valor2025;
AcumInterno += valorInterno;
}

Console.WriteLine("[Eval4_1 ] Acumula: {0," + ANCHO_ANALISIS + "}",

```

```

        AcumValor2025);

    Console.WriteLine("[Interno  ] Acumula: {0," + ANCHO_ANALISIS + "}",
        AcumInterno);

    Console.WriteLine("\r\n\r\n[Eval4_1  ] Análisis: {0," + ANCHO_ANALISIS
+ "} Evalúa: {1," + ANCHO_EVALUA + "}",
        TotalTiempo2025Analiza, TotalTiempo2025Evalua);

    Console.WriteLine("[Eval4_1  ] Análisis y evaluación: {0," +
ANCHO_ANALISIS + "}",
        TotalTiempo2025Analiza + TotalTiempo2025Evalua);

    Console.WriteLine("[Interno  ] Análisis y evaluación: {0," +
ANCHO_ANALISIS + "}", TotalTiempoInterno);
}

public static string EcuacionAzar(int Longitud, Random Azar) {
    int cont = 0;
    int numParentesisAbre = 0;

    string Ecuacion = "";
    while (cont < Longitud) {

        //Función o paréntesis o nada
        if (Azar.NextDouble() < 0.5) {
            Ecuacion += "(";
            numParentesisAbre++;
            cont++;
        }

        //Variable o número
        cont++;
        switch (Azar.Next(4)) {
            case 0: Ecuacion += NumeroAzar(Azar); break;
            case 1: Ecuacion += "x"; break;
            case 2: Ecuacion += "y"; break;
            case 3: Ecuacion += "z"; break;
        }

        //Paréntesis que cierra
        int numParentesisCierra = Azar.Next(numParentesisAbre + 1);
        for (int num = 1; num <= numParentesisCierra; num++) {
            Ecuacion += ")";
            numParentesisAbre--;
            cont++;
        }
    }
}

```

```

        //Operador
        cont++;
        Ecuacion += OperadorAzar(Azar);
    }

    //Variable o número
    switch (Azar.Next(4)) {
        case 0: Ecuacion += NumeroAzar(Azar); break;
        case 1: Ecuacion += "x"; break;
        case 2: Ecuacion += "y"; break;
        case 3: Ecuacion += "z"; break;
    }

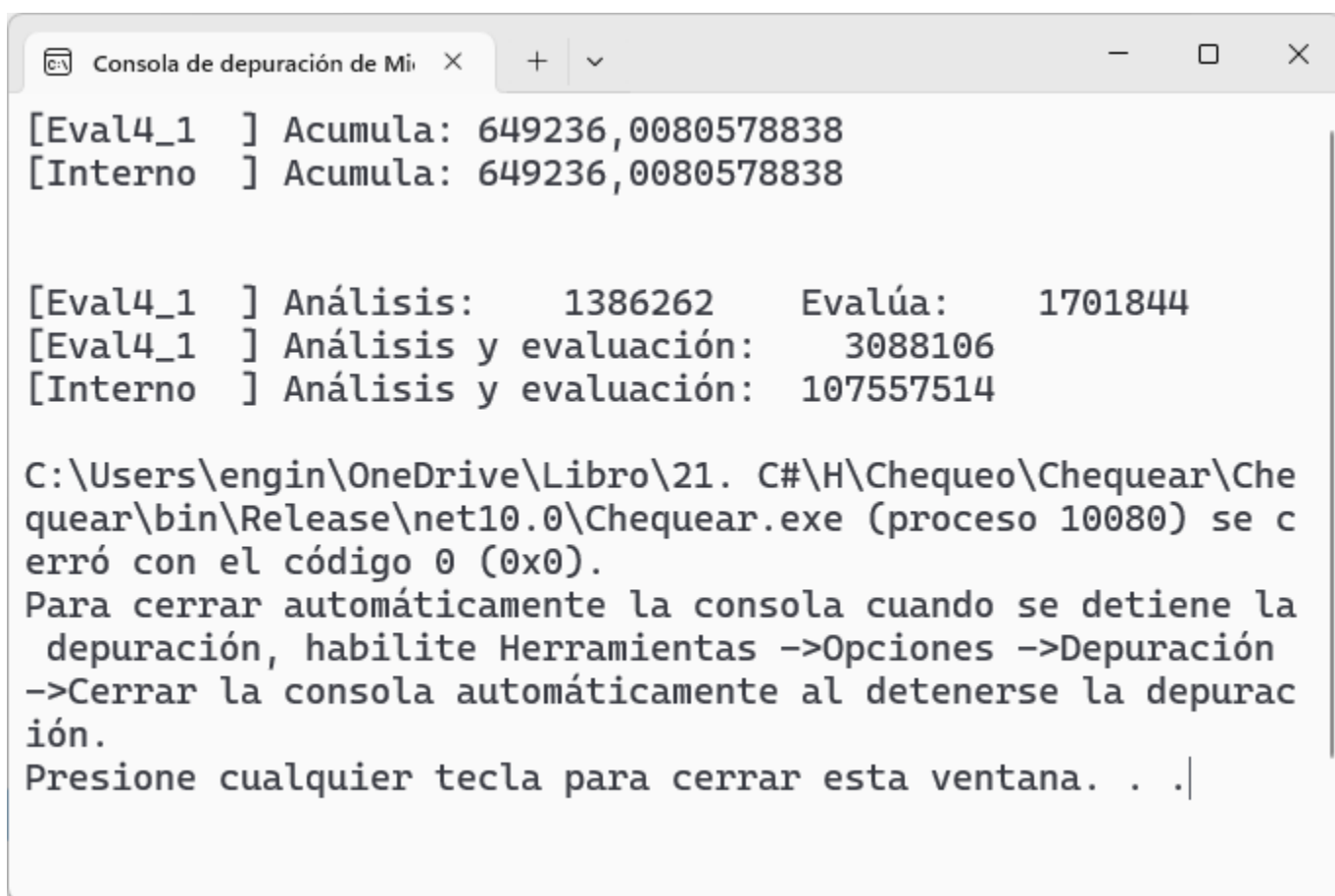
    for (int num = 0; num < numParentesisAbre; num++)
        Ecuacion += ") ";

    return Ecuacion;
}

private static string OperadorAzar(Random azar) {
    string[] operadores = { "+", "-", "*" };
    return operadores[azar.Next(operadores.Length)];
}

private static string NumeroAzar(Random azar) {
    return "0." + Convert.ToString(azar.Next(1000000) + 1);
}
}
}

```



```
[Eval4_1 ] Acumula: 649236,0080578838
[Interno  ] Acumula: 649236,0080578838

[Eval4_1 ] Análisis:      1386262      Evalúa:      1701844
[Eval4_1 ] Análisis y evaluación:      3088106
[Interno  ] Análisis y evaluación:      107557514

C:\Users\engin\OneDrive\Libro\21. C#\H\Chequeo\Chequear\Chequear\bin\Release\net10.0\Chequear.exe (proceso 10080) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .|
```

*Ilustración 3: Métrica compara ambos evaluadores*

Como se puede observar, ambos evaluadores obtienen los mismos resultados evaluando 1000 ecuaciones, cada una del tamaño de 350 caracteres y 300 valores distintos de variables por ecuación. Luego está correcto.

En el lado de desempeño, el Evaluador 4.1 es más rápido que el evaluador interno. La explicación a esta diferencia tan alta (a favor del Evaluador 4.1) es que el evaluador está diseñado, escrito y optimizado sólo para ecuaciones algebraicas, mientras el evaluador interno de C# es más genérico para diversos tipos de expresiones. La especialización vence.



## Evaluador de expresiones usando un árbol binario. Programación recursiva

## Fase 1. Sumas y restas

Se hace uso de la estructura de datos conocida como árbol binario. En este libro se aborda como fue construyéndose la solución hasta el programa final.

Se inicia con una expresión simple que tiene sólo números y los operadores son la suma y resta. Hay dos fases diferenciadas en cuanto a evaluar la expresión matemática:

1. El análisis que es la fase que construye el árbol binario.
2. La evaluación que es la fase en la que se recorre el árbol binario para dar con el resultado.

Como es una estructura de datos, el Nodo de un árbol binario tendrá esta estructura:

```
internal class Nodo {
    public int ID;
    public char Operador; //+ o -
    public double Numero;
    public Nodo Izquierda, Derecha;

    //Si el nodo tiene operador + o -
    public Nodo(char Operador, int identifica) {
        Numero = 0;
        this.Operador = Operador;
        Izquierda = null;
        Derecha = null;
        ID = identifica;
    }

    //Si el Nodo sólo tiene número
    public Nodo(double Numero, int identifica) {
        this.Numero = Numero;
        Operador = '#';
        Izquierda = null;
        Derecha = null;
        ID = identifica;
    }

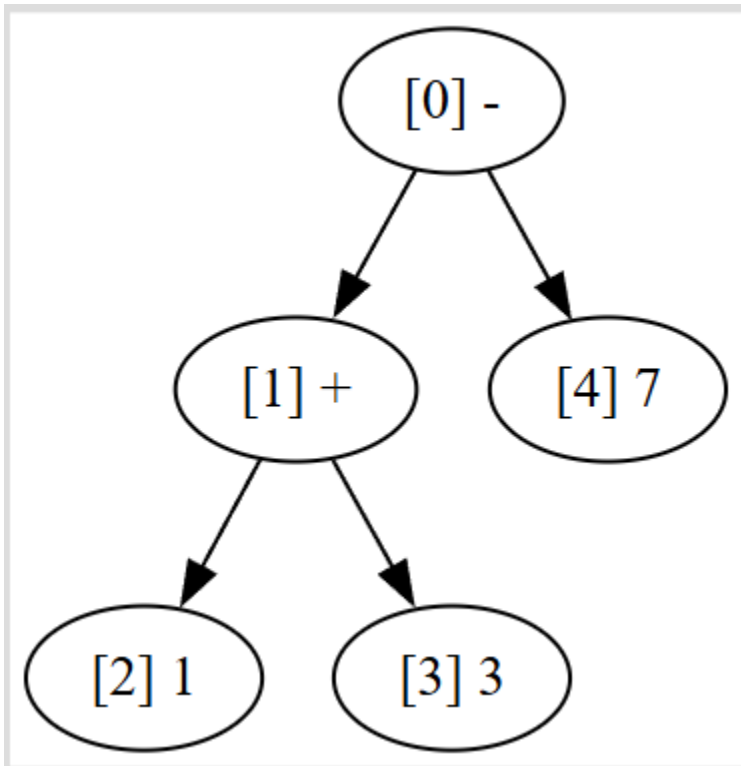
    public void Imprime() {
        if (Operador != '#')
            Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
        else
            Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
    }
}
```

El Nodo maneja dos tipos de datos: el número o el operador (suma o resta). Para este libro se añade un tercer atributo que es el ID que va a ser usado para poder dibujar el árbol usando la herramienta en línea <http://viz-js.com> pero más adelante se retirará. Como se trabaja con árboles binarios, entonces el Nodo maneja dos apuntadores: Izquierda y Derecha.

Una expresión matemática como:

1+3-7

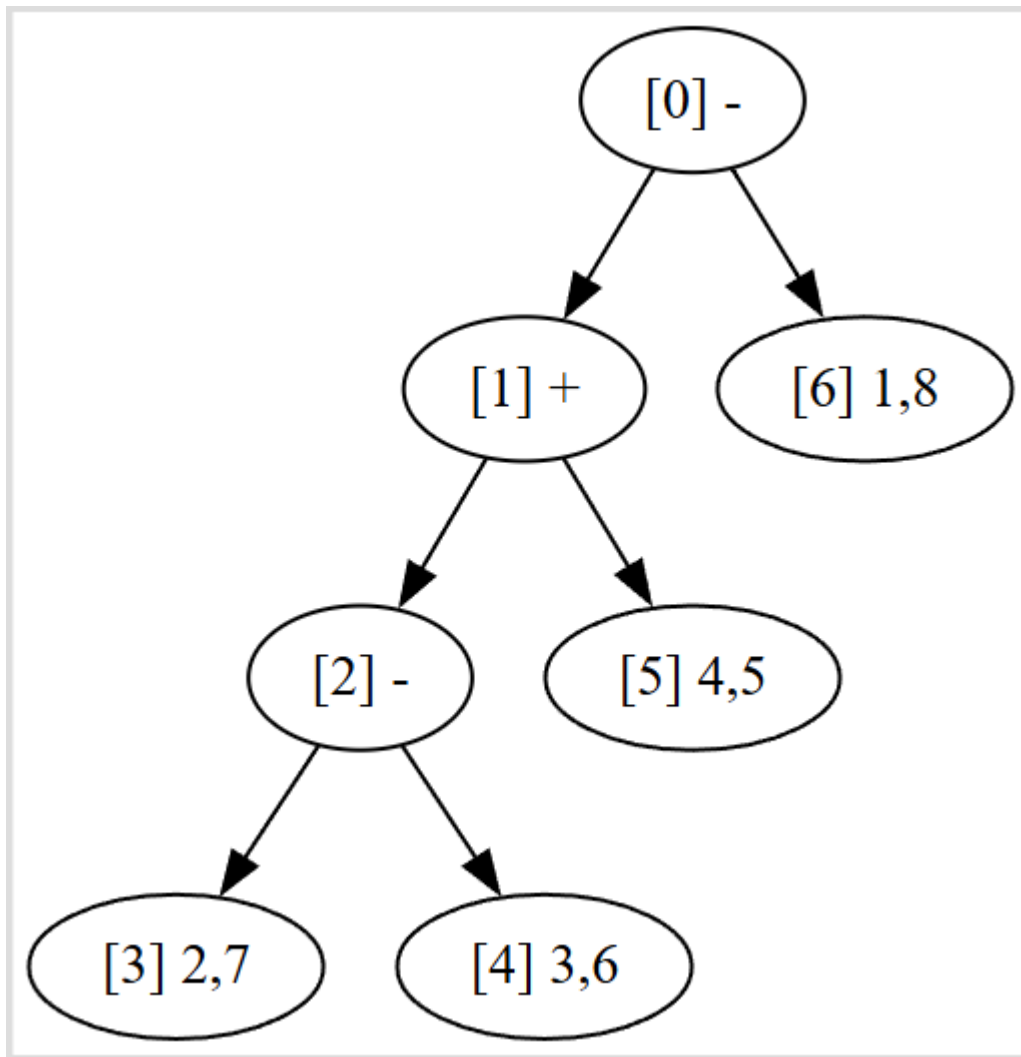
Debe formar un árbol binario así:



*Ilustración 4: Árbol binario generado*

Un segundo ejemplo:

$2.7-3.6+4.5-1.8$



*Ilustración 5: Árbol binario generado*

```
using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de números y operadores suma y resta

namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ o -
        public double Numero;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + o -
        public Nodo(char Operador, int identifica) {
            Numero = 0;
            this.Operador = Operador;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            this.Numero = Numero;
            Operador = '#';
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        public void Imprime() {
            if (Operador != '#')
                Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
            else
                Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
        }
    }

    internal class Program {
        static int Identifica = 0;

        static void Main(string[] args) {
            //Ecuación con sólo sumas y restas
            string Ecuacion = "8+3-4+1-9";
            Nodo MiArbol = null;
        }
    }
}
```

```

MiArbol = CreaArbol(Ecuacion, MiArbol);

//Probarlo en: http://viz-js.com
Console.WriteLine("digraph testgraph{");
Dibujar(MiArbol);
Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cadena, Nodo Arbol) {
    //Busca +, -
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '+' || Cadena[Cont] == '-') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Sólo queda el número y se le crea el nodo
    double Numero;
    Numero = double.Parse(Cadena, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
    return Arbol;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}
}
}
}
}

```

```
Consola de depuración de Mi × +
digraph testgraph{
  "[0] -" -> "[1] +"
  "[1] +" -> "[2] -"
  "[2] -" -> "[3] +"
  "[3] +" -> "[4] 8"
  "[3] +" -> "[5] 3"
  "[2] -" -> "[6] 4"
  "[1] +" -> "[7] 1"
  "[0] -" -> "[8] 9"
}
```

Ilustración 6: Ejemplo de ejecución del programa

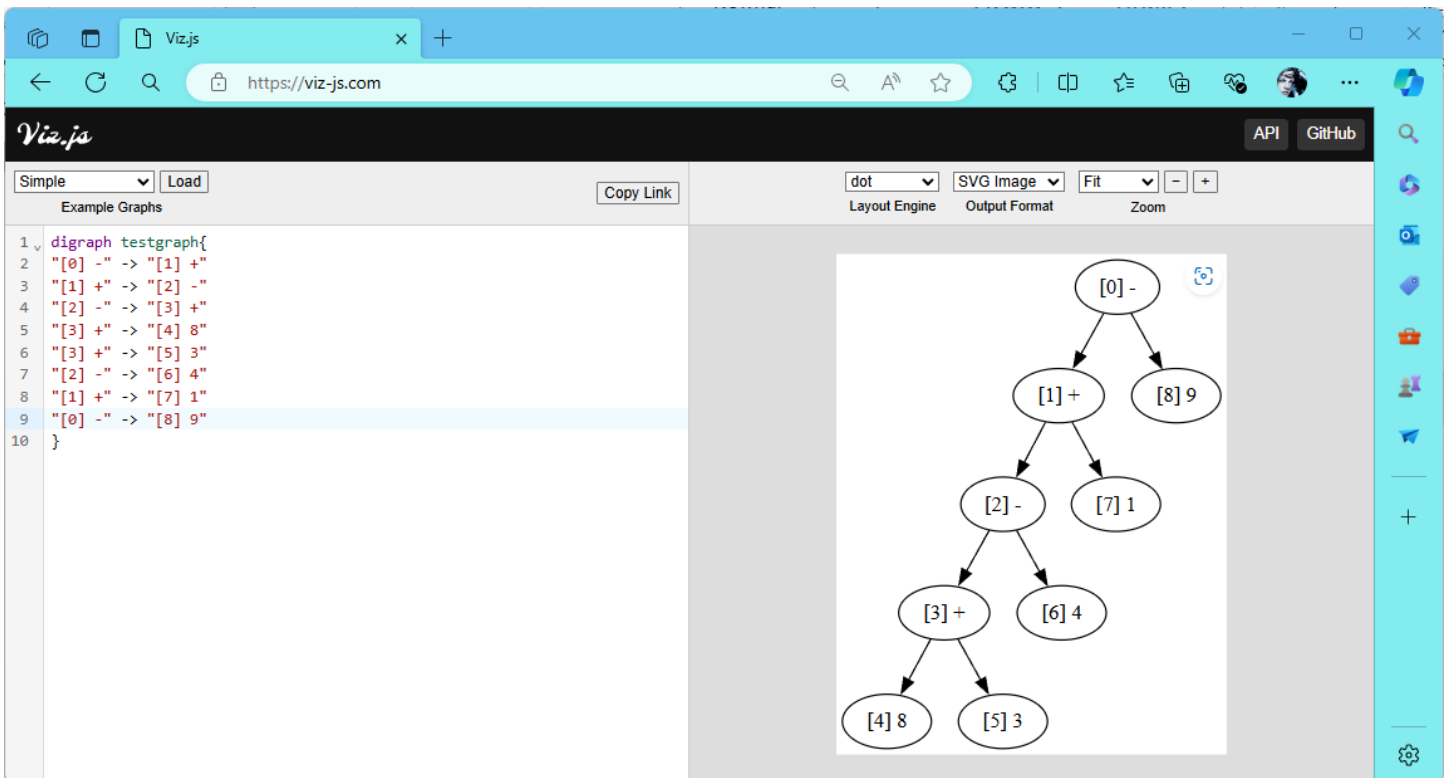


Ilustración 7: Interpretación en viz-js.com

Para evaluar la expresión, ya generado el árbol binario, es recorrerlo en post-orden (izquierda, derecha, raíz).

H/Árbol/001.zip

```
using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de números y operadores suma y resta
//Luego evalúa la expresión
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ o -
        public double Numero;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + o -
        public Nodo(char Operador, int identifica) {
            Numero = 0;
            this.Operador = Operador;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            this.Numero = Numero;
            Operador = '#';
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        public void Imprime() {
            if (Operador != '#')
                Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
            else
                Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
        }
    }

    internal class Program {
        static int Identifica = 0;

        static void Main(string[] args) {
            //Ecuación con sólo sumas y restas
            string Ecuacion = "1+2-3+4-5";
        }
    }
}
```



```

    Nodo MiArbol = null;
    MiArbol = CreaArbol(Ecuacion, MiArbol);

    //Evalúa la expresión
    double Resultado = EvaluaArbol(MiArbol);
    Console.WriteLine("Resultado es: " + Resultado);

    //Probarlo en: http://viz-js.com
    Console.WriteLine("digraph testgraph{");
    Dibujar(MiArbol);
    Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cadena, Nodo Arbol) {
    //Busca +, -
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '+' || Cadena[Cont] == '-') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Sólo queda el número y se le crea el nodo
    double Numero;
    Numero = double.Parse(Cadena, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
    return Arbol;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}

```

```

    }
}

// Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay ramas hijas, entonces es un número
    if (Arbol.Izquierda == null)
        return Arbol.Numero;

    //Recorrido Post-Orden
    double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
    double ValDerecha = EvaluaArbol(Arbol.Derecha);

    //Aplica operador
    switch (Arbol.Operador) {
        case '+': return ValIzquierda + ValDerecha;
        case '-': return ValIzquierda - ValDerecha;
    }
    return 0;
}
}
}

```

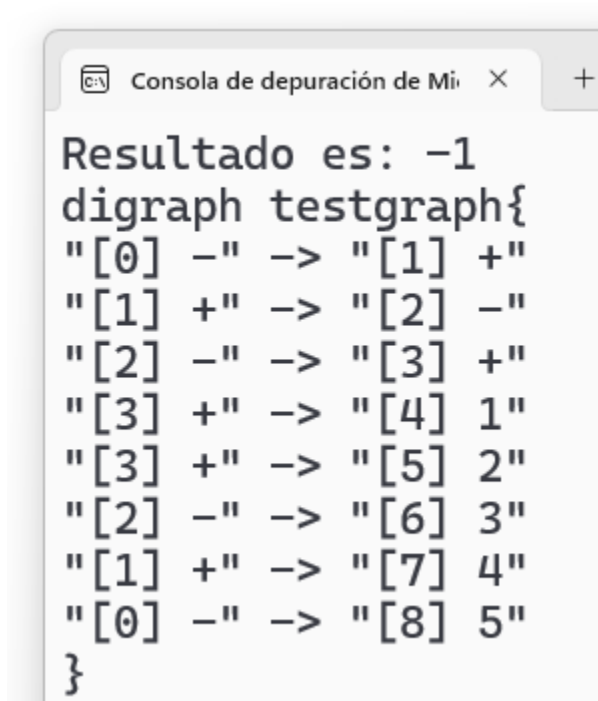


Ilustración 8: Resultado al evaluar

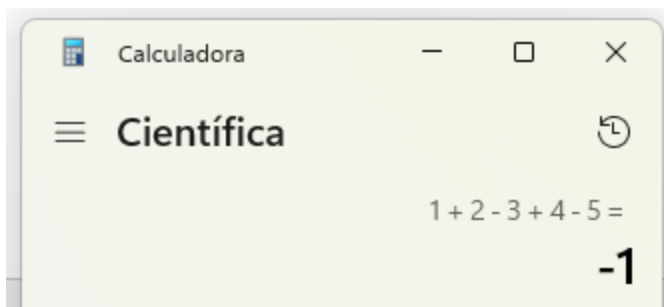


Ilustración 9: Validado con la calculadora

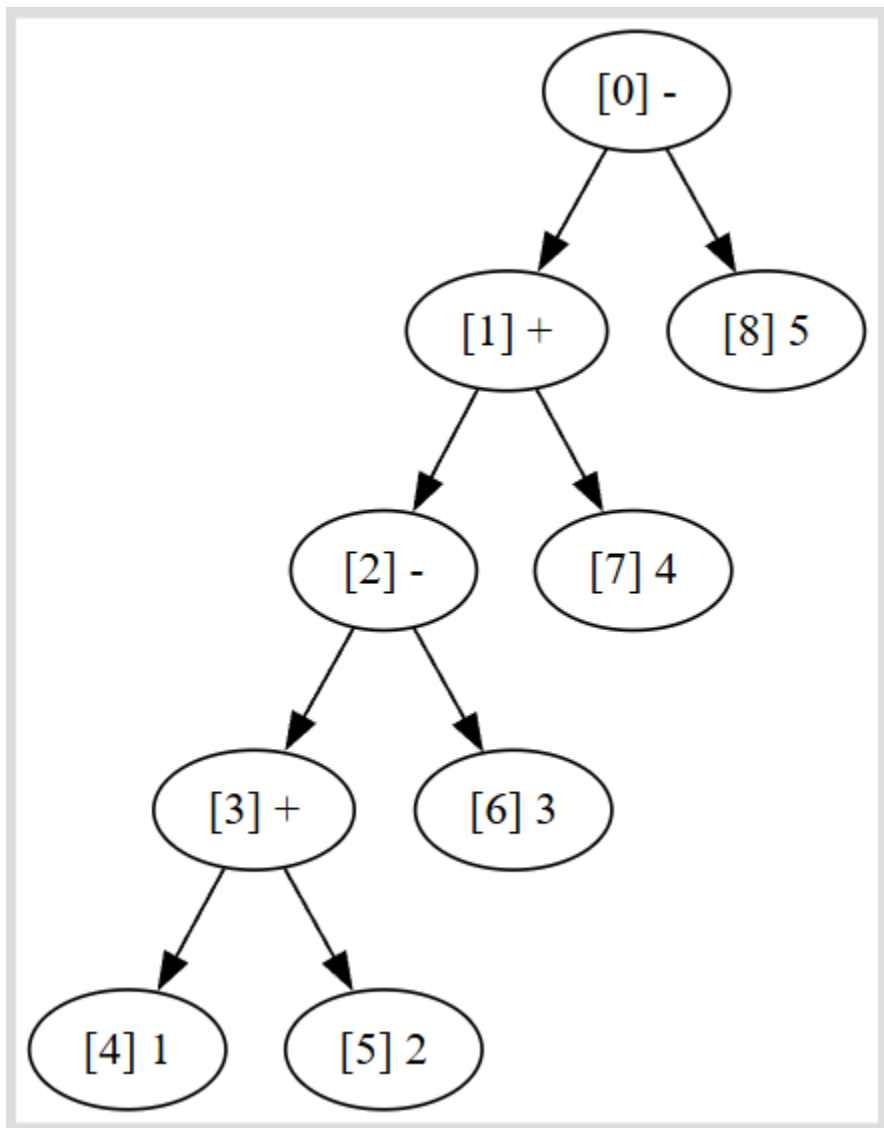


Ilustración 10: Árbol binario generado

## Fase 2. Multiplicación y división

El siguiente paso es implementar los operadores de multiplicación y división. Según las reglas matemáticas, primero deben evaluarse estos operadores. Luego ante una expresión así:

$$2*3+4-5*6+7/8+9$$

El árbol binario generado debe ser este:

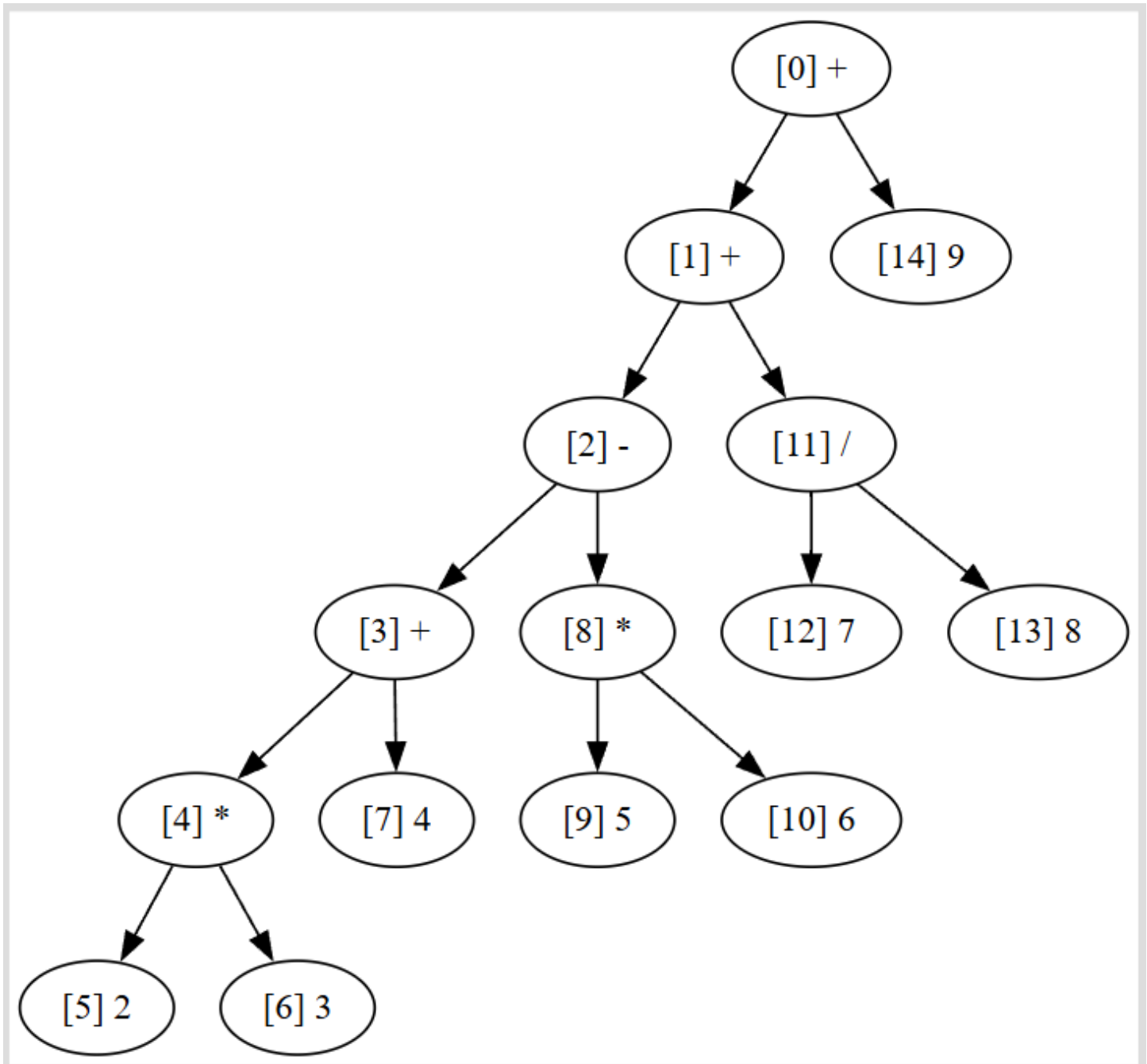


Ilustración 11: Árbol binario generado

Al evaluarse en post-orden se obtiene el resultado. Este es el programa:

```

using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de números y los operadores:
//Suma, resta, multiplicación y división
//Luego evalúa la expresión
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ - * /
        public double Numero;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * /
        public Nodo(char Operador, int identifica) {
            Numero = 0;
            this.Operador = Operador;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            this.Numero = Numero;
            Operador = '#';
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        public void Imprime() {
            if (Operador != '#')
                Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
            else
                Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
        }
    }

    internal class Program {
        static int Identifica = 0;

        static void Main(string[] args) {
            //Ecuación los cuatro operadores
            string Ecuacion = "1+2*3+4/5-6";
            Nodo MiArbol = null;
            MiArbol = CreaArbol(Ecuacion, MiArbol);
        }
    }
}

```

```

//Evalúa la expresión
double Resultado = EvaluaArbol(MiArbol);
Console.WriteLine("Resultado es: " + Resultado);

//Probarlo en: http://viz-js.com
Console.WriteLine("digraph testgraph{");
Dibujar(MiArbol);
Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cadena, Nodo Arbol) {
    //Busca +, -
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '+' || Cadena[Cont] == '-') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Busca *, /
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '*' || Cadena[Cont] == '/') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Sólo queda el número y se le crea el nodo
    double Numero;
    Numero = double.Parse(Cadena, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
    return Arbol;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
        }
    }
}

```

```

        Arbol.Izquierda.Imprime();
        Console.WriteLine(" ");
        Dibujar(Arbol.Izquierda);
    }
    if (Arbol.Derecha != null) {
        Arbol.Imprime();
        Console.Write(" -> ");
        Arbol.Derecha.Imprime();
        Console.WriteLine(" ");
        Dibujar(Arbol.Derecha);
    }
}
}

// Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay ramas hijas, luego es un número
    if (Arbol.Izquierda == null)
        return Arbol.Numero;

    //Recorrido Post-Orden
    double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
    double ValDerecha = EvaluaArbol(Arbol.Derecha);

    //Aplica operador
    switch (Arbol.Operador) {
        case '+': return ValIzquierda + ValDerecha;
        case '-': return ValIzquierda - ValDerecha;
        case '*': return ValIzquierda * ValDerecha;
        case '/': return ValIzquierda / ValDerecha;
    }
    return 0;
}
}
}

```

Así ejecuta:

```
Consola de depuración de Mi  ×  +  v

Resultado es: -10,125
digraph testgraph{
"[0] +" -> "[1] +"
"[1] +" -> "[2] -"
"[2] -" -> "[3] +"
"[3] +" -> "[4] *"
"[4] *" -> "[5] 2"
"[4] *" -> "[6] 3"
"[3] +" -> "[7] 4"
"[2] -" -> "[8] *"
"[8] *" -> "[9] 5"
"[8] *" -> "[10] 6"
"[1] +" -> "[11] /"
"[11] /" -> "[12] 7"
"[11] /" -> "[13] 8"
"[0] +" -> "[14] 9"
}
```

Ilustración 12: Ejecución del evaluador

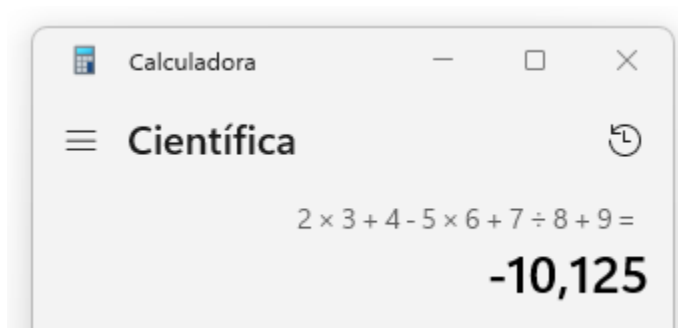


Ilustración 13: Prueba con calculadora



### Fase 3. Potencia

El último operador es la potencia, que es con el símbolo  $^$ . Este operador debe ser evaluado primero, luego multiplicación y división, por último suma y resta.

Una expresión como:

$$2^3 - 8 + 7 * 6 - 1 + 7 / 5$$

Genera este árbol binario:

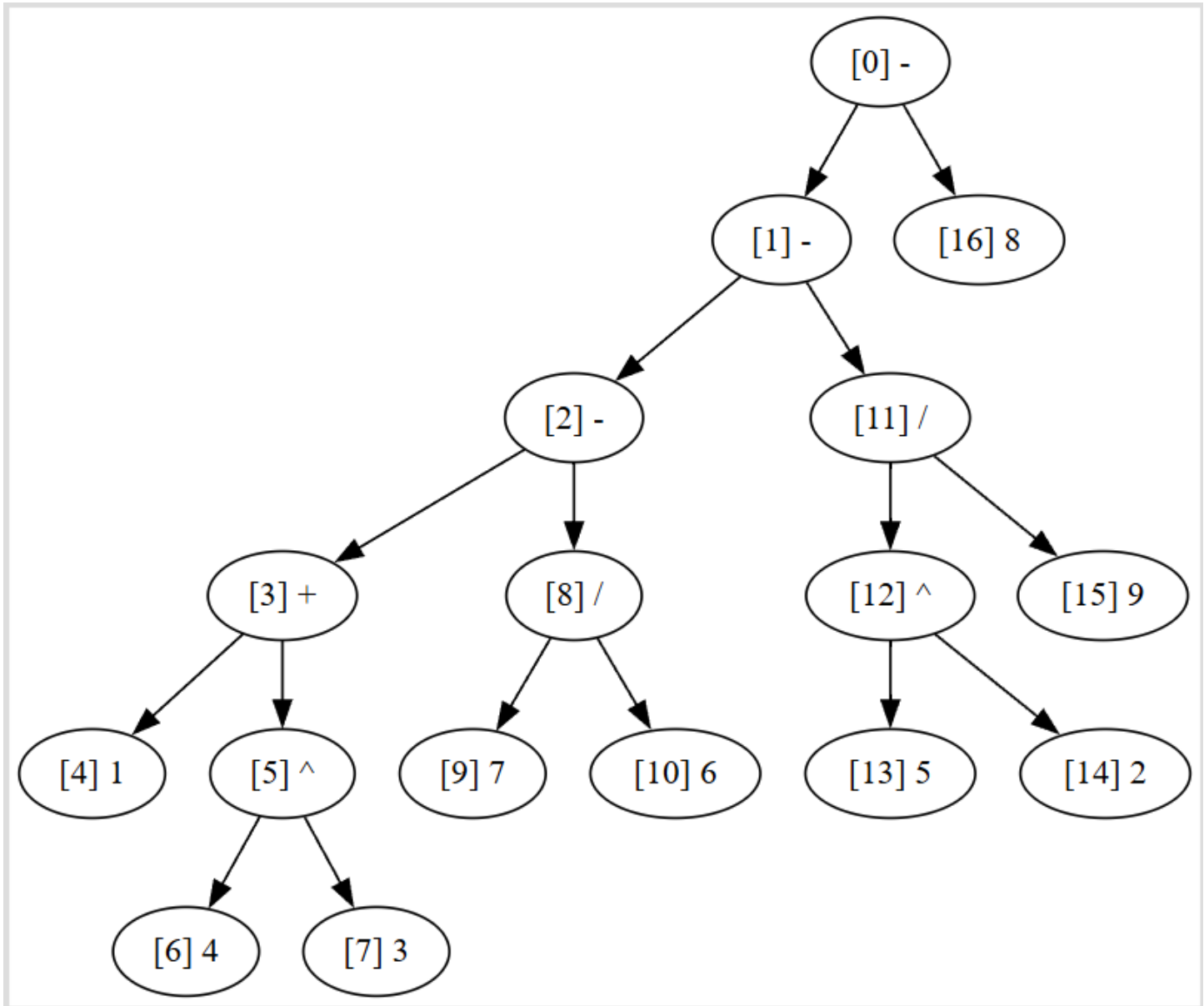


Ilustración 14: Árbol binario generado

Al recorrerlo en Post-Orden se evalúa la expresión.

```

using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de números y los operadores:
//Suma, resta, multiplicación, división y potencia
//Luego evalúa la expresión
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ - * / ^
        public double Numero;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * / ^
        public Nodo(char Operador, int identifica) {
            Numero = 0;
            this.Operador = Operador;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            this.Numero = Numero;
            Operador = '#';
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        public void Imprime() {
            if (Operador != '#')
                Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
            else
                Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
        }
    }

    internal class Program {
        static int Identifica = 0;

        static void Main(string[] args) {
            //Ecuación los cuatro operadores
            string Ecuacion = "1+4^3-7/6-5^2/9-8";

```

```

    Nodo MiArbol = null;
    MiArbol = CreaArbol(Ecuacion, MiArbol);

    //Evalúa la expresión
    double Resultado = EvaluaArbol(MiArbol);
    Console.WriteLine("Resultado es: " + Resultado);

    //Probarlo en: http://viz-js.com
    Console.WriteLine("digraph testgraph{");
    Dibujar(MiArbol);
    Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cadena, Nodo Arbol) {
    //Busca +, -
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '+' || Cadena[Cont] == '-') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Busca *, /
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '*' || Cadena[Cont] == '/') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Busca ^
    for (int Cont = Cadena.Length - 1; Cont >= 0; Cont--) {
        if (Cadena[Cont] == '^') {
            string Izquierda = Cadena.Substring(0, Cont);
            string Derecha = Cadena.Substring(Cont + 1);
            Arbol = new Nodo(Cadena[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }
}

```

```

}

//Sólo queda el número y se le crea el nodo
double Numero;
Numero = double.Parse(Cadena, CultureInfo.InvariantCulture);
Arbol = new Nodo(Numero, Identifica++);
return Arbol;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}

// Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay ramas hijas entonces es un número
    if (Arbol.Izquierda == null)
        return Arbol.Numero;

    //Recorrido Post-Orden
    double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
    double ValDerecha = EvaluaArbol(Arbol.Derecha);

    //Aplica operador
    switch (Arbol.Operador) {
        case '+': return ValIzquierda + ValDerecha;
        case '-': return ValIzquierda - ValDerecha;
        case '*': return ValIzquierda * ValDerecha;
        case '/': return ValIzquierda / ValDerecha;
        case '^': return Math.Pow(ValIzquierda, ValDerecha);
    }
    return 0;
}

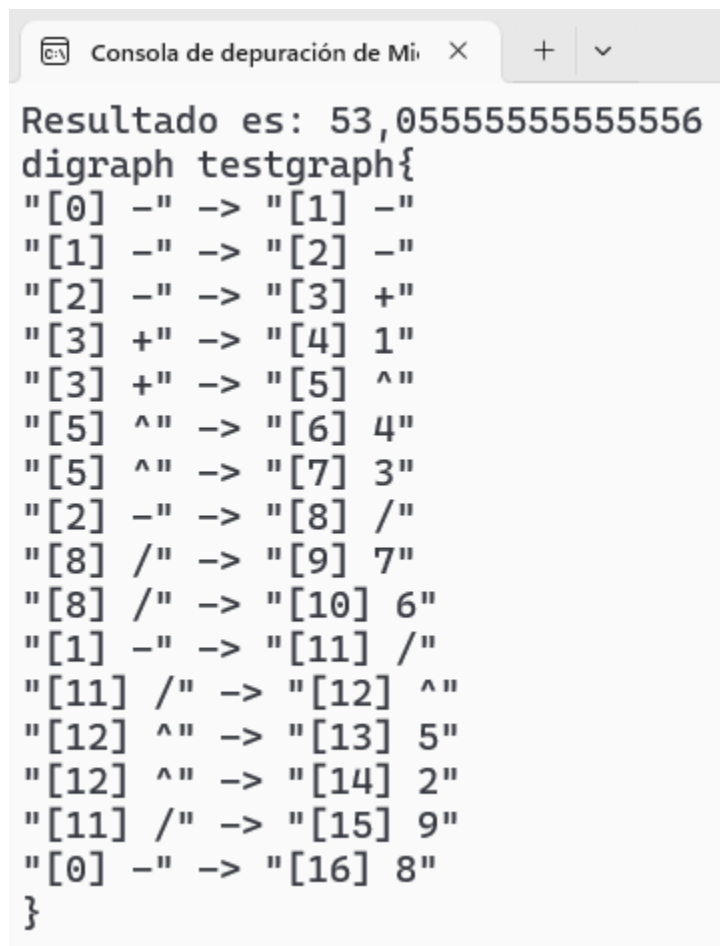
```

```

}
}
}

```

Ejemplo de ejecución:



Consola de depuración de Mi

Resultado es: 53,05555555555556

```

digraph testgraph{
"[0] -" -> "[1] -"
"[1] -" -> "[2] -"
"[2] -" -> "[3] +"
"[3] +" -> "[4] 1"
"[3] +" -> "[5] ^"
"[5] ^" -> "[6] 4"
"[5] ^" -> "[7] 3"
"[2] -" -> "[8] /"
"[8] /" -> "[9] 7"
"[8] /" -> "[10] 6"
"[1] -" -> "[11] /"
"[11] /" -> "[12] ^"
"[12] ^" -> "[13] 5"
"[12] ^" -> "[14] 2"
"[11] /" -> "[15] 9"
"[0] -" -> "[16] 8"
}

```

Ilustración 15: Ejemplo de ejecución

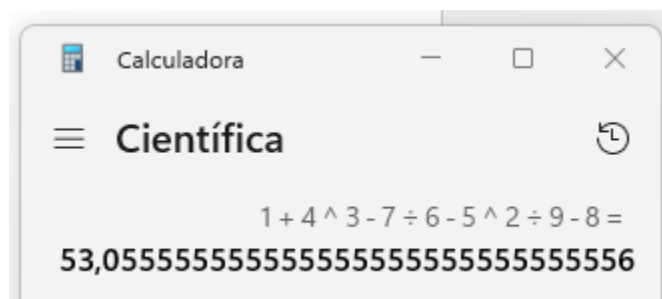


Ilustración 16: Prueba con la calculadora

## Fase 4. Paréntesis

Los paréntesis cambian el orden de interpretación de una expresión matemática, hay que considerar que puede haber paréntesis internos. Incluso hay casos como que hay paréntesis internos sin ninguna función, ejemplo:

$46-(((792))) + (((125)))$

Puede convertirse a:

$46-792+125$

Luego el software debe eliminar ese tipo de paréntesis.

Una expresión como:

$(1+6)-((8*3)+2/9)+2/9$

Genera un árbol binario así:

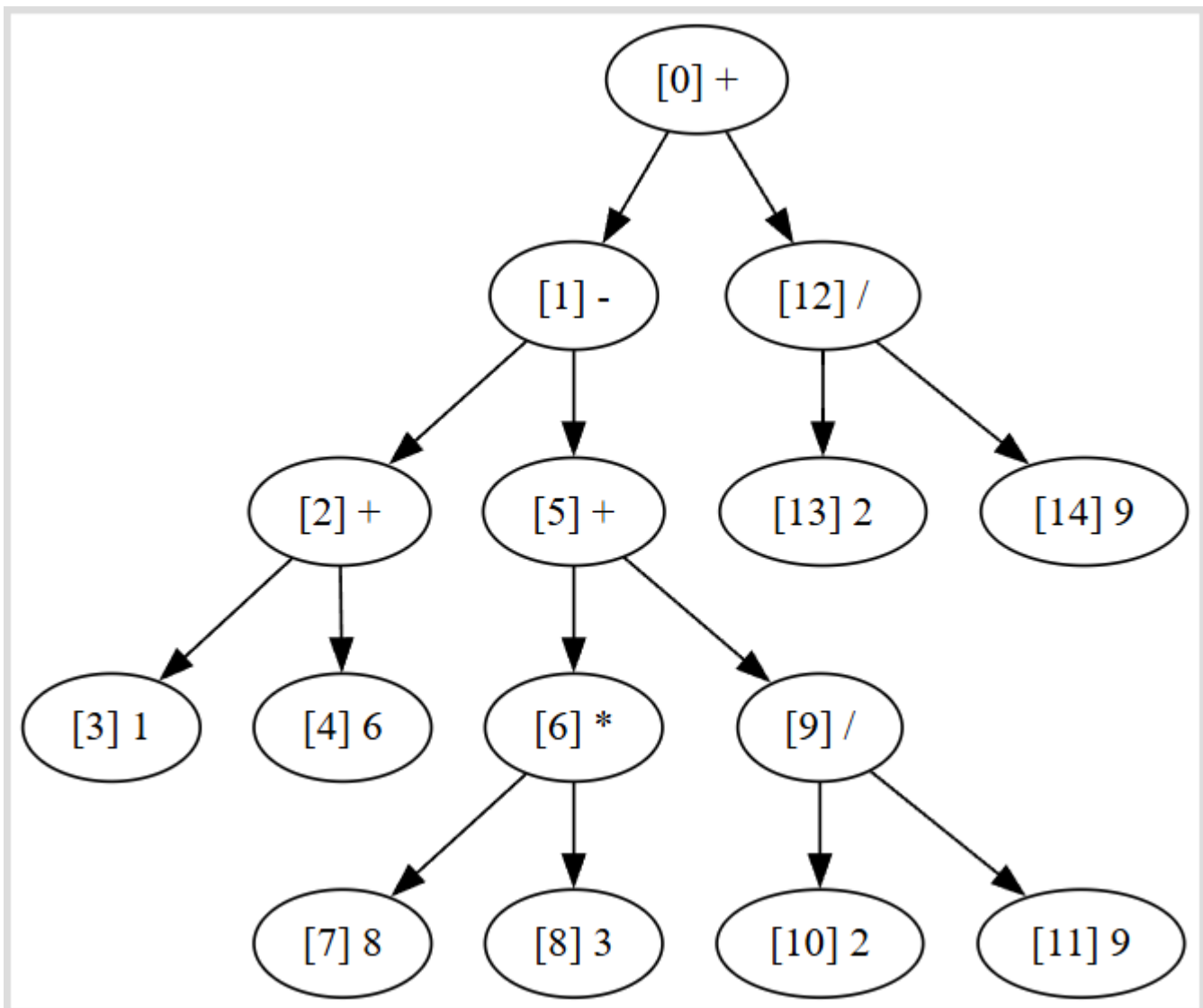


Ilustración 17: Árbol binario generado

Este es el programa:

H/Árbol/004.zip

```
using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de paréntesis, números y los operadores:
//Suma, resta, multiplicación, división y potencia
//Luego evalúa la expresión
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ - * / ^
        public double Numero;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * / ^
        public Nodo(char Operador, int identifica) {
            Numero = 0;
            this.Operador = Operador;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            this.Numero = Numero;
            Operador = '#';
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        public void Imprime() {
            if (Operador != '#')
                Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
            else
                Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
        }
    }

    internal class Program {
        static int Identifica = 0;

        static void Main(string[] args) {
```

```

//Ecuación los cuatro operadores
string Ecuacion = "(1+6)-((8*3)+2/9)+2/9";
Nodo MiArbol = null;
MiArbol = CreaArbol(Ecuacion, MiArbol);

//Evalúa la expresión
double Resultado = EvaluaArbol(MiArbol);
Console.WriteLine("Resultado es: " + Resultado);

//Probarlo en: http://viz-js.com
Console.WriteLine("digraph testgraph{");
Dibujar(MiArbol);
Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cad, Nodo Arbol) {
    /* Elimina paréntesis al inicio y al final si es
       una expresión del tipo:
       (expresión de números y operadores)
       la convierte en:
       expresión de números y operadores
       Ejemplo:
       (2^3-8+7*2-14+7/2)
       Se vuelve:
       2^3-8+7*2-14+7/2

       Pero si encuentra algo así:
       (2+4) * (5-2)
       No aplica tal conversión porque no son
       paréntesis que cubren toda la expresión
    */
    int Prntss;
    bool EsFinal;
    do {
        EsFinal = false;
        if (Cad[0] == '(') {
            EsFinal = true;
            Prntss = 0;
            for (int Cont = 1; Cont < Cad.Length - 1; Cont++) {
                if (Cad[Cont] == '(') Prntss++;
                if (Cad[Cont] == ')') Prntss--;
                if (Prntss < 0) {
                    EsFinal = false;
                    break;
                }
            }
        }
    }
    if (EsFinal)

```



```

        Cad = Cad.Substring(1, Cad.Length - 2);
    } while (EsFinal == true);

    //Busca +, -
    Prntss = 0;
    for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
        if (Cad[Cont] == '(') Prntss++;
        if (Cad[Cont] == ')') Prntss--;
        if ((Cad[Cont] == '+' || Cad[Cont] == '-') && Prntss == 0) {
            string Izquierda = Cad.Substring(0, Cont);
            string Derecha = Cad.Substring(Cont + 1);
            Arbol = new Nodo(Cad[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Busca *, /
    for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
        if (Cad[Cont] == '(') Prntss++;
        if (Cad[Cont] == ')') Prntss--;
        if ((Cad[Cont] == '*' || Cad[Cont] == '/') && Prntss == 0) {
            string Izquierda = Cad.Substring(0, Cont);
            string Derecha = Cad.Substring(Cont + 1);
            Arbol = new Nodo(Cad[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Busca ^
    for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
        if (Cad[Cont] == '(') Prntss++;
        if (Cad[Cont] == ')') Prntss--;
        if (Cad[Cont] == '^' && Prntss == 0) {
            string Izquierda = Cad.Substring(0, Cont);
            string Derecha = Cad.Substring(Cont + 1);
            Arbol = new Nodo(Cad[Cont], Identifica++);
            Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
            Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
            return Arbol;
        }
    }

    //Sólo queda el número y se le crea el nodo
    double Numero = double.Parse(Cad, CultureInfo.InvariantCulture);

```

```

    Arbol = new Nodo(Numero, Identifica++);
    return Arbol;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}

//Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay ramas hijas entonces es un número
    if (Arbol.Izquierda == null)
        return Arbol.Numero;

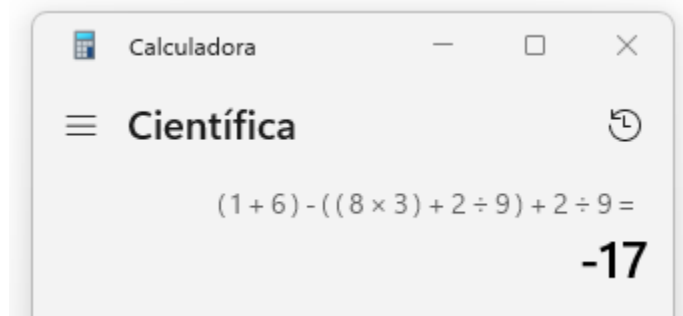
    //Recorrido Post-Orden
    double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
    double ValDerecha = EvaluaArbol(Arbol.Derecha);

    //Aplica operador
    switch (Arbol.Operador) {
        case '+': return ValIzquierda + ValDerecha;
        case '-': return ValIzquierda - ValDerecha;
        case '*': return ValIzquierda * ValDerecha;
        case '/': return ValIzquierda / ValDerecha;
        case '^': return Math.Pow(ValIzquierda, ValDerecha);
    }
    return 0;
}
}
}

```

Así ejecuta:

```
Consola de depuración de Mi X +
Resultado es: -17
digraph testgraph{
"[0] +" -> "[1] -"
"[1] -" -> "[2] +"
"[2] +" -> "[3] 1"
"[2] +" -> "[4] 6"
"[1] -" -> "[5] +"
"[5] +" -> "[6] *"
"[6] *" -> "[7] 8"
"[6] *" -> "[8] 3"
"[5] +" -> "[9] /"
"[9] /" -> "[10] 2"
"[9] /" -> "[11] 9"
"[0] +" -> "[12] /"
"[12] /" -> "[13] 2"
"[12] /" -> "[14] 9"
}
```



## Fase 5. Variables

Una expresión matemática puede manejar variables, en este caso permite variables de la 'a' hasta la 'z'.

En un arreglo unidimensional se tiene el valor de las variables:

```
static double[] Valores = new double[26];
```

Y las variables se tratan igual que con el evaluador 4.1

```
string Ecuacion = "x+y-z/q";  
DarValorVariable('x', 2);  
DarValorVariable('y', 3);  
DarValorVariable('z', 9);  
DarValorVariable('q', 3);
```

Y la función DarValorVariable

```
/* Da valor a las variables que tendrá  
 * la expresión algebraica */  
public static void DarValorVariable(char varAlgebra, double Valor) {  
    Valores[varAlgebra - 'a'] = Valor;  
}
```

Este sería el código:

H/Árbol/005.zip

```
using System.Globalization;  
  
//Forma el árbol binario dada una expresión matemática  
//de números, variables, paréntesis y los operadores:  
//Suma, resta, multiplicación, división y potencia  
//Luego evalúa la expresión  
namespace Ejemplo {  
    internal class Nodo {  
        public int ID;  
        public char Operador; //+ - * / ^  
        public double Numero;  
        public int Variable;
```

```

public Nodo Izquierda, Derecha;

//Si el nodo tiene operador + - * / ^
public Nodo(char Operador, int identifica) {
    Variable = -1;
    Numero = 0;
    this.Operador = Operador;
    Izquierda = null;
    Derecha = null;
    ID = identifica;
}

//Si el Nodo sólo tiene número
public Nodo(double Numero, int identifica) {
    Variable = -1;
    this.Numero = Numero;
    Operador = '#';
    Izquierda = null;
    Derecha = null;
    ID = identifica;
}

//Si el Nodo sólo tiene variable
public Nodo(int Variable, int identifica) {
    this.Variable = Variable;
    Numero = 0;
    Operador = '#';
    Izquierda = null;
    Derecha = null;
    ID = identifica;
}

public void Imprime() {
    if (Operador != '#')
        Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
    else if (Variable != -1) {
        int Ascii = 'a' + Variable;
        char Letra = (char)Ascii;
        Console.WriteLine("\n[" + ID + "] " + Letra + "\n");
    }
    else
        Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
}

internal class Program {
    static int Identifica = 0;
    static double[] Valores = new double[26];
}

```

```

static void Main(string[] args) {
    //Ecuación los cuatro operadores
    string Ecuacion = "x+y-z/q";
    DarValorVariable('x', 2);
    DarValorVariable('y', 3);
    DarValorVariable('z', 9);
    DarValorVariable('q', 3);

    Nodo MiArbol = null;
    MiArbol = CreaArbol(Ecuacion, MiArbol);

    //Evalúa la expresión
    double Resultado = EvaluaArbol(MiArbol);
    Console.WriteLine("Resultado es: " + Resultado);

    //Probarlo en: http://viz-js.com
    Console.WriteLine("digraph testgraph{");
    Dibujar(MiArbol);
    Console.WriteLine("}");
}

public static Nodo CreaArbol(string Cad, Nodo Arbol) {
    /* Elimina paréntesis al inicio y al final si es
       una expresión del tipo:
       (expresión de números y operadores)
       la convierte en:
       expresión de números y operadores
       Ejemplo:
       (2^3-8+7*2-14+7/2)
       Se vuelve:
       2^3-8+7*2-14+7/2

       Pero si encuentra algo así:
       (2+4) * (5-2)
       No aplica tal conversión porque no son
       paréntesis que cubren toda la expresión
    */
    int Prntss;
    bool EsFinal;
    do {
        EsFinal = false;
        if (Cad[0] == '(') {
            EsFinal = true;
            Prntss = 0;
            for (int Cont = 1; Cont < Cad.Length - 1; Cont++) {
                if (Cad[Cont] == '(') Prntss++;
                if (Cad[Cont] == ')') Prntss--;
            }
        }
    } while (!EsFinal);
}

```

```

        if (Prntss < 0) {
            EsFinal = false;
            break;
        }
    }
}
if (EsFinal)
    Cad = Cad.Substring(1, Cad.Length - 2);
} while (EsFinal == true);

//Busca +, -
Prntss = 0;
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '+' || Cad[Cont] == '-') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca *, /
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '*' || Cad[Cont] == '/') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca ^
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if (Cad[Cont] == '^' && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);

```

```

        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Sólo queda el número o la variable y se le crea el nodo
if (Cad[0] >= 'a' && Cad[0] <= 'z') {
    int Variable = (int)Cad[0] - 'a';
    Arbol = new Nodo(Variable, Identifica++);
}
else {
    double Numero;
    Numero = double.Parse(Cad, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
}

return Arbol;
}

/* Da valor a las variables que tendrá
 * la expresión algebraica */
public static void DarValorVariable(char varAlgebra, double Valor) {
    Valores[varAlgebra - 'a'] = Valor;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}

//Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay rama hija entonces es número o variable

```



```

if (Arbol.Izquierda == null)
    if (Arbol.Variable != -1)
        return Valores[Arbol.Variable];
    else
        return Arbol.Numero;

//Recorrido Post-Orden
double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
double ValDerecha = EvaluaArbol(Arbol.Derecha);

//Aplica operador
switch (Arbol.Operador) {
    case '+': return ValIzquierda + ValDerecha;
    case '-': return ValIzquierda - ValDerecha;
    case '*': return ValIzquierda * ValDerecha;
    case '/': return ValIzquierda / ValDerecha;
    case '^': return Math.Pow(ValIzquierda, ValDerecha);
}
return 0;
}
}
}

```

Una expresión como:

$x+y-z/q$

Genera un árbol binario así:

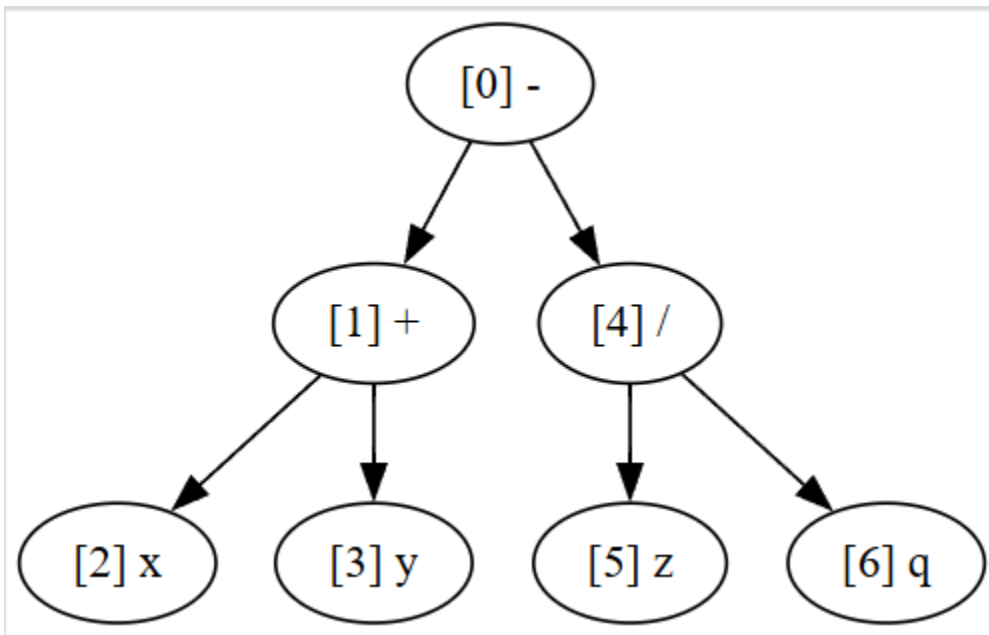
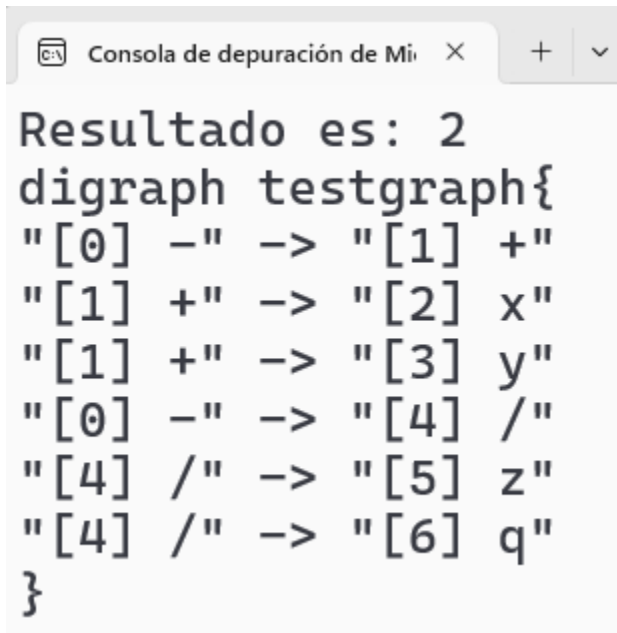


Ilustración 18: Árbol binario generado

Y esta es su ejecución:



```
Resultado es: 2
digraph testgraph{
"[0] -" -> "[1] +"
"[1] +" -> "[2] x"
"[1] +" -> "[3] y"
"[0] -" -> "[4] /"
"[4] /" -> "[5] z"
"[4] /" -> "[6] q"
}
```

*Ilustración 19: Ejemplo de ejecución*

## Fase 6. Funciones matemáticas

Finalmente vienen las funciones matemáticas como seno, coseno, tangente. En este caso, la función tiene su propio nodo, la rama izquierda es la que tiene toda la operación interna que va a ser luego evaluada por la función, como requiere una rama derecha, entonces esta rama derecha es simplemente un número cero.

Las funciones son de tres letras. Una expresión como:

$$3-\text{sen}(x*y+2)$$

Se reemplaza a:

$$3-A(x*y+2)$$

Así facilita evaluar la expresión.

Esta es la tabla de equivalencias:

| Función | Descripción       | Letra con que se reemplaza |
|---------|-------------------|----------------------------|
| Sen     | Seno              | A                          |
| Cos     | Coseno            | B                          |
| Tan     | Tangente          | C                          |
| Abs     | Valor absoluto    | D                          |
| Asn     | Arcoseno          | E                          |
| Acs     | Arcocoseno        | F                          |
| Atn     | Arcotangente      | G                          |
| Log     | Logaritmo Natural | H                          |
| Exp     | Exponencial       | I                          |
| Sqr     | Raíz cuadrada     | J                          |

```
using System.Globalization;

//Forma el árbol binario dada una expresión matemática
//de números, variables, paréntesis, funciones y los operadores:
//Suma, resta, multiplicación, división y potencia
//Luego evalúa la expresión
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ - * / ^
        public double Numero;
        public int Variable;
        public int Funcion;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * / ^
        public Nodo(char Operador, int identifica) {
            Variable = -1;
            Numero = 0;
            this.Operador = Operador;
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            Variable = -1;
            this.Numero = Numero;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene variable
        public Nodo(int Variable, int identifica) {
            this.Variable = Variable;
            Numero = 0;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
        }
    }
}
```

```

    ID = identifica;
}

//Si el Nodo es de una función
public Nodo(int Funcion, int identifica, bool EsFuncion) {
    Variable = -1;
    Numero = 0;
    Operador = '#';
    this.Funcion = Funcion;
    Izquierda = null;
    Derecha = null;
    ID = identifica;
}

public void Imprime() {
    if (Operador != '#')
        Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
    else if (Variable != -1) {
        int Ascii = 'a' + Variable;
        char Letra = (char)Ascii;
        Console.WriteLine("\n[" + ID + "] " + Letra + "\n");
    }
    else if (Funcion != -1) {
        Console.WriteLine("\n[" + ID + "] ");
        /* Código de la función 0:seno, 1:coseno, 2:tangente,
        * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
        * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
        * 9: exponencial, 10: raíz cuadrada */
        switch (Funcion) {
            case 0: Console.WriteLine("seno \n"); break;
            case 1: Console.WriteLine("coseno \n"); break;
            case 2: Console.WriteLine("tangente \n"); break;
            case 3: Console.WriteLine("absoluto \n"); break;
            case 4: Console.WriteLine("arcoseno \n"); break;
            case 5: Console.WriteLine("arcocoseno \n"); break;
            case 6: Console.WriteLine("arcotangente \n"); break;
            case 7: Console.WriteLine("log \n"); break;
            case 8: Console.WriteLine("ceil \n"); break;
            case 9: Console.WriteLine("exp \n"); break;
            case 10: Console.WriteLine("sqrt \n"); break;
        }
    }
    else
        Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
}
}

```

```

internal class Program {
    static int Identifica = 0;
    static double[] Valores = new double[26];

    static void Main(string[] args) {
        //Ecuación operadores, variables, funciones, paréntesis
        string Ecuacion = "sen(cos(x)+sen(y+z))/cos(78-q)";
        string Convertir = Convierte(Ecuacion);
        DarValorVariable('x', 120);
        DarValorVariable('y', 150);
        DarValorVariable('z', 45);
        DarValorVariable('q', -10);

        Nodo MiArbol = null;
        MiArbol = CreaArbol(Convertir, MiArbol);

        //Evalúa la expresión
        double Resultado = EvaluaArbol(MiArbol);
        Console.WriteLine("Resultado es: " + Resultado);

        //Probarlo en: http://viz-js.com
        Console.WriteLine("digraph testgraph{");
        Dibujar(MiArbol);
        Console.WriteLine("}");
    }

    public static Nodo CreaArbol(string Cad, Nodo Arbol) {
        int Prntss;
        bool EsFinal;

        //Detecta si es función
        if (Cad[0] >= 'A' && Cad[0] <= 'J') {
            //Busca el paréntesis que cierra la función
            EsFinal = true;
            Prntss = 0;
            for (int Cont = 2; Cont < Cad.Length - 1; Cont++) {
                if (Cad[Cont] == '(') Prntss++;
                if (Cad[Cont] == ')') Prntss--;
                if (Prntss < 0) {
                    EsFinal = false;
                    break;
                }
            }
        }

        //¿Es una función en solitario?
        if (EsFinal) {
            int Ascii = Cad[0] - 'A';
            Arbol = new Nodo(Ascii, Identifica++, true);
        }
    }
}

```

```

Arbol.Derecha = new Nodo(0.0, Identifica);

//Retira la letra A, el primer y último paréntesis
Cad = Cad.Substring(2, Cad.Length - 3);
Arbol.Izquierda = CreaArbol(Cad, Arbol.Izquierda);
return Arbol;
}
}

/* Elimina paréntesis al inicio y al final si es
una expresión del tipo:
(expresión de números y operadores)
la convierte en:
expresión de números y operadores
Ejemplo:
(2^3-8+7*2-14+7/2)
Se vuelve:
2^3-8+7*2-14+7/2

Pero si encuentra algo así:
(2+4) * (5-2)
No aplica tal conversión porque no son
paréntesis que cubren toda la expresión
*/
do {
    EsFinal = false;
    if (Cad[0] == '(') {
        EsFinal = true;
        Prntss = 0;
        for (int Cont = 1; Cont < Cad.Length - 1; Cont++) {
            if (Cad[Cont] == '(') Prntss++;
            if (Cad[Cont] == ')') Prntss--;
            if (Prntss < 0) {
                EsFinal = false;
                break;
            }
        }
    }
    if (EsFinal)
        Cad = Cad.Substring(1, Cad.Length - 2);
} while (EsFinal == true);

//Busca +, -
Prntss = 0;
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '+' || Cad[Cont] == '-') && Prntss == 0) {

```

```

        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca *, /
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '*' || Cad[Cont] == '/') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca ^
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if (Cad[Cont] == '^' && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Sólo queda el número o la variable y se le crea el nodo
if (Cad[0] >= 'a' && Cad[0] <= 'z') {
    int Variable = Cad[0] - 'a';
    Arbol = new Nodo(Variable, Identifica++);
}
else {
    double Numero;
    Numero = double.Parse(Cad, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
}
}

```



```

    return Arbol;
}

/* Da valor a las variables que tendrá
 * la expresión algebraica */
public static void DarValorVariable(char varAlgebra, double Valor) {
    Valores[varAlgebra - 'a'] = Valor;
}

static void Dibujar(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            Dibujar(Arbol.Derecha);
        }
    }
}

//Recorrido en Post-Orden para
//evaluar el árbol binario
public static double EvaluaArbol(Nodo Arbol) {
    //No hay rama hija entonces es número o variable
    if (Arbol.Izquierda == null)
        if (Arbol.Variable != -1)
            return Valores[Arbol.Variable];
        else
            return Arbol.Numero;

    //Recorrido Post-Orden
    double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
    double ValDerecha = EvaluaArbol(Arbol.Derecha);

    /* Código de la función 0:seno, 1:coseno, 2:tangente,
     * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
     * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
     * 9: exponencial, 10: raíz cuadrada */
    if (Arbol.Funcion != -1) {
        switch (Arbol.Funcion) {

```

```

        case 0: return Math.Sin(ValIzquierda);
        case 1: return Math.Cos(ValIzquierda);
        case 2: return Math.Tan(ValIzquierda);
        case 3: return Math.Abs(ValIzquierda);
        case 4: return Math.Asin(ValIzquierda);
        case 5: return Math.Acos(ValIzquierda);
        case 6: return Math.Atan(ValIzquierda);
        case 7: return Math.Log(ValIzquierda);
        case 8: return Math.Ceiling(ValIzquierda);
        case 9: return Math.Exp(ValIzquierda);
        case 10: return Math.Sqrt(ValIzquierda);
    }
}

//Aplica operador
switch (Arbol.Operador) {
    case '+': return ValIzquierda + ValDerecha;
    case '-': return ValIzquierda - ValDerecha;
    case '*': return ValIzquierda * ValDerecha;
    case '/': return ValIzquierda / ValDerecha;
    case '^': return Math.Pow(ValIzquierda, ValDerecha);
}
return 0;
}

//Convierte la expresión algebraica, escrita por el usuario,
//a un formato que pueda ser interpretado por el evaluador
static string Convierte(string ExpOrig) {
    /* Primero a minúsculas */
    string Minusculas = ExpOrig.ToLower();

    /* Cadena a evaluar */
    string Cadena = Minusculas.Replace("sen", "A");
    Cadena = Cadena.Replace("cos", "B");
    Cadena = Cadena.Replace("tan", "C");
    Cadena = Cadena.Replace("abs", "D");
    Cadena = Cadena.Replace("asn", "E");
    Cadena = Cadena.Replace("acs", "F");
    Cadena = Cadena.Replace("atn", "G");
    Cadena = Cadena.Replace("log", "H");
    Cadena = Cadena.Replace("exp", "I");
    Cadena = Cadena.Replace("sqr", "J");

    return Cadena;
}
}
}

```

```
Consola de depuración de Mi X + v
Resultado es: 0,8597038048551232
digraph testgraph{
"[0] /" -> "[1] seno "
"[1] seno " -> "[2] +"
"[2] +" -> "[3] coseno "
"[3] coseno " -> "[4] x"
"[3] coseno " -> "[4] 0"
"[2] +" -> "[5] seno "
"[5] seno " -> "[6] +"
"[6] +" -> "[7] y"
"[6] +" -> "[8] z"
"[5] seno " -> "[6] 0"
"[1] seno " -> "[2] 0"
"[0] /" -> "[9] coseno "
"[9] coseno " -> "[10] -"
"[10] -" -> "[11] 78"
"[10] -" -> "[12] q"
"[9] coseno " -> "[10] 0"
}
```

Ilustración 20: Ejemplo de ejecución

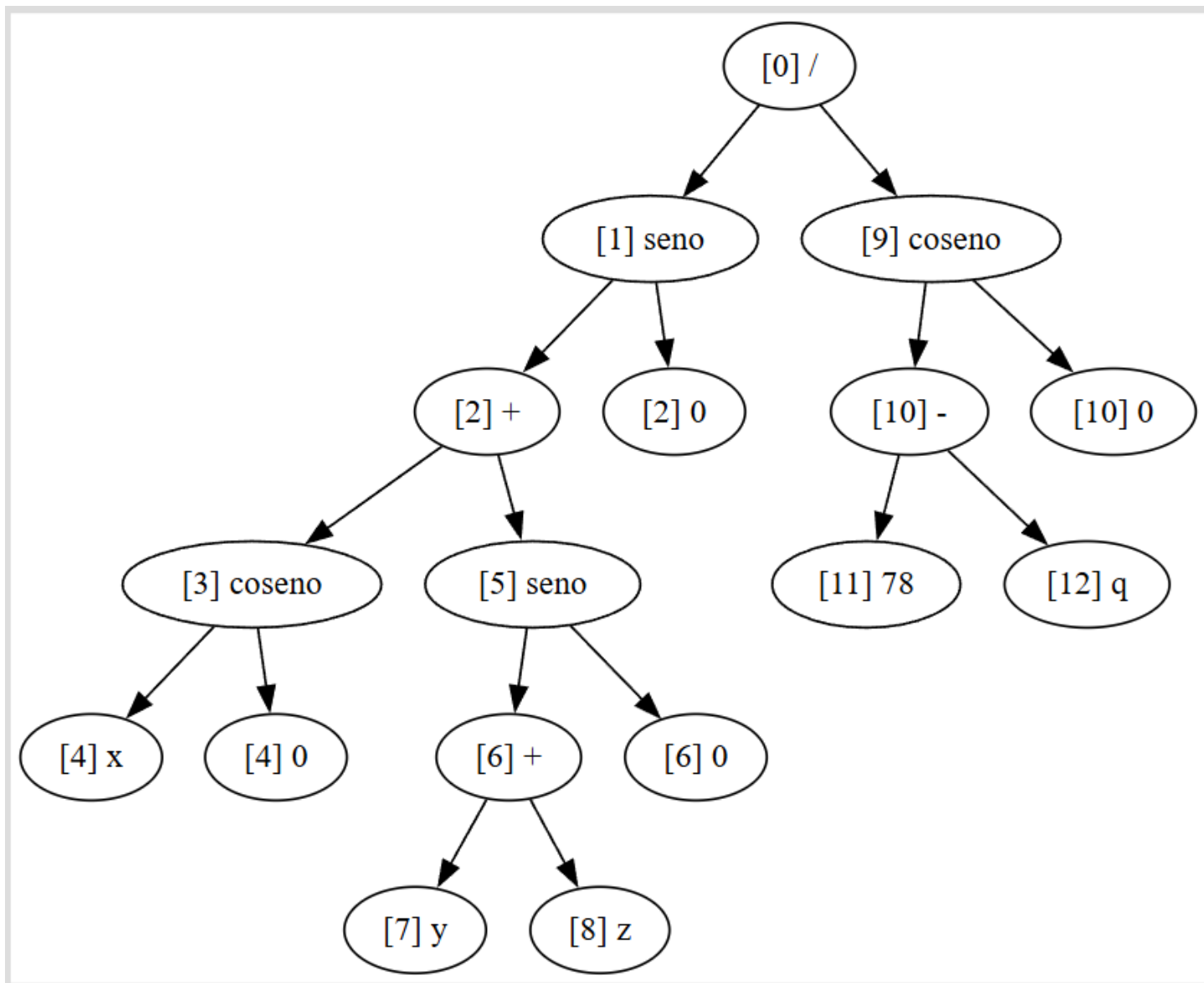


Ilustración 21: Árbol binario generado

## Fase 7. Orientado a objetos

Ahora es volver el evaluador como clases (aplicar la programación orientada a objetos). La primera clase es el Nodo.

H/Árbol/007.zip

```
namespace Ejemplo {
    internal class Nodo {
        public int ID;
        public char Operador; //+ - * / ^
        public double Numero;
        public int Variable;
        public int Funcion;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * / ^
        public Nodo(char Operador, int identifica) {
            Variable = -1;
            Numero = 0;
            this.Operador = Operador;
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero, int identifica) {
            Variable = -1;
            this.Numero = Numero;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }

        //Si el Nodo sólo tiene variable
        public Nodo(int Variable, int identifica) {
            this.Variable = Variable;
            Numero = 0;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
            ID = identifica;
        }
    }
}
```

```

}

//Si el Nodo es de una función
public Nodo(int Funcion, int identifica, bool EsFuncion) {
    Variable = -1;
    Numero = 0;
    Operador = '#';
    this.Funcion = Funcion;
    Izquierda = null;
    Derecha = null;
    ID = identifica;
}

public void Imprime() {
    if (Operador != '#')
        Console.WriteLine("\n[" + ID + "] " + Operador + "\n");
    else if (Variable != -1) {
        int Ascii = 'a' + Variable;
        char Letra = (char)Ascii;
        Console.WriteLine("\n[" + ID + "] " + Letra + "\n");
    }
    else if (Funcion != -1) {
        Console.WriteLine("\n[" + ID + "] ");
        /* Código de la función 0:seno, 1:coseno, 2:tangente,
        * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
        * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
        * 9: exponencial, 10: raíz cuadrada */
        switch (Funcion) {
            case 0: Console.WriteLine("seno \n"); break;
            case 1: Console.WriteLine("coseno \n"); break;
            case 2: Console.WriteLine("tangente \n"); break;
            case 3: Console.WriteLine("absoluto \n"); break;
            case 4: Console.WriteLine("arcoseno \n"); break;
            case 5: Console.WriteLine("arcocoseno \n"); break;
            case 6: Console.WriteLine("arcotangente \n"); break;
            case 7: Console.WriteLine("log \n"); break;
            case 8: Console.WriteLine("ceil \n"); break;
            case 9: Console.WriteLine("exp \n"); break;
            case 10: Console.WriteLine("sqrt \n"); break;
        }
    }
    else
        Console.WriteLine("\n[" + ID + "] " + Numero + "\n");
}
}
}

```

```

using System.Globalization;

/* Evaluador de expresiones usando un árbol binario */
namespace Ejemplo {
    internal class EvalArbolBin {

        /* Usado para dibujar el árbol */
        private int Identifica = 0;

        /* Las 26 variables que puede tener la expresión */
        private double[] Valores = new double[26];

        /* Árbol binario */
        private Nodo MiArbol;

        public void Analizar(string Ecuacion) {
            string Convertir = Convierte(Ecuacion);

            MiArbol = null;
            MiArbol = CreaArbol(Convertir, MiArbol);
        }

        /* Da valor a las variables que tendrá
         * la expresión algebraica */
        public void DarValorVariable(char varAlgebra, double Valor) {
            Valores[varAlgebra - 'a'] = Valor;
        }

        /* Dibuja el árbol generado */
        public void Dibujar() {
            //Probarlo en: http://viz-js.com
            Console.WriteLine("digraph testgraph{");
            DibujaArbol(MiArbol);
            Console.WriteLine("}");
        }

        /* Evalúa la expresión ya analizada */
        public double Evaluar() {
            return EvaluaArbol(MiArbol);
        }

        /* Analiza la expresión generando un árbol binario */
        private Nodo CreaArbol(string Cad, Nodo Arbol) {
            int Prntss;
            bool EsFinal;

```

```

//Detecta si es función
if (Cad[0] >= 'A' && Cad[0] <= 'J') {
    //Busca el paréntesis que cierra la función
    EsFinal = true;
    Prntss = 0;
    for (int Cont = 2; Cont < Cad.Length - 1; Cont++) {
        if (Cad[Cont] == '(') Prntss++;
        if (Cad[Cont] == ')') Prntss--;
        if (Prntss < 0) {
            EsFinal = false;
            break;
        }
    }
}

//¿Es una función en solitario?
if (EsFinal) {
    int Ascii = Cad[0] - 'A';
    Arbol = new Nodo(Ascii, Identifica++, true);
    Arbol.Derecha = new Nodo(0.0, Identifica);

    //Retira la letra A, el primer y último paréntesis
    Cad = Cad.Substring(2, Cad.Length - 3);
    Arbol.Izquierda = CreaArbol(Cad, Arbol.Izquierda);
    return Arbol;
}
}

/* Elimina paréntesis al inicio y al final si es
una expresión del tipo:
(expresión de números y operadores)
la convierte en:
expresión de números y operadores
Ejemplo:
(2^3-8+7*2-14+7/2)
Se vuelve:
2^3-8+7*2-14+7/2

Pero si encuentra algo así:
(2+4) * (5-2)
No aplica tal conversión porque no son
paréntesis que cubren toda la expresión
*/
do {
    EsFinal = false;
    if (Cad[0] == '(') {
        EsFinal = true;
        Prntss = 0;
    }
}

```



```

    for (int Cont = 1; Cont < Cad.Length - 1; Cont++) {
        if (Cad[Cont] == '(') Prntss++;
        if (Cad[Cont] == ')') Prntss--;
        if (Prntss < 0) {
            EsFinal = false;
            break;
        }
    }
}
if (EsFinal)
    Cad = Cad.Substring(1, Cad.Length - 2);
} while (EsFinal == true);

//Busca +, -
Prntss = 0;
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '+' || Cad[Cont] == '-') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca *, /
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '*' || Cad[Cont] == '/') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca ^
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if (Cad[Cont] == '^' && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);

```

```

        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont], Identifica++);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Sólo queda el número o la variable y se le crea el nodo
if (Cad[0] >= 'a' && Cad[0] <= 'z') {
    int Variable = Cad[0] - 'a';
    Arbol = new Nodo(Variable, Identifica++);
}
else {
    double Numero;
    Numero = double.Parse(Cad, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero, Identifica++);
}

return Arbol;
}

/* Dibuja el árbol binario */
private void DibujaArbol(Nodo Arbol) {
    if (Arbol != null) {
        if (Arbol.Izquierda != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Izquierda.Imprime();
            Console.WriteLine(" ");
            DibujaArbol(Arbol.Izquierda);
        }
        if (Arbol.Derecha != null) {
            Arbol.Imprime();
            Console.Write(" -> ");
            Arbol.Derecha.Imprime();
            Console.WriteLine(" ");
            DibujaArbol(Arbol.Derecha);
        }
    }
}

/* Recorrido en Post-Orden para
evaluar el árbol binario */
private double EvaluaArbol(Nodo Arbol) {
    //No hay rama hija entonces es número o variable
    if (Arbol.Izquierda == null)
        if (Arbol.Variable != -1)

```

```

        return Valores[Arbol.Variable];
    else
        return Arbol.Numero;

//Recorrido Post-Orden
double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
double ValDerecha = EvaluaArbol(Arbol.Derecha);

/* Código de la función 0:seno, 1:coseno, 2:tangente,
 * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
 * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
 * 9: exponencial, 10: raíz cuadrada */
if (Arbol.Funcion != -1) {
    switch (Arbol.Funcion) {
        case 0: return Math.Sin(ValIzquierda);
        case 1: return Math.Cos(ValIzquierda);
        case 2: return Math.Tan(ValIzquierda);
        case 3: return Math.Abs(ValIzquierda);
        case 4: return Math.Asin(ValIzquierda);
        case 5: return Math.Acos(ValIzquierda);
        case 6: return Math.Atan(ValIzquierda);
        case 7: return Math.Log(ValIzquierda);
        case 8: return Math.Ceiling(ValIzquierda);
        case 9: return Math.Exp(ValIzquierda);
        case 10: return Math.Sqrt(ValIzquierda);
    }
}

//Aplica operador
switch (Arbol.Operador) {
    case '+': return ValIzquierda + ValDerecha;
    case '-': return ValIzquierda - ValDerecha;
    case '*': return ValIzquierda * ValDerecha;
    case '/': return ValIzquierda / ValDerecha;
    case '^': return Math.Pow(ValIzquierda, ValDerecha);
}
return 0;
}

/* Convierte la expresión algebraica, escrita por el usuario,
a un formato que pueda ser interpretado por el evaluador */
private string Convierte(string ExpOrig) {
    /* Primero a minúsculas */
    string Minusculas = ExpOrig.ToLower();

    /* Cadena a evaluar */
    string Cadena = Minusculas.Replace("sen", "A");
    Cadena = Cadena.Replace("cos", "B");

```

```

Cadena = Cadena.Replace("tan", "C");
Cadena = Cadena.Replace("abs", "D");
Cadena = Cadena.Replace("asn", "E");
Cadena = Cadena.Replace("acs", "F");
Cadena = Cadena.Replace("atn", "G");
Cadena = Cadena.Replace("log", "H");
Cadena = Cadena.Replace("exp", "I");
Cadena = Cadena.Replace("sqr", "J");

    return Cadena;
}
}
}

```

Y el que lo utiliza

H/Árbol/007.zip

```

namespace Ejemplo {
//Forma el árbol binario dada una expresión matemática
//de números, variables, paréntesis, funciones y los operadores:
//Suma, resta, multiplicación, división y potencia
//Luego evalúa la expresión
    internal class Program {

        static void Main(string[] args) {
            //Ecuación operadores, variables, funciones, paréntesis
            string Ecuacion = "sen(cos(x)+sen(y+z))/cos(78-q)";

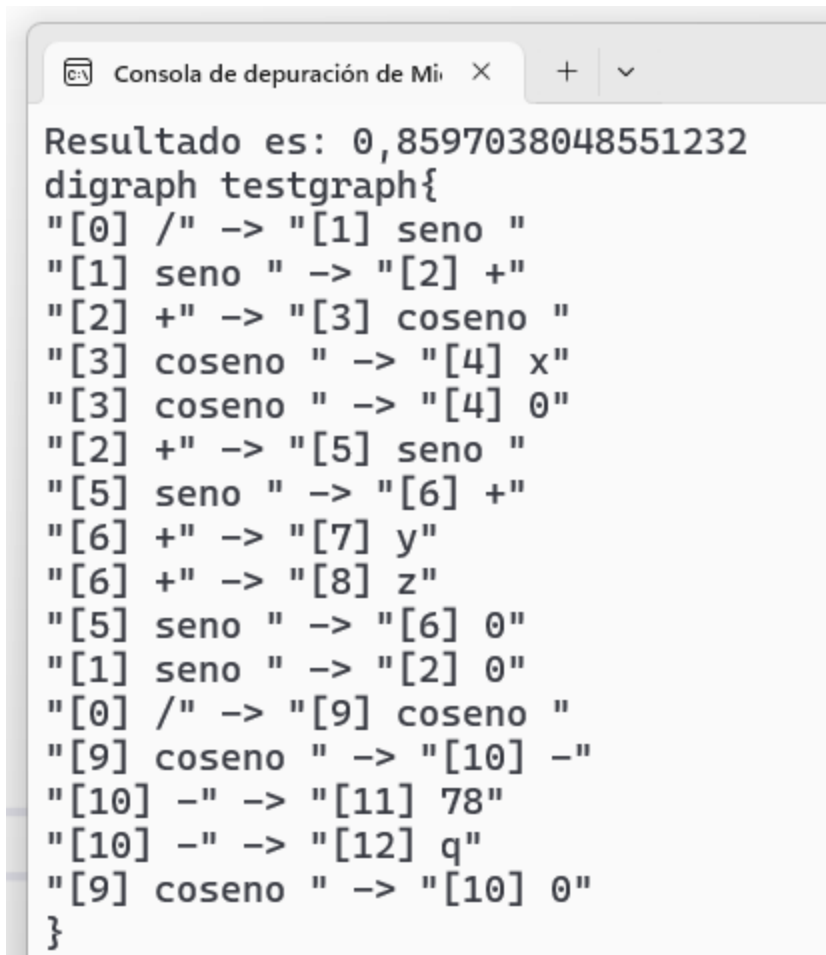
            EvalArbolBin obj = new();
            obj.Analizar(Ecuacion);
            obj.DarValorVariable('x', 120);
            obj.DarValorVariable('y', 150);
            obj.DarValorVariable('z', 45);
            obj.DarValorVariable('q', -10);

            //Evalúa la expresión
            double Resultado = obj.Evaluar();
            Console.WriteLine("Resultado es: " + Resultado);

            //Probarlo en: http://viz-js.com
            obj.Dibujar();
        }
    }
}

```

Ejemplo de ejecución:



The image shows a browser window with a developer console open. The console title is "Consola de depuración de Mi". The output of the console is as follows:

```
Resultado es: 0,8597038048551232
digraph testgraph{
"[0] /" -> "[1] seno "
"[1] seno " -> "[2] +"
"[2] +" -> "[3] coseno "
"[3] coseno " -> "[4] x"
"[3] coseno " -> "[4] 0"
"[2] +" -> "[5] seno "
"[5] seno " -> "[6] +"
"[6] +" -> "[7] y"
"[6] +" -> "[8] z"
"[5] seno " -> "[6] 0"
"[1] seno " -> "[2] 0"
"[0] /" -> "[9] coseno "
"[9] coseno " -> "[10] -"
"[10] -" -> "[11] 78"
"[10] -" -> "[12] q"
"[9] coseno " -> "[10] 0"
}
```

Ilustración 22: Ejemplo de ejecución

## Fase 8. Evaluación de sintaxis y optimización

Finalmente se le agrega la evaluación de sintaxis que es muy similar a la del evaluador 4.1. Se retira el atributo ID, porque no es necesario dibujar el árbol binario (además le resta velocidad).

H/Árbol/008.zip

```
namespace Ejemplo {
    internal class Nodo {
        public char Operador; //+ - * / ^
        public double Numero;
        public int Variable;
        public int Funcion;
        public Nodo Izquierda, Derecha;

        //Si el nodo tiene operador + - * / ^
        public Nodo(char Operador) {
            Variable = -1;
            Numero = 0;
            this.Operador = Operador;
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
        }

        //Si el Nodo sólo tiene número
        public Nodo(double Numero) {
            Variable = -1;
            this.Numero = Numero;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
        }

        //Si el Nodo sólo tiene variable
        public Nodo(int Variable) {
            this.Variable = Variable;
            Numero = 0;
            Operador = '#';
            Funcion = -1;
            Izquierda = null;
            Derecha = null;
        }

        //Si el Nodo es de una función
        public Nodo(int Funcion, bool EsFuncion) {
```

```

    Variable = -1;
    Numero = 0;
    Operador = '#';
    this.Funcion = Funcion;
    Izquierda = null;
    Derecha = null;
}
}
}

```

H/Árbol/008.zip

```

using System.Globalization;

namespace Ejemplo {
    internal class EvalArbolBin {
        /* Las 26 variables que puede tener la expresión */
        private double[] Valores = new double[26];

        /* Árbol binario */
        private Nodo MiArbol;

        public int Analizar(string ExpresionOriginal) {
            int Sintaxis = ChequeaSintaxis(ExpresionOriginal);
            if (Sintaxis == 0) {
                for (int cont = 0; cont < Valores.Length; cont++)
                    Valores[cont] = 0;
                string Convertir = Convierte(ExpresionOriginal);
                MiArbol = null;
                MiArbol = CreaArbol(Convertir, MiArbol);
            }
            return Sintaxis;
        }

        /* Da valor a las variables que tendrá
         * la expresión algebraica */
        public void DarValorVariable(char varAlgebra, double Valor) {
            Valores[varAlgebra - 'a'] = Valor;
        }

        /* Evalúa la expresión ya analizada */
        public double Evaluar() {
            return EvaluaArbol(MiArbol);
        }

        /* Retorna mensaje de error sintáctico */
    }
}

```

```
public string MensajeError(int CodigoError) {  
    string Msj = "";  
    switch (CodigoError) {  
        case 1:  
            Msj = "1. Número seguido de letra";  
            break;  
  
        case 2:  
            Msj = "2. Número seguido de paréntesis que abre";  
            break;  
  
        case 3:  
            Msj = "3. Doble punto seguido";  
            break;  
  
        case 4:  
            Msj = "4. Punto seguido de operador";  
            break;  
  
        case 5:  
            Msj = "5. Punto y sigue una letra";  
            break;  
  
        case 6:  
            Msj = "6. Punto seguido de paréntesis que abre";  
            break;  
  
        case 7:  
            Msj = "7. Punto seguido de paréntesis que cierra";  
            break;  
  
        case 8:  
            Msj = "8. Operador seguido de un punto";  
            break;  
  
        case 9:  
            Msj = "9. Dos operadores estén seguidos";  
            break;  
  
        case 10:  
            Msj = "10. Operador seguido de paréntesis que cierra";  
            break;  
  
        case 11:  
            Msj = "11. Letra seguida de número";  
            break;  
  
        case 12:
```



```
    Msj = "12. Letra seguida de punto";  
    break;  
  
case 13:  
    Msj = "13. Letra seguida de otra letra";  
    break;  
  
case 14:  
    Msj = "14. Letra seguida de paréntesis que abre";  
    break;  
  
case 15:  
    Msj = "15. Paréntesis que abre seguido de punto";  
    break;  
  
case 16:  
    Msj = "16. Paréntesis que abre y sigue operador";  
    break;  
  
case 17:  
    Msj = "17. Paréntesis que abre y luego cierra";  
    break;  
  
case 18:  
    Msj = "18. Paréntesis que cierra y sigue número";  
    break;  
  
case 19:  
    Msj = "19. Paréntesis que cierra y sigue punto";  
    break;  
  
case 20:  
    Msj = "20. Paréntesis que cierra y sigue letra";  
    break;  
  
case 21:  
    Msj = "21. Paréntesis que cierra y luego abre";  
    break;  
  
case 22:  
    Msj = "22. Inicia con operador";  
    break;  
  
case 23:  
    Msj = "23. Finaliza con operador";  
    break;  
  
case 24:
```

```

    Msj = "24. No hay correspondencia entre paréntesis";
    break;

case 25:
    Msj = "25. Paréntesis desbalanceados";
    break;

case 26:
    Msj = "26. Dos o más puntos en número real";
    break;
}
return Msj;
}

/* Analiza la expresión generando un árbol binario */
private Nodo CreaArbol(string Cad, Nodo Arbol) {
    int Prntss;
    bool EsFinal;

    //Detecta si es función
    if (Cad[0] >= 'A' && Cad[0] <= 'J') {
        //Busca el paréntesis que cierra la función
        EsFinal = true;
        Prntss = 0;
        for (int Cont = 2; Cont < Cad.Length - 1; Cont++) {
            if (Cad[Cont] == '(') Prntss++;
            if (Cad[Cont] == ')') Prntss--;
            if (Prntss < 0) {
                EsFinal = false;
                break;
            }
        }

        //¿Es una función en solitario?
        if (EsFinal) {
            int Ascii = Cad[0] - 'A';
            Arbol = new Nodo(Ascii, true);
            Arbol.Derecha = new Nodo(0.0);

            //Retira la letra A, el primer y último paréntesis
            Cad = Cad.Substring(2, Cad.Length - 3);
            Arbol.Izquierda = CreaArbol(Cad, Arbol.Izquierda);
            return Arbol;
        }
    }
}

/* Elimina paréntesis al inicio y al final si es
una expresión del tipo:

```

(expresión de números y operadores)

la convierte en:

expresión de números y operadores

Ejemplo:

$(2^3 - 8 + 7 * 2 - 14 + 7 / 2)$

Se vuelve:

$2^3 - 8 + 7 * 2 - 14 + 7 / 2$

Pero si encuentra algo así:

$(2 + 4) * (5 - 2)$

No aplica tal conversión porque no son  
paréntesis que cubren toda la expresión

```
*/
do {
    EsFinal = false;
    if (Cad[0] == '(') {
        EsFinal = true;
        Prntss = 0;
        for (int Cont = 1; Cont < Cad.Length - 1; Cont++) {
            if (Cad[Cont] == '(') Prntss++;
            if (Cad[Cont] == ')') Prntss--;
            if (Prntss < 0) {
                EsFinal = false;
                break;
            }
        }
    }
    if (EsFinal)
        Cad = Cad.Substring(1, Cad.Length - 2);
} while (EsFinal == true);

//Busca +, -
Prntss = 0;
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if ((Cad[Cont] == '+' || Cad[Cont] == '-') && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont]);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Busca *, /
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
```

```

if (Cad[Cont] == '(') Prntss++;
if (Cad[Cont] == ')') Prntss--;
if ((Cad[Cont] == '*' || Cad[Cont] == '/') && Prntss == 0) {
    string Izquierda = Cad.Substring(0, Cont);
    string Derecha = Cad.Substring(Cont + 1);
    Arbol = new Nodo(Cad[Cont]);
    Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
    Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
    return Arbol;
}
}

//Busca ^
for (int Cont = Cad.Length - 1; Cont >= 0; Cont--) {
    if (Cad[Cont] == '(') Prntss++;
    if (Cad[Cont] == ')') Prntss--;
    if (Cad[Cont] == '^' && Prntss == 0) {
        string Izquierda = Cad.Substring(0, Cont);
        string Derecha = Cad.Substring(Cont + 1);
        Arbol = new Nodo(Cad[Cont]);
        Arbol.Izquierda = CreaArbol(Izquierda, Arbol.Izquierda);
        Arbol.Derecha = CreaArbol(Derecha, Arbol.Derecha);
        return Arbol;
    }
}

//Sólo queda el número o la variable y se le crea el nodo
if (Cad[0] >= 'a' && Cad[0] <= 'z') {
    int Variable = Cad[0] - 'a';
    Arbol = new Nodo(Variable);
}
else {
    double Numero;
    Numero = double.Parse(Cad, CultureInfo.InvariantCulture);
    Arbol = new Nodo(Numero);
}

return Arbol;
}

/* Recorrido en Post-Orden para
evaluar el árbol binario */
private double EvaluaArbol(Nodo Arbol) {
    //No hay nodos hijos, entonces
    //es un número o una variable
    if (Arbol.Izquierda == null)
        if (Arbol.Variable != -1)
            return Valores[Arbol.Variable];
}

```

```

else
    return Arbol.Numero;

//Recorrido Post-Orden
double ValIzquierda = EvaluaArbol(Arbol.Izquierda);
double ValDerecha = EvaluaArbol(Arbol.Derecha);

/* Código de la función 0:seno, 1:coseno, 2:tangente,
 * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
 * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
 * 9: exponencial, 10: raíz cuadrada, -1 no es función */
switch (Arbol.Funcion) {
    case 0: return Math.Sin(ValIzquierda);
    case 1: return Math.Cos(ValIzquierda);
    case 2: return Math.Tan(ValIzquierda);
    case 3: return Math.Abs(ValIzquierda);
    case 4: return Math.Asin(ValIzquierda);
    case 5: return Math.Acos(ValIzquierda);
    case 6: return Math.Atan(ValIzquierda);
    case 7: return Math.Log(ValIzquierda);
    case 8: return Math.Exp(ValIzquierda);
    case 9: return Math.Sqrt(ValIzquierda);
}

//Aplica operador
return Arbol.Operador switch {
    '+' => ValIzquierda + ValDerecha,
    '-' => ValIzquierda - ValDerecha,
    '*' => ValIzquierda * ValDerecha,
    '/' => ValIzquierda / ValDerecha,
    '^' => Math.Pow(ValIzquierda, ValDerecha),
    _ => 0,
};
}

/* Chequea la sintaxis de la expresión algebraica */
private int ChequeaSintaxis(string ExpOrig) {
    /* Primero a minúsculas */
    string Minusculas = ExpOrig.ToLower();

    /* Sólo los caracteres válidos */
    string Valido = "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()";
    HashSet<char> Permite = new HashSet<char>(Valido);
    string ConLetrasValidas = "";
    for (int Cont = 0; Cont < Minusculas.Length; Cont++)
        if (Permite.Contains(Minusculas[Cont]))
            ConLetrasValidas += Minusculas[Cont].ToString();
}

```

```

/* Validación de sintaxis, se genera una copia
 * y allí se reemplaza las funciones por a+( */
string sbSintax = ConLetrasValidas;
sbSintax = sbSintax.Replace("sen(", "a+(");
sbSintax = sbSintax.Replace("cos(", "a+(");
sbSintax = sbSintax.Replace("tan(", "a+(");
sbSintax = sbSintax.Replace("abs(", "a+(");
sbSintax = sbSintax.Replace("asn(", "a+(");
sbSintax = sbSintax.Replace("acs(", "a+(");
sbSintax = sbSintax.Replace("atn(", "a+(");
sbSintax = sbSintax.Replace("log(", "a+(");
sbSintax = sbSintax.Replace("exp(", "a+(");
sbSintax = sbSintax.Replace("sqr(", "a+(");

for (int Cont = 0; Cont < sbSintax.Length - 1; Cont++) {
    char cA = sbSintax[Cont];
    char cB = sbSintax[Cont + 1];

    if (Char.IsDigit(cA) && Char.IsLower(cB)) return 1;
    if (Char.IsDigit(cA) && cB == '(') return 2;
    if (cA == '.' && cB == '.') return 3;
    if (cA == '.' && EsUnOperador(cB)) return 4;
    if (cA == '.' && Char.IsLower(cB)) return 5;
    if (cA == '.' && cB == '(') return 6;
    if (cA == '.' && cB == ')') return 7;
    if (EsUnOperador(cA) && cB == '.') return 8;
    if (EsUnOperador(cA) && EsUnOperador(cB)) return 9;
    if (EsUnOperador(cA) && cB == ')') return 10;
    if (Char.IsLower(cA) && Char.IsDigit(cB)) return 11;
    if (Char.IsLower(cA) && cB == '.') return 12;
    if (Char.IsLower(cA) && Char.IsLower(cB)) return 13;
    if (Char.IsLower(cA) && cB == '(') return 14;
    if (cA == '(' && cB == '.') return 15;
    if (cA == '(' && EsUnOperador(cB)) return 16;
    if (cA == '(' && cB == ')') return 17;
    if (cA == ')' && Char.IsDigit(cB)) return 18;
    if (cA == ')' && cB == '.') return 19;
    if (cA == ')' && Char.IsLower(cB)) return 20;
    if (cA == ')' && cB == '(') return 21;
}

/* Valida el inicio y fin de la expresión */
if (EsUnOperador(sbSintax[0])) return 22;
if (EsUnOperador(sbSintax[sbSintax.Length - 1])) return 23;

/* Valida balance de paréntesis */
int ParentesisAbre = 0; /* Contador de paréntesis que abre */
int ParentesisCierra = 0; /* Contador de paréntesis que cierra */

```

```

for (int Cont = 0; Cont < sbSintax.Length; Cont++) {
    switch (sbSintax[Cont]) {
        case '(': ParentesisAbre++; break;
        case ')': ParentesisCierra++; break;
    }
    if (ParentesisCierra > ParentesisAbre) return 24;
}
if (ParentesisAbre != ParentesisCierra) return 25;

/* Validar los puntos decimales de un número real */
int TotalPuntos = 0;
for (int Cont = 0; Cont < sbSintax.Length; Cont++) {
    if (EsUnOperador(sbSintax[Cont])) TotalPuntos = 0;
    if (sbSintax[Cont] == '.') TotalPuntos++;
    if (TotalPuntos > 1) return 26;
}

return 0;
}

/* Convierte la expresión algebraica, escrita por el usuario,
a un formato que pueda ser interpretado por el evaluador */
private string Convierte(string ExpOrig) {
    /* Primero a minúsculas */
    string Minusculas = ExpOrig.ToLower();

    /* Sólo los caracteres válidos */
    string Valido = "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()";
    HashSet<char> Permite = new HashSet<char>(Valido);
    string ConLetrasValidas = "";
    for (int Cont = 0; Cont < Minusculas.Length; Cont++)
        if (Permite.Contains(Minusculas[Cont]))
            ConLetrasValidas += Minusculas[Cont].ToString();

    /* Cadena a evaluar */
    string Cadena = ConLetrasValidas;
    Cadena = Cadena.Replace("sen", "A");
    Cadena = Cadena.Replace("cos", "B");
    Cadena = Cadena.Replace("tan", "C");
    Cadena = Cadena.Replace("abs", "D");
    Cadena = Cadena.Replace("asn", "E");
    Cadena = Cadena.Replace("acs", "F");
    Cadena = Cadena.Replace("atn", "G");
    Cadena = Cadena.Replace("log", "H");
    Cadena = Cadena.Replace("exp", "I");
    Cadena = Cadena.Replace("sqr", "J");

    return Cadena;
}

```

```

}

/* Retorna true si el Caracter es
 * un operador matemático */
private bool EsUnOperador(char Caracter) {
    return Caracter switch {
        '+' or '-' or '*' or '/' or '^' => true,
        _ => false,
    };
}
}
}
}

```

Y el programa que lo usa:

H/Árbol/008.zip

```

namespace Ejemplo {
    //Forma el árbol binario dada una expresión matemática
    //de números, variables, paréntesis, funciones y los operadores:
    //Suma, resta, multiplicación, división y potencia
    //Luego evalúa la expresión
    internal class Program {

        static void Main(string[] args) {
            //Ecuación operadores, variables, funciones, paréntesis
            string Ecuacion = "sen(cos(x)+sen(y*z))/cos(q^3-z)";

            EvalArbolBin obj = new();
            int Sintaxis = obj.Analizar(Ecuacion);

            if (Sintaxis == 0) {

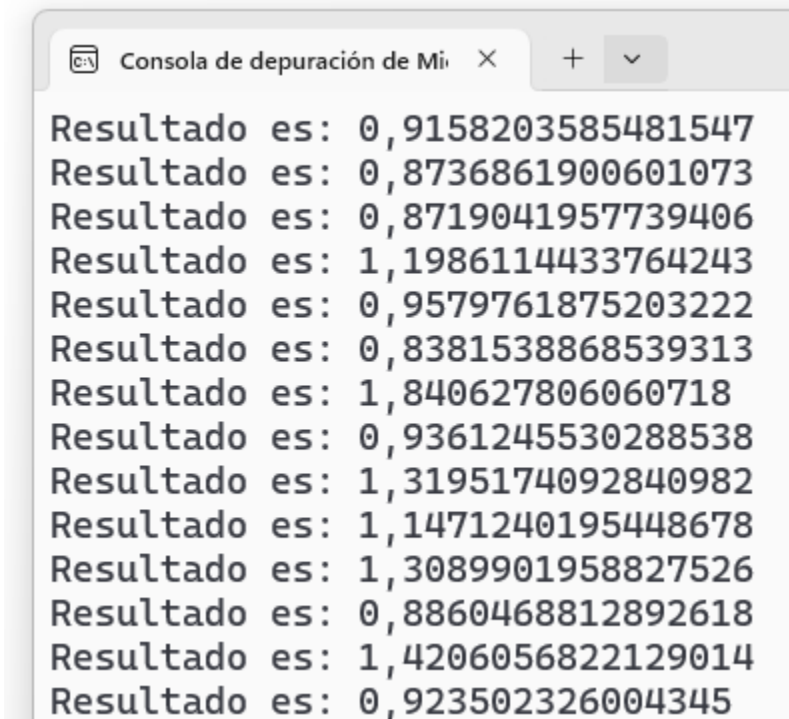
                //En un ciclo se cambian los valores de las
                //variables
                Random Azar = new();
                for (int Cont = 1; Cont <= 30; Cont++) {
                    obj.DarValorVariable('x', Azar.NextDouble());
                    obj.DarValorVariable('y', Azar.NextDouble());
                    obj.DarValorVariable('z', Azar.NextDouble());
                    obj.DarValorVariable('q', Azar.NextDouble());

                    //Evalúa la expresión
                    double Resultado = obj.Evaluar();
                    Console.WriteLine("Resultado es: " + Resultado);
                }
            }
            else {

```



```
//Hay un error de sintaxis
Console.WriteLine(obj.MensajeError(Sintaxis));
}
}
}
}
```



*Ilustración 23: Ejemplo de ejecución*

## Comparativa de desempeño entre evaluadores

¿Cuál de los dos evaluadores es más rápido? Ambos evaluadores tienen dos fases frente a una expresión matemática: el análisis y la evaluación. Si sólo requiere tan sólo un valor de la expresión (raro), entonces hay que fijarse en el tiempo de análisis, pero sí de la misma expresión se requiere generar miles de valores al cambiar los valores de las variables (algo muy común), hay que fijarse en el tiempo de la evaluación.

En H/Comparativa está la comparativa entre ambos evaluadores. Los pasos que se hacen son:

1. Generar una ecuación al azar con variable X, operadores, paréntesis y funciones.
2. Analizar y evaluar con el Evaluador 4.1
3. Analizar y evaluar con el evaluador de árbol binario.

Se varía el tamaño de la expresión matemática, desde 50 hasta 400. Se generan unas 10000 ecuaciones y cada una se evalúa unas 3000 veces. Para validar que ambos evaluadores están generando valores correctos, se comparan los resultados.

Estos son los programas usados para hacer la comparativa:

H/Comparativa.zip

```
using System.Diagnostics;

//Comparar los dos evaluadores
namespace Comparativa {
    internal class Program {

        const int ANCHO_ANALISIS = 10; // ancho de la columna "Análisis"
        const int ANCHO_EVALUA = 10; // ancho de la columna "Evalúa"

        static void Main(string[] args) {
            long TAnalisisEval4_1 = 0, TEvaluaEval4_1 = 0;
            long TAnalisisEval4 = 0, TEvaluaEval4 = 0;
            long TAnalisisArbol = 0, TEvaluaArbol = 0;

            EvalArbolBin arbol = new();
            Evaluador4 eval4 = new();
            Evaluador4_1 eval4_1 = new();
            EcuacionAzar ecu = new();

            //Toma el tiempo
            Stopwatch temporizador = new();

            //Parámetros
            int TotalEcuaciones = 700;
            int VecesEvalua = 3000;
            int Tamano = 400;

            //Valores de X al azar
            double[] valX = new double[VecesEvalua];
```

```

Random azar = new();

for (int cont = 1; cont <= TotalEcuaciones; cont++) {
    //Ecuación al azar
    string Ecuacion = ecu.Ecuacion(Tamano);

    //Valores de X al azar
    for (int xvalor = 0; xvalor < valX.Length; xvalor++) {
        valX[xvalor] = azar.NextDouble() - azar.NextDouble();
    }

    //Analiza la expresión con el árbol binario
    temporizador.Reset();
    temporizador.Start();
    arbol.Analizar(Ecuacion);
    temporizador.Stop();
    TAnalisisArbol += temporizador.ElapsedTicks;

    //Analiza la expresión con el evaluador 4.0
    temporizador.Reset();
    temporizador.Start();
    eval4.Analizar(Ecuacion);
    temporizador.Stop();
    TAnalisisEval4 += temporizador.ElapsedTicks;

    //Analiza la expresión con el evaluador 4.1
    temporizador.Reset();
    temporizador.Start();
    eval4_1.Analizar(Ecuacion);
    temporizador.Stop();
    TAnalisisEval4_1 += temporizador.ElapsedTicks;

    //Evalúa la expresión con el árbol binario
    temporizador.Reset();
    temporizador.Start();
    double ResArbol = 0;
    for (int xvalor = 0; xvalor < valX.Length; xvalor++) {
        arbol.DarValorVariable('x', valX[xvalor]);
        ResArbol += arbol.Evaluar();
    }
    temporizador.Stop();
    TEvaluaArbol += temporizador.ElapsedTicks;

    //Evalúa la expresión con el evaluador 4.0
    temporizador.Reset();
    temporizador.Start();
    double ResEval4 = 0;
    for (int xvalor = 0; xvalor < valX.Length; xvalor++) {

```

```

        eval4.DarValorVariable('x', valX[xvalor]);
        ResEval4 += eval4.Evaluar();
    }
    temporizador.Stop();
    TEvaluaEval4 += temporizador.ElapsedTicks;

    //Evalúa la expresión con el evaluador 4.0
    temporizador.Reset();
    temporizador.Start();
    double ResEval4_1 = 0;
    for (int xvalor = 0; xvalor < valX.Length; xvalor++) {
        eval4_1.DarValorVariable('x', valX[xvalor]);
        ResEval4_1 += eval4_1.Evaluar();
    }
    temporizador.Stop();
    TEvaluaEval4_1 += temporizador.ElapsedTicks;

    //Chequea si hay una diferencia entre ambos evaluadores
    if (Math.Abs(ResArbol - ResEval4) > 0.01 || Math.Abs(ResArbol -
ResEval4_1) > 0.01) {
        Console.WriteLine(Ecuacion);
        Console.WriteLine("Arbol:    " + ResArbol);
        Console.WriteLine("Eval4:    " + ResEval4);
        Console.WriteLine("Eval4_1: " + ResEval4_1);
        break;
    }
}

Console.WriteLine("Tamaño ecuación: " + Tamano);
Console.WriteLine("Total ecuaciones: " + TotalEcuaciones);
Console.WriteLine("Veces evalúa: " + VecesEvalua);
Console.WriteLine("\r\n[Eval4_1 ] Análisis: {0," + ANCHO_ANALISIS +
"} Evalúa: {1," + ANCHO_EVALUA + "}",
    TAnalisisEval4_1, TEvaluaEval4_1);
Console.WriteLine("\r\n[Eval4 ] Análisis: {0," + ANCHO_ANALISIS + "}
Evalúa: {1," + ANCHO_EVALUA + "}",
    TAnalisisEval4, TEvaluaEval4);
Console.WriteLine("\r\n[Árbol ] Análisis: {0," + ANCHO_ANALISIS + "}
Evalúa: {1," + ANCHO_EVALUA + "}",
    TAnalisisArbol, TEvaluaArbol);
Console.WriteLine("FIN");
}
}
}

```

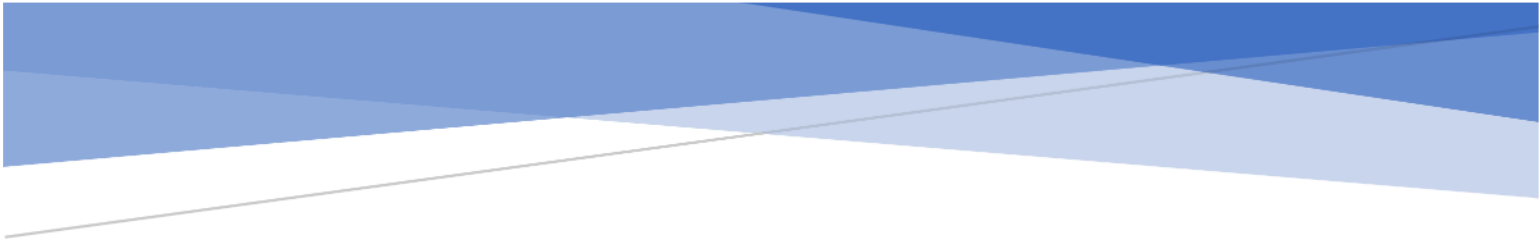
```
Consola de depuración de Mi x + v - □ X  
Tamaño ecuación: 400  
Total ecuaciones: 700  
Veces evalúa: 3000  
  
[Eval4_1 ] Análisis:      918682      Evalúa:      19975165  
[Eval4   ] Análisis:     1083422      Evalúa:      22281897  
[Árbol   ] Análisis:     1518247      Evalúa:      29222143  
FIN
```

Ilustración 24: Comparativa de desempeño

```
Consola de depuración de Mi x + v - □ X  
Tamaño ecuación: 400  
Total ecuaciones: 700  
Veces evalúa: 3000  
  
[Eval4_1 ] Análisis:      945676      Evalúa:      20560596  
[Eval4   ] Análisis:     1095550      Evalúa:      23443423  
[Árbol   ] Análisis:     1479635      Evalúa:      28849520  
FIN
```

Ilustración 25: Comparativa de desempeño

El evaluador más rápido en la fase de análisis y evaluación es el 4.1.



# C# Y .NET 10

## Parte 9. Simulaciones

2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                          |    |
|----------------------------------------------------------|----|
| Tabla de ilustraciones.....                              | 3  |
| Acerca del autor.....                                    | 4  |
| Licencia de este libro .....                             | 4  |
| Licencia del software .....                              | 4  |
| Marcas registradas .....                                 | 5  |
| En memoria de .....                                      | 6  |
| Números aleatorios .....                                 | 7  |
| Generadores de números aleatorios .....                  | 9  |
| Generador congruencial lineal.....                       | 9  |
| Blum Blum Shub .....                                     | 11 |
| Pruebas a generadores de números aleatorios.....         | 12 |
| Variables aleatorias .....                               | 18 |
| Distribuciones.....                                      | 19 |
| Distribución Normal .....                                | 19 |
| Distribución Triangular .....                            | 21 |
| Distribución Uniforme.....                               | 23 |
| Problema del viajero.....                                | 24 |
| Resolviendo un sudoku .....                              | 29 |
| El problema de las tres puertas .....                    | 33 |
| Área bajo la curva .....                                 | 36 |
| Dos robots se encuentran .....                           | 43 |
| Simulación de Inventarios .....                          | 47 |
| Simulación de Inventarios: Buscando el mejor valor ..... | 57 |
| Jugadores y apuestas .....                               | 64 |



## Tabla de ilustraciones

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Ilustración 1: Generando números aleatorios .....                                    | 8  |
| Ilustración 2: Números aleatorios .....                                              | 10 |
| Ilustración 3: Prueba del generador de números aleatorios .....                      | 16 |
| Ilustración 4: Prueba del generador de números aleatorios .....                      | 17 |
| Ilustración 5: Distribución Normal .....                                             | 19 |
| Ilustración 6: Distribución Normal .....                                             | 20 |
| Ilustración 7: Distribución Triangular .....                                         | 21 |
| Ilustración 8: Distribución Triangular .....                                         | 22 |
| Ilustración 9: Distribución Uniforme .....                                           | 23 |
| Ilustración 10: Ruta más corta .....                                                 | 28 |
| Ilustración 11: Resuelve el Sudoku .....                                             | 32 |
| Ilustración 12: Problema de las tres puertas .....                                   | 35 |
| Ilustración 13: Curva $Y = F(X)$ .....                                               | 36 |
| Ilustración 14: Los límites $X_{min}$ y $X_{max}$ para hallar el área interna .....  | 37 |
| Ilustración 15: Se dibuja un rectángulo que cubra la curva .....                     | 38 |
| Ilustración 16: Se lanzan puntos al azar al interior del rectángulo .....            | 39 |
| Ilustración 17: Área hallada .....                                                   | 42 |
| Ilustración 18: Comparando con un servicio externo .....                             | 42 |
| Ilustración 19: Dos robots moviéndose .....                                          | 46 |
| Ilustración 20: Archivo en Excel con las entradas del modelo de simulación .....     | 48 |
| Ilustración 21: Semillas de los generadores de números pseudo-aleatorios .....       | 49 |
| Ilustración 22: Comportamiento día a día de la simulación .....                      | 50 |
| Ilustración 23: Programa en C# que replica el comportamiento mostrado en Excel ..... | 50 |
| Ilustración 24: Inicia la simulación .....                                           | 63 |
| Ilustración 25: Finaliza la simulación .....                                         | 63 |
| Ilustración 26: Valor en porcentaje a apostar .....                                  | 66 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL "Lesser General Public License" [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

En memoria de

En memoria de Tinita



# Números aleatorios

.NET 10 tiene su propio generador de números pseudoaleatorios (no existe la aleatoriedad perfecta construida en un lenguaje de programación, para tal propósito, se requeriría hardware especializado que genere números aleatorios). El generador de .NET 10 requiere un valor semilla, por defecto, .NET 10 usa el reloj de la máquina. Cuando se trabaja con números aleatorios en modelos de simulación, entonces son números reales de tal manera que  $0 \leq r < 1$ , donde  $r$  es la denominación del número aleatorio

I/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Generando números al azar.

        //La semilla del generador la define el .NET
        Random azar = new();

        //Números enteros al azar
        Console.WriteLine("Enteros positivos: ");
        for (int Contador = 1; Contador <= 10; Contador++) {
            int numEntero = azar.Next();
            Console.WriteLine(numEntero + " | ");
        }

        //Números entre 0 y 1 al azar
        Console.WriteLine("\r\n\r\nReales: ");
        for (int Contador = 1; Contador <= 10; Contador++) {
            double numReal = azar.NextDouble();
            Console.WriteLine(numReal + " | ");
        }

        //Números entre 15 y 44 al azar.
        Console.WriteLine("\r\n\r\nEnteros positivos entre 15 y 44: ");
        for (int Contador = 1; Contador <= 10; Contador++) {
            //El segundo parámetro debe ser +1 del
            //rango máximo que se busca
            int numEntero = azar.Next(15, 45);
            Console.WriteLine(numEntero + " | ");
        }

        //Generando los mismos valores
        Console.WriteLine("\r\n\r\nGenerando los mismos valores");
        Console.WriteLine("al usar la misma semilla: ");
        Random AleatorioA = new(500);
        Random AleatorioB = new(500);
```

```

    for (int Contador = 1; Contador <= 20; Contador++) {
        int numA = AleatorioA.Next(55, 95);
        int numB = AleatorioB.Next(55, 95);
        Console.Write(numA + " y " + numB + " | ");
    }

    Console.WriteLine(" Final");
}
}

```

```

Enteros positivos: 1349122097 | 347272714 | 630598533 | 20
60364881 | 274356213 | 1109777197 | 745846173 | 640017399
| 1031440606 | 925710697 |

Reales: 0,27050603987428146 | 0,14320314102765974 | 0,4307
797309069017 | 0,22110153550851241 | 0,6256506375048103 |
0,9985297394636624 | 0,4262641617587756 | 0,45914040500129
36 | 0,612075621820961 | 0,32945016716730047 |

Enteros positivos entre 15 y 44: 25 | 17 | 15 | 15 | 33 |
32 | 42 | 23 | 32 | 26 |

Generando los mismos valores al usar la misma semilla: 92 y
92 | 76 y 76 | 65 y 65 | 66 y 66 | 92 y 92 | 75 y 75 | 92
y 92 | 66 y 66 | 88 y 88 | 80 y 80 | 81 y 81 | 87 y 87 |
69 y 69 | 78 y 78 | 93 y 93 | 66 y 66 | 56 y 56 | 94 y 94
| 63 y 63 | 91 y 91 | Final

```

Ilustración 1: Generando números aleatorios

## Generadores de números aleatorios

Es posible hacer un propio generador de números pseudoaleatorios, hay varios algoritmos que pueden ser útiles

([https://es.wikipedia.org/wiki/Generador\\_de\\_n%C3%BAmeros\\_pseudoaleatorios](https://es.wikipedia.org/wiki/Generador_de_n%C3%BAmeros_pseudoaleatorios)).

### Generador congruencial lineal

Hace uso de la siguiente fórmula:

$$X_1 = (A * X_0 + B) \% N \quad r_1 = X_1 / N$$

$$X_2 = (A * X_1 + B) \% N \quad r_2 = X_2 / N$$

$$X_3 = (A * X_2 + B) \% N \quad r_3 = X_3 / N$$

$$X_n = (A * X_{n-1} + B) \% N$$

A, B,  $X_0$  y N son valores llamados "semillas" que son ingresados por el usuario, a partir de ese punto el algoritmo genera los números pseudoaleatorios. Y la operación  $\%$  retorna el residuo de la división.

```

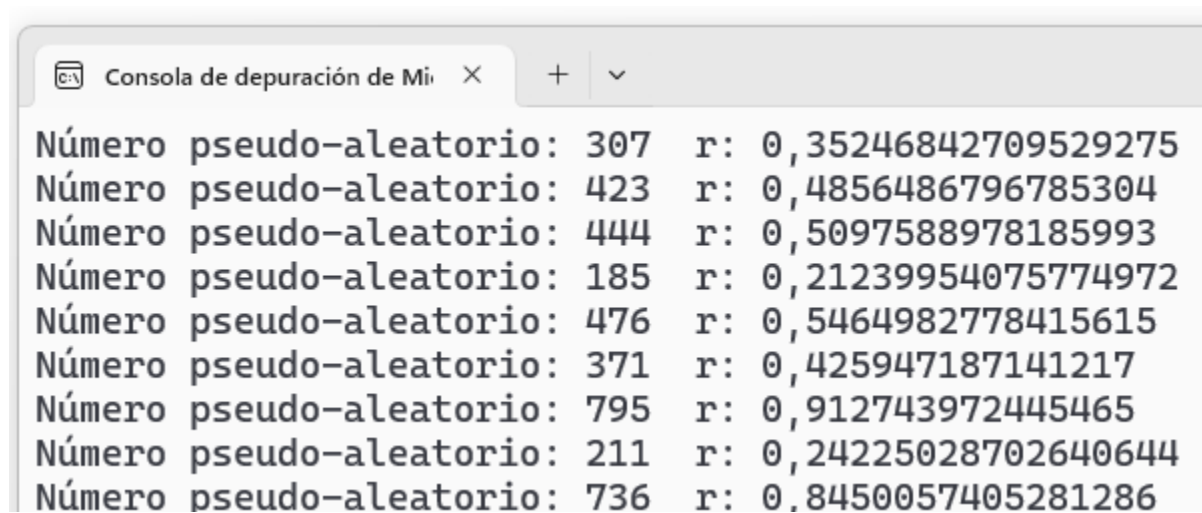
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Generador congruencial lineal
        long X0, A, B, N;

        //Valores de inicio. Se los da el usuario.
        X0 = 302;
        A = 278;
        B = 435;
        N = 871;

        for (int contador = 1; contador <= 100; contador++) {
            X0 = (A * X0 + B) % N;
            double r = (double)X0 / N;
            Console.WriteLine("Número pseudo-aleatorio: ");
            Console.WriteLine(X0 + "    r: " + r);
        }
    }
}

```



| Número pseudo-aleatorio | r                   |
|-------------------------|---------------------|
| 307                     | 0,35246842709529275 |
| 423                     | 0,4856486796785304  |
| 444                     | 0,5097588978185993  |
| 185                     | 0,21239954075774972 |
| 476                     | 0,5464982778415615  |
| 371                     | 0,425947187141217   |
| 795                     | 0,912743972445465   |
| 211                     | 0,24225028702640644 |
| 736                     | 0,8450057405281286  |

Ilustración 2: Números aleatorios



## Blum Blum Shub

[https://es.wikipedia.org/wiki/Blum\\_Blum\\_Shub](https://es.wikipedia.org/wiki/Blum_Blum_Shub)

Hace uso de la siguiente fórmula:

$$X_1 = (X_0)^2 \% M \quad r_1 = X_1/M$$

$$X_2 = (X_1)^2 \% M \quad r_2 = X_2/M$$

$$X_3 = (X_2)^2 \% M \quad r_3 = X_3/M$$

$$X_n = (X_{n-1})^2 \% M$$

Donde M es el producto de la multiplicación de dos números primos muy grandes.

I/003.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Generador Blum Blum Shub
        long X0, M;

        //Valores de inicio. Se los da el usuario.
        X0 = 3;
        M = 11 * 19;

        for (int contador = 1; contador <= 100; contador++) {
            X0 = (X0 * X0) % M;
            double r = (double)X0 / M;
            Console.WriteLine("Número pseudo-aleatorio: ");
            Console.WriteLine(X0 + "   r: " + r);
        }
    }
}
```

En el código se probó como demostración con números primos muy pequeños, eso se debe cambiar.

## Pruebas a generadores de números aleatorios

¿Cómo saber si un generador de números aleatorios es aceptable? Hay diversas pruebas estadísticas para probar. A continuación, se muestran unas cuantas:

1. Los números generados deben dar como promedio un número muy cercano a 0.5
2. La varianza debe ser cercana a 1/12
3. Los números deben estar uniformemente distribuidos en el rango 0 a 1
4. Cada número es independiente del anterior y no afecta al siguiente

I/004.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {

        List<double> Numeros = [];

        //Generador congruencial lineal
        long X0, A, B, Ndiv;

        //Valores de inicio. Se los da el usuario.
        X0 = 7302; //debe ser menor de Ndiv
        A = 5278;
        B = 3435;
        Ndiv = 16832929;

        for (int contador = 1; contador <= 7000; contador++) {
            X0 = (A * X0 + B) % Ndiv;
            double r = (double)X0 / Ndiv;
            Numeros.Add(r);
        }

        //=====
        //Prueba de promedio
        //=====
        double Acumula = 0;
        for (int cont = 0; cont < Numeros.Count; cont++) {
            Acumula += Numeros[cont];
        }
        double Promedio = (double)Acumula / Numeros.Count;
        Console.WriteLine("Promedio es: {0:0.00000}", Promedio);
        Console.WriteLine("y debe ser similar a 0,5");

        //=====
        //Prueba de varianza
        //=====
    }
}
```

```

double Sumatoria = 0;
for (int cont = 0; cont < Numeros.Count; cont++) {
    double Valor = (Numeros[cont] - Promedio);
    Sumatoria += Valor * Valor;
}
double Varianza = Sumatoria / Numeros.Count;
Console.Write("Varianza es: {0:0.00000} y debe", Varianza);
Console.WriteLine(" ser similar a {0:0.00000}", (1.0 / 12.0));

//=====
//Prueba de Uniformidad
//=====
int[] Rango = new int[10];
for (int cont = 0; cont < Numeros.Count; cont++) {
    if (Numeros[cont] < 0.1) Rango[0]++;
    else if (Numeros[cont] < 0.2) Rango[1]++;
    else if (Numeros[cont] < 0.3) Rango[2]++;
    else if (Numeros[cont] < 0.4) Rango[3]++;
    else if (Numeros[cont] < 0.5) Rango[4]++;
    else if (Numeros[cont] < 0.6) Rango[5]++;
    else if (Numeros[cont] < 0.7) Rango[6]++;
    else if (Numeros[cont] < 0.8) Rango[7]++;
    else if (Numeros[cont] < 0.9) Rango[8]++;
    else Rango[9]++;
}

Console.WriteLine("\n\nPrueba de Uniformidad");
Console.WriteLine("Números por rango");
Console.Write(" Rango\t");
Console.Write("\tFrecuencia Obtenida Fo");
Console.Write("\tFrecuencia Esperada Fe");
Console.WriteLine("\t((Fe-Fo)^2)/Fe");

double FEspera = Numeros.Count / 10;
double SumaRango = 0;
for (int cont = 0; cont < Rango.Length; cont++) {
    double Minimo = cont / 10.0;
    double Maximo = (cont + 1) / 10.0;
    double Valor = FEspera - Rango[cont];
    double Diferencia = (double)Valor * Valor / FEspera;
    Console.Write(" {0:0.0} y {1:0.0}", Minimo, Maximo);
    Console.Write("\t\t{0:#}\t\t\t\t{1:#}", Rango[cont], FEspera);
    Console.WriteLine("\t\t{0:0.00000}", Diferencia);

    SumaRango += Diferencia;
}

```

```

if (SumaRango < 16.9) {
    Console.Write("Pasa la prueba de uniformidad porque ");
    Console.WriteLine(SumaRango + " debe ser menor a 16.9");
}
else {
    Console.Write("NO pasó la prueba de uniformidad porque ");
    Console.WriteLine(SumaRango + " fue mayor o igual a 16.9");
}

//=====
//Prueba de Uniformidad de Kolmogorov-Smirnov
//=====
double[] ProbAcum = new double[10];
ProbAcum[0] = (double)Rango[0] / Numeros.Count;
for (int cont = 1; cont < Rango.Length; cont++) {
    double Valor = (double)Rango[cont] / Numeros.Count;
    ProbAcum[cont] = ProbAcum[cont - 1] + Valor;
}

Console.WriteLine("\n\nUniformidad de Kolmogorov-Smirnov");
Console.WriteLine("Probabilidad acumulada");
Console.WriteLine(" Rango\t\t\tEsperada\tObtenida\tDiferencia");
double MaxDiferencia = 0;
for (int cont = 0; cont < ProbAcum.Length; cont++) {
    double Minimo = cont / 10.0;
    double Maximo = (cont + 1) / 10.0;
    double Espera = (double)(cont + 1) / 10;
    double Diferencia = Math.Abs(ProbAcum[cont] - Espera);

    if (Diferencia > MaxDiferencia)
        MaxDiferencia = Diferencia;

    Console.Write(" {0:0.0} y {1:0.0}", Minimo, Maximo);
    Console.Write("\t\t{0:0.00}", Espera);
    Console.Write("\t\t{0:0.00}", ProbAcum[cont]);
    Console.WriteLine("\t\t{0:0.00}", Diferencia);
}

double Compara = 1.36 / Math.Sqrt(Numeros.Count);
if (MaxDiferencia < Compara) {
    Console.Write("Prueba de uniformidad aprobada porque ");
    Console.WriteLine(MaxDiferencia + " < " + Compara);
}
else {
    Console.WriteLine("NO aprobó prueba de uniformidad porque ");
    Console.WriteLine(MaxDiferencia + " >= " + Compara);
}

```

```

}

//=====
// Prueba de Independencia - Wald-Wolfowitz
//=====
Console.WriteLine("\n\nPrueba de Independencia - Wald-Wolfowitz");

//Deduce el valor de R, N1, N2, N
double N1 = 0, N2 = 0, R = 0;
int Bloque = 0;
for (int cont = 0; cont < Numeros.Count; cont++) {
    if (Numeros[cont] < Promedio) {
        if (Bloque != 1) R++;
        Bloque = 1;
        N1++;
    }
    else {
        if (Bloque != 2) R++;
        Bloque = 2;
        N2++;
    }
}

double N = Numeros.Count;

Console.WriteLine("N = " + N);
Console.WriteLine("N1 = " + N1);
Console.WriteLine("N2 = " + N2);
Console.WriteLine("R = " + R);

//Deduce la media
double Media = (2 * N1 * N2) / (N + 1);
double Variar = (Media - 1) * (Media - 2) / (N - 1);
double Z = (R - Media) / Math.Sqrt(Variar);

Console.WriteLine("Media = " + Media);
Console.WriteLine("Variación = " + Variar);
Console.WriteLine("Z = " + Z);

if (Z >= -1.96 && Z <= 1.96) {
    Console.WriteLine("Pasa la prueba de independencia");
    Console.WriteLine(" porque Z está entre -1.96 y 1.96");
}
else {
    Console.WriteLine("NO pasó la prueba de independencia porque Z");
    Console.WriteLine(" está fuera del rango de -1.96 y 1.96");
}
}

```

}

```
Consola de depuración de Mi × + v

Promedio es: 0,50444y debe ser similar a 0,5
Varianza es: 0,08361 y debe ser similar a 0,08333

Prueba de Uniformidad
Números por rango
Rango      Frecuencia Obtenida Fo  Frecuencia Esperada Fe  ((Fe-Fo)^2)/Fe
0,0 y 0,1      662      700      2,06286
0,1 y 0,2      722      700      0,69143
0,2 y 0,3      685      700      0,32143
0,3 y 0,4      719      700      0,51571
0,4 y 0,5      674      700      0,96571
0,5 y 0,6      702      700      0,00571
0,6 y 0,7      694      700      0,05143
0,7 y 0,8      693      700      0,07000
0,8 y 0,9      732      700      1,46286
0,9 y 1,0      717      700      0,41286
Pasa la prueba de uniformidad porque 6,56 debe ser menor a 16.9

Uniformidad de Kolmogorov-Smirnov
Probabilidad acumulada
Rango      Esperada      Obtenida      Diferencia
0,0 y 0,1      0,10      0,09      0,01
0,1 y 0,2      0,20      0,20      0,00
0,2 y 0,3      0,30      0,30      0,00
0,3 y 0,4      0,40      0,40      0,00
0,4 y 0,5      0,50      0,49      0,01
0,5 y 0,6      0,60      0,59      0,01
0,6 y 0,7      0,70      0,69      0,01
0,7 y 0,8      0,80      0,79      0,01
0,8 y 0,9      0,90      0,90      0,00
0,9 y 1,0      1,00      1,00      0,00
Prueba de uniformidad aprobada porque 0,007000000000000117 < 0,016255109086947755
```

Ilustración 3: Prueba del generador de números aleatorios

## Prueba de Independencia - Wald-Wolfowitz

$N = 7000$

$N_1 = 3494$

$N_2 = 3506$

$R = 3515$

Media = 3499,489787173261

Variación = 1748,2400773208224

$Z = 0,3709515765225129$

Pasa la prueba de independencia porque  $Z$  está entre  $-1.96$  y  $1.96$

*Ilustración 4: Prueba del generador de números aleatorios*

## Variables aleatorias

Una variable aleatoria es el resultado de una ecuación que hace uso de números aleatorios, se representa así:

$$V = F(r)$$

Donde  $r$  es el número aleatorio entre 0 y 1. La variable aleatoria es usada dentro de los modelos de simulación, como la parte no controlable, pero si estimable. Por ejemplo, lanzar una moneda, puede generar cara o cruz. Ese valor es la variable aleatoria y puede expresarse de esta forma en C#:

```
Random Azar = new(); //Para el número aleatorio entre 0 y 1

//La variable aleatoria
if (Azar.NextDouble() < 0.5)
    return "Cara";
else
    return "Cruz";
```

Las funciones que pueden tener las variables aleatorias pueden ser discretas como la mostrada anteriormente con solo dos valores devueltos, que es "cara" o "cruz", o también pueden ser continuas, es decir, retornan un número real. A esas funciones continuas se les conocen como distribuciones. Ejemplo:

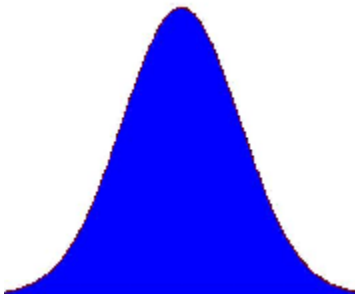
$$V = \textit{seno}(r)$$



# Distribuciones

## Distribución Normal

La distribución normal es una de las más utilizadas porque ciertos fenómenos tienen un comportamiento que sigue esta distribución:



*Ilustración 5: Distribución Normal*

Los parámetros recibidos para poder usar esta distribución son M (media) y D (Desviación)

Para generar la variable aleatoria que utilice estos parámetros se usa la ecuación:

$$V = M + D * c$$

Donde c es un valor aleatorio que se deduce con la siguiente fórmula:

$$c = \cos(2\pi r_2) * \sqrt{-2 * \ln(r_1)}$$

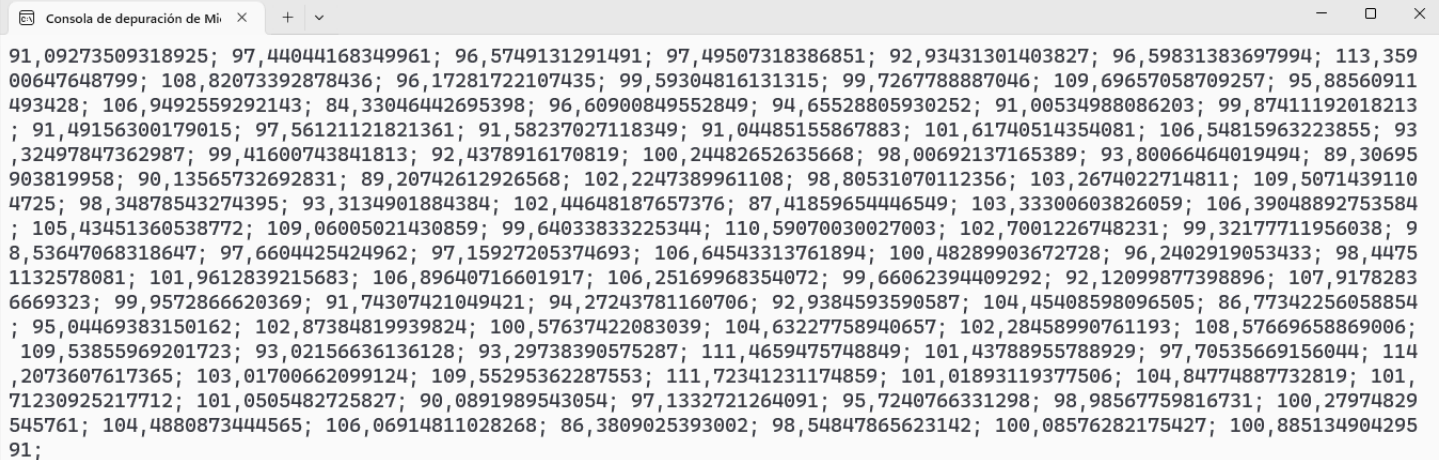
Donde  $r_1$  y  $r_2$  son números aleatorios.

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Variable aleatoria
           Distribución Normal.
           Generar una variable aleatoria con media M y desviación D
           variable = M + D * c
           donde c = cos(2*PI*r2)*raizcuadrada(-2*LogarimoNatural(r1))
           r1 y r2 son números aleatorios */
        Random Azar = new();
        double M = 100;
        double D = 7;
        for (int cont = 1; cont <= 100; cont++) {
            double r1 = Azar.NextDouble();
            double r2 = Azar.NextDouble();
            double valA = Math.Cos(2 * Math.PI * r2);
            double valB = Math.Sqrt(-2 * Math.Log(r1));
            double c = valA * valB;
            double variable = M + D * c;
            Console.Write(variable + "; ");
        }
    }
}

```



Consola de depuración de Mi

```

91,09273509318925; 97,44044168349961; 96,5749131291491; 97,49507318386851; 92,93431301403827; 96,59831383697994; 113,359
00647648799; 108,82073392878436; 96,17281722107435; 99,59304816131315; 99,7267788887046; 109,69657058709257; 95,88560911
493428; 106,9492559292143; 84,33046442695398; 96,60900849552849; 94,65528805930252; 91,00534988086203; 99,87411192018213
; 91,49156300179015; 97,56121121821361; 91,58237027118349; 91,04485155867883; 101,61740514354081; 106,54815963223855; 93
,32497847362987; 99,41600743841813; 92,4378916170819; 100,24482652635668; 98,00692137165389; 93,80066464019494; 89,30695
903819958; 90,13565732692831; 89,20742612926568; 102,2247389961108; 98,80531070112356; 103,2674022714811; 109,5071439110
4725; 98,34878543274395; 93,3134901884384; 102,44648187657376; 87,41859654446549; 103,33300603826059; 106,39048892753584
; 105,43451360538772; 109,06005021430859; 99,64033833225344; 110,59070030027003; 102,7001226748231; 99,32177711956038; 9
8,53647068318647; 97,6604425424962; 97,15927205374693; 106,64543313761894; 100,48289903672728; 96,2402919053433; 98,4475
1132578081; 101,9612839215683; 106,89640716601917; 106,25169968354072; 99,66062394409292; 92,12099877398896; 107,9178283
669323; 99,9572866620369; 91,74307421049421; 94,27243781160706; 92,9384593590587; 104,45408598096505; 86,77342256058854
; 95,04469383150162; 102,87384819939824; 100,57637422083039; 104,63227758940657; 102,28458990761193; 108,57669658869006;
109,53855969201723; 93,02156636136128; 93,29738390575287; 111,4659475748849; 101,43788955788929; 97,70535669156044; 114
,2073607617365; 103,01700662099124; 109,55295362287553; 111,72341231174859; 101,01893119377506; 104,84774887732819; 101,
71230925217712; 101,0505482725827; 90,0891989543054; 97,1332721264091; 95,7240766331298; 98,98567759816731; 100,27974829
545761; 104,4880873444565; 106,06914811028268; 86,3809025393002; 98,54847865623142; 100,08576282175427; 100,885134904295
91;

```

Ilustración 6: Distribución Normal

## Distribución Triangular



*Ilustración 7: Distribución Triangular*

Los parámetros recibidos para poder usar esta distribución son un valor mínimo (a), un valor más probable (b) y un valor máximo (c)

Para calcular el valor de la variable aleatorio está el siguiente algoritmo:

Si  $r < \frac{b-a}{c-a}$  entonces

$$V = a + \sqrt{r * (c - a) * (b - a)}$$

De lo contrario

$$V = c - \sqrt{(1 - r) * (c - a) * (c - b)}$$

Donde r es un número aleatorio.

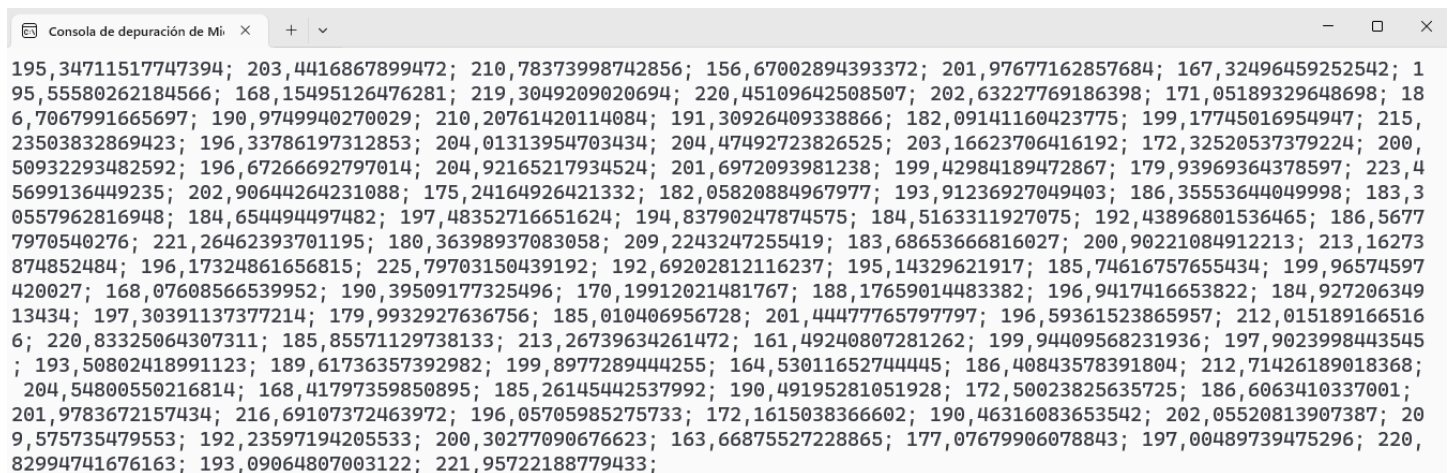
```

namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Distribución Triangular. Generar una variable aleatoria
           con valor mínimo A,
           valor más probable B y valor máximo C
           Si  $r < (B-A)/(C-A)$ 
               Variable =  $A + \text{raizcuadrada}(r * (C-A) * (B-A))$ 
           de lo contrario
               Variable =  $C - \text{raizcuadrada}((1-r) * (C-A) * (C-B))$ 
        */
        Random Azar = new();

        double A = 150;
        double B = 190;
        double C = 230;
        double variable;
        for (int cont = 1; cont <= 100; cont++) {
            double r = Azar.NextDouble();
            if (r < (B - A) / (C - A))
                variable = A + Math.Sqrt(r * (C - A) * (B - A));
            else
                variable = C - Math.Sqrt((1 - r) * (C - A) * (C - B));
            Console.Write(variable + "; ");
        }
    }
}

```



Consola de depuración de Mi

195,34711517747394; 203,4416867899472; 210,78373998742856; 156,67002894393372; 201,97677162857684; 167,32496459252542; 195,55580262184566; 168,15495126476281; 219,3049209020694; 220,45109642508507; 202,63227769186398; 171,05189329648698; 186,7067991665697; 190,9749940270029; 210,20761420114084; 191,30926409338866; 182,09141160423775; 199,17745016954947; 215,23503832869423; 196,33786197312853; 204,01313954703434; 204,47492723826525; 203,16623706416192; 172,32520537379224; 200,50932293482592; 196,67266692797014; 204,92165217934524; 201,6972093981238; 199,42984189472867; 179,93969364378597; 223,45699136449235; 202,90644264231088; 175,24164926421332; 182,05820884967977; 193,91236927049403; 186,35553644049998; 183,30557962816948; 184,654494497482; 197,48352716651624; 194,83790247874575; 184,5163311927075; 192,43896801536465; 186,56777970540276; 221,26462393701195; 180,36398937083058; 209,2243247255419; 183,68653666816027; 200,90221084912213; 213,16273874852484; 196,17324861656815; 225,79703150439192; 192,69202812116237; 195,14329621917; 185,74616757655434; 199,96574597420027; 168,07608566539952; 190,39509177325496; 170,19912021481767; 188,17659014483382; 196,9417416653822; 184,92720634913434; 197,30391137377214; 179,9932927636756; 185,010406956728; 201,44477765797797; 196,59361523865957; 212,0151891665166; 220,83325064307311; 185,85571129738133; 213,26739634261472; 161,49240807281262; 199,94409568231936; 197,9023998443545; 193,50802418991123; 189,61736357392982; 199,8977289444255; 164,53011652744445; 186,40843578391804; 212,71426189018368; 204,54800550216814; 168,41797359850895; 185,26145442537992; 190,49195281051928; 172,50023825635725; 186,6063410337001; 201,9783672157434; 216,69107372463972; 196,05705985275733; 172,1615038366602; 190,46316083653542; 202,05520813907387; 209,575735479553; 192,23597194205533; 200,30277090676623; 163,66875527228865; 177,07679906078843; 197,00489739475296; 220,82994741676163; 193,09064807003122; 221,95722188779433;

Ilustración 8: Distribución Triangular

## Distribución Uniforme

Se requiere un valor aleatorio entre A y B, y que su ocurrencia sea uniforme entre esos dos valores.

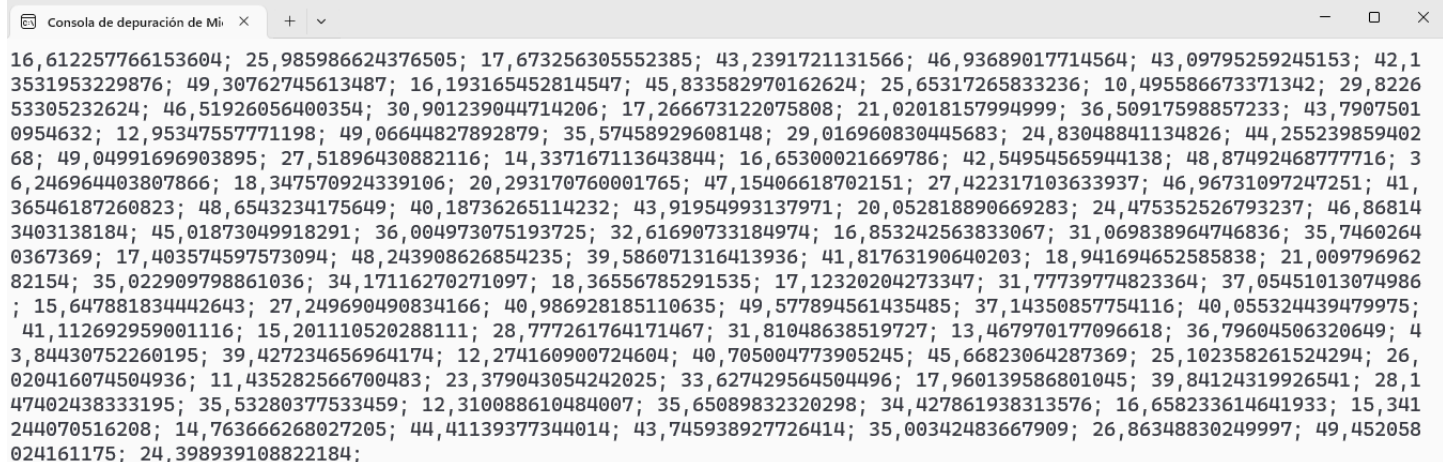
$$V = r * (B - A) + A$$

Donde r es un número aleatorio.

I/007.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Variable aleatoria
           Distribución uniforme. Generar un número
           entero entre A y B
           variable = r * (B - A) + A */
        Random Azar = new();
        double A = 10;
        double B = 50;
        for (int cont = 1; cont <= 100; cont++) {
            double r = Azar.NextDouble();
            double variable = r * (B - A) + A;
            Console.Write(variable + "; ");
        }
    }
}
```



16,612257766153604; 25,985986624376505; 17,673256305552385; 43,2391721131566; 46,93689017714564; 43,09795259245153; 42,13531953229876; 49,30762745613487; 16,193165452814547; 45,833582970162624; 25,65317265833236; 10,495586673371342; 29,822653305232624; 46,51926056400354; 30,901239044714206; 17,266673122075808; 21,02018157994999; 36,50917598857233; 43,79075010954632; 12,95347557771198; 49,06644827892879; 35,57458929608148; 29,016960830445683; 24,83048841134826; 44,25523985940268; 49,04991696903895; 27,51896430882116; 14,337167113643844; 16,65300021669786; 42,54954565944138; 48,87492468777716; 36,246964403807866; 18,347570924339106; 20,293170760001765; 47,15406618702151; 27,422317103633937; 46,96731097247251; 41,36546187260823; 48,6543234175649; 40,18736265114232; 43,91954993137971; 20,052818890669283; 24,475352526793237; 46,868143403138184; 45,01873049918291; 36,004973075193725; 32,61690733184974; 16,853242563833067; 31,069838964746836; 35,74602640367369; 17,403574597573094; 48,243908626854235; 39,586071316413936; 41,81763190640203; 18,941694652585838; 21,00979696282154; 35,022909798861036; 34,17116270271097; 18,36556785291535; 17,12320204273347; 31,77739774823364; 37,05451013074986; 15,647881834442643; 27,249690490834166; 40,986928185110635; 49,577894561435485; 37,14350857754116; 40,055324439479975; 41,112692959001116; 15,201110520288111; 28,777261764171467; 31,81048638519727; 13,467970177096618; 36,79604506320649; 43,84430752260195; 39,427234656964174; 12,274160900724604; 40,705004773905245; 45,66823064287369; 25,102358261524294; 26,020416074504936; 11,435282566700483; 23,379043054242025; 33,627429564504496; 17,960139586801045; 39,84124319926541; 28,147402438333195; 35,53280377533459; 12,310088610484007; 35,65089832320298; 34,427861938313576; 16,658233614641933; 15,341244070516208; 14,763666268027205; 44,41139377344014; 43,745938927726414; 35,00342483667909; 26,86348830249997; 49,452058024161175; 24,398939108822184;

*Ilustración 9: Distribución Uniforme*

## Problema del viajero

Hay varias ciudades por visitar, sólo puede visitarse una por vez y cada viaje de una ciudad a otra tiene un costo, inclusive ir de la ciudad C a D puede tener un valor distinto de ir de D a C. El viajero debe planificar el viaje de tal manera que visite todas las ciudades, sólo una vez por ciudad y que le salga lo más económico posible.

A continuación, una tabla de costos de viaje de ir de una ciudad (vertical) a otra ciudad (horizontal). Por ejemplo, ir de F a B cuesta \$37, o ir de K a E cuesta \$24

| Ciudad | A     | B     | C     | D     | E     | F     | G     | H     | I     | J     | K     |
|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| A      | \$ 0  | \$ 74 | \$ 13 | \$ 10 | \$ 75 | \$ 49 | \$ 21 | \$ 60 | \$ 60 | \$ 97 | \$ 89 |
| B      | \$ 43 | \$ 0  | \$ 31 | \$ 86 | \$ 41 | \$ 92 | \$ 75 | \$ 56 | \$ 72 | \$ 41 | \$ 10 |
| C      | \$ 67 | \$ 39 | \$ 0  | \$ 64 | \$ 88 | \$ 75 | \$ 82 | \$ 50 | \$ 68 | \$ 14 | \$ 38 |
| D      | \$ 12 | \$ 52 | \$ 28 | \$ 0  | \$ 16 | \$ 32 | \$ 51 | \$ 77 | \$ 85 | \$ 67 | \$ 28 |
| E      | \$ 22 | \$ 85 | \$ 19 | \$ 51 | \$ 0  | \$ 51 | \$ 19 | \$ 11 | \$ 37 | \$ 62 | \$ 63 |
| F      | \$ 10 | \$ 37 | \$ 13 | \$ 35 | \$ 12 | \$ 0  | \$ 43 | \$ 53 | \$ 49 | \$ 98 | \$ 77 |
| G      | \$ 18 | \$ 75 | \$ 62 | \$ 64 | \$ 32 | \$ 58 | \$ 0  | \$ 81 | \$ 86 | \$ 93 | \$ 46 |
| H      | \$ 22 | \$ 37 | \$ 51 | \$ 41 | \$ 12 | \$ 71 | \$ 76 | \$ 0  | \$ 92 | \$ 99 | \$ 15 |
| I      | \$ 36 | \$ 33 | \$ 71 | \$ 25 | \$ 92 | \$ 10 | \$ 93 | \$ 52 | \$ 0  | \$ 98 | \$ 61 |
| J      | \$ 38 | \$ 94 | \$ 39 | \$ 52 | \$ 34 | \$ 10 | \$ 58 | \$ 92 | \$ 87 | \$ 0  | \$ 26 |
| K      | \$ 59 | \$ 32 | \$ 30 | \$ 92 | \$ 24 | \$ 12 | \$ 51 | \$ 36 | \$ 38 | \$ 98 | \$ 0  |

Una idea de un trayecto sería este:  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow K$

El costo del viaje sería:  $\$74 + \$31 + \$64 + \$16 + \$51 + \$43 + \$81 + \$92 + \$98 + \$26 = \$576$

Si se modificara el viaje a:  $B \rightarrow A \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I \rightarrow J \rightarrow K$

El costo del viaje sería:  $\$43 + \$13 + \$64 + \$16 + \$51 + \$43 + \$81 + \$92 + \$98 + \$26 = \$527$

Una ruta más económica.

¿Cuántas rutas posibles habría? Es la función factorial del total de ciudades, en el ejemplo serían  $11!$  Rutas = 39.916.800 rutas. Y de todas esas rutas habrá una que será la más económica, pero hay que hacer cálculo de costos por cada ruta. Si fuesen unas 20 ciudades, las rutas posibles serían: 2.432.902.008.176.640.000, haciendo cálculos, y teniendo un computador que sea capaz de calcular el costo de 1000 millones de rutas por segundo, tardaría 2.432.902.008 segundos aproximadamente en evaluar todas las rutas, que equivalen a aproximadamente 77 años. Eso es muchísimo tiempo.

El método propuesto es mucho más rápido y da con una ruta con un costo bajo (no el más bajo, pero si lo suficiente). Tarda pocos segundos. ¿En qué consiste? Parte de una ruta inicial a la cual se le calcula su costo, al azar se toman dos ciudades de la ruta y se intercambian, se vuelve a calcular el nuevo costo, si es menor, entonces se deja la nueva ruta, caso contrario, se repone como estaba anteriormente. Se repite ese proceso por el tiempo deseado, y la última ruta hallada será lo mejor encontrado por el momento.

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Hay N ciudades a recorrer (0 a N-1).
           Sólo se puede visitar una ciudad por vez.
           En la tabla aparece cuanto cuesta ir de una
           ciudad (origen) a otra ciudad (destino).
           ¿Qué ruta tomar para visitar todas las
           ciudades con el mínimo costo? */
        Random Azar = new();

        int ciudad = 20; //Número de ciudades

        //Valor mínimo que tendrá ir de una ciudad a otra
        int minValor = 15;

        //Valor máximo que tendrá ir de una ciudad a otra
        int maxValor = 85;

        //Ciudad origen a Ciudad destino
        int dest1, dest2;

        //Genera valores de viaje al azar
        int[,] valorviajes = Valores(ciudad, minValor, maxValor);

        //Imprime los valores
        Imprime(valorviajes);

        //Inicia con una ruta predeterminada 0, 1, 2, 3, .... N
        int[] ruta = IniciaRuta(ciudad);

        //Deduce el costo de esa ruta predeterminada
        int costo = DeduceCosto(ruta, valorviajes);

        Console.WriteLine("\r\nRutas:");
        ImprimeRuta(ruta, costo);

        //Usando el método Monte Carlo se buscarán
        //otras rutas con menor costo
        for (int pruebas = 1; pruebas <= 700000; pruebas++) {
            double r1 = Azar.NextDouble();
            dest1 = (int)Math.Floor(r1 * ruta.Length);

            do {
                double r2 = Azar.NextDouble();
                dest2 = (int)Math.Floor(r2 * ruta.Length);
            } while (dest1 == dest2);
        }
    }
}

```

```

        } while (dest2 == dest1);

        ModificaRuta(ruta, dest1, dest2);

        int costoNuevo = DeduceCosto(ruta, valorviajes);

        if (costoNuevo < costo) {
            costo = costoNuevo;
            ImprimeRuta(ruta, costo);
        }
        else {
            //Dejar la ruta como antes
            ModificaRuta(ruta, dest1, dest2);
        }
    }
}

//Modifica la ruta de viaje
static void ModificaRuta(int[] ruta, int dest1, int dest2) {
    int tmp = ruta[dest1];
    ruta[dest1] = ruta[dest2];
    ruta[dest2] = tmp;
}

//Inicia el arreglo bidimensional de rutas
static int[] IniciaRuta(int limite) {
    int[] ruta = new int[limite];
    for (int cont = 0; cont < limite; cont++)
        ruta[cont] = cont;
    return ruta;
}

//Deduce el costo de la ruta de viaje
static int DeduceCosto(int[] ruta, int[,] costos) {
    int acum = 0;
    for (int cont = 0; cont < ruta.Length - 1; cont++)
        acum += costos[ruta[cont], ruta[cont + 1]];
    return acum;
}

//Llena de valores al azar la tabla de costos de viajes
//de una ciudad a otra
static int[,] Valores(int ciudad, int minValor, int maxValor) {
    int[,] tablero = new int[ciudad, ciudad];
    Random Azar = new();
    for (int fila = 0; fila < ciudad; fila++) {
        for (int columna = 0; columna < ciudad; columna++) {
            tablero[fila, columna] = Azar.Next(minValor, maxValor);
        }
    }
}

```



```

        }
        tablero[fila, fila] = 0;
    }
    return tablero;
}

//Imprime el tablero
static void Imprime(int[,] tablero) {
    Console.WriteLine("Tabla de costos");
    Console.Write("    ");
    for (int col = 0; col < tablero.GetLength(1); col++)
        Console.Write((char)(col + 65) + " ");
    Console.WriteLine(" ");

    for (int fila = 0; fila < tablero.GetLength(0); fila++) {
        Console.Write((char)(fila + 65) + ": ");
        for (int col = 0; col < tablero.GetLength(1); col++)
            Console.Write(tablero[fila, col] + " ");
        Console.WriteLine(" ");
    }
}

//Imprime la ruta y su costo
static void ImprimeRuta(int[] ruta, int costo) {
    for (int col = 0; col < ruta.Length; col++) {
        Console.Write((char)(ruta[col] + 65) + "->");
    }
    Console.WriteLine(" $" + costo);
}
}

```

La ejecución del programa genera una tabla de costos al azar para unas 20 ciudades, luego muestra el costo de una ruta inicial y el proceso para encontrar rutas de menor costo cada vez.

### Tabla de costos

|    | A  | B  | C  | D  | E  | F  | G  | H  | I  | J  | K  | L  | M  | N  | O  | P  | Q  | R  | S  | T  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| A: | 0  | 56 | 80 | 26 | 64 | 47 | 51 | 27 | 38 | 58 | 58 | 42 | 15 | 20 | 62 | 54 | 62 | 58 | 75 | 34 |
| B: | 36 | 0  | 20 | 58 | 59 | 47 | 18 | 65 | 23 | 33 | 36 | 46 | 25 | 33 | 58 | 32 | 45 | 41 | 21 | 44 |
| C: | 71 | 48 | 0  | 32 | 20 | 50 | 55 | 83 | 56 | 21 | 62 | 48 | 48 | 43 | 56 | 47 | 33 | 77 | 34 | 19 |
| D: | 63 | 37 | 76 | 0  | 16 | 21 | 65 | 60 | 44 | 17 | 81 | 24 | 43 | 71 | 43 | 53 | 64 | 28 | 49 | 66 |
| E: | 16 | 50 | 64 | 71 | 0  | 79 | 57 | 82 | 78 | 84 | 20 | 84 | 66 | 39 | 76 | 61 | 49 | 77 | 28 | 58 |
| F: | 60 | 61 | 65 | 61 | 24 | 0  | 27 | 57 | 68 | 25 | 61 | 48 | 81 | 82 | 20 | 56 | 82 | 17 | 34 | 53 |
| G: | 53 | 84 | 51 | 39 | 15 | 51 | 0  | 27 | 38 | 44 | 36 | 35 | 30 | 63 | 57 | 75 | 19 | 45 | 50 | 83 |
| H: | 27 | 60 | 77 | 54 | 66 | 53 | 57 | 0  | 62 | 79 | 32 | 74 | 47 | 21 | 19 | 16 | 28 | 69 | 52 | 82 |
| I: | 73 | 49 | 67 | 80 | 32 | 48 | 37 | 69 | 0  | 43 | 70 | 56 | 57 | 54 | 38 | 27 | 73 | 20 | 32 | 65 |
| J: | 55 | 64 | 28 | 73 | 41 | 28 | 72 | 49 | 41 | 0  | 59 | 38 | 74 | 47 | 26 | 21 | 17 | 77 | 53 | 71 |
| K: | 16 | 80 | 70 | 39 | 61 | 16 | 18 | 19 | 73 | 52 | 0  | 83 | 74 | 79 | 79 | 31 | 38 | 40 | 24 | 73 |
| L: | 61 | 54 | 47 | 71 | 76 | 27 | 24 | 73 | 37 | 40 | 18 | 0  | 42 | 82 | 33 | 23 | 68 | 78 | 75 | 17 |
| M: | 15 | 82 | 17 | 71 | 33 | 52 | 70 | 31 | 54 | 36 | 71 | 17 | 0  | 53 | 44 | 39 | 55 | 47 | 17 | 73 |
| N: | 16 | 65 | 73 | 21 | 30 | 45 | 25 | 15 | 80 | 82 | 39 | 81 | 74 | 0  | 73 | 19 | 82 | 65 | 56 | 16 |
| O: | 68 | 22 | 77 | 81 | 22 | 51 | 17 | 70 | 22 | 48 | 68 | 47 | 66 | 74 | 0  | 37 | 16 | 40 | 63 | 39 |
| P: | 52 | 64 | 50 | 77 | 30 | 72 | 57 | 51 | 22 | 46 | 38 | 22 | 19 | 26 | 43 | 0  | 45 | 43 | 61 | 50 |
| Q: | 41 | 27 | 48 | 84 | 29 | 26 | 78 | 81 | 32 | 52 | 40 | 44 | 47 | 64 | 41 | 27 | 0  | 81 | 46 | 19 |
| R: | 49 | 53 | 62 | 28 | 61 | 15 | 68 | 34 | 74 | 70 | 33 | 33 | 42 | 63 | 56 | 80 | 52 | 0  | 74 | 48 |
| S: | 19 | 38 | 44 | 63 | 42 | 22 | 72 | 41 | 80 | 28 | 15 | 81 | 44 | 34 | 70 | 84 | 22 | 19 | 0  | 65 |
| T: | 38 | 30 | 43 | 56 | 67 | 71 | 28 | 36 | 58 | 18 | 66 | 24 | 57 | 82 | 63 | 23 | 72 | 16 | 38 | 0  |

### Rutas:

|                                                              |       |
|--------------------------------------------------------------|-------|
| A->B->C->D->E->F->G->H->I->J->K->L->M->N->O->P->Q->R->S->T-> | \$974 |
| A->B->C->D->E->F->G->M->I->J->K->L->H->N->O->P->Q->R->S->T-> | \$968 |
| A->B->C->D->E->F->G->M->I->J->K->L->O->N->H->P->Q->R->S->T-> | \$902 |
| A->B->C->D->E->F->G->M->I->J->S->L->O->N->H->P->Q->R->K->T-> | \$861 |
| A->B->C->D->E->F->G->M->I->J->S->K->O->N->H->P->Q->R->L->T-> | \$785 |
| A->B->C->D->E->T->G->M->I->J->S->K->O->N->H->P->Q->R->L->F-> | \$775 |
| A->B->C->D->E->T->G->M->J->I->S->K->O->N->H->P->Q->R->L->F-> | \$734 |
| A->B->C->D->E->T->G->M->J->I->S->R->O->N->H->P->Q->K->L->F-> | \$724 |
| A->B->C->D->E->T->L->M->J->I->S->R->O->N->H->P->Q->K->G->F-> | \$691 |
| J->B->C->D->E->T->L->M->A->I->S->R->O->N->H->P->Q->K->G->F-> | \$675 |
| J->B->C->D->E->T->L->M->A->I->S->R->F->N->H->P->Q->K->G->O-> | \$648 |
| N->B->C->D->E->T->L->M->A->I->S->R->F->J->H->P->Q->K->G->O-> | \$626 |
| N->B->C->D->E->T->L->M->A->I->S->R->F->J->H->P->Q->O->G->K-> | \$605 |
| N->T->C->D->E->B->L->M->A->I->S->R->F->J->H->P->Q->O->G->K-> | \$593 |
| N->T->C->D->E->B->L->M->A->I->S->R->F->J->Q->P->H->O->G->K-> | \$556 |
| N->T->C->D->E->B->L->M->A->H->S->R->F->J->Q->P->I->O->G->K-> | \$555 |
| N->T->C->D->E->B->L->H->A->M->S->R->F->J->Q->P->I->O->G->K-> | \$551 |
| N->T->C->D->E->B->P->H->A->M->S->R->F->J->Q->L->I->O->G->K-> | \$547 |
| N->T->C->D->E->L->P->H->A->M->S->R->F->J->Q->B->I->O->G->K-> | \$541 |
| N->T->C->D->G->L->P->H->A->M->S->R->F->J->Q->B->I->O->E->K-> | \$530 |
| N->D->C->T->G->L->P->H->A->M->S->R->F->J->Q->B->I->O->E->K-> | \$518 |
| N->D->C->T->P->L->G->H->A->M->S->R->F->J->Q->B->I->O->E->K-> | \$477 |

Ilustración 10: Ruta más corta

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        Random Azar = new(); //con semilla 700 usaría 1429497 ciclos

        /* SUDOKU original, los ceros son los números
           que se deben hallar */
        int[][] Original = [
            [1, 0, 0, 0, 0, 7, 0, 9, 0],
            [0, 3, 0, 0, 2, 0, 0, 0, 8],
            [0, 0, 9, 6, 0, 0, 5, 0, 0],
            [0, 0, 5, 3, 0, 0, 9, 0, 0],
            [0, 1, 0, 0, 8, 0, 0, 0, 2],
            [6, 0, 0, 0, 0, 4, 0, 0, 0],
            [3, 0, 0, 0, 0, 0, 0, 0, 1],
            [0, 4, 0, 0, 0, 0, 0, 0, 7],
            [0, 0, 7, 0, 0, 0, 3, 0, 0]
        ];

        /* Mantiene el ciclo hasta que resuelva el Sudoku */
        bool Finalizar;

        /* Lleva el número de iteraciones */
        int Ciclos = 0;

        /* Copia el SUDOKU original para trabajar en esta copia */
        int[][] Copia = new int[9][];
        for (int i = 0; i < 9; i++)
            Copia[i] = new int[9];

        /* Cada cuantos ciclos borra números para destrabar */
        int DESTRUYE = 700;

        /* Ciclo que llenará el sudoku completamente */
        do {
            /* Copia el sudoku original */
            for (int Fil = 0; Fil < 9; Fil++) {
                for (int Col = 0; Col < 9; Col++)
                    if (Original[Fil][Col] != 0)
                        Copia[Fil][Col] = Original[Fil][Col];
            }

            bool numValido;
            int Fila, Columna, Numero;
```

```

do {
    /* Busca un número al azar para colocar en alguna celda */
    Fila = Azar.Next(9); /* Una posición X de 0 a 8 */
    Columna = Azar.Next(9); /* Una columna de 0 a 8 */
    Numero = Azar.Next(1, 10); /* Un número al azar de 1 a 9 */
    numValido = true;

    /* Chequea si el número no se repite ni vertical
       ni horizontalmente */
    for (int Cont = 0; Cont < 9; Cont++)
        if (Copia[Cont][Columna] == Numero ||
            Copia[Fila][Cont] == Numero)
            numValido = false;

} while (!numValido);

/* Si el número no se repite entonces valida que
   no se repita dentro del cuadro interno */
int cuadroFila = Fila - Fila % 3;
int cuadroColumna = Columna - Columna % 3;
for (int i = cuadroFila; i < cuadroFila + 3; i++) {
    for (int j = cuadroColumna; j < cuadroColumna + 3; j++) {
        if (Copia[i][j] == Numero) {
            numValido = false;
        }
    }
}

/* Si todo está bien, entonces se pone el número */
if (numValido)
    Copia[Fila][Columna] = Numero;

/* Chequea si se completó el sudoku completamente */
Finalizar = true;
for (Fila = 0; Fila < 9; Fila++)
    for (Columna = 0; Columna < 9; Columna++)
        if (Copia[Fila][Columna] == 0)
            Finalizar = false;

/* Cada cierto número de ciclos, para destrabar,
   borra la tercera parte de lo completado */
Ciclos++;
if (Ciclos % DESTRUYE == 0) {
    Console.WriteLine("Ciclos: " + Ciclos);
    for (Fila = 0; Fila < 9; Fila++)
        for (Columna = 0; Columna < 9; Columna++)
            if (Azar.NextDouble() < 0.34)
                Copia[Fila][Columna] = 0;
}

```

```

    }
} while (!Finalizar);

//Imprime el sudoku original
Console.WriteLine("Sudoku Original");
for (int Fila = 0; Fila < 9; Fila++) {
    for (int Columna = 0; Columna < 9; Columna++)
        if (Original[Fila][Columna] == 0)
            Console.Write("_ ");
        else
            Console.Write(Original[Fila][Columna] + " ");
    Console.WriteLine(" ");
}

//Imprime el sudoku resuelto
Console.WriteLine("\r\nCiclos totales usados: " + Ciclos);
Console.WriteLine("Sudoku Resuelto");
for (int Fila = 0; Fila < 9; Fila++) {
    for (int Columna = 0; Columna < 9; Columna++)
        Console.Write(Copia[Fila][Columna] + " ");
    Console.WriteLine(" ");
}
}
}

```

```
Consola de depuración de Mi X + v

Sudoku Original
1 _ _ _ 7 _ 9 _
_ 3 _ _ 2 _ _ 8
_ _ 9 6 _ _ 5 _
_ _ 5 3 _ _ 9 _
_ 1 _ _ 8 _ _ 2
6 _ _ _ 4 _ _ _
3 _ _ _ _ _ 1 _
_ 4 _ _ _ _ _ 7
_ _ 7 _ _ _ 3 _

Ciclos totales usados: 6035430
Sudoku Resuelto
1 6 2 8 5 7 4 9 3
5 3 4 1 2 9 6 7 8
7 8 9 6 4 3 5 2 1
4 7 5 3 1 2 9 8 6
9 1 3 5 8 6 7 4 2
6 2 8 7 9 4 1 3 5
3 5 6 4 7 8 2 1 9
2 4 1 9 3 5 8 6 7
8 9 7 2 6 1 3 5 4
```

Ilustración 11: Resuelve el Sudoku

## El problema de las tres puertas

El problema de las tres puertas, también conocido como el problema de Monty Hall, es un acertijo matemático de probabilidad que se presenta en diferentes variantes. En una de las variantes, se tienen tres puertas etiquetadas como "coche", "cabra 1" y "cabra 2". Detrás de una de las puertas se encuentra un coche, mientras que detrás de las otras dos hay cabras. El objetivo del acertijo es determinar qué puerta contiene el coche, utilizando solo una elección de puerta y sin abrir ninguna de las puertas.

La mecánica del acertijo es la siguiente: el concursante debe elegir una puerta entre tres (todas cerradas); el premio consiste en llevarse lo que se encuentra detrás de la elegida. Se sabe con certeza que tras una de ellas se oculta un automóvil, y tras las otras dos hay cabras. Una vez que el concursante haya elegido una puerta y comunicado su elección a los presentes, el presentador, que sabe lo que hay detrás de cada puerta, abrirá una de las otras dos, en la que habrá una cabra. A continuación, le da la opción al concursante de cambiar, si lo desea, de puerta (tiene dos opciones). ¿Debe el concursante mantener su elección original o escoger la otra puerta? ¿Hay alguna diferencia?

La respuesta correcta es que el concursante debe cambiar de puerta. Si el concursante cambia de puerta, la probabilidad de ganar el coche es de  $2/3$ , mientras que, si mantiene su elección original, la probabilidad de ganar el coche es de  $1/3$ .

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Generador de números pseudo-aleatorios */
        Random Azar = new();

        /* Contador de éxitos si el jugador
           cambia su decisión inicial */
        int CambiaDecision = 0;

        /* Contador de éxitos si el jugador
           insiste en mantener su decisión inicial */
        int InsisteNOCambiar = 0;

        for (int prueba = 1; prueba <= 1000000; prueba++) {
            /* Selecciona una caja al azar que será la ganadora */
            int cajaGanadora = Azar.Next(3);

            /* El jugador selecciona una caja al azar */
            int cajaJugador = Azar.Next(3);

            /* Si el jugador escogió por casualidad la
               caja ganadora, el dueño escoge al azar
               alguna de las dos cajas restantes */
            int cajaDueno;
            if (cajaGanadora == cajaJugador)
                do {
                    cajaDueno = Azar.Next(3);
                } while (cajaDueno == cajaGanadora);
            else
                /* El dueño sólo puede escoger la caja
                   que no tiene premio */
                do {
                    cajaDueno = Azar.Next(3);
                } while (cajaDueno == cajaGanadora ||
                    cajaDueno == cajaJugador);

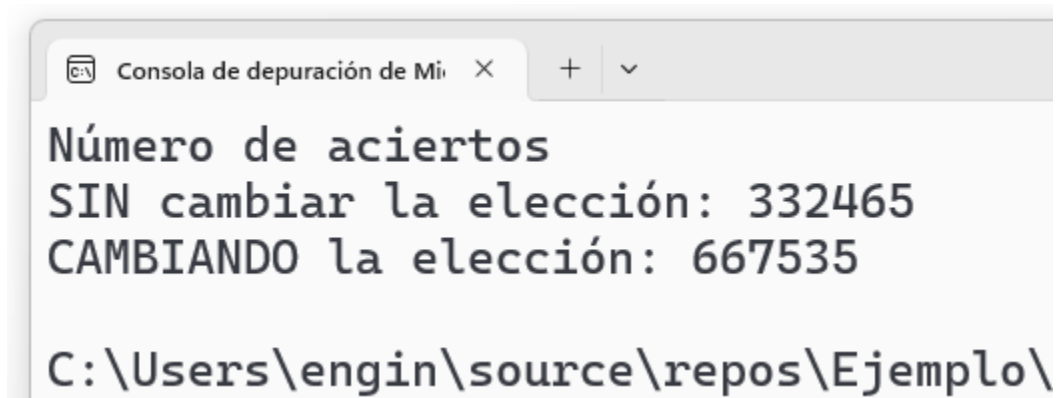
            /* El jugador NO cambia su elección */
            if (cajaGanadora == cajaJugador)
                InsisteNOCambiar++;

            /* El jugador SI cambia su elección */
            int nuevaCaja;
            do {
                nuevaCaja = Azar.Next(3);
            } while (nuevaCaja == cajaDueno || nuevaCaja == cajaJugador);
        }
    }
}

```



```
        if (nuevaCaja == cajaGanadora) CambiaDecision++;  
    }  
  
    Console.WriteLine("Número de aciertos");  
    Console.WriteLine("SIN cambiar la elección: " + InsisteNOCambiar);  
    Console.WriteLine("CAMBIANDO la elección: " + CambiaDecision);  
}  
}
```



*Ilustración 12: Problema de las tres puertas*

# Área bajo la curva

El área bajo la curva es un concepto matemático que se utiliza para calcular el área de una región limitada por una curva y el eje x en un intervalo cerrado. Para encontrar el área bajo una curva, se utilizan integrales definidas. En general, el proceso para encontrar el área bajo una curva implica los siguientes pasos:

1. Formar una integral definida con la ecuación  $Y=F(X)$  dada.
2. Obtener la integral de la función.
3. Se evalúan los límites superior e inferior en la expresión integrada.
4. Se obtiene la diferencia de los valores obtenidos del límite superior y límite inferior.

¿Pero en el caso que la integral definida sea muy compleja de resolver? En ese caso, se hace uso del siguiente método

Paso 1: Tiene la ecuación  $Y=F(X)$

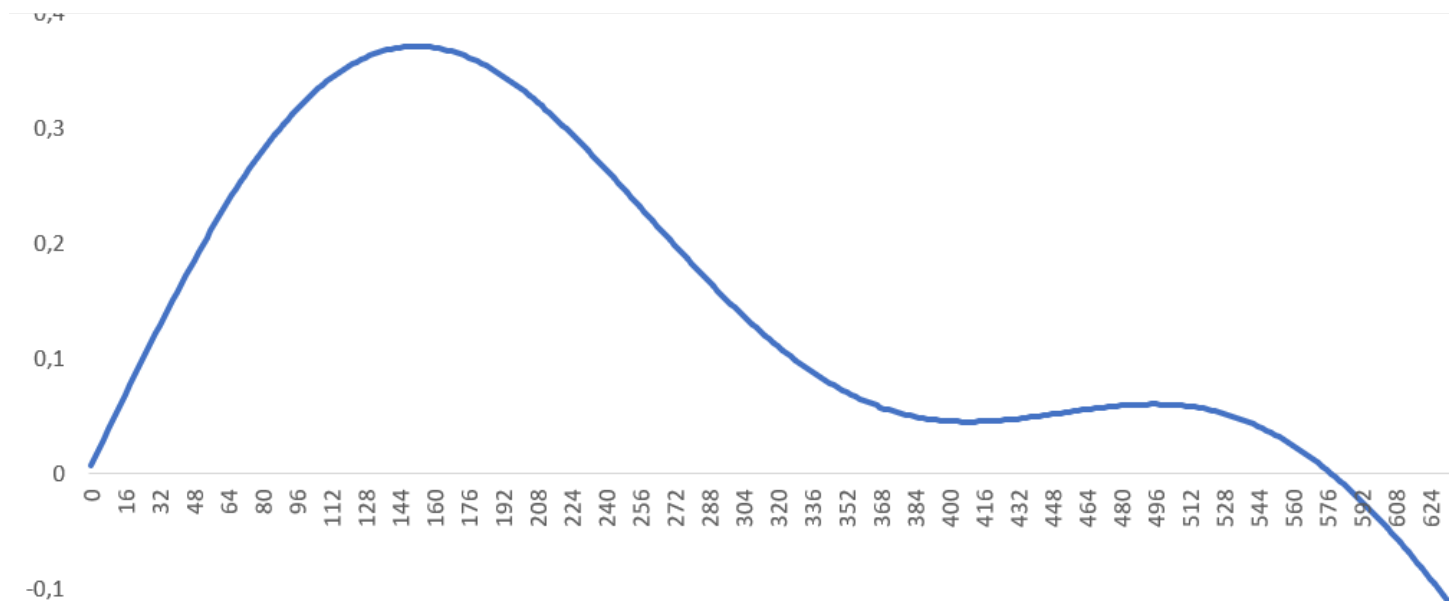


Ilustración 13: Curva  $Y = F(X)$

Paso 2: Se dibujan los límites X mínimo y X máximo para calcular el área, con eso se tiene el tamaño de la base

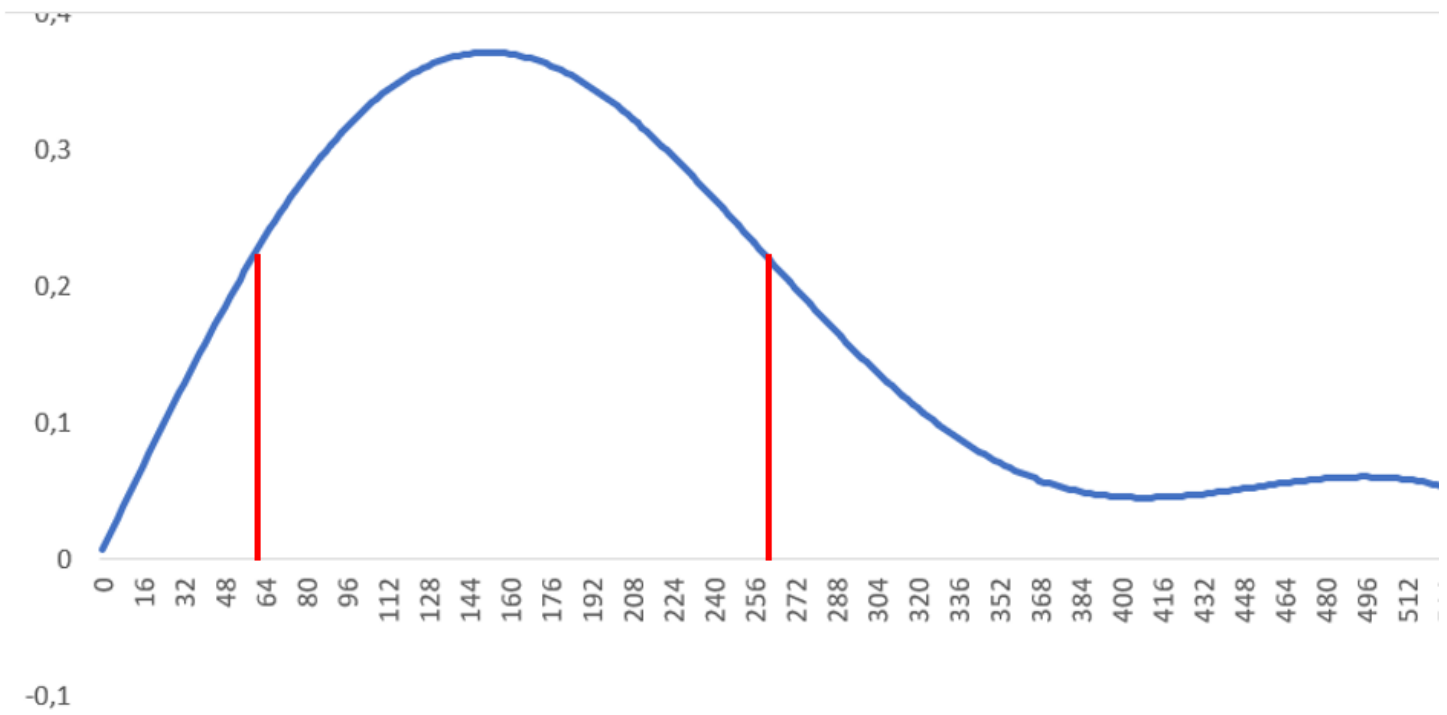


Ilustración 14: Los límites  $X_{min}$  y  $X_{max}$  para hallar el área interna

Paso 3: Se dibuja un rectángulo que contenga la curva en su interior. Luego ya se tiene el valor del tamaño de la base y la altura del rectángulo (que sería el mayor valor de Y entre X mínimo y Xmáximo)

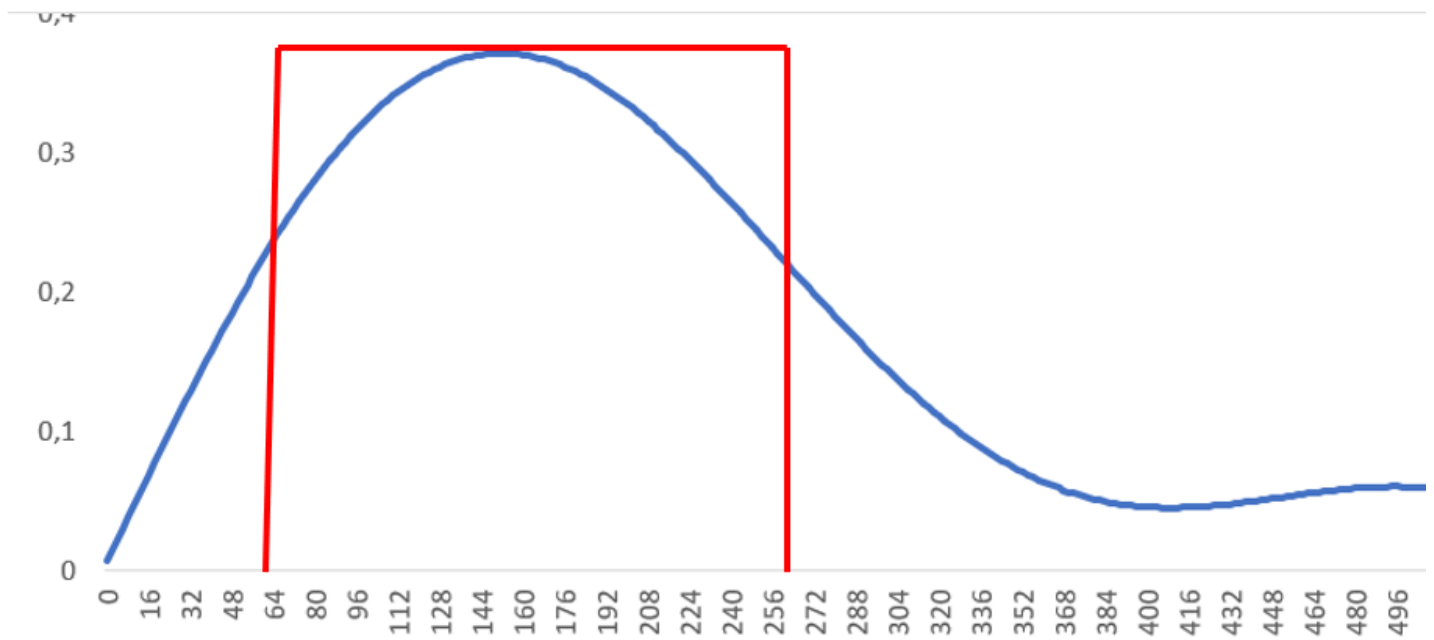


Ilustración 15: Se dibuja un rectángulo que cubra la curva

Paso 4: Se lanzan muchos puntos al azar al interior del rectángulo

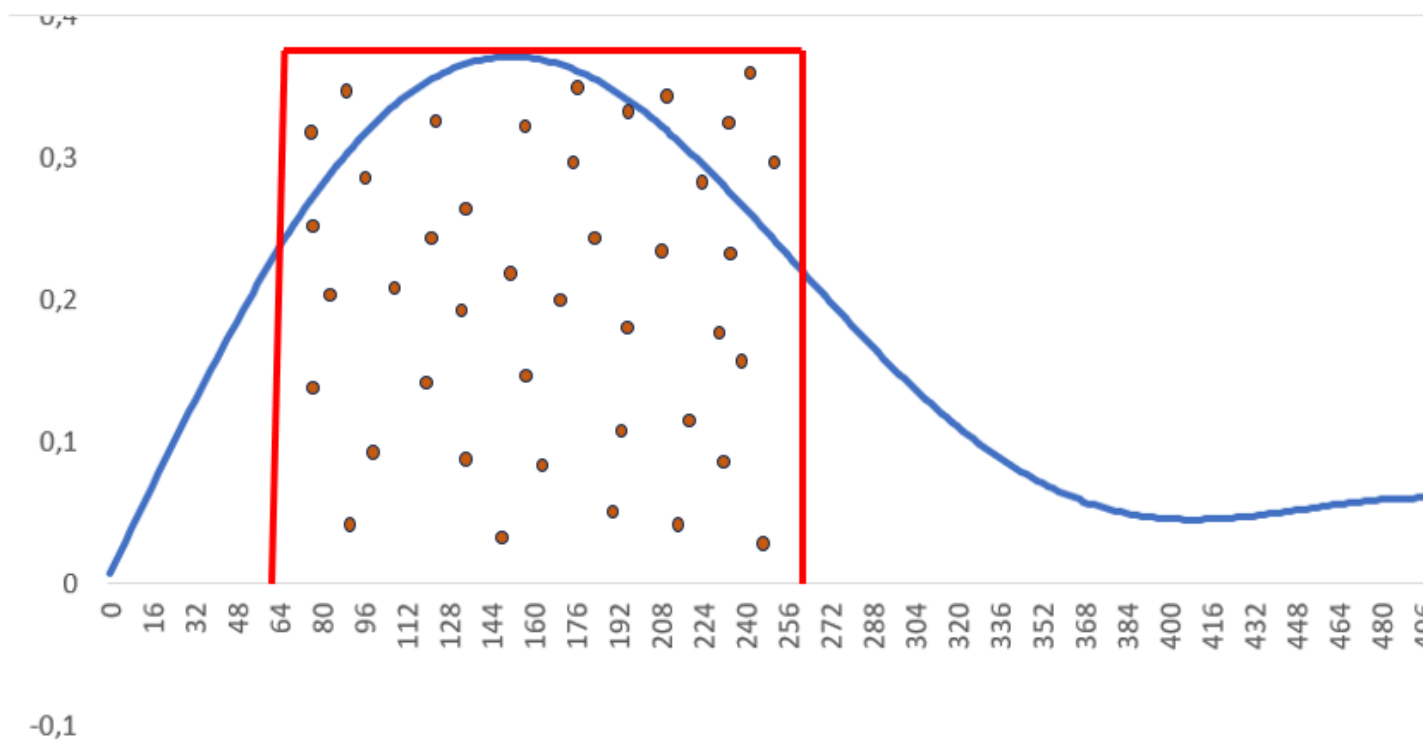


Ilustración 16: Se lanzan puntos al azar al interior del rectángulo

Paso 5: Se calcula el área bajo la curva con la siguiente ecuación:

$$\text{Área bajo la curva} = \text{Área del rectángulo} * \frac{\text{Total puntos al interior de la curva}}{\text{Total puntos lanzados}}$$

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Área bajo la curva

        //Mínimo valor en X, máximo valor en X
        double MinValX = -5;
        double MaxValX = 5;

        //Validar que no hayan puntos de corte
        if (HayPuntosCorte(MinValX, MaxValX)) {
            Console.Write("Hay puntos de corte, ");
            Console.WriteLine("no es válido para hallar el área");
        }
        else {
            double Ymax = MaxMinY(MinValX, MaxValX);
            double Area = AreaC(MinValX, MaxValX, Ymax);
            Console.WriteLine("Área es: " + Area);
        }
    }

    //Retorna true si hay puntos de corte entre los valores
    //de X mínimo y máximo dados
    static public bool HayPuntosCorte(double MinX, double MaxX) {
        int Positivos = 0;
        int Negativos = 0;
        for (double X = MinX; X <= MaxX; X += 0.001) {
            double Y = Ecuacion(X);
            if (Y > 0) Positivos++;
            if (Y < 0) Negativos++;
        }
        if (Positivos != 0 && Negativos != 0) return true;
        return false;
    }

    //Retorna el mínimo valor de Y (área está por debajo del eje X)
    //o el máximo valor de Y (área está por encima del eje X)
    static public double MaxMinY(double MinX, double MaxX) {
        double MaximoY = double.MinValue;
        double MinimoY = double.MaxValue;
        bool Orienta = false;
        for (double X = MinX; X <= MaxX; X += 0.001) {
            double Y = Ecuacion(X);
            if (Y > 0) Orienta = true;
            if (Y > MaximoY) MaximoY = Y;
            if (Y < MinimoY) MinimoY = Y;
        }
    }
}

```

```

    }
    if (Orienta) return MaximoY;
    return MinimoY;
}

//Calcula el área bajo la curva usando el método Monte Carlo.
//Siguiendo las directrices matemáticas,
//el área será positiva si la curva está por encima del eje X,
//el área sera negativa si la curva está por debajo del eje X
static public double AreaC(double MinX, double MaxX, double Ymax) {
    Random Azar = new();
    int PuntosDentro = 0;
    int PuntosTotal = 7000000;
    for (int puntos = 1; puntos <= PuntosTotal; puntos++) {
        double Xazar = Azar.NextDouble() * (MaxX - MinX) + MinX;
        double Yazar = Azar.NextDouble() * Ymax;
        double Y = Ecuacion(Xazar);
        if (Yazar <= Y && Ymax > 0) PuntosDentro++;
        if (Yazar >= Y && Ymax < 0) PuntosDentro++;
    }
    double Valor = (double)PuntosDentro / PuntosTotal;
    double AreaTotal = (MaxX - MinX) * Ymax * Valor;
    return AreaTotal;
}

static public double Ecuacion(double X) {
    double Y = -1 * Math.Abs(Math.Cos(X) - Math.Sin(X) * X);
    return Y;
}
}

```

Área es: -18,23635566437144

Ilustración 17: Área hallada

DE LOS CREADORES DE WOLFRAM LANGUAGE Y MATHEMATICA



integrate Y = -1 \* abs(cos(X) - sin(X) \* X) from X= -5 to 5

LENGUAJE NATURAL

ENTRADA MATEMÁTICA

TECLADO EXTENDIDO

EJEMPLOS

CARGAR

ALEATORIO

Integral definida

Más dígitos

Solución paso a paso

$$\int_{-5}^5 -|\cos(X) - \sin(X) X| dX =$$

$$-2 \left( \cos\left(\text{la raíz de } \cos(x) - x \sin(x) \text{ cercana a } x = -3,42562\right) \right.$$

$$\left. \cos\left(\text{la raíz de } \cos(x) - x \sin(x) \text{ cercana a } x = -0,860334\right) \right.$$

$$\left. \cos\left(\text{la raíz de } \cos(x) - x \sin(x) \text{ cercana a } x = 0,860334\right) \right.$$

$$\left. \cos\left(\text{la raíz de } \cos(x) - x \sin(x) \text{ cercana a } x = 3,42562\right) \right.$$

$$\left. \cos(5) \right) \approx -18,234$$

Ilustración 18: Comparando con un servicio externo



## Dos robots se encuentran

En un arreglo bidimensional se ubican dos robots, cada uno en una casilla seleccionada al azar. No tienen sensores visuales, luego cada robot no sabe dónde está el otro robot. La simulación es averiguar en cuál escenario es más probable que haya un encuentro de los dos robots:

1. Un robot quieto mientras el otro se mueve al azar por el tablero.
2. Los dos robots moviéndose al tiempo al azar por el tablero.

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        /* Encuentro de dos robots sin sensor visual
           en un tablero.
           ¿Cuál es la mejor estrategia?
           1. Un robot quieto y el otro moviéndose.
           2. Los dos robots moviéndose.
        */

        Random Azar = new();

        //Tamaño del tablero
        int Filas = 50;
        int Columnas = 50;

        int Estrategia1 = 0;
        int Estrategia2 = 0;
        for (int Pruebas = 1; Pruebas <= 500; Pruebas++) {
            //Fila, Columna => Robot A
            int FilaRobotA = Azar.Next(Filas);
            int ColumnaRobotA = Azar.Next(Columnas);

            //Fila, Columna => Robot B
            int FilaRobotB, ColumnaRobotB;
            do {
                FilaRobotB = Azar.Next(Filas);
                ColumnaRobotB = Azar.Next(Columnas);
            } while (FilaRobotA == FilaRobotB || ColumnaRobotA == ColumnaRobotB);

            Estrategia1 += Program.Estrategia1(Azar, FilaRobotA, ColumnaRobotA, FilaRobotB, ColumnaRobotB, Filas, Columnas);
        }
    }
}
```

```

        Estrategia2 += Program.Estrategia2(Azar, FilaRobotA, ColumnaRobotA, FilaRobotB, ColumnaRobotB, Filas, Columnas);
    }

    Console.Write("Uno quieto y el otro moviéndose");
    Console.WriteLine(". Movimientos: " + Estrategia1);
    Console.Write("Ambos robots se están moviendo.");
    Console.WriteLine("  Movimientos: " + Estrategia2);
}

static int Estrategia1(Random Azar, int FilaRobotA, int ColumnaRobotA, int FilaRobotB, int ColumnaRobotB,
    int Filas, int Columnas) {

    //Estrategia 1. Un robot quieto (B) y el otro moviéndose (A)
    int TotalMovimientos = 0;
    do {
        TotalMovimientos++;
        int FilaMueve, ColumnaMueve;
        do {
            FilaMueve = Azar.Next(-1, 2);
            ColumnaMueve = Azar.Next(-1, 2);
        } while (FilaRobotA + FilaMueve < 0 || FilaRobotA + FilaMueve >= Filas ||
            ColumnaRobotA + ColumnaMueve < 0 || ColumnaRobotA + ColumnaMueve >= Columnas ||
            (FilaMueve == 0 && ColumnaMueve == 0));
        FilaRobotA += FilaMueve;
        ColumnaRobotA += ColumnaMueve;
    } while (FilaRobotA != FilaRobotB || ColumnaRobotA != ColumnaRobotB);
    return TotalMovimientos;
}

static int Estrategia2(Random Azar, int FilaRobotA, int ColumnaRobotA, int FilaRobotB, int ColumnaRobotB,
    int Filas, int Columnas) {
    //Estrategia 2. Los dos robots moviéndose
    int TotalMovimientos = 0;
    do {
        TotalMovimientos++;

```

```
//Mueve Robot A
int RobotA_FilaMueve, RobotA_ColumnaMueve;
do {
    RobotA_FilaMueve = Azar.Next(-1, 2);
    RobotA_ColumnaMueve = Azar.Next(-1, 2);
} while (FilaRobotA + RobotA_FilaMueve < 0 || FilaRobotA + RobotA_FilaMueve >= Filas ||
        ColumnaRobotA + RobotA_ColumnaMueve < 0 || ColumnaRobotA + RobotA_ColumnaMueve >= Columnas ||
        (RobotA_FilaMueve == 0 && RobotA_ColumnaMueve == 0));
FilaRobotA += RobotA_FilaMueve;
ColumnaRobotA += RobotA_ColumnaMueve;

//Mueve RobotB
int RobotB_FilaMueve, RobotB_ColumnaMueve;
do {
    RobotB_FilaMueve = Azar.Next(-1, 2);
    RobotB_ColumnaMueve = Azar.Next(-1, 2);
} while (FilaRobotB + RobotB_FilaMueve < 0 || FilaRobotB + RobotB_FilaMueve >= Filas ||
        ColumnaRobotB + RobotB_ColumnaMueve < 0 || ColumnaRobotB + RobotB_ColumnaMueve >= Columnas ||
        (RobotB_FilaMueve == 0 && RobotB_ColumnaMueve == 0));
FilaRobotB += RobotB_FilaMueve;
ColumnaRobotB += RobotB_ColumnaMueve;

} while (FilaRobotA != FilaRobotB || ColumnaRobotA != ColumnaRobotB);
return TotalMovimientos;
}
}
```

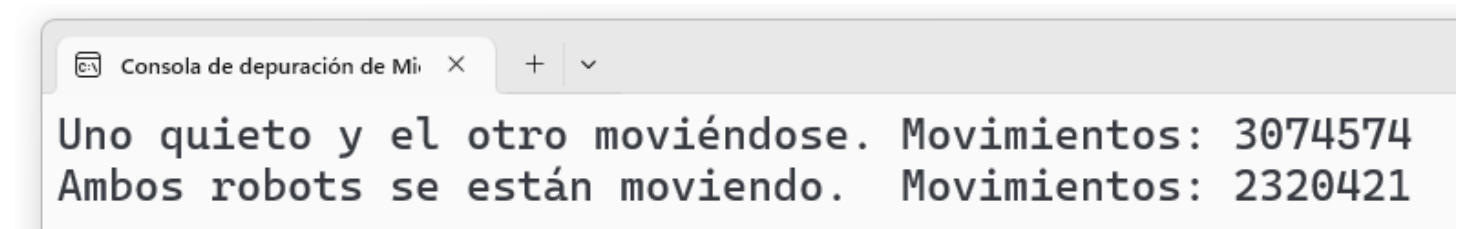


Ilustración 19: Dos robots moviéndose

# Simulación de Inventarios

Tener un inventario genera costos. Luego la meta es disminuirlos.

Para simular el comportamiento de un inventario, se empieza por algo sencillo: es el inventario de un solo producto en particular.

Un inventario tiene lo siguiente:

1. Cada unidad del producto en el inventario genera un costo diario de inventario. Por ejemplo: si tiene almacenadas 400 unidades y cada unidad genera un costo de \$30, entonces, el costo de ese día es de  $400 * \$30 = \$12000$ . Nota: Ese costo se genera al final del día con las unidades que quedan en el inventario.
2. Una demanda  $D$  variable diaria de unidades de ese producto que va disminuyendo el número de productos del inventario. Por ejemplo: hay 700 unidades y ese día la demanda fue de 80 unidades, luego al final del día habría  $700 - 80 = 620$  unidades.
3. Si la demanda de ese día supera lo que había en el inventario, se entrega lo que queda del inventario, pero al no poder suplir a la demanda se paga por costo de imagen (se pierde credibilidad y eso es medible). Por ejemplo: hay 150 unidades y la demanda fue de 210 unidades. Se logra suplir 150, pero el resto  $210 - 150 = 60$  unidades se pagan por pérdida de imagen. El costo de pérdida de imagen se calcula así: unidades que no se pudo suplir \* costo de pérdida de imagen por unidad que no fue posible suplir. Supongamos que ese costo por unidad que no fue posible vender es de \$950. Luego la pérdida de imagen es de  $60 * \$950 = \$57.000$
4. Si quien gestiona el inventario, ve que el inventario está por debajo de un punto  $R$  ese día, hace el pedido al proveedor de ese producto, la cantidad a pedir es  $Q$ . El sólo hecho de hacer el pedido genera un costo de pedido  $\$CP$ . Cabe anotar que es hasta el final del día que no se sabe si el inventario está igual o por debajo de  $R$ . Si al finalizar el día es menor o igual a  $R$ , entonces se hace el pedido, pero los días de entrega de ese pedido cuentan a partir del siguiente día, un ejemplo: al final del lunes se hace el pedido y tarda 3 días (esperaría martes, miércoles, jueves) y es hasta el viernes que recibe el pedido a primera hora.
5. El proveedor tardará un tiempo  $T$  variable de días para llegar con ese pedido. Mientras tanto, quien gestiona el inventario tendrá que defenderse con lo que le queda del inventario. No se pueden hacer más pedidos hasta que llegue el proveedor. El proveedor siempre llega con el pedido justo antes de abrir el inventario el día de la entrega. Por ejemplo: el día amanece con 175 unidades, el proveedor llega con el pedido  $Q$  de 400 unidades, luego cuando abra el inventario tendrá  $175 + 400 = 575$  unidades.

El archivo de Excel "Inventario.xlsx" muestra la simulación del comportamiento del inventario.

|    |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
|----|------------------------------------------------------------------------|----------|---|-----------------------------------|---|------------------------------------------------------------------|--------------|--------|--------|
| B5 | 120                                                                    |          |   |                                   |   |                                                                  |              |        |        |
|    | A                                                                      | B        | C | D                                 | E | F                                                                | G            | H      | I      |
| 1  |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
| 2  |                                                                        |          |   | ENTRADAS DEL MODELO DE SIMULACIÓN |   |                                                                  |              |        |        |
| 3  |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
| 4  |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
| 5  | R                                                                      | 120      |   |                                   |   | Unidades<br>compradas diarias<br>por los clientes                | Probabilidad | Minimo | Maximo |
| 6  | Q                                                                      | 240      |   |                                   |   | 25                                                               | 10%          | 0      | 0.1    |
| 7  | Inventario Inicial                                                     | 570      |   |                                   |   | 26                                                               | 20%          | 0.1    | 0.3    |
| 8  | Costo de pérdida de<br>prestigio por unidad<br>no entregada al cliente | \$ 400   |   |                                   |   | 27                                                               | 30%          | 0.3    | 0.6    |
| 9  | Costo de hacer el<br>pedido                                            | \$ 7,000 |   |                                   |   | 28                                                               | 25%          | 0.6    | 0.85   |
| 10 | Costo de almacenar<br>una unidad por día                               | \$ 125   |   |                                   |   | 29                                                               | 15%          | 0.85   | 1      |
| 11 |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
| 12 |                                                                        |          |   |                                   |   |                                                                  |              |        |        |
| 13 |                                                                        |          |   |                                   |   | Tiempo en días<br>para que el<br>proveedor<br>entregue el pedido | Probabilidad | Minimo | Maximo |
| 14 |                                                                        |          |   |                                   |   | 3                                                                | 20%          | 0      | 0.2    |
| 15 |                                                                        |          |   |                                   |   | 4                                                                | 30%          | 0.2    | 0.5    |
| 16 |                                                                        |          |   |                                   |   | 5                                                                | 35%          | 0.5    | 0.85   |
| 17 |                                                                        |          |   |                                   |   | 6                                                                | 10%          | 0.85   | 0.95   |
| 18 |                                                                        |          |   |                                   |   | 7                                                                | 5%           | 0.95   | 1      |

Ilustración 20: Archivo en Excel con las entradas del modelo de simulación

|    | L                                                      | M       | N       | O       | P | Q |
|----|--------------------------------------------------------|---------|---------|---------|---|---|
| 1  | Simulación de Inventario de punto RQ                   |         |         |         |   |   |
| 2  |                                                        |         |         |         |   |   |
| 3  |                                                        |         |         |         |   |   |
| 4  |                                                        |         |         |         |   |   |
| 5  | Generador congruencial lineal de números aleatorios #1 |         |         |         |   |   |
| 6  | A                                                      | X0      | B       | N       |   |   |
| 7  | 2678917                                                | 4579991 | 1317513 | 9824217 |   |   |
| 8  |                                                        |         |         |         |   |   |
| 9  |                                                        |         |         |         |   |   |
| 10 |                                                        |         |         |         |   |   |
| 11 | Generador congruencial lineal de números aleatorios #2 |         |         |         |   |   |
| 12 | A                                                      | X0      | B       | N       |   |   |
| 13 | 7921083                                                | 6731297 | 9021531 | 9420811 |   |   |
| 14 |                                                        |         |         |         |   |   |

*Ilustración 21: Semillas de los generadores de números pseudo-aleatorios*

K1031

▼

:

✕

✓

fx

▼

=SUMA(E1031:J1031)

|    | A             | B                | C             | D                                             | E                | F             | G      | H            | I                   | J                          |
|----|---------------|------------------|---------------|-----------------------------------------------|------------------|---------------|--------|--------------|---------------------|----------------------------|
| 20 |               |                  |               | RESULTADOS O SALIDAS DEL MODELO DE SIMULACIÓN |                  |               |        |              |                     |                            |
| 21 |               |                  |               |                                               |                  |               |        |              |                     |                            |
| 22 |               |                  |               |                                               |                  |               |        |              |                     |                            |
| 23 | Costo Total   | Costo Inventario | Costo Ordenar | Costo pérdida de prestigio                    |                  |               |        |              |                     |                            |
| 24 | \$ 14,675,475 | \$ 12,940,875    | \$ 707,000    | \$ 1,027,600                                  |                  |               |        |              |                     |                            |
| 25 |               |                  |               |                                               |                  |               |        |              |                     |                            |
| 26 |               |                  |               |                                               |                  |               |        |              |                     |                            |
| 27 |               |                  |               |                                               |                  |               |        |              |                     |                            |
| 28 | Día           | #Azar            | Demanda       | Inventario                                    | Costo Inventario | Costo Ordenar | #Azar  | Llega Pedido | Cuenta atrás pedido | Costo pérdida de prestigio |
| 29 | 0             |                  |               | 570                                           |                  |               |        |              | -1                  |                            |
| 30 | 1             | 0.71211436       | 28            | 542                                           | \$ 67,750        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 31 | 2             | 0.398659761      | 27            | 515                                           | \$ 64,375        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 32 | 3             | 0.544731555      | 27            | 488                                           | \$ 61,000        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 33 | 4             | 0.756398194      | 28            | 460                                           | \$ 57,500        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 34 | 5             | 0.115803326      | 26            | 434                                           | \$ 54,250        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 35 | 6             | 0.6317107        | 28            | 406                                           | \$ 50,750        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 36 | 7             | 0.667960001      | 28            | 378                                           | \$ 47,250        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 37 | 8             | 0.536886349      | 27            | 351                                           | \$ 43,875        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 38 | 9             | 0.102354315      | 26            | 325                                           | \$ 40,625        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 39 | 10            | 0.848191159      | 28            | 297                                           | \$ 37,125        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 40 | 11            | 0.848249789      | 28            | 269                                           | \$ 33,625        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 41 | 12            | 0.9148325        | 29            | 240                                           | \$ 30,000        | \$ 0          | -1.000 | 0            | -1                  |                            |
| 42 | 13            | 0.470878646      | 27            | 213                                           | \$ 26,625        | \$ 0          | -1.000 | 0            | -1                  |                            |

Ilustración 22: Comportamiento día a día de la simulación

Se replica esa simulación en C# con una aplicación de consola.

```

Suma de costo de inventario;12940875
Costo total de ordenar;707000
Costo total de pérdida de prestigio;1027600
Suma de todos los costos;14675475

```

Ilustración 23: Programa en C# que replica el comportamiento mostrado en Excel



```

namespace Ejemplo;

//Generador congruencia lineal de números
//pseudo-aleatorios
internal class Aleatorio {
    private long A, B, X0, N;
    public Aleatorio(long A, long X0, long B, long N) {
        this.A = A;
        this.X0 = X0;
        this.B = B;
        this.N = N;
    }

    public double Numero() {
        X0 = (A * X0 + B) % N;
        return (double)X0 / N;
    }
}

```

```

namespace Ejemplo;

//Demanda diaria del producto
internal class Demanda {
    private int Compra;
    private int Probabilidad;

    public Demanda(int Compra, int Probabilidad) {
        this.Compra = Compra;
        this.Probabilidad = Probabilidad;
    }

    public int getCompra() { return Compra; }
    public int getProbabilidad() { return Probabilidad; }
}

```

```

namespace Ejemplo;

//Los días que tarda el proveedor en entregar el producto
//desde el pedido
internal class Proveedor {
    int Dias;
    int Probabilidad;

    public Proveedor(int Dias, int Probabilidad) {
        this.Dias = Dias;
        this.Probabilidad = Probabilidad;
    }

    public int getDias() { return Dias; }
    public int getProbabilidad() { return Probabilidad; }
}

```

```

namespace Ejemplo;

//Simula el día a día del inventario de punto RQ
internal class Simular {
    int R, Q;
    int Inicial;
    int CostoPerdida;
    int CostoPedido;
    int CostoAlmacena;
    List<Demanda> DemandaCliente;
    List<Proveedor> DiasProveedor;
    int DiasSimula;
    Aleatorio AzarA;
    Aleatorio AzarB;

    public void Parametros() {
        /* Punto de Reorden.Cuando el inventario al final del día queda
        * igual o por debajo de este valor, entonces se hace el pedido
        * al proveedor */
        R = 120;

        /* Esta es la cantidad fija que se le pide al proveedor */
        Q = 240;

        /* Inventario Inicial */
        Inicial = 570;
    }
}

```

```

    /* Costo de pérdida de prestigio por unidad no entregada al cliente
    */
    CostoPerdida = 400;

    /* Costo de hacer el pedido */
    CostoPedido = 7000;

    /* Costo de almacenar una unidad por día */
    CostoAlmacena = 125;

    /* Comportamiento de la demanda,
    * las probabilidades (columna 2) deben sumar 100 */
    DemandaCliente =
    [
        new Demanda(25, 10),
        new Demanda(26, 20),
        new Demanda(27, 30),
        new Demanda(28, 25),
        new Demanda(29, 15),
    ];

    /* Dias de entrega del proveedor,
    * las probabilidades (columna 2) deben sumar 100 */
    DiasProveedor =
    [
        new Proveedor(3, 20),
        new Proveedor(4, 30),
        new Proveedor(5, 35),
        new Proveedor(6, 10),
        new Proveedor(7, 5),
    ];

    /* Días a simular */
    DiasSimula = 1000;

    /* Generadores de números aleatorios */
    AzarA = new Aleatorio(2678917, 4579991, 1317513, 9824217);
    AzarB = new Aleatorio(7921083, 6731297, 9021531, 9420811);
}

/* Retorna la demanda de unidades dependiendo del
* del número aleatorio */

public int Demanda(double Aleatorio) {
    int Acumula = 0;
    int Tope = (int) (Aleatorio * 100);
    int Cont;

```

```

        for (Cont = 0; Cont < DemandaCliente.Count; Cont++) {
            Acumula += DemandaCliente[Cont].getProbabilidad();
            if (Acumula > Tope) break;
        }
        return DemandaCliente[Cont].getCompra();
    }

    /* Retorna los días dependiendo del número aleatorio */
    public int Dias(double Aleatorio) {
        int Acumula = 0;
        int Tope = (int) (Aleatorio * 100);
        int Cont;
        for (Cont = 0; Cont < DiasProveedor.Count; Cont++) {
            Acumula += DiasProveedor[Cont].getProbabilidad();
            if (Acumula > Tope) break;
        }
        return DiasProveedor[Cont].getDias();
    }

    /* Proceso de simulación */
    public void Proceso() {
        //Día cero
        AzarA.Numero();
        AzarB.Numero();

        //Inicia el día cero con un inventario inicial
        int Inventario = Inicial;

        //Días para llegar el pedido del proveedor
        int DiasLlegaPedido = 0, CuentaAtrasPedido = -1;

        //Costos diarios
        int CostoInventario, CostoOrdenar, CostoPrestigio;

        //Acumula Costos
        int AcumCostoInventario = 0, AcumCostoOrdenar = 0, AcumCostoPrestigio
= 0;

        Console.WriteLine("Dia;Azar;Demanda;Inventario;Costo Inventario;Costo
Ordenar;Azar;Llega Pedido;Cuenta atrás pedido;Costo pérdida de
prestigio");

        for (int dias = 1; dias <= DiasSimula; dias++) {

            //Cuenta Atrás para llegar el pedido del proveedor
            if (DiasLlegaPedido > 0)
                CuentaAtrasPedido = DiasLlegaPedido;
            else {

```

```

        if (CuentaAtrasPedido > 0)
            CuentaAtrasPedido--;
        else
            CuentaAtrasPedido = -1;
    }

    //Valores aleatorios del día
    double AleatorioA = AzarA.Numero();
    double AleatorioB = AzarB.Numero();

    //Valor del inventario
    int NuevoValor = 0;
    if (Inventario > 0) NuevoValor = Inventario;

    //Compra de los clientes
    int Compra = Demanda(AleatorioA);
    NuevoValor -= Compra;

    //Si llega el pedido
    if (CuentaAtrasPedido == 0) NuevoValor += Q;

    //El nuevo valor de inventario
    Inventario = NuevoValor;

    //Costo del inventario
    if (Inventario > 0)
        CostoInventario = Inventario * CostoAlmacena;
    else
        CostoInventario = 0;

    //Costo ordenar
    CostoOrdenar = 0;
    if (Inventario <= R && CuentaAtrasPedido <= 0)
        CostoOrdenar = CostoPedido;

    //Segundo Azar si hay pedido
    double SegundoAzar = -1;
    if (CostoOrdenar > 0)
        SegundoAzar = AleatorioB;

    //Llega Pedido
    DiasLlegaPedido = 0;
    if (SegundoAzar > 0)
        DiasLlegaPedido = Dias(SegundoAzar);

    //Pérdida de prestigio
    CostoPrestigio = 0;
    if (Inventario < 0)

```

```

        CostoPrestigio = Inventario * CostoPerdida * -1;

        //Imprime el día
        Console.WriteLine(dias + ";" + AleatorioA + ";" + Compra + ";" +
Inventario + ";" + CostoInventario + ";" + CostoOrdenar + ";" +
SegundoAzar + ";" + DiasLlegaPedido + ";" + CuentaAtrasPedido + ";" +
CostoPrestigio);

        //Acumula costos
        AcumCostoInventario += CostoInventario;
        AcumCostoOrdenar += CostoOrdenar;
        AcumCostoPrestigio += CostoPrestigio;
    }

    //Imprime los costos totales
    Console.WriteLine("Suma de costo de inventario;" +
AcumCostoInventario);
    Console.WriteLine("Costo total de ordenar;" + AcumCostoOrdenar);
    Console.WriteLine("Costo total de pérdida de prestigio;" +
AcumCostoPrestigio);

    //Costo total
    int CostoTotal = AcumCostoInventario + AcumCostoOrdenar +
AcumCostoPrestigio;
    Console.WriteLine("Suma de todos los costos;" + CostoTotal);
}
}

```

I/013.zip

```

namespace Ejemplo;
//Inventario
class Program {
    static void Main() {
        Simular simular = new();
        simular.Parametros();
        simular.Proceso();
    }
}

```

Hay que considerar que una simulación es sensible a los valores iniciales, en el caso de esta simulación, es el inventario inicial. Luego, se debería escoger un valor de inventario inicial muy cercano a Q (pedido) o hacer los totales de costos a partir de varios días, por ejemplo, del 100 en adelante, cuando se considera que el modelo se ha estabilizado.

## Simulación de Inventarios: Buscando el mejor valor

En la simulación anterior, la decisión recae en los valores R y Q, en búsqueda de disminuir el costo total. Una técnica es probar sistemáticamente valores R y Q hasta dar con aquellos que obtengan el mínimo costo. Para esto, se modifica el programa anterior para que se dedique a calcular, no imprimir y además retorne el costo total. Importante: Debido a que las simulaciones son sensibles a los valores iniciales (en este caso, al valor del inventario inicial) y tanto Q como R tienen diversos valores, entonces los totales se calculan a partir de varios días de simulación para que esta se estabilice, en el código, a partir del día 500 (se puede variar).

I/014.zip

```
namespace Ejemplo;

//Generador congruencia lineal de números
//pseudo-aleatorios
internal class Aleatorio {
    private long A, B, X0, N;
    public Aleatorio(long A, long X0, long B, long N) {
        this.A = A;
        this.X0 = X0;
        this.B = B;
        this.N = N;
    }

    public double Numero() {
        X0 = (A * X0 + B) % N;
        return (double) X0 / N;
    }
}
```

I/014.zip

```
namespace Ejemplo;

//Demanda diaria del producto
internal class Demanda {
    private int Compra;
    private int Probabilidad;

    public Demanda(int Compra, int Probabilidad) {
        this.Compra = Compra;
        this.Probabilidad = Probabilidad;
    }

    public int getCompra() { return Compra; }
```

```
    public int getProbabilidad() { return Probabilidad; }  
}
```

I/014.zip

```
namespace Ejemplo;  
  
//Los días que tarda el proveedor en entregar el producto  
//desde el pedido  
internal class Proveedor {  
    int Dias;  
    int Probabilidad;  
  
    public Proveedor(int Dias, int Probabilidad) {  
        this.Dias = Dias;  
        this.Probabilidad = Probabilidad;  
    }  
  
    public int getDias() { return Dias; }  
    public int getProbabilidad() { return Probabilidad; }  
}
```

I/014.zip

```
namespace Ejemplo;  
  
//Simula el día a día del inventario de punto RQ  
internal class Simular {  
    int R, Q;  
    int Inicial;  
    int CostoPerdida;  
    int CostoPedido;  
    int CostoAlmacena;  
    List<Demanda> DemandaCliente;  
    List<Proveedor> DiasProveedor;  
    int DiasSimula;  
    int IniciaDiaCostoAcumula;  
    Aleatorio AzarA;  
    Aleatorio AzarB;  
  
    public void Parametros(int R, int Q) {  
        /* Punto de Reorden.Cuando el inventario al final  
        * del día queda igual o por debajo de este valor,  
        * entonces se hace el pedido al proveedor */  
        this.R = R;  
    }  
}
```



```

/* Esta es la cantidad fija que se le pide al proveedor */
this.Q = Q;

/* Inventario Inicial */
Inicial = 570;

/* Costo de pérdida de prestigio por
 * unidad no entregada al cliente */
CostoPerdida = 400;

/* Costo de hacer el pedido */
CostoPedido = 7000;

/* Costo de almacenar una unidad por día */
CostoAlmacena = 125;

/* Comportamiento de la demanda,
 * las probabilidades deben sumar 100 */
DemandaCliente =
[
    new Demanda(25, 10),
    new Demanda(26, 20),
    new Demanda(27, 30),
    new Demanda(28, 25),
    new Demanda(29, 15),
];

/* Dias de entrega del proveedor,
 * las probabilidades deben sumar 100 */
DiasProveedor =
[
    new Proveedor(3, 20),
    new Proveedor(4, 30),
    new Proveedor(5, 35),
    new Proveedor(6, 10),
    new Proveedor(7, 5),
];

/* Días a simular */
DiasSimula = 2000;

/* A partir de que día se acumulan los costos.
 * La razón es evitar que el valor inicial del
 * inventario, afecte a la simulación. Hay que
 * esperar que se estabilice */
IniciaDiaCostoAcumula = 500;

```

```

    /* Generadores de números aleatorios */
    AzarA = new Aleatorio(2678917, 4579991, 1317513, 9824217);
    AzarB = new Aleatorio(7921083, 6731297, 9021531, 9420811);
}

/* Retorna la demanda de unidades dependiendo del
 * del número aleatorio */

public int Demanda(double Aleatorio) {
    int Acumula = 0;
    int Tope = (int) (Aleatorio * 100);
    int Cont;
    for (Cont = 0; Cont < DemandaCliente.Count; Cont++) {
        Acumula += DemandaCliente[Cont].getProbabilidad();
        if (Acumula > Tope) break;
    }
    return DemandaCliente[Cont].getCompra();
}

/* Retorna los días dependiendo del número aleatorio */
public int Dias(double Aleatorio) {
    int Acumula = 0;
    int Tope = (int) (Aleatorio * 100);
    int Cont;
    for (Cont = 0; Cont < DiasProveedor.Count; Cont++) {
        Acumula += DiasProveedor[Cont].getProbabilidad();
        if (Acumula > Tope) break;
    }
    return DiasProveedor[Cont].getDias();
}

/* Proceso de simulación */
public int Proceso() {
    //Día cero
    AzarA.Numero();
    AzarB.Numero();

    //Inicia el día cero con un inventario inicial
    int Inventario = Inicial;

    //Días para llegar el pedido del proveedor
    int DiasLlegaPedido = 0, CuentaAtrasPedido = -1;

    //Costos diarios
    int CostoInventario, CostoOrdenar, CostoPrestigio;

    //Acumula Costos

```

```

int AcumCostoInventario = 0, AcumCostoOrdenar = 0;
int AcumCostoPrestigio = 0;

for (int dias = 1; dias <= DiasSimula; dias++) {

    //Cuenta Atrás para llegar el pedido del proveedor
    if (DiasLlegaPedido > 0)
        CuentaAtrasPedido = DiasLlegaPedido;
    else {
        if (CuentaAtrasPedido > 0)
            CuentaAtrasPedido--;
        else
            CuentaAtrasPedido = -1;
    }

    //Valores aleatorios del día
    double NumA = AzarA.Numero();
    double NumB = AzarB.Numero();

    //Valor del inventario
    int NuevoValor = 0;
    if (Inventario > 0) NuevoValor = Inventario;

    //Compra de los clientes
    int Compra = Demanda(NumA);
    NuevoValor -= Compra;

    //Si llega el pedido
    if (CuentaAtrasPedido == 0) NuevoValor += Q;

    //El nuevo valor de inventario
    Inventario = NuevoValor;

    //Costo del inventario
    if (Inventario > 0)
        CostoInventario = Inventario * CostoAlmacena;
    else
        CostoInventario = 0;

    //Costo ordenar
    CostoOrdenar = 0;
    if (Inventario <= R && CuentaAtrasPedido <= 0)
        CostoOrdenar = CostoPedido;

    //Segundo Azar si hay pedido
    double SegundoAzar = -1;
    if (CostoOrdenar > 0)
        SegundoAzar = NumB;
}

```

```

//Llega Pedido
DiasLlegaPedido = 0;
if (SegundoAzar > 0)
    DiasLlegaPedido = Dias(SegundoAzar);

//Pérdida de prestigio
CostoPrestigio = 0;
if (Inventario < 0)
    CostoPrestigio = Inventario * CostoPerdida * -1;

//Acumula costos
if (dias >= IniciaDiaCostoAcumula) {
    AcumCostoInventario += CostoInventario;
    AcumCostoOrdenar += CostoOrdenar;
    AcumCostoPrestigio += CostoPrestigio;
}
}

//Costo total
int CostoTotal = AcumCostoInventario;
CostoTotal += AcumCostoOrdenar;
CostoTotal += AcumCostoPrestigio;
return CostoTotal;
}
}

```

I/014.zip

```

namespace Ejemplo;
//Inventario
class Program {
    static void Main() {
        int CostoTotal = int.MaxValue;

        int MejorR = -1, MejorQ = -1;
        for (int R = 1; R <= 1000; R++)
            for (int Q = 1; Q <= 1000; Q++) {
                int Costo = SimularInventario(R, Q);
                if (Costo < CostoTotal) {
                    CostoTotal = Costo;
                    MejorR = R;
                    MejorQ = Q;
                    Console.WriteLine("Costo: " + CostoTotal + " R = " + MejorR +
" Q = " + MejorQ);
                }
            }
    }
}

```

```

        Console.WriteLine("Costo más bajo: " + CostoTotal);
        Console.WriteLine("R = " + MejorR);
        Console.WriteLine("Q = " + MejorQ);
    }

    static public int SimularInventario(int R, int Q) {
        Simular simular = new();
        simular.Parametros(R, Q);
        return simular.Proceso();
    }
}

```

```

Costo: 18110400 R = 1 Q = 1
Costo: 18002400 R = 1 Q = 2
Costo: 17894400 R = 1 Q = 3
Costo: 17786400 R = 1 Q = 4
Costo: 17678400 R = 1 Q = 5
Costo: 17570400 R = 1 Q = 6
Costo: 17462400 R = 1 Q = 7
Costo: 17354400 R = 1 Q = 8
Costo: 17246400 R = 1 Q = 9
Costo: 17138400 R = 1 Q = 10
Costo: 17030400 R = 1 Q = 11
Costo: 16922400 R = 1 Q = 12
Costo: 16814400 R = 1 Q = 13

```

*Ilustración 24: Inicia la simulación*

```

Costo: 12017900 R = 78 Q = 103
Costo: 12012025 R = 79 Q = 104
Costo: 12008275 R = 80 Q = 105
Costo: 11996350 R = 82 Q = 107
Costo más bajo: 11996350
R = 82
Q = 107

```

*Ilustración 25: Finaliza la simulación*

Después de varios segundos (o minutos, dependiendo de la velocidad del PC), se obtienen los valores R y Q que generan el menor costo posible.

## Jugadores y apuestas

Se da la siguiente situación: un grupo de jugadores, cada uno con un mismo capital inicial, empieza una serie de rondas de apuestas. Cada jugador apuesta N% de su capital, y ese N% es lo que puede perder o puede ganar. Por ejemplo, un jugador tiene \$100.000 de capital y apuesta el 15%, es decir, \$15.000; si gana, le suma \$15.000 a su capital, si pierde, le resta \$15.000 a su capital. ¿Y cómo es el juego? Se lanza un número entero al azar entre 1 y 100, si ese número es menor o igual a N, el jugador pierde, caso contrario, gana. ¿Cuál sería el mejor N para aumentar el capital?

I/015. zip

```
namespace Ejemplo;
internal class Apostador {
    public int Dinero;
    public int Riesgo;

    public Apostador(int dinero, int riesgo) {
        Dinero = dinero;
        Riesgo = riesgo;
    }
}
```

I/015.zip

```
namespace Ejemplo;
internal class Poblacion {
    int TotalJugadores;
    int DineroInicial;
    int DineroMinimo;
    int TotalCiclos;
    List<Apostador> Jugador;

    public void Parametros() {
        TotalJugadores = 900;
        Jugador = [];

        //Con cuánto dinero arranca cada jugador
        DineroInicial = 20000;

        //Si el capital de un jugador cae por debajo
        //de este número, se retira
        DineroMinimo = 1500;

        TotalCiclos = 7000;
    }
}
```

```

public void Proceso() {
    Random Azar = new();

    //Crea la población
    for (int cont = 1; cont <= TotalJugadores; cont++)
        Jugador.Add(new Apostador(DineroInicial, Azar.Next(1, 101)));

    //Ciclo de juegos
    for (int Ciclo = 1; Ciclo <= TotalCiclos; Ciclo++) {
        int num = 0;
        while (num < Jugador.Count) {

            //Si el jugador queda con capital por debajo
            //de DineroMinimo, se elimina
            if (Jugador[num].Dinero < DineroMinimo) {
                Jugador.RemoveAt(num);
                continue;
            }

            //Se enfrenta a una apuesta
            int Ganancia = Jugador[num].Dinero * Jugador[num].Riesgo /
100;

            if (Azar.Next(1, 101) <= Jugador[num].Riesgo)
                Jugador[num].Dinero -= Ganancia;
            else
                Jugador[num].Dinero += Ganancia;

            //Siguiente jugador
            num++;
        }

        //¿La población quedó en cero?
        if (Jugador.Count == 0) {
            Console.WriteLine("Colapso de la población");
            Environment.Exit(0);
        }

        //Si la población está por debajo de TotalJugadores,
        //entonces se busca el que más dinero tiene y se le
        //sacan copias modificadas
        if (Jugador.Count < TotalJugadores) {

            //Busca el que tiene más dinero
            int MaximoDinero = int.MinValue;
            int Cual = -1;
            for (num = 0; num < Jugador.Count; num++) {
                if (Jugador[num].Dinero > MaximoDinero) {
                    MaximoDinero = Jugador[num].Dinero;
                }
            }
        }
    }
}

```

```

        Cual = num;
    }
}

//Saca copias modificadas del individuo con más dinero
while (Jugador.Count < TotalJugadores) {
    int CambiaRiesgo = 1;
    if (Azar.NextDouble() < 0.5) CambiaRiesgo = -1;
    int NuevoRiesgo = Jugador[Cual].Riesgo + CambiaRiesgo;
    Jugador.Add(new Apostador(DineroInicial, NuevoRiesgo));
}
}
}

Console.WriteLine("Los jugadores que se mantienen en la población,
arriesgaron porcentaje de su capital en promedio: ");
int Acumula = 0;
for (int num = 0; num < Jugador.Count; num++)
    Acumula += Jugador[num].Riesgo;
double Promedio = (double) Acumula / Jugador.Count;
Console.WriteLine(Promedio);
}
}

```

I/015.zip

```

namespace Ejemplo;
internal class Program {
    static void Main() {
        Poblacion poblacion = new();
        poblacion.Parametros();
        poblacion.Proceso();
    }
}

```

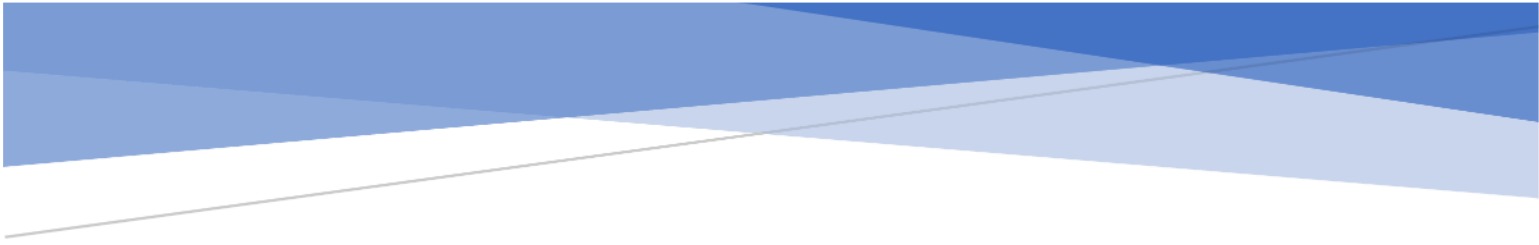


*Ilustración 26: Valor en porcentaje a apostar*

El valor de N en el programa es la variable “Riesgo”, luego según la simulación con esos valores de dinero inicial (capital inicial), informa que N debe ser del 22.46% para obtener las mayores ganancias posibles.







# C# Y .NET 10

## Parte 10. Algoritmos Evolutivos

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                       | 3  |
| Acerca del autor.....                                             | 4  |
| Licencia de este libro .....                                      | 4  |
| Licencia del software .....                                       | 4  |
| Marcas registradas .....                                          | 5  |
| Dedicatoria .....                                                 | 6  |
| Definición .....                                                  | 7  |
| Búsqueda aleatoria.....                                           | 8  |
| El operador Mutación .....                                        | 10 |
| El operador Cruce .....                                           | 17 |
| Combinando los operadores cruce y mutación.....                   | 24 |
| Buscar el máximo en una función .....                             | 29 |
| De genotipos y fenotipos .....                                    | 35 |
| Máximos y mínimos locales.....                                    | 43 |
| Buscar el mayor valor en una ecuación de múltiples variables..... | 45 |
| Simplificar una ecuación.....                                     | 51 |

## Tabla de ilustraciones

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| Ilustración 1: El programa es muy improbable que se detenga .....                       | 9  |
| Ilustración 2: Algoritmo evolutivo con el operador mutación .....                       | 16 |
| Ilustración 3: Algoritmo evolutivo con el operador cruce .....                          | 22 |
| Ilustración 4: Algoritmo evolutivo: Combinando operador cruce y mutación .....          | 28 |
| Ilustración 5: Polinomio de grado 6 .....                                               | 29 |
| Ilustración 6: Buscar el máximo en una función .....                                    | 34 |
| Ilustración 7: Gráfico de una ecuación .....                                            | 35 |
| Ilustración 8: Operador cruce y mutación .....                                          | 42 |
| Ilustración 9: Gráfico matemático .....                                                 | 43 |
| Ilustración 10: Buscar el mayor valor de Y con múltiples variables independientes ..... | 50 |
| Ilustración 11: Algoritmo Evolutivo .....                                               | 59 |
| Ilustración 12: Algoritmo Evolutivo .....                                               | 60 |
| Ilustración 13: Ejemplo de simplificación de curvas .....                               | 61 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelif@hotmai.com](mailto:enginelif@hotmai.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

## Definición

Los algoritmos evolutivos, inspirados en la biología, sirven para resolver problemas que pueden llegar a ser bastante complejos. Su mecánica es la aplicación de principios evolutivos como la reproducción y la selección natural de soluciones a un problema. A medida que pasa el tiempo (en algoritmos es en cada iteración o ciclo), se van encontrando mejores soluciones a ese problema dado. La utilidad de este tipo de algoritmos es que ofrecen un camino a encontrar soluciones mejores (no las ideales) a problemas complejos donde una solución clásica o sistemática tardaría muchísimo. En la parte de "Simulaciones" se trabajó con soluciones de tipo Montecarlo para hallar la ruta de menor costo, o resolver un Sudoku, o hallar el área bajo la curva. Los algoritmos evolutivos siguen ese estilo.



# Búsqueda aleatoria

Está la siguiente cadena:

*esta es una prueba de texto*

¿Es posible generar cadenas de caracteres al azar de tal manera que coincida con la cadena anterior? En realidad, sí, pero es altamente improbable que la cadena generada al azar acierte a esa cadena en particular.

Probando con código:

J/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main(string[] args) {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Cadena original a "adivinar" */
        string Original = "naturaleza";

        /* Iniciar el proceso */
        Proceso(Azar, Original);
    }

    /* Método que realiza el proceso de "adivinar" la cadena original */
    static void Proceso(Random Azar, string Original) {
        int Contador = 0;
        for (; ; ) {

            /* Generar una cadena aleatoria */
            string Cadena = GenerarCadenaAzar(Azar, Original.Length);

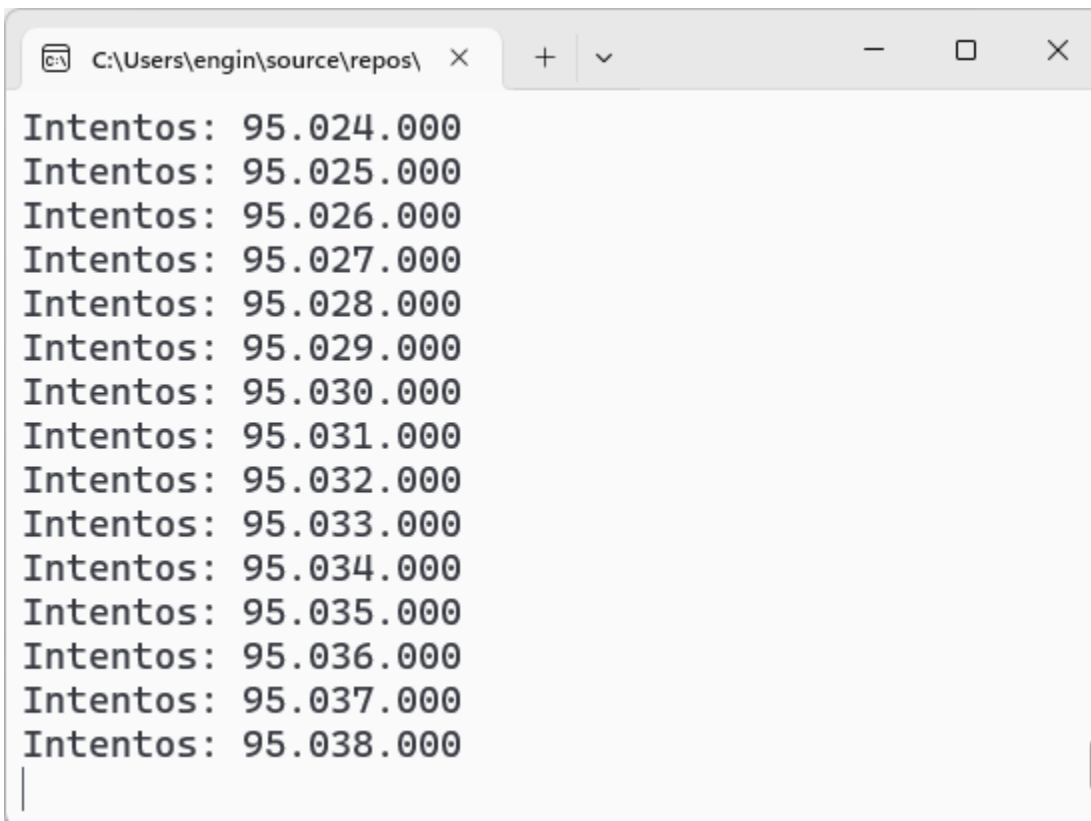
            /* Comparar con la original */
            if (string.Compare(Original, Cadena) == 0) {
                break;
            }

            /* Incrementar el contador e informar cada 1000 intentos */
            Contador++;
            if (Contador % 1000 == 0) {
                Console.WriteLine($"Intentos: {Contador:N0}");
            }
        }
    }
}
```

```
/* Método que genera una cadena aleatoria de una longitud dada */
static string GenerarCadenaAzar(Random Azar, int longitud) {
    char[] caracteres = new char[longitud];
    for (int i = 0; i < longitud; i++) {
        caracteres[i] = LetraAzar(Azar);
    }
    return new string(caracteres);
}

/* Método que genera una letra minúscula aleatoria */
static char LetraAzar(Random Azar) {
    return (char)Azar.Next('a', 'z' + 1);
}
}
```

Es muy improbable que la cadena generada al azar acierte a la cadena original, luego es muy improbable el programa se detenga.



```
C:\Users\engin\source\repos\ x + - □ ×
Intentos: 95.024.000
Intentos: 95.025.000
Intentos: 95.026.000
Intentos: 95.027.000
Intentos: 95.028.000
Intentos: 95.029.000
Intentos: 95.030.000
Intentos: 95.031.000
Intentos: 95.032.000
Intentos: 95.033.000
Intentos: 95.034.000
Intentos: 95.035.000
Intentos: 95.036.000
Intentos: 95.037.000
Intentos: 95.038.000
```

*Ilustración 1: El programa es muy improbable que se detenga*

## El operador Mutación

Como el algoritmo anterior falla en dar con la cadena. Se hace uso entonces del concepto "algoritmo evolutivo", en este caso, se aplica el operador mutación.

El algoritmo:

### Algoritmo Evolutivo

#### Inicio

Generar población de N individuos al azar

#### Inicio ciclo

Seleccionar al azar dos individuos de esa población: A y B

Evaluar adaptación de A

Evaluar adaptación de B

**Si** adaptación de A es mejor que adaptación de B **entonces**

Eliminar individuo B

Duplicar individuo A

Modificar levemente al azar el nuevo duplicado

#### de lo contrario

Eliminar individuo A

Duplicar individuo B

Modificar levemente al azar el nuevo duplicado

#### Fin Si

#### Fin ciclo

Buscar individuo con mejor adaptación de la población

Imprimir individuo

#### Fin

Explicación del algoritmo:

"Generar población de N individuos al azar": Generar N cadenas al azar y guardarlas en un arreglo o lista.

"Evaluar adaptación de": Evaluar cuantitativamente esa cadena con respecto a la original. En ese caso, una medida puede ser sumar un punto por cada letra que coincida con la cadena original. A mayor puntaje, mejor es la adaptación.

Al principio se crea una población con varios individuos generados al azar, por ejemplo, con 10 individuos:

```
1: oTGBVáDgzD!óxëVoÓsbY AoVVRwkTj
2: Áh?JÄgÉmmÍ;!óQ UbFtdfymNtNSjmF
3: rEVáa Z;oaÓC¿rxy?Äx¿pÑj íHmfGÓ
4: ÜkJñzU;mCoQEZe! ÄmúÖN¿Ghj zoá
5: tWsLótMRFhoB;íÄpkkDgYq;LrXëÓU
6: LFopAİéóüYj!ëÖÄvÚMycCÄİİFPLÜ?
```

7: ÑýérMQwlgMXaYTgúGsFfQÑIQBÜsíb  
8: jOYnÚtAEH¿Ë;HVvPB ÁëÜáx¿fuÛch  
9: ñróQEx;MqöüïEKÜiNTpRBóBWúofsö  
10: ëWÄUPLúíiBVëÁÑLaómpKPÓ!İWXYÚQ

Se seleccionan dos individuos al azar, por ejemplo, el 3 y el 9:

1: oTGBVáDgzD!óxëVoÓsbYAoVVRwkTj  
2: Áh?JÄgÉmmÍ;!óQUbFtdfymNtNSjmf  
3: rEVáa Z;oaÓC¿rxy?Äx¿pÑjíHmfGÓ  
4: ÜkJñzU;mCoQEZe! ÄmúÖN¿Ghj zoá  
5: tWsLótMRFhoB;íÄpkkDgYq;LrXëÓU  
6: LFopAİéóüYj!ëÖÄvÚMycCÄİİFPLÜ?  
7: ÑýérMQwlgMXaYTgúGsFfQÑIQBÜsíb  
8: jOYnÚtAEH¿Ë;HVvPB ÁëÜáx¿fuÛch  
9: ñróQEx;MqöüïEKÜiNTpRBóBWúofsö  
10: ëWÄUPLúíiBVëÁÑLaómpKPÓ!İWXYÚQ

Ahora se evalúa cual individuo, de los dos seleccionados, es el mejor. Una forma de hacerlo es comparar letra por letra entre la cadena del individuo y la cadena original, si coinciden entonces suma 1 al puntaje.

Evaluar: rEVáa Z;oaÓC¿rxy?Äx¿pÑjíHmfGÓ vs Esta es una prueba de texto Es 1

Evaluar: ñróQEx;MqöüïEKÜiNTpRBóBWúofsö vs Esta es una prueba de texto Es 0

El individuo ganador es rEVáa Z;oaÓC¿rxy?Äx¿pÑjíHmfGÓ con puntaje de 1, el otro pierde con un puntaje de 0

El individuo perdedor es eliminado de la población

```
1: oTGBVáDgzD!óxëVoÓsbYAoVVRwkTj
2: Áh?JÄgÉmmÍ;!óQUbFtdfymNtNSjmF
3: rEVáa Z;oaÓC;rxY?Äx;pÑj íHmfGÓ
4: ÜkJñzU;mCoQEZe! ÄmúÖN;Ghj zoá
5: tWsLótMRfhoB;íÄpkkDgYq;LrXëÓU
6: LFopAİéóüYj!ëÖÄvÚMycCÄİİFPLÜ?
7: ÑyérMQwlgMXaYTgúGsFfQÑIQBÜsíb
8: jOYnÚtAEH;Ë;HVvPB ÄëÜáx;fuÜch
9: ñróQEEx;MqöüiEKÜiNTpRBóBWúofse
10: ëWÄUPLúíiBVëÄÑLaómpKPÓ!İWXYÚQ
```

El mejor individuo se premia creando una copia de sí mismo

```
3: rEVáa Z;oaÓC;rxY?Äx;pÑj íHmfGÓ
```

Ahora esa copia se modifica en algún punto al azar

```
¿?: rEVáa ZhhoaÓC;rxY?Äx;pÑj íHmfGÓ
```

Esa copia modificada ahora hace parte de la población ocupando el espacio dejado por el perdedor

```
1: oTGBVáDgzD!óxëVoÓsbYAoVVRwkTj
2: Áh?JÄgÉmmÍ;!óQUbFtdfymNtNSjmF
3: rEVáa Z;oaÓC;rxY?Äx;pÑj íHmfGÓ
4: ÜkJñzU;mCoQEZe! ÄmúÖN;Ghj zoá
5: tWsLótMRfhoB;íÄpkkDgYq;LrXëÓU
6: LFopAİéóüYj!ëÖÄvÚMycCÄİİFPLÜ?
7: ÑyérMQwlgMXaYTgúGsFfQÑIQBÜsíb
8: jOYnÚtAEH;Ë;HVvPB ÄëÜáx;fuÜch
9: rEVáa ZhhoaÓC;rxY?Äx;pÑj íHmfGÓ
10: ëWÄUPLúíiBVëÄÑLaómpKPÓ!İWXYÚQ
```

El proceso se repite varias veces hasta que se encuentra el mejor individuo que soluciona el problema.

```
namespace Ejemplo;

internal class Program {
    public static char[] Letras;

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Alfabeto de letras */
        Letras = "abcdefghijklmnopqrstuvwxyz".ToCharArray();

        /* Cadena original a "adivinar" */
        string Original = "esta es una prueba para dar con una cadena";

        /* Iniciar el proceso */
        Proceso(Azar, Original);
    }

    /* Método que realiza el proceso de "adivinar" la cadena original */
    static void Proceso(Random Azar, string Original) {
        /* Trabaja con arreglos de caracteres */
        char[] OriginalArray = Original.ToCharArray();

        /* Genera población inicial con individuos que tengan la
           misma longitud de la cadena original */
        int TotalIndividuos = 1000;
        char[][] Individuos = new char[TotalIndividuos][];

        int Tamano = OriginalArray.Length;
        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
```

```

Individuos[indiv] = new char[Tamano];
for (int pos = 0; pos < Tamano; pos++)
    Individuos[indiv][pos] = Letras[Azar.Next(Letras.Length)];
}

int Contador = 0;
int MejorPuntaje = -1;
int MejorIndividuo = -1;

while (true) {

    /* Seleccionar dos individuos aleatoriamente */
    int Indice1 = Azar.Next(TotalIndividuos);
    int Indice2;
    do {
        Indice2 = Azar.Next(TotalIndividuos);
    } while (Indice2 == Indice1);

    /* Evalúa cada individuo */
    int Puntaje1 = Puntuar(Individuos[Indice1], OriginalArray);
    int Puntaje2 = Puntuar(Individuos[Indice2], OriginalArray);

    if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
    if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

    /* Evalúa si un individuo ha acertado a la cadena original */
    if (Puntaje1 == OriginalArray.Length || Puntaje2 == OriginalArray.Length) {
        break;
    } /* Evalúa si el individuo 1 es mejor que el individuo 2 */
    else if (Puntaje1 > Puntaje2) {
        // El ganador sobre-escribe al perdedor
        Array.Copy(Individuos[Indice1], Individuos[Indice2], Individuos[Indice1].Length);

        //Modifica la copia
        Individuos[Indice1][Azar.Next(OriginalArray.Length)] = Letras[Azar.Next(Letras.Length)];
    } /* Evalúa si el individuo 2 es mejor que el individuo 1 */
}

```

```

else if (Puntaje2 > Puntaje1) {
    // El ganador sobre-escribe al perdedor
    Array.Copy(Individuos[Indice2], Individuos[Indice1], Individuos[Indice2].Length);

    //Modifica la copia
    Individuos[Indice2][Azar.Next(OriginalArray.Length)] = Letras[Azar.Next(Letras.Length)];
}

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");
}
}

Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");
}

/* Evalúa la similitud entre dos arreglos de caracteres */
static int Puntuar(char[] charArreglo1, char[] charArreglo2) {
    int Similitud = 0;
    for (int i = 0; i < charArreglo1.Length; i++) {
        if (charArreglo1[i] == charArreglo2[i]) {
            Similitud++;
        }
    }
    return Similitud;
}
}

```



```
Consola de depuración de Mi X + v - □ X
Intento: 128.000 Mejor individuo: [esta es una prueba fadt dar cmn una cadena]
Intento: 129.000 Mejor individuo: [esta es una prueba fadt dar cmn una cadena]
Intento: 130.000 Mejor individuo: [esta es una prueba fadt dar cmn una cadena]
Intento: 131.000 Mejor individuo: [esta es una prueba paaa dar cju una cadena]
Intento: 132.000 Mejor individuo: [esta es una prueba paaa dar cju una cadena]
Intento: 133.000 Mejor individuo: [esta es una prueba paaa oar cjn una cadena]
Intento: 134.000 Mejor individuo: [estahes una prueba dara dar crn una cadena]
Intento: 135.000 Mejor individuo: [estahes una prueba dara dar crn una cadena]
Intento: 136.000 Mejor individuo: [estahes una prueba dara dar crn una cadena]
Intento: 137.000 Mejor individuo: [esta es unagprueba pama dar cyn una cadena]
Intento: 138.000 Mejor individuo: [esta es unagprueba pama dar cyn una cadena]
Intento: 139.000 Mejor individuo: [esta es unaapruueba xaca dar con una cadena]
Intento: 140.000 Mejor individuo: [esta es mna prueba kara dsr con una cadena]
Intento: 141.000 Mejor individuo: [esta es mna prueba kara dsr con una cadena]
Intento: 142.000 Mejor individuo: [esta es una prueba para dar crn una gadena]
Intento: 143.000 Mejor individuo: [esta es una prueba para dar crn una gadena]
Intento: 144.000 Mejor individuo: [esta es una prueba para darfcrn una gadena]
Intento: 145.000 Mejor individuo: [esta es una prueba para darfcrn una gadena]
Intento: 146.000 Mejor individuo: [esta es una prueba para darfcrn una gadena]
Intento: 147.000 Mejor individuo: [esta es una prueba para darfcrn una gadena]
Intento: 148.000 Mejor individuo: [esta es una prueba iara dar chn una cadena]
Intento: 149.000 Mejor individuo: [esta es una prueba iara dar chn una cadena]
Intento: 150.000 Mejor individuo: [esta es una prueba iara dar chf una cadena]
Intento: 151.000 Mejor individuo: [esta es unakprueba paaa dar con una cadena]
Intento: 152.000 Mejor individuo: [esta es una rueba kara dar con una cadena]
Intento: 152.944 Mejor individuo: [esta es una prueba para dar con una cadena]
```

Ilustración 2: Algoritmo evolutivo con el operador mutación

Después de varios ciclos, se logró dar con la cadena.

## El operador Cruce

Dada la cadena de destino:

```
String original = "¡Esta es una prueba de texto!";
```

Se genera una población donde cada individuo es una cadena, por ejemplo:

```
1: oTGBVáDgzD!óxëVoÓsbYAoVVRwkTj
2: Áh?JÄgÉmmÍ;!óQUbFtdfymNtNSjmF
3: rEVáaqZ;oaÓC;rxY?Äx;pÑjíHmfGÓ
4: ÜkJñzU;mCoQEZe! ÄmúÖN;Ghj zoá
5: tWsLótMRFhoB;íÄpkkDgYq;LrXëÓU
6: LFopAİéÓüYj!ëÖÄvÚMycCÄİİFPLÜ?
7: ÑyérMQwlgMXaYTgúGsFfQÑIQBÜsíb
8: jOYnÚtAEH;Ë;HVvPB ÄëÜáx;fuÜch
9: ñróQEx;MqöúïEKÜïINTpRBóBWúofsö
10: ëWÄUPLúíïBVëÄÑLaómpKPÓ!İWXYÚQ
```

Se seleccionan dos individuos al azar

```
1: oTGBVáDgzD!óxëVoÓsbYAoVVRwkTj
2: Áh?JÄgÉmmÍ;!óQUbFtdfymNtNSjmF
3: rEVáaqZ;oaÓC;rxY?Äx;pÑjíHmfGÓ
4: ÜkJñzU;mCoQEZe! ÄmúÖN;Ghj zoá
5: tWsLótMRFhoB;íÄpkkDgYq;LrXëÓU
6: LFopAİéÓüYj!ëÖÄvÚMycCÄİİFPLÜ?
7: ÑyérMQwlgMXaYTgúGsFfQÑIQBÜsíb
8: jOYnÚtAEH;Ë;HVvPB ÄëÜáx;fuÜch
9: ñróQEx;MqöúïEKÜïINTpRBóBWúofsö
10: ëWÄUPLúíïBVëÄÑLaómpKPÓ!İWXYÚQ
```

Esos dos individuos generan dos hijos con la operación de cruce:

### Hijo A

Al azar se escoge un punto de corte, la primera parte la pone el individuo A y el resto el Individuo B (en fondo verde son las partes de los individuos que constituirán el Hijo A)

rEVáaqZ;oaÓC;rxY?Äx;pÑj íHmfGÓ y ñróQEx;MqöúïEKÜïNTpRBóBWúofsö

Hijo A nace al unir las dos piezas: rEVáax;MqöúïEKÜïNTpRBóBWúofsö

### Hijo B

Usando el mismo punto de corte anterior, la primera parte la pone el individuo B y el resto el Individuo A (en fondo azul son las partes de los individuos que constituirán el Hijo B)

rEVáaqZ;oaÓC;rxY?Äx;pÑj íHmfGÓ y ñróQEx;MqöúïEKÜïNTpRBóBWúofsö

Hijo B nace al unir las dos piezas: ñróQEqZ;oaÓC;rxY?Äx;pÑj íHmfGÓ

Ambos hijos son evaluados, si superan a los padres en puntaje, entonces esos hijos reemplazan a sus padres.

```
namespace Ejemplo;

internal class Program {
    public static char[] Letras;

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Alfabeto de letras */
        Letras = "abcdefghijklmnopqrstuvwxyz ".ToCharArray();

        /* Cadena original a "adivinar" */
        string Original = "esta es una prueba para dar con una cadena";

        /* Iniciar el proceso */
        Proceso(Azar, Original);
    }

    /* Método que realiza el proceso de "adivinar" la cadena original */
    static void Proceso(Random Azar, string Original) {
        /* Trabaja con arreglos de caracteres */
        char[] OriginalArray = Original.ToCharArray();

        /* Genera población inicial con individuos que tengan la
           misma longitud de la cadena original */
        int TotalIndividuos = 1000;
        char[][] Individuos = new char[TotalIndividuos][];

        int Tamano = OriginalArray.Length;
        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
            Individuos[indiv] = new char[Tamano];
        }
    }
}
```

```

    for (int pos = 0; pos < Tamano; pos++)
        Individuos[indiv][pos] = Letras[Azar.Next(Letras.Length)];
}

int Contador = 0;
int MejorPuntaje = -1;
int MejorIndividuo = -1;

while (true) {

    /* Seleccionar dos individuos aleatoriamente */
    int Indice1 = Azar.Next(TotalIndividuos);
    int Indice2;
    do {
        Indice2 = Azar.Next(TotalIndividuos);
    } while (Indice2 == Indice1);

    /* Genera los hijos */
    char[] HijoA = new char[Tamano];
    int Pos = Azar.Next(Tamano);

    // Copiar desde Padre [0..Pos]
    Array.Copy(Individuos[Indice1], 0, HijoA, 0, Pos + 1);

    // Copiar desde Madre [Pos+1..fin]
    Array.Copy(Individuos[Indice2], Pos + 1, HijoA, Pos + 1, Individuos[Indice2].Length - (Pos + 1));

    char[] HijoB = new char[Tamano];

    // Copiar desde Padre [0..Pos]
    Array.Copy(Individuos[Indice2], 0, HijoB, 0, Pos + 1);

    // Copiar desde Madre [Pos+1..fin]
    Array.Copy(Individuos[Indice1], Pos + 1, HijoB, Pos + 1, Individuos[Indice1].Length - (Pos + 1));

    /* Evalúa cada individuo */

```

```

int Puntaje1 = Puntuar(Individuos[Indice1], OriginalArray);
int Puntaje2 = Puntuar(Individuos[Indice2], OriginalArray);
int PuntajeHijoA = Puntuar(HijoA, OriginalArray);
int PuntajeHijoB = Puntuar(HijoB, OriginalArray);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (PuntajeHijoA > Puntaje1)
    Array.Copy(HijoA, Individuos[Indice1], Individuos[Indice1].Length);
if (PuntajeHijoA > Puntaje2)
    Array.Copy(HijoA, Individuos[Indice2], Individuos[Indice2].Length);

if (PuntajeHijoB > Puntaje1)
    Array.Copy(HijoB, Individuos[Indice1], Individuos[Indice1].Length);
if (PuntajeHijoB > Puntaje2)
    Array.Copy(HijoB, Individuos[Indice2], Individuos[Indice2].Length);

if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

/* Evalúa si un individuo ha acertado a la cadena original */
if (Puntaje1 == OriginalArray.Length || Puntaje2 == OriginalArray.Length) {
    break;
}

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");
}
}

Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");
}

/* Evalúa la similitud entre dos arreglos de caracteres */
static int Puntuar(char[] charArreglo1, char[] charArreglo2) {

```

```
int Similitud = 0;
for (int i = 0; i < charArreglo1.Length; i++) {
    if (charArreglo1[i] == charArreglo2[i]) {
        Similitud++;
    }
}
return Similitud;
}
```

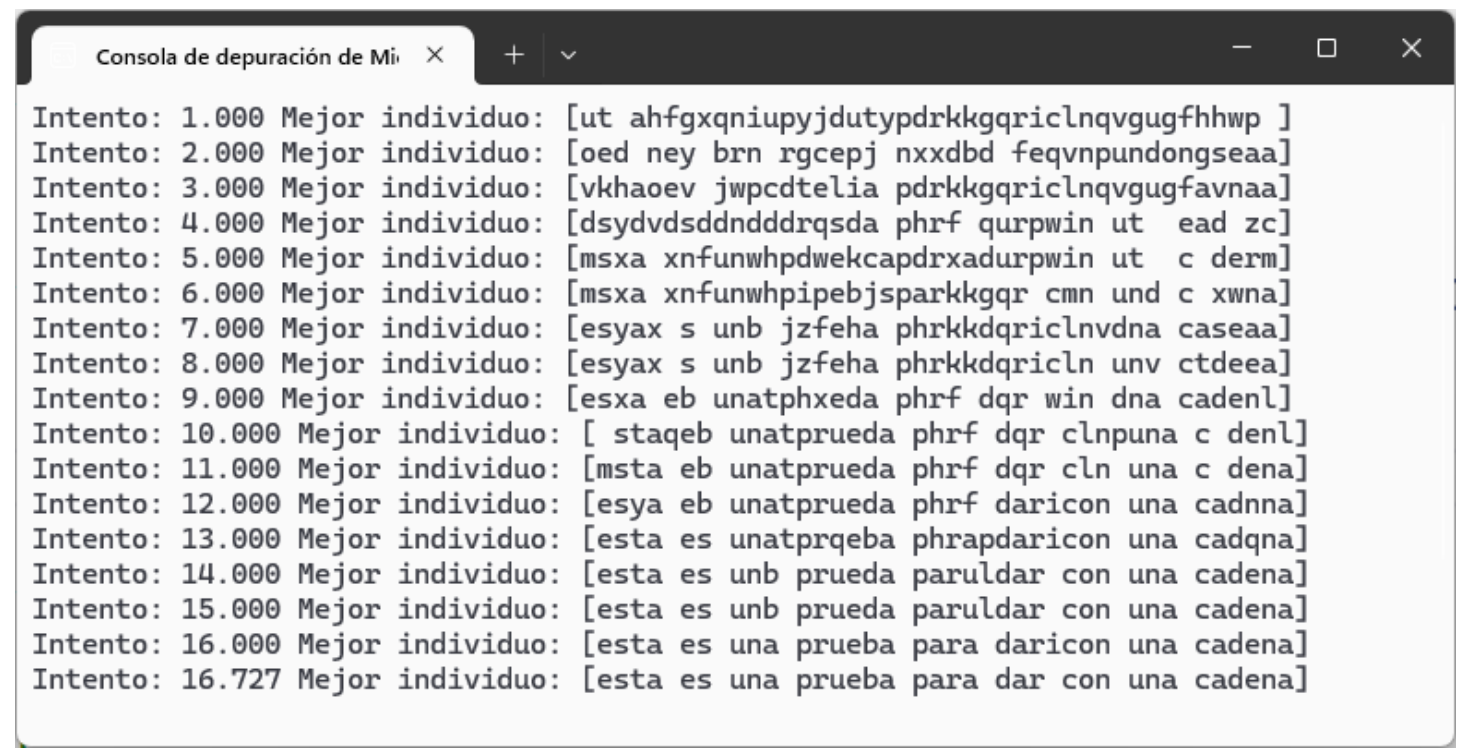


Ilustración 3: Algoritmo evolutivo con el operador cruce

¡OJO! Sucedió que, en ciertas ejecuciones, la simulación no se detenía, la razón es que si en la cadena original, si una letra en una determinada posición no aparece en esa posición en ningún individuo de la población inicial, no hay forma que los cruces logren dar con la cadena original.

Por ejemplo, obsérvese que en la posición 2 (contando desde cero) de la cadena original esta la letra "t". Puede suceder que ningún individuo de la población inicial tenga la letra "t" en la posición 2 y esto hace que no pueda generarse la cadena original.

¿Solución? Aumentar bastante la población minimiza el riesgo (pero no desaparece), sin embargo, el precio a pagar es que se tarda más en dar con la cadena original.



## Combinando los operadores cruce y mutación

Mutar es confiable pero lento, cruzar es rápido, pero hay riesgo de no encontrar la cadena. La solución es combinar ambos operadores.

```
namespace Ejemplo;

internal class Program {
    public static char[] Letras;

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Alfabeto de letras */
        Letras = "abcdefghijklmnopqrstuvwxyz ".ToCharArray();

        /* Cadena original a "adivinar" */
        string Original = "esta es una prueba para dar con una cadena";

        /* Iniciar el proceso */
        Proceso(Azar, Original);
    }

    /* Método que realiza el proceso de "adivinar" la cadena original */
    static void Proceso(Random Azar, string Original) {
        /* Trabaja con arreglos de caracteres */
        char[] OriginalArray = Original.ToCharArray();

        /* Genera población inicial con individuos que tengan la
           misma longitud de la cadena original */
        int TotalIndividuos = 10;
        char[][] Individuos = new char[TotalIndividuos][];

        int Tamano = OriginalArray.Length;
        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
            Individuos[indiv] = new char[Tamano];
        }
    }
}
```

```

    for (int pos = 0; pos < Tamano; pos++)
        Individuos[indiv][pos] = Letras[Azar.Next(Letras.Length)];
}

int Contador = 0;
int MejorPuntaje = -1;
int MejorIndividuo = -1;

while (true) {

    /* Seleccionar dos inviduos aleatoriamente */
    int Indice1 = Azar.Next(TotalIndividuos);
    int Indice2;
    do {
        Indice2 = Azar.Next(TotalIndividuos);
    } while (Indice2 == Indice1);

    /* Genera los hijos */
    char[] HijoA = new char[Tamano];
    int Pos = Azar.Next(Tamano);

    // Copiar desde Padre [0..Pos]
    Array.Copy(Individuos[Indice1], 0, HijoA, 0, Pos + 1);

    // Copiar desde Madre [Pos+1..fin]
    Array.Copy(Individuos[Indice2], Pos + 1, HijoA, Pos + 1, Individuos[Indice2].Length - (Pos + 1));

    char[] HijoB = new char[Tamano];

    // Copiar desde Padre [0..Pos]
    Array.Copy(Individuos[Indice2], 0, HijoB, 0, Pos + 1);

    // Copiar desde Madre [Pos+1..fin]
    Array.Copy(Individuos[Indice1], Pos + 1, HijoB, Pos + 1, Individuos[Indice1].Length - (Pos + 1));

    /* Muta los hijos */

```

```

HijoA[Azar.Next(OriginalArray.Length)] = Letras[Azar.Next(Letras.Length)];
HijoB[Azar.Next(OriginalArray.Length)] = Letras[Azar.Next(Letras.Length)];

/* Evalúa cada individuo */
int Puntaje1 = Puntuar(Individuos[Indice1], OriginalArray);
int Puntaje2 = Puntuar(Individuos[Indice2], OriginalArray);
int PuntajeHijoA = Puntuar(HijoA, OriginalArray);
int PuntajeHijoB = Puntuar(HijoB, OriginalArray);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (PuntajeHijoA > Puntaje1)
    Array.Copy(HijoA, Individuos[Indice1], Individuos[Indice1].Length);
if (PuntajeHijoA > Puntaje2)
    Array.Copy(HijoA, Individuos[Indice2], Individuos[Indice2].Length);

if (PuntajeHijoB > Puntaje1)
    Array.Copy(HijoB, Individuos[Indice1], Individuos[Indice1].Length);
if (PuntajeHijoB > Puntaje2)
    Array.Copy(HijoB, Individuos[Indice2], Individuos[Indice2].Length);

if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

/* Evalúa si un individuo ha acertado a la cadena original */
if (Puntaje1 == OriginalArray.Length || Puntaje2 == OriginalArray.Length) {
    break;
}

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");
}
}

Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{new string(Individuos[MejorIndividuo])}]");

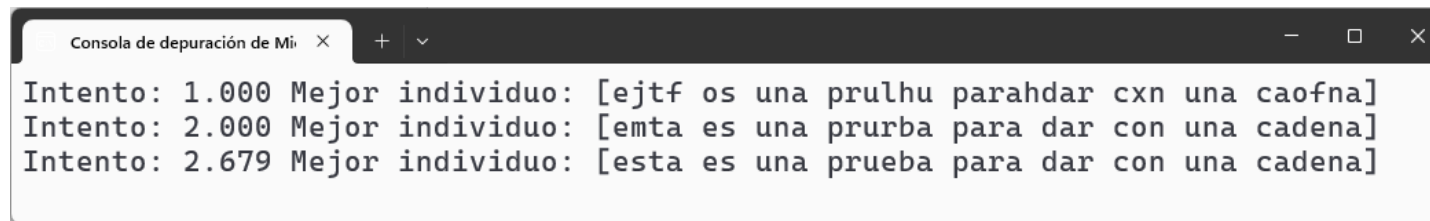
```

```

}

/* Evalúa la similitud entre dos arreglos de caracteres */
static int Puntuar(char[] charArreglo1, char[] charArreglo2) {
    int Similitud = 0;
    for (int i = 0; i < charArreglo1.Length; i++) {
        if (charArreglo1[i] == charArreglo2[i]) {
            Similitud++;
        }
    }
    return Similitud;
}
}

```



```

Consola de depuración de Mi
Intento: 1.000 Mejor individuo: [ejtf os una prulhu parahdar cxn una caofna]
Intento: 2.000 Mejor individuo: [emta es una prurba para dar con una cadena]
Intento: 2.679 Mejor individuo: [esta es una prueba para dar con una cadena]

```

*Ilustración 4: Algoritmo evolutivo: Combinando operador cruce y mutación*

No solo es mucho más rápido encontrando la cadena, es que además es confiable. El ejemplo usó una población muy pequeña inicial, de diez individuos.

## Buscar el máximo en una función

En este segundo problema, dada una función  $y=f(x)$ , un valor inicial para  $x=a$  y un valor final para  $x=b$  donde  $a < b$ , se pide encontrar en cuál valor de " $x$ ", se obtiene el máximo valor de " $y$ " en ese rango  $[a, b]$ . El procedimiento es derivar la función  $y=f(x)$ , igualar a cero y así se obtiene el valor de  $x$ . Pero puede que  $y=f(x)$  sea una ecuación bastante compleja, que complique el derivarla y resolverla a  $y'=0$ . En estos casos es posible aplicar los algoritmos evolutivos.

Por ejemplo, se tiene esta curva:

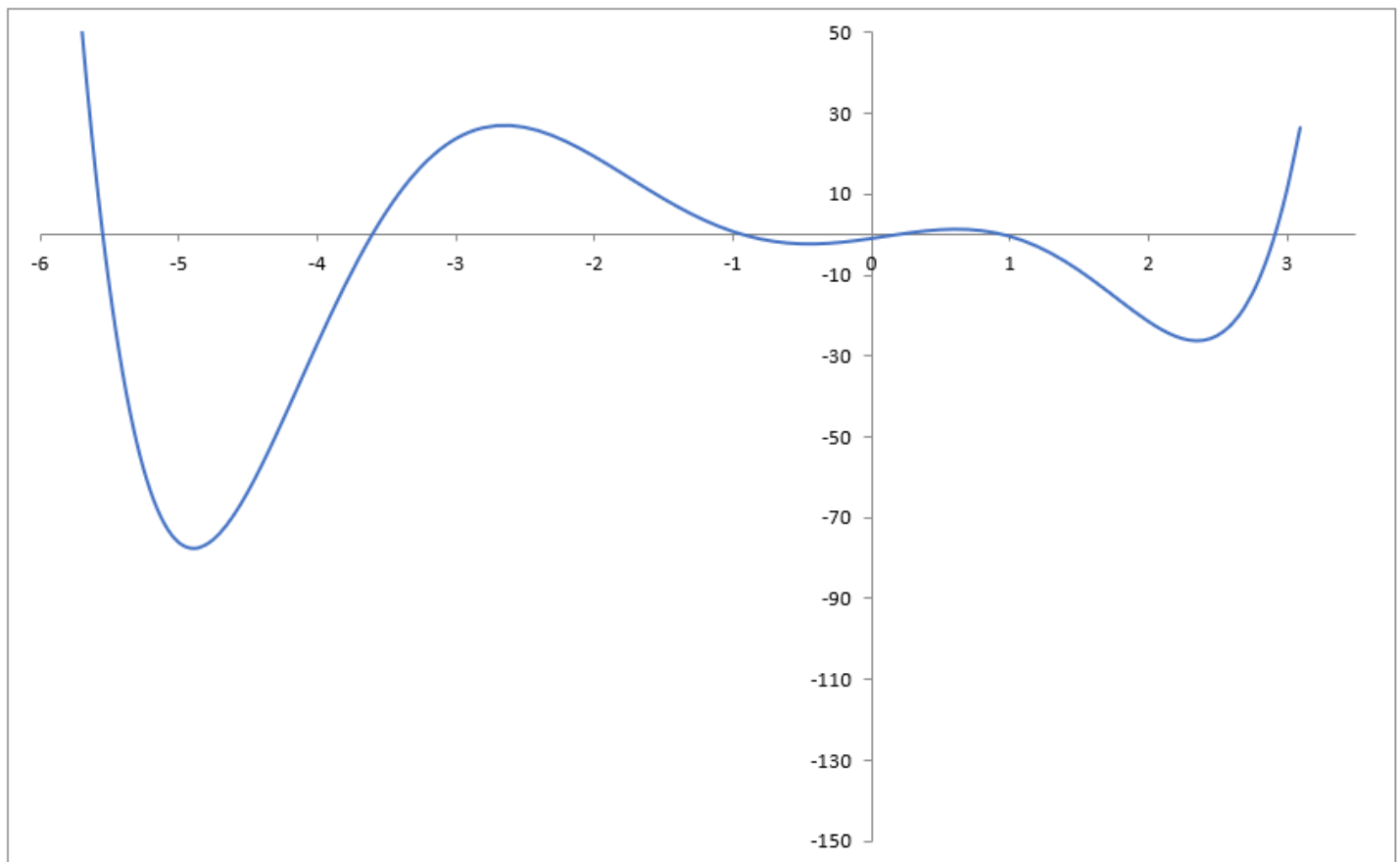


Ilustración 5: Polinomio de grado 6

Para hallar el valor de  $X$  que obtenga el mayor valor de  $Y$  en el rango  $X_{min}$  y  $X_{max}$ . La curva es:

$$Y = 0.1 * X^6 + 0.6 * X^5 - 0.9 * X^4 - 6.2 * X^3 + 2 * X^2 + 5 * X - 1$$

El primer paso con los algoritmos evolutivos es determinar cómo construir al individuo que represente una solución. Una forma de hacerlo es que los individuos sean números reales de doble precisión. Así que se crea una población de varios individuos como números reales aleatorios entre 0 y -1. Aplicando la distribución uniforme entonces esos valores quedarían entre  $a$  y  $b$ .

En cada ciclo del proceso, se escogen dos individuos al azar. Se aplica el operador cruce (que sería el promedio de los dos escogidos), eso genera un hijo, luego a ese hijo se le aplica el operador mutación (un ligero cambio aleatorio). Si el hijo es mejor (genera mayor valor en Y) que sus progenitores, entonces ese hijo sobre escribe a los padres.

El código es el siguiente:

```
namespace Ejemplo;

internal class Program {

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Iniciar el proceso */
        double Xini = -4;
        double Xfin = 1;
        int TotalIndividuos = 100;
        int NumeroCiclos = 10000;
        MaximoValor(Azar, Xini, Xfin, TotalIndividuos, NumeroCiclos);
    }

    /* Método que realiza el proceso de encontrar el valor máximo */
    static void MaximoValor(Random Azar, double Xini, double Xfin, int TotalIndividuos, int NumeroCiclos) {

        /* Genera población inicial con individuos con valores aleatorios */
        double[] Individuos = new double[TotalIndividuos];

        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
            Individuos[indiv] = Azar.NextDouble();
        }

        int Contador = 0;
        double MejorPuntaje = double.MinValue;
        int MejorIndividuo = -1;
        double MejorValorX, MayorValorY;

        for (int ciclos = 1; ciclos <= NumeroCiclos; ciclos++) {
```



```

/* Seleccionar dos individuos aleatoriamente */
int Indice1 = Azar.Next(TotalIndividuos);
int Indice2;
do {
    Indice2 = Azar.Next(TotalIndividuos);
} while (Indice2 == Indice1);

/* Evalúa cada individuo */
double Puntaje1 = Ecuacion(Individuos[Indice1] * (Xfin - Xini) + Xini);
double Puntaje2 = Ecuacion(Individuos[Indice2] * (Xfin - Xini) + Xini);

if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

/* Genera un hijo */
double Hijo = (Individuos[Indice1] + Individuos[Indice2]) / 2.0;

/* Muta el hijo */
Hijo += (Azar.NextDouble() - 0.5) * 0.1;

/* Evalúa el hijo */
double PuntajeHijo = Ecuacion(Hijo * (Xfin - Xini) + Xini);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (PuntajeHijo > Puntaje1)
    Individuos[Indice1] = Hijo;

if (PuntajeHijo > Puntaje2)
    Individuos[Indice2] = Hijo;

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    MejorValorX = Individuos[MejorIndividuo] * (Xfin - Xini) + Xini;
    MayorValorY = Ecuacion(MejorValorX);
}

```

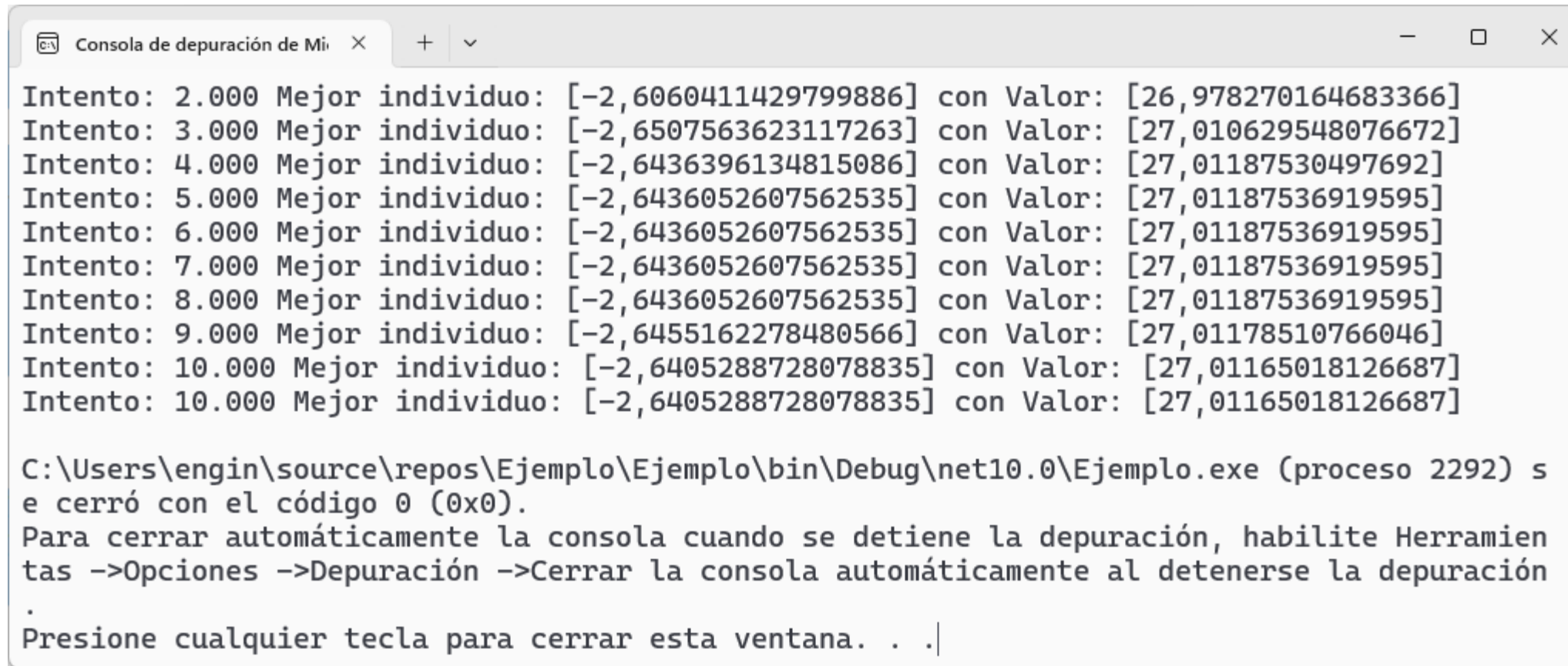
```

        Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorValorX}] con Valor: [{MayorValorY}]");
    }
}

MejorValorX = Individuos[MejorIndividuo] * (Xfin - Xini) + Xini;
MayorValorY = Ecuacion(MejorValorX);
Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorValorX}] con Valor: [{MayorValorY}]");
}

static double Ecuacion(double x) {
    return 0.1 * Math.Pow(x, 6) + 0.6 * Math.Pow(x, 5) + (-0.9 * Math.Pow(x, 4)) - 6.2 * Math.Pow(x, 3) + 2 * x * x + 5 * x - 1;
}
}

```



```
Consola de depuración de Mi  ×  +  ▾  
Intento: 2.000 Mejor individuo: [-2,6060411429799886] con Valor: [26,978270164683366]  
Intento: 3.000 Mejor individuo: [-2,6507563623117263] con Valor: [27,010629548076672]  
Intento: 4.000 Mejor individuo: [-2,6436396134815086] con Valor: [27,01187530497692]  
Intento: 5.000 Mejor individuo: [-2,6436052607562535] con Valor: [27,01187536919595]  
Intento: 6.000 Mejor individuo: [-2,6436052607562535] con Valor: [27,01187536919595]  
Intento: 7.000 Mejor individuo: [-2,6436052607562535] con Valor: [27,01187536919595]  
Intento: 8.000 Mejor individuo: [-2,6436052607562535] con Valor: [27,01187536919595]  
Intento: 9.000 Mejor individuo: [-2,6455162278480566] con Valor: [27,01178510766046]  
Intento: 10.000 Mejor individuo: [-2,6405288728078835] con Valor: [27,01165018126687]  
Intento: 10.000 Mejor individuo: [-2,6405288728078835] con Valor: [27,01165018126687]  
  
C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Debug\net10.0\Ejemplo.exe (proceso 2292) s  
e cerró con el código 0 (0x0).  
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramien  
tas ->Opciones ->Depuración ->Cerrar la consola automáticamente al detenerse la depuración  
.  
Presione cualquier tecla para cerrar esta ventana. . .|
```

Ilustración 6: Buscar el máximo en una función

## De genotipos y fenotipos

En el anterior problema, los individuos fueron números reales de doble precisión en C#, la mutación fue adicionar o sustraer un valor aleatorio. ¿Y si se quiere mayor control en esa modificación para poder aplicar el operador cruce?

En los seres vivos, las instrucciones para su desarrollo y funcionamiento están en el ADN. Esa información genética es el genotipo. La interpretación de ese genotipo es el fenotipo. Un ejemplo grosso modo:

| Genotipo                     | Fenotipo                 |
|------------------------------|--------------------------|
| ACCTGAACTTGGCCAATGGCCTAACCTG | Forma del pico de un ave |

En algoritmos evolutivos se hace una analogía similar: el genotipo es una cadena de números binarios, esta es interpretada, generando un fenotipo (por ejemplo, un valor real) el cuál se evalúa. Si el fenotipo de ese individuo le permite ser el “ganador”, entonces es al genotipo de ese individuo “ganador” al que se le hacen las operaciones de mutación.

Del ejemplo anterior, está la siguiente función:

$$Y = -1.78 * \text{seno}(X)^2 + 6.40 * \text{coseno}(X)^2 + 3.07 * \text{seno}(X)^3$$

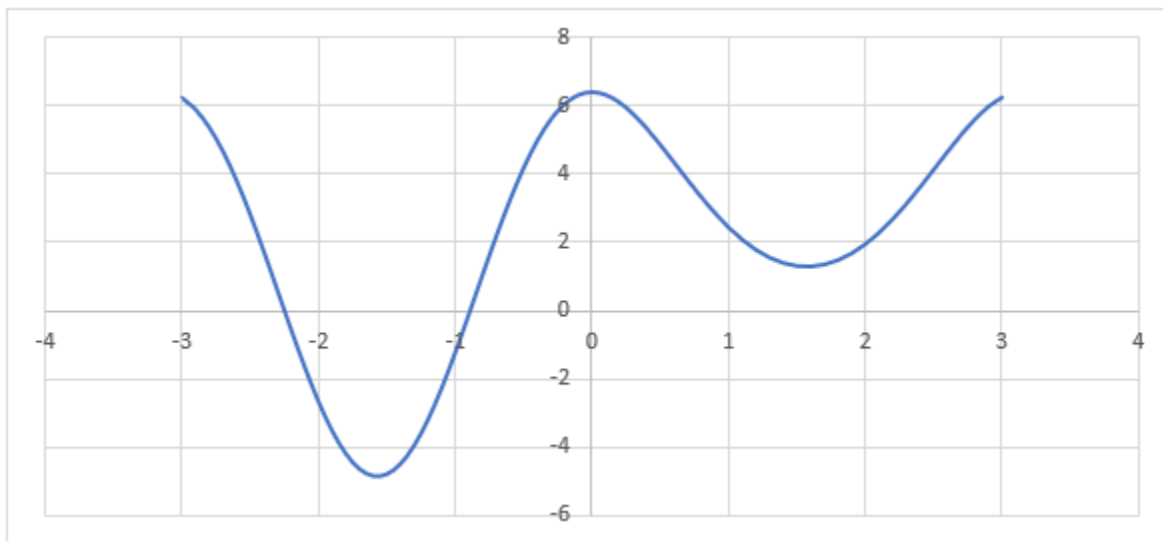


Ilustración 7: Gráfico de una ecuación

Se pide encontrar el valor de “x” entre -3 y 2, donde se obtenga el mínimo valor de “y”. A ojo, viendo la gráfica es posible afirmar que el valor aproximado para x es -1.6, pero eso no es preciso. Se requiere precisión y este sería el proceso:

## Paso 1

Se define que tan largo será el genotipo. Es una cadena de números binarios, entre más cifras, más preciso serán los resultados, pero tomará más tiempo en procesamiento. Una cadena puede ser: 011101101

Para el ejemplo, el valor mínimo -3 será representado por "000000000" (9 cifras) y el valor máximo 2 será representado por "111111111". ¿Cuántas combinaciones hay? Es  $2^{\text{total\_cifras}}$ , luego en este caso concreto sería  $2^9 = 512$ . ¿Qué significa eso? Que se va a dividir en 512 partes iguales el rango entre -3 y 2, y es en una de esas partes, donde estará el valor de "x" que se obtiene el mínimo valor de "y".

Nota aclaratoria: El rango mínimo sin importar su valor será representado como "000000000" y el rango máximo sin importar su valor será representado como "111111111". ¡No se debe hacer la conversión valor binario a valor decimal aquí!

## Paso 2

Interpretando el genotipo. Si -3 será "000000000" y 2 será "111111111", entonces ¿Qué sería, por ejemplo, "011011101"? ¿A qué valor x correspondería? Así se calcula:

$$x = Xmin + \text{binarioadecimal} * \frac{Xmax - Xmin}{2^{\text{TotalCifras}} - 1}$$

Probando con "000000000" (literal es un 0 en decimal)

$$x = -3 + 0 * \frac{2 - -3}{2^9 - 1} = -3 + 0 * \frac{5}{512 - 1} = -3 + 0 * \frac{5}{511} = -3$$

Probando con "111111111" (literal es un 511 en decimal)

$$x = -3 + 511 * \frac{2 - -3}{2^9 - 1} = -3 + 511 * \frac{5}{511} = -3 + 5 = 2$$

Probando con "011011101" (literal es un 221 en decimal)

$$x = -3 + 221 * \frac{2 - -3}{2^9 - 1} = -3 + 221 * \frac{5}{511} = -0,837573385518591$$

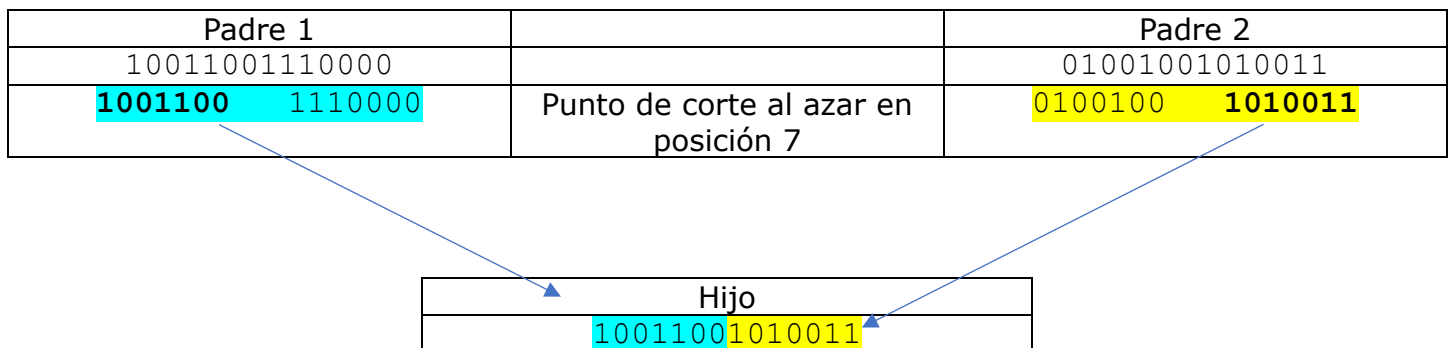
### Paso 3

La representación binaria de un individuo da como ventaja la facilidad de implementar el operador mutación, porque es ir a una determinada posición de la cadena binaria y si esa tenía un "0" se cambia a "1" y viceversa. Por otro lado, la totalidad del individuo puede cambiarse porque cualquier posición de la cadena binaria está al alcance de ese operador. Por ejemplo:

|                                 |                         |
|---------------------------------|-------------------------|
| Individuo                       | 10011001110000          |
| Posición al azar 9 que es       | 10011001 <b>1</b> 10000 |
| Cambiando valor de esa posición | 10011001 <b>0</b> 10000 |

### Paso 4

Otra ventaja de la representación binaria es la facilidad de implementar el operador cruce, porque es simplemente dividir la cadena de los padres, combinar esos pedazos para dar origen al nuevo individuo. Por ejemplo:



### Paso 5

¿Cómo el lenguaje de programación elegido para implementar el algoritmo evolutivo puede operar con valores binarios? ¿Tendrá operadores de fácil uso? ¿El desempeño será bueno? C# permite operaciones a nivel de bit, la cadena binaria estaría almacenada en una variable de tipo entero.

### Paso 6

Definir una población. Los algoritmos evolutivos inician con una población inicial de individuos generados al azar. Siguiendo el enfoque anterior, la solución a esto es una lista de números de tipo entero que serán tratados como cadenas de bits.

El algoritmo sería el siguiente:

**Algoritmo BuscaXparaMinimoValorY**

**Inicio**

Generar población de N individuos al azar (cadenas en binario)

**Inicio ciclo**

Seleccionar al azar dos individuos de esa población: A y B

Generar Hijo cruzando los genes de A y B

Mutar Hijo

Evaluar valorY generado por el individuo A

Evaluar valorY generado por el individuo B

Evaluar valorY generado por el Hijo

**Si** valorY de Hijo es mayor que valorY de A **entonces** Hijo reemplaza a A

**Si** valorY de Hijo es mayor que valorY de B **entonces** Hijo reemplaza a B

**Fin ciclo**

Buscar individuo que genere mayor Y de la población

Imprimir individuo

**Fin**

```
namespace Ejemplo;

internal class Program {

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Iniciar el proceso */
        double Xini = -4;
        double Xfin = 1;
        int TotalIndividuos = 100;
        int NumeroCiclos = 10000;
        int TotalBits = 10; //Número de bits para representar cada individuo
        MaximoValor(Azar, Xini, Xfin, TotalIndividuos, NumeroCiclos, TotalBits);
    }

    /* Método que realiza el proceso de encontrar el valor máximo */
    static void MaximoValor(Random Azar, double Xini, double Xfin, int TotalIndividuos, int NumeroCiclos, int TotalBits) {

        /* Genera población inicial con individuos con valores aleatorios */
        int[] Individuos = new int[TotalIndividuos];

        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
            Individuos[indiv] = Azar.Next((int)Math.Pow(2, TotalBits));
        }

        int Contador = 0;
        double MejorPuntaje = double.MinValue;
        int MejorIndividuo = -1;
        double MejorValorX, MayorValorY;
```



```

//El factor de conversión
double Divide = Math.Pow(2, TotalBits) - 1;
double Factor = (Xfin - Xini) / Divide;

for (int ciclos = 1; ciclos <= NumeroCiclos; ciclos++) {

    /* Seleccionar dos individuos aleatoriamente */
    int Indice1 = Azar.Next(TotalIndividuos);
    int Indice2;
    do {
        Indice2 = Azar.Next(TotalIndividuos);
    } while (Indice2 == Indice1);

    /* Evalúa cada individuo */
    double Puntaje1 = Ecuacion(Xini + Individuos[Indice1] * Factor);
    double Puntaje2 = Ecuacion(Xini + Individuos[Indice2] * Factor);

    if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
    if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

    /* Genera un hijo con operador cruce */

    //En que posicion corta el genotipo de cada padre
    int Posicion = Azar.Next(sizeof(int) * 8); //Entero por los 8 bits de un byte

    //Extrae las partes de cada progenitor
    int Mascara = (1 << Posicion) - 1;
    int ParteA = Individuos[Indice1] >> Posicion;
    int ParteB = Individuos[Indice2] & Mascara;

    //Une las partes: inicial de A y la final de B
    int Hijo = (ParteA << Posicion) | ParteB;

    /* Muta el hijo */
    Mascara = 1 << Azar.Next(TotalBits);
    Hijo ^= Mascara;

```

```

/* Evalúa el hijo */
double PuntajeHijo = Ecuacion(Xini + Hijo * Factor);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (PuntajeHijo > Puntaje1)
    Individuos[Indice1] = Hijo;

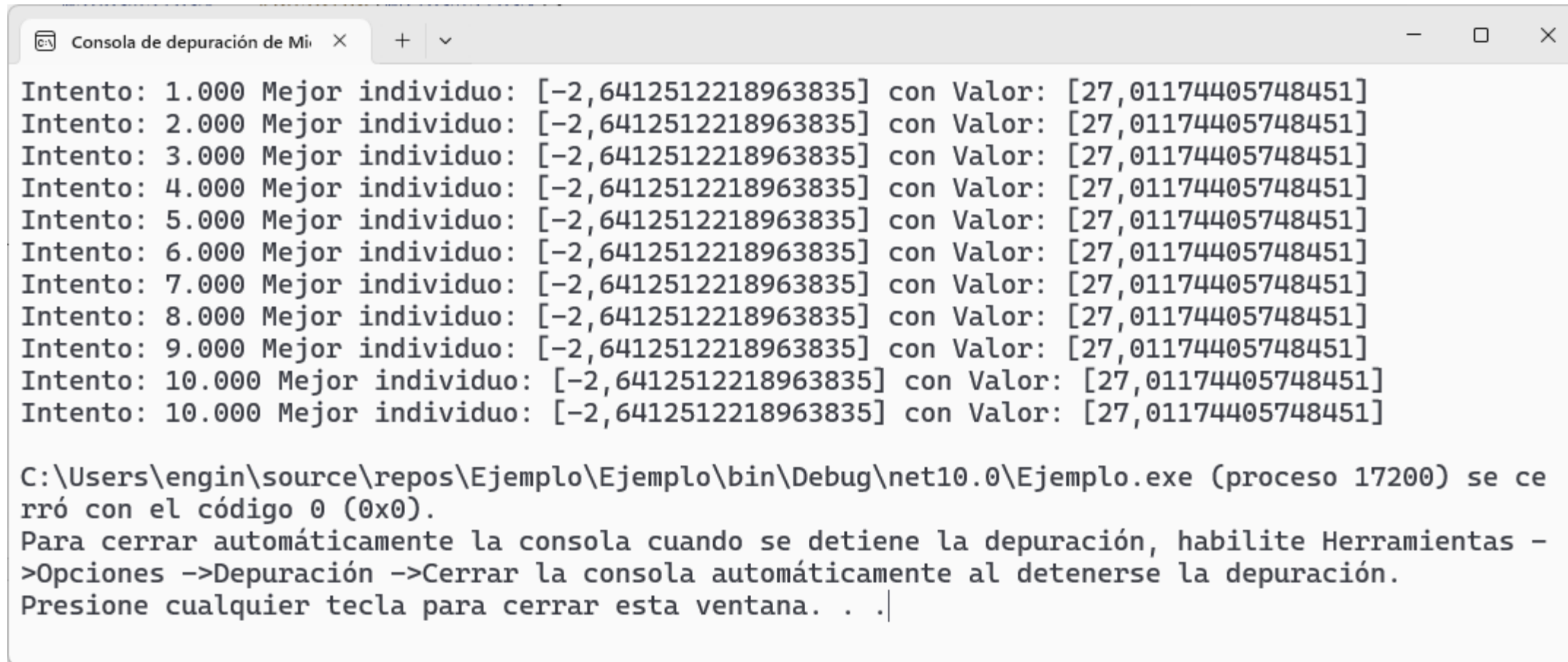
if (PuntajeHijo > Puntaje2)
    Individuos[Indice2] = Hijo;

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    MejorValorX = Xini + Individuos[MejorIndividuo] * Factor;
    MayorValorY = Ecuacion(MejorValorX);
    Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorValorX}] con Valor: [{MayorValorY}]");
}
}

MejorValorX = Xini + Individuos[MejorIndividuo] * Factor;
MayorValorY = Ecuacion(MejorValorX);
Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorValorX}] con Valor: [{MayorValorY}]");
}

static double Ecuacion(double x) {
    return 0.1 * Math.Pow(x, 6) + 0.6 * Math.Pow(x, 5) + (-0.9 * Math.Pow(x, 4)) - 6.2 * Math.Pow(x, 3) + 2 * x * x + 5 * x - 1;
}
}

```



```
Consola de depuración de Mi  X + v
Intento: 1.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 2.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 3.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 4.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 5.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 6.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 7.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 8.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 9.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 10.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]
Intento: 10.000 Mejor individuo: [-2,6412512218963835] con Valor: [27,01174405748451]

C:\Users\engin\source\repos\Ejemplo\Ejemplo\bin\Debug\net10.0\Ejemplo.exe (proceso 17200) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .|
```

Ilustración 8: Operador cruce y mutación

## Máximos y mínimos locales

En el siguiente gráfico se puede apreciar visualmente un problema con los algoritmos evolutivos. Se busca obtener el valor de X con el que se obtiene el mayor valor de Y. Los círculos rellenos representan diferentes individuos generados al azar dentro de la población.

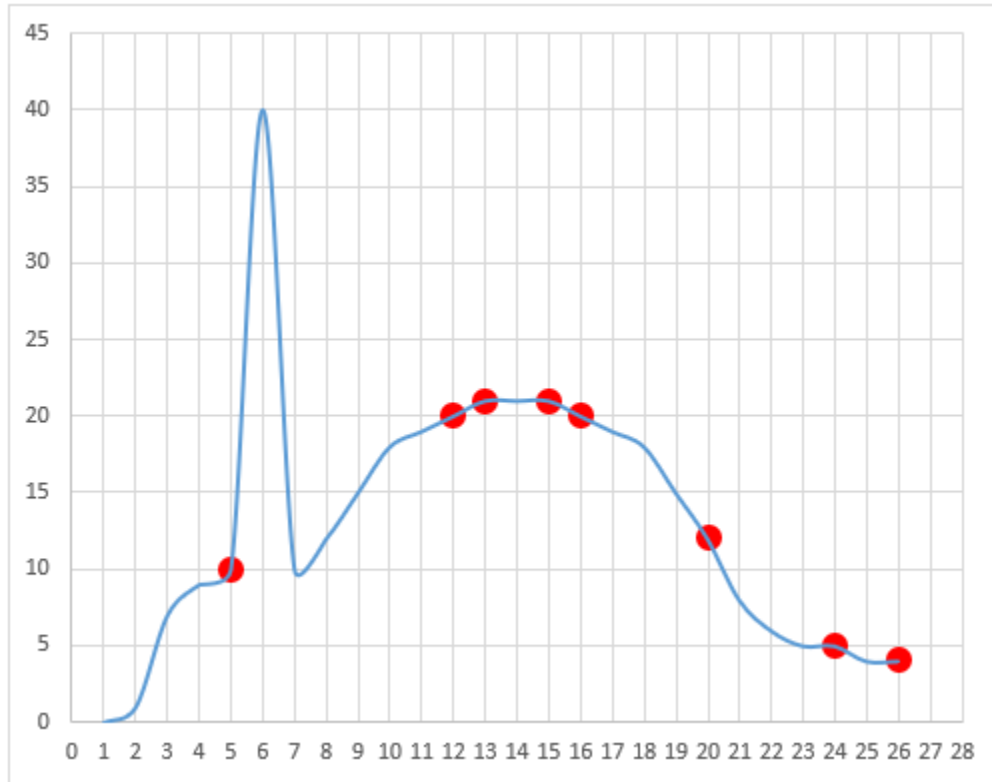


Ilustración 9: Gráfico matemático

Visualmente, el máximo "Y" se encuentra entre los valores de  $5 \leq X \leq 7$ . Hay muchos individuos entre  $11 \leq X \leq 17$  que al competir contra el único individuo en la posición  $X=5$ , ganarían y se reproducirían eliminando de la población al individuo que en verdad estaba cerca de la solución global. El algoritmo evolutivo dará más puntaje a los individuos entre  $11 \leq X \leq 17$ , y deduce erróneamente que el máximo está entre  $11 \leq X \leq 17$ . Eso se le conoce como máximo local y una vez que el algoritmo evolutivo privilegie a los individuos de ese máximo local, se borrará cualquier esperanza de que se llegue al máximo global real.

¿Cómo solucionarlo? La variedad de los individuos dentro de la población debe mantenerse, pero no es fácil porque el mismo algoritmo premia a los ganadores con reproducirse y a los perdedores los castiga con la eliminación, así al perdedor le faltase muy poco para dar con la respuesta correcta.

El mismo problema se tendría con un mínimo local y un mínimo global.

Técnicas para evitar caer en máximos y mínimos locales:

1. Ejecutar varias veces el algoritmo evolutivo.

2. Tener varias poblaciones separadas.
3. Cada cierto número de ciclos generar aleatoriamente algunos individuos que reemplacen a algunos de la población existente.
4. Hacer uso del operador cruce que asemeja a la reproducción sexual, la cual es una técnica utilizada en la naturaleza para mantener la variabilidad.

## Buscar el mayor valor en una ecuación de múltiples variables

Si hay una ecuación del tipo:

$$Y = F(a, b, c, d, e)$$

Donde a, b, c, d, e son variables independientes. ¿Cómo encontrar el mayor valor de Y si nos dan un rango en el que oscilan esas variables independientes? Este es un problema sencillo de solucionar, pero difícil de llevarlo a la práctica, porque se tendrían los siguientes ciclos anidados:

```
for (a = Minimo; a <= Maximo; a += 0.001)
  for (b = Minimo; b <= Maximo; b += 0.001)
    for (c = Minimo; c <= Maximo; c += 0.001)
      for (d = Minimo; d <= Maximo; d += 0.001)
        for (e = Minimo; e <= Maximo; e += 0.001)
```

Al terminar de ejecutar se obtendría el mayor valor de Y, pero haciendo cuentas, suponiendo que cada ciclo itera unas 1000 veces, se tiene que el total de iteraciones es  $1000^5 = 1.000.000.000.000.000$  de veces, una cantidad enorme que puede consumir mucho tiempo. ¿Y si requiere mayor precisión? ¿Y si son más variables? En esos casos hay un gran problema.

Los algoritmos evolutivos ofrecen una solución muy buena (no perfecta) para encontrar el mayor valor, el algoritmo es el siguiente:

**Algoritmo BuscaXparaMinimoValorY****Inicio**

Generar población de N individuos al azar (cadenas en binario)

**Inicio ciclo**

Seleccionar al azar dos individuos de esa población: A y B

Generar Hijo cruzando los genes de A y B

Mutar Hijo

Evaluar valorY generado por el individuo A

Evaluar valorY generado por el individuo B

Evaluar valorY generado por el Hijo

**Si** valorY de Hijo es mayor que valorY de A **entonces** Hijo reemplaza a A

**Si** valorY de Hijo es mayor que valorY de B **entonces** Hijo reemplaza a B

**Fin ciclo**

Buscar individuo que genere mayor Y de la población

Imprimir individuo

**Fin**

```
namespace Ejemplo;

internal class Program {

    static void Main() {
        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Iniciar el proceso */
        double MinValor = -10;
        double MaxValor = 10;
        int TotalIndividuos = 100;
        int NumeroCiclos = 50000;
        int TotalBits = 10; //Número de bits para representar cada individuo
        int NumeroVariables = 5; //Número de variables en la ecuación
        MaximoValor(Azar, MinValor, MaxValor, TotalIndividuos, NumeroCiclos, TotalBits, NumeroVariables);
    }

    /* Método que realiza el proceso de encontrar el valor máximo */
    static void MaximoValor(Random Azar, double MinValor, double MaxValor, int TotalIndividuos, int NumeroCiclos, int TotalBits, int
NumeroVariables) {

        /* Genera población inicial con individuos con valores aleatorios */
        int[][] Individuos = new int[TotalIndividuos][];

        for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
            Individuos[indiv] = new int[NumeroVariables];
            for (int varInterna = 0; varInterna < NumeroVariables; varInterna++) {
                Individuos[indiv][varInterna] = Azar.Next((int)Math.Pow(2, TotalBits));
            }
        }
    }
}
```



```

int Contador = 0;
double MejorPuntaje = double.MinValue;
int MejorIndividuo = -1;
double MayorValorY;

//El factor de conversión
double Divide = Math.Pow(2, TotalBits) - 1;
double Factor = (MaxValor - MinValor) / Divide;

for (int ciclos = 1; ciclos <= NumeroCiclos; ciclos++) {

    /* Seleccionar dos individuos aleatoriamente */
    int Indice1 = Azar.Next(TotalIndividuos);
    int Indice2;
    do {
        Indice2 = Azar.Next(TotalIndividuos);
    } while (Indice2 == Indice1);

    /* Evalúa cada individuo */
    double Puntaje1 = Ecuacion(Individuos[Indice1], MinValor, Factor);
    double Puntaje2 = Ecuacion(Individuos[Indice2], MinValor, Factor);

    if (Puntaje1 > MejorPuntaje) { MejorPuntaje = Puntaje1; MejorIndividuo = Indice1; }
    if (Puntaje2 > MejorPuntaje) { MejorPuntaje = Puntaje2; MejorIndividuo = Indice2; }

    /* Genera un hijo con operador cruce */
    int[] Hijo = new int[NumeroVariables];

    int PuntoCruce = Azar.Next(NumeroVariables);
    for (int varInterna = 0; varInterna < NumeroVariables; varInterna++) {
        if (varInterna <= PuntoCruce)
            Hijo[varInterna] = Individuos[Indice1][varInterna];
        else
            Hijo[varInterna] = Individuos[Indice2][varInterna];
    }
}

```

```

/* Muta el hijo */
int PuntoMutacion = Azar.Next(NumeroVariables);
Hijo[PuntoMutacion] = Azar.Next((int)Math.Pow(2, TotalBits));

/* Evalúa el hijo */
double PuntajeHijo = Ecuacion(Hijo, MinValor, Factor);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (PuntajeHijo > Puntaje1)
    Individuos[Indice1] = Hijo;

if (PuntajeHijo > Puntaje2)
    Individuos[Indice2] = Hijo;

/* Incrementar el contador e informar cada 1000 intentos */
Contador++;
if (Contador % 1000 == 0) {
    MayorValorY = Ecuacion(Individuos[MejorIndividuo], MinValor, Factor);
    Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorIndividuo}] con Valor: [{MayorValorY}]");
}
}

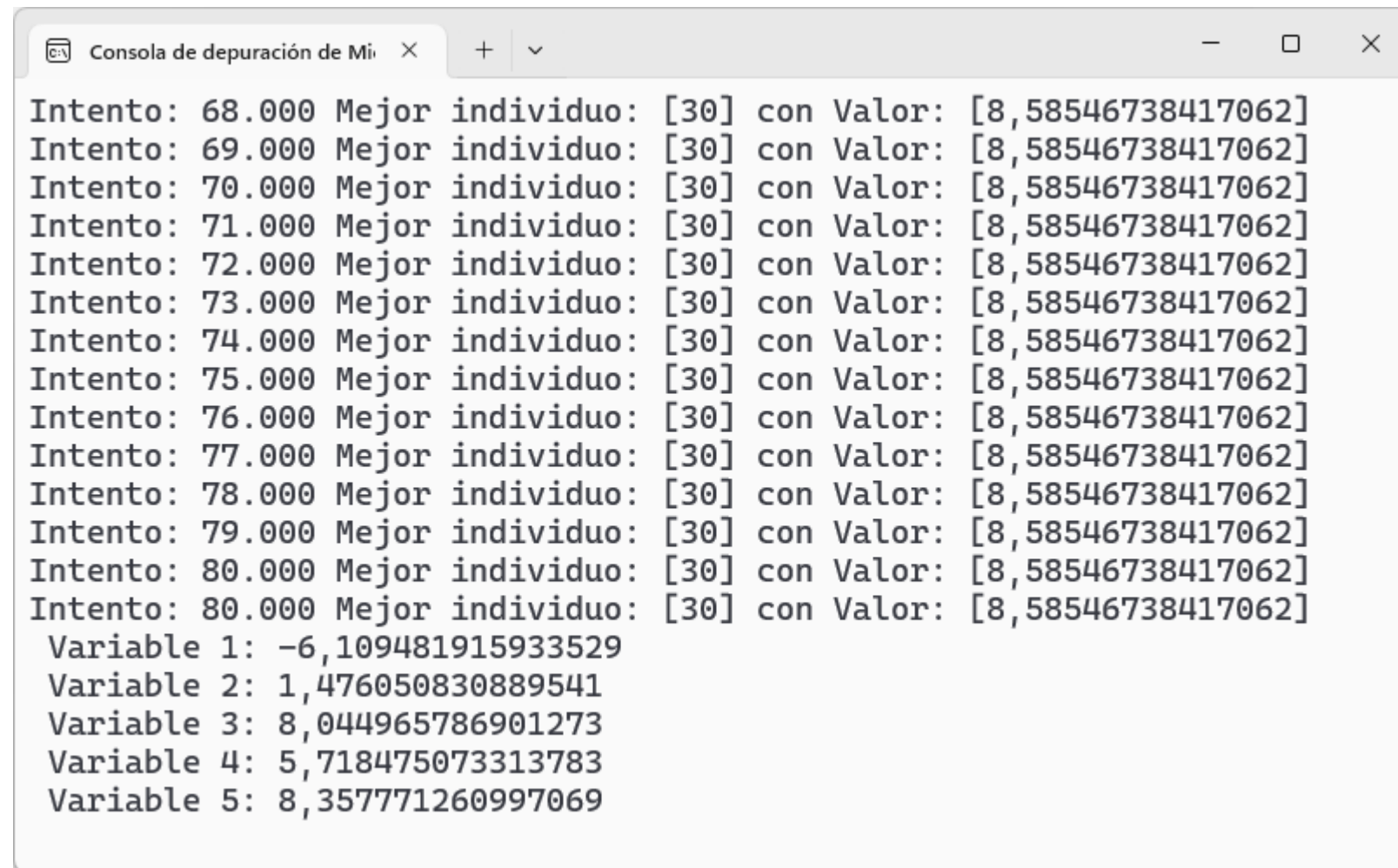
MayorValorY = Ecuacion(Individuos[MejorIndividuo], MinValor, Factor);
Console.WriteLine($"Intento: {Contador:N0} Mejor individuo: [{MejorIndividuo}] con Valor: [{MayorValorY}]");
}

static double Ecuacion(int[] Variables, double MinValor, double Factor) {
    double a = MinValor + Variables[0] * Factor;
    double b = MinValor + Variables[1] * Factor;
    double c = MinValor + Variables[2] * Factor;
    double d = MinValor + Variables[3] * Factor;
    double e = MinValor + Variables[4] * Factor;

    return 0.3 * Math.Sin(a * c - d) +
        1.7 * Math.Sin(e * b + c) +
        2.8 * Math.Cos(3.1 * b - 4.4 * a) -

```

```
    3.1 * Math.Sin(a * d - e * c) +  
    0.7 * Math.Cos(a + b * c - d);  
}  
}
```



```
Consola de depuración de Mi  
Intento: 68.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 69.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 70.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 71.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 72.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 73.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 74.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 75.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 76.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 77.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 78.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 79.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 80.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Intento: 80.000 Mejor individuo: [30] con Valor: [8,58546738417062]  
Variable 1: -6,109481915933529  
Variable 2: 1,476050830889541  
Variable 3: 8,044965786901273  
Variable 4: 5,718475073313783  
Variable 5: 8,357771260997069
```

Ilustración 10: Buscar el mayor valor de Y con múltiples variables independientes

## Simplificar una ecuación

Dada una ecuación original del tipo:

$$Y = a * \textit{seno}(b * X + c) + d * \textit{seno}(e * X + f) + \dots + p * \textit{seno}(q * X + r)$$

Como se puede observar, tiene varios bloques del tipo:

$$Y = a * \textit{seno}(b * X + c)$$

¿Será posible generar una ecuación que tenga menos bloques que se acerque al comportamiento de la ecuación original? De esa forma se simplifican los cálculos y se hacen más rápido.

Se hace uso de los algoritmos evolutivos. En el ejemplo, la ecuación original tiene 30 bloques, y se busca una ecuación de 7 bloques que se acerque lo más posible a lo que hace la de 30.

```

using System.Diagnostics;

/* Algoritmo evolutivo para simplificar ecuaciones
 * Autor: Rafael Alberto Moreno Parra
 *
 * Problema:
 * Dada una ecuación compleja del tipo
 *  $Y = a \cdot \text{seno}(b \cdot X + c) + \dots + p \cdot \text{seno}(q \cdot X + r) + \dots$ 
 * Con múltiples sumandos
 *
 * donde X es la variable independiente y
 * Y la variable dependiente, hallar una función
 * que simplifique la ecuación anterior.
 *
 * Solución:
 * Usar un algoritmo evolutivo para dar con esa
 * función que se acerque al comportamiento de la
 * función compleja.
 * ¿Cómo?
 * La ecuación compleja genera una serie de datos, luego
 * se busca la ecuación simple que se acerque a esa serie
 * de datos.
 * */

namespace Ejemplo;

/* Cada individuo es una solución posible */
public class Individuo {
    const double Grado_A_Radian = Math.PI / 180.0;
    public double Distancia;
    public double[] Coef;

```

```

public Individuo(int TotalBloques, Random Azar, double CoefMinimo, double CoefMaximo) {
    Distancia = -1;
    Coef = new double[TotalBloques * 3];

    double Multiplica = CoefMaximo - CoefMinimo;
    for (int cont = 0; cont < Coef.Length; cont += 3) {
        Coef[cont] = Azar.NextDouble() * Multiplica + CoefMinimo;
        Coef[cont + 1] = (Azar.NextDouble() * Multiplica + CoefMinimo) * Grado_A_Radian;
        Coef[cont + 2] = (Azar.NextDouble() * Multiplica + CoefMinimo) * Grado_A_Radian;
    }
}

/* Aquí se hace el proceso evolutivo */
internal class Program {

    //Valores X de entrada
    public static double[] Xentrada;

    //Valores Y generados por la ecuación
    public static double[] Yesperado;

    static void Main() {
        //Toma el tiempo
        Stopwatch temporizador = new();
        temporizador.Reset();
        temporizador.Start();

        /* Único generador de números aleatorios para todo el programa */
        Random Azar = new();

        /* Genera el dataset de la ecuación compleja */
        int BloquesDataset = 30;
        double CoefMinimo = -3;
        double CoefMaximo = 3;
    }
}

```

```

double Xmin = -720;
double Xmax = 720;
int TotalDatos = 100;
GeneraDataset(Azar, BloquesDataset, CoefMinimo, CoefMaximo, Xmin, Xmax, TotalDatos);

/* Buscar la ecuación más simple */
int TotalIndividuos = 400;
int NumeroCiclos = 4000000;
int BloquesMinimo = 7;
BuscaEcuacion(Azar, BloquesMinimo, CoefMinimo, CoefMaximo, TotalIndividuos, NumeroCiclos);

temporizador.Stop();
Console.WriteLine("Tiempo tomado: " + temporizador.ElapsedMilliseconds);

/* Datos */
Console.WriteLine("\r\nConfiguración dataset original");
Console.WriteLine("Número de bloques originales: " + BloquesDataset);
Console.WriteLine("Valor mínimo de los coeficientes: " + CoefMinimo);
Console.WriteLine("Valor máximo de los coeficientes: " + CoefMaximo);
Console.WriteLine("X mínimo: " + Xmin);
Console.WriteLine("X máximo: " + Xmax);
Console.WriteLine("Total datos generados: " + TotalDatos);

Console.WriteLine("\r\nConfiguración población");
Console.WriteLine("Total individuos: " + TotalIndividuos);
Console.WriteLine("Número de ciclos: " + NumeroCiclos);
Console.WriteLine("Número de bloques de cada individuo: " + BloquesMinimo);
}

/* Genera los datos de la ecuación compleja */
static void GeneraDataset(Random Azar, int Bloques, double CoefMinimo, double CoefMaximo, double Xmin, double Xmax, int
TotalDatos) {
    /* Genera coeficientes aleatorios */
    Individuo Ambiente = new(Bloques, Azar, CoefMinimo, CoefMaximo);

    /* Genera valores X e Y */

```

```

Xentrada = new double[TotalDatos];
for (int Cont = 0; Cont < TotalDatos; Cont++)
    Xentrada[Cont] = Azar.NextDouble() * (Xmax - Xmin) + Xmin;
Xentrada.Sort();

Yesperado = new double[TotalDatos];
for (int Cont = 0; Cont < TotalDatos; Cont++) {
    Yesperado[Cont] = Ecuacion(Ambiente, Xentrada[Cont]);
}
}

/* Método que realiza el proceso de encontrar el valor máximo */
static void BuscaEcuacion(Random Azar, int Bloques, double CoefMinimo, double CoefMaximo, int TotalIndividuos, int NumeroCiclos)
{

    /* Genera población inicial con coeficientes aleatorios */
    Individuo[] Individuos = new Individuo[TotalIndividuos];
    Individuo Hijo = new(Bloques, Azar, CoefMinimo, CoefMaximo);

    for (int indiv = 0; indiv < TotalIndividuos; indiv++) {
        Individuos[indiv] = new Individuo(Bloques, Azar, CoefMinimo, CoefMaximo);
    }

    double MejorDistancia = double.MaxValue;
    int MejorIndividuo = -1;
    int numCoef = Bloques * 3;

    for (int ciclos = 1; ciclos <= NumeroCiclos; ciclos++) {

        /* Seleccionar dos individuos aleatoriamente */
        int Indice1 = Azar.Next(TotalIndividuos);
        int Indice2;
        do {
            Indice2 = Azar.Next(TotalIndividuos);
        } while (Indice2 == Indice1);
    }
}

```



```

var Indiv1 = Individuos[Indice1];
var Indiv2 = Individuos[Indice2];

/* Evalúa cada individuo */
Distancia(Indiv1);
Distancia(Indiv2);

if (Indiv1.Distancia < MejorDistancia) {
    MejorDistancia = Indiv1.Distancia;
    MejorIndividuo = Indice1;
}

if (Indiv2.Distancia < MejorDistancia) {
    MejorDistancia = Indiv2.Distancia;
    MejorIndividuo = Indice2;
}

/* Genera un hijo con operador cruce (se cruzan los coeficientes) */
var coef1 = Indiv1.Cof;
var coef2 = Indiv2.Cof;
var coefHijo = Hijo.Cof;

int puntoCruce = Azar.Next(numCof);
Array.Copy(coef1, 0, coefHijo, 0, puntoCruce + 1);
Array.Copy(coef2, puntoCruce + 1, coefHijo, puntoCruce + 1, numCof - (puntoCruce + 1));

/* Muta el hijo */
int PuntoMutacion = Azar.Next(numCof);
coefHijo[PuntoMutacion] += Azar.NextDouble() * 2 - 1;

/* Evalúa el hijo */
Hijo.Distancia = -1;
Distancia(Hijo);

/* Si el hijo es mejor que algún progenitor, entonces se sobre-escribe el progenitor */
if (Hijo.Distancia < Indiv1.Distancia) {

```

```

        Array.Copy(coefHijo, coef1, coefHijo.Length);
        Indiv1.Distance = Hijo.Distance;
    }

    if (Hijo.Distance < Indiv2.Distance) {
        Array.Copy(coefHijo, coef2, coefHijo.Length);
        Indiv2.Distance = Hijo.Distance;
    }

    /* Informar cada 100000 intentos */
    if (ciclos % 100000 == 0) {
        Console.WriteLine($"Intento: {ciclos:N0} Mejor individuo: [{MejorIndividuo}] con Valor:
[{{Individuos[MejorIndividuo].Distance}}]");
    }
}

Console.WriteLine($"Mejor individuo: [{MejorIndividuo}] con Valor: [{Individuos[MejorIndividuo].Distance}]");
for (int varInterna = 0; varInterna < Individuos[MejorIndividuo].Coef.Length; varInterna++) {
    Console.WriteLine($" Variable {varInterna + 1}: {Individuos[MejorIndividuo].Coef[varInterna]}");
}

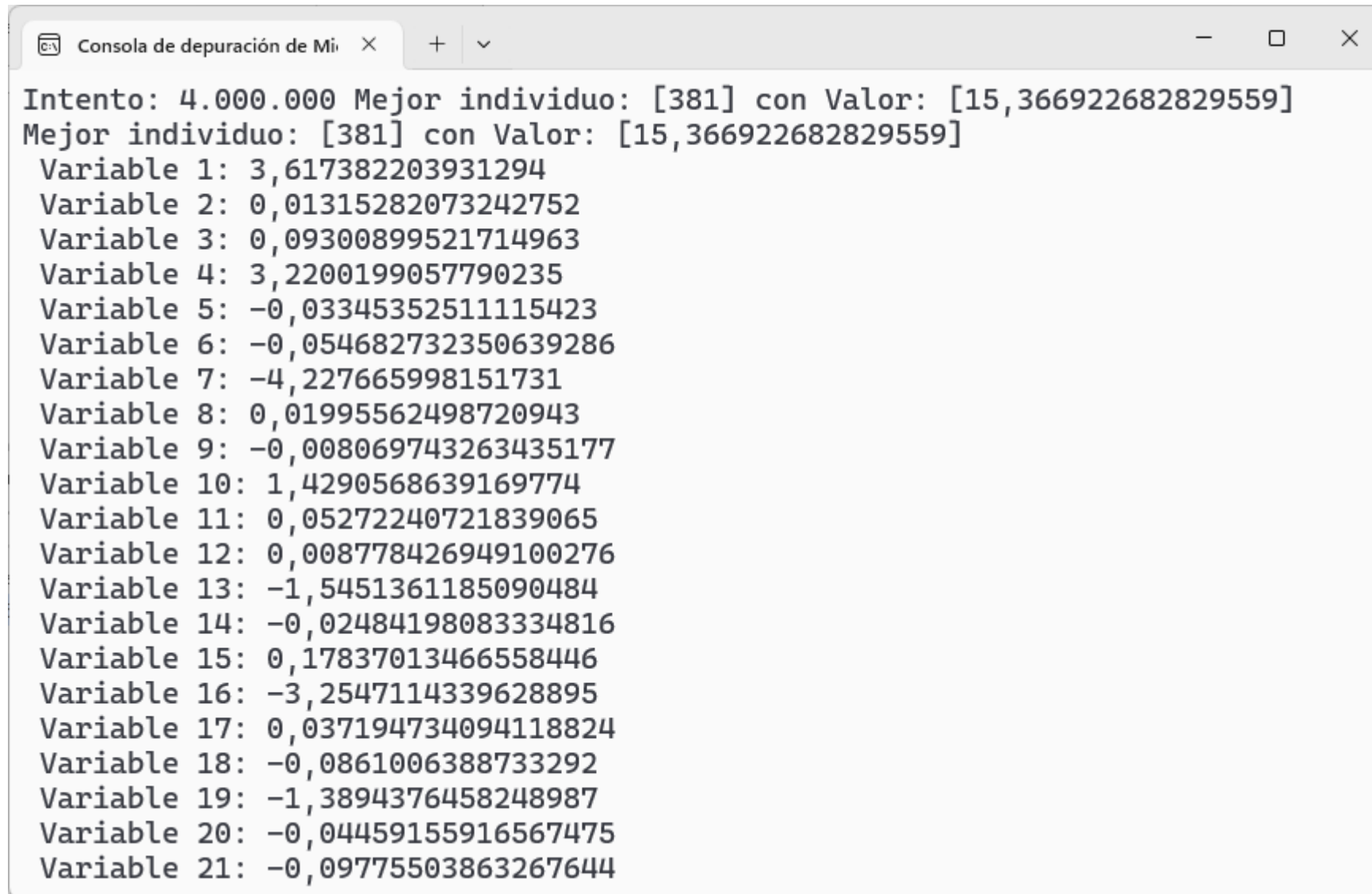
Console.WriteLine("Registros");
for (int cont = 0; cont < Xentrada.Length; cont++) {
    Console.Write(Xentrada[cont] + ";" + Yesperado[cont] + ";");
    Console.WriteLine(Ecuacion(Individuos[MejorIndividuo], Xentrada[cont]));
}
}

static void Distancia(Individuo Indiv) {
    if (Indiv.Distance != -1) return;
    Indiv.Distance = 0;
    for (int Cont = 0; Cont < Xentrada.Length; Cont++) {
        double ValorEcuacion = Ecuacion(Indiv, Xentrada[Cont]);
        Indiv.Distance += Math.Abs(ValorEcuacion - Yesperado[Cont]);
    }
}
}

```

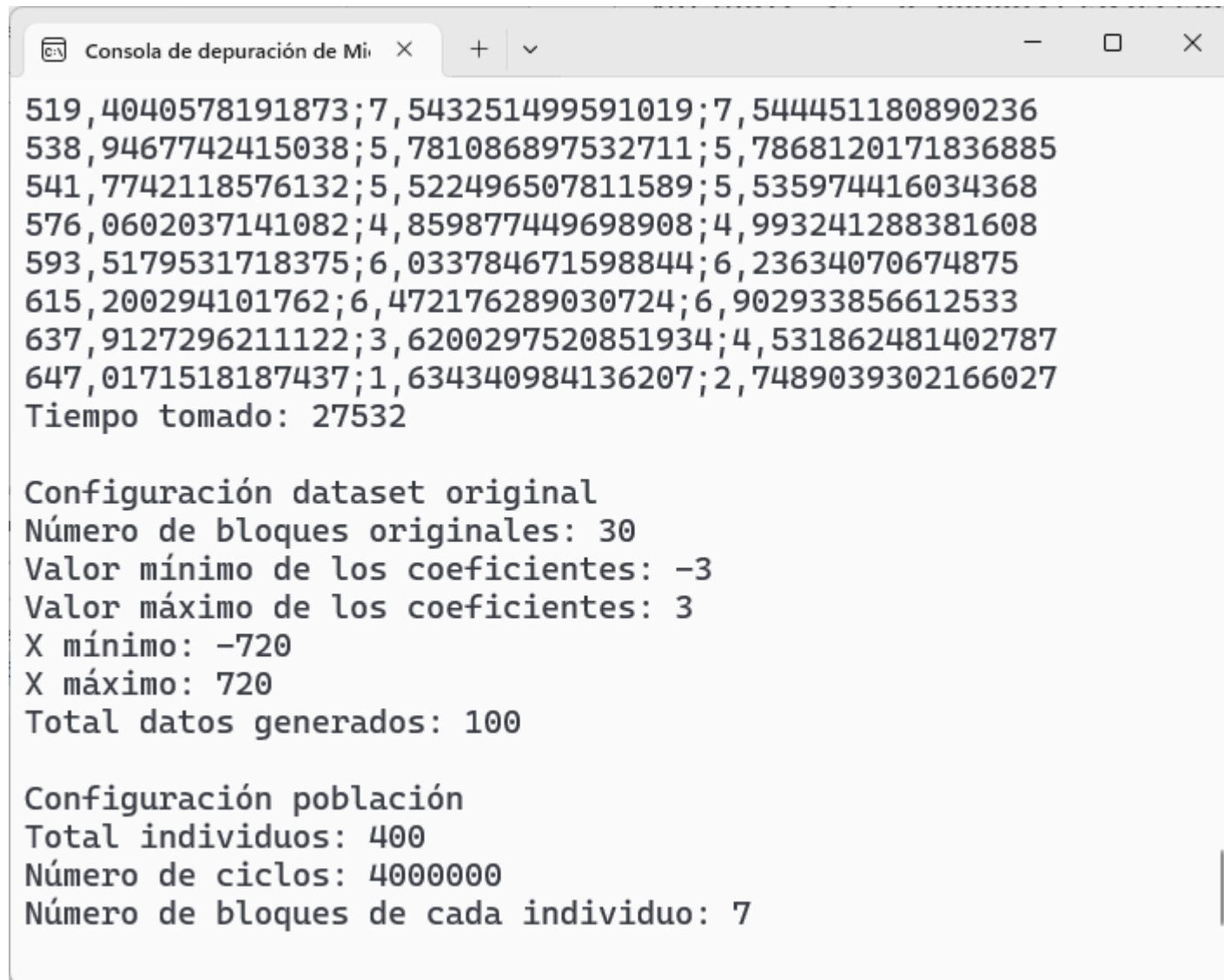
```
static double Ecuacion(Individuo Indiv, double X) {  
    double Acumula = 0;  
    for (int bloque = 0; bloque < Indiv.Coeff.Length; bloque += 3) {  
        Acumula += Indiv.Coeff[bloque] * Math.Sin(Indiv.Coeff[bloque + 1] * X + Indiv.Coeff[bloque + 2]);  
    }  
    return Acumula;  
}  
}
```

Ejemplo de ejecución:



```
Intento: 4.000.000 Mejor individuo: [381] con Valor: [15,366922682829559]
Mejor individuo: [381] con Valor: [15,366922682829559]
Variable 1: 3,617382203931294
Variable 2: 0,01315282073242752
Variable 3: 0,09300899521714963
Variable 4: 3,2200199057790235
Variable 5: -0,03345352511115423
Variable 6: -0,054682732350639286
Variable 7: -4,227665998151731
Variable 8: 0,01995562498720943
Variable 9: -0,008069743263435177
Variable 10: 1,4290568639169774
Variable 11: 0,05272240721839065
Variable 12: 0,008778426949100276
Variable 13: -1,5451361185090484
Variable 14: -0,02484198083334816
Variable 15: 0,17837013466558446
Variable 16: -3,2547114339628895
Variable 17: 0,037194734094118824
Variable 18: -0,0861006388733292
Variable 19: -1,3894376458248987
Variable 20: -0,04459155916567475
Variable 21: -0,09775503863267644
```

Ilustración 11: Algoritmo Evolutivo

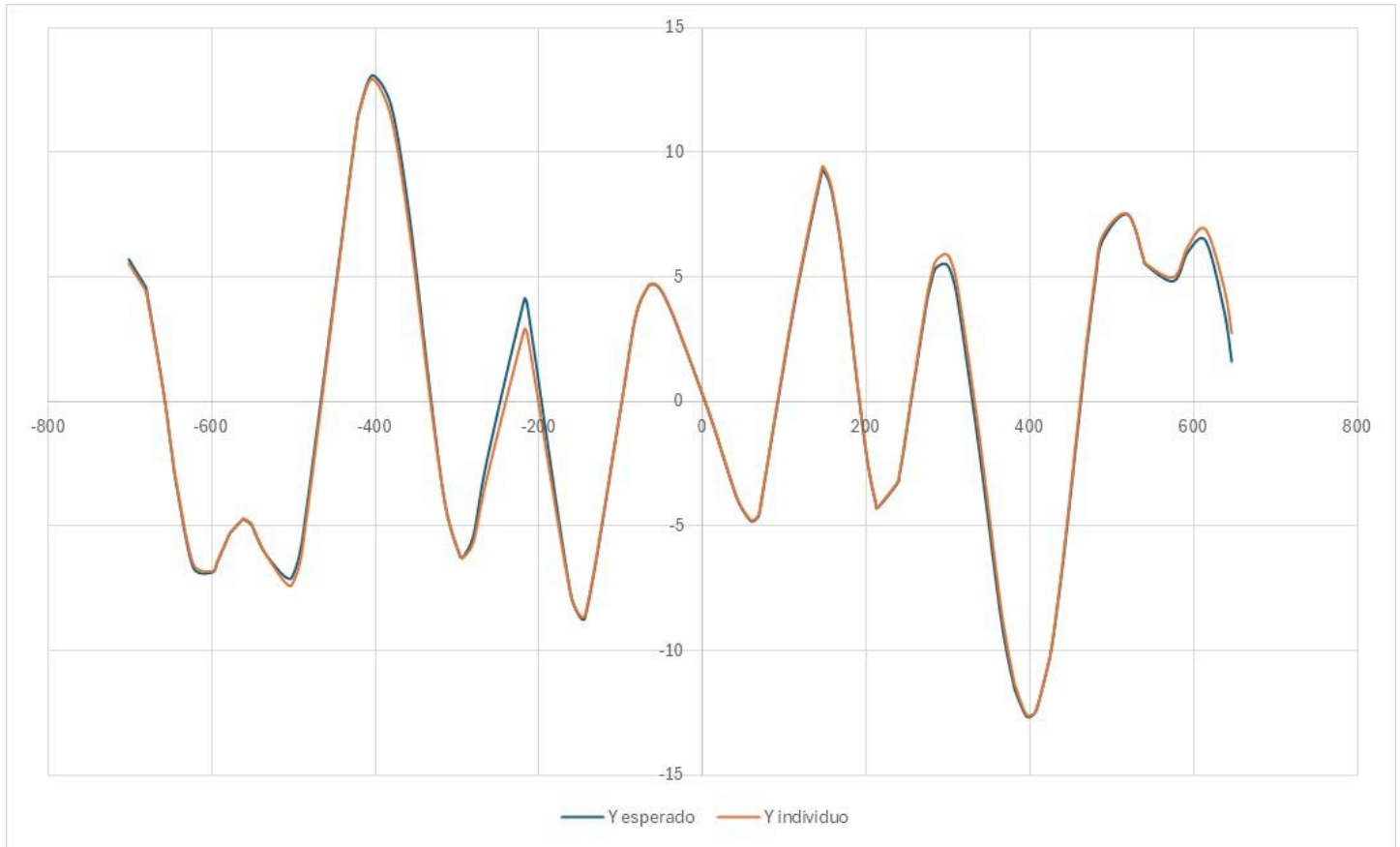


```
519,4040578191873;7,543251499591019;7,544451180890236
538,9467742415038;5,781086897532711;5,7868120171836885
541,7742118576132;5,522496507811589;5,535974416034368
576,0602037141082;4,859877449698908;4,993241288381608
593,5179531718375;6,033784671598844;6,23634070674875
615,200294101762;6,472176289030724;6,902933856612533
637,9127296211122;3,6200297520851934;4,531862481402787
647,0171518187437;1,634340984136207;2,7489039302166027
Tiempo tomado: 27532

Configuración dataset original
Número de bloques originales: 30
Valor mínimo de los coeficientes: -3
Valor máximo de los coeficientes: 3
X mínimo: -720
X máximo: 720
Total datos generados: 100

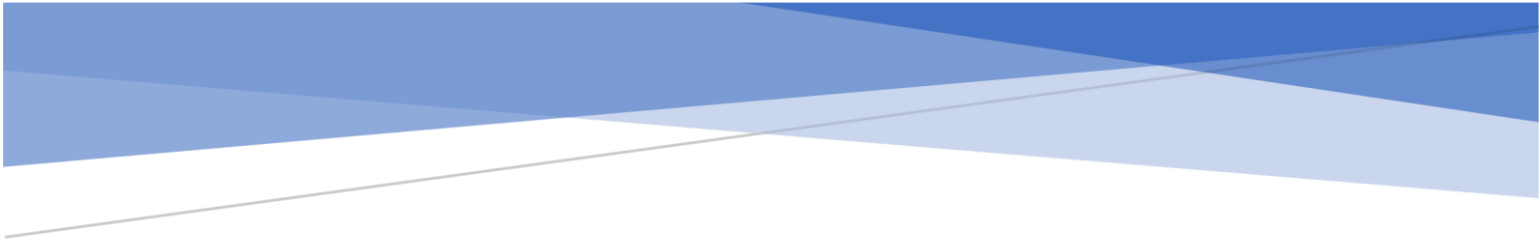
Configuración población
Total individuos: 400
Número de ciclos: 4000000
Número de bloques de cada individuo: 7
```

Ilustración 12: Algoritmo Evolutivo



*Ilustración 13: Ejemplo de simplificación de curvas*

Se puede apreciar que el mejor individuo de sólo 7 bloques (es decir, 21 coeficientes), logra un comportamiento bastante similar al ambiente construido con 30 bloques (es decir, 90 coeficientes).



# C# Y .NET 10

## Parte 11. Redes Neuronales. Perceptrón Multicapa

2025-12

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                                    |     |
|------------------------------------------------------------------------------------|-----|
| Tabla de ilustraciones.....                                                        | 3   |
| Acerca del autor.....                                                              | 5   |
| Licencia de este libro .....                                                       | 5   |
| Licencia del software .....                                                        | 5   |
| Marcas registradas .....                                                           | 6   |
| Dedicatoria .....                                                                  | 7   |
| Iniciando .....                                                                    | 8   |
| Perceptrón simple .....                                                            | 11  |
| Fórmula de Frank Rosenblatt.....                                                   | 19  |
| Perceptrón simple: Aprendiendo la tabla del OR .....                               | 22  |
| Límites del Perceptrón Simple .....                                                | 23  |
| Encontrando el mínimo en una ecuación .....                                        | 26  |
| Descenso del gradiente .....                                                       | 31  |
| Mínimos locales y globales.....                                                    | 35  |
| Búsqueda de mínimos y redes neuronales .....                                       | 37  |
| Perceptrón Multicapa .....                                                         | 38  |
| Las neuronas.....                                                                  | 39  |
| Pesos y como nombrarlos .....                                                      | 43  |
| La función de activación de la neurona .....                                       | 46  |
| Introducción al algoritmo "Backpropagation" (backward propagation of errors) ..... | 52  |
| Nombrando las entradas, pesos, umbrales, salidas y capas .....                     | 54  |
| Regla de la cadena .....                                                           | 59  |
| Derivadas parciales .....                                                          | 60  |
| Las derivadas en el algoritmo de propagación hacia atrás .....                     | 62  |
| Variando los pesos y umbrales con el algoritmo de propagación hacia atrás .....    | 99  |
| Implementación del perceptrón multicapa .....                                      | 100 |
| Algoritmo de retro propagación .....                                               | 112 |
| Código completo del perceptrón.....                                                | 120 |
| Reconocimiento de números de un reloj digital.....                                 | 129 |
| Detección de patrones en series de tiempo.....                                     | 143 |
| El problema del sobre-entrenamiento .....                                          | 145 |
| Leyendo archivos CSV .....                                                         | 145 |



## Tabla de ilustraciones

|                                                                                |     |
|--------------------------------------------------------------------------------|-----|
| Ilustración 1: Caja negra, entradas y salidas .....                            | 8   |
| Ilustración 2: Pesos al interior de la caja .....                              | 8   |
| Ilustración 3: Esos pesos ya operan con el ejemplo A.....                      | 9   |
| Ilustración 4: Los mismos pesos funcionan para el ejemplo B .....              | 9   |
| Ilustración 5: Y esos mismos pesos funcionan para el ejemplo C.....            | 10  |
| Ilustración 6: Perceptrón simple .....                                         | 11  |
| Ilustración 7: Funcionamiento del perceptrón simple .....                      | 12  |
| Ilustración 8: Los pesos funcionan para esa regla .....                        | 14  |
| Ilustración 9: Esos pesos fallan con la segunda regla .....                    | 15  |
| Ilustración 10: Dar con los pesos con sólo azar .....                          | 17  |
| Ilustración 11: Dar con los pesos con sólo azar .....                          | 17  |
| Ilustración 12: Dar con los pesos con sólo azar .....                          | 17  |
| Ilustración 13: Encontrando los pesos más rápido .....                         | 21  |
| Ilustración 14: Encontrando los pesos más rápido .....                         | 21  |
| Ilustración 15: Encontrando los pesos más rápido .....                         | 21  |
| Ilustración 16: Tabla del AND .....                                            | 23  |
| Ilustración 17: Tabla del XOR .....                                            | 24  |
| Ilustración 18: Red neuronal con 3 neuronas .....                              | 25  |
| Ilustración 19: Red neuronal con otro tipo de conexiones.....                  | 25  |
| Ilustración 20: Tabla y gráfico de la ecuación hechos con Microsoft Excel..... | 26  |
| Ilustración 21: Derivada usando en WolframAlpha .....                          | 27  |
| Ilustración 22: Buscando el mínimo.....                                        | 30  |
| Ilustración 23: Pendientes .....                                               | 32  |
| Ilustración 24: Gráfico de un polinomio de quinto grado .....                  | 35  |
| Ilustración 25: Red neuronal con varias conexiones distintas.....              | 37  |
| Ilustración 26: Perceptrón multicapa .....                                     | 38  |
| Ilustración 27: Esquema de una neurona .....                                   | 39  |
| Ilustración 28: Esquema de un perceptrón multicapa .....                       | 43  |
| Ilustración 29: Nombrando los pesos con una letra .....                        | 44  |
| Ilustración 30: Gráfico de una función sigmoide .....                          | 47  |
| Ilustración 31: Derivada de la función sigmoide .....                          | 49  |
| Ilustración 32: Comparativa de derivadas.....                                  | 50  |
| Ilustración 33: Esquema de un perceptrón multicapa .....                       | 52  |
| Ilustración 34: Partes de un perceptrón multicapa .....                        | 54  |
| Ilustración 35: Derivada parcial .....                                         | 61  |
| Ilustración 36: Nombramiento de pesos, umbrales y salidas.....                 | 62  |
| Ilustración 37: Esquema de un perceptrón multicapa .....                       | 76  |
| Ilustración 38: Primer camino para ese peso .....                              | 77  |
| Ilustración 39: Segundo camino para ese peso.....                              | 78  |
| Ilustración 40: Primer camino .....                                            | 81  |
| Ilustración 41: Segundo camino .....                                           | 82  |
| Ilustración 42: Tercer camino .....                                            | 83  |
| Ilustración 43: Cuarto camino .....                                            | 84  |
| Ilustración 44: Dos entradas y dos salidas.....                                | 88  |
| Ilustración 45: Representación del error.....                                  | 89  |
| Ilustración 46: Modelo del perceptrón.....                                     | 100 |

|                                                                                          |     |
|------------------------------------------------------------------------------------------|-----|
| Ilustración 47: Modelo del perceptrón.....                                               | 100 |
| Ilustración 48: Un perceptrón multicapa .....                                            | 110 |
| Ilustración 49: Perceptrón aprendiendo la tabla del XOR.....                             | 128 |
| Ilustración 50: Números en un reloj digital.....                                         | 129 |
| Ilustración 51: Aprendiendo los patrones para identificar el número digital. ....        | 142 |
| Ilustración 52: Aprendió los patrones para identificar el número digital. ....           | 142 |
| Ilustración 53: Gráfico uniendo los puntos.....                                          | 144 |
| Ilustración 54: Archivo CSV .....                                                        | 146 |
| Ilustración 55: Resultado del entrenamiento teniendo en cuenta en NO sobre-ajustar ..... | 160 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

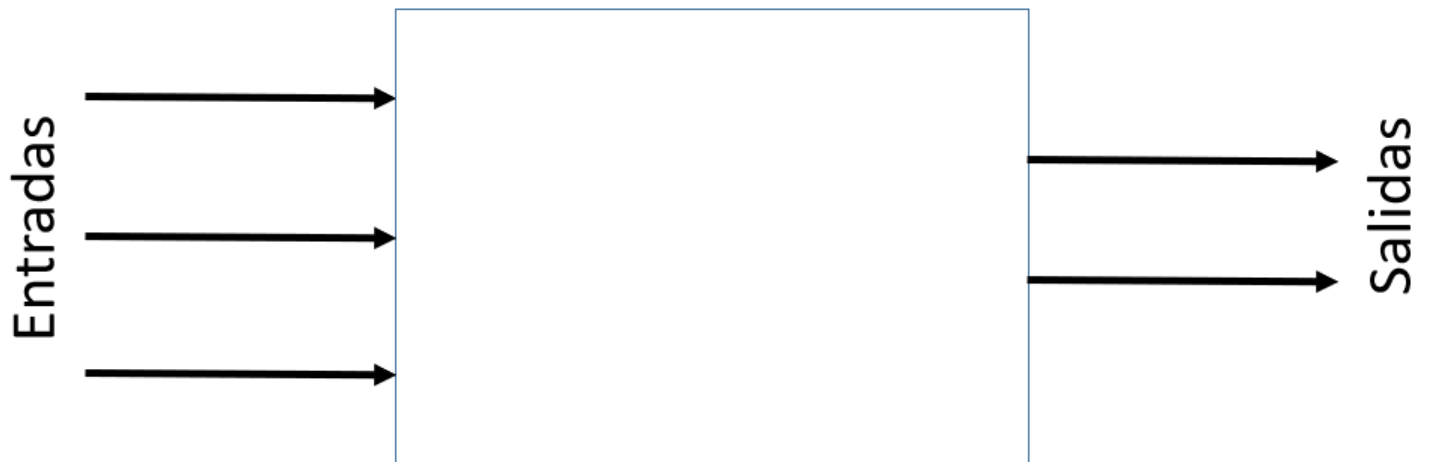
## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

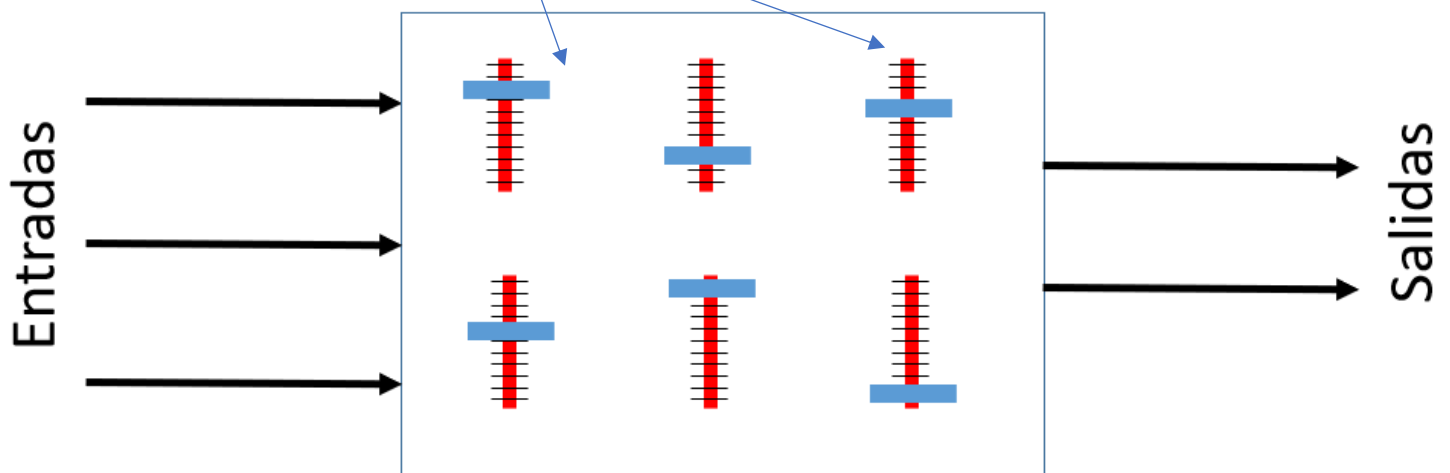
## Iniciando

Las redes neuronales son como una caja negra en la cual hay unas entradas, la caja en sí y unas salidas.



*Ilustración 1: Caja negra, entradas y salidas*

Lo particular es que hay unos pesos que dependiendo de su valor (más arriba o más abajo) afectan el valor de las salidas.



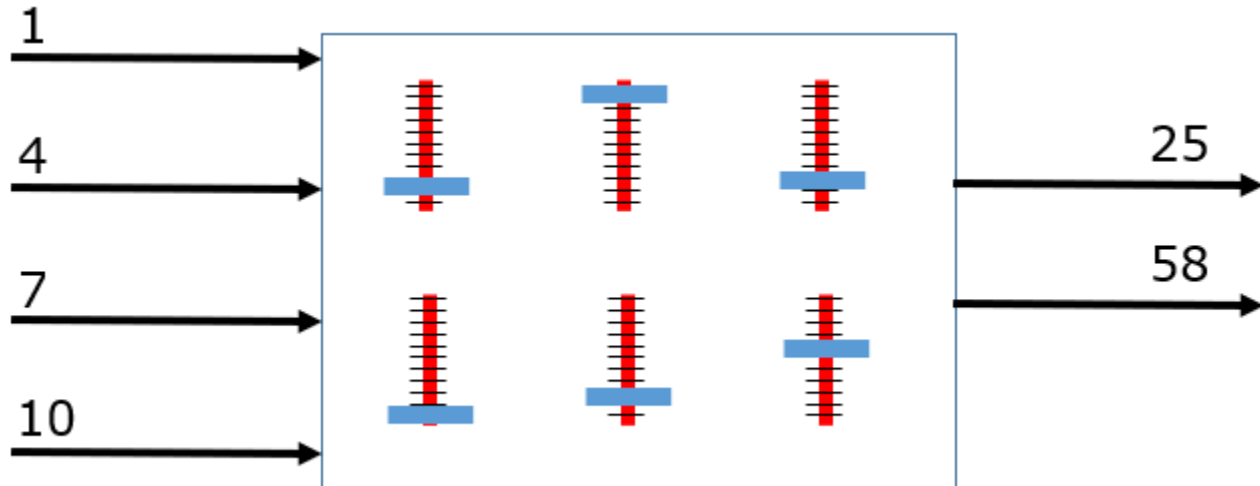
*Ilustración 2: Pesos al interior de la caja*

En el ejemplo, están las siguientes entradas y salidas:

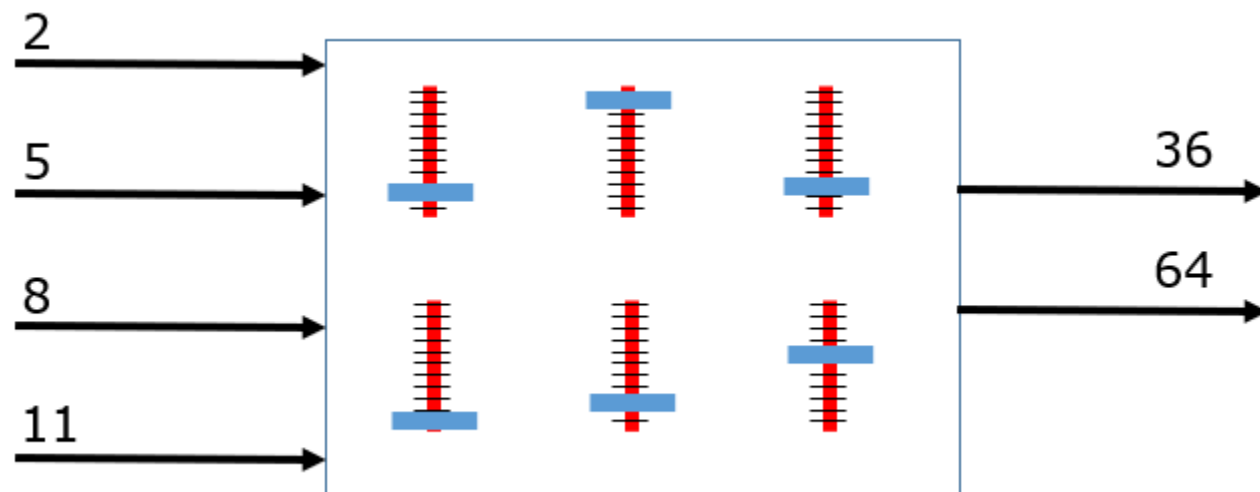
|                  | Entrada 1 | Entrada 2 | Entrada 3 | Entrada 4 | Salida deseada 1 | Salida deseada 2 |
|------------------|-----------|-----------|-----------|-----------|------------------|------------------|
| <b>Ejemplo A</b> | 1         | 4         | 7         | 10        | 25               | 58               |
| <b>Ejemplo B</b> | 2         | 5         | 8         | 11        | 36               | 64               |
| <b>Ejemplo C</b> | 3         | 6         | 9         | 12        | 47               | 70               |

Significa que, si entran los números 1, 4, 7, 10, (ejemplo A), deberían salir 25 y 58. Luego hay que ajustar pesos (moviéndolos arriba o a abajo) hasta obtener esa salida deseada.

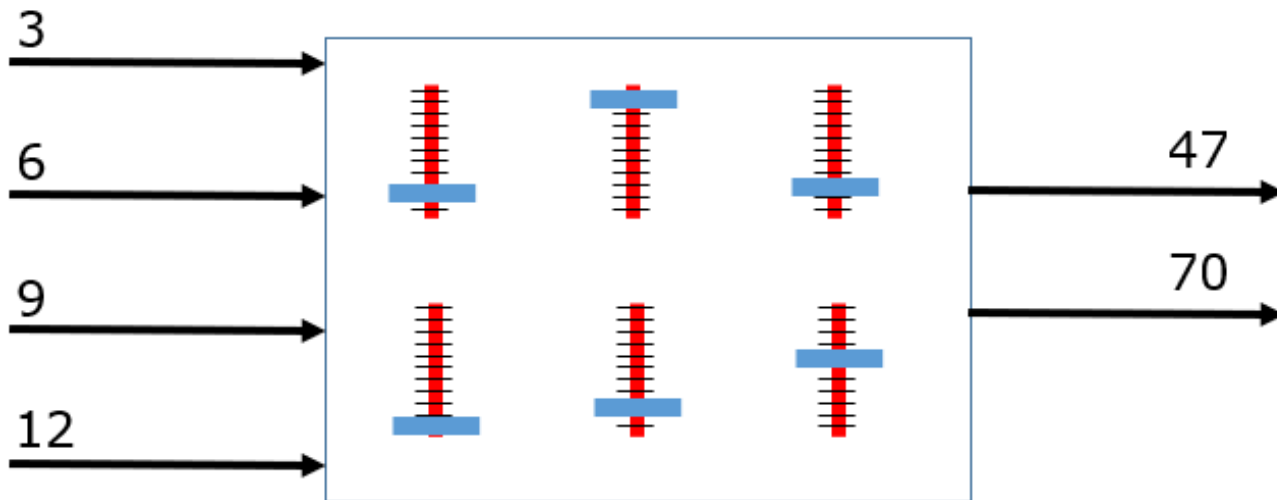
Luego se prueban esos pesos con el nuevo conjunto de datos (ejemplo B). Se ingresa 2, 5, 8, 11 y debería salir 36 y 64. ¿Qué pasaría si eso no sucede? Que hay que cambiar los pesos con nuevos valores y probar desde el inicio (con el ejemplo A). ¿Cuándo termina? Cuando los valores de los pesos encontrados funcionen para los tres ejemplos (A, B y C).



*Ilustración 3: Esos pesos ya operan con el ejemplo A*



*Ilustración 4: Los mismos pesos funcionan para el ejemplo B*



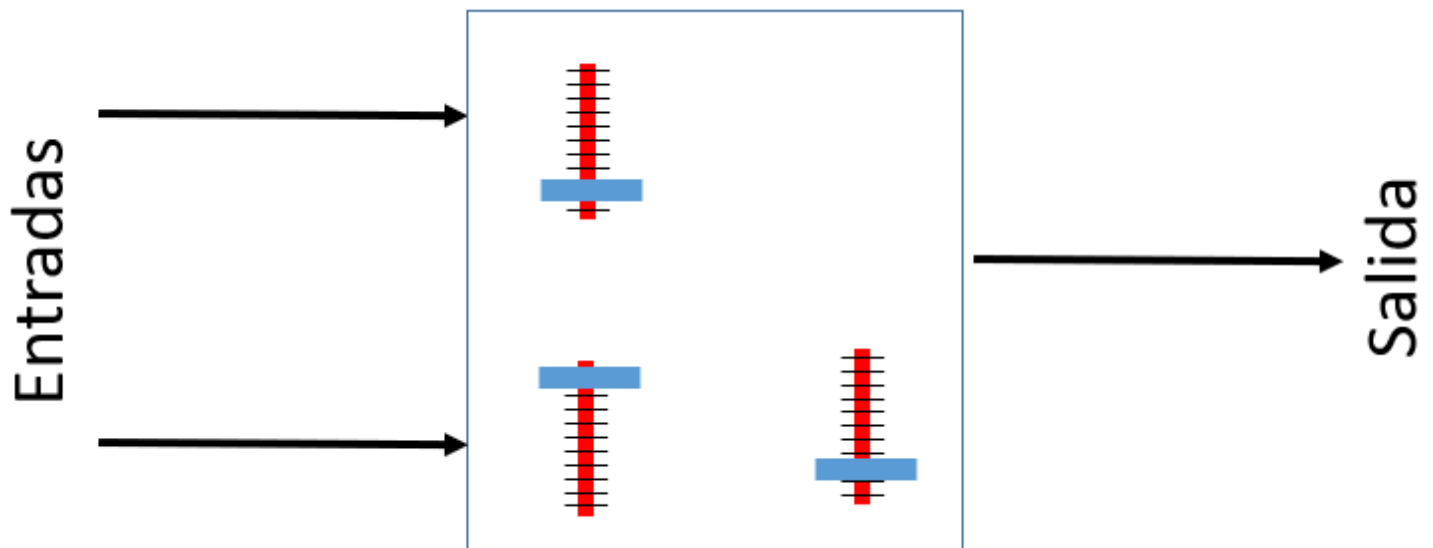
*Ilustración 5: Y esos mismos pesos funcionan para el ejemplo C*

¿Cómo es el proceso? Al iniciar, esos pesos tienen valores al azar y poco a poco se van ajustando. Existen técnicas matemáticas que colaboran en encontrar esos pesos rápidamente porque de lo contrario, sería un ajuste al azar continuamente hasta que por suerte se encuentren los valores correctos.



## Perceptrón simple

Se inicia con una neurona. Esta es su representación:



*Ilustración 6: Perceptrón simple*

Dos entradas, una salida y tres pesos. Se demostrará que esta neurona puede “aprender” como opera la tabla del AND:

| A         | B         | Resultado (A AND B) |
|-----------|-----------|---------------------|
| Verdadero | Verdadero | Verdadero           |
| Verdadero | Falso     | Falso               |
| Falso     | Verdadero | Falso               |
| Falso     | Falso     | Falso               |

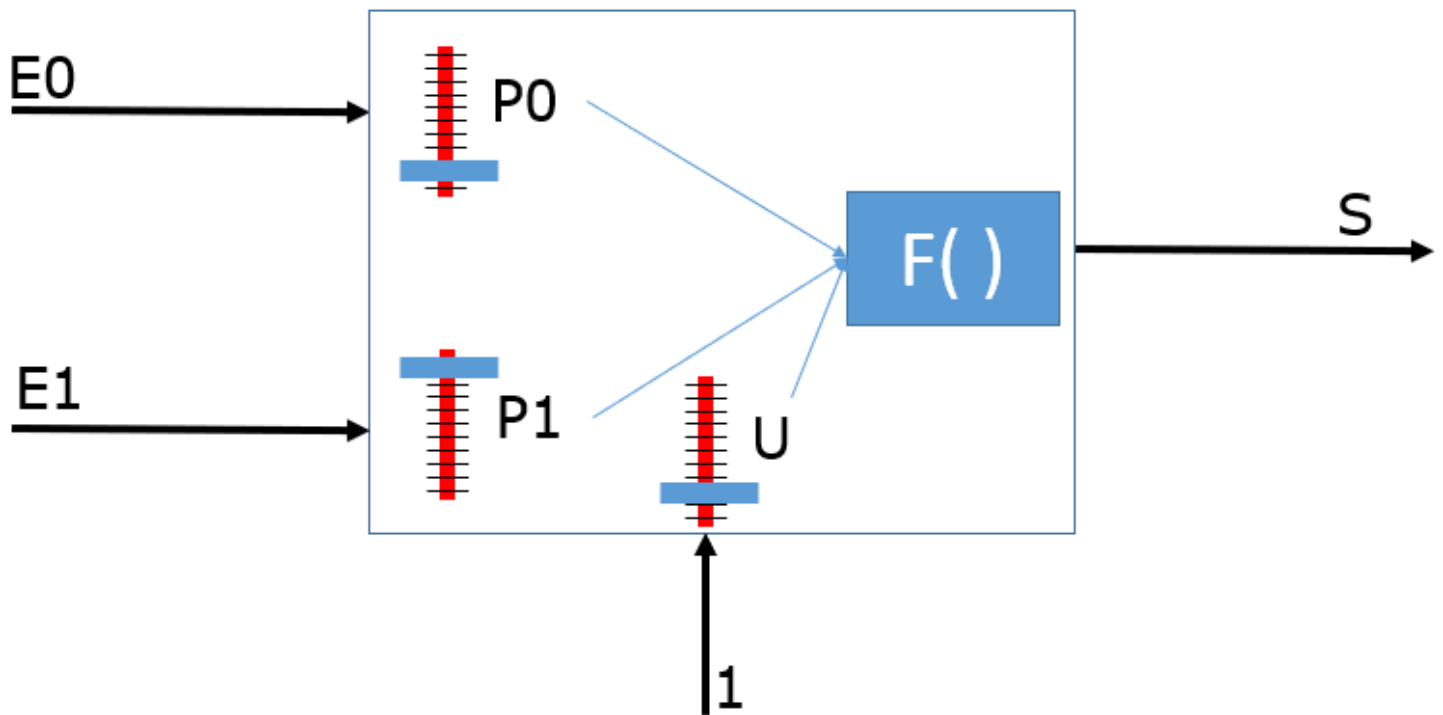
Esa neurona se le conoce con el nombre de Perceptrón Simple.

**Paso 1:** Hacerlo cuantitativo (1 es verdadero, 0 es falso)

| A | B | Resultado (A AND B) |
|---|---|---------------------|
| 1 | 1 | 1                   |
| 1 | 0 | 0                   |
| 0 | 1 | 0                   |
| 0 | 0 | 0                   |

La razón de este cambio es que se requieren valores cuantitativos para ser usados dentro de fórmulas matemáticas.

**Paso 2:** Diseñando la neurona:



*Ilustración 7: Funcionamiento del perceptrón simple*

Un peso por cada entrada y se le adiciona una entrada interna que se llama umbral y tiene el valor de 1 con su propio peso.

E0 y E1 son las entradas

P0, P1 son los pesos de las entradas

U es el peso del umbral

S es la salida

F( ) es la función que le da el valor a S

**Paso 3:** Haciendo los cálculos:

La salida S se calcula con la siguiente fórmula matemática

$$S = F ( E0 * P0 + E1 * P1 + 1 * U )$$

Se inicia con la primera regla de la tabla AND (verdadero y verdadero, da verdadero), en este caso se ingresa 1 y 1, la salida debería ser 1.

E0 = 1 (verdadero)

E1 = 1 (verdadero)

$P0 = 0.6172$  (un valor al azar)  
 $P1 = 0.4501$  (un valor real al azar)  
 $U = 0.3789$  (un valor real al azar)

Entonces la salida sería:

$S = F ( E0 * P0 + E1 * P1 + 1 * U )$

$S = F ( 1 * 0.6172 + 1 * 0.4501 + 1 * 0.3789 )$

$S = F ( 1.4462 )$

¿Y que es  $F( )$ ? una función que podría ser matemática o un algoritmo. Así:

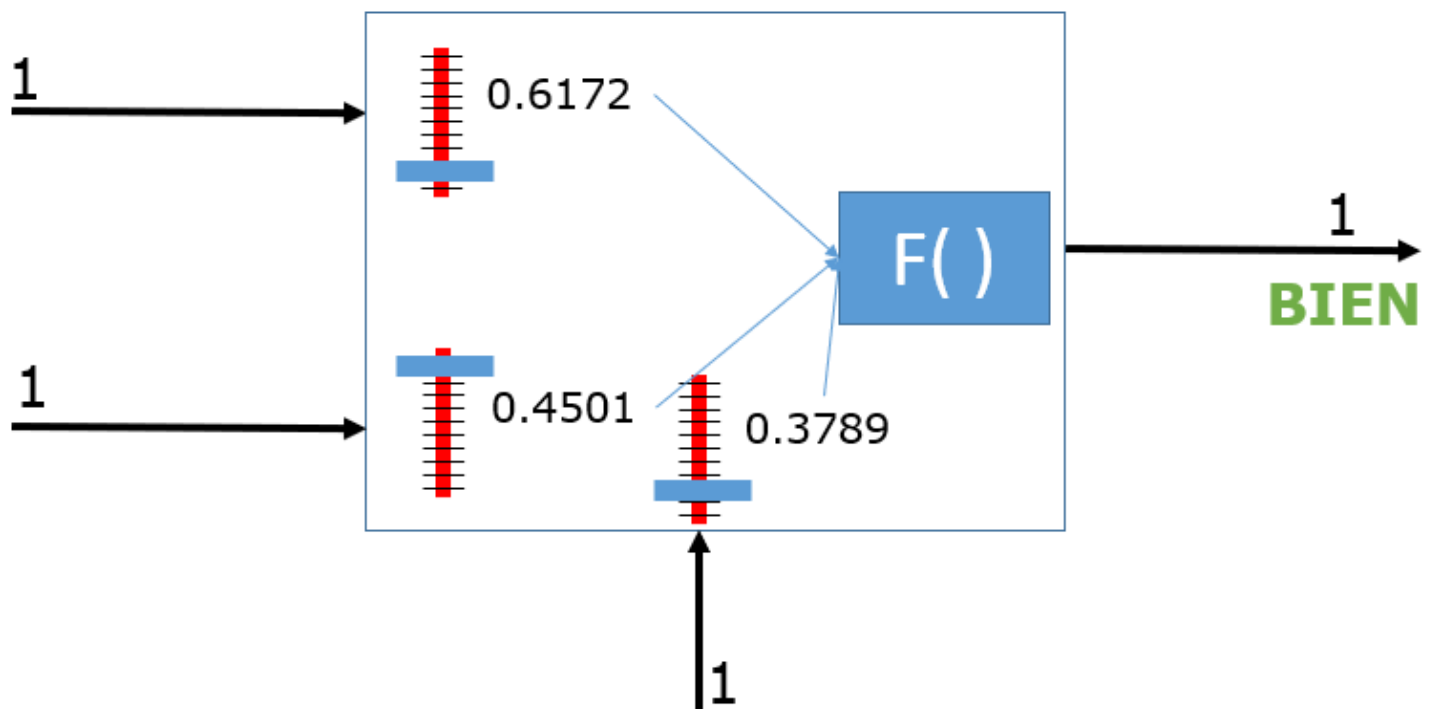
```
Función F(Valor)
Inicio
  Si Valor > 0.5 entonces
    retorne 1
  de lo contrario
    retorne 0
  fin si
Fin
```

Continuando con el ejemplo:

$S = F ( 1.4462 )$

$S = 1$

Y ese es el valor esperado. Los pesos funcionan para esas entradas.



*Ilustración 8: Los pesos funcionan para esa regla*

¿Funcionarán esos pesos para las otras reglas de la tabla del AND? Se prueba entonces Verdadero y Falso, debería dar Falso

$E0 = 1$  (verdadero)

$E1 = 0$  (falso)

$S = F ( E0 * P0 + E1 * P1 + 1 * U )$

$S = F ( 1 * 0.6172 + 0 * 0.4501 + 1 * 0.3789 )$

$S = F ( 0.9961 )$

$S = 1$

No, no funcionó, debería haber dado cero.

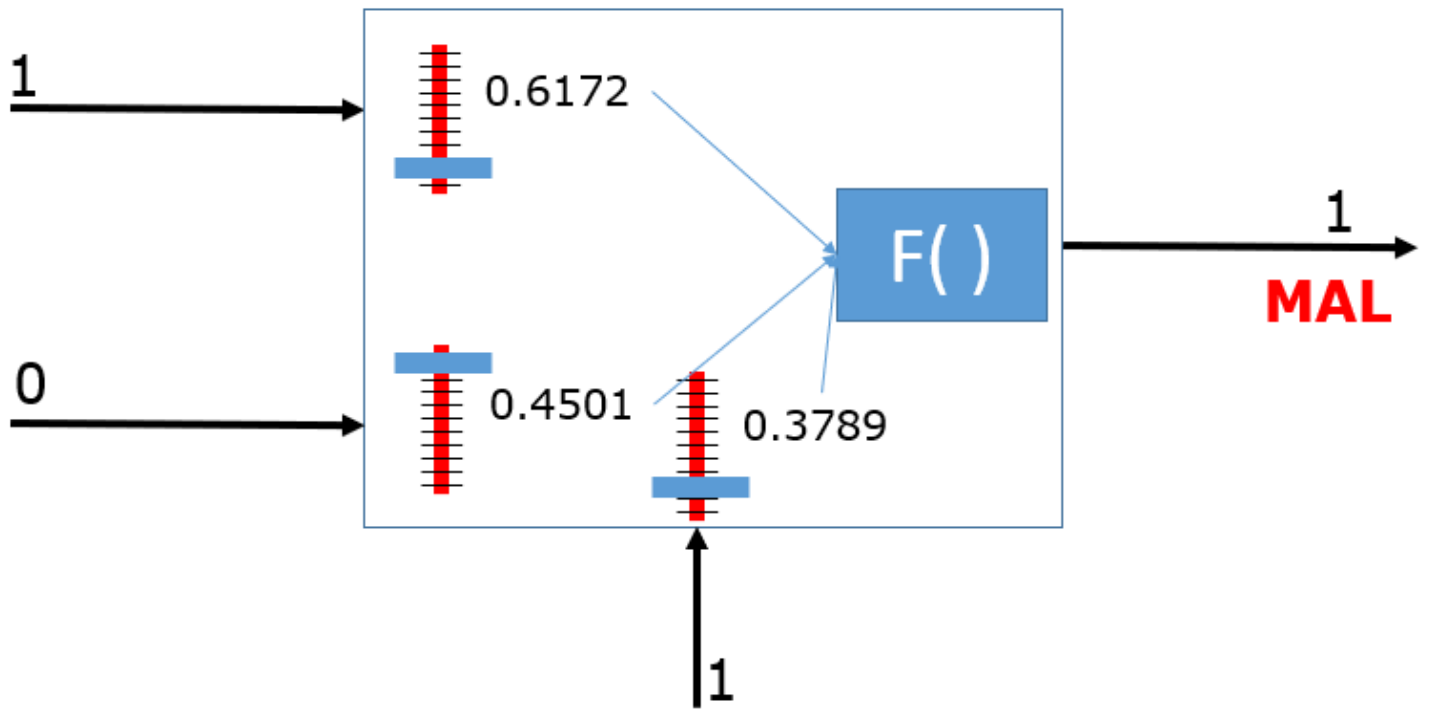


Ilustración 9: Esos pesos fallan con la segunda regla

¿Qué sigue? Habrá que utilizar otros valores para los pesos. Una forma es darle otros valores al azar. Ejecutar de nuevo el proceso, probar con todas las reglas hasta que finalmente de las salidas esperadas.

K/001.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Único generador de números aleatorios
        Random Azar = new();

        //Tabla AND
        int[][] Entra = [
            [1, 1],
            [1, 0],
            [0, 1],
            [0, 0]
        ];
        int[] Sale = [1, 0, 0, 0];

        //Los pesos
        double P0, P1, U;
```

```

//Mantiene el proceso activo
bool Proceso;

//Número de iteraciones
int Iteracion = 0;

//Hasta que aprenda la tabla AND
do {
    Iteracion++;

    //Pesos al azar
    P0 = Azar.NextDouble();
    P1 = Azar.NextDouble();
    U = Azar.NextDouble();

    //Prueba la tabla AND
    Proceso = false;
    for (int Con = 0; Con < Entra.GetLength(0); Con++) {

        //Calcula el valor de entrada a la función
        double Oper = Entra[Con][0] * P0 + Entra[Con][1] * P1 + U;

        //Función de activación
        int Salida = Oper > 0.5 ? 1 : 0;

        //Si la salida no coincide con lo esperado,
        //cambia los pesos
        if (Salida != Sale[Con]) {
            Proceso = true;
            break;
        }
    }
} while (Proceso);

//Muestra aprendizaje perceptrón simple
for (int Cont = 0; Cont < Entra.GetLength(0); Cont++) {
    double Oper = Entra[Cont][0] * P0 + Entra[Cont][1] * P1 + U;

    //Función de activación
    int Salida = Oper > 0.5 ? 1 : 0;

    Console.WriteLine("Entradas: " + Entra[Cont][0]);
    Console.WriteLine(" y " + Entra[Cont][1] + " = ");
    Console.WriteLine(Sale[Cont] + " red: " + Salida);
}

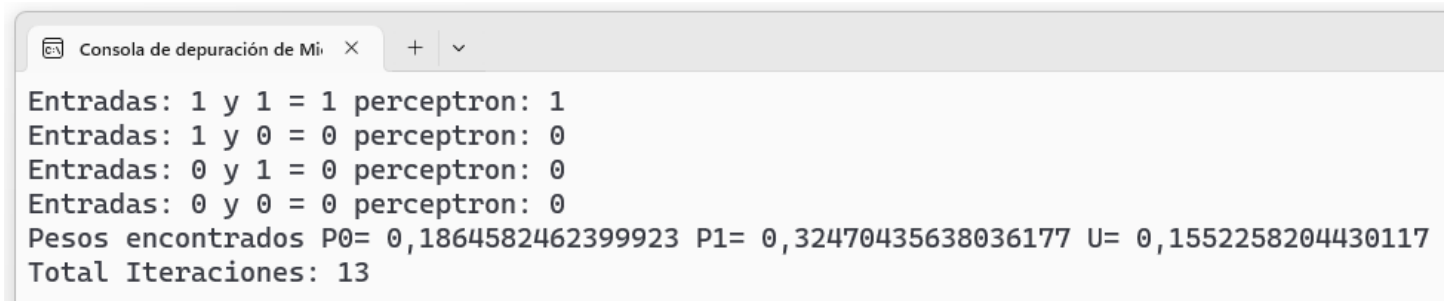
Console.WriteLine("Pesos encontrados P0= " + P0);

```

```

        Console.WriteLine(" P1= " + P1 + " U= " + U);
        Console.WriteLine("Total Iteraciones: " + Iteracion);
    }
}

```

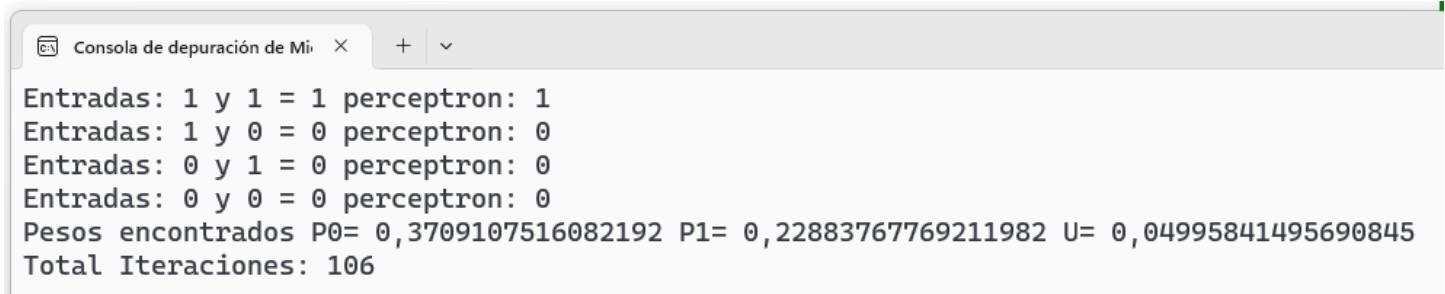


```

Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,1864582462399923 P1= 0,32470435638036177 U= 0,1552258204430117
Total Iteraciones: 13

```

*Ilustración 10: Dar con los pesos con sólo azar*

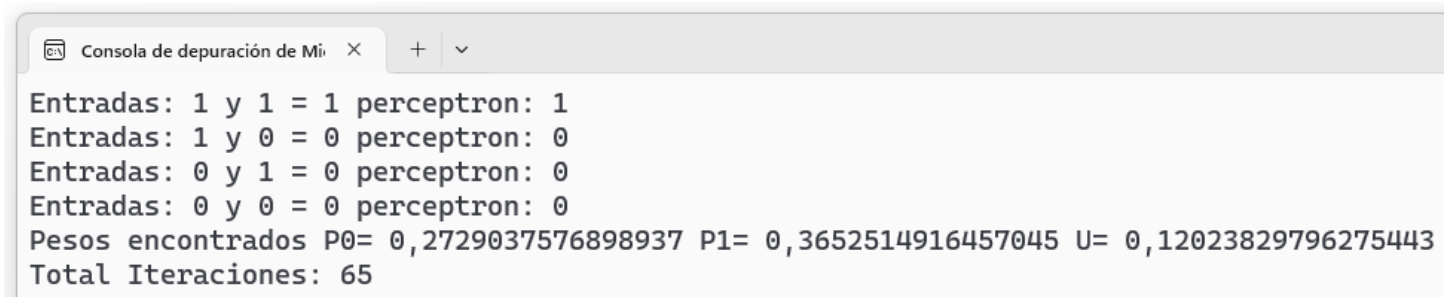


```

Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,3709107516082192 P1= 0,22883767769211982 U= 0,04995841495690845
Total Iteraciones: 106

```

*Ilustración 11: Dar con los pesos con sólo azar*



```

Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,2729037576898937 P1= 0,3652514916457045 U= 0,12023829796275443
Total Iteraciones: 65

```

*Ilustración 12: Dar con los pesos con sólo azar*

Los valores de los pesos no es una respuesta única, pueden ser distintos y son números reales (en C# se implementaron de tipo double). También se observa que en una ejecución requirió 106 iteraciones. El cambio de pesos sucede en estas líneas:

```
P0 = azar.NextDouble();  
P1 = azar.NextDouble();  
U = azar.NextDouble();
```

En caso de que no funcionasen los pesos, el programa simplemente los cambiaba al azar en un valor que oscila entre 0 y 1. Eso puede ser muy ineficiente y riesgoso porque limita los valores a estar entre 0 y 1 ¿Y si los pesos requieren valores mucho más altos o bajos?

Afortunadamente, hay un método matemático que minimiza el uso del azar y puede dar con valores de los pesos en cualquier rango. ¿Cómo funciona? Al principio los pesos tienen un valor al azar, pero de allí en adelante el cálculo de esos pesos se basa en comparar la salida esperada con la salida obtenida, si difieren, ese error sirve para ir cuadrando poco a poco los pesos.



# Fórmula de Frank Rosenblatt

Los pesos se cambian haciendo uso de una fórmula matemática:

```
Error = SalidaEsperada - SalidaReal
Si Error != cero entonces
    P0 = P0 + tasaAprende * Error * E0
    P1 = P1 + tasaAprende * Error * E1
    U = U + tasaAprende * Error * 1
Fin Si
```

tasaAprende es un valor constante de tipo real y de valor entre 0 y 1 (sin tomar el 0, ni el 1). A continuación, el código en C#

K/002.cs

```
namespace Ejemplo;

internal class Program {
    static void Main() {
        //Único generador de números aleatorios
        Random Azar = new();

        //Tabla AND
        int[][] Entra = [
            [1, 1],
            [1, 0],
            [0, 1],
            [0, 0]
        ];
        int[] Sale = [1, 0, 0, 0];

        //Los pesos
        double P0, P1, U;

        //Mantiene el proceso activo
        bool Proceso;

        //Número de iteraciones
        int Iteracion = 0;

        //Tasa de aprendizaje
        double TasaAprende = 0.3;

        //Pesos inician al azar
        P0 = Azar.NextDouble();
```

```

P1 = Azar.NextDouble();
U = Azar.NextDouble();

//Hasta que aprenda la tabla AND
do {
    Iteracion++;

    //Prueba la tabla AND
    Proceso = false;
    for (int Cont = 0; Cont < Entra.GetLength(0); Cont++) {

        //Calcula el valor de entrada a la función
        double Oper = Entra[Cont][0] * P0 + Entra[Cont][1] * P1 + U;

        //Función de activación
        int Salida = Oper > 0.5 ? 1 : 0;

        //El error
        int Error = Sale[Cont] - Salida;

        //Si hay error, cambia los pesos con
        //la Tasa de Aprendizaje
        if (Error != 0) {
            P0 += TasaAprende * Error * Entra[Cont][0];
            P1 += TasaAprende * Error * Entra[Cont][1];
            U += TasaAprende * Error * 1;
            Proceso = true;
        }
    }
} while (Proceso);

//Muestra aprendizaje perceptrón simple
for (int Cont = 0; Cont < Entra.GetLength(0); Cont++) {
    double Oper = Entra[Cont][0] * P0 + Entra[Cont][1] * P1 + U;

    //Función de activación
    int Salida = Oper > 0.5 ? 1 : 0;

    Console.Write("Entradas: " + Entra[Cont][0]);
    Console.Write(" y " + Entra[Cont][1] + " = ");
    Console.WriteLine(Sale[Cont] + " red: " + Salida);
}

Console.Write("Pesos encontrados P0= " + P0);
Console.WriteLine(" P1= " + P1 + " U= " + U);
Console.WriteLine("Total Iteraciones: " + Iteracion);
}
}

```

```
Consola de depuración de Mi X + v
Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,5218730868070858 P1= 0,6514357002751983 U= -0,4271222277132198
Total Iteraciones: 7
```

Ilustración 13: Encontrando los pesos más rápido

```
Consola de depuración de Mi X + v
Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,42514198541349996 P1= 0,6067804235862224 U= -0,3399274573373506
Total Iteraciones: 7
```

Ilustración 14: Encontrando los pesos más rápido

```
Consola de depuración de Mi X + v
Entradas: 1 y 1 = 1 perceptron: 1
Entradas: 1 y 0 = 0 perceptron: 0
Entradas: 0 y 1 = 0 perceptron: 0
Entradas: 0 y 0 = 0 perceptron: 0
Pesos encontrados P0= 0,5705308605404625 P1= 0,5039279097470348 U= -0,5078508611506496
Total Iteraciones: 3
```

Ilustración 15: Encontrando los pesos más rápido

Como se puede observar, se necesitan menos iteraciones en promedio usando la fórmula para hallar los pesos.

## Perceptrón simple: Aprendiendo la tabla del OR

El ejemplo anterior el perceptrón simple aprendía la tabla AND, ¿y con la OR?

| A         | B         | Resultado (A OR B) |
|-----------|-----------|--------------------|
| Verdadero | Verdadero | Verdadero          |
| Verdadero | Falso     | Verdadero          |
| Falso     | Verdadero | Verdadero          |
| Falso     | Falso     | Falso              |

Paso 1: Volver cuantitativa esa tabla (1 es verdadero, 0 es falso)

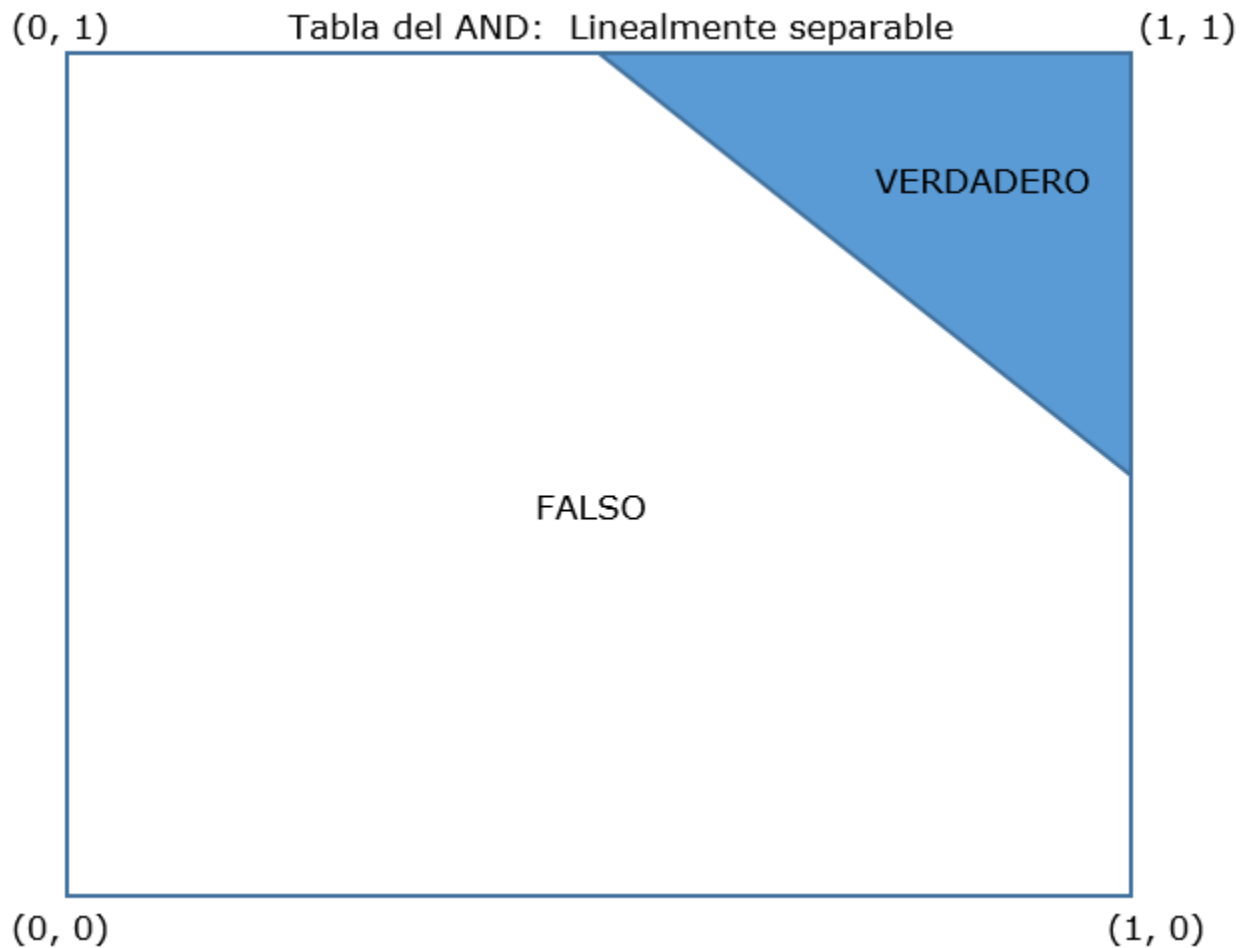
| A | B | Resultado (A OR B) |
|---|---|--------------------|
| 1 | 1 | 1                  |
| 1 | 0 | 1                  |
| 0 | 1 | 1                  |
| 0 | 0 | 0                  |

Es sólo cambiar estas líneas del programa:

```
//Tabla OR
int[][] Entradas = new int[][] {
    new int[] {1, 1},
    new int[] {1, 0},
    new int[] {0, 1},
    new int[] {0, 0}
};
int[] Salidas = new int[] { 1, 1, 1, 0 };
```

## Límites del Perceptrón Simple

El perceptrón simple tiene un límite: que sólo sirve cuando la solución se puede separar con **una** recta. Se explica a continuación:



*Ilustración 16: Tabla del AND*

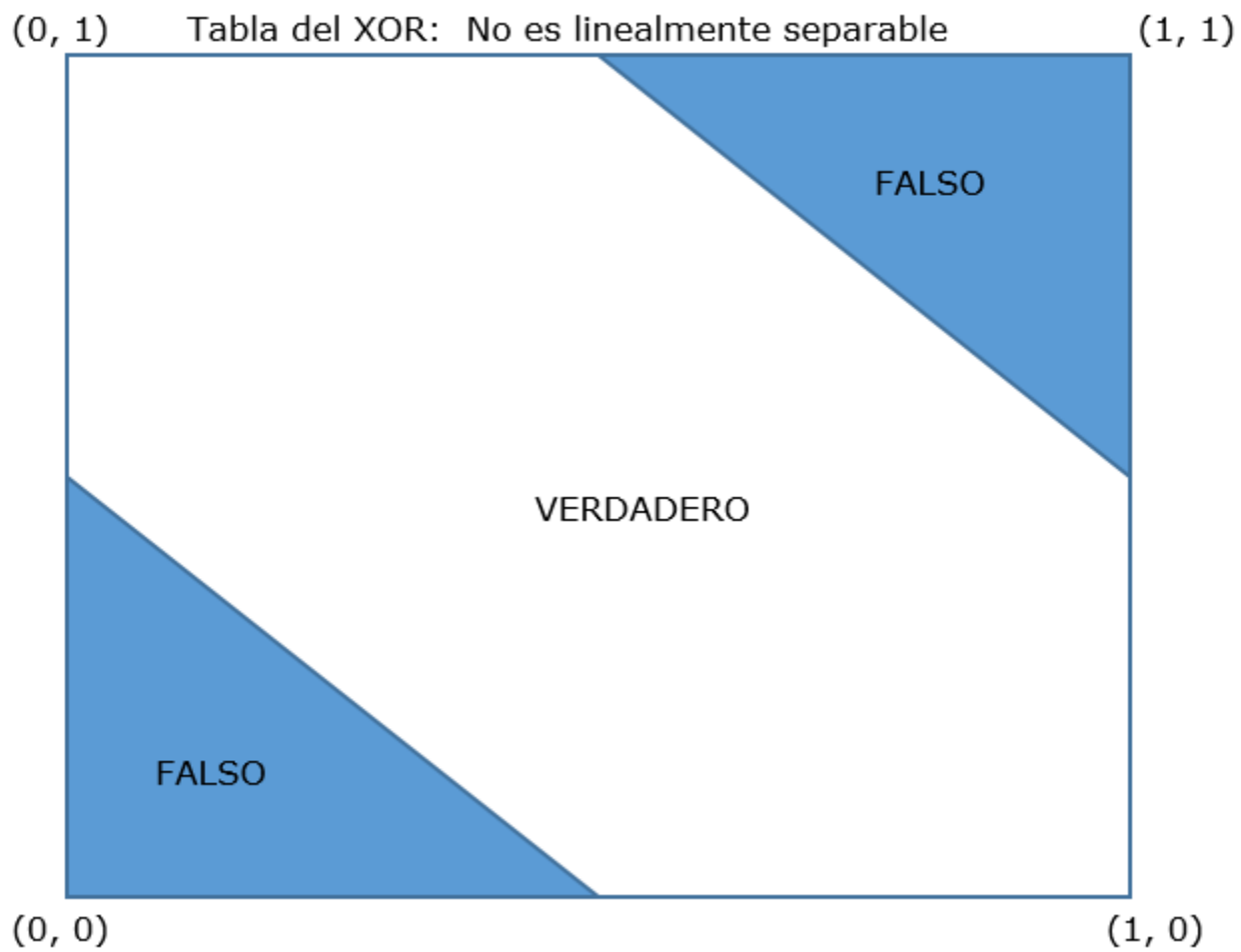
En cambio, si se quiere abordar un problema que requiera dos separaciones, no lo podría hacer el perceptrón simple. El ejemplo clásico es la tabla XOR:

| A         | B         | Resultado (A XOR B) |
|-----------|-----------|---------------------|
| Verdadero | Verdadero | Falso               |
| Verdadero | Falso     | Verdadero           |
| Falso     | Verdadero | Verdadero           |
| Falso     | Falso     | Falso               |

Volviendo cuantitativa esa tabla:

| A | B | Resultado (A XOR B) |
|---|---|---------------------|
| 1 | 1 | 0                   |
| 1 | 0 | 1                   |

|   |   |   |
|---|---|---|
| 0 | 1 | 1 |
| 0 | 0 | 0 |



*Ilustración 17: Tabla del XOR*

Para solucionar ese problema es necesario usar más neuronas puestas en varias capas. Por ejemplo:

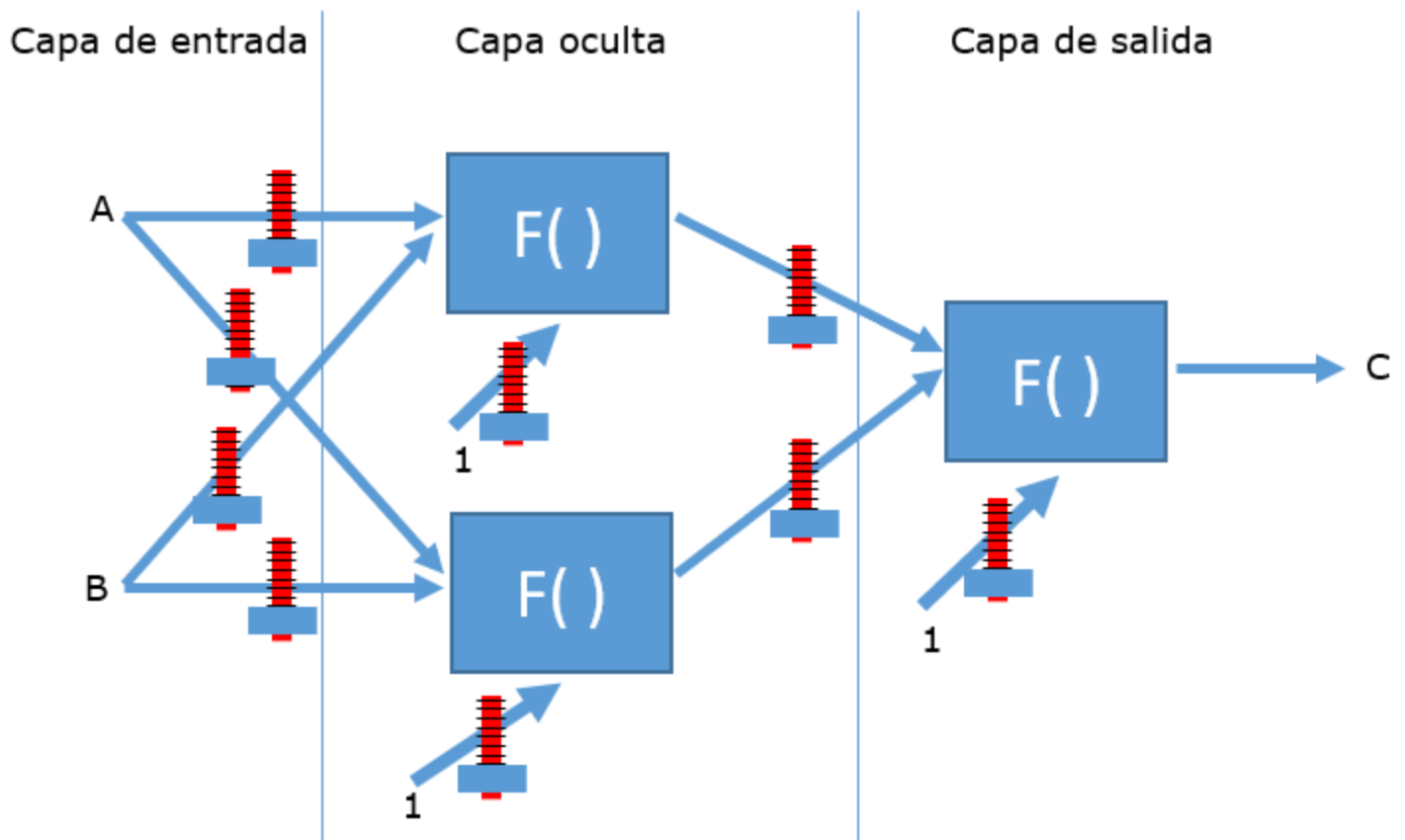


Ilustración 18: Red neuronal con 3 neuronas

O así

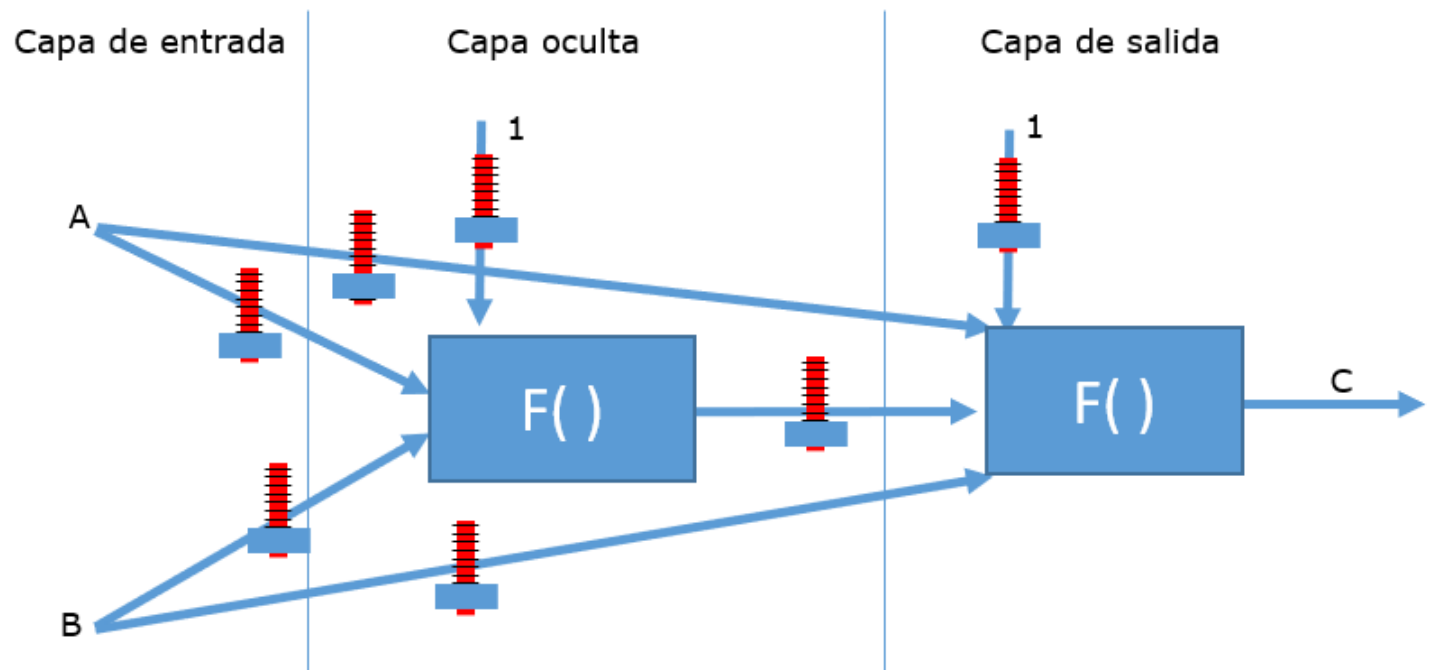


Ilustración 19: Red neuronal con otro tipo de conexiones

Muchos más pesos, luego el reto es cómo dar con cada peso para que se cumplan las salidas. Hay entonces un modelo matemático para lograr esto.

## Encontrando el mínimo en una ecuación

A continuación, se explica la matemática que ayudará a deducir los pesos en una red neuronal.

Para dar con el mínimo de una ecuación se hace uso de las derivadas. Por ejemplo, dada la ecuación:

$$y = 5 * x^2 - 7 * x - 13$$

Tabla de datos y gráfico

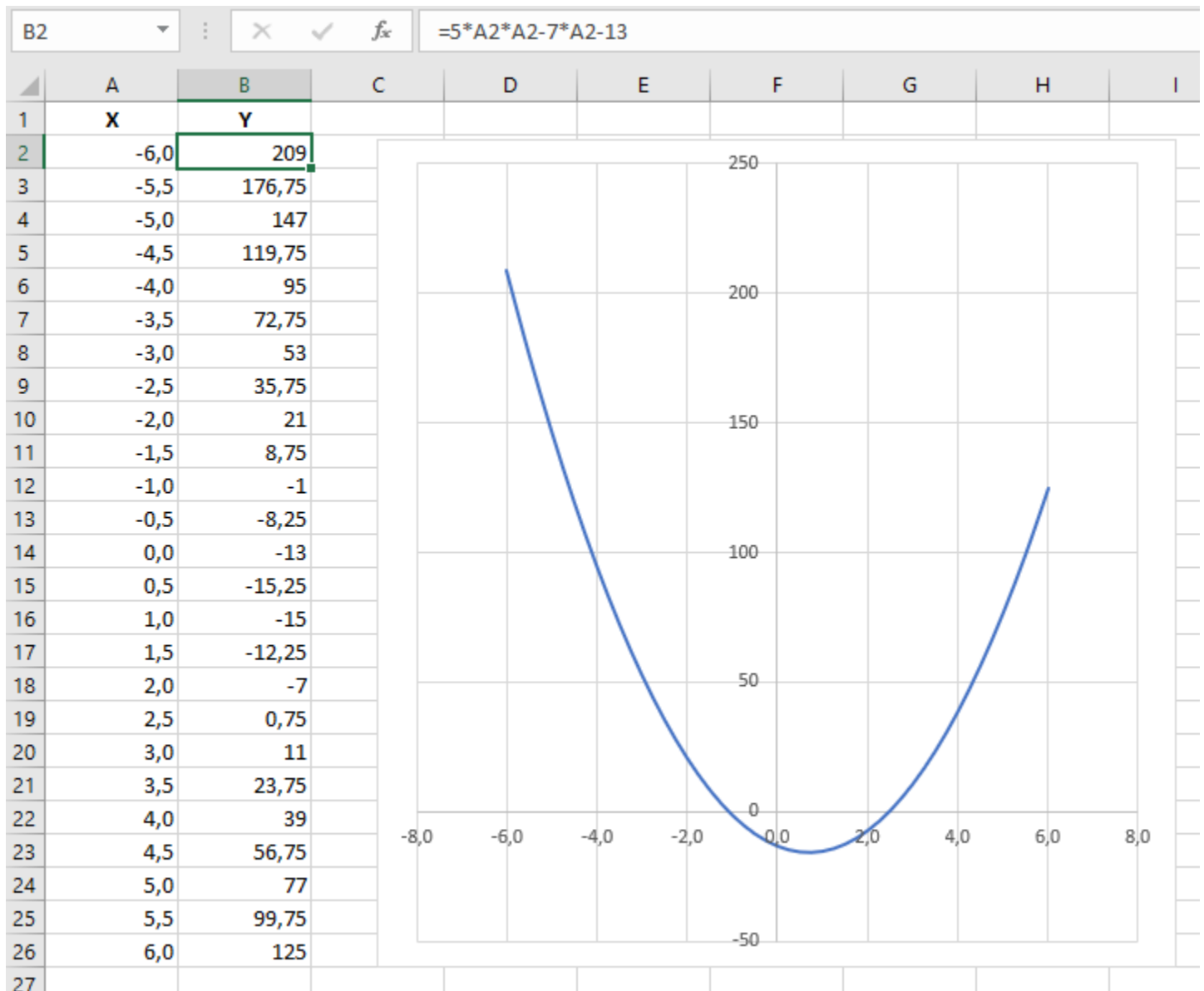


Ilustración 20: Tabla y gráfico de la ecuación hechos con Microsoft Excel

Si se quiere dar con el valor de  $x$  para que  $y$  sea el mínimo valor, el primer paso es derivar

$$y' = 10 * x - 7$$



derivate 5\*x^2-7\*x-13



Extended Keyboard



Upload



Examples



Random

Derivative:

☒ Step-by-step solution

$$\frac{d}{dx}(5x^2 - 7x - 13) = 10x - 7$$

Ilustración 21: Derivada usando en WolframAlpha

Luego esa derivada se iguala a cero

$$0 = 10 * x - 7$$

Se resuelve el valor de x

$$x = 7/10$$

$$x = 0.7$$

Se deduce el valor de x con el que se obtiene el mínimo valor de y

$$y = 5 * x^2 - 7 * x - 13$$

$$y = 5 * 0.7^2 - 7 * 0.7 - 13$$

$$y = -15.45$$

En este caso fue fácil dar con la derivada, porque fue un polinomio de grado 2, el problema sucede cuando la ecuación es compleja, derivarla se torna un desafío y despejar  $x$  sea muy complicado.

Otra forma de dar con el mínimo es iniciar con algún punto  $x$  al azar, por ejemplo,  $x = 1.0$

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 1.0        | -15                        |

Luego un desplazamiento tanto a la izquierda como a la derecha de 0.5 en 0.5, es decir,  $x=0.5$  y  $x=1.5$

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.5        | -15.25                     |
| 1.0        | -15                        |
| 1.5        | -12,25                     |

Se obtiene un nuevo valor de X más prometedor que es 0.5, luego se repite el procedimiento, izquierda y derecha, es decir,  $x=0.0$  y  $x=1.0$

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.0        | -13                        |
| 0.5        | -15.25                     |
| 1.0        | -15                        |

El valor de 0.5 se mantiene como el mejor, luego se hace izquierda y derecha a un paso menor de 0.25

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.25       | -14,4375                   |
| 0.50       | -15.25                     |
| 0.75       | -15.4375                   |

El valor de  $x=0.75$  es el que muestra mejor comportamiento, luego se hace izquierda y derecha a un paso de 0.25

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.50       | -15.25                     |
| 0.75       | -15.4375                   |
| 1.00       | -15                        |

Sigue  $x=0.75$  como mejor valor, luego se prueba a izquierda y derecha, pero en una variación menor de 0.125

| Valor de X  | $y = 5 * x^2 - 7 * x - 13$ |
|-------------|----------------------------|
| 0.625       | -15.421875                 |
| <b>0.75</b> | <b>-15.4375</b>            |
| 0.875       | -15.296875                 |

Sigue  $x=0.75$  como mejor valor, luego se prueba a izquierda y derecha, pero en una variación menor de 0.0625

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.6875     | -15.4492188                |
| 0.75       | -15.4375                   |
| 0.8125     | -15.3867188                |

Ahora es x=0.6875 como mejor valor, luego se prueba a izquierda y derecha en una variación de 0.0625

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.625      | -15.421875                 |
| 0.6875     | -15.4492188                |
| 0.75       | -15.4375                   |

Sigue x=0.6875 como mejor valor, luego se prueba a izquierda y derecha, pero en una variación menor de 0.03125

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.65625    | -15.4404297                |
| 0.6875     | -15.4492188                |
| 0.71875    | -15.4482422                |

Sigue x=0.6875 como mejor valor, luego se prueba a izquierda y derecha, pero en una variación menor de 0.015625

| Valor de X | $y = 5 * x^2 - 7 * x - 13$ |
|------------|----------------------------|
| 0.671875   | -15.4460449                |
| 0.6875     | -15.4492188                |
| 0.703125   | -15.4499512                |

Ahora es x=0,703125 como mejor valor. Este método poco a poco se aproxima a x=0.7 que es el resultado que se dedujo con las derivadas. Esta es su implementación en C#:

```

namespace Ejemplo;

internal class Program {
    static void Main() {
        double X = 1; //valor inicial
        double Yini = Ecuacion(X);
        double Variacion = 1;

        while (Math.Abs(Variacion) > 0.00001) {
            double Ysigue = Ecuacion(X + Variacion);

            //Si en vez de disminuir Y,
            //lo que hace es aumentar,
            //cambia de dirección a un paso menor
            if (Ysigue > Yini) {
                Variacion *= -1;
                Variacion /= 10;
            }
            else { //Está disminuyendo Y
                Yini = Ysigue;
                X += Variacion;
                Console.WriteLine("X: " + X + " Y:" + Yini);
            }
        }
        Console.WriteLine("Respuesta: " + X);
    }

    //Ecuación a analizar
    static double Ecuacion(double X) {
        double Y = 2 * Math.Sin(3 * X - 4) + 5 * Math.Sin(-4 * X + 5) - 4 *
Math.Sin(5 * X - 7);
        return Y;
    }
}

```

```

X: 0,9 Y:-15,25
X: 0,8 Y:-15,4
X: 0,7000000000000001 Y:-15,45
Respuesta: 0,7000000000000001

```

Ilustración 22: Buscando el mínimo

## Descenso del gradiente

Anteriormente se mostró, con las aproximaciones, como buscar el mínimo valor de Y modificando el valor de X, ya sea yendo por la izquierda (disminuyendo) o por la derecha (aumentando). Matemáticamente para saber en qué dirección ir, es con esta expresión:

$$\Delta x = -y'$$

¿Qué significa? Que  $x$  debe modificarse en contra de la derivada de la ecuación.

¿Por qué? La derivada muestra la tangente que pasa por el punto que se seleccionó al azar al inicio. Esa tangente es una línea recta y como toda línea recta tiene una pendiente. Si la pendiente es positiva entonces X se debe ir hacia la izquierda (el valor de X debe disminuir), en cambio, si la pendiente es negativa entonces X debe ir hacia la derecha (el valor de X debe aumentar). Con esa indicación ya se sabe por dónde ir para dar con el valor de X que obtiene el mínimo Y.

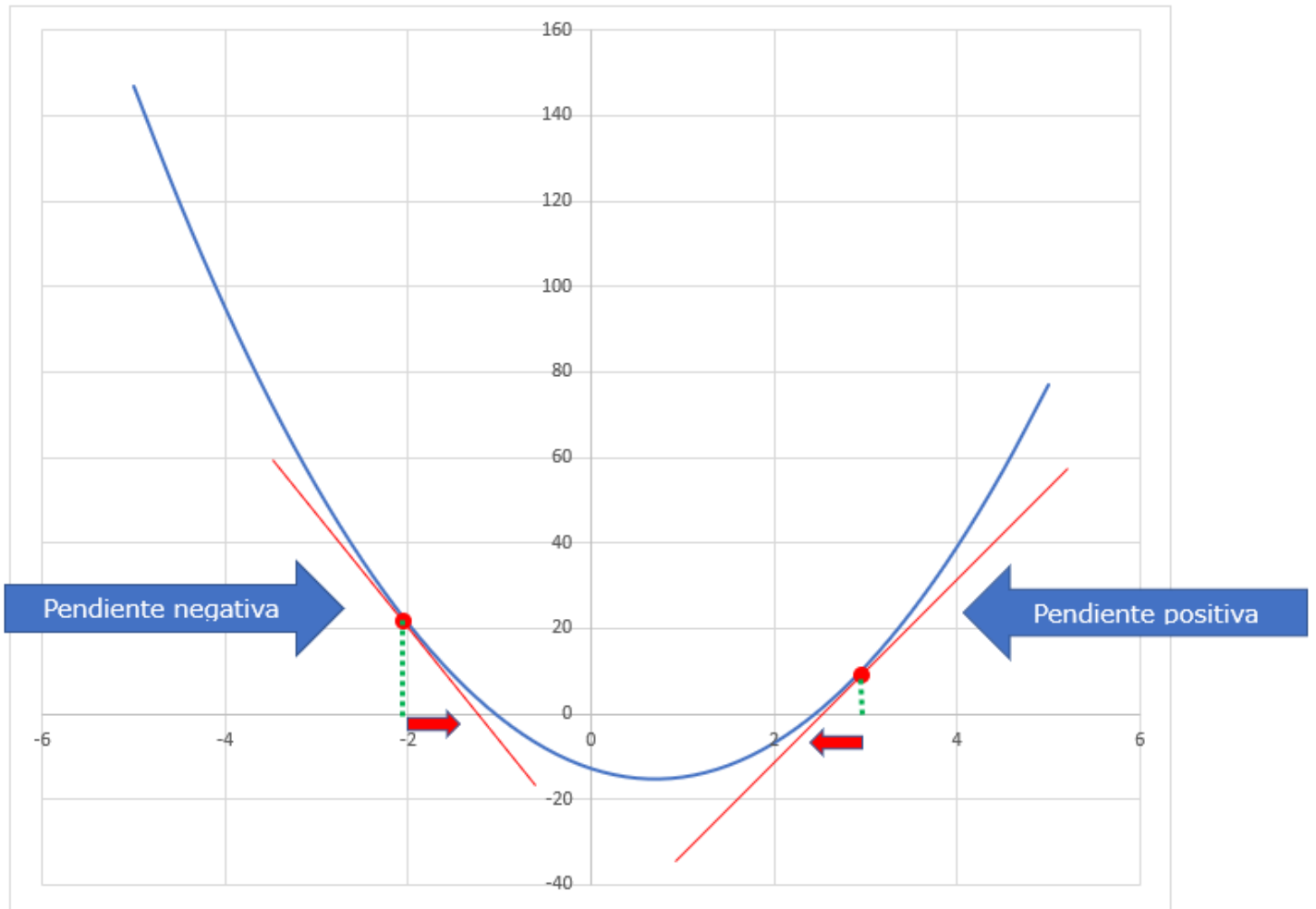


Ilustración 23: Pendientes

Para dar con el nuevo valor de X esta sería la expresión:

$$x_{nuevo} = x_{anterior} + \Delta x$$

Reemplazando

$$x_{nuevo} = x_{anterior} - y'$$

## EJEMPLO

Con la ecuación anterior

$$y = 5 * x^2 - 7 * x - 13$$

$$y' = 10 * x - 7$$

$$x_{nuevo} = x_{anterior} - y'$$

$$x_{nuevo} = x_{anterior} - (10 * x - 7)$$

X inicia en 0.4 por ejemplo, luego

$$x_{nuevo} = 0.4 - (10 * 0.4 - 7)$$

$$x_{nuevo} = -3.4$$

Ahora hay un nuevo valor para X que es -2. En la siguiente tabla se muestra como progresa X

| <b>Xanterior</b> | <b>Xnuevo</b> | <b>Y</b>    |
|------------------|---------------|-------------|
| 0.4              | 3.4           | 21          |
| 3.4              | -23.6         | 2937        |
| -23.6            | 219.4         | 239133      |
| 219.4            | -1967.6       | 19371009    |
| -1967.6          | 17715.4       | 1569052965  |
| 17715.4          | -159431.6     | 1.27093E+11 |
| -159431.6        | 1434891.4     | 1.02946E+13 |
| 1434891.4        | -12914015.6   | 8.33859E+14 |
| -12914015.6      | 116226147.4   | 6.75426E+16 |
| 116226147.4      | -1046035320   | 5.47095E+18 |
| -1046035320      | 9414317883    | 4.43147E+20 |

El valor de X se dispara, se vuelve extremo hacía la izquierda o derecha. Se debe arreglar agregando una constante a la ecuación:

$$x_{nuevo} = x_{anterior} - \alpha * y'$$

Se agrega entonces un  $\alpha$  que es una constante muy pequeña, por ejemplo  $\alpha=0.05$  y esto es lo que sucede

$$x_{nuevo} = x_{anterior} - 0.05 * (10 * x_{anterior} - 7)$$

| <b>Xanterior</b> | <b>Xnuevo</b> | <b>Y</b>    |
|------------------|---------------|-------------|
| 0.4              | 0.55          | -15.3375    |
| 0.55             | 0.625         | -15.421875  |
| 0.625            | 0.6625        | -15.4429688 |
| 0.6625           | 0.68125       | -15.4482422 |
| 0.68125          | 0.690625      | -15.4495605 |
| 0.690625         | 0.6953125     | -15.4498901 |

|             |             |             |
|-------------|-------------|-------------|
| 0.6953125   | 0.69765625  | -15.4499725 |
| 0.69765625  | 0.698828125 | -15.4499931 |
| 0.698828125 | 0.699414063 | -15.4499983 |
| 0.699414063 | 0.699707031 | -15.4499996 |
| 0.699707031 | 0.699853516 | -15.4499999 |

Tiene más sentido y se acerca a  $X=0.7$  que es la respuesta correcta.

Este método se le conoce como el descenso del gradiente que se expresa así:

$$x_{nuevo} = x_{anterior} - \alpha * f'(x_{anterior})$$

En formato matemático

$$x_{n+1} = x_n - \alpha * f'(x_n)$$



## Mínimos locales y globales

La siguiente curva es generada por el siguiente polinomio

$$y = 0.1 * x^6 + 0.6 * x^5 - 0.7 * x^4 - 6 * x^3 + 2 * x^2 + 2 * x + 1$$

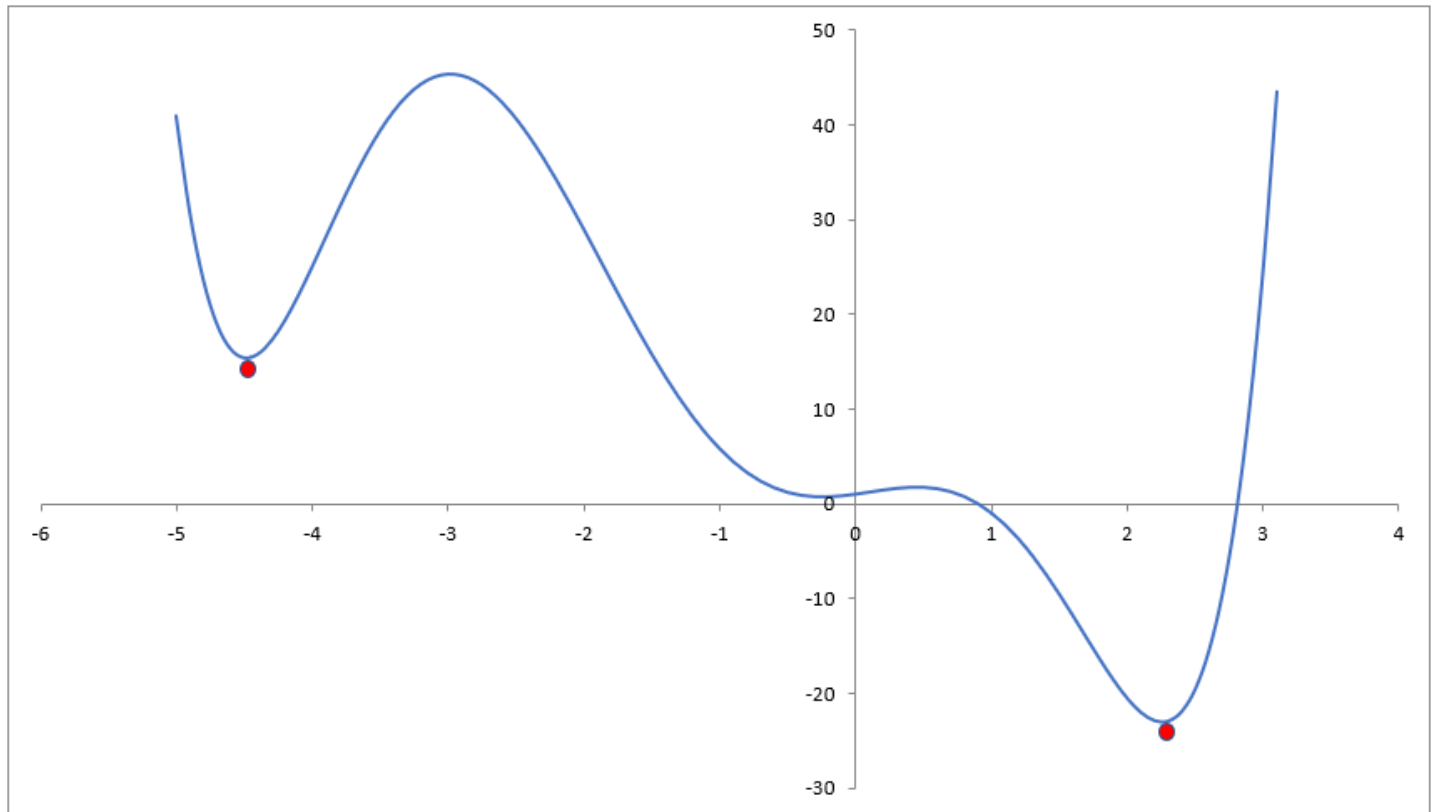


Ilustración 24: Gráfico de un polinomio de quinto grado

Se aprecian dos puntos donde claramente la curva desciende y vuelve a ascender (se han marcado con puntos en rojo), por supuesto, el segundo a la derecha es el mínimo real, pero ¿Qué pasaría si se hubiese hecho una búsqueda iniciando en  $x = -4$ ? La respuesta es que el algoritmo se hubiese decantado por el mínimo de la izquierda:

$$x_{nuevo} = x_{anterior} - \alpha * y'$$

$$x_{nuevo} = x_{anterior} - 0.01 * (0.6 * x^5 + 3 * x^4 - 2.8 * x^3 - 18 * x^2 + 4 * x + 2)$$

| Xanterior | Xnuevo  | Y       |
|-----------|---------|---------|
| -4        | -4.308  | 25      |
| -4.308    | -4.4838 | 17.0485 |

|         |         |         |
|---------|---------|---------|
| -4.4838 | -4.4815 | 15.3935 |
| -4.4815 | -4.4822 | 15.3933 |
| -4.4822 | -4.482  | 15.3933 |
| -4.482  | -4.482  | 15.3933 |

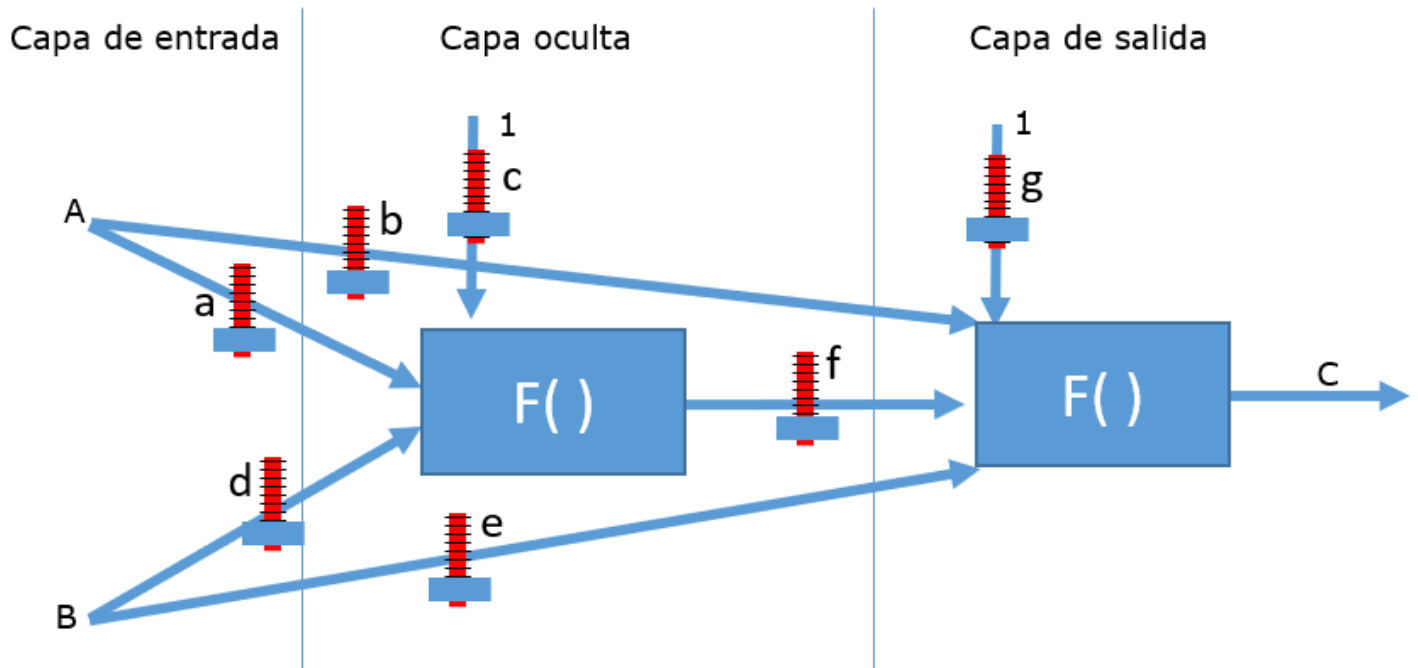
Este problema se le conoce como caer en mínimo local y también lo sufren los algoritmos evolutivos. Así que se deben probar otros valores de X para iniciar, si fuese  $X=2$  se observa que si acierta con el mínimo real:

| <b>Xanterior</b> | <b>Xnuevo</b> | <b>Y</b> |
|------------------|---------------|----------|
| 2                | 2.172         | -20.6    |
| 2.172            | 2.24349       | -22.777  |
| 2.24349          | 2.25489       | -23.08   |
| 2.25489          | 2.25559       | -23.087  |
| 2.25559          | 2.25562       | -23.087  |
| 2.25562          | 2.25563       | -23.087  |
| 2.25563          | 2.25563       | -23.087  |

Fue fácil darse cuenta donde está el mínimo real viendo la gráfica, pero el problema estará vigente cuando no sea fácil generar el gráfico o peor aún, cuando no sea una sola variable independiente  $f(x)$  sino varias, como funciones del tipo  $f(a, b, c, d, e)$

## Búsqueda de mínimos y redes neuronales

En la figura, hay dos entradas: A y B, y una salida: C, todo eso son constantes porque son los datos de entrenamiento, no hay control sobre estos. Lo que, si se puede variar, son los pesos. Si se quiere saber que tanto se debe ajustar cada peso, el procedimiento matemático de obtener mínimos, se enfoca solamente en esos pesos.



*Ilustración 25: Red neuronal con varias conexiones distintas*

En la figura se aprecian 7 pesos: a, b, c, d, e, f, g. ¿Cómo obtener un mínimo? En ese caso se utilizan derivadas parciales, es decir, se deriva por 'a' dejando el resto como constantes, luego por 'b' dejando el resto constantes y así sucesivamente. Esos mínimos servirán para ir ajustando los pesos.

# Perceptrón Multicapa

Es un tipo de red neuronal en donde hay varias capas:

1. Capa de entrada
2. Capas ocultas
3. Capa de salida

En la siguiente figura se muestra un ejemplo de perceptrón multicapa, los círculos representan las neuronas. Tiene dos capas ocultas. Las capas ocultas donde cada una tiene 3 neuronas y la capa de salida con 2 neuronas.

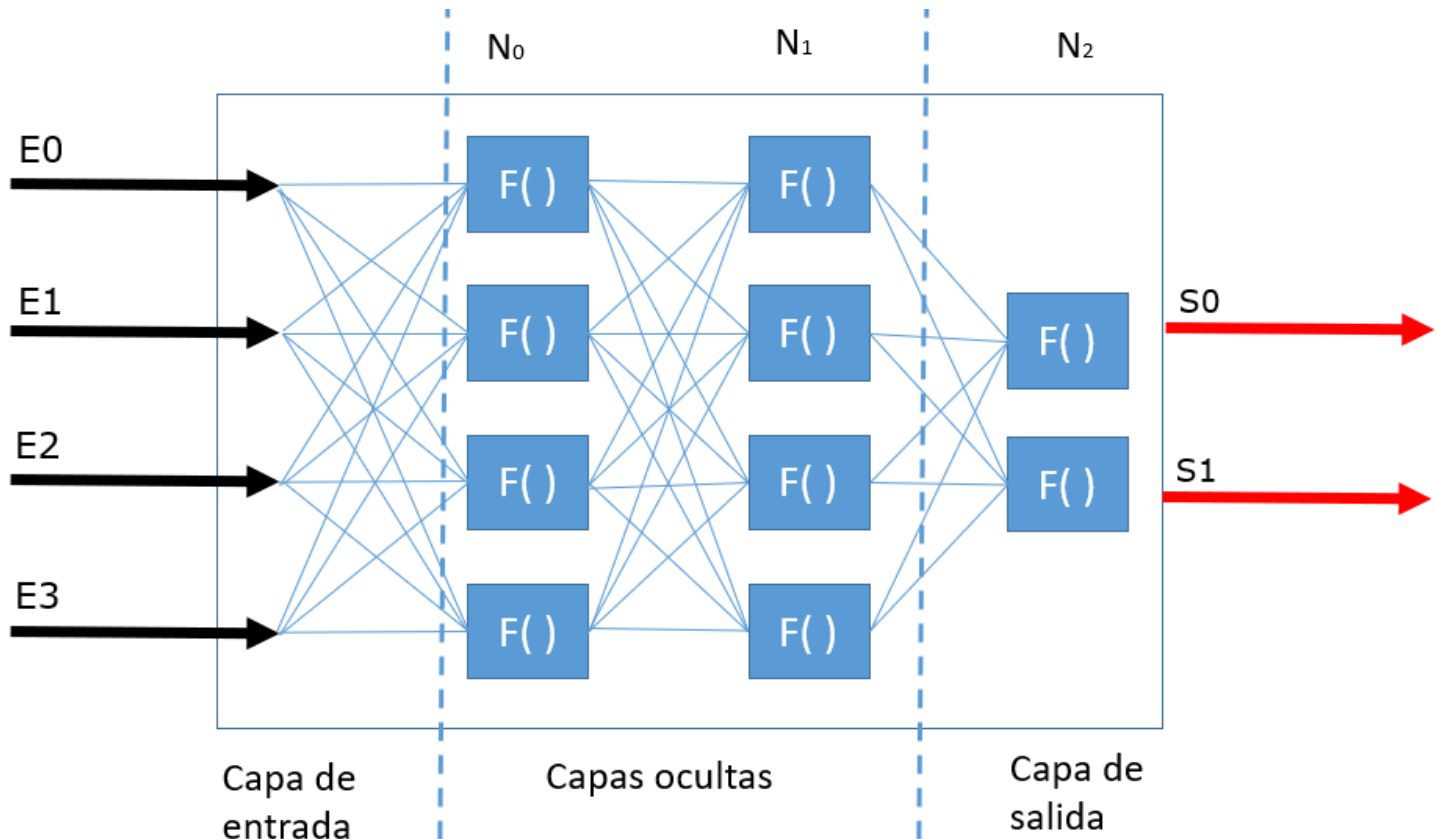


Ilustración 26: Perceptrón multicapa

Las capas se denotarán con la letra 'N', luego

$N_0=4$  (capa 0, que es oculta, tiene 4 neuronas)

$N_1=4$  (capa 1, que es oculta, tiene 4 neuronas)

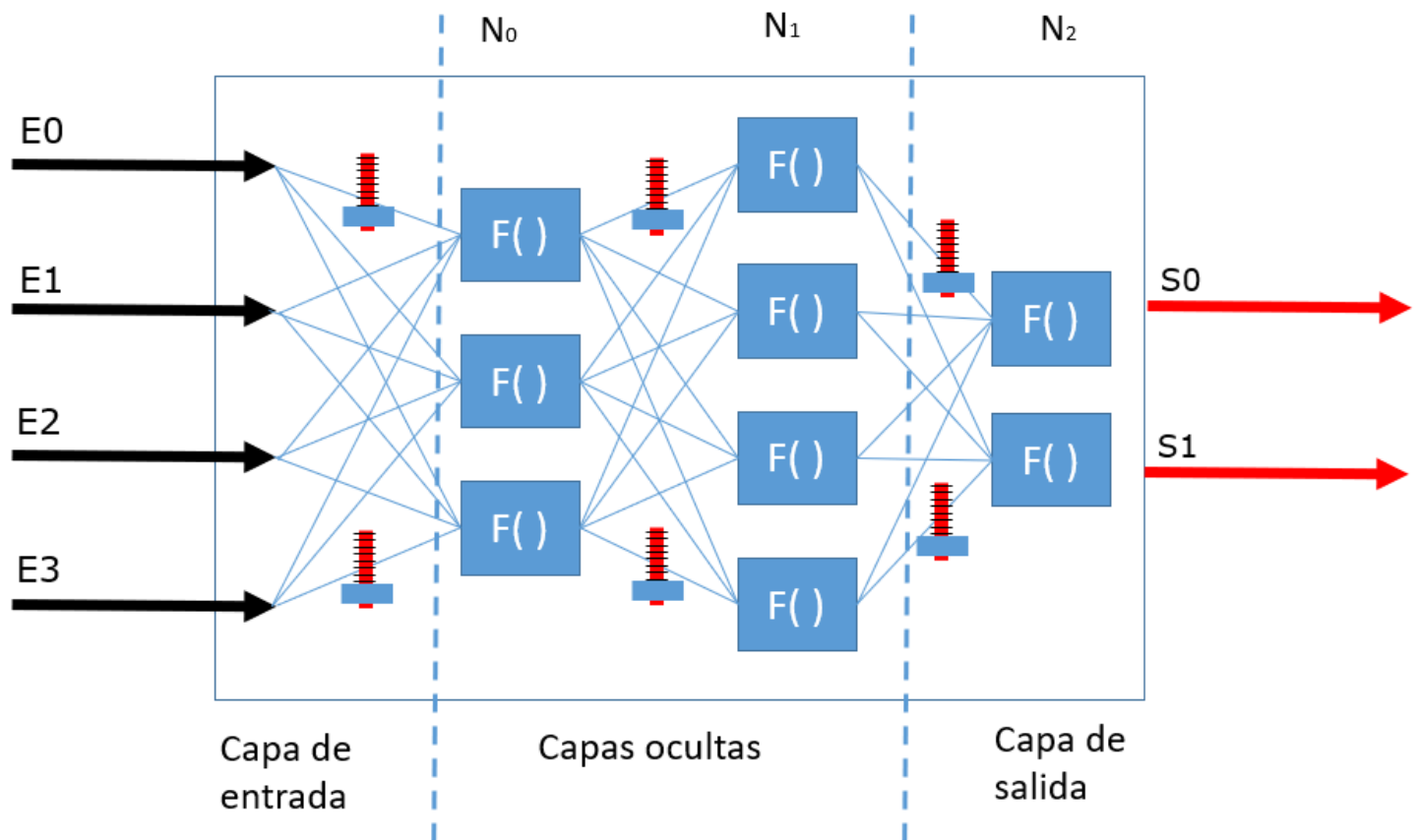
$N_2=2$  (capa 2, que es la de salida, tiene 2 neuronas)

La capa de entrada no hace ningún proceso, sólo recibe los datos de entrada.

En el perceptrón multicapa, las neuronas de la capa 0 se conectan con las neuronas de la capa 1, las neuronas de la capa 1 con las neuronas de la capa 2. No está permitido conectar neuronas de la capa 0 con las neuronas de la capa 2 por ejemplo, ese salto podrá suceder en otro tipo de redes neuronales, pero no en el perceptrón multicapa.

## Las neuronas

De nuevo se muestra un esquema de cómo es una neurona con dos entradas externas y su salida.



*Ilustración 27: Esquema de una neurona*

Mostrado como una clase en C#, esta sería su implementación:

K/004.cs

```
namespace Ejemplo;

class Neurona {
    public double CalculaSalida(double E0, double E1) {
        double S = 0;

        //Se hace una operación aquí

        return S;
    }
}

class Program {
    static void Main() {
        Neurona objA = new();
        Neurona objB = new();
    }
}
```

En cada entrada hay un peso P0 y P1. Para la entrada interna, que siempre es 1, el peso se llama U

K/005.cs

```
namespace Ejemplo;

class Neurona {
    //Pesos para cada entrada P0 y P1; y el peso de la entrada interna U
    private double P0;
    private double P1;
    private double U;

    public double CalculaSalida(double E0, double E1) {
        double S = 0;

        //Se hace una operación aquí

        return S;
    }
}

class Program {
    static void Main() {
        Neurona objA = new();
        Neurona objB = new();
    }
}
```

Los pesos se inicializan con un valor al azar y un buen sitio es hacerlo en el constructor. En el ejemplo se hace uso de la clase Random y luego NextDouble() que retorna un número real al azar entre 0 y 1.

K/006.cs

```
namespace Ejemplo;

class Neurona {
    //Pesos para cada entrada P0 y P1; y el peso de la entrada interna U
    private double P0;
    private double P1;
    private double U;

    public Neurona(Random Azar) { //Constructor
        P0 = Azar.NextDouble();
        P1 = Azar.NextDouble();
        U = Azar.NextDouble();
    }

    public double CalculaSalida(double E0, double E1) {
        double S = 0;

        //Se hace una operación aquí

        return S;
    }
}

class Program {
    static void Main() {
        Random Azar = new();
        Neurona objA = new(Azar);
        Neurona objB = new(Azar);
    }
}
```



## Pesos y como nombrarlos

En el gráfico se dibujan algunos pesos y como se podrá dilucidar, el número de estos pesos crece rápidamente a medida que se agregan capas y neuronas.

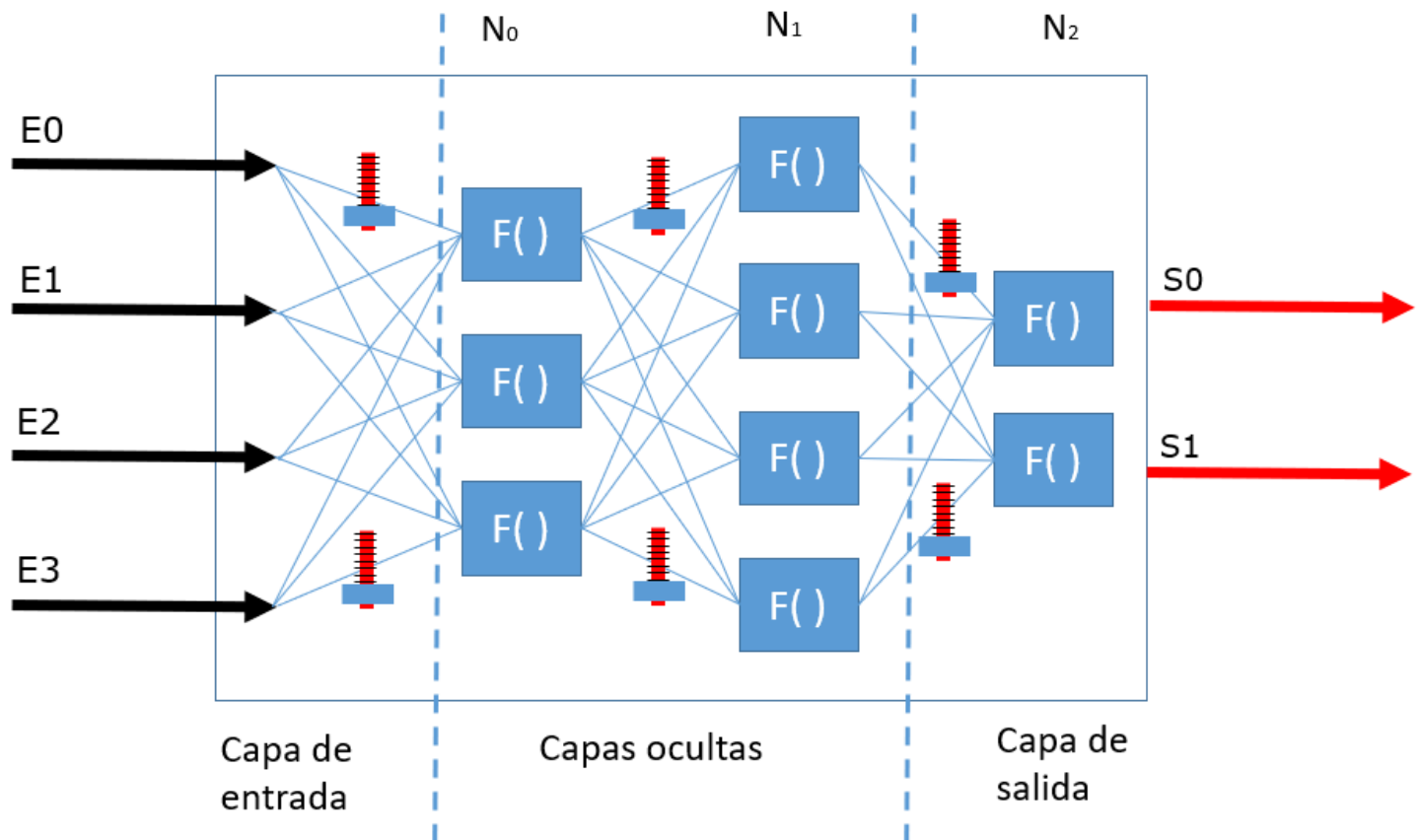


Ilustración 28: Esquema de un perceptrón multicapa

Un ejemplo: Capa 0 tiene 3 neuronas, capa 1 tiene 4 neuronas, luego el total de conexiones entre Capa 0 y Capa 1 son  $3 \times 4 = 12$  conexiones, luego son 12 pesos. Si se usaran más neuronas por capa, habría tantos pesos que nombrarlos con una sola letra no sería conveniente. Por tal motivo, hay otra forma de nombrarlos y es el siguiente:

$$w_{\text{neurona inicial, neurona final}}^{(\text{capa a donde llega la conexión})}$$

W es la letra inicial de la palabra peso en inglés: Weight.

(Capa de donde sale la conexión) Las capas se enumeran desde 0 que sería en este caso la primera capa oculta. ¿Cómo nombrar los pesos iniciales? Esos pesos de la capa de entrada tendrán como "capa de donde sale la conexión" la letra E.

Neurona inicial, de donde parte la conexión. Se enumeran desde 0 de arriba abajo en la capa.

Neurona final, a donde llega la conexión. Se enumeran desde 0 de arriba abajo en la capa.

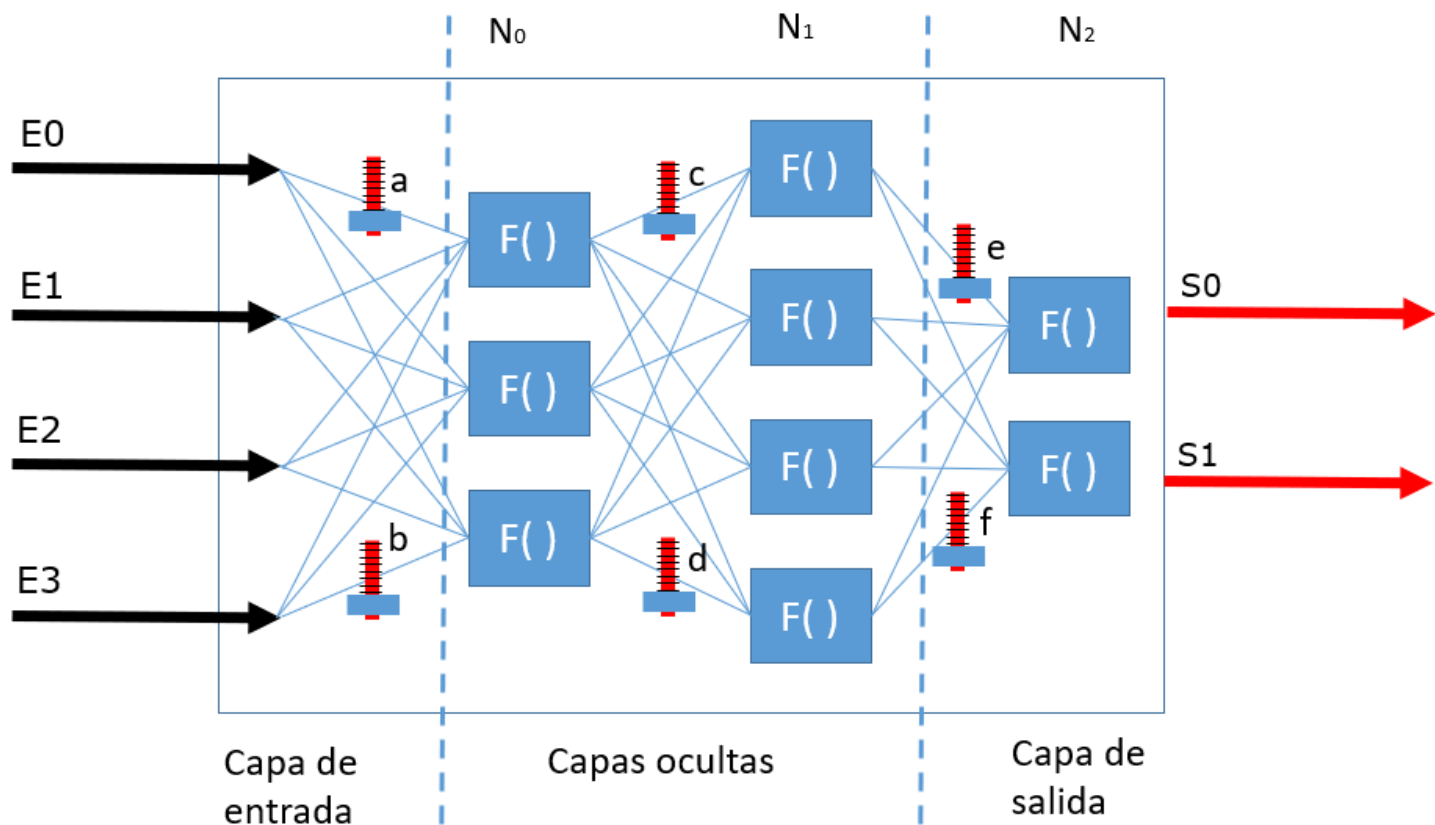


Ilustración 29: Nombrando los pesos con una letra

Para nombrar el peso mostrado con la letra 'a' sería entonces:

$$w_{\text{neurona inicial, neurona final}}^{(\text{capa a donde llega la conexión})}$$

$$w_{0,0}^{(0)}$$

En esta tabla se muestra como se nombrarían los pesos que se han puesto en la gráfica:

| Peso | Se nombra       |
|------|-----------------|
| a    | $w_{0,0}^{(0)}$ |
| b    | $w_{3,2}^{(0)}$ |
| c    | $w_{0,0}^{(1)}$ |
| d    | $w_{2,3}^{(1)}$ |
| e    | $w_{0,0}^{(2)}$ |
| f    | $w_{3,1}^{(2)}$ |

## La función de activación de la neurona

Anteriormente se había mostrado que la función de activación era esta:

```
Función F(valor)
Inicio
  Si valor > 0.5 entonces
    retorne 1
  de lo contrario
    retorne 0
  fin si
Fin
```

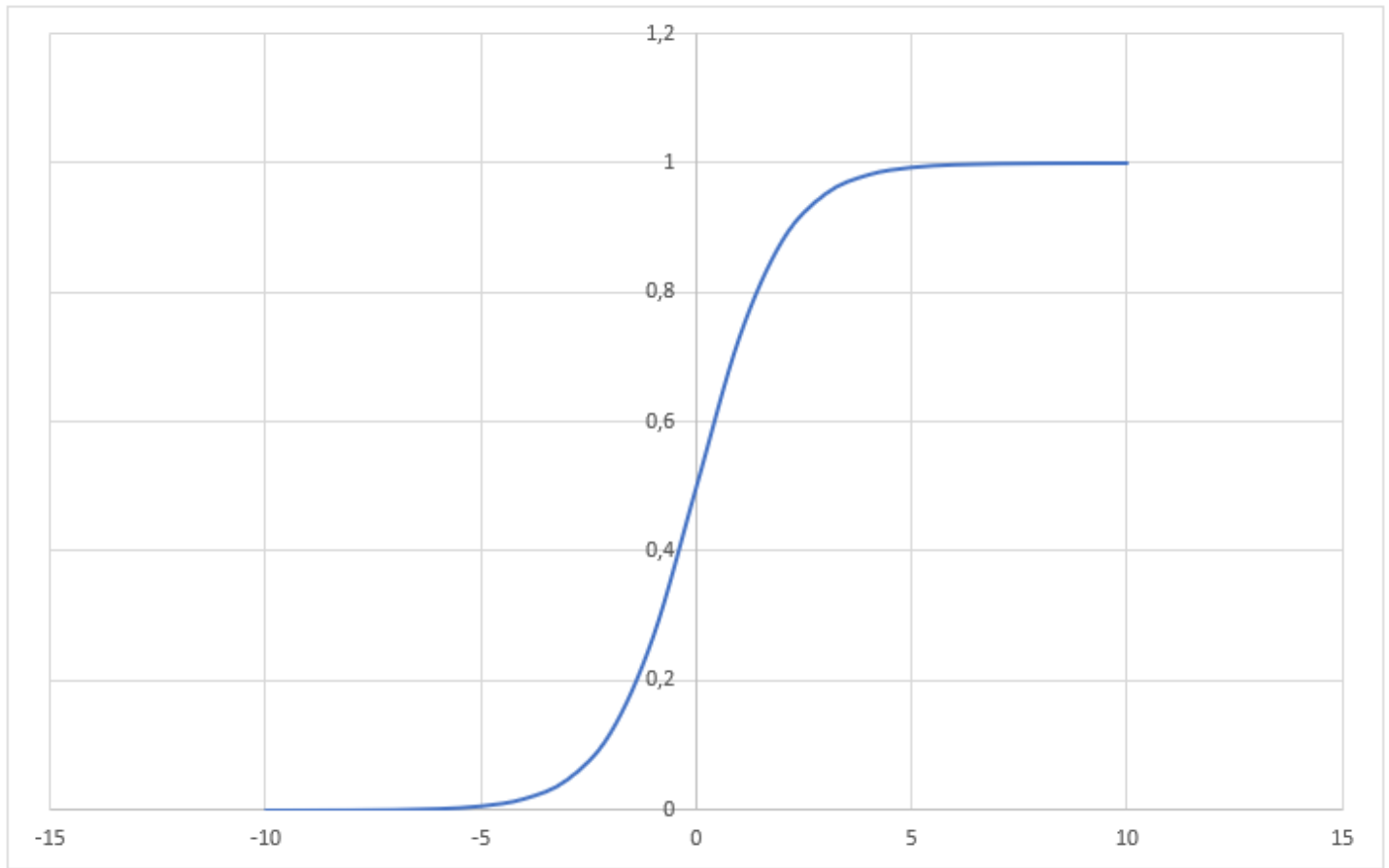
En otros problemas, por lo general, esa función es la sigmoide que tiene la siguiente ecuación:

$$y = \frac{1}{1 + e^{-x}}$$

Esta sería una tabla de valores generados con esa función

| <b>x</b> | <b>y</b> |
|----------|----------|
| -10      | 4.5E-05  |
| -9       | 0.00012  |
| -8       | 0.00034  |
| -7       | 0.00091  |
| -6       | 0.00247  |
| -5       | 0.00669  |
| -4       | 0.01799  |
| -3       | 0.04743  |
| -2       | 0.1192   |
| -1       | 0.26894  |
| 0        | 0.5      |
| 1        | 0.73106  |
| 2        | 0.8808   |
| 3        | 0.95257  |
| 4        | 0.98201  |
| 5        | 0.99331  |
| 6        | 0.99753  |
| 7        | 0.99909  |
| 8        | 0.99966  |
| 9        | 0.99988  |
| 10       | 0.99995  |

Y esta sería su gráfica



*Ilustración 30: Gráfico de una función sigmoide*

Al moverse a la izquierda el valor que toma es 0 y al moverse a la derecha toma el valor de 1. Hay una transición pronunciada de 0 a 1 en el rango [-5 y 5].

¿Qué tiene de especial esta función sigmoide? Su derivada.

Ecuación original:

$$y = \frac{1}{1 + e^{-x}}$$

Derivada no negativa de esa ecuación:

$$\partial y = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Que equivale a esto:

$$\partial y = y * (1 - y)$$

Demostración de la equivalencia:

$$\begin{aligned}\partial y &= \frac{1}{1 + e^{-x}} * \left[ 1 - \frac{1}{1 + e^{-x}} \right] \\ \partial y &= \frac{1}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} * \frac{1}{1 + e^{-x}} \\ \partial y &= \frac{1}{1 + e^{-x}} - \frac{1}{(1 + e^{-x})^2} \\ \partial y &= \frac{1 + e^{-x} - 1}{(1 + e^{-x})^2} \\ \partial y &= \frac{e^{-x}}{(1 + e^{-x})^2}\end{aligned}$$

Nota: Si hace uso de WolframAlpha y deriva la sigmoidea, sucede esto:

derive  $y=1/(1+e^{-x})$

 Extended Keyboard

 Upload

Input interpretation:

differentiate

$$y = \frac{1}{1 + e^{-x}}$$

with respect to

$x$

Result:

$$y'(x) = \frac{e^x}{(e^x + 1)^2}$$

*Ilustración 31: Derivada de la función sigmoide*

A primera vista parece diferente a la derivada mostrada anteriormente, pero si se compara

$$e^x/(e^x+1)^2 = e^{-x}/(1+e^{-x})^2$$

 Extended Keyboard

 Upload

Input:

$$\frac{e^x}{(e^x + 1)^2} = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Alternate form:

True

*Ilustración 32: Comparativa de derivadas*



El código del perceptrón multicapa progresa así:

K/007.cs

```
namespace Ejemplo;

class Neurona {
    //Pesos para cada entrada P0 y P1; y el peso de la entrada interna U
    private double P0;
    private double P1;
    private double U;

    public Neurona(Random Azar) { //Constructor
        P0 = Azar.NextDouble();
        P1 = Azar.NextDouble();
        U = Azar.NextDouble();
    }

    //Calcula la salida de la neurona con las dos entradas E0 y E1
    public double CalculaSalida(double E0, double E1) {
        double Valor = E0 * P0 + E1 * P1 + 1 * U;
        return 1 / (1 + Math.Exp(-Valor));
    }
}

class Program {
    static void Main() {
        Random Azar = new();
        Neurona objA = new(Azar);
        Neurona objB = new(Azar);
    }
}
```

El método calculaSalida implementa el procesamiento de la neurona. Tiene como parámetros las entradas, en este caso, dos entradas E0 y E1. En el interior cada entrada se multiplica con su peso respectivo, se suman, incluyendo la entrada interna (umbral). Una vez con ese valor, se calcula la salida con la sigmoide.

## Introducción al algoritmo “Backpropagation” (backward propagation of errors)

En el siguiente ejemplo se observa una conexión entre tres neuronas

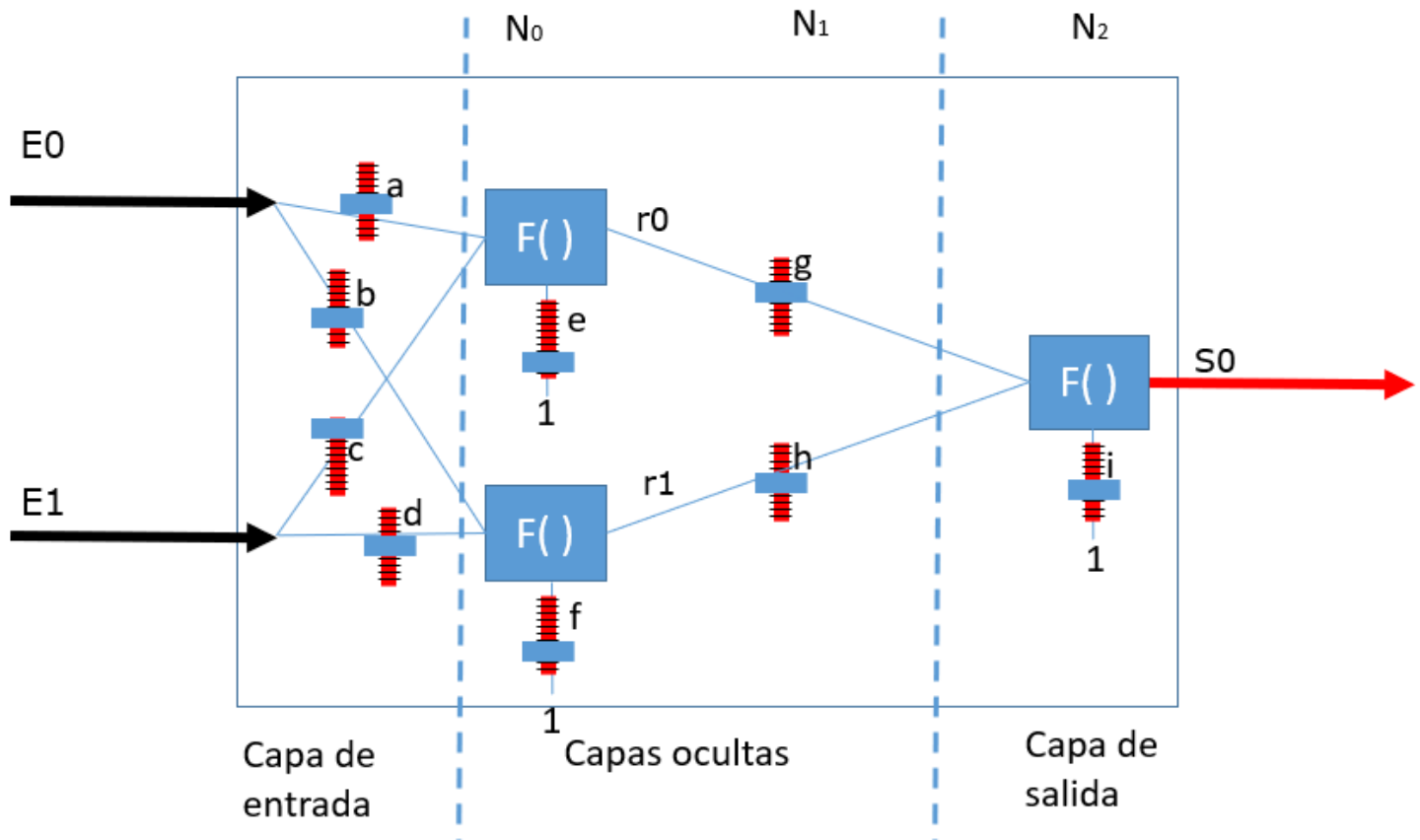


Ilustración 33: Esquema de un perceptrón multicapa

E<sub>0</sub> y E<sub>1</sub> son las entradas externas. Se observa que E<sub>0</sub> entra con el peso 'a' en la neurona de arriba y con peso 'b' en la neurona de abajo. Sucede lo mismo con la entrada E<sub>1</sub>.

Lo interesante viene después, porque el resultado de la neurona de arriba 'r<sub>0</sub>' y el resultado de la neurona de abajo 'r<sub>1</sub>' se convierten en entradas para la neurona de la derecha y esas entradas a su vez tienen sus propios pesos. Al final el sistema genera una salida S<sub>0</sub> que es la respuesta final de la red neuronal.

Obsérvese que cada neurona tiene la entrada 1 y su peso llamado umbral.

¿Qué importancia tiene eso? Que, si se quiere ajustar S<sub>0</sub> al resultado esperado, entonces se retrocede a las entradas de esa neurona de la derecha ajustando sus pesos respectivos y por supuesto, ese ajuste hace retroceder más aún hasta mirar los pesos de las neuronas de arriba y abajo. Eso se conocerá como el algoritmo “Backpropagation”.

Luego:

$$r0 = F(E0 * a + E1 * c + 1 * e)$$

$$r1 = F(E0 * b + E1 * d + 1 * f)$$

$$S0 = F(r0 * g + r1 * h + 1 * i)$$

Se concluye entonces que:

$$S0 = F(F(E0 * a + E1 * c + 1 * e) * g + F(E0 * b + E1 * d + 1 * f) * h + 1 * i)$$

## Nombrando las entradas, pesos, umbrales, salidas y capas

Volviendo al perceptrón multicapa. Es momento de ponerle nombres a las diversas partes de la red neuronal:

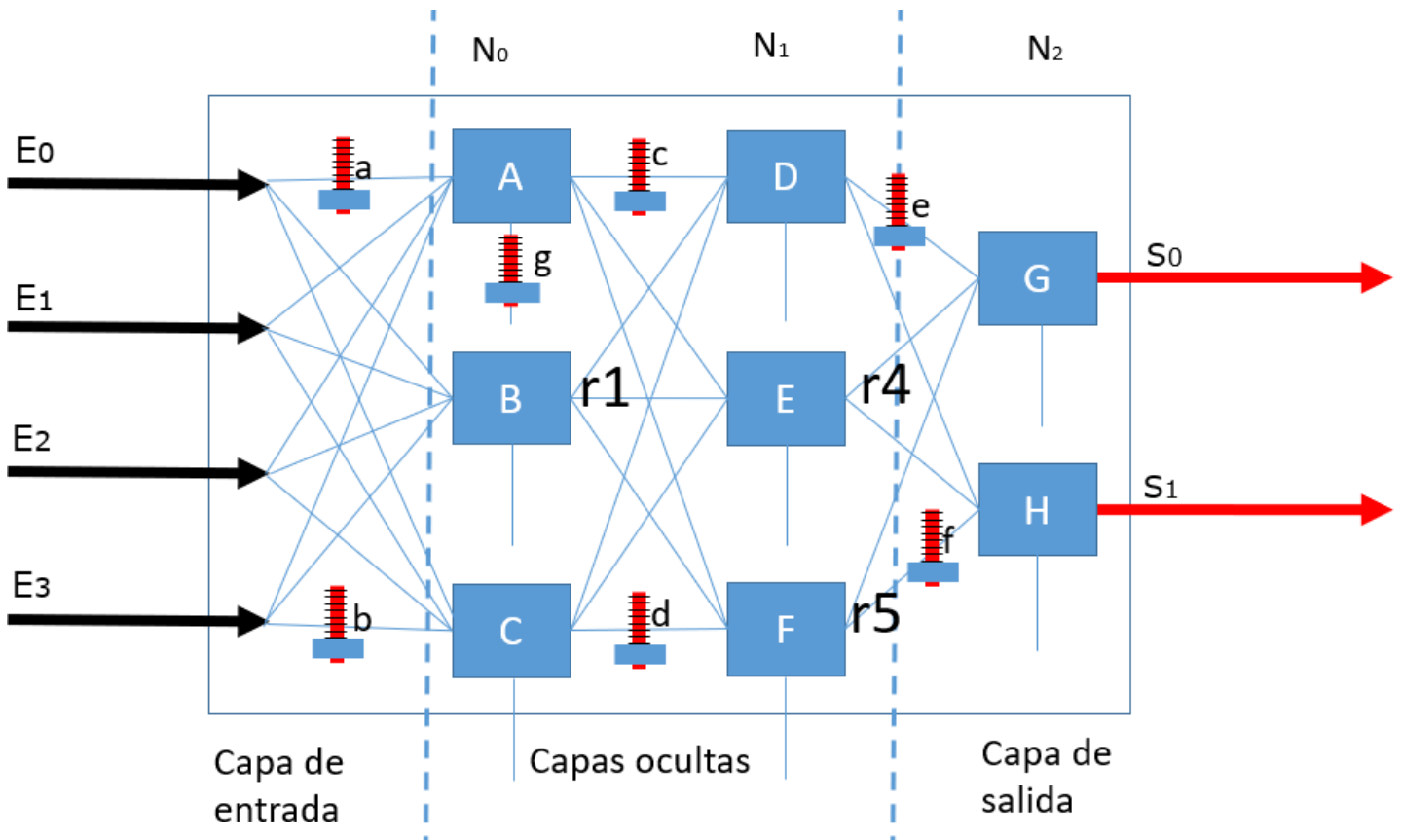


Ilustración 34: Partes de un perceptrón multicapa

Las entradas:

$$E_{\text{número de la entrada}}$$

Los pesos:

$$w_{\text{neurona inicial, neurona final}}^{(\text{capa a donde llega la conexión})}$$

Los umbrales:

$$u_{\text{neurona que tiene esa entrada interna}}^{(\text{capa de la neurona que tiene esa entrada interna})}$$

Las salidas internas de cada neurona:

$$a_{\text{neurona de esa salida}}^{(\text{capa de la neurona de esa salida})}$$

Las capas:

$n$  número de la capa

Entonces, viendo el gráfico:

| Como se nombró antes | Nueva nomenclatura |
|----------------------|--------------------|
| $E_0$                | $E_0$              |
| $E_1$                | $E_1$              |
| $E_2$                | $E_2$              |
| $E_3$                | $E_3$              |
| a                    | $w_{0,0}^{(0)}$    |
| b                    | $w_{3,3}^{(0)}$    |
| c                    | $w_{0,0}^{(1)}$    |
| d                    | $w_{2,2}^{(1)}$    |
| e                    | $w_{0,0}^{(2)}$    |
| f                    | $w_{2,1}^{(2)}$    |
| g                    | $u_0^{(0)}$        |
| r1                   | $a_1^{(0)}$        |
| r4                   | $a_1^{(1)}$        |
| r5                   | $a_2^{(1)}$        |
| $S_0$                | $a_0^{(2)}$        |
| $S_1$                | $a_1^{(2)}$        |

Luego:

$$a_0^{(2)} = F \left( a_0^{(1)} * w_{0,0}^{(2)} + a_1^{(1)} * w_{1,0}^{(2)} + a_2^{(1)} * w_{2,0}^{(2)} + u_0^{(2)} \right)$$

$$a_1^{(2)} = F \left( a_0^{(1)} * w_{0,1}^{(2)} + a_1^{(1)} * w_{1,1}^{(2)} + a_2^{(1)} * w_{2,1}^{(2)} + u_1^{(2)} \right)$$

$$a_0^{(1)} = F \left( a_0^{(0)} * w_{0,0}^{(1)} + a_1^{(0)} * w_{1,0}^{(1)} + a_2^{(0)} * w_{2,0}^{(1)} + u_0^{(1)} \right)$$

$$a_1^{(1)} = F \left( a_0^{(0)} * w_{0,1}^{(1)} + a_1^{(0)} * w_{1,1}^{(1)} + a_2^{(0)} * w_{2,1}^{(1)} + u_1^{(1)} \right)$$

$$a_2^{(1)} = F \left( a_0^{(0)} * w_{0,2}^{(1)} + a_1^{(0)} * w_{1,2}^{(1)} + a_2^{(0)} * w_{2,2}^{(1)} + u_2^{(1)} \right)$$

$$a_0^{(0)} = F \left( E_0 * w_{0,0}^{(0)} + E_1 * w_{1,0}^{(0)} + E_2 * w_{2,0}^{(0)} + E_3 * w_{3,0}^{(0)} + u_0^{(0)} \right)$$

$$a_1^{(0)} = F \left( E_0 * w_{0,1}^{(0)} + E_1 * w_{1,1}^{(0)} + E_2 * w_{2,1}^{(0)} + E_3 * w_{3,1}^{(0)} + u_1^{(0)} \right)$$

$$a_2^{(0)} = F \left( E_0 * w_{0,2}^{(0)} + E_1 * w_{1,2}^{(0)} + E_2 * w_{2,2}^{(0)} + E_3 * w_{3,2}^{(0)} + u_2^{(0)} \right)$$

¿Qué hay de  $E_0$ ,  $E_1$ ,  $E_2$  y  $E_3$ ? Podrían tomarse como salidas de las neuronas de la capa E (Entrada). Cabe recordar que en esa capa no hay procesamiento. Luego:

| Como se nombró antes | Nueva nomenclatura |
|----------------------|--------------------|
| $E_0$                | $a_0^{(E)}$        |
| $E_1$                | $a_1^{(E)}$        |
| $E_2$                | $a_2^{(E)}$        |
| $E_3$                | $a_3^{(E)}$        |

Luego las tres últimas ecuaciones quedan así:

$$a_0^{(0)} = F \left( a_0^{(E)} * w_{0,0}^{(0)} + a_1^{(E)} * w_{1,0}^{(0)} + a_2^{(E)} * w_{2,0}^{(0)} + a_3^{(E)} * w_{3,0}^{(0)} + u_0^{(0)} \right)$$

$$a_1^{(0)} = F \left( a_0^{(E)} * w_{0,1}^{(0)} + a_1^{(E)} * w_{1,1}^{(0)} + a_2^{(E)} * w_{2,1}^{(0)} + a_3^{(E)} * w_{3,1}^{(0)} + u_1^{(0)} \right)$$

$$a_2^{(0)} = F \left( a_0^{(E)} * w_{0,2}^{(0)} + a_1^{(E)} * w_{1,2}^{(0)} + a_2^{(E)} * w_{2,2}^{(0)} + a_3^{(E)} * w_{3,2}^{(0)} + u_2^{(0)} \right)$$

¡OJO! Las capas tendrían este orden  $E, 0, 1, 2$

Y generalizando se puede decir que:

$$a_i^{(k)} = F \left( a_j^{(k-1)} * w_{j,i}^{(k)} + a_{j+1}^{(k-1)} * w_{j+1,i}^{(k)} + a_{j+2}^{(k-1)} * w_{j+2,i}^{(k)} + a_{j+3}^{(k-1)} * w_{j+3,i}^{(k)} + 1 * u_i^{(k)} \right)$$

Es decir que si  $k-1 < 0$  entonces se está refiriendo a la capa E la de entrada.

Se puede simplificar más:

$$a_i^{(k)} = f( u_i^{(k)} + \sum_{j=0}^{n_k-1} a_j^{(k-1)} * w_{j,i}^{(k)} )$$

Donde k inicia en 0 y termina en el número de la última capa.



## Regla de la cadena

Para continuar con el algoritmo de "Backpropagation", cabe recordar esta regla matemática llamada regla de la cadena.

$$\partial\{F[g(x)]\} = \partial F[g(x)] * \partial g(x)$$

Un ejemplo, hay dos funciones:

$$g(x) = 3 * x^2$$

$$F[p] = 7 - p^3$$

Luego:

$$F[g(x)] = 7 - (3 * x^2)^3 = 7 - 27 * x^6$$

Reemplazando "p" con lo que tiene g(x)

Derivando

$$\partial\{F[g(x)]\} = -162 * x^5$$

Usando la regla de la cadena

$$\begin{aligned}\partial\{F[g(x)]\} &= \partial F[g(x)] * \partial g(x) = \partial(7 - p^3) * \partial(3 * x^2) = \\ &= (0 - 3 * p^2) * (6 * x) = (0 - 3 * (3 * x^2)^2) * (6 * x) = \\ &= (0 - 3 * (9 * x^4)) * (6 * x) = -162 * x^5\end{aligned}$$

La regla de la cadena funciona.

## Derivadas parciales

Dada una ecuación que tenga dos o más variables independientes, es posible derivar por una variable considerando las demás constantes, eso es conocido como derivada parcial.

Ejemplo de una ecuación con tres variables independientes:

$$q = a^2 + b^3 + c^4$$

Su derivada parcial con respecto a la variable **a** sería

$$\frac{\partial q}{\partial a}(a, b, c) = 2 * a + 0 + 0$$

$$\frac{\partial q}{\partial a}(a, b, c) = 2 * a$$

Su derivada parcial con respecto a la variable **b** sería

$$\frac{\partial q}{\partial b}(a, b, c) = 0 + 3 * b^2 + 0$$

$$\frac{\partial q}{\partial b}(a, b, c) = 3 * b^2$$

Su derivada parcial con respecto a la variable **c** sería

$$\frac{\partial q}{\partial c}(a, b, c) = 0 + 0 + 4 * c^3$$

$$\frac{\partial q}{\partial c}(a, b, c) = 4 * c^3$$

d/da a^2+b^3+c^4

 Extended Keyboard

 Upload

 Examp

Derivative:

☒ St

$$\frac{\partial}{\partial a}(a^2 + b^3 + c^4) = 2a$$

*Ilustración 35: Derivada parcial*

## Las derivadas en el algoritmo de propagación hacia atrás

Obsérvese el siguiente gráfico muy sencillo de un perceptrón multicapa. Hay que recordar que la capa de entrada no hace proceso.

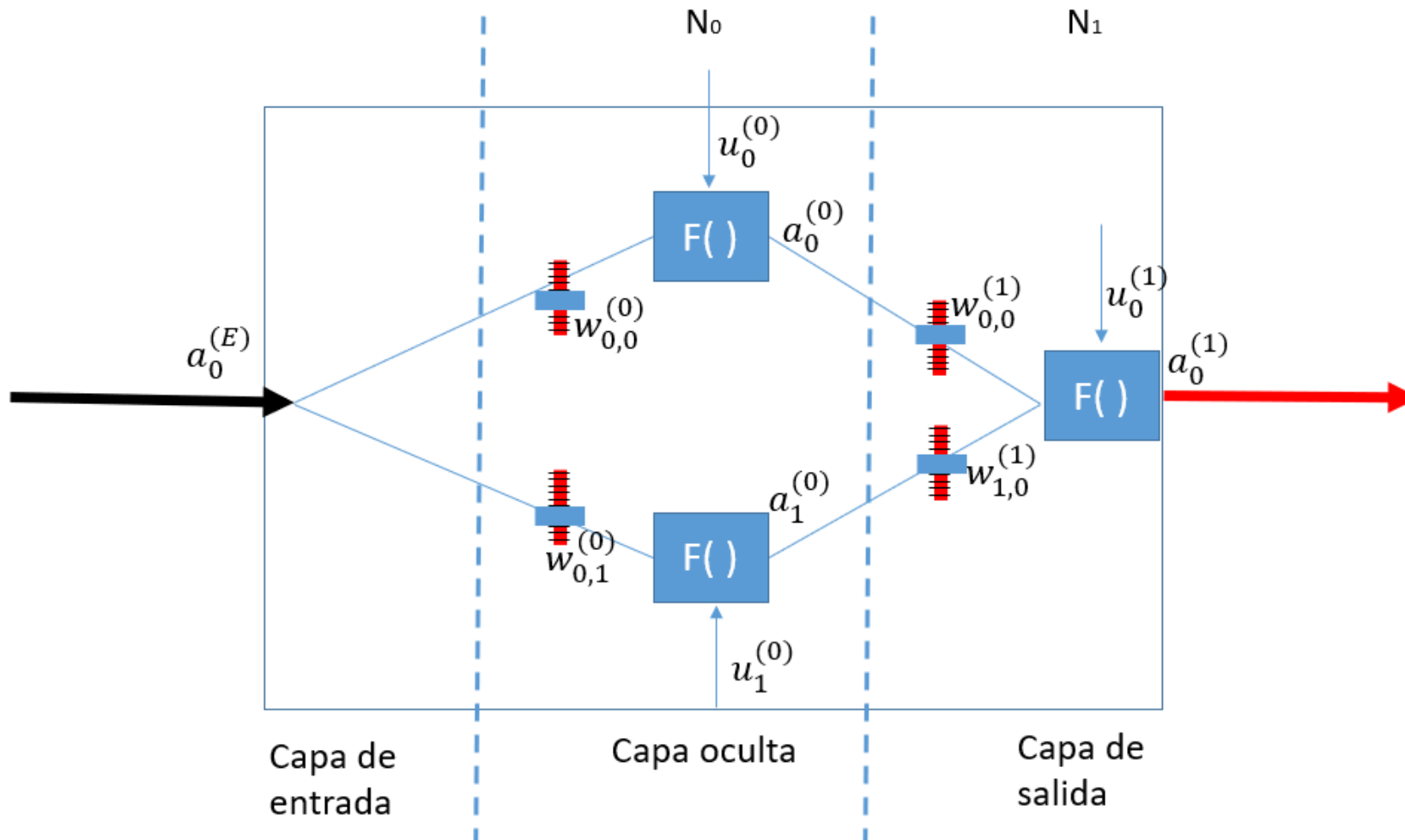


Ilustración 36: Nombramiento de pesos, umbrales y salidas

$$a_0^{(1)} = F(a_0^{(0)} * w_{0,0}^{(1)} + a_1^{(0)} * w_{1,0}^{(1)} + 1 * u_0^{(1)})$$

$$a_0^{(0)} = F(a_0^{(E)} * w_{0,0}^{(0)} + 1 * u_0^{(0)})$$

$$a_1^{(0)} = F(a_0^{(E)} * w_{0,1}^{(0)} + 1 * u_1^{(0)})$$

Luego reemplazando

$$a_0^{(1)} = F(F(a_0^{(E)} * w_{0,0}^{(0)} + 1 * u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + 1 * u_1^{(0)}) * w_{1,0}^{(1)} + 1 * u_0^{(1)})$$

Simplificando

$$a_0^{(1)} = F(F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)})$$

Como la función  $F[]$  es una sigmoidea y la derivada de las sigmoideas es así:

$$\partial y = y * (1 - y)$$

Luego:

$$\partial F[r] = F[r] * (1 - F[r])$$

¿Qué sucedería si  $r$  es a su vez una función sigmoidea?

$$r = g(k)$$

Tener en cuenta que  $g()$  es una sigmoidea. Y además de eso,  $k$  es una función polinómica.

Recordando la regla de la cadena, esto sucedería:

$$\partial\{F[g(k)]\} = \partial F[g(k)] * \partial g(k)$$

Y como  $F()$  es sigmoidea, entonces al derivar

$$\partial\{F[g(k)]\} = F(g(k)) * (1 - F(g(k))) * \partial g(k)$$

$g(k)$  es una función sigmoidea y  $k$  es una función polinómica, luego la derivada de  $g(k)$  aplicando la regla de la cadena sería:

$$\partial g(k) = \partial g(k) * \partial k = g(k) * (1 - g(k)) * \partial k$$

La derivada queda así

$$\partial\{F[g(k)]\} = F(g(k)) * (1 - F(g(k))) * g(k) * (1 - g(k)) * \partial k$$

Simplificando

$$\partial\{F[g(k)]\} = F(r) * (1 - F(r)) * g(k) * (1 - g(k)) * \partial k$$

Esta es la ecuación (la salida total de esa red neuronal) que se va a derivar parcialmente con respecto a un peso en particular

$$a_0^{(1)} = F(\textcolor{red}{F}(\textcolor{red}{a}_0^{(E)} * \textcolor{red}{w}_{0,0}^{(0)} + \textcolor{red}{u}_0^{(0)}) * w_{0,0}^{(1)} + \textcolor{blue}{F}(\textcolor{blue}{a}_0^{(E)} * w_{0,1}^{(0)} + \textcolor{blue}{u}_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)})$$

En el ejemplo, se deriva con respecto a  $w_{0,0}^{(0)}$  (una derivada parcial). En rojo se pone que ecuación interna es derivable con respecto a  $w_{0,0}^{(0)}$

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(0)}} = \partial F \left( \textcolor{red}{\partial} \left[ \textcolor{red}{F} \left( \textcolor{green}{a}_0^{(E)} * w_{0,0}^{(0)} + \textcolor{green}{u}_0^{(0)} \right) * \textcolor{red}{w}_{0,0}^{(0)} \right] + \textcolor{blue}{0} + 0 \right)$$

Para derivar entonces se deriva la F externa (que está en negro y es F(r)), luego la F interna (que está en rojo y es g(k) y que la multiplica la constante  $w_{0,0}^{(1)}$ ) y por último el polinomio (que es k) que está en verde porque allí está  $w_{0,0}^{(0)}$ . Hay tres derivaciones.

Entonces, se sabe que:

$$F(r) = a_0^{(1)}$$

$$g(k) = a_0^{(0)}$$

$$k = \textcolor{green}{a}_0^{(E)} * w_{0,0}^{(0)} + \textcolor{green}{u}_0^{(0)}$$

$$\partial k = \textcolor{green}{a}_0^{(E)}$$



$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(E)}} = \partial\{F[g(k)]\}$$

$$\partial\{F[g(k)]\} = F(r) * (1 - F(r)) * g(k) * (1 - g(k)) * \partial k$$

Entonces:

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(0)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_0^{(0)} * (1 - a_0^{(0)}) * a_0^{(E)} * w_{0,0}^{(1)}$$

Esta es la ecuación (la salida total de esa red neuronal) que se va a derivar parcialmente con respecto a un peso en particular

$$a_0^{(1)} = F(F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)})$$

En el ejemplo, se deriva con respecto a  $w_{0,1}^{(0)}$  (una derivada parcial). En azul se pone que ecuación interna es derivable con respecto a  $w_{0,1}^{(0)}$

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(0)}} = \partial F \left( 0 + \partial [F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)}] + 0 \right)$$

Para derivar entonces se deriva la F externa (que está en negro y es  $F(r)$ ), luego la F interna (que está en azul y es  $g(k)$ ) y que la multiplica la constante  $w_{1,0}^{(1)}$  y por último el polinomio (que es  $k$ ) que está en verde porque allí está  $w_{0,0}^{(0)}$ . Hay tres derivaciones.

Entonces, se sabe que:

$$F(r) = a_0^{(1)}$$

$$g(k) = a_1^{(0)}$$

$$k = a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}$$

$$\partial k = a_0^{(E)}$$

$$\frac{\partial a_0^{(1)}}{\partial w_{0,1}^{(0)}} = \partial \{F[g(k)]\}$$

$$\partial \{F[g(k)]\} = F(r) * (1 - F(r)) * g(k) * (1 - g(k)) * \partial k$$

Entonces:

$$\frac{\partial a_0^{(1)}}{\partial w_{0,1}^{(0)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_1^{(0)} * (1 - a_1^{(0)}) * a_0^{(E)} * w_{1,0}^{(1)}$$

Generalizando

$$\frac{\partial a_0^{(1)}}{\partial w_{0,j}^{(0)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_j^{(0)} * (1 - a_j^{(0)}) * a_0^{(E)} * w_{j,0}^{(1)}$$

Donde j puede ser 0 o 1. Esa sería la generalización para los pesos  $w_{0,0}^{(0)}$  y  $w_{0,1}^{(0)}$

Para el peso  $w_{0,0}^{(1)}$  este sería el tratamiento:

$$a_0^{(1)} = F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)}$$

En el ejemplo, se deriva con respecto a  $w_{0,0}^{(1)}$  (una derivada parcial). Lo que está en rojo es lo que "sobrevive" de esa derivada parcial (se comporta como una constante porque se deriva parcialmente por  $w_{0,0}^{(1)}$ ), quedando así:

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(1)}} = \partial F \left( F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) + 0 + 0 \right)$$

Entonces

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(1)}} = a_0^{(1)} * \left( 1 - a_0^{(1)} \right) * F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)})$$

Se comporta como una constante

Y como

$$a_0^{(0)} = F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)})$$

Entonces

$$\frac{\partial a_0^{(1)}}{\partial w_{0,0}^{(1)}} = a_0^{(1)} * \left( 1 - a_0^{(1)} \right) * a_0^{(0)}$$

Para el peso  $w_{1,0}^{(1)}$  este sería el tratamiento:

$$a_0^{(1)} = F(\textcolor{red}{F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)})} * w_{0,0}^{(1)} + \textcolor{blue}{F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)})} * w_{1,0}^{(1)} + u_0^{(1)})$$

En el ejemplo, se deriva con respecto a  $w_{1,0}^{(1)}$  (una derivada parcial). Lo que está en azul es lo que “sobrevive” de esa derivada parcial (se comporta como una constante porque se deriva parcialmente por  $w_{1,0}^{(1)}$ ), quedando así:

$$\frac{\partial a_0^{(1)}}{\partial w_{1,0}^{(1)}} = \partial F \left( \textcolor{red}{0} + \textcolor{blue}{F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)})} + 0 \right)$$

Entonces

$$\frac{\partial a_0^{(1)}}{\partial w_{1,0}^{(1)}} = a_0^{(1)} * \left( 1 - a_0^{(1)} \right) * \textcolor{blue}{F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)})}$$

Se comporta como una constante

Y como

$$a_1^{(0)} = \textcolor{blue}{F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)})}$$

Entonces

$$\frac{\partial a_0^{(1)}}{\partial w_{1,0}^{(1)}} = a_0^{(1)} * \left( 1 - a_0^{(1)} \right) * \textcolor{blue}{a_1^{(0)}}$$

Generalizando

$$\frac{\partial a_0^{(1)}}{\partial w_{j,0}^{(1)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_j^{(0)}$$

Donde j=0 o 1

Para el umbral  $u_0^{(0)}$  este sería el tratamiento:

$$a_0^{(1)} = F(F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)})$$

En el ejemplo, se deriva con respecto a  $u_0^{(0)}$  (una derivada parcial). En rojo se pone que ecuación interna es derivable con respecto a  $u_0^{(0)}$

$$\frac{\partial a_0^{(1)}}{\partial u_0^{(0)}} = \partial F \left( \partial F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + 0 + 0 \right)$$

Para derivar entonces se deriva la F externa (que está en negro y es F(r)), luego la F interna (que está en rojo y es g(k) y que la multiplica la constante  $w_{0,0}^{(1)}$ ) y por último el polinomio (que es k) que está en verde porque allí está  $u_0^{(0)}$ . Hay tres derivaciones.

Entonces, se sabe que:

$$F(r) = a_0^{(1)}$$

$$g(k) = a_0^{(0)}$$

$$k = a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}$$

$$\partial k = 1$$

$$\frac{\partial a_0^{(1)}}{\partial u_0^{(0)}} = \partial \{F[g(k)]\}$$

$$\partial \{F[g(k)]\} = F(r) * (1 - F(r)) * g(k) * (1 - g(k)) * \partial k$$

Entonces:

$$\frac{\partial a_0^{(1)}}{\partial u_0^{(0)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_0^{(0)} * (1 - a_0^{(0)}) * 1 * w_{0,0}^{(1)}$$

Para el umbral  $u_1^{(0)}$  este sería el tratamiento:

$$a_0^{(1)} = F(F(a_0^{(E)} * w_{0,0}^{(0)} + u_0^{(0)}) * w_{0,0}^{(1)} + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + u_0^{(1)})$$

En el ejemplo, se deriva con respecto a  $u_1^{(0)}$  (una derivada parcial). En azul se pone que ecuación interna es derivable con respecto a  $u_1^{(0)}$

$$\frac{\partial a_0^{(1)}}{\partial u_1^{(0)}} = \partial F \left( 0 + F(a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}) * w_{1,0}^{(1)} + 0 \right)$$

Para derivar entonces se deriva la F externa (que está en negro y es F(r)), luego la F interna (que está en azul y es g(k) y que la multiplica la constante  $w_{1,0}^{(1)}$ ) y por último el polinomio (que es k) que está en verde porque allí está  $u_1^{(0)}$ . Hay tres derivaciones.

Entonces, se sabe que:

$$F(r) = a_0^{(1)}$$

$$g(k) = a_1^{(0)}$$

$$k = a_0^{(E)} * w_{0,1}^{(0)} + u_1^{(0)}$$

$$\partial k = 1$$



$$\frac{\partial a_0^{(1)}}{\partial u_1^{(0)}} = \partial\{F[g(k)]\}$$

$$\partial\{F[g(k)]\} = F(r) * (1 - F(r)) * g(k) * (1 - g(k)) * \partial k$$

Entonces:

$$\frac{\partial a_0^{(1)}}{\partial u_1^{(0)}} = a_0^{(1)} * (1 - a_0^{(1)}) * a_1^{(0)} * (1 - a_1^{(0)}) * 1 * w_{1,0}^{(1)}$$

Con un ejemplo más complejo en el que la capa oculta tiene dos capas de neuronas y cada capa tiene dos neuronas.

Luego  $N_0 = 2$ ,  $N_1 = 2$ ,  $N_2 = 1$

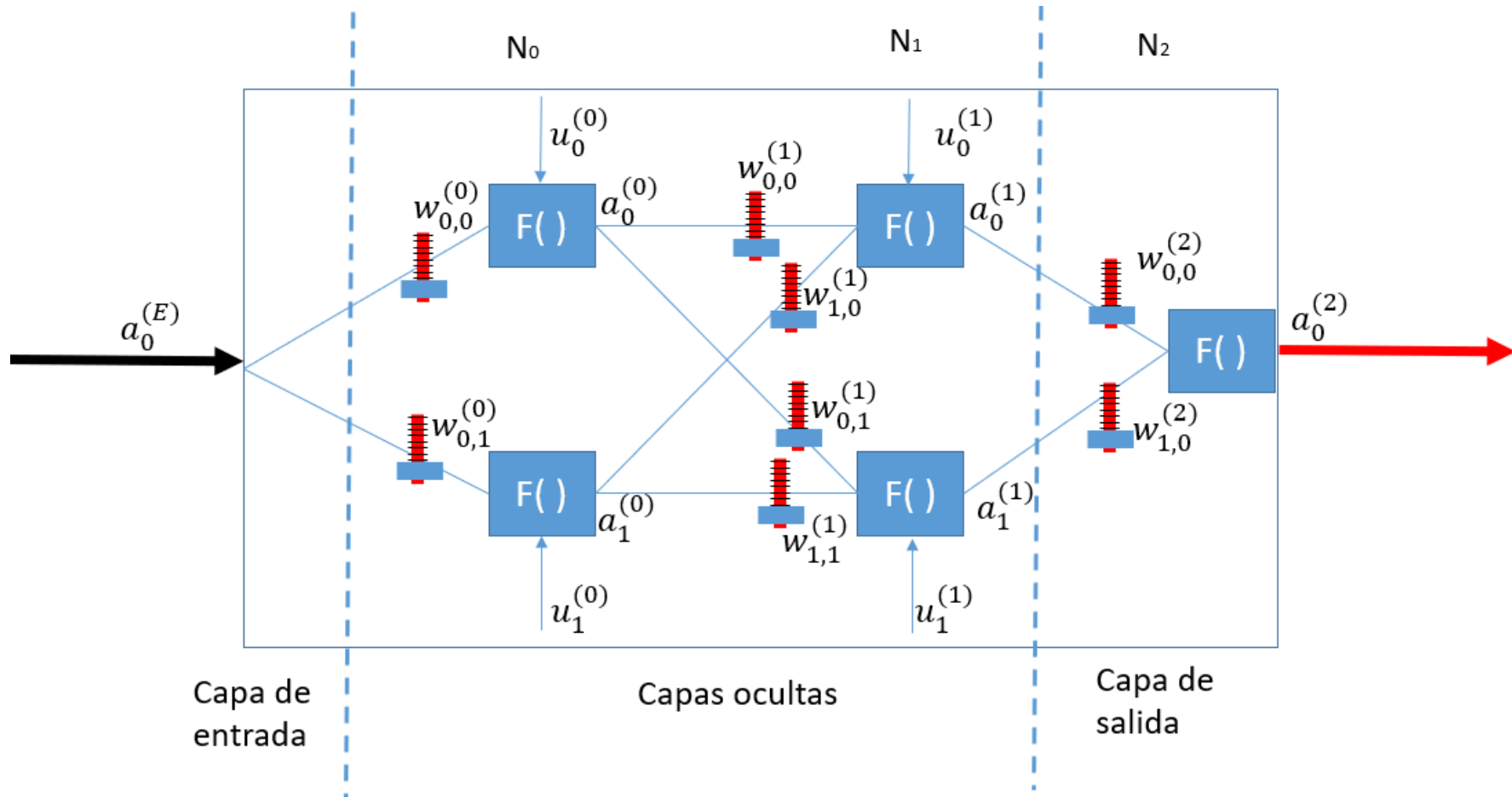


Ilustración 37: Esquema de un perceptrón multicapa

Se buscan los caminos para  $w_{0,0}^{(0)}$ , entonces hay dos marcados en rojo

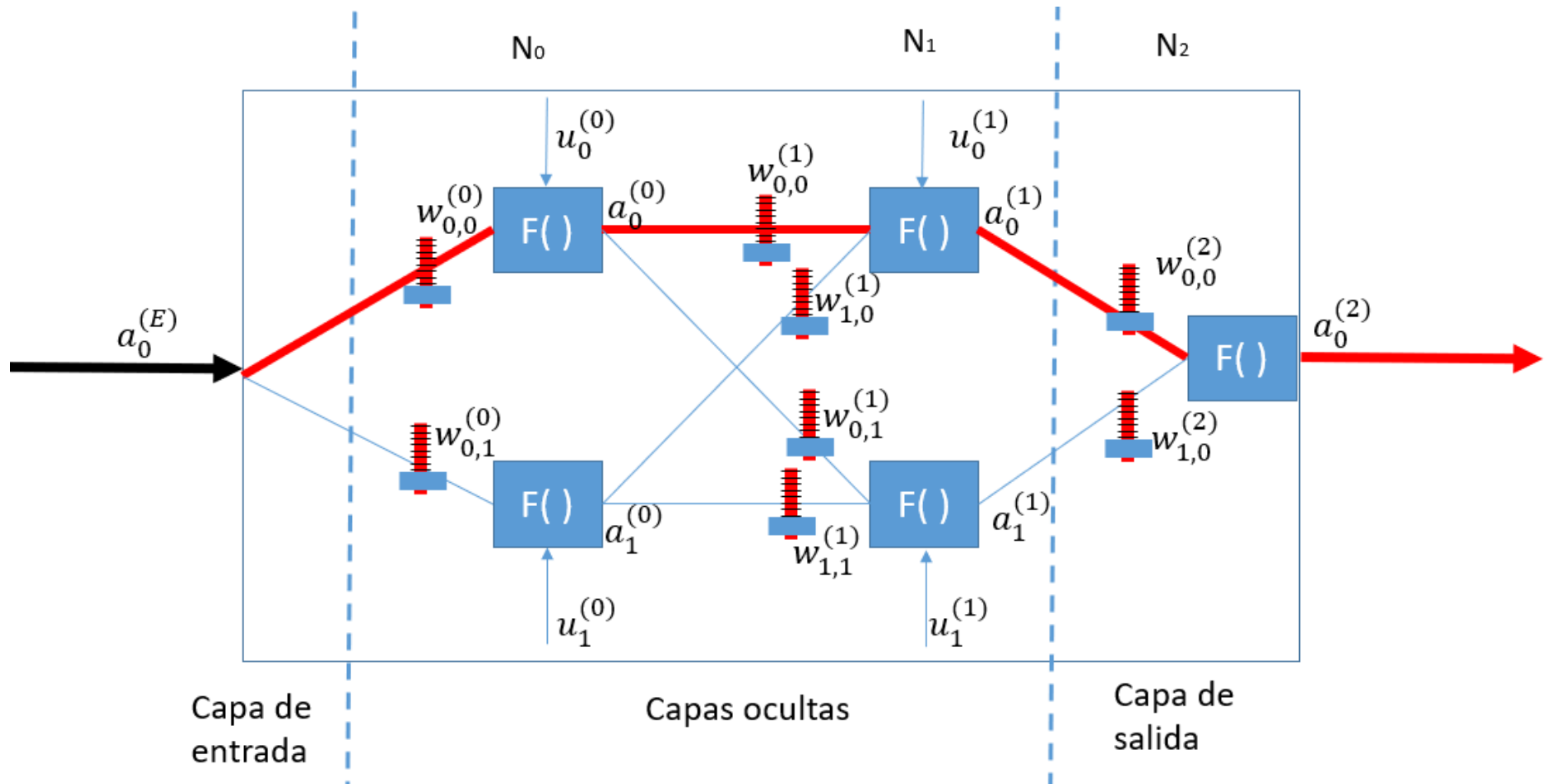


Ilustración 38: Primer camino para ese peso

Y

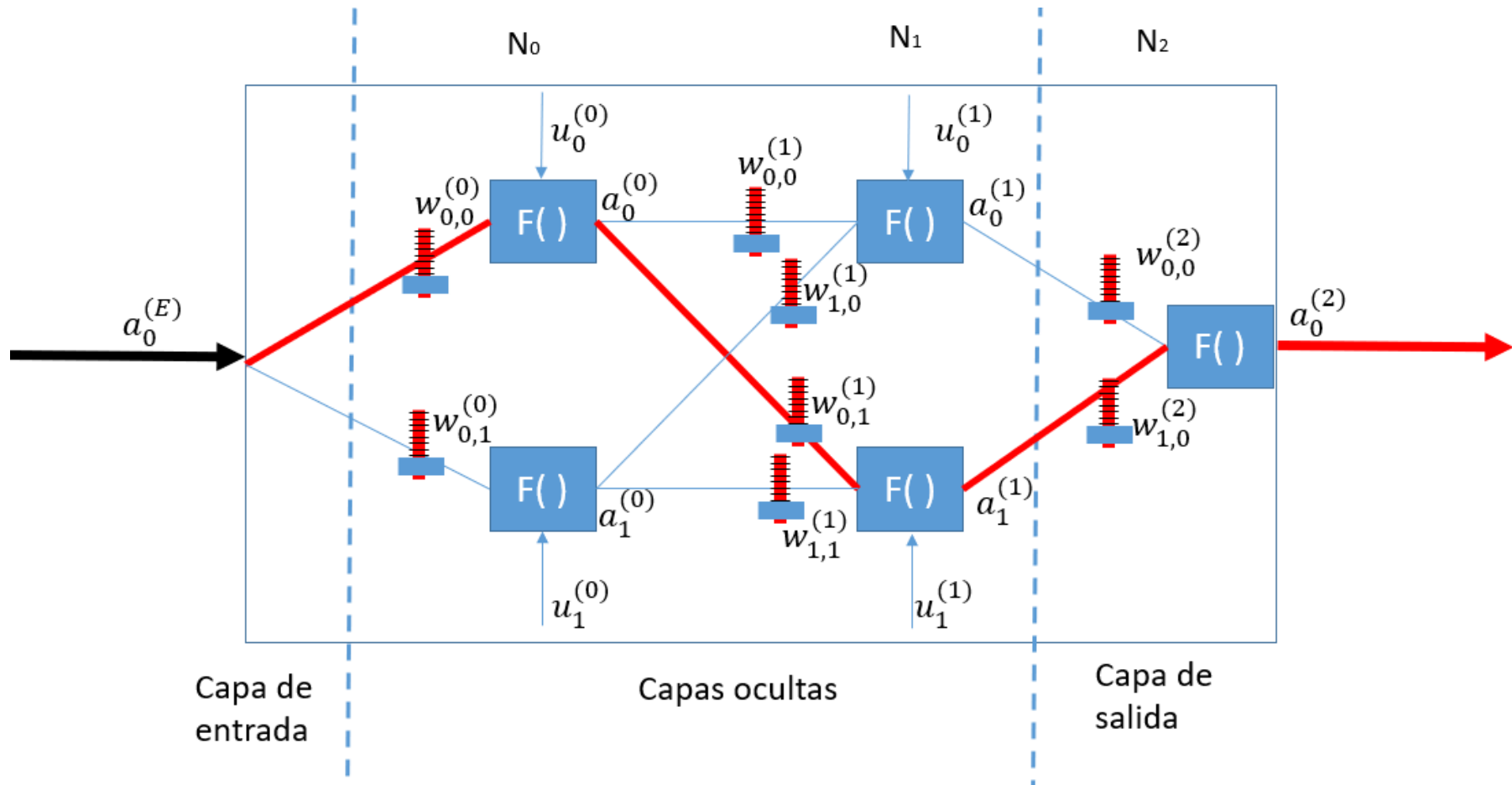


Ilustración 39: Segundo camino para ese peso

Luego la siguiente expresión para la derivada parcial con respecto a  $w_{0,0}^{(0)}$  es seguir los dos caminos, sumando ambos

$$\begin{aligned} \frac{\partial a_0^{(2)}}{\partial w_{0,0}^{(0)}} &= a_0^{(2)} * \left(1 - a_0^{(2)}\right) * w_{0,0}^{(2)} * a_0^{(1)} * \left(1 - a_0^{(1)}\right) * w_{0,0}^{(1)} * a_0^{(0)} * \left(1 - a_0^{(0)}\right) * a_0^{(E)} \\ &+ \\ &a_0^{(2)} * \left(1 - a_0^{(2)}\right) * w_{1,0}^{(2)} * a_1^{(1)} * \left(1 - a_1^{(1)}\right) * w_{0,1}^{(1)} * a_0^{(0)} * \left(1 - a_0^{(0)}\right) * a_0^{(E)} \end{aligned}$$

Recomendado ir de la entrada a la salida para ver cómo se incrementa el nivel de las capas

$$\begin{aligned} \frac{\partial a_0^{(2)}}{\partial w_{0,0}^{(0)}} &= a_0^{(E)} * a_0^{(0)} * \left(1 - a_0^{(0)}\right) * w_{0,0}^{(1)} * a_0^{(1)} * \left(1 - a_0^{(1)}\right) * w_{0,0}^{(2)} * a_0^{(2)} * \left(1 - a_0^{(2)}\right) \\ &+ \\ &a_0^{(E)} * a_0^{(0)} * \left(1 - a_0^{(0)}\right) * w_{0,1}^{(1)} * a_1^{(1)} * \left(1 - a_1^{(1)}\right) * w_{1,0}^{(2)} * a_0^{(2)} * \left(1 - a_0^{(2)}\right) \end{aligned}$$

Y así poder generalizar

$$\frac{\partial a_0^{(2)}}{\partial w_{0,0}^{(0)}} = a_0^{(E)} * a_0^{(0)} * \left(1 - a_0^{(0)}\right) * \left[ \sum_{j=0}^{N_2-1} w_{0,j}^{(1)} * a_j^{(1)} * \left(1 - a_j^{(1)}\right) * w_{j,0}^{(2)} \right] * a_0^{(2)} * \left(1 - a_0^{(2)}\right)$$

La ventaja es que, si las capas ocultas tienen más neuronas, sería cambiar el límite máximo en la sumatoria.

Renombrando la entrada y salida del perceptrón así:

$$a_0^{(E)} = x_0$$

$$a_0^{(2)} = y_0$$

Entonces

$$\frac{\partial y_0}{\partial w_{0,0}^{(0)}} = x_0 * a_0^{(0)} * (1 - a_0^{(0)}) * \left[ \sum_{j=0}^{N_2-1} w_{0,j}^{(1)} * a_j^{(1)} * (1 - a_j^{(1)}) * w_{j,0}^{(2)} \right] * y_0 * (1 - y_0)$$

Con un perceptrón con más entradas y salidas,  $N_0 = 4$ ,  $N_1 = 4$ ,  $N_2 = 2$ , y si se desea dar con  $\frac{\partial y_0}{\partial w_{0,0}^{(0)}}$ , hay que considerar los diferentes caminos:

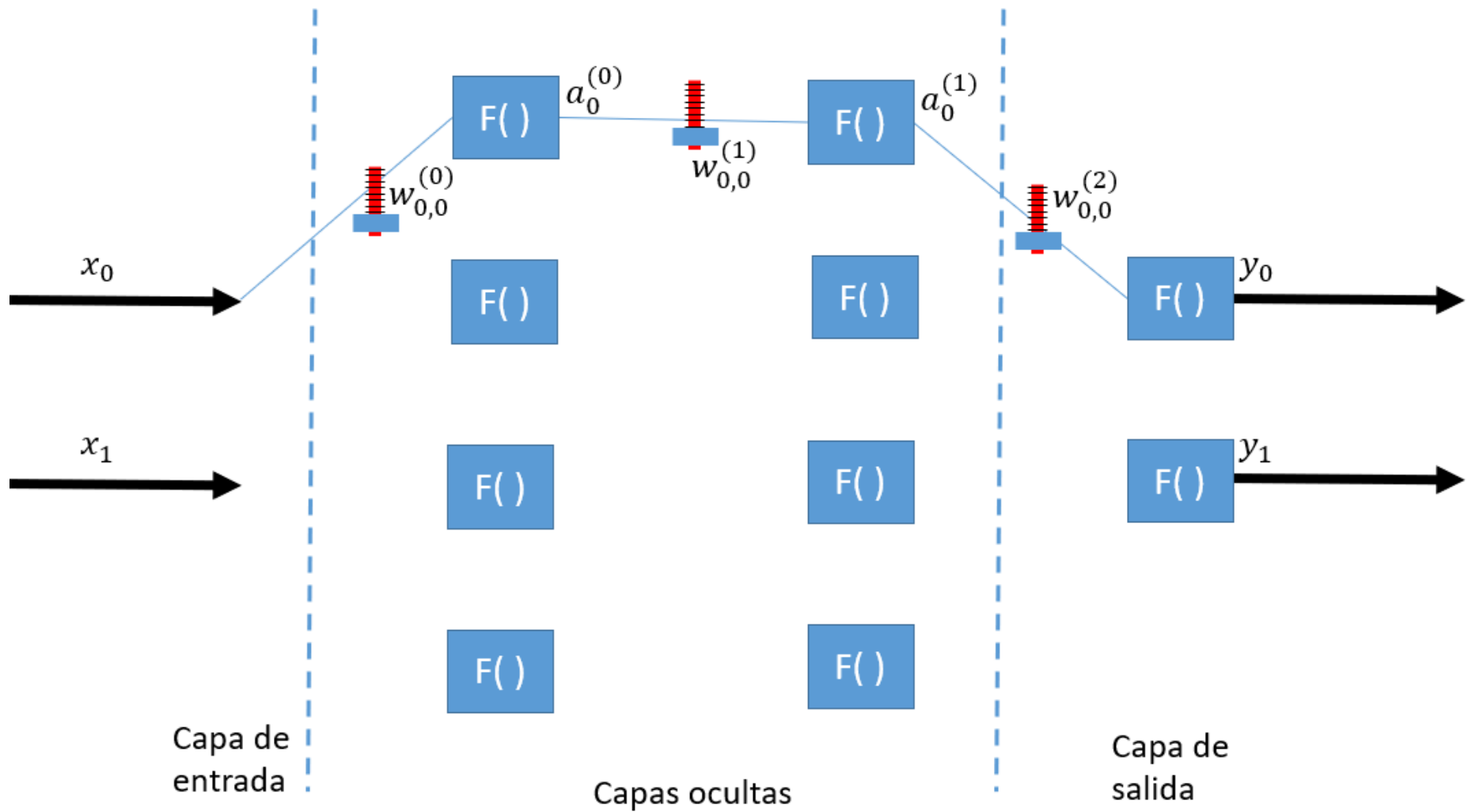


Ilustración 40: Primer camino

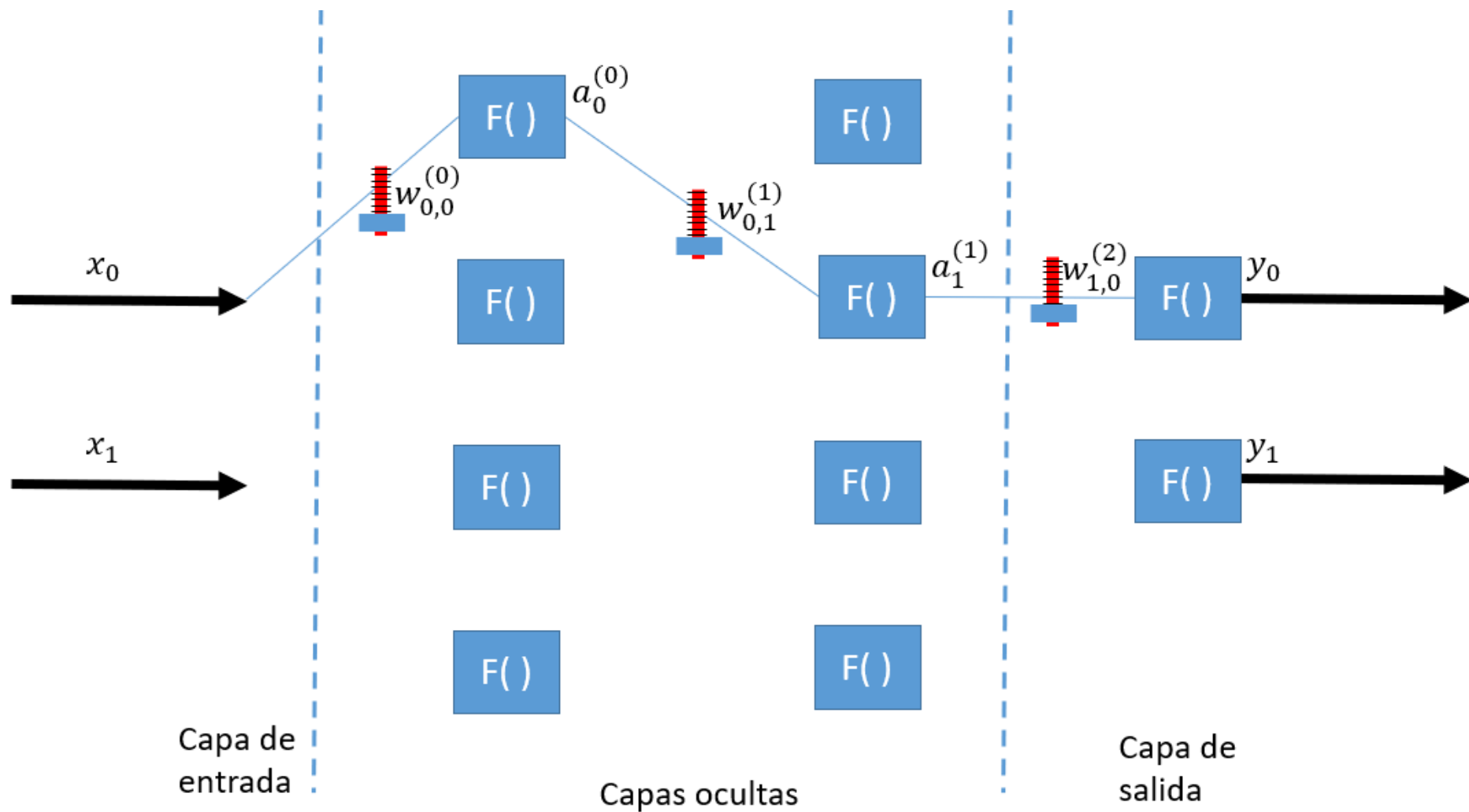


Ilustración 41: Segundo camino



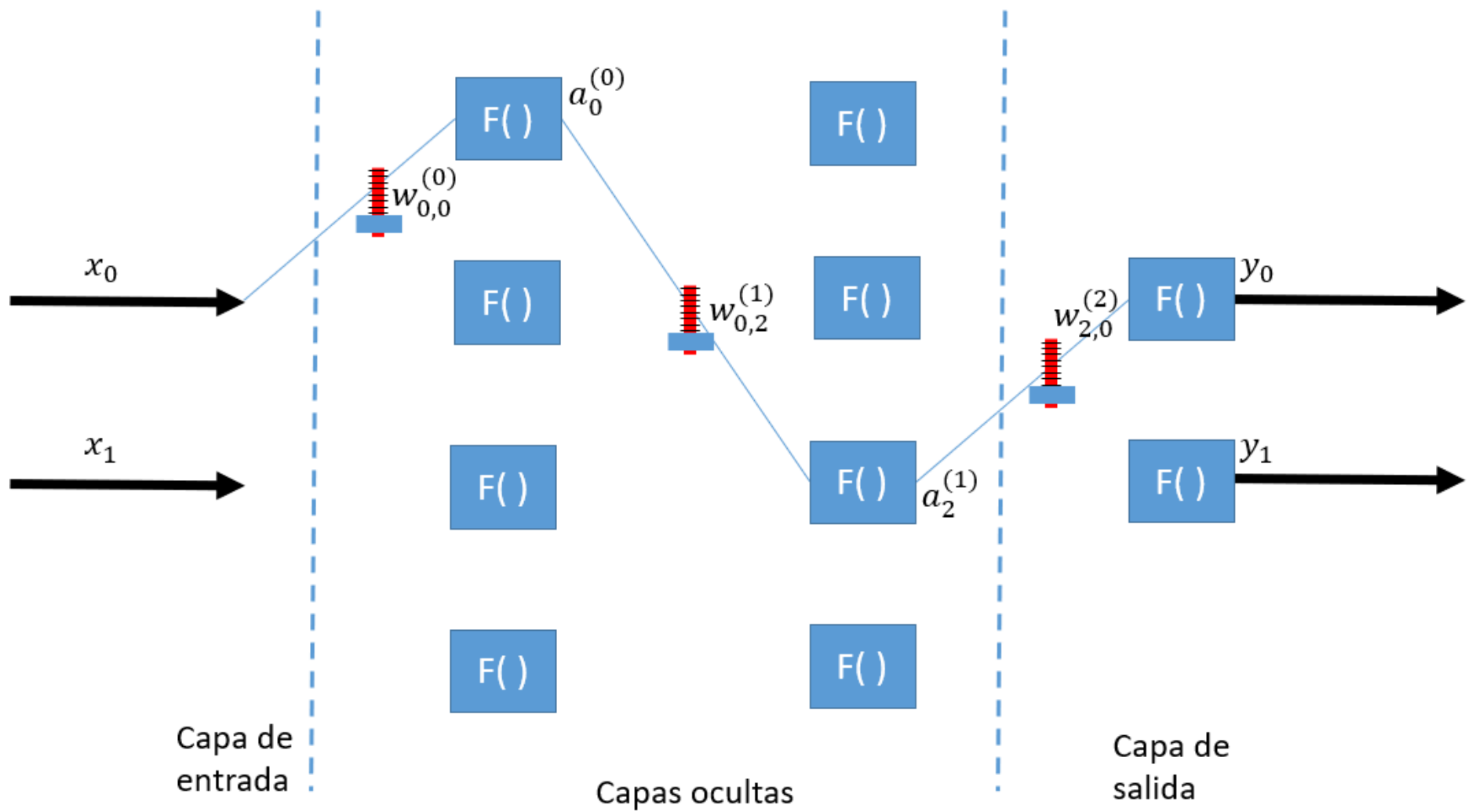


Ilustración 42: Tercer camino

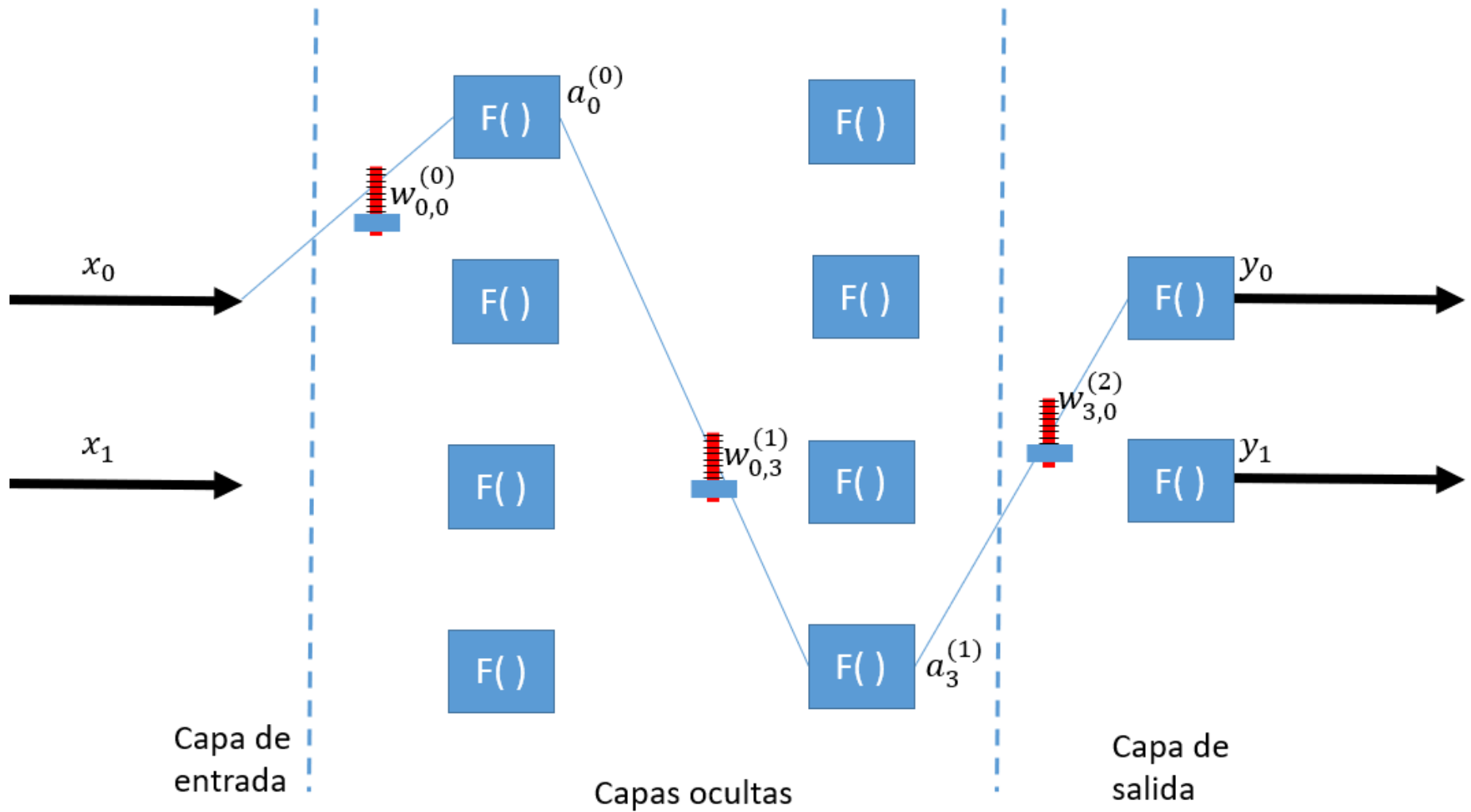


Ilustración 43: Cuarto camino

Luego

$$\frac{\partial y_0}{\partial w_{0,0}^{(0)}} = x_0 * a_0^{(0)} * (1 - a_0^{(0)}) * \left[ \sum_{p=0}^{N_1-1} w_{0,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,0}^{(2)} \right] * y_0 * (1 - y_0)$$

Generalizando

$$\frac{\partial y_i}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \left[ \sum_{p=0}^{N_1-1} w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,i}^{(2)} \right] * y_i * (1 - y_i)$$

Donde i=0 a 1, j=0 a 3, k=0 a 3

¿Y para los  $w^{(1)}$ ?

$$\frac{\partial y_i}{\partial w_{j,k}^{(1)}} = a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) * w_{k,i}^{(2)} * y_i * (1 - y_i)$$

¿Y para los  $w^{(2)}$ ?

$$\frac{\partial y_i}{\partial w_{j,i}^{(2)}} = a_j^{(1)} * y_i * (1 - y_i)$$

¿Y los umbrales  $u^{(0)}$ ?

$$\frac{\partial y_i}{\partial u_k^{(0)}} = 1 * a_k^{(0)} * (1 - a_k^{(0)}) * \left[ \sum_{p=0}^{N_1-1} w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,i}^{(2)} \right] * y_i * (1 - y_i)$$

Donde i=0 a 1, k=0 a 3

¿Y los umbrales  $u^{(1)}$ ?

$$\frac{\partial y_i}{\partial u_k^{(1)}} = 1 * a_k^{(1)} * (1 - a_k^{(1)}) * w_{k,i}^{(2)} * y_i * (1 - y_i)$$

¿Y los umbrales  $u^{(2)}$ ?

$$\frac{\partial y_i}{\partial u_i^{(2)}} = 1 * y_i * (1 - y_i)$$

## Tratamiento del error en el algoritmo de propagación hacia atrás

Se tiene la siguiente tabla

| Entrada $X_0$ | Entrada $X_1$ | Valor esperado de salida $S_0$ | Valor esperado de salida $S_1$ |
|---------------|---------------|--------------------------------|--------------------------------|
| 1             | 0             | 0                              | 1                              |
| 0             | 0             | 1                              | 1                              |
| 0             | 1             | 0                              | 0                              |

Pero en realidad se está obteniendo con el perceptrón estas salidas

| Entrada $X_0$ | Entrada $X_1$ | Salida real $Y_0$ | Salida real $Y_1$ |
|---------------|---------------|-------------------|-------------------|
| 1             | 0             | 1                 | 1                 |
| 0             | 0             | 1                 | 0                 |
| 0             | 1             | 0                 | 1                 |

Hay un error evidente con las salidas porque no coinciden con lo esperado. ¿Qué hacer? Ajustar los pesos y los umbrales.

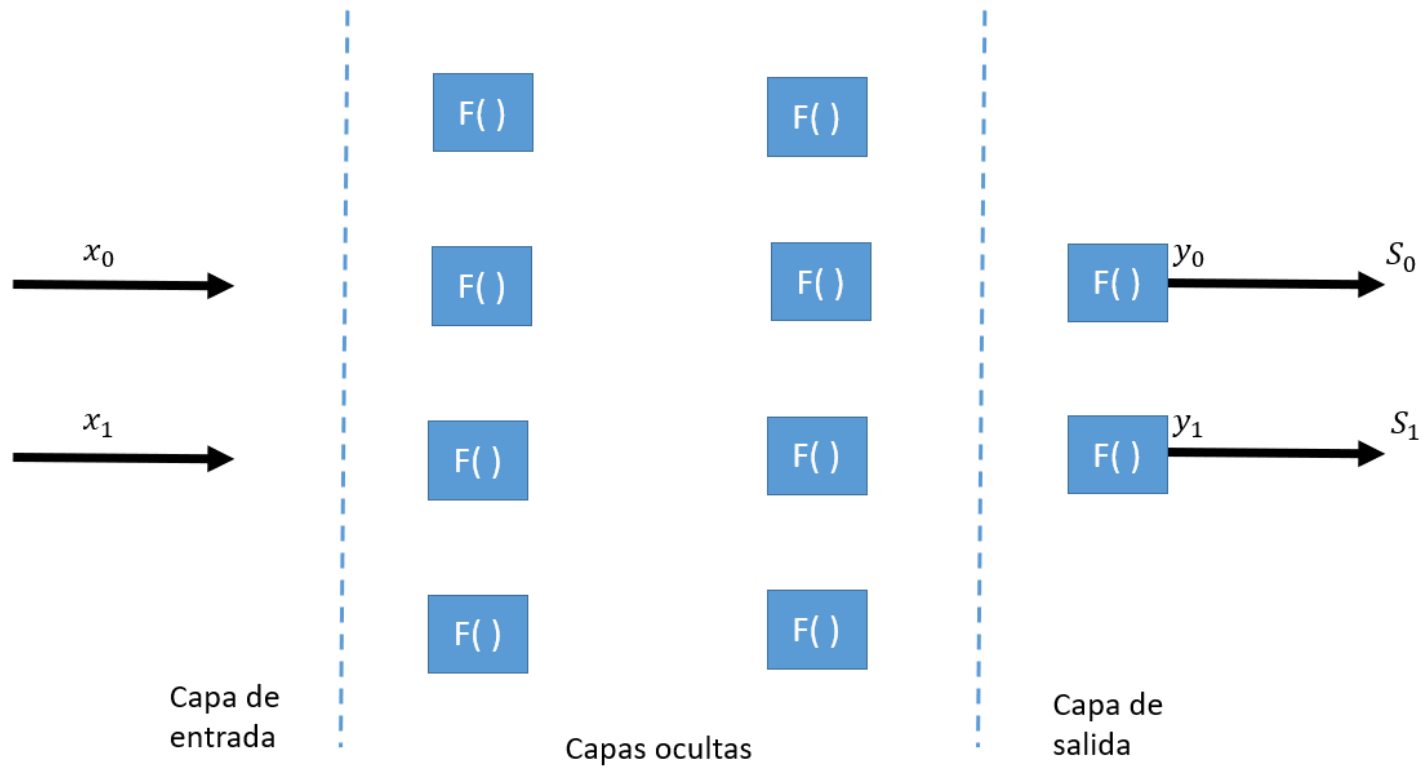
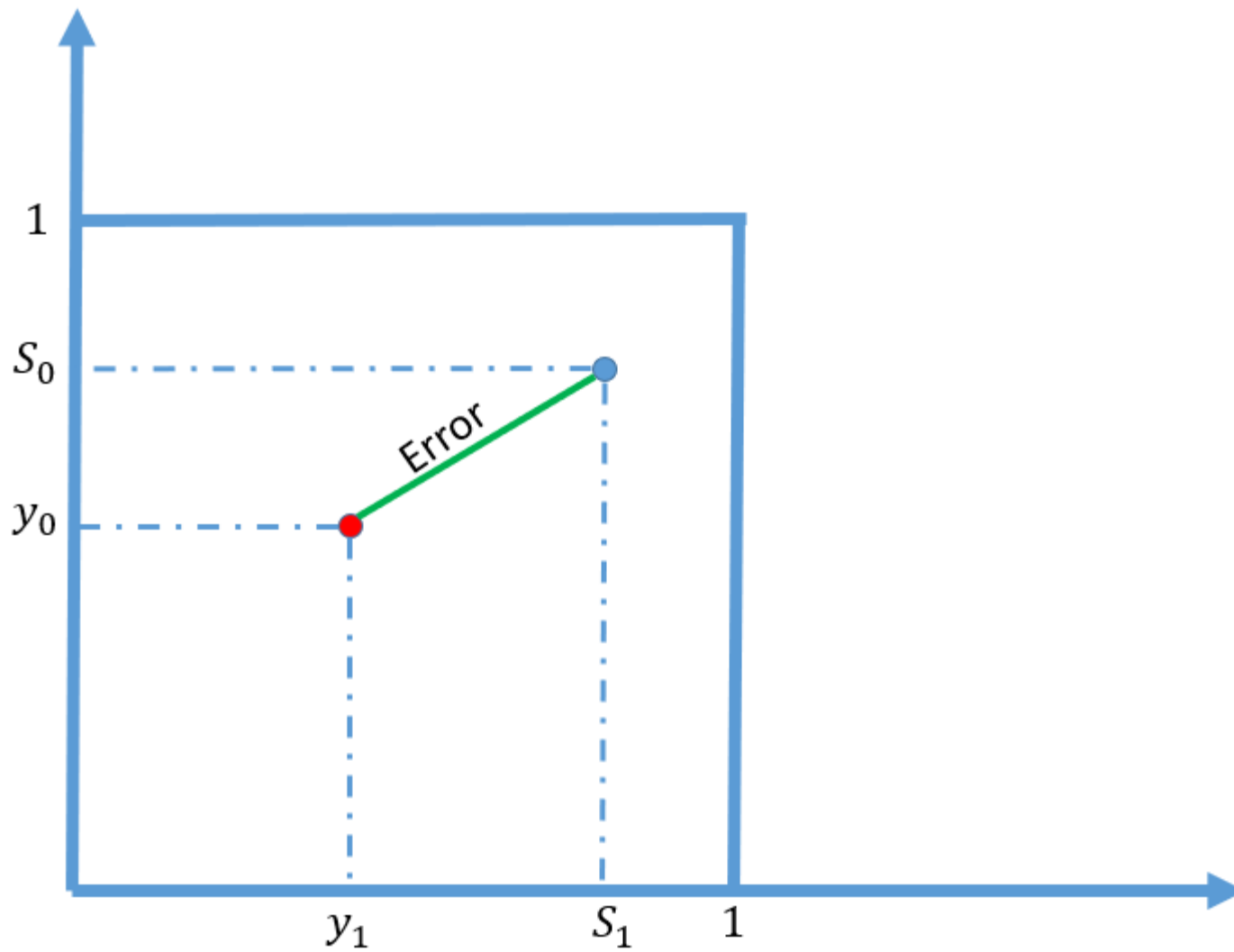


Ilustración 44: Dos entradas y dos salidas

Si se tomaran las salidas  $y_0$  y  $y_1$  como coordenadas e igualmente  $S_0$  y  $S_1$ , se obtiene lo siguiente:



*Ilustración 45: Representación del error*

Como la función de salida de las neuronas es la sigmoidea, la salida está entre 0 y 1.

En el gráfico de color verde está el error y para calcularlo es usar la fórmula de distancia entre dos puntos en un plano:

$$Error = \sqrt{(S_1 - y_1)^2 + (S_0 - y_0)^2}$$

Para minimizar ese error hay que considerar que dado:

$$F(x) = \sqrt{g(x)}$$

Al derivar:

$$\partial F(x) = \frac{\partial g(x)}{2 * \sqrt{g(x)}}$$

Y como hay que minimizar se iguala esa derivada a cero

$$\partial F(x) = \frac{\partial g(x)}{2 * \sqrt{g(x)}} = 0$$

Luego

$$\partial g(x) = 0$$

En otras palabras, la raíz cuadrada de  $F(x)$ , es irrelevante cuando se busca minimizar, porque lo importante es minimizar el interior. Luego la ecuación del error pasa a ser:

$$Error = (S_1 - y_1)^2 + (S_0 - y_0)^2$$

Que es más sencilla de evaluar. El siguiente paso es multiplicarla por unas constantes quedando así:

$$Error = \frac{1}{2} (S_1 - y_1)^2 + \frac{1}{2} (S_0 - y_0)^2$$



¿Y por qué se hizo eso? Para hacer que la derivada de Error sea más sencilla. Y no hay que preocuparse porque afecte los resultados: como se busca minimizar, esas constantes no afectan el procedimiento.

¡OJO! Hay que recordar que  $y_0$ ,  $y_1$ , varían, en cambio,  $S_0$ ,  $S_1$  son constantes porque son los valores esperados.

Hay que considerar esta regla matemática: Si P es una función con varias variables independientes, es decir: P(m, n) y Q también es otra función con esas mismas variables independientes, es decir: Q(m, n) y hay una *superfunción* que hace uso de P y Q, es decir: K(P,Q), entonces para derivar a K por una de las variables independientes, tenemos:

$$\frac{\partial K}{\partial m} = \frac{\partial K}{\partial P} * \frac{\partial P}{\partial m} + \frac{\partial K}{\partial Q} * \frac{\partial Q}{\partial m}$$

o

$$\frac{\partial K}{\partial n} = \frac{\partial K}{\partial P} * \frac{\partial P}{\partial n} + \frac{\partial K}{\partial Q} * \frac{\partial Q}{\partial n}$$

Luego

$$\frac{\partial Error}{\partial \blacksquare} = \frac{\partial Error}{\partial y_0} * \frac{\partial y_0}{\partial \blacksquare} + \frac{\partial Error}{\partial y_1} * \frac{\partial y_1}{\partial \blacksquare}$$

¿Y qué es ese cuadro relleno negro? Puede ser algún peso o algún umbral. Generalizando:

$$\frac{\partial Error}{\partial \blacksquare} = \sum_{i=0}^{N_2-1} \left( \frac{\partial Error}{\partial y_i} * \frac{\partial y_i}{\partial \blacksquare} \right)$$

Sabiendo que

$$Error = \frac{1}{2} (S_1 - y_1)^2 + \frac{1}{2} (S_0 - y_0)^2$$

Entonces la derivada de Error con respecto a  $y_0$  es:

$$\frac{\partial Error}{\partial y_0} = y_0 - S_0$$

Generalizando

$$\frac{\partial Error}{\partial y_i} = y_i - S_i$$

En el ejemplo, hay dos salidas  $Y_0$  y  $Y_1$  que están en la capa de salida  $N_2$

Luego, la sumatoria sería de 0 a 1

$$\frac{\partial Error}{\partial \blacksquare} = \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * \frac{\partial y_i}{\partial \blacksquare} \right)$$

Queda entonces el cuadro relleno negro que como se mencionó anteriormente puede ser un peso o un umbral. Entonces si se tiene por ejemplo que:

$$\blacksquare = w_{j,i}^{(2)}$$

Entonces como hay una  $i$  en particular, la sumatoria se retira luego.

$$\frac{\partial Error}{\partial w_{j,i}^{(2)}} = (y_i - S_i) * \frac{\partial y_i}{\partial w_{j,i}^{(2)}}$$

Y como

$$\frac{\partial y_i}{\partial w_{j,i}^{(2)}} = a_j^{(1)} * y_i * (1 - y_i)$$

Luego

$$\frac{\partial Error}{\partial w_{j,i}^{(2)}} = (y_i - S_i) * a_j^{(1)} * y_i * (1 - y_i)$$

Ordenando

$$\frac{\partial Error}{\partial w_{j,i}^{(2)}} = a_j^{(1)} * (y_i - S_i) * y_i * (1 - y_i)$$

De nuevo la derivada del error

$$\frac{\partial Error}{\partial \blacksquare} = \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * \frac{\partial y_i}{\partial \blacksquare} \right)$$

Suponiendo que

$$\blacksquare = w_{j,k}^{(1)}$$

Entonces

$$\frac{\partial Error}{\partial w_{j,k}^{(1)}} = \sum_{i=1}^{N_2-1} \left( (y_i - s_i) * \frac{\partial y_i}{\partial w_{j,k}^{(1)}} \right)$$

Y como

$$\frac{\partial y_i}{\partial w_{j,k}^{(1)}} = a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) * w_{k,i}^{(2)} * y_i * (1 - y_i)$$

Entonces

$$\frac{\partial Error}{\partial w_{j,k}^{(1)}} = \sum_{i=0}^{N_2-1} \left( (y_i - s_i) * a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) * w_{k,i}^{(2)} * y_i * (1 - y_i) \right)$$

Simplificando

$$\frac{\partial Error}{\partial w_{j,k}^{(1)}} = a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) * \sum_{i=0}^{N_2-1} \left( (y_i - s_i) * w_{k,i}^{(2)} * y_i * (1 - y_i) \right)$$

De nuevo la derivada del error

$$\frac{\partial Error}{\partial \blacksquare} = \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * \frac{\partial y_i}{\partial \blacksquare} \right)$$

Suponiendo que

$$\blacksquare = w_{j,k}^{(0)}$$

Luego la derivada del error es:

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * \frac{\partial y_i}{\partial w_{j,k}^{(0)}} \right)$$

Y como se vio anteriormente que

$$\frac{\partial y_i}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \left[ \sum_{p=0}^{N_1-1} w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,i}^{(2)} \right] * y_i * (1 - y_i)$$

Luego reemplazando en la expresión se obtiene:

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \left[ \sum_{p=0}^{N_1-1} w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,i}^{(2)} \right] * y_i * (1 - y_i) \right)$$

Simplificando

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \sum_{i=0}^{N_2-1} \left( (y_i - S_i) * \left[ \sum_{p=0}^{N_1-1} w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * w_{p,i}^{(2)} \right] * y_i * (1 - y_i) \right)$$

Luego

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \sum_{p=0}^{N_1-1} \left[ w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{p,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right) \right]$$

En limpio las fórmulas para los pesos son:

$$\frac{\partial Error}{\partial w_{j,i}^{(2)}} = a_j^{(1)} * (y_i - S_i) * y_i * (1 - y_i)$$

$$\frac{\partial Error}{\partial w_{j,k}^{(1)}} = a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{k,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right)$$

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \sum_{p=0}^{N_1-1} \left[ w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{p,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right) \right]$$

Y para los umbrales sería:

$$\frac{\partial Error}{\partial u_i^{(2)}} = (y_i - S_i) * y_i * (1 - y_i)$$

$$\frac{\partial Error}{\partial u_k^{(1)}} = a_k^{(1)} * (1 - a_k^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{k,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right)$$

$$\frac{\partial Error}{\partial u_k^{(0)}} = a_k^{(0)} * (1 - a_k^{(0)}) * \sum_{p=0}^{N_1-1} \left[ w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{p,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right) \right]$$



## Variando los pesos y umbrales con el algoritmo de propagación hacia atrás

La fórmula de variación de los pesos y umbrales es:

$$w_{j,i}^{(2)} \leftarrow w_{j,i}^{(2)} - \alpha * \frac{\partial Error}{\partial w_{j,i}^{(2)}}$$

$$w_{j,k}^{(1)} \leftarrow w_{j,k}^{(1)} - \alpha * \frac{\partial Error}{\partial w_{j,k}^{(1)}}$$

$$w_{j,k}^{(0)} \leftarrow w_{j,k}^{(0)} - \alpha * \frac{\partial Error}{\partial w_{j,k}^{(0)}}$$

$$u_i^{(2)} \leftarrow u_i^{(2)} - \alpha * \frac{\partial Error}{\partial u_i^{(2)}}$$

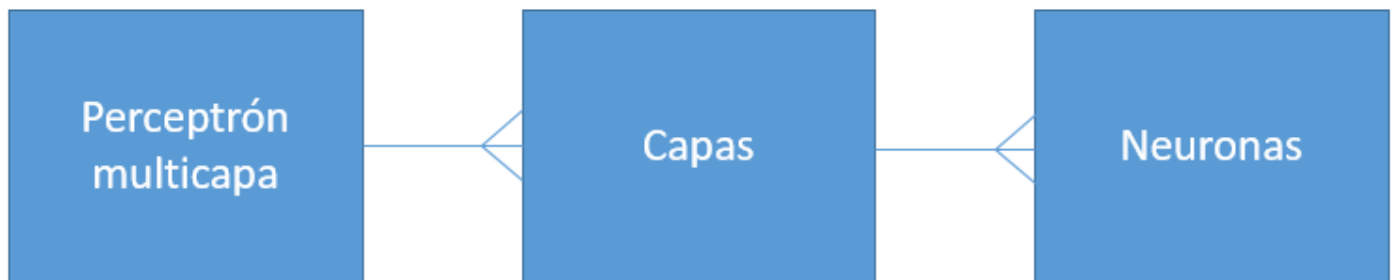
$$u_k^{(1)} \leftarrow u_k^{(1)} - \alpha * \frac{\partial Error}{\partial u_k^{(1)}}$$

$$u_k^{(0)} \leftarrow u_k^{(0)} - \alpha * \frac{\partial Error}{\partial u_k^{(0)}}$$

Donde  $\alpha$  es el factor de aprendizaje con un valor pequeño entre 0.1 y 0.9

## Implementación del perceptrón multicapa

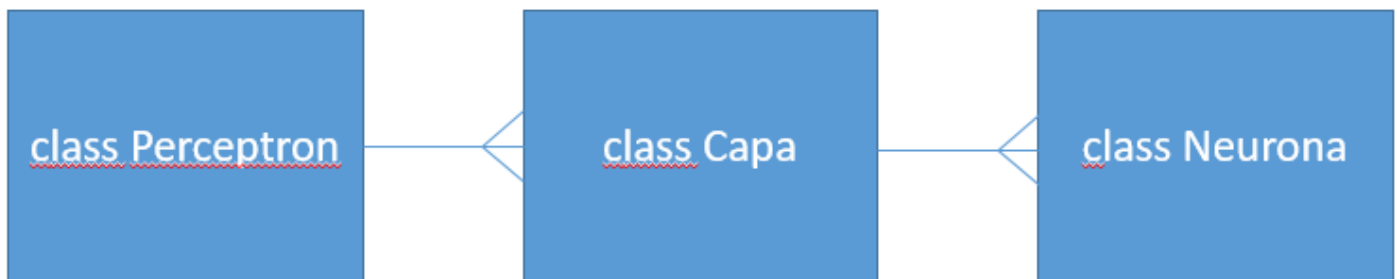
El siguiente modelo entidad-relación muestra cómo se compone un perceptrón multicapa



*Ilustración 46: Modelo del perceptrón*

Un perceptrón tiene dos o más capas (mínimo una oculta y la de salida). Una capa tiene uno o más neuronas.

Para implementarlo se hace uso de clases y listas.



*Ilustración 47: Modelo del perceptrón*

Esta sería la plantilla del programa:

K/008.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa> Capas;
}

class Capa {
    List<Neurona> Neuronas;
}

class Neurona {
}
```

Cada neurona tiene los pesos de entrada y el umbral. En el constructor se inicializan los pesos y el umbral al azar. Así quedaría el código:

K/009.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa> Capas;
}

class Capa {
    List<Neurona> Neuronas;
}

class Neurona {
    private List<double> Pesos; //Los pesos para cada entrada
    double Umbral; //El peso del umbral

    //Inicializa los pesos y umbral con valores al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++)
            Pesos.Add(Azar.NextDouble());
        Umbral = Azar.NextDouble();
    }
}
```

Se añade a la clase neurona, el método CalculaSalida() que tiene como parámetro un arreglo unidimensional, el cual tiene los datos de entrada.

K/010.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa> Capas;
}

class Capa {
    List<Neurona> Neuronas;
}

class Neurona {
    private List<double> Pesos; //Los pesos para cada entrada
    double Umbral; //El peso del umbral

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++)
            Pesos.Add(Azar.NextDouble());
        Umbral = Azar.NextDouble();
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }
}
```

La clase **Capa** almacena sus propias neuronas y la salida de cada una de esas neuronas en una lista salidas para facilitar los cálculos más adelante. Se añade el método en que calcula la salida de cada neurona y guarda ese resultado en el listado de "salidas".

K/011.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa> Capas;
}

class Capa {
    List<Neurona> Neuronas; //Las neuronas que tendrá la capa
    List<double> Salidas; //Almacena la salida de cada neurona

    public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
        Neuronas = [];
        Salidas = [];

        //Genera las neuronas e inicializa sus salidas
        for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
            Neuronas.Add(new Neurona(Azar, TotalEntradas));
            Salidas.Add(0);
        }
    }

    //Calcula la salida de cada neurona de la capa
    public void CalculaCapa(List<double> Entradas) {
        for (int cont = 0; cont < Neuronas.Count; cont++)
            Salidas[cont] = Neuronas[cont].CalculaSalida(Entradas);
    }
}

class Neurona {
    private List<double> Pesos; //Los pesos para cada entrada
    double Umbral; //El peso del umbral

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++)
            Pesos.Add(Azar.NextDouble());
    }
}
```

```
        Umbral = Azar.NextDouble();
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }
}
```

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa>? Capas;

    //Crea las diversas capas
    public void CreaCapas(Random Azar, int Entradas,
                          int NeuronasCapa0, int NeuronasCapa1,
                          int NeuronasCapa2) {
        Capas =
        [
            //Crea la capa 0
            new Capa(Azar, NeuronasCapa0, Entradas),

            //Crea la capa 1 (el número de entradas es
            //el número de neuronas de la capa anterior)
            new Capa(Azar, NeuronasCapa1, NeuronasCapa0),

            //Crea la capa 2 (el número de entradas es
            //el número de neuronas de la capa anterior)
            new Capa(Azar, NeuronasCapa2, NeuronasCapa1),
        ];
    }
}

class Capa {
    List<Neurona> Neuronas; //Las neuronas que tendrá la capa
    List<double> Salidas; //Almacena la salida de cada neurona

    public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
        Neuronas = [];
        Salidas = [];

        //Genera las neuronas e inicializa sus salidas
        for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
            Neuronas.Add(new Neurona(Azar, TotalEntradas));
            Salidas.Add(0);
        }
    }
}
```



```

    //Calcula la salida de cada neurona de la capa
    public void CalculaCapa(List<double> Entradas) {
        for (int cont = 0; cont < Neuronas.Count; cont++)
            Salidas[cont] = Neuronas[cont].CalculaSalida(Entradas);
    }
}

class Neurona {
    private List<double> Pesos; //Los pesos para cada entrada
    double Umbral; //El peso del umbral

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++)
            Pesos.Add(Azar.NextDouble());
        Umbral = Azar.NextDouble();
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }
}

```

En Perceptron se hace el cálculo de cada capa. Cabe recordar que la salida de la capa 0 es la entrada de la capa 1, y la salida de la capa 1 es la entrada de la capa 2.

K/013.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
    }
}

class Perceptron {
    List<Capa>? Capas;

    //Crea las diversas capas
    public void CreaCapas(Random Azar, int Entradas,
        int NeuronasCapa0, int NeuronasCapa1, int NeuronasCapa2) {
        Capas =
        [
            //Crea la capa 0
            new Capa(Azar, NeuronasCapa0, Entradas),

            //Crea la capa 1 (el número de entradas
            //es el número de neuronas de la capa anterior)
            new Capa(Azar, NeuronasCapa1, NeuronasCapa0),

            //Crea la capa 2 (el número de entradas
            //es el número de neuronas de la capa anterior)
            new Capa(Azar, NeuronasCapa2, NeuronasCapa1),
        ];
    }

    public void calculaSalida(List<double> Entradas) {
        Capas[0].CalculaCapa(Entradas);

        //Las salidas de la capa anterior son
        //las entradas de la siguiente capa
        Capas[1].CalculaCapa(Capas[0].Salidas);
        Capas[2].CalculaCapa(Capas[1].Salidas);
    }
}

class Capa {
    //Las neuronas que tendrá la capa
    List<Neurona> Neuronas;

    //Almacena la salida de cada neurona
```

```

public List<double> Salidas;

public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
    Neuronas = [];
    Salidas = [];

    //Genera las neuronas e inicializa sus salidas
    for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
        Neuronas.Add(new Neurona(Azar, TotalEntradas));
        Salidas.Add(0);
    }
}

//Calcula la salida de cada neurona de la capa
public void CalculaCapa(List<double> Entradas) {
    for (int cont = 0; cont < Neuronas.Count; cont++)
        Salidas[cont] = Neuronas[cont].CalculaSalida(Entradas);
}
}

class Neurona {
    private List<double> Pesos; //Los pesos para cada entrada
    double Umbral; //El peso del umbral

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++)
            Pesos.Add(Azar.NextDouble());
        Umbral = Azar.NextDouble();
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }
}

```

Ejemplo de uso de la clase Perceptron para generar este diseño en particular:

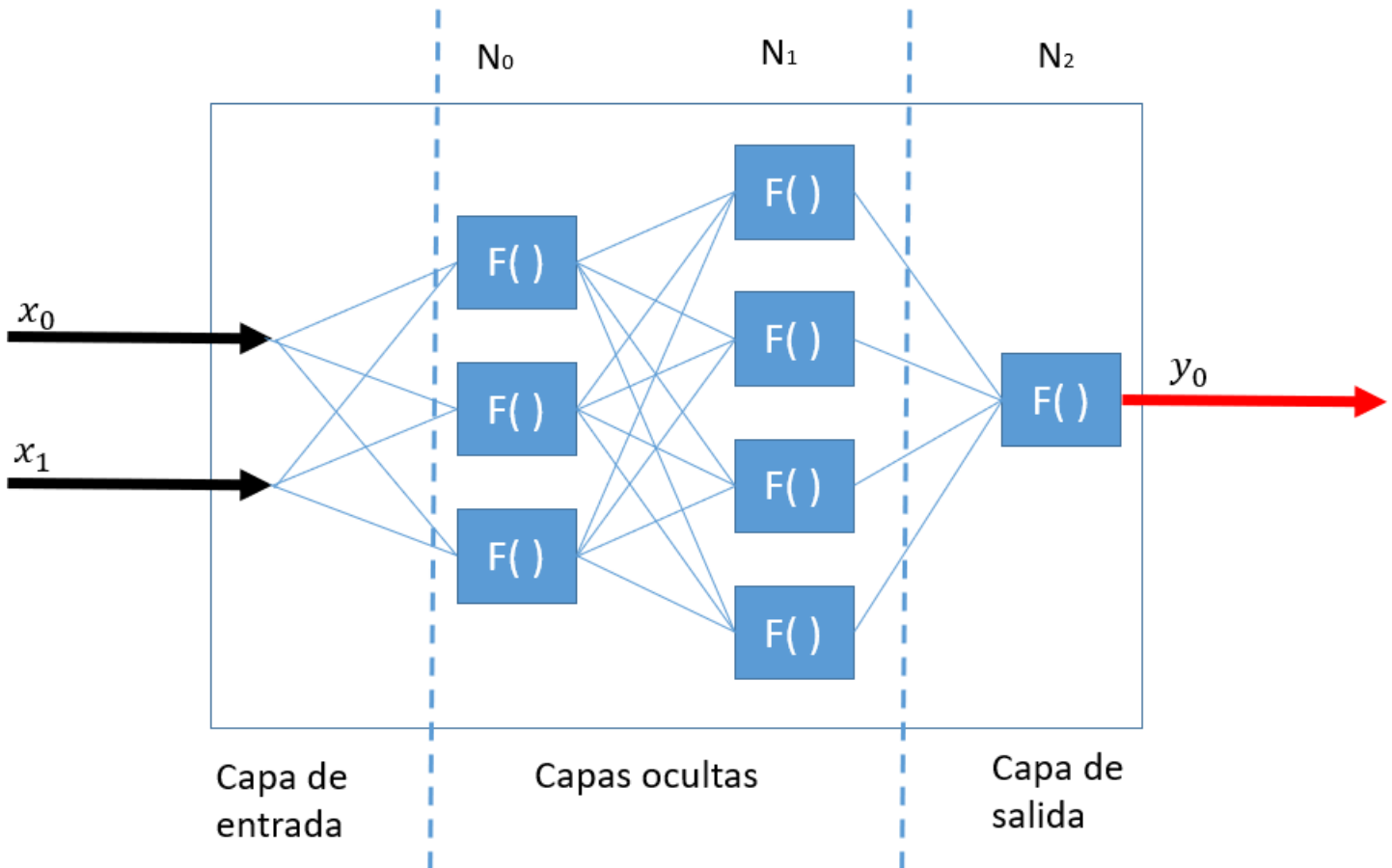


Ilustración 48: Un perceptrón multicapa

K/014.cs

```
class Program {
    static void Main() {
        //Un solo generador de números pseudo-aleatorios
        Random Azar = new();
        Perceptron perceptron = new Perceptron();

        int numEntradas = 2; //Número de entradas
        int capa0 = 3; //Total neuronas en la capa 0
        int capa1 = 4; //Total neuronas en la capa 1
        int capa2 = 1; //Total neuronas en la capa 2
        perceptron.CreaCapas(Azar, numEntradas, capa0, capa1, capa2);

        //Estas serán las entradas externas al perceptrón
        List<double> Entradas = new List<double>();
        Entradas.Add(1);
        Entradas.Add(0);
    }
}
```

```
        //Se hace el cálculo
        perceptron.calculaSalida(Entradas);
    }
}
```

Es en el programa principal que se crea el objeto que genera los números pseudoaleatorios y se le envía al perceptrón.

## Algoritmo de retro propagación

Las fórmulas del algoritmo de retro propagación para la capa 2:

$$\frac{\partial Error}{\partial w_{j,i}^{(2)}} = a_j^{(1)} * (y_i - S_i) * y_i * (1 - y_i)$$

Y

$$w_{j,i}^{(2)} \leftarrow w_{j,i}^{(2)} - \alpha * \frac{\partial Error}{\partial w_{j,i}^{(2)}}$$

Se implementa así:

```
for (int j = 0; j < NeuronasCapa1; j++)  
    //Va de neurona en neurona de la capa 1  
    for (int i = 0; i < NeuronasCapa2; i++) {  
        //Va de neurona en neurona de la capa de salida (capa 2)  
        double Yi = Capas[2].Salidas[i]; //Salida de la neurona de la capa de salida  
        double Si = SalidaEsperada[i]; //Salida esperada  
        double a1j = Capas[1].Salidas[j]; //Salida de la capa 1  
        double dE2 = a1j * (Yi - Si) * Yi * (1 - Yi); //La fórmula del error  
        Capas[2].Neuronas[i].NuevosPesos[j] = Capas[2].Neuronas[i].Pesos[j] -  
        Alpha * dE2; //Ajusta el nuevo peso  
    }
```

Para la capa 1:

$$\frac{\partial Error}{\partial w_{j,k}^{(1)}} = a_j^{(0)} * a_k^{(1)} * (1 - a_k^{(1)}) \\ * \sum_{i=0}^{N_2-1} (w_{k,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i))$$

Y

$$w_{j,k}^{(1)} \leftarrow w_{j,k}^{(1)} - \alpha * \frac{\partial Error}{\partial w_{j,k}^{(1)}}$$

Se implementa así:

```
for (int j = 0; j < NeuronasCapa0; j++)
    //Va de neurona en neurona de la capa 0
    for (int k = 0; k < NeuronasCapa1; k++) {
//Va de neurona en neurona de la capa 1
        double Acumula = 0;
        for (int i = 0; i < NeuronasCapa2; i++) { //Va de neurona en neurona
de la capa 2
            double Yi = Capas[2].Salidas[i]; //Salida de la capa 2
            double Si = SalidaEsperada[i]; //Salida esperada
            double W2ki = Capas[2].Neuronas[i].Pesos[k];
            Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi); //Sumatoria
        }
        double a0j = Capas[0].Salidas[j];
        double a1k = Capas[1].Salidas[k];
        double dE1 = a0j * a1k * (1 - a1k) * Acumula;
        Capas[1].Neuronas[k].NuevosPesos[j] = Capas[1].Neuronas[k].Pesos[j] -
Alpha * dE1;
    }
```

Para la capa 0:

$$\frac{\partial Error}{\partial w_{j,k}^{(0)}} = x_j * a_k^{(0)} * (1 - a_k^{(0)}) * \sum_{p=0}^{N_1-1} \left[ w_{k,p}^{(1)} * a_p^{(1)} * (1 - a_p^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{p,i}^{(2)} * (y_i - s_i) * y_i * (1 - y_i) \right) \right]$$

Y

$$w_{j,k}^{(0)} \leftarrow w_{j,k}^{(0)} - \alpha * \frac{\partial Error}{\partial w_{j,k}^{(0)}}$$

Se implementa así:

```
for (int j = 0; j < Entradas.Count; j++)
//Va de entrada en entrada
    for (int k = 0; k < NeuronasCapa0; k++) { //Va de neurona en neurona de
la capa 0
        double Acumula = 0;
        for (int p = 0; p < NeuronasCapa1; p++) { //Va de neurona en neurona
de la capa 1
            double InternoAcumula = 0;
            for (int i = 0; i < NeuronasCapa2; i++) { //Va de neurona en
neurona de la capa 2
                double Yi = Capas[2].Salidas[i];
                double Si = SalidaEsperada[i]; //Salida esperada
                double W2pi = Capas[2].Neuronas[i].Pesos[p];
                InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi); //Sumatoria
interna
            }
            double W1kp = Capas[1].Neuronas[p].Pesos[k];
            double alp = Capas[1].Salidas[p];
            Acumula += W1kp * alp * (1 - alp) * InternoAcumula; //Sumatoria
externa
        }
        double xj = Entradas[j];
        double a0k = Capas[0].Salidas[k];
        double dE0 = xj * a0k * (1 - a0k) * Acumula;
```



```
double W0jk = Capas[0].Neuronas[k].Pesos[j];  
Capas[0].Neuronas[k].NuevosPesos[j] = W0jk - Alpha * dE0;  
}
```

Para los umbrales de la capa 2

$$\frac{\partial Error}{\partial u_i^{(2)}} = (y_i - S_i) * y_i * (1 - y_i)$$

$$u_i^{(2)} \leftarrow u_i^{(2)} - \alpha * \frac{\partial Error}{\partial u_i^{(2)}}$$

Se implementa así:

```
for (int i = 0; i < NeuronasCapa2; i++) {  
    //Va de neurona en neurona de la capa de salida (capa 2)  
    double Yi = Capas[2].Salidas[i]; //Salida de la neurona de la capa de  
    salida  
    double Si = SalidaEsperada[i]; //Salida esperada  
    double dE2 = (Yi - Si) * Yi * (1 - Yi);  
    Capas[2].Neuronas[i].NuevoUmbral = Capas[2].Neuronas[i].Umbral - Alpha *  
    dE2;  
}
```

Para los umbrales de la capa 1

$$\frac{\partial Error}{\partial u_k^{(1)}} = a_k^{(1)} * (1 - a_k^{(1)}) * \sum_{i=0}^{N_2-1} \left( w_{k,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right)$$

$$u_k^{(1)} \leftarrow u_k^{(1)} - \alpha * \frac{\partial Error}{\partial u_k^{(1)}}$$

Se implementa así:

```
for (int k = 0; k < NeuronasCapa1; k++) {  
    //Va de neurona en neurona de la capa 1  
    double Acumula = 0;  
    for (int i = 0; i < NeuronasCapa2; i++) { //Va de neurona en neurona de  
        la capa 2  
        double Yi = Capas[2].Salidas[i]; //Salida de la capa 2  
        double Si = SalidaEsperada[i];  
        double W2ki = Capas[2].Neuronas[i].Pesos[k];  
        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi);  
    }  
    double alk = Capas[1].Salidas[k];  
    double dE1 = alk * (1 - alk) * Acumula;  
    Capas[1].Neuronas[k].NuevoUmbral = Capas[1].Neuronas[k].Umbral - Alpha *  
    dE1;  
}
```

Para los umbrales de la capa 0

$$\frac{\partial Error}{\partial u_k^{(0)}} = a_k^{(0)} * \left(1 - a_k^{(0)}\right) * \sum_{p=0}^{N_1-1} \left[ w_{k,p}^{(1)} * a_p^{(1)} * \left(1 - a_p^{(1)}\right) * \sum_{i=0}^{N_2-1} \left( w_{p,i}^{(2)} * (y_i - S_i) * y_i * (1 - y_i) \right) \right]$$

$$u_k^{(0)} \leftarrow u_k^{(0)} - \alpha * \frac{\partial Error}{\partial u_k^{(0)}}$$

Se implementa así:

```
for (int k = 0; k < NeuronasCapa0; k++) {
//Va de neurona en neurona de la capa 0
    double Acumula = 0;
    for (int p = 0; p < NeuronasCapa1; p++) { //Va de neurona en neurona de
la capa 1
        double InternoAcumula = 0;
        for (int i = 0; i < NeuronasCapa2; i++) { //Va de neurona en neurona
de la capa 2
            double Yi = Capas[2].Salidas[i];
            double Si = SalidaEsperada[i];
            double W2pi = Capas[2].Neuronas[i].Pesos[p];
            InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
        }
        double W1kp = Capas[1].Neuronas[p].Pesos[k];
        double alp = Capas[1].Salidas[p];
        Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
    }
    double a0k = Capas[0].Salidas[k];
    double dE0 = a0k * (1 - a0k) * Acumula;
    Capas[0].Neuronas[k].NuevoUmbral = Capas[0].Neuronas[k].Umbral - Alpha *
dE0;
}
```

Como se puede observar en los códigos, se deducen nuevos pesos y umbrales. Esos nuevos valores posteriormente deben reemplazar los viejos. Luego hay que hacer un cambio a la clase Neurona para que almacene temporalmente los nuevos valores para que cuando se termine de calcularlos, entonces reemplaza los viejos.

El código del encabezado sería así:

```
class Neurona {  
    public List<double> Pesos; //Los pesos para cada entrada  
    public List<double> NuevosPesos; //Nuevos pesos dados por el algoritmo  
    de "backpropagation"  
    public double Umbral; //El peso del umbral  
    public double NuevoUmbral; //Nuevo umbral dado por el algoritmo de  
    "backpropagation"
```

Y tendría un nuevo método que actualizaría los pesos con los nuevos valores, **después** de ejecutar el algoritmo de "backpropagation"

```
public void Actualiza() {  
    for (int Contador = 0; Contador < Pesos.Count; Contador++)  
        Pesos[Contador] = NuevosPesos[Contador];  
    Umbral = NuevoUmbral;  
}
```

Significa que en la clase Capa debe haber un método que llama la actualización de las neuronas

```
public void Actualiza() {  
    for (int Contador = 0; Contador < Neuronas.Count; Contador++)  
        Neuronas[Contador].Actualiza();  
}
```

## Código completo del perceptrón

A continuación, se muestra el código completo en el que se ha creado una clase que implementa el perceptrón (creación, proceso, entrenamiento) y en la clase principal se pone como datos de prueba la tabla del XOR que el perceptrón debe aprender.

K/015.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //Tabla XOR
        int[][] XORentra = [
            [1, 1],
            [1, 0],
            [0, 1],
            [0, 0]
        ];
        int[] XORsale = [0, 1, 1, 0];

        int TotalEntradas = 2; //Número de entradas
        int NeuronasCapa0 = 3; //Total neuronas en la capa 0
        int NeuronasCapa1 = 2; //Total neuronas en la capa 1
        int NeuronasCapa2 = 1; //Total neuronas en la capa 2
        Perceptron RedNeuronal = new(TotalEntradas, NeuronasCapa0,
                                      NeuronasCapa1, NeuronasCapa2);

        //Estas serán las dos entradas externas al perceptrón
        List<double> Entradas = [0, 0];

        //Esta será la salida esperada externa al perceptrón
        List<double> SalidaEsperada = [0];

        //Ciclo que entrena la red neuronal
        int TotalCiclos = 90000; //Ciclos de entrenamiento
        for (int Ciclo = 1; Ciclo <= TotalCiclos; Ciclo++) {

            if (Ciclo % 200 == 0) Console.WriteLine("\r\nCiclo: " + Ciclo);

            //Por cada ciclo, se entrena el
            //perceptrón con toda la tabla de XOR
            for (int Conjunto = 0; Conjunto < XORsale.Length; Conjunto++) {

                //Entradas y salidas esperadas
                Entradas[0] = XORentra[Conjunto][0];
                Entradas[1] = XORentra[Conjunto][1];
                SalidaEsperada[0] = XORsale[Conjunto];
            }
        }
    }
}
```

```

        //Primero calcula la salida del
        //perceptrón con esas entradas
        RedNeuronal.CalculaSalida(Entradas);

        //Luego entrena el perceptrón para
        //ajustar los pesos y umbrales
        RedNeuronal.Entrena(Entradas, SalidaEsperada);

        //Cada 200 ciclos muestra como
        //progresas el entrenamiento
        if (Ciclo % 200 == 0)
            RedNeuronal.SalidaPerceptron(Entradas, SalidaEsperada[0]);
    }
}

Console.WriteLine("Finaliza el entrenamiento");
}
}

class Perceptron {
    public List<Capa> Capas;

    //Imprime los datos de las diferentes capas
    public void SalidaPerceptron(List<double> Entradas,
                                double SalidaEsperada) {

        for (int Cont = 0; Cont < Entradas.Count; Cont++)
            Console.Write(Entradas[Cont] + " | ");

        Console.Write(" Esperada: " + SalidaEsperada + " Calculada: ");

        for (int Cont = 0; Cont < Capas[2].Salidas.Count; Cont++) {
            if (Capas[2].Salidas[Cont] >= 0.5)
                Console.Write(" 1 | ");
            else
                Console.Write(" 0 | ");
            Console.Write(Capas[2].Salidas[Cont] + " | ");
        }
        Console.WriteLine(" ");
    }

    //Crea las diversas capas
    public Perceptron(int TotalEntradas, int NeuronasCapa0,
                      int NeuronasCapa1, int NeuronasCapa2) {
        Random Azar = new();
        Capas =
        [

```

```

        new Capa(Azar, NeuronasCapa0, TotalEntradas), //Crea la capa 0
        new Capa(Azar, NeuronasCapa1, NeuronasCapa0), //Crea la capa 1
        new Capa(Azar, NeuronasCapa2, NeuronasCapa1), //Crea la capa 2
    ];
}

//Dada las entradas al perceptrón, se calcula
//la salida de cada capa. Con eso se sabrá que salidas
//se obtienen con los pesos y umbrales actuales. Esas
//salidas son requeridas para el algoritmo de entrenamiento.
public void CalculaSalida(List<double> Entradas) {
    Capas[0].CalculaCapa(Entradas);
    Capas[1].CalculaCapa(Capas[0].Salidas);
    Capas[2].CalculaCapa(Capas[1].Salidas);
}

//Con las salidas previamente calculadas con
//unas determinadas entradas se ejecuta el algoritmo
//de entrenamiento "Backpropagation"
public void Entrena(List<double> Entradas,
    List<double> SalidaEsperada) {
    int NeuronasCapa0 = Capas[0].Neuronas.Count;
    int NeuronasCapa1 = Capas[1].Neuronas.Count;
    int NeuronasCapa2 = Capas[2].Neuronas.Count;

    //Factor de aprendizaje
    double Alpha = 0.4;

    //=====
    //Procesa pesos capa 2
    //=====

    //Va de neurona en neurona de la capa 1
    for (int j = 0; j < NeuronasCapa1; j++)

        //Va de neurona en neurona de la capa de salida (capa 2)
        for (int i = 0; i < NeuronasCapa2; i++) {

            //Salida de la neurona de la capa de salida
            double Yi = Capas[2].Salidas[i];

            //Salida esperada
            double Si = SalidaEsperada[i];

            //Salida de la capa 1
            double a1j = Capas[1].Salidas[j];

```



```

        //La fórmula del error
        double dE2 = a1j * (Yi - Si) * Yi * (1 - Yi);

        //Ajusta el nuevo peso
        double Nuevo = Capas[2].Neuronas[i].Pesos[j] - Alpha * dE2;
        Capas[2].Neuronas[i].NuevosPesos[j] = Nuevo;
    }

    //=====
    //Procesa pesos capa 1
    //=====

    //Va de neurona en neurona de la capa 0
    for (int j = 0; j < NeuronasCapa0; j++)

        //Va de neurona en neurona de la capa 1
        for (int k = 0; k < NeuronasCapa1; k++) {
            double Acumula = 0;

            //Va de neurona en neurona de la capa 2
            for (int i = 0; i < NeuronasCapa2; i++) {

                //Salida de la capa 2
                double Yi = Capas[2].Salidas[i];

                //Salida esperada
                double Si = SalidaEsperada[i];
                double W2ki = Capas[2].Neuronas[i].Pesos[k];
                Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi); //Sumatoria
            }
            double a0j = Capas[0].Salidas[j];
            double a1k = Capas[1].Salidas[k];
            double dE1 = a0j * a1k * (1 - a1k) * Acumula;
            double Nuevo = Capas[1].Neuronas[k].Pesos[j] - Alpha * dE1;
            Capas[1].Neuronas[k].NuevosPesos[j] = Nuevo;
        }

    //=====
    //Procesa pesos capa 0
    //=====

    //Va de entrada en entrada
    for (int j = 0; j < Entradas.Count; j++)

        //Va de neurona en neurona de la capa 0
        for (int k = 0; k < NeuronasCapa0; k++) {
            double Acumula = 0;

```

```

//Va de neurona en neurona de la capa 1
for (int p = 0; p < NeuronasCapa1; p++) {
    double InternoAcumula = 0;

    //Va de neurona en neurona de la capa 2
    for (int i = 0; i < NeuronasCapa2; i++) {
        double Yi = Capas[2].Salidas[i];
        double Si = SalidaEsperada[i]; //Salida esperada
        double W2pi = Capas[2].Neuronas[i].Pesos[p];

        //Sumatoria interna
        InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
    }
    double W1kp = Capas[1].Neuronas[p].Pesos[k];
    double alp = Capas[1].Salidas[p];

    //Sumatoria externa
    Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
}
double xj = Entradas[j];
double a0k = Capas[0].Salidas[k];
double dE0 = xj * a0k * (1 - a0k) * Acumula;
double W0jk = Capas[0].Neuronas[k].Pesos[j];
Capas[0].Neuronas[k].NuevosPesos[j] = W0jk - Alpha * dE0;
}

//=====
//Procesa umbrales capa 2
//=====

//Va de neurona en neurona de la capa de salida (capa 2)
for (int i = 0; i < NeuronasCapa2; i++) {

    //Salida de la neurona de la capa de salida
    double Yi = Capas[2].Salidas[i];

    //Salida esperada
    double Si = SalidaEsperada[i];
    double dE2 = (Yi - Si) * Yi * (1 - Yi);
    double Nuevo = Capas[2].Neuronas[i].Umbral - Alpha * dE2;
    Capas[2].Neuronas[i].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 1
//=====

//Va de neurona en neurona de la capa 1

```

```

for (int k = 0; k < NeuronasCapa1; k++) {
    double Acumula = 0;

    //Va de neurona en neurona de la capa 2
    for (int i = 0; i < NeuronasCapa2; i++) {

        //Salida de la capa 2
        double Yi = Capas[2].Salidas[i];
        double Si = SalidaEsperada[i];
        double W2ki = Capas[2].Neuronas[i].Pesos[k];
        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi);
    }
    double alk = Capas[1].Salidas[k];
    double dE1 = alk * (1 - alk) * Acumula;
    double Nuevo = Capas[1].Neuronas[k].Umbral - Alpha * dE1;
    Capas[1].Neuronas[k].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 0
//=====

//Va de neurona en neurona de la capa 0
for (int k = 0; k < NeuronasCapa0; k++) {
    double Acumula = 0;

    //Va de neurona en neurona de la capa 1
    for (int p = 0; p < NeuronasCapa1; p++) {
        double InternoAcumula = 0;

        //Va de neurona en neurona de la capa 2
        for (int i = 0; i < NeuronasCapa2; i++) {
            double Yi = Capas[2].Salidas[i];
            double Si = SalidaEsperada[i];
            double W2pi = Capas[2].Neuronas[i].Pesos[p];
            InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
        }
        double W1kp = Capas[1].Neuronas[p].Pesos[k];
        double alp = Capas[1].Salidas[p];
        Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
    }
    double a0k = Capas[0].Salidas[k];
    double dE0 = a0k * (1 - a0k) * Acumula;
    double Nuevo = Capas[0].Neuronas[k].Umbral - Alpha * dE0;
    Capas[0].Neuronas[k].NuevoUmbral = Nuevo;
}

//Actualiza los pesos

```

```

        Capas[0].Actualizar();
        Capas[1].Actualizar();
        Capas[2].Actualizar();
    }

}

class Capa {
    //Las neuronas que tendrá la capa
    public List<Neurona> Neuronas;

    //Almacena las salidas de cada neurona
    public List<double> Salidas;

    public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
        Neuronas = [];
        Salidas = [];

        //Genera las neuronas
        for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
            Neuronas.Add(new Neurona(Azar, TotalEntradas));
            Salidas.Add(0);
        }
    }

    //Calcula las salidas de cada neurona de la capa
    public void CalculaCapa(List<double> Entradas) {
        for (int Contador = 0; Contador < Neuronas.Count; Contador++)
            Salidas[Contador] = Neuronas[Contador].CalculaSalida(Entradas);
    }

    //Actualiza los pesos y umbrales de las neuronas
    public void Actualiza() {
        for (int Contador = 0; Contador < Neuronas.Count; Contador++)
            Neuronas[Contador].Actualizar();
    }
}

class Neurona {
    //Los pesos para cada entrada
    public List<double> Pesos;

    //Nuevos pesos dados por el algoritmo de "backpropagation"
    public List<double> NuevosPesos;

    //El peso del umbral
    public double Umbral;
}

```

```

//Nuevo umbral dado por el algoritmo de "backpropagation"
public double NuevoUmbral;

//Inicializa los pesos y umbral con un valor al azar
public Neurona(Random Azar, int TotalEntradas) {
    Pesos = [];
    NuevosPesos = [];
    for (int Contador = 0; Contador < TotalEntradas; Contador++) {
        Pesos.Add(Azar.NextDouble());
        NuevosPesos.Add(0);
    }
    Umbral = Azar.NextDouble();
    NuevoUmbral = 0;
}

//Calcula la salida de la neurona dependiendo de las entradas
public double CalculaSalida(List<double> Entradas) {
    double Valor = 0;
    for (int Contador = 0; Contador < Pesos.Count; Contador++)
        Valor += Entradas[Contador] * Pesos[Contador];
    Valor += Umbral;
    return 1 / (1 + Math.Exp(-Valor));
}

//Reemplaza viejos pesos por nuevos
public void Actualiza() {
    for (int Contador = 0; Contador < Pesos.Count; Contador++)
        Pesos[Contador] = NuevosPesos[Contador];
    Umbral = NuevoUmbral;
}
}

```

Ciclo: 89800

|   |  |   |  |             |              |  |                       |  |
|---|--|---|--|-------------|--------------|--|-----------------------|--|
| 1 |  | 1 |  | Esperada: 0 | Calculada: 0 |  | 0,003702174383143816  |  |
| 1 |  | 0 |  | Esperada: 1 | Calculada: 1 |  | 0,9953775432424248    |  |
| 0 |  | 1 |  | Esperada: 1 | Calculada: 1 |  | 0,9953696734810018    |  |
| 0 |  | 0 |  | Esperada: 0 | Calculada: 0 |  | 0,0031923050449013304 |  |

Ciclo: 90000

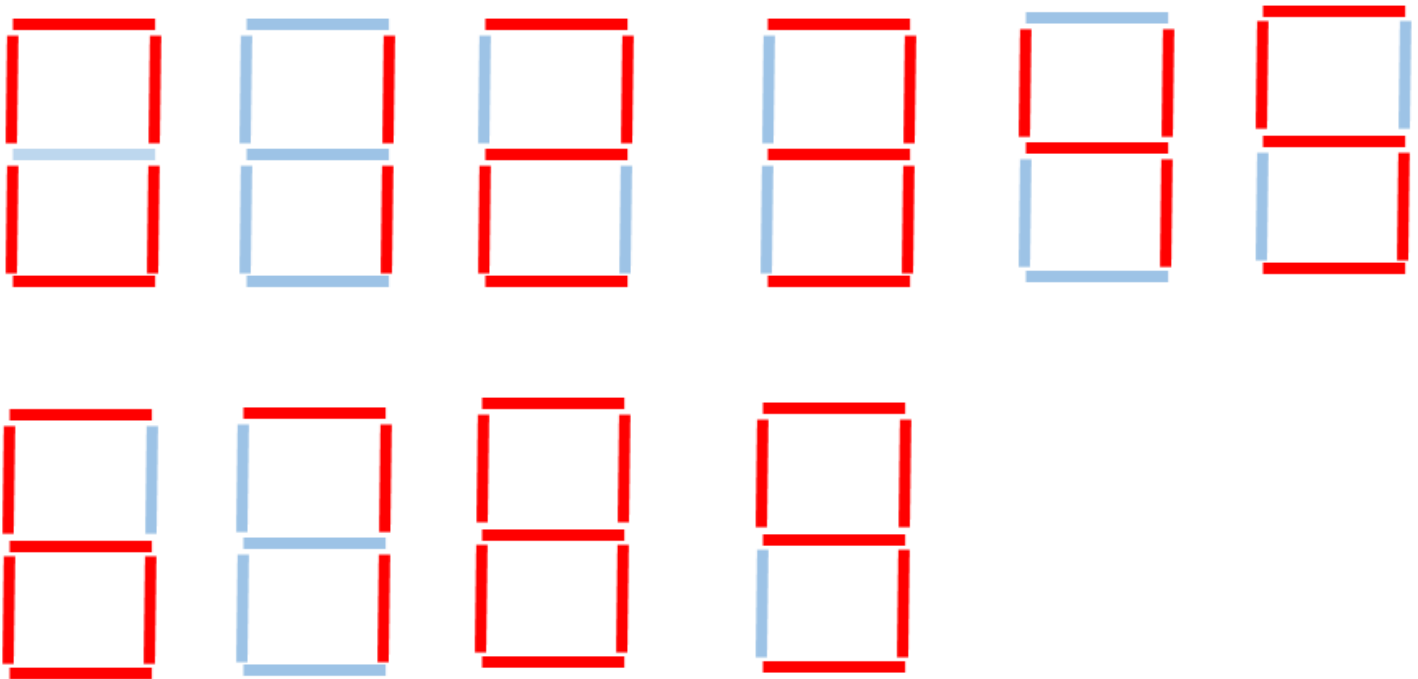
|   |  |   |  |             |              |  |                       |  |
|---|--|---|--|-------------|--------------|--|-----------------------|--|
| 1 |  | 1 |  | Esperada: 0 | Calculada: 0 |  | 0,003695850644007681  |  |
| 1 |  | 0 |  | Esperada: 1 | Calculada: 1 |  | 0,9953851916858888    |  |
| 0 |  | 1 |  | Esperada: 1 | Calculada: 1 |  | 0,9953773370654849    |  |
| 0 |  | 0 |  | Esperada: 0 | Calculada: 0 |  | 0,0031871208730737985 |  |

Finaliza el entrenamiento

*Ilustración 49: Perceptrón aprendiendo la tabla del XOR*

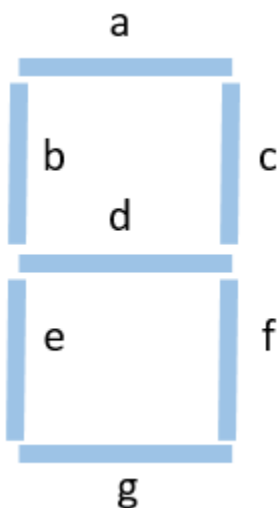
## Reconocimiento de números de un reloj digital

En la imagen, los números del 0 al 9 contruidos usando las barras verticales y horizontales. Típicos de un reloj digital.

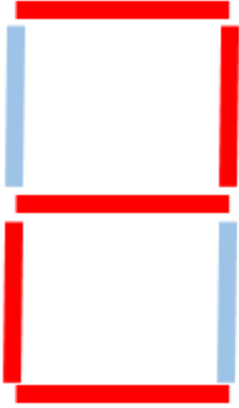


*Ilustración 50: Números en un reloj digital*

Se quiere construir una red neuronal tipo perceptrón multicapa que dado ese número al estilo reloj digital se pueda deducir el número como tal. Para iniciar se pone un identificador a cada barra



Luego se le da un valor de "1" a la barra que queda en rojo al construir el número y "0" a la barra que queda en azul claro. Por ejemplo:



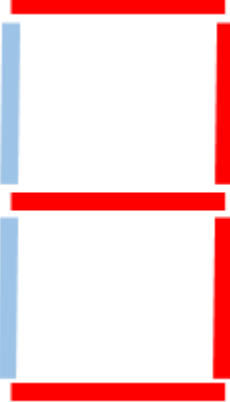
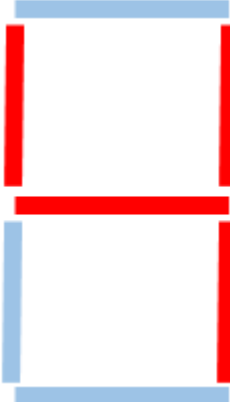
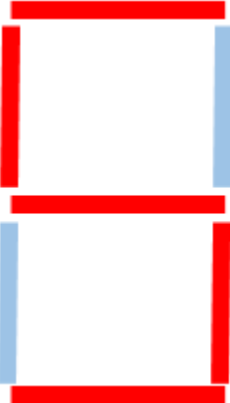
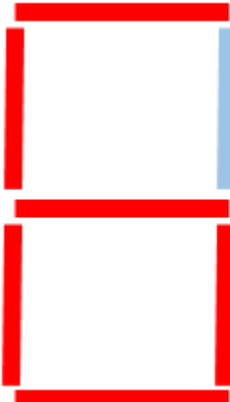
Los valores de a,b,c,d,e,f,g serían: 1,0,1,1,1,0,1

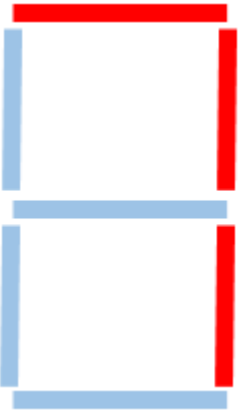
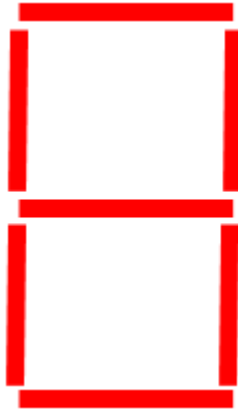
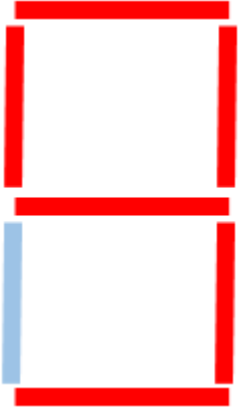
Como se debe deducir que es un 2, este valor se convierte a binario  $10_2$  o 0,0,1,0 (para convertirlo en salida del perceptrón, como el máximo valor del reloj digital es 9 y este es 1,0,0,1 entonces el número de salidas es 4)



La tabla de entradas y salidas esperadas es:

| Imagen                                                                              | Valor de entrada | Valor de salida esperado |
|-------------------------------------------------------------------------------------|------------------|--------------------------|
|    | 1,1,1,0,1,1,1    | 0,0,0,0                  |
|   | 0,0,1,0,0,1,0    | 0,0,0,1                  |
|  | 1,0,1,1,1,0,1    | 0,0,1,0                  |
|                                                                                     | 1,0,1,1,0,1,1    | 0,0,1,1                  |

|                                                                                     |               |         |
|-------------------------------------------------------------------------------------|---------------|---------|
|    |               |         |
|    | 0,1,1,1,0,1,0 | 0,1,0,0 |
|  | 1,1,0,1,0,1,1 | 0,1,0,1 |
|  | 1,1,0,1,1,1,1 | 0,1,1,0 |

|                                                                                     |               |         |
|-------------------------------------------------------------------------------------|---------------|---------|
|    | 1,0,1,0,0,1,0 | 0,1,1,1 |
|   | 1,1,1,1,1,1,1 | 1,0,0,0 |
|  | 1,1,1,1,0,1,1 | 1,0,0,1 |

El código es el siguiente:

K/016.cs

```
namespace Ejemplo;

class Program {
    static void Main() {
        //El número "dibujado" en el reloj digital
        int[][] Reloj = [
            [1, 1, 1, 0, 1, 1, 1],
            [0, 0, 1, 0, 0, 1, 0],
            [1, 0, 1, 1, 1, 0, 1],
            [1, 0, 1, 1, 0, 1, 1],
            [0, 1, 1, 1, 0, 1, 0],
            [1, 1, 0, 1, 0, 1, 1],
            [1, 1, 0, 1, 1, 1, 1],
            [1, 0, 1, 0, 0, 1, 0],
            [1, 1, 1, 1, 1, 1, 1],
            [1, 1, 1, 1, 0, 1, 1]
        ];

        //Número esperado en binario
        int[][] Esperado = [
            [0, 0, 0, 0],
            [0, 0, 0, 1],
            [0, 0, 1, 0],
            [0, 0, 1, 1],
            [0, 1, 0, 0],
            [0, 1, 0, 1],
            [0, 1, 1, 0],
            [0, 1, 1, 1],
            [1, 0, 0, 0],
            [1, 0, 0, 1]
        ];

        int TotalEntradas = 7; //Número de entradas
        int NeuronasCapa0 = 5; //Total neuronas en la capa 0
        int NeuronasCapa1 = 5; //Total neuronas en la capa 1
        int NeuronasCapa2 = 4; //Total neuronas en la capa 2
        Perceptron RedNeuronal = new(TotalEntradas, NeuronasCapa0,
                                      NeuronasCapa1, NeuronasCapa2);

        //Estas serán las dos entradas externas al perceptrón
        List<double> Entradas = [0, 0, 0, 0, 0, 0, 0];

        //Esta será la salida esperada externa al perceptrón
        List<double> SalidaEsperada = [0, 0, 0, 0];
    }
}
```

```

//Ciclo que entrena la red neuronal
int TotalCiclos = 10000; //Ciclos de entrenamiento
for (int Ciclo = 1; Ciclo <= TotalCiclos; Ciclo++) {

    if (Ciclo % 1000 == 0)
        Console.WriteLine("\r\nCiclo: " + Ciclo);

    //Por cada ciclo, se entrena el perceptrón
    //con toda la tabla
    for (int Conj = 0; Conj < Esperado.GetLength(0); Conj++) {

        //Entradas y salidas esperadas
        for (int Entra = 0; Entra < Reloj[0].GetLength(0); Entra++)
            Entradas[Entra] = Reloj[Conj][Entra];

        for (int Sale = 0; Sale < Esperado[0].GetLength(0); Sale++)
            SalidaEsperada[Sale] = Esperado[Conj][Sale];

        //Primero calcula la salida del
        //perceptrón con esas entradas
        RedNeuronal.CalculaSalida(Entradas);

        //Luego entrena el perceptrón para
        //ajustar los pesos y umbrales
        RedNeuronal.Entrena(Entradas, SalidaEsperada);

        //Cada 1000 ciclos muestra como
        //progresas el entrenamiento
        if (Ciclo % 1000 == 0)
            RedNeuronal.SalidaPerceptron(Entradas, SalidaEsperada);
    }
}

Console.WriteLine("Finaliza el entrenamiento");
}

class Perceptron {
    public List<Capa> Capas;

    //Imprime los datos de las diferentes capas
    public void SalidaPerceptron(List<double> Entradas,
                                List<double> SalidaEsperada) {

        for (int cont = 0; cont < Entradas.Count; cont++) {
            Console.Write(Entradas[cont] + " ");
        }
    }
}

```

```

    Console.WriteLine(" Esperada: ");

    for (int cont = 0; cont < SalidaEsperada.Count; cont++) {
        Console.WriteLine(SalidaEsperada[cont] + " ");
    }

    Console.WriteLine(" Calculada: ");

    for (int cont = 0; cont < Capas[2].Salidas.Count; cont++) {
        if (Capas[2].Salidas[cont] >= 0.5)
            Console.WriteLine("1 ");
        else
            Console.WriteLine("0 ");
    }
    Console.WriteLine(" ");
}

//Crea las diversas capas
public Perceptron(int TotalEntradas, int NeuronasCapa0,
                  int NeuronasCapa1, int NeuronasCapa2) {
    Random Azar = new();
    Capas =
    [
        new Capa(Azar, NeuronasCapa0, TotalEntradas), //Crea la capa 0
        new Capa(Azar, NeuronasCapa1, NeuronasCapa0), //Crea la capa 1
        new Capa(Azar, NeuronasCapa2, NeuronasCapa1), //Crea la capa 2
    ];
}

//Dada las entradas al perceptrón, se calcula
//la salida de cada capa. Con eso se sabrá que salidas
//se obtienen con los pesos y umbrales actuales. Esas
//salidas son requeridas para el algoritmo de entrenamiento.
public void CalculaSalida(List<double> Entradas) {
    Capas[0].CalculaCapa(Entradas);
    Capas[1].CalculaCapa(Capas[0].Salidas);
    Capas[2].CalculaCapa(Capas[1].Salidas);
}

//Con las salidas previamente calculadas con
//unas determinadas entradas se ejecuta el algoritmo
//de entrenamiento "Backpropagation"
public void Entrena(List<double> Entradas,
                    List<double> SalidaEsperada) {
    int NeuronasCapa0 = Capas[0].Neuronas.Count;
    int NeuronasCapa1 = Capas[1].Neuronas.Count;
    int NeuronasCapa2 = Capas[2].Neuronas.Count;

```

```

//Factor de aprendizaje
double Alpha = 0.4;

//=====
//Procesa pesos capa 2
//=====

//Va de neurona en neurona de la capa 1
for (int j = 0; j < NeuronasCapa1; j++)

    //Va de neurona en neurona de la capa de salida (capa 2)
    for (int i = 0; i < NeuronasCapa2; i++) {

        //Salida de la neurona de la capa de salida
        double Yi = Capas[2].Salidas[i];

        //Salida esperada
        double Si = SalidaEsperada[i];

        //Salida de la capa 1
        double alj = Capas[1].Salidas[j];

        //La fórmula del error
        double dE2 = alj * (Yi - Si) * Yi * (1 - Yi);

        //Ajusta el nuevo peso
        double Nuevo = Capas[2].Neuronas[i].Pesos[j] - Alpha * dE2;
        Capas[2].Neuronas[i].NuevosPesos[j] = Nuevo;
    }

//=====
//Procesa pesos capa 1
//=====

//Va de neurona en neurona de la capa 0
for (int j = 0; j < NeuronasCapa0; j++)

    //Va de neurona en neurona de la capa 1
    for (int k = 0; k < NeuronasCapa1; k++) {
        double Acumula = 0;

        //Va de neurona en neurona de la capa 2
        for (int i = 0; i < NeuronasCapa2; i++) {

            //Salida de la capa 2
            double Yi = Capas[2].Salidas[i];

```

```

        //Salida esperada
        double Si = SalidaEsperada[i];
        double W2ki = Capas[2].Neuronas[i].Pesos[k];
        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi); //Sumatoria
    }
    double a0j = Capas[0].Salidas[j];
    double a1k = Capas[1].Salidas[k];
    double dE1 = a0j * a1k * (1 - a1k) * Acumula;
    double Nuevo = Capas[1].Neuronas[k].Pesos[j] - Alpha * dE1;
    Capas[1].Neuronas[k].NuevosPesos[j] = Nuevo;
}

//=====
//Procesa pesos capa 0
//=====

//Va de entrada en entrada
for (int j = 0; j < Entradas.Count; j++)

    //Va de neurona en neurona de la capa 0
    for (int k = 0; k < NeuronasCapa0; k++) {
        double Acumula = 0;

        //Va de neurona en neurona de la capa 1
        for (int p = 0; p < NeuronasCapa1; p++) {
            double InternoAcumula = 0;

            //Va de neurona en neurona de la capa 2
            for (int i = 0; i < NeuronasCapa2; i++) {
                double Yi = Capas[2].Salidas[i];
                double Si = SalidaEsperada[i]; //Salida esperada
                double W2pi = Capas[2].Neuronas[i].Pesos[p];

                //Sumatoria interna
                InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
            }
            double W1kp = Capas[1].Neuronas[p].Pesos[k];
            double alp = Capas[1].Salidas[p];

            //Sumatoria externa
            Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
        }
        double xj = Entradas[j];
        double a0k = Capas[0].Salidas[k];
        double dE0 = xj * a0k * (1 - a0k) * Acumula;
        double W0jk = Capas[0].Neuronas[k].Pesos[j];
        Capas[0].Neuronas[k].NuevosPesos[j] = W0jk - Alpha * dE0;
    }
}

```



```

//=====
//Procesa umbrales capa 2
//=====

//Va de neurona en neurona de la capa de salida (capa 2)
for (int i = 0; i < NeuronasCapa2; i++) {

    //Salida de la neurona de la capa de salida
    double Yi = Capas[2].Salidas[i];

    //Salida esperada
    double Si = SalidaEsperada[i];
    double dE2 = (Yi - Si) * Yi * (1 - Yi);
    double Nuevo = Capas[2].Neuronas[i].Umbral - Alpha * dE2;
    Capas[2].Neuronas[i].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 1
//=====

//Va de neurona en neurona de la capa 1
for (int k = 0; k < NeuronasCapa1; k++) {
    double Acumula = 0;

    //Va de neurona en neurona de la capa 2
    for (int i = 0; i < NeuronasCapa2; i++) {

        //Salida de la capa 2
        double Yi = Capas[2].Salidas[i];
        double Si = SalidaEsperada[i];
        double W2ki = Capas[2].Neuronas[i].Pesos[k];
        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi);
    }
    double alk = Capas[1].Salidas[k];
    double dE1 = alk * (1 - alk) * Acumula;
    double Nuevo = Capas[1].Neuronas[k].Umbral - Alpha * dE1;
    Capas[1].Neuronas[k].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 0
//=====

//Va de neurona en neurona de la capa 0
for (int k = 0; k < NeuronasCapa0; k++) {
    double Acumula = 0;

```

```

        //Va de neurona en neurona de la capa 1
        for (int p = 0; p < NeuronasCapa1; p++) {
            double InternoAcumula = 0;

            //Va de neurona en neurona de la capa 2
            for (int i = 0; i < NeuronasCapa2; i++) {
                double Yi = Capas[2].Salidas[i];
                double Si = SalidaEsperada[i];
                double W2pi = Capas[2].Neuronas[i].Pesos[p];
                InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
            }
            double W1kp = Capas[1].Neuronas[p].Pesos[k];
            double alp = Capas[1].Salidas[p];
            Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
        }
        double a0k = Capas[0].Salidas[k];
        double dE0 = a0k * (1 - a0k) * Acumula;
        double Nuevo = Capas[0].Neuronas[k].Umbral - Alpha * dE0;
        Capas[0].Neuronas[k].NuevoUmbral = Nuevo;
    }

    //Actualiza los pesos
    Capas[0].Actualiza();
    Capas[1].Actualiza();
    Capas[2].Actualiza();
}

}

class Capa {
    //Las neuronas que tendrá la capa
    public List<Neurona> Neuronas;

    //Almacena las salidas de cada neurona
    public List<double> Salidas;

    public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
        Neuronas = [];
        Salidas = [];

        //Genera las neuronas
        for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
            Neuronas.Add(new Neurona(Azar, TotalEntradas));
            Salidas.Add(0);
        }
    }
}

```

```

//Calcula las salidas de cada neurona de la capa
public void CalculaCapa(List<double> Entradas) {
    for (int Contador = 0; Contador < Neuronas.Count; Contador++)
        Salidas[Contador] = Neuronas[Contador].CalculaSalida(Entradas);
}

//Actualiza los pesos y umbrales de las neuronas
public void Actualiza() {
    for (int Contador = 0; Contador < Neuronas.Count; Contador++)
        Neuronas[Contador].Actualiza();
}
}

class Neurona {
    //Los pesos para cada entrada
    public List<double> Pesos;

    //Nuevos pesos dados por el algoritmo de "backpropagation"
    public List<double> NuevosPesos;

    //El peso del umbral
    public double Umbral;

    //Nuevo umbral dado por el algoritmo de "backpropagation"
    public double NuevoUmbral;

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        NuevosPesos = [];
        for (int Contador = 0; Contador < TotalEntradas; Contador++) {
            Pesos.Add(Azar.NextDouble());
            NuevosPesos.Add(0);
        }
        Umbral = Azar.NextDouble();
        NuevoUmbral = 0;
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }

    //Reemplaza viejos pesos por nuevos

```

```

public void Actualiza() {
    for (int Contador = 0; Contador < Pesos.Count; Contador++)
        Pesos[Contador] = NuevosPesos[Contador];
    Umbral = NuevoUmbral;
}
}

```

Consola de depuración de Mi X + v

Ciclo: 1000

|               |                   |                    |
|---------------|-------------------|--------------------|
| 1 1 1 0 1 1 1 | Esperada: 0 0 0 0 | Calculada: 0 0 0 0 |
| 0 0 1 0 0 1 0 | Esperada: 0 0 0 1 | Calculada: 0 0 0 1 |
| 1 0 1 1 1 0 1 | Esperada: 0 0 1 0 | Calculada: 0 0 1 0 |
| 1 0 1 1 0 1 1 | Esperada: 0 0 1 1 | Calculada: 0 0 1 1 |
| 0 1 1 1 0 1 0 | Esperada: 0 1 0 0 | Calculada: 1 0 0 0 |
| 1 1 0 1 0 1 1 | Esperada: 0 1 0 1 | Calculada: 0 1 0 1 |
| 1 1 0 1 1 1 1 | Esperada: 0 1 1 0 | Calculada: 0 0 1 0 |
| 1 0 1 0 0 1 0 | Esperada: 0 1 1 1 | Calculada: 0 1 1 1 |
| 1 1 1 1 1 1 1 | Esperada: 1 0 0 0 | Calculada: 1 0 0 0 |
| 1 1 1 1 0 1 1 | Esperada: 1 0 0 1 | Calculada: 0 1 0 1 |

Ilustración 51: Aprendiendo los patrones para identificar el número digital.

Ciclo: 10000

|               |                   |                    |
|---------------|-------------------|--------------------|
| 1 1 1 0 1 1 1 | Esperada: 0 0 0 0 | Calculada: 0 0 0 0 |
| 0 0 1 0 0 1 0 | Esperada: 0 0 0 1 | Calculada: 0 0 0 1 |
| 1 0 1 1 1 0 1 | Esperada: 0 0 1 0 | Calculada: 0 0 1 0 |
| 1 0 1 1 0 1 1 | Esperada: 0 0 1 1 | Calculada: 0 0 1 1 |
| 0 1 1 1 0 1 0 | Esperada: 0 1 0 0 | Calculada: 0 1 0 0 |
| 1 1 0 1 0 1 1 | Esperada: 0 1 0 1 | Calculada: 0 1 0 1 |
| 1 1 0 1 1 1 1 | Esperada: 0 1 1 0 | Calculada: 0 1 1 0 |
| 1 0 1 0 0 1 0 | Esperada: 0 1 1 1 | Calculada: 0 1 1 1 |
| 1 1 1 1 1 1 1 | Esperada: 1 0 0 0 | Calculada: 1 0 0 0 |
| 1 1 1 1 0 1 1 | Esperada: 1 0 0 1 | Calculada: 1 0 0 1 |

Finaliza el entrenamiento

Ilustración 52: Aprendió los patrones para identificar el número digital.

## Detección de patrones en series de tiempo

En los dos ejemplos anteriores, los valores de las entradas externas del perceptrón fueron 0 o 1. Pero no está limitado a eso, las entradas externas pueden tener valores entre 0 y 1 (incluyendo el 0 y el 1) por ejemplo: 0.7321, 0.21896, 0.9173418

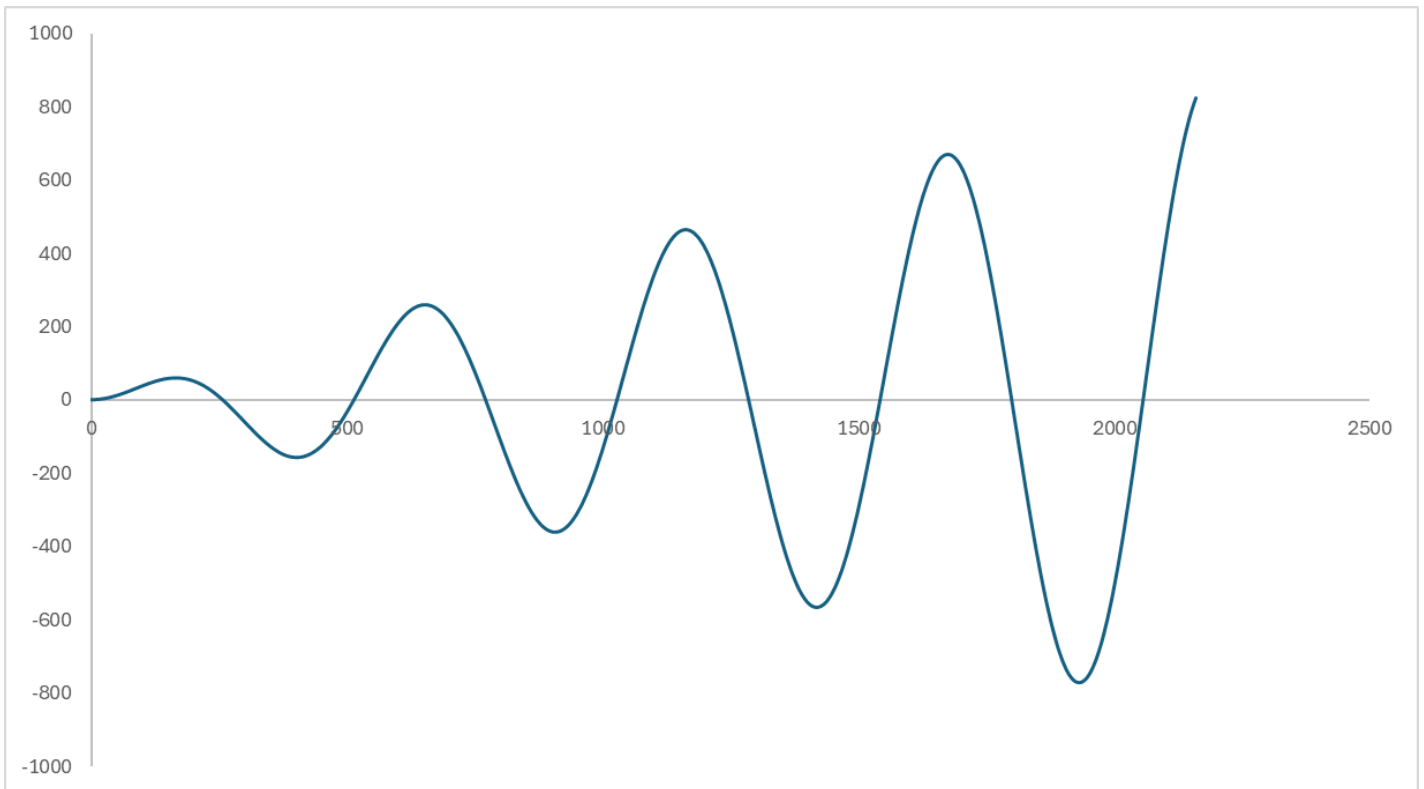
El problema que se plantea es dado el comportamiento de un evento en el tiempo, ¿podrá la red neuronal deducir el patrón?

Ejemplo: Se tiene esta tabla

| <b>X</b> | <b>Y</b> |
|----------|----------|
| 0        | 0        |
| 3        | 0,05025  |
| 6        | 0,1883   |
| 9        | 0,41377  |
| 12       | 0,72605  |
| 15       | 1,12429  |
| 18       | 1,6074   |
| 21       | 2,17408  |
| 24       | 2,82279  |
| 27       | 3,55176  |
| 30       | 4,35901  |
| 33       | 5,24235  |
| 36       | 6,19936  |
| 39       | 7,22742  |
| 42       | 8,32371  |
| 45       | 9,4852   |
| 48       | 10,7087  |
| 51       | 11,9908  |
| 54       | 13,328   |
| 57       | 14,7164  |
| 60       | 16,1523  |
| 63       | 17,6315  |
| 66       | 19,1499  |
| 69       | 20,7031  |
| 72       | 22,2866  |

X es la variable independiente mientras Y es la variable dependiente, en otras palabras,  $Y=F(X)$ . El problema es que no se sabe  $F( )$ , sólo están los datos. Esta sería la gráfica.

Uniendo los puntos, se obtendría



*Ilustración 53: Gráfico uniendo los puntos*

El primer paso es convertir los valores de X y de Y que dan origen al gráfico, en valores entre 0 y 1 porque la función de activación del perceptrón multicapa  $y = \frac{1}{1+e^{-x}}$  solo genera números entre 0 y 1.

Se hace entonces una normalización usando la siguiente fórmula

$$X_{normalizado} = \frac{X_{original} - MinimoX}{MaximoX - MinimoX}$$

$$Y_{normalizado} = \frac{Y_{original} - MinimoY}{MaximoY - MinimoY}$$

Como indica la fórmula, habría que recorrer todos los datos de X y Y para deducir el mayor valor de X, menor valor de X, mayor valor de Y y menor valor de Y. Con esos datos normalizados, se alimenta la red neuronal para entrenarla.

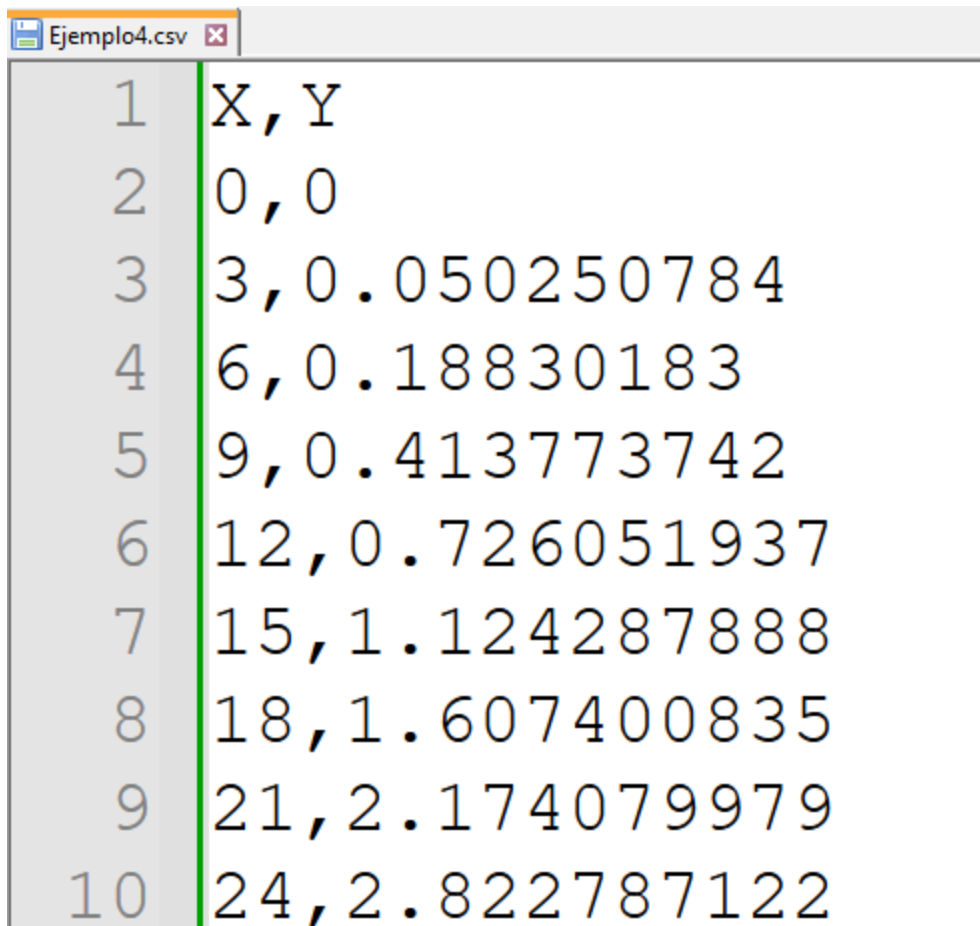
## El problema del sobre-entrenamiento

Al entrenar con un número fijo de ciclos (épocas) a una red neuronal, el resultado es que el modelo se ajusta mucho a la curva XY, pero puede sobreajustarse, es decir, aprender demasiado los datos específicos y no generalizar bien a nuevos datos. El objetivo de un modelo no es memorizar los datos, sino aprender patrones generales que funcionen bien con datos nuevos. ¿Cómo se logra esto? Se usa un subconjunto para entrenar (70%) y un subconjunto de prueba (30%), el entrenamiento se detiene cuando el perceptrón va perdiendo ajuste al subconjunto de prueba.

Eso significa que debe haber un número mínimo de registros, mínimo cien(100).

## Leyendo archivos CSV

Se escribe el programa en C# para que lea el archivo de datos en formato .CSV que es el usual en análisis de datos.



| 1  | X, Y            |
|----|-----------------|
| 2  | 0, 0            |
| 3  | 3, 0.050250784  |
| 4  | 6, 0.18830183   |
| 5  | 9, 0.413773742  |
| 6  | 12, 0.726051937 |
| 7  | 15, 1.124287888 |
| 8  | 18, 1.607400835 |
| 9  | 21, 2.174079979 |
| 10 | 24, 2.822787122 |

*Ilustración 54: Archivo CSV*



```
namespace Ejemplo;

class Program {
    static void Main() {
        Random Azar = new();

        //=====
        //Leer los datos del archivo CSV
        //=====
        DatosArchivo Datos = new();
        Datos.LeeXYdeCSV("Ejemplo1.csv");

        //Pasa los datos a dos listas: una de entrenamiento y otra de prueba
        List<double> Xentrenamiento =[.. Datos.Xentrada];
        List<double> Yentrenamiento =[.. Datos.Ysalidas];
        List<double> Xprueba = [];
        List<double> Yprueba = [];

        int PorcentajePrueba = 30;
        int Extrae = Xentrenamiento.Count * PorcentajePrueba / 100;

        //Con los datos de entrada y salida (el dataset), ahora procede a normalizarlos
        double MinX = Xentrenamiento.Min();
        double MaxX = Xentrenamiento.Max();
        double MinY = Yentrenamiento.Min();
        double MaxY = Yentrenamiento.Max();

        for (int Cont = 0; Cont < Yentrenamiento.Count; Cont++) {
            Xentrenamiento[Cont] = (Xentrenamiento[Cont] - MinX) / (MaxX - MinX);
            Yentrenamiento[Cont] = (Yentrenamiento[Cont] - MinY) / (MaxY - MinY);
        }
    }
}
```

```

//Genera las dos listas: de entrenamiento y prueba
while (Xprueba.Count < Extrae) {
    int Pos = Azar.Next(Xentrenna.Count);
    Xprueba.Add(Xentrenna[Pos]);
    Yprueba.Add(Yentrenna[Pos]);
    Xentrenna.RemoveAt(Pos);
    Yentrenna.RemoveAt(Pos);
}

//Entrena la red neuronal para que detecte el patrón
int TotalEntradas = 1; //Número de entradas
int NeuronasCapa0 = 7; //Total neuronas en la capa 0
int NeuronasCapa1 = 7; //Total neuronas en la capa 1
int NeuronasCapa2 = 1; //Total neuronas en la capa 2
Perceptron RedNeuronal = new(Azar, TotalEntradas, NeuronasCapa0,
                             NeuronasCapa1, NeuronasCapa2);

//Esta será la única entrada externa al perceptrón, es decir, X
List<double> Entrada = [0];

//Esta será la salida esperada externa al perceptrón, es decir, Y
List<double> SalidaEsperada = [0];

//Ciclo que entrena la red neuronal
bool SigueEntrenando = true;
int Ciclo = 0;
double Ajuste = double.MaxValue;
while (SigueEntrenando) {
    Ciclo++;

    //Entrena la red neuronal
    for (int Cont = 0; Cont < Xentrenna.Count; Cont++) {

        //Entrada y salida esperadas
        Entrada[0] = Xentrenna[Cont];
        SalidaEsperada[0] = Yentrenna[Cont];
    }
}

```

```

        //Primero calcula la salida del perceptrón con esa entrada
        RedNeuronal.CalculaSalida(Entrada);

        //Luego entrena el perceptrón para ajustar los pesos y umbrales
        RedNeuronal.Entrena(Entrada, SalidaEsperada);
    }

    if (Ciclo % 200 == 0) {

        //Entrada y salida de prueba
        double Diferencia = 0;
        for (int Cont = 0; Cont < Xprueba.Count; Cont++) {
            Entrada[0] = Xprueba[Cont];

            //Calcula la salida del perceptrón con esa entrada
            RedNeuronal.CalculaSalida(Entrada);
            double Salida = RedNeuronal.Capas[2].Salidas[0];

            //Acumula el error
            Diferencia += (Salida - Yprueba[Cont]) * (Salida - Yprueba[Cont]);
        }

        //Compara con el ajuste anterior
        if (Diferencia <= Ajuste) {
            Ajuste = Diferencia;
            Console.WriteLine("Ciclos: " + Ciclo + " Nuevo Ajuste: " + Ajuste);
        }
        else { //Si el ajuste anterior era menor, significa que ha
            //comenzado un sobreajuste, luego detiene el entrenamiento
            Console.WriteLine("Ciclos: " + Ciclo + " Desajuste: " + Diferencia);
            SigueEntrenando = false;
        }
    }
}

```

```

        Console.WriteLine("Entrada;Salida esperada;Salida perceptrón");

        for (int Cont = 0; Cont < Datos.Xentrada.Count; Cont++) {
            //Entradas y salidas esperadas
            Entrada[0] = (Datos.Xentrada[Cont]-MinX)/(MaxX-MinX);

            //Calcula la salida del perceptrón con esas entradas
            RedNeuronal.CalcularSalida(Entrada);
            double Salida = RedNeuronal.Capas[2].Salidas[0] * (MaxY - MinY) + MinY;

            //Imprime
            Console.WriteLine(Datos.Xentrada[Cont] + ";" + Datos.Ysalidas[Cont] + ";" + Salida);
        }
        Console.WriteLine("Finaliza el entrenamiento");
    }
}

```

```

using System.Globalization;

namespace Ejemplo;

internal class DatosArchivo {
    //Los datos seleccionados para
    //encontrar patrón de comportamiento
    public List<double> Xentrada;
    public List<double> Ysalidas;

    //Lee los valores X y Y.
    public void LeerXYdeCSV(string urlArchivo) {
        //Empieza a leer el archivo
        var Archivo = new StreamReader(urlArchivo);

        //Inicializa las listas
    }
}

```

```

Xentrada = [];
Ysalidas = [];

//Lee la linea de los dos datos numéricos
string LineaDato;
double valX, valY;
Archivo.ReadLine();
while ((LineaDato = Archivo.ReadLine()) != null) {
    int Coma = LineaDato.IndexOf(',');
    string Xc = LineaDato[..Coma];
    string Yc = LineaDato[(Coma + 1)..];
    valX = double.Parse(Xc, CultureInfo.InvariantCulture);
    valY = double.Parse(Yc, CultureInfo.InvariantCulture);
    Xentrada.Add(valX);
    Ysalidas.Add(valY);
}
}
}

```

```

namespace Ejemplo;

internal class Perceptron {
    public List<Capa> Capas;

    //Crea las diversas capas
    public Perceptron(Random Azar, int TotalEntradas, int NeuronasCapa0,
        int NeuronasCapa1, int NeuronasCapa2) {
        Capas =
        [
            new Capa(Azar, NeuronasCapa0, TotalEntradas), //Crea la capa 0
            new Capa(Azar, NeuronasCapa1, NeuronasCapa0), //Crea la capa 1
            new Capa(Azar, NeuronasCapa2, NeuronasCapa1), //Crea la capa 2
        ];
    }
}

```

```

}

//Dada las entradas al perceptrón, se calcula
//la salida de cada capa. Con eso se sabrá que salidas
//se obtienen con los pesos y umbrales actuales. Esas
//salidas son requeridas para el algoritmo de entrenamiento.
public void CalculaSalida(List<double> Entradas) {
    Capas[0].CalculaCapa(Entradas);
    Capas[1].CalculaCapa(Capas[0].Salidas);
    Capas[2].CalculaCapa(Capas[1].Salidas);
}

//Con las salidas previamente calculadas con
//unas determinadas entradas se ejecuta el algoritmo
//de entrenamiento "Backpropagation"
public void Entrena(List<double> Entradas,
    List<double> SalidaEsperada) {
    int NeuronasCapa0 = Capas[0].Neuronas.Count;
    int NeuronasCapa1 = Capas[1].Neuronas.Count;
    int NeuronasCapa2 = Capas[2].Neuronas.Count;

    //Factor de aprendizaje
    double Alpha = 0.4;

    //=====
    //Procesa pesos capa 2
    //=====

    //Va de neurona en neurona de la capa 1
    for (int j = 0; j < NeuronasCapa1; j++)

        //Va de neurona en neurona de la capa de salida (capa 2)
        for (int i = 0; i < NeuronasCapa2; i++) {

            //Salida de la neurona de la capa de salida

```

```

    double Yi = Capas[2].Salidas[i];

    //Salida esperada
    double Si = SalidaEsperada[i];

    //Salida de la capa 1
    double alj = Capas[1].Salidas[j];

    //La fórmula del error
    double dE2 = alj * (Yi - Si) * Yi * (1 - Yi);

    //Ajusta el nuevo peso
    double Nuevo = Capas[2].Neuronas[i].Pesos[j] - Alpha * dE2;
    Capas[2].Neuronas[i].NuevosPesos[j] = Nuevo;
}

//=====
//Procesa pesos capa 1
//=====

//Va de neurona en neurona de la capa 0
for (int j = 0; j < NeuronasCapa0; j++)

    //Va de neurona en neurona de la capa 1
    for (int k = 0; k < NeuronasCapa1; k++) {
        double Acumula = 0;

        //Va de neurona en neurona de la capa 2
        for (int i = 0; i < NeuronasCapa2; i++) {

            //Salida de la capa 2
            double Yi = Capas[2].Salidas[i];

            //Salida esperada
            double Si = SalidaEsperada[i];
            double W2ki = Capas[2].Neuronas[i].Pesos[k];

```

```

        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi); //Sumatoria
    }
    double a0j = Capas[0].Salidas[j];
    double a1k = Capas[1].Salidas[k];
    double dE1 = a0j * a1k * (1 - a1k) * Acumula;
    double Nuevo = Capas[1].Neuronas[k].Pesos[j] - Alpha * dE1;
    Capas[1].Neuronas[k].NuevosPesos[j] = Nuevo;
}

//=====
//Procesa pesos capa 0
//=====

//Va de entrada en entrada
for (int j = 0; j < Entradas.Count; j++)

    //Va de neurona en neurona de la capa 0
    for (int k = 0; k < NeuronasCapa0; k++) {
        double Acumula = 0;

        //Va de neurona en neurona de la capa 1
        for (int p = 0; p < NeuronasCapa1; p++) {
            double InternoAcumula = 0;

            //Va de neurona en neurona de la capa 2
            for (int i = 0; i < NeuronasCapa2; i++) {
                double Yi = Capas[2].Salidas[i];
                double Si = SalidaEsperada[i]; //Salida esperada
                double W2pi = Capas[2].Neuronas[i].Pesos[p];

                //Sumatoria interna
                InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
            }
            double W1kp = Capas[1].Neuronas[p].Pesos[k];
            double alp = Capas[1].Salidas[p];

```



```

        //Sumatoria externa
        Acumula += Wlkp * alp * (1 - alp) * InternoAcumula;
    }
    double xj = Entradas[j];
    double a0k = Capas[0].Salidas[k];
    double dE0 = xj * a0k * (1 - a0k) * Acumula;
    double W0jk = Capas[0].Neuronas[k].Pesos[j];
    Capas[0].Neuronas[k].NuevosPesos[j] = W0jk - Alpha * dE0;
}

//=====
//Procesa umbrales capa 2
//=====

//Va de neurona en neurona de la capa de salida (capa 2)
for (int i = 0; i < NeuronasCapa2; i++) {

    //Salida de la neurona de la capa de salida
    double Yi = Capas[2].Salidas[i];

    //Salida esperada
    double Si = SalidaEsperada[i];
    double dE2 = (Yi - Si) * Yi * (1 - Yi);
    double Nuevo = Capas[2].Neuronas[i].Umbral - Alpha * dE2;
    Capas[2].Neuronas[i].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 1
//=====

//Va de neurona en neurona de la capa 1
for (int k = 0; k < NeuronasCapa1; k++) {
    double Acumula = 0;

    //Va de neurona en neurona de la capa 2

```

```

    for (int i = 0; i < NeuronasCapa2; i++) {

        //Salida de la capa 2
        double Yi = Capas[2].Salidas[i];
        double Si = SalidaEsperada[i];
        double W2ki = Capas[2].Neuronas[i].Pesos[k];
        Acumula += W2ki * (Yi - Si) * Yi * (1 - Yi);
    }
    double a1k = Capas[1].Salidas[k];
    double dE1 = a1k * (1 - a1k) * Acumula;
    double Nuevo = Capas[1].Neuronas[k].Umbral - Alpha * dE1;
    Capas[1].Neuronas[k].NuevoUmbral = Nuevo;
}

//=====
//Procesa umbrales capa 0
//=====

//Va de neurona en neurona de la capa 0
for (int k = 0; k < NeuronasCapa0; k++) {
    double Acumula = 0;

    //Va de neurona en neurona de la capa 1
    for (int p = 0; p < NeuronasCapa1; p++) {
        double InternoAcumula = 0;

        //Va de neurona en neurona de la capa 2
        for (int i = 0; i < NeuronasCapa2; i++) {
            double Yi = Capas[2].Salidas[i];
            double Si = SalidaEsperada[i];
            double W2pi = Capas[2].Neuronas[i].Pesos[p];
            InternoAcumula += W2pi * (Yi - Si) * Yi * (1 - Yi);
        }
        double W1kp = Capas[1].Neuronas[p].Pesos[k];
        double alp = Capas[1].Salidas[p];
        Acumula += W1kp * alp * (1 - alp) * InternoAcumula;
    }
}

```

```

    }
    double a0k = Capas[0].Salidas[k];
    double dE0 = a0k * (1 - a0k) * Acumula;
    double Nuevo = Capas[0].Neuronas[k].Umbral - Alpha * dE0;
    Capas[0].Neuronas[k].NuevoUmbral = Nuevo;
}

//Actualiza los pesos
Capas[0].Actualiza();
Capas[1].Actualiza();
Capas[2].Actualiza();
}
}

```

```

namespace Ejemplo;

internal class Capa {
    //Las neuronas que tendrá la capa
    public List<Neurona> Neuronas;

    //Almacena las salidas de cada neurona
    public List<double> Salidas;

    public Capa(Random Azar, int TotalNeuronas, int TotalEntradas) {
        Neuronas = [];
        Salidas = [];

        //Genera las neuronas
        for (int Contador = 0; Contador < TotalNeuronas; Contador++) {
            Neuronas.Add(new Neurona(Azar, TotalEntradas));
            Salidas.Add(0);
        }
    }
}

```

```

//Calcula las salidas de cada neurona de la capa
public void CalculaCapa(List<double> Entradas) {
    for (int Contador = 0; Contador < Neuronas.Count; Contador++)
        Salidas[Contador] = Neuronas[Contador].CalculaSalida(Entradas);
}

//Actualiza los pesos y umbrales de las neuronas
public void Actualiza() {
    for (int Contador = 0; Contador < Neuronas.Count; Contador++)
        Neuronas[Contador].Actualiza();
}
}

```

```

namespace Ejemplo;

internal class Neurona {
    //Los pesos para cada entrada
    public List<double> Pesos;

    //Nuevos pesos dados por el algoritmo de "backpropagation"
    public List<double> NuevosPesos;

    //El peso del umbral
    public double Umbral;

    //Nuevo umbral dado por el algoritmo de "backpropagation"
    public double NuevoUmbral;

    //Inicializa los pesos y umbral con un valor al azar
    public Neurona(Random Azar, int TotalEntradas) {
        Pesos = [];
        NuevosPesos = [];
    }
}

```

```

        for (int Contador = 0; Contador < TotalEntradas; Contador++) {
            Pesos.Add(Azar.NextDouble());
            NuevosPesos.Add(0);
        }
        Umbral = Azar.NextDouble();
        NuevoUmbral = 0;
    }

    //Calcula la salida de la neurona dependiendo de las entradas
    public double CalculaSalida(List<double> Entradas) {
        double Valor = 0;
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Valor += Entradas[Contador] * Pesos[Contador];
        Valor += Umbral;
        return 1 / (1 + Math.Exp(-Valor));
    }

    //Reemplaza viejos pesos por nuevos
    public void Actualiza() {
        for (int Contador = 0; Contador < Pesos.Count; Contador++)
            Pesos[Contador] = NuevosPesos[Contador];
        Umbral = NuevoUmbral;
    }
}

```

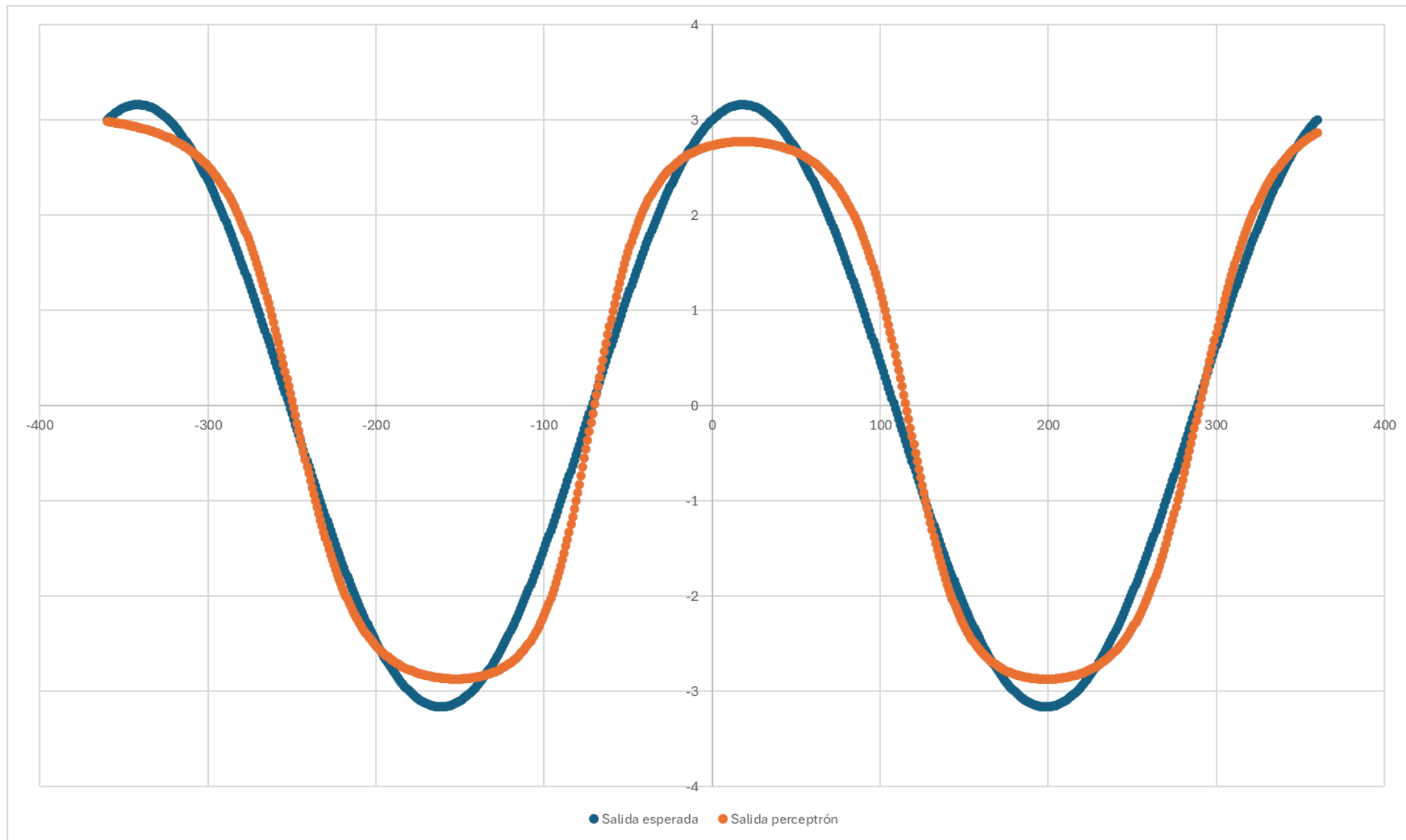
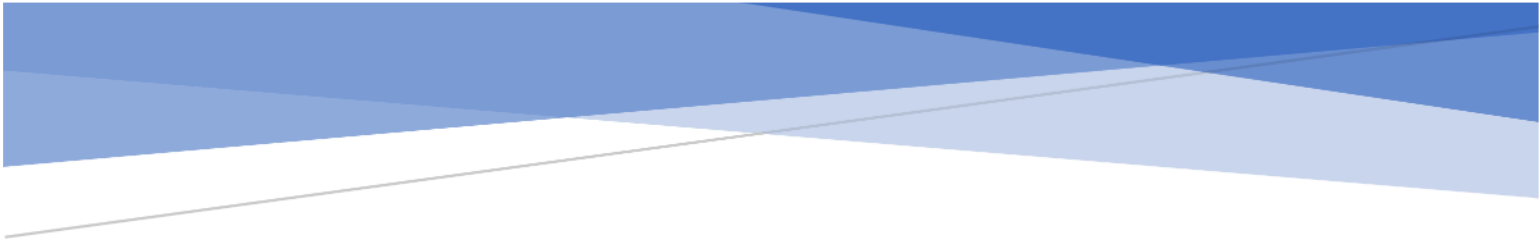


Ilustración 55: Resultado del entrenamiento teniendo en cuenta en NO sobre-ajustar



# C# Y .NET 10

## Parte 12. Gráficos 2D

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                            |    |
|------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                | 4  |
| Acerca del autor.....                                      | 6  |
| Licencia de este libro .....                               | 6  |
| Licencia del software .....                                | 6  |
| Marcas registradas .....                                   | 7  |
| En memoria de .....                                        | 8  |
| Primitivas Gráficas.....                                   | 9  |
| Línea.....                                                 | 15 |
| Arco .....                                                 | 16 |
| Rectángulo .....                                           | 17 |
| Rectángulo relleno.....                                    | 18 |
| Curva Bézier.....                                          | 19 |
| Varias líneas rectas .....                                 | 20 |
| Polígono.....                                              | 22 |
| Polígono relleno.....                                      | 24 |
| Curva cerrada .....                                        | 26 |
| Curva cerrada rellena .....                                | 28 |
| Curva que une varios puntos .....                          | 30 |
| Elipse .....                                               | 32 |
| Elipse rellena .....                                       | 33 |
| Letras.....                                                | 34 |
| Diagrama de pastel.....                                    | 36 |
| Diagrama de pastel relleno .....                           | 37 |
| Líneas horizontales .....                                  | 39 |
| Líneas verticales.....                                     | 41 |
| Líneas más juntas en el centro .....                       | 43 |
| Líneas generan rombo.....                                  | 44 |
| Líneas en esquina superior derecha .....                   | 46 |
| Líneas en esquina inferior derecha .....                   | 48 |
| Líneas en esquina inferior izquierda .....                 | 50 |
| Triángulos apuntan a la derecha.....                       | 52 |
| Triángulos apuntan a la izquierda .....                    | 54 |
| Triángulos apuntan a la izquierda alternando colores ..... | 56 |



|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Rectángulos concéntricos.....                                                    | 58 |
| Dibuja formas que semejan cuadrados a las que les falta el techo o la base ..... | 60 |
| Dibuja ángulos rectos.....                                                       | 62 |
| Líneas diagonales cruzadas .....                                                 | 64 |
| Celdas .....                                                                     | 66 |
| Líneas rectas simulan curvas .....                                               | 68 |
| Algoritmo de Bresenham para dibujar líneas .....                                 | 70 |
| Transformaciones en 2D .....                                                     | 72 |
| Traslado de figuras en un plano .....                                            | 72 |
| Giro de figuras en un plano .....                                                | 74 |
| Giro de figuras en un plano calculando el centroide .....                        | 76 |
| Gráfico matemático en 2D .....                                                   | 79 |
| Gráfico polar .....                                                              | 85 |
| Uso de Bitmap para control a nivel de pixel .....                                | 90 |
| Control total del píxel .....                                                    | 90 |
| Persistencia del dibujo.....                                                     | 90 |
| Separación de lógica y presentación .....                                        | 90 |
| Mayor eficiencia en renderizados complejos .....                                 | 90 |
| Uso de Bitmap y Graphics.....                                                    | 92 |
| Uso de LockBits para mayor velocidad .....                                       | 93 |
| LockBits dibujando un círculo .....                                              | 95 |

## Tabla de ilustraciones

|                                                                                                  |    |
|--------------------------------------------------------------------------------------------------|----|
| Ilustración 1: Inicia proyecto en Visual Studio 2026 .....                                       | 9  |
| Ilustración 2: Selecciona "Aplicación de Windows Forms" .....                                    | 10 |
| Ilustración 3: Pone nombre al proyecto, para este libro es "Graficos" (sin tilde) .....          | 11 |
| Ilustración 4: Selecciona .NET 10.0 .....                                                        | 12 |
| Ilustración 5: Una aplicación de escritorio típica .....                                         | 12 |
| Ilustración 6: Clic botón derecho sobre la ventana y selecciona "Propiedades" .....              | 13 |
| Ilustración 7: En la ventana de "Propiedades", selecciona "Eventos" .....                        | 13 |
| Ilustración 8: En "Eventos" se da doble clic al frente del evento "Paint" .....                  | 14 |
| Ilustración 9: Esta es el código que se genera automáticamente .....                             | 14 |
| Ilustración 10: Línea .....                                                                      | 15 |
| Ilustración 11: Arco .....                                                                       | 16 |
| Ilustración 12: Rectángulo .....                                                                 | 17 |
| Ilustración 13: Rectángulo relleno .....                                                         | 18 |
| Ilustración 14: Curva Bézier .....                                                               | 19 |
| Ilustración 15: Varias líneas rectas .....                                                       | 21 |
| Ilustración 16: Polígono .....                                                                   | 23 |
| Ilustración 17: Polígono relleno .....                                                           | 25 |
| Ilustración 18: Curva cerrada .....                                                              | 27 |
| Ilustración 19: Curva cerrada rellena .....                                                      | 29 |
| Ilustración 20: Curva que une varios puntos .....                                                | 31 |
| Ilustración 21: Elipse .....                                                                     | 32 |
| Ilustración 22: Elipse rellena .....                                                             | 33 |
| Ilustración 23: Letras .....                                                                     | 35 |
| Ilustración 24: Diagrama de pastel .....                                                         | 36 |
| Ilustración 25: Diagrama de pastel relleno .....                                                 | 38 |
| Ilustración 26: Líneas horizontales .....                                                        | 40 |
| Ilustración 27: Líneas verticales .....                                                          | 42 |
| Ilustración 28: Líneas más juntas en el centro .....                                             | 43 |
| Ilustración 29: Líneas generan rombo .....                                                       | 45 |
| Ilustración 30: Líneas en esquina superior derecha .....                                         | 47 |
| Ilustración 31: Líneas en esquina inferior derecha .....                                         | 49 |
| Ilustración 32: Líneas en esquina inferior izquierda .....                                       | 51 |
| Ilustración 33: Triángulos apuntan a la derecha .....                                            | 53 |
| Ilustración 34: Triángulos apuntan a la izquierda .....                                          | 55 |
| Ilustración 35: Triángulos apuntan a la izquierda alternando colores .....                       | 57 |
| Ilustración 36: Rectángulos concéntricos .....                                                   | 59 |
| Ilustración 37: Dibuja formas que semejan cuadrados a las que les falta el techo o la base ..... | 61 |
| Ilustración 38: Dibuja ángulos rectos .....                                                      | 63 |
| Ilustración 39: Líneas diagonales cruzadas .....                                                 | 65 |
| Ilustración 40: Celdas .....                                                                     | 67 |
| Ilustración 41: Líneas rectas simulan curvas .....                                               | 69 |
| Ilustración 42: Algoritmo de Bresenham para dibujar líneas .....                                 | 71 |
| Ilustración 43: Traslado de figuras en un plano .....                                            | 73 |
| Ilustración 44: Traslado de figuras en un plano mueveX = 70 .....                                | 73 |
| Ilustración 45: Giro de figuras en un plano .....                                                | 75 |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Ilustración 46: Giro de figuras en un plano calculando el centroide ..... | 78 |
| Ilustración 47: Gráfico matemático en 2D .....                            | 84 |
| Ilustración 48: Gráfico polar.....                                        | 89 |
| Ilustración 49: Línea dibujada a nivel de pixel.....                      | 91 |
| Ilustración 50: Uso de Bitmap y Graphics .....                            | 92 |
| Ilustración 51: Uso de LockBits .....                                     | 94 |
| Ilustración 52: Dibujando un círculo con LockBits .....                   | 96 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelif@hotmai.com](mailto:enginelif@hotmai.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

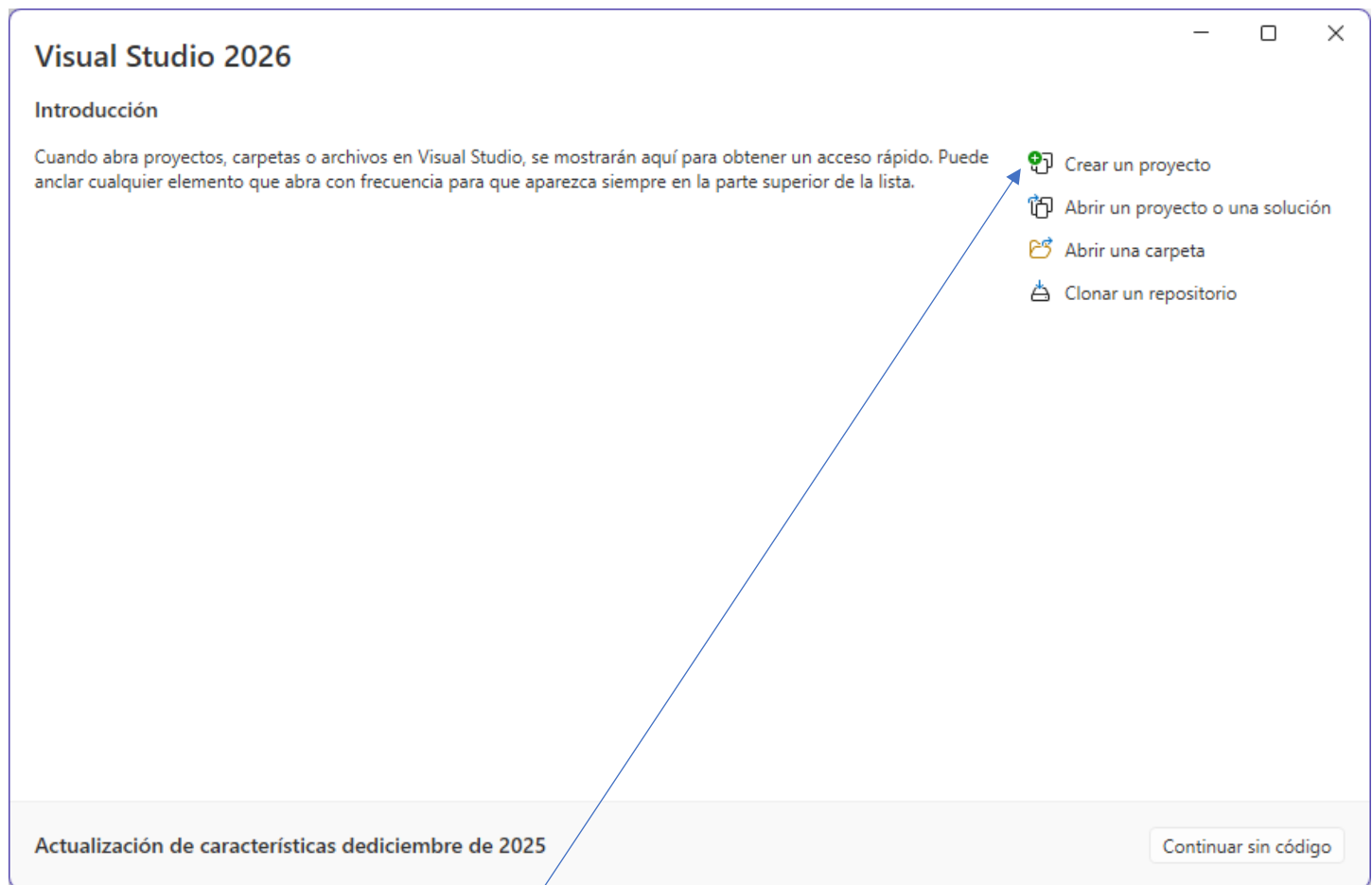
En memoria de

Sally



# Primitivas Gráficas

Para trabajar con gráficos, se debe crear un proyecto de escritorio gráfico



*Ilustración 1: Inicia proyecto en Visual Studio 2026*

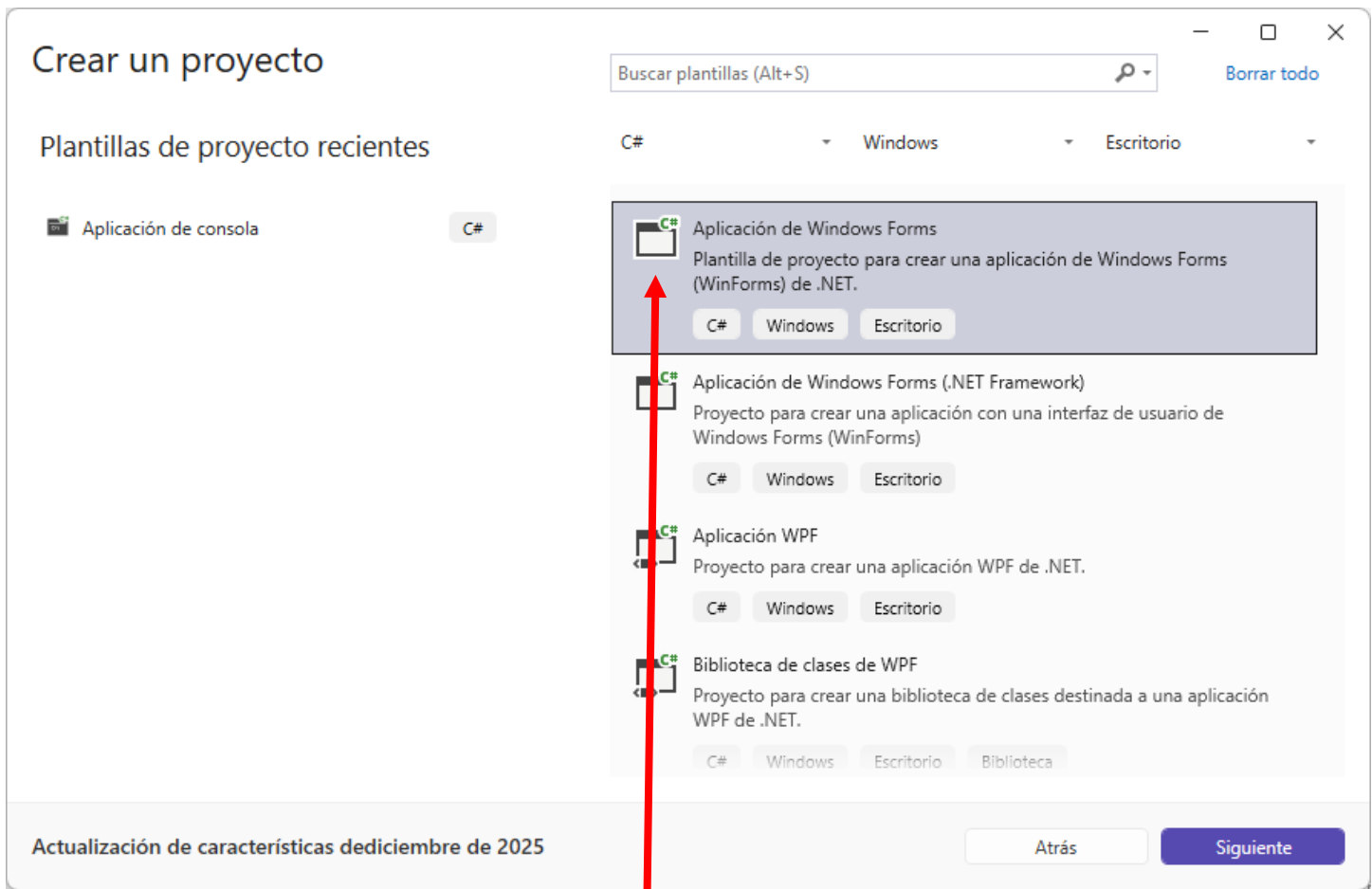


Ilustración 2: Selecciona "Aplicación de Windows Forms"

Una aplicación de tipo Windows Forms



Configure su nuevo proyecto

Aplicación de Windows Forms C# Windows Escritorio

Nombre del proyecto

Graficos

Ubicación

C:\Users\engin\source\repos

Nombre de la solución ⓘ

Graficos

☐ Colocar la solución y el proyecto en el mismo directorio

Proyecto se creará en "C:\Users\engin\source\repos\Graficos\Graficos\"

Actualización de características dediciembre de 2025

Atrás Siguiente

*Ilustración 3: Pone nombre al proyecto, para este libro es "Graficos" (sin tilde)*

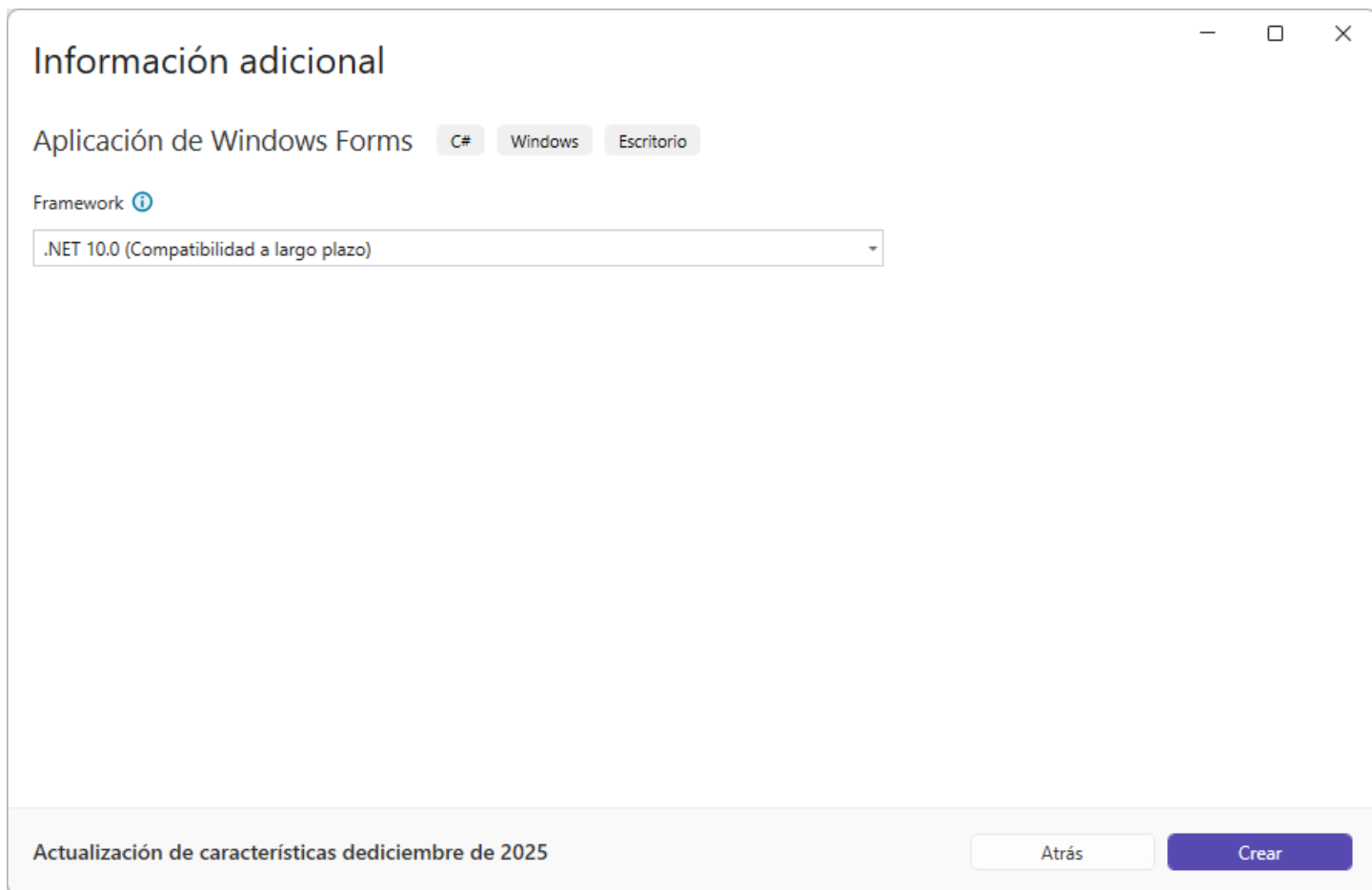


Ilustración 4: Selecciona .NET 10.0

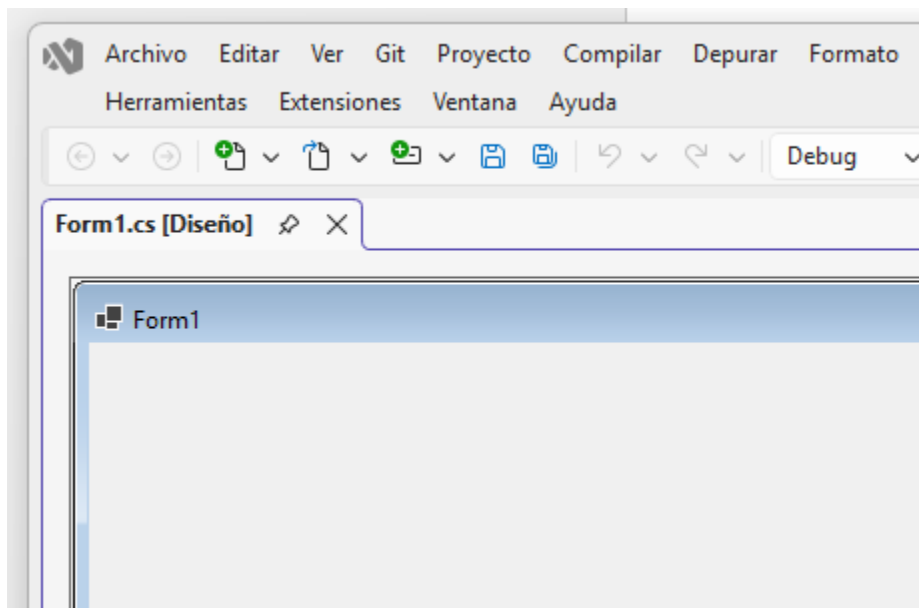


Ilustración 5: Una aplicación de escritorio típica

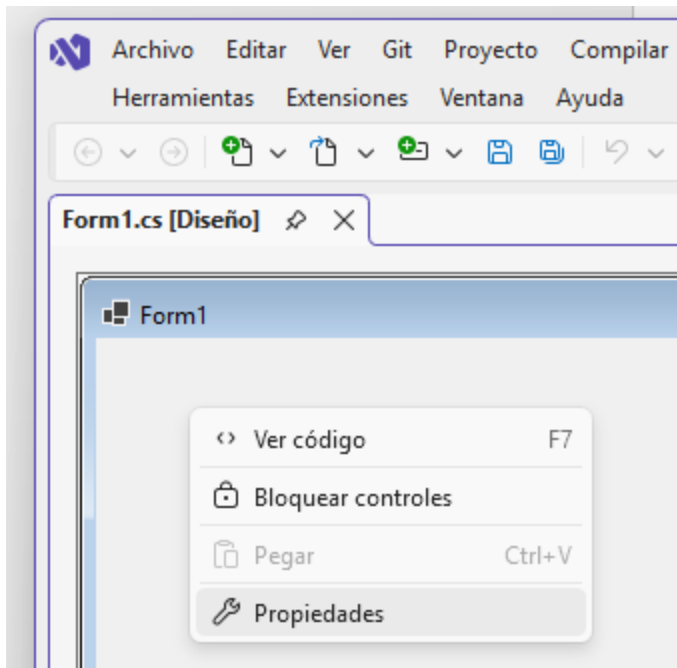


Ilustración 6: Clic botón derecho sobre la ventana y selecciona "Propiedades"

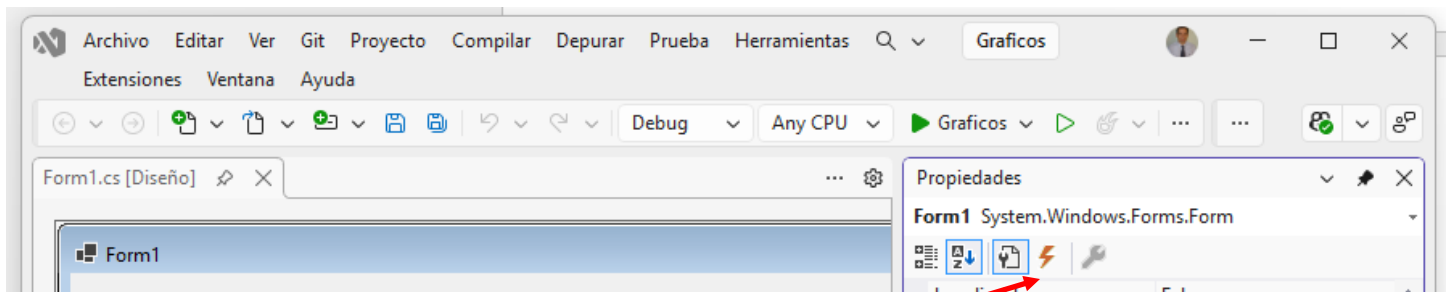


Ilustración 7: En la ventana de "Propiedades", selecciona "Eventos"

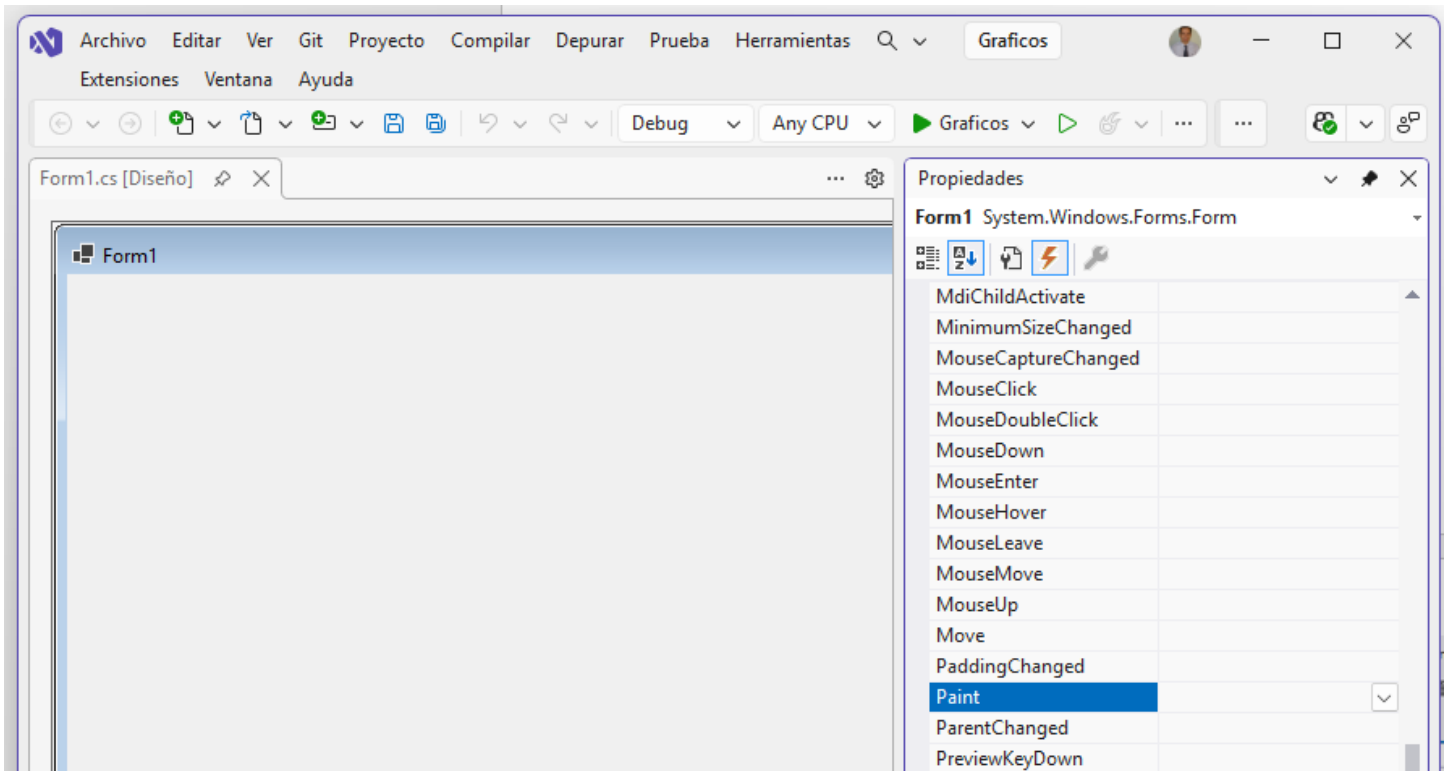


Ilustración 8: En "Eventos" se da doble clic al frente del evento "Paint"

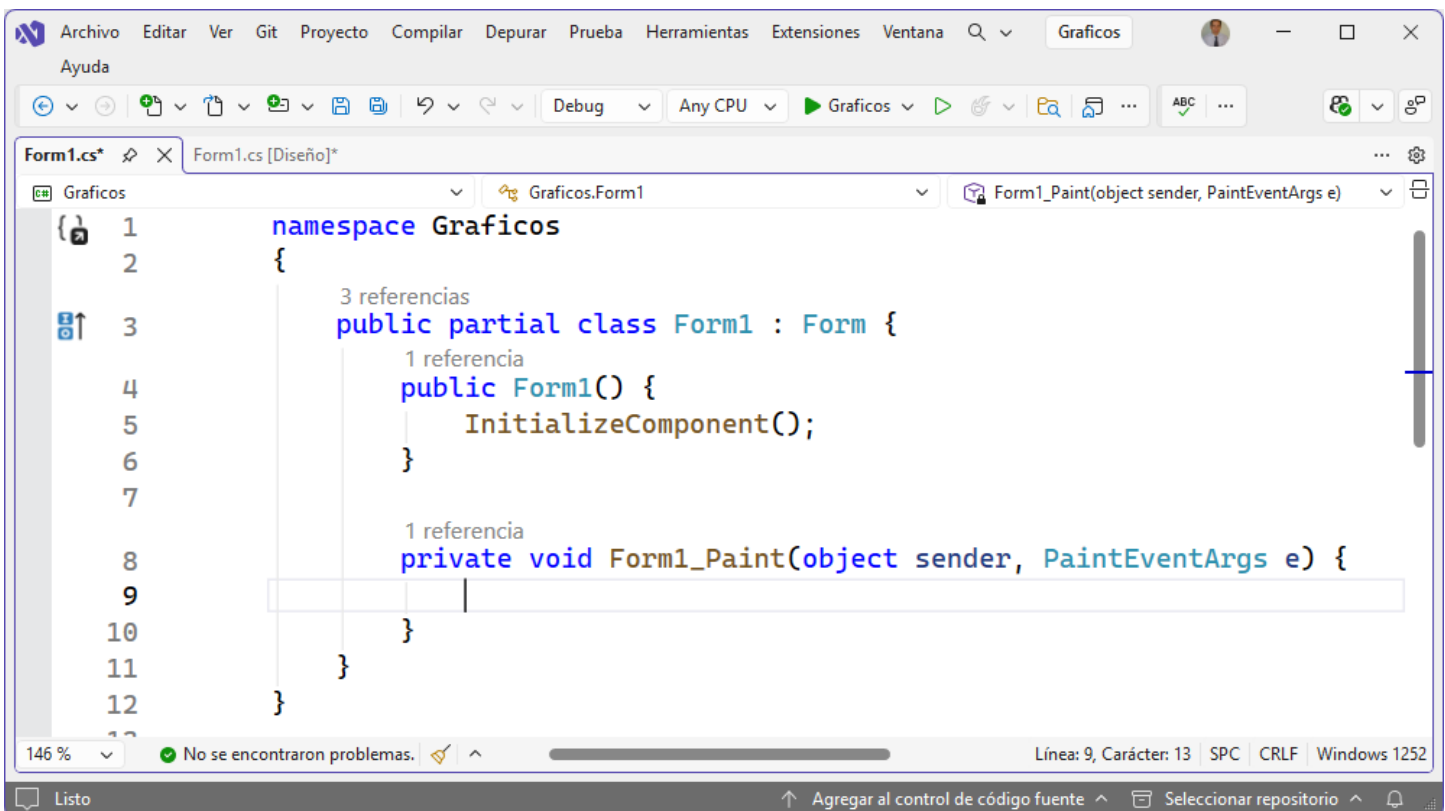


Ilustración 9: Esta es el código que se genera automáticamente

```
namespace Graficos
{
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new(Color.Blue, 2);

            //=====
            //Línea: Xini, Yini, Xfin, Yfin
            //=====
            lienzo.DrawLine(lapiz, 10, 10, 130, 140);
        }
    }
}
```



*Ilustración 10: Línea*

Cuando se usa el objeto "Graphics" proporcionado por el evento "Paint", se está dibujando directamente en la pantalla. Esto tiene algunas limitaciones:

- No se puede acceder ni modificar píxeles individuales fácilmente.
- No se puede conservar lo que se ha dibujado entre repintados (por ejemplo, al minimizar/restaurar la ventana).
- No se puede aplicar lógica de profundidad (Z-buffer) fácilmente, porque no se tiene control sobre el orden de los píxeles una vez dibujados.

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Lápiz con que dibuja. Color, grosor  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            //=====  
            //Arco: Xpos, Ypos, Ancho, Alto, Ángulo Inicial, Ángulo Final  
            //=====  
            lienzo.DrawArc(lapiz, 10, 90, 160, 170, 0, 270);  
        }  
    }  
}
```

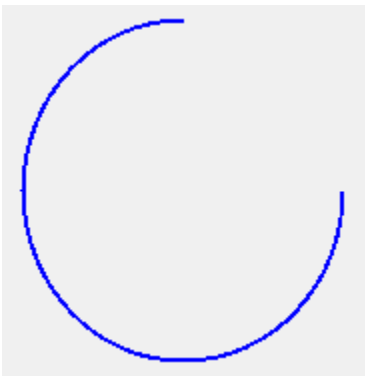


Ilustración 11: Arco

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Lápiz con que dibuja. Color, grosor  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            //=====  
            //Rectángulo: Xpos, Ypos, Ancho, Alto  
            //=====  
            lienzo.DrawRectangle(lapiz, 100, 100, 200, 150);  
        }  
    }  
}
```

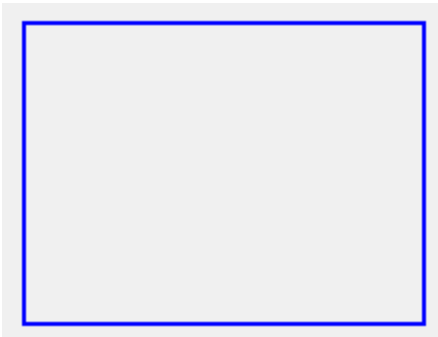


Ilustración 12: Rectángulo

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Con qué color se rellenan las figuras  
            SolidBrush Relleno = new SolidBrush(Color.Red);  
  
            //=====  
            //Rectángulo: Xpos, Ypos, Ancho, Alto  
            //=====  
            lienzo.FillRectangle(Relleno, 100, 100, 200, 150);  
        }  
    }  
}
```



Ilustración 13: Rectángulo relleno



```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Lápiz con que dibuja. Color, grosor  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            //=====  
            //Dibujar una Curva de Bézier  
            //=====  
            Point P0 = new Point(100, 180);  
            Point P1 = new Point(200, 10);  
            Point P2 = new Point(350, 50);  
            Point P3 = new Point(500, 180);  
            lienzo.DrawBezier(lapiz, P0, P1, P2, P3);  
        }  
    }  
}
```



Ilustración 14: Curva Bézier

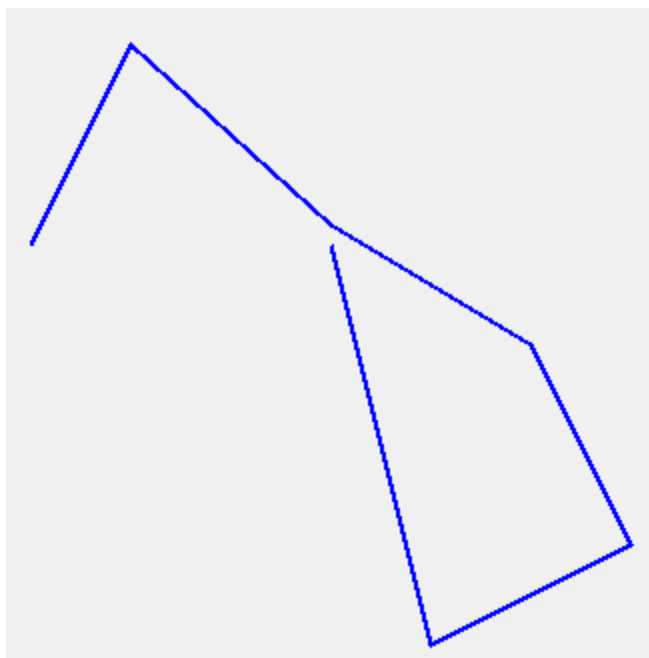
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new Pen(Color.Blue, 2);

            //Conjunto de puntos
            PointF p1 = new PointF(150.0F, 150.0F);
            PointF p2 = new PointF(200.0F, 50.0F);
            PointF p3 = new PointF(300.0F, 140.0F);
            PointF p4 = new PointF(400.0F, 200.0F);
            PointF p5 = new PointF(450.0F, 300.0F);
            PointF p6 = new PointF(350.0F, 350.0F);
            PointF p7 = new PointF(300.0F, 150.0F);
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };

            //Dibuja líneas rectas para unir los puntos
            lienzo.DrawLines(lapiz, Puntos);
        }
    }
}
```



*Ilustración 15: Varías líneas rectas*

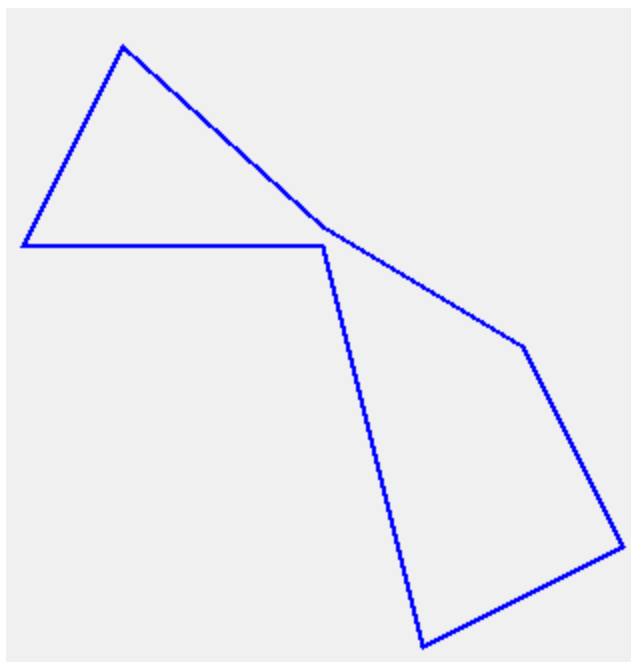
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new Pen(Color.Blue, 2);

            //Conjunto de puntos
            PointF p1 = new PointF(150.0F, 150.0F);
            PointF p2 = new PointF(200.0F, 50.0F);
            PointF p3 = new PointF(300.0F, 140.0F);
            PointF p4 = new PointF(400.0F, 200.0F);
            PointF p5 = new PointF(450.0F, 300.0F);
            PointF p6 = new PointF(350.0F, 350.0F);
            PointF p7 = new PointF(300.0F, 150.0F);
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };

            //Dibuja el polígono
            lienzo.DrawPolygon(lapiz, Puntos);
        }
    }
}
```



*Ilustración 16: Polígono*

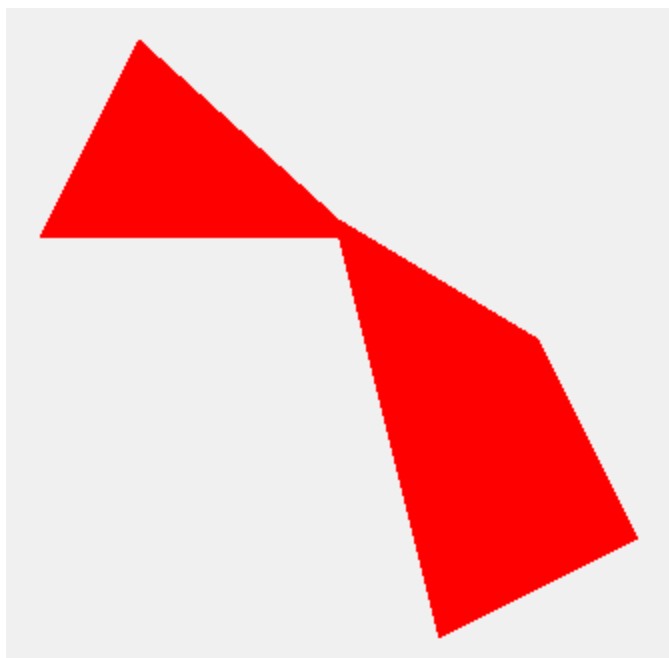
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Con qué color se rellenan las figuras
            SolidBrush Relleno = new SolidBrush(Color.Red);

            //Conjunto de puntos
            PointF p1 = new PointF(150.0F, 150.0F);
            PointF p2 = new PointF(200.0F, 50.0F);
            PointF p3 = new PointF(300.0F, 140.0F);
            PointF p4 = new PointF(400.0F, 200.0F);
            PointF p5 = new PointF(450.0F, 300.0F);
            PointF p6 = new PointF(350.0F, 350.0F);
            PointF p7 = new PointF(300.0F, 150.0F);
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };

            //Dibuja el polígono
            lienzo.FillPolygon(Relleno, Puntos);
        }
    }
}
```



*Ilustración 17: Polígono relleno*

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

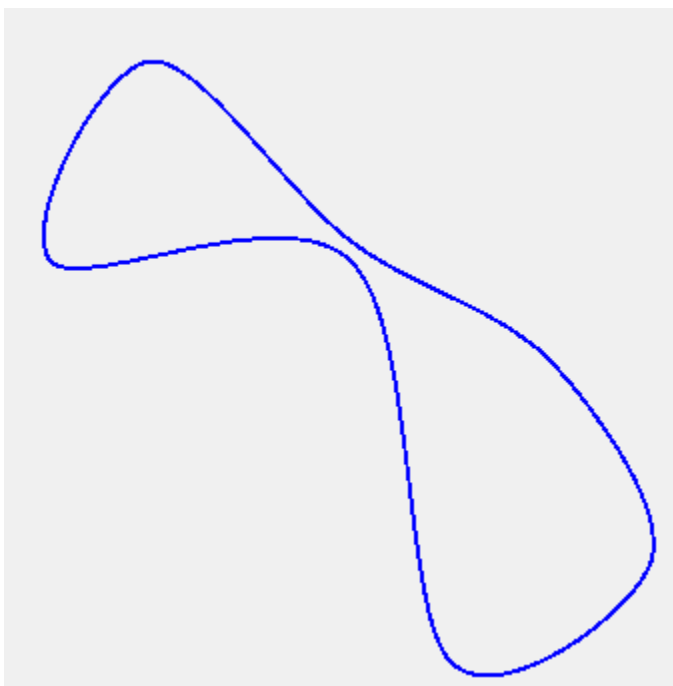
        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new Pen(Color.Blue, 2);

            //Conjunto de Puntos
            PointF p1 = new PointF(150.0F, 150.0F);
            PointF p2 = new PointF(200.0F, 50.0F);
            PointF p3 = new PointF(300.0F, 140.0F);
            PointF p4 = new PointF(400.0F, 200.0F);
            PointF p5 = new PointF(450.0F, 300.0F);
            PointF p6 = new PointF(350.0F, 350.0F);
            PointF p7 = new PointF(300.0F, 150.0F);
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };

            //Dibuja una curva cerrada que une esos puntos
            lienzo.DrawClosedCurve(lapiz, Puntos);
        }
    }
}
```





*Ilustración 18: Curva cerrada*

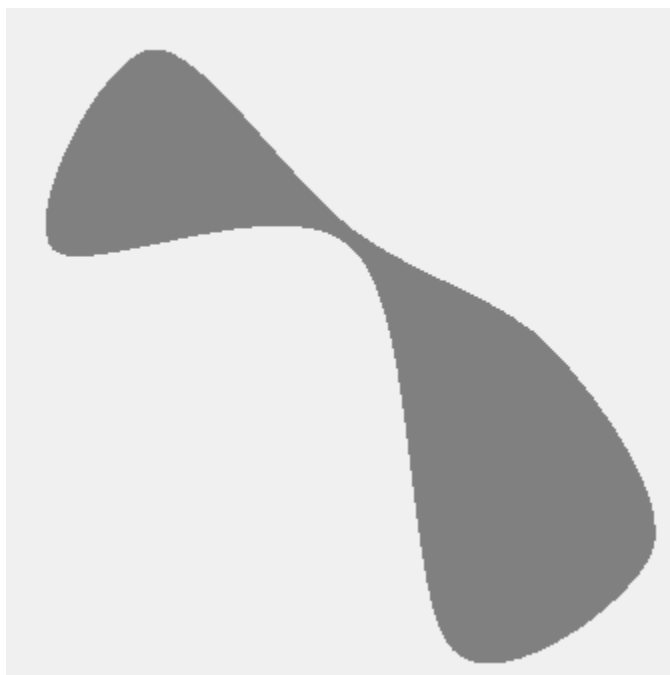
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Relleno
            SolidBrush Relleno = new SolidBrush(Color.Gray);

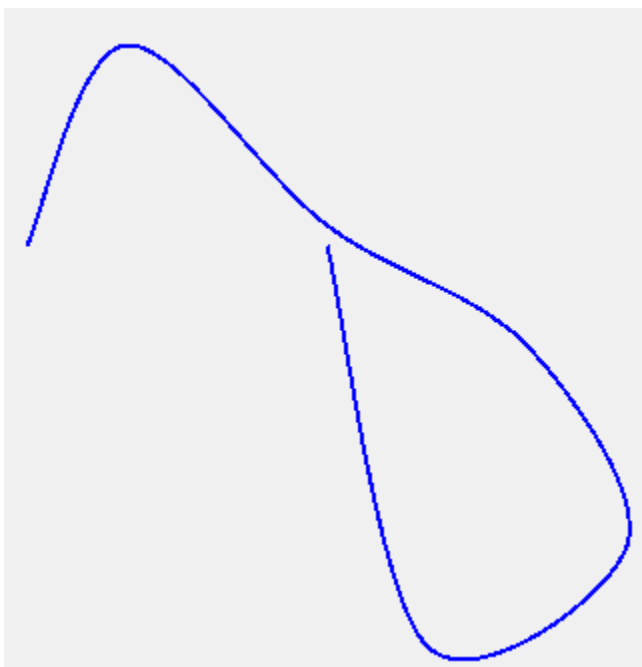
            //Conjunto de Puntos
            PointF p1 = new PointF(150.0F, 150.0F);
            PointF p2 = new PointF(200.0F, 50.0F);
            PointF p3 = new PointF(300.0F, 140.0F);
            PointF p4 = new PointF(400.0F, 200.0F);
            PointF p5 = new PointF(450.0F, 300.0F);
            PointF p6 = new PointF(350.0F, 350.0F);
            PointF p7 = new PointF(300.0F, 150.0F);
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };

            //Dibuja una curva cerrada rellena que une esos puntos
            lienzo.FillClosedCurve(Relleno, Puntos);
        }
    }
}
```



*Ilustración 19: Curva cerrada rellena*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Lápiz con que dibuja. Color, grosor  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            //Conjunto de Puntos  
            PointF p1 = new PointF(150.0F, 150.0F);  
            PointF p2 = new PointF(200.0F, 50.0F);  
            PointF p3 = new PointF(300.0F, 140.0F);  
            PointF p4 = new PointF(400.0F, 200.0F);  
            PointF p5 = new PointF(450.0F, 300.0F);  
            PointF p6 = new PointF(350.0F, 350.0F);  
            PointF p7 = new PointF(300.0F, 150.0F);  
            PointF[] Puntos = { p1, p2, p3, p4, p5, p6, p7 };  
  
            //Dibuja la curva que une esos puntos  
            lienzo.DrawCurve(lapiz, Puntos);  
        }  
    }  
}
```



*Ilustración 20: Curva que une varios puntos*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Lápiz con que dibuja. Color, grosor  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            //=====  
            //Ellipse: Xpos, Ypos, ancho, alto  
            //=====  
            lienzo.DrawEllipse(lapiz, 200, 30, 250, 90);  
        }  
    }  
}
```



Ilustración 21: Elipse

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo donde va a hacer el gráfico  
            Graphics lienzo = e.Graphics;  
  
            //Relleno  
            SolidBrush Relleno = new SolidBrush(Color.Chocolate);  
  
            //=====  
            //Ellipse: Xpos, Ypos, ancho, alto  
            //=====  
            lienzo.FillEllipse(Relleno, 200, 30, 250, 90);  
        }  
    }  
}
```



Ilustración 22: Elipse rellena

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new Pen(Color.Blue, 2);

            //=====
            //Letras
            //=====
            string Cadena = "Esta es una prueba";

            //Fuente y la brocha con que se pinta.
            Font Fuente = new Font("Tahoma", 16);
            SolidBrush Brocha = new SolidBrush(Color.Black);

            //Punto arriba a la izquierda para pintar la cadena
            PointF PuntoCadena = new PointF(100.0F, 140.0F);

            //Formato para dibujar
            StringFormat Formato = new StringFormat();
            FormatoDibuja.FormatFlags = StringFormatFlags.DirectionVertical;

            //Dibuja la cadena
            lienzo.DrawString(Cadena, Fuente, Brocha, PuntoCadena, Formato);
        }
    }
}
```



Esta es una prueba

*Ilustración 23: Letras*

```

namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Lápiz con que dibuja. Color, grosor
            Pen lapiz = new Pen(Color.Blue, 2);

            //=====
            //Dibuja un pastel

            //Crea un rectángulo para la elipse que dibujará el pastel
            Rectangle rectangulo = new Rectangle(10, 10, 400, 400);

            //Pastel: rectángulo que contiene:
            //ángulo inicial, ángulo de apertura
            lienzo.DrawPie(lapiz, rectangulo, 0F, 45F);
            lienzo.DrawPie(lapiz, rectangulo, 90F, 45F);
            lienzo.DrawPie(lapiz, rectangulo, 150F, 45F);
        }
    }
}

```

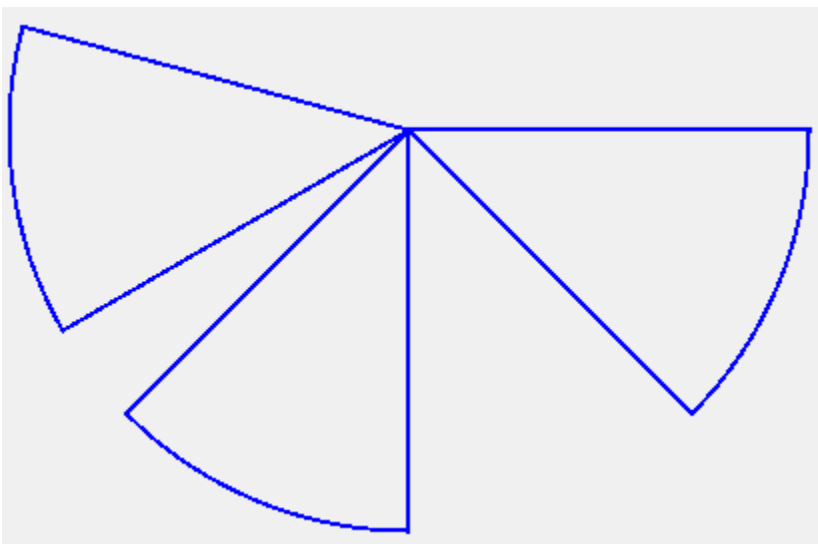


Ilustración 24: Diagrama de pastel

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

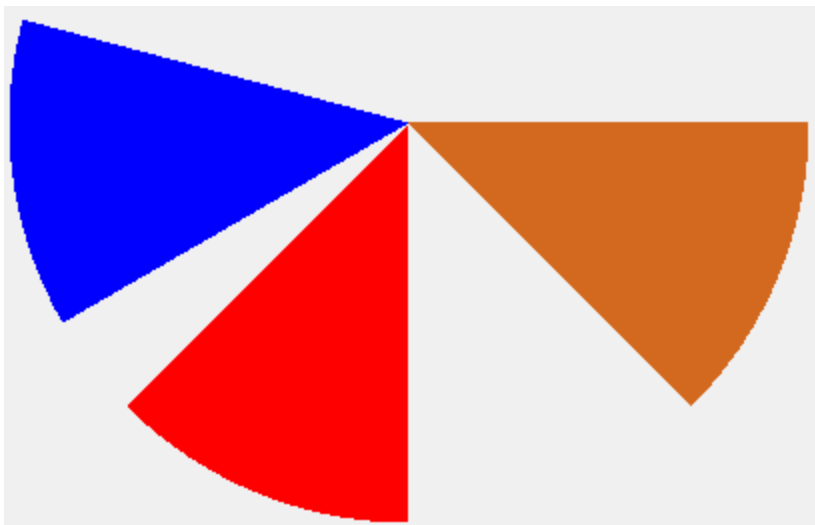
        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo donde va a hacer el gráfico
            Graphics lienzo = e.Graphics;

            //Rellenos
            SolidBrush Relleno1 = new SolidBrush(Color.Chocolate);
            SolidBrush Relleno2 = new SolidBrush(Color.Red);
            SolidBrush Relleno3 = new SolidBrush(Color.Blue);

            //=====
            //Dibuja un pastel
            //=====

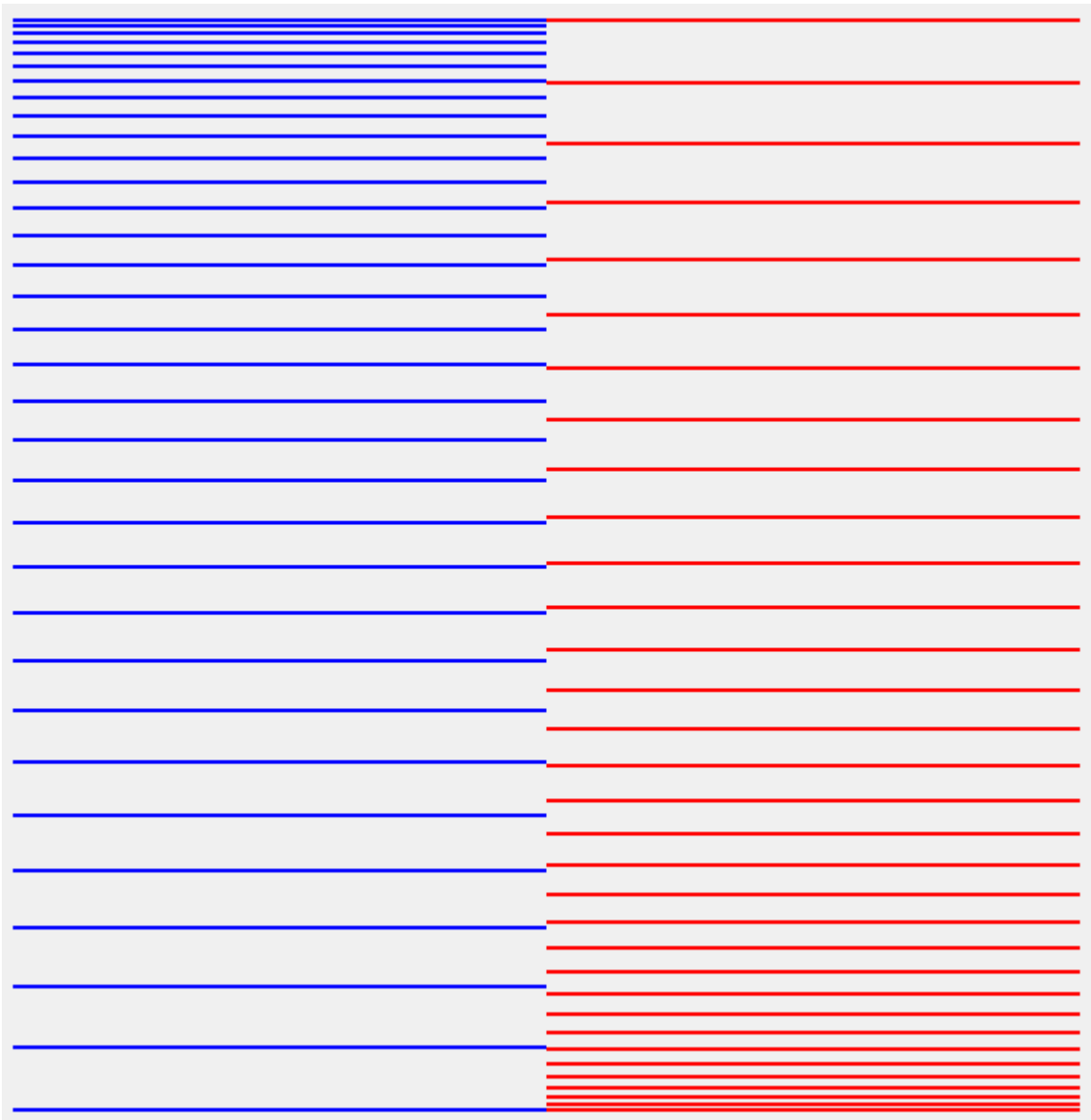
            //Crea un rectángulo para la elipse que dibujará el pastel
            Rectangle rectangulo = new Rectangle(10, 10, 400, 400);

            //Pastel: rectángulo que contiene:
            //ángulo inicial, ángulo de apertura
            lienzo.FillPie(Relleno1, rectangulo, 0F, 45F);
            lienzo.FillPie(Relleno2, rectangulo, 90F, 45F);
            lienzo.FillPie(Relleno3, rectangulo, 150F, 45F);
        }
    }
}
```



*Ilustración 25: Diagrama de pastel relleno*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            //Lienzo  
            Graphics grafico = e.Graphics;  
  
            //Primer gráfico  
            Pen lapiz = new Pen(Color.Blue, 2);  
            int contador = 10;  
            int incremento = 2;  
  
            do {  
                grafico.DrawLine(lapiz, 10, contador, 300, contador);  
                incremento++;  
                //Incrementa el espacio entre línea y línea  
                contador += incremento;  
            } while (contador <= 602);  
            contador -= incremento;  
  
            //Segundo gráfico  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            incremento = 2;  
            do {  
                grafico.DrawLine(lapiz2, 300, contador, 590, contador);  
                incremento++;  
                contador -= incremento;  
            } while (contador >= 10);  
        }  
    }  
}
```



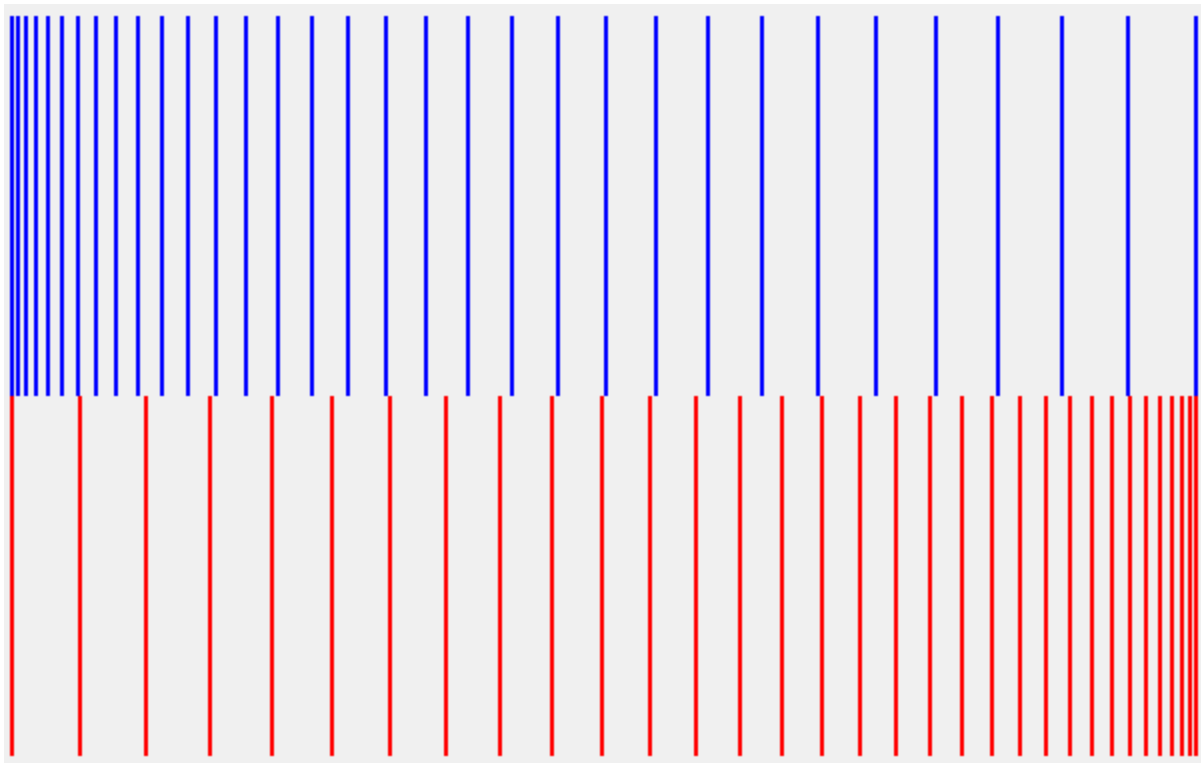
*Ilustración 26: Líneas horizontales*

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Lienzo
            Graphics grafico = e.Graphics;
            Pen lapiz = new Pen(Color.Blue, 2);
            int contador = 10;
            int incremento = 2;

            //Primer gráfico
            do {
                grafico.DrawLine(lapiz, contador, 10, contador, 200);
                incremento++;
                //Incrementa el espacio entre línea y línea
                contador += incremento;
            }
            while (contador <= 602);
            contador -= incremento;

            //Segundo gráfico
            Pen lapiz2 = new Pen(Color.Red, 2);
            incremento = 2;
            do {
                grafico.DrawLine(lapiz2, contador, 200, contador, 380);
                incremento++;
                contador -= incremento;
            }
            while (contador >= 10);
        }
    }
}
```



*Ilustración 27: Líneas verticales*



```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics lienzo = e.Graphics;
            Pen lapiz = new Pen(Color.Blue, 2);
            Pen lapiz2 = new Pen(Color.Red, 2);
            int lim = 500;
            int cont = lim;
            int incremento = 2;

            do {
                int comun = lim - (cont - lim);
                lienzo.DrawLine(lapiz, cont, 10, cont, 200);
                lienzo.DrawLine(lapiz, comun, 10, comun, 200);
                lienzo.DrawLine(lapiz2, cont, 200, cont, 380);
                lienzo.DrawLine(lapiz2, comun, 200, comun, 380);
                incremento++;
                cont -= incremento;
            }
            while (cont >= 2 * lim);
        }
    }
}
```

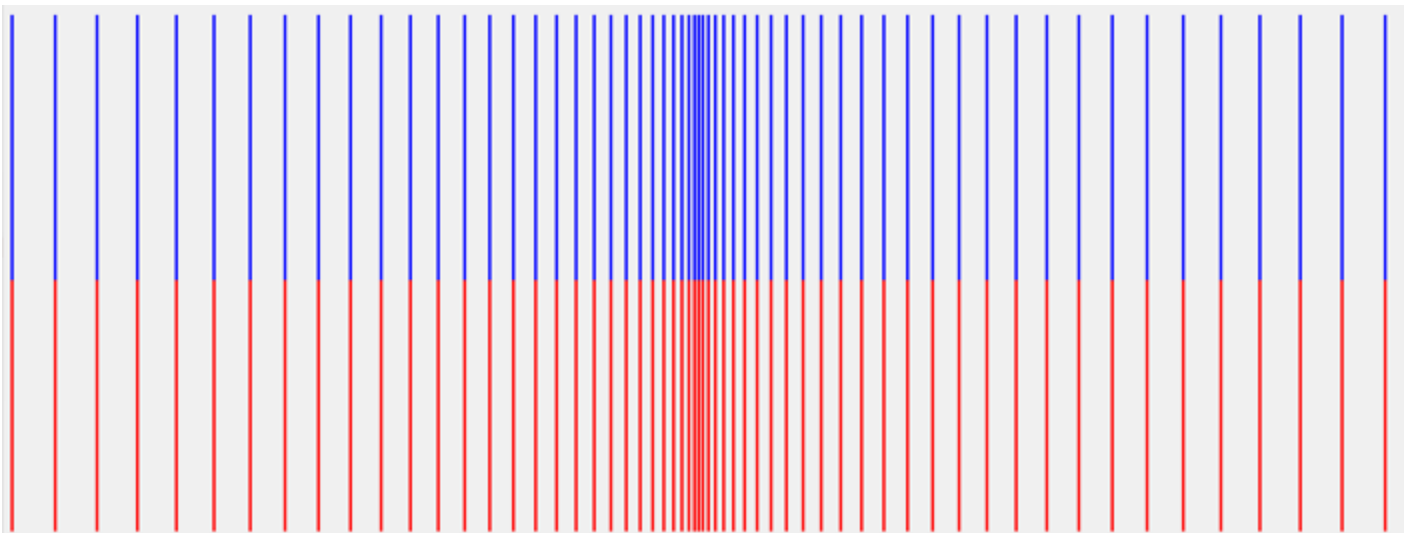


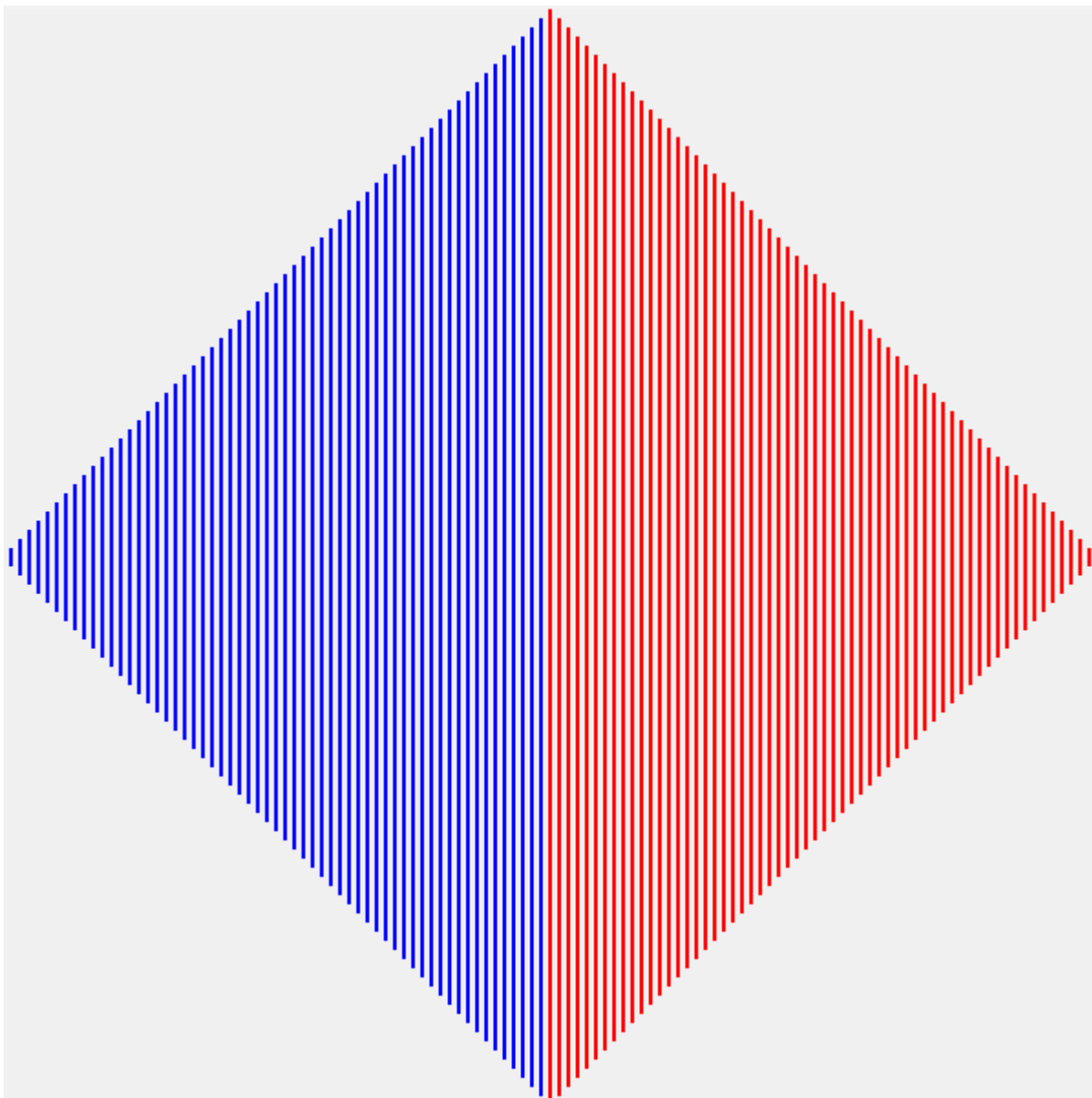
Ilustración 28: Líneas más juntas en el centro

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics grafico = e.Graphics;
            Pen lapiz = new Pen(Color.Blue, 2);
            Pen lapiz2 = new Pen(Color.Red, 2);
            int xval = 10;
            int yval = 5;

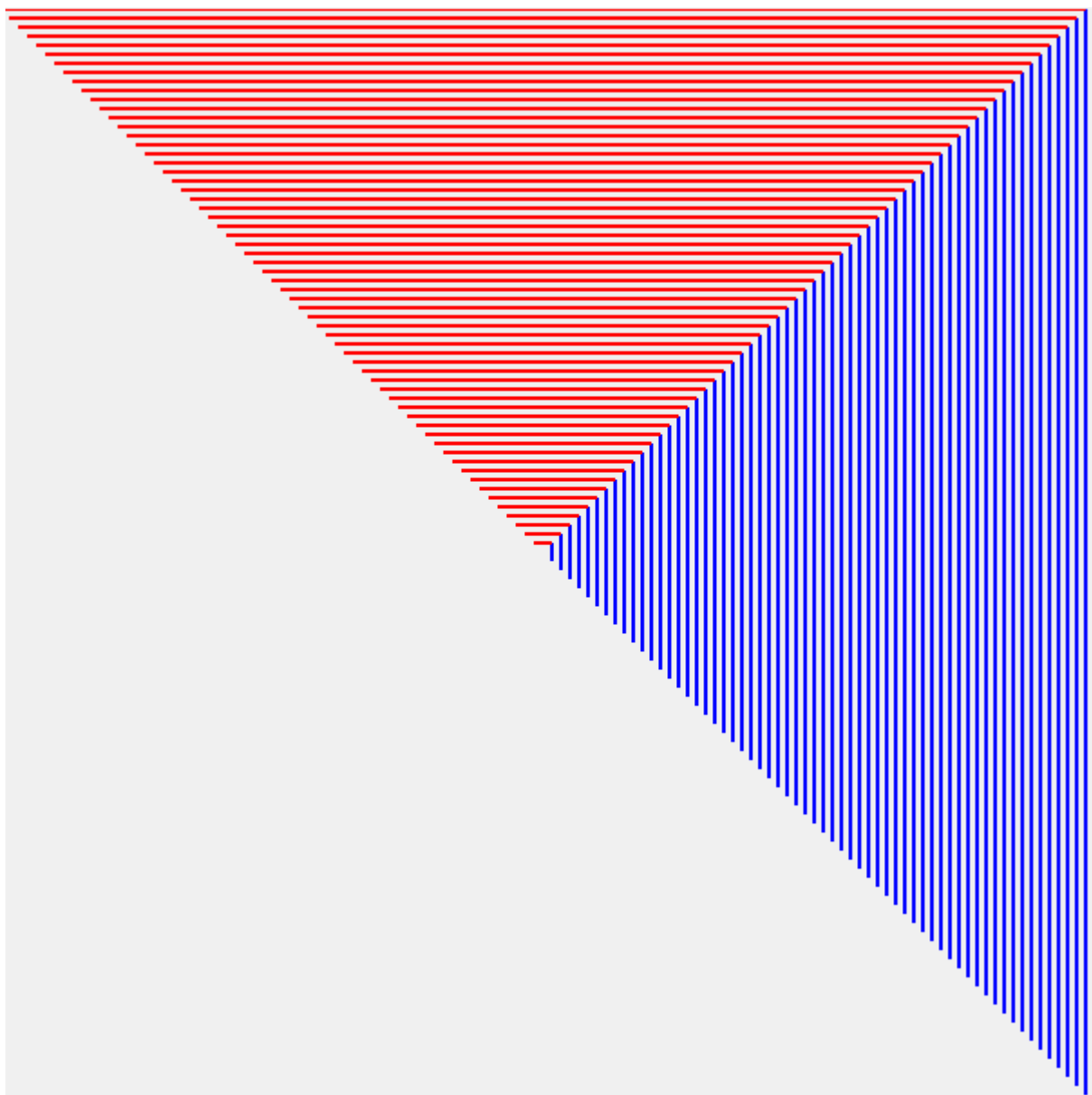
            do {
                grafico.DrawLine(lapiz, xval, 300 - yval, xval, 300 + yval);
                xval += 5;
                yval += 5;
            }
            while (yval < 300);

            do {
                grafico.DrawLine(lapiz2, xval, 300 - yval, xval, 300 + yval);
                xval += 5;
                yval -= 5;
            }
            while (yval > 0);
        }
    }
}
```



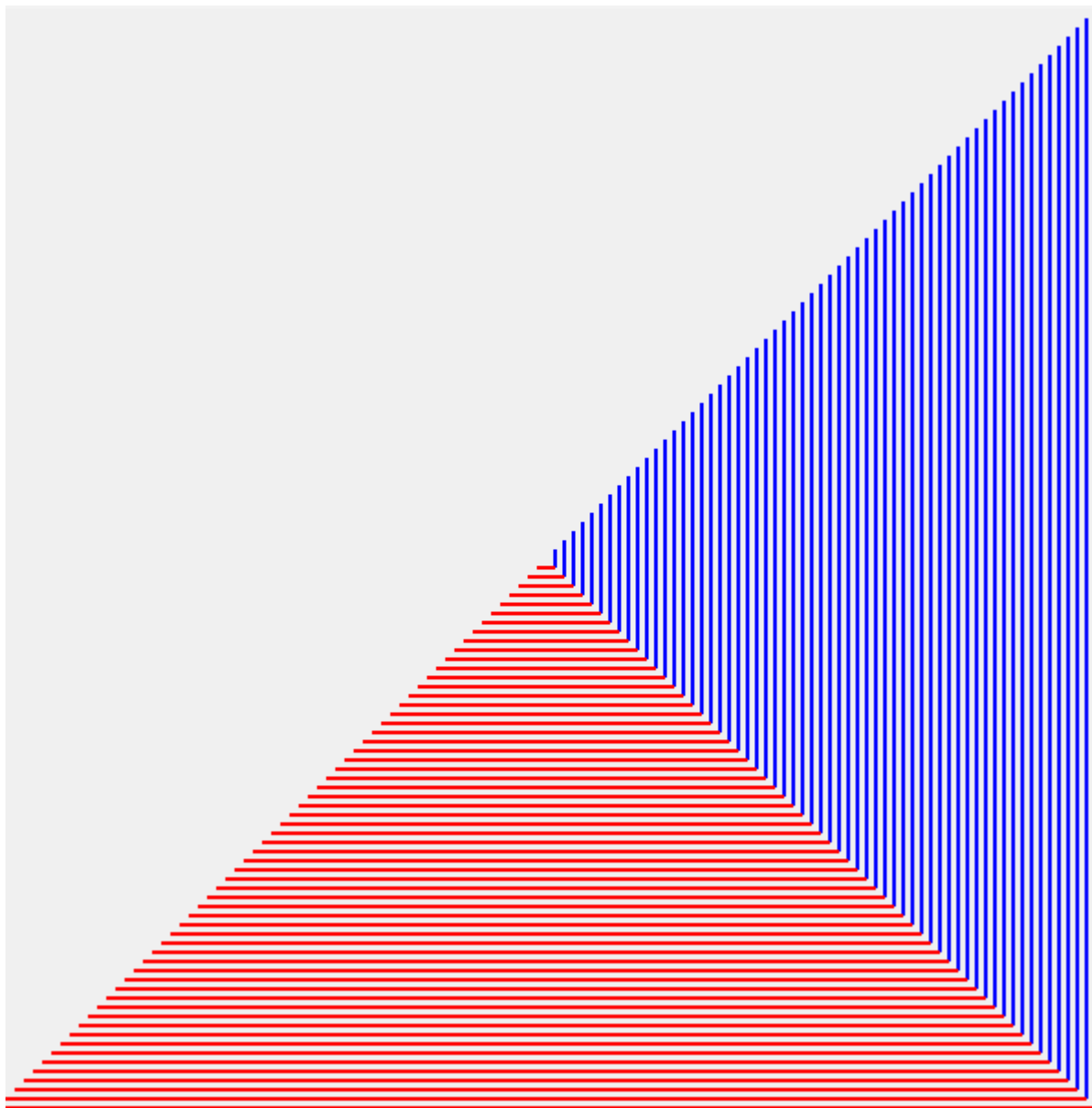
*Ilustración 29: Líneas generan rombo*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics grafico = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            int xval = 600;  
            int yval = 300;  
  
            do {  
                grafico.DrawLine(lapiz, xval, 300 - yval, xval, 300 + yval);  
                xval -= 5;  
                yval -= 5;  
            }  
            while (yval >= 0);  
  
            xval = 300;  
            yval = 0;  
            do {  
                grafico.DrawLine(lapiz2, 300 - xval, yval, 300 + xval, yval);  
                xval -= 5;  
                yval += 5;  
            }  
            while (yval <= 300);  
        }  
    }  
}
```



*Ilustración 30: Líneas en esquina superior derecha*

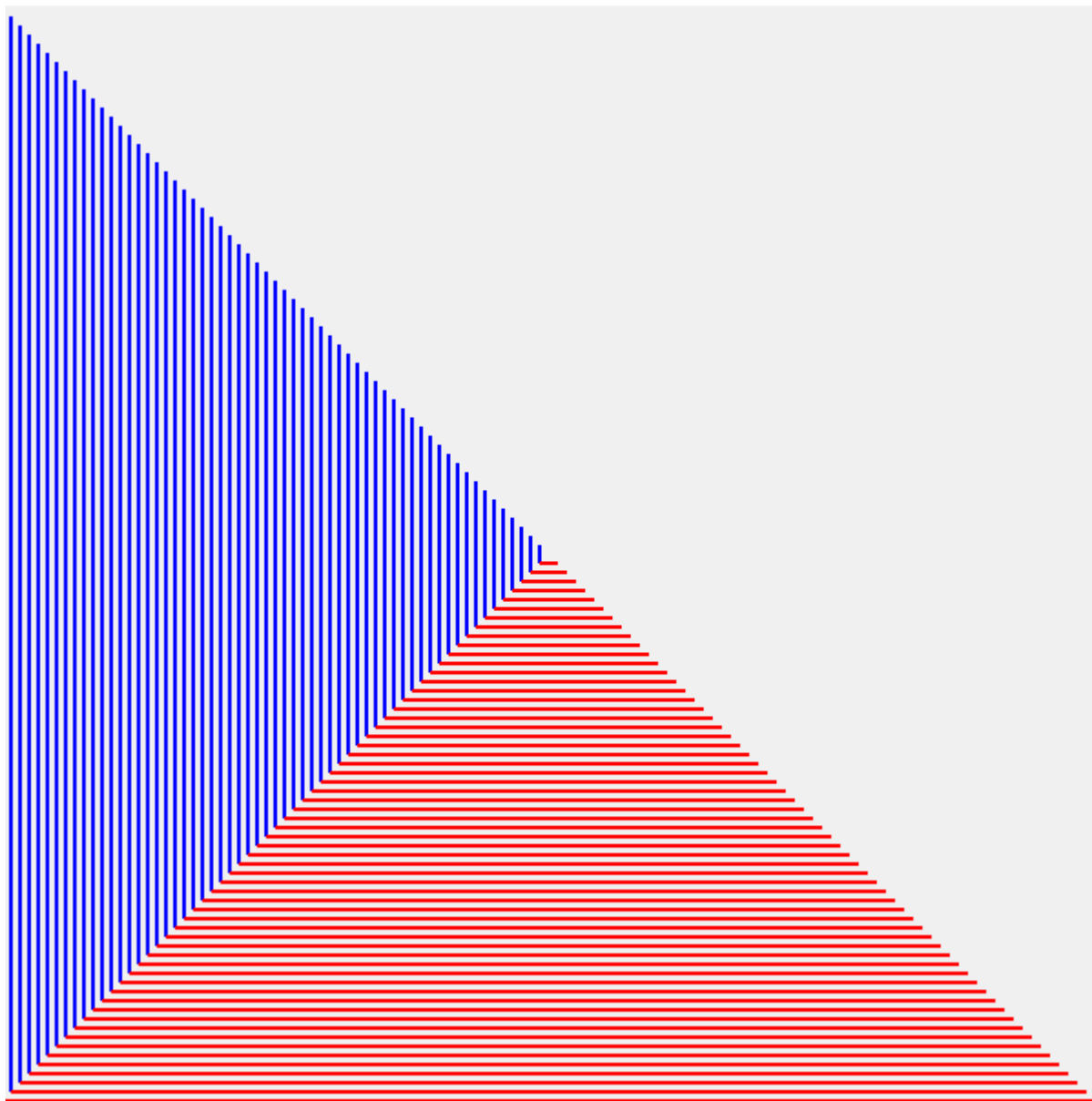
```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics grafico = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            int xval = 600;  
            int yval = 300;  
  
            do {  
                grafico.DrawLine(lapiz, xval, 300 - yval, xval, 300 + yval);  
                xval -= 5;  
                yval -= 5;  
            }  
            while (yval >= 0);  
  
            xval = 300;  
            yval = 600;  
            do {  
                grafico.DrawLine(lapiz2, 300 - xval, yval, 300 + xval, yval);  
                xval -= 5;  
                yval -= 5;  
            }  
            while (yval >= 300);  
        }  
    }  
}
```



*Ilustración 31: Líneas en esquina inferior derecha*

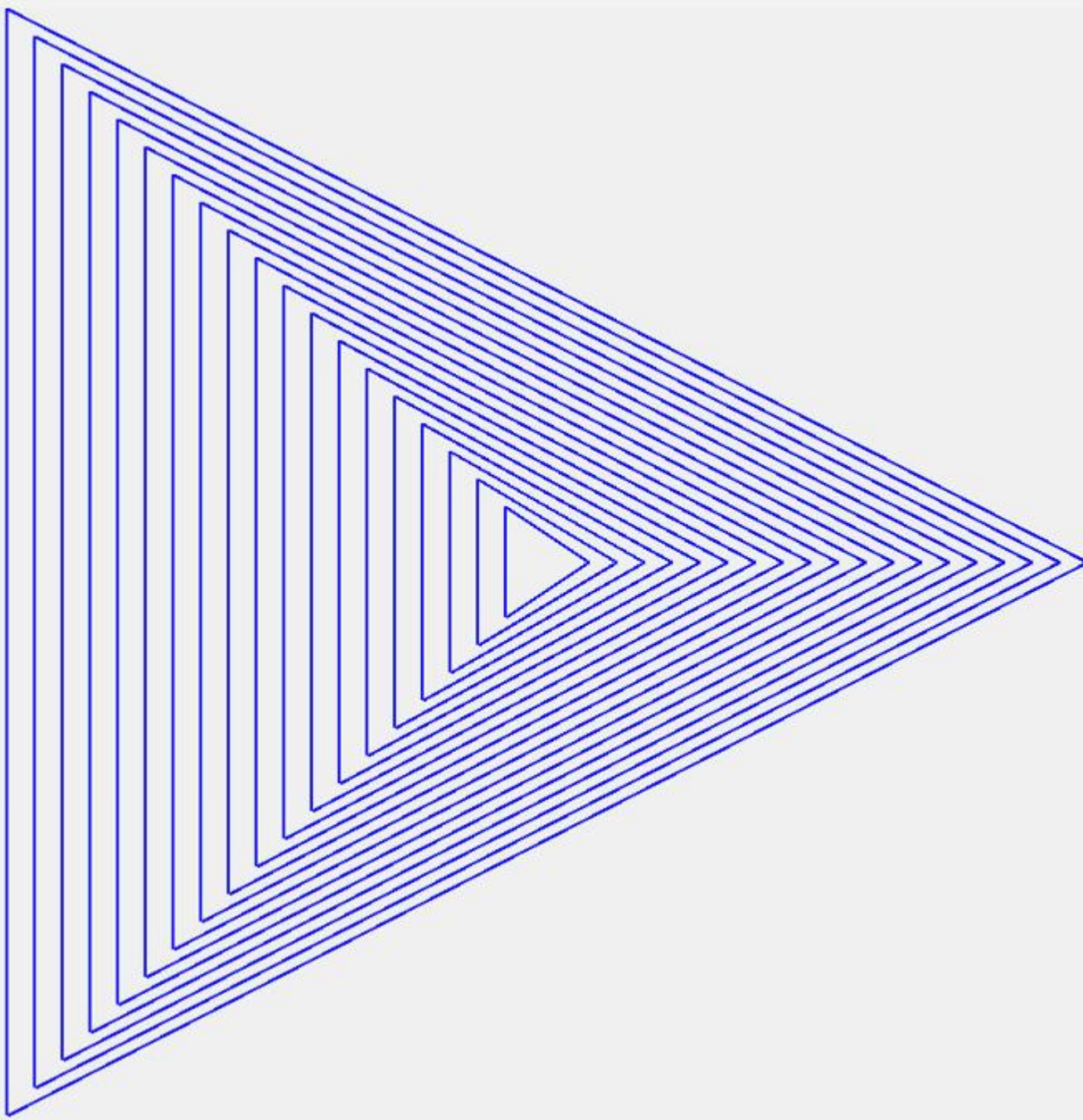
```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics grafico = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            int xval = 0;  
            int yval = 300;  
  
            do {  
                grafico.DrawLine(lapiz, xval, 300 - yval, xval, 300 + yval);  
                xval += 5;  
                yval -= 5;  
            }  
            while (yval >= 0);  
  
            xval = 300;  
            yval = 600;  
            do {  
                grafico.DrawLine(lapiz2, 300 - xval, yval, 300 + xval, yval);  
                xval -= 5;  
                yval -= 5;  
            }  
            while (yval >= 300);  
        }  
    }  
}
```





*Ilustración 32: Líneas en esquina inferior izquierda*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics grafico = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
  
            int pX1, pY1, pX2, pY2, pX3, pY3, constante;  
            pX1 = 420;  
            pY1 = 360;  
            pX2 = 480;  
            pY2 = 400;  
            pX3 = 420;  
            pY3 = 440;  
            constante = 20;  
            do {  
                grafico.DrawLine(lapiz, pX1, pY1, pX2, pY2);  
                grafico.DrawLine(lapiz, pX2, pY2, pX3, pY3);  
                grafico.DrawLine(lapiz, pX1, pY1, pX3, pY3);  
                pX1 -= constante;  
                pY1 -= constante;  
                pX2 += constante;  
                pX3 -= constante;  
                pY3 += constante;  
            }  
            while (pY1 >= 0);  
        }  
    }  
}
```

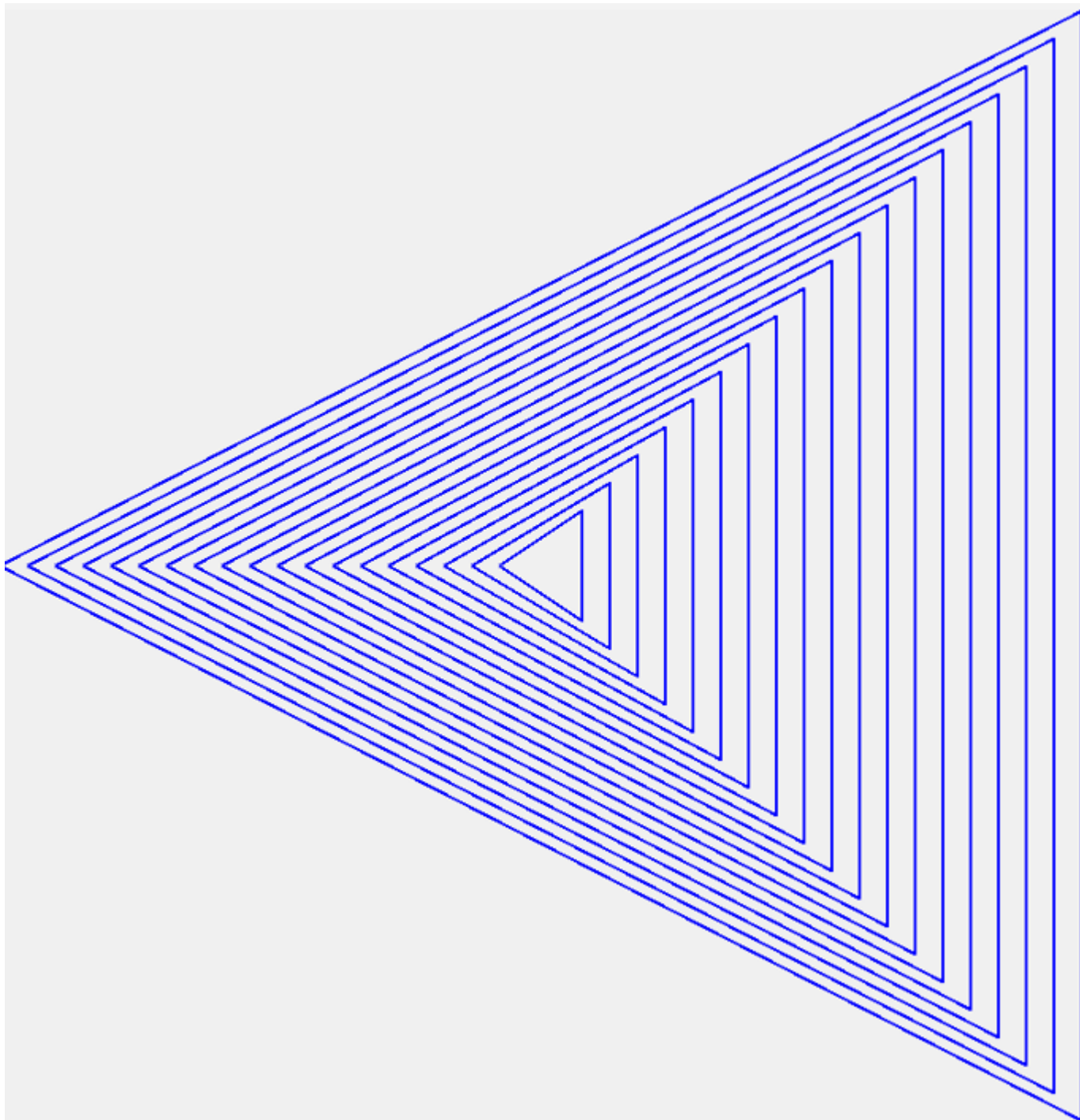


*Ilustración 33: Triángulos apuntan a la derecha*

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics grafico = e.Graphics;
            Pen lapiz = new Pen(Color.Blue, 2);

            int pX1, pY1, pX2, pY2, pX3, pY3, constante;
            pX1 = 420;
            pY1 = 360;
            pX2 = 360;
            pY2 = 400;
            pX3 = 420;
            pY3 = 440;
            constante = 20;
            do {
                grafico.DrawLine(lapiz, pX1, pY1, pX2, pY2);
                grafico.DrawLine(lapiz, pX2, pY2, pX3, pY3);
                grafico.DrawLine(lapiz, pX1, pY1, pX3, pY3);
                pX1 += constante;
                pY1 -= constante;
                pX2 -= constante;
                pX3 += constante;
                pY3 += constante;
            }
            while (pY1 >= 0);
        }
    }
}
```



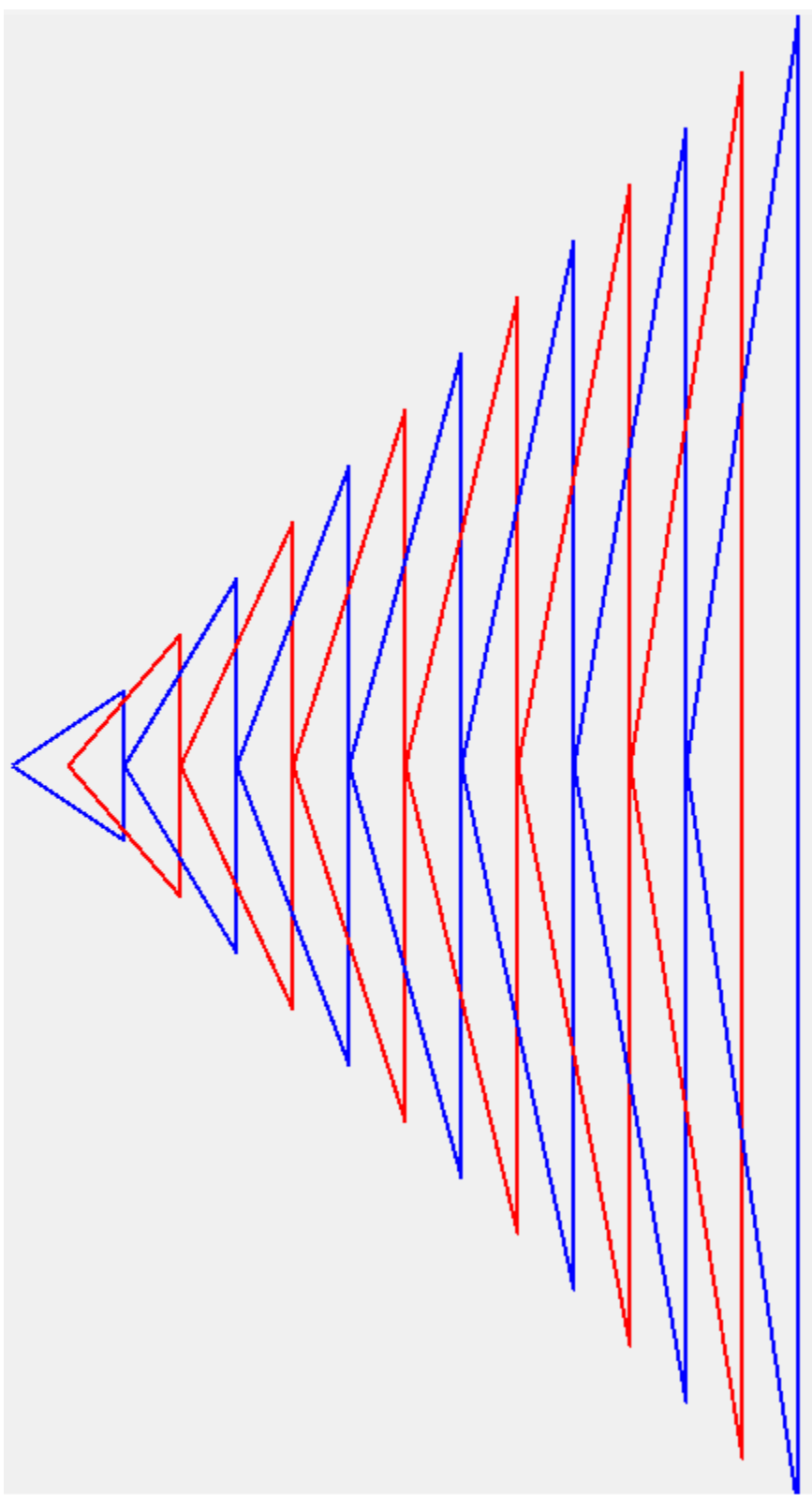
*Ilustración 34: Triángulos apuntan a la izquierda*

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics grafico = e.Graphics;
            Pen lapiz;
            Pen lapizRojo = new Pen(Color.Red, 2);
            Pen lapizAzul = new Pen(Color.Blue, 2);

            int pX1, pY1, pX2, pY2, pX3, pY3, constante, cambia;
            pX1 = 220;
            pY1 = 360;
            pX2 = 160;
            pY2 = 400;
            pX3 = 220;
            pY3 = 440;
            constante = 30;
            cambia = 1;
            do {
                if (cambia == 1) {
                    cambia = 2;
                    lapiz = lapizAzul;
                }
                else {
                    cambia = 1;
                    lapiz = lapizRojo;
                }

                grafico.DrawLine(lapiz, pX1, pY1, pX2, pY2);
                grafico.DrawLine(lapiz, pX2, pY2, pX3, pY3);
                grafico.DrawLine(lapiz, pX1, pY1, pX3, pY3);
                pX1 += constante;
                pY1 -= constante;
                pX2 += constante;
                pX3 += constante;
                pY3 += constante;
            }
            while (pY1 >= 0);
        }
    }
}
```



*Ilustración 35: Triángulos apuntan a la izquierda alternando colores*

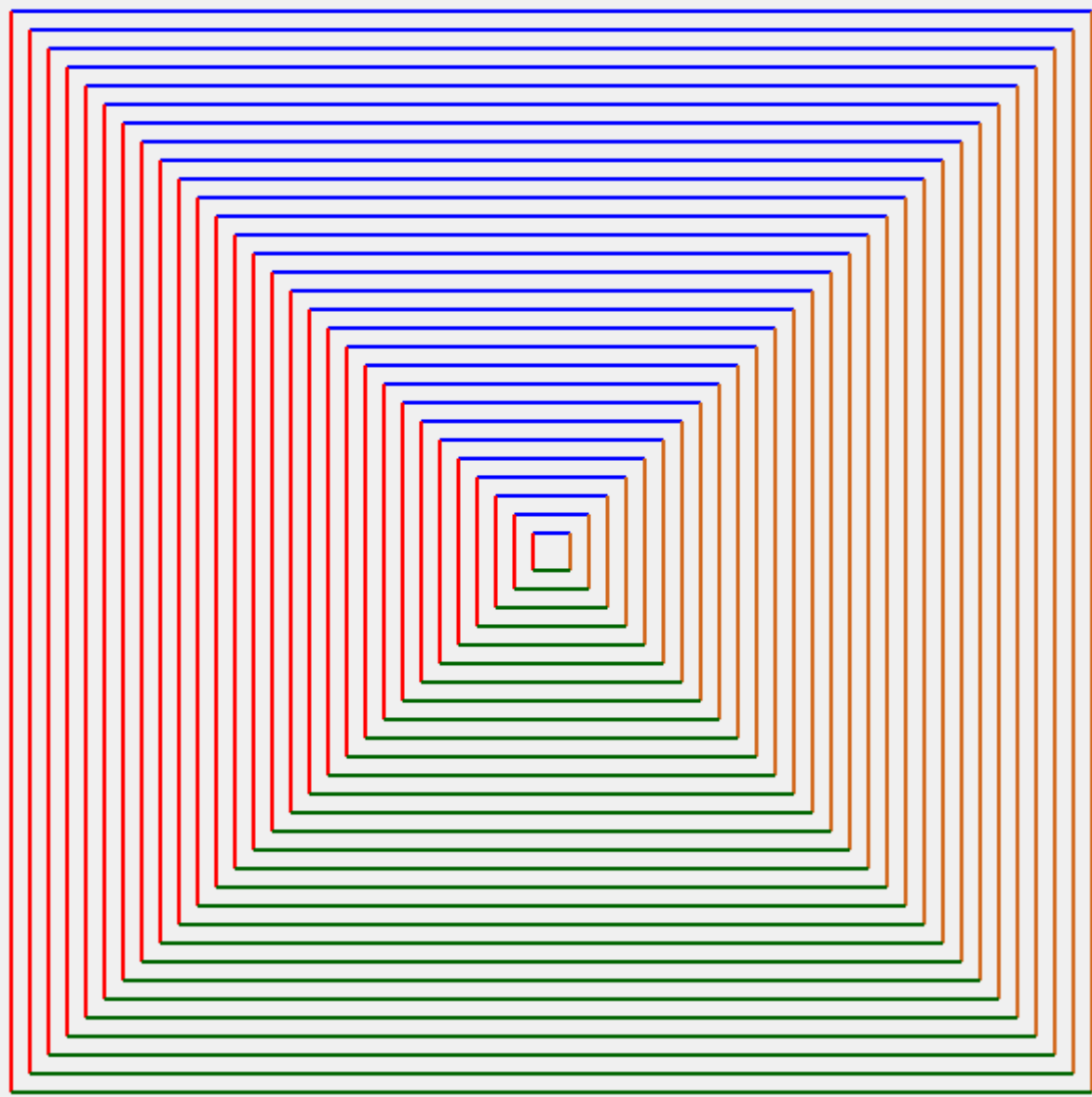
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics grafico = e.Graphics;
            Pen lapizA = new Pen(Color.Blue, 2);
            Pen lapizB = new Pen(Color.Red, 2);
            Pen lapizC = new Pen(Color.Chocolate, 2);
            Pen lapizD = new Pen(Color.DarkGreen, 2);

            int variar = 0;
            int centro = 380;

            for (var cont = 1; cont <= 30; cont += 1) {
                int valA = centro - variar;
                int valB = centro + variar;
                grafico.DrawLine(lapizA, valA, valA, valB, valA);
                grafico.DrawLine(lapizB, valA, valA, valA, valB);
                grafico.DrawLine(lapizC, valB, valA, valB, valB);
                grafico.DrawLine(lapizD, valA, valB, valB, valB);
                variar += 10;
            }
        }
    }
}
```





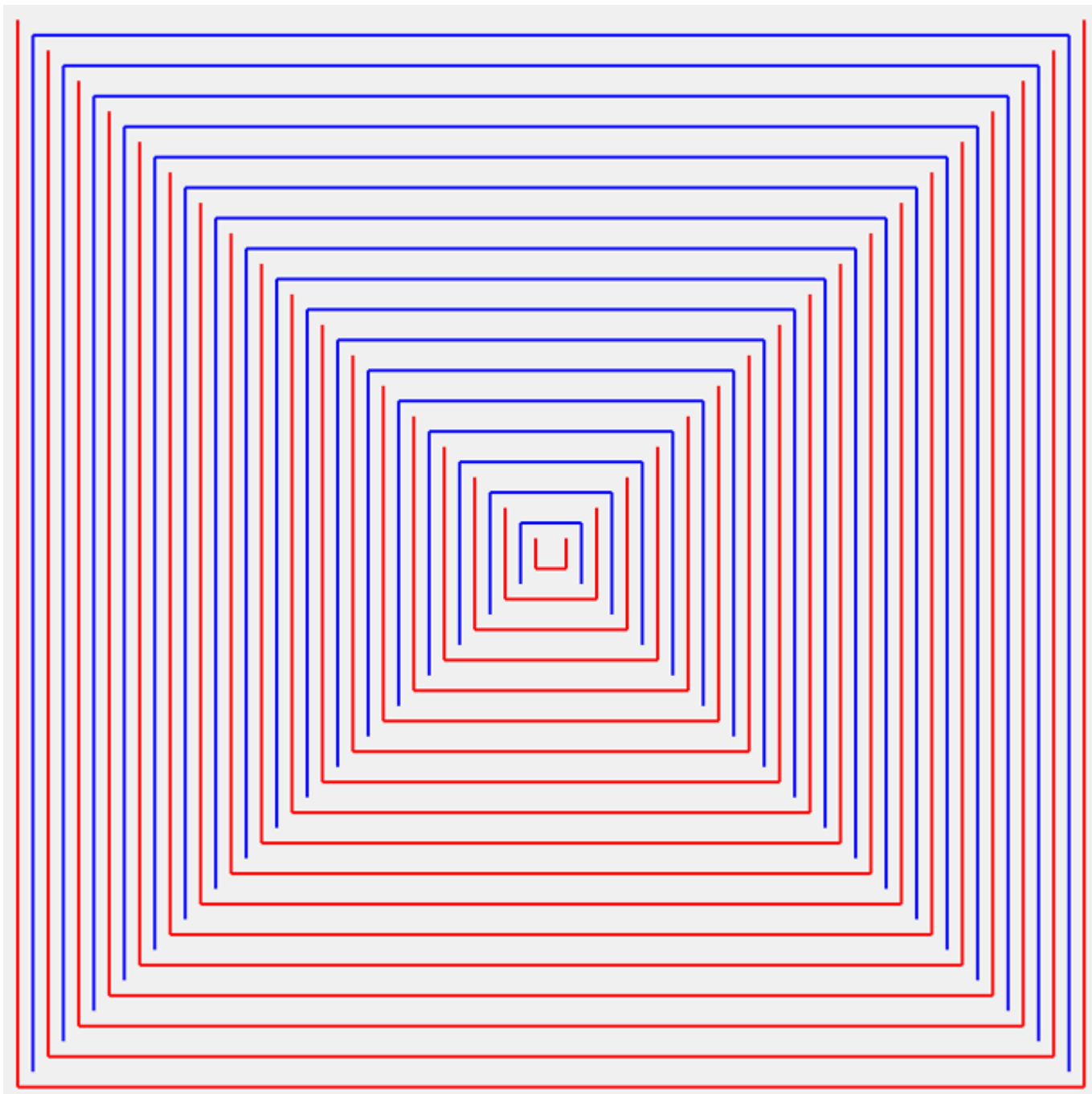
*Ilustración 36: Rectángulos concéntricos*

```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics lienzo = e.Graphics;
            Pen lapiz = new Pen(Color.Blue, 2);
            Pen lapiz2 = new Pen(Color.Red, 2);
            int variar = 0;

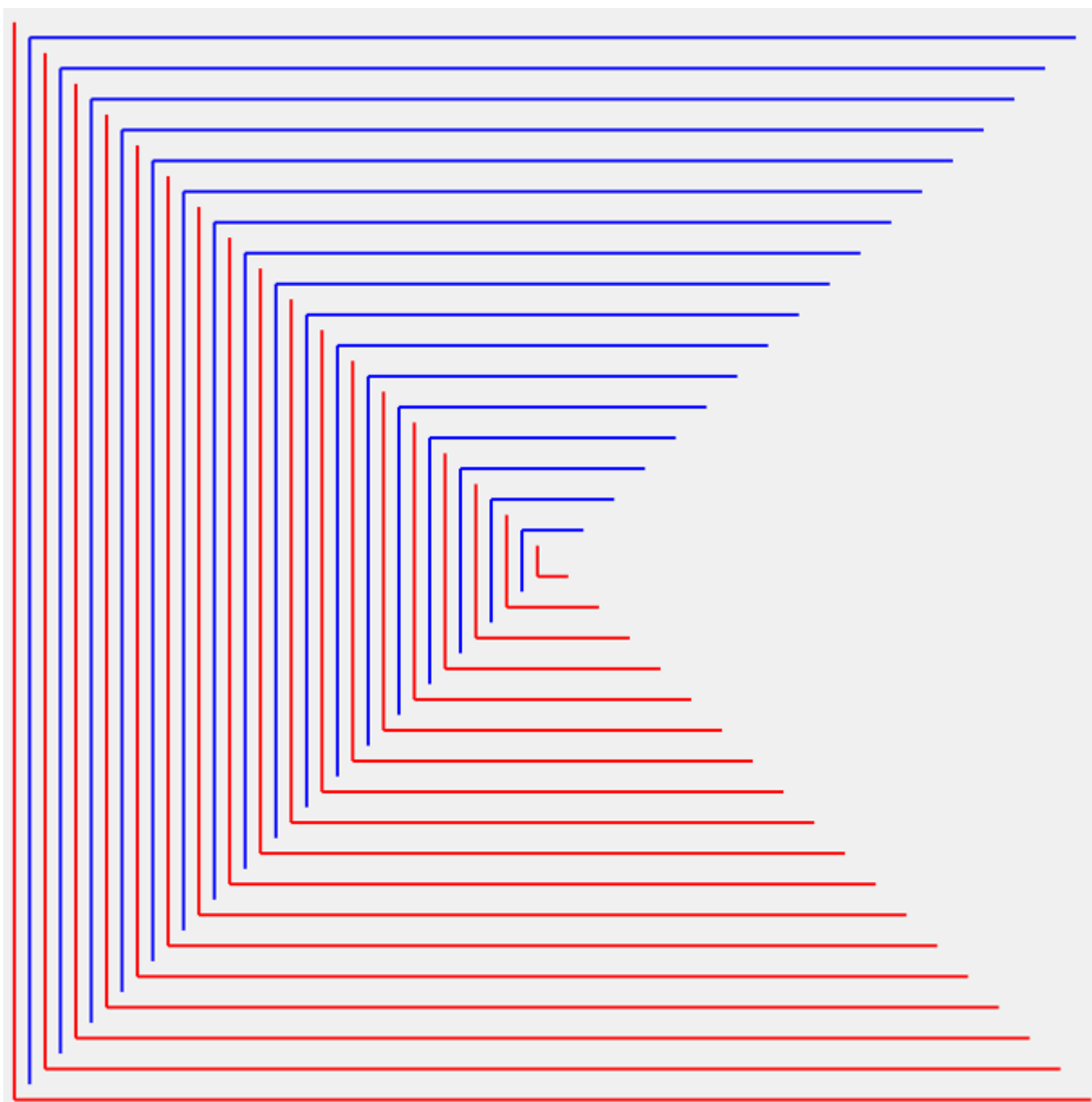
            for (var cont = 1; cont <= 18; cont += 1) {
                int valA = 380 - variar;
                int valB = 380 + variar;
                lienzo.DrawLine(lapiz, valA, valA, valB, valA);
                lienzo.DrawLine(lapiz, valA, valA, valA, valB);
                lienzo.DrawLine(lapiz, valB, valA, valB, valB);
                variar += 10;

                lienzo.DrawLine(lapiz2, valA, valB, valB, valB);
                lienzo.DrawLine(lapiz2, valA, valA, valA, valB);
                lienzo.DrawLine(lapiz2, valB, valA, valB, valB);
                variar += 10;
            }
        }
    }
}
```



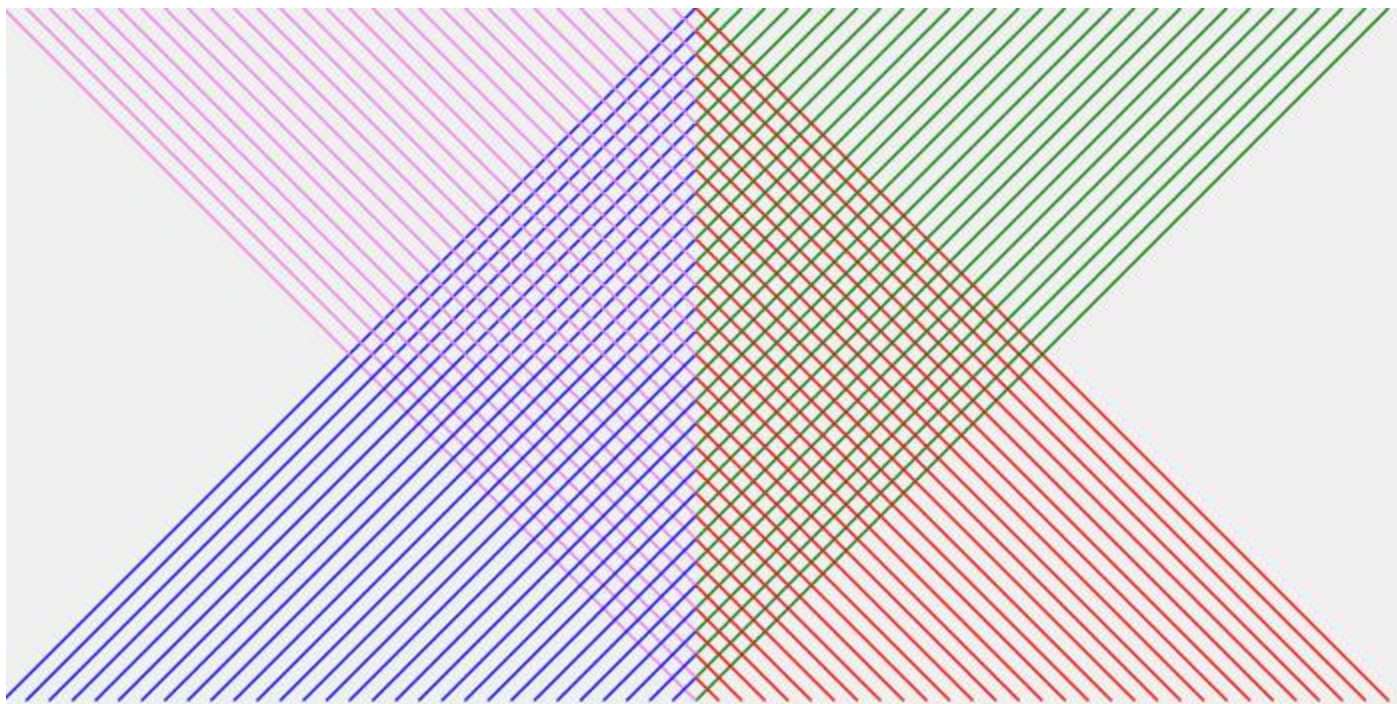
*Ilustración 37: Dibuja formas que semejan cuadrados a las que les falta el techo o la base*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics lienzo = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            int variar = 0;  
  
            for (var cont = 1; cont <= 18; cont += 1) {  
                int valA = 380 - variar;  
                int valB = 380 + variar;  
                lienzo.DrawLine(lapiz, valA, valA, valB, valA);  
                lienzo.DrawLine(lapiz, valA, valA, valA, valB);  
                variar += 10;  
  
                lienzo.DrawLine(lapiz2, valA, valB, valB, valB);  
                lienzo.DrawLine(lapiz2, valA, valA, valA, valB);  
                variar += 10;  
            }  
        }  
    }  
}
```



*Ilustración 38: Dibuja ángulos rectos*

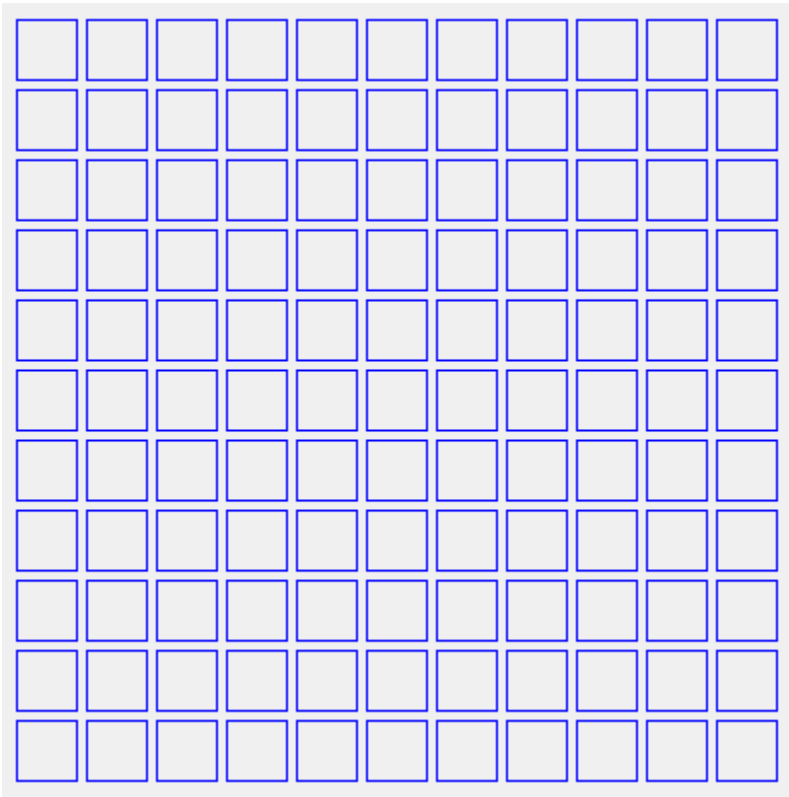
```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            Graphics grafico = e.Graphics;  
            Pen lapiz = new Pen(Color.Blue, 2);  
            Pen lapiz2 = new Pen(Color.Red, 2);  
            Pen lapiz3 = new Pen(Color.Green, 2);  
            Pen lapiz4 = new Pen(Color.Violet, 2);  
            int cont = 0;  
  
            do {  
                grafico.DrawLine(lapiz, cont, 600, 600, cont);  
                grafico.DrawLine(lapiz2, 1200 - cont, 600, 600, cont);  
                grafico.DrawLine(lapiz3, 1200 - cont, 0, 600, 600 - cont);  
                grafico.DrawLine(lapiz4, cont, 0, 600, 600 - cont);  
                cont += 20;  
            }  
            while (cont <= 600);  
        }  
    }  
}
```



*Ilustración 39: Líneas diagonales cruzadas*

```
namespace Graficos {  
    public partial class Form1 : Form {  
        public Form1() {  
            InitializeComponent();  
        }  
  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            // Necesario para los gráficos GDI+  
            Graphics grafico = e.Graphics;  
  
            // Lápiz para el color de la línea y con un ancho de 1  
            Pen lapiz = new Pen(Color.Blue, 1);  
  
            for (int Fila = 0; Fil <= 10; Fil++) {  
                for (int Col = 0; Col <= 10; Col++){  
                    int valA = Fil * 35 + 80;  
                    int valB = Col * 35 + 40;  
                    grafico.DrawRectangle(lapiz, valA, valB, 30, 30);  
                }  
            }  
        }  
    }  
}
```





*Ilustración 40: Celdas*

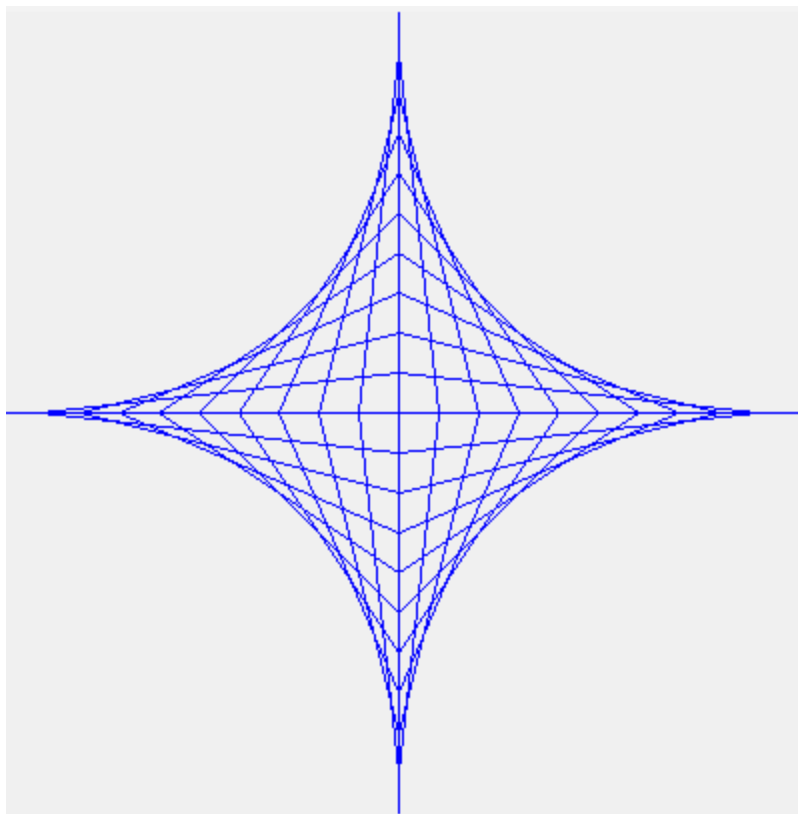
```
namespace Graficos {
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics grafico = e.Graphics;

            // Lápiz para el color de la línea y con un ancho de 1
            Pen lapiz = new Pen(Color.Blue, 1);

            // Parte superior
            for (int Posicion = 0; Posicion <= 200; Posicion += 20) {
                // Línea requiere: Lapiz, X1, Y1, X2, Y2
                grafico.DrawLine(lapiz, 200, Posicion, Posicion + 200, 200);
                grafico.DrawLine(lapiz, 200, Posicion, 200 - Posicion, 200);
            }

            //Parte inferior
            for (int Posicion = 400; Posicion >= 200; Posicion += -20) {
                // Línea requiere: Lapiz, X1, Y1, X2, Y2
                grafico.DrawLine(lapiz, 200, Posicion, 600 - Posicion, 200);
                grafico.DrawLine(lapiz, 200, Posicion, Posicion - 200, 200);
            }
        }
    }
}
```



*Ilustración 41: Líneas rectas simulan curvas*

## Algoritmo de Bresenham para dibujar líneas

Este algoritmo dibuja líneas rectas dadas las coordenadas inicial y final. Lo interesante es que no hace operaciones de punto flotante (que sería lento) sino operaciones de enteros (más rápido). Ese es el algoritmo de la función DrawLine

L/033.cs

```
//Algoritmo de Bresenham, para dibujar líneas rectas rápidamente
namespace Graficos {
    public partial class Form1 : Form {

        public Form1() {
            InitializeComponent();
        }

        public void Linea(Graphics lienzo,
            int iniX, int iniY,
            int finX, int finY) {
            int Contador, Distancia;
            int Xerror=0, Yerror=0;
            int CambioX, CambioY, incrementoX, incrementoY;

            CambioX = finX - iniX;
            CambioY = finY - iniY;

            if (CambioX > 0) incrementoX = 1;
            else if (CambioX == 0) incrementoX = 0;
            else incrementoX = -1;

            if (CambioY > 0) incrementoY = 1;
            else if (CambioY == 0) incrementoY = 0;
            else incrementoY = -1;

            if (CambioX < 0) CambioX *= -1;
            if (CambioY < 0) CambioY *= -1;

            if (CambioX > CambioY)
                Distancia = CambioX;
            else
                Distancia = CambioY;

            for (Contador = 0; Contador <= Distancia + 1; Contador++) {
                lienzo.FillRectangle(Brushes.Black, iniX, iniY, 1, 1);
                Xerror += CambioX;
                Yerror += CambioY;
                if (Xerror > Distancia) {
                    Xerror -= Distancia;
                }
            }
        }
    }
}
```

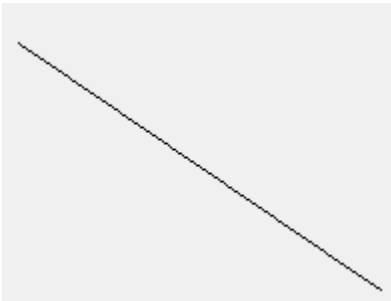
```

        iniX += incrementoX;
    }

    if (Yerror > Distancia) {
        Yerror -= Distancia;
        iniY += incrementoY;
    }
}

private void Form1_Paint(object sender, PaintEventArgs e) {
    //Ejemplo
    Linea(e.Graphics, 16, 83, 197, 206);
}
}

```



*Ilustración 42: Algoritmo de Bresenham para dibujar líneas*

# Transformaciones en 2D

## Traslado de figuras en un plano

Mover la figura en el eje X y en el eje Y, es simplemente sumar o restar a los valores de las coordenadas:

$NuevaPosicionX = PosicionOriginalX + MueveX$

$NuevaPosicionY = PosicionOriginalY + MueveY$

L/034.cs

```
//Traslado de figuras en un plano
namespace Graficos {
    public partial class Form1 : Form {

        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics lienzo = e.Graphics;

            //Traslado horizontal
            int mueveX = 0;

            //Traslado vertical
            int mueveY = 0;

            //Nuevas coordenadas
            int posX1 = 10 + mueveX;
            int posX2 = 400 + mueveX;
            int posY1 = 10 + mueveY;
            int posY2 = 250 + mueveY;

            //Dibuja un rectángulo con cuatro líneas
            lienzo.DrawLine(Pens.Black, posX1, posY1, posX2, posY1);
            lienzo.DrawLine(Pens.Black, posX1, posY2, posX2, posY2);
            lienzo.DrawLine(Pens.Black, posX2, posY1, posX2, posY2);
            lienzo.DrawLine(Pens.Black, posX1, posY1, posX1, posY2);
        }
    }
}
```



*Ilustración 43: Traslado de figuras en un plano*



*Ilustración 44: Traslado de figuras en un plano mueveX = 70*

## Giro de figuras en un plano

Para el giro en determinado ángulo, se usa la siguiente fórmula para una coordenada plana:

$$\text{PosicionNuevaX} = \text{PosicionOriginalX} * \cos(\text{Angulo}) - \text{PosicionOriginalY} * \sin(\text{Angulo})$$
$$\text{PosicionNuevaY} = \text{PosicionOriginalX} * \sin(\text{Angulo}) + \text{PosicionOriginalY} * \cos(\text{Angulo})$$

L/035.cs

```
//Giro de figuras en un plano
namespace Graficos {
    public partial class Form1 : Form {

        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Coordenadas de la figura
            int posXa, posYa, posXb, posYb, posXc, posYc;

            posXa = 80;
            posYa = 80;
            posXb = 400;
            posYb = 300;
            posXc = 500;
            posYc = 200;

            //Dibuja un triángulo con tres líneas
            e.Graphics.DrawLine(Pens.Black, posXa, posYa, posXb, posYb);
            e.Graphics.DrawLine(Pens.Black, posXb, posYb, posXc, posYc);
            e.Graphics.DrawLine(Pens.Black, posXc, posYc, posXa, posYa);

            //Ángulo de giro
            int AnguloGiro = 15;
            double AnguloRadianes = AnguloGiro * Math.PI / 180;
            double CosA = Math.Cos(AnguloRadianes);
            double SinA = Math.Sin(AnguloRadianes);

            //Cálcula el giro
            int posXga = Convert.ToInt32(posXa * CosA - posYa * SinA);
            int posYga = Convert.ToInt32(posXa * SinA + posYa * CosA);

            int posXgb = Convert.ToInt32(posXb * CosA - posYb * SinA);
            int posYgb = Convert.ToInt32(posXb * SinA + posYb * CosA);

            int posXgc = Convert.ToInt32(posXc * CosA - posYc * SinA);
```

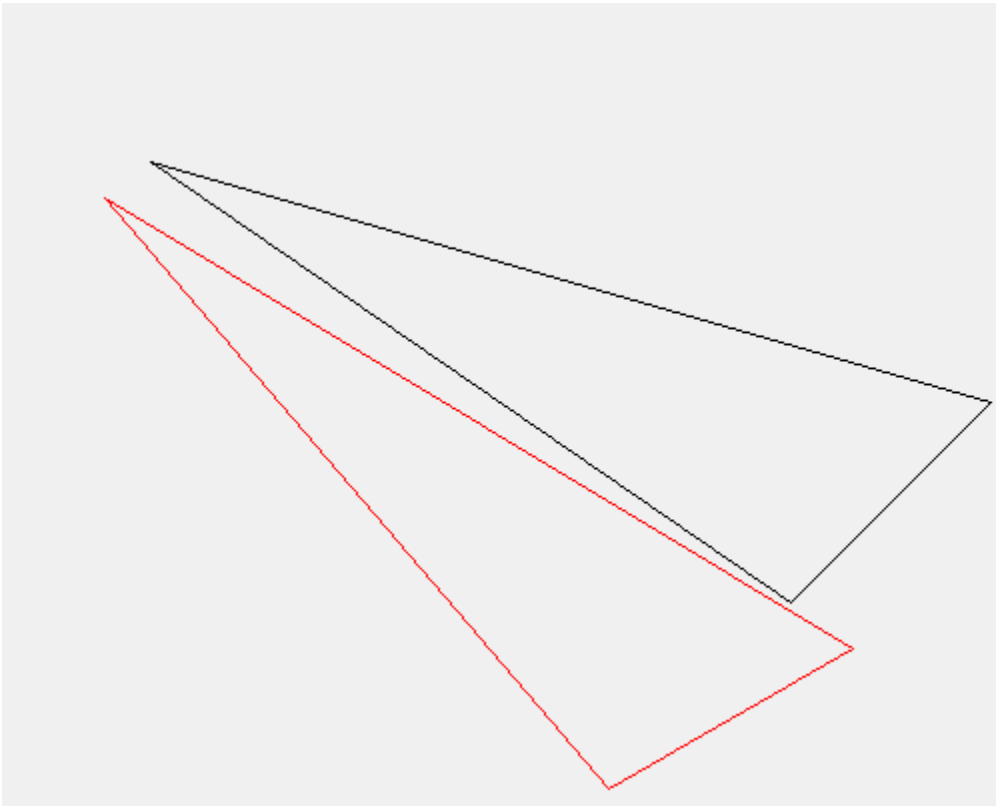


```

    int posYgc = Convert.ToInt32(posXc * SinA + posYc * CosA);

    //Dibuja el triángulo con el giro
    e.Graphics.DrawLine(Pens.Red, posXga, posYga, posXgb, posYgb);
    e.Graphics.DrawLine(Pens.Red, posXgb, posYgb, posXgc, posYgc);
    e.Graphics.DrawLine(Pens.Red, posXgc, posYgc, posXga, posYga);
}
}
}

```



*Ilustración 45: Giro de figuras en un plano*

En el ejemplo, se usó un giro de 5 grados (que se deben convertir en radianes antes de hacer uso de las funciones de seno y coseno). El ángulo se aplica desde la posición 0,0 de la ventana.

## Giro de figuras en un plano calculando el centroide

Se requiere calcular el centroide de la figura (fácil si es un rectángulo), se aplica el giro y se compara cuánto se desplazó en X y Y con respecto a la posición original. Una vez obtenidos esos datos, se restaura para toda la figura y se obtiene el giro sobre sí misma.

L/036.cs

```
//Giro de figuras en un plano calculando el centroide
namespace Graficos {
    public partial class Form1 : Form {

        public Form1() {
            InitializeComponent();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics lienzo = e.Graphics;
            Pen lapizA = new(Color.Black, 1);
            pen lapizB = new(Color.Red, 2);

            //Datos del rectángulo
            int Xa, Ya, Largo, Alto;
            Xa = 80;
            Ya = 80;
            Largo = 400;
            Alto = 180;

            //Deduce las otras tres coordenadas del rectángulo
            int Xb, Yb, Xc, Yc, Xd, Yd;
            Xb = Xa + Largo;
            Yb = Ya;
            Xc = Xa + Largo;
            Yc = Ya + Alto;
            Xd = Xa;
            Yd = Ya + Alto;

            //Dibuja un rectángulo con cuatro líneas
            lienzo.DrawLine(lapizA, Xa, Ya, Xb, Yb);
            lienzo.DrawLine(lapizA, Xb, Yb, Xc, Yc);
            lienzo.DrawLine(lapizA, Xc, Yc, Xd, Yd);
            lienzo.DrawLine(lapizA, Xd, Yd, Xa, Ya);

            //Ángulo de giro
            int AnguloGiro = 15;
            double AnguloRadianes = AnguloGiro * Math.PI / 180;
            double CosA = Math.Cos(AnguloRadianes);
            double SinA = Math.Sin(AnguloRadianes);
```

```

//Centroide
int Xcentro = Xa + Largo / 2;
int Ycentro = Ya + Alto / 2;

//Cálcula el giro
int Xga = Convert.ToInt32(Xa * CosA - Ya * SinA);
int Yga = Convert.ToInt32(Xa * SinA + Ya * CosA);

int Xgb = Convert.ToInt32(Xb * CosA - Yb * SinA);
int Ygb = Convert.ToInt32(Xb * SinA + Yb * CosA);

int Xgc = Convert.ToInt32(Xc * CosA - Yc * SinA);
int Ygc = Convert.ToInt32(Xc * SinA + Yc * CosA);

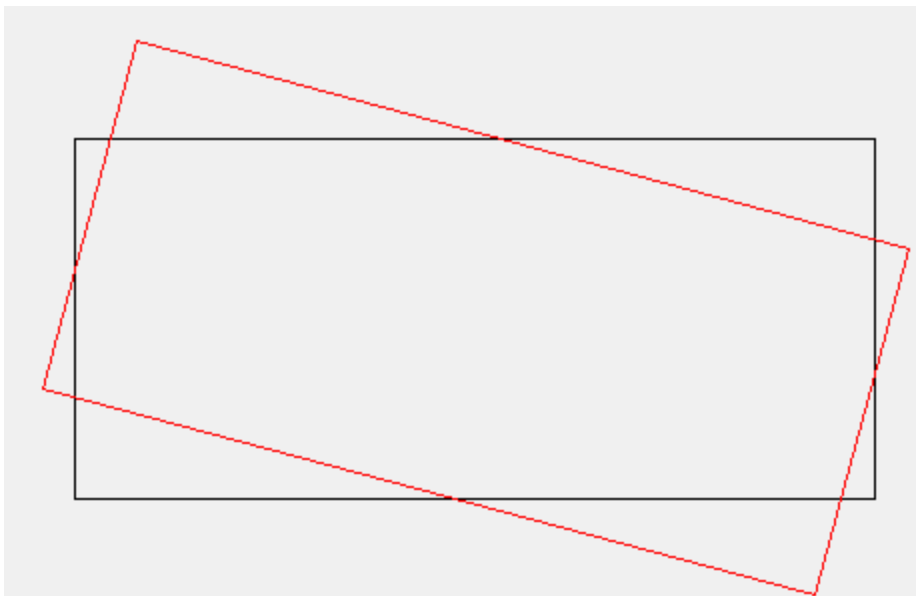
int Xgd = Convert.ToInt32(Xd * CosA - Yd * SinA);
int Ygd = Convert.ToInt32(Xd * SinA + Yd * CosA);

//Giro del centroide
int Xgcentro = Convert.ToInt32(Xcentro * CosA - Ycentro * SinA);
int Ygcentro = Convert.ToInt32(Xcentro * SinA + Ycentro * CosA);

//¿Cuánto se desplazo el centroide?
int dX = Xgcentro - Xcentro;
int dY = Ygcentro - Ycentro;

//Dibuja el triángulo con el giro
lienzo.DrawLine(lapizB, Xga - dX, Yga - dY, Xgb - dX, Ygb - dY);
lienzo.DrawLine(lapizB, Xgb - dX, Ygb - dY, Xgc - dX, Ygc - dY);
lienzo.DrawLine(lapizB, Xgc - dX, Ygc - dY, Xgd - dX, Ygd - dY);
lienzo.DrawLine(lapizB, Xga - dX, Yga - dY, Xgd - dX, Ygd - dY);
    }
}
}

```



*Ilustración 46: Giro de figuras en un plano calculando el centroide*

## Gráfico matemático en 2D

Dada una ecuación del tipo  $Y=F(X)$ , por ejemplo:

$$Y = 0.1X^6 + 0.6X^5 - 0.7X^4 - 5.7X^3 + 2X^2 + 2X + 1$$

Para graficarla, se deben hacer estos pasos:

1. Saber el valor de inicio de X y el valor final de X. Se llamarán Xini y Xfin respectivamente.
2. También se necesita el avance de X desde el inicio hasta el final, se llamará Xavance.
3. Con esos datos, se hace uso de un ciclo que calcule los valores de Y. Se almacena el par X, Y
4. Debido a que el eje Y aumenta hacia abajo en una pantalla o ventana, habrá que darles la vuelta a los valores de Y calculados, es decir, multiplicarlos por -1. Luego realmente se almacena X, -Y
5. Se calculan cuatro datos:
  - a. El mínimo valor de X. Se llamará Xmin.
  - b. El máximo valor de X. Se llamará Xmax.
  - c. El mínimo valor de Y obtenido. Se llamará Ymin.
  - d. El máximo valor de Y obtenido. Se llamará Ymax.
6. Se requieren dos datos para poder ajustar el gráfico matemático a un área rectangular definida en la ventana:
  - a. Coordenada superior izquierda en pantalla. Serán las coordenadas enteras positivas XpantallaIni, YpantallaIni
  - b. Coordenada inferior derecha en pantalla. Serán las coordenadas enteras positivas XpantallaFin, YpantallaFin
7. Se calculan unas constantes de conversión con estas fórmulas:
  - a. `convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xmin)`
  - b. `convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin)`
8. Tomar cada coordenada calculada de la ecuación (X, Y) y hacerle la conversión a pantalla plana con la siguiente fórmula:
  - a. `pantX = convierteX * (X - Xmin) + XpantallaIni`
  - b. `pantY = convierteY * (Y - Ymin) + YpantallaIni`
9. Se grafican esos puntos y se unen con líneas.

La clase Linea es para almacenar los valores reales de la ecuación (valorXa, valorYa, valorXb, valorYb) y los valores convertidos para que cuadren en pantalla (pantXa, pantYa, pantXb, pantYb)

```

using System;
using System.Collections.Generic;
using System.Text;

namespace Graficos {
    internal class Linea {
        //Valor X, Y reales de la ecuación para dibujar la línea
        public double Xa, Ya;
        public double Xb, Yb;

        //Puntos convertidos a coordenadas enteras de pantalla
        public int pantXa, pantYa;
        public int pantXb, pantYb;

        public Linea(double Xa, double Ya, double Xb, double Yb) {
            this.Xa = Xa;
            this.Ya = Ya;
            this.Xb = Xb;
            this.Yb = Yb;
        }

        //Convierte las coordenadas reales a coordenadas en pantalla
        public void Convierte(double convierteX, double convierteY, double
Xmin, double Ymin, int XpantallaIni, int YpantallaIni) {
            pantXa = Convert.ToInt32(convierteX * (Xa - Xmin) + XpantallaIni);
            pantXb = Convert.ToInt32(convierteX * (Xb - Xmin) + XpantallaIni);
            pantYa = Convert.ToInt32(convierteY * (Ya - Ymin) + YpantallaIni);
            pantYb = Convert.ToInt32(convierteY * (Yb - Ymin) + YpantallaIni);
        }
    }
}

```

```

namespace Graficos
{
    public partial class Form1 : Form {
        List<Linea> Lineas;
        List<Linea> Ejes;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        public Form1() {
            InitializeComponent();
            Lineas = [];
        }
    }
}

```

```

Ejes = [];

//Área dónde se dibujará el gráfico matemático
XpantallaIni = 20;
YpantallaIni = 40;
XpantallaFin = 600;
YpantallaFin = 550;

//Algoritmo que calcula los puntos del gráfico
double Xini = -100;
double Xfin = 750;
double Xavance = 0.01;
Logica(Xini, Xfin, Xavance);
}

public void Logica(double Xini, double Xfin, double Xavance) {

    Lineas.Clear();
    Ejes.Clear();

    //Calcula las líneas que tendrá el gráfico
    double valXa = Xini;
    double valYa = -Ecuacion(valXa);
    while (valXa < Xfin) {
        double valXb = Math.Round(valXa + Xavance, 5);
        double valYb = -Ecuacion(valXb);

        //Si la línea es válida, se agrega a la lista
        if (double.IsFinite(valYa) && double.IsFinite(valYb)) {
            Lineas.Add(new Linea(valXa, valYa, valXb, valYb));
        }

        valXa = valXb;
        valYa = valYb;
    }

    //Determina los valores máximos y mínimos en X y Y que tienen las
líneas
    double Xmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymin = double.MaxValue;
    double Ymax = double.MinValue;
    foreach (var linea in Lineas) {
        Xmin = Math.Min(Xmin, Math.Min(linea.Xa, linea.Xb));
        Xmax = Math.Max(Xmax, Math.Max(linea.Xa, linea.Xb));
        Ymin = Math.Min(Ymin, Math.Min(linea.Ya, linea.Yb));
        Ymax = Math.Max(Ymax, Math.Max(linea.Ya, linea.Yb));
    }
}

```

```

//Extremos para poder dibujar los ejes
Xmin = Math.Min(Xmin, Xini);
Xmax = Math.Max(Xmax, Xfin);
Ymin = Math.Min(Ymin, 0);
Ymax = Math.Max(Ymax, 0);

//Ejes
Ejes.Add(new Linea(Xmin, 0, Xmax, 0)); //Eje X
Ejes.Add(new Linea(0, Ymin, 0, Ymax)); //Eje Y

//Calcula los puntos a poner en la pantalla
double convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xmin);
double convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin);

for (int Cont = 0; Cont < Lineas.Count; Cont++) {
    Lineas[Cont].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
}

//Calcula donde poner los ejes
Ejes[0].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
Ejes[1].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
}

//Aquí está la ecuación que se desee graficar con variable
independiente X
public double Ecuacion(double X) {
    //return Math.Sqrt(Math.Cos(X*Math.PI/180));
    //return Math.Sqrt(X);
    //return (X * X - 1) / (X - 1);
    return Math.Sin(X * Math.PI / 180);
}

//Pinta la ecuación
private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics Lienzo = e.Graphics;
    Pen LapizGrafica = new(Color.Blue, 2);
    Pen LapizPerimetro = new(Color.White, 2);
    Pen LapizEje = new(Color.Red, 2);
    Brush Relleno = new SolidBrush(Color.White);

    //Un recuadro para ver el área del gráfico
    Lienzo.DrawRectangle(LapizPerimetro, XpantallaIni, YpantallaIni,
        XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

```



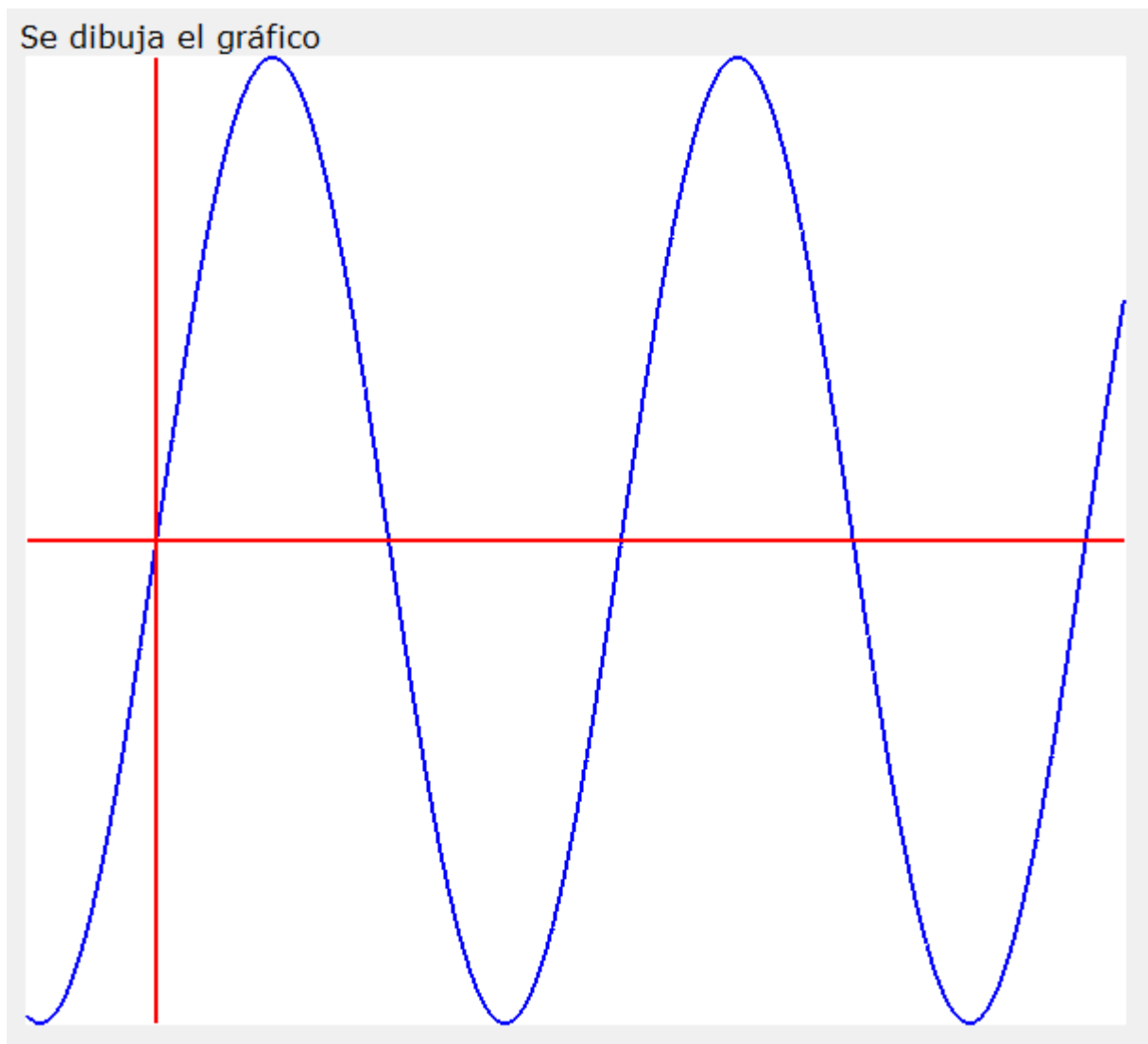
```

        Lienzo.FillRectangle(Relleno, XpantallaIni, YpantallaIni,
            XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

        //Dibuja el gráfico matemático
        for (int Cont = 0; Cont < Lineas.Count; Cont++) {
            Lienzo.DrawLine(LapizGrafica, Lineas[Cont].pantXa,
                Lineas[Cont].pantYa,
                Lineas[Cont].pantXb, Lineas[Cont].pantYb);
        }

        //Dibuja los ejes
        Lienzo.DrawLine(LapizEje, Ejes[0].pantXa, Ejes[0].pantYa,
            Ejes[0].pantXb, Ejes[0].pantYb);
        Lienzo.DrawLine(LapizEje, Ejes[1].pantXa, Ejes[1].pantYa,
            Ejes[1].pantXb, Ejes[1].pantYb);
    }
}
}

```



*Ilustración 47: Gráfico matemático en 2D*

## Gráfico polar

Dada una ecuación del tipo  $r=F(\Theta)$ , donde  $\Theta$  es el ángulo y  $r$  el radio, por ejemplo:

$$r = 2 * \text{seno}(4 * \theta * \pi/180)$$

Se procede a graficarlo de esta forma:

1. Saber el valor de inicio de  $\Theta$  y el valor final de  $\Theta$ . Se llamarán minTheta y maxTheta respectivamente.
2. También se necesita el avance de  $\Theta$  desde el inicio hasta el final, se llamará avanceTheta.
3. Con esos datos, se hace uso de un ciclo que calcule los valores de  $r$ . Luego se hace esta conversión:

$r = \text{Ecuacion}(\text{theta});$

$X = r * \text{Math.Cos}(\text{theta} * \text{Math.PI} / 180);$

$Y = r * \text{Math.Sin}(\text{theta} * \text{Math.PI} / 180);$

4. Debido a que el eje Y aumenta hacia abajo en una pantalla o ventana, habrá que darles la vuelta a los valores de Y calculados, es decir, multiplicarlos por -1. Luego realmente se almacena X, -Y
5. Se calculan cuatro datos:
  - a. El mínimo valor de X. Se llamará Xmin.
  - b. El máximo valor de X. Se llamará Xmax.
  - c. El mínimo valor de Y obtenido. Se llamará Ymin.
  - d. El máximo valor de Y obtenido. Se llamará Ymax.
6. Se requieren dos datos para poder ajustar el gráfico matemático a un área rectangular definida en la ventana:
  - a. Coordenada superior izquierda en pantalla. Serán las coordenadas enteras positivas XpantallaIni, YpantallaIni
  - b. Coordenada inferior derecha en pantalla. Serán las coordenadas enteras positivas XpantallaFin, YpantallaFin
7. Se calculan unas constantes de conversión con estas fórmulas:
  - a.  $\text{convierteX} = (\text{XpantallaFin} - \text{XpantallaIni}) / (\text{Xmax} - \text{Xmin});$
  - b.  $\text{convierteY} = (\text{YpantallaFin} - \text{YpantallaIni}) / (\text{Ymax} - \text{Ymin});$
8. Tomar cada coordenada calculada de la ecuación (X, Y) y hacerle la conversión a pantalla plana con la siguiente fórmula:
  - a.  $\text{pantX} = \text{convierteX} * (X - \text{Xmin}) + \text{XpantallaIni}$
  - b.  $\text{pantY} = \text{convierteY} * (Y - \text{Ymin}) + \text{YpantallaIni}$
9. Se grafican esos puntos y se unen con líneas.

La clase Linea es para almacenar los valores reales de la ecuación (valorXa, valorYa, valorXb, valorYb) y los valores convertidos para que cuadren en pantalla (pantXa, pantYa, pantXb, pantYb)

L/038.zip

```
namespace Graficos {
    internal class Linea {
        //Valor X, Y reales de la ecuación
        public double Xa, Ya;
        public double Xb, Yb;

        //Puntos convertidos a coordenadas enteras de pantalla
        public int pantXa, pantYa;
        public int pantXb, pantYb;

        public Linea(double Xa, double Ya, double Xb, double Yb) {
            this.Xa = Xa;
            this.Ya = Ya;
            this.Xb = Xb;
            this.Yb = Yb;
        }

        //Convierte las coordenadas reales a coordenadas en pantalla
        public void Convierte(double convierteX, double convierteY, double
Xmin, double Ymin, int XpantallaIni, int YpantallaIni) {
            pantXa = Convert.ToInt32(convierteX * (Xa - Xmin) + XpantallaIni);
            pantXb = Convert.ToInt32(convierteX * (Xb - Xmin) + XpantallaIni);
            pantYa = Convert.ToInt32(convierteY * (Ya - Ymin) + YpantallaIni);
            pantYb = Convert.ToInt32(convierteY * (Yb - Ymin) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    public partial class Form1 : Form {
        List<Linea> Lineas;
        List<Linea> Ejes;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        public Form1() {
            InitializeComponent();
            Lineas = [];
            Ejes = [];

            //Área dónde se dibujará el gráfico matemático
            XpantallaIni = 20;
        }
    }
}
```

```

YpantallaIni = 40;
XpantallaFin = 600;
YpantallaFin = 550;

//Algoritmo que calcula los puntos del gráfico
double ThetaIni = 0;
double ThetaFin = 720;
double AvanceTheta = 1;
Logica(ThetaIni, ThetaFin, AvanceTheta);
}

public void Logica(double ThetaIni, double ThetaFin, double
AvanceTheta) {

    Lineas.Clear();
    Ejes.Clear();

    //Calcula las líneas que tendrá el gráfico polar
    double Theta = ThetaIni;
    double Ra = -Ecuacion(Theta);
    while (Theta < ThetaFin) {
        double valXa = Ra * Math.Cos(Theta * Math.PI / 180);
        double valYa = -1 * Ra * Math.Sin(Theta * Math.PI / 180);

        Theta = Math.Round(Theta + AvanceTheta, 5);
        double Rb = -Ecuacion(Theta);
        double valXb = Rb * Math.Cos(Theta * Math.PI / 180);
        double valYb = -1 * Rb * Math.Sin(Theta * Math.PI / 180);

        if (double.IsFinite(valYa) && double.IsFinite(valYb)) {
            Lineas.Add(new Linea(valXa, valYa, valXb, valYb));
        }

        Ra = Rb;
    }

    //Determina los valores máximos y mínimos en X y Y que tienen las
líneas
    double Xmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymin = double.MaxValue;
    double Ymax = double.MinValue;
    foreach (var linea in Lineas) {
        Xmin = Math.Min(Xmin, Math.Min(linea.Xa, linea.Xb));
        Xmax = Math.Max(Xmax, Math.Max(linea.Xa, linea.Xb));
        Ymin = Math.Min(Ymin, Math.Min(linea.Ya, linea.Yb));
        Ymax = Math.Max(Ymax, Math.Max(linea.Ya, linea.Yb));
    }
}

```

```

//Extremos para poder dibujar los ejes
Ymin = Math.Min(Ymin, 0);
Ymax = Math.Max(Ymax, 0);

//Ejes
Ejes.Add(new Linea(Xmin, 0, Xmax, 0)); //Eje X
Ejes.Add(new Linea(0, Ymin, 0, Ymax)); //Eje Y

//Calcula los puntos a poner en la pantalla
double convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xmin);
double convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin);

for (int Cont = 0; Cont < Lineas.Count; Cont++) {
    Lineas[Cont].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
}

//Calcula donde poner los ejes
Ejes[0].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
Ejes[1].Convierte(convierteX, convierteY, Xmin, Ymin,
XpantallaIni, YpantallaIni);
}

//Aquí está la ecuación que se desee graficar con variable
independiente Theta
public double Ecuacion(double Theta) {
    return 2*Theta;
}

//Pinta la ecuación
private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics Lienzo = e.Graphics;
    Pen LapizGrafica = new(Color.Blue, 2);
    Pen LapizPerimetro = new(Color.White, 2);
    Pen LapizEje = new(Color.Red, 2);
    Brush Relleno = new SolidBrush(Color.White);

    //Un recuadro para ver el área del gráfico
    Lienzo.DrawRectangle(LapizPerimetro, XpantallaIni, YpantallaIni,
        XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

    Lienzo.FillRectangle(Relleno, XpantallaIni, YpantallaIni,
        XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

    //Dibuja el gráfico matemático
    for (int Cont = 0; Cont < Lineas.Count; Cont++) {

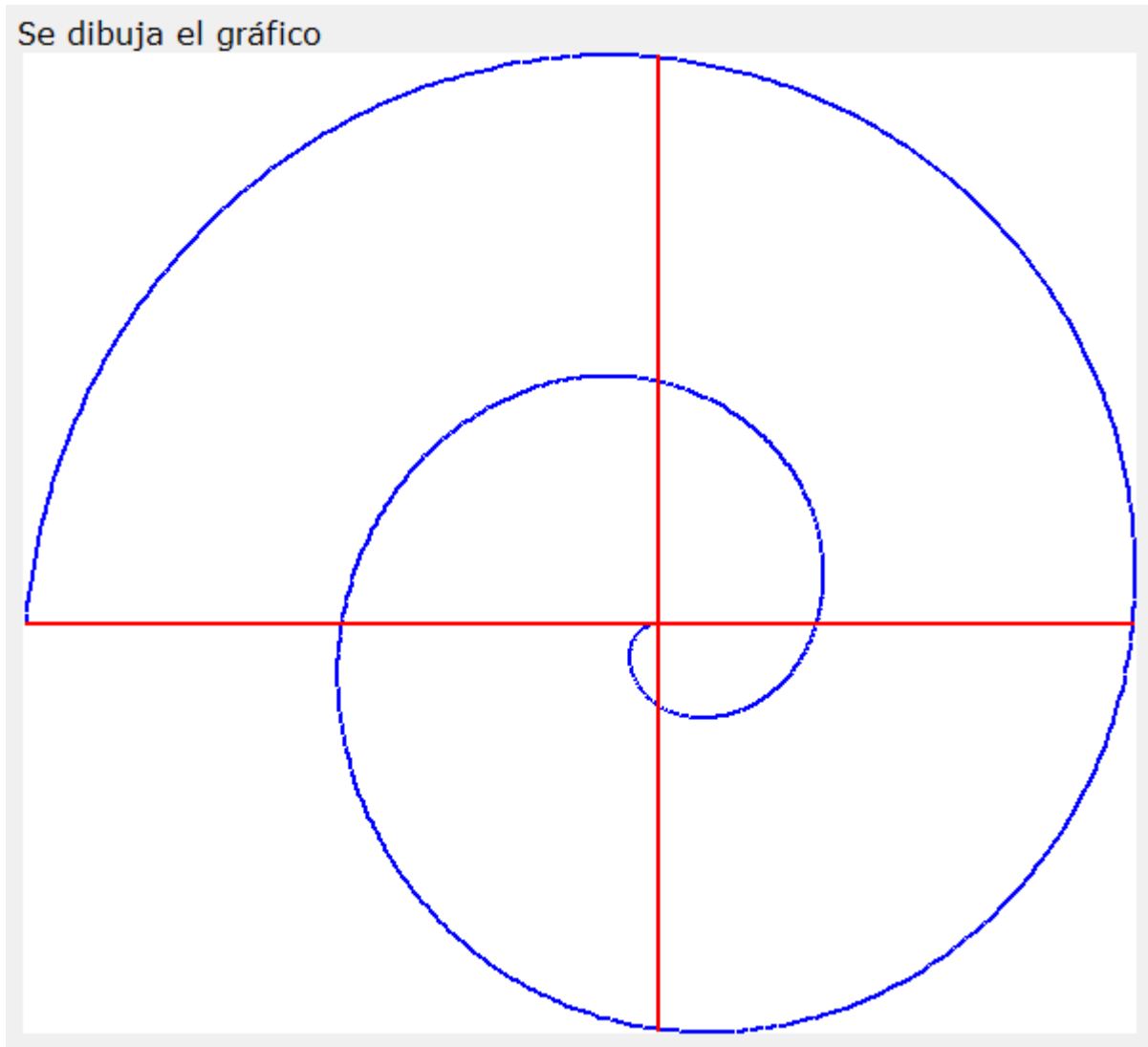
```

```

        Lienzo.DrawLine(LapizGrafica, Lineas[Cont].pantXa,
Lineas[Cont].pantYa,
        Lineas[Cont].pantXb, Lineas[Cont].pantYb);
    }

    //Dibuja los ejes
    Lienzo.DrawLine(LapizEje, Ejes[0].pantXa, Ejes[0].pantYa,
        Ejes[0].pantXb, Ejes[0].pantYb);
    Lienzo.DrawLine(LapizEje, Ejes[1].pantXa, Ejes[1].pantYa,
        Ejes[1].pantXb, Ejes[1].pantYb);
}
}
}

```



*Ilustración 48: Gráfico polar*

# Uso de Bitmap para control a nivel de pixel

C# trae Bitmap para control sobre el pixel, estas son sus características:

## Control total del píxel

1. Se puede usar "SetPixel(x, y, color)" para modificar cada píxel individualmente.
2. Es Ideal para algoritmos como Z-buffer, ray tracing, rasterización manual, etc.

## Persistencia del dibujo

1. El contenido del "Bitmap" se mantiene en memoria.
2. Se puede redibujar la imagen fácilmente en cada "Paint" sin recalcularlo todo.

## Separación de lógica y presentación

1. Se puede hacer todo el cálculo y renderizado en memoria.
2. Luego simplemente es mostrar el resultado con "Graphics.DrawImage()".

## Mayor eficiencia en renderizados complejos

1. Aunque "SetPixel" no es muy rápido, se puede optimizar usando "LockBits" si es necesario.
2. Se evita redibujar todo desde cero en cada evento "Paint".

L/039.cs

```
namespace Graficos {
    public partial class Form1 : Form {
        private Bitmap Lienzo;

        public Form1() {
            InitializeComponent();
            Lienzo = new Bitmap(400, 300);
        }

        //Pintar
        private void Form1_Paint(object sender, PaintEventArgs e) {
            DibujarLinea(50, 50, 300, 200, Color.Blue); // Línea de ejemplo
            e.Graphics.DrawImage(Lienzo, 0, 0);
        }

        //Algoritmo de Bresenham para líneas
        private void DibujarLinea(int Xini, int Yini, int Xfin, int Yfin,
            Color color) {
```



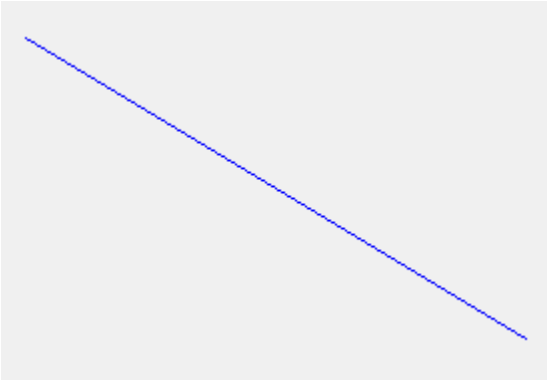
```

int dx = Math.Abs(Xfin - Xini), sx = Xini < Xfin ? 1 : -1;
int dy = -Math.Abs(Yfin - Yini), sy = Yini < Yfin ? 1 : -1;
int error = dx + dy, e2;

while (true) {
    if (Xini >= 0 && Xini < Lienzo.Width && Yini >= 0 && Yini <
Lienzo.Height)
        Lienzo.SetPixel(Xini, Yini, color); //Dibuja el pixel

    if (Xini == Xfin && Yini == Yfin) break;
    e2 = 2 * error;
    if (e2 >= dy) { error += dy; Xini += sx; }
    if (e2 <= dx) { error += dx; Yini += sy; }
}
}
}
}

```



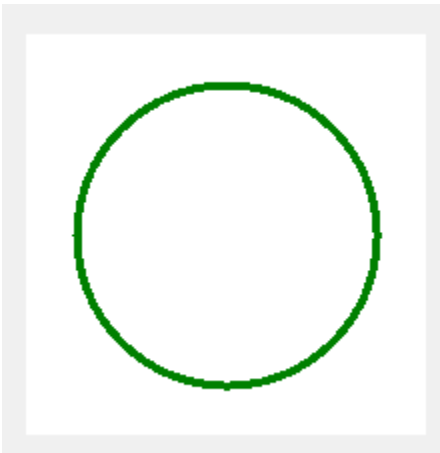
*Ilustración 49: Línea dibujada a nivel de pixel*

# Uso de Bitmap y Graphics

Se puede hacer uso de la API de Graphics dentro de Bitmap.

L/040.cs

```
namespace Graficos {  
  
    public partial class Form1 : Form {  
        private Bitmap Lienzo;  
  
        public Form1() {  
            InitializeComponent();  
  
            //Uso de bitmap para hacer gráficos  
            Lienzo = new Bitmap(200, 200);  
            Graphics Grafico = Graphics.FromImage(Lienzo);  
            Grafico.Clear(Color.White);  
            Pen Lapiz = new(Color.Green, 4);  
            Grafico.DrawEllipse(Lapiz, 25, 25, 150, 150); // Círculo  
        }  
  
        //Pintar  
        private void Form1_Paint(object sender, PaintEventArgs e) {  
            e.Graphics.DrawImage(Lienzo, 50, 50);  
        }  
    }  
}
```



*Ilustración 50: Uso de Bitmap y Graphics*

## Uso de LockBits para mayor velocidad

El código es más complejo y largo pero trae como beneficio la velocidad.

L/041.cs

```
using System.Drawing.Imaging;
using System.Runtime.InteropServices;

namespace Graficos {

    public partial class Form1 : Form {
        private Bitmap Lienzo;

        public Form1() {
            Lienzo = new Bitmap(400, 300, PixelFormat.Format24bppRgb);
            InitializeComponent();
        }

        //Pintar
        private void Form1_Paint(object sender, PaintEventArgs e) {
            DibujaLineaUsandoLockBits(100, 100, 301, 201, Color.Red);
            e.Graphics.DrawImage(Lienzo, 0, 0);
        }

        /*
         * LockBits bloquea una parte del Bitmap en memoria para que pueda
         acceder directamente a los datos de píxeles
         * mediante punteros o arrays. Esto es mucho más rápido que usar
         GetPixel y SetPixel, que son lentos porque
         * implican muchas llamadas a funciones nativas.
         *
         * Usa LockBits cuando:
         * Es necesario modificar muchos píxeles rápidamente (por ejemplo,
         filtros de imagen, efectos, procesamiento de imágenes).
         * Se está haciendo animaciones por píxel o manipulación de texturas.
         * Se desea leer o escribir datos de píxeles en bloque.
         * */
        private void DibujaLineaUsandoLockBits(int Xini, int Yini, int Xfin,
        int Yfin, Color color) {
            Rectangle Rectangulo = new(0, 0, Lienzo.Width, Lienzo.Height);
            BitmapData Datos = Lienzo.LockBits(Rectangulo,
            ImageLockMode.ReadWrite, Lienzo.PixelFormat);

            int bytesPorPixel = Image.GetPixelFormatSize(Lienzo.PixelFormat) /
            8;
        }
    }
}
```

```

int Paso = Datos.Stride;
IntPtr ptr = Datos.Scan0;
int Alto = Lienzo.Height;
int Ancho = Lienzo.Width;

byte[] Pixeles = new byte[Paso * Alto];
Marshal.Copy(ptr, Pixeles, 0, Pixeles.Length);

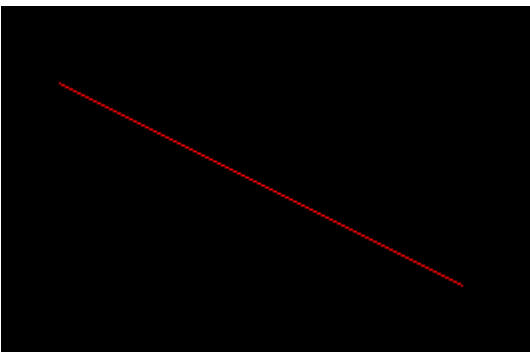
// Algoritmo de Bresenham
int dx = Math.Abs(Xfin - Xini), sx = Xini < Xfin ? 1 : -1;
int dy = -Math.Abs(Yfin - Yini), sy = Yini < Yfin ? 1 : -1;
int Error = dx + dy, Error2;

while (true) {
    if (Xini >= 0 && Xini < Ancho && Yini >= 0 && Yini < Alto) {
        int Indice = Yini * Paso + Xini * bytesPorPixel;
        Pixeles[Indice + 0] = color.B;
        Pixeles[Indice + 1] = color.G;
        Pixeles[Indice + 2] = color.R;
    }

    if (Xini == Xfin && Yini == Yfin) break;
    Error2 = 2 * Error;
    if (Error2 >= dy) { Error += dy; Xini += sx; }
    if (Error2 <= dx) { Error += dx; Yini += sy; }
}

Marshal.Copy(Pixeles, 0, ptr, Pixeles.Length);
Lienzo.UnlockBits(Datos);
}
}
}

```



*Ilustración 51: Uso de LockBits*

# LockBits dibujando un círculo

L/042.cs

```
/*
Si se está dibujando formas geométricas, usar Graphics.
Es más simple, legible y suficientemente rápido para
la mayoría de los casos.

Si se necesita modificar píxeles directamente
(como en procesamiento de imágenes), entonces
usar LockBits.
*/
using System.Drawing.Imaging;
using System.Runtime.InteropServices;

namespace Graficos {
    public partial class Form1 : Form {
        private Bitmap Lienzo;

        public Form1() {
            InitializeComponent();

            // Crear el bitmap
            Lienzo = new Bitmap(200, 200, PixelFormat.Format24bppRgb);

            // Usar LockBits para dibujar un círculo
            Rectangle rectangulo = new Rectangle(0, 0, Lienzo.Width, Lienzo.Height);
            BitmapData bmpDato = Lienzo.LockBits(rectangulo, ImageLockMode.ReadWrite,
            Lienzo.PixelFormat);

            int paso = bmpDato.Stride;
            IntPtr ptr = bmpDato.Scan0;
            int bytes = Math.Abs(paso) * Lienzo.Height;
            byte[] rgbValues = new byte[bytes];

            Marshal.Copy(ptr, rgbValues, 0, bytes);

            int centroX = 100;
            int centroY = 100;
            int radio = 75;

            for (int y = 0; y < Lienzo.Height; y++) {
                for (int x = 0; x < Lienzo.Width; x++) {
                    int dx = x - centroX;
                    int dy = y - centroY;
                    int distancia = dx * dx + dy * dy;
```

```

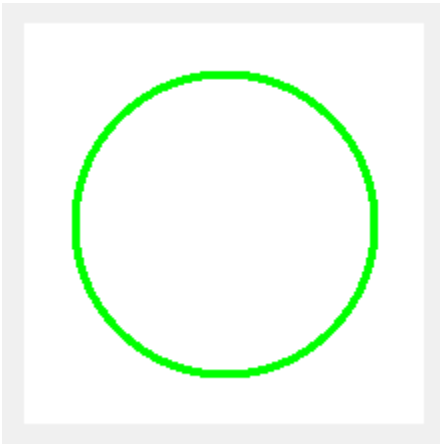
        // Dibujar solo el borde del círculo (ancho de 4 píxeles)
        if (distancia >= radio * radio - 4 * radio && distancia <= radio * radio + 4 *
radio) {

            int indice = y * paso + x * 3;
            rgbValues[indice] = 0;        // Blue
            rgbValues[indice + 1] = 255; // Green
            rgbValues[indice + 2] = 0;    // Red
        }
        else {
            int indice = y * paso + x * 3;
            rgbValues[indice] = 255;
            rgbValues[indice + 1] = 255;
            rgbValues[indice + 2] = 255;
        }
    }
}

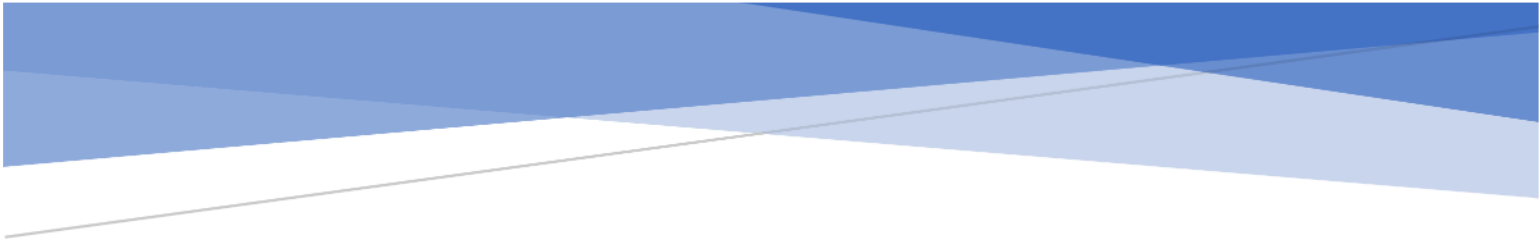
Marshal.Copy(rgbValues, 0, ptr, bytes);
Lienzo.UnlockBits(bmpDato);
}

private void Form1_Paint(object sender, PaintEventArgs e) {
    e.Graphics.DrawImage(Lienzo, 50, 50);
}
}
}

```



*Ilustración 52: Dibujando un círculo con LockBits*



# C# Y .NET 10

## Parte 13. Gráficos 3D

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Tabla de ilustraciones.....                                                    | 3  |
| Acerca del autor.....                                                          | 4  |
| Licencia de este libro .....                                                   | 4  |
| Licencia del software .....                                                    | 4  |
| Marcas registradas .....                                                       | 5  |
| En memoria .....                                                               | 6  |
| Proyección 3D a 2D .....                                                       | 7  |
| Girando un objeto 3D y mostrarlo en 2D .....                                   | 12 |
| Matriz de Giro en X.....                                                       | 12 |
| Matriz de Giro en Y .....                                                      | 13 |
| Matriz de Giro en Z.....                                                       | 14 |
| Aplicación .....                                                               | 15 |
| Giro centrado: Cálculo de constantes.....                                      | 20 |
| Giro centrado: Uso de las constantes calculadas previamente.....               | 24 |
| Giro centrado: Icosaedro .....                                                 | 30 |
| Combinando los tres giros: Cálculo de constantes.....                          | 36 |
| Combinando los tres giros: Uso de constantes .....                             | 39 |
| Al interior de un cubo .....                                                   | 43 |
| Líneas ocultas.....                                                            | 48 |
| Líneas ocultas. Optimización del código.....                                   | 54 |
| Gráfico Matemático en 3D.....                                                  | 58 |
| Gráfico Matemático en 3D. Colores según altura y evaluador de expresiones..... | 67 |
| Gráfico Polar en 3D .....                                                      | 83 |
| Gráfico de sólido de revolución .....                                          | 90 |



## Tabla de ilustraciones

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Ilustración 1: Proyección de objeto 3D en pantalla 2D .....                           | 7  |
| Ilustración 2: Esquema de proyección.....                                             | 8  |
| Ilustración 3: Proyección 3D a 2D de una figura tridimensional: un cubo .....         | 11 |
| Ilustración 4: Giro figura en 3D .....                                                | 18 |
| Ilustración 5: Giro en el eje X de la figura 3D. No hay giro en los otros ejes.....   | 18 |
| Ilustración 6: Giro en el eje Y de la figura 3D. No hay giro en los otros ejes.....   | 19 |
| Ilustración 7: Giro en el eje Z de la figura 3D. No hay giro en los otros ejes.....   | 19 |
| Ilustración 8: Constantes para después calcular los puntos en pantalla .....          | 22 |
| Ilustración 9: Cubo proyectado por defecto .....                                      | 27 |
| Ilustración 10: Cubo proyectado con giro en X .....                                   | 28 |
| Ilustración 11: Cubo proyectado con giro en Y .....                                   | 28 |
| Ilustración 12: Cubo proyectado con giro en Z .....                                   | 29 |
| Ilustración 13: Icosaedro por defecto .....                                           | 34 |
| Ilustración 14: Giro en X .....                                                       | 34 |
| Ilustración 15: Giro en Y .....                                                       | 35 |
| Ilustración 16: Giro en Z .....                                                       | 35 |
| Ilustración 17: Constantes para cuadrar el cubo en la pantalla.....                   | 38 |
| Ilustración 18: Cubo proyectado con giros.....                                        | 42 |
| Ilustración 19: El icosaedro está dentro del cubo.....                                | 47 |
| Ilustración 20: Se gira el cubo, entonces gira el icosaedro. Todo pre-calculado. .... | 47 |
| Ilustración 21: Icosaedro pintado con líneas ocultas.....                             | 52 |
| Ilustración 22: Icosaedro pintado con líneas ocultas.....                             | 53 |
| Ilustración 23: Ecuación en 3D proyectada .....                                       | 61 |
| Ilustración 24: Ecuación en 3D proyectada .....                                       | 62 |
| Ilustración 25: Gráfico ecuación 3D con colores según altura.....                     | 81 |
| Ilustración 26: Gráfico ecuación 3D con colores según altura.....                     | 82 |
| Ilustración 27: Gráfico polar 3D.....                                                 | 89 |
| Ilustración 28: Gráfico de sólido de revolución .....                                 | 96 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL "Lesser General Public License" [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

En memoria

En memoria de mi gata Capuchina.



Proyección 3D a 2D

Dado un objeto tridimensional que flota entre la persona y una pantalla plana, ¿Cómo se proyectaría este objeto tridimensional en la pantalla? Ese objeto tiene coordenadas (X, Y, Z) y deben convertirse a coordenadas planas (Xplano, Yplano). Hay que considerar la distancia de la persona u observador a ese objeto (Zpersona). Si el observador está muy cerca del objeto 3D, lo verá grande, si se aleja el observador, lo verá pequeño.



Ilustración 1: Proyección de objeto 3D en pantalla 2D

Las ecuaciones para pasar coordenadas (X, Y, Z) a coordenadas planas Xplano, Yplano considerando la distancia del observador (ZPersona) es:

$$X_{plano} = \frac{Z_{Persona} * X}{(Z_{Persona} - Z)}$$
$$Y_{plano} = \frac{Z_{Persona} * Y}{(Z_{Persona} - Z)}$$

Demostración:

**Paso 1**

Se pone el valor de  $X = 0$  para ver sólo el comportamiento de  $Y$ , este sería el gráfico

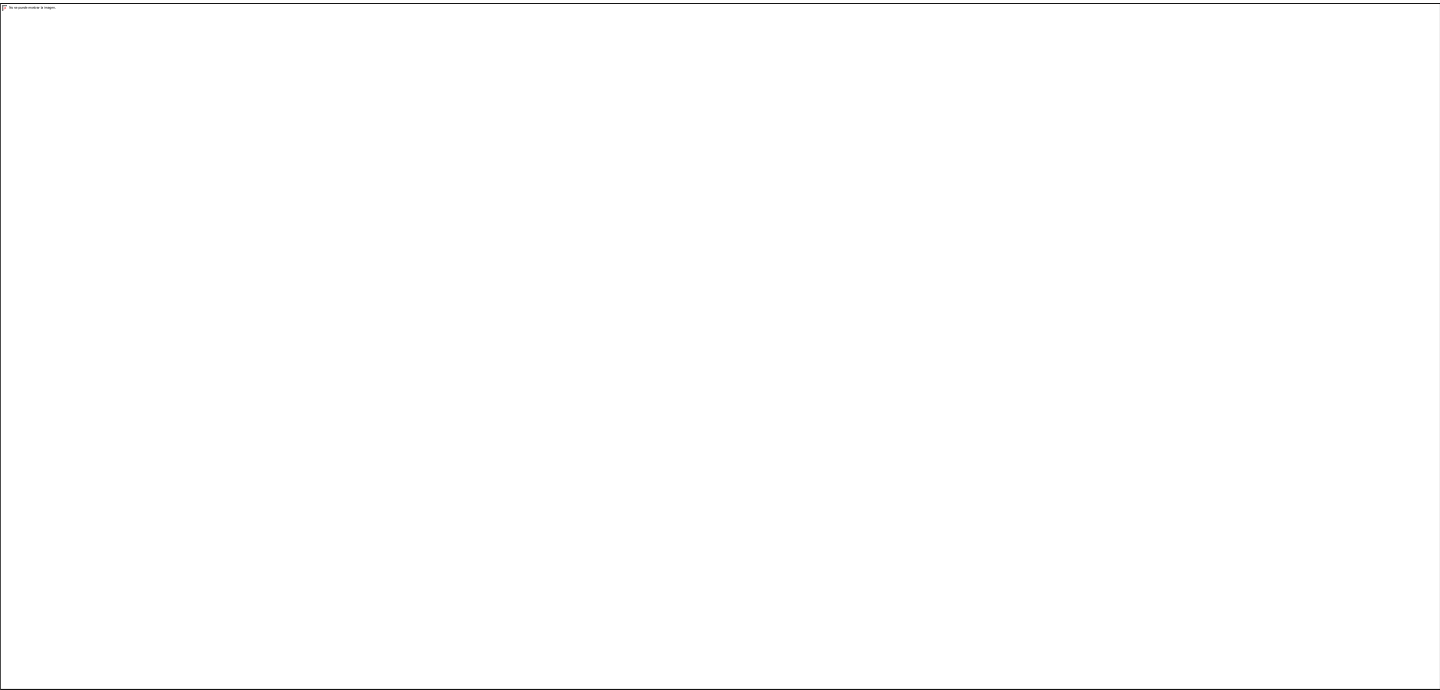


Ilustración 2: Esquema de proyección

**Paso 2**

Es conocido entonces los valores de  $Y$ ,  $Z$  y  $Z_{persona}$ . Se necesita hallar el valor de  $i^?$  Que sería  $Y_{plano}$ , ¿Cómo? La respuesta está en el ángulo  $\theta$ , más concretamente en la tangente que sería así:

$$\tan \theta = \frac{Y}{Z_{persona} - Z}$$

Pero también se sabe que:

$$\tan \theta = \frac{i^?}{Z_{persona}}$$

Igualando:

$$\frac{i^?}{Z_{persona}} = \frac{Y}{Z_{persona} - Z}$$

Despejando:

$$i^? = \frac{Z_{persona} * Y}{Z_{persona} - Z}$$

Luego:

$$Y_{plano} = \frac{Z_{persona} * Y}{Z_{persona} - Z}$$

Igualmente, para  $X_{plano}$  sería:

$$X_{plano} = \frac{Z_{persona} * X}{Z_{persona} - Z}$$

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private int X, Y, Z; //Coordenadas originales
        public int PlanoX, PlanoY; //Proyección o sombra

        public Punto(int X, int Y, int Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(int ZPersona) {
            PlanoX = X * ZPersona / (ZPersona - Z);
            PlanoY = Y * ZPersona / (ZPersona - Z);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}
```

```

namespace Graficos {
    internal class Cubo {
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Un cubo tiene 12 líneas de conexión
        private List<Conexion> conexiones;

        //Constructor
        public Cubo(int ZPersona) {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = new List<Punto> {
                new Punto(50, 50, 50),
                new Punto(100, 50, 50),
                new Punto(100, 100, 50),
                new Punto(50, 100, 50),
                new Punto(50, 50, 100),
                new Punto(100, 50, 100),
                new Punto(100, 100, 100),
                new Punto(50, 100, 100)
            };

            //Las 12 líneas para dibujar el cubo
            //punto inicial ---> punto final
            conexiones = new List<Conexion> {
                new Conexion(0, 1),
                new Conexion(1, 2),
                new Conexion(2, 3),
                new Conexion(3, 0),
                new Conexion(4, 5),
                new Conexion(5, 6),
                new Conexion(6, 7),
                new Conexion(7, 4),
                new Conexion(0, 4),
                new Conexion(1, 5),
                new Conexion(2, 6),
                new Conexion(3, 7)
            };

            //Convierte de 3D a 2D
            for (int Cont = 0; Cont < puntos.Count; Cont++)
                puntos[Cont].Proyecta(ZPersona);
        }

        //Dibuja el cubo
        public void Dibuja(Graphics lienzo, Pen lapiz) {
            for (int Cont = 0; Cont < conexiones.Count; Cont++) {
                int Inicio = conexiones[Cont].punto1;
                int Final = conexiones[Cont].punto2;
                lienzo.DrawLine(lapiz, puntos[Inicio].PlanoX, puntos[Inicio].PlanoY,
                    puntos[Final].PlanoX, puntos[Final].PlanoY);
            }
        }
    }
}

```



```

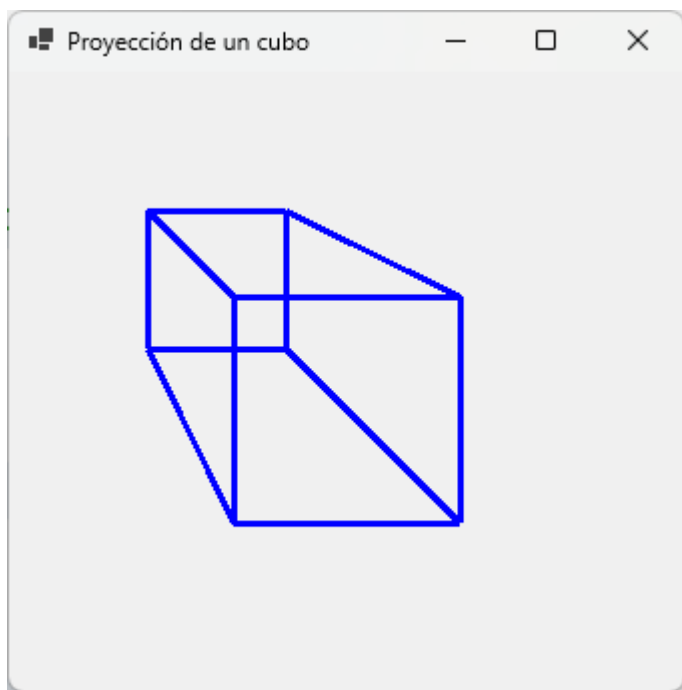
namespace Graficos {
    //Proyección 3D a 2D de una figura tridimensional: un cubo
    public partial class Form1 : Form {
        public Form1() {
            InitializeComponent();
        }

        //Pinta la proyección
        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Distancia de la persona (observador) al plano donde se
            //proyecta la "sombra" del cubo
            int ZPersona = 180;

            //Crea el objeto tridimensional
            Cubo Figura3D = new Cubo(ZPersona);

            //Dibuja el objeto tridimensional
            Graphics Lienzo = e.Graphics;
            Pen Lapis = new Pen(Color.Blue, 3);
            Figura3D.Dibuja(Lienzo, Lapis);
        }
    }
}

```



*Ilustración 3: Proyección 3D a 2D de una figura tridimensional: un cubo*

## Girando un objeto 3D y mostrarlo en 2D

Dada una coordenada tridimensional (X, Y, Z) se quiere aplicar un giro en un determinado ángulo. Ese ángulo puede ser en X o en Y o en Z. Para aplicar el giro, se requiere una matriz de transformación. Estas serían las matrices:

### Matriz de Giro en X

$$\begin{bmatrix} X_{giro} \\ Y_{giro} \\ Z_{giro} \end{bmatrix} = [X \quad Y \quad Z] * \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(angX) & \text{sen}(angX) \\ 0 & -\text{sen}(angX) & \cos(angX) \end{bmatrix}$$

Aplicando:

$$X_{giro} = X * 1 + Y * 0 + Z * 0$$

$$Y_{giro} = X * 0 + Y * \cos(angX) - Z * \text{sen}(angX)$$

$$Z_{giro} = X * 0 + Y * \text{sen}(angX) + Z * \cos(angX)$$

Simplificando:

$$X_{giro} = X$$

$$Y_{giro} = Y * \cos(angX) - Z * \text{sen}(angX)$$

$$Z_{giro} = Y * \text{sen}(angX) + Z * \cos(angX)$$

$$\begin{bmatrix} X_{giro} \\ Y_{giro} \\ Z_{giro} \end{bmatrix} = [X \quad Y \quad Z] * \begin{bmatrix} \cos(angY) & 0 & -\text{sen}(angY) \\ 0 & 1 & 0 \\ \text{sen}(angY) & 0 & \cos(angY) \end{bmatrix}$$

Aplicando:

$$X_{giro} = X * \cos(angY) + Y * 0 + Z * \text{sen}(angY)$$

$$Y_{giro} = X * 0 + Y * 1 + Z * 0$$

$$Z_{giro} = -X * \text{sen}(angY) + Y * 0 + Z * \cos(angY)$$

Simplificando:

$$X_{giro} = X * \cos(angY) + Z * \text{sen}(angY)$$

$$Y_{giro} = Y$$

$$Z_{giro} = -X * \text{sen}(angY) + Z * \cos(angY)$$

$$\begin{bmatrix} X_{giro} \\ Y_{giro} \\ Z_{giro} \end{bmatrix} = [X \quad Y \quad Z] * \begin{bmatrix} \cos(angZ) & \sin(angZ) & 0 \\ -\sin(angZ) & \cos(angZ) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Aplicando:

$$X_{giro} = X * \cos(angZ) - Y * \sin(angZ) + Z * 0$$

$$Y_{giro} = X * \sin(angZ) + Y * \cos(angZ) + Z * 0$$

$$Z_{giro} = X * 0 + Y * 0 + Z * 1$$

Simplificando:

$$X_{giro} = X * \cos(angZ) - Y * \sin(angZ)$$

$$Y_{giro} = X * \sin(angZ) + Y * \cos(angZ)$$

$$Z_{giro} = Z$$

Más información en: [https://www.cs.buap.mx/~iolmos/graficacion/5\\_Transformaciones\\_geometricas\\_3D.pdf](https://www.cs.buap.mx/~iolmos/graficacion/5_Transformaciones_geometricas_3D.pdf)

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        private double Xg, Yg, Zg; //Coordenadas al girar
        public int PlanoX, PlanoY; //Proyección o sombra

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(double ZPersona) {
            PlanoX = Convert.ToInt32(Xg * ZPersona / (ZPersona - Zg));
            PlanoY = Convert.ToInt32(Yg * ZPersona / (ZPersona - Zg));
        }

        //Gira en X
        public void GiroX(double angulo) {
            double[,] Mt = new double[3, 3] {
                {1, 0, 0},
                {0, Math.Cos(angulo), Math.Sin(angulo)},
                {0, -Math.Sin(angulo), Math.Cos(angulo)}
            };

            Xg = X * Mt[0, 0] + Y * Mt[1, 0] + Z * Mt[2, 0];
            Yg = X * Mt[0, 1] + Y * Mt[1, 1] + Z * Mt[2, 1];
            Zg = X * Mt[0, 2] + Y * Mt[1, 2] + Z * Mt[2, 2];
        }

        //Gira en Y
        public void GiroY(double angulo) {
            double[,] Mt = new double[3, 3] {
                {Math.Cos(angulo), 0, -Math.Sin(angulo)},
                {0, 1, 0},
                {Math.Sin(angulo), 0, Math.Cos(angulo)}
            };

            Xg = X * Mt[0, 0] + Y * Mt[1, 0] + Z * Mt[2, 0];
            Yg = X * Mt[0, 1] + Y * Mt[1, 1] + Z * Mt[2, 1];
            Zg = X * Mt[0, 2] + Y * Mt[1, 2] + Z * Mt[2, 2];
        }

        //Gira en Z
        public void GiroZ(double angulo) {
            double[,] Mt = new double[3, 3] {
                {Math.Cos(angulo), Math.Sin(angulo), 0},
                {-Math.Sin(angulo), Math.Cos(angulo), 0},
                {0, 0, 1}
            };

            Xg = X * Mt[0, 0] + Y * Mt[1, 0] + Z * Mt[2, 0];
            Yg = X * Mt[0, 1] + Y * Mt[1, 1] + Z * Mt[2, 1];
            Zg = X * Mt[0, 2] + Y * Mt[1, 2] + Z * Mt[2, 2];
        }
    }
}
```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    internal class Cubo {
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z
        public List<Punto> puntos;

        //Un cubo tiene 12 líneas de conexión
        private List<Conexion> conexiones;

        //Constructor
        public Cubo() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto(50, 50, 50),
                new Punto(100, 50, 50),
                new Punto(100, 100, 50),
                new Punto(50, 100, 50),
                new Punto(50, 50, 100),
                new Punto(100, 50, 100),
                new Punto(100, 100, 100),
                new Punto(50, 100, 100)
            ];

            //Las 12 líneas para dibujar el cubo
            //punto inicial ---> punto final
            conexiones = [
                new Conexion(0, 1),
                new Conexion(1, 2),
                new Conexion(2, 3),
                new Conexion(3, 0),
                new Conexion(4, 5),
                new Conexion(5, 6),
                new Conexion(6, 7),
                new Conexion(7, 4),
                new Conexion(0, 4),
                new Conexion(1, 5),
                new Conexion(2, 6),
                new Conexion(3, 7)
            ];
        }

        public void AplicaGiro(int CualAnguloGira, int ValorAngulo, int ZPersona) {
            double ang = ValorAngulo * Math.PI / 180;

            //Dependiendo de cuál ángulo gira
            for (int Cont = 0; Cont < puntos.Count; Cont++) {
                switch (CualAnguloGira) {
                    case 0: puntos[Cont].GiroX(ang); break;
                    case 1: puntos[Cont].GiroY(ang); break;
                    case 2: puntos[Cont].GiroZ(ang); break;
                }
                puntos[Cont].Proyecta(ZPersona);
            }
        }

        //Dibuja el cubo
        public void Dibuja(Graphics lienzo, Pen lapiz) {
            for (int Cont = 0; Cont < conexiones.Count; Cont++) {
                int Inicio = conexiones[Cont].punto1;
                int Final = conexiones[Cont].punto2;
                lienzo.DrawLine(lapiz, puntos[Inicio].PlanoX, puntos[Inicio].PlanoY,

```



```

        puntos[Final].PlanoX, puntos[Final].PlanoY);
    }
}
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D. Uso de giros en X, Y, Z pero tan sólo en uno

    public partial class Form1 : Form {
        //El cubo que se proyecta y gira
        Cubo Figura3D;
        int ZPersona = 180;

        public Form1() {
            InitializeComponent();

            Figura3D = new Cubo();
            Figura3D.AplicaGiro(0, 0, ZPersona);
        }

        private void numGiroX_ValueChanged(object sender, System.EventArgs e) {
            //Se anulan los dos valores de los otros ángulos
            numGiroY.Value = 0;
            numGiroZ.Value = 0;

            //Sólo puede girar en un ángulo
            int AnguloX = Convert.ToInt32(numGiroX.Value);
            Figura3D.AplicaGiro(0, AnguloX, ZPersona); //0 es giro en X
            Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            numGiroX.Value = 0;
            numGiroZ.Value = 0;
            int AnguloY = Convert.ToInt32(numGiroY.Value);
            Figura3D.AplicaGiro(1, AnguloY, ZPersona); //1 es giro en Y
            Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            numGiroX.Value = 0;
            numGiroY.Value = 0;
            int AnguloZ = Convert.ToInt32(numGiroZ.Value);
            Figura3D.AplicaGiro(2, AnguloZ, ZPersona); //2 es giro en Z
            Refresh();
        }

        //Pinta la proyección
        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics Lienzo = e.Graphics;
            Pen Lapis = new(Color.Blue, 3);
            Figura3D.Dibuja(Lienzo, Lapis);

            //Dibuja la línea de referencia del giro que parte de 0,0
            Pen Lapis2 = new(Color.Red, 3);
            Lienzo.DrawLine(Lapis2, 0, 0, Figura3D.puntos[0].PlanoX, Figura3D.puntos[0].PlanoY);
        }
    }
}

```

Ejemplo de ejecución:

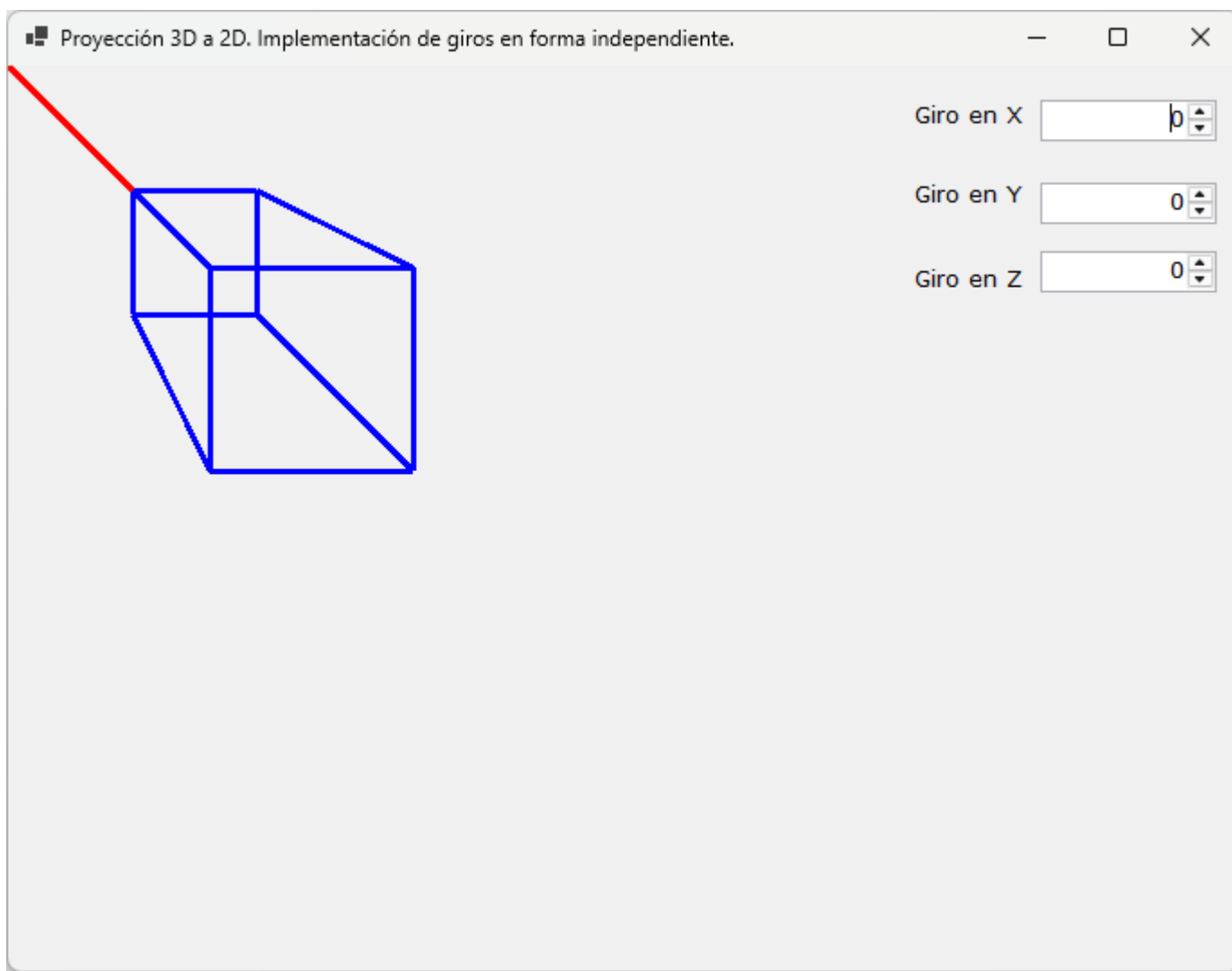


Ilustración 4: Giro figura en 3D

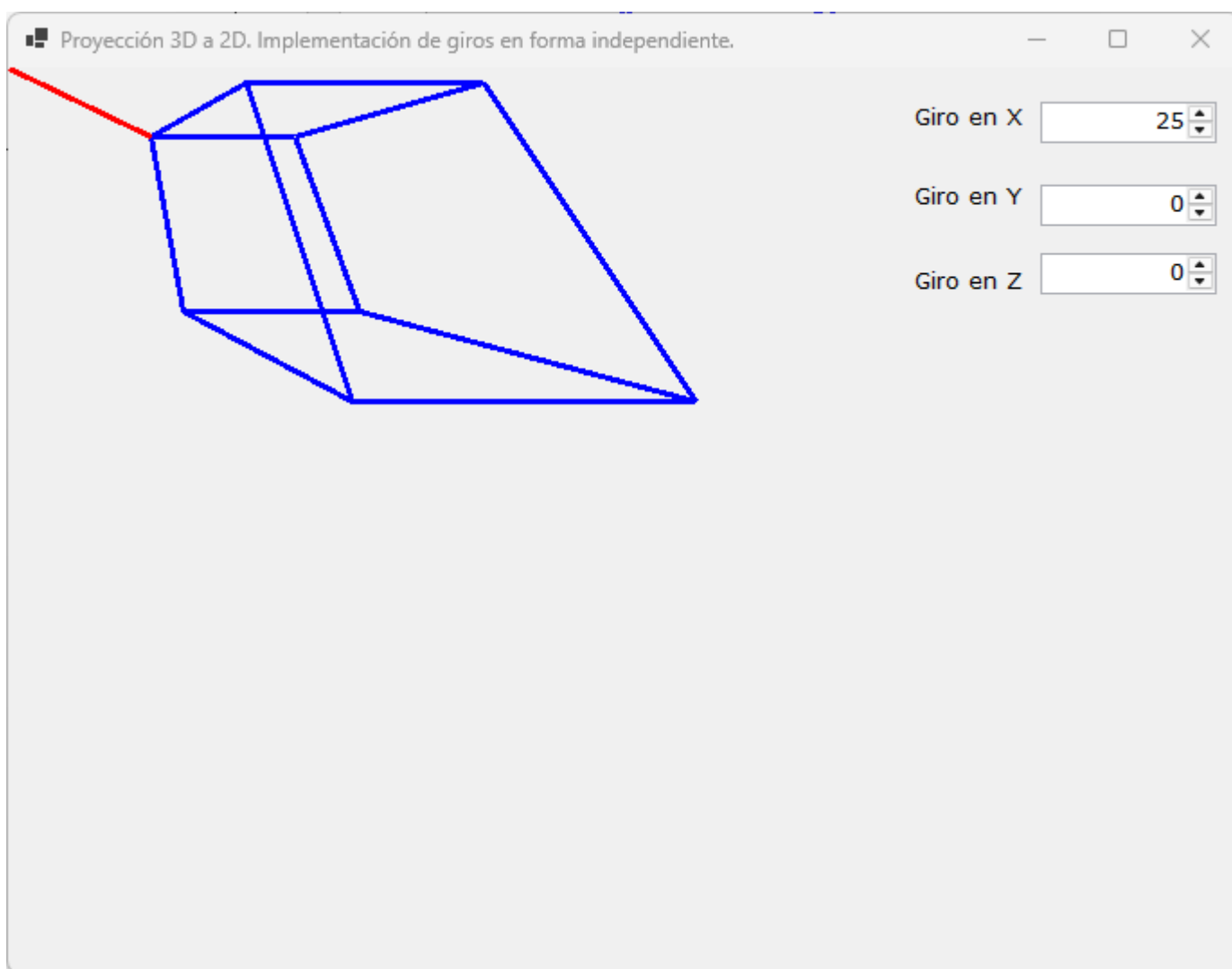


Ilustración 5: Giro en el eje X de la figura 3D. No hay giro en los otros ejes.



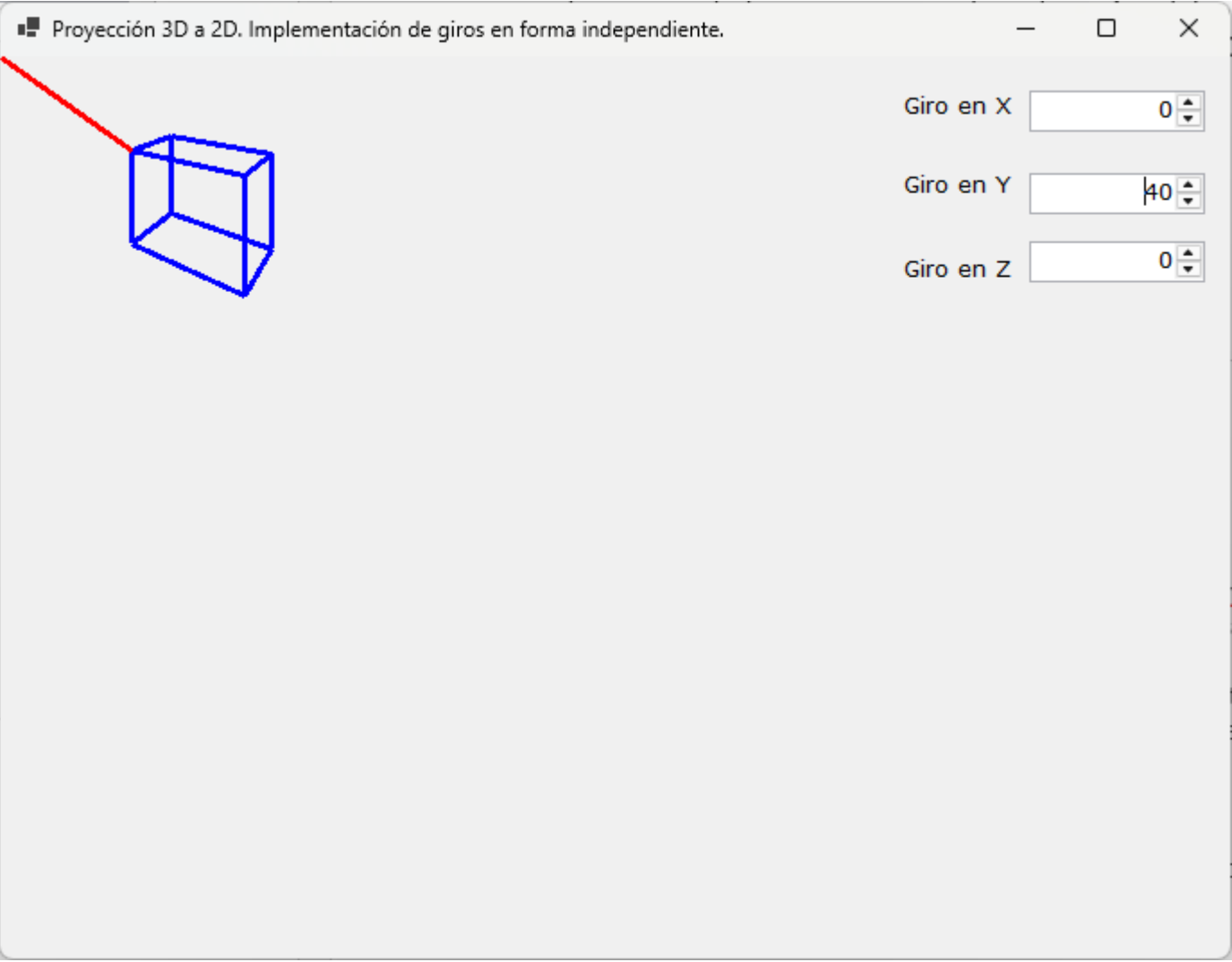


Ilustración 6: Giro en el eje Y de la figura 3D. No hay giro en los otros ejes.

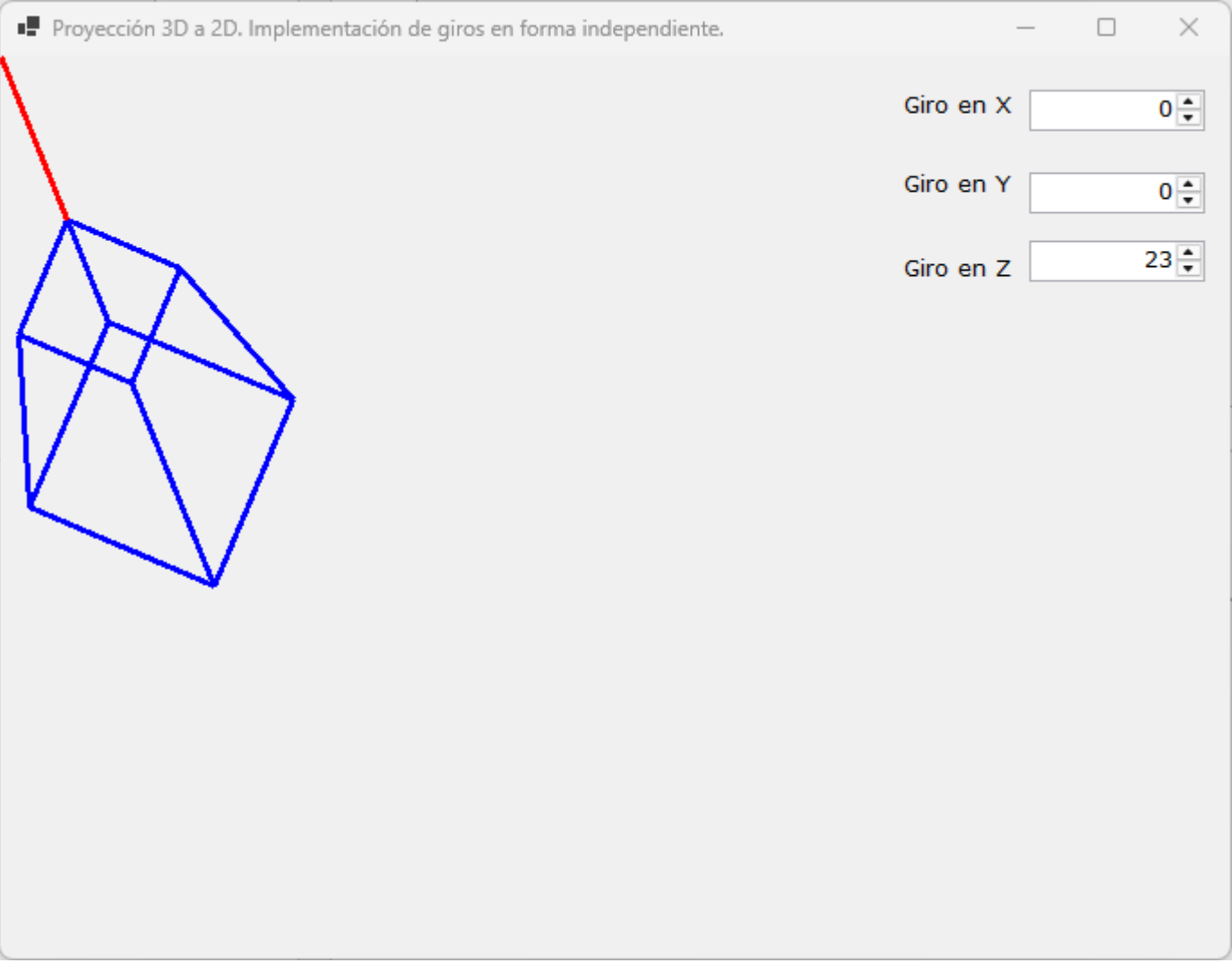


Ilustración 7: Giro en el eje Z de la figura 3D. No hay giro en los otros ejes.

La línea roja representa desde donde se hace el giro, en el punto (0,0,0). Por ese motivo, la figura no solo gira sino que se desplaza.

## Giro centrado: Cálculo de constantes

En el ejemplo anterior, el punto (0,0,0) se encuentra en la parte superior izquierda de la ventana, el cubo está a un lado, no en el centro, por lo que girar el cubo también implica desplazarlo. Llega a un punto que se sale de la ventana y no se puede ver. Se hace un cambio al cubo para que su centroide esté en la posición (0,0,0), además de mejorar la proyección en pantalla.

Pero aparece un problema: al girar el cubo, en ciertos ángulos, su proyección queda por fuera de la ventana. ¿Cómo evitar esto? Habría que calcular para cada punto del cubo, su proyección girando desde 0 a 360 grados, tanto en X como en Y como en Z.

Se sabe que el cubo tiene coordenadas de -0.5 a 0.5 (su lado mide 1) tanto en X, Y, y Z. Se deja fija la distancia del observador, en este caso con un valor de 5 (suficientemente alejado de la figura). Con esos valores entonces se probaron todos los ángulos de giro en X de 0 a 360 grados, en Y de 0 a 360 grados y en Z de 0 a 360 grados, así:

Desde ángulo en X = 0 grados hasta 360 grados ➔ Calcule X plano mínimo, Y plano mínimo, X plano máximo, Y plano máximo

Desde ángulo en Y = 0 grados hasta 360 grados ➔ Calcule X plano mínimo, Y plano mínimo, X plano máximo, Y plano máximo

Desde ángulo en Z = 0 grados hasta 360 grados ➔ Calcule X plano mínimo, Y plano mínimo, X plano máximo, Y plano máximo

Con eso se obtienen los valores X plano mínimo, Y plano mínimo, X plano máximo, Y plano máximo. Estos valores son del plano matemático.

M/003.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        public double PlanoX, PlanoY; //Proyección o sombra

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Gira en X
        public void GiroX(double angulo) {
            double ang = angulo * Math.PI / 180;

            double[,] Matriz = new double[3, 3] {
                {1, 0, 0},
                {0, Math.Cos(ang), Math.Sin(ang)},
                {0, -Math.Sin(ang), Math.Cos(ang)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Y
        public void GiroY(double angulo) {
            double ang = angulo * Math.PI / 180;

            double[,] Matriz = new double[3, 3] {
                {Math.Cos(ang), 0, -Math.Sin(ang)},
                {0, 1, 0},
                {Math.Sin(ang), 0, Math.Cos(ang)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Z
        public void GiroZ(double angulo) {
            double ang = angulo * Math.PI / 180;

            double[,] Matriz = new double[3, 3] {
                {Math.Cos(ang), Math.Sin(ang), 0},
                {-Math.Sin(ang), Math.Cos(ang), 0},
                {0, 0, 1}
            };

            AplicaMatrizGiro(Matriz);
        }
    }
}
```

```

//Aplica la matriz de giro
private void AplicaMatrizGiro(double[,] Matriz) {
    //Hace el giro
    Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
    Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
    Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
}

//Convierte de 3D a 2D (segunda dimensión)
public void Proyecta(int ZPersona, ref double minimoX, ref double maximoX, ref double minimoY, ref double maximoY) {
    PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
    PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

    if (PlanoX < minimoX) minimoX = PlanoX;
    if (PlanoX > maximoX) maximoX = PlanoX;
    if (PlanoY < minimoY) minimoY = PlanoY;
    if (PlanoY > maximoY) maximoY = PlanoY;
}
}
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    internal class Cubo {
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Un cubo tiene 12 líneas de conexión
        private List<Conexion> conexiones;

        //Constructor
        public Cubo() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto(-0.5, -0.5, -0.5),
                new Punto(0.5, -0.5, -0.5),
                new Punto(0.5, 0.5, -0.5),
                new Punto(-0.5, 0.5, -0.5),
                new Punto(-0.5, -0.5, 0.5),
                new Punto(0.5, -0.5, 0.5),
                new Punto(0.5, 0.5, 0.5),
                new Punto(-0.5, 0.5, 0.5)
            ];

            //Las 12 líneas para dibujar el cubo
            //punto inicial ---> punto final
            conexiones = [
                new Conexion(0, 1),
                new Conexion(1, 2),
                new Conexion(2, 3),
                new Conexion(3, 0),
                new Conexion(4, 5),
                new Conexion(5, 6),
                new Conexion(6, 7),
                new Conexion(7, 4),
                new Conexion(0, 4),
                new Conexion(1, 5),
                new Conexion(2, 6),

```

```

        new Conexion(3, 7)
    ];
}

//Calcula los extremos de las coordenadas del cubo al girar y proyectarse
public void CalculaExtremo(int ZPersona) {
    double maximoX = double.MinValue;
    double minimoX = double.MaxValue;
    double maximoY = double.MinValue;
    double minimoY = double.MaxValue;

    //Va de punto en punto
    for (int Cont = 0; Cont < puntos.Count; Cont++) {

        //Va de angulo en angulo
        for (double angulo = 0; angulo <= 360; angulo++) {
            puntos[Cont].GiroX(angulo);
            puntos[Cont].Proyecta(ZPersona, ref minimoX, ref maximoX, ref minimoY, ref maximoY);

            puntos[Cont].GiroY(angulo);
            puntos[Cont].Proyecta(ZPersona, ref minimoX, ref maximoX, ref minimoY, ref maximoY);

            puntos[Cont].GiroZ(angulo);
            puntos[Cont].Proyecta(ZPersona, ref minimoX, ref maximoX, ref minimoY, ref maximoY);
        }
    }

    Console.WriteLine("Extremos son:");
    Console.WriteLine("MinimoX: " + minimoX.ToString());
    Console.WriteLine("MaximoX: " + maximoX.ToString());
    Console.WriteLine("MinimoY: " + minimoY.ToString());
    Console.WriteLine("MaximoY: " + maximoY.ToString());
}
}
}

```

```

namespace Graficos
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Cubo Figura3D = new();
            Figura3D.CalculaExtremo(5);
        }
    }
}

```

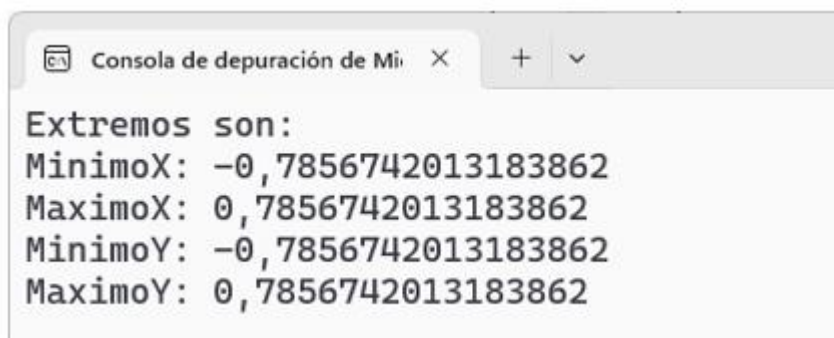


Ilustración 8: Constantes para después calcular los puntos en pantalla

Volviendo a las ecuaciones:

$$\text{convierteX} = (\text{XpantallaFin} - \text{XpantallaIni}) / (\text{MaximoX} - \text{MinimoX})$$
$$\text{convierteY} = (\text{YpantallaFin} - \text{YpantallaIni}) / (\text{MaximoY} - \text{MinimoY})$$

Entonces:

$$\text{convierteX} = (\text{XpantallaFin} - \text{XpantallaIni}) / (0,7856742013183862 - -0,7856742013183862)$$
$$\text{convierteY} = (\text{YpantallaFin} - \text{YpantallaIni}) / (0,7856742013183862 - -0,7856742013183862)$$

Es decir:

$$\text{convierteX} = (\text{XpantallaFin} - \text{XpantallaIni}) / 1,57134840263677$$
$$\text{convierteY} = (\text{YpantallaFin} - \text{YpantallaIni}) / 1,57134840263677$$

Esas constantes son las que se usan para generar las coordenadas en pantalla física.

Nota 1: ¡OJO! Si cambia la distancia del observador al plano o cambia el tamaño de la figura 3D habría que recalcular nuevas constantes.

Nota 2: Esas constantes se usarán así:

```
//Los valores extremos de las coordenadas del cubo
double maximoX = 0.785674201318386;
double minimoX = -0.785674201318386;
double maximoY = 0.785674201318386;
double minimoY = -0.785674201318386;

//Las constantes de transformación
double convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
double convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);
```

## Giro centrado: Uso de las constantes calculadas previamente

Se rehace el programa de proyección del cubo con el uso de constantes calculadas en el programa anterior.

M/004.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        private double PlanoX, PlanoY; //Proyección o sombra
        public int Xpantalla, Ypantalla; //Coordenadas en la pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Gira en X
        public void GiroX(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {1, 0, 0},
                {0, Math.Cos(AnguloRad), Math.Sin(AnguloRad)},
                {0, -Math.Sin(AnguloRad), Math.Cos(AnguloRad)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Y
        public void GiroY(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {Math.Cos(AnguloRad), 0, -Math.Sin(AnguloRad)},
                {0, 1, 0},
                {Math.Sin(AnguloRad), 0, Math.Cos(AnguloRad)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Z
        public void GiroZ(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {Math.Cos(AnguloRad), Math.Sin(AnguloRad), 0},
                {-Math.Sin(AnguloRad), Math.Cos(AnguloRad), 0},
                {0, 0, 1}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Aplica la matriz de giro
        private void AplicaMatrizGiro(double[,] Matriz) {
            //Hace el giro
            Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(int ZPersona) {
            PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);
        }

        //Cuadra en pantalla
        public void CuadraPantalla(double convierteX, double convierteY,
            double minimoX, double minimoY,
            int XpantallaIni, int XpantallaFin,
            int YpantallaIni, int YpantallaFin) {
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX - minimoX) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY - minimoY) + YpantallaIni);
        }
    }
}
```



```
}
```

```
namespace Graficos {  
    //Conecta con una línea recta un punto con otro  
    internal class Conexion {  
        public int punto1, punto2;  
  
        public Conexion(int punto1, int punto2) {  
            this.punto1 = punto1;  
            this.punto2 = punto2;  
        }  
    }  
}
```

```
namespace Graficos {  
    internal class Cubo {  
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z  
        private List<Punto> puntos;  
  
        //Un cubo tiene 12 líneas de conexión  
        private List<Conexion> conexiones;  
  
        //Valores extremos  
        private double minimoX, maximoX, minimoY, maximoY, convierteX, convierteY;  
  
        //Despliegue en pantalla  
        private int XpantallaIni, XpantallaFin, YpantallaIni, YpantallaFin;  
  
        //Distancia del observador  
        private int ZPersona;  
  
        //Constructor  
        public Cubo(int XpantallaIni, int XpantallaFin, int YpantallaIni, int YpantallaFin) {  
            //Ejemplo de coordenadas espaciales X,Y,Z  
            puntos = [  
                new Punto(-0.5, -0.5, -0.5),  
                new Punto(0.5, -0.5, -0.5),  
                new Punto(0.5, 0.5, -0.5),  
                new Punto(-0.5, 0.5, -0.5),  
                new Punto(-0.5, -0.5, 0.5),  
                new Punto(0.5, -0.5, 0.5),  
                new Punto(0.5, 0.5, 0.5),  
                new Punto(-0.5, 0.5, 0.5)  
            ];  
  
            //Las 12 líneas para dibujar el cubo  
            //punto inicial ---> punto final  
            conexiones = [  
                new Conexion(0, 1),  
                new Conexion(1, 2),  
                new Conexion(2, 3),  
                new Conexion(3, 0),  
                new Conexion(4, 5),  
                new Conexion(5, 6),  
                new Conexion(6, 7),  
                new Conexion(7, 4),  
                new Conexion(0, 4),  
                new Conexion(1, 5),  
                new Conexion(2, 6),  
                new Conexion(3, 7)  
            ];  
  
            this.XpantallaIni = XpantallaIni;  
            this.XpantallaFin = XpantallaFin;  
            this.YpantallaIni = YpantallaIni;  
            this.YpantallaFin = YpantallaFin;  
            this.ZPersona = 5;  
  
            //Los valores extremos de las coordenadas del cubo  
            this.maximoX = 0.785674201318386;  
            this.minimoX = -0.785674201318386;
```

```

        this.maximoY = 0.785674201318386;
        this.minimoY = -0.785674201318386;

        //Las constantes de transformación
        convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
        convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);
    }

    public void GiroX(int Angulo) {
        double Radian = Angulo * Math.PI / 180;
        for (int cont = 0; cont < puntos.Count; cont++) {
            puntos[cont].GiroX(Radian);
            puntos[cont].Proyecta(ZPersona);
            puntos[cont].CuadraPantalla(convierteX, convierteY,
                minimoX, minimoY,
                XpantallaIni, XpantallaFin,
                YpantallaIni, YpantallaFin);
        }
    }

    public void GiroY(int Angulo) {
        double Radian = Angulo * Math.PI / 180;
        for (int cont = 0; cont < puntos.Count; cont++) {
            puntos[cont].GiroY(Radian);
            puntos[cont].Proyecta(ZPersona);
            puntos[cont].CuadraPantalla(convierteX, convierteY,
                minimoX, minimoY,
                XpantallaIni, XpantallaFin,
                YpantallaIni, YpantallaFin);
        }
    }

    public void GiroZ(int Angulo) {
        double Radian = Angulo * Math.PI / 180;
        for (int cont = 0; cont < puntos.Count; cont++) {
            puntos[cont].GiroZ(Radian);
            puntos[cont].Proyecta(ZPersona);
            puntos[cont].CuadraPantalla(convierteX, convierteY,
                minimoX, minimoY,
                XpantallaIni, XpantallaFin,
                YpantallaIni, YpantallaFin);
        }
    }

    //Dibuja el cubo
    public void Dibuja(Graphics lienzo, Pen lapiz) {
        for (int Cont = 0; Cont < conexiones.Count; Cont++) {
            int Inicio = conexiones[Cont].punto1;
            int Final = conexiones[Cont].punto2;
            lienzo.DrawLine(lapiz, puntos[Inicio].Xpantalla, puntos[Inicio].Ypantalla,
                puntos[Final].Xpantalla, puntos[Final].Ypantalla);
        }
    }
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada.
    public partial class Form1 : Form {
        //El cubo que se proyecta y gira
        Cubo Figura3D;

        public Form1() {
            InitializeComponent();

            int XpantallaIni = 30, XpantallaFin = 700, YpantallaIni = 30, YpantallaFin = 700;
            Figura3D = new Cubo(XpantallaIni, XpantallaFin, YpantallaIni, YpantallaFin);
            Figura3D.GiroX(0);
        }

        private void numGiroX_ValueChanged(object sender, System.EventArgs e) {
            numGiroY.Value = 0;
            numGiroZ.Value = 0;
            int AnguloX = Convert.ToInt32(numGiroX.Value);

```



```

        Figura3D.GiroX(AnguloX);
        Refresh();
    }

    private void numGiroY_ValueChanged(object sender, EventArgs e) {
        numGiroX.Value = 0;
        numGiroZ.Value = 0;
        int AnguloY = Convert.ToInt32(numGiroY.Value);
        Figura3D.GiroY(AnguloY);
        Refresh();
    }

    private void numGiroZ_ValueChanged(object sender, EventArgs e) {
        numGiroX.Value = 0;
        numGiroY.Value = 0;
        int AnguloZ = Convert.ToInt32(numGiroZ.Value);
        Figura3D.GiroZ(AnguloZ);
        Refresh();
    }

    //Pinta la proyección
    private void Form1_Paint(object sender, PaintEventArgs e) {
        Graphics Lienzo = e.Graphics;
        Pen Lapis = new(Color.Blue, 3);
        Figura3D.Dibuja(Lienzo, Lapis);
    }
}

```

Ejemplo de ejecución:

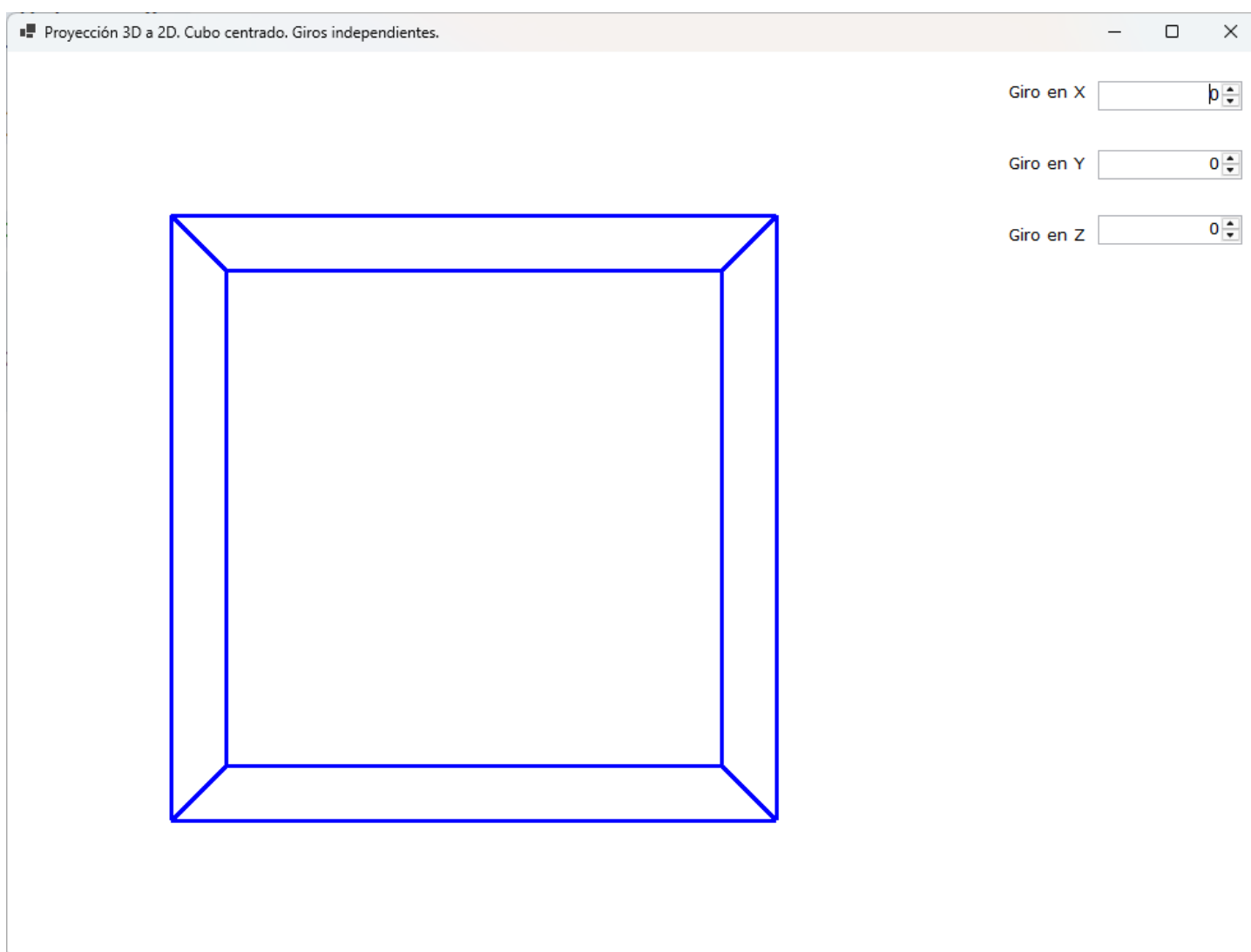
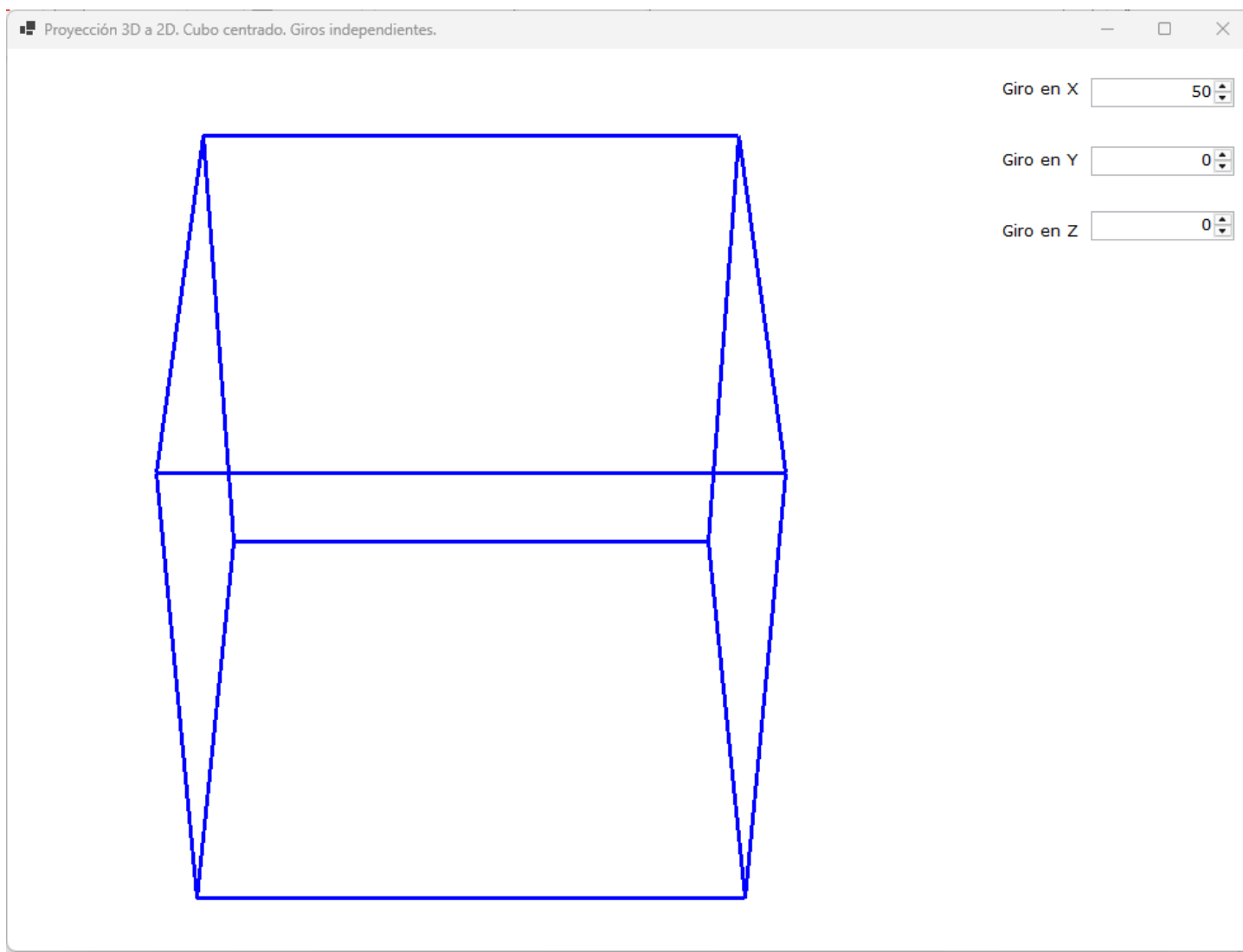
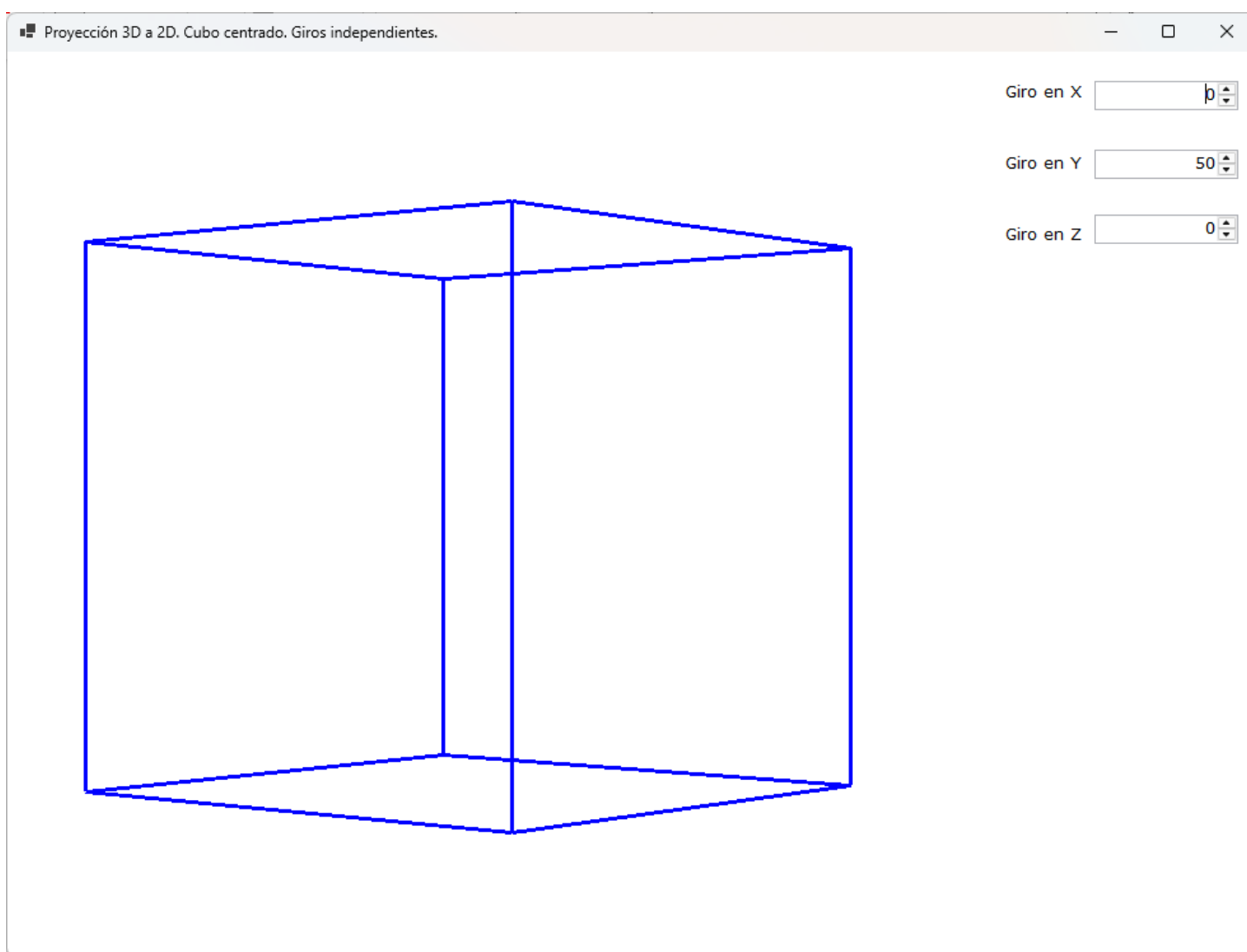


Ilustración 9: Cubo proyectado por defecto



*Ilustración 10: Cubo proyectado con giro en X*



*Ilustración 11: Cubo proyectado con giro en Y*

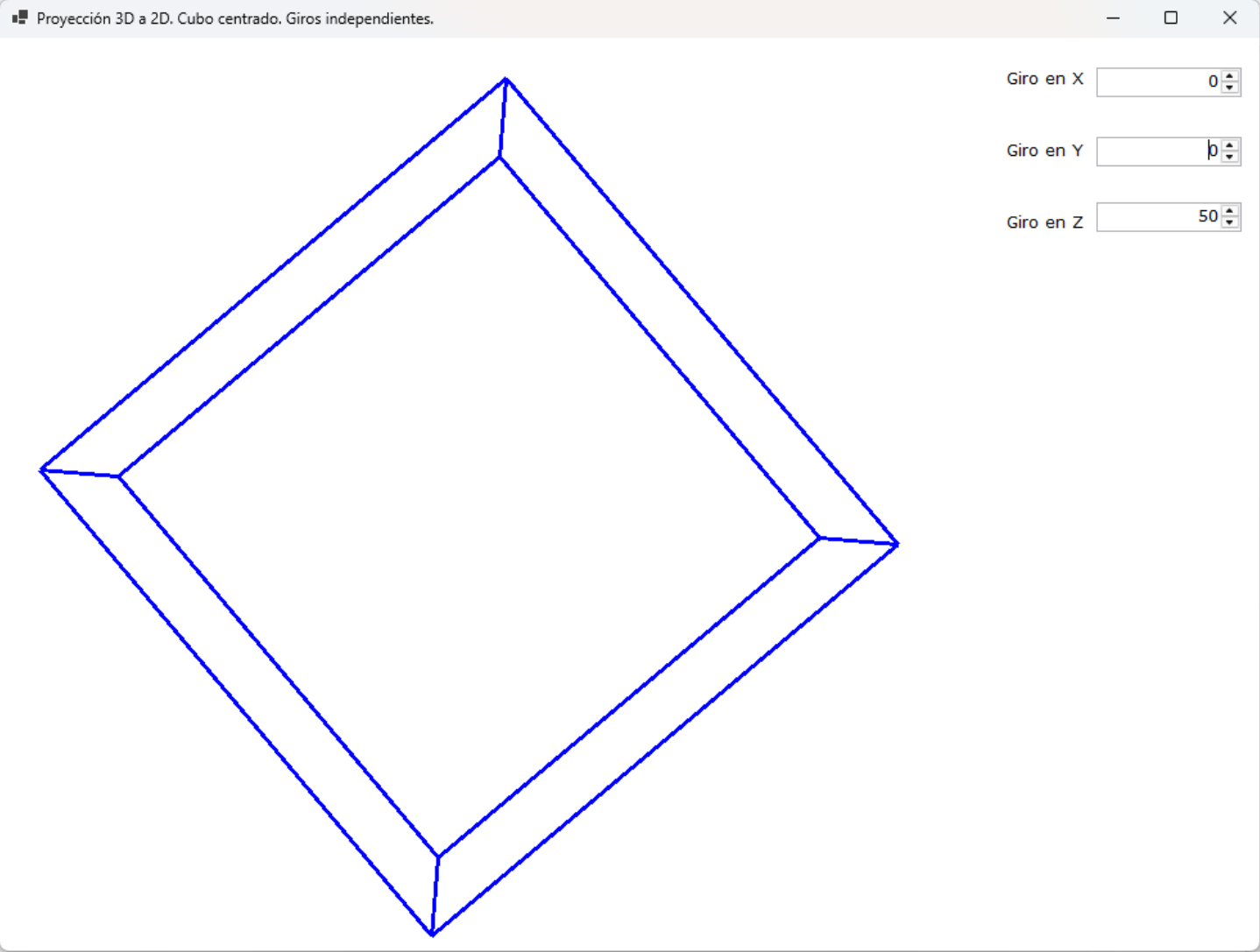


Ilustración 12: Cubo proyectado con giro en Z

# Giro centrado: Icosaedro

El programa cambia a usar un icosaedro. Esta figura está dentro de ese cubo de lado = 1.

M/005.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        private double PlanoX, PlanoY; //Proyección o sombra
        public int Xpantalla, Ypantalla; //Coordenadas en la pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Gira en X
        public void GiroX(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {1, 0, 0},
                {0, Math.Cos(AnguloRad), Math.Sin(AnguloRad)},
                {0, -Math.Sin(AnguloRad), Math.Cos(AnguloRad)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Y
        public void GiroY(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {Math.Cos(AnguloRad), 0, -Math.Sin(AnguloRad)},
                {0, 1, 0},
                {Math.Sin(AnguloRad), 0, Math.Cos(AnguloRad)}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Gira en Z
        public void GiroZ(double AnguloRad) {
            double[,] Matriz = new double[3, 3] {
                {Math.Cos(AnguloRad), Math.Sin(AnguloRad), 0},
                {-Math.Sin(AnguloRad), Math.Cos(AnguloRad), 0},
                {0, 0, 1}
            };

            AplicaMatrizGiro(Matriz);
        }

        //Aplica la matriz de giro
        private void AplicaMatrizGiro(double[,] Matriz) {
            //Hace el giro
            Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(int ZPersona) {
            PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);
        }

        //Cuadra en pantalla
        public void CuadraPantalla(double convierteX, double convierteY,
            double minimoX, double minimoY,
            int XpantallaIni, int XpantallaFin,
            int YpantallaIni, int YpantallaFin) {
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX - minimoX) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY - minimoY) + YpantallaIni);
        }
    }
}
```

```
}
```

```
namespace Graficos {  
    //Triángulo  
    internal class Poligono {  
        public int punto1, punto2, punto3;  
  
        public Poligono(int punto1, int punto2, int punto3) {  
            this.punto1 = punto1;  
            this.punto2 = punto2;  
            this.punto3 = punto3;  
        }  
    }  
}
```

```
namespace Graficos {  
    internal class Objeto3D {  
        //Coordenadas espaciales X, Y, Z  
        private List<Punto> puntos;  
  
        //Coordenadas del polígono (triángulo)  
        private List<Poligono> poligonos;  
  
        //Valores extremos  
        private double minimoX, maximoX, minimoY, maximoY, convierteX, convierteY;  
  
        //Despliegue en pantalla  
        private int XpantallaIni, XpantallaFin, YpantallaIni, YpantallaFin;  
  
        //Distancia del observador  
        private int ZPersona;  
  
        //Constructor  
        public Objeto3D(int XpantallaIni, int XpantallaFin, int YpantallaIni, int YpantallaFin) {  
            //Ejemplo de coordenadas espaciales X,Y,Z  
            puntos = [  
                new Punto( 0.000000000000, 0.361803, 0.5),  
                new Punto( 0.000000000000, 0.361803, -0.5),  
                new Punto( 0.000000000000, -0.361803, 0.5),  
                new Punto( 0.000000000000, -0.361803, -0.5),  
                new Punto( 0.361803, 0.5, 0.000000000000),  
                new Punto( 0.361803, -0.5, 0.000000000000),  
                new Punto(-0.361803, 0.5, 0.000000000000),  
                new Punto(-0.361803, -0.5, 0.000000000000),  
                new Punto( 0.5, 0.000000000000, 0.361803),  
                new Punto( 0.5, 0.000000000000, -0.361803),  
                new Punto(-0.5, 0.000000000000, 0.361803),  
                new Punto(-0.5, 0.000000000000, -0.361803)  
            ];  
  
            //Los polígonos que dibujan el icosaedro  
            poligonos = [  
                new Poligono(0, 6, 4),  
                new Poligono(0, 4, 8),  
                new Poligono(0, 8, 2),  
                new Poligono(0, 2, 10),  
                new Poligono(0, 10, 6),  
                new Poligono(1, 9, 4),  
                new Poligono(1, 4, 6),  
                new Poligono(1, 6, 11),  
                new Poligono(1, 11, 3),  
                new Poligono(1, 3, 9),  
                new Poligono(2, 8, 5),  
                new Poligono(2, 5, 7),  
                new Poligono(2, 7, 10),  
                new Poligono(3, 11, 7),  
                new Poligono(3, 7, 5),  
                new Poligono(3, 5, 9),  
                new Poligono(6, 10, 11),  
                new Poligono(5, 8, 9),  
            ];  
        }  
    }  
}
```

```

        new Poligono(7, 11, 10),
        new Poligono(9, 5, 8)
];

this.XpantallaIni = XpantallaIni;
this.XpantallaFin = XpantallaFin;
this.YpantallaIni = YpantallaIni;
this.YpantallaFin = YpantallaFin;
this.ZPersona = 5;

//Los valores extremos
this.maximoX = 0.785674201318386;
this.minimoX = -0.785674201318386;
this.maximoY = 0.785674201318386;
this.minimoY = -0.785674201318386;

//Las constantes de transformación
convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);
}

public void GiroX(int Angulo) {
    double Radian = Angulo * Math.PI / 180;
    for (int cont = 0; cont < puntos.Count; cont++) {
        puntos[cont].GiroX(Radian);
        puntos[cont].Proyecta(ZPersona);
        puntos[cont].CuadraPantalla(convierteX, convierteY,
            minimoX, minimoY,
            XpantallaIni, XpantallaFin,
            YpantallaIni, YpantallaFin);
    }
}

public void GiroY(int Angulo) {
    double Radian = Angulo * Math.PI / 180;
    for (int cont = 0; cont < puntos.Count; cont++) {
        puntos[cont].GiroY(Radian);
        puntos[cont].Proyecta(ZPersona);
        puntos[cont].CuadraPantalla(convierteX, convierteY,
            minimoX, minimoY,
            XpantallaIni, XpantallaFin,
            YpantallaIni, YpantallaFin);
    }
}

public void GiroZ(int Angulo) {
    double Radian = Angulo * Math.PI / 180;
    for (int cont = 0; cont < puntos.Count; cont++) {
        puntos[cont].GiroZ(Radian);
        puntos[cont].Proyecta(ZPersona);
        puntos[cont].CuadraPantalla(convierteX, convierteY,
            minimoX, minimoY,
            XpantallaIni, XpantallaFin,
            YpantallaIni, YpantallaFin);
    }
}

//Dibuja el icosaedro
public void Dibuja(Graphics lienzo, Pen lapiz) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++) {
        int puntoA = poligonos[Cont].punto1;
        int puntoB = poligonos[Cont].punto2;
        int puntoC = poligonos[Cont].punto3;
        lienzo.DrawLine(lapiz, puntos[puntoA].Xpantalla, puntos[puntoA].Ypantalla,
            puntos[puntoB].Xpantalla, puntos[puntoB].Ypantalla);
        lienzo.DrawLine(lapiz, puntos[puntoA].Xpantalla, puntos[puntoA].Ypantalla,
            puntos[puntoC].Xpantalla, puntos[puntoC].Ypantalla);
        lienzo.DrawLine(lapiz, puntos[puntoC].Xpantalla, puntos[puntoC].Ypantalla,
            puntos[puntoB].Xpantalla, puntos[puntoB].Ypantalla);
    }
}
}
}
}

```

```
namespace Graficos {
```



```

//Proyección 3D a 2D y giros. Figura centrada.
public partial class Form1 : Form {
    //El icosaedro que se proyecta y gira
    Objeto3D Figura3D;

    public Form1() {
        InitializeComponent();

        int XpantallaIni = 0, XpantallaFin = 700, YpantallaIni = 0, YpantallaFin = 700;
        Figura3D = new Objeto3D(XpantallaIni, XpantallaFin, YpantallaIni, YpantallaFin);
        Figura3D.GiroX(0);
    }

    private void numGiroX_ValueChanged(object sender, System.EventArgs e) {
        numGiroY.Value = 0;
        numGiroZ.Value = 0;
        int AnguloX = Convert.ToInt32(numGiroX.Value);
        Figura3D.GiroX(AnguloX);
        Refresh();
    }

    private void numGiroY_ValueChanged(object sender, EventArgs e) {
        numGiroX.Value = 0;
        numGiroZ.Value = 0;
        int AnguloY = Convert.ToInt32(numGiroY.Value);
        Figura3D.GiroY(AnguloY);
        Refresh();
    }

    private void numGiroZ_ValueChanged(object sender, EventArgs e) {
        numGiroX.Value = 0;
        numGiroY.Value = 0;
        int AnguloZ = Convert.ToInt32(numGiroZ.Value);
        Figura3D.GiroZ(AnguloZ);
        Refresh();
    }

    //Pinta la proyección
    private void Form1_Paint(object sender, PaintEventArgs e) {
        Graphics Lienzo = e.Graphics;
        Pen Lapis = new(Color.Blue, 3);
        Figura3D.Dibuja(Lienzo, Lapis);
    }
}

```

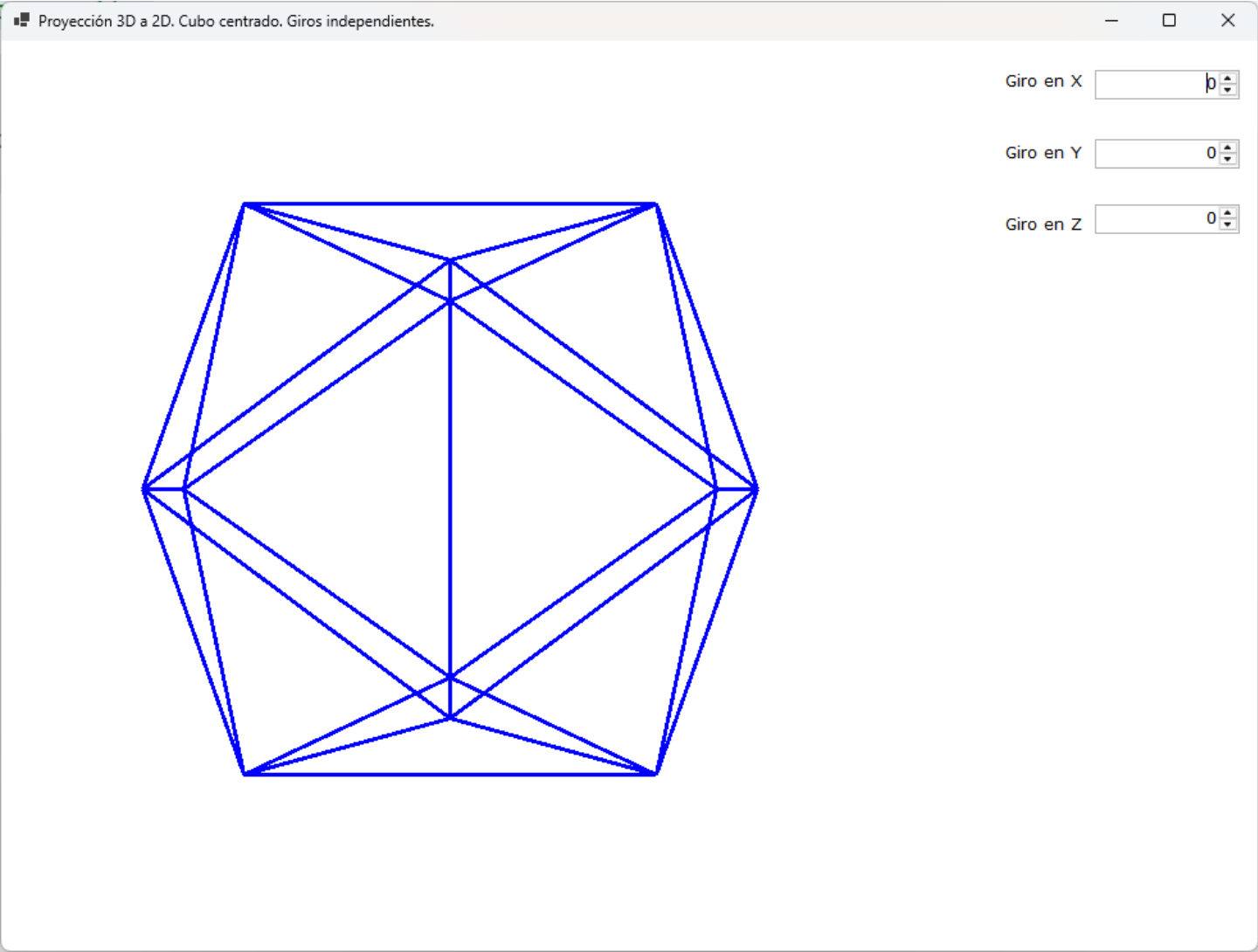


Ilustración 13: Icosaedro por defecto

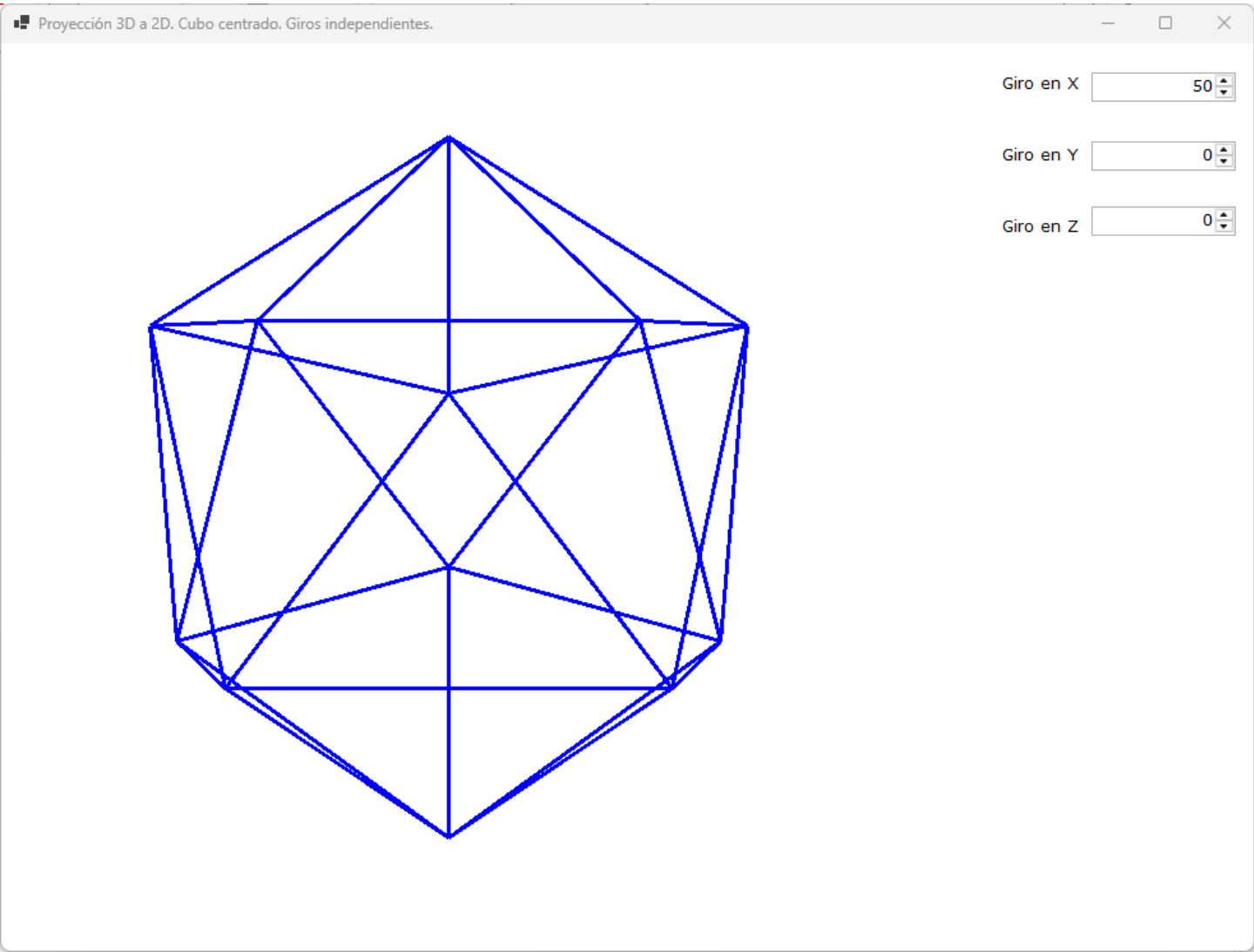


Ilustración 14: Giro en X



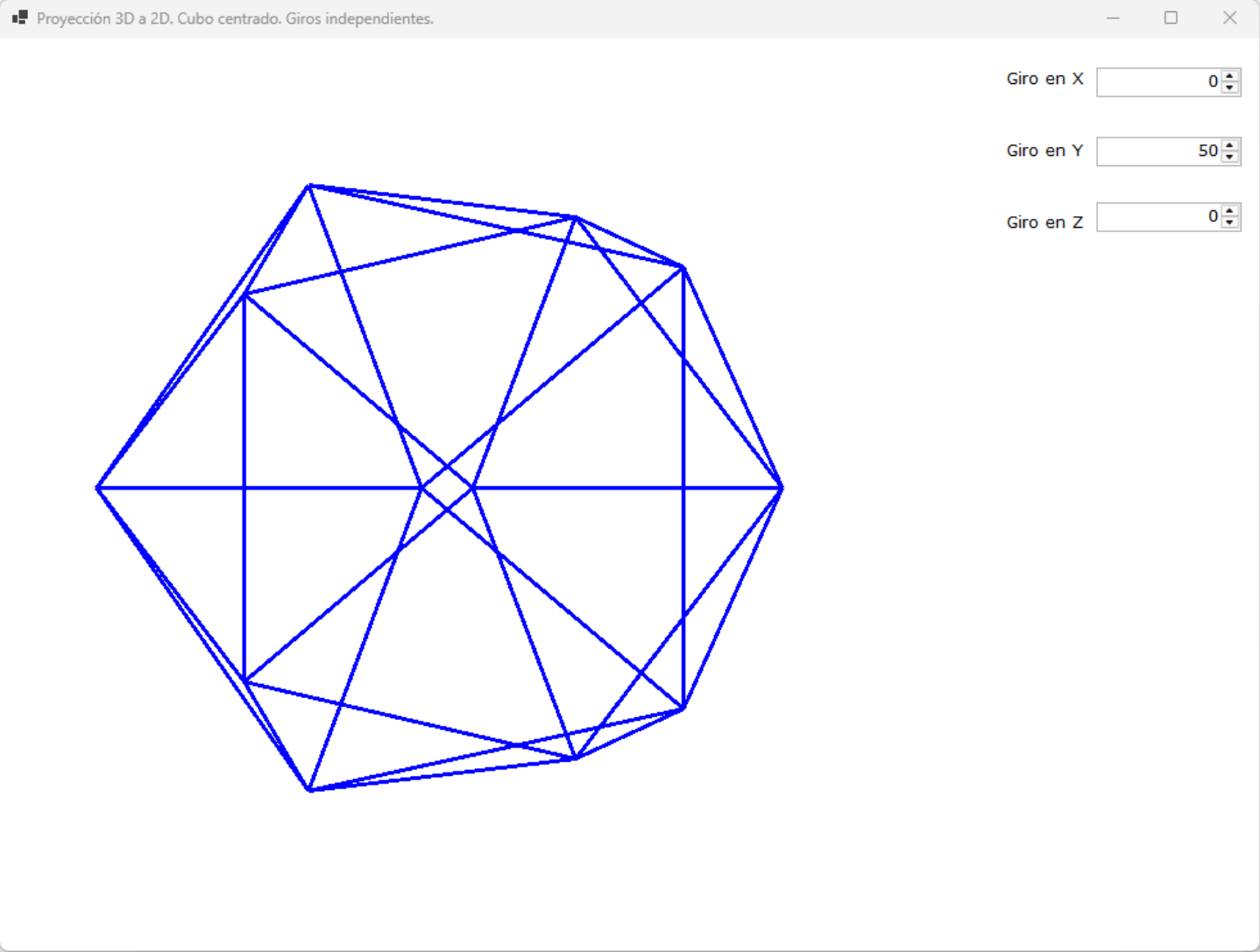


Ilustración 15: Giro en Y

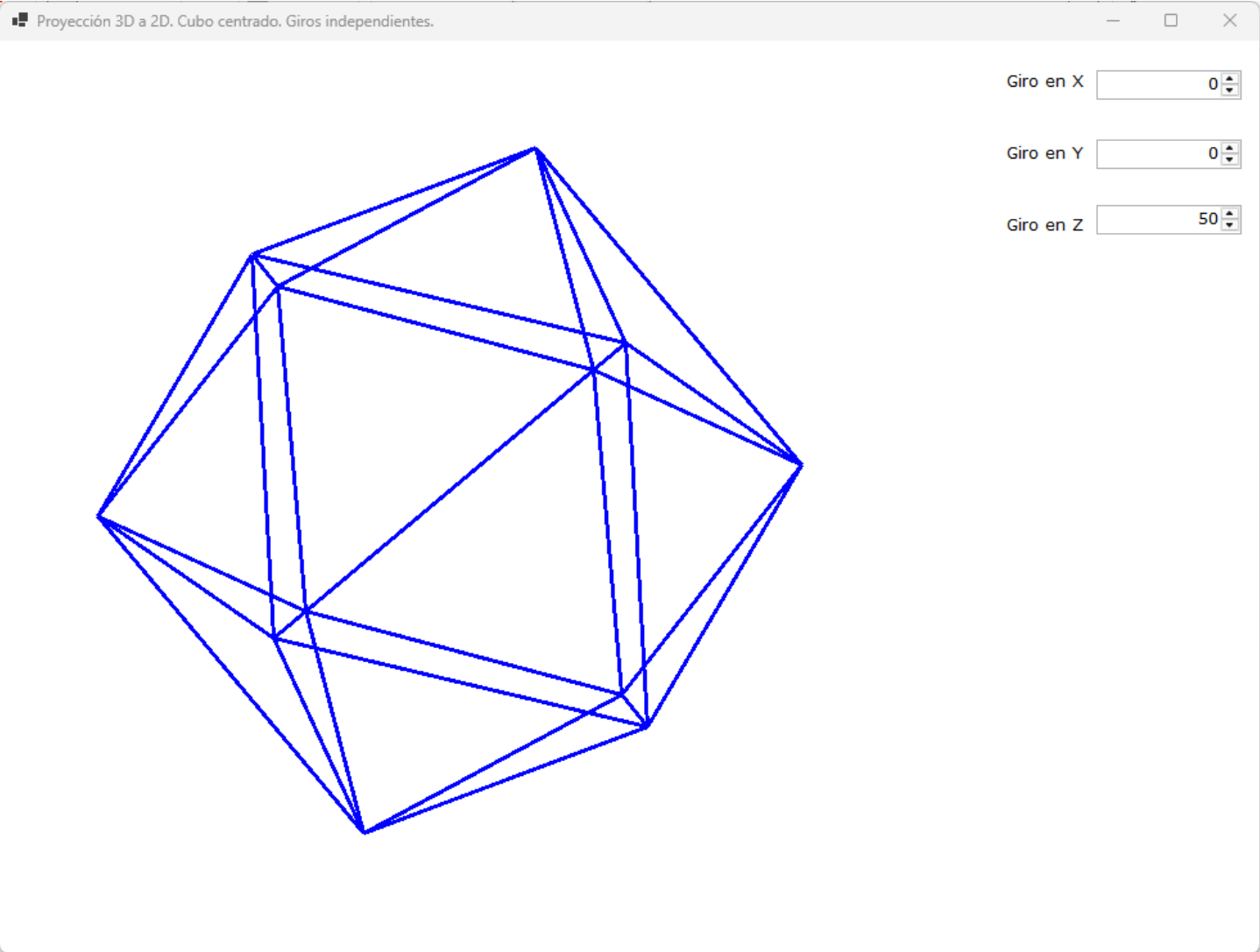


Ilustración 16: Giro en Z

Combinando los tres giros: Cálculo de constantes

En el programa anterior, sólo se puede girar en un ángulo, no hay forma de girarlo en los tres ángulos. ¿Cómo lograr el giro en los tres ángulos? La respuesta está en multiplicar las tres matrices de giro y con la matriz resultante se procede a hacer los giros.

$$\begin{aligned} \cos X &= \cos(angX) \\ \sin X &= \sin(angX) \\ \cos Y &= \cos(angY) \\ \sin Y &= \sin(angY) \\ \cos Z &= \cos(angZ) \\ \sin Z &= \sin(angZ) \end{aligned}$$

$$\begin{bmatrix} X_{giro} \\ Y_{giro} \\ Z_{giro} \end{bmatrix} = \begin{bmatrix} \cos Y * \cos Z & -\cos X * \sin Z + \sin X * \sin Y * \cos Z & \sin X * \sin Z + \cos X * \sin Y * \cos Z \\ \cos Y * \sin Z & \cos X * \cos Z + \sin X * \sin Y * \sin Z & -\sin X * \cos Z + \cos X * \sin Y * \sin Z \\ -\sin Y & \sin X * \cos Y & \cos X * \cos Y \end{bmatrix} * \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

Los pasos para dibujar el cubo fueron los siguientes:

**Paso 1**  
Coordenadas del cubo, donde su centro esté en la posición 0, 0, 0. Las coordenadas van de -0.5 a 0.5, luego cada lado mide 1. Cada coordenada tiene valores posX, posY, posZ

**Paso 2**  
Se gira cada coordenada del cubo usando la matriz de giro XYZ. Cada coordenada girada es posXg, posYg, posZg.

**Paso 3**  
Se proyecta la coordenada girada a un plano XY, el valor Z queda en cero. Esa coordenada plana es planoX, planoY. Pero aquí se tiene un problema: como se pueden combinar los tres giros, hay que recalcular de nuevo las constantes. Se deja fija la distancia del observador, en este caso con un valor de 5 (suficientemente alejado de la figura). Con esos valores entonces se probaron todos los ángulos de giro en X de 0 a 360 grados, en Y de 0 a 360 grados y en Z de 0 a 360 grados, así:

Desde anguloX = 0 grados hasta 360 grados

Desde anguloY = 0 grados hasta 360 grados

Desde anguloZ = 0 grados hasta 360 grados

Eso significa que se probaron 361\*361\*361 = 47.045.881 situaciones posibles. Un número muy alto que tomó tiempo en hacerlo (depende de la velocidad del computador) para saber las coordenadas máximas planas al girarlo en todos esos ángulos.

M/006.zip

```
namespace Graficos {
//Cada punto espacial es almacenado y convertido
internal class Punto {
private double X, Y, Z; //Coordenadas originales
private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
public double PlanoX, PlanoY; //Proyección o sombra

public Punto(double X, double Y, double Z) {
this.X = X;
this.Y = Y;
this.Z = Z;
}

public void Giro(double angX, double angY, double angZ) {
double angXr = angX * Math.PI / 180;
double angYr = angY * Math.PI / 180;
double angZr = angZ * Math.PI / 180;

double CosX = Math.Cos(angXr);
double SinX = Math.Sin(angXr);
double CosY = Math.Cos(angYr);
double SinY = Math.Sin(angYr);
double CosZ = Math.Cos(angZr);
double SinZ = Math.Sin(angZr);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
{ CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
{ CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
{-SinY, SinX * CosY, CosX * CosY }
};
};
};
```

```

        //Hace el giro
        Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
        Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
        Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
    }

    //Convierte de 3D a 2D (segunda dimensión)
    public void Proyecta(int ZPersona, ref double minimoX, ref double maximoX, ref double minimoY, ref
double maximoY) {
        PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
        PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

        if (PlanoX < minimoX) minimoX = PlanoX;
        if (PlanoX > maximoX) maximoX = PlanoX;
        if (PlanoY < minimoY) minimoY = PlanoY;
        if (PlanoY > maximoY) maximoY = PlanoY;
    }
}
}
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}
}

```

```

namespace Graficos {
    internal class Cubo {
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Constructor
        public Cubo() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto(-0.5, -0.5, -0.5),
                new Punto(0.5, -0.5, -0.5),
                new Punto(0.5, 0.5, -0.5),
                new Punto(-0.5, 0.5, -0.5),
                new Punto(-0.5, -0.5, 0.5),
                new Punto(0.5, -0.5, 0.5),
                new Punto(0.5, 0.5, 0.5),
                new Punto(-0.5, 0.5, 0.5)
            ];
        }

        //Calcula los extremos de las coordenadas del cubo al girar y proyectarse
        public void CalculaExtremo(int ZPersona) {
            double maximoX = double.MinValue;
            double minimoX = double.MaxValue;
            double maximoY = double.MinValue;
            double minimoY = double.MaxValue;

            //Va de punto en punto
            for (int Cont = 0; Cont < puntos.Count; Cont++) {

                //Va de angulo en angulo
                for (double angX = 0; angX <= 360; angX++)
                    for (double angY = 0; angY <= 360; angY++)
                        for (double angZ = 0; angZ <= 360; angZ++) {
                            puntos[Cont].Giro(angX, angY, angZ);
                            puntos[Cont].Proyecta(ZPersona, ref minimoX, ref maximoX, ref minimoY, ref maximoY);
                        }
            }
        }
    }
}

```

```

        Console.WriteLine("Extremos son:");
        Console.WriteLine("MinimoX: " + minimoX.ToString());
        Console.WriteLine("MaximoX: " + maximoX.ToString());
        Console.WriteLine("MinimoY: " + minimoY.ToString());
        Console.WriteLine("MaximoY: " + maximoY.ToString());
    }
}
}

```

```

namespace Graficos
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Cubo Figura3D = new();
            Figura3D.CalculaExtremo(5);
        }
    }
}

```

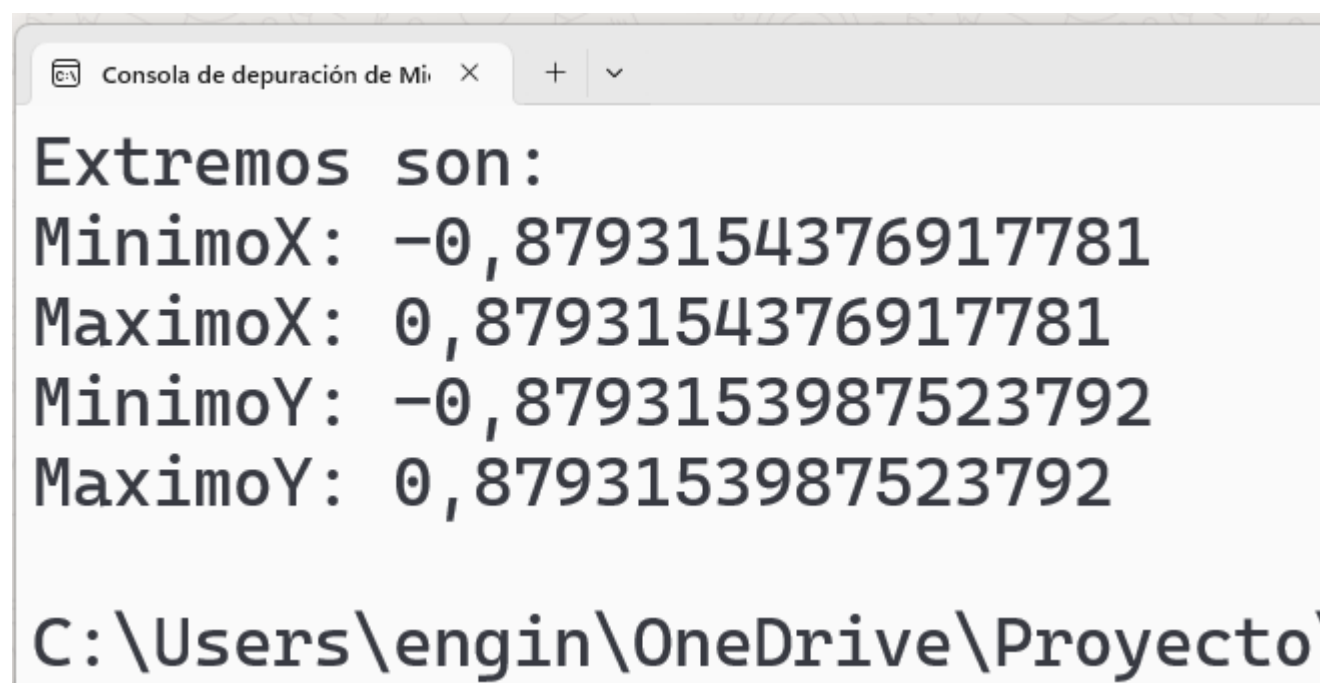


Ilustración 17: Constantes para cuadrar el cubo en la pantalla

```

namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        public double PlanoX, PlanoY; //Proyección o sombra
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double angX, double angY, double angZ) {
            double angXr = angX * Math.PI / 180;
            double angYr = angY * Math.PI / 180;
            double angZr = angZ * Math.PI / 180;

            double CosX = Math.Cos(angXr);
            double SinX = Math.Sin(angXr);
            double CosY = Math.Cos(angYr);
            double SinY = Math.Sin(angYr);
            double CosZ = Math.Cos(angZr);
            double SinZ = Math.Sin(angZr);

            //Matriz de Rotación
            //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
            double[,] Matriz = new double[3, 3] {
                { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
                { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
                {-SinY, SinX * CosY, CosX * CosY }
            };

            //Hace el giro
            Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(double ZPersona) {
            PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);
        }

        //Cuadra en pantalla
        public void CuadraPantalla(int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {
            //Los valores extremos de las coordenadas del cubo
            double maximoX = 0.879315437691778;
            double minimoX = -0.879315437691778;
            double maximoY = 0.879315398752379;
            double minimoY = -0.879315437691778;

            //Las constantes de transformación
            double convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
            double convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX - minimoX) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY - minimoY) + YpantallaIni);
        }
    }
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    internal class Cubo {
        //Un cubo tiene 8 coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Un cubo tiene 12 líneas de conexión
        private List<Conexion> conexiones;

        //Constructor
        public Cubo() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto(-0.5, -0.5, -0.5),
                new Punto(0.5, -0.5, -0.5),
                new Punto(0.5, 0.5, -0.5),
                new Punto(-0.5, 0.5, -0.5),
                new Punto(-0.5, -0.5, 0.5),
                new Punto(0.5, -0.5, 0.5),
                new Punto(0.5, 0.5, 0.5),
                new Punto(-0.5, 0.5, 0.5)
            ];

            //Las 12 líneas para dibujar el cubo
            //punto inicial ---> punto final
            conexiones = [
                new Conexion(0, 1),
                new Conexion(1, 2),
                new Conexion(2, 3),
                new Conexion(3, 0),
                new Conexion(4, 5),
                new Conexion(5, 6),
                new Conexion(6, 7),
                new Conexion(7, 4),
                new Conexion(0, 4),
                new Conexion(1, 5),
                new Conexion(2, 6),
                new Conexion(3, 7)
            ];
        }

        public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {
            //Gira los 8 puntos
            for (int cont = 0; cont < puntos.Count; cont++) {
                puntos[cont].Giro(angX, angY, angZ);
                puntos[cont].Proyecta(ZPersona);
                puntos[cont].CuadraPantalla(XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);
            }
        }

        //Dibuja el cubo
        public void Dibuja(Graphics lienzo, Pen lapiz) {
            for (int Cont = 0; Cont < conexiones.Count; Cont++) {
                int Inicio = conexiones[Cont].punto1;
                int Final = conexiones[Cont].punto2;
                lienzo.DrawLine(lapiz, puntos[Inicio].Xpantalla, puntos[Inicio].Ypantalla,
                    puntos[Final].Xpantalla, puntos[Final].Ypantalla);
            }
        }
    }
}

```



```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada.
    public partial class Form1 : Form {

        //El cubo que se proyecta y gira
        Cubo Figura3D;

        //Tamaño de pantalla
        double ZPersona;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        public Form1() {
            InitializeComponent();

            //Distancia del observador
            ZPersona = 5;

            //Tamaño de pantalla
            XpantallaIni = 20;
            YpantallaIni = 20;
            XpantallaFin = 500;
            YpantallaFin = 500;

            Figura3D = new Cubo();

            //Gira, proyecta y cuadra cada punto de la figura 3D
            Figura3D.GirarFigura(0, 0, 0, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            double AnguloX = Convert.ToDouble(numGiroX.Value);
            double AnguloY = Convert.ToDouble(numGiroY.Value);
            double AnguloZ = Convert.ToDouble(numGiroZ.Value);

            Graphics Lienzo = e.Graphics;

            //Dibuja el rectángulo que contiene la figura 3D
            Pen LapisMarco = new(Color.Blue, 1);
            Lienzo.DrawRectangle(LapisMarco, XpantallaIni, YpantallaIni, XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

            //Gira, proyecta y cuadra cada punto de la figura 3D
            Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);

            //Dibuja la figura 3D
            Pen Lapis3D = new(Color.Black, 1);
            Figura3D.Dibuja(Lienzo, Lapis3D);
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }
    }
}

```

Ejemplo de ejecución:

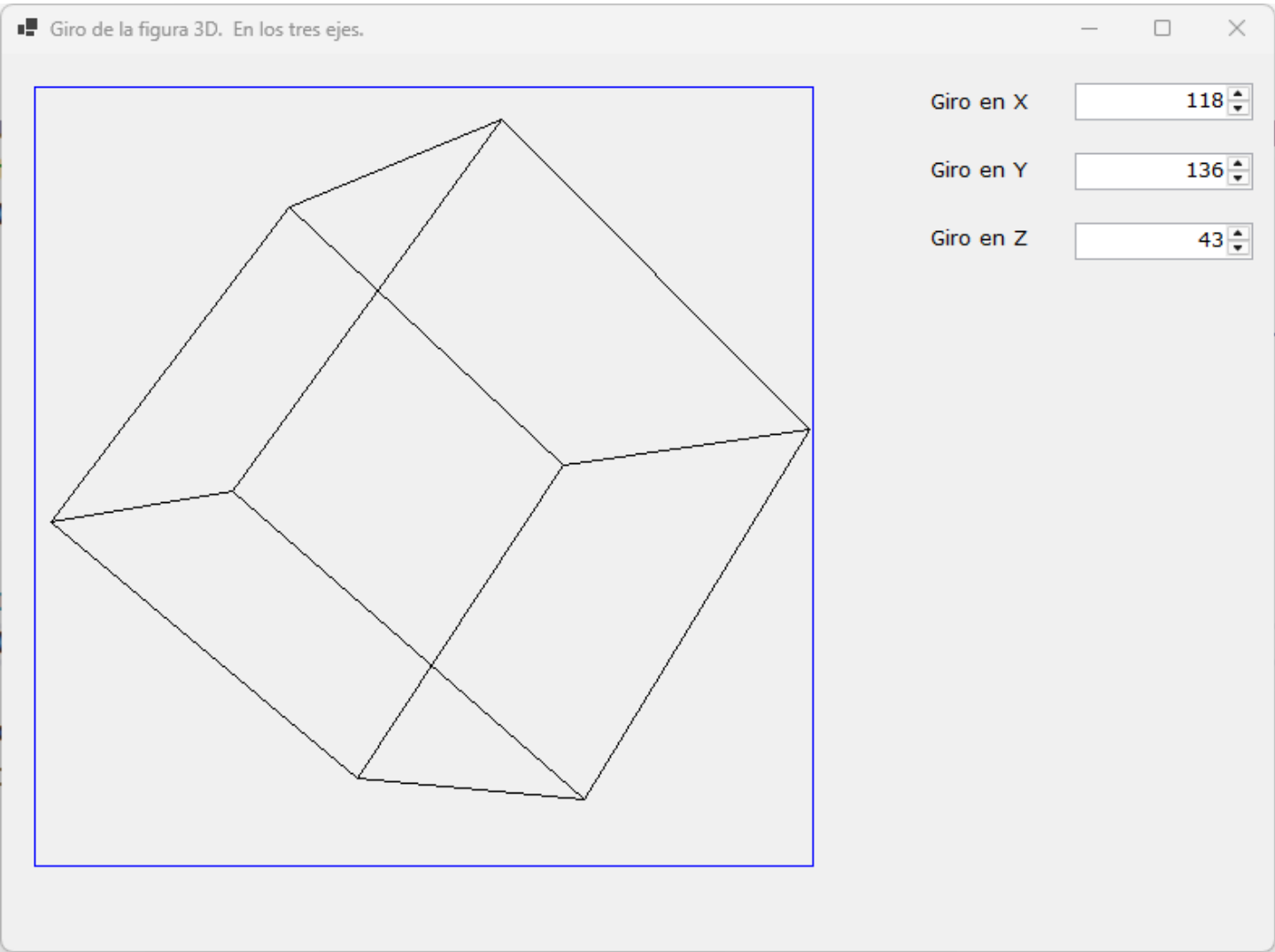


Ilustración 18: Cubo proyectado con giros



Las figuras 3D se dibujan al interior de un cubo “imaginario” de lado con valor uno(1) de esa manera los giros ya están precalculados.

M/008.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        private double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        public double PlanoX, PlanoY; //Proyección o sombra
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double angX, double angY, double angZ) {
            double angXr = angX * Math.PI / 180;
            double angYr = angY * Math.PI / 180;
            double angZr = angZ * Math.PI / 180;

            double CosX = Math.Cos(angXr);
            double SinX = Math.Sin(angXr);
            double CosY = Math.Cos(angYr);
            double SinY = Math.Sin(angYr);
            double CosZ = Math.Cos(angZr);
            double SinZ = Math.Sin(angZr);

            //Matriz de Rotación
            //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
            double[,] Matriz = new double[3, 3] {
                { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
                { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
                {-SinY, SinX * CosY, CosX * CosY }
            };

            //Hace el giro
            Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(double ZPersona) {
            PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);
        }

        //Cuadra en pantalla
        public void CuadraPantalla(int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {
            //Los valores extremos de las coordenadas del cubo
            double maximoX = 0.879315437691778;
            double minimoX = -0.879315437691778;
            double maximoY = 0.879315398752379;
            double minimoY = -0.879315437691778;

            //Las constantes de transformación
            double convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
            double convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX - minimoX) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY - minimoY) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
```

```

internal class Conexion {
    public int punto1, punto2;

    public Conexion(int punto1, int punto2) {
        this.punto1 = punto1;
        this.punto2 = punto2;
    }
}

```

```

namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3;

        public Poligono(int punto1, int punto2, int punto3) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Un cubo tiene 12 líneas de conexión
        private List<Conexion> conexiones;

        //Constructor
        public Objeto3D() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto( 0.000000000000, 0.361803, 0.5),
                new Punto( 0.000000000000, 0.361803, -0.5),
                new Punto( 0.000000000000, -0.361803, 0.5),
                new Punto( 0.000000000000, -0.361803, -0.5),
                new Punto( 0.361803, 0.5, 0.000000000000),
                new Punto( 0.361803, -0.5, 0.000000000000),
                new Punto(-0.361803, 0.5, 0.000000000000),
                new Punto(-0.361803, -0.5, 0.000000000000),
                new Punto( 0.5, 0.000000000000, 0.361803),
                new Punto( 0.5, 0.000000000000, -0.361803),
                new Punto(-0.5, 0.000000000000, 0.361803),
                new Punto(-0.5, 0.000000000000, -0.361803),

                //Coordenadas del cubo que contiene al icosaedro
                new Punto(-0.5, -0.5, -0.5),
                new Punto(0.5, -0.5, -0.5),
                new Punto(0.5, 0.5, -0.5),
                new Punto(-0.5, 0.5, -0.5),
                new Punto(-0.5, -0.5, 0.5),
                new Punto(0.5, -0.5, 0.5),
                new Punto(0.5, 0.5, 0.5),
                new Punto(-0.5, 0.5, 0.5)
            ];

            //Los polígonos que dibujan el icosaedro
            poligonos = [
                new Poligono(0, 6, 4),
                new Poligono(0, 4, 8),
                new Poligono(0, 8, 2),
                new Poligono(0, 2, 10),
                new Poligono(0, 10, 6),
                new Poligono(1, 9, 4),
            ];
        }
    }
}

```

```

        new Poligono(1, 4, 6),
        new Poligono(1, 6, 11),
        new Poligono(1, 11, 3),
        new Poligono(1, 3, 9),
        new Poligono(2, 8, 5),
        new Poligono(2, 5, 7),
        new Poligono(2, 7, 10),
        new Poligono(3, 11, 7),
        new Poligono(3, 7, 5),
        new Poligono(3, 5, 9),
        new Poligono(6, 10, 11),
        new Poligono(5, 8, 9),
        new Poligono(7, 11, 10),
        new Poligono(4, 9, 8)
    ];

    //Las 12 líneas para dibujar el cubo
    //punto inicial ---> punto final
    conexiones = [
        new Conexion(0+12, 1+12),
        new Conexion(1+12, 2+12),
        new Conexion(2+12, 3+12),
        new Conexion(3+12, 0+12),
        new Conexion(4+12, 5+12),
        new Conexion(5+12, 6+12),
        new Conexion(6+12, 7+12),
        new Conexion(7+12, 4+12),
        new Conexion(0+12, 4+12),
        new Conexion(1+12, 5+12),
        new Conexion(2+12, 6+12),
        new Conexion(3+12, 7+12)
    ];
}

public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int
YpantallaIni, int XpantallaFin, int YpantallaFin) {
    //Gira los 8 puntos
    for (int cont = 0; cont < puntos.Count; cont++) {
        puntos[cont].Giro(angX, angY, angZ);
        puntos[cont].Proyecta(ZPersona);
        puntos[cont].CuadraPantalla(XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);
    }
}

//Dibuja el icosaedro
public void Dibuja(Graphics lienzo, Pen lapizIcosaedro, Pen lapizCubo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++) {
        int puntoA = poligonos[Cont].punto1;
        int puntoB = poligonos[Cont].punto2;
        int puntoC = poligonos[Cont].punto3;
        lienzo.DrawLine(lapizIcosaedro, puntos[puntoA].Xpantalla, puntos[puntoA].Ypantalla,
            puntos[puntoB].Xpantalla, puntos[puntoB].Ypantalla);
        lienzo.DrawLine(lapizIcosaedro, puntos[puntoA].Xpantalla, puntos[puntoA].Ypantalla,
            puntos[puntoC].Xpantalla, puntos[puntoC].Ypantalla);
        lienzo.DrawLine(lapizIcosaedro, puntos[puntoC].Xpantalla, puntos[puntoC].Ypantalla,
            puntos[puntoB].Xpantalla, puntos[puntoB].Ypantalla);
    }

    for (int Cont = 0; Cont < conexiones.Count; Cont++) {
        int Inicio = conexiones[Cont].punto1;
        int Final = conexiones[Cont].punto2;
        lienzo.DrawLine(lapizCubo, puntos[Inicio].Xpantalla, puntos[Inicio].Ypantalla,
            puntos[Final].Xpantalla, puntos[Final].Ypantalla);
    }
}
}
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada.
    public partial class Form1 : Form {

        //El icosaedro que se proyecta y gira
        Objeto3D Figura3D;

        //Tamaño de pantalla
        double ZPersona;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        public Form1() {
            InitializeComponent();

            //Distancia del observador
            ZPersona = 5;

            //Tamaño de pantalla
            XpantallaIni = 20;
            YpantallaIni = 20;
            XpantallaFin = 500;
            YpantallaFin = 500;

            Figura3D = new Objeto3D();

            //Gira, proyecta y cuadra cada punto de la figura 3D
            Figura3D.GirarFigura(0, 0, 0, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            double AnguloX = Convert.ToDouble(numGiroX.Value);
            double AnguloY = Convert.ToDouble(numGiroY.Value);
            double AnguloZ = Convert.ToDouble(numGiroZ.Value);

            Graphics Lienzo = e.Graphics;

            //Dibuja el rectángulo que contiene la figura 3D
            Pen LapisMarco = new(Color.Blue, 1);
            Lienzo.DrawRectangle(LapisMarco, XpantallaIni, YpantallaIni, XpantallaFin - XpantallaIni, YpantallaFin - YpantallaIni);

            //Gira, proyecta y cuadra cada punto de la figura 3D
            Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);

            //Dibuja la figura 3D
            Pen LapisIcosaedro = new(Color.Black, 1);
            Pen LapisCubo = new(Color.Red, 1);
            Figura3D.Dibuja(Lienzo, LapisIcosaedro, LapisCubo);
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            Refresh();
        }
    }
}

```

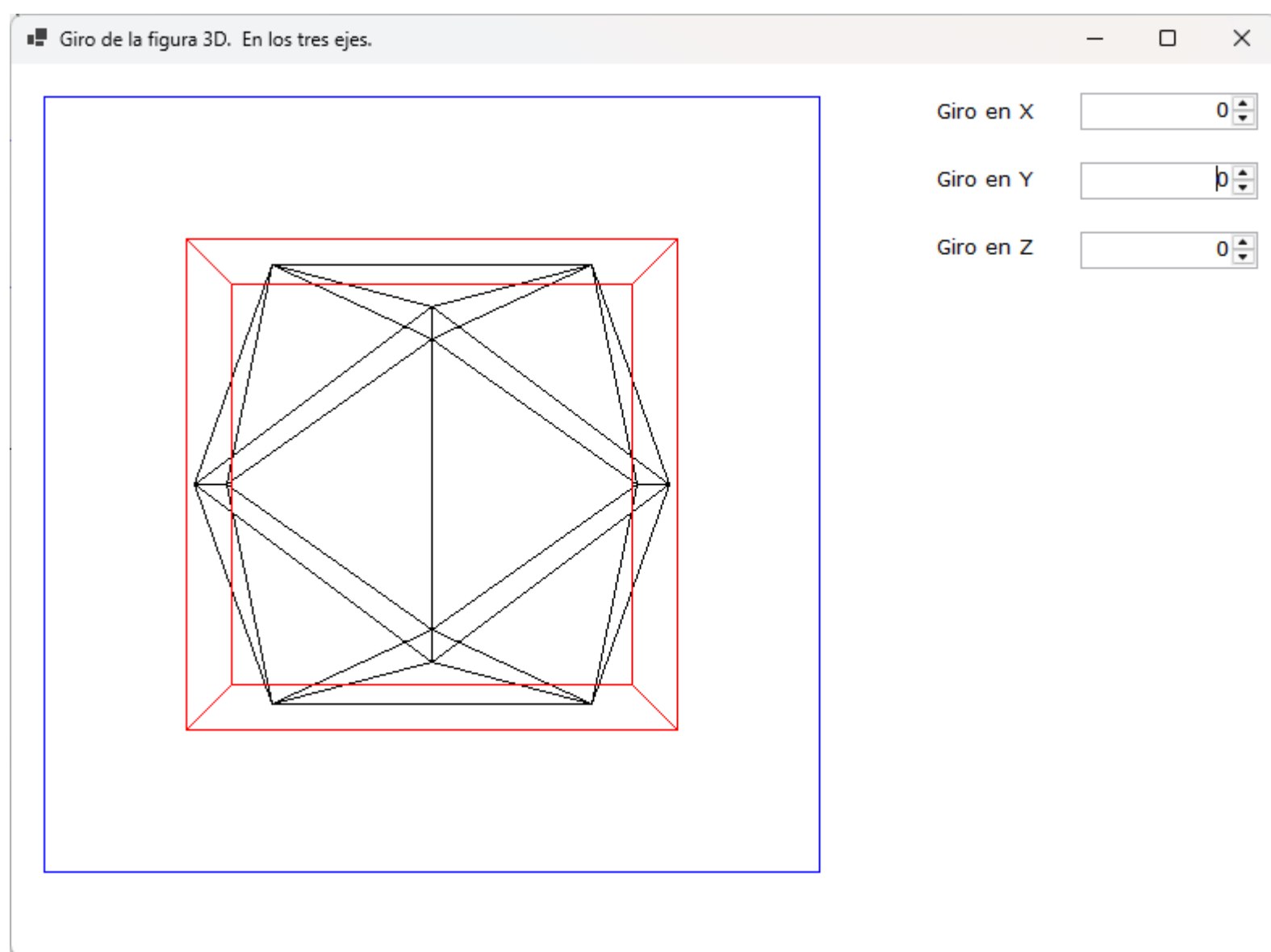


Ilustración 19: El icosaedro está dentro del cubo

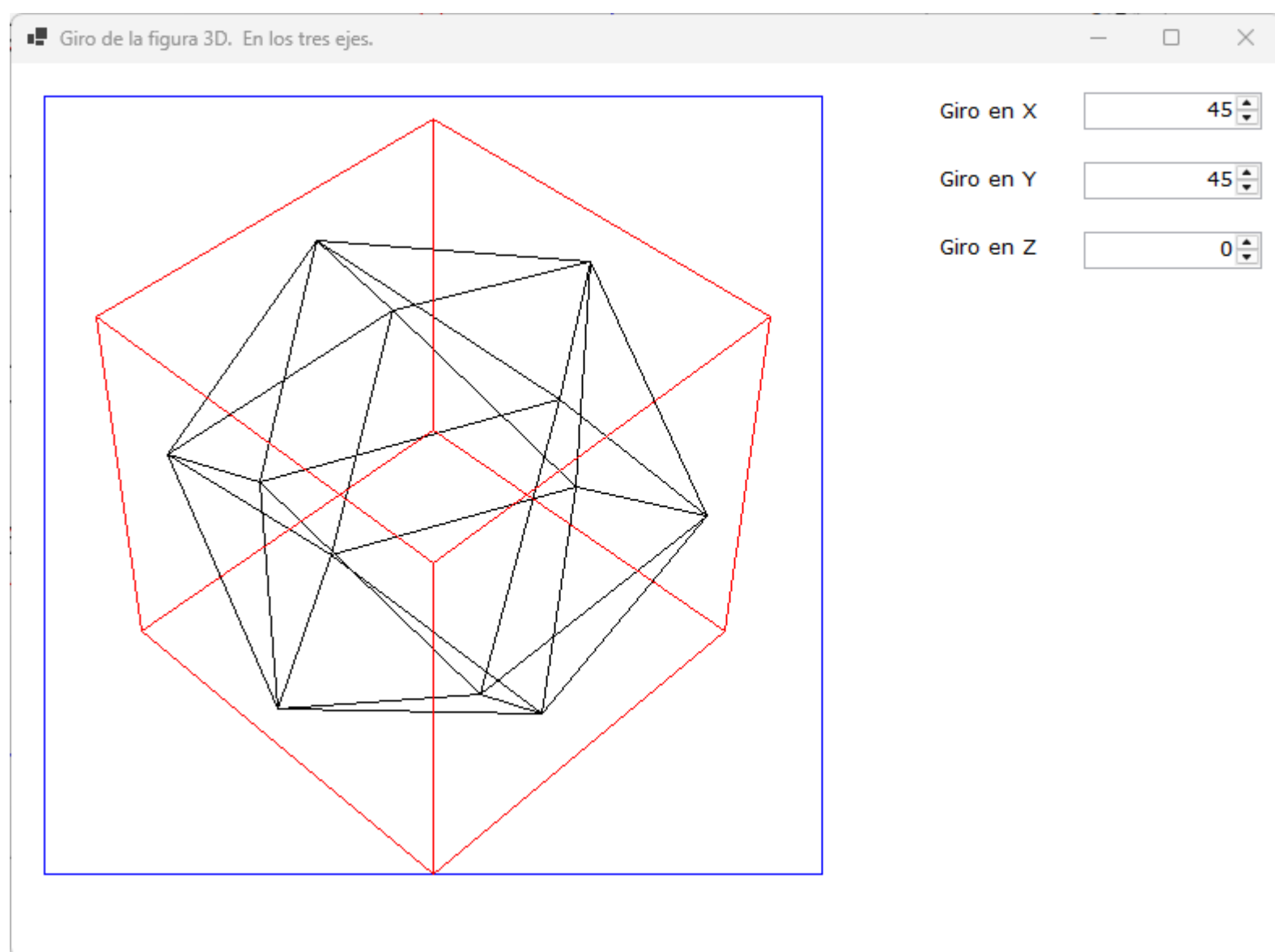


Ilustración 20: Se gira el cubo, entonces gira el icosaedro. Todo pre-calculado.

El problema con los ejemplos anteriores de giros es la vista, en vez de interpretar que el icosaedro gira, lo que a veces se observa es que la figura se distorsiona y es porque el icosaedro hecho de líneas verticales, horizontales y oblicuas llega a confundir, y no se sabe que está atrás y que está adelante. Para evitar esto, se hace un cambio y es dibujar el icosaedro no con líneas sino con polígonos [2]. Un icosaedro tiene veinte caras, luego serían veinte polígonos para dibujarlo. Estos serían los pasos:

- 1. Escribir las tres coordenadas XYZ que servirán para componer cada uno de los veinte polígonos.
- 2. Girar cada coordenada XYZ de cada polígono.
- 3. Determinar el valor Z promedio de cada polígono.
- 4. Ordenar los polígonos del Z promedio más pequeño al más grande. Eso daría que polígono está más lejos y cuál más cerca al observador.

Dibujar en ese orden cada polígono, primero rellenando el área del polígono con el color del fondo y luego dibujando el perímetro con algún color visible.

M/009.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        public double Xgiro, Ygiro, Zgiro; //Al girar los puntos
        public double PlanoX, PlanoY; //Proyección o sombra
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double angX, double angY, double angZ) {
            double angXr = angX * Math.PI / 180;
            double angYr = angY * Math.PI / 180;
            double angZr = angZ * Math.PI / 180;

            double CosX = Math.Cos(angXr);
            double SinX = Math.Sin(angXr);
            double CosY = Math.Cos(angYr);
            double SinY = Math.Sin(angYr);
            double CosZ = Math.Cos(angZr);
            double SinZ = Math.Sin(angZr);

            //Matriz de Rotación
            //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
            double[,] Matriz = new double[3, 3] {
                { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
                { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
                {-SinY, SinX * CosY, CosX * CosY }
            };

            //Hace el giro
            Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];
        }

        //Convierte de 3D a 2D (segunda dimensión)
        public void Proyecta(double ZPersona) {
            PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);
        }

        //Cuadra en pantalla
        public void CuadraPantalla(int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {
            //Los valores extremos de las coordenadas del cubo
            double maximoX = 0.879315437691778;
            double minimoX = -0.879315437691778;
            double maximoY = 0.879315398752379;
            double minimoY = -0.879315437691778;

            //Las constantes de transformación
            double convierteX = (XpantallaFin - XpantallaIni) / (maximoX - minimoX);
            double convierteY = (YpantallaFin - YpantallaIni) / (maximoY - minimoY);
        }
    }
}
```



```

        //Deduce las coordenadas de pantalla
        Xpantalla = Convert.ToInt32(convierteX * (PlanoX - minimoX) + XpantallaIni);
        Ypantalla = Convert.ToInt32(convierteY * (PlanoY - minimoY) + YpantallaIni);
    }
}
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3;
        public double Centro;

        //Coordenadas de dibujo del polígono
        Point Punto1, Punto2, Punto3;

        public Poligono(int punto1, int punto2, int punto3) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
        }

        //Calcula la profundidad y crea los vértices del polígo
        public void CalculaProfundidad(List<Punto> puntos) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            Centro = (Z1+ Z2 +Z3) / 3;

            Punto1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Punto2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Punto3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo, Pen Lapiz, Brush Relleno) {
            Point[] ListaPuntos = [Punto1, Punto2, Punto3];

            //Dibuja el polígono relleno y eso borra los polígonos que no se ven
            Lienzo.FillPolygon(Relleno, ListaPuntos);

            //Dibuja el perímetro del polígono haciéndolo visible.
            Lienzo.DrawPolygon(Lapiz, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Constructor
        public Objeto3D() {

```

```

//Ejemplo de coordenadas espaciales X,Y,Z
puntos = [
    new Punto( 0.000000000000,  0.361803,  0.5),
    new Punto( 0.000000000000,  0.361803, -0.5),
    new Punto( 0.000000000000, -0.361803,  0.5),
    new Punto( 0.000000000000, -0.361803, -0.5),
    new Punto( 0.361803,  0.5,  0.000000000000),
    new Punto( 0.361803, -0.5,  0.000000000000),
    new Punto(-0.361803,  0.5,  0.000000000000),
    new Punto(-0.361803, -0.5,  0.000000000000),
    new Punto( 0.5,  0.000000000000,  0.361803),
    new Punto( 0.5,  0.000000000000, -0.361803),
    new Punto(-0.5,  0.000000000000,  0.361803),
    new Punto(-0.5,  0.000000000000, -0.361803),
];

//Los polígonos que dibujan el icosaedro
poligonos = [
    new Poligono(0, 6, 4),
    new Poligono(0, 4, 8),
    new Poligono(0, 8, 2),
    new Poligono(0, 2, 10),
    new Poligono(0, 10, 6),
    new Poligono(1, 9, 4),
    new Poligono(1, 4, 6),
    new Poligono(1, 6, 11),
    new Poligono(1, 11, 3),
    new Poligono(1, 3, 9),
    new Poligono(2, 8, 5),
    new Poligono(2, 5, 7),
    new Poligono(2, 7, 10),
    new Poligono(3, 11, 7),
    new Poligono(3, 7, 5),
    new Poligono(3, 5, 9),
    new Poligono(6, 10, 11),
    new Poligono(5, 8, 9),
    new Poligono(7, 11, 10),
    new Poligono(4, 9, 8)
];
}

public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int
YpantallaIni, int XpantallaFin, int YpantallaFin) {
    //Gira los 8 puntos
    for (int cont = 0; cont < puntos.Count; cont++) {
        puntos[cont].Giro(angX, angY, angZ);
        puntos[cont].Proyecta(ZPersona);
        puntos[cont].CuadraPantalla(XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);
    }

    //Calcula la profundidad promedio
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].CalculaProfundidad(puntos);

    //Ordena del polígono más alejado al más cercano,
    //de esa manera los polígonos de adelante son
    //visibles y los de atrás son borrados.
    //Algoritmo de pintor.
    poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja el icosaedro
public void Dibuja(Graphics lienzo, Pen lapizIcosaedro, Brush relleno) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo, lapizIcosaedro, relleno);
}
}
}

```



```

using System.Diagnostics;

namespace Graficos
{
    //Proyección 3D a 2D y giros. Figura centrada.
    public partial class Form1 : Form {

        //Para calcular los FPS (Frames Per Second)
        private Stopwatch cronometro = new Stopwatch();
        private int fps = 0;
        private int contadorFrames = 0;
        private long tiempoAcumulado = 0;

        //El icosaedro que se proyecta y gira
        Objeto3D Figura3D;

        //Tamaño de pantalla
        double ZPersona;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        public Form1() {
            InitializeComponent();

            //Para calcular los FPS (Frames Per Second)
            cronometro.Start();
            System.Windows.Forms.Timer timer = new System.Windows.Forms.Timer();
            timer.Interval = 16; // ~60 FPS
            timer.Tick += (s, e) => this.Invalidate(); // Forzar redibujo
            timer.Start();

            //Distancia del observador
            ZPersona = 5;

            //Tamaño de pantalla
            XpantallaIni = 20;
            YpantallaIni = 20;
            XpantallaFin = 500;
            YpantallaFin = 500;

            Figura3D = new Objeto3D();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            double AnguloX = Convert.ToDouble(numGiroX.Value);
            double AnguloY = Convert.ToDouble(numGiroY.Value);
            double AnguloZ = Convert.ToDouble(numGiroZ.Value);

            //Gira, proyecta y cuadra cada punto de la figura 3D
            Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin);

            //Dibuja la figura 3D
            Graphics Lienzo = e.Graphics;
            Pen LapizIcosaedro = new(Color.Black, 1);
            Brush Relleno = new SolidBrush(Color.White);
            Figura3D.Dibuja(Lienzo, LapizIcosaedro, Relleno);

            //Calcular los FPS
            contadorFrames++;
            tiempoAcumulado += cronometro.ElapsedMilliseconds;
            cronometro.Restart();

            if (tiempoAcumulado >= 1000) // cada segundo
            {
                fps = contadorFrames;
                contadorFrames = 0;
                tiempoAcumulado = 0;
            }

            // Dibujar texto con los FPS
            Lienzo.DrawString($"FPS: {fps}", this.Font, Brushes.Red, 10, 10);
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {

```

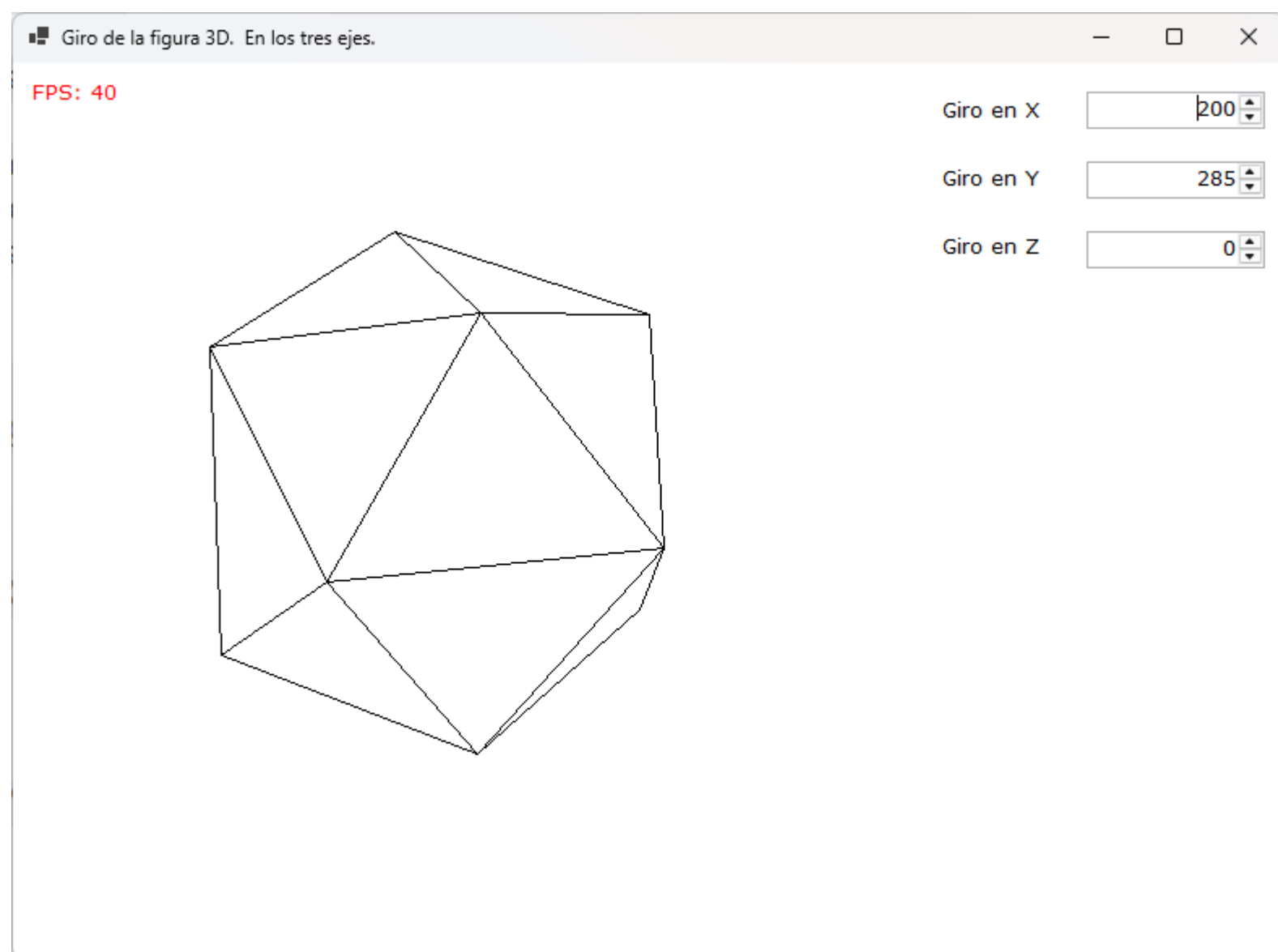
```

    Refresh();
}

private void numGiroY_ValueChanged(object sender, EventArgs e) {
    Refresh();
}

private void numGiroZ_ValueChanged(object sender, EventArgs e) {
    Refresh();
}
}
}

```



*Ilustración 21: Icosaedro pintado con líneas ocultas*

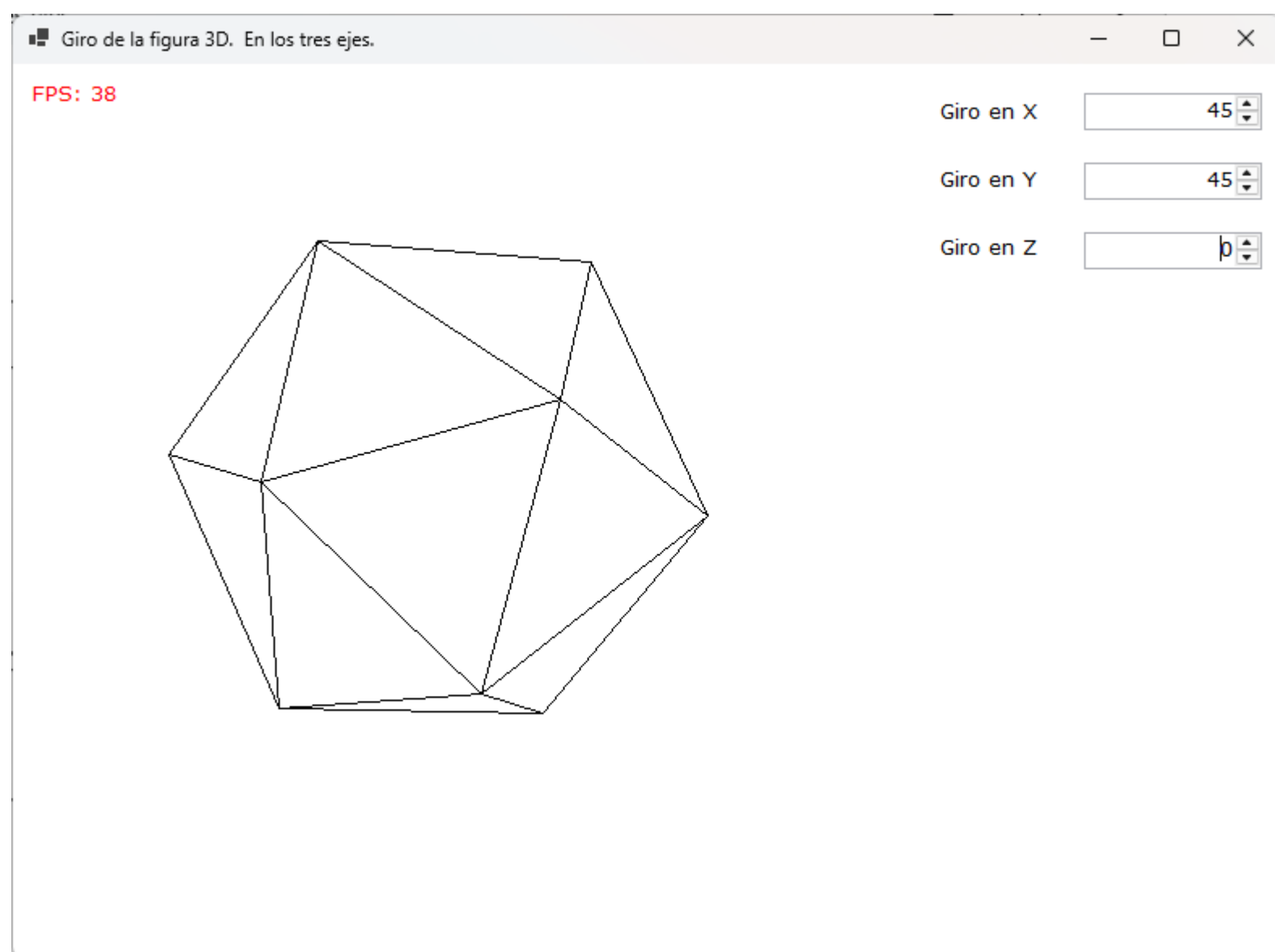


Ilustración 22: Icosaedro pintado con líneas ocultas

Líneas ocultas. Optimización del código.

Algunas operaciones no son necesarias repetirlas cuando el icosaedro gira. Luego se analiza el código buscando como mejorar su velocidad.

M/010.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        private double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}
```

```
namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3;
        private Point[] ListaPuntos;

        public Poligono(int punto1, int punto2, int punto3) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos) {
            double Z1 = puntos[punto1].Zgiro;
```

```

double Z2 = puntos[punto2].Zgiro;
double Z3 = puntos[punto3].Zgiro;
Centro = (Z1 + Z2 + Z3) / 3;

Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
ListaPuntos = [Polig1, Polig2, Polig3];
}

//Hace el gráfico del polígono
public void Dibuja(Graphics Lienzo, Pen Lapiz, Brush Relleno) {
    //Dibuja el polígono relleno y eso borra los polígonos que no se ven
    Lienzo.FillPolygon(Relleno, ListaPuntos);

    //Dibuja el perímetro del polígono haciéndolo visible.
    Lienzo.DrawPolygon(Lapiz, ListaPuntos);
}
}
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Constructor
        public Objeto3D() {
            //Ejemplo de coordenadas espaciales X,Y,Z
            puntos = [
                new Punto( 0.000000000000, 0.361803, 0.5),
                new Punto( 0.000000000000, 0.361803, -0.5),
                new Punto( 0.000000000000, -0.361803, 0.5),
                new Punto( 0.000000000000, -0.361803, -0.5),
                new Punto( 0.361803, 0.5, 0.000000000000),
                new Punto( 0.361803, -0.5, 0.000000000000),
                new Punto(-0.361803, 0.5, 0.000000000000),
                new Punto(-0.361803, -0.5, 0.000000000000),
                new Punto( 0.5, 0.000000000000, 0.361803),
                new Punto( 0.5, 0.000000000000, -0.361803),
                new Punto(-0.5, 0.000000000000, 0.361803),
                new Punto(-0.5, 0.000000000000, -0.361803),
            ];

            //Los polígonos que dibujan el icosaedro
            poligonos = [
                new Poligono(0, 6, 4),
                new Poligono(0, 4, 8),
                new Poligono(0, 8, 2),
                new Poligono(0, 2, 10),
                new Poligono(0, 10, 6),
                new Poligono(1, 9, 4),
                new Poligono(1, 4, 6),
                new Poligono(1, 6, 11),
                new Poligono(1, 11, 3),
                new Poligono(1, 3, 9),
                new Poligono(2, 8, 5),
                new Poligono(2, 5, 7),
                new Poligono(2, 7, 10),
                new Poligono(3, 11, 7),
                new Poligono(3, 7, 5),
                new Poligono(3, 5, 9),
                new Poligono(6, 10, 11),
                new Poligono(5, 8, 9),
                new Poligono(7, 11, 10),
                new Poligono(4, 9, 8)
            ];
        }
    }
}

```

```

    public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int
YpantallaIni, int XpantallaFin, int YpantallaFin) {
    //Para la matriz de rotación
    double CosX = Math.Cos(angX);
    double SinX = Math.Sin(angX);
    double CosY = Math.Cos(angY);
    double SinY = Math.Sin(angY);
    double CosZ = Math.Cos(angZ);
    double SinZ = Math.Sin(angZ);

    //Matriz de Rotación
    //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
    double[,] Matriz = new double[3, 3] {
        { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
        { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
        {-SinY, SinX * CosY, CosX * CosY }
    };

    //Las constantes de transformación para cuadrar en pantalla
    double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
    double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

    //Gira los 8 puntos
    for (int cont = 0; cont < puntos.Count; cont++)
        puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

    //Calcula la profundidad y forma el polígono
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].ProfundidadFigura(puntos);

    //Algoritmo de pintor.
    //Ordena del polígono más alejado al más cercano,
    //los polígonos de adelante son visibles y los de atrás son borrados.
    poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja el icosaedro
public void Dibuja(Graphics lienzo, Pen lapizIcosaedro, Brush relleno) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo, lapizIcosaedro, relleno);
}
}
}

```

```

using System.Diagnostics;

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {

        //Para calcular los FPS (Frames Per Second)
        private Stopwatch cronometro = new Stopwatch();
        private int fps = 0;
        private int contadorFrames = 0;
        private long tiempoAcumulado = 0;

        //El icosaedro que se proyecta y gira
        Objeto3D Figura3D;

        //Tamaño de pantalla
        double AnguloX, AnguloY, AnguloZ, ZPersona;
        const double Radianes = Math.PI / 180;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;

        //Para dibujar
        Pen LapizIcosaedro = new(Color.Black, 1);
        Brush Relleno = new SolidBrush(Color.White);

        public Form1() {
            InitializeComponent();

            //Para calcular los FPS (Frames Per Second)
            cronometro.Start();

```



```

System.Windows.Forms.Timer timer = new System.Windows.Forms.Timer();
timer.Interval = 15; // ~60 FPS
timer.Tick += (s, e) => this.Invalidate(); // Forzar redibujo
timer.Start();

//Distancia del observador
ZPersona = 5;

//Tamaño de pantalla
XpantallaIni = 20;
YpantallaIni = 20;
XpantallaFin = 500;
YpantallaFin = 500;

Figura3D = new Objeto3D();

//Gira, proyecta y cuadra cada punto de la figura 3D
Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
}

private void Form1_Paint(object sender, PaintEventArgs e) {

    //Dibuja la figura 3D
    Figura3D.Dibuja(e.Graphics, LapizIcosaedro, Relleno);

    //Calcular los FPS
    contadorFrames++;
    tiempoAcumulado += cronometro.ElapsedMilliseconds;
    cronometro.Restart();

    if (tiempoAcumulado >= 1000) { // cada segundo
        fps = contadorFrames;
        contadorFrames = 0;
        tiempoAcumulado = 0;
    }

    // Dibujar texto con los FPS
    e.Graphics.DrawString($"FPS: {fps}", this.Font, Brushes.Red, 10, 10);
}

private void numGiroX_ValueChanged(object sender, EventArgs e) {
    AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}

private void numGiroY_ValueChanged(object sender, EventArgs e) {
    AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}

private void numGiroZ_ValueChanged(object sender, EventArgs e) {
    AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}
}
}

```

Gráfico Matemático en 3D

Cuando tenemos una ecuación del tipo  $Z = F(X,Y)$ , tenemos una ecuación con dos variables independientes y una variable dependiente. Su representación es en 3D. Por ejemplo:

$$Z = \sqrt[2]{X^2 + Y^2} + 3 * Cos(\sqrt[2]{X^2 + Y^2}) + 5$$

Los pasos para hacer el gráfico son los siguientes (implementación en M/011):

Paso 1

Saber el valor mínimo de X (Xini) hasta el valor máximo de X (Xfin). Igual sucede con Y, el valor mínimo de Y (Yini) hasta el valor máximo de Y (Yfin). También se pregunta cuantas líneas tendrá el gráfico (NumLineas), entre más líneas, más detallado será el gráfico.

```
double Xini = -10;
double Yini = -10;
double Xfin = 10;
double Yfin = 10;
int NumLineas = 40;
```

Paso 2

Se calcula el avance en X que sería:  $IncrX = (Xfin - Xini) / (NumLineas-1)$

Se calcula el avance en Y que sería:  $IncrY = (Yfin - Yini) / (NumLineas-1)$

```
double IncrX = (Xfin - Xini) / (NumLineas-1);
double IncrY = (Yfin - Yini) / (NumLineas-1);
double X = Xini, Y = Yini;
```

Paso 3

El gráfico requiere una lista de objetos tipo punto. Cada objeto tiene:

- a. Los puntos X, Y, Z (Z es calculado). Si el valor de Z no es real (infinito o NaN), hace que el punto no sea válido, luego no se usará más adelante.
- b. Los puntos Xg, Yg, Zg (más adelante se calculan)
- c. Los puntos planoX, planoY (más adelante se calculan)
- d. Los puntos Xp, Yp (más adelante se calculan)

```
//Coordenadas espaciales X,Y,Z
puntos = [];
for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
    for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
        double Z = Ecuacion(X, Y);
        puntos.Add(new Punto(X, Y, Z));
        X += IncrX;
    }
    X = Xini;
    Y += IncrY;
}
```

Paso 4

Se normalizan los valores de X, Y, Z de cada objeto, para que queden entre 0 y 1, luego se le resta -0.5 ¿Para qué? Para que los puntos queden contenidos dentro de un cubo de lado=1, cuyo centro está en 0,0,0. Ver los dos temas anteriores como se muestra el cubo.

```
//Debe normalizar las coordenadas y ponerlas entre -0.5 y 0.5
double Xmin = double.MaxValue;
double Ymin = double.MaxValue;
double Zmin = double.MaxValue;
double Xmax = double.MinValue;
double Ymax = double.MinValue;
double Zmax = double.MinValue;
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    if (puntos[Cont].X < Xmin) Xmin = puntos[Cont].X;
    if (puntos[Cont].Y < Ymin) Ymin = puntos[Cont].Y;
    if (puntos[Cont].Z < Zmin) Zmin = puntos[Cont].Z;
    if (puntos[Cont].X > Xmax) Xmax = puntos[Cont].X;
```



```

        if (puntos[Cont].Y > Ymax) Ymax = puntos[Cont].Y;
        if (puntos[Cont].Z > Zmax) Zmax = puntos[Cont].Z;
    }

    for (int Cont = 0; Cont < puntos.Count; Cont++) {
        if (puntos[Cont].Valido == false) continue;
        puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
        puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
        puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
    }

```

## Paso 5

Se forman los polígonos con cuatro puntos

```

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    // siempre y cuando los vértices sean válidos
    if (puntos[coordenadaActual].Valido &&
        puntos[coordenadaActual + 1].Valido &&
        puntos[coordenadaActual + NumLineas + 1].Valido &&
        puntos[coordenadaActual + NumLineas].Valido) {

        poligonos.Add(new Poligono(
            coordenadaActual,
            coordenadaActual + 1,
            coordenadaActual + NumLineas + 1,
            coordenadaActual + NumLineas
        ));
    }

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

```

## Paso 6

Se gira cada polígono. En el código es sencillo: cada polígono tiene 4 puntos donde cada punto está en una lista, luego en vez de girar polígono a polígono, se gira toda la lista de puntos.

```

public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int
YpantallaIni, int XpantallaFin, int YpantallaFin) {
    //Para la matriz de rotación
    double CosX = Math.Cos(angX);
    double SinX = Math.Sin(angX);
    double CosY = Math.Cos(angY);
    double SinY = Math.Sin(angY);
    double CosZ = Math.Cos(angZ);
    double SinZ = Math.Sin(angZ);

    //Matriz de Rotación
    //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
    double[,] Matriz = new double[3, 3] {
        { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
        { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
        {-SinY, SinX * CosY, CosX * CosY }
    };

    //Las constantes de transformación para cuadrar en pantalla
    double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;

```

```

double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

//Gira los puntos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

```

Y así se cuadra cada punto en pantalla

```

//Giro en los tres ejes
public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

    //Hace el giro
    double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
    double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
    Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

    //Convierte de 3D a 2D (segunda dimensión)
    double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
    double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

    //Deduce las coordenadas de pantalla
    Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
    Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
}

```

## Paso 7

Se dibuja cada polígono

```

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo, Pen lapizIcosaedro, Brush relleno) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo, lapizIcosaedro, relleno);
}

```

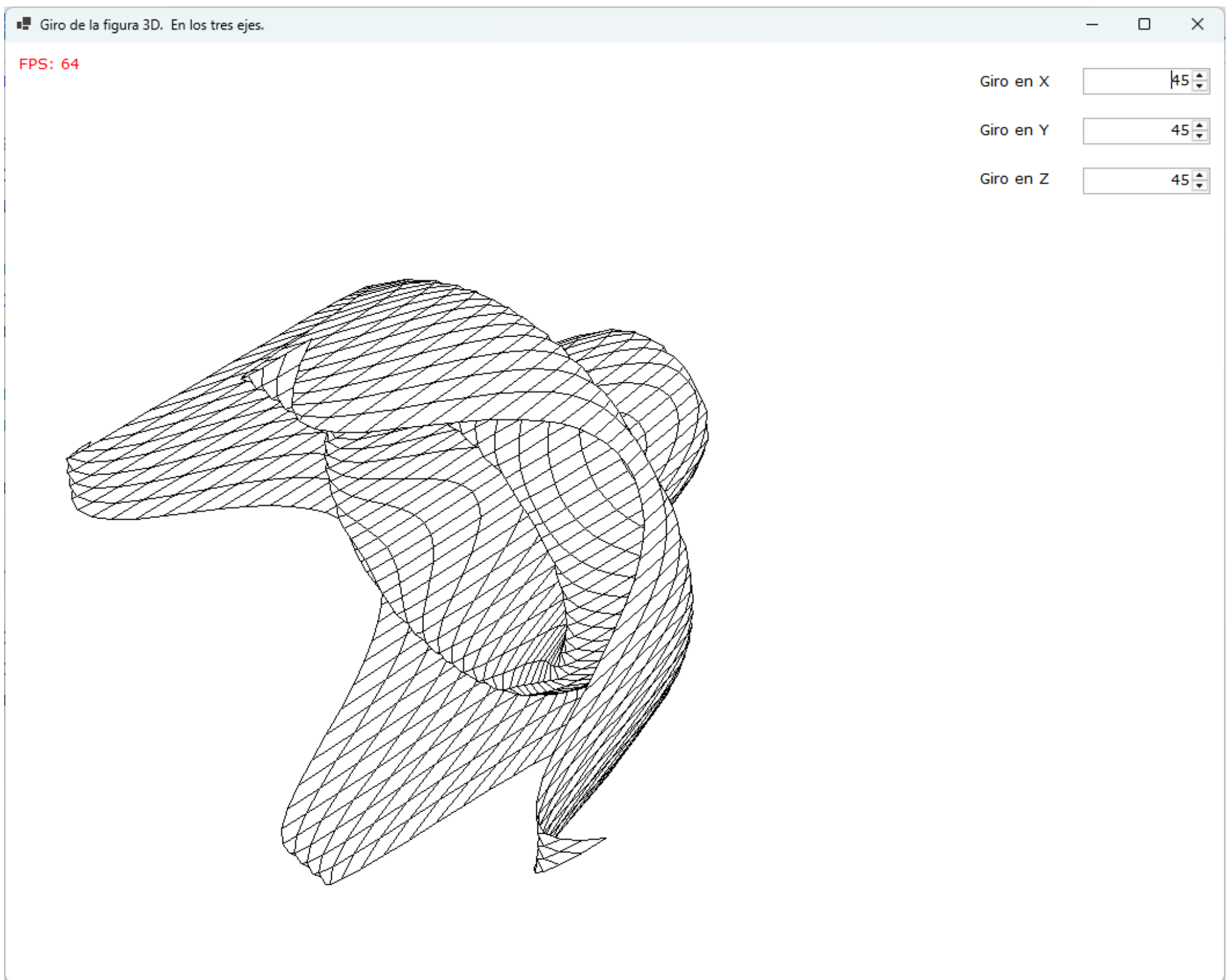


Ilustración 23: Ecuación en 3D proyectada

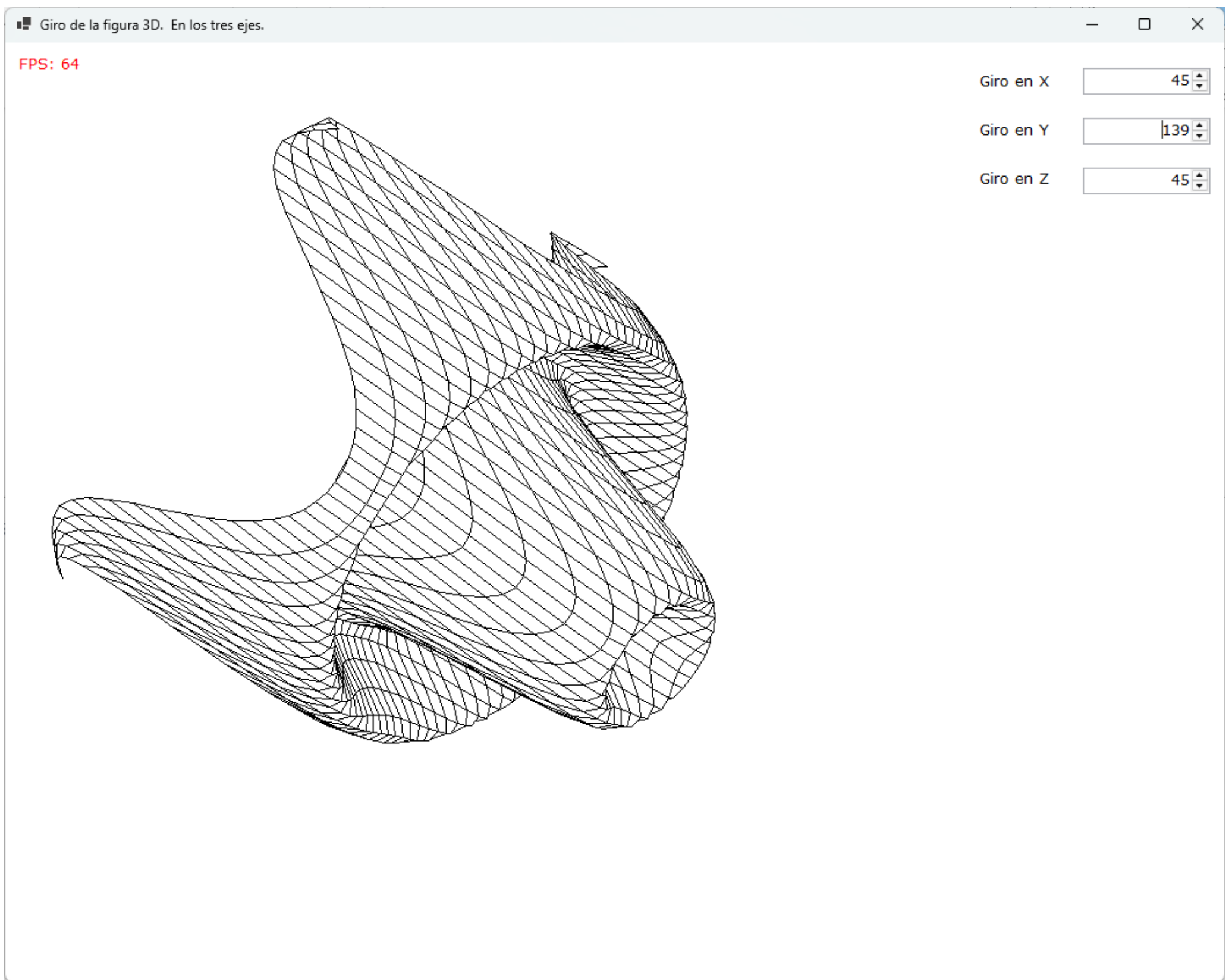


Ilustración 24: Ecuación en 3D proyectada

M/011.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public bool Valido; //Si el punto es válido
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            Valido = true;
            if (double.IsFinite(Z)) {
                this.Z = Z;
            } else {
                Valido = false;
            }
        }
    }

    //Giro en los tres ejes
    public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
    convierteX, double convierteY) {
        //Si el punto no es válido, no hace nada
        if (Valido == false) return;

        //Hace el giro
        double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
        double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
    }
}
```

```

        Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

        //Convierte de 3D a 2D (segunda dimensión)
        double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
        double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

        //Deduce las coordenadas de pantalla
        Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
        Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
    }
}
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            double Z4 = puntos[punto4].Zgiro;
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;

            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo, Pen Lapiz, Brush Relleno) {
            //Dibuja el polígono relleno y eso borra los polígonos que no se ven
            Lienzo.FillPolygon(Relleno, ListaPuntos);

            //Dibuja el perímetro del polígono haciéndolo visible.
            Lienzo.DrawPolygon(Lapiz, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
    }
}

```

```

private List<Poligono> poligonos;

//Constructor
public Objeto3D() {
    double Xini = -10;
    double Yini = -10;
    double Xfin = 10;
    double Yfin = 10;
    int NumLineas = 40;

    double IncrX = (Xfin - Xini) / (NumLineas-1);
    double IncrY = (Yfin - Yini) / (NumLineas-1);
    double X = Xini, Y = Yini;

    //Coordenadas espaciales X,Y,Z
    puntos = [];
    for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            double Z = Ecuacion(X, Y);
            puntos.Add(new Punto(X, Y, Z));
            X += IncrX;
        }
        X = Xini;
        Y += IncrY;
    }

    //Debe normalizar las coordenadas y ponerlas entre -0.5 y 0.5
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;
    for (int Cont = 0; Cont < puntos.Count; Cont++) {
        if (puntos[Cont].Valido == false) continue;
        if (puntos[Cont].X < Xmin) Xmin = puntos[Cont].X;
        if (puntos[Cont].Y < Ymin) Ymin = puntos[Cont].Y;
        if (puntos[Cont].Z < Zmin) Zmin = puntos[Cont].Z;
        if (puntos[Cont].X > Xmax) Xmax = puntos[Cont].X;
        if (puntos[Cont].Y > Ymax) Ymax = puntos[Cont].Y;
        if (puntos[Cont].Z > Zmax) Zmax = puntos[Cont].Z;
    }

    for (int Cont = 0; Cont < puntos.Count; Cont++) {
        if (puntos[Cont].Valido == false) continue;
        puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
        puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
        puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
    }

    // Inicializa la lista de polígonos
    poligonos = [];

    int coordenadaActual = 0;
    int filaActual = 1;
    int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

    for (int Cont = 0; Cont < totalPoligonos; Cont++) {
        // Crea un polígono con las coordenadas de los vértices
        // siempre y cuando los vértices sean válidos
        if (puntos[coordenadaActual].Valido &&
            puntos[coordenadaActual + 1].Valido &&
            puntos[coordenadaActual + NumLineas + 1].Valido &&
            puntos[coordenadaActual + NumLineas].Valido) {

            poligonos.Add(new Poligono(
                coordenadaActual,
                coordenadaActual + 1,
                coordenadaActual + NumLineas + 1,
                coordenadaActual + NumLineas
            ));
        }

        coordenadaActual++;

        // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    }

```



```

        if (coordenadaActual == NumLineas * filaActual - 1) {
            coordenadaActual++;
            filaActual++;
        }
    }
}

public double Ecuacion(double X, double Y) {
    double Z = Math.Sqrt(X*X + Y*Y) + 3 * Math.Cos(Math.Sqrt(X * X + Y * Y)) + 5;
    return Z;
}

public void GirarFigura(double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {
    //Para la matriz de rotación
    double CosX = Math.Cos(angX);
    double SinX = Math.Sin(angX);
    double CosY = Math.Cos(angY);
    double SinY = Math.Sin(angY);
    double CosZ = Math.Cos(angZ);
    double SinZ = Math.Sin(angZ);

    //Matriz de Rotación
    //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
    double[,] Matriz = new double[3, 3] {
        { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
        { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
        {-SinY, SinX * CosY, CosX * CosY }
    };

    //Las constantes de transformación para cuadrar en pantalla
    double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
    double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

    //Gira los puntos
    for (int cont = 0; cont < puntos.Count; cont++)
        puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

    //Calcula la profundidad y forma el polígono
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].ProfundidadFigura(puntos);

    //Algoritmo de pintor.
    //Ordena del polígono más alejado al más cercano,
    //los polígonos de adelante son visibles y los de atrás son borrados.
    poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo, Pen lapizIcosaedro, Brush relleno) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo, lapizIcosaedro, relleno);
}
}
}

```

```

using System.Diagnostics;

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {

        //Para calcular los FPS (Frames Per Second)
        private Stopwatch cronometro = new Stopwatch();
        private int fps = 0;
        private int contadorFrames = 0;
        private long tiempoAcumulado = 0;

        //El icosaedro que se proyecta y gira
        Objeto3D Figura3D;

        //Tamaño de pantalla
        double AnguloX, AnguloY, AnguloZ, ZPersona;
        const double Radianes = Math.PI / 180;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;
    }
}

```

```

//Para dibujar
Pen LapizIcosaedro = new(Color.Black, 1);
Brush Relleno = new SolidBrush(Color.White);

public Form1() {
    InitializeComponent();

    //Para calcular los FPS (Frames Per Second)
    cronometro.Start();
    System.Windows.Forms.Timer timer = new System.Windows.Forms.Timer();
    timer.Interval = 15; // ~60 FPS
    timer.Tick += (s, e) => this.Invalidate(); // Forzar redibujo
    timer.Start();

    //Distancia del observador
    ZPersona = 5;

    //Tamaño de pantalla
    XpantallaIni = 20;
    YpantallaIni = 20;
    XpantallaFin = 800;
    YpantallaFin = 800;

    Figura3D = new Objeto3D();

    //Gira, proyecta y cuadra cada punto de la figura 3D
    AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
    AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
    AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
}

private void Form1_Paint(object sender, PaintEventArgs e) {

    //Dibuja la figura 3D
    Figura3D.Dibuja(e.Graphics, LapizIcosaedro, Relleno);

    //Calcular los FPS
    contadorFrames++;
    tiempoAcumulado += cronometro.ElapsedMilliseconds;
    cronometro.Restart();

    if (tiempoAcumulado >= 1000) { // cada segundo
        fps = contadorFrames;
        contadorFrames = 0;
        tiempoAcumulado = 0;
    }

    // Dibujar texto con los FPS
    e.Graphics.DrawString($"FPS: {fps}", this.Font, Brushes.Red, 10, 10);
}

private void numGiroX_ValueChanged(object sender, EventArgs e) {
    AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}

private void numGiroY_ValueChanged(object sender, EventArgs e) {
    AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}

private void numGiroZ_ValueChanged(object sender, EventArgs e) {
    AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
    Figura3D.GirarFigura(AnguloX, AnguloY, AnguloZ, ZPersona, XpantallaIni, YpantallaIni, XpantallaFin,
YpantallaFin);
    Refresh();
}
}
}

```



Gráfico Matemático en 3D. Colores según altura y evaluador de expresiones.

En cada polígono a dibujar se tiene en cuenta el valor de Z original sin girar (promedio de los 4 puntos que conforman al polígono) de la ecuación. Luego se hace un gradiente de color entre los dos extremos valores de Z y así se dibuja por colores por niveles.

La ecuación no es fija sino que el usuario la ingresa y es evaluada por el programa.

M/012.zip

```
using System.Globalization;
using System.Text;

/* Evaluador de expresiones versión 4 (enero de 2024)
 * Autor: Rafael Alberto Moreno Parra
 * Correo: ramsoftware@gmail.com ;  enginelife@hotmail.com
 * URL: http://darwin.50webs.com
 * GitHub: https://github.com/ramsoftware
 * */

namespace Graficos {
    internal class ParteEvl4 {
        /* Constantes de los diferentes tipos
         * de datos que tendrán las piezas */
        private const int ESFUNCION = 1;
        private const int ESPARABRE = 2;
        private const int ESOPERADOR = 4;
        private const int ESNUMERO = 5;
        private const int ESVARIABLE = 6;

        /* Acumulador, función, paréntesis que abre,
         * paréntesis que cierra, operador,
         * número, variable */
        public int Tipo;

        /* Código de la función 0:seno, 1:coseno, 2:tangente,
         * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
         * 6: arcotangente, 7: logaritmo natural, 8: exponencial
         * 9: raíz cuadrada */
        public int Funcion;

        /* + suma - resta * multiplicación / división ^ potencia */
        public int Operador;

        /* Posición en lista de valores del número literal
         * por ejemplo: 3.141592 */
        public int posNumero;

        /* Posición en lista de valores del
         * valor de la variable algebraica */
        public int posVariable;

        /* Posición en lista de valores del valor de la pieza.
         * Por ejemplo:
         * 3 + 2 / 5 se convierte así:
         * |3| |+| |2| | / | |5|
         * |3| |+| |A| A es un identificador de acumulador */
        public int posAcumula;

        public ParteEvl4(int TipoParte, int Valor) {
            this.Tipo = TipoParte;
            switch (TipoParte) {
                case ESFUNCION: this.Funcion = Valor; break;
                case ESNUMERO: this.posNumero = Valor; break;
                case ESVARIABLE: this.posVariable = Valor; break;
                case ESPARABRE: this.Funcion = -1; break;
            }
        }

        public ParteEvl4(char Operador) {
            this.Tipo = ESOPERADOR;
            switch (Operador) {
                case '+': this.Operador = 0; break;
                case '-': this.Operador = 1; break;
                case '*': this.Operador = 2; break;
                case '/': this.Operador = 3; break;
                case '^': this.Operador = 4; break;
            }
        }
    }
}
```

```

    }
}

internal class PiezaEvl4 {
    /* Posición donde se almacena el valor que genera
    * la pieza al evaluarse */
    public int PosResultado;

    /* Código de la función 0:seno, 1:coseno, 2:tangente,
    * 3: valor absoluto, 4: arcoseno, 5: arcocoseno,
    * 6: arcotangente, 7: logaritmo natural, 8: valor tope,
    * 9: exponencial, 10: raíz cuadrada */
    public int Funcion;

    /* Posición donde se almacena la primera parte de la pieza */
    public int pA;

    /* 0 suma 1 resta 2 multiplicación 3 división 4 potencia */
    public int Operador;

    /* Posición donde se almacena la segunda parte de la pieza */
    public int pB;
}

internal class Evaluador4 {
    /* Constantes de los diferentes tipos
    * de datos que tendrán las piezas */
    private const int ESFUNCION = 1;
    private const int ESPARABRE = 2;
    private const int ESPARCIERRA = 3;
    private const int ESOPERADOR = 4;
    private const int ESNUMERO = 5;
    private const int ESVARIABLE = 6;
    private const int ESACUMULA = 7;

    /* Expresión algebraica convertida de la
    * original dada por el usuario */
    private StringBuilder Analizada = new();

    /* Donde guarda los valores de variables, constantes y piezas */
    private List<double> Valores = new List<double>();

    /* Listado de partes en que se divide la expresión
    Toma una expresión, por ejemplo:
    1.68 + sen( 3 / x ) * ( 2.92 - 9 )
    y la divide en partes así:
    [1.68] [+] [sen() [3] [/] [x] []] [*] [(] [2.92] [-] [9] []]
    Cada parte puede tener un número, un operador, una función,
    un paréntesis que abre o un paréntesis que cierra.
    En esta versión 4.0, las constantes, piezas y variables guardan
    sus valores en la lista Valores.
    En partes, se almacena la posición en Valores */
    private List<ParteEvl4> Partes = new List<ParteEvl4>();

    /* Listado de piezas que ejecutan
    Toma las partes y las divide en piezas con
    la siguiente estructura:

    acumula = funcion valor operador valor

    donde valor puede ser un número, una variable o
    un acumulador

    Siguiendo el ejemplo anterior sería:
    A = 2.92 - 9
    B = sen( 3 / x )
    C = B * A
    D = 1.68 + C

    Esas piezas se evalúan de arriba a abajo y así
    se interpreta la ecuación */
    private List<PiezaEvl4> Piezas = new List<PiezaEvl4>();

    /* Analiza la expresión */

```

```

public int Analizar(string ExpresionOriginal) {
    Partes.Clear();
    Piezas.Clear();
    Valores.Clear();

    /* Hace espacio para las 26 variables que
    * puede manejar el evaluador */
    for (int Variables = 1; Variables <= 26; Variables++)
        Valores.Add(0);

    int Sintaxis = ChequeaSintaxis(ExpresionOriginal);
    if (Sintaxis == 0) {
        CrearPartes();
        CrearPiezas();
    }
    return Sintaxis;
}

/* Retorna mensaje de error sintáctico */
public string MensajeError(int CodigoError) {
    string Msj = "";
    switch (CodigoError) {
        case 1:
            Msj = "1. Número seguido de letra";
            break;

        case 2:
            Msj = "2. Número seguido de paréntesis que abre";
            break;

        case 3:
            Msj = "3. Doble punto seguido";
            break;

        case 4:
            Msj = "4. Punto seguido de operador";
            break;

        case 5:
            Msj = "5. Punto y sigue una letra";
            break;

        case 6:
            Msj = "6. Punto seguido de paréntesis que abre";
            break;

        case 7:
            Msj = "7. Punto seguido de paréntesis que cierra";
            break;

        case 8:
            Msj = "8. Operador seguido de un punto";
            break;

        case 9:
            Msj = "9. Dos operadores estén seguidos";
            break;

        case 10:
            Msj = "10. Operador seguido de paréntesis que cierra";
            break;

        case 11:
            Msj = "11. Letra seguida de número";
            break;

        case 12:
            Msj = "12. Letra seguida de punto";
            break;

        case 13:
            Msj = "13. Letra seguida de otra letra";
            break;

        case 14:
            Msj = "14. Letra seguida de paréntesis que abre";

```

```

        break;

    case 15:
        Msj = "15. Paréntesis que abre seguido de punto";
        break;

    case 16:
        Msj = "16. Paréntesis que abre y sigue operador";
        break;

    case 17:
        Msj = "17. Paréntesis que abre y luego cierra";
        break;

    case 18:
        Msj = "18. Paréntesis que cierra y sigue número";
        break;

    case 19:
        Msj = "19. Paréntesis que cierra y sigue punto";
        break;

    case 20:
        Msj = "20. Paréntesis que cierra y sigue letra";
        break;

    case 21:
        Msj = "21. Paréntesis que cierra y luego abre";
        break;

    case 22:
        Msj = "22. Inicia con operador";
        break;

    case 23:
        Msj = "23. Finaliza con operador";
        break;

    case 24:
        Msj = "24. No hay correspondencia entre paréntesis";
        break;

    case 25:
        Msj = "25. Paréntesis desbalanceados";
        break;

    case 26:
        Msj = "26. Dos o más puntos en número real";
        break;

    case 27:
        Msj = "27. Expresión vacía";
        break;
    }
    return Msj;
}

/* Da valor a las variables que tendrá
 * la expresión algebraica */
public void DarValorVariable(char varAlgebra, double Valor) {
    Valores[varAlgebra - 'a'] = Valor;
}

/* Evalúa la expresión convertida en piezas */
public double Evaluar() {
    double Resulta = 0;
    PiezaEvl4 tmp;

    /* Va de pieza en pieza */
    for (int Posicion = 0; Posicion < Piezas.Count; Posicion++) {
        tmp = Piezas[Posicion];

        switch (tmp.Operador) {
            case 0:
                Resulta = Valores[tmp.pA] + Valores[tmp.pB];
                break;

```

```

        case 1:
            Resulta = Valores[tmp.pA] - Valores[tmp.pB];
            break;
        case 2:
            Resulta = Valores[tmp.pA] * Valores[tmp.pB];
            break;
        case 3:
            Resulta = Valores[tmp.pA] / Valores[tmp.pB];
            break;
        default:
            Resulta = Math.Pow(Valores[tmp.pA], Valores[tmp.pB]);
            break;
    }

    switch (tmp.Funcion) {
        case 0: Resulta = Math.Sin(Resulta); break;
        case 1: Resulta = Math.Cos(Resulta); break;
        case 2: Resulta = Math.Tan(Resulta); break;
        case 3: Resulta = Math.Abs(Resulta); break;
        case 4: Resulta = Math.Asin(Resulta); break;
        case 5: Resulta = Math.Acos(Resulta); break;
        case 6: Resulta = Math.Atan(Resulta); break;
        case 7: Resulta = Math.Log(Resulta); break;
        case 8: Resulta = Math.Exp(Resulta); break;
        case 9: Resulta = Math.Sqrt(Resulta); break;
    }

    Valores[tmp.PosResultado] = Resulta;
}
return Resulta;
}

/* Divide la expresión en partes, yendo de caracter en caracter */
private void CrearPartes() {
    StringBuilder Numero = new StringBuilder();
    for (int Pos = 0; Pos < this.Analizada.Length; Pos++) {
        char Letra = this.Analizada[Pos];
        switch (Letra) {
            case '.':
            case '0':
            case '1':
            case '2':
            case '3':
            case '4':
            case '5':
            case '6':
            case '7':
            case '8':
            case '9': /* Si es un dígito o un punto
                       * va acumulando el número */
                Numero.Append(Letra); break;
            case '+':
            case '-':
            case '*':
            case '/':
            case '^':
                /* Si es un operador matemático entonces verifica
                 * si hay un número que se ha acumulado */
                if (Numero.Length > 0) {
                    Valores.Add(Convierte(Numero));
                    Partes.Add(new ParteEv14(ESNUMERO, Valores.Count - 1));
                    Numero.Clear();
                }
                /* Agregar el operador matemático */
                Partes.Add(new ParteEv14(Letra));
                break;
            case '(': /* Es paréntesis que abre */
                Partes.Add(new ParteEv14(ESPARABRE, 0));
                break;
            case ')':
                /* Si es un paréntesis que cierra entonces verifica
                 * si hay un número que se ha acumulado */
                if (Numero.Length > 0) {
                    Valores.Add(Convierte(Numero));

```

```

        Partes.Add(new ParteEvl4(ESNUMERO, Valores.Count - 1));
        Numero.Clear();
    }

    /* Si sólo había un número o variable
    * dentro del paréntesis le agrega + 0
    * (por ejemplo: sen(x) o 3*(2) ) */
    if (Partes[Partes.Count - 2].Tipo == ESPARABRE ||
        Partes[Partes.Count - 2].Tipo == ESFUNCION) {
        Partes.Add(new ParteEvl4(ESOPERADOR, 0));
        Valores.Add(0);
        Partes.Add(new ParteEvl4(ESNUMERO, Valores.Count - 1));
    }

    /* Adiciona el paréntesis que cierra */
    Partes.Add(new ParteEvl4(ESPARCIERRA, 0));
    break;
case 'A':    /* Seno */
case 'B':    /* Coseno */
case 'C':    /* Tangente */
case 'D':    /* Valor absoluto */
case 'E':    /* Arcoseno */
case 'F':    /* Arcocoseno */
case 'G':    /* Arcotangente */
case 'H':    /* Logaritmo natural */
case 'I':    /* Exponencial */
case 'J':    /* Raíz cuadrada */
    Partes.Add(new ParteEvl4(ESFUNCION, Letra - 'A'));
    Pos++;
    break;
default:
    Partes.Add(new ParteEvl4(ESVARIABLE, Letra - 'a'));
    break;
}
}
}

private double Convierte(StringBuilder Numero) {
    string Cad = Numero.ToString();
    return double.Parse(Cad, CultureInfo.InvariantCulture);
}

/* Convierte las partes en las piezas finales de ejecución */
private void CrearPiezas() {
    int Contador = Partes.Count - 1;
    do {
        ParteEvl4 tmpParte = Partes[Contador];
        if (tmpParte.Tipo == ESPARABRE || tmpParte.Tipo == ESFUNCION) {

            /* Evalúa las potencias */
            GeneraPiezaOpera(4, 4, Contador);

            /* Luego evalúa multiplicar y dividir */
            GeneraPiezaOpera(2, 3, Contador);

            /* Finalmente evalúa sumar y restar */
            GeneraPiezaOpera(0, 1, Contador);

            /* Agrega la función a la última pieza */
            if (tmpParte.Tipo == ESFUNCION) {
                Piezas[Piezas.Count - 1].Funcion = tmpParte.Funcion;
            }

            /* Quita el paréntesis/función que abre y
            * el que cierra, dejando el centro */
            Partes.RemoveAt(Contador);
            Partes.RemoveAt(Contador + 1);
        }
        Contador--;
    } while (Contador >= 0);
}

/* Genera las piezas buscando determinado operador */
private void GeneraPiezaOpera(int OperA, int OperB, int Inicia) {
    int Contador = Inicia + 1;
    do {

```



```

ParteEvl4 tmpParte = Partes[Contador];
if (tmpParte.Tipo == ESOPERADOR &&
    (tmpParte.Operador == OperA || tmpParte.Operador == OperB)) {
    ParteEvl4 tmpParteIzq = Partes[Contador - 1];
    ParteEvl4 tmpParteDer = Partes[Contador + 1];

    /* Crea Pieza */
    PiezaEvl4 NuevaPieza = new PiezaEvl4();
    NuevaPieza.Funcion = -1;

    switch (tmpParteIzq.Tipo) {
        case ESNUMERO:
            NuevaPieza.pA = tmpParteIzq.posNumero;
            break;

        case ESVARIABLE:
            NuevaPieza.pA = tmpParteIzq.posVariable;
            break;

        default:
            NuevaPieza.pA = tmpParteIzq.posAcumula;
            break;
    }

    NuevaPieza.Operador = tmpParte.Operador;

    switch (tmpParteDer.Tipo) {
        case ESNUMERO:
            NuevaPieza.pB = tmpParteDer.posNumero;
            break;

        case ESVARIABLE:
            NuevaPieza.pB = tmpParteDer.posVariable;
            break;

        default:
            NuevaPieza.pB = tmpParteDer.posAcumula;
            break;
    }

    /* Añade a lista de piezas y crea una
     * nueva posición en Valores */
    Valores.Add(0);
    NuevaPieza.PosResultado = Valores.Count - 1;
    Piezas.Add(NuevaPieza);

    /* Elimina la parte del operador y la siguiente */
    Partes.RemoveAt(Contador);
    Partes.RemoveAt(Contador);

    /* Retorna el contador en uno para tomar
     * la siguiente operación */
    Contador--;

    /* Cambia la parte anterior por parte que acumula */
    tmpParteIzq.Tipo = ESACUMULA;
    tmpParteIzq.posAcumula = NuevaPieza.PosResultado;
}
Contador++;
} while (Partes[Contador].Tipo != ESPARCIERRA);
}

private int ChequeaSintaxis(string ExpOrig) {
    /* Primero a minúsculas */
    StringBuilder Minusculas = new StringBuilder(ExpOrig.ToLower());

    /* Sólo los caracteres válidos */
    string Valido = "abcdefghijklmnopqrstuvwxyz0123456789.+*/^()";
    HashSet<char> Permite = new HashSet<char>(Valido);
    StringBuilder ConLetrasValidas = new StringBuilder("(");
    for (int Cont = 0; Cont < Minusculas.Length; Cont++)
        if (Permite.Contains(Minusculas[Cont]))
            ConLetrasValidas.Append(Minusculas[Cont]);
    ConLetrasValidas.Append(')');

    /* Agrega +0) donde exista )) porque es

```



```

    * necesario para crear las piezas */
string nuevo = ConLetrasValidas.ToString();
if (nuevo.Length == 2) return 27;
while (nuevo.IndexOf(")") != -1)
    nuevo = nuevo.Replace(")", "+0)");
ConLetrasValidas = new StringBuilder(nuevo);

/* Validación de sintaxis, se genera una copia
 * y allí se reemplaza las funciones por a+( */
StringBuilder sbSintax = new StringBuilder();
sbSintax.Append(ConLetrasValidas);
sbSintax.Replace("sen(", "a+(");
sbSintax.Replace("cos(", "a+(");
sbSintax.Replace("tan(", "a+(");
sbSintax.Replace("abs(", "a+(");
sbSintax.Replace("asn(", "a+(");
sbSintax.Replace("acs(", "a+(");
sbSintax.Replace("atn(", "a+(");
sbSintax.Replace("log(", "a+(");
sbSintax.Replace("exp(", "a+(");
sbSintax.Replace("sqr(", "a+(");

for (int Cont = 1; Cont < sbSintax.Length - 2; Cont++) {
    char cA = sbSintax[Cont];
    char cB = sbSintax[Cont + 1];

    if (Char.IsDigit(cA) && Char.IsLower(cB)) return 1;
    if (Char.IsDigit(cA) && cB == '(') return 2;
    if (cA == '.' && cB == '.') return 3;
    if (cA == '.' && EsUnOperador(cB)) return 4;
    if (cA == '.' && Char.IsLower(cB)) return 5;
    if (cA == '.' && cB == '(') return 6;
    if (cA == '.' && cB == ')') return 7;
    if (EsUnOperador(cA) && cB == '.') return 8;
    if (EsUnOperador(cA) && EsUnOperador(cB)) return 9;
    if (EsUnOperador(cA) && cB == ')') return 10;
    if (Char.IsLower(cA) && Char.IsDigit(cB)) return 11;
    if (Char.IsLower(cA) && cB == '.') return 12;
    if (Char.IsLower(cA) && Char.IsLower(cB)) return 13;
    if (Char.IsLower(cA) && cB == '(') return 14;
    if (cA == '(' && cB == '.') return 15;
    if (cA == '(' && EsUnOperador(cB)) return 16;
    if (cA == '(' && cB == ')') return 17;
    if (cA == ')') && Char.IsDigit(cB)) return 18;
    if (cA == ')') && cB == '.') return 19;
    if (cA == ')') && Char.IsLower(cB)) return 20;
    if (cA == ')') && cB == '(') return 21;
}

/* Valida el inicio y fin de la expresión */
if (EsUnOperador(sbSintax[1])) return 22;
if (EsUnOperador(sbSintax[sbSintax.Length - 2])) return 23;

/* Valida balance de paréntesis */
int ParentesisAbre = 0; /* Contador de paréntesis que abre */
int ParentesisCierra = 0; /* Contador de paréntesis que cierra */
for (int Cont = 1; Cont < sbSintax.Length - 1; Cont++) {
    switch (sbSintax[Cont]) {
        case '(': ParentesisAbre++; break;
        case ')': ParentesisCierra++; break;
    }
    if (ParentesisCierra > ParentesisAbre) return 24;
}
if (ParentesisAbre != ParentesisCierra) return 25;

/* Validar los puntos decimales de un número real */
int TotalPuntos = 0;
for (int Cont = 0; Cont < sbSintax.Length; Cont++) {
    if (EsUnOperador(sbSintax[Cont])) TotalPuntos = 0;
    if (sbSintax[Cont] == '.') TotalPuntos++;
    if (TotalPuntos > 1) return 26;
}

/* Deja la expresión para ser analizada.
 * Reemplaza las funciones de tres letras
 * por una letra mayúscula. Cambia los ))

```



```

    * por )+0) porque es requerido al crear las piezas */
this.Analizada.Length = 0;
this.Analizada.Append(ConLetrasValidas);
this.Analizada.Replace("sen", "A");
this.Analizada.Replace("cos", "B");
this.Analizada.Replace("tan", "C");
this.Analizada.Replace("abs", "D");
this.Analizada.Replace("asn", "E");
this.Analizada.Replace("acs", "F");
this.Analizada.Replace("atn", "G");
this.Analizada.Replace("log", "H");
this.Analizada.Replace("exp", "I");
this.Analizada.Replace("sqr", "J");

/* Sintaxis correcta */
return 0;
}

/* Retorna si el Caracter es un operador matemático */
private static bool EsUnOperador(char Caracter) {
    switch (Caracter) {
        case '+':
        case '-':
        case '*':
        case '/':
        case '^':
            return true;
        default:
            return false;
    }
}
}
}
}

```

```

namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public bool Valido; //Si el punto es válido
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            Valido = true;
            if (double.IsFinite(Z)) {
                this.Z = Z;
            }
            else {
                Valido = false;
            }
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
        convierteX, double convierteY) {
            //Si el punto no es válido, no hace nada
            if (Valido == false) return;

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
}

```

```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        //Color de relleno del polígono
        Color ColorZ;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            double Z4 = puntos[punto4].Zgiro;
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;

            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

            int TotalColores = colorList.Count;
            double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2) /
4;
            int ColorEscoge = (int)Math.Floor(TotalColores * PromedioZ);
            ColorZ = colorList[ColorEscoge];
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo) {
            //Dibuja el polígono relleno
            Brush Relleno2 = new SolidBrush(ColorZ);
            Lienzo.FillPolygon(Relleno2, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Colores para pintar la malla
    }
}

```

```

List<Color> ListaColores;

//Ecuación
string EcuacionAnterior = "";
Evaluador4 Evaluador = new();

public Objeto3D() {
    int NumColores = 40; // Número de colores a generar

    //Genera listado de colores para el gráfico
    ListaColores = [];
    int Mitad = NumColores / 2;

    // Gradiente de azul a amarillo
    for (int Cont = 0; Cont < Mitad; Cont++) {
        int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación  $Z = F(X,Y)$ 
//para dibujarla
public void CalcularFigura3D(string Ecuacion, double Xini, double Yini, double Xfin, double Yfin, int
NumLineas, double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int
XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double IncrX = (Xfin - Xini) / (NumLineas - 1);
    double IncrY = (Yfin - Yini) / (NumLineas - 1);
    double X = Xini, Y = Yini;

    //Coordenadas espaciales X,Y,Z
    X = Xini;
    Y = Yini;
    puntos = [];
    for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            Evaluador.DarValorVariable('y', Y);
            double Z = Evaluador.Evaluar();
            puntos.Add(new Punto(X, Y, Z));
            X += IncrX;
        }
        X = Xini;
        Y += IncrY;
    }

    //Debe normalizar las coordenadas y ponerlas entre -0.5 y 0.5
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;

```

```

for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    if (puntos[Cont].X < Xmin) Xmin = puntos[Cont].X;
    if (puntos[Cont].Y < Ymin) Ymin = puntos[Cont].Y;
    if (puntos[Cont].Z < Zmin) Zmin = puntos[Cont].Z;
    if (puntos[Cont].X > Xmax) Xmax = puntos[Cont].X;
    if (puntos[Cont].Y > Ymax) Ymax = puntos[Cont].Y;
    if (puntos[Cont].Z > Zmax) Zmax = puntos[Cont].Z;
}

if (Math.Abs(Xmin - Xmax) < 0.0001 || Math.Abs(Ymin - Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
    MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    // siempre y cuando los vértices sean válidos
    if (puntos[coordenadaActual].Valido &&
        puntos[coordenadaActual + 1].Valido &&
        puntos[coordenadaActual + NumLineas + 1].Valido &&
        puntos[coordenadaActual + NumLineas].Valido) {

        poligonos.Add(new Poligono(
            coordenadaActual,
            coordenadaActual + 1,
            coordenadaActual + NumLineas + 1,
            coordenadaActual + NumLineas
        ));
    }

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
    {-SinY, SinX * CosY, CosX * CosY }
};

//Las constantes de transformación para cuadrar en pantalla
double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

```

```

//Gira los puntos válidos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;
            FrmGrafico.HuboCambio = true;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numMinimoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMinimoX.Value = numMaximoX.Value - 1;

            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numMinimoY_ValueChanged(object sender, EventArgs e) {
            if (numMinimoY.Value >= numMaximoY.Value)

```

```

        numMinimoY.Value = numMaximoY.Value - 1;

        FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }

    private void numMaximoX_ValueChanged(object sender, EventArgs e) {
        if (numMinimoX.Value >= numMaximoX.Value)
            numMaximoX.Value = numMinimoX.Value + 1;

        FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }

    private void numMaximoY_ValueChanged(object sender, EventArgs e) {
        if (numMinimoY.Value >= numMaximoY.Value)
            numMaximoY.Value = numMinimoY.Value + 1;

        FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }

    private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
        FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }

    private void btnProcesar_Click(object sender, EventArgs e) {
        FrmGrafico.Ecuacion = txtEcuacion.Text;
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Giro de figura
        public double AnguloX, AnguloY, AnguloZ;

        //Distancia del observador
        public double ZPersona;

        //Rango de valores
        public double Xini, Yini, Xfin, Yfin;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        //En caso de que el usuario haya hecho algún cambio a los valores
        //se recalcula todo de nuevo.
        public bool HuboCambio;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {

            //Dibuja la figura 3D
            if (HuboCambio) {

```



```
Figura3D.CalcularFigura3D(Ecuacion, Xini, Yini, Xfin, Yfin, NumLineas, AnguloX, AnguloY, AnguloZ,
ZPersona, 0, 0, this.ClientSize.Width, this.ClientSize.Height);
    HuboCambio = false;
}

Figura3D.Dibuja(e.Graphics);
}

private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
    Application.Exit();
}

private void Form1_Resize(object sender, EventArgs e) {
    HuboCambio = true;
    Refresh();
}
}
```

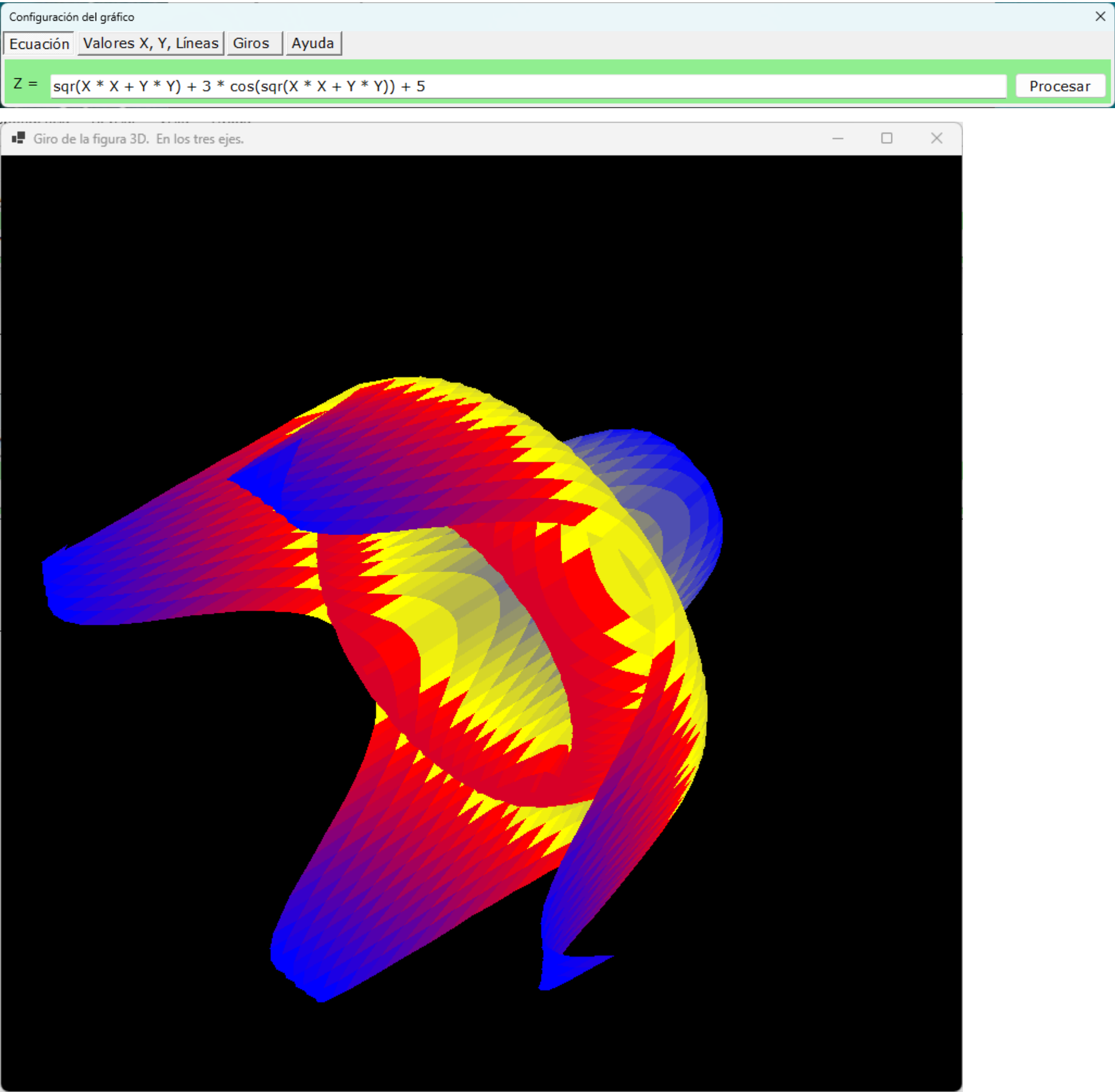


Ilustración 25: Gráfico ecuación 3D con colores según altura



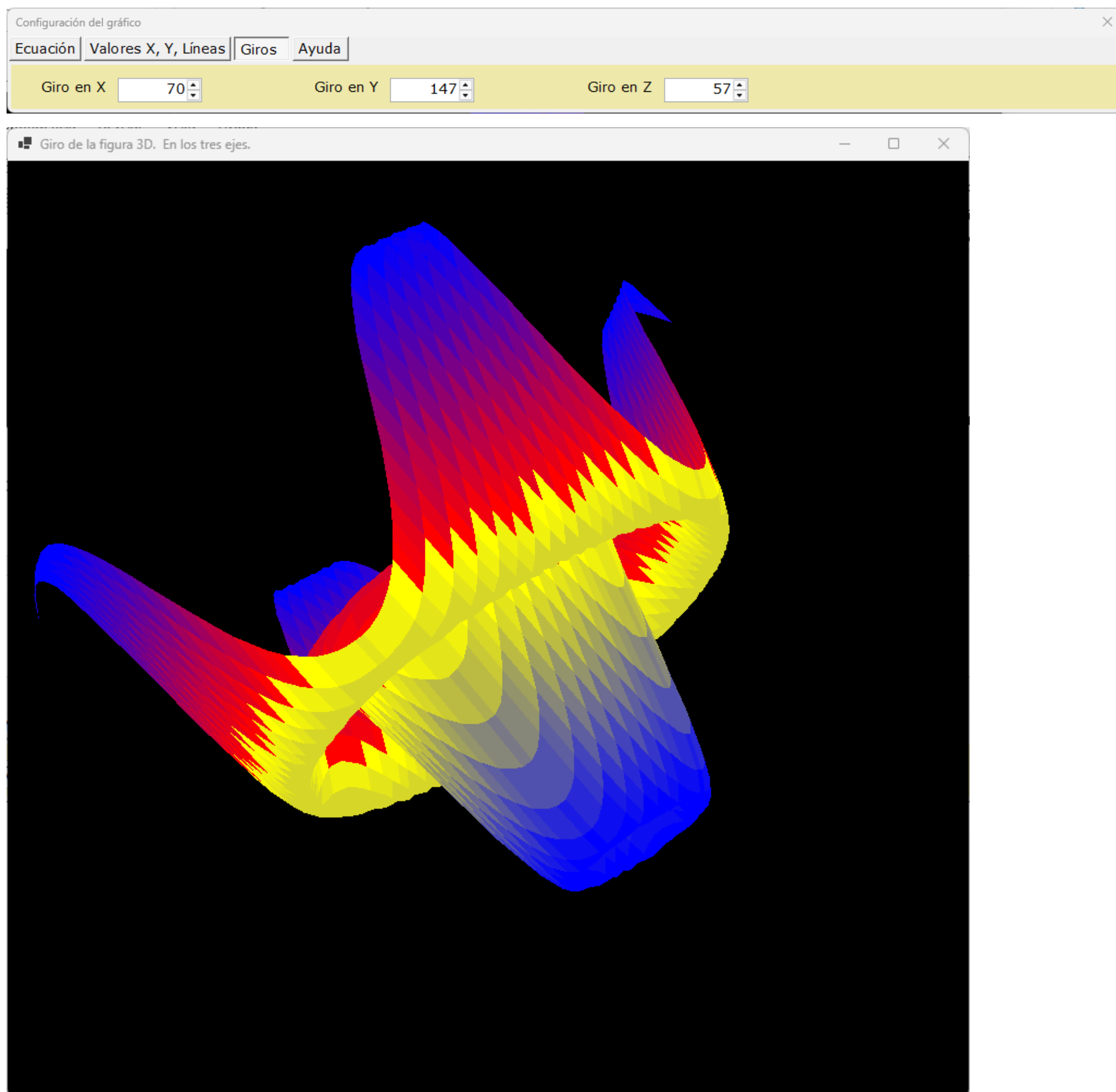


Ilustración 26: Gráfico ecuación 3D con colores según altura



Gráfico Polar en 3D

Los gráficos polares en 3D trabajan con dos ángulos: Theta y Phi. Ver: <https://mathematica.stackexchange.com/questions/83867/how-to-make-a-3d-plot-using-polar-coordinates> o <https://stackoverflow.com/questions/55031161/polar3d-plot-with-theta-phi-and-radius> o <https://mathworld.wolfram.com/SphericalCoordinates.html> o [https://en.wikipedia.org/wiki/Spherical\\_coordinate\\_system](https://en.wikipedia.org/wiki/Spherical_coordinate_system)

Por ejemplo, esta ecuación:

$$r = Cos(\varphi + \theta) - Sen(\theta - \varphi)$$

Los pasos para dibujar el gráfico polar 3D son:

Paso 1: φ (phi) va de 0 a 2π, Θ (theta) va de 0 a 2π y con eso se obtienen los valores de r

Paso 2: Luego se hace la traducción a coordenadas XYZ con estas fórmulas:

$$x = r * Cos(\varphi) * Sen(\theta)$$

$$y = r * Sen(\varphi) * Sen(\theta)$$

$$z = r * Cos(\theta)$$

Paso 3: Se hace el mismo procedimiento con los gráficos 3D de XYZ.

M/013.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public bool Valido; //Si el punto es válido
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            Valido = true;
            if (double.IsFinite(Z)) {
                this.Z = Z;
            }
            else {
                Valido = false;
            }
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {
            //Si el punto no es válido, no hace nada
            if (Valido == false) return;

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
```

```

public int punto1, punto2;

public Conexion(int punto1, int punto2) {
    this.punto1 = punto1;
    this.punto2 = punto2;
}
}
}

```

```

namespace Graficos {
    //Polígono
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        //Color de relleno del polígono
        Color ColorZ;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            double Z4 = puntos[punto4].Zgiro;
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;

            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

            int TotalColores = colorList.Count;
            double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2) /
4;
            int ColorEscoge = (int)Math.Floor(TotalColores * PromedioZ);
            ColorZ = colorList[ColorEscoge];
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo) {
            //Dibuja el polígono relleno
            Brush Relleno2 = new SolidBrush(ColorZ);
            Lienzo.FillPolygon(Relleno2, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Colores para pintar la malla
        List<Color> ListaColores;
    }
}

```

```

//Ecuación
string EcuacionAnterior = "";
Evaluador4 Evaluador = new();

public Objeto3D() {
    int NumColores = 40; // Número de colores a generar

    //Genera listado de colores para el gráfico
    ListaColores = [];
    int Mitad = NumColores / 2;

    // Gradiente de azul a amarillo
    for (int Cont = 0; Cont < Mitad; Cont++) {
        int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación Z = F(X,Y)
//para dibujarla
public void CalcularFigura3D(string Ecuacion, int NumLineas, double angX, double angY, double angZ,
double ZPersona, int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double MinTheta = 0;
    double MinPhi = 0;
    double MaxTheta = 360;
    double MaxPhi = 360;
    double IncrTheta = (MaxTheta - MinTheta) / (NumLineas - 1);
    double IncrPhi = (MaxPhi - MinPhi) / (NumLineas - 1);
    double Theta = MinTheta, Phi = MinPhi;

    //Coordenadas espaciales X,Y,Z
    Theta = MinTheta;
    Phi = MinPhi;
    puntos = [];
    for (int AngTheta = 1; AngTheta <= NumLineas; AngTheta++) {
        for (int AngPhi = 1; AngPhi <= NumLineas; AngPhi++) {
            double ThetaR = Theta * Math.PI / 180;
            double PhiR = Phi * Math.PI / 180;

            Evaluador.DarValorVariable('t', ThetaR);
            Evaluador.DarValorVariable('p', PhiR);
            double R = Evaluador.Evaluar();

            double X = R * Math.Cos(PhiR) * Math.Sin(ThetaR);
            double Y = R * Math.Sin(PhiR) * Math.Sin(ThetaR);
            double Z = R * Math.Cos(ThetaR);

            puntos.Add(new Punto(X, Y, Z));
            Theta += IncrTheta;
        }
        Theta = MinTheta;
        Phi += IncrPhi;
    }
}

```

```

}

//Debe normalizar las coordenadas y ponerlas entre -0.5 y 0.5
double Xmin = double.MaxValue;
double Ymin = double.MaxValue;
double Zmin = double.MaxValue;
double Xmax = double.MinValue;
double Ymax = double.MinValue;
double Zmax = double.MinValue;
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    if (puntos[Cont].X < Xmin) Xmin = puntos[Cont].X;
    if (puntos[Cont].Y < Ymin) Ymin = puntos[Cont].Y;
    if (puntos[Cont].Z < Zmin) Zmin = puntos[Cont].Z;
    if (puntos[Cont].X > Xmax) Xmax = puntos[Cont].X;
    if (puntos[Cont].Y > Ymax) Ymax = puntos[Cont].Y;
    if (puntos[Cont].Z > Zmax) Zmax = puntos[Cont].Z;
}

if (Math.Abs(Xmin - Xmax) < 0.0001 || Math.Abs(Ymin - Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
    MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    // siempre y cuando los vértices sean válidos
    if (puntos[coordenadaActual].Valido &&
        puntos[coordenadaActual + 1].Valido &&
        puntos[coordenadaActual + NumLineas + 1].Valido &&
        puntos[coordenadaActual + NumLineas].Valido) {

        poligonos.Add(new Poligono(
            coordenadaActual,
            coordenadaActual + 1,
            coordenadaActual + NumLineas + 1,
            coordenadaActual + NumLineas
        ));
    }

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions

```

```

double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
    {-SinY, SinX * CosY, CosX * CosY }
};

//Las constantes de transformación para cuadrar en pantalla
double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

//Gira los puntos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;
            FrmGrafico.HuboCambio = true;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numMinimoX_ValueChanged(object sender, EventArgs e) {

        }
    }
}

```

```

private void numMinimoY_ValueChanged(object sender, EventArgs e) {

}

private void numMaximoX_ValueChanged(object sender, EventArgs e) {

}

private void numMaximoY_ValueChanged(object sender, EventArgs e) {

}

private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
    FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
    FrmGrafico.HuboCambio = true;
    FrmGrafico.Refresh();
}

private void btnProcesar_Click(object sender, EventArgs e) {
    FrmGrafico.Ecuacion = txtEcuacion.Text;
    FrmGrafico.HuboCambio = true;
    FrmGrafico.Refresh();
}
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Giro de figura
        public double AnguloX, AnguloY, AnguloZ;

        //Distancia del observador
        public double ZPersona;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        //En caso de que el usuario haya hecho algún cambio a los valores
        //se recalcula todo de nuevo.
        public bool HuboCambio;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {

            //Dibuja la figura 3D
            if (HuboCambio) {
                Figura3D.CalcularFigura3D(Ecuacion, NumLineas, AnguloX, AnguloY, AnguloZ, ZPersona, 0, 0,
this.ClientSize.Width, this.ClientSize.Height);
                HuboCambio = false;
            }

            Figura3D.Dibuja(e.Graphics);
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
            Application.Exit();
        }

        private void Form1_Resize(object sender, EventArgs e) {
            HuboCambio = true;
            Refresh();
        }
    }
}

```



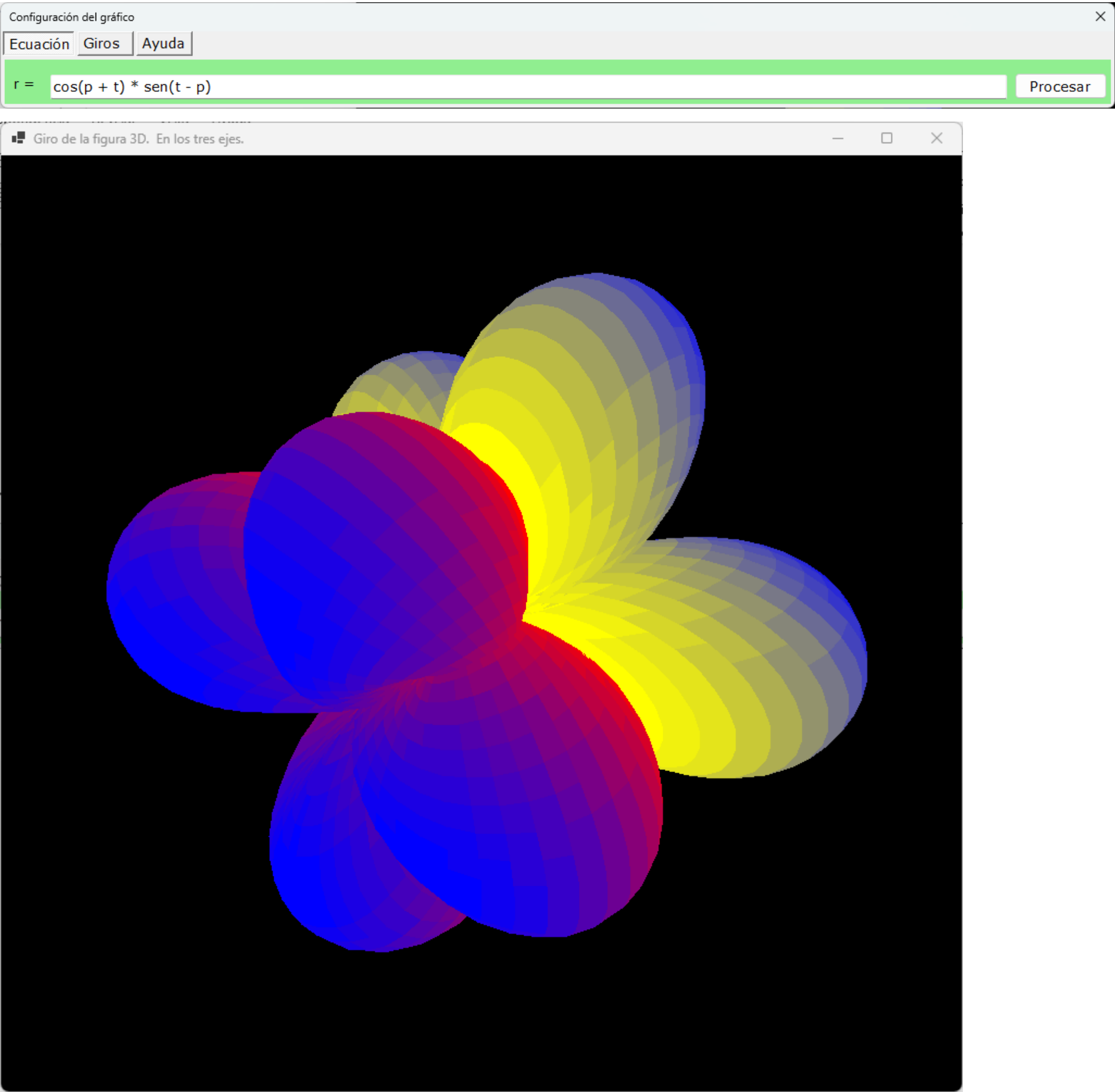


Ilustración 27: Gráfico polar 3D



Gráfico de sólido de revolución

Un sólido de revolución puede nacer de tomar una ecuación del tipo  $Y=F(X)$ , y luego poner a girar el gráfico resultante en el eje X. Ver más en: [https://es.wikipedia.org/wiki/S%C3%B3lido\\_de\\_revoluci%C3%B3n](https://es.wikipedia.org/wiki/S%C3%B3lido_de_revoluci%C3%B3n)

Los pasos para hacer el gráfico son los siguientes:

Paso 1: Saber el valor donde inicia X hasta donde termina. Así se calcula Y. Un típico gráfico  $Y=F(X)$  en 2D.

Paso 2: Ese gráfico en 2D va a girar sobre el eje X.

Paso 3: Se aplica la matriz de giro en el eje X. Cómo se va a hacer uso de polígonos, se calculan cuatro coordenadas (X,Y,Z) para formar cada polígono que conformará la figura 3D.

Paso 4: Se normalizan los valores de (X,Y,Z) para que queden entre 0 y 1, luego se le resta -0.5 ¿Para qué? Para que los puntos (realmente polígonos) queden contenidos dentro de un cubo de lado=1, cuyo centro está en 0,0,0. Ver los dos temas anteriores como se muestra el cubo.

Paso 5: Para cada valor (X,Y,Z) se aplica el giro en los tres ángulos (la matriz para hacer girar en 3D). Se obtiene Xg, Yg, Zg

Paso 6: Se ordenan los polígonos del más profundo (menor valor de Zg) al más superficial (mayor valor de Zg). Por esa razón, la clase Polígono hereda de IComparable

Paso 7: Con Xg, Yg, Zg se proyecta a la pantalla, obteniéndose el planoX, planoY.

Paso 8: Con planoX, planoY se aplican las constantes que se calcularon para proyectar el cubo y se obtiene los datos de pantalla, es decir, pantallaX, pantallaY

Paso 9: Se dibujan los polígonos, primero rellenándolos con el color del fondo y luego se gráfica el perímetro de ese polígono.

M/014.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public bool Valido; //Si el punto es válido
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            Valido = true;
            if (double.IsFinite(Z)) {
                this.Z = Z;
            }
            else {
                Valido = false;
            }
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {
            //Si el punto no es válido, no hace nada
            if (Valido == false) return;

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
```



```

internal class Conexion {
    public int punto1, punto2;

    public Conexion(int punto1, int punto2) {
        this.punto1 = punto1;
        this.punto2 = punto2;
    }
}
}

```

```

namespace Graficos {
    //Polígono
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        //Color de relleno del polígono
        Color ColorZ;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            double Z4 = puntos[punto4].Zgiro;
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;

            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

            int TotalColores = colorList.Count;
            double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2) /
4;
            int ColorEscoge = (int)Math.Floor((TotalColores-1) * PromedioZ);
            ColorZ = colorList[ColorEscoge];
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo) {
            //Dibuja el polígono relleno
            Brush Relleno2 = new SolidBrush(ColorZ);
            Lienzo.FillPolygon(Relleno2, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Colores para pintar la malla
        List<Color> ListaColores;
    }
}

```

```

//Ecuación
string EcuacionAnterior = "";
Evaluador4 Evaluador = new();

public Objeto3D() {
    int NumColores = 40; // Número de colores a generar

    //Genera listado de colores para el gráfico
    ListaColores = [];
    int Mitad = NumColores / 2;

    // Gradiente de azul a amarillo
    for (int Cont = 0; Cont < Mitad; Cont++) {
        int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación Z = F(X,Y)
//para dibujarla
public void CalcularFigura3D(string Ecuacion, double Xini, double Xfin, int NumLineas, double angX,
double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int XpantallaFin, int
YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double IncrX = (Xfin - Xini) / (NumLineas - 1);
    double IncAng = (double) 360 / NumLineas;
    double X = Xini;
    double AnguloGiro = 0;

    //Coordenadas espaciales X,Y,Z
    X = Xini;
    puntos = [];
    AnguloGiro = 0;
    for (int GiroLinea = 1; GiroLinea <= NumLineas+1; GiroLinea++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            double Y = Evaluador.Evaluar();

            double Yg = Y * Math.Cos(AnguloGiro * Math.PI / 180);
            double Zg = Y * Math.Sin(AnguloGiro * Math.PI / 180);

            puntos.Add(new Punto(X, Yg, Zg));
            X += IncrX;
        }
        AnguloGiro += IncAng;
        X = Xini;
    }

    //Debe normalizar las coordenadas y ponerlas entre -0.5 y 0.5
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;

```

```

double Ymax = double.MinValue;
double Zmax = double.MinValue;
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    if (puntos[Cont].X < Xmin) Xmin = puntos[Cont].X;
    if (puntos[Cont].Y < Ymin) Ymin = puntos[Cont].Y;
    if (puntos[Cont].Z < Zmin) Zmin = puntos[Cont].Z;
    if (puntos[Cont].X > Xmax) Xmax = puntos[Cont].X;
    if (puntos[Cont].Y > Ymax) Ymax = puntos[Cont].Y;
    if (puntos[Cont].Z > Zmax) Zmax = puntos[Cont].Z;
}

if (Math.Abs(Xmin - Xmax) < 0.0001 || Math.Abs(Ymin - Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
    MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    if (puntos[Cont].Valido == false) continue;
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1) + (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    // siempre y cuando los vértices sean válidos
    if (puntos[coordenadaActual].Valido &&
        puntos[coordenadaActual + 1].Valido &&
        puntos[coordenadaActual + NumLineas + 1].Valido &&
        puntos[coordenadaActual + NumLineas].Valido) {

        poligonos.Add(new Poligono(
            coordenadaActual,
            coordenadaActual + 1,
            coordenadaActual + NumLineas + 1,
            coordenadaActual + NumLineas
        ));
    }

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
    {-SinY, SinX * CosY, CosX * CosY }
};

//Las constantes de transformación para cuadrar en pantalla

```

```

double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

//Gira los puntos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;
            FrmGrafico.HuboCambio = true;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numMinimoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMinimoX.Value = numMaximoX.Value - 1;

            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }
    }
}

```

```

private void numMaximoX_ValueChanged(object sender, EventArgs e) {
    if (numMinimoX.Value >= numMaximoX.Value)
        numMaximoX.Value = numMinimoX.Value + 1;

    FrmGrafico.Xfin = Convert.ToDouble((double) numMaximoX.Value);
    FrmGrafico.HuboCambio = true;
    FrmGrafico.Refresh();
}

private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
    FrmGrafico.NumLineas = Convert.ToInt32((double) numTotalLineas.Value);
    FrmGrafico.HuboCambio = true;
    FrmGrafico.Refresh();
}

private void btnProcesar_Click(object sender, EventArgs e) {
    FrmGrafico.Ecuacion = txtEcuacion.Text;
    FrmGrafico.HuboCambio = true;
    FrmGrafico.Refresh();
}
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Giro de figura
        public double AnguloX, AnguloY, AnguloZ;

        //Distancia del observador
        public double ZPersona;

        //Rango de valores
        public double Xini, Yini, Xfin, Yfin;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        //En caso de que el usuario haya hecho algún cambio a los valores
        //se recalcula todo de nuevo.
        public bool HuboCambio;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {

            //Dibuja la figura 3D
            if (HuboCambio) {
                Figura3D.CalcularFigura3D(Ecuacion, Xini, Xfin, NumLineas, AnguloX, AnguloY, AnguloZ, ZPersona, 0,
0, this.ClientSize.Width, this.ClientSize.Height);
                HuboCambio = false;
            }

            Figura3D.Dibuja(e.Graphics);
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
            Application.Exit();
        }

        private void Form1_Resize(object sender, EventArgs e) {
            HuboCambio = true;
            Refresh();
        }
    }
}

```

```
}  
}
```

Configuración del gráfico

Ecuación   Valores X, Líneas   Giros   Ayuda

Y =    Procesar

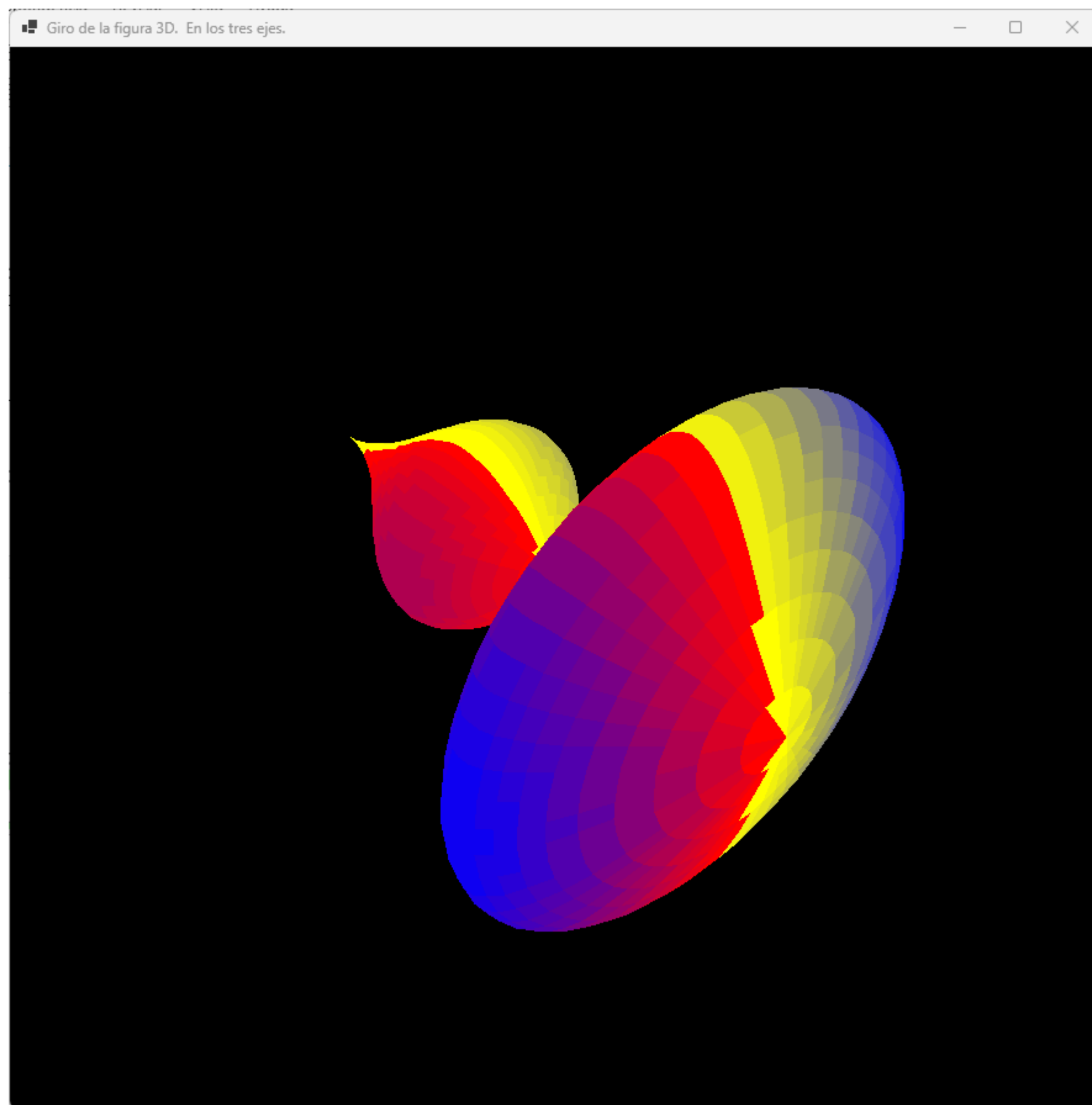
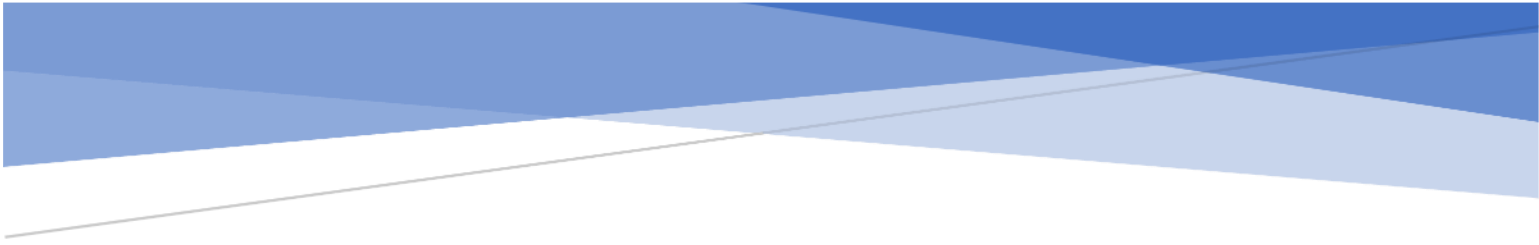


Ilustración 28: Gráfico de sólido de revolución





# C# Y .NET 10

## Parte 14. Animación

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                               |    |
|-----------------------------------------------|----|
| Tabla de ilustraciones.....                   | 3  |
| Acerca del autor.....                         | 4  |
| Licencia de este libro .....                  | 4  |
| Licencia del software .....                   | 4  |
| Marcas registradas .....                      | 5  |
| En memoria de .....                           | 6  |
| Haciendo uso del Paint().....                 | 7  |
| Rebotando en pantalla .....                   | 11 |
| Lógica y representación.....                  | 12 |
| Cazador y Presa .....                         | 14 |
| El juego de la vida.....                      | 18 |
| Población e infección.....                    | 21 |
| Gráfico matemático en 2D y animado .....      | 24 |
| Gráfico polar animado.....                    | 28 |
| Gráfico Matemático en 3D. Giros animados..... | 32 |
| Gráfico Matemático en 4D.....                 | 40 |
| Gráfico Polar en 4D .....                     | 49 |
| Sólido de revolución animado.....             | 57 |



## Tabla de ilustraciones

|                                                                                       |                                      |
|---------------------------------------------------------------------------------------|--------------------------------------|
| Ilustración 1: Control Timer .....                                                    | 7                                    |
| Ilustración 2: Propiedades del control Timer. Enabled debe estar en <b>True</b> ..... | 7                                    |
| Ilustración 3: Propiedad DoubleBuffered del formulario en True .....                  | 8                                    |
| Ilustración 4: Habilitar el evento Tick del Timer.....                                | 8                                    |
| Ilustración 5: Evento para escribir el código.....                                    | 9                                    |
| Ilustración 6: Cuadro desplazándose .....                                             | 10                                   |
| Ilustración 7: Cuadro desplazándose .....                                             | 10                                   |
| Ilustración 8: Rebote .....                                                           | 11                                   |
| Ilustración 9: Rebote en arreglo bidimensional.....                                   | 13                                   |
| Ilustración 10: Cambiando el tamaño de la ventana .....                               | 13                                   |
| Ilustración 11: Depredador y presa .....                                              | 17                                   |
| Ilustración 12: Juego de la vida .....                                                | 20                                   |
| Ilustración 13: Infección propagándose.....                                           | 23                                   |
| Ilustración 14: Gráfico matemático 2D animado .....                                   | 27                                   |
| Ilustración 15: Gráfico matemático 2D animado .....                                   | <b>iError! Marcador no definido.</b> |
| Ilustración 16: Gráfico polar animado .....                                           | 31                                   |
| Ilustración 17: Gráfico polar animado .....                                           | 31                                   |
| Ilustración 18: Gráfico Matemático en 3D. Giros animados .....                        | 38                                   |
| Ilustración 19: Gráfico Matemático en 3D. Giros animados .....                        | 39                                   |
| Ilustración 20: Gráfico Matemático en 4D.....                                         | 47                                   |
| Ilustración 21: Gráfico Matemático en 4D.....                                         | 48                                   |
| Ilustración 22: Gráfico polar 4D.....                                                 | 55                                   |
| Ilustración 23: Gráfico polar 4D.....                                                 | 56                                   |
| Ilustración 24: Sólido de revolución animado .....                                    | 63                                   |
| Ilustración 25: Sólido de revolución animado .....                                    | 64                                   |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

En memoria de

Snow



# Haciendo uso del Paint()

C# tiene un sistema fácil para hacer animaciones. Sin embargo, no es para hacer animaciones muy complejas. Es necesario agregar el control “Timer” al formulario:

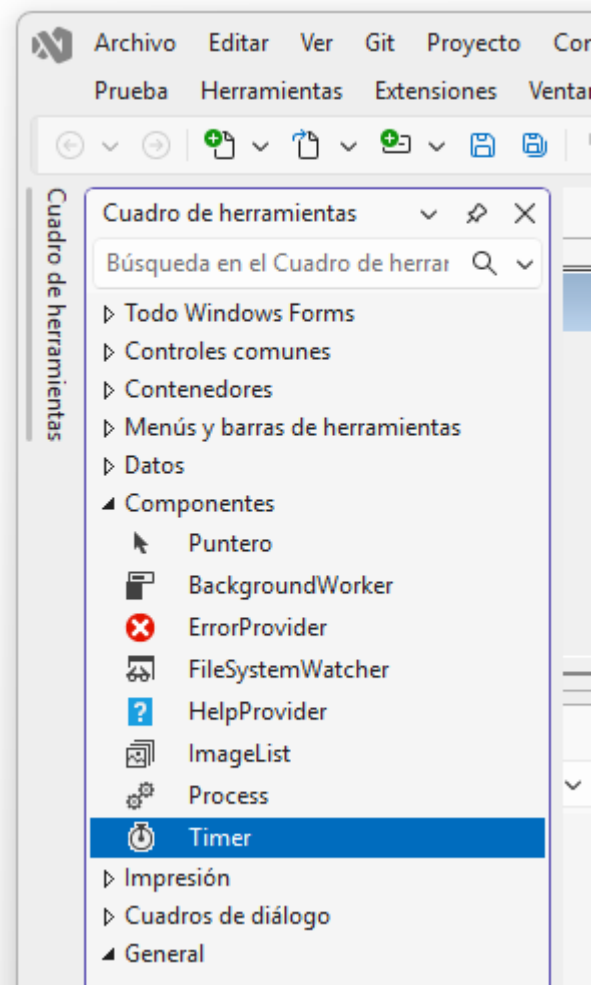


Ilustración 1: Control Timer

Se ajustan las propiedades de este control: (Name) y Enabled que debe estar en **True**

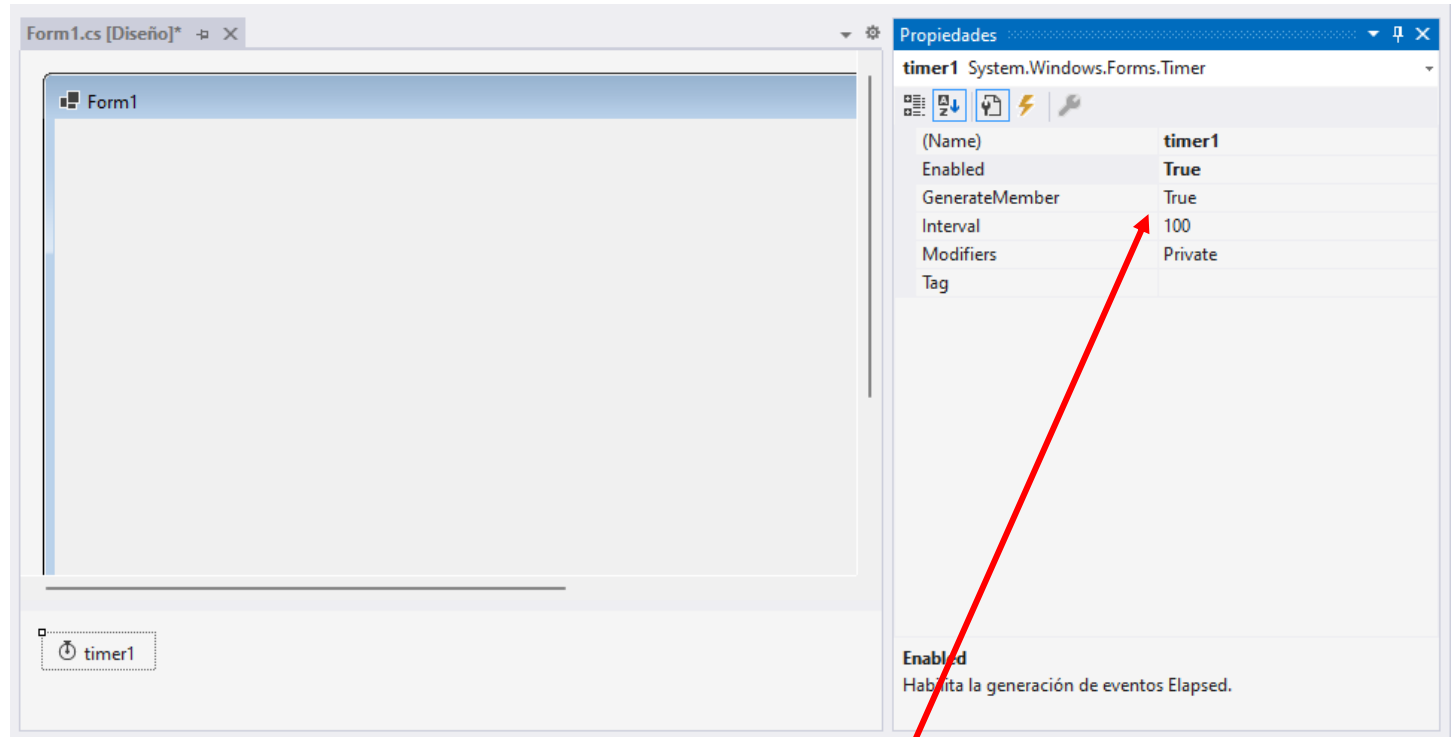


Ilustración 2: Propiedades del control Timer. Enabled debe estar en **True**

¡OJO!: Hay que poner en **True** la propiedad DoubleBuffered del formulario

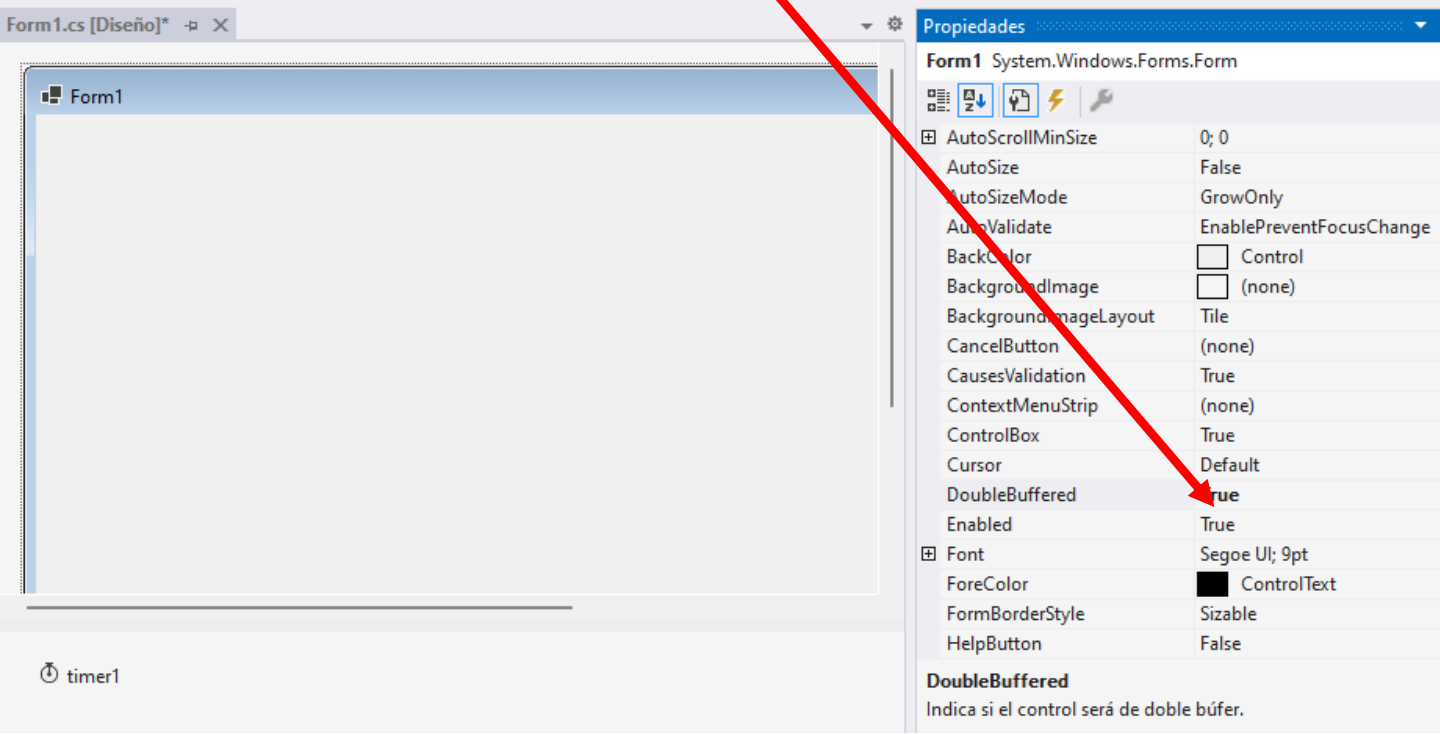


Ilustración 3: Propiedad DoubleBuffered del formulario en True

Ahora se habilita el evento Tick del Timer

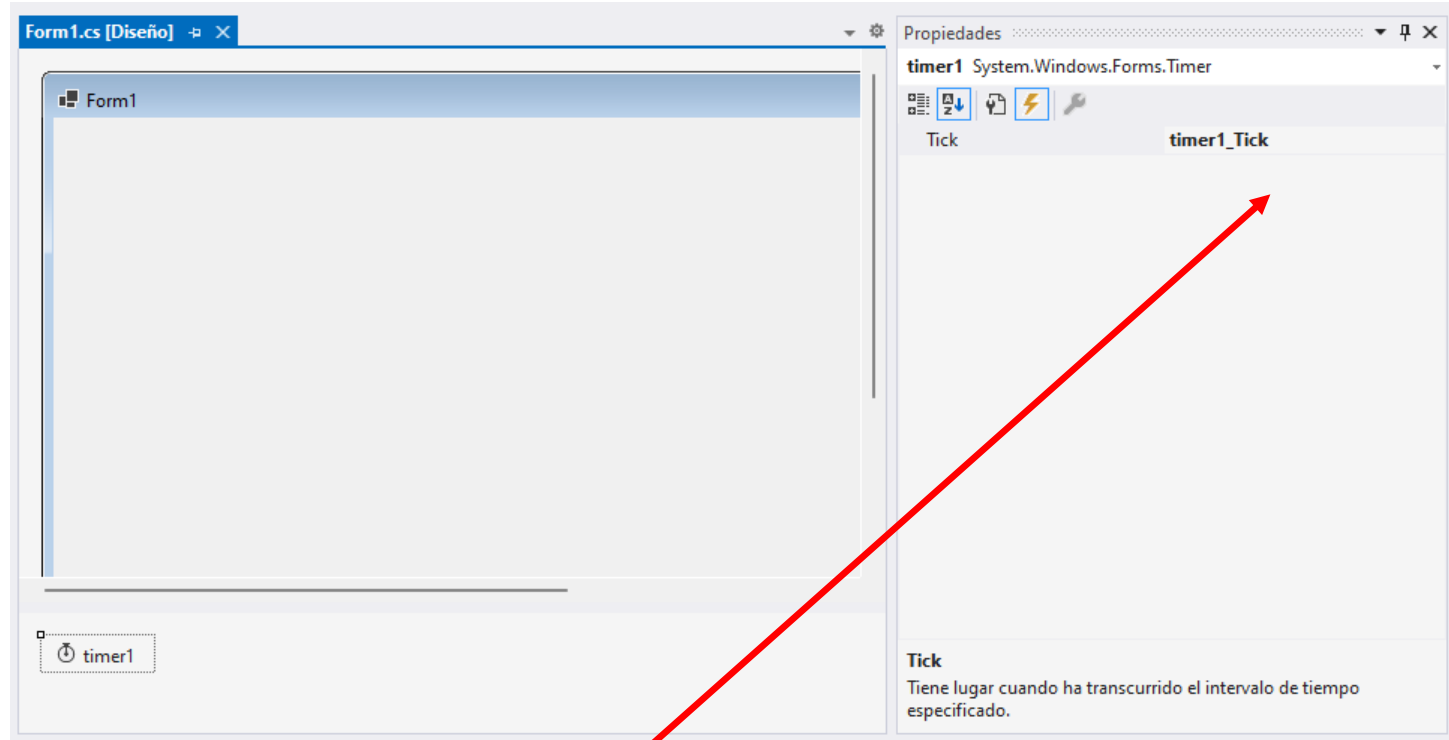


Ilustración 4: Habilitar el evento Tick del Timer

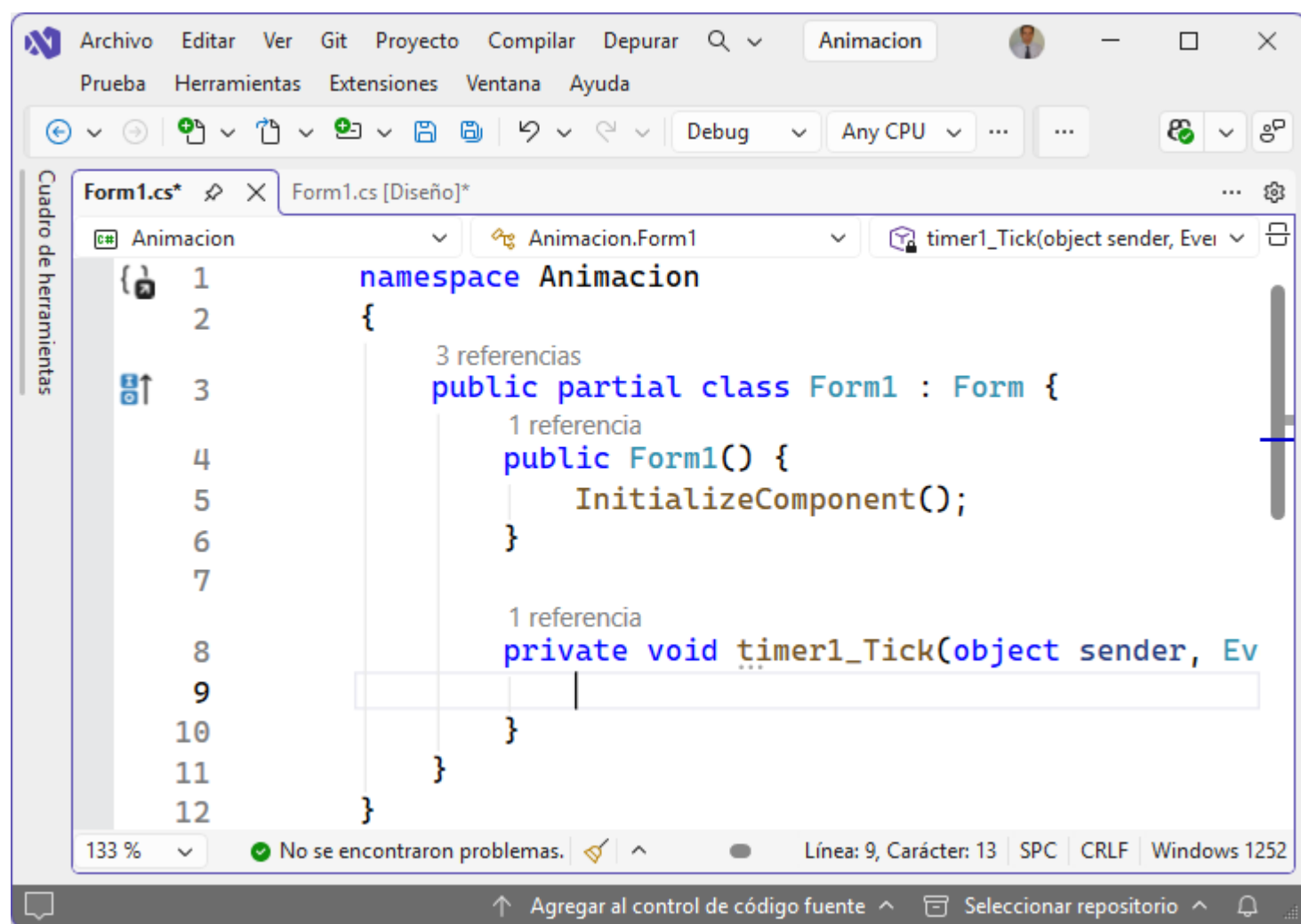


Ilustración 5: Evento para escribir el código

```

namespace Animacion {
    public partial class Form1 : Form {
        int PosX, PosY; //Coordenadas del cuadrado relleno
        public Form1() {
            InitializeComponent();

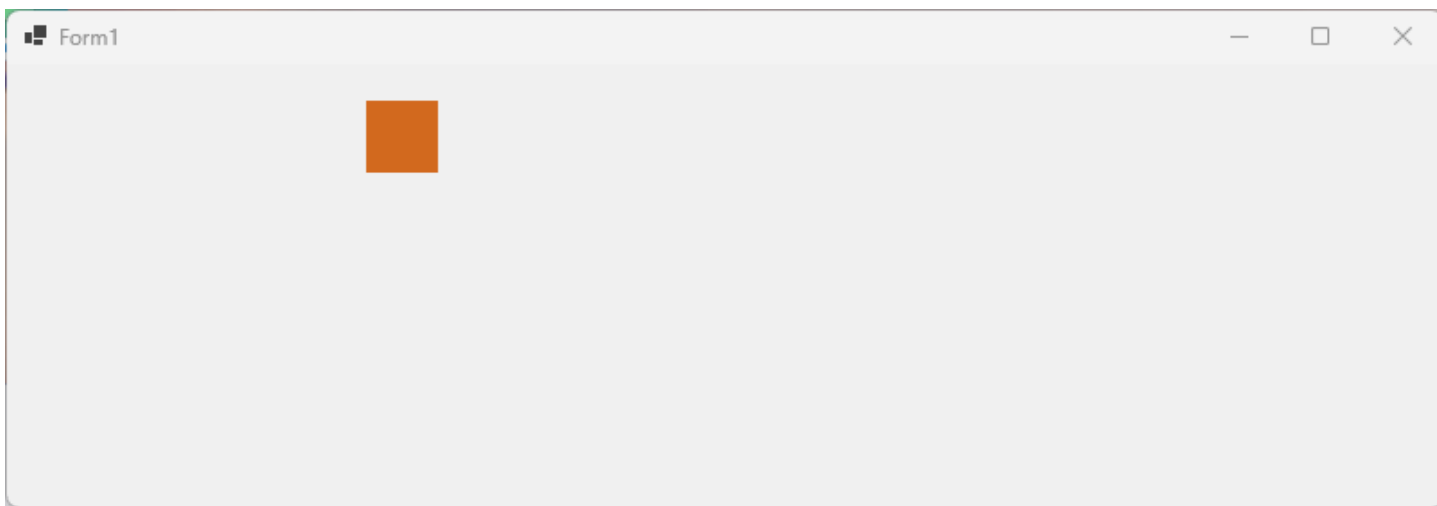
            //Inicializa las posiciones
            PosX = 10;
            PosY = 20;
        }

        private void timer1_Tick(object sender, EventArgs e) {
            //Por cada tick, incrementa en 10 el valor de
            //la posición X del cuadrado relleno
            PosX += 10;
            Refresh(); //Refresca el formulario y llama a Paint()
        }

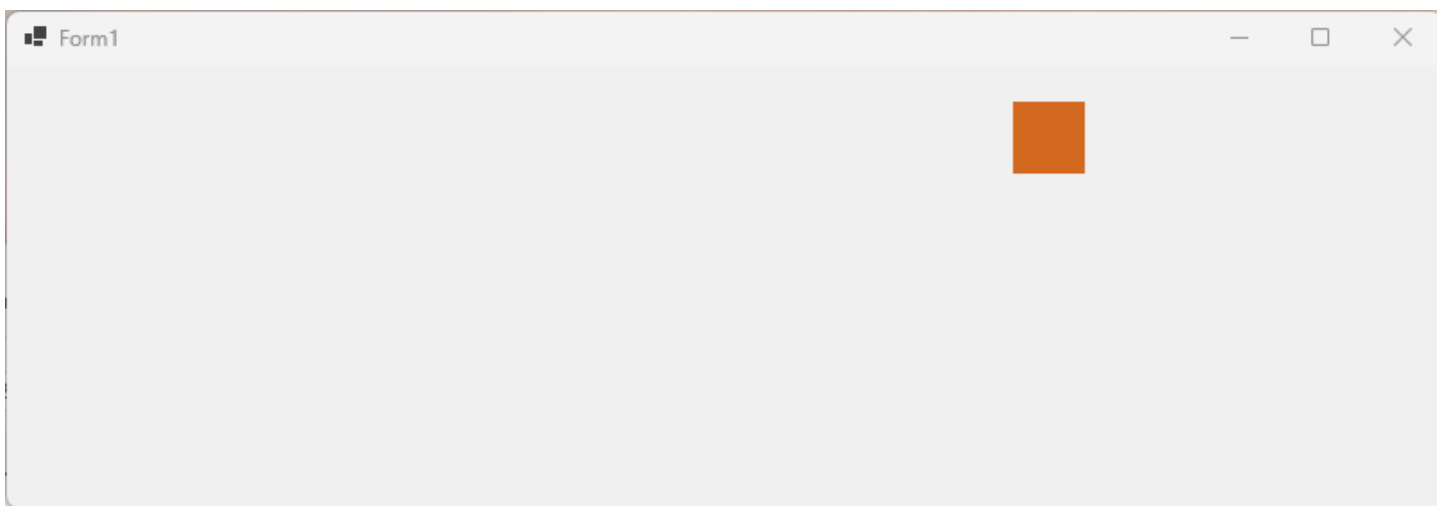
        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics Lienzo = e.Graphics;
            SolidBrush Relleno = new(Color.Chocolate);

            //=====
            //Rectángulo: Xpos, Ypos, ancho, alto
            //=====
            Lienzo.FillRectangle(Relleno, PosX, PosY, 40, 40);
        }
    }
}

```



*Ilustración 6: Cuadro desplazándose*



*Ilustración 7: Cuadro desplazándose*

¡OJO! La animación no será suave, es una limitante que tiene Winforms. En algunos foros recomiendan cambiar la instrucción `Refresh()` por `Invalidate()`; . Ambas instrucciones redibujan, pero operan de distinta forma:

`Invalidate()` : Envía el mensaje al sistema operativo para que redibuje, pero no lo hace inmediatamente.

`Refresh()` : Fuerza a redibujar inmediatamente.



# Rebotando en pantalla

El siguiente ejemplo, muestra un cuadrado relleno rebotando en las paredes de la ventana.

N/002.zip

```
namespace Animacion {
    public partial class Form1 : Form {
        int PosX, PosY; //Coordenadas del cuadrado relleno
        int Tamano; //Tamaño del lado del cuadrado
        int IncrementoX, IncrementoY; //Desplazamiento del cuadrado relleno

        public Form1() {
            InitializeComponent();

            //Inicializa la posición y tamaño del cuadrado relleno
            PosX = 10;
            PosY = 20;
            Tamano = 40;

            //Velocidad con que se desplaza el cuadrado relleno
            IncrementoX = 5;
            IncrementoY = 5;
        }

        private void timer1_Tick(object sender, System.EventArgs e) {
            Logica(); //Lógica de la animación
            Refresh(); //Visual de la animación
        }

        void Logica() {
            //Si colisiona con alguna pared cambia el incremento
            if (PosX + Tamano > this.ClientSize.Width || PosX < 0)
                IncrementoX *= -1;

            if (PosY + Tamano > this.ClientSize.Height || PosY < 0)
                IncrementoY *= -1;

            //Cambia la posición de X y Y
            PosX += IncrementoX;
            PosY += IncrementoY;
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics Lienzo = e.Graphics;

            //Fondo de la ventana
            Rectangle rect = new Rectangle(0, 0, this.Width, this.Height);
            Lienzo.FillRectangle(Brushes.Black, rect);

            //Gráfico a animar
            Lienzo.FillRectangle(Brushes.Red, PosX, PosY, Tamano, Tamano);
        }
    }
}
```

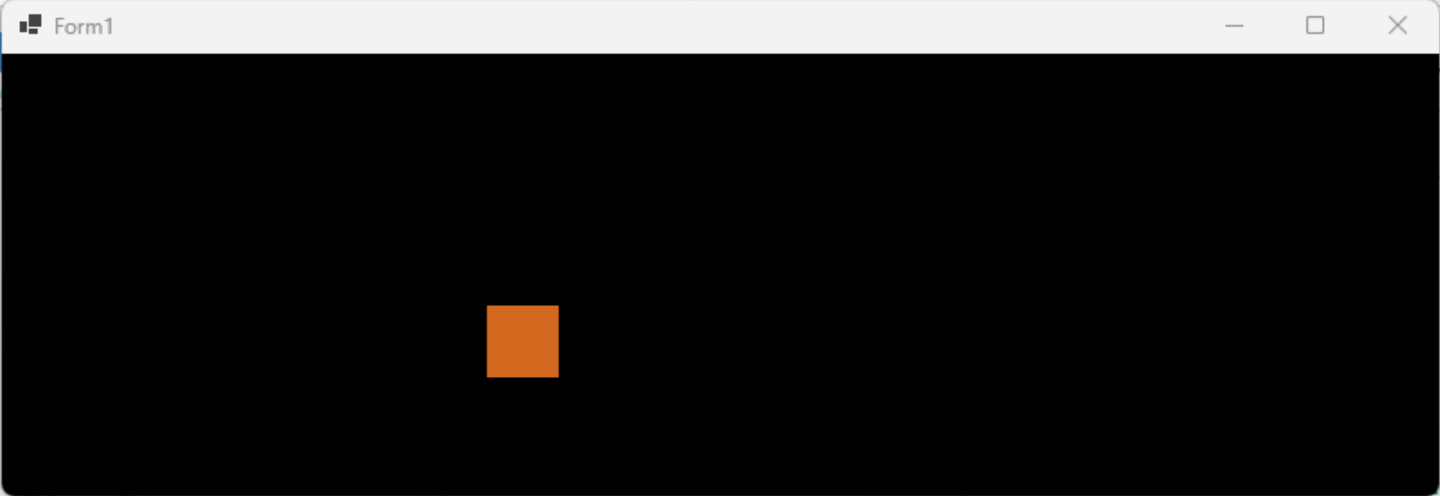


Ilustración 8: Rebote

# Lógica y representación

Es muy recomendable que la parte lógica de la aplicación gráfica se concentre sobre un arreglo bidimensional y la parte visual es reflejar lo que sucede en ese arreglo bidimensional en pantalla.

Se reescribe el programa de rebote anterior, haciendo uso de un arreglo bidimensional.

N/003.zip

```
namespace Animacion {
    public partial class Form1 : Form {
        int[,] Plano; //Dónde ocurre realmente la acción
        int PosX, PosY; //Coordenadas del cuadrado relleno
        int IncrX, IncrY; //Desplazamiento del cuadrado relleno

        public Form1() {
            InitializeComponent();

            //Inicializa el tablero
            Plano = new int[30, 30];

            //Inicializa la posición del cuadrado relleno
            Random azar = new();
            PosX = azar.Next(0, Plano.GetLength(0));
            PosY = azar.Next(0, Plano.GetLength(1));

            //Desplaza el cuadrado relleno
            IncrX = 1;
            IncrY = 1;
        }

        private void timer1_Tick(object sender, System.EventArgs e) {
            Logica(); //Lógica de la animación
            Refresh(); //Visual de la animación
        }

        void Logica() {
            //Borra la posición anterior
            Plano[PosX, PosY] = 0;

            //Si colisiona con alguna pared cambia el incremento
            if (PosX + IncrX >= Plano.GetLength(0) || PosX + IncrX < 0)
                IncrX *= -1;

            if (PosY + IncrY >= Plano.GetLength(1) || PosY + IncrY < 0)
                IncrY *= -1;

            //Cambia la posición de X y Y
            PosX += IncrX;
            PosY += IncrY;

            //Nueva posición
            Plano[PosX, PosY] = 1;
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Graphics lienzo = e.Graphics;
            Pen Lapis = new(Color.Blue, 1);
            Brush Llena = new SolidBrush(Color.Red);

            //Tamaño de cada celda
            int tX = ClientSize.Width / Plano.GetLength(0);
            int tY = ClientSize.Height / Plano.GetLength(1);

            //Fondo de la ventana
            Rectangle rect = new(0, 0, this.Width, this.Height);
            lienzo.FillRectangle(Brushes.Black, rect);

            //Dibuja la malla y la posición del rectángulo relleno
            for (int Fil = 0; Fil < Plano.GetLength(0); Fil++) {
                for (int Col = 0; Col < Plano.GetLength(1); Col++)
                    if (Plano[Fil, Col] == 0)
                        lienzo.DrawRectangle(Lapis, Fil * tX, Col * tY, tX, tY);
                    else
                        lienzo.FillRectangle(Llena, Fil * tX, Col * tY, tX, tY);
            }
        }
    }
}
```

```
}  
}  
}
```

Así ejecuta

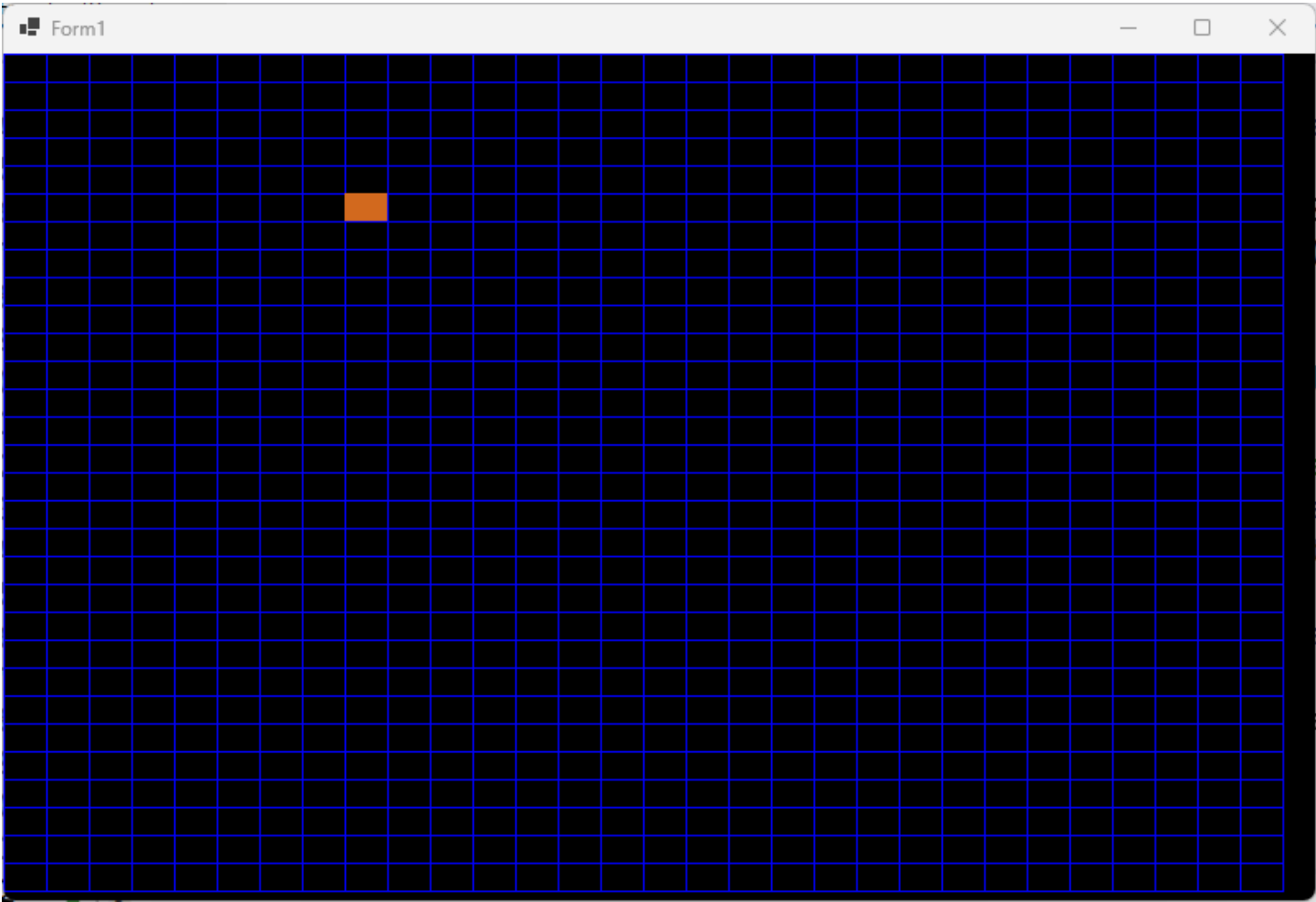


Ilustración 9: Rebote en arreglo bidimensional

Es mucho mejor así, porque hay pleno control del tamaño del tablero, cómo se desplaza, dónde rebota. En el código de ejemplo, si se cambia el tamaño de la ventana, el programa reacciona al nuevo tamaño.

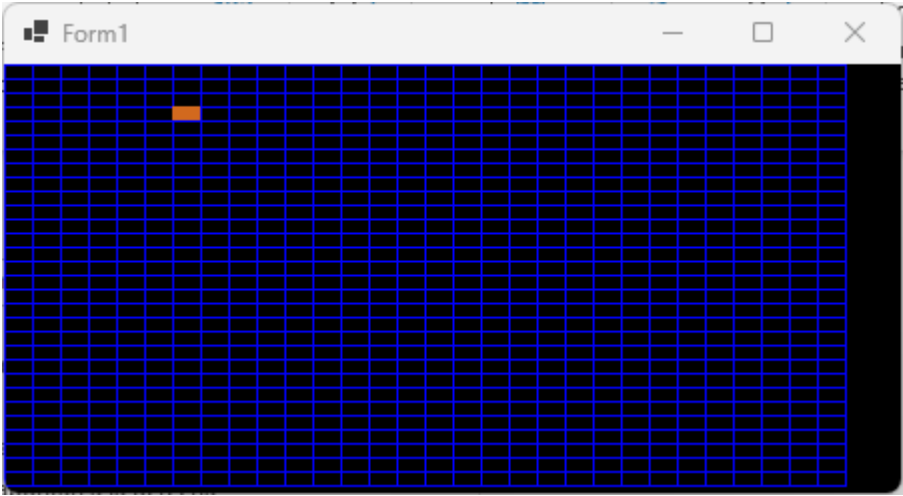


Ilustración 10: Cambiando el tamaño de la ventana

# Cazador y Presa

Dado un arreglo bidimensional de 30\*30, se ubica en una casilla (seleccionada al azar) un cazador y en otra casilla (seleccionada al azar) una presa. El objetivo es hacer que el cazador salte de casilla en casilla, este salto puede ser horizontal o vertical o diagonal hasta alcanzar la casilla de la presa. El tamaño del salto del depredador es de una casilla cada vez. La presa se mantiene inmóvil.

Se deben agregar obstáculos al azar a este tablero bidimensional (casillas en las cuales el cazador **NO** puede saltar). Y allí está el problema, porque el cazador se puede topar con una pared que le impide llegar a la presa. ¿Y qué se hace en ese caso? Una solución planteada es generar al azar alguna coordenada temporal e instruir al cazador para que olvide la presa y se dirija a esa ubicación temporal, una vez el cazador llega a esa ubicación temporal, de nuevo se le instruye para que se dirija hacia la presa con la esperanza que ahora si encuentre un camino libre hasta la presa.

N/004.zip

```
namespace Animacion {
    public partial class Form1 : Form {
        //Constantes para calificar cada celda del Plano
        private const int CAMINO = 0;
        private const int OBSTACULO = 1;
        private const int CAZADOR = 2;
        private const int PRESA = 3;

        //Dónde ocurre realmente la acción
        int[,] Plano;

        //Coordenadas del cazador
        int CazaX, CazaY;

        //Desplazamiento del cazador
        int CazaMX, CazaMY;

        //Coordenadas de la presa
        int PresaX, PresaY;

        //Coordenadas temporales para desatorar
        int tmpX, tmpY;

        //Alternar entre buscar la presa real o
        //ir a la coordenada temporal
        bool BuscaTmp;

        //Único generador de números aleatorios.
        Random Azar;

        public Form1() {
            InitializeComponent();
            Azar = new Random();
            IniciarParametros();
        }

        public void IniciarParametros() {
            //Inicializa el Plano.
            Plano = new int[30, 30];

            //Total de paredes dentro del plano
            int Obstaculos = 150;
            for (int cont = 1; cont <= Obstaculos; cont++) {
                int obstaculoX, obstaculoY;
                do {
                    obstaculoX = Azar.Next(0, Plano.GetLength(0));
                    obstaculoY = Azar.Next(0, Plano.GetLength(1));
                } while (Plano[obstaculoX, obstaculoY] != 0);
                Plano[obstaculoX, obstaculoY] = OBSTACULO;
            }

            //Inicializa la posición del cazador
            do {
                CazaX = Azar.Next(0, Plano.GetLength(0));
                CazaY = Azar.Next(0, Plano.GetLength(1));
            } while (Plano[CazaX, CazaY] != 0);
            Plano[CazaX, CazaY] = CAZADOR;

            //Inicializa la posición de la presa
```

```

do {
    PresaX = Azar.Next(0, Plano.GetLength(0));
    PresaY = Azar.Next(0, Plano.GetLength(1));
} while (Plano[PresaX, PresaY] != 0);
Plano[PresaX, PresaY] = PRESA;

//Desplazamiento del cazador
CazaMX = 1;
CazaMY = 1;

timer1.Start();
}

private void timer1_Tick(object sender, System.EventArgs e) {
    if (Logica())
        Refresh();
    else {
        timer1.Stop();
        MessageBox.Show("El cazador alcanzó a la presa");
        IniciarParametros();
    }
}

//Retorna true si la animación continúa, false si el depredador
//alcanza a la presa y detiene la animación.
public bool Logica() {
    Plano[CazaX, CazaY] = CAMINO;

    if (BuscaTmp == false) {
        //Esta buscando la presa
        if (CazaX > PresaX) CazaMX = -1;
        else if (CazaX < PresaX) CazaMX = 1;
        else CazaMX = 0;

        if (CazaY > PresaY) CazaMY = -1;
        else if (CazaY < PresaY) CazaMY = 1;
        else CazaMY = 0;

        //Verifica si ya llegó a la presa
        if (CazaX == PresaX && CazaY == PresaY) {
            return false;
        }
        //Si no, verifica si puede desplazarse a la nueva ubicación
        else if (Plano[CazaX + CazaMX, CazaY + CazaMY] == CAMINO ||
            Plano[CazaX + CazaMX, CazaY + CazaMY] == PRESA) {
            CazaX += CazaMX;
            CazaY += CazaMY;
        }
        //Si no, entonces está atorado con los obstáculos.
        //Luego genera ubicación temporal para ir allí
        else {
            do {
                tmpX = Azar.Next(0, Plano.GetLength(0));
                tmpY = Azar.Next(0, Plano.GetLength(1));
            } while (Plano[tmpX, tmpY] != CAMINO);
            BuscaTmp = true;
        }
    }
    else { //Está yendo a la ubicación temporal
        if (CazaX > tmpX) CazaMX = -1;
        else if (CazaX < tmpX) CazaMX = 1;
        else CazaMX = 0;

        if (CazaY > tmpY) CazaMY = -1;
        else if (CazaY < tmpY) CazaMY = 1;
        else CazaMY = 0;

        //Si ha llegado a la ubicación temporal o se queda atorado
        //deja de ir a esa ubicación temporal
        if (CazaX == tmpX && CazaY == tmpY ||
            Plano[CazaX + CazaMX, CazaY + CazaMY] == OBSTACULO)
            BuscaTmp = false;
        else {
            CazaX += CazaMX;
            CazaY += CazaMY;
        }
    }
}

```

```

    }

Plano[CazaX, CazaY] = CAZADOR;
return true;
}

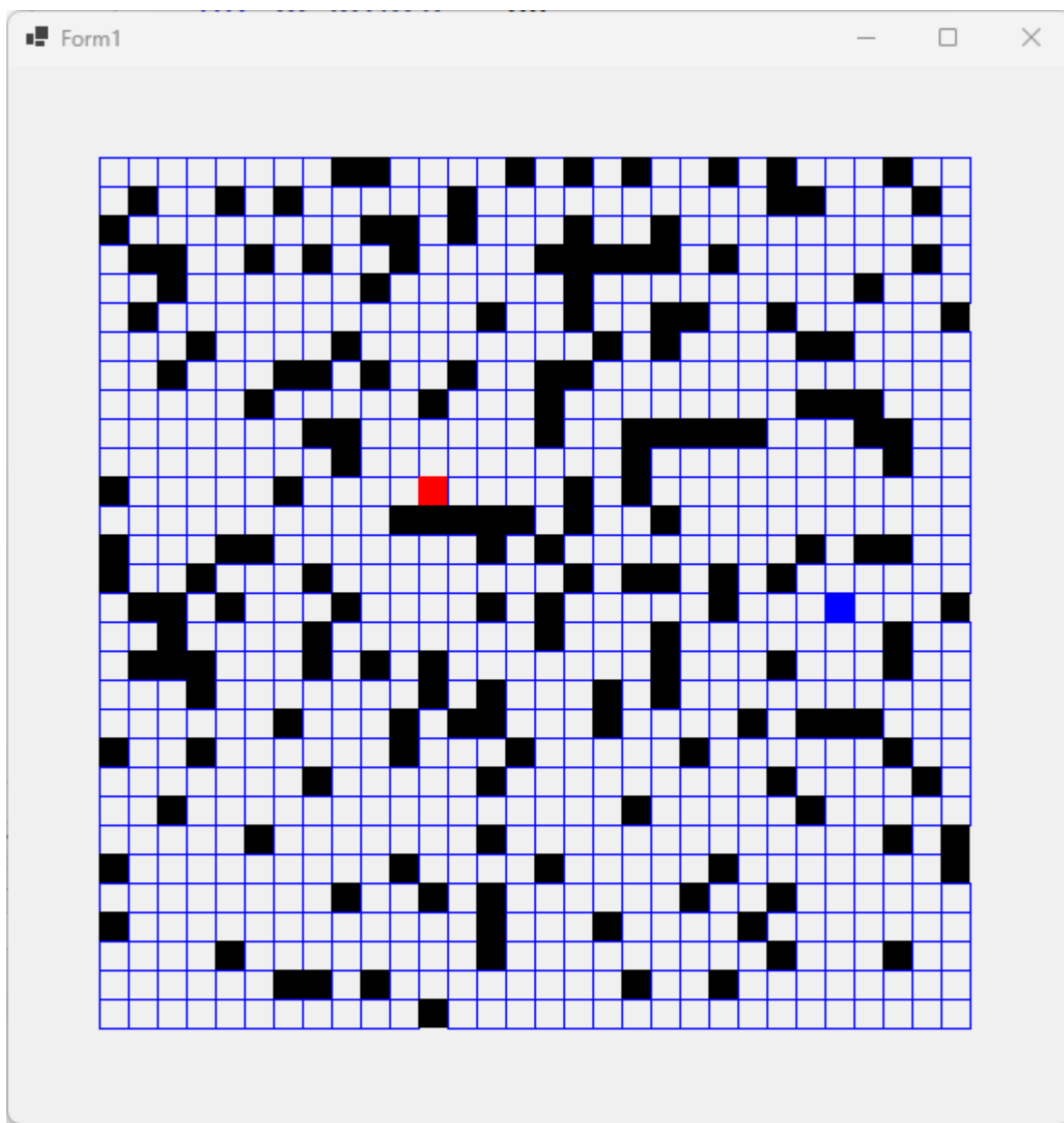
private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics lienzo = e.Graphics;
    Pen Lapisz = new(Color.Blue, 1);
    Brush Llena1 = new SolidBrush(Color.Black);
    Brush Llena2 = new SolidBrush(Color.Red);
    Brush Llena3 = new SolidBrush(Color.Blue);

    //Tamaño de cada celda
    int tX = 500 / Plano.GetLength(0);
    int tY = 500 / Plano.GetLength(1);
    int desplaza = 50;

    //Dibuja el arreglo bidimensional
    for (int Fil = 0; Fil < Plano.GetLength(0); Fil++) {
        for (int Col = 0; Col < Plano.GetLength(1); Col++) {
            int uX = Fil * tX + desplaza;
            int uY = Col * tY + desplaza;
            switch (Plano[Fil, Col]) {
                case CAMINO:
                    lienzo.DrawRectangle(Lapisz, uX, uY, tX, tY);
                    break;
                case OBSTACULO:
                    lienzo.FillRectangle(Llena1, uX, uY, tX, tY);
                    break;
                case CAZADOR:
                    lienzo.FillRectangle(Llena2, uX, uY, tX, tY);
                    break;
                case PRESA:
                    lienzo.FillRectangle(Llena3, uX, uY, tX, tY);
                    break;
            }
        }
    }
}

```





*Ilustración 11: Depredador y presa*

¡OJO! El algoritmo de búsqueda de la presa es bastante ineficiente en su cometido: El cazador puede tardar un tiempo considerable en encontrar la presa, además este algoritmo no valida que exista un camino viable por lo que se puede quedar operando en forma infinita buscando como llegar a la presa.

## El juego de la vida

Este juego [https://es.wikipedia.org/wiki/Juego\\_de\\_la\\_vida](https://es.wikipedia.org/wiki/Juego_de_la_vida) inventado por John Horton Conway, ha llamado mucho la atención por como dada una serie de unas reglas muy sencillas, genera patrones complejos.

N/005.zip

```
namespace Animacion {
    public partial class Form1 : Form {
        //Constantes para calificar cada celda del tablero
        private const int INACTIVA = 0;
        private const int ACTIVA = 1;

        int[,] Plano; //Dónde ocurre realmente la acción
        Random Azar; //Único generador de números aleatorios.

        public Form1() {
            InitializeComponent();
            Azar = new Random();
            IniciarParametros();
        }

        private void mnuReiniciar_Click(object sender, EventArgs e) {
            IniciarParametros();
        }

        public void IniciarParametros() {
            //Inicializa el tablero.
            Plano = new int[40, 40];

            //Habrán celdas activas en el 20%
            int Activos = 200;
            for (int cont = 1; cont <= Activos; cont++) {
                int posX, posY;
                do {
                    posX = Azar.Next(0, Plano.GetLength(0));
                    posY = Azar.Next(0, Plano.GetLength(1));
                } while (Plano[posX, posY] != 0);
                Plano[posX, posY] = ACTIVA;
            }

            timer1.Start();
        }

        private void timer1_Tick(object sender, System.EventArgs e) {
            Logica(); //Lógica de la animación
            Refresh(); //Visual de la animación
        }

        public void Logica() {
            //Copia el tablero a uno temporal
            int[,] PlanoTmp = Plano.Clone() as int[,];

            //Va de celda en celda
            for (int posX = 0; posX < Plano.GetLength(0); posX++) {
                for (int posY = 0; posY < Plano.GetLength(1); posY++) {

                    //Determina el número de vecinos activos
                    int BordeXizq;
                    if (posX - 1 < 0)
                        BordeXizq = Plano.GetLength(0) - 1;
                    else
                        BordeXizq = posX - 1;

                    int BordeYarr;
                    if (posY - 1 < 0)
                        BordeYarr = Plano.GetLength(1) - 1;
                    else
                        BordeYarr = posY - 1;

                    int BordeXder;
                    if (posX + 1 >= Plano.GetLength(0))
                        BordeXder = 0;
                }
            }
        }
    }
}
```



```

else
    BordeXder = posX + 1;

int BordeYaba;
if (posY + 1 >= Plano.GetLength(1))
    BordeYaba = 0;
else
    BordeYaba = posY + 1;

int Activos = Plano[BordeXizq, BordeYarr];
Activos += Plano[posX, BordeYarr];
Activos += Plano[BordeXder, BordeYarr];

Activos += Plano[BordeXizq, posY];
Activos += Plano[BordeXder, posY];

Activos += Plano[BordeXizq, BordeYaba];
Activos += Plano[posX, BordeYaba];
Activos += Plano[BordeXder, BordeYaba];

//Los cambios se registran en el tablero temporal

//Si la celda está inactiva y tiene 3 celdas activas
//alrededor, entonces la celda se activa
if (Plano[posX, posY] == INACTIVA && Activos == 3)
    PlanoTmp[posX, posY] = ACTIVA;

//Si la celda está activa y tiene menos de 2 celdas o
//más de 3 celdas, entonces la celda se inactiva
if (Plano[posX, posY] == ACTIVA &&
    (Activos < 2 || Activos > 3))
    PlanoTmp[posX, posY] = INACTIVA;
    }
}

//El cambio en el tablero temporal se copia sobre el tablero
Plano = PlanoTmp.Clone() as int[,];
}

private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics lienzo = e.Graphics;
    Pen Lapisz = new(Color.Blue, 1);
    Brush Llena = new SolidBrush(Color.Black);

    //Tamaño de cada celda
    int tX = 600 / Plano.GetLength(0);
    int tY = 600 / Plano.GetLength(1);
    int desplaza = 50;

    //Dibuja el arreglo bidimensional
    for (int Fil = 0; Fil < Plano.GetLength(0); Fil++) {
        for (int Col = 0; Col < Plano.GetLength(1); Col++) {
            int uX = Fil * tX + desplaza;
            int uY = Col * tY + desplaza;
            switch (Plano[Fil, Col]) {
                case INACTIVA:
                    lienzo.DrawRectangle(Lapisz, uX, uY, tX, tY);
                    break;
                case ACTIVA:
                    lienzo.FillRectangle(Llena, uX, uY, tX, tY);
                    break;
            }
        }
    }
}
}

```

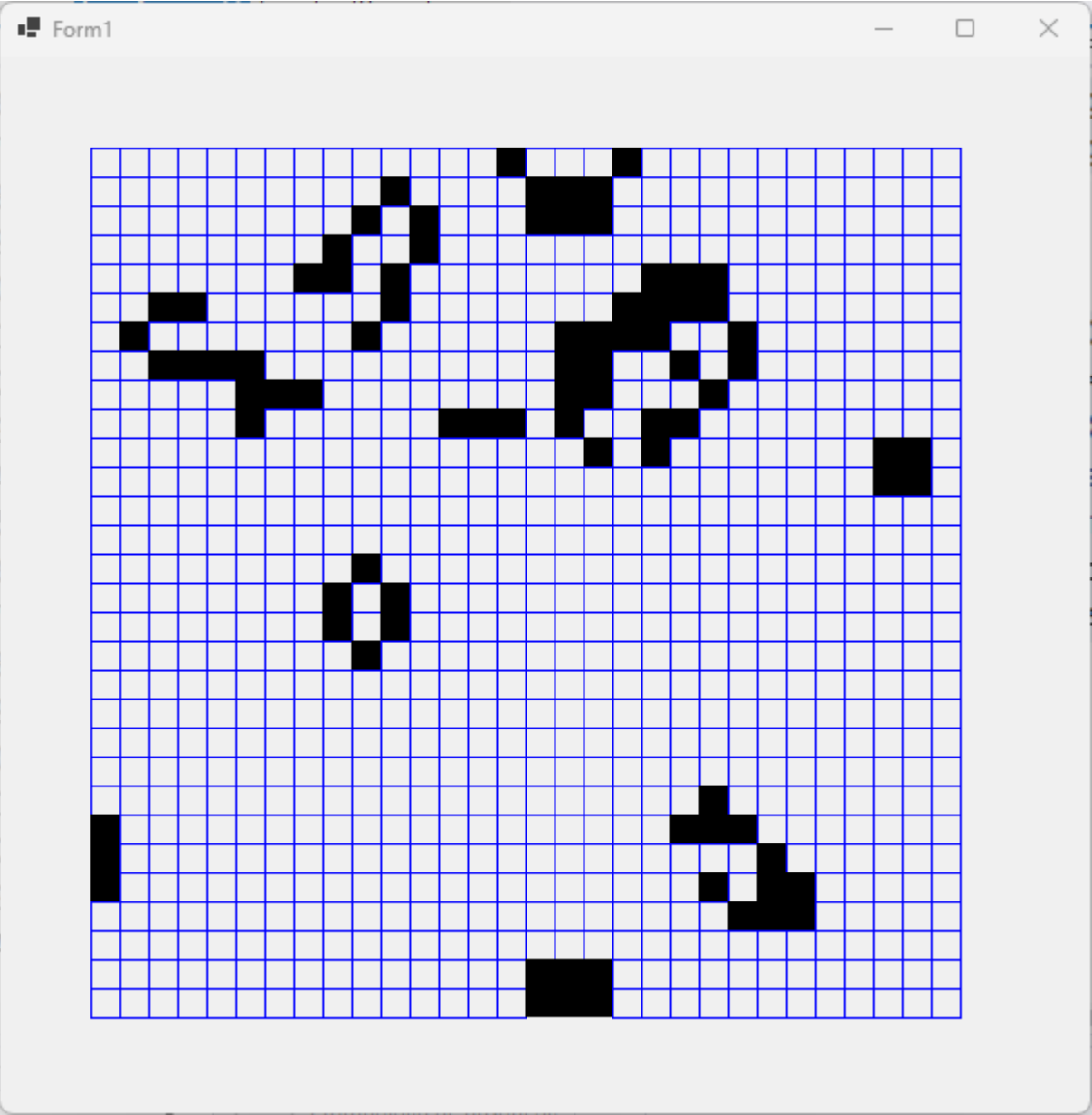


Ilustración 12: Juego de la vida

# Población e infección

Simula una población de individuos en la cual uno tiene una infección y como esta se va esparciendo por la población. Los individuos se mueven al azar y si un infectado (en rojo) coincide en la misma casilla que uno sano (en negro), el individuo sano se infecta.

N/006.zip

```
namespace Animacion {
    public partial class Form1 : Form {
        //Constantes para calificar cada celda del tablero
        private const int VACIO = 0;
        private const int SANO = 1;
        private const int INFECTADO = 2;

        int[,] Plano; //Dónde ocurre realmente la acción

        Random azar;

        //Tamaño de cada celda
        int tamF, tamC, desplaza;

        //Población
        List<Individuo> Pobl;

        public Form1() {
            InitializeComponent();
            IniciarParametros();
        }

        public void IniciarParametros() {
            Plano = new int[30, 30]; //Inicializa el tablero.
            azar = new Random();

            //Inicializa la población
            Pobl = new List<Individuo>();
            int NumIndividuos = 100;
            for (int cont = 1; cont <= NumIndividuos; cont++) {
                int Fila, Columna;
                do {
                    Fila = azar.Next(Plano.GetLength(0));
                    Columna = azar.Next(Plano.GetLength(1));
                } while (Plano[Fila, Columna] != VACIO);
                Plano[Fila, Columna] = SANO;
                Pobl.Add(new Individuo(Fila, Columna, SANO));
            }

            //Inicia con un individuo infectado
            Pobl[azar.Next(NumIndividuos)].Estado = INFECTADO;

            timer1.Start();
        }

        private void timer1_Tick(object sender, System.EventArgs e) {
            Logica();
            Refresh(); //Visual de la animación
        }

        //Lógica de la población
        private void Logica() {
            int NumFil = Plano.GetLength(0);
            int NumCol = Plano.GetLength(1);

            //Mueve los individuos
            for (int cont = 0; cont < Pobl.Count; cont++)
                Pobl[cont].Mover(azar.Next(8), NumFil, NumCol);

            //Limpia el tablero
            for (int Fil = 0; Fil < NumFil; Fil++)
                for (int Col = 0; Col < NumCol; Col++)
                    Plano[Fil, Col] = VACIO;

            //Refleja ese movimiento en el tablero
            for (int cont = 0; cont < Pobl.Count; cont++) {
```

```

        int Fila = Pobl[cont].Fila;
        int Columna = Pobl[cont].Columna;
        Plano[Fila, Columna] = Pobl[cont].Estado;
    }

    //Chequea si un individuo infectado coincide con un
    //individuo sano en la misma casilla para infectarlo
    for (int cont = 0; cont < Pobl.Count; cont++) {
        if (Pobl[cont].Estado == INFECTADO) {
            for (int busca = 0; busca < Pobl.Count; busca++) {
                if (Pobl[cont].Fila == Pobl[busca].Fila &&
                    Pobl[cont].Columna == Pobl[busca].Columna)
                    Pobl[busca].Estado = INFECTADO;
            }
        }
    }
}

private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics lienzo = e.Graphics;
    Pen Lapiz = new(Color.Blue, 1);
    Brush Llena1 = new SolidBrush(Color.Black);
    Brush Llena2 = new SolidBrush(Color.Red);

    //Tamaño de cada celda
    tamF = 500 / Plano.GetLength(0);
    tamC = 500 / Plano.GetLength(1);
    desplaza = 50;

    //Dibuja el arreglo bidimensional
    for (int Fil = 0; Fil < Plano.GetLength(0); Fil++) {
        for (int Col = 0; Col < Plano.GetLength(1); Col++) {
            int uF = Fil * tamF + desplaza;
            int uC = Col * tamC + desplaza;
            switch (Plano[Fil, Col]) {
                case VACIO:
                    lienzo.DrawRectangle(Lapiz, uF, uC, tamF, tamC);
                    break;
                case SANO:
                    lienzo.FillRectangle(Llena1, uF, uC, tamF, tamC);
                    break;
                case INFECTADO:
                    lienzo.FillRectangle(Llena2, uF, uC, tamF, tamC);
                    break;
            }
        }
    }
}

internal class Individuo {
    public int Fila, Columna, Estado;

    public Individuo(int Fila, int Columna, int Estado) {
        this.Fila = Fila;
        this.Columna = Columna;
        this.Estado = Estado;
    }

    public void Mover(int direccion, int NumFilas, int NumColumnas) {
        switch (direccion) {
            case 0: Fila--; Columna--; break;
            case 1: Fila--; break;
            case 2: Fila--; Columna++; break;
            case 3: Columna--; break;
            case 4: Columna++; break;
            case 5: Fila++; Columna--; break;
            case 6: Fila++; break;
            case 7: Fila++; Columna++; break;
        }

        if (Fila < 0) Fila = 0;
        if (Columna < 0) Columna = 0;
        if (Fila >= NumFilas) Fila = NumFilas - 1;
        if (Columna >= NumColumnas) Columna = NumColumnas - 1;
    }
}

```

```
}  
}
```

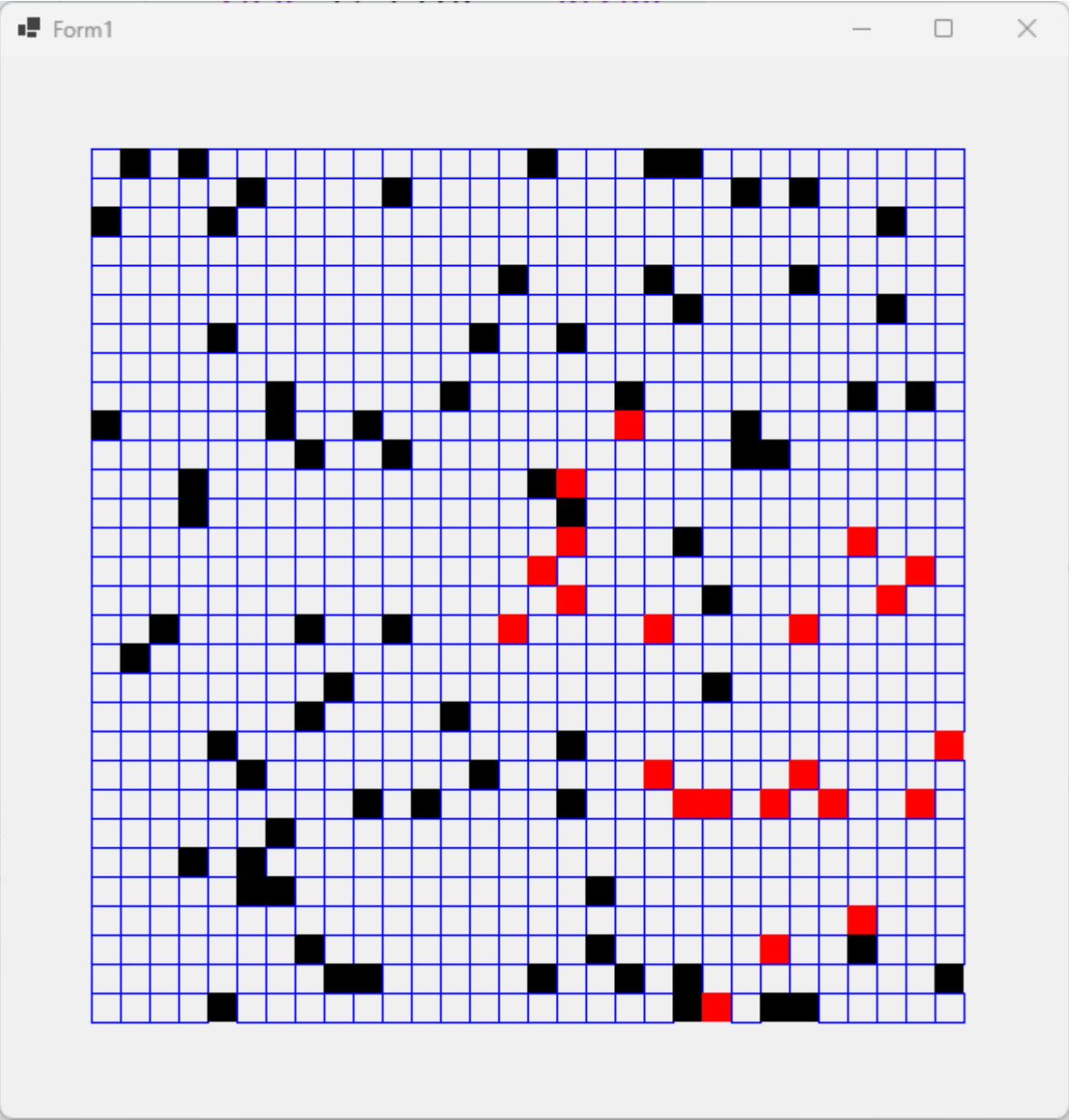


Ilustración 13: Infección propagándose

# Gráfico matemático en 2D y animado

Dada una ecuación del tipo  $Y = F(X,T)$ , donde X es la variable independiente y T es el tiempo, por ejemplo:

$$Y = X * (T + 1) * seno(X^2 * T^2)$$

Se procede a graficarlo de esta forma:

1. Saber el valor de inicio de T y el valor final de T. Se llamarán Tminimo y Tmaximo respectivamente.
2. Saber la tasa de incremento del tiempo, para ir desde Tminimo hasta Tmaximo, una vez se alcance Tmaximo se devuelve decrementando a la misma tasa hasta Tminimo y vuelve a empezar.
3. Saber el valor de inicio de X y el valor final de X. Se llamarán Xini y Xfin respectivamente.
4. También se necesita el avance de X desde el inicio hasta el final, se llamará Xavance.
5. Con un control Timer, en cada tick se da un valor a T y se hacen los siguientes cálculos:
  - a. Se hace uso de un ciclo que calcule los valores de Y. Se almacena el par X, Y
  - b. Debido a que el eje Y aumenta hacia abajo en una pantalla o ventana, habrá que darles la vuelta a los valores de Y calculados, es decir, multiplicarlos por -1. Luego realmente se almacena X, -Y
  - c. Se requieren cuatro datos:
    - i. El mínimo valor de X. Es el mismo Xini.
    - ii. El máximo valor de X. Se llamará Xmax.
    - iii. El mínimo valor de Y obtenido. Se llamará Ymin.
    - iv. El máximo valor de Y obtenido. Se llamará Ymax.
  - d. También se requieren estos dos datos para poder ajustar el gráfico matemático a un área rectangular definida en la ventana.
    - i. Coordenada superior izquierda en pantalla. Serán las coordenadas enteras positivas XpantallaIni, YpantallaIni
    - ii. Coordenada inferior derecha en pantalla. Serán las coordenadas enteras positivas XpantallaFin, YpantallaFin
  - e. Se calculan unas constantes de conversión con estas fórmulas:
    - i.  $convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xini)$
    - ii.  $convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin)$
  - f. Tomar cada coordenada calculada de la ecuación (valor, valorY) y hacerle la conversión a pantalla plana con la siguiente fórmula:
    - i.  $pantX = convierteX * (valorX - Xini) + XpantallaIni$
    - ii.  $pantY = convierteY * (valorY - Ymin) + YpantallaIni$
  - g. Se grafican esos puntos y se unen con líneas.
  - h. Se borra la pantalla y se da otro valor de T, eso da la sensación de animación.

N/007.zip

```
namespace Animacion {
    internal class Linea {
        //Valor X, Y reales de la ecuación para dibujar la línea
        public double Xa, Ya;
        public double Xb, Yb;

        //Puntos convertidos a coordenadas enteras de pantalla
        public int pantXa, pantYa;
        public int pantXb, pantYb;

        public Linea(double Xa, double Ya, double Xb, double Yb) {
            this.Xa = Xa;
            this.Ya = Ya;
            this.Xb = Xb;
            this.Yb = Yb;
        }

        //Convierte las coordenadas reales a coordenadas en pantalla
        public void Convierte(double convierteX, double convierteY, double Xmin, double Ymin, int
XpantallaIni, int YpantallaIni) {
            pantXa = Convert.ToInt32(convierteX * (Xa - Xmin) + XpantallaIni);
            pantXb = Convert.ToInt32(convierteX * (Xb - Xmin) + XpantallaIni);
            pantYa = Convert.ToInt32(convierteY * (Ya - Ymin) + YpantallaIni);
            pantYb = Convert.ToInt32(convierteY * (Yb - Ymin) + YpantallaIni);
        }
    }
}
```



```

namespace Animacion {
    public partial class Form1 : Form {
        List<Linea> Lineas;
        List<Linea> Ejes;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;
        private double TiempoValor, Tminimo, Tmaximo, Tincrementa;
        private double Xini, Xfin, Xavance;

        public Form1() {
            InitializeComponent();
            Lineas = [];
            Ejes = [];

            //Área dónde se dibujará el gráfico matemático
            XpantallaIni = 20;
            YpantallaIni = 40;
            XpantallaFin = 600;
            YpantallaFin = 550;

            //Inicia el tiempo
            Tminimo = 1;
            Tmaximo = 5;
            Tincrementa = 0.05;
            TiempoValor = Tminimo;

            //Algoritmo que calcula los puntos del gráfico
            Xini = -50;
            Xfin = 50;
            Xavance = 1;
            Logica(Xini, Xfin, Xavance);
        }

        public void Logica(double Xini, double Xfin, double Xavance) {

            Lineas.Clear();
            Ejes.Clear();

            //Calcula las líneas que tendrá el gráfico
            double valXa = Xini;
            double valYa = -Ecuacion(valXa, TiempoValor);
            while (valXa < Xfin) {
                double valXb = Math.Round(valXa + Xavance, 5);
                double valYb = -Ecuacion(valXb, TiempoValor);

                //Si la línea es válida, se agrega a la lista
                if (double.IsFinite(valYa) && double.IsFinite(valYb)) {
                    Lineas.Add(new Linea(valXa, valYa, valXb, valYb));
                }

                valXa = valXb;
                valYa = valYb;
            }

            //Determina los valores máximos y mínimos en X y Y que tienen las líneas
            double Xmin = double.MaxValue;
            double Xmax = double.MinValue;
            double Ymin = double.MaxValue;
            double Ymax = double.MinValue;
            foreach (var linea in Lineas) {
                Xmin = Math.Min(Xmin, Math.Min(linea.Xa, linea.Xb));
                Xmax = Math.Max(Xmax, Math.Max(linea.Xa, linea.Xb));
                Ymin = Math.Min(Ymin, Math.Min(linea.Ya, linea.Yb));
                Ymax = Math.Max(Ymax, Math.Max(linea.Ya, linea.Yb));
            }

            //Extremos para poder dibujar los ejes
            Xmin = Math.Min(Xmin, Xini);
            Xmax = Math.Max(Xmax, Xfin);
            Ymin = Math.Min(Ymin, 0);
            Ymax = Math.Max(Ymax, 0);
        }
    }
}

```

```

//Ejes
Ejes.Add(new Linea(Xmin, 0, Xmax, 0)); //Eje X
Ejes.Add(new Linea(0, Ymin, 0, Ymax)); //Eje Y

//Calcula los puntos a poner en la pantalla
double convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xmin);
double convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin);

for (int Cont = 0; Cont < Lineas.Count; Cont++) {
    Lineas[Cont].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
}

//Calcula donde poner los ejes
Ejes[0].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
Ejes[1].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
}

//Aquí está la ecuación que se desee graficar con variable
//independiente X y T
public double Ecuacion(double X, double T) {
    return Math.Sin(X * T * Math.PI / 180);
}

//Pinta la ecuación
private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics Lienzo = e.Graphics;
    Pen LapizGrafica = new(Color.Blue, 2);
    Pen LapizEje = new(Color.Red, 2);

    //Dibuja el gráfico matemático
    for (int Cont = 0; Cont < Lineas.Count; Cont++) {
        Lienzo.DrawLine(LapizGrafica, Lineas[Cont].pantXa, Lineas[Cont].pantYa,
            Lineas[Cont].pantXb, Lineas[Cont].pantYb);
    }

    //Dibuja los ejes
    Lienzo.DrawLine(LapizEje, Ejes[0].pantXa, Ejes[0].pantYa,
        Ejes[0].pantXb, Ejes[0].pantYb);
    Lienzo.DrawLine(LapizEje, Ejes[1].pantXa, Ejes[1].pantYa,
        Ejes[1].pantXb, Ejes[1].pantYb);
}

private void timer1_Tick(object sender, EventArgs e) {
    TiempoValor += Tincrementa;
    if (TiempoValor <= Tminimo || TiempoValor >= Tmaximo)
        Tincrementa = -Tincrementa;

    Logica(Xini, Xfin, Xavance);
    Refresh();
}
}
}

```



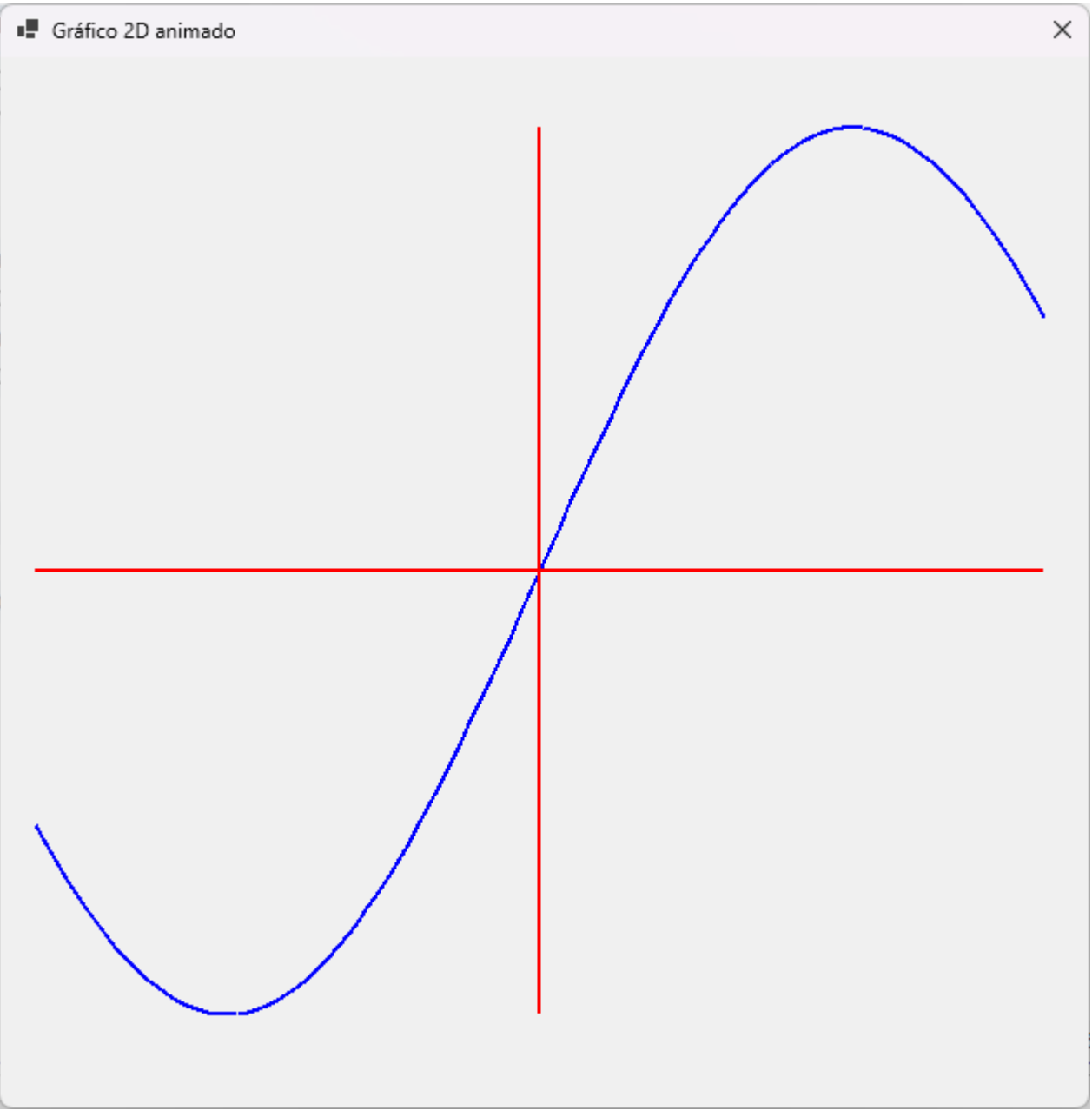


Ilustración 14: Gráfico matemático 2D animado

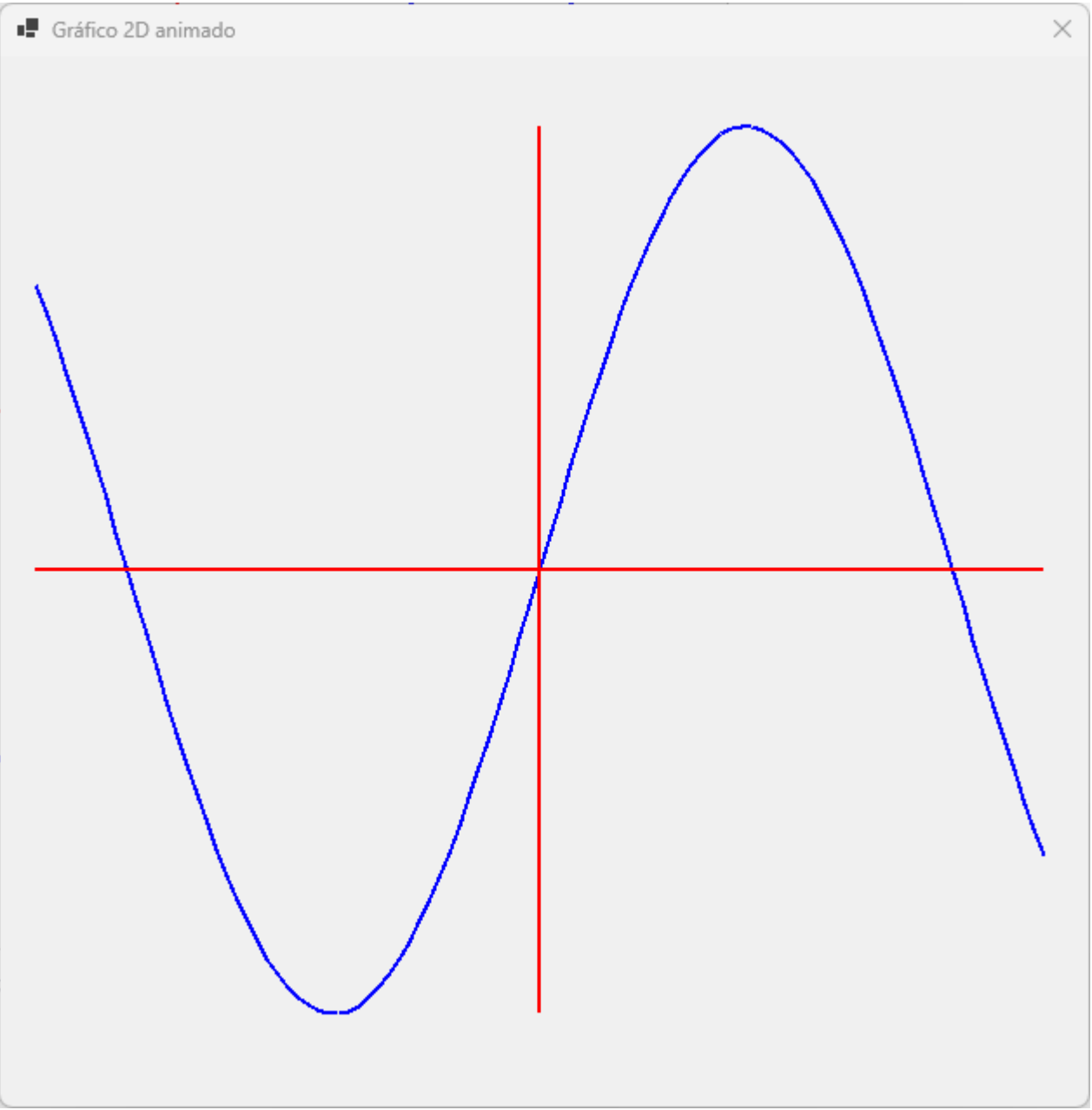


Ilustración 15: : Gráfico matemático 2D animado

# Gráfico polar animado

Dada una ecuación del tipo  $r=F(\theta, T)$ , donde  $\theta$  es el ángulo,  $r$  el radio y  $T$  el tiempo, por ejemplo:

$$r = 2 * seno(4 * \theta * T * \pi/180)$$

Se procede a graficarlo de esta forma:

1. Saber el valor de inicio de  $T$  y el valor final de  $T$ . Se llamarán  $T_{\text{minimo}}$  y  $T_{\text{maximo}}$  respectivamente.
2. Saber la tasa de incremento del tiempo, para ir desde  $T_{\text{minimo}}$  hasta  $T_{\text{maximo}}$ , una vez se alcance  $T_{\text{maximo}}$  se devuelve decrementando a la misma tasa hasta  $T_{\text{minimo}}$  y vuelve a empezar.
3. Saber el valor de inicio de  $\theta$  y el valor final de  $\theta$ . Se llamarán  $\text{minTheta}$  y  $\text{maxTheta}$  respectivamente.
4. Con un control `Timer`, en cada tick se da un valor a  $T$  y se hacen los siguientes cálculos:
  - a. Con  $\theta$  desde  $\text{minTheta}$  y  $\text{maxTheta}$  se calculan los valores de  $r$ . Luego se hace esta conversión:

```
r = Ecuacion(theta, T);  
  
X = r * Math.Cos(theta * Math.PI / 180);  
  
Y = r * Math.Sin(theta * Math.PI / 180);
```
  - b. Debido a que el eje  $Y$  aumenta hacia abajo en una pantalla o ventana, habrá que darles la vuelta a los valores de  $Y$  calculados, es decir, multiplicarlos por  $-1$ . Luego realmente se almacena  $X$ ,  $-Y$
  - c. Se calculan cuatro datos:
    - i. El mínimo valor de  $X$ . Se llamará  $X_{\text{min}}$ .
    - ii. El máximo valor de  $X$ . Se llamará  $X_{\text{max}}$ .
    - iii. El mínimo valor de  $Y$  obtenido. Se llamará  $Y_{\text{min}}$ .
    - iv. El máximo valor de  $Y$  obtenido. Se llamará  $Y_{\text{max}}$ .
  - d. Se requieren dos datos para poder ajustar el gráfico matemático a un área rectangular definida en la ventana.
    - i. Coordenada superior izquierda en pantalla. Serán las coordenadas enteras positivas  $X_{\text{pantallaIni}}$ ,  $Y_{\text{pantallaIni}}$
    - ii. Coordenada inferior derecha en pantalla. Serán las coordenadas enteras positivas  $X_{\text{pantallaFin}}$ ,  $Y_{\text{pantallaFin}}$
  - e. Se calculan unas constantes de conversión con estas fórmulas:
    - i.  $\text{convierteX} = (X_{\text{pantallaFin}} - X_{\text{pantallaIni}}) / (X_{\text{max}} - X_{\text{min}})$
    - ii.  $\text{convierteY} = (Y_{\text{pantallaFin}} - Y_{\text{pantallaIni}}) / (Y_{\text{max}} - Y_{\text{min}})$
  - f. Tomar cada coordenada calculada de la ecuación (valor, valor $Y$ ) y hacerle la conversión a pantalla plana con la siguiente fórmula:
    - i.  $\text{pantX} = \text{convierteX} * (\text{valorX} - X_{\text{min}}) + X_{\text{pantallaIni}}$
    - ii.  $\text{pantY} = \text{convierteY} * (\text{valorY} - Y_{\text{min}}) + Y_{\text{pantallaIni}}$
  - g. Se grafican esos puntos y se unen con líneas.

Se borra la pantalla y se da otro valor de  $T$ , eso da la sensación de animación.

N/008.zip

```
namespace Animacion {  
    internal class Linea {  
        //Valor X, Y reales de la ecuación para dibujar la línea  
        public double Xa, Ya;  
        public double Xb, Yb;  
  
        //Puntos convertidos a coordenadas enteras de pantalla  
        public int pantXa, pantYa;  
        public int pantXb, pantYb;  
  
        public Linea(double Xa, double Ya, double Xb, double Yb) {  
            this.Xa = Xa;  
            this.Ya = Ya;  
            this.Xb = Xb;  
            this.Yb = Yb;  
        }  
  
        //Convierte las coordenadas reales a coordenadas en pantalla  
        public void Convierte(double convierteX, double convierteY, double Xmin, double Ymin, int  
XpantallaIni, int YpantallaIni) {  
            pantXa = Convert.ToInt32(convierteX * (Xa - Xmin) + XpantallaIni);  
            pantXb = Convert.ToInt32(convierteX * (Xb - Xmin) + XpantallaIni);  
            pantYa = Convert.ToInt32(convierteY * (Ya - Ymin) + YpantallaIni);  
            pantYb = Convert.ToInt32(convierteY * (Yb - Ymin) + YpantallaIni);  
        }  
    }  
}
```

```

namespace Animacion {
    public partial class Form1 : Form {
        List<Linea> Lineas;
        List<Linea> Ejes;
        int XpantallaIni, YpantallaIni, XpantallaFin, YpantallaFin;
        private double TiempoValor, Tminimo, Tmaximo, Tincrementa;
        private double minTheta, maxTheta, Theta_avance;

        public Form1() {
            InitializeComponent();
            Lineas = [];
            Ejes = [];

            //Área dónde se dibujará el gráfico matemático
            XpantallaIni = 20;
            YpantallaIni = 40;
            XpantallaFin = 600;
            YpantallaFin = 550;

            //Inicia el tiempo
            Tminimo = 1;
            Tmaximo = 7;
            Tincrementa = 0.05;
            TiempoValor = Tminimo;

            //Algoritmo que calcula los puntos del gráfico
            minTheta = 0;
            maxTheta = 360;
            Theta_avance = 1;
            Logica(minTheta, maxTheta, Theta_avance);
        }

        public void Logica(double minTheta, double maxTheta, double Theta_avance) {

            Lineas.Clear();
            Ejes.Clear();

            //Calcula las líneas que tendrá el gráfico
            double theta = minTheta;
            double valorR = Ecuacion(theta, TiempoValor);
            double valXa = valorR * Math.Cos(theta * Math.PI / 180);
            double valYa = -1 * valorR * Math.Sin(theta * Math.PI / 180);

            while (theta <= maxTheta) {
                theta = Math.Round(theta + Theta_avance, 5);
                valorR = Ecuacion(theta, TiempoValor);
                double valXb = valorR * Math.Cos(theta * Math.PI / 180);
                double valYb = -1 * valorR * Math.Sin(theta * Math.PI / 180);

                //Si la línea es válida, se agrega a la lista
                if (double.IsFinite(valYa) && double.IsFinite(valYb)) {
                    Lineas.Add(new Linea(valXa, valYa, valXb, valYb));
                }

                valXa = valXb;
                valYa = valYb;
            }

            //Determina los valores máximos y mínimos en X y Y que tienen las líneas
            double Xmin = double.MaxValue;
            double Xmax = double.MinValue;
            double Ymin = double.MaxValue;
            double Ymax = double.MinValue;
            foreach (var linea in Lineas) {
                Xmin = Math.Min(Xmin, Math.Min(linea.Xa, linea.Xb));
                Xmax = Math.Max(Xmax, Math.Max(linea.Xa, linea.Xb));
                Ymin = Math.Min(Ymin, Math.Min(linea.Ya, linea.Yb));
                Ymax = Math.Max(Ymax, Math.Max(linea.Ya, linea.Yb));
            }

            //Extremos para poder dibujar los ejes
            Xmin = Math.Min(Xmin, 0);
            Xmax = Math.Max(Xmax, 0);
            Ymin = Math.Min(Ymin, 0);
            Ymax = Math.Max(Ymax, 0);
        }
    }
}

```

```

//Ejes
Ejes.Add(new Linea(Xmin, 0, Xmax, 0)); //Eje X
Ejes.Add(new Linea(0, Ymin, 0, Ymax)); //Eje Y

//Calcula los puntos a poner en la pantalla
double convierteX = (XpantallaFin - XpantallaIni) / (Xmax - Xmin);
double convierteY = (YpantallaFin - YpantallaIni) / (Ymax - Ymin);

for (int Cont = 0; Cont < Lineas.Count; Cont++) {
    Lineas[Cont].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
}

//Calcula donde poner los ejes
Ejes[0].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
Ejes[1].Convierte(convierteX, convierteY, Xmin, Ymin, XpantallaIni, YpantallaIni);
}

//Aquí está la ecuación que se desee graficar
//con variable Theta y T (tiempo)
public double Ecuacion(double Theta, double T) {
    return 2 * (1 + Math.Sin(Theta * T * Math.PI / 180));
}

//Pinta la ecuación
private void Form1_Paint(object sender, PaintEventArgs e) {
    Graphics Lienzo = e.Graphics;
    Pen LapizGrafica = new(Color.Blue, 2);
    Pen LapizEje = new(Color.Red, 2);

    //Dibuja el gráfico matemático
    for (int Cont = 0; Cont < Lineas.Count; Cont++) {
        Lienzo.DrawLine(LapizGrafica, Lineas[Cont].pantXa, Lineas[Cont].pantYa,
            Lineas[Cont].pantXb, Lineas[Cont].pantYb);
    }

    //Dibuja los ejes
    Lienzo.DrawLine(LapizEje, Ejes[0].pantXa, Ejes[0].pantYa,
        Ejes[0].pantXb, Ejes[0].pantYb);
    Lienzo.DrawLine(LapizEje, Ejes[1].pantXa, Ejes[1].pantYa,
        Ejes[1].pantXb, Ejes[1].pantYb);
}

private void timer1_Tick(object sender, EventArgs e) {
    TiempoValor += Tincrementa;
    if (TiempoValor <= Tminimo || TiempoValor >= Tmaximo)
        Tincrementa = -Tincrementa;

    Logica(minTheta, maxTheta, Theta_avance);
    Refresh();
}
}
}

```

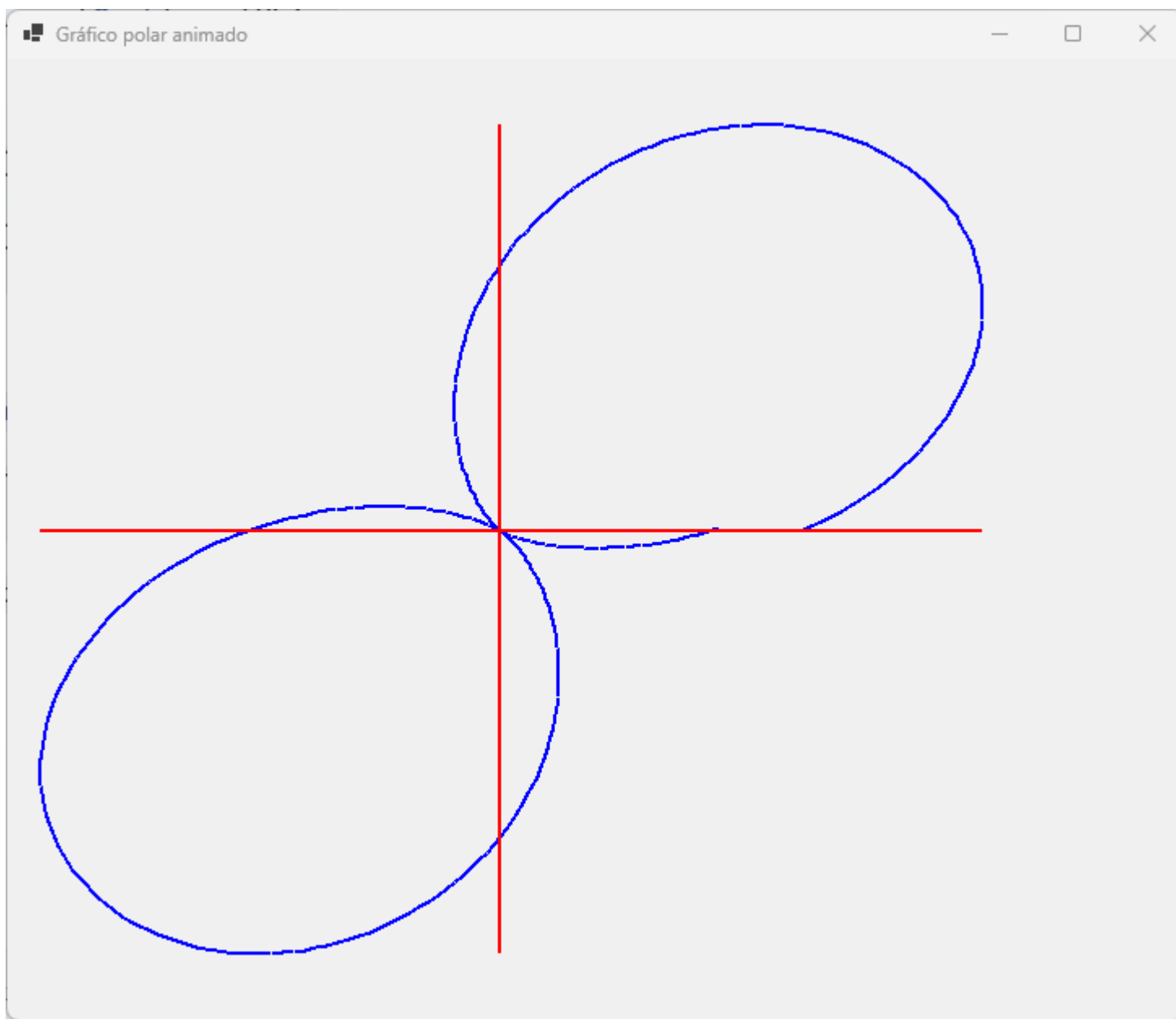


Ilustración 16: Gráfico polar animado

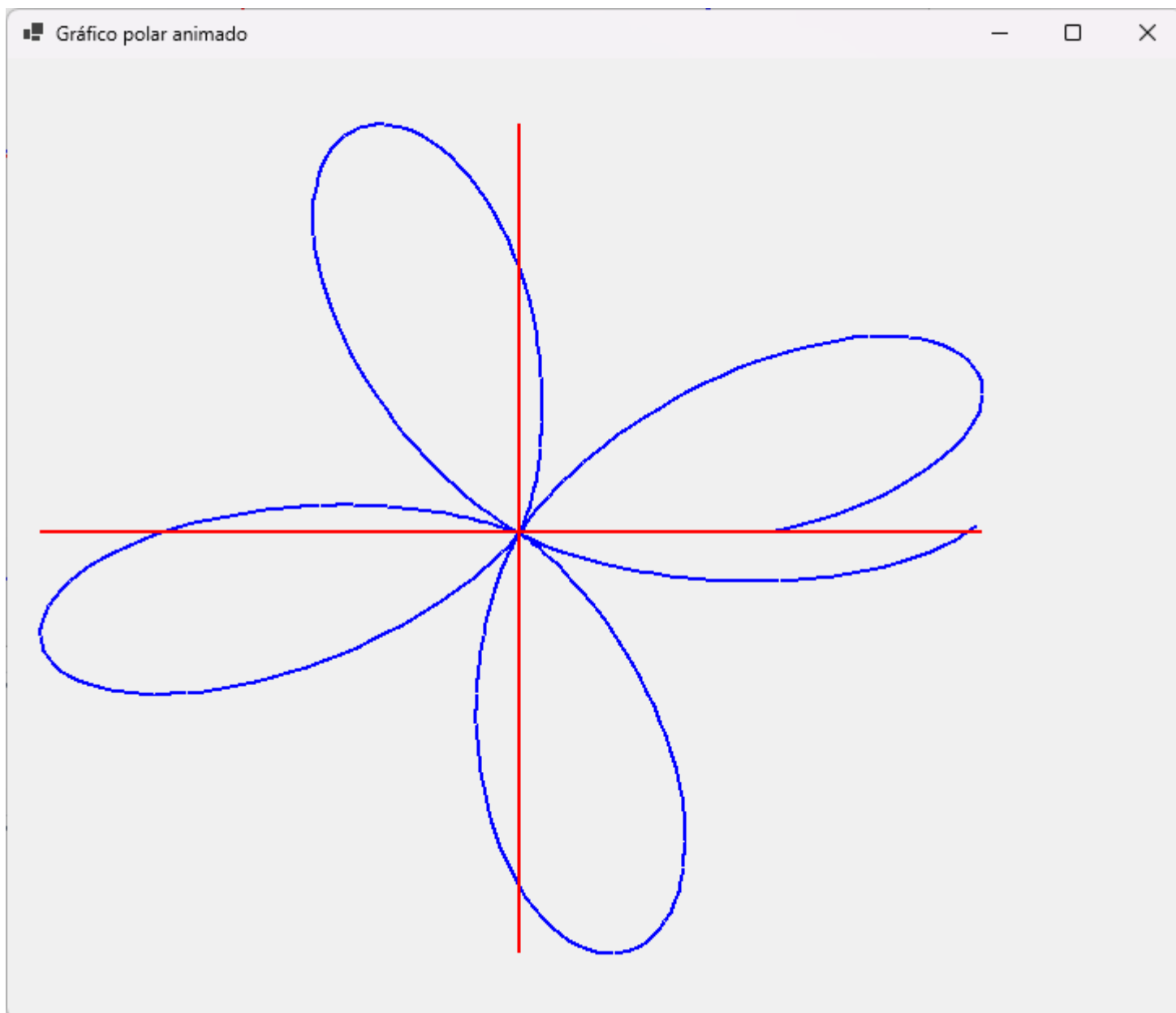


Ilustración 17: Gráfico polar animado

## Gráfico Matemático en 3D. Giros animados.

Cuando tenemos una ecuación del tipo  $Z = F(X,Y)$ , tenemos una ecuación con dos variables independientes y una variable dependiente. Se van a hacer giros en los tres ejes de forma aleatoria y continua por lo que da la sensación de una animación. Por ejemplo:

$$Z = \sqrt[2]{X^2 + Y^2} + 3 * Cos(\sqrt[2]{X^2 + Y}) + 5$$

Los pasos para hacer el gráfico son parecidos al anterior:

Paso 1: Saber el valor donde inicia X hasta donde termina. Igual sucede con Y, donde inicia Y hasta donde termina.

Paso 2: Con esos valores se calcula el valor Z. Al final se tiene un conjunto de coordenadas posX, posY, posZ. Se va a hacer uso de polígonos, por lo que se calculan cuatro coordenadas para formar el polígono.

Paso 3: Se normalizan los valores de posX, posY, posZ para que queden entre 0 y 1, luego se le resta -0.5 ¿Para qué? Para que los puntos (realmente polígonos) queden contenidos dentro de un cubo de lado=1, cuyo centro está en 0,0,0. Ver los dos temas anteriores como se muestra el cubo.

Paso 4: Luego se aplica un giro en los tres ángulos. Se obtiene posXg, posYg, posZg

Paso 5: Se ordenan los polígonos del más profundo (menor valor de posZg) al más superficial (mayor valor de posZg). Por esa razón, la clase Polígono hereda de IComparable

Paso 6: Con Xg, Yg, Zg se proyecta a la pantalla, obteniéndose el planoX, planoY.

Paso 7: Con planoX, planoY se aplican las constantes que se calcularon anteriormente y se obtiene la proyección en pantalla, es decir, pantallaX, pantallaY

Paso 8: Se dibujan los polígonos, primero rellenándolos con el color del fondo y luego se gráfica el perímetro de ese polígono.

Paso 9: Se calcula un nuevo giro en los tres ángulos. Y se vuelve al Paso 4.

Nota 1: El gráfico se ve animado por lo que es necesario que el formulario tenga en "true" la propiedad DoubleBuffered

Nota 2: El gráfico animado está contenido dentro de un cubo invisible de lado = 1. Eso se logra con la normalización. Sin importar el valor que tome Z, el gráfico estará contenido dentro de ese cubo, luego si Z crece mucho, el resultado visual es que el gráfico se empequeñece para que se ajuste todo a ese cubo

N/009.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```



```

namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}

```

```

namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        //Color de relleno del polígono
        Color ColorZ;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }

        //Calcula la profundidad y crea los vértices del polígono
        public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
            double Z1 = puntos[punto1].Zgiro;
            double Z2 = puntos[punto2].Zgiro;
            double Z3 = puntos[punto3].Zgiro;
            double Z4 = puntos[punto4].Zgiro;
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;

            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

            int TotalColores = colorList.Count;
            double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2)
/ 4;

            int ColorEscoge = (int)Math.Floor(TotalColores * PromedioZ);
            ColorZ = colorList[ColorEscoge];
        }

        //Hace el gráfico del polígono
        public void Dibuja(Graphics Lienzo) {
            //Dibuja el polígono relleno
            Brush Relleno2 = new SolidBrush(ColorZ);
            Lienzo.FillPolygon(Relleno2, ListaPuntos);
        }
    }
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;
    }
}

```

```

//Colores para pintar la malla
List<Color> ListaColores;

//Ecuación
string EcuacionAnterior = "";
Evaluador4 Evaluador = new();

public Objeto3D() {
    int NumColores = 40; // Número de colores a generar

    //Genera listado de colores para el gráfico
    ListaColores = [];
    int Mitad = NumColores / 2;

    // Gradiente de azul a amarillo
    for (int Cont = 0; Cont < Mitad; Cont++) {
        int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación  $Z = F(X,Y)$ 
//para dibujarla
public void CalcularFigura3D(string Ecuacion, double Xini, double Yini, double Xfin, double Yfin, int
NumLineas, double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int
XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double IncrX = (Xfin - Xini) / (NumLineas - 1);
    double IncrY = (Yfin - Yini) / (NumLineas - 1);
    double X = Xini, Y = Yini;

    //Valida que la ecuación tenga valores para graficar y de paso calcula
    //las constantes para normalizar
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;
    for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            Evaluador.DarValorVariable('y', Y);
            double Z = Evaluador.Evaluar();
            if (double.IsNaN(Z) || double.IsInfinity(Z)) Z = 0;

            if (X < Xmin) Xmin = X;
            if (Y < Ymin) Ymin = Y;
            if (Z < Zmin) Zmin = Z;
            if (X > Xmax) Xmax = X;
            if (Y > Ymax) Ymax = Y;
            if (Z > Zmax) Zmax = Z;
        }
    }
}

```



```

        X += IncrX;
    }
    X = Xini;
    Y += IncrY;
}

if ( Math.Abs(Xmin-Xmax) < 0.0001 || Math.Abs(Ymin-Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
    MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

//Coordenadas espaciales X,Y,Z
X = Xini;
Y = Yini;
puntos = [];
for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
    for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
        Evaluador.DarValorVariable('x', X);
        Evaluador.DarValorVariable('y', Y);
        double Z = Evaluador.Evaluar();
        if (double.IsNaN(Z) || double.IsInfinity(Z)) Z = 0;
        puntos.Add(new Punto(X, Y, Z));
        X += IncrX;
    }
    X = Xini;
    Y += IncrY;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    poligonos.Add(new Poligono(
        coordenadaActual,
        coordenadaActual + 1,
        coordenadaActual + NumLineas + 1,
        coordenadaActual + NumLineas
    ));

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},

```

```

        {-SinY, SinX * CosY, CosX * CosY }
    };

    //Las constantes de transformación para cuadrar en pantalla
    double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
    double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

    //Gira los 8 puntos
    for (int cont = 0; cont < puntos.Count; cont++)
        puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

    //Calcula la profundidad y forma el polígono
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

    //Algoritmo de pintor.
    //Ordena del polígono más alejado al más cercano,
    //los polígonos de adelante son visibles y los de atrás son borrados.
    poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;

            FrmGrafico.Show();
        }

        private void numMinimoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMinimoX.Value = numMaximoX.Value - 1;

            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Refresh();
        }

        private void numMinimoY_ValueChanged(object sender, EventArgs e) {
            if (numMinimoY.Value >= numMaximoY.Value)
                numMinimoY.Value = numMaximoY.Value - 1;

            FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
            FrmGrafico.Refresh();
        }

        private void numMaximoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMaximoX.Value = numMinimoX.Value + 1;

            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.Refresh();
        }

        private void numMaximoY_ValueChanged(object sender, EventArgs e) {

```

```

        if (numMinimoY.Value >= numMaximoY.Value)
            numMaximoY.Value = numMinimoY.Value + 1;

        FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
        FrmGrafico.Refresh();
    }

    private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
        FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
        FrmGrafico.Refresh();
    }

    private void btnProcesar_Click(object sender, EventArgs e) {
        FrmGrafico.Ecuacion = txtEcuacion.Text;
        FrmGrafico.Refresh();
    }
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Distancia del observador
        public double ZPersona;

        //Rango de valores
        public double Xini, Yini, Xfin, Yfin;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        //Para el movimiento aleatorio del gráfico 3D
        private double AnguloX, AnguloY, AnguloZ;
        private double IncrAngX, IncrAngY, IncrAngZ;
        private double AnguloXNuevo, AnguloYNuevo, AnguloZNuevo;
        Random Azar;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();

            //Ángulos aleatorios para simular animación
            Azar = new Random();
            AnguloX = Azar.Next(0, 360);
            AnguloY = Azar.Next(0, 360);
            AnguloZ = Azar.Next(0, 360);
            AnguloXNuevo = Azar.Next(0, 360);
            AnguloYNuevo = Azar.Next(0, 360);
            AnguloZNuevo = Azar.Next(0, 360);
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            //Dibuja la figura 3D
            Figura3D.CalcularFigura3D(Ecuacion, Xini, Yini, Xfin, Yfin, NumLineas, AnguloX*Math.PI/180,
            AnguloY * Math.PI / 180, AnguloZ * Math.PI / 180, ZPersona, 0, 0, this.ClientSize.Width,
            this.ClientSize.Height);
            Figura3D.Dibuja(e.Graphics);
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
            Application.Exit();
        }

        private void Form1_Resize(object sender, EventArgs e) {
            Refresh();
        }
    }
}

```

```

private void timer1_Tick(object sender, EventArgs e) {
    //Cambio de los ángulos de giro
    if (AnguloX > AnguloXNuevo) IncrAngX = -1; else IncrAngX = 1;
    if (AnguloY > AnguloYNuevo) IncrAngY = -1; else IncrAngY = 1;
    if (AnguloZ > AnguloZNuevo) IncrAngZ = -1; else IncrAngZ = 1;

    AnguloX += IncrAngX;
    AnguloY += IncrAngY;
    AnguloZ += IncrAngZ;

    if (AnguloX == AnguloXNuevo) AnguloXNuevo = Azar.Next(0, 360);
    if (AnguloY == AnguloYNuevo) AnguloYNuevo = Azar.Next(0, 360);
    if (AnguloZ == AnguloZNuevo) AnguloZNuevo = Azar.Next(0, 360);
    Refresh();
}
}
}

```

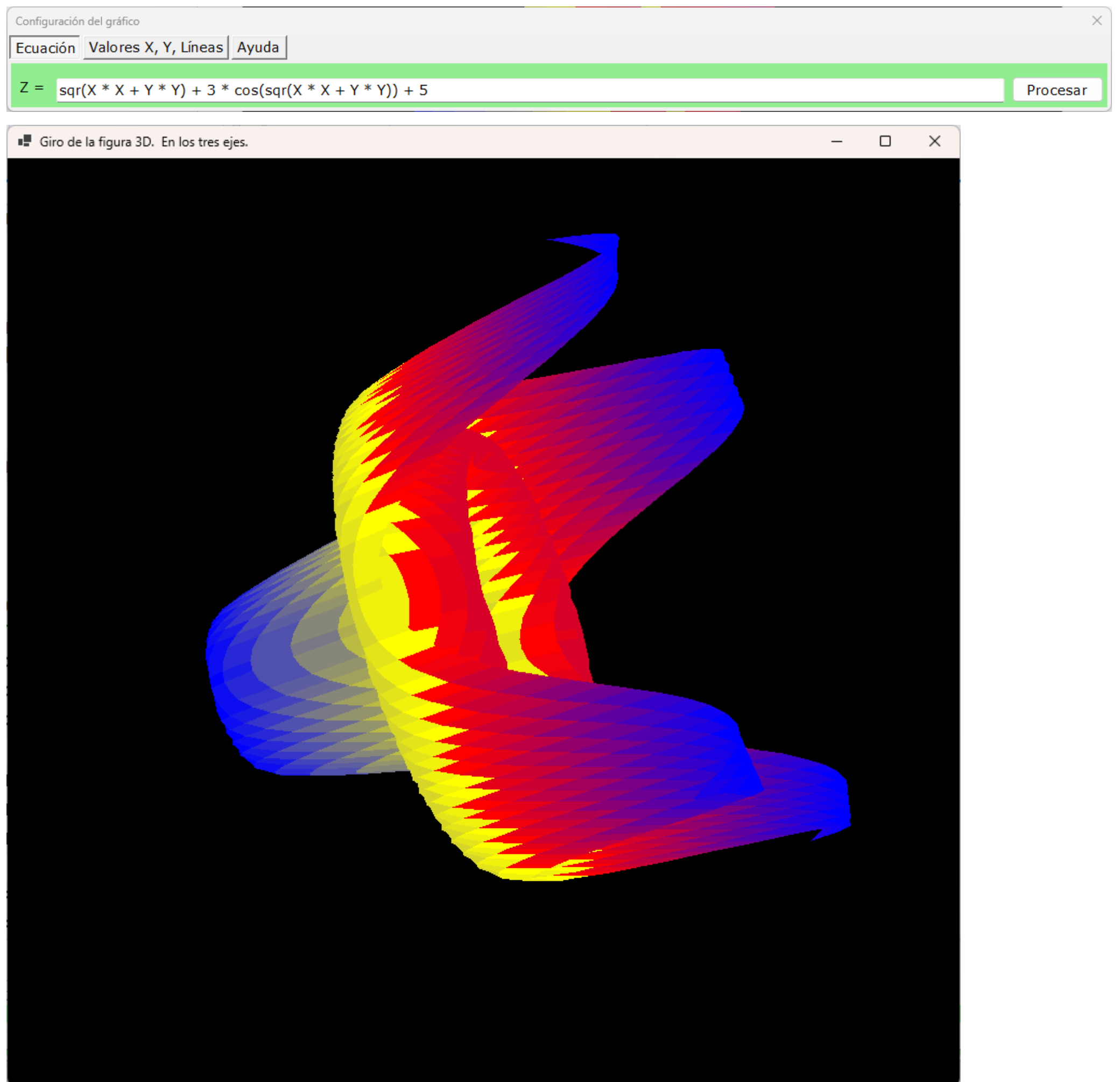
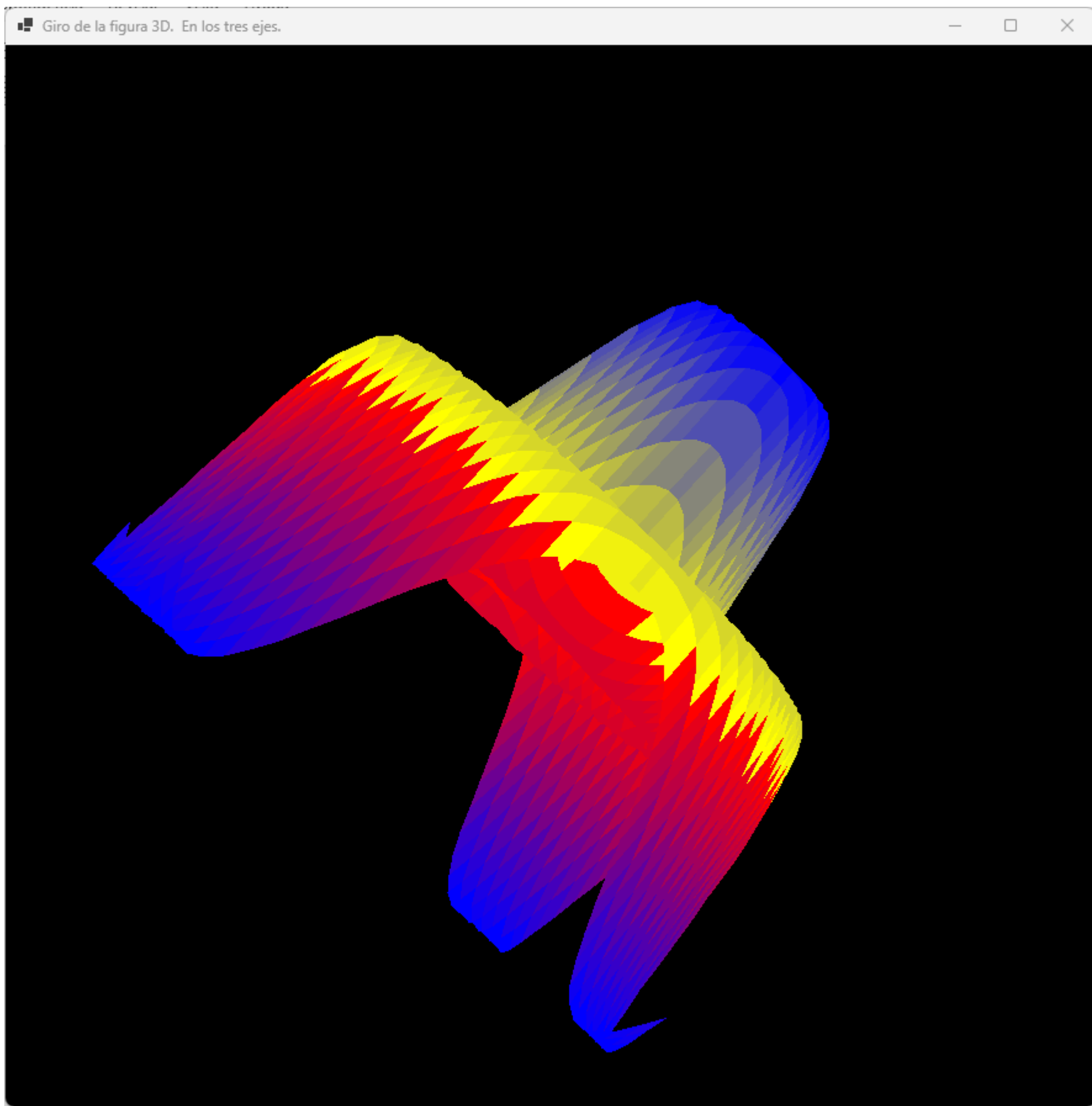


Ilustración 18: Gráfico Matemático en 3D. Giros animados



*Ilustración 19: Gráfico Matemático en 3D. Giros animados*

# Gráfico Matemático en 4D

Cuando tenemos una ecuación del tipo  $Z = F(X,Y,T)$ , tenemos una ecuación con tres variables independientes (entre ellas, la del tiempo por lo que se conoce como 4D) y una variable dependiente. Su representación es 3D animado. Por ejemplo:

$$Z = \sqrt[2]{T * X^2 + T * Y^2} + 3 * Cos(\sqrt[2]{T * X^2 + T * Y^2}) + 5$$

Los pasos para hacer el gráfico son los siguientes:

Paso 0: Saber el valor del tiempo mínimo y el valor del tiempo máximo. Se hace uso de un control “timer” que cada vez que se dispara el evento “Tick” varía el valor de T desde el mínimo hasta el máximo paso a paso, una vez alcanzado el máximo se decrementa paso a paso hasta el mínimo y repite el ciclo. De resto es igual a hacer un gráfico de ecuación tipo  $Z = F(X,Y)$ .

Paso 1: Saber el valor donde inicia X hasta donde termina. Igual sucede con Y, donde inicia Y hasta donde termina.

Paso 2: Con esos valores se calcula el valor Z. Al final se tiene un conjunto de coordenadas posX, posY, posZ. Se va a hacer uso de polígonos, por lo que se calculan cuatro coordenadas para formar el polígono.

Paso 3: Se normalizan los valores de posX, posY, posZ para que queden entre 0 y 1, luego se le resta -0.5 ¿Para qué? Para que los puntos (realmente polígonos) queden contenidos dentro de un cubo de lado=1, cuyo centro está en 0,0,0. Ver los dos temas anteriores como se muestra el cubo.

Paso 4: Luego se aplica el giro en los tres ángulos. Se obtiene posXg, posYg, posZg

Paso 5: Se ordenan los polígonos del más profundo (menor valor de posZg) al más superficial (mayor valor de posZg). Por esa razón, la clase Polígono hereda de IComparable

Paso 6: Con Xg, Yg, Zg se proyecta a la pantalla, obteniéndose el planoX, planoY.

Paso 7: Con planoX, planoY se aplican las constantes que se calcularon anteriormente y se obtiene la proyección en pantalla, es decir, pantallaX, pantallaY

Paso 8: Se dibujan los polígonos, primero rellenándolos con el color del fondo y luego se gráfica el perímetro de ese polígono.

Nota 1: El gráfico se ve animado por lo que es necesario que el formulario tenga en “true” la propiedad DoubleBuffered

Nota 2: El gráfico animado está contenido dentro de un cubo invisible de lado = 1. Eso se logra con la normalización. Sin importar el valor que tome Z, el gráfico estará contenido dentro de ese cubo, luego si Z crece mucho, el resultado visual es que el gráfico se empequeñece para que se ajuste todo a ese cubo.

N/010.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```



```
}
```

```
namespace Graficos {  
    //Conecta con una línea recta un punto con otro  
    internal class Conexion {  
        public int punto1, punto2;  
  
        public Conexion(int punto1, int punto2) {  
            this.punto1 = punto1;  
            this.punto2 = punto2;  
        }  
    }  
}
```

```
namespace Graficos {  
    //Triángulo  
    internal class Poligono {  
        public int punto1, punto2, punto3, punto4;  
        public double Centro;  
  
        //Coordenadas de dibujo del polígono  
        private Point Polig1, Polig2, Polig3, Polig4;  
        private Point[] ListaPuntos;  
  
        //Color de relleno del polígono  
        Color ColorZ;  
  
        public Poligono(int punto1, int punto2, int punto3, int punto4) {  
            this.punto1 = punto1;  
            this.punto2 = punto2;  
            this.punto3 = punto3;  
            this.punto4 = punto4;  
        }  
  
        //Calcula la profundidad y crea los vértices del polígono  
        public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {  
            double Z1 = puntos[punto1].Zgiro;  
            double Z2 = puntos[punto2].Zgiro;  
            double Z3 = puntos[punto3].Zgiro;  
            double Z4 = puntos[punto4].Zgiro;  
            Centro = (Z1 + Z2 + Z3 + Z4) / 4;  
  
            Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);  
            Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);  
            Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);  
            Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);  
            ListaPuntos = [Polig1, Polig2, Polig3, Polig4];  
  
            int TotalColores = colorList.Count;  
            double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2)  
/ 4;  
            int ColorEscoge = (int)Math.Floor(TotalColores * PromedioZ);  
            ColorZ = colorList[ColorEscoge];  
        }  
  
        //Hace el gráfico del polígono  
        public void Dibuja(Graphics Lienzo) {  
            //Dibuja el polígono relleno  
            Brush Relleno2 = new SolidBrush(ColorZ);  
            Lienzo.FillPolygon(Relleno2, ListaPuntos);  
        }  
    }  
}
```

```
namespace Graficos {  
    internal class Objeto3D {  
        //Coordenadas espaciales X, Y, Z  
        private List<Punto> puntos;
```



```

//Coordenadas del polígono (triángulo)
private List<Poligono> poligonos;

//Colores para pintar la malla
List<Color> ListaColores;

//Ecuación
string EcuacionAnterior = "";
Evaluador4 Evaluador = new();

public Objeto3D() {
    int NumColores = 40; // Número de colores a generar

    //Genera listado de colores para el gráfico
    ListaColores = [];
    int Mitad = NumColores / 2;

    // Gradiente de azul a amarillo
    for (int Cont = 0; Cont < Mitad; Cont++) {
        int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación  $Z = F(X,Y)$ 
//para dibujarla
public void CalcularFigura3D(string Ecuacion, double Xini, double Yini, double Xfin, double Yfin, int
NumLineas, double valorT, double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int
YpantallaIni, int XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double IncrX = (Xfin - Xini) / (NumLineas - 1);
    double IncrY = (Yfin - Yini) / (NumLineas - 1);
    double X = Xini, Y = Yini;

    //Valida que la ecuación tenga valores para graficar y de paso calcula
    //las constantes para normalizar
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;
    for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            Evaluador.DarValorVariable('y', Y);
            Evaluador.DarValorVariable('t', valorT);
            double Z = Evaluador.Evaluar();
            if (double.IsNaN(Z) || double.IsInfinity(Z)) Z = 0;

            if (X < Xmin) Xmin = X;
            if (Y < Ymin) Ymin = Y;
            if (Z < Zmin) Zmin = Z;
        }
    }
}

```



```

        if (X > Xmax) Xmax = X;
        if (Y > Ymax) Ymax = Y;
        if (Z > Zmax) Zmax = Z;

        X += IncrX;
    }
    X = Xini;
    Y += IncrY;
}

if ( Math.Abs(Xmin-Xmax) < 0.0001 || Math.Abs(Ymin-Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
    MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    return;
}

//Coordenadas espaciales X,Y,Z
X = Xini;
Y = Yini;
puntos = [];
for (int EjeY = 1; EjeY <= NumLineas; EjeY++) {
    for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
        Evaluador.DarValorVariable('x', X);
        Evaluador.DarValorVariable('y', Y);
        Evaluador.DarValorVariable('t', valorT);
        double Z = Evaluador.Evaluar();
        if (double.IsNaN(Z) || double.IsInfinity(Z)) Z = 0;
        puntos.Add(new Punto(X, Y, Z));
        X += IncrX;
    }
    X = Xini;
    Y += IncrY;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    poligonos.Add(new Poligono(
        coordenadaActual,
        coordenadaActual + 1,
        coordenadaActual + NumLineas + 1,
        coordenadaActual + NumLineas
    ));

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

```

```

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
    {-SinY, SinX * CosY, CosX * CosY }
};

//Las constantes de transformación para cuadrar en pantalla
double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

//Gira los 8 puntos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
            FrmGrafico.Tini = Convert.ToDouble((double)numMinimoT.Value);
            FrmGrafico.Tfin = Convert.ToDouble((double)numMaximoT.Value);
            FrmGrafico.Tavance = Convert.ToDouble((double)numAvanceT.Value);

            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;

```

```

        FrmGrafico.Refresh();
    }

    private void numMinimoX_ValueChanged(object sender, EventArgs e) {
        if (numMinimoX.Value >= numMaximoX.Value)
            numMinimoX.Value = numMaximoX.Value - 1;

        FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
        FrmGrafico.Refresh();
    }

    private void numMinimoY_ValueChanged(object sender, EventArgs e) {
        if (numMinimoY.Value >= numMaximoY.Value)
            numMinimoY.Value = numMaximoY.Value - 1;

        FrmGrafico.Yini = Convert.ToDouble((double)numMinimoY.Value);
        FrmGrafico.Refresh();
    }

    private void numMaximoX_ValueChanged(object sender, EventArgs e) {
        if (numMinimoX.Value >= numMaximoX.Value)
            numMaximoX.Value = numMinimoX.Value + 1;

        FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
        FrmGrafico.Refresh();
    }

    private void numMaximoY_ValueChanged(object sender, EventArgs e) {
        if (numMinimoY.Value >= numMaximoY.Value)
            numMaximoY.Value = numMinimoY.Value + 1;

        FrmGrafico.Yfin = Convert.ToDouble((double)numMaximoY.Value);
        FrmGrafico.Refresh();
    }

    private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
        FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
        FrmGrafico.Refresh();
    }

    private void btnProcesar_Click(object sender, EventArgs e) {
        FrmGrafico.Ecuacion = txtEcuacion.Text;
        FrmGrafico.Refresh();
    }

    private void numMinimoT_ValueChanged(object sender, EventArgs e) {
        if (numMinimoY.Value >= numMaximoY.Value)
            numMinimoY.Value = numMaximoY.Value - 1;

        FrmGrafico.Tini = Convert.ToDouble((double)numMinimoT.Value);
        FrmGrafico.valorT = FrmGrafico.Tini + 0.01;
        FrmGrafico.Refresh();
    }

    private void numMaximoT_ValueChanged(object sender, EventArgs e) {
        if (numMinimoY.Value >= numMaximoY.Value)
            numMaximoY.Value = numMinimoY.Value + 1;

        FrmGrafico.Tfin = Convert.ToDouble((double)numMaximoT.Value);
        FrmGrafico.valorT = FrmGrafico.Tini + 0.01;
        FrmGrafico.Refresh();
    }

    private void numAvanceT_ValueChanged(object sender, EventArgs e) {
        FrmGrafico.Tavance = Convert.ToDouble((double)numAvanceT.Value);
        FrmGrafico.Refresh();
    }
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira

```

```

Objeto3D Figura3D;

//Giro de figura
public double AnguloX, AnguloY, AnguloZ;

//Distancia del observador
public double ZPersona;

//Rango de valores
public double Xini, Yini, Xfin, Yfin, Tini, Tfin, Tavance, valorT;

//Número de líneas que tendrá el gráfico
public int NumLineas;

//Ecuación
public string Ecuacion;

public Form1() {
    InitializeComponent();
    Figura3D = new Objeto3D();

    valorT = Tini+0.01;
}

private void Form1_Paint(object sender, PaintEventArgs e) {

    //Dibuja la figura 3D
    Figura3D.CalcularFigura3D(Ecuacion, Xini, Yini, Xfin, Yfin, NumLineas, valorT, AnguloX, AnguloY,
AnguloZ, ZPersona, 0, 0, this.ClientSize.Width, this.ClientSize.Height);
    Figura3D.Dibuja(e.Graphics);
}

private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
    Application.Exit();
}

private void Form1_Resize(object sender, EventArgs e) {
    Refresh();
}

private void timer1_Tick(object sender, EventArgs e) {
    valorT += Tavance;
    if (valorT >= Tfin) {
        valorT = Tfin;
        Tavance = -Tavance;
    }

    if (valorT <= Tini) {
        valorT = Tini;
        Tavance = -Tavance;
    }

    Refresh();
}
}
}

```

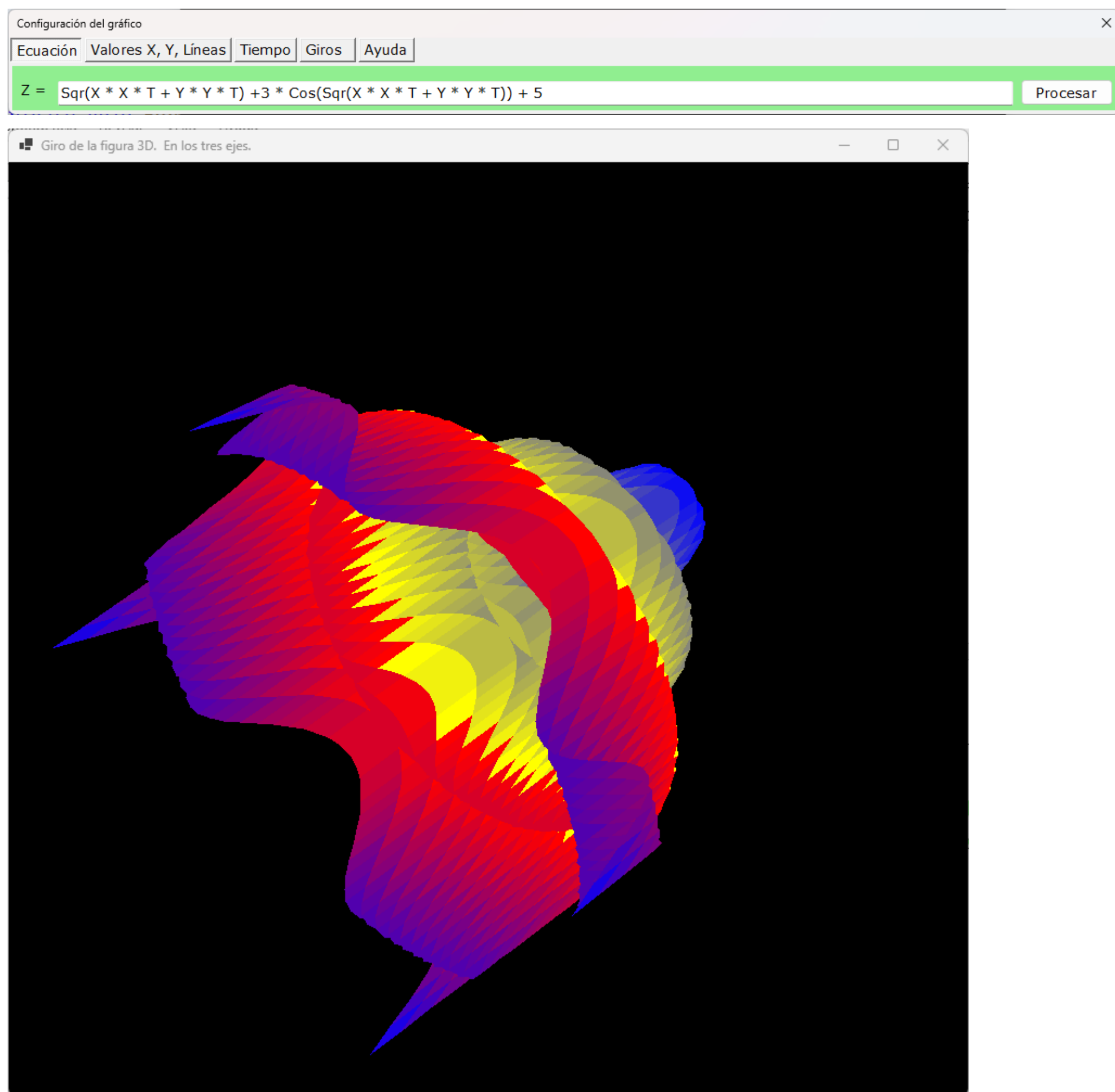


Ilustración 20: Gráfico Matemático en 4D

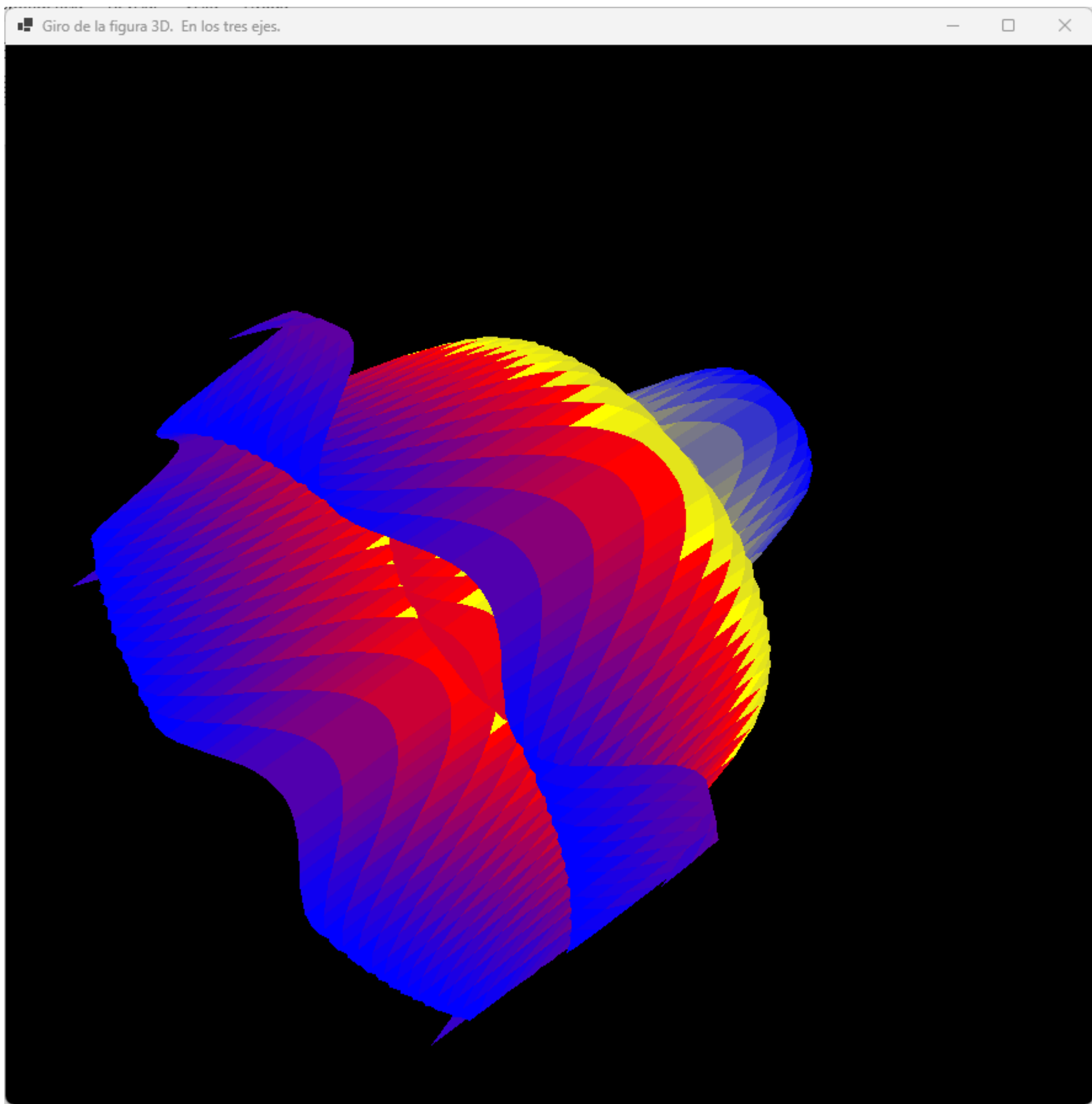


Ilustración 21: Gráfico Matemático en 4D



# Gráfico Polar en 4D

Cuando tenemos una ecuación polar del tipo  $r = F(\theta, \varphi, t)$ , tenemos una ecuación con tres variables independientes (una de ellas es el tiempo) y una variable dependiente. Su representación es en 3D animada. Un ejemplo de este tipo de ecuación:

$$r = Cos(\varphi * t) + Sen(\theta * t)$$

Los pasos para hacer el gráfico son los siguientes:

Paso 0: Saber el valor del tiempo mínimo y el valor del tiempo máximo. Se hace uso de un control “timer” que cada vez que se dispara el evento “Tick” varía el valor de T desde el mínimo hasta el máximo paso a paso, una vez alcanzado el máximo se decrementa paso a paso hasta el mínimo y repite el ciclo. De resto es igual a hacer un gráfico de ecuación tipo  $r = F(\theta, \varphi)$ .

En este código se cambia además los ángulos de giro.

N/011.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}
```

```
namespace Graficos {
    //Triángulo
    internal class Poligono {
```

```

public int punto1, punto2, punto3, punto4;
public double Centro;

//Coordenadas de dibujo del polígono
private Point Polig1, Polig2, Polig3, Polig4;
private Point[] ListaPuntos;

//Color de relleno del polígono
Color ColorZ;

public Poligono(int punto1, int punto2, int punto3, int punto4) {
    this.punto1 = punto1;
    this.punto2 = punto2;
    this.punto3 = punto3;
    this.punto4 = punto4;
}

//Calcula la profundidad y crea los vértices del polígono
public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
    double Z1 = puntos[punto1].Zgiro;
    double Z2 = puntos[punto2].Zgiro;
    double Z3 = puntos[punto3].Zgiro;
    double Z4 = puntos[punto4].Zgiro;
    Centro = (Z1 + Z2 + Z3 + Z4) / 4;

    Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
    Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
    Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
    Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
    ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

    int TotalColores = colorList.Count;
    double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2)
/ 4;

    int ColorEscoge = (int)Math.Floor(TotalColores * PromedioZ);
    ColorZ = colorList[ColorEscoge];
}

//Hace el gráfico del polígono
public void Dibuja(Graphics Lienzo) {
    //Dibuja el polígono relleno
    Brush Relleno2 = new SolidBrush(ColorZ);
    Lienzo.FillPolygon(Relleno2, ListaPuntos);
}
}
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Colores para pintar la malla
        List<Color> ListaColores;

        //Ecuación
        string EcuacionAnterior = "";
        Evaluador4 Evaluador = new();

        public Objeto3D() {
            int NumColores = 40; // Número de colores a generar

            //Genera listado de colores para el gráfico
            ListaColores = [];
            int Mitad = NumColores / 2;

            // Gradiente de azul a amarillo
            for (int Cont = 0; Cont < Mitad; Cont++) {
                int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
                int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
                int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
            }
        }
    }
}

```



```

        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }

    // Gradiente de rojo a azul
    for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
        int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
        int Verde = 0;
        int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
        ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
    }
}

//Hace los cálculos de la ecuación Z = F(X,Y)
//para dibujarla
public void CalcularFigura3D(string Ecuacion, int NumLineas, double angX, double angY, double angZ,
double valorT, double ZPersona, int XpantallaIni, int YpantallaIni, int XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double MinTheta = 0;
    double MinPhi = 0;
    double MaxTheta = 360;
    double MaxPhi = 360;
    double IncrTheta = (MaxTheta - MinTheta) / (NumLineas - 1);
    double IncrPhi = (MaxPhi - MinPhi) / (NumLineas - 1);
    double Theta = MinTheta, Phi = MinPhi;

    //Valida que la ecuación tenga valores para graficar y de paso calcula
    //las constantes para normalizar
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;
    for (int AngTheta = 1; AngTheta <= NumLineas; AngTheta++) {
        for (int AngPhi = 1; AngPhi <= NumLineas; AngPhi++) {
            double ThetaR = Theta * Math.PI / 180;
            double PhiR = Phi * Math.PI / 180;

            Evaluador.DarValorVariable('p', PhiR);
            Evaluador.DarValorVariable('h', ThetaR);
            Evaluador.DarValorVariable('t', valorT);
            double R = Evaluador.Evaluar();
            if (double.IsNaN(R) || double.IsInfinity(R)) R = 0;

            double X = R * Math.Cos(PhiR) * Math.Sin(ThetaR);
            double Y = R * Math.Sin(PhiR) * Math.Sin(ThetaR);
            double Z = R * Math.Cos(ThetaR);

            if (X < Xmin) Xmin = X;
            if (Y < Ymin) Ymin = Y;
            if (Z < Zmin) Zmin = Z;
            if (X > Xmax) Xmax = X;
            if (Y > Ymax) Ymax = Y;
            if (Z > Zmax) Zmax = Z;

            Theta += IncrTheta;
        }
        Theta = MinTheta;
        Phi += IncrPhi;
    }

    if (Math.Abs(Xmin-Xmax) < 0.0001 || Math.Abs(Ymin-Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {

```

```

        MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    //Coordenadas espaciales X,Y,Z
    Theta = MinTheta;
    Phi = MinPhi;
    puntos = [];
    for (int AngTheta = 1; AngTheta <= NumLineas; AngTheta++) {
        for (int AngPhi = 1; AngPhi <= NumLineas; AngPhi++) {
            double ThetaR = Theta * Math.PI / 180;
            double PhiR = Phi * Math.PI / 180;

            Evaluador.DarValorVariable('p', PhiR);
            Evaluador.DarValorVariable('h', ThetaR);
            Evaluador.DarValorVariable('t', valorT);
            double R = Evaluador.Evaluar();
            if (double.IsNaN(R) || double.IsInfinity(R)) R = 0;

            double X = R * Math.Cos(PhiR) * Math.Sin(ThetaR);
            double Y = R * Math.Sin(PhiR) * Math.Sin(ThetaR);
            double Z = R * Math.Cos(ThetaR);

            puntos.Add(new Punto(X, Y, Z));
            Theta += IncrTheta;
        }
        Theta = MinTheta;
        Phi += IncrPhi;
    }

    //Normaliza
    for (int Cont = 0; Cont < puntos.Count; Cont++) {
        puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
        puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
        puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
    }

    // Inicializa la lista de polígonos
    poligonos = [];

    int coordenadaActual = 0;
    int filaActual = 1;
    int totalPoligonos = (NumLineas - 1) * (NumLineas - 1);

    for (int Cont = 0; Cont < totalPoligonos; Cont++) {
        // Crea un polígono con las coordenadas de los vértices
        poligonos.Add(new Poligono(
            coordenadaActual,
            coordenadaActual + 1,
            coordenadaActual + NumLineas + 1,
            coordenadaActual + NumLineas
        ));

        coordenadaActual++;

        // Salta al inicio de la siguiente fila si se alcanza el final de la actual
        if (coordenadaActual == NumLineas * filaActual - 1) {
            coordenadaActual++;
            filaActual++;
        }
    }

    //Para la matriz de rotación
    double CosX = Math.Cos(angX);
    double SinX = Math.Sin(angX);
    double CosY = Math.Cos(angY);
    double SinY = Math.Sin(angY);
    double CosZ = Math.Cos(angZ);
    double SinZ = Math.Sin(angZ);

    //Matriz de Rotación
    //https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
    double[,] Matriz = new double[3, 3] {
        { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},

```

```

        { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
        {-SinY, SinX * CosY, CosX * CosY }
    };

    //Las constantes de transformación para cuadrar en pantalla
    double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
    double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

    //Gira los 8 puntos
    for (int cont = 0; cont < puntos.Count; cont++)
        puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

    //Calcula la profundidad y forma el polígono
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

    //Algoritmo de pintor.
    //Ordena del polígono más alejado al más cercano,
    //los polígonos de adelante son visibles y los de atrás son borrados.
    poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {
    for (int Cont = 0; Cont < poligonos.Count; Cont++)
        poligonos[Cont].Dibuja(lienzo);
}
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Tini = Convert.ToDouble((double)numMinimoT.Value);
            FrmGrafico.Tfin = Convert.ToDouble((double)numMaximoT.Value);
            FrmGrafico.Tavance = Convert.ToDouble((double)numAvanceT.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;
            FrmGrafico.HuboCambio = true;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.HuboCambio = true;
            FrmGrafico.Refresh();
        }

        private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.HuboCambio = true;
        }
    }
}

```

```

        FrmGrafico.Refresh();
    }

    private void btnProcesar_Click(object sender, EventArgs e) {
        FrmGrafico.Ecuacion = txtEcuacion.Text;
        FrmGrafico.HuboCambio = true;
        FrmGrafico.Refresh();
    }
}
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Giro de figura
        public double AnguloX, AnguloY, AnguloZ, Tini, Tfin, Tavance, valorT;

        //Distancia del observador
        public double ZPersona;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        //En caso de que el usuario haya hecho algún cambio a los valores
        //se recalcula todo de nuevo.
        public bool HuboCambio;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();

            valorT = Tini + 0.01;
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Figura3D.CalcularFigura3D(Ecuacion, NumLineas, AnguloX, AnguloY, AnguloZ, valorT, ZPersona, 0, 0,
this.ClientSize.Width, this.ClientSize.Height);
            Figura3D.Dibuja(e.Graphics);
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
            Application.Exit();
        }

        private void Form1_Resize(object sender, EventArgs e) {
            Refresh();
        }

        private void timer1_Tick(object sender, EventArgs e) {
            valorT += Tavance;
            if (valorT >= Tfin) {
                valorT = Tfin;
                Tavance = -Tavance;
            }

            if (valorT <= Tini) {
                valorT = Tini;
                Tavance = -Tavance;
            }

            Refresh();
        }
    }
}

```

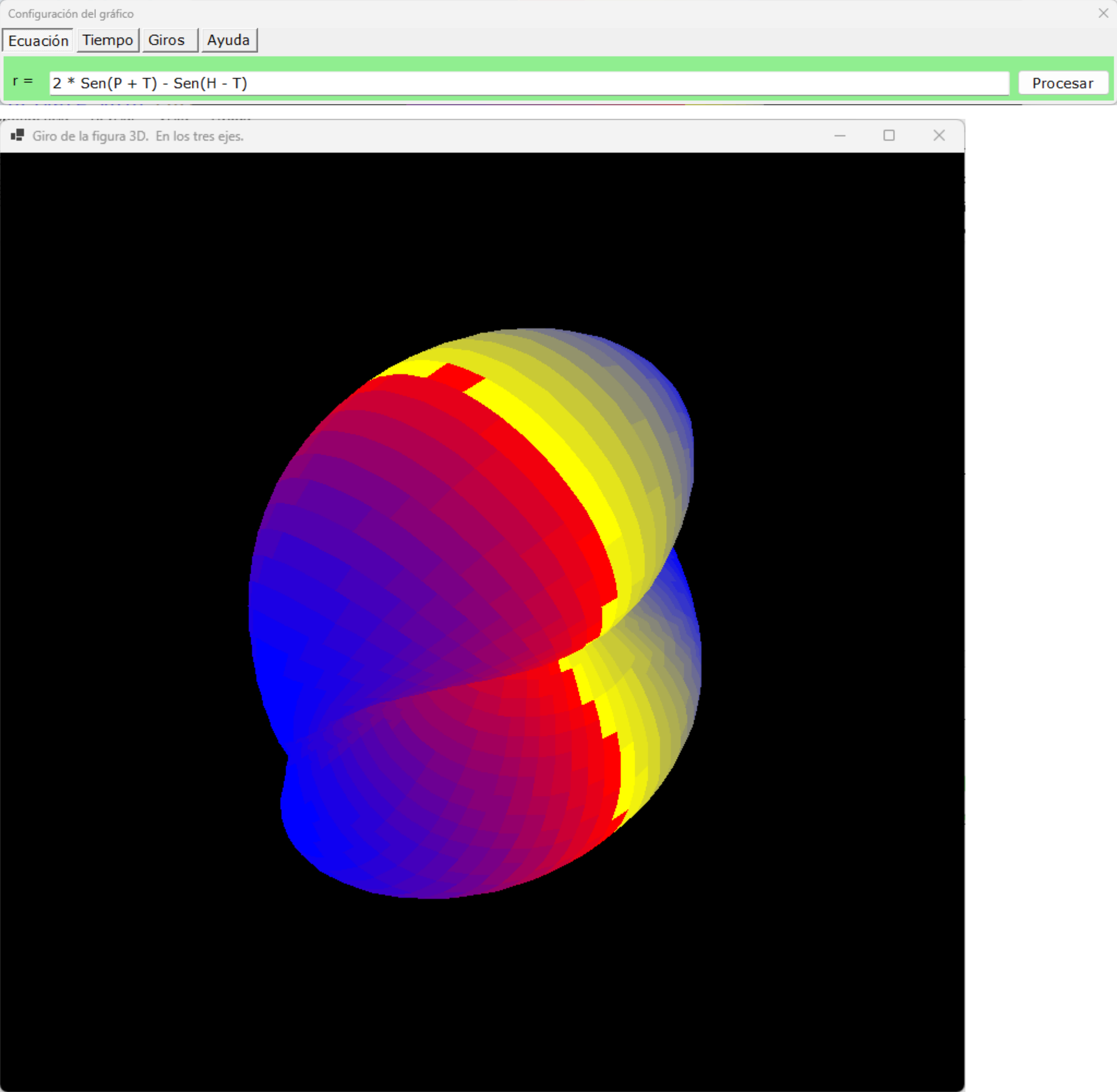


Ilustración 22: Gráfico polar 4D

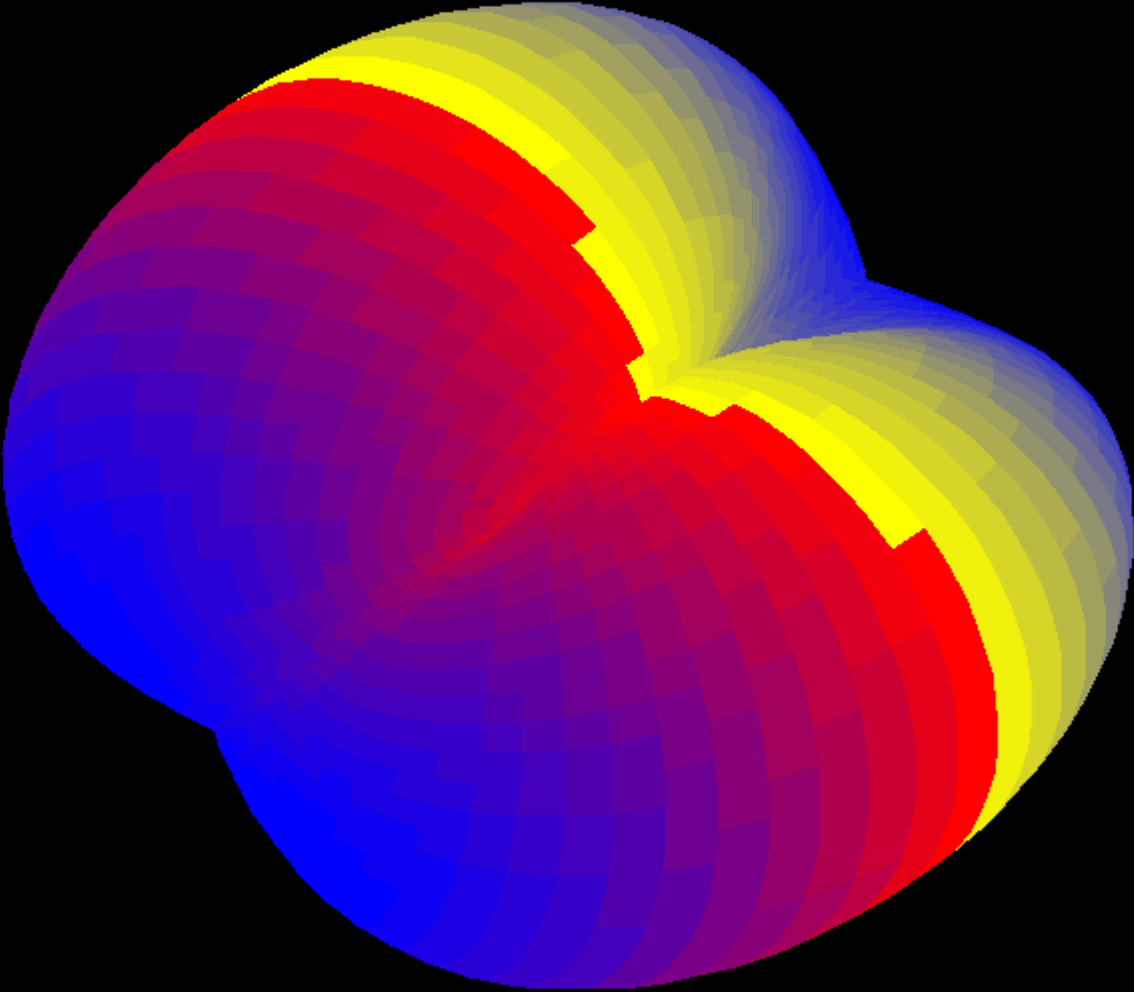


Ilustración 23: Gráfico polar 4D



# Sólido de revolución animado

Una ecuación del tipo  $Y = F(X,t)$  donde t es el tiempo. Y similar al anterior, con esa ecuación al girarla en el eje X se obtiene el sólido de revolución. Por cada t se cambia el gráfico. Un control timer se encarga de cambiar t y luego evaluar toda la ecuación con ese valor de t en particular, generar el sólido de revolución y mostrarlo en pantalla.

N/012.zip

```
namespace Graficos {
    //Cada punto espacial es almacenado y convertido
    internal class Punto {
        public double X, Y, Z; //Coordenadas originales
        public double Zgiro; //Al girar los puntos
        public int Xpantalla, Ypantalla; //Puntos en pantalla

        public Punto(double X, double Y, double Z) {
            this.X = X;
            this.Y = Y;
            this.Z = Z;
        }

        //Giro en los tres ejes
        public void Giro(double[,] Matriz, double ZPersona, int XpantallaIni, int YpantallaIni, double
convierteX, double convierteY) {

            //Hace el giro
            double Xgiro = X * Matriz[0, 0] + Y * Matriz[1, 0] + Z * Matriz[2, 0];
            double Ygiro = X * Matriz[0, 1] + Y * Matriz[1, 1] + Z * Matriz[2, 1];
            Zgiro = X * Matriz[0, 2] + Y * Matriz[1, 2] + Z * Matriz[2, 2];

            //Convierte de 3D a 2D (segunda dimensión)
            double PlanoX = Xgiro * ZPersona / (ZPersona - Zgiro);
            double PlanoY = Ygiro * ZPersona / (ZPersona - Zgiro);

            //Deduce las coordenadas de pantalla
            Xpantalla = Convert.ToInt32(convierteX * (PlanoX + 0.879315437691778) + XpantallaIni);
            Ypantalla = Convert.ToInt32(convierteY * (PlanoY + 0.879315437691778) + YpantallaIni);
        }
    }
}
```

```
namespace Graficos {
    //Conecta con una línea recta un punto con otro
    internal class Conexion {
        public int punto1, punto2;

        public Conexion(int punto1, int punto2) {
            this.punto1 = punto1;
            this.punto2 = punto2;
        }
    }
}
```

```
namespace Graficos {
    //Triángulo
    internal class Poligono {
        public int punto1, punto2, punto3, punto4;
        public double Centro;

        //Coordenadas de dibujo del polígono
        private Point Polig1, Polig2, Polig3, Polig4;
        private Point[] ListaPuntos;

        //Color de relleno del polígono
        Color ColorZ;

        public Poligono(int punto1, int punto2, int punto3, int punto4) {
            this.punto1 = punto1;
            this.punto2 = punto2;
            this.punto3 = punto3;
            this.punto4 = punto4;
        }
    }
}
```

```

    }

    //Calcula la profundidad y crea los vértices del polígono
    public void ProfundidadFigura(List<Punto> puntos, List<Color> colorList) {
        double Z1 = puntos[punto1].Zgiro;
        double Z2 = puntos[punto2].Zgiro;
        double Z3 = puntos[punto3].Zgiro;
        double Z4 = puntos[punto4].Zgiro;
        Centro = (Z1 + Z2 + Z3 + Z4) / 4;

        Polig1 = new(puntos[punto1].Xpantalla, puntos[punto1].Ypantalla);
        Polig2 = new(puntos[punto2].Xpantalla, puntos[punto2].Ypantalla);
        Polig3 = new(puntos[punto3].Xpantalla, puntos[punto3].Ypantalla);
        Polig4 = new(puntos[punto4].Xpantalla, puntos[punto4].Ypantalla);
        ListaPuntos = [Polig1, Polig2, Polig3, Polig4];

        int TotalColores = colorList.Count;
        double PromedioZ = (puntos[punto1].Z + puntos[punto2].Z + puntos[punto3].Z + puntos[punto4].Z + 2)
/ 4;

        int ColorEscoge = (int)Math.Floor((TotalColores-1) * PromedioZ);
        ColorZ = colorList[ColorEscoge];
    }

    //Hace el gráfico del polígono
    public void Dibuja(Graphics Lienzo) {
        //Dibuja el polígono relleno
        Brush Relleno2 = new SolidBrush(ColorZ);
        Lienzo.FillPolygon(Relleno2, ListaPuntos);
    }
}
}

```

```

namespace Graficos {
    internal class Objeto3D {
        //Coordenadas espaciales X, Y, Z
        private List<Punto> puntos;

        //Coordenadas del polígono (triángulo)
        private List<Poligono> poligonos;

        //Colores para pintar la malla
        List<Color> ListaColores;

        //Ecuación
        string EcuacionAnterior = "";
        Evaluador4 Evaluador = new();

        public Objeto3D() {
            int NumColores = 40; // Número de colores a generar

            //Genera listado de colores para el gráfico
            ListaColores = [];
            int Mitad = NumColores / 2;

            // Gradiente de azul a amarillo
            for (int Cont = 0; Cont < Mitad; Cont++) {
                int Rojo = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
                int Verde = (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
                int Azul = 255 - (int)(255 * (Cont / (float)(NumColores - Mitad - 1)));
                ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
            }

            // Gradiente de rojo a azul
            for (int Cont = 0; Cont < NumColores - Mitad; Cont++) {
                int Rojo = 255 - (int)(255 * (Cont / (float)(Mitad - 1)));
                int Verde = 0;
                int Azul = (int)(255 * (Cont / (float)(Mitad - 1)));
                ListaColores.Add(Color.FromArgb(Rojo, Verde, Azul));
            }
        }

        //Hace los cálculos de la ecuación Z = F(X,Y)
        //para dibujarla
    }
}

```



```

    public void CalcularFigura3D(string Ecuacion, double Xini, double Xfin, int NumLineas, double valorT,
double angX, double angY, double angZ, double ZPersona, int XpantallaIni, int YpantallaIni, int
XpantallaFin, int YpantallaFin) {

    //Evalúa la ecuación. Si es nueva, la analiza.
    if (Ecuacion.Equals(EcuacionAnterior) == false) {
        int Sintaxis = Evaluador.Analizar(Ecuacion);
        if (Sintaxis > 0) { //Tiene un error de sintaxis
            string MensajeError = Evaluador.MensajeError(Sintaxis);
            MessageBox.Show(MensajeError, "Error de sintaxis",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    EcuacionAnterior = Ecuacion;

    //Incrementos X, Y
    double IncrX = (Xfin - Xini) / (NumLineas - 1);
    double IncAng = (double) 360 / NumLineas;
    double X = Xini;

    //Valida que la ecuación tenga valores para graficar y de paso calcula
    //las constantes para normalizar
    double Xmin = double.MaxValue;
    double Ymin = double.MaxValue;
    double Zmin = double.MaxValue;
    double Xmax = double.MinValue;
    double Ymax = double.MinValue;
    double Zmax = double.MinValue;

    double AnguloGiro = 0;
    for (int GiroLinea = 1; GiroLinea <= NumLineas + 1; GiroLinea++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            Evaluador.DarValorVariable('t', valorT);
            double Y = Evaluador.Evaluar();
            if (double.IsNaN(Y) || double.IsInfinity(Y)) Y = 0;

            double Yg = Y * Math.Cos(AnguloGiro * Math.PI / 180);
            double Zg = Y * Math.Sin(AnguloGiro * Math.PI / 180);

            if (X < Xmin) Xmin = X;
            if (Yg < Ymin) Ymin = Yg;
            if (Zg < Zmin) Zmin = Zg;

            if (X > Xmax) Xmax = X;
            if (Yg > Ymax) Ymax = Yg;
            if (Zg > Zmax) Zmax = Zg;

            X += IncrX;
        }
        AnguloGiro += IncAng;
        X = Xini;
    }

    if ( Math.Abs(Xmin-Xmax) < 0.0001 || Math.Abs(Ymin-Ymax) < 0.0001 || Math.Abs(Zmin - Zmax) <
0.0001) {
        MessageBox.Show("La ecuación digitada no puede generar un gráfico", "Error de cálculo",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    //Coordenadas espaciales X,Y,Z
    X = Xini;
    puntos = [];
    AnguloGiro = 0;
    for (int GiroLinea = 1; GiroLinea <= NumLineas+1; GiroLinea++) {
        for (int EjeX = 1; EjeX <= NumLineas; EjeX++) {
            Evaluador.DarValorVariable('x', X);
            Evaluador.DarValorVariable('t', valorT);
            double Y = Evaluador.Evaluar();
            if (double.IsNaN(Y) || double.IsInfinity(Y)) Y = 0;

            double Yg = Y * Math.Cos(AnguloGiro * Math.PI / 180);
            double Zg = Y * Math.Sin(AnguloGiro * Math.PI / 180);

```

```

        puntos.Add(new Punto(X, Yg, Zg));
        X += IncrX;
    }
    AnguloGiro += IncAng;
    X = Xini;
}

//Normaliza
for (int Cont = 0; Cont < puntos.Count; Cont++) {
    puntos[Cont].X = (puntos[Cont].X - Xmin) / (Xmax - Xmin) - 0.5;
    puntos[Cont].Y = (puntos[Cont].Y - Ymin) / (Ymax - Ymin) - 0.5;
    puntos[Cont].Z = (puntos[Cont].Z - Zmin) / (Zmax - Zmin) - 0.5;
}

// Inicializa la lista de polígonos
poligonos = [];

int coordenadaActual = 0;
int filaActual = 1;
int totalPoligonos = (NumLineas - 1) * (NumLineas - 1) + (NumLineas - 1);

for (int Cont = 0; Cont < totalPoligonos; Cont++) {
    // Crea un polígono con las coordenadas de los vértices
    poligonos.Add(new Poligono(
        coordenadaActual,
        coordenadaActual + 1,
        coordenadaActual + NumLineas + 1,
        coordenadaActual + NumLineas
    ));

    coordenadaActual++;

    // Salta al inicio de la siguiente fila si se alcanza el final de la actual
    if (coordenadaActual == NumLineas * filaActual - 1) {
        coordenadaActual++;
        filaActual++;
    }
}

//Para la matriz de rotación
double CosX = Math.Cos(angX);
double SinX = Math.Sin(angX);
double CosY = Math.Cos(angY);
double SinY = Math.Sin(angY);
double CosZ = Math.Cos(angZ);
double SinZ = Math.Sin(angZ);

//Matriz de Rotación
//https://en.wikipedia.org/wiki/Rotation_formalisms_in_three_dimensions
double[,] Matriz = new double[3, 3] {
    { CosY * CosZ, -CosX * SinZ + SinX * SinY * CosZ, SinX * SinZ + CosX * SinY * CosZ},
    { CosY * SinZ, CosX * CosZ + SinX * SinY * SinZ, -SinX * CosZ + CosX * SinY * SinZ},
    {-SinY, SinX * CosY, CosX * CosY }
};

//Las constantes de transformación para cuadrar en pantalla
double convierteX = (XpantallaFin - XpantallaIni) / 1.758630875383556;
double convierteY = (YpantallaFin - YpantallaIni) / 1.758630875383556;

//Gira los 8 puntos
for (int cont = 0; cont < puntos.Count; cont++)
    puntos[cont].Giro(Matriz, ZPersona, XpantallaIni, YpantallaIni, convierteX, convierteY);

//Calcula la profundidad y forma el polígono
for (int Cont = 0; Cont < poligonos.Count; Cont++)
    poligonos[Cont].ProfundidadFigura(puntos, ListaColores);

//Algoritmo de pintor.
//Ordena del polígono más alejado al más cercano,
//los polígonos de adelante son visibles y los de atrás son borrados.
poligonos.Sort((p1, p2) => p1.Centro.CompareTo(p2.Centro));
}

//Dibuja la figura 3D
public void Dibuja(Graphics lienzo) {

```

```

        for (int Cont = 0; Cont < poligonos.Count; Cont++)
            poligonos[Cont].Dibuja(lienzo);
    }
}
}

```

```

namespace Graficos {
    public partial class Valores : Form {

        Form1 FrmGrafico = new();
        const double Radianes = Math.PI / 180;
        public Valores() {
            InitializeComponent();

            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Tini = Convert.ToDouble((double)numMinimoT.Value);
            FrmGrafico.Tfin = Convert.ToDouble((double)numMaximoT.Value);
            FrmGrafico.Tavance = Convert.ToDouble((double)numAvanceT.Value);
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.ZPersona = 5;

            FrmGrafico.Show();
        }

        private void numGiroX_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloX = Convert.ToDouble(numGiroX.Value) * Radianes;
            FrmGrafico.Refresh();
        }

        private void numGiroY_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloY = Convert.ToDouble(numGiroY.Value) * Radianes;
            FrmGrafico.Refresh();
        }

        private void numGiroZ_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.AnguloZ = Convert.ToDouble(numGiroZ.Value) * Radianes;
            FrmGrafico.Refresh();
        }

        private void numMinimoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMinimoX.Value = numMaximoX.Value - 1;

            FrmGrafico.Xini = Convert.ToDouble((double)numMinimoX.Value);
            FrmGrafico.Refresh();
        }

        private void numMaximoX_ValueChanged(object sender, EventArgs e) {
            if (numMinimoX.Value >= numMaximoX.Value)
                numMaximoX.Value = numMinimoX.Value + 1;

            FrmGrafico.Xfin = Convert.ToDouble((double)numMaximoX.Value);
            FrmGrafico.Refresh();
        }

        private void numTotalLineas_ValueChanged(object sender, EventArgs e) {
            FrmGrafico.NumLineas = Convert.ToInt32((double)numTotalLineas.Value);
            FrmGrafico.Refresh();
        }

        private void btnProcesar_Click(object sender, EventArgs e) {
            FrmGrafico.Ecuacion = txtEcuacion.Text;
            FrmGrafico.Refresh();
        }
    }
}

```

```

namespace Graficos {
    //Proyección 3D a 2D y giros. Figura centrada. Optimización.
    public partial class Form1 : Form {
        //La figura que se proyecta y gira
        Objeto3D Figura3D;

        //Giro de figura
        public double AnguloX, AnguloY, AnguloZ;

        //Distancia del observador
        public double ZPersona;

        //Rango de valores
        public double Xini, Yini, Xfin, Yfin, Tini, Tfin, Tavance, valorT;

        //Número de líneas que tendrá el gráfico
        public int NumLineas;

        //Ecuación
        public string Ecuacion;

        public Form1() {
            InitializeComponent();
            Figura3D = new Objeto3D();

            valorT = Tini + 0.01;
        }

        private void Form1_Paint(object sender, PaintEventArgs e) {
            Figura3D.CalcularFigura3D(Ecuacion, Xini, Xfin, NumLineas, valorT, AnguloX, AnguloY, AnguloZ,
            ZPersona, 0, 0, this.ClientSize.Width, this.ClientSize.Height);
            Figura3D.Dibuja(e.Graphics);
        }

        private void Form1_FormClosing(object sender, FormClosingEventArgs e) {
            Application.Exit();
        }

        private void Form1_Resize(object sender, EventArgs e) {
            Refresh();
        }

        private void timer1_Tick(object sender, EventArgs e) {
            valorT += Tavance;
            if (valorT >= Tfin) {
                valorT = Tfin;
                Tavance = -Tavance;
            }

            if (valorT <= Tini) {
                valorT = Tini;
                Tavance = -Tavance;
            }

            Refresh();
        }
    }
}

```

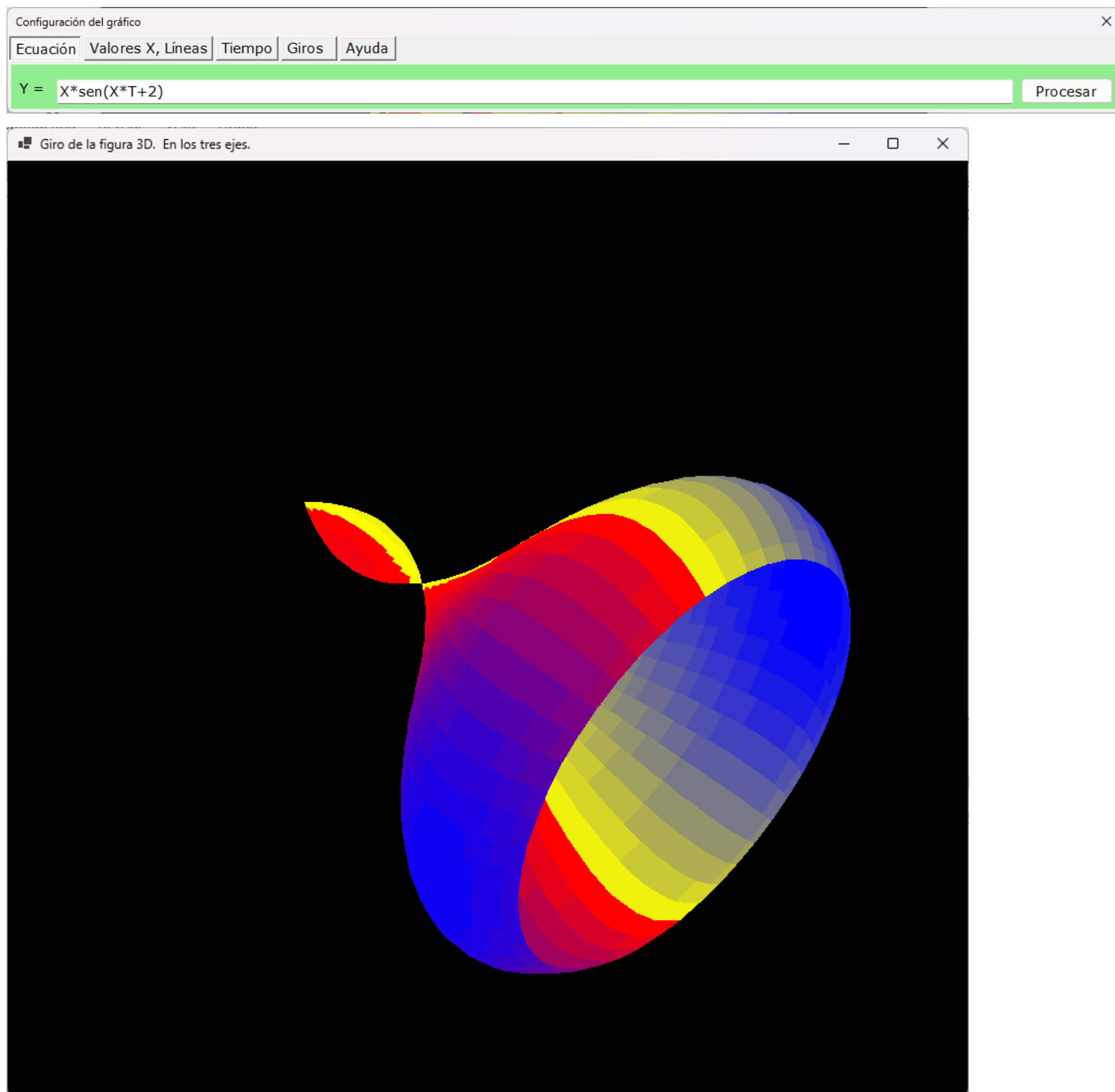


Ilustración 24: Sólido de revolución animado

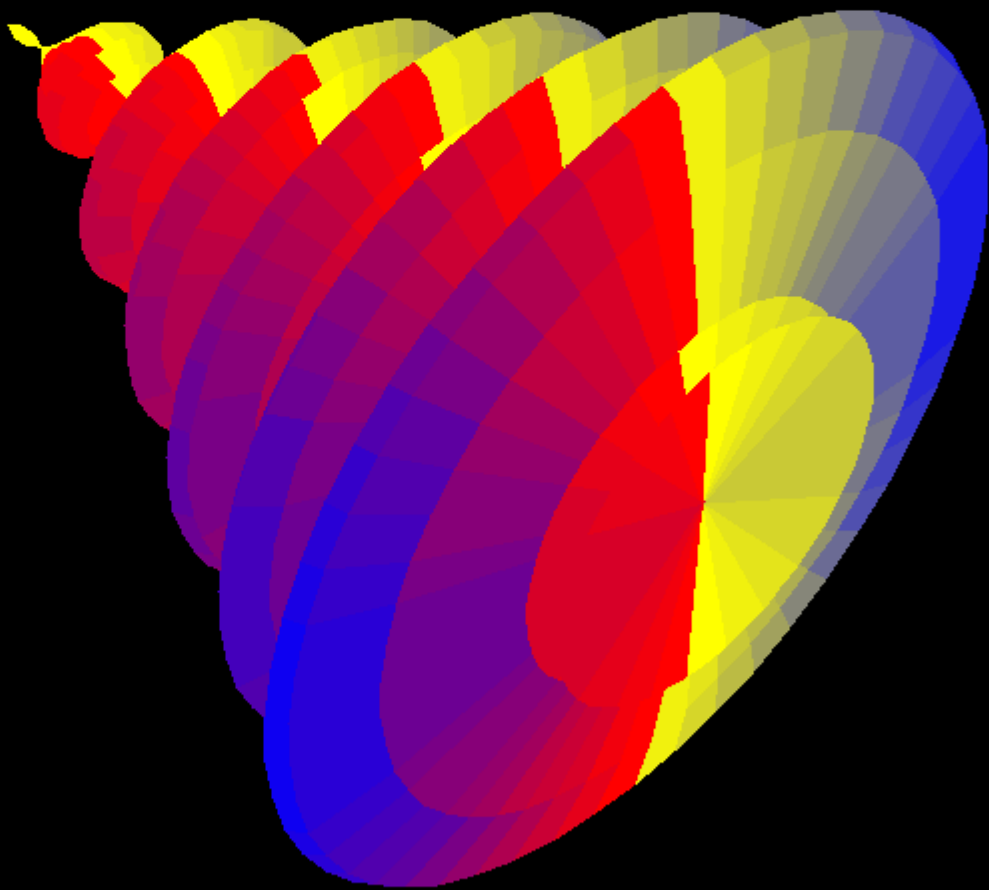
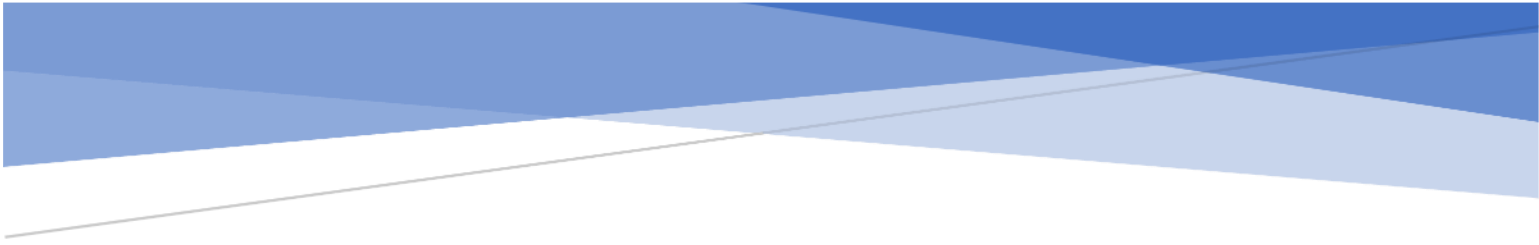


Ilustración 25: Sólido de revolución animado





# C# Y .NET 10

## Parte 15. Imágenes

2026-01

Rafael Alberto Moreno Parra  
ramsoftware@gmail.com

## Contenido

|                                                    |    |
|----------------------------------------------------|----|
| Tabla de ilustraciones.....                        | 4  |
| Acerca del autor.....                              | 6  |
| Licencia de este libro .....                       | 6  |
| Licencia del software .....                        | 6  |
| Marcas registradas .....                           | 7  |
| Dedicatoria .....                                  | 8  |
| Iniciando con imágenes.....                        | 9  |
| Convertir a escala de grises .....                 | 20 |
| Disminuir el tamaño de la imagen .....             | 21 |
| Reflejo Horizontal.....                            | 23 |
| Reflejo Vertical .....                             | 24 |
| Giro de la imagen.....                             | 25 |
| Filtro blanco y negro.....                         | 26 |
| Invierte colores.....                              | 27 |
| Filtro Kodachrome .....                            | 28 |
| Filtro Sepia .....                                 | 29 |
| Filtro Lomograph.....                              | 30 |
| Filtro Polaroid .....                              | 31 |
| Filtro umbral .....                                | 32 |
| Filtro borde .....                                 | 33 |
| Filtro pixelado.....                               | 34 |
| Filtro de saturación de color .....                | 35 |
| Filtro viñeta.....                                 | 36 |
| Cambiar pixel a pixel .....                        | 37 |
| Invertir colores .....                             | 38 |
| Aplicando un núcleo o “kernel” .....               | 40 |
| Una aplicación WPF que carga una imagen .....      | 42 |
| Una aplicación WPF que aplica varios filtros ..... | 53 |
| Información imágenes .....                         | 62 |
| Dibujando en imágenes.....                         | 63 |
| Dibujar una línea .....                            | 66 |
| Dibujar un círculo .....                           | 68 |
| Dibujar un polígono .....                          | 70 |



|                                   |    |
|-----------------------------------|----|
| Dibujar Líneas.....               | 72 |
| Dibujar Círculos .....            | 74 |
| Dibujar rectángulos al azar ..... | 76 |
| Dibujar letras.....               | 78 |
| Texto girado .....                | 80 |

## Tabla de ilustraciones

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| Ilustración 1: Inicia proyecto en Visual Studio 2026 .....                                   | 9  |
| Ilustración 2: Aplicación de consola .....                                                   | 10 |
| Ilustración 3: Ponerle un nombre al proyecto .....                                           | 11 |
| Ilustración 4: Seleccionar .NET 10 .....                                                     | 12 |
| Ilustración 5: Aplicación de consola de ejemplo .....                                        | 13 |
| Ilustración 6: Dar clic en "Herramientas" y luego en "Administrador de paquetes NuGet" ..... | 14 |
| Ilustración 7: Instalación de SixLabors.ImageSharp .....                                     | 15 |
| Ilustración 8: Se instalan las librerías en el proyecto .....                                | 16 |
| Ilustración 9: Versión de la librería .....                                                  | 17 |
| Ilustración 10: Licencia de la librería .....                                                | 18 |
| Ilustración 11: Paquete instalado .....                                                      | 19 |
| Ilustración 12: Escala de grises .....                                                       | 20 |
| Ilustración 13: Cambio de tamaño de imagen .....                                             | 22 |
| Ilustración 14: Reflejo horizontal .....                                                     | 23 |
| Ilustración 15: Reflejo vertical .....                                                       | 24 |
| Ilustración 16: Giro de la imagen .....                                                      | 25 |
| Ilustración 17: Filtro blanco y negro .....                                                  | 26 |
| Ilustración 18: Invierte colores .....                                                       | 27 |
| Ilustración 19: Filtro Kodachrome .....                                                      | 28 |
| Ilustración 20: Filtro Sepia .....                                                           | 29 |
| Ilustración 21: Filtro Lomograph .....                                                       | 30 |
| Ilustración 22: Filtro Polaroid .....                                                        | 31 |
| Ilustración 23: Filtro umbral .....                                                          | 32 |
| Ilustración 24: Filtro borde .....                                                           | 33 |
| Ilustración 25: Filtro pixelado .....                                                        | 34 |
| Ilustración 26: Filtro de saturación de color .....                                          | 35 |
| Ilustración 27: Filtro viñeta .....                                                          | 36 |
| Ilustración 28: Modificando pixel a pixel .....                                              | 37 |
| Ilustración 29: Invertir colores .....                                                       | 39 |
| Ilustración 30: Aplicando un núcleo .....                                                    | 41 |
| Ilustración 31: Aplicación WPF .....                                                         | 42 |
| Ilustración 32: Nombre del proyecto .....                                                    | 43 |
| Ilustración 33: Seleccionar .NET 10 .....                                                    | 44 |
| Ilustración 34: Aplicación WPF típica .....                                                  | 45 |
| Ilustración 35: Código en C# .....                                                           | 46 |
| Ilustración 36: Carga las librerías .....                                                    | 47 |
| Ilustración 37: Se pone el objeto "Image" en la ventana .....                                | 48 |
| Ilustración 38: Se pone nombre a ese objeto "Image" .....                                    | 49 |
| Ilustración 39: Ejecución del proyecto .....                                                 | 52 |
| Ilustración 40: Una ventana de diálogo para escoger el archivo de imagen .....               | 53 |
| Ilustración 41: Imagen cargada .....                                                         | 54 |
| Ilustración 42: Aplicando el filtro sepia .....                                              | 55 |
| Ilustración 43: Aplicando reflejo horizontal y doble vez Kodachrome .....                    | 56 |
| Ilustración 44: Información de la imagen .....                                               | 62 |
| Ilustración 45: Se agrega SixLabors.ImageSharp.Drawing .....                                 | 63 |

|                                                               |    |
|---------------------------------------------------------------|----|
| Ilustración 46: Agregar Drawing .....                         | 64 |
| Ilustración 47: Licencia de esa librería .....                | 65 |
| Ilustración 48: Dibuja una línea en imagen.....               | 67 |
| Ilustración 49: Dibuja círculo en imagen .....                | 69 |
| Ilustración 50: Dibuja un polígono dentro de una imagen ..... | 71 |
| Ilustración 51: Dibuja líneas en una imagen .....             | 73 |
| Ilustración 52: Dibuja círculos en una imagen .....           | 75 |
| Ilustración 53: Rectángulos rellenos al azar .....            | 77 |

## Acerca del autor

Rafael Alberto Moreno Parra

[ramsoftware@gmail.com](mailto:ramsoftware@gmail.com) o [enginelifelife@hotmail.com](mailto:enginelifelife@hotmail.com)

Sitio Web: <http://darwin.50webs.com> (dedicado a la investigación de algoritmos evolutivos y vida artificial).

Github: <https://github.com/ramsoftware>

Youtube: <https://www.youtube.com/@RafaelMorenoP>

## Licencia de este libro



## Licencia del software

Todo el software desarrollado aquí tiene licencia LGPL “Lesser General Public License” [1]



## Marcas registradas

En este libro se hace uso de las siguientes tecnologías registradas:

Microsoft ® Windows ® Enlace: <http://windows.microsoft.com/en-US/windows/home>

Microsoft ® Visual Studio 2026 ® Enlace: <https://visualstudio.microsoft.com/es/vs/>

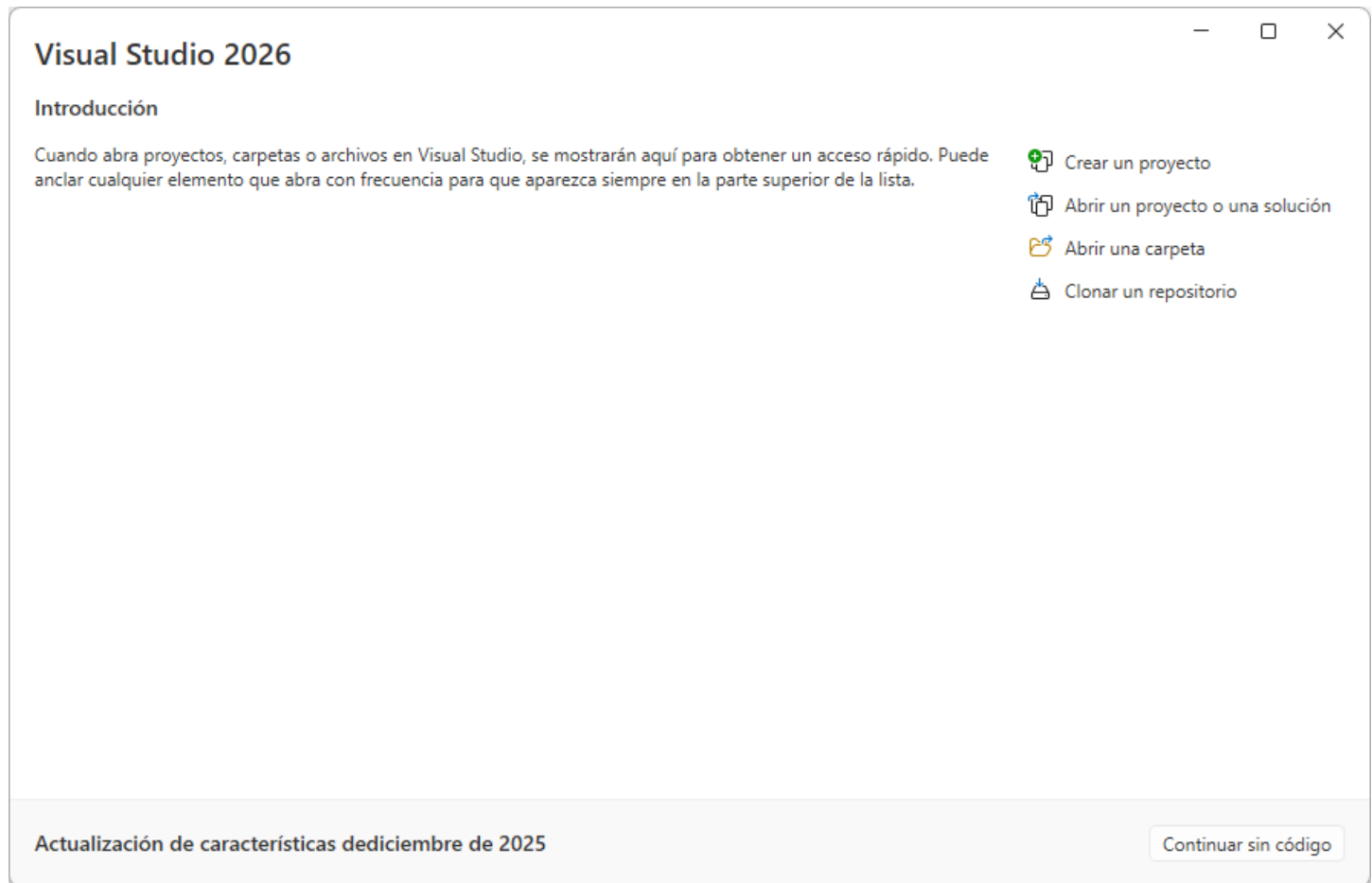
## Dedicatoria

A mis padres, a mi hermana....

Y a mi tropa gatuna: Suini, Grisú, Milú, Arián, Frac y mis recordados Sally, Capuchina, Tinita, Tammy, Vikingo y Michu.

# Iniciando con imágenes

Para trabajar con gráficos, se debe crear un proyecto de consola



*Ilustración 1: Inicia proyecto en Visual Studio 2026*

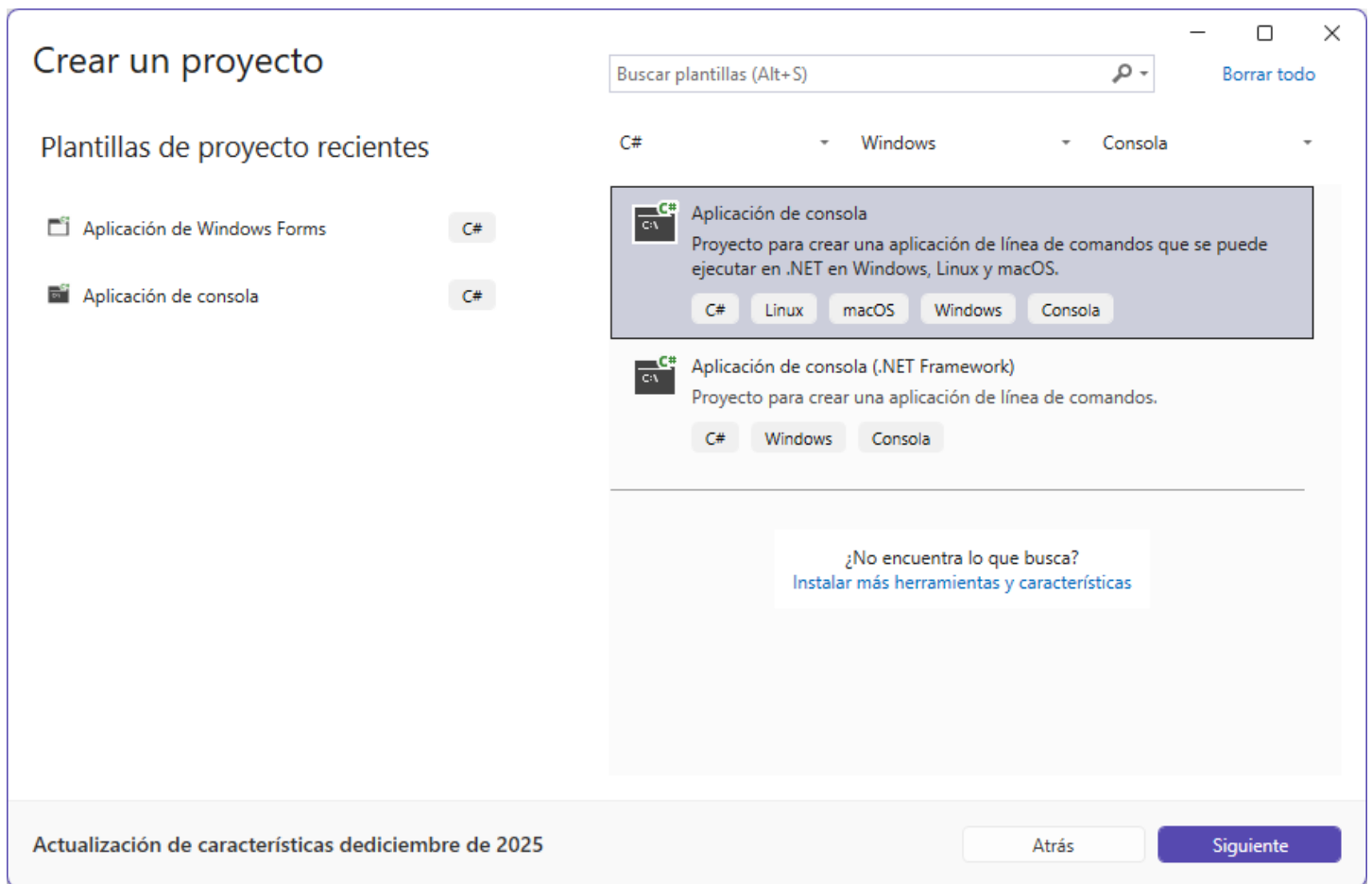


Ilustración 2: Aplicación de consola



—□×

# Configure su nuevo proyecto

Aplicación de consola

C#LinuxmacOSWindowsConsola

Nombre del proyecto

Ejemplo

Ubicación

C:\Users\engin\source\repos

...

Nombre de la solución ⓘ

Ejemplo

☐ Colocar la solución y el proyecto en el mismo directorio

Proyecto se creará en "C:\Users\engin\source\repos\Ejemplo\Ejemplo"

Actualización de características dediciembre de 2025

Atrás

Siguiente

Ilustración 3: Ponerle un nombre al proyecto

—□×

## Información adicional

Aplicación de consola

C#LinuxmacOSWindowsConsola

Framework ⓘ

.NET 10.0 (Compatibilidad a largo plazo) ▾

☒ No usar instrucciones de nivel superior ⓘ  
☐ Habilitar publicación de AOT native ⓘ

Actualización de características dediciembre de 2025

Atrás

Crear

Ilustración 4: Seleccionar .NET 10

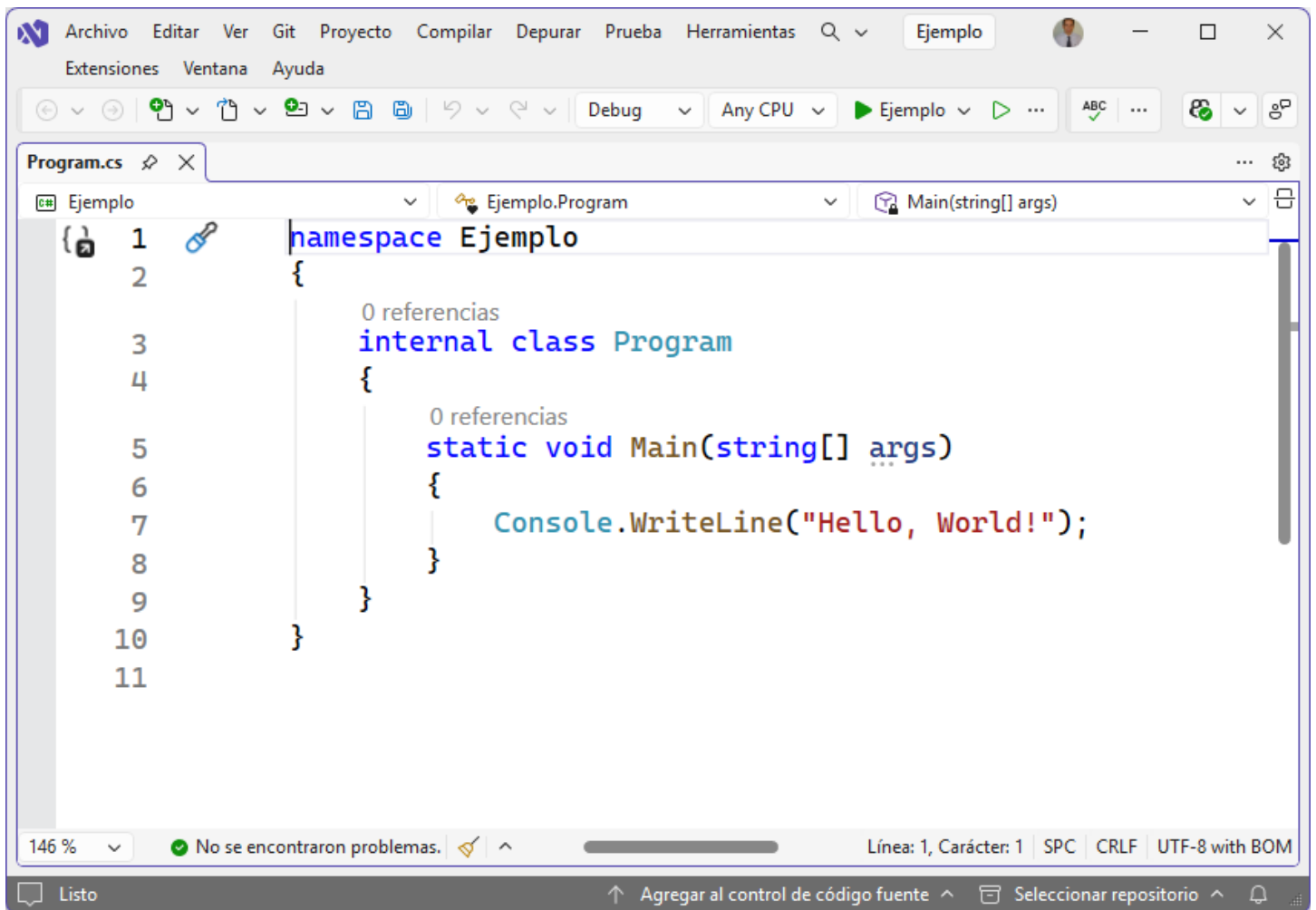


Ilustración 5: Aplicación de consola de ejemplo

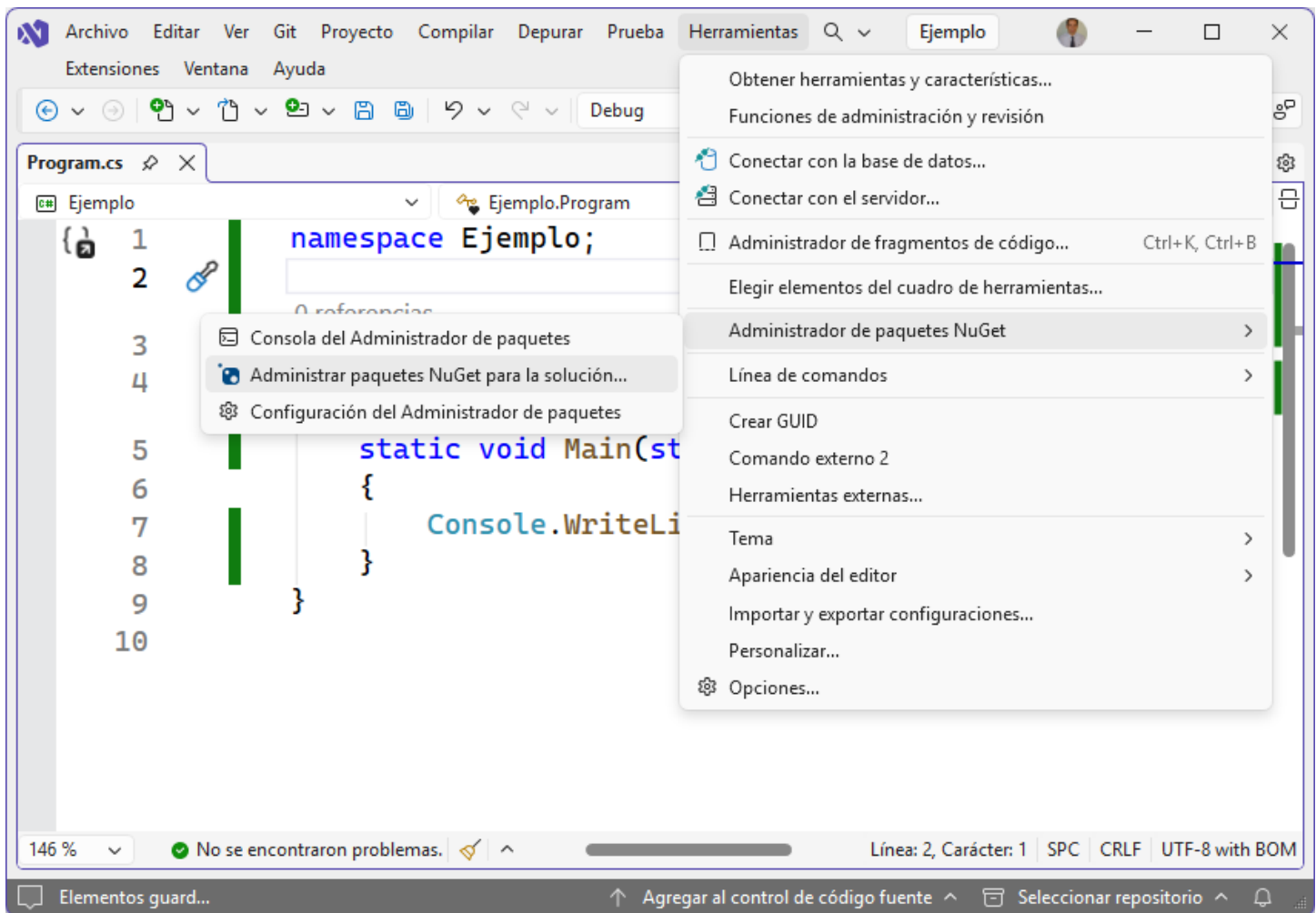


Ilustración 6: Dar clic en "Herramientas" y luego en "Administrador de paquetes NuGet"

Se requiere el paquete "SIX LABORS – ImageShark", esta es su instalación:

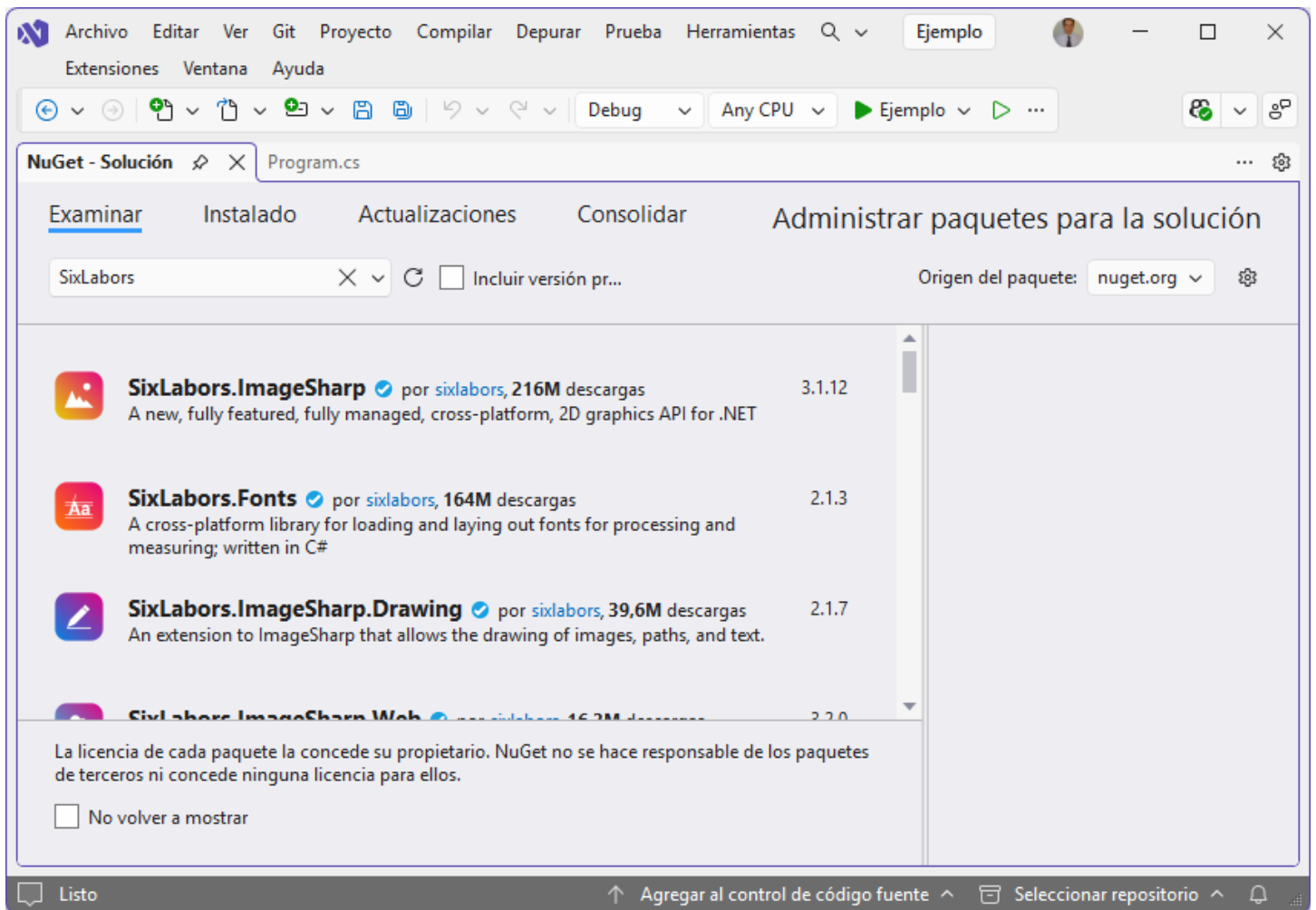


Ilustración 7: Instalación de SixLabors.ImageSharp

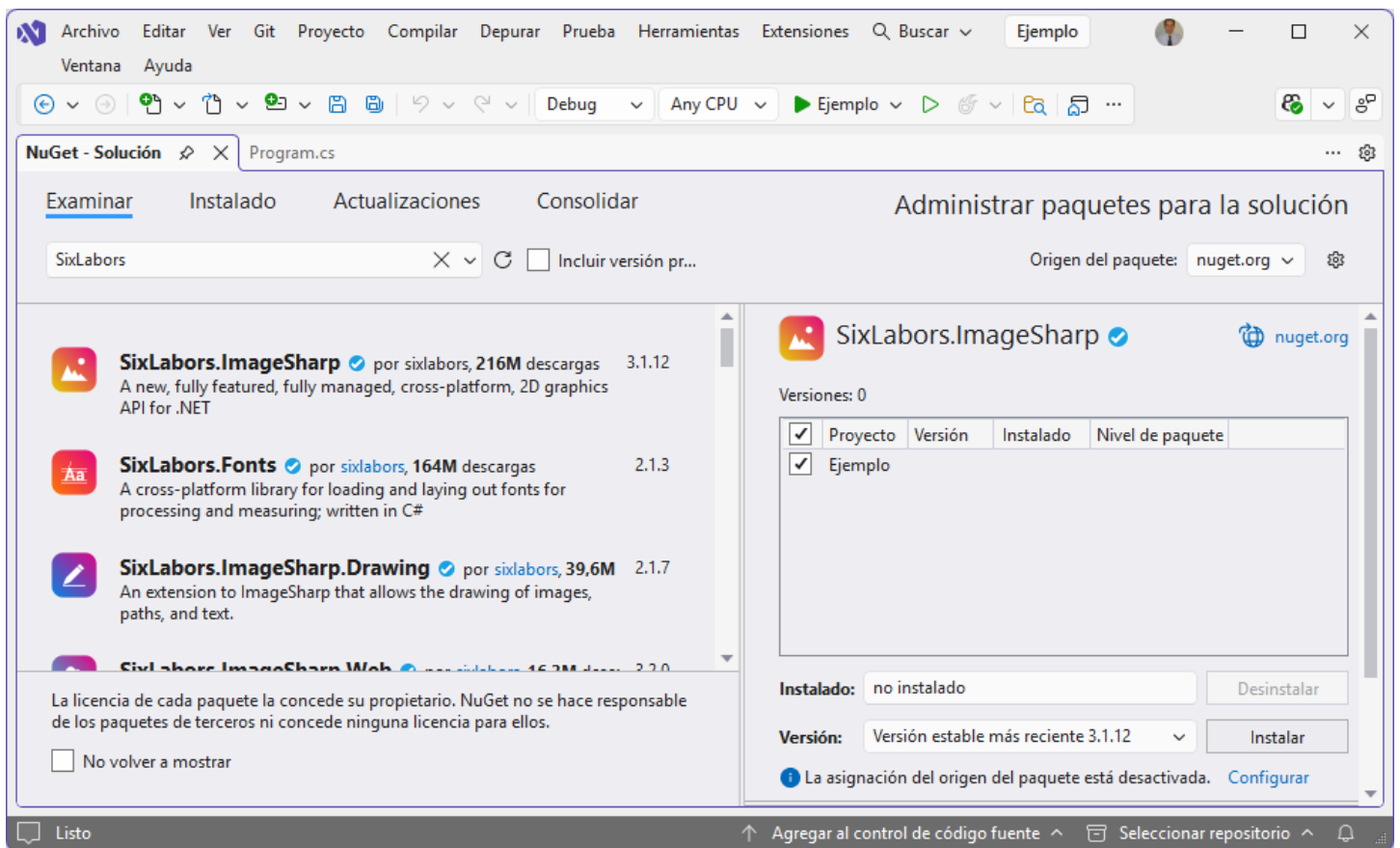
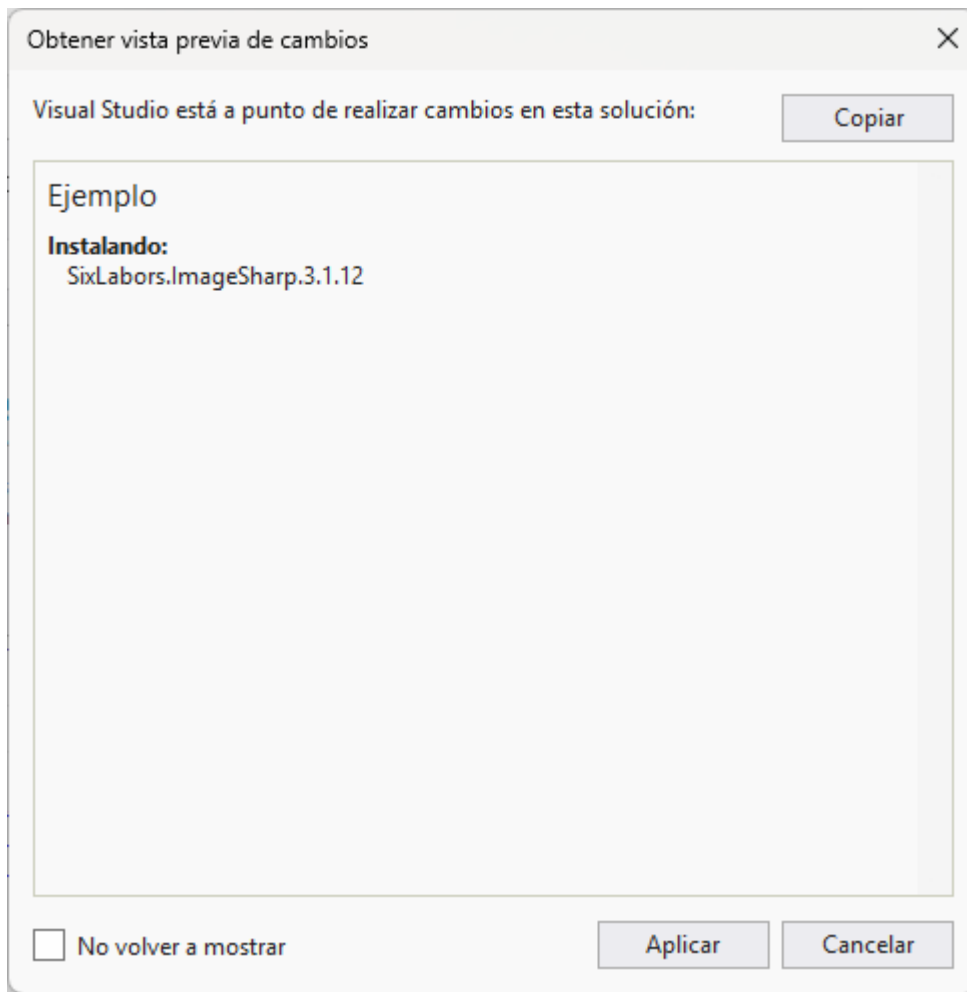
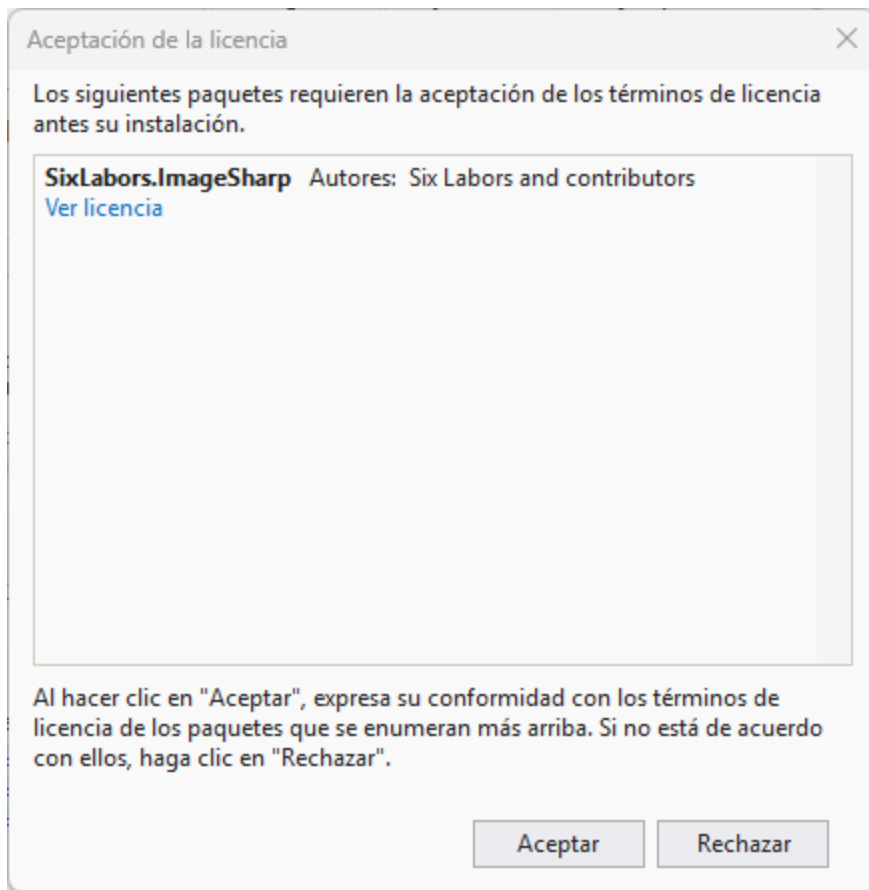


Ilustración 8: Se instalan las librerías en el proyecto



*Ilustración 9: Versión de la librería*



*Ilustración 10: Licencia de la librería*



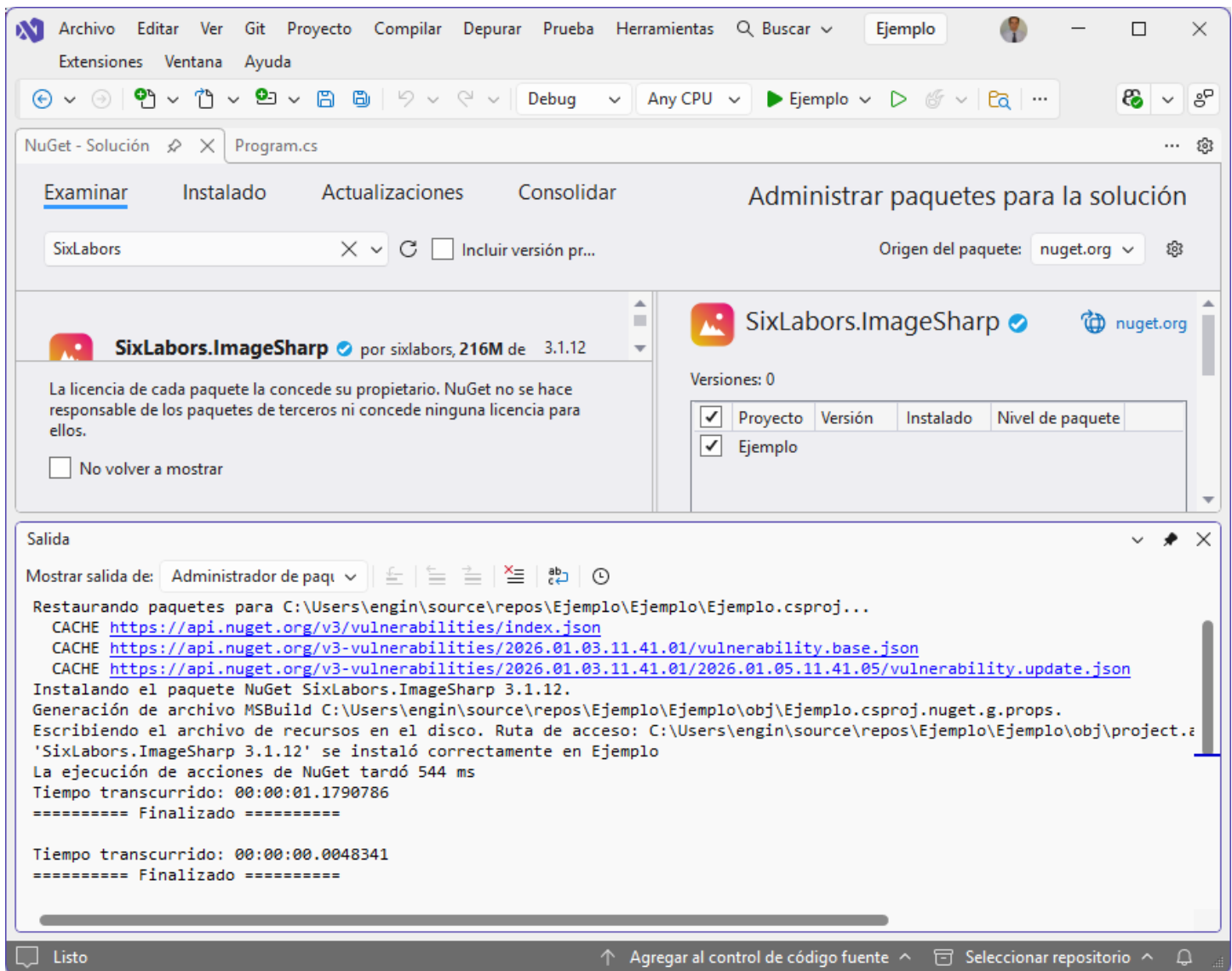


Ilustración 11: Paquete instalado

# Convertir a escala de grises

O/001.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro de escala de grises
            Foto.Mutate(x => x.Grayscale());

            //Guarda la nueva imagen de escala de grises
            string Salida = "C:\\\\TEMP\\\\GrisúEscalaGrises.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```

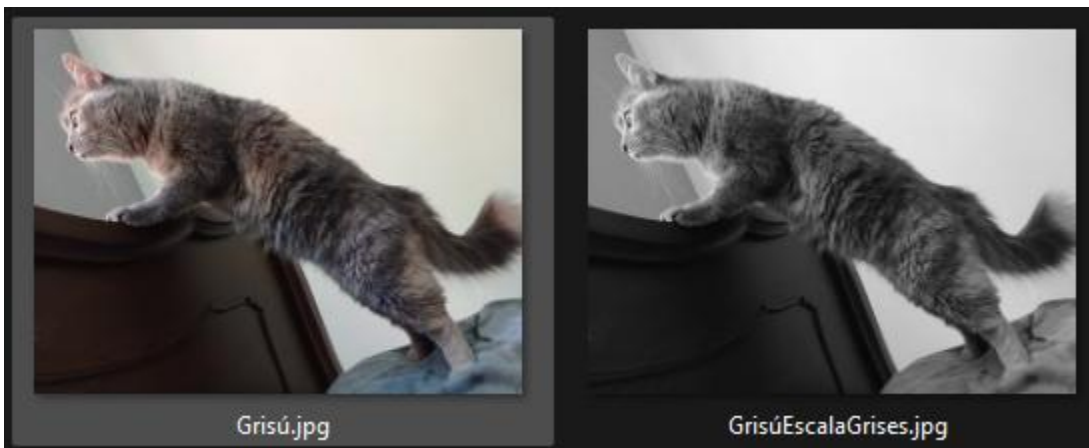


Ilustración 12: Escala de grises

## Disminuir el tamaño de la imagen

O/002.cs

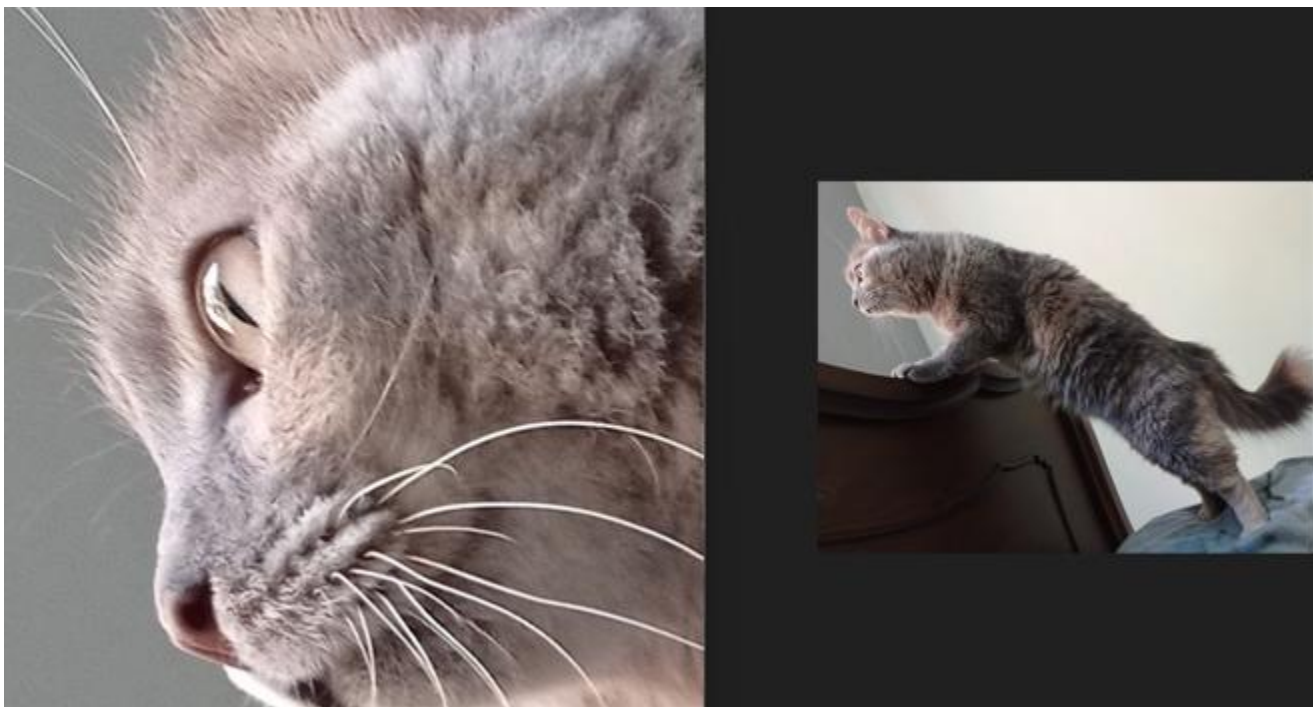
```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro para disminuir
            Foto.Mutate(x => x.Resize(Foto.Width / 10, Foto.Height / 10));

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúEscala.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



*Ilustración 13: Cambio de tamaño de imagen*

# Reflejo Horizontal

O/003.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            // Aplicar el reflejo horizontal
            Foto.Mutate(x => x.Flip(FlipMode.Horizontal));

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúReflejoHorizontal.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Ilustración 14: Reflejo horizontal

# Reflejo Vertical

O/004.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            // Aplicar el reflejo vertical
            Foto.Mutate(x => x.Flip(FlipMode.Vertical));

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúReflejoVertical.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```

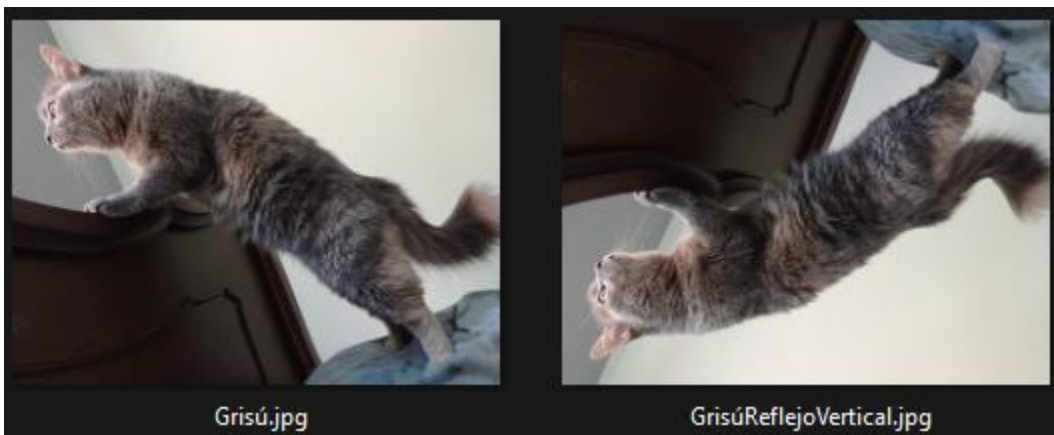


Ilustración 15: Reflejo vertical

# Giro de la imagen

O/005.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            // Aplicar el giro en 45 grados
            Foto.Mutate(x => x.Rotate(45));

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúGiro.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Ilustración 16: Giro de la imagen

## Filtro blanco y negro

O/006.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro de blanco y negro
            Foto.Mutate(x => x.BlackWhite());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúBlancoNegro.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```

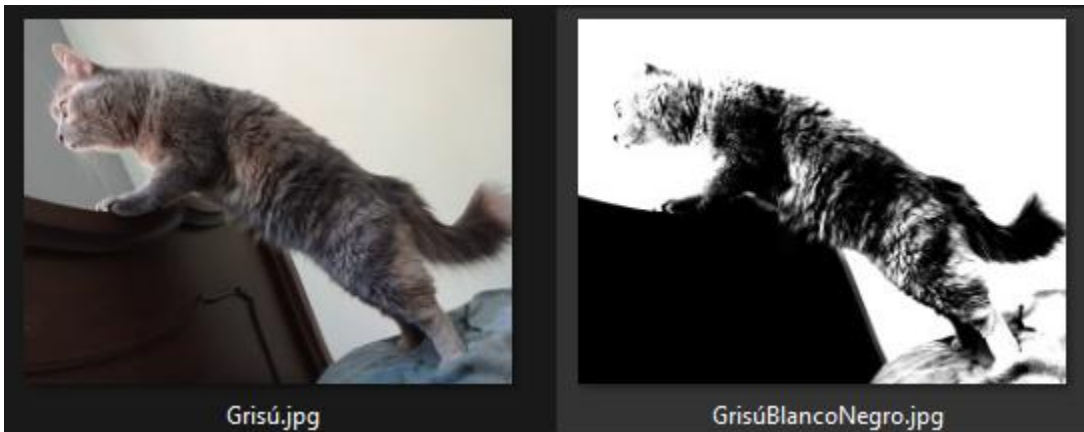


Ilustración 17: Filtro blanco y negro



# Invierte colores

O/007.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro de invertir colores
            Foto.Mutate(x => x.Invert());

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúInvierte.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```

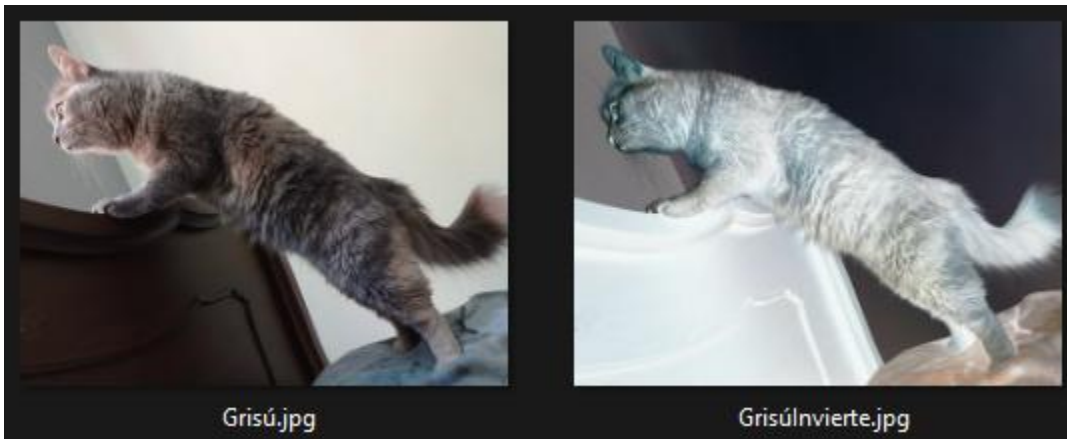


Ilustración 18: Invierte colores

# Filtro Kodachrome

Aplica un efecto similar a las cámaras antiguas Kodachrome.

O/008.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica un efecto similar a las cámaras antiguas Kodachrome
            Foto.Mutate(x => x.Kodachrome());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúKodachrome.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```

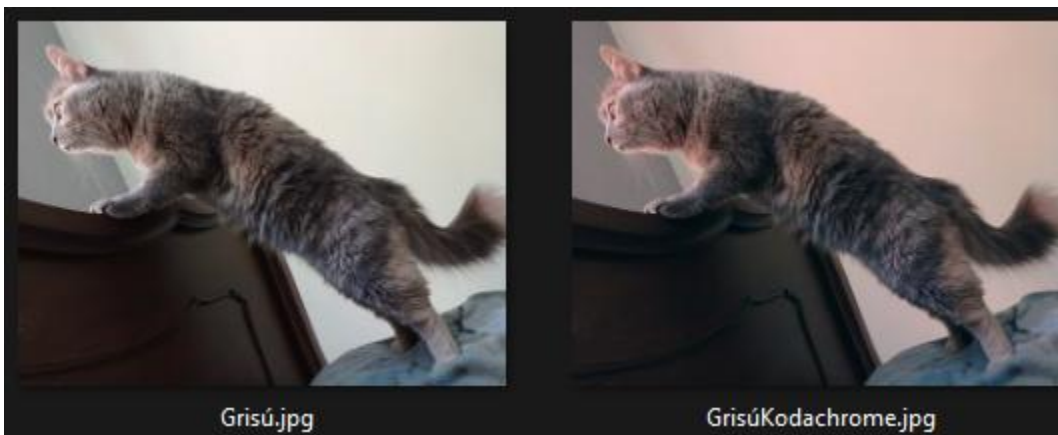


Ilustración 19: Filtro Kodachrome

# Filtro Sepia

O/009.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro sepia
            Foto.Mutate(x => x.Sepia());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúSepia.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúSepia.jpg

*Ilustración 20: Filtro Sepia*

# Filtro Lomograph

O/010.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro Lomograph
            Foto.Mutate(x => x.Lomograph());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúLomograph.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Ilustración 21: Filtro Lomograph

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro Polaroid
            Foto.Mutate(x => x.Polaroid());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúPolaroid.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúPolaroid.jpg

*Ilustración 22: Filtro Polaroid*

## Filtro umbral

O/012.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro Threshold
            Foto.Mutate(x => x.AdaptiveThreshold());

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúThreshold.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúThreshold.jpg

Ilustración 23: Filtro umbral

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro detección de bordes
            Foto.Mutate(x => x.DetectEdges());

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúBordes.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúBordes.jpg

Ilustración 24: Filtro borde

## Filtro pixelado

Pixelado de la imagen, el parámetro es el tamaño del pixel, entre más alto, más se pixela.

O/014.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\\\TEMP\\\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro pixelado
            Foto.Mutate(x => x.Pixelate(100));

            //Guarda la nueva imagen
            string Salida = "C:\\\\TEMP\\\\GrisúPixelado.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúPixelado.jpg

*Ilustración 25: Filtro pixelado*



## Filtro de saturación de color

Recibe un parámetro entero, entre más alto, más satura el color.

O/015.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro de saturación de color
            Foto.Mutate(x => x.Saturate((float)10));

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúSaturado.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúSaturado.jpg

*Ilustración 26: Filtro de saturación de color*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

internal class Program {
    static void Main() {
        //Carga imagen original
        string Entrada = "C:\\TEMP\\Grisú.jpg";
        using (Image<Rgba32> Foto = Image.Load<Rgba32>(Entrada)) {
            //Aplica el filtro viñeta
            Foto.Mutate(x => x.Vignette(Color.Blue));

            //Guarda la nueva imagen
            string Salida = "C:\\TEMP\\GrisúVineta.jpg";
            Foto.Save(Salida);
        }

        Console.WriteLine("Conversión terminada.");
    }
}
```



Grisú.jpg



GrisúVineta.jpg

Ilustración 27: Filtro viñeta

# Cambiar pixel a pixel

O/017.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

namespace Ejemplo;

internal class Program {
    static void Main() {
        // Cargar la imagen original
        using (var Foto = Image.Load<Rgba32>("C:\\TEMP\\Grisú.jpg")) {
            // Recorrer cada píxel y convertirlo a escala de grises
            for (int y = 0; y < Foto.Height; y++) {
                for (int x = 0; x < Foto.Width; x++) {
                    var pixel = Foto[x, y];
                    // Calcular el valor de gris usando la fórmula de luminancia
                    byte gris = (byte) (0.3 * pixel.R + 0.59 * pixel.G + 0.11 *
pixel.B);
                    var Pixelgris = new Rgba32(gris, gris, gris, pixel.A);
                    Foto[x, y] = Pixelgris;
                }
            }

            // Guardar la imagen en escala de grises
            Foto.Save("C:\\TEMP\\GrisúPixelGris.jpg");
        }
        Console.WriteLine("Proceso terminado");
    }
}
```



Grisú.jpg



GrisúPixelGris.jpg

Ilustración 28: Modificando pixel a pixel

# Invertir colores

O/018.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

namespace Ejemplo;

internal class Program {
    static void Main() {
        // Cargar la imagen original
        using (var Foto = Image.Load<Rgba32>("C:\\TEMP\\Grisú.jpg")) {

            // Recorrer cada píxel y convertirlo
            for (int y = 0; y < Foto.Height; y++) {
                for (int x = 0; x < Foto.Width; x++) {
                    var pixel = Foto[x, y];

                    //Invierte el color
                    byte NuevoR = (byte) (255 - pixel.R);
                    byte NuevoG = (byte) (255 - pixel.G);
                    byte NuevoB = (byte) (255 - pixel.B);

                    //El pixel con el nuevo color
                    var PixelNuevo = new Rgba32(NuevoR, NuevoG, NuevoB, pixel.A);

                    //Cambia la imagen
                    Foto[x, y] = PixelNuevo;
                }
            }

            // Guardar la imagen en escala de grises
            Foto.Save("C:\\TEMP\\GrisúInvierte.jpg");
        }
        Console.WriteLine("Proceso terminado");
    }
}
```



Grisú.jpg



GrisúInvierte.jpg

*Ilustración 29: Invertir colores*

## Aplicando un núcleo o “kernel”

Un kernel es una matriz cuadrada pequeña (por ejemplo, 3×3 o 5×5) que contiene valores numéricos.

Se utiliza en la operación llamada convolución, donde cada píxel de la imagen se recalcula combinando los valores de sus vecinos según los coeficientes del kernel.

El resultado depende de los valores dentro del kernel: algunos resaltan bordes, otros suavizan, otros enfocan.

| Tipo de operación   | Matriz                                                                      | Resultado           |
|---------------------|-----------------------------------------------------------------------------|---------------------|
| Identidad           | $\begin{vmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{vmatrix}$         | No cambia la imagen |
| Detección de bordes | $\begin{vmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{vmatrix}$ | Resalta los bordes  |
| Enfoque             | $\begin{vmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{vmatrix}$     | Aumenta nitidez     |

O/019.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;

namespace Ejemplo;

class Program {
    static void Main() {
        int[,] Nucleo = {
            { 1, 0, -1 },
            { 1, 0, -1 },
            { 1, 0, -1 }
        };

        int nAlto = Nucleo.GetLength(0);
        int nAncho = Nucleo.GetLength(1);

        using (var Foto = Image.Load<Rgba32>("C:\\\\TEMP\\\\Grisú.jpg")) {
            var pixel = Foto[0, 0];
        }
    }
}
```

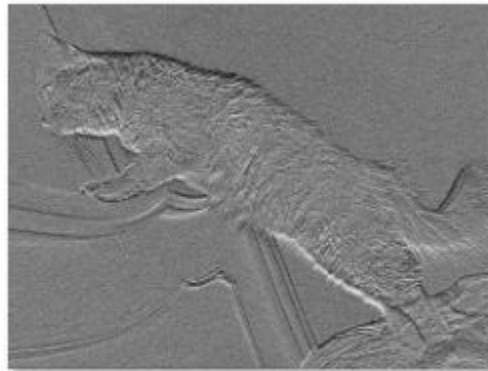
```

for (int y = 0; y < Foto.Height - nAlto + 1; y++) {
    for (int x = 0; x < Foto.Width - nAncho + 1; x++) {
        int Acumula = 0;
        for (int nY = 0; nY < nAlto; nY++) {
            for (int nX = 0; nX < nAncho; nX++) {
                pixel = Foto[x + nX, y + nY];
                int gris = (int)(0.3 * pixel.R + 0.59 * pixel.G + 0.11 *
pixel.B);
                Acumula += gris * Nucleo[nX, nY];
            }
        }
        byte suma = (byte)Acumula;
        Foto[x, y] = new Rgba32(suma, suma, suma, pixel.A);
    }
}
// Guardar la imagen que se le aplicó el núcleo
Foto.Save("C:\\TEMP\\GrisúAplicaNucleo.jpg");
}
Console.WriteLine("Proceso terminado");
}
}

```



Grisú.jpg



GrisúAplicaNucleo.jpg

*Ilustración 30: Aplicando un núcleo*

## Una aplicación WPF que carga una imagen

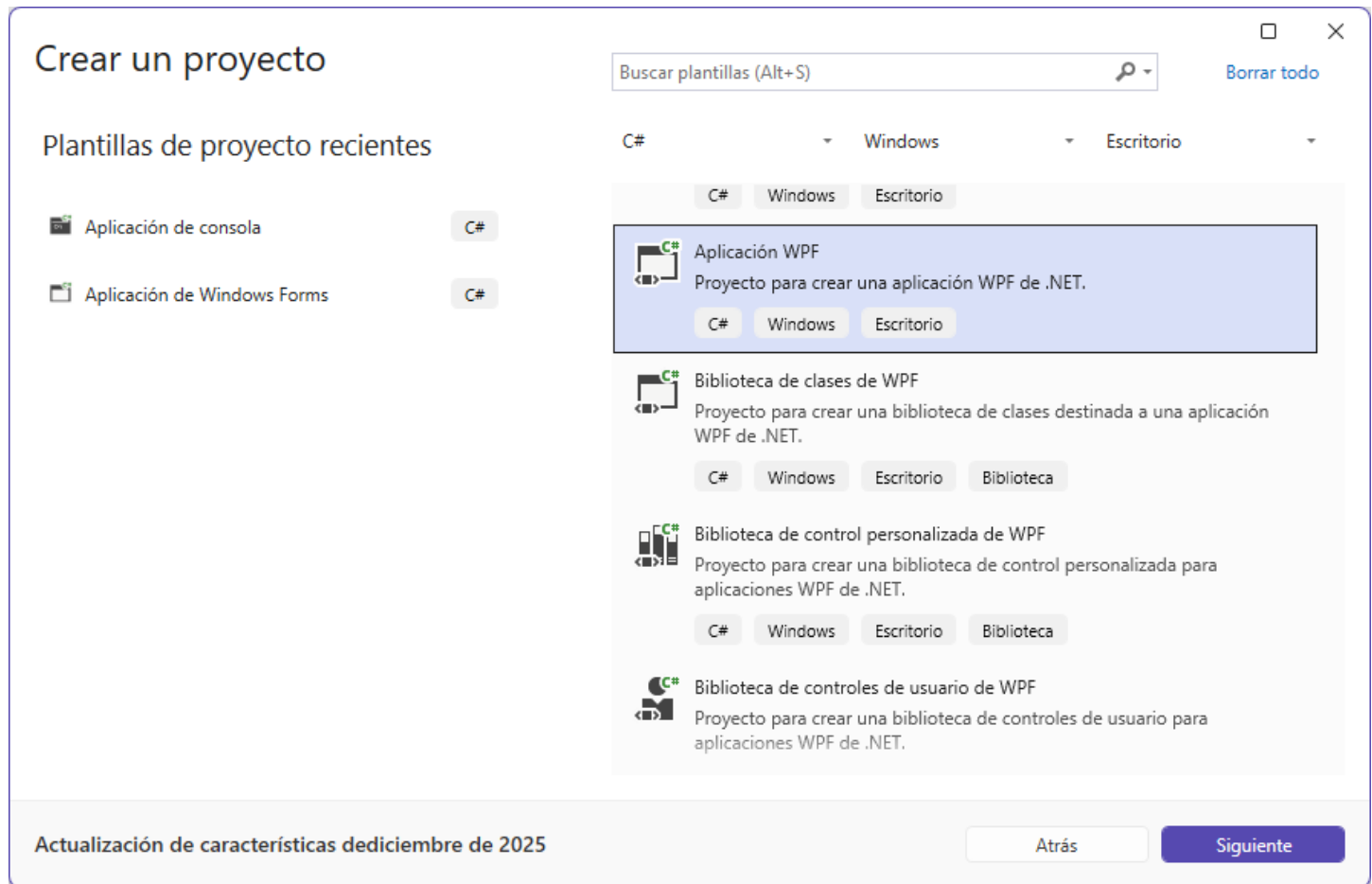


Ilustración 31: Aplicación WPF



Configure su nuevo proyecto

Aplicación WPF

C#

Windows

Escritorio

Nombre del proyecto

Sharplimagen

Ubicación

C:\Users\engin\source\repos

...

Nombre de la solución ⓘ

Sharplimagen

☐ Colocar la solución y el proyecto en el mismo directorio

Proyecto se creará en "C:\Users\engin\source\repos\Sharplimagen\Sharplimagen\"

Actualización de características dediciembre de 2025

Atrás

Siguiente

Ilustración 32: Nombre del proyecto

Información adicional

Aplicación WPF

C#WindowsEscritorio

Framework ⓘ

.NET 10.0 (Compatibilidad a largo plazo)

Actualización de características dediciembre de 2025

Atrás

Crear

Ilustración 33: Seleccionar .NET 10

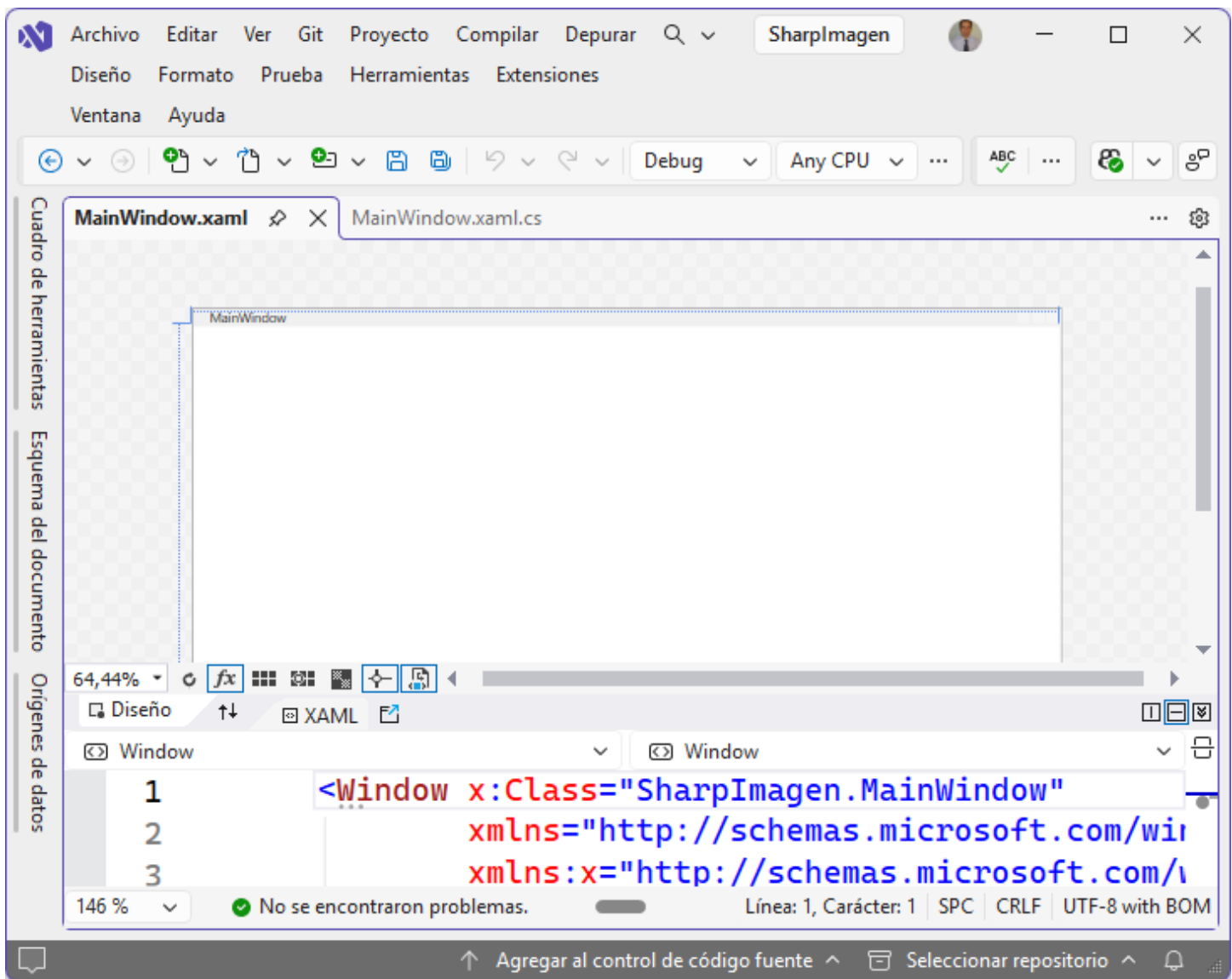


Ilustración 34: Aplicación WPF típica

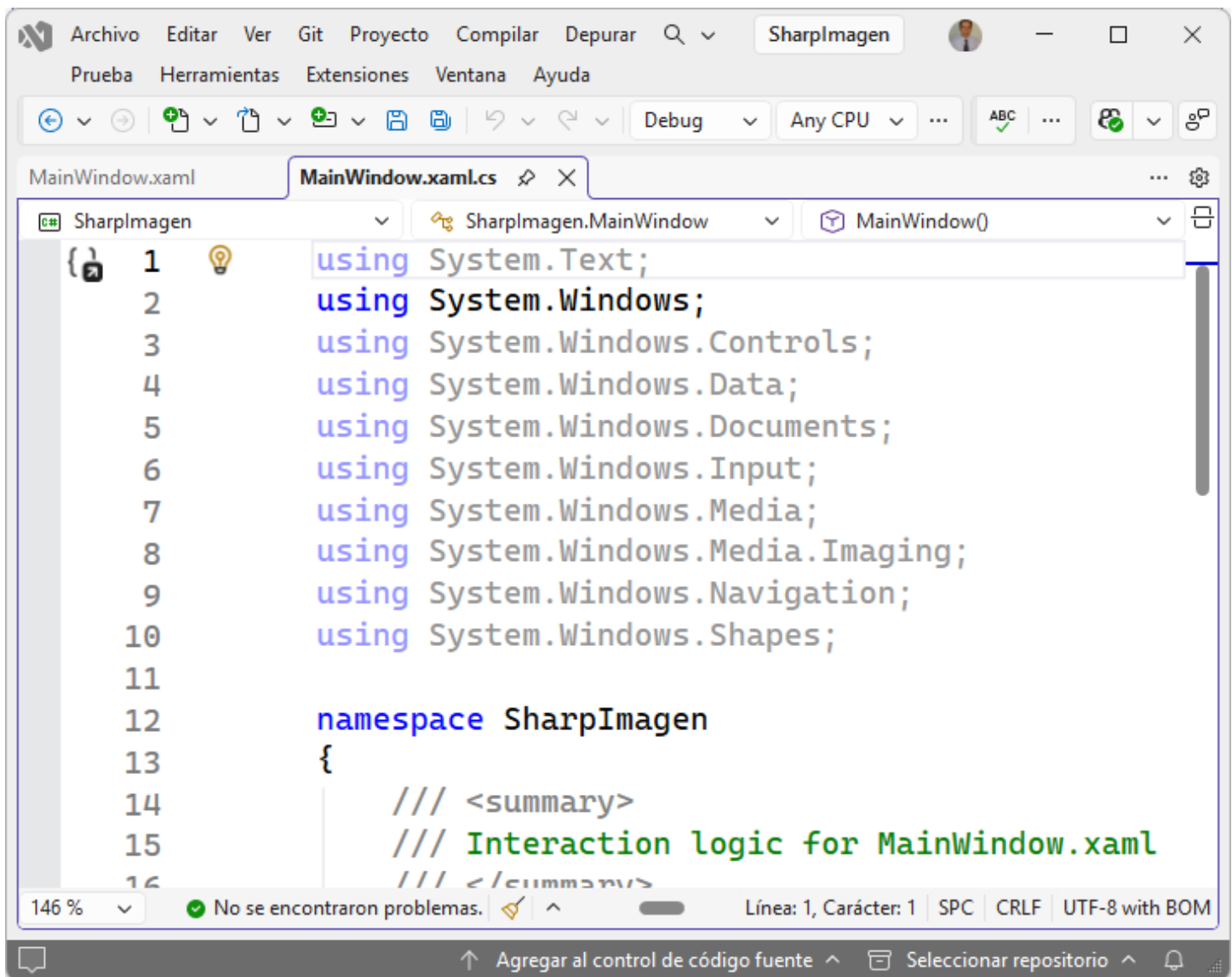


Ilustración 35: Código en C#

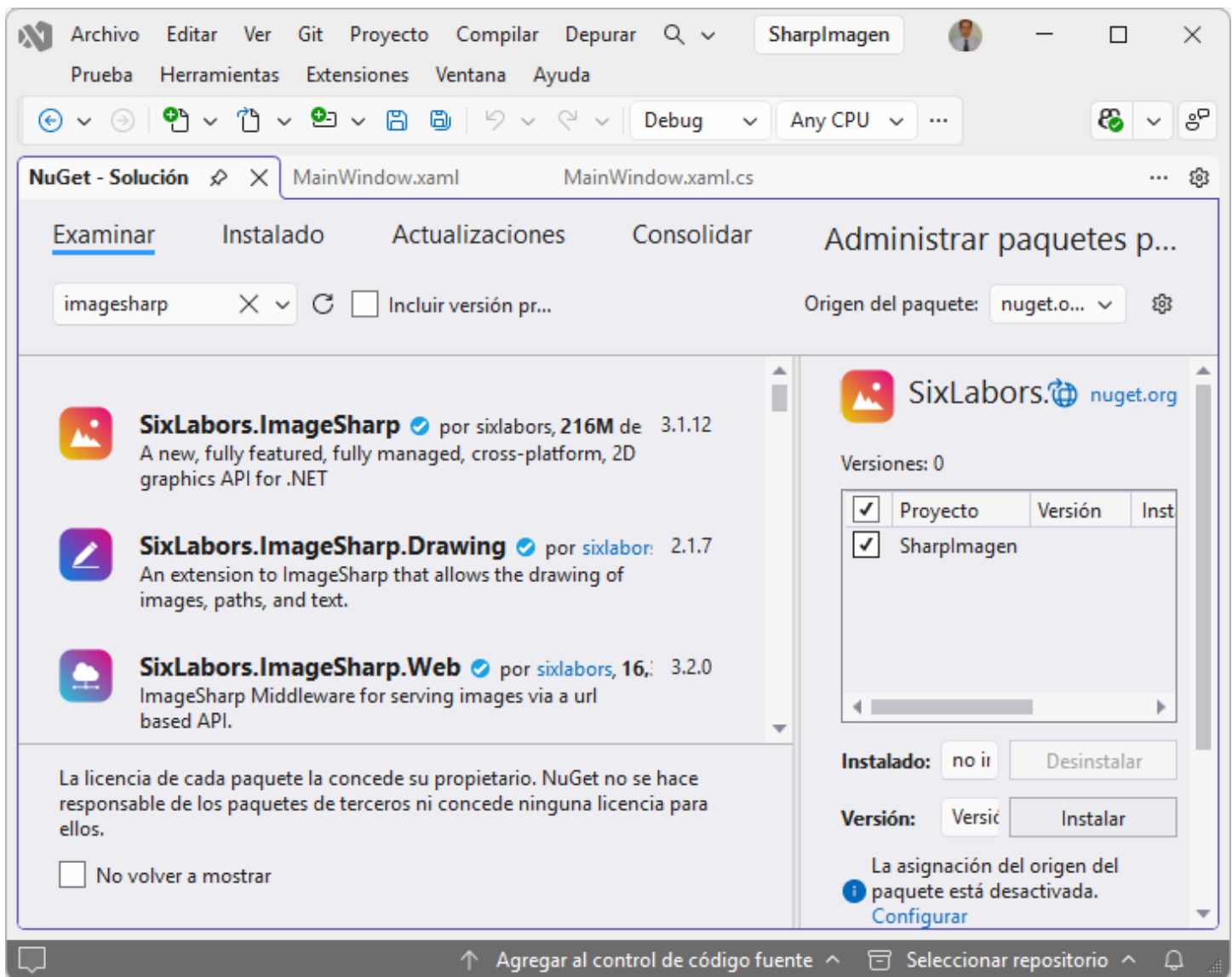


Ilustración 36: Carga las librerías

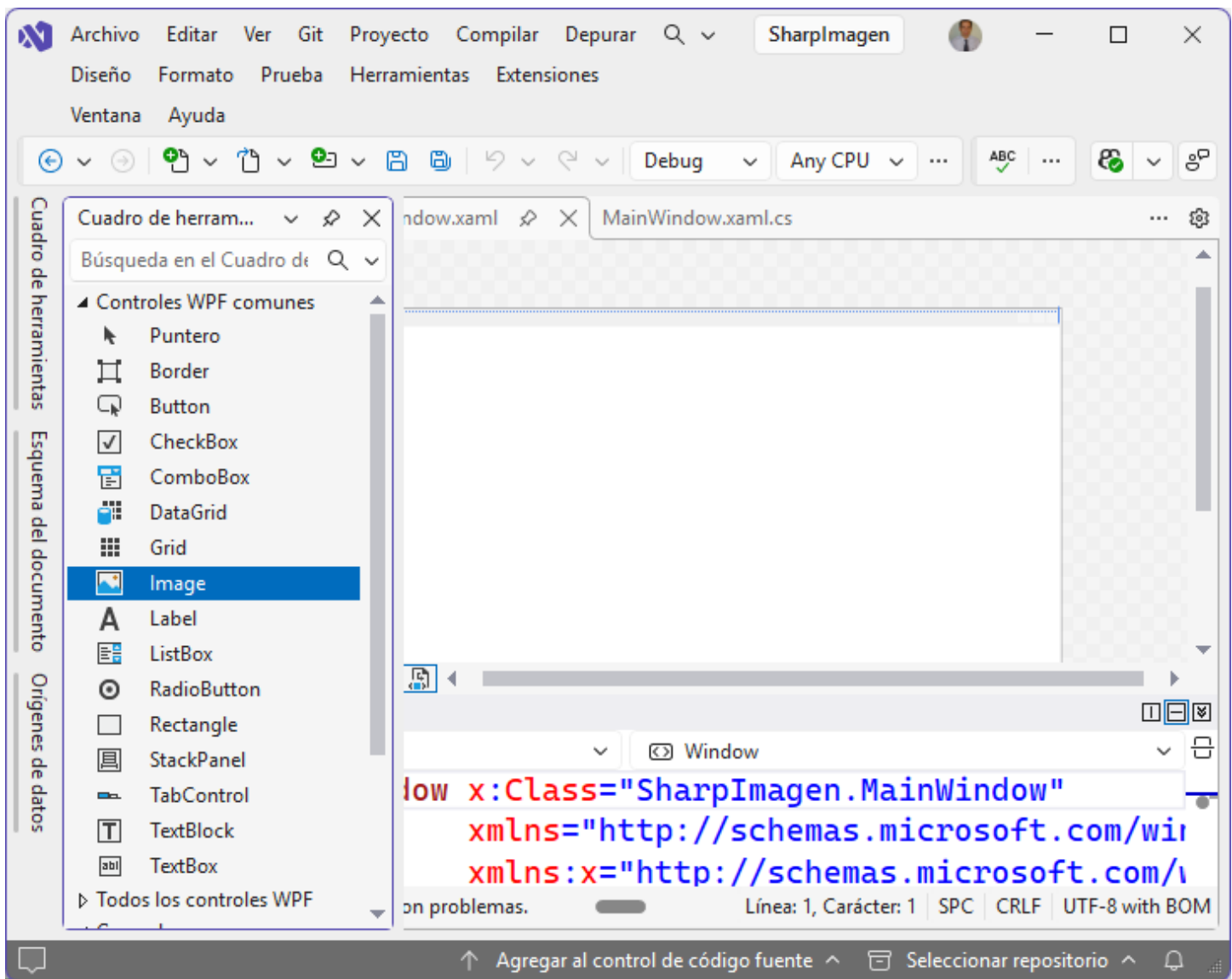


Ilustración 37: Se pone el objeto "Image" en la ventana

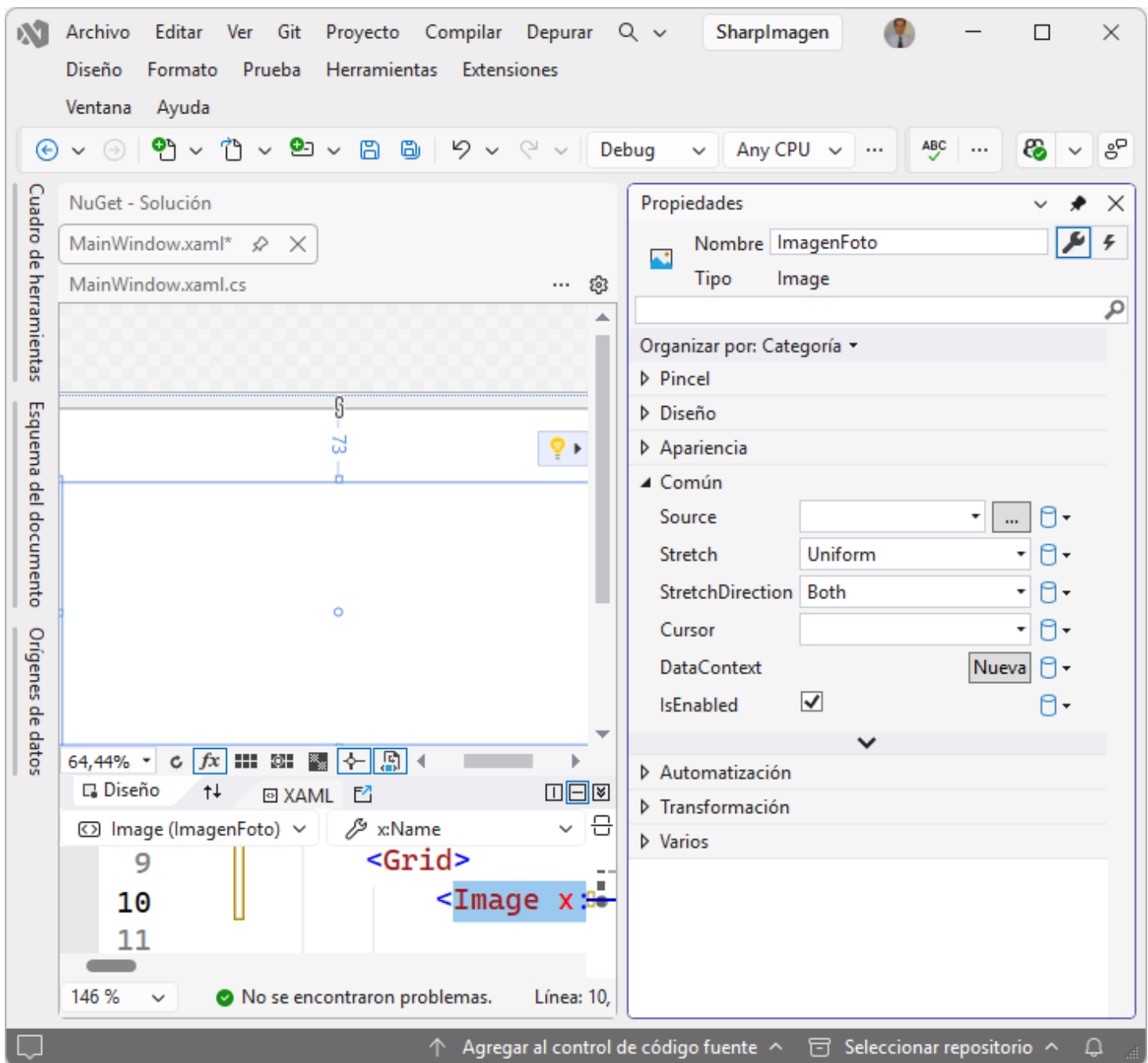


Ilustración 38: Se pone nombre a ese objeto "Image"

O/020.cs

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;
```

```

namespace SharpImagen {
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window {
        public MainWindow() {
            InitializeComponent();
            LoadImage("C:\\TEMP\\Grisú.jpg");
        }

        //Carga la imagen y la muestra en el control
        private void LoadImage(string RutaImagen) {
            try {
                using (var image =
SixLabors.ImageSharp.Image.Load<Rgba32>(RutaImagen)) {
                    using (var FlujoMemoria = new MemoryStream()) {
                        image.SaveAsBmp(FlujoMemoria);

                        /* Mover la posición actual dentro del flujo de memoria
                        (FlujoMemoria) a un punto específico.
                        FlujoMemoria es usado para almacenar temporalmente la imagen
                        en memoria.
                        Seek(0, SeekOrigin.Begin): Es el método que mueve la
                        posición actual dentro del flujo.
                        Este método toma dos parámetros:
                        0: Este es el desplazamiento en bytes desde la posición
                        especificada por el segundo parámetro.
                        En este caso, 0 significa que no se está moviendo
                        desde la posición especificada, simplemente se está
                        estableciendo la posición en el inicio del flujo.
                        SeekOrigin.Begin: Este es un enumerador que especifica
                        el punto de referencia desde el cual
                        se calcula la nueva posición.
                        SeekOrigin.Begin indica que el punto
                        de referencia es el comienzo del flujo. */
                        FlujoMemoria.Seek(0, SeekOrigin.Begin);

                        var bitmap = new BitmapImage();

                        /* BeginInit() marca el inicio de un bloque de inicialización
                        * para el objeto BitmapImage. Durante este bloque, se puede
                        * establecer varias propiedades del BitmapImage sin que el
                        * objeto intente cargarse o procesarse inmediatamente */
                        bitmap.BeginInit();
                        bitmap.StreamSource = FlujoMemoria;

                        /* Se utiliza para configurar cómo se almacena en caché la
                        imagen cuando se carga en un objeto BitmapImage.

```



bitmap: Es el objeto BitmapImage que se está utilizando para mostrar la imagen en la aplicación WPF.

CacheOption: Es una propiedad del BitmapImage que determina cómo se almacena en caché la imagen. Esto afecta el rendimiento y el uso de memoria de la aplicación.

BitmapCacheOption.OnLoad: Es uno de los valores del enumerador BitmapCacheOption. Especifica que la imagen debe cargarse completamente en memoria cuando se llama al método EndInit(). Esto significa que toda la imagen se carga y se almacena en memoria de una vez.

```

*/
bitmap.CacheOption = BitmapCacheOption.OnLoad;

/* En este punto, el BitmapImage se carga y se procesa utilizando
 * las propiedades que se establecieron durante el bloque
 * de inicialización. */
bitmap.EndInit();
ImagenFoto.Source = bitmap;
}
}
}
catch (Exception ex) {
    MessageBox.Show($"Error al cargar la imagen: {ex.Message}");
}
}
}
}
}

```

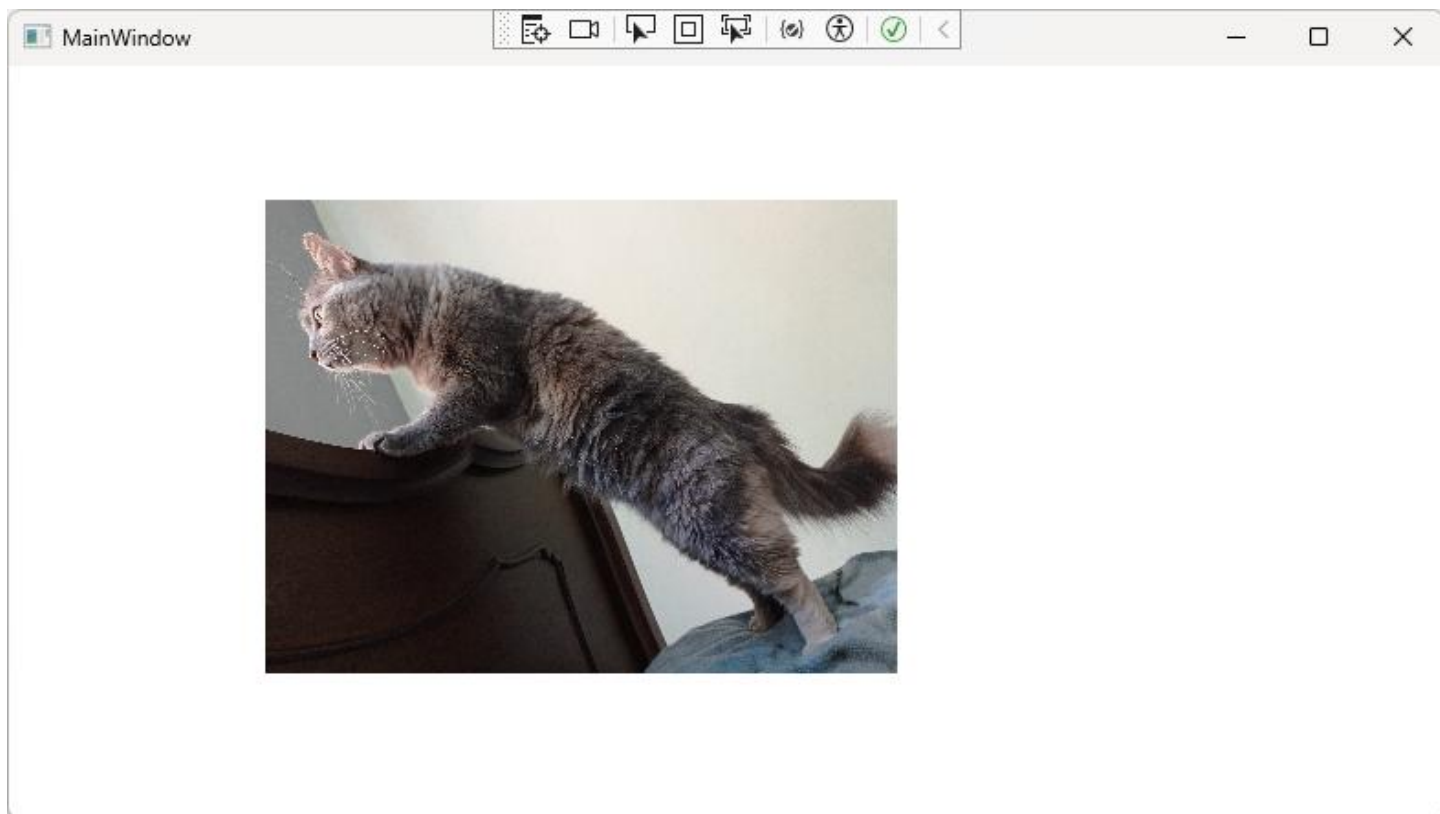
O/020.xaml

```

<Window x:Class="SharpImagen.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:SharpImagen"
    mc:Ignorable="d"
    Title="MainWindow" Height="450" Width="800">
    <Grid>
        <Image x:Name="ImagenFoto" HorizontalAlignment="Left" Height="333"
Margin="45,41,0,0" VerticalAlignment="Top" Width="676"/>
    </Grid>
</Window>

```

El proyecto completo se puede descargar en O/020.zip



*Ilustración 39: Ejecución del proyecto*

## Una aplicación WPF que aplica varios filtros

Se muestra una aplicación en WPF que permite al usuario cargar una imagen (bmp, jpg, jpeg, gif, webp, tga) y luego aplicarle varios filtros, inclusive combinándolos. El proyecto completo se puede descargar en O/021.zip

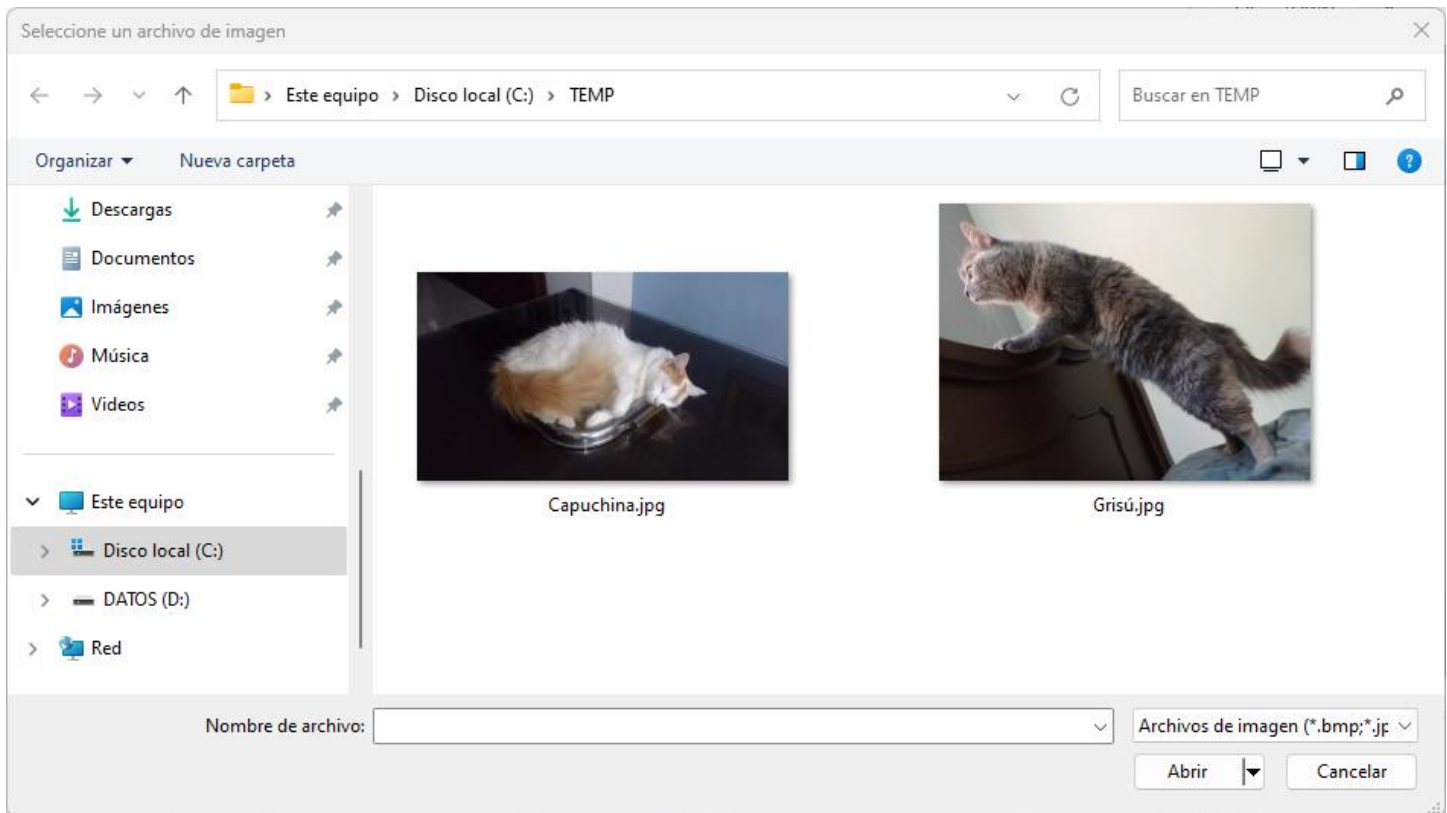
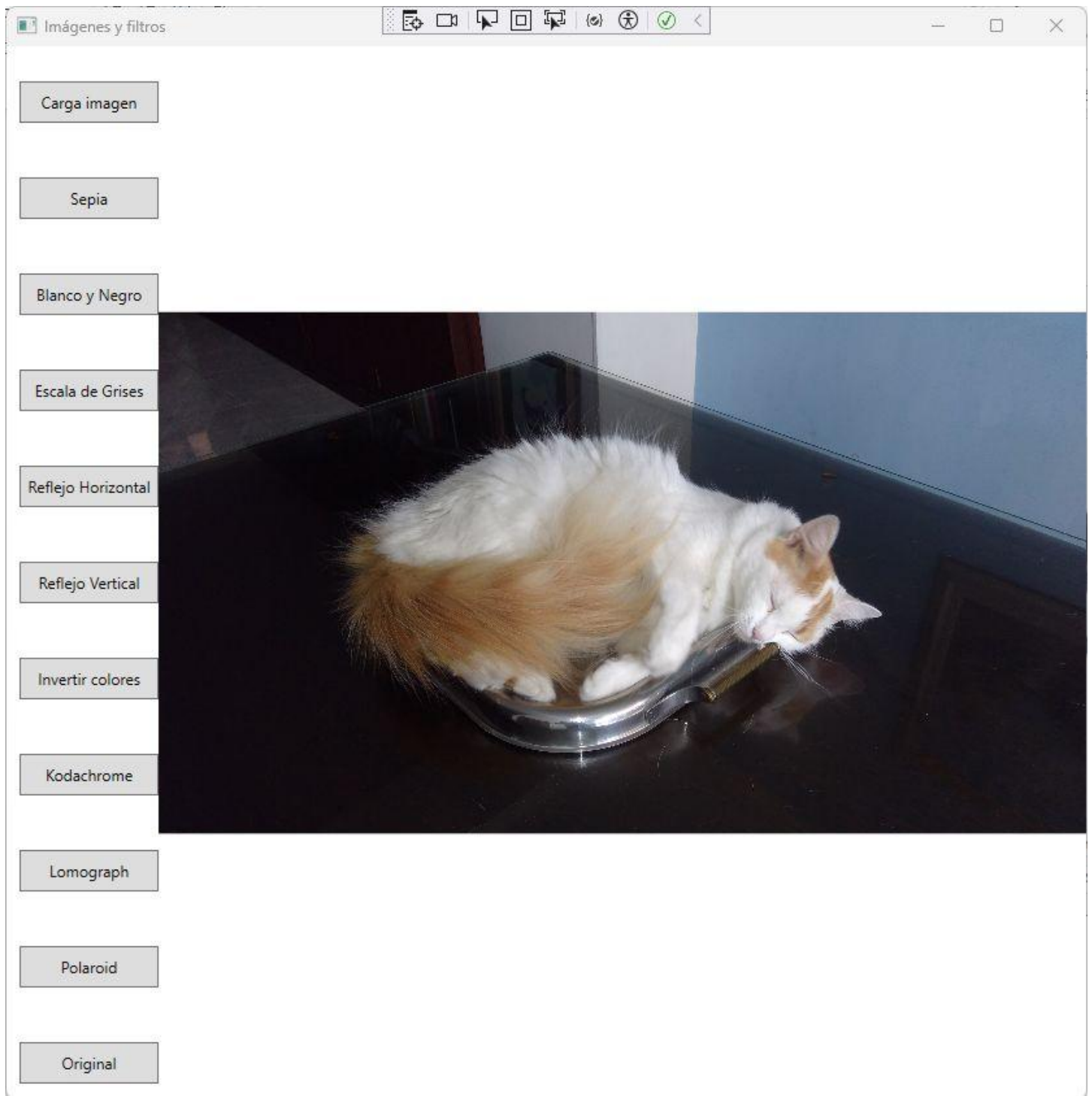
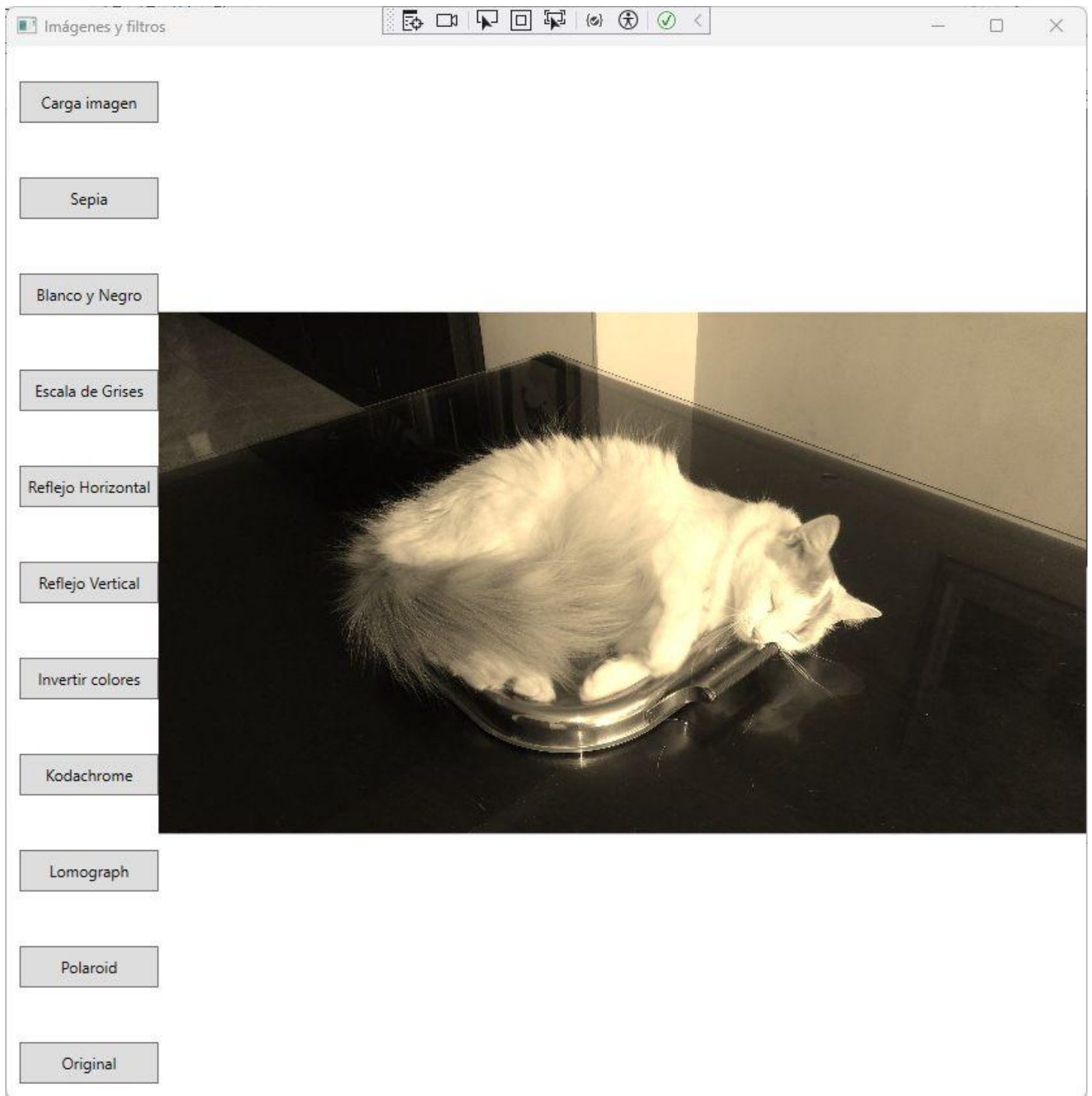


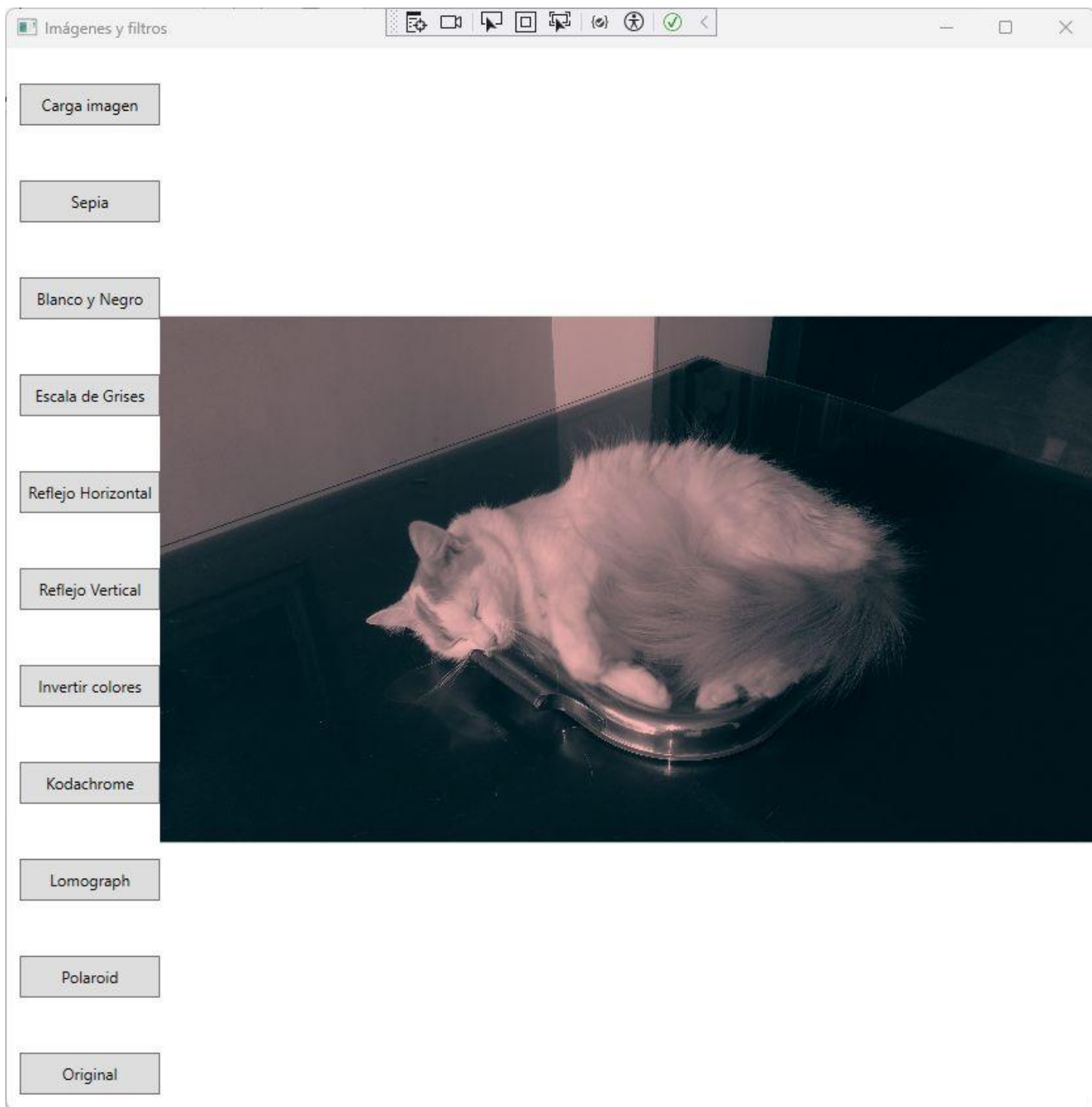
Ilustración 40: Una ventana de diálogo para escoger el archivo de imagen



*Ilustración 41: Imagen cargada*



*Ilustración 42: Aplicando el filtro sepia*



*Ilustración 43: Aplicando reflejo horizontal y doble vez Kodachrome*

```

using Microsoft.Win32;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;
using System.IO;
using System.Windows;
using System.Windows.Media.Imaging;

namespace SharpImagen {
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window {
        SixLabors.ImageSharp.Image Foto, Copia;

        public MainWindow() {
            InitializeComponent();
        }

        //Carga la imagen y la muestra en el control
        private void MuestraFoto() {
            var FlujoMemoria = new MemoryStream();
            Copia.SaveAsBmp(FlujoMemoria);

            /* Mover la posición actual dentro del flujo de memoria
            (FlujoMemoria) a un punto específico.
            FlujoMemoria es usado para almacenar temporalmente la imagen
            en memoria.
            Seek(0, SeekOrigin.Begin): Es el método que mueve la
            posición actual dentro del flujo.
            Este método toma dos parámetros:
            0: Este es el desplazamiento en bytes desde la posición
            especificada por el segundo parámetro.
            En este caso, 0 significa que no se está moviendo
            desde la posición especificada, simplemente se está
            estableciendo la posición en el inicio del flujo.
            SeekOrigin.Begin: Este es un enumerador que especifica
            el punto de referencia desde el cual
            se calcula la nueva posición.
            SeekOrigin.Begin indica que el punto
            de referencia es el comienzo del flujo. */
            FlujoMemoria.Seek(0, SeekOrigin.Begin);

            var bitmap = new BitmapImage();

            /* BeginInit() marca el inicio de un bloque de inicialización
            * para el objeto BitmapImage. Durante este bloque, se puede

```



```

    * establecer varias propiedades del BitmapImage sin que el
    * objeto intente cargarse o procesarse inmediatamente */
bitmap.BeginInit();
bitmap.StreamSource = FlujoMemoria;

/* Se utiliza para configurar cómo se almacena en caché la
imagen cuando se carga en un objeto BitmapImage.
bitmap: Es el objeto BitmapImage que se está utilizando
para mostrar la imagen en la aplicación WPF.
CacheOption: Es una propiedad del BitmapImage que determina
cómo se almacena en caché la imagen.
Esto afecta el rendimiento y el uso de memoria
de la aplicación.
BitmapCacheOption.OnLoad: Es uno de los valores del enumerador
BitmapCacheOption. Especifica que la
imagen debe cargarse completamente en
memoria cuando se llama al método
EndInit(). Esto significa que toda
la imagen se carga y se almacena en
memoria de una vez.
*/
bitmap.CacheOption = BitmapCacheOption.OnLoad;

/* En este punto, el BitmapImage se carga y se procesa utilizando
* las propiedades que se establecieron durante el bloque
* de inicialización. */
bitmap.EndInit();
ImagenFoto.Source = bitmap;

}

private void btnSepia_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Sepia());
    MuestraFoto();
}

private void btnBlancoNegro_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.BlackWhite());
    MuestraFoto();
}

private void btnGrises_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Grayscale());
    MuestraFoto();
}

private void btnOriginal_Click(object sender, RoutedEventArgs e) {
    Copia = Foto.CloneAs<Rgba32>();
}

```



```

    MuestraFoto();
}

private void btnReflejoH_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Flip(FlipMode.Horizontal));
    MuestraFoto();
}

private void btnReflejoV_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Flip(FlipMode.Vertical));
    MuestraFoto();
}

private void btnInvierte_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Invert());
    MuestraFoto();
}

private void btnKodachrome_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Kodachrome());
    MuestraFoto();
}

private void btnLomograph_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Lomograph());
    MuestraFoto();
}

private void btnPolaroid_Click(object sender, RoutedEventArgs e) {
    Copia.Mutate(x => x.Polaroid());
    MuestraFoto();
}

private void btnCarga_Click(object sender, RoutedEventArgs e) {
    OpenFileDialog dlgAbrir = new OpenFileDialog();
    dlgAbrir.Filter = "Archivos de
imagen|*.bmp;*.jpg;*.jpeg;*.gif;*.webp;*.tga";
    dlgAbrir.Title = "Seleccione un archivo de imagen";

    if (dlgAbrir.ShowDialog() == true) {
        try {
            Foto = SixLabors.ImageSharp.Image.Load<Rgba32>(dlgAbrir.FileName);
            Copia = Foto.CloneAs<Rgba32>();
            MuestraFoto();
            btnBlancoNegro.IsEnabled = true;
            btnGrises.IsEnabled = true;
            btnSepia.IsEnabled = true;
            btnOriginal.IsEnabled = true;
        }
    }
}

```

```

        btnReflejoH.IsEnabled = true;
        btnReflejoV.IsEnabled = true;
        btnInvierte.IsEnabled = true;
        btnKodachrome.IsEnabled = true;
        btnLomograph.IsEnabled = true;
        btnPolaroid.IsEnabled = true;
    }
    catch (Exception ex) {
        MessageBox.Show($"Error al cargar la imagen: {ex.Message}");
    }
}
}
}
}
}

```

```

<Window x:Class="SharpImagen.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:SharpImagen"
    mc:Ignorable="d"
    Title="Imágenes y filtros" Height="850" Width="800"
    WindowStartupLocation="CenterScreen" WindowState="Maximized">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto"></ColumnDefinition>
            <ColumnDefinition Width="*"></ColumnDefinition>
        </Grid.ColumnDefinitions>
        <Image x:Name="ImagenFoto" Grid.Column="1"/>
        <Button x:Name="btnCarga" Content="Carga imagen"
            HorizontalAlignment="Left" Height="32" Margin="10,27,0,0"
            VerticalAlignment="Top" Width="107" Click="btnCarga_Click"/>
        <Button x:Name="btnSepia" Content="Sepia" HorizontalAlignment="Left"
            Height="32" Margin="10,101,0,0" VerticalAlignment="Top" Width="107"
            IsEnabled="False" Click="btnSepia_Click"/>
        <Button x:Name="btnBlancoNegro" Content="Blanco y Negro"
            HorizontalAlignment="Left" Height="32" Margin="10,175,0,0"
            VerticalAlignment="Top" Width="107" IsEnabled="False"
            Click="btnBlancoNegro_Click"/>
        <Button x:Name="btnGris" Content="Escala de Grises"
            HorizontalAlignment="Left" Height="32" Margin="10,249,0,0"
            VerticalAlignment="Top" Width="107" IsEnabled="False"
            Click="btnGris_Click"/>
        <Button x:Name="btnReflejoH" Content="Reflejo Horizontal"
            HorizontalAlignment="Left" Height="32" Margin="10,323,0,0"

```

```

VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnReflejoH_Click"/>
    <Button x:Name="btnReflejoV" Content="Reflejo Vertical"
HorizontalAlignment="Left" Height="32" Margin="10,397,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnReflejoV_Click"/>
    <Button x:Name="btnInvierte" Content="Invertir colores"
HorizontalAlignment="Left" Height="32" Margin="10,471,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnInvierte_Click"/>
    <Button x:Name="btnKodachrome" Content="Kodachrome"
HorizontalAlignment="Left" Height="32" Margin="10,545,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnKodachrome_Click"/>
    <Button x:Name="btnLomograph" Content="Lomograph"
HorizontalAlignment="Left" Height="32" Margin="10,619,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnLomograph_Click"/>
    <Button x:Name="btnPolaroid" Content="Polaroid"
HorizontalAlignment="Left" Height="32" Margin="10,693,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnPolaroid_Click"/>
    <Button x:Name="btnOriginal" Content="Original"
HorizontalAlignment="Left" Height="32" Margin="10,767,0,0"
VerticalAlignment="Top" Width="107" IsEnabled="False"
Click="btnOriginal_Click"/>
</Grid>
</Window>

```

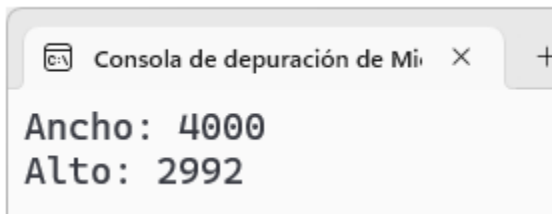
# Información imágenes

O/022.cs

```
using SixLabors.ImageSharp;

namespace Ejemplo;

class Program {
    static void Main() {
        ImageInfo InformacionImagen = Image.Identify(@"C:\\TEMP\\Grisú.jpg");
        Console.WriteLine($"Ancho: {InformacionImagen.Width}");
        Console.WriteLine($"Alto: {InformacionImagen.Height}");
    }
}
```



*Ilustración 44: Información de la imagen*

## Dibujando en imágenes

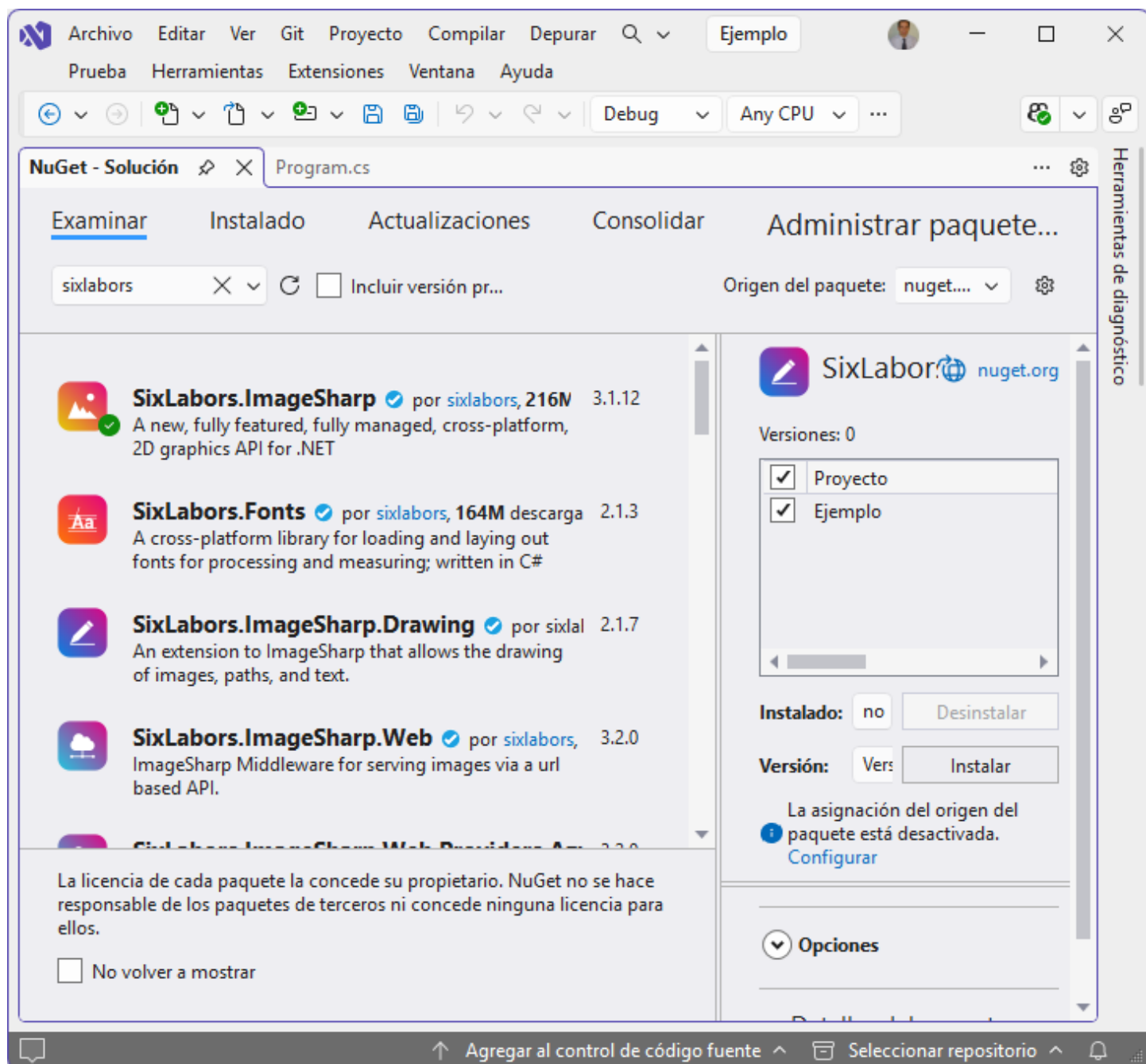
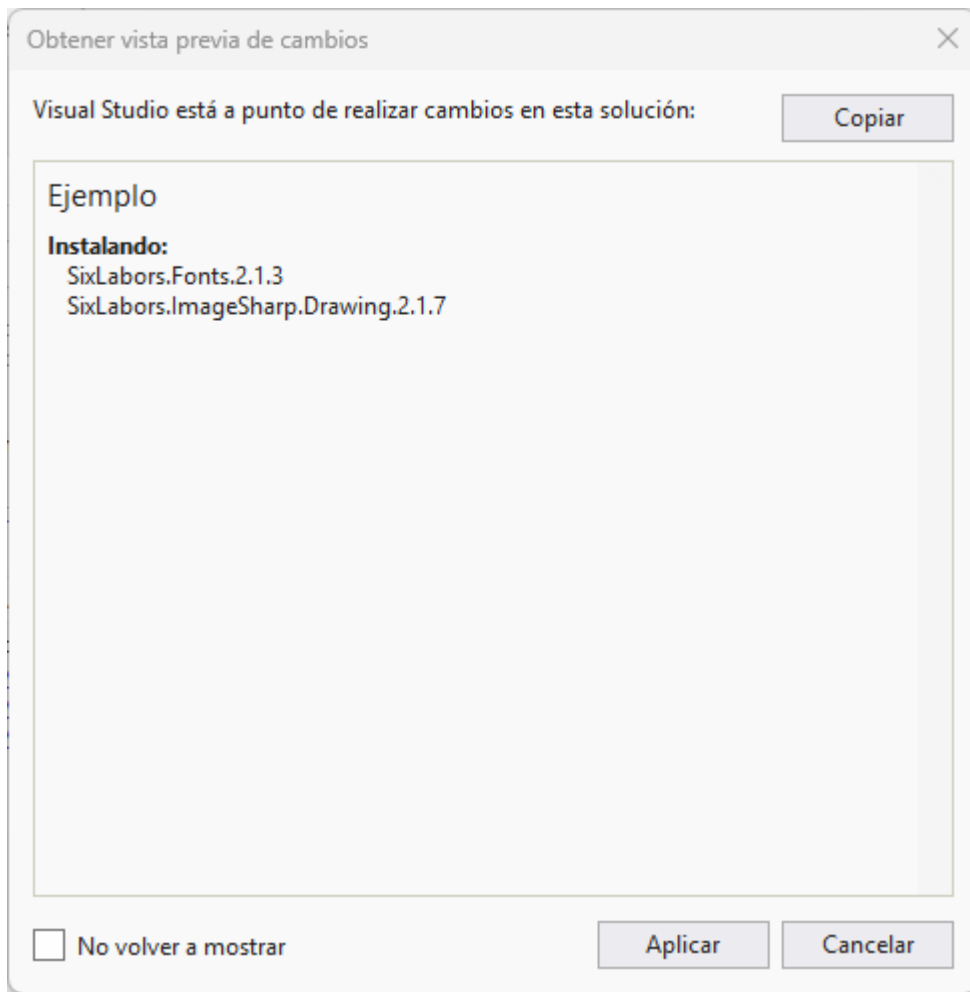
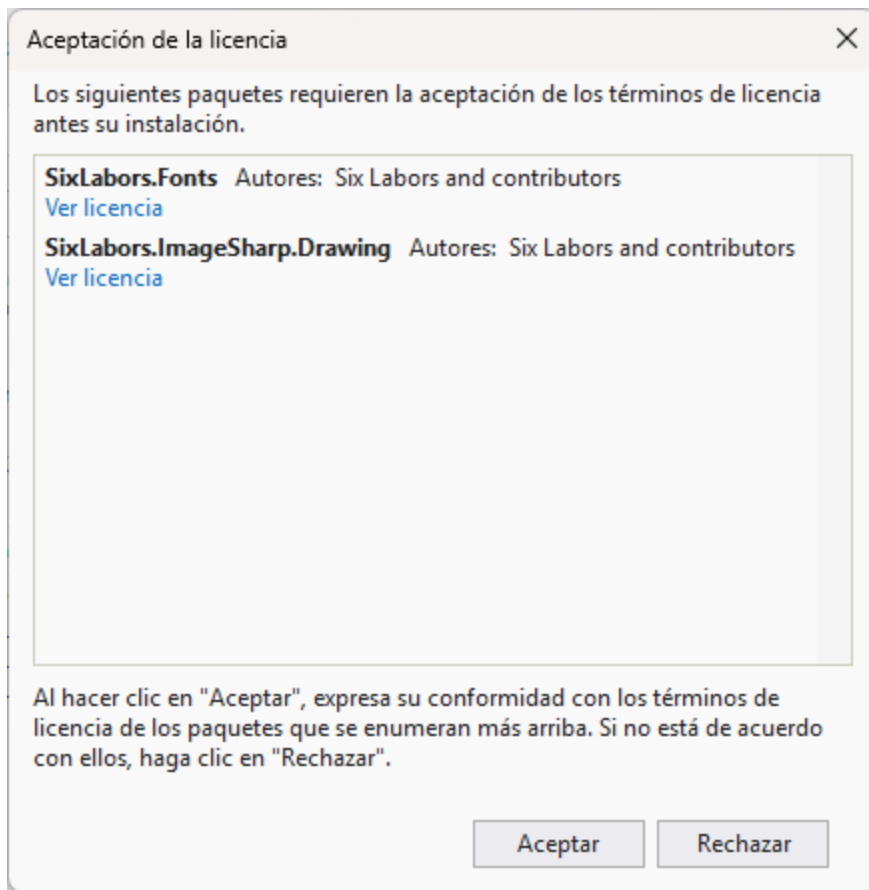


Ilustración 45: Se agrega SixLabors.ImageSharp.Drawing



*Ilustración 46: Agregar Drawing*



*Ilustración 47: Licencia de esa librería*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.Processing;
using SixLabors.ImageSharp.PixelFormats;

namespace Ejemplo;

class Program {
    static void Main() {
        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

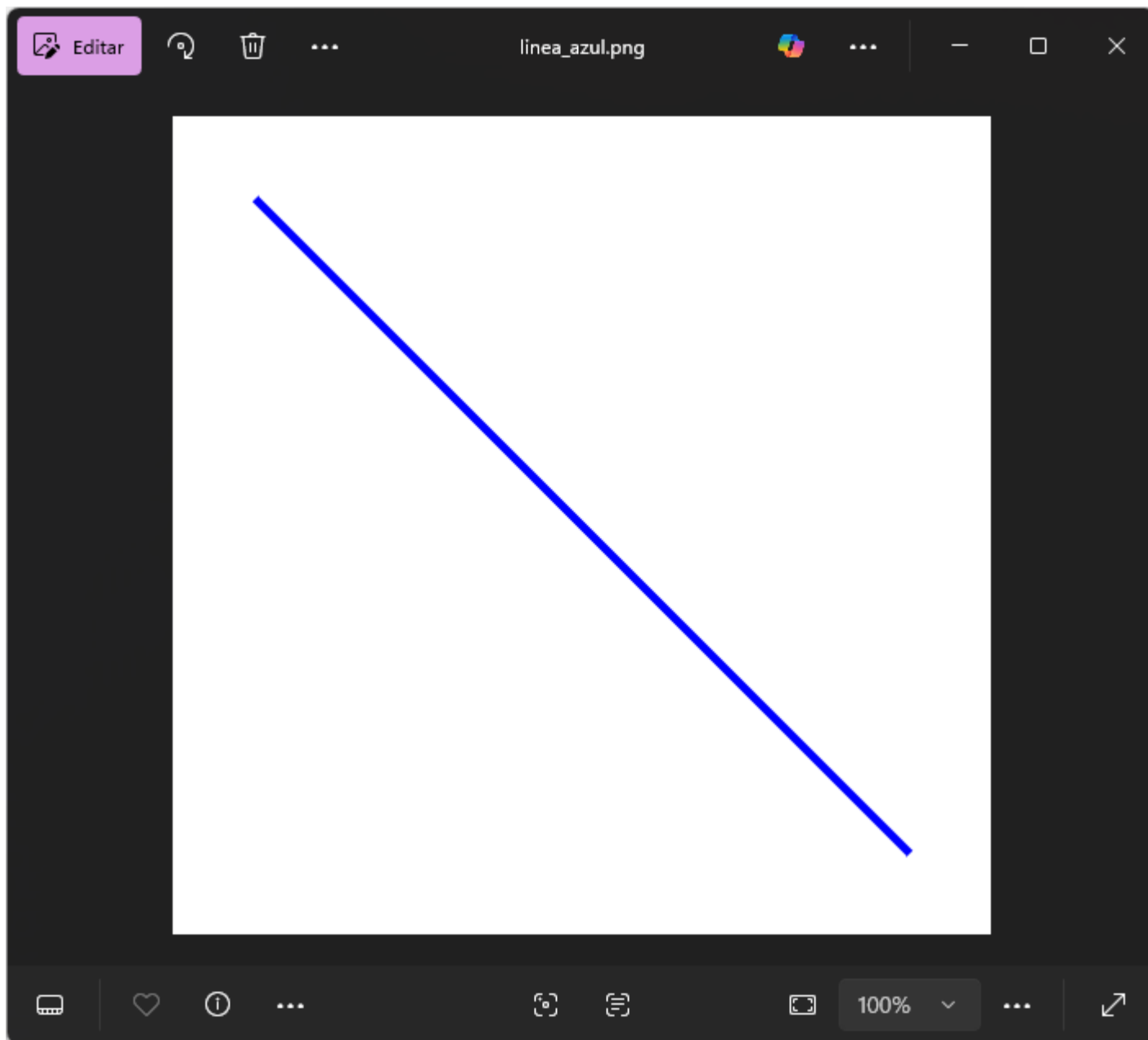
                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapis = Pens.Solid(Color.Blue, 5);

                // Dibujar una línea desde (50, 50) hasta (450, 450)
                var Linea = new SixLabors.ImageSharp.Drawing.PathBuilder()
                    .AddLine(new PointF(50, 50), new PointF(450, 450))
                    .Build();

                Lienzo.Draw(Lapis, Linea);
            });

            // Guardar la imagen
            NuevaImagen.Save("linea_azul.png");
        }
    }
}
```





*Ilustración 48: Dibuja una línea en imagen*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

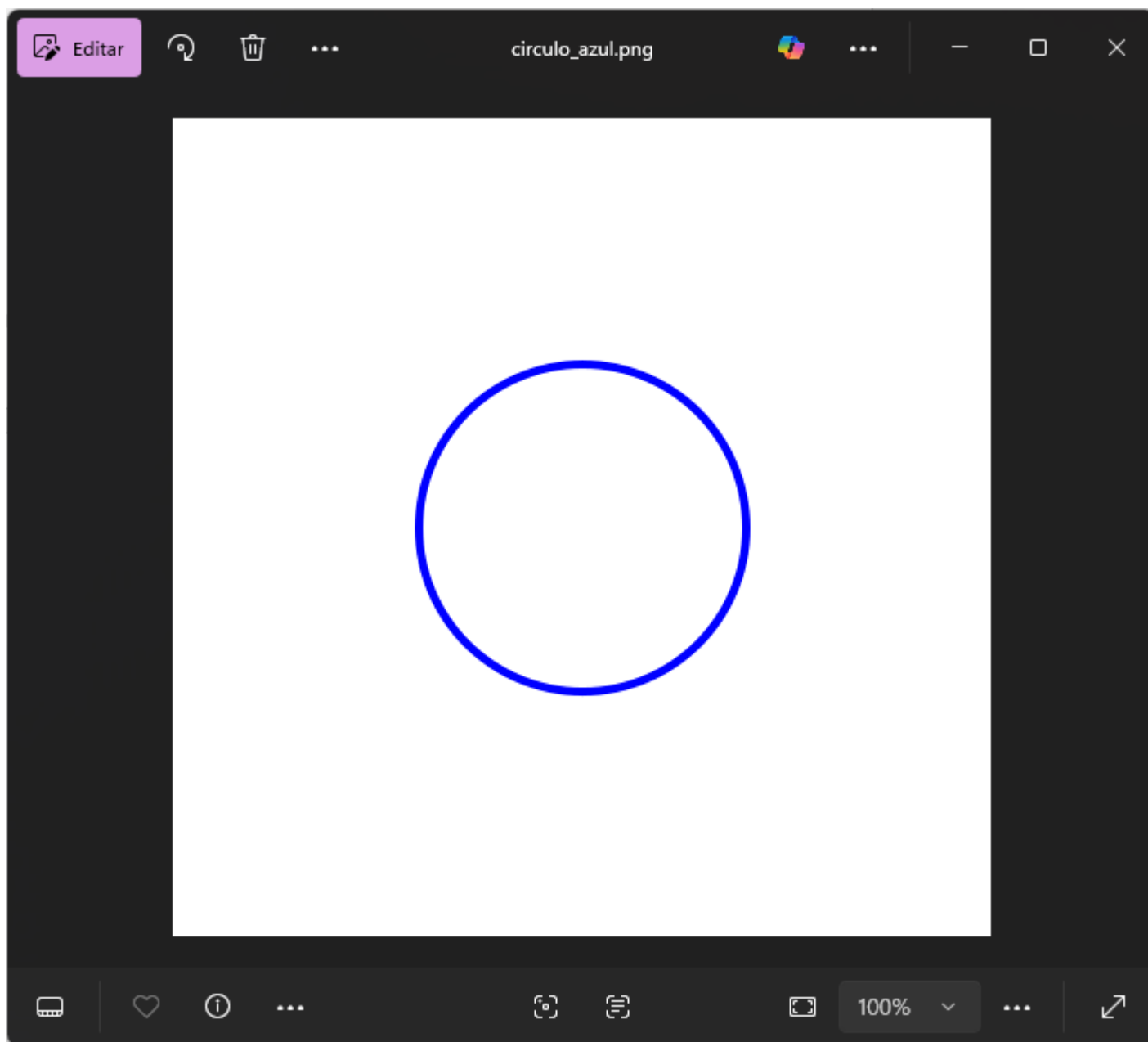
class Program {
    static void Main() {
        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapiz = Pens.Solid(Color.Blue, 5);

                // Dibujar un círculo centrado en (250, 250) con radio 100
                var Circulo = new EllipsePolygon(new PointF(250, 250), 100);

                Lienzo.Draw(Lapiz, Circulo);
            });

            // Guardar la imagen
            NuevaImagen.Save("circulo_azul.png");
        }
    }
}
```



*Ilustración 49: Dibuja círculo en imagen*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

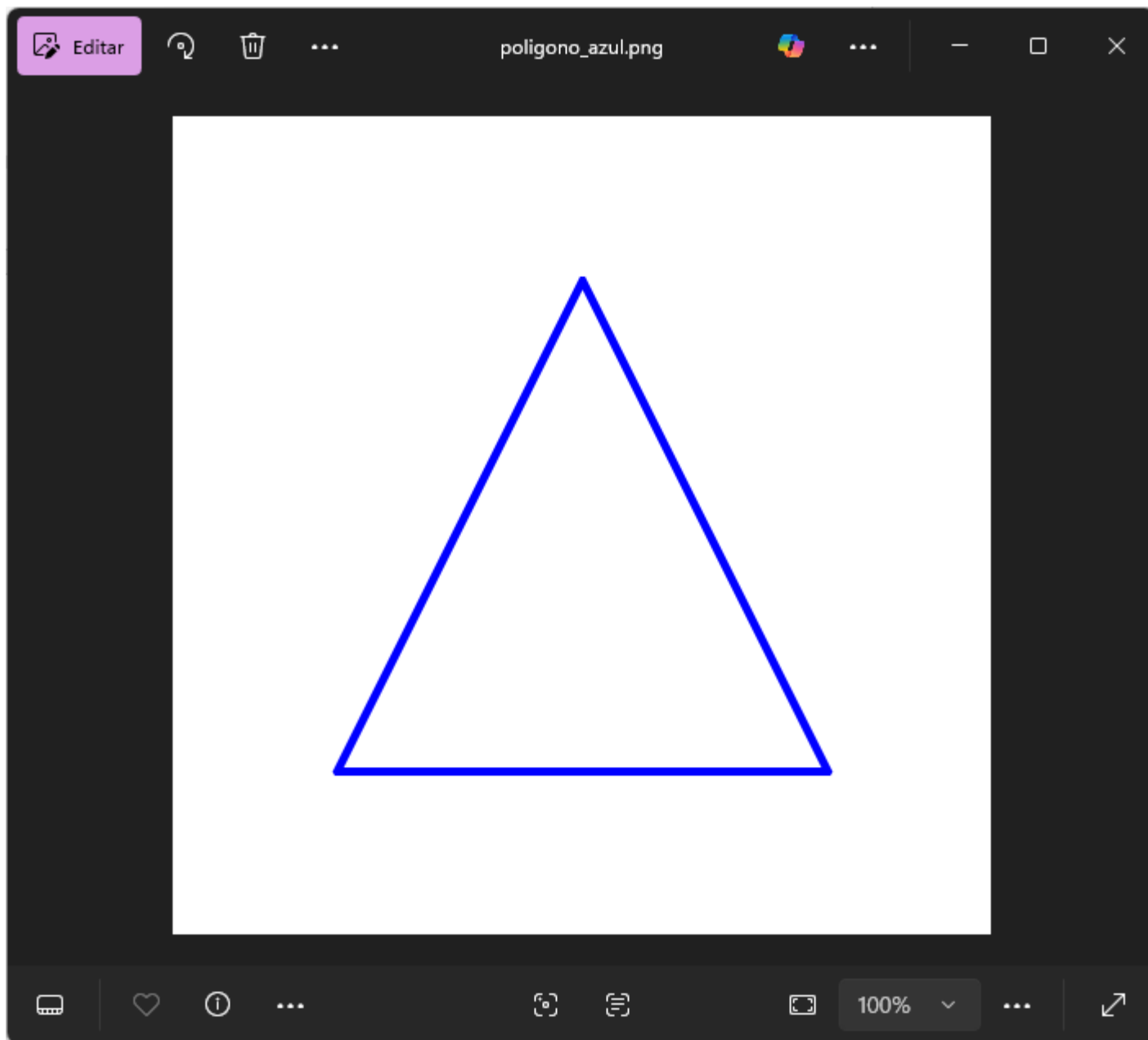
class Program {
    static void Main() {
        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapiz = Pens.Solid(Color.Blue, 5);

                // Definir los puntos del polígono (triángulo en este caso)
                var Poligono = new Polygon(new LinearLineSegment(
                    new PointF(250, 100),
                    new PointF(100, 400),
                    new PointF(400, 400),
                    new PointF(250, 100) // cerrar el triángulo
                ));

                Lienzo.Draw(Lapiz, Poligono);
            });

            // Guardar la imagen
            NuevaImagen.Save("poligono_azul.png");
        }
    }
}
```



*Ilustración 50: Dibuja un polígono dentro de una imagen*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

class Program {
    static void Main() {
        int Ancho = 500;
        int Alto = 500;
        int cantidadLineas = 20;
        Random Azar = new();

        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

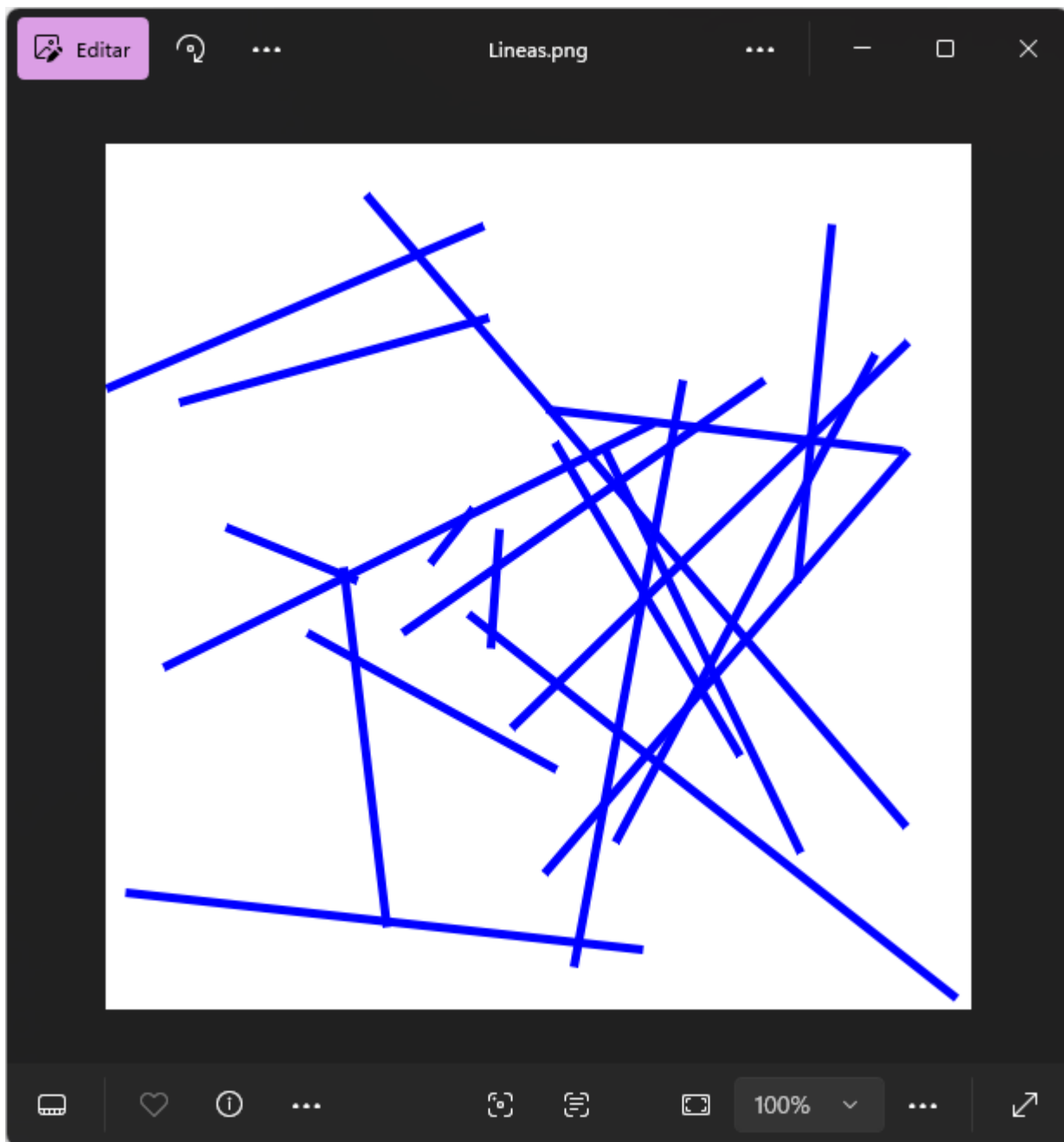
                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapiz = Pens.Solid(Color.Blue, 5);

                for (int i = 0; i < cantidadLineas; i++) {
                    // Generar puntos aleatorios dentro del área de la imagen
                    var x1 = Azar.Next(Ancho);
                    var y1 = Azar.Next(Alto);
                    var x2 = Azar.Next(Ancho);
                    var y2 = Azar.Next(Alto);

                    var Linea = new SixLabors.ImageSharp.Drawing.PathBuilder()
                        .AddLine(new PointF(x1, y1), new PointF(x2, y2))
                        .Build();

                    Lienzo.Draw(Lapiz, Linea);
                }
            });

            // Guardar la imagen
            NuevaImagen.Save("Lineas.png");
        }
    }
}
```



*Ilustración 51: Dibuja líneas en una imagen*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

class Program {
    static void Main() {
        int Ancho = 500;
        int Alto = 500;
        int cantidadCirculos = 20;
        Random Azar = new();

        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapiz = Pens.Solid(Color.Blue, 5);

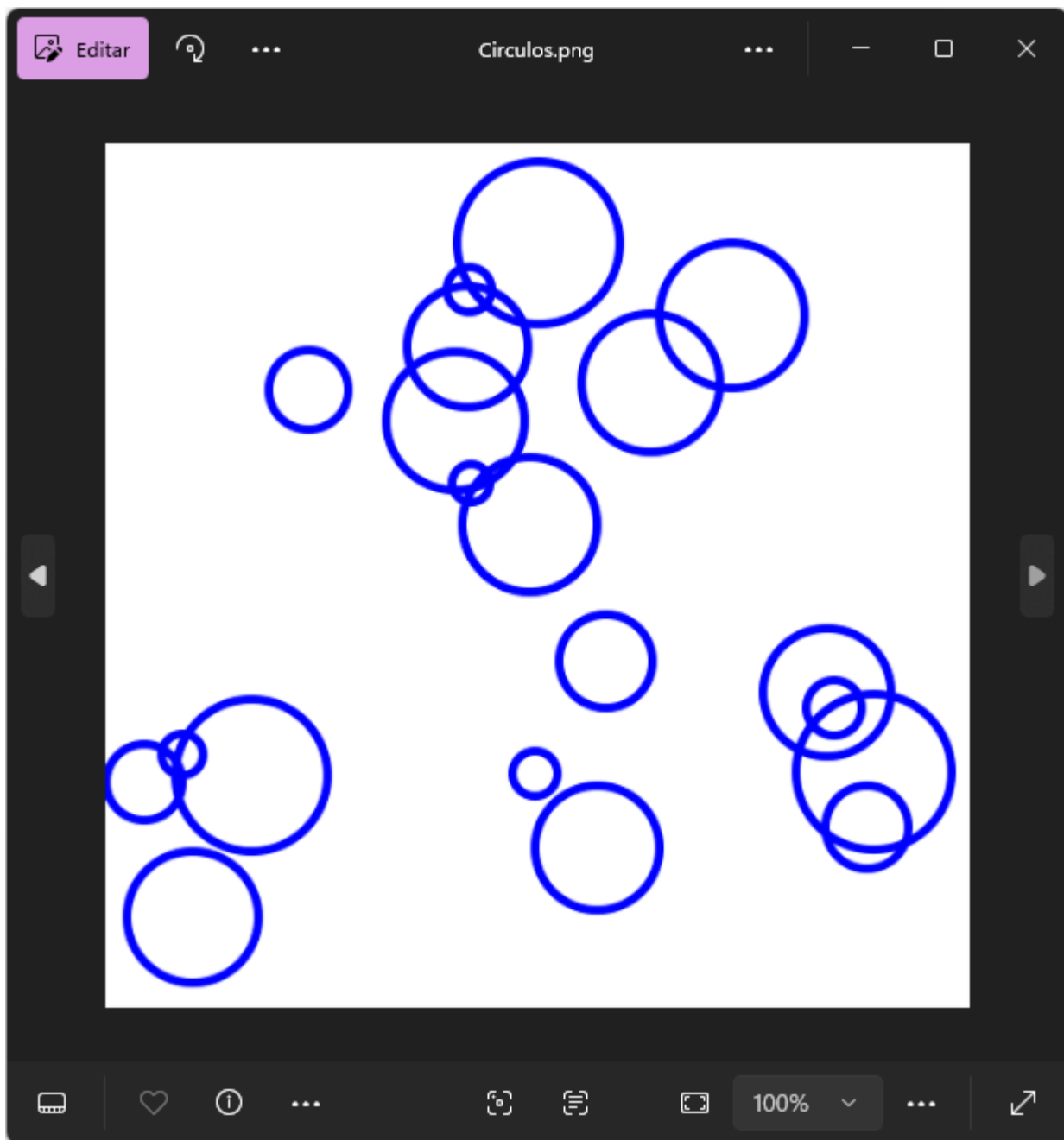
                for (int i = 0; i < cantidadCirculos; i++) {
                    // Radio aleatorio entre 10 y 50
                    float radio = Azar.Next(10, 51);

                    // Posición aleatoria dentro de los límites de la imagen
                    float x = Azar.Next((int)radio, Ancho - (int)radio);
                    float y = Azar.Next((int)radio, Alto - (int)radio);

                    var circle = new EllipsePolygon(new PointF(x, y), radio);
                    Lienzo.Draw(Lapiz, circle);
                }
            });

            // Guardar la imagen
            NuevaImagen.Save("Circulos.png");
        }
    }
}
```





*Ilustración 52: Dibuja círculos en una imagen*

```
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

class Program {
    static void Main() {
        int Ancho = 500;
        int Alto = 500;
        int cantidadRectangulos = 20;
        Random Azar = new();

        // Crear una imagen de 500x500 píxeles con fondo blanco
        using (var NuevaImagen = new Image<Rgba32>(500, 500)) {
            NuevaImagen.Mutate(Lienzo => {
                // Fondo blanco
                Lienzo.Fill(Color.White);

                // Crear un lapiz azul con grosor de 5 píxeles
                var Lapiz = Pens.Solid(Color.Blue, 5);

                for (int i = 0; i < cantidadRectangulos; i++) {
                    // Generar posición y tamaño aleatorios
                    int w = Azar.Next(40, 200);
                    int h = Azar.Next(40, 200);
                    int x = Azar.Next(0, Ancho - w);
                    int y = Azar.Next(0, Alto - h);

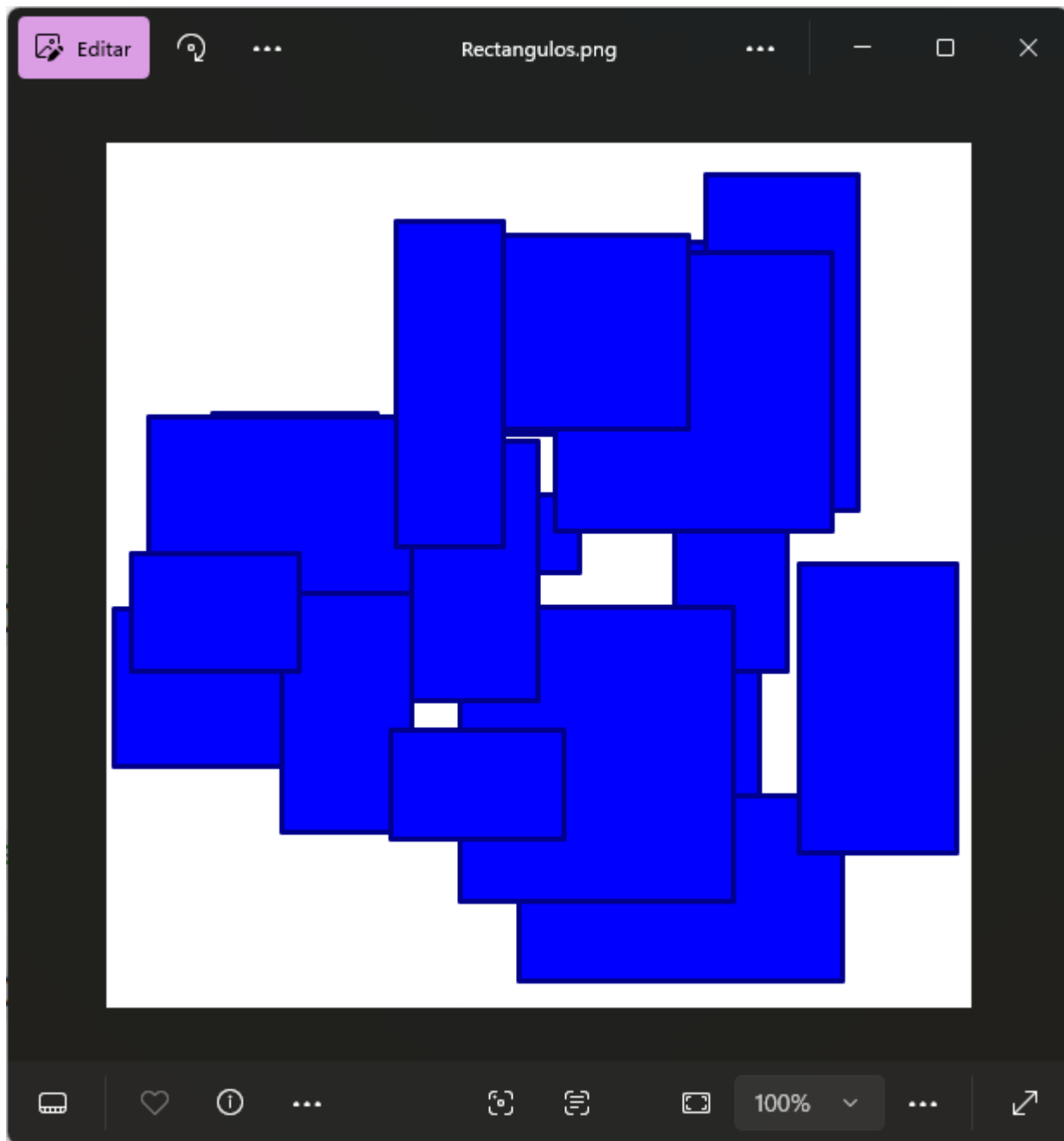
                    var rectangulo = new RectangularPolygon(x, y, w, h);

                    // Dibujar relleno azul
                    Lienzo.Fill(Color.Blue, rectangulo);

                    // Dibujar borde opcional
                    Lienzo.Draw(Color.DarkBlue, 3f, rectangulo);
                }
            });

            // Guardar la imagen
            NuevaImagen.Save("Rectangulos.png");
        }
    }
}
```

}



*Ilustración 53: Rectángulos rellenos al azar*

```
using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

class Program {
    static void Main() {
        int width = 800, height = 600;

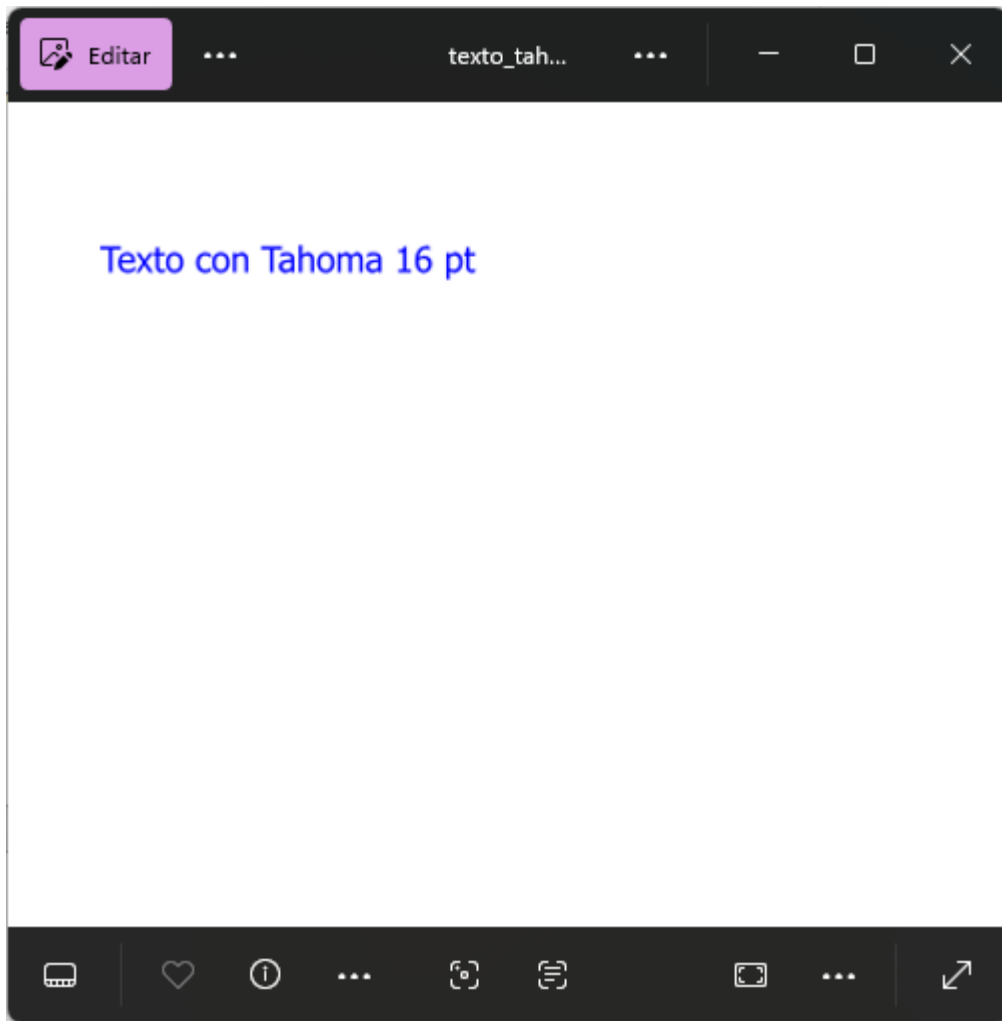
        using var img = new Image<Rgba32>(width, height, Color.White);

        // Intentar obtener Tahoma desde fuentes del sistema
        Font font;
        if (SystemFonts.TryGet("Tahoma", out FontFamily tahomaFamily)) {
            font = tahomaFamily.CreateFont(16, FontStyle.Regular);
        }
        else {
            throw new InvalidOperationException("La fuente 'Tahoma' no está disponible en el sistema.");
        }

        string texto = "Texto con Tahoma 16 pt";
        var color = Color.Blue; // Cambia el color según necesites
        var position = new PointF(50, 80); // Posición superior izquierda del texto

        img.Mutate(ctx => {
            // Dibujar texto
            ctx.DrawText(texto, font, color, position);
        });

        img.Save("texto_tahoma.png");
        Console.WriteLine("Imagen generada: texto_tahoma.png");
    }
}
```



```

using SixLabors.Fonts;
using SixLabors.ImageSharp;
using SixLabors.ImageSharp.Drawing.Processing;
using SixLabors.ImageSharp.PixelFormats;
using SixLabors.ImageSharp.Processing;

namespace Ejemplo;

class Program {
    static void Main() {
        int width = 800, height = 600;

        using var img = new Image<Rgba32>(width, height, Color.White);

        // Intentar obtener Tahoma desde fuentes del sistema
        Font fuenteLetra;
        if (SystemFonts.TryGet("Tahoma", out FontFamily tahomaFamily)) {
            fuenteLetra = tahomaFamily.CreateFont(31, FontStyle.Regular);
        }
        else {
            throw new InvalidOperationException("La fuente 'Tahoma' no está
disponible en el sistema.");
        }

        string Texto = "Texto con Tahoma 31 pt";

        img.Mutate(x => x
            .DrawText(
                new DrawingOptions {
                    Transform = Matrix3x2Extensions.CreateRotationDegrees(45, new
PointF(50, 50))
                },
                Texto,
                fuenteLetra,
                Color.Black,
                new PointF(50, 50)));

        img.Save("texto_tahoma.png");
        Console.WriteLine("Imagen generada: texto_tahoma.png");
    }
}

```

